SQL Handbook

Queries · joins · window functions · data modeling · normalization · indexes · transactions · scaling · 40+ interview problems — PostgreSQL first

20 parts · one place to study: DesignGurus courses + classic books + the web, combined, beginner to intermediate

what it is | problem | how it works | trade-offs | real world | interview | self-check | TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Chapter 1 — Databases and the Relational Model**
   - 1.1 Before databases: the file processing era
   - 1.2 What a database is
   - 1.3 What a DBMS does
   - 1.4 Inside a DBMS: the core components
   - 1.5 Database architectures
   - 1.6 The relational model
   - 1.7 RDBMS — relational database management systems
   - 1.8 Relational vs non-relational databases
   - 1.9 What SQL is
   - 1.10 Your private PostgreSQL server and psql
   - 1.11 The sample database
   - Key terms (chapter 1)
   - Chapter 1 TL;DR
2. **Chapter 2 — SELECT Basics: Reading, Filtering, Sorting and Paging**
   - 2.1 SELECT: choosing what comes back
   - 2.2 WHERE: filtering rows
   - 2.3 Operators
   - 2.4 Ranges, lists and patterns: BETWEEN, IN, LIKE/ILIKE, regex
   - 2.5 NULL and three-valued logic
   - 2.6 DISTINCT: removing duplicate rows
   - 2.7 ORDER BY: sorting results
   - 2.8 LIMIT, OFFSET and FETCH: returning a page
   - 2.9 How a SELECT is evaluated (logical processing order)
   - Putting it together
   - Chapter 2 TL;DR
3. **Chapter 3 — Functions and Expressions**
   - 3.1 Data types and conversions
   - 3.2 Number functions
   - 3.3 String functions
   - 3.4 Date and time functions
   - 3.5 Conditional expressions: CASE, COALESCE, NULLIF, GREATEST, LEAST
   - 3.6 Worked exercises
   - Chapter 3 TL;DR
4. **Chapter 4 — Aggregation: Summarising Many Rows**
   - 4.1 Aggregate functions
   - 4.2 GROUP BY
   - 4.3 HAVING
   - 4.4 Conditional aggregation: FILTER and CASE
   - 4.5 The logical order of query processing
   - 4.6 Subtotals: GROUPING SETS, ROLLUP and CUBE
   - Chapter 4 TL;DR
5. **Chapter 5 — Joins: Combining Tables**
   - 5.1 Join theory: the Cartesian product and CROSS JOIN
   - 5.2 The four main join types at a glance
   - 5.3 INNER JOIN
   - 5.4 LEFT OUTER JOIN
   - 5.5 RIGHT OUTER JOIN
   - 5.6 FULL OUTER JOIN
   - 5.7 SELF JOIN
   - 5.8 Semi-joins and anti-joins
   - 5.9 Join pitfalls
   - 5.10 USING, NATURAL JOIN, multi-table joins, and how joins are executed
   - Chapter 5 TL;DR
6. **Chapter 6 — Subqueries and Set Operations**
   - 6.1 Subqueries: queries inside queries
   - 6.2 Multi-row subqueries: IN and NOT IN
   - 6.3 Correlated subqueries
   - 6.4 EXISTS and NOT EXISTS
   - 6.5 ANY, SOME and ALL
   - 6.6 Table subqueries: derived tables and row subqueries
   - 6.7 LATERAL subqueries
   - 6.8 Set operations and UNION
   - 6.9 INTERSECT
   - 6.10 EXCEPT
   - 6.11 Precedence, sorting, and choosing the right tool
   - Chapter 6 TL;DR
7. **Chapter 7 — CTEs, Views and Recursion: Naming Queries and Walking Trees**
   - 7.1 Common table expressions (WITH)
   - 7.2 Data-modifying CTEs (PostgreSQL)
   - 7.3 Recursive CTEs: the mechanics
   - 7.4 Hierarchies: trees stored as parent pointers
   - 7.5 Graphs and cycles
   - 7.6 generate_series: making rows (PostgreSQL)
   - 7.7 Views
   - 7.8 Materialized views
   - 7.9 Choosing the right tool
   - Chapter 7 TL;DR
8. **Chapter 8 — Window Functions: Rankings, Running Totals and Row-to-Row Comparisons**
   - 8.1 What a window function is
   - 8.2 PARTITION BY: per-group values next to each row
   - 8.3 ORDER BY inside OVER: running calculations and the default frame
   - 8.4 Frames: ROWS, RANGE, GROUPS and EXCLUDE

- 8.5 Ranking functions
- 8.6 Value (offset) functions: LAG, LEAD, FIRST_VALUE, LAST_VALUE, NTH_VALUE
- 8.7 Running totals, moving averages and growth
- 8.8 Top-N per group and de-duplication
- 8.9 Gaps and islands
- 8.10 Named windows, FILTER, evaluation order and performance
- Chapter 8 TL;DR

9. **Chapter 9 — DDL, DML and Constraints: Building Tables and Changing Data Safely**
   - 9.1 Data types
   - 9.2 CREATE: databases, schemas and tables
   - 9.3 Keys and constraints
   - 9.4 Generated keys: identity, sequences, serial, UUID, generated columns
   - 9.5 ALTER: changing existing tables
   - 9.6 DROP and TRUNCATE (vs DELETE)
   - 9.7 Indexes (in brief)
   - 9.8 INSERT
   - 9.9 UPDATE
   - 9.10 DELETE
   - 9.11 Upsert: INSERT … ON CONFLICT, and MERGE
   - 9.12 Backup and restore
   - Chapter 9 TL;DR
10. **Chapter 10 — Data Modeling and ER Diagrams**
   - 10.1 What data modeling is
   - 10.2 The three levels of a data model
   - 10.3 The data modeling process
   - 10.4 Best practices in data modeling
   - 10.5 The Entity-Relationship (ER) model and its notations
   - 10.6 Entities and the five kinds of attribute
   - 10.7 Relationships: degree, cardinality and participation
   - 10.8 Attributes of relationships
   - 10.9 Weak entities
   - 10.10 Worked example — an employee-management ER diagram
   - 10.11 The relational model
   - 10.12 Relations, tuples, attributes — the precise vocabulary
   - 10.13 Keys in relational databases
   - 10.14 Relational integrity constraints
   - 10.15 Converting an ER diagram to tables
   - 10.16 Best practices for ER → relational mapping
   - Key terms
   - Chapter 10 TL;DR
11. **Chapter 11 — Normalization**
   - 11.1 Functional dependencies
   - 11.2 Types of functional dependencies
   - 11.3 Armstrong's inference rules
   - 11.4 Attribute closure and finding candidate keys
   - 11.5 Minimal (canonical) cover
   - 11.6 Anomalies — why normalization exists
   - 11.7 First normal form (1NF)
   - 11.8 Second normal form (2NF)
   - 11.9 Third normal form (3NF)
   - 11.10 Decomposition done right: lossless joins and dependency preservation
   - 11.11 Boyce–Codd normal form (BCNF)
   - 11.12 Fourth normal form (4NF) — independent multivalued facts
   - 11.13 Fifth normal form (5NF) — facts that need three tables
   - 11.14 Putting it together — a normalization procedure
   - 11.15 Exercises (with solutions)
   - 11.16 Denormalization — when and how to break the rules
   - Key terms
   - Chapter 11 TL;DR
12. **Chapter 12 — Indexes, Storage and Query Performance**
   - 12.1 How a database stores data
   - 12.2 Data compression and encoding
   - 12.3 File organization
   - 12.4 What an index is
   - 12.5 Single-level indexing, with numbers
   - 12.6 Tree-based indexes: B-trees and B+ trees
   - 12.7 Hash-based indexing
   - 12.8 Bloom filters
   - 12.9 PostgreSQL's other index types: GIN, GiST, BRIN
   - 12.10 Reading EXPLAIN: the four ways to read a table
   - 12.11 Composite indexes and column order
   - 12.12 Covering, partial and expression indexes
   - 12.13 Anti-patterns that disable indexes
   - 12.14 OFFSET vs keyset pagination
   - 12.15 Join algorithms
   - 12.16 Statistics: how the planner estimates
   - 12.17 A tuning workflow
   - Chapter TL;DR
13. **Chapter 13 — Transactions and Concurrency**
   - 13.1 What a transaction is
   - 13.2 Atomicity
   - 13.3 Consistency
   - 13.4 Isolation — anomalies and isolation levels
   - 13.5 Durability
   - 13.6 Watching isolation anomalies on a real server
   - 13.7 Lost updates and how to prevent them
   - 13.8 Write skew and Serializable isolation
   - 13.9 Concurrency control — the big picture
   - 13.10 Lock-based concurrency control
   - 13.11 Deadlocks
   - 13.12 Row locks for work queues: `FOR UPDATE`, `NOWAIT`, `SKIP LOCKED`
   - 13.13 Timestamp-based concurrency control

- 13.14 Multiversion concurrency control (MVCC)
- 13.15 Putting it together
- Chapter TL;DR

14. **Chapter 14 — Advanced SQL and Using SQL from Code**
    - 14.1 User-defined functions
    - 14.2 Stored procedures
    - 14.3 Triggers
    - 14.4 JSON and JSONB
    - 14.5 Arrays
    - 14.6 Full-text search
    - 14.7 Pivot and unpivot
    - 14.8 Security: roles, privileges and row-level security
    - 14.9 SQL injection and parameterized queries
    - 14.10 Using SQL from Python
    - 14.11 Using SQL from Java (JDBC)
    - 14.12 Connection pooling
    - 14.13 ORMs vs raw SQL
    - Chapter TL;DR
15. **Chapter 15 — Scaling Databases: Partitioning, Sharding, Replication and Recovery**
    - 15.1 Distributed databases: the big picture
    - 15.2 Partitioning
    - 15.3 Sharding
    - 15.4 Distributed query processing
    - 15.5 Replication
    - 15.6 Consistency models
    - 15.7 The CAP theorem (and PACELC)
    - 15.8 Consensus and leader election
    - 15.9 Fault tolerance and recovery
    - 15.10 High availability: replication plus failover
    - 15.11 Data resilience: redundancy, mirroring, backups
    - Chapter 15 TL;DR
16. **Chapter 16 — Beyond Relational: NoSQL, NewSQL and Specialised Databases**
    - 16.1 Why NoSQL?
    - 16.2 Key-value stores
    - 16.3 Document databases
    - 16.4 Column-family and wide-column stores
    - 16.5 Graph databases
    - 16.6 In-memory databases
    - 16.7 NewSQL / distributed SQL
    - 16.8 Time-series and spatial databases
    - 16.9 Big data and distributed processing frameworks
    - 16.10 Choosing a database
    - 16.11 Inside two workhorses: Elasticsearch (and the ELK stack) and Redis
    - Chapter 16 TL;DR
17. **Chapter 17 — Schema Design Case Studies**
    - 17.0 The template
    - 17.1 Hospital management system
    - 17.2 Hotel reservation system
    - 17.3 Banking system
    - 17.4 Instagram-style photo sharing (runnable)
    - 17.5 E-commerce store (runnable)
    - 17.6 Food delivery platform
    - 17.7 Library management system (runnable)
    - 17.8 Ticket booking system (runnable, with seat-hold concurrency)
    - 17.9 Warehouse inventory system
    - 17.10 Quora (questions and answers)
    - 17.11 TikTok (short-video platform)
    - 17.12 Airbnb (short-term rentals)
    - 17.13 Snapchat (ephemeral messaging)
    - 17.14 Recurring modeling patterns
    - Chapter 17 TL;DR
18. **Chapter 18 — SQL Interview Problems I: Method and 42 Warm-Up to Medium Problems**
    - 18.1 How to approach a SQL interview question
    - 18.2 Filtering problems (P1–P8)
    - 18.3 Aggregation problems (P9–P16)
    - 18.4 Join problems (P17–P22)
    - 18.5 Anti-join problems (P23–P26)
    - 18.6 Conditional aggregation (P27–P28)
    - 18.7 Date-logic problems (P29–P33)
    - 18.8 Ratios and percentages (P34–P38)
    - 18.9 Window and ranking basics (P39–P42)
    - 18.10 Pattern table
    - Chapter 18 TL;DR
19. **Chapter 19 — SQL Interview Problems 2: Medium and Hard Patterns**
    - 19.1 How to attack a medium or hard problem
    - 19.2 Warm-ups: joins, counts and grids
    - 19.3 Nth highest, top-N per group and "first row per group"
    - 19.4 Running totals and row-to-row comparisons
    - 19.5 Gaps and islands: consecutive rows and streaks
    - 19.6 Pivot and unpivot (static and dynamic)
    - 19.7 Self-joins, "both directions" and graph-like questions
    - 19.8 Date bucketing and retention
    - 19.9 Ratios and cancellation rates
    - 19.10 String building and expression evaluation
    - 19.11 Pattern catalogue
    - Chapter 19 TL;DR
20. **Chapter 20 — Cheat Sheets, Interview Q&A and Glossary**
    - 20.1 The logical query-processing order
    - 20.2 SELECT, filtering, NULL, sorting and paging
    - 20.3 Functions and expressions
    - 20.4 Aggregation
    - 20.5 Joins
    - 20.6 Subqueries and set operations

- 20.7 CTEs, recursion and views
- 20.8 Window functions and frames
- 20.9 DDL, DML, constraints and upsert
- 20.10 Data modeling and normalization quick rules
- 20.11 Indexes and reading EXPLAIN
- 20.12 Transactions, isolation and concurrency
- 20.13 Advanced PostgreSQL: PL/pgSQL, JSONB, full-text search, security
- 20.14 Scaling: partitioning, sharding, replication
- 20.15 Which database?
- 20.16 Dialect differences
- 20.17 Verified behaviours
- 20.18 Interview pattern catalogue
- 20.19 Interview questions and answers
- 20.20 Glossary (A–Z)
- Chapter 20 TL;DR

Chapter 1 — Databases and the Relational Model

Almost every program you use keeps its facts somewhere: the orders in a shop, the balance in a bank, the messages in a chat app. This chapter explains where those facts live and why the answer is usually "in a relational database". You will see the problems that existed before databases, what a database management system (DBMS) does for you, how a DBMS is built inside, how relational databases compare with NoSQL stores (databases that keep data in shapes other than tables, covered in 1.8), and what SQL is. The chapter ends with a working PostgreSQL server (PostgreSQL is a free, open-source relational database product; a *server* is a program that runs in the background and answers requests from other programs) on your machine and the **sample database** that the next chapters query.

What you'll learn

#	Topic	You will be able to…
1.1	Life before databases	name the seven problems of file-based systems
1.2	What a database is	define database, DBMS and database system precisely
1.3	What a DBMS does	list its six jobs and explain data independence
1.4	Inside a DBMS	follow one query through parser, optimizer, executor, storage and log
1.5	Database architectures	compare two-tier, three-tier, n-tier, cloud and distributed set-ups
1.6	The relational model	use the vocabulary (relation, tuple, key…) and the four table rules
1.7	RDBMS products	describe what an RDBMS adds and how the main products differ
1.8	Relational vs non-relational	choose a store by data shape and access pattern
1.9	What SQL is	sort any statement into DDL, DML, DQL, DCL or TCL
1.10	Your private PostgreSQL and psql	start the server, connect, and use the psql commands you need daily
1.11	The sample database	know the `company` tables used in chapters 2 onward

All code is in `code/`. Every output below was pasted from a real run on the book's private PostgreSQL 14 server (section 1.10 shows how to start it).

> **Dialect note.** This book writes PostgreSQL first. Where MySQL, SQLite or SQL Server (three other widely used database products, compared in 1.7) behave differently, a small table says so. (Each product speaks its own *dialect* — its version of SQL; see 1.9.) Most of what you learn transfers directly; the differences are mainly in functions (chapter 3), limits/paging (chapter 2) and data types (chapter 9).

1.1 Before databases: the file processing era

What it is

Until the late 1960s most business software stored data in **files owned by one program each**. A payroll program had its payroll files, the billing program had its billing files. The layout of each file (which bytes meant "name", which meant "salary") was written into the program's code, not stored with the file. This arrangement is called a **file processing system**.

Think of a school where the admissions office, the library and the exam office each keep their own paper register of students. Each register works fine on its own. Trouble starts when a student moves house.

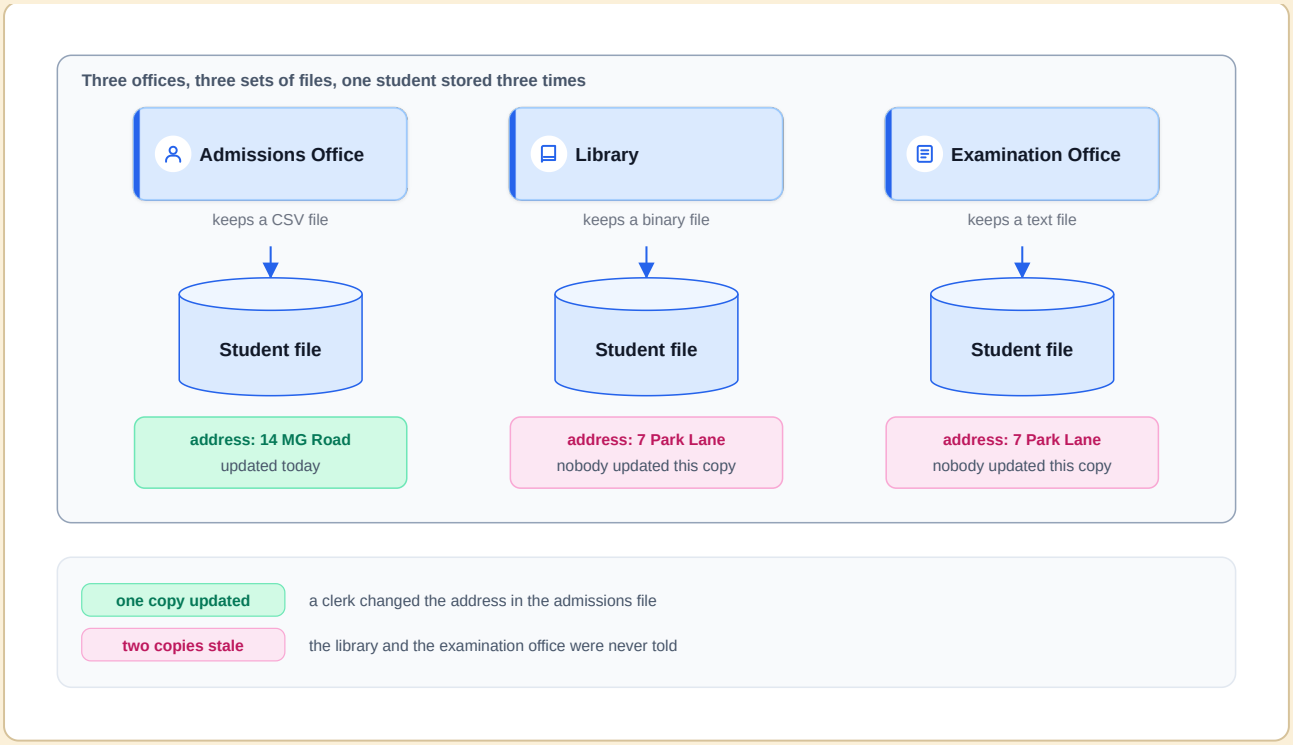


Figure — One fact stored in three separate files: an update reaches one of them and the other two go stale (course diagram).

Records in such files were found in one of two ways. **Sequential access** reads the file from the beginning until the wanted record appears. **Indexed access** keeps a second, hand-maintained file that maps a key (say, a student number) to a byte position, which is faster but is one more thing to keep in sync.

The problem it solves — seven recurring failures

As companies added programs and data grew, the same seven problems appeared everywhere. Each one later became a feature of the DBMS.

#	Problem	What went wrong	What a DBMS does instead
1	**Redundancy → inconsistency**	A customer's address is copied into several files; an update reaches one copy, the copies disagree, and nothing notices.	Store each fact once; every program reads the same copy.
2	**Data isolation**	Files sit on different machines in different formats (CSV = plain text with commas between values; fixed-width binary = raw bytes at set positions); a cross-department question needs custom parsing code.	One store with one self-describing format.
3	**No ad-hoc questions**	No query language: "which customers owe more than ₹500?" means days of writing a new program, so many questions are never asked.	A declarative query language (SQL — you state *what* you want, not *how* to get it): a new question is a new query.
4	**Integrity rules buried in code**	"Marks must be 0–100" lives as an `if` in one program. Any other program writing that file can skip it.	Constraints (rules declared on the table, such as "0 to 100") declared once and enforced for every writer.
5	**Unsafe concurrent access**	Two clerks both read "1 copy left" and both save 0: two copies go out, the count dropped once — a **lost update**. Locking the whole file blocks everybody.	Row-level locks (a marker that makes others wait while one program changes a row) and transactions (all-or-nothing groups of changes, defined in 1.4).
6	**No crash recovery**	A transfer is two writes; power fails after the first and the money has left one account and reached none. Only an old backup can repair it.	A log (an append-only diary of every change) that lets the system undo or redo half-finished work.
7	**All-or-nothing security**	File permissions cover the whole file. A clerk who needs addresses can also read salaries.	Permissions per table, column and even row.

How the DBMS changed things

A DBMS is a layer of software that **every application must go through**; no program touches the stored files directly.

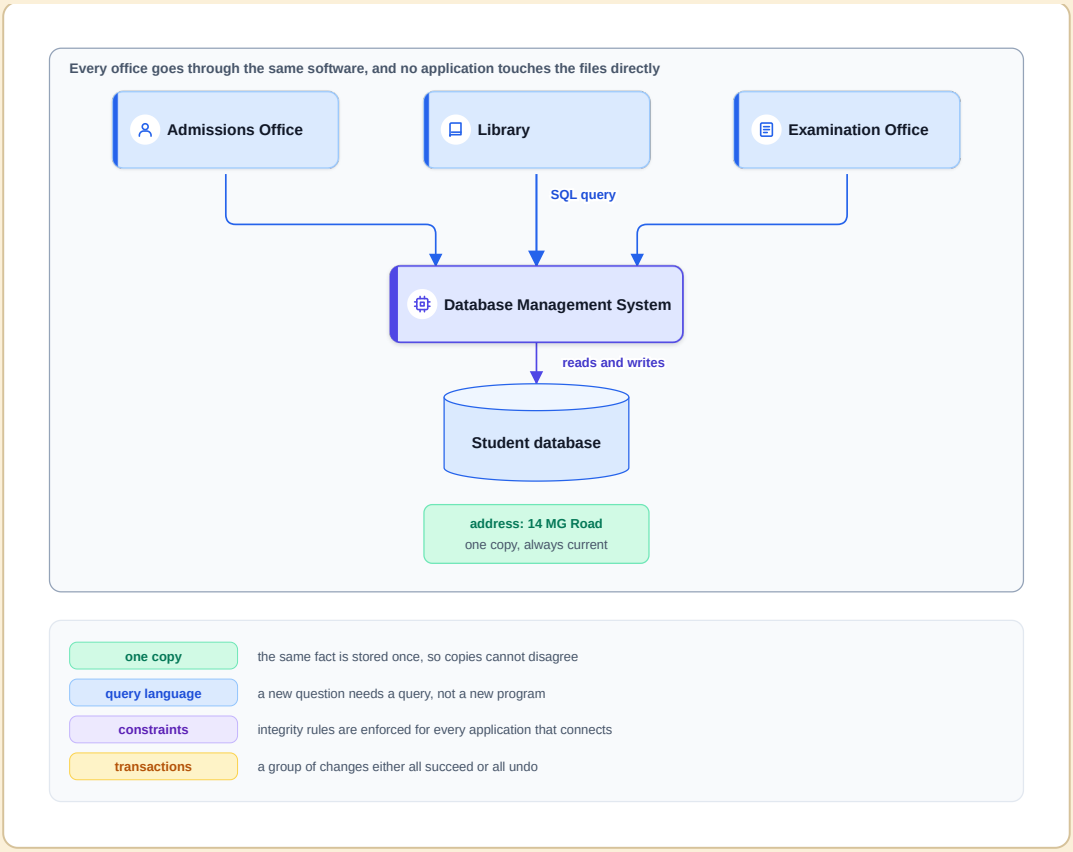


Figure — With a DBMS in the middle, a single shared copy of the data lives alongside its protecting rules (course diagram).

Because every read and write passes through one place, that place can apply all seven fixes in the table above.

A short timeline helps place the ideas:

Year	Milestone
1960s	Navigational databases: IBM's IMS (hierarchical, a tree of records) and Charles Bachman's IDS / the CODASYL network model (records linked in a web, not just a tree). Programs followed stored pointers (saved disk addresses of other records) from record to record.
1970	Edgar F. Codd (IBM) publishes *A Relational Model of Data for Large Shared Data Banks*.
1974–79	IBM's System R and the query language SEQUEL (later renamed SQL); UC Berkeley's Ingres; Oracle ships the first commercial SQL database (1979).
1986–87	SQL becomes an ANSI (1986), then ISO (1987) standard (ANSI and ISO are the US and international standards bodies). In 1986 UC Berkeley also starts POSTGRES, the ancestor of PostgreSQL.
2000s	Open-source (free, with public source code) RDBMSs (relational DBMSs, see 1.7) dominate the web (MySQL, PostgreSQL); SQLite runs inside phones and browsers.
late 2000s	"NoSQL" systems (key-value, document, wide-column, graph) for web-scale workloads.
2010s–now	Cloud-managed databases (run for you by a provider such as AWS) and distributed SQL ("NewSQL": relational databases spread over many machines).

Self-check

1. *Why does storing the same fact in three files lead to wrong answers, not just wasted space?* → Updates reach some copies and not others, so the copies drift apart (inconsistency), and nothing checks that they match.
2. *What is a lost update?* → Two programs read the same value, each computes a new value from it, and the second write overwrites the first, so one change disappears.

3. *Which file-system problem does a transaction log solve?* → Recovery from a crash in the middle of a multi-step change.

1.2 What a database is

What it is

A **database** is an organised, persistent collection of related data that you can query and update, stored *along with a description of its own structure*. Your phone's contact list is a (tiny) database; so is a bank's decade of transactions. Size changes the hardware and the cost, not the definition.

Four properties make a pile of data a database:

Property	Meaning	Example
Related	Everything describes one chosen slice of reality, often called the *mini-world* (a school, a shop, a hospital).	A shop database holds customers, products and orders — not your holiday photos.
Self-describing	The database stores **metadata** (data about data: table names, column names, types, rules) in its **catalog**, also called the **data dictionary**.	Any program can ask "which columns does `employees` have?" instead of hard-coding the layout.
Purposeful	Built for specific people and the questions they ask; requirements are gathered before any table exists.	A hospital's schema is shaped by what doctors, nurses and billing staff need to look up.
Persistent	Data survives after the program ends and the machine restarts.	An in-memory list that disappears on exit is not a database.

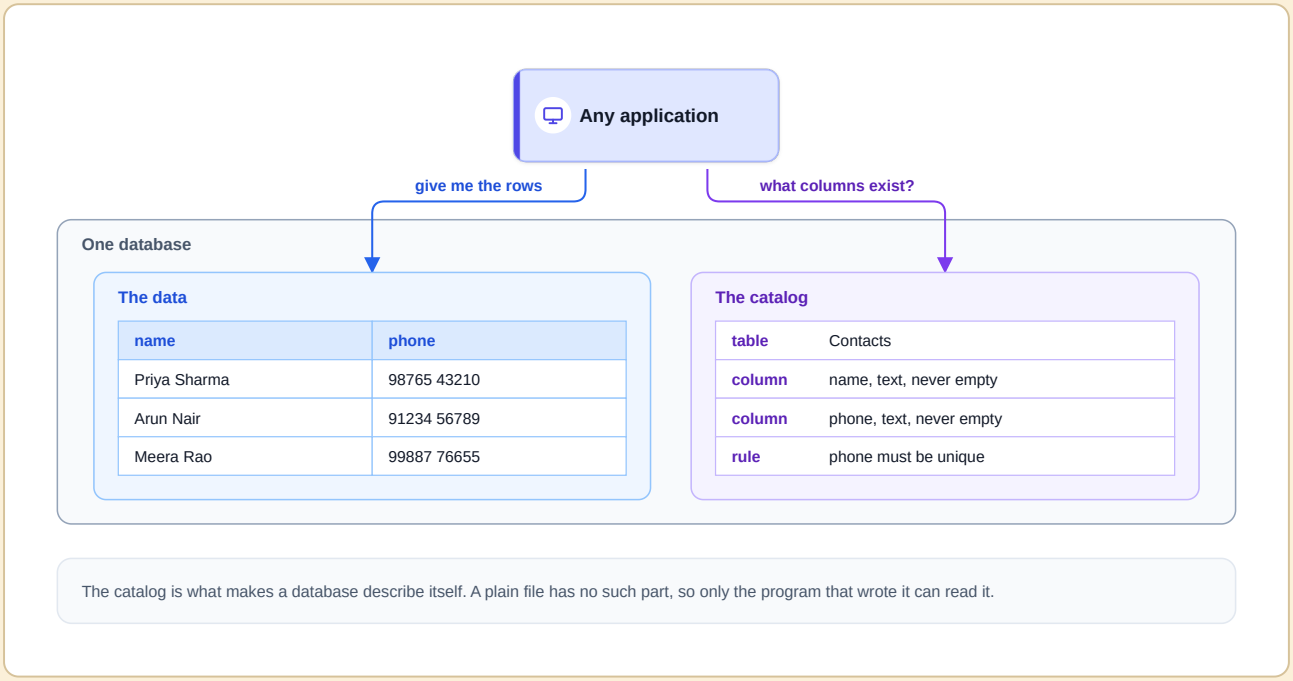


Figure — The catalog sits inside the database next to the data it describes (course diagram).

You can see self-description in action: PostgreSQL answers questions about its own tables through the standard `information_schema` views (a **view** is a saved query you can read like a table — chapter 7) and its own `pg_catalog` tables.

```sql
-- code/02_catalog_metadata.sql (part)
SELECT column_name, data_type, is_nullable
FROM information_schema.columns
WHERE table_name = 'departments'
ORDER BY ordinal_position;
```

Output:

```
 column_name | data_type | is_nullable
-------------+-----------+-------------
 dept_id     | integer   | NO
 name        | text      | NO
 location    | text      | YES
 budget      | numeric   | NO
(4 rows)
```

```sql
SELECT conname AS constraint_name, contype AS kind, conrelid::regclass AS on_table
FROM pg_constraint
WHERE conrelid = 'employees'::regclass
ORDER BY conname;
```

Output:

```
      constraint_name      | kind | on_table
---------------------------+------+-----------
 employees_dept_id_fkey    | f    | employees
 employees_email_key       | u    | employees
 employees_manager_id_fkey | f    | employees
 employees_pkey            | p    | employees
 employees_salary_check    | c    | employees
(5 rows)
```

The `kind` letters are `p` primary key, `f` foreign key, `u` unique, `c` check. A **constraint** is a rule attached to a table that the DBMS checks on every write: a *primary key* names the column that identifies each row, a *foreign key* says a value must exist in another table (both defined properly in 1.6), *unique* forbids repeats, and a *check* is a yes/no test every row must pass (here: salary must be positive). The rules you declared live in the catalog, which is how the DBMS can enforce them for every program.

Database vs DBMS vs database system

These three are often mixed up — including in interviews.

Term	What it is	Example
Database	The stored data plus its description	the `company` tables in this book
DBMS (database management system)	The software that creates, stores, queries and protects databases	PostgreSQL, MySQL, Oracle
Database system	DBMS + databases + the applications that use them	PostgreSQL + `company` + the web shop that talks to it

Why it matters

Databases make large amounts of data fast to find, keep it accurate (integrity rules) and private (access control — deciding who may see what; encryption — scrambling data so only key holders can read it), let many users work at once, and feed the reports behind business decisions. A good one is well structured, scalable (able to handle more data and users by adding hardware), reliable through failures, richly queryable and usable from many tools and languages.

Real-world examples

A shop must never lose a paid order; a hospital must never show records to the wrong person; a bank must never lose or double-apply a transfer; a social app must not fall over when millions read the same post. Each requirement maps to a DBMS job in the next section.

Self-check

1. *What is metadata and where is it kept?* → Data describing the data (tables, columns, types, constraints, permissions); in the catalog / data dictionary inside the database.
2. *Is MySQL a database?* → Strictly, no — it is a DBMS. The tables you create inside it form a database.

1.3 What a DBMS does

What it is

A **DBMS** is the program that stands between applications and stored data. Applications send requests ("give me these rows", "store this order"); the DBMS decides where bytes go on disk, how to find them again, and how to keep them safe.

How it works — the six jobs

Job	Meaning	In SQL / PostgreSQL
Define	Describe tables, columns, types and rules; saved in the catalog	`CREATE TABLE`, `ALTER TABLE`
Construct	Physically write the data to storage; you never choose file names or byte offsets	PostgreSQL chooses files under its data directory (the folder where the server keeps its files)
Manipulate	Read, add, change, remove data	`SELECT`, `INSERT`, `UPDATE`, `DELETE`
Share	Let many users and programs work on the same data at once without corrupting it	transactions, locks, MVCC (keeping several versions of a row so readers and writers don't block each other; chapter 13)
Protect	*Security* (who may read/write what) and *failure protection* (stay correct after a crash)	`GRANT` / `REVOKE`, the write-ahead log (a change diary saved to disk *before* the data itself, used to repair after a crash)
Maintain	Let the schema and storage evolve for years without rewriting applications	add a column, add an index (a lookup structure, explained below), move to faster disks

File processing systems managed the first three poorly and the last three not at all.

Who talks to a DBMS

Role	What they do
Application programs	Issue SQL for the people using them (a shopper clicks buttons, never writes SQL).
Database administrators (DBAs)	Create users and permissions, plan backups, tune and monitor.
Database designers	Decide which tables exist and how they connect (chapter 10).
Analysts	Query the data themselves to produce reports and insights.

Data independence

Data independence means a change at one level of the system's description does not force changes at the level above. It is the reason a 20-year-old report query can still run after the database moved to new hardware.

Kind	You change…	…and this keeps working	Example
Physical data independence	how data is stored (indexes, files, disks, page layout)	the tables and every query	add an index; move to SSDs (fast flash-memory disks)
Logical data independence	the tables (add a column or a table)	applications that don't use the changed part	add `phone` to `employees`; a report reading names is unaffected

What does "logical is separate from physical" mean? The **logical level** is what you query — tables, columns, views. The **physical level** is how the bytes are laid out — data files, 8 kB pages (fixed-size chunks the database reads and writes as one unit), index trees.

> **Key term — index:** An index is an extra, sorted structure the database keeps beside a table so it can find rows by a column's value without reading every row — like the index at the back of a book. It speeds up reads but costs disk space and a little extra work on every insert or update. PostgreSQL's default index is a *B-tree*, a balanced tree that stays sorted as rows change. Chapter 12 covers indexes in depth. SQL names only logical objects, so the DBMS may reorganise the physical side freely and your queries keep their meaning.

The demo below changes both levels and runs the same "application" query each time.

```sql
-- code/06_data_independence.sql
SELECT first_name, last_name, job_title FROM employees WHERE last_name = 'Verma';
EXPLAIN (COSTS OFF) SELECT first_name, last_name, job_title FROM employees WHERE last_name = 'Verma';

-- Physical change: add an index. The query text is unchanged.
CREATE INDEX employees_last_name_idx ON employees (last_name);
SET enable_seqscan = off;   -- the table is tiny; nudge the planner so we can SEE it use the index
EXPLAIN (COSTS OFF) SELECT first_name, last_name, job_title FROM employees WHERE last_name = 'Verma';
SELECT first_name, last_name, job_title FROM employees WHERE last_name = 'Verma';
RESET enable_seqscan;

-- Logical change: add a column the query does not use. Still unchanged.
ALTER TABLE employees ADD COLUMN phone text;
SELECT first_name, last_name, job_title FROM employees WHERE last_name = 'Verma';
```

Output (abridged to the interesting parts):

```
 first_name | last_name | job_title
------------+-----------+-----------
 Rahul      | Verma     | Engineer
(1 row)

              QUERY PLAN
---------------------------------------
 Seq Scan on employees
   Filter: (last_name = 'Verma'::text)
(2 rows)

                      QUERY PLAN
------------------------------------------------------
 Index Scan using employees_last_name_idx on employees
   Index Cond: (last_name = 'Verma'::text)
(2 rows)

...
ALTER TABLE
 first_name | last_name | job_title
------------+-----------+-----------
 Rahul      | Verma     | Engineer
(1 row)
```

`EXPLAIN` shows the **plan** — the method the DBMS chose. The **planner** is PostgreSQL's name for the optimizer, the part that picks the plan. Before the index it scans every row (`Seq Scan`); after, it jumps through the index (`Index Scan`). The result and the query text never changed. (`enable_seqscan = off` is a teaching switch only; on a 15-row table the planner would rightly prefer the full scan. Chapter 12 covers plans properly.)

Self-check

1. *Name the six jobs of a DBMS.* → Define, construct, manipulate, share, protect, maintain.
2. *Adding an index without touching any query is an example of…?* → Physical data independence.

1.4 Inside a DBMS: the core components

What it is

A DBMS is several cooperating parts, each with one job. Knowing them makes database behaviour predictable: why the first run of a query is slower than the second, why a crash does not lose committed data, why two users can update different rows at once.

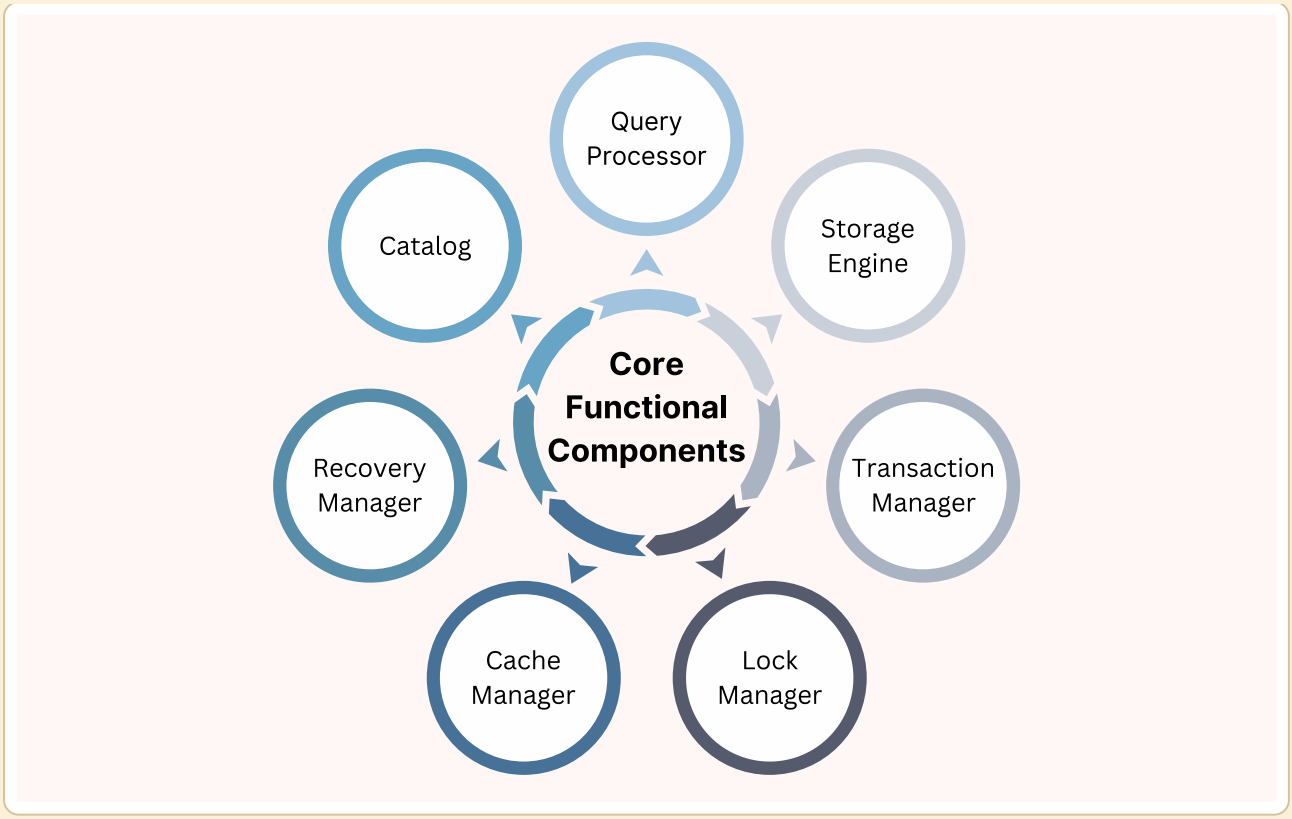
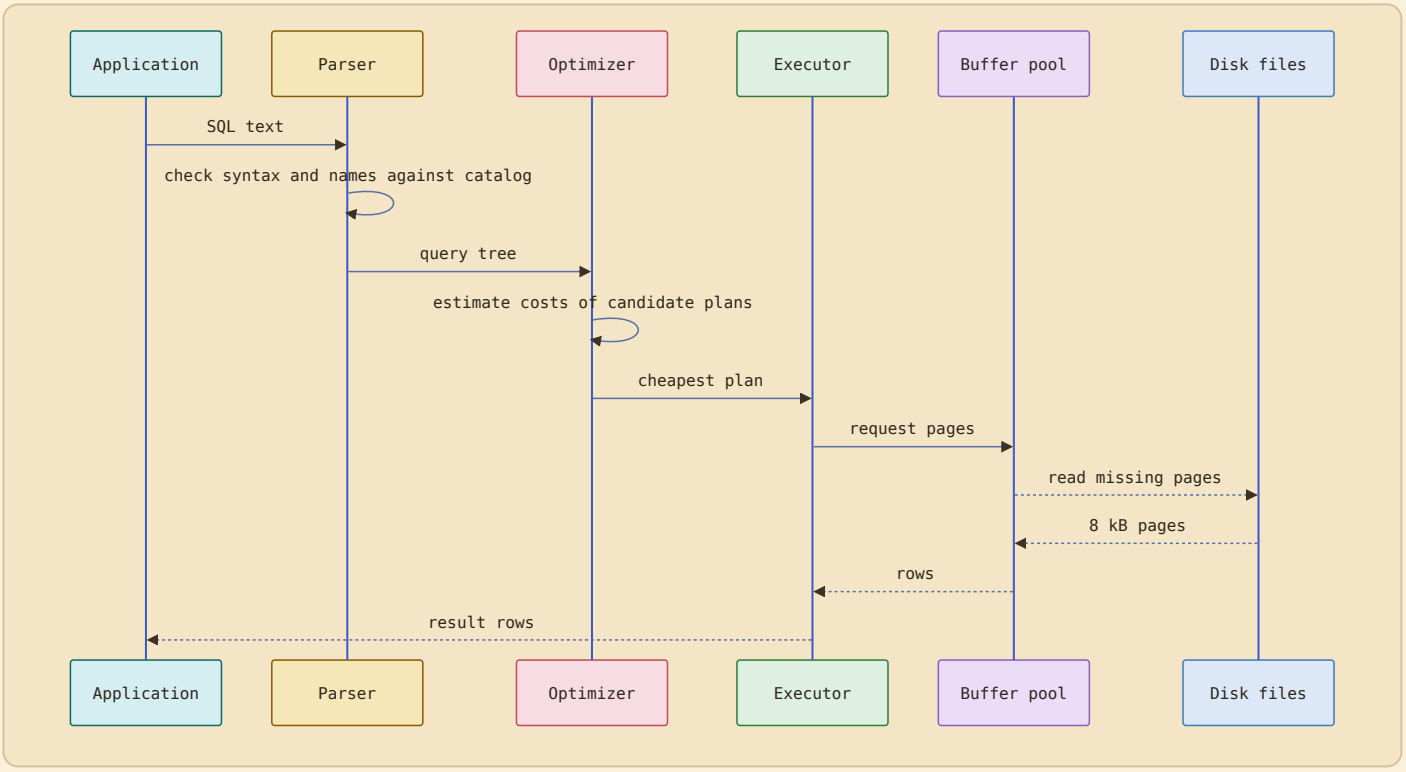


Figure — Query processor, storage engine, transaction manager, lock manager, cache manager, recovery manager and catalog (course diagram).

Component	Job	Sub-parts / tools
Query processor	Turn SQL text into results	**Parser** checks syntax and meaning (do these tables/columns exist? may you use them?) and builds an internal tree. **Optimizer** considers alternative plans and picks the cheapest by estimated cost. **Execution engine** runs the plan, asking the storage engine for rows.
Storage engine (storage manager)	Store and fetch bytes	Organises data into fixed-size **pages** (blocks) inside files; maintains indexes; may store rows (row store) or columns (column store). Examples: InnoDB in MySQL, WiredTiger in MongoDB, PostgreSQL's heap + index access methods.
Cache / buffer manager	Keep hot pages in memory	The **buffer pool** holds recently used pages; a query whose pages are already cached avoids disk reads.
Transaction manager	Make groups of operations all-or-nothing and isolated	Begins, commits (makes permanent) and rolls back (cancels) transactions; guarantees ACID (all-or-nothing, rule-preserving, isolated, durable — defined in 1.8; chapter 13).
Lock manager	Coordinate concurrent access	**Shared (S) locks**: many readers may hold one at once. **Exclusive (X) locks**: one writer, nobody else. Prevents lost updates and dirty reads (seeing another transaction's uncommitted change); must also detect **deadlocks** (two transactions each waiting for the other's lock).
Recovery manager	Restore a consistent state after a crash	Writes a **log** of every change *before* applying it; after a crash, **redoes** committed work (roll forward) and **undoes** uncommitted work (roll back).
Catalog (data dictionary)	Describe everything	Tables, columns, types, constraints, indexes, views, users and permissions, storage sizes. Every other component reads it.

A **transaction** bundles several operations into one all-or-nothing unit: if any step fails, none of the steps remain. The classic example is a bank transfer — the debit and the credit succeed together or not at all.

How it works — the journey of one query



The optimizer's candidate plans differ in scan method (full scan or index), join order and join algorithm; it estimates each one's cost from **statistics** about the data. That division of labour is what **declarative** means: your SQL names the result, the DBMS picks the algorithm. For writes, the change is first recorded in the log, the lock manager holds off conflicting writers, and the change becomes durable at `COMMIT`.

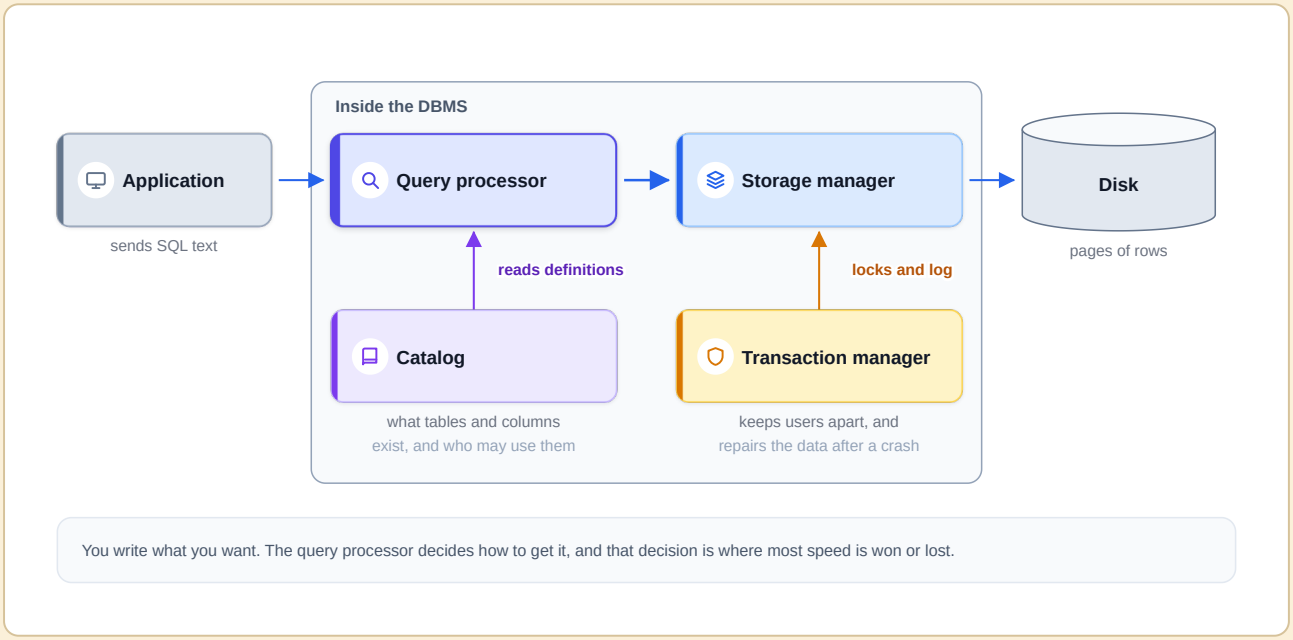


Figure — A query flows from the application through the query processor to storage; the catalog and the transaction manager support every step (course diagram).

Real-world example — the same parts in PostgreSQL

Generic component	PostgreSQL name
Parser / optimizer / executor	parser → rewriter → **planner** → executor (each connection gets its own server process, i.e. its own running copy of the server program)
Pages	8 kB blocks (`SHOW block_size` → `8192`) in files under the data directory
Buffer pool	`shared_buffers` (default `128MB`) plus the operating-system cache (file data the OS itself keeps in spare memory)
Recovery log	**WAL** (write-ahead log) in `pg_wal/`
Lock manager + concurrency	table/row locks **plus MVCC** (multi-version concurrency control: readers see a snapshot and don't block writers)
Catalog	`pg_catalog` tables and the standard `information_schema` views

(Both numbers were read on the book's server by `code/02_catalog_metadata.sql`.)

Common mistakes

- Assuming SQL runs "top to bottom as written" — the optimizer may reorder joins and filters as long as the result is the same.
- Judging a query by its first (cold-cache) run — the second may be served from the buffer pool.
- Confusing the cache (a fast in-memory copy, lost in a crash) with durability (the guarantee that committed data survives — provided by the log on disk).

In the interview

> **30-second answer:** "A DBMS has a query processor — parser, optimizer, executor — that turns SQL into a plan and runs it; a storage engine that keeps data in fixed-size pages and indexes; a buffer pool that caches pages in memory; a transaction manager and lock manager that give ACID and safe concurrency; a recovery manager that uses a write-ahead log to redo or undo work after a crash; and a catalog that describes everything."

- *Why does the optimizer need statistics?* → To estimate how many rows each step returns, which decides whether an index or a full scan, and which join order, is cheaper.
- *What is write-ahead logging?* → The log record describing a change is flushed to disk before the changed data page, so after a crash the system can replay or undo from the log.
- *Shared vs exclusive lock?* → Shared allows concurrent readers; exclusive allows one writer and blocks others.

Self-check

1. *Which component decides between an index scan and a full scan?* → The optimizer (planner).
2. *What does the recovery manager do for a committed transaction whose pages weren't yet written?* → Redoes it from the log (roll forward).

1.5 Database architectures

What it is

An **architecture** describes where the database runs relative to its users and how requests travel. The same PostgreSQL can sit in any of these set-ups.

Two-tier: client–server

The **client** application (the program asking for data) talks **directly** to the database server: it opens a **connection** (a logged-in, two-way network channel to the server), sends SQL, gets rows back.

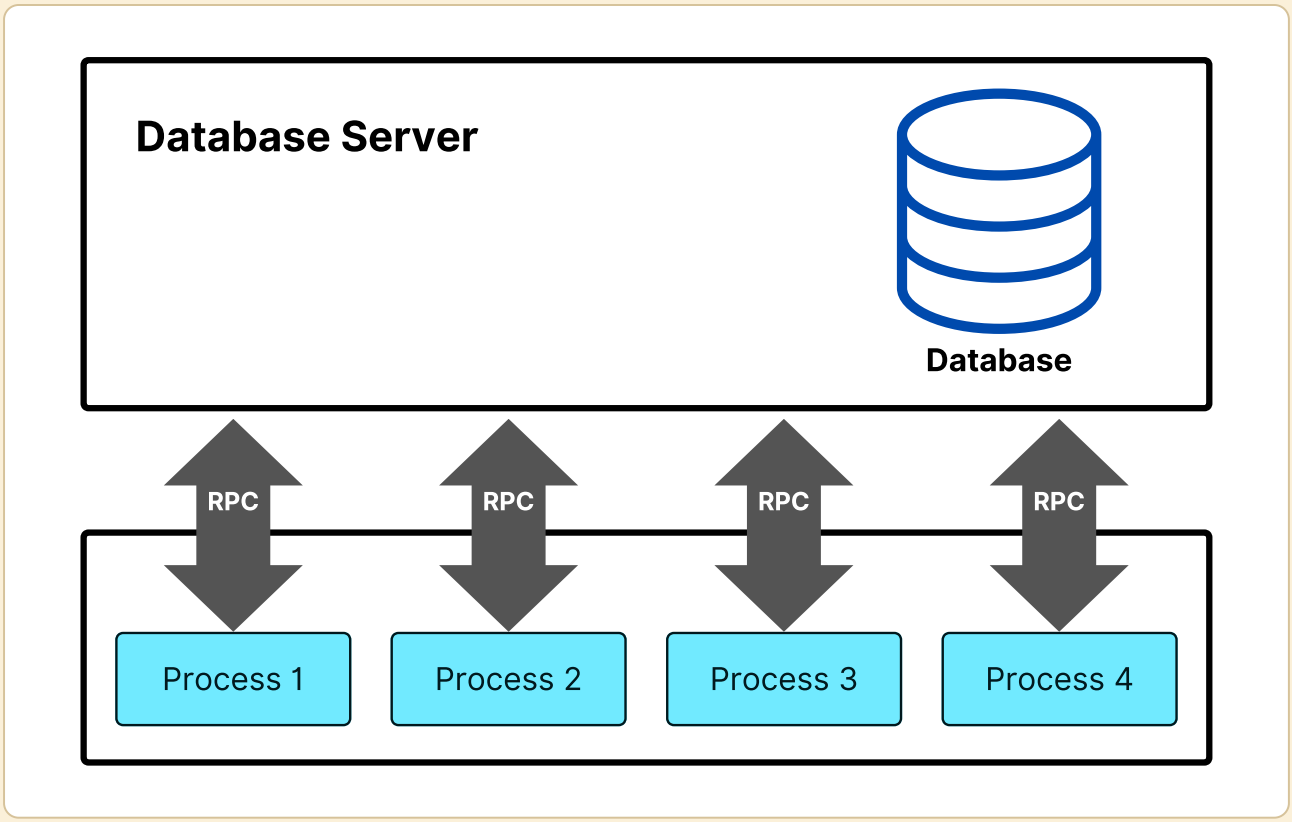


Figure — Two-tier client–server: many clients, one database server (course diagram).

- **Good:** simple; fine for small teams and internal tools (a desktop accounting app on an office server, `psql` itself).
- **Bad:** one server struggles with many client connections; business rules live in every client, so a change means redeploying (reinstalling an updated version of) them all; clients hold database credentials (user name and password).

Three-tier

A middle **application tier** sits between the user interface and the database.

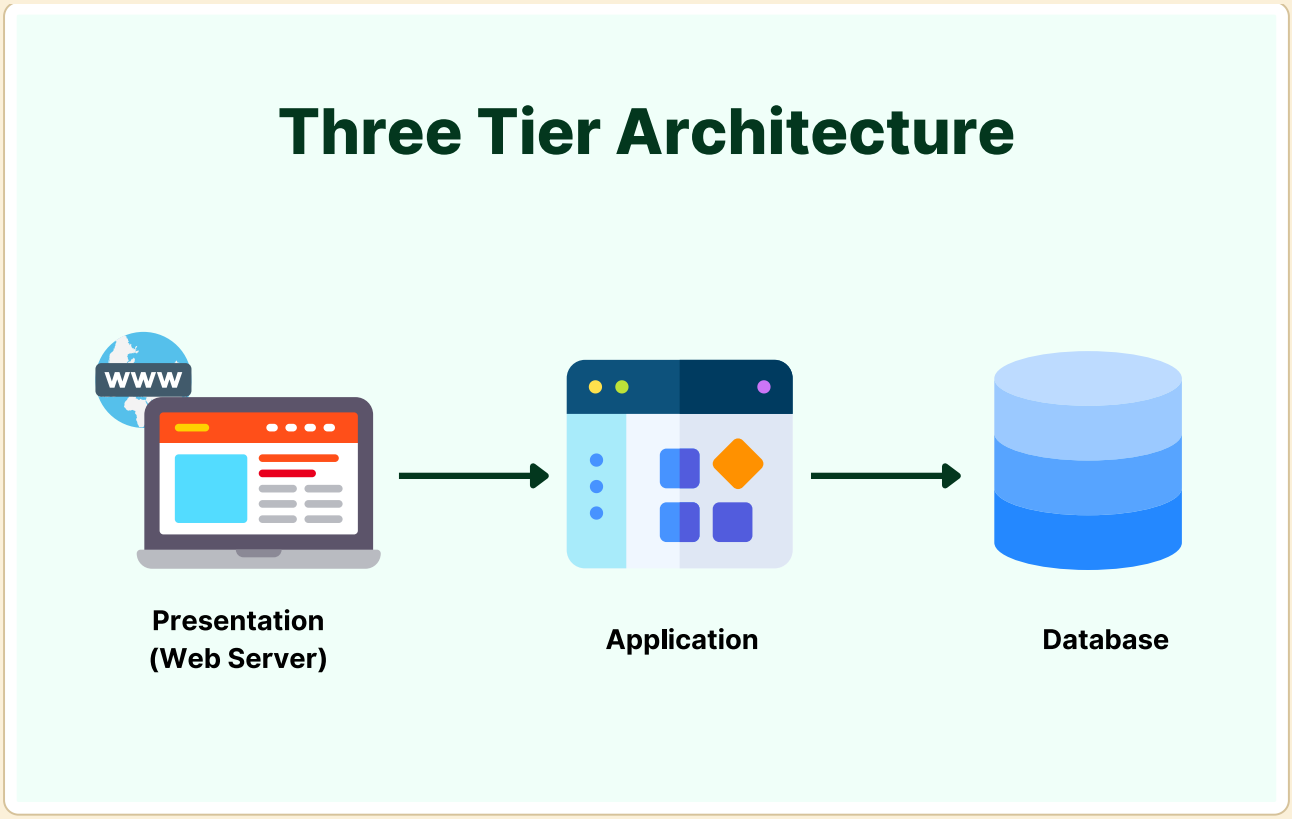


Figure — Three-tier: presentation, application (business logic) and data layers (course diagram).

Tier	Job	Example
Presentation	User interface	browser page or mobile app
Application (middle tier)	Business rules, validation, coordination, connection pooling (reusing a few open connections for many users)	a Java/Python/Node web service
Data	Storage and queries	PostgreSQL

- **Good:** tiers scale separately (more app servers behind a **load balancer** — a component that spreads incoming requests across several servers); rules change in one place; the database stays off the internet. **Bad:** more moving parts, a little **latency** (delay, the time a request takes) per hop (each network trip between tiers). Both terms are explained further in System Design Fundamentals.
- **Typical use:** almost every web application — shops, ERP (enterprise resource planning) and CRM (customer relationship management) systems.

N-tier

Split further into specialised layers: an authentication service (checks who a user is), a payment service, a cache tier (e.g. Redis — a fast in-memory store that keeps copies of hot data), a search tier, message queues (buffers where one service drops work for another to pick up later).

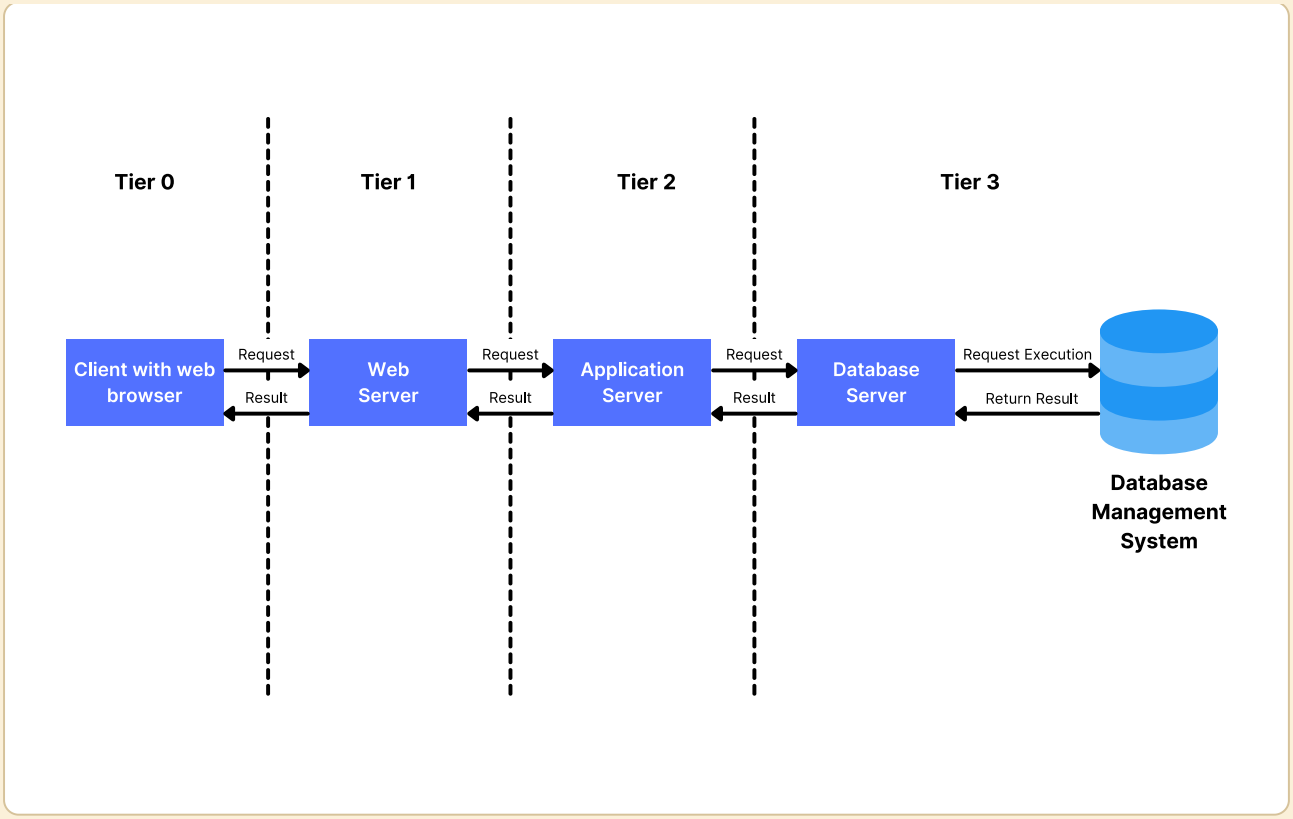


Figure — N-tier: extra service, integration and caching layers (course diagram).

- **Good:** layers scale, deploy and fail independently. **Bad:** harder to operate and debug; more network hops. Microservices (an app split into many small, independently deployed services) are an extreme form.

Cloud-hosted databases

The database runs on a cloud provider's infrastructure. Three service models:

Model	You manage	Provider manages	Example
IaaS (Infrastructure as a Service)	OS, DBMS install, backups, upgrades	hardware, virtual machines (software-simulated computers)	PostgreSQL you install on an AWS EC2 or Azure VM
PaaS / DBaaS (Platform / Database as a Service)	schema, queries, some settings	install, patching, backups, failover (switching to a standby copy when the main one dies)	Amazon RDS, Google Cloud SQL, Azure Database for PostgreSQL
SaaS (Software as a Service)	only your data inside the app	everything, including the database	Salesforce, Google Workspace

- **Good:** elastic capacity (grow or shrink on demand), less operational work, global reach. **Bad:** depends on the network; cost can grow quietly; data location must satisfy compliance (data-residency) rules.

Distributed databases

Data is spread over several machines, often in several regions. Each machine in such a group is called a **node**.

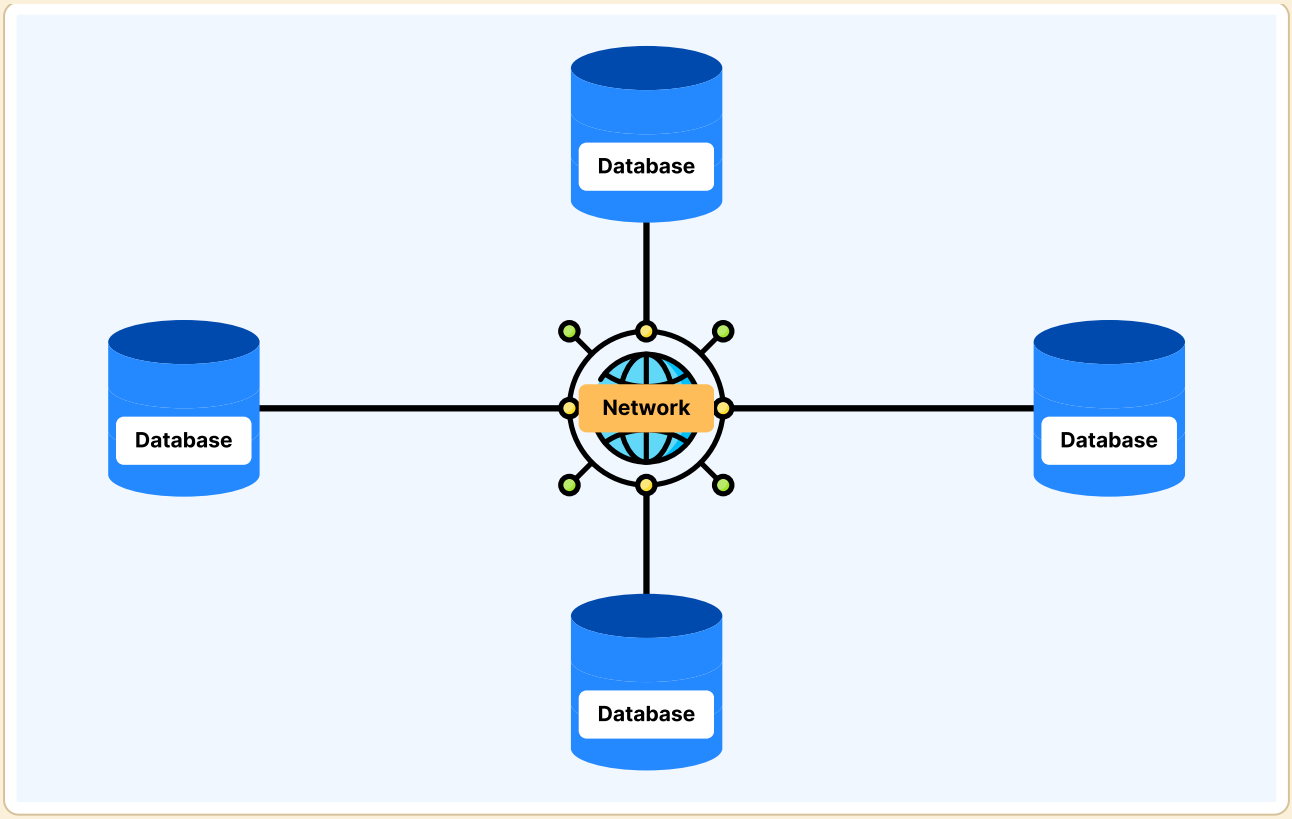


Figure — A distributed database: nodes in different locations, data partitioned and replicated (course diagram).

- **Partitioning / sharding** — split data into pieces, each on a different node (e.g. customers A–M on node 1, N–Z on node 2).
- **Replication** — keep copies (**replicas**) of the same data on several nodes for availability (staying up when a machine fails) and read scaling (spreading reads over the copies). See System Design Fundamentals for more on sharding and replication.
- **Consistency models** — how quickly all copies agree, from **strong** (every read sees the latest write) to **eventual** (copies converge after a while).
- **Good:** scales past one machine, survives node failures, puts data near users. **Bad:** much harder to reason about; cross-node queries and transactions pay network costs.

All of this is chapter 15 (Scaling databases); see also the DDIA notes (DDIA = the book *Designing Data-Intensive Applications*; `../../../ddia/`).

Self-check

1. *Why is exposing the database directly to thousands of browser clients a bad idea?* → Too many connections, credentials on clients, and business rules duplicated in every client; a middle tier pools connections and centralises rules.
2. *Amazon RDS is which cloud model?* → PaaS / managed database service.
3. *Partitioning vs replication?* → Partitioning splits different data across nodes; replication copies the same data to several nodes.

1.6 The relational model

What it is

The **relational model** stores *all* data in tables and connects tables using **values they share**, not pointers or file addresses. Codd proposed it in 1970 because the navigational databases of the day forced programs to follow stored pointers, so reorganising data meant rewriting programs. His proposal:

represent everything as plain values in tables and leave the retrieval strategy to the system — the query states the goal, the DBMS finds the route.

A common misconception: "relational" does **not** come from relationships between tables. It comes from **relation**, the mathematical word for a table (a set of tuples).

The vocabulary

Formal term	Everyday term	Meaning
Relation	table	a named set of rows that all have the same columns
Tuple	row, record	one item — one employee, one order
Attribute	column, field	one property — a name, a price
Domain	data type (+ rules)	the allowed values for a column (a *data type* is the kind of value: integer, text, date…)
Degree	number of columns	`departments` has degree 4
Cardinality	number of rows	`departments` has cardinality 6
Schema	table/database definition	the list of tables, columns, types and constraints

(Careful: "cardinality" also has a second meaning in ER modelling — one-to-many, many-to-many — covered in chapter 10.)

The four rules every table obeys

1. **Same columns in every row.** No row has extra private fields. A missing value is stored as `NULL`, meaning "unknown" or "not applicable" (chapter 2 treats `NULL` in depth).
2. **One value per cell (atomic values).** Not a list, not a nested record. This is the heart of *First Normal Form* (chapter 11).
3. **Row order means nothing.** The table is a set; if you need an order, store it in a column and ask for it with `ORDER BY`.
4. **No duplicate rows — in theory.** A mathematical set cannot contain the same element twice. SQL is looser: a table without a primary key or unique constraint will happily store identical rows, which is why good designs always declare one.

Keys and referential integrity

- A **primary key (PK)** is a column (or set of columns) whose value identifies exactly one row; it may not repeat and may not be `NULL`. Example: `customers.customer_id`.
- A **foreign key (FK)** is a column whose values must match a primary (or unique) key in another table. Example: `orders.customer_id` → `customers.customer_id`.
- **Referential integrity** is the DBMS rule that every foreign-key value points at a real row. Inserting an order for a customer who doesn't exist is rejected. Deleting a customer who still has orders is either refused or **cascades** (automatically repeats the delete on the dependent rows) to delete the orders too, depending on the `ON DELETE` rule chosen when the table was created (chapter 9).
- **Entity integrity** is the rule that a primary key is never `NULL`.

Here the sample database enforces all of these (errors are deliberate):

```sql
-- code/04_integrity.sql
INSERT INTO orders (order_id, customer_id, placed_at, status)
VALUES (200, 99, '2025-07-01 10:00', 'pending');          -- no customer 99
DELETE FROM customers WHERE customer_id = 1;               -- customer 1 has orders
INSERT INTO orders (order_id, customer_id, placed_at, status)
VALUES (201, 1, '2025-07-01 10:00', 'lost');              -- not an allowed status
INSERT INTO departments VALUES (1, 'Legal', 'Pune', 1000);  -- duplicate key
INSERT INTO departments VALUES (NULL, 'Legal', 'Pune', 1000);
SELECT count(*) AS orders_still FROM orders;
```

Output:

```
ERROR:  insert or update on table "orders" violates foreign key constraint "orders_customer_id_fkey"
DETAIL:  Key (customer_id)=(99) is not present in table "customers".
ERROR:  update or delete on table "customers" violates foreign key constraint "orders_customer_id_fkey"
on table "orders"
DETAIL:  Key (customer_id)=(1) is still referenced from table "orders".
ERROR:  new row for relation "orders" violates check constraint "orders_status_check"
DETAIL:  Failing row contains (201, 1, null, 2025-07-01 10:00:00, lost, null).
ERROR:  duplicate key value violates unique constraint "departments_pkey"
DETAIL:  Key (dept_id)=(1) already exists.
ERROR:  null value in column "dept_id" of relation "departments" violates not-null constraint
DETAIL:  Failing row contains (null, Legal, Pune, 1000.00).
 orders_still
--------------
           14
(1 row)
```

No application code checked anything — the rules live in the schema, so they hold for every program that connects.

Connecting tables with a join

A **join** combines rows from two tables whose shared values match. Joins are the everyday workhorse of relational querying (chapter 5 covers every kind). Which products did customer 1 buy?

```sql
-- code/01_first_queries.sql (part)
SELECT c.first_name, c.last_name, o.order_id, p.name AS product
FROM customers   c
JOIN orders      o  ON o.customer_id = c.customer_id
JOIN order_items oi ON oi.order_id   = o.order_id
JOIN products    p  ON p.product_id  = oi.product_id
WHERE c.customer_id = 1
ORDER BY o.order_id, p.name;
```

Output:

```
 first_name | last_name | order_id |       product
------------+-----------+----------+---------------------
 Anita      | Shah      |      101 | Laptop Pro 14
 Anita      | Shah      |      101 | Wireless Mouse
 Anita      | Shah      |      104 | Ergonomic Chair
 Anita      | Shah      |      104 | Standing Desk
 Anita      | Shah      |      113 | Laptop Pro 14
 Anita      | Shah      |      113 | Mechanical Keyboard
(6 rows)
```

Nothing in the tables "points" anywhere: the value `1` in `orders.customer_id` and the value `1` in `customers.customer_id` are the whole connection.

Because queries name only tables and columns, the model gives data independence and lets the optimizer choose (and later improve) the method; because it is mathematical, it also gives exact tests for design flaws — the normal forms of chapter 11.

Common mistakes

- Relying on "the order rows came back in" without `ORDER BY` — it can change at any time.
- Storing lists in one cell (`'red,green,blue'`), which breaks rule 2 and makes querying painful.
- Tables without a primary key, which lets exact duplicates creep in.

In the interview

> **30-second answer:** "The relational model, from Codd in 1970, stores data as relations — tables of tuples with typed attributes — and links them only by values: a foreign-key column stores the key of a row in another table, and the DBMS guarantees that row exists. Rows are unordered and cells hold single values. Because queries are declarative and never mention storage, the DBMS can optimise and reorganise freely."

- *Where does the word 'relational' come from?* → From *relation*, the mathematical name for a table — not from relationships.
- *What happens if you delete a parent row that has children?* → Depends on the FK's `ON DELETE` action: `NO ACTION`/`RESTRICT` (error, the default), `CASCADE`, `SET NULL` or `SET DEFAULT`.

Self-check

1. *Degree and cardinality of `products`?* → Degree 5 (columns), cardinality 9 (rows).
2. *Why is a primary key not allowed to be NULL?* → It must identify a row; an unknown identifier identifies nothing (entity integrity).
3. *Which rule does storing `'555-1234, 555-9876'` in one phone column break?* → One value per cell (First Normal Form).

1.7 RDBMS — relational database management systems

What it is

An **RDBMS** is a DBMS that implements the relational model and speaks SQL. Its defining features:

Feature	What it gives you
Tables of rows and columns	one uniform structure for all data
Primary and foreign keys	identity and enforced relationships
SQL	one standard language to define, change, query and secure data
Transactions with **ACID** guarantees	Atomicity, Consistency, Isolation, Durability (defined in 1.8, in depth in chapter 13)
Integrity constraints	types, `NOT NULL`, `UNIQUE`, `CHECK`, keys — bad data is refused at the door
Separation of logical and physical	data independence (1.3)

Advantages: the same tables answer countless questions without being reorganised, SQL is far shorter than file-access code, ACID and constraints keep data correct, and modern products handle large volumes and many concurrent users.

The main products

Product	Licence	Known for	Notes for this book
PostgreSQL	open source (PostgreSQL licence)	standards compliance, correctness, extensibility (JSONB = stored JSON documents, PostGIS = maps and geography, custom types)	this book's main dialect
MySQL (and fork MariaDB — a copy that became its own project)	open source (GPL, a licence that keeps derived code open) + commercial	ubiquity on the web, simple operations	the SQL course's original dialect; differences noted per chapter
Oracle Database	commercial	enterprise features, PL/SQL (Oracle's procedural programming language)	mentioned where syntax differs notably
Microsoft SQL Server	commercial	Microsoft ecosystem integration, T-SQL (Transact-SQL, its SQL dialect)	differences noted in small tables
SQLite	public domain	serverless (no separate server program): a whole database in one file, embedded in apps, phones and browsers	`*.sqlite.sql` examples run in `sqlite3`

Self-check

1. *What makes SQLite different from the others?* → No separate server process; a library that stores the whole database in a single file.

1.8 Relational vs non-relational databases

What it is

Non-relational databases store data in shapes other than tables. The umbrella name is **NoSQL**; people now tend to expand it as "**not only SQL**", since many such products later gained SQL-flavoured query languages. Neither family is "better": the shape of the data decides which questions are cheap.

Picture a food-delivery app. Orders and payments fit tables perfectly. Restaurant menus do not: one restaurant has thirty dishes with sizes and toppings, another four fixed meals, a third rewrites its menu weekly. That is the moment teams consider a second kind of store.

The four NoSQL families

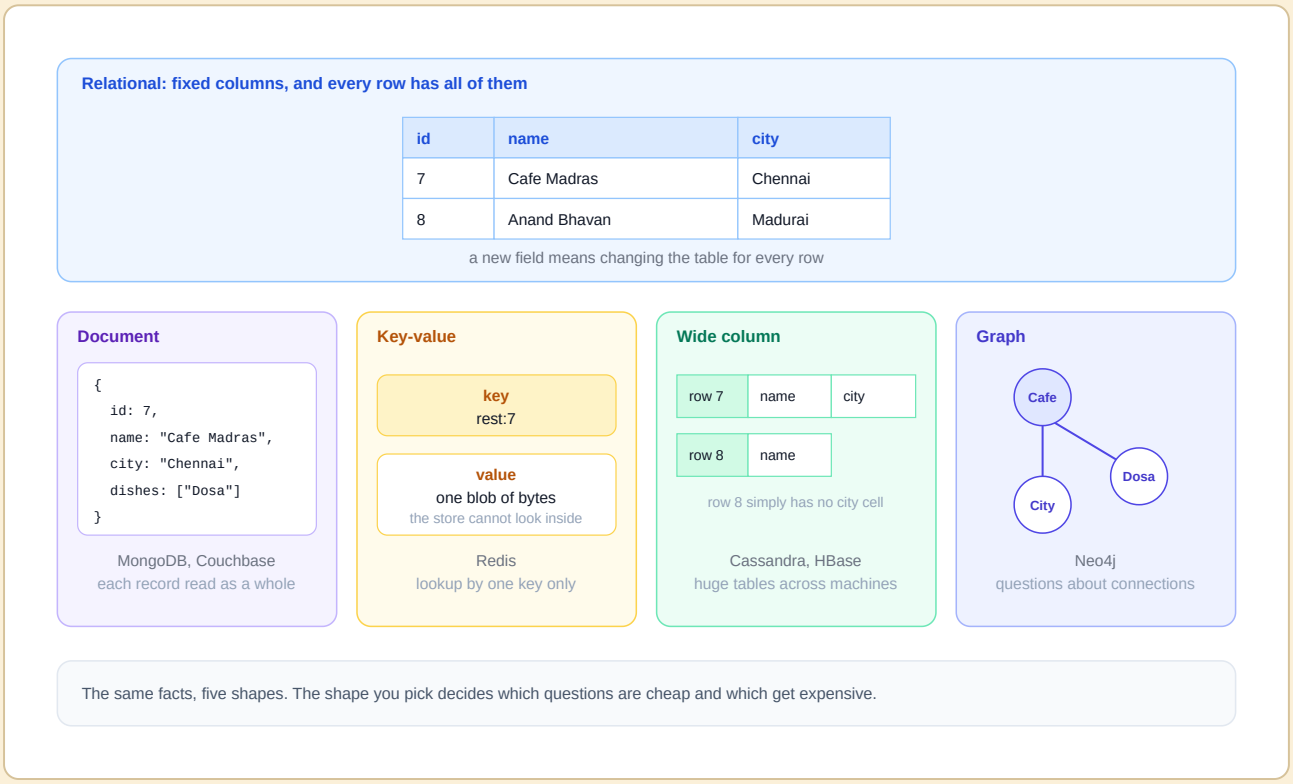


Figure — One dataset, four non-relational shapes, next to the relational version (course diagram).

Family	Data shape	Strength	Weakness	Examples	Typical use
Document	self-contained JSON/BSON (or XML) documents (JSON = text of nested `{"key": value}` pairs; BSON = binary JSON; XML = tag-based text); documents in one collection may differ in fields	reading/writing one whole object; varied shapes	cross-document joins are limited	MongoDB, Couchbase, CouchDB	catalogs, content, user profiles, menus
Key-value	a value stored under a key; mainly `put` and `get`	extremely fast lookups by key	cannot search inside values	Redis, Amazon DynamoDB, Riak	caches, sessions (a logged-in user's temporary state), feature flags (on/off switches for app features)
Wide-column (column-family)	rows grouped into column families; rows may have different columns; partitioned by key	very high write throughput (writes per second) across many machines	queries must follow the partition key (the column that decides which machine holds a row)	Apache Cassandra, HBase	time series (values recorded over time), event logs, IoT (Internet of Things — sensors and devices)
Graph	nodes, edges and properties	multi-hop relationship questions ("friends of friends who…")	not built for bulk aggregation	Neo4j, Amazon Neptune	social networks, recommendations, fraud detection

Chapter 16 (Beyond relational) builds each of these in practice.

ACID vs BASE

ACID — the guarantees of a relational transaction:

Letter	Guarantee
Atomicity	all operations in the transaction happen, or none do
Consistency	after a transaction finishes, every declared rule (keys, checks, constraints) is still satisfied
Isolation	transactions running side by side behave as if they ran alone; half-done changes stay invisible to others
Durability	once committed, changes survive crashes and power loss

BASE — how many distributed NoSQL systems describe themselves:

Letter(s)	Meaning
Basically Available	requests still get answers while some nodes are failing, though the data returned may be stale (out of date)
Soft state	stored state may keep changing without new writes as replicas catch up
Eventual consistency	if writes stop, all replicas converge; until then two readers may see different values

BASE gives up immediate agreement to stay available — acceptable for a product catalogue, unacceptable for an account balance.

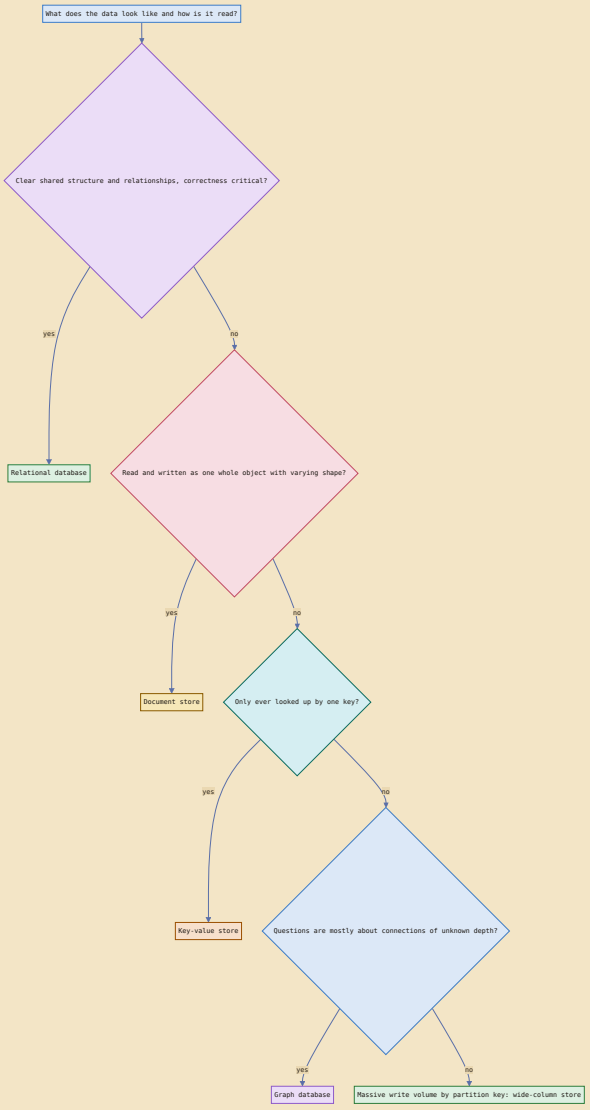
Three claims that need care

1. **"Relational databases only scale vertically."** *False.* **Vertical scaling** = a bigger machine; **horizontal scaling** = more machines. Relational systems scale out with read replicas (read-only copies that share the query load), partitioning/sharding and distributed SQL (CockroachDB, Google Spanner, YugabyteDB, Citus for PostgreSQL). The real issue is *cost*: cross-machine joins and transactions are expensive, and NoSQL models avoid them by design.
2. **"NoSQL databases don't support transactions."** *Out of date.* MongoDB has had multi-document ACID transactions since version 4.0 (2018); DynamoDB added transactions in 2018; Cassandra offers lightweight (compare-and-set: "write only if the value is still what I last saw") transactions per partition. Limits and costs vary, so check the specific product.
3. **"NoSQL databases have no schema."** *Misleading.* The database doesn't *enforce* one, but your code still expects `price` to be a number — the schema has moved into the application. The names: **schema-on-write** (the database checks data as it's written — relational) vs **schema-on-read** (the reader interprets whatever is stored). Schema-on-read makes change cheap and bad data easy.

Comparison

Aspect	Relational	Non-relational
Schema	checked by the DBMS as rows are written (schema-on-write)	mostly checked by application code when reading (schema-on-read)
Query language	SQL, standardised	product-specific (MongoDB's query API, Cassandra's CQL, Neo4j's Cypher…), some SQL-like
Joins	built in, optimised	few or none; applications join in code or embed related data instead
Transactions	full ACID across tables	available in many products, often scoped (one document / one partition) or slower
Best fit	structured, related data where correctness matters (orders, payments, bookings, inventory, ERP, CRM)	huge volumes, varied/changing record shapes, one dominant access pattern (catalogues, caches, feeds, telemetry, real-time analytics)

How to choose



Real systems often combine several: orders and payments in PostgreSQL, menus in a document store, sessions in Redis. Using different stores for different jobs is **polyglot persistence**. It is not free — every extra store must be run, backed up, monitored and kept in step with the others — so start with one

database (a relational one is the safe default) and add another only when a concrete requirement demands it.

In the interview

> **30-second answer:** "I pick from the access pattern, not the fashion. Orders need multi-table transactions and constraints, so relational. Menus differ per restaurant and are fetched as one unit, which suits a document store. Sessions are key lookups, so Redis. Relational databases do scale horizontally — replicas, sharding, distributed SQL — the real cost is cross-node joins and transactions."

- *Why not one database for everything?* → Often you should! Each extra store adds operational cost and consistency problems between stores; add one only when a requirement justifies it.

Self-check

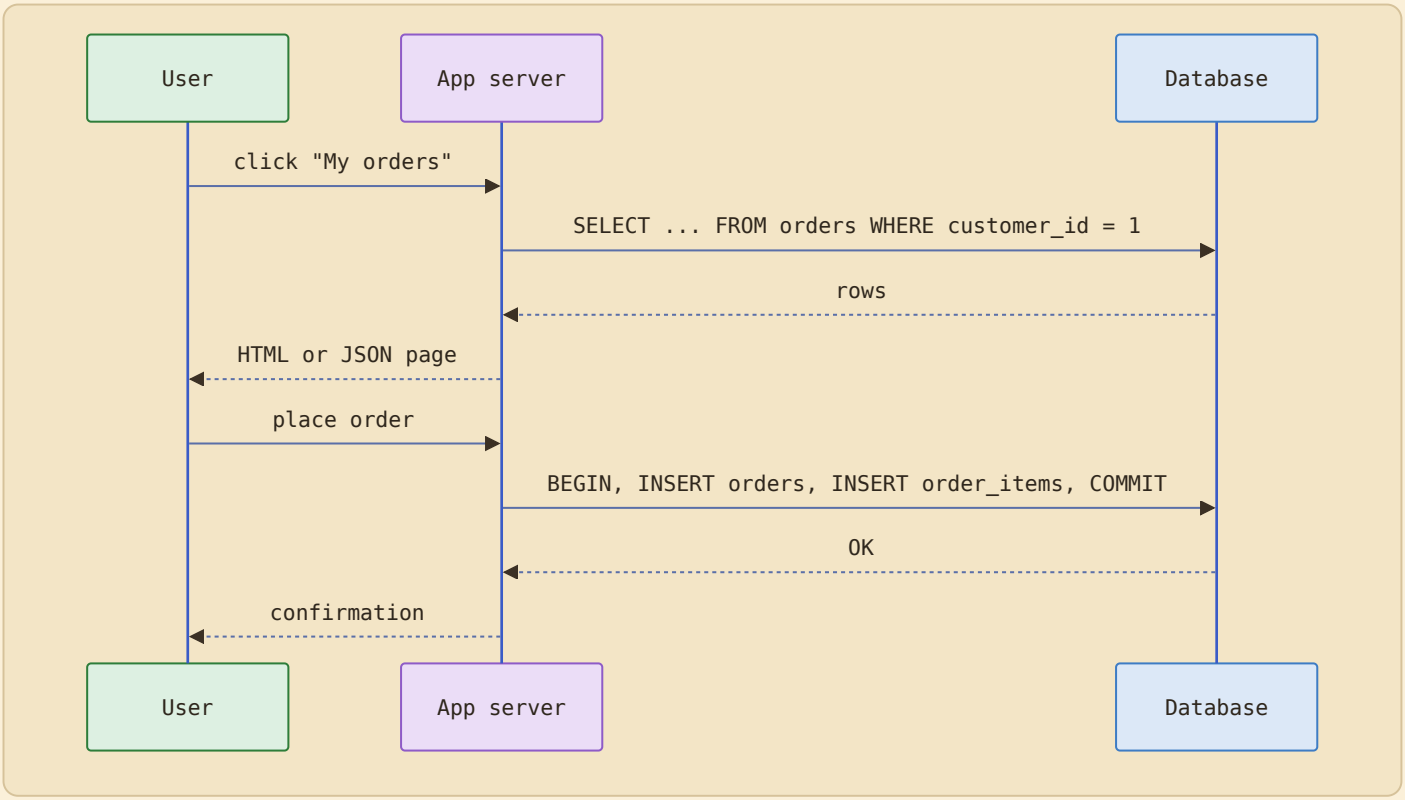
1. *A feature-flag service reads flags by name millions of times per second. Which family?* → Key-value.
2. *"Who are the friends of my friends who also like jazz?" Which family?* → Graph.
3. *Horizontal vs vertical scaling?* → More machines vs a bigger machine.

1.9 What SQL is

What it is

SQL (Structured Query Language) is the standard language for defining, changing, querying and securing data in relational databases. It is **declarative**: you describe the result you want, not the steps to compute it. Developed at IBM in the 1970s as SEQUEL (renamed SQL for trademark reasons), it became an ANSI standard in 1986 and an ISO standard in 1987; the current edition is **SQL:2023**. Every RDBMS implements the standard's core plus its own extensions — a **dialect** (PostgreSQL's, MySQL's, T-SQL for SQL Server, PL/SQL-flavoured Oracle SQL).

SQL is the traffic between applications and databases:



The five sub-languages

SQL statements are traditionally grouped into five families. (Some texts fold DQL into DML; interviewers accept either, but know both.)

Sub-language	Purpose	Main statements
DDL — Data Definition Language	define and change *structure*	`CREATE`, `ALTER`, `DROP`, `TRUNCATE`, `COMMENT`, `RENAME` (via `ALTER`)
DML — Data Manipulation Language	change *data*	`INSERT`, `UPDATE`, `DELETE`, `MERGE` (PostgreSQL 15+), `COPY` (PostgreSQL)
DQL — Data Query Language	read data	`SELECT`
DCL — Data Control Language	control *access*	`GRANT`, `REVOKE` (and role management such as `CREATE ROLE`)
TCL — Transaction Control Language	group changes into transactions	`BEGIN` / `START TRANSACTION`, `COMMIT`, `ROLLBACK`, `SAVEPOINT`, `ROLLBACK TO SAVEPOINT`, `RELEASE SAVEPOINT`

A frequent mix-up: **granting and revoking permissions is DCL, not TCL.** TCL is only about transaction boundaries (commit, rollback, savepoints).

How it works — one statement of each kind

```sql
-- code/03_sql_sublanguages.sql
-- DDL: define structure
CREATE TABLE suppliers (
    supplier_id integer PRIMARY KEY,
    name        text NOT NULL,
    country     text
);

-- DML: change data
INSERT INTO suppliers VALUES (1, 'Deskworks', 'India'), (2, 'Clicky Ltd', 'UK');
UPDATE suppliers SET country = 'Ireland' WHERE supplier_id = 2;

-- DQL: read data
SELECT * FROM suppliers ORDER BY supplier_id;

-- DCL: control who may do what
CREATE ROLE sql01_report_reader;
GRANT SELECT ON suppliers TO sql01_report_reader;
SELECT grantee, privilege_type
FROM information_schema.role_table_grants
WHERE table_name = 'suppliers' AND grantee = 'sql01_report_reader';
REVOKE SELECT ON suppliers FROM sql01_report_reader;

-- TCL: group changes into a transaction, with a savepoint
BEGIN;
INSERT INTO suppliers VALUES (3, 'Chairmakers', 'India');
SAVEPOINT before_mistake;
DELETE FROM suppliers;                -- oops: no WHERE clause
ROLLBACK TO SAVEPOINT before_mistake;  -- undo only the delete
COMMIT;
SELECT * FROM suppliers ORDER BY supplier_id;
```

Output (psql echoes a *command tag* such as `INSERT 0 2` after each change — `0` is a legacy OID (object identifier) field, `2` is the number of rows):

```
CREATE TABLE
INSERT 0 2
UPDATE 1
 supplier_id |    name    | country
-------------+------------+---------
           1 | Deskworks  | India
           2 | Clicky Ltd | Ireland
(2 rows)

CREATE ROLE
GRANT
       grantee       | privilege_type
---------------------+----------------
 sql01_report_reader | SELECT
(1 row)

REVOKE
BEGIN
INSERT 0 1
SAVEPOINT
DELETE 3
ROLLBACK
COMMIT
 supplier_id |    name     | country
-------------+-------------+---------
           1 | Deskworks   | India
           2 | Clicky Ltd  | Ireland
           3 | Chairmakers | India
(3 rows)
```

The `DELETE 3` wiped every row, but `ROLLBACK TO SAVEPOINT` undid just that step; the insert before the savepoint survived the `COMMIT`.

SQL syntax basics you'll use from chapter 2

Rule	Detail
Statements end with `;`	psql waits for the semicolon before running anything
Keywords are case-insensitive	`select` = `SELECT`; this book capitalises keywords for readability
Unquoted identifiers fold to lower case in PostgreSQL	`Employees` means `employees`; `"Employees"` (double quotes) keeps the case exactly
Strings use single quotes	`'Mumbai'`; a quote inside a string is doubled: `'O''Brien'`
Comments	`-- to end of line` and `/* block */`
Whitespace is free	break long queries over lines

Dialect differences at a glance

Feature	PostgreSQL	MySQL	SQLite	SQL Server
Limit rows	`LIMIT n` / `FETCH FIRST n ROWS ONLY`	`LIMIT n`	`LIMIT n`	`TOP n` / `OFFSET … FETCH`
Identifier quotes	`"name"`	`` `name` `` (or `"` in ANSI mode)	`"name"`	`[name]` or `"name"`
String concat	`||`, `concat()`	`concat()` (`||` is OR by default)	`||`	`+`, `concat()`
Auto-increment	`GENERATED … AS IDENTITY` / `serial`	`AUTO_INCREMENT`	`INTEGER PRIMARY KEY`	`IDENTITY(1,1)`
DDL inside transactions	yes, can be rolled back	no (implicit commit)	yes	mostly yes

Self-check

1. *Classify:* *`TRUNCATE`*, *`GRANT`*, *`SAVEPOINT`*, *`MERGE`*, *`SELECT`*. → DDL, DCL, TCL, DML, DQL.
2. *What does "declarative" mean for SQL?* → You state the result; the DBMS decides the algorithm.
3. *Why did `ROLLBACK TO SAVEPOINT` not remove supplier 3?* → The insert happened before the savepoint; only work after the savepoint is undone.

TL;DR

SQL is the declarative, standardised language of relational databases, split into DDL (structure), DML (changes), DQL (queries), DCL (permissions) and TCL (transactions). Each product speaks a dialect; this book uses PostgreSQL's.

1.10 Your private PostgreSQL server and psql

What it is

The book ships a small script, `tools_pg.sh`, in the project root. It runs a **private PostgreSQL 14 server** that belongs only to this book: its data lives in `~/.pg-books-system-design/` (outside the iCloud-synced Documents folder), it listens on port **5544** (a *port* is a numbered network door a server waits on) through a Unix socket in `/tmp` (a special file programs on the same machine use to talk to each other), and it never touches any other PostgreSQL you may have installed (which normally uses port 5432). **psql** is PostgreSQL's command-line client (a text-only program run in a terminal): you type SQL, it prints results.

How it works — step by step

Run these from the project root (`system_design_notes/`).

1. **Start the server** (the first run also creates the data directory):

```
bash tools_pg.sh start
```

```
postgres ready on port 5544 (socket /tmp)
```

2. **Create a database** for the sample data:

```
bash tools_pg.sh psql -d postgres -c "CREATE DATABASE company ENCODING 'UTF8' TEMPLATE template0"
```

```
CREATE DATABASE
```

An **encoding** is the rule for turning characters into bytes; **UTF-8** is the universal one that covers every language. Asking for `UTF8` explicitly guarantees that accented and non-Latin text (`café`, `मुंबई`) is stored and measured correctly, whatever encoding the server was initialised with; `TEMPLATE template0` is required when the encoding differs from the default template's.

3. **Load the sample schema** (section 1.11):

```
bash tools_pg.sh psql -d company -q -f sql/chapters/01-databases-and-the-relational-model/code/00_schema.sql
```

4. **Check it**:

```
bash tools_pg.sh psql -d company -c '\conninfo' -c 'SELECT count(*) FROM employees'
```

```
You are connected to database "company" as user "postgres" via socket in "/tmp" at port "5544".
 count
-------
    15
(1 row)
```

5. **Open an interactive session**: `bash tools_pg.sh psql -d company`. The prompt `company=#` means "connected to `company` as a superuser" (an account allowed to do everything) (`=>` for ordinary users). If you press Enter before the `;`, the prompt changes to `company-#` — psql is waiting for the rest of the statement. Type `\q` to leave.
6. **Stop the server** when you're done: `bash tools_pg.sh stop`.

`tools_pg.sh psql` is a thin wrapper around `psql -h /tmp -p 5544 -U postgres -X`: `-h /tmp` uses the socket directory, `-p` the port, `-U` the user, and `-X` skips any personal `~/.psqlrc` so output looks the same for every reader.

psql meta-commands

Lines starting with a backslash are **meta-commands**: psql handles them itself instead of sending them to the server. They do not need a semicolon.

Command	Does
`\?` / `\h SELECT`	help on meta-commands / SQL syntax help for a statement
`\l`	list databases
`\c company`	connect to another database
`\conninfo`	show current connection
`\dt` / `\dt+`	list tables (with sizes)
`\d employees`	describe a table: columns, types, indexes, constraints, references
`\dn`, `\df`, `\dv`, `\di`, `\du`	schemas, functions, views, indexes, roles
`\x` (or `\x auto`)	expanded display: one field per line, good for wide rows
`\pset null '(null)'`	show NULLs visibly instead of as blanks (used throughout this book)
`\timing`	print how long each statement took
`\i file.sql`	run a file of SQL
`\e`	edit the last query in your `$EDITOR` (the environment variable naming your preferred text editor)
`\copy`	import/export CSV from your machine (client side)
`\q`	quit

Meta-commands also work inside script files:

```
-- code/05_psql_meta_commands.sql
\dt
\d departments
\x on
SELECT * FROM employees WHERE emp_id = 7;
\x off
```

Output (abridged):

```
               List of relations
 Schema |       Name        | Type  |  Owner
--------+-------------------+-------+----------
 public | customers         | table | postgres
 public | departments       | table | postgres
 ...
 public | projects          | table | postgres
(8 rows)

                Table "public.departments"
  Column  |     Type      | Collation | Nullable | Default
----------+---------------+-----------+----------+---------
 dept_id  | integer       |           | not null |
 name     | text          |           | not null |
 location | text          |           |          |
 budget   | numeric(12,2) |           | not null |
Indexes:
    "departments_pkey" PRIMARY KEY, btree (dept_id)
    "departments_name_key" UNIQUE CONSTRAINT, btree (name)
Check constraints:
    "departments_budget_check" CHECK (budget >= 0::numeric)
Referenced by:
    TABLE "employees" CONSTRAINT "employees_dept_id_fkey" FOREIGN KEY (dept_id) REFERENCES
departments(dept_id)
    TABLE "projects" CONSTRAINT "projects_dept_id_fkey" FOREIGN KEY (dept_id) REFERENCES
departments(dept_id)

Expanded display is on.
-[ RECORD 1 ]--+--------------------------
emp_id         | 7
first_name     | Priya
last_name      | Nair
email          | priya.nair@company.example
...
hire_date      | 2020-02-10

Expanded display is off.
```

Running the book's code

- Run **one file** against your `company` database: `bash tools_pg.sh psql -d company -f sql/chapters/02-select-basics/code/02_where.sql`. (Files that change data, like 1.3's demo, are best run on a fresh copy: re-run step 3 to reset.)
- Run **every chapter's code** the way the book was checked: `bash tools_run_sql.sh sql`. For each file it creates a fresh database, loads that chapter's `00_schema.sql`, runs the file, and prints `OK` or `FAIL`.

Common mistakes

- Forgetting the `;` — psql shows `company-#` and waits. Type `;` and Enter.
- Typing SQL at the shell prompt (`$`) instead of inside psql.
- Connecting to the wrong server: always go through `tools_pg.sh psql` (port 5544), not a bare `psql` (default port 5432).
- A typo inside a transaction puts it in an aborted state (`current transaction is aborted…`); type `ROLLBACK;` to start again.

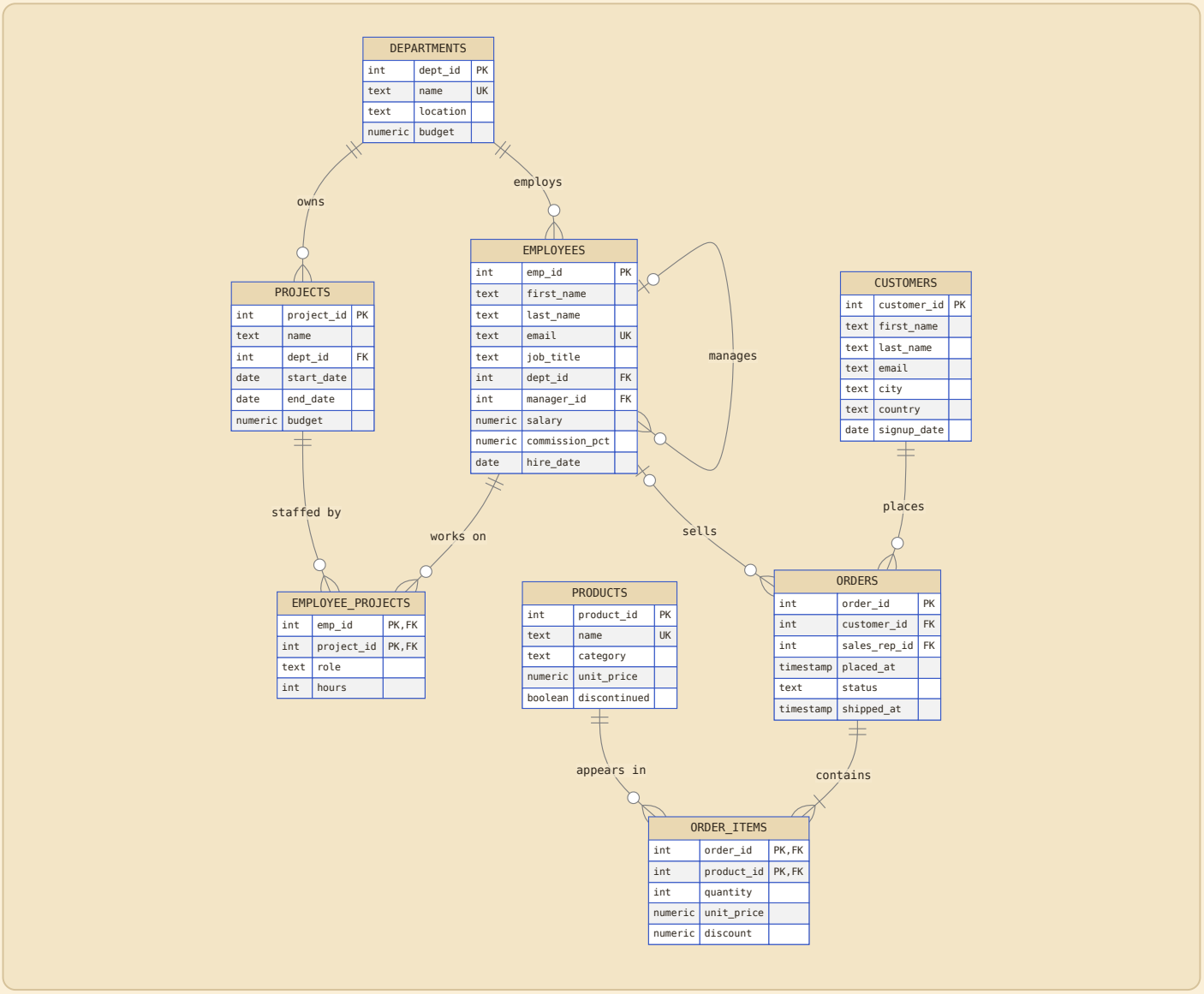
TL;DR

`bash tools_pg.sh start` → create the UTF-8 `company` database → load `00_schema.sql` → `bash tools_pg.sh psql -d company`. Use `\dt`, `\d table`, `\x`, `\pset null`, `\timing`, `\q`. `tools_run_sql.sh sql` runs all examples on fresh databases.

1.11 The sample database

What it is

Chapters 2 onward query one small, realistic dataset: **company**, a fictional firm that sells office equipment. It has staff organised in departments and projects, and a small shop side with customers, products and orders. Every chapter's `code/00_schema.sql` is a copy of `code/00_schema.sql`, so you can load it once and follow along.



(Columns that may be NULL are listed in the table below. How to read the crow's-foot symbols — `||` exactly one, `o{` zero or more, `|{` one or more, `|o` zero or one — is explained in chapter 10.)

The tables

Table	Rows	What a row is	Notable columns
`departments`	6	a department	`location` may be NULL, `budget`
`employees`	15	a member of staff	`manager_id` → another employee (a self-reference: a foreign key pointing at its own table), `commission_pct` only for some sales staff
`projects`	5	an internal project owned by a department	`end_date` NULL = still running
`employee_projects`	13	"employee X works on project Y" (a many-to-many link)	composite key (a key made of two or more columns together) `(emp_id, project_id)`, `role`, `hours`
`customers`	10	a shop customer	`city`, `email` may be NULL
`products`	9	a product for sale	`category` may be NULL, `discontinued`
`orders`	14	an order header	`placed_at`, `status` ∈ pending / shipped / delivered / cancelled, `shipped_at`, `sales_rep_id` NULL for online orders
`order_items`	22	one product line in an order	`quantity`, `unit_price` at time of sale, `discount` (0.10 = 10 %)

Deliberate quirks (so later chapters have something to find)

Quirk	Where	Useful for
CEO (Asha) and a new intern (Rohan) have no department	`employees.dept_id` NULL	NULL handling, outer joins
Research department has no employees; Support has no location	`departments`	outer joins (keep unmatched rows), anti-joins (find rows with *no* match), NULL sorting
Salary ties: Rahul & Meera 98,000; Arjun & Sara 68,000	`employees.salary`	`DISTINCT`, `WITH TIES`, ranking functions
Missing emails (Meera, Ananya) and commissions	`employees`	`IS NULL`, `COALESCE`, `count(col)`
One email stored in mixed case (`Neha.Gupta@Company.example`)	`employees.email`	case-insensitive matching
Anita Shah entered twice, second time with spaces and capitals in the email and city `mumbai`	`customers` 1 and 8	de-duplication, `trim`, `lower`
Customers 9 and 10 never ordered; product "Desk Lamp" never sold; "Gift Card" has no category	`customers`, `products`	anti-joins, NULL groups
Orders across Jan–Jun 2025, one at 23:55 on 31 January	`orders.placed_at`	date ranges, month boundaries
Pending/cancelled orders have no `shipped_at`	`orders`	NULLs in date arithmetic

A first look:

```sql
SELECT * FROM departments;
```

Output:

```
 dept_id |    name     | location  |  budget
---------+-------------+-----------+-----------
       1 | Engineering | Bengaluru | 900000.00
       2 | Sales       | Mumbai    | 400000.00
       3 | Marketing   | Mumbai    | 250000.00
       4 | Finance     | Pune      | 200000.00
       5 | Support     | (null)    | 150000.00
       6 | Research    | Pune      | 300000.00
(6 rows)
```

Key terms (chapter 1)

Every term is defined where it is first used (in **bold**). For flash-card revision: file processing system, redundancy/inconsistency, lost update, metadata, catalog/data dictionary, DBMS, database system, data independence (physical/logical), buffer pool, page, WAL, transaction, ACID, BASE, relation/tuple/attribute/domain, degree, cardinality, primary key, foreign key, entity and referential integrity, schema-on-write/-on-read, horizontal/vertical scaling, polyglot persistence, DDL/DML/DQL/DCL/TCL, declarative. The book-wide glossary is in chapter 20.

Chapter 1 TL;DR

- File-based systems suffered seven problems (redundancy, isolation, no queries, rules in code, lost updates, no recovery, coarse security); the **DBMS** fixes them by mediating every access.
- A **database** is related, self-describing (catalog), purposeful, persistent data; the **DBMS** is the software that **defines, constructs, manipulates, shares, protects and maintains** it, with **physical and logical data independence**.
- Inside: **parser → optimizer → executor**, a **storage engine** with pages, a **buffer pool**, **transaction/lock managers**, a **recovery manager** with a write-ahead log, and a **catalog**.
- Architectures: two-tier, three-tier, n-tier, cloud (IaaS/PaaS/SaaS), distributed (partitioning + replication).
- The **relational model**: relations of tuples and attributes; same columns per row, atomic cells, no row order, no duplicates; **primary keys** identify, **foreign keys** link, the DBMS enforces **referential integrity**.
- NoSQL families (document, key-value, wide-column, graph) trade enforced structure and joins for flexible shapes and easier scale-out; choose by access pattern.
- **SQL** is declarative and standard: **DDL, DML, DQL, DCL, TCL**.
- Your lab: `bash tools_pg.sh start`, `bash tools_pg.sh psql -d company`; the `company` sample database powers the next chapters.

Next: Chapter 2 — SELECT basics.

Chapter 2 — SELECT Basics: Reading, Filtering, Sorting and Paging

Reading data is most of what anyone does with SQL. This chapter teaches the single-table `SELECT` statement completely: choosing columns, computing new ones, filtering rows with `WHERE`, the operators you can use there, the peculiar but essential rules of `NULL`, removing duplicates with `DISTINCT`, sorting with `ORDER BY`, and returning one page of results with `LIMIT`/`OFFSET` or the standard `FETCH`. Joins (combining tables), grouping and subqueries (queries nested inside another query) build on exactly these pieces in chapters 4–6.

What you'll learn

#	Topic	You will be able to…
2.1	`SELECT` and aliases	pick columns, compute expressions, rename outputs
2.2	`WHERE` and logical operators	filter rows and combine conditions without precedence bugs
2.3	Operators	use arithmetic, comparison, logical and string operators correctly (incl. integer division)
2.4	`BETWEEN`, `IN`, `LIKE`/`ILIKE`, regex	express ranges, lists and text patterns
2.5	`NULL` and three-valued logic	test for missing values and avoid the classic `NOT IN` trap
2.6	`DISTINCT` / `DISTINCT ON`	remove duplicates and keep one row per group
2.7	`ORDER BY`	sort by several keys, control where NULLs go, custom orders
2.8	`LIMIT`/`OFFSET`/`FETCH`	page through results, handle ties, prefer keyset paging
2.9	Logical processing order	know why an alias works in `ORDER BY` but not in `WHERE`

Setup. All examples use the `company` sample database from chapter 1, section 1.11. Files are in `code/`; each starts with `\pset null '(null)'` so missing values print as `(null)` instead of blanks. To follow along: `bash tools_pg.sh psql -d company`, then type `\pset null '(null)'`.

2.1 SELECT: choosing what comes back

What it is

`SELECT` reads rows from a table and returns a new, temporary table called the **result set**. Think of the table as a spreadsheet: `SELECT` chooses which columns you see, and (with `WHERE`, next section) which rows.

```
SELECT column1, column2, ...     -- what to show
FROM   table_name;               -- where to read it
```

How it works

1. `FROM` names the table to read.
2. The `SELECT` list says which columns (or computed values) to return, in which order.
3. `*` is a wildcard meaning "every column, in table order".

All columns:

```sql
-- code/01_select_columns.sql
SELECT * FROM products;
```

Output:

```
 product_id |            name             |  category   | unit_price | discontinued
------------+-----------------------------+-------------+------------+--------------
          1 | Laptop Pro 14               | Electronics |    1299.00 | f
          2 | Wireless Mouse              | Electronics |      24.99 | f
          3 | USB-C Hub                   | Accessories |      49.50 | f
          4 | Mechanical Keyboard         | Accessories |      89.00 | f
          5 | Standing Desk               | Furniture   |     499.00 | f
          6 | Ergonomic Chair             | Furniture   |     349.99 | f
          7 | Noise-Cancelling Headphones | Electronics |     199.95 | f
          8 | Desk Lamp                   | Furniture   |      35.00 | t
          9 | Gift Card                   | (null)      |      50.00 | f
(9 rows)
```

(`f`/`t` are how psql prints boolean false/true; a **boolean** is a true/false value.)

Chosen columns — list them after `SELECT`, in the order you want:

```sql
SELECT first_name, last_name, job_title FROM employees;
```

Output (first rows):

```
 first_name | last_name |      job_title
------------+-----------+---------------------
 Asha       | Rao       | CEO
 Vikram     | Menon     | Engineering Manager
 Neha       | Gupta     | Senior Engineer
 Rahul      | Verma     | Engineer
 ...
(15 rows)
```

Expressions and column aliases

The `SELECT` list can hold any **expression** — arithmetic, function calls (a *function* is a named built-in operation, such as `round(x, 2)`, that takes inputs and returns a value), constants — not just column names. An **alias** (`AS new_name`) names the output column. Use double quotes if the alias has spaces or capitals you want to keep.

```sql
SELECT name,
       unit_price,
       unit_price * 1.18        AS price_with_gst,
       'per unit'               AS note,
       unit_price * 1.18 AS "Price incl. GST"
FROM products
WHERE product_id <= 3;
```

Output:

```
      name      | unit_price | price_with_gst |   note   | Price incl. GST
----------------+------------+----------------+----------+-----------------
 Laptop Pro 14  |    1299.00 |      1532.8200 | per unit |       1532.8200
 Wireless Mouse |      24.99 |        29.4882 | per unit |         29.4882
 USB-C Hub      |      49.50 |        58.4100 | per unit |         58.4100
(3 rows)
```

(GST is India's 18 % goods-and-services sales tax, hence `* 1.18`.) The table is not changed — computed columns exist only in the result. (Rounding to 2 decimals is `round(x, 2)`, chapter 3.)

Table aliases give a table a short name, which you'll need constantly once queries involve several tables. The `AS` keyword is optional for both kinds of alias, but writing it for column aliases avoids confusing bugs (see *Common mistakes*).

```
SELECT e.first_name, e.salary / 12 monthly
FROM employees AS e
WHERE e.emp_id IN (1, 2);
```

Output:

```
 first_name |      monthly
------------+--------------------
 Asha       | 20833.333333333333
 Vikram     | 15416.666666666667
(2 rows)
```

SELECT without FROM — PostgreSQL (and MySQL, SQLite, SQL Server) let you evaluate expressions directly, which makes psql a handy calculator. Oracle required `FROM dual` (a built-in one-row dummy table) until version 23ai made `FROM` optional. Below, `now()` returns the current date and time, and `::date` **casts** it — converts it to another type, here a plain date.

```
SELECT 2 + 3 AS five, now()::date >= DATE '2025-01-01' AS after_2025;
```

Output:

```
 five | after_2025
------+------------
    5 | t
(1 row)
```

Common mistakes

- **`SELECT *` in application code.** It returns columns you don't need (more I/O — disk input/output — and network traffic), and code breaks or silently changes when someone adds a column. Name the columns. `SELECT *` is fine for exploring in psql.
- **A missing comma turns a column into an alias.** `SELECT first_name last_name FROM employees` returns *one* column named `last_name` containing first names. Writing `AS` for every alias makes this mistake visible.
- **Quoting aliases with single quotes.** `'total'` is a string literal (a fixed text value typed into the query); aliases use double quotes: `"Total"`.

Self-check

1. *What does `SELECT *` return for a table with 5 columns?* → All 5 columns in table-definition order.
2. *Does `SELECT unit_price * 2 FROM products` change prices?* → No; it only computes a value in the result set.
3. *How do you name an output column `Net Price` exactly?* → `AS "Net Price"`.

TL;DR

`SELECT cols FROM table`. Use `*` only to explore. Any expression can be a column; `AS alias` names it; table aliases shorten references.

2.2 WHERE: filtering rows

What it is

`WHERE condition` keeps only the rows for which the condition is **true**. It works the same way in `SELECT`, `UPDATE` and `DELETE` statements (chapter 9), so a filter you test with `SELECT` can be reused safely in a `DELETE`.

```
SELECT column1, column2
FROM   table_name
WHERE  condition;
```

How it works

For each row, the database evaluates the condition. The result is `TRUE`, `FALSE` or — when `NULL` is involved — `UNKNOWN` (section 2.5). Only `TRUE` rows pass.

```
-- code/02_where.sql
SELECT name, category, unit_price
FROM products
WHERE category = 'Electronics';
```

Output:

```
            name             |  category   | unit_price
-----------------------------+-------------+------------
 Laptop Pro 14               | Electronics |    1299.00
 Wireless Mouse              | Electronics |      24.99
 Noise-Cancelling Headphones | Electronics |     199.95
(3 rows)
```

Comparisons work on numbers, text and dates alike. Date literals are written as strings in ISO format `'YYYY-MM-DD'` (the international standard date layout: year, month, day), which PostgreSQL converts to the column's type:

```
SELECT first_name, hire_date FROM employees WHERE hire_date >= '2023-01-01' ORDER BY hire_date;
```

Output:

```
 first_name | hire_date
------------+------------
 Li         | 2023-02-14
 Arjun      | 2023-11-20
 Sara       | 2024-04-01
 Rohan      | 2025-06-02
(4 rows)
```

Combining conditions: AND, OR, NOT

Operator	True when	Example
`AND`	both sides are true	`dept_id = 2 AND salary > 70000`
`OR`	at least one side is true	`city = 'Delhi' OR city = 'Mumbai'`
`NOT`	the condition is false	`NOT discontinued`

Precedence: `NOT` binds tightest, then `AND`, then `OR` — just as `*` binds before `+`. So `a OR b AND c` means `a OR (b AND c)`. This silently changes results:

```
-- Intended: Engineering or Sales staff earning over 100k
SELECT first_name, dept_id, salary
FROM employees
WHERE dept_id = 1 OR dept_id = 2 AND salary > 100000
ORDER BY emp_id;
```

Output:

```
 first_name | dept_id |  salary
------------+---------+-----------
 Vikram     |       1 | 185000.00
 Neha       |       1 | 142000.00
 Rahul      |       1 |  98000.00
 Meera      |       1 |  98000.00
 Daniel     |       2 | 150000.00
(5 rows)
```

Rahul and Meera earn less than 100k but appear, because the query really says "*everyone* in dept 1, or dept-2 staff over 100k". Parentheses fix it:

```
SELECT first_name, dept_id, salary
FROM employees
WHERE (dept_id = 1 OR dept_id = 2) AND salary > 100000
ORDER BY emp_id;
```

Output:

```
 first_name | dept_id |  salary
------------+---------+-----------
 Vikram     |       1 | 185000.00
 Neha       |       1 | 142000.00
 Daniel     |       2 | 150000.00
(3 rows)
```

`NOT` negates the condition after it:

```
SELECT name FROM products WHERE NOT discontinued AND NOT unit_price < 100 ORDER BY name;
```

Output:

```
            name
----------------------------
 Ergonomic Chair
 Laptop Pro 14
 Noise-Cancelling Headphones
 Standing Desk
(4 rows)
```

Case sensitivity

In PostgreSQL, text comparison with `=` is **case-sensitive**: `'Mumbai' = 'mumbai'` is false. Our customers table has one `Mumbai` and one `mumbai`, and `WHERE city = 'Mumbai'` finds only one:

```
 exact_mumbai
--------------
            1
```

MySQL's default collations (the rules a database uses to compare and sort text, see 2.7) are case-*insensitive*, so the same query there returns 2 — a real portability trap (code that behaves differently when moved to another database). In PostgreSQL, compare `lower(city) = 'mumbai'` or use `ILIKE` (2.4).

Common mistakes

- Mixing `AND` and `OR` without parentheses. Rule of thumb: whenever both appear, add parentheses.
- Comparing with `= NULL` — never true; use `IS NULL` (2.5).
- Writing `"Mumbai"` (double quotes) for a string: PostgreSQL reads it as a column name and errors with *`column "Mumbai" does not exist`*.

Self-check

1. *How is `a AND b OR c` evaluated?* → `(a AND b) OR c`.
2. *Does `WHERE` also apply to `DELETE`?* → Yes; `DELETE FROM t WHERE …` removes only matching rows (and without `WHERE`, all rows).
3. *Why does `WHERE city = 'Mumbai'` miss `'mumbai'` in PostgreSQL?* → String equality is case-sensitive.

TL;DR

`WHERE` keeps rows whose condition is TRUE. `NOT` > `AND` > `OR` in precedence — parenthesise mixed conditions. Text equality is case-sensitive in PostgreSQL.

2.3 Operators

What it is

An **operator** is a symbol or keyword that combines one or more values (its **operands**) into a new value: `+`, `=`, `AND`, `||`.

Arithmetic operators

Operator	Meaning	Example	Result
`+`	add	`5 + 3`	8
`-`	subtract	`10 - 4`	6
`*`	multiply	`3 * 7`	21
`/`	divide	`20 / 5`	4
`%`	remainder (modulo)	`17 % 3`	2
`^`	power (PostgreSQL)	`2 ^ 10`	1024
`|/`, `@`	square root, absolute value (PostgreSQL)	`|/ 25`, `@ -5`	5, 5

```sql
-- code/08_operators.sql
SELECT 5 + 3 AS add, 10 - 4 AS sub, 3 * 7 AS mul, 20 / 5 AS div, 17 % 3 AS modulo;
SELECT 7 / 2 AS int_div, 7 / 2.0 AS num_div, 7::numeric / 2 AS cast_div, -7 / 2 AS neg_int_div;
SELECT 2 ^ 10 AS power, |/ 25 AS sqrt, @ -5 AS abs;
SELECT 2 + 3 * 4 AS no_parens, (2 + 3) * 4 AS parens;
```

Output:

```
 add | sub | mul | div | modulo
-----+-----+-----+-----+--------
   8 |   6 |  21 |   4 |      2
(1 row)

 int_div |      num_div       |      cast_div      | neg_int_div
---------+--------------------+--------------------+-------------
       3 | 3.5000000000000000 | 3.5000000000000000 |          -3
(1 row)

 power | sqrt | abs
-------+------+-----
  1024 |    5 |   5
(1 row)

 no_parens | parens
-----------+--------
        14 |     20
(1 row)
```

Integer division truncates toward zero (simply drops the fractional part) in PostgreSQL, SQL Server and SQLite: `7 / 2` is `3`, and `-7 / 2` is `-3`. If either side is a decimal (`2.0`) or cast to `numeric` (PostgreSQL's exact-decimal number type), you get the fraction. MySQL is the odd one out: `/` always returns a decimal (`7 / 2` = `3.5000`), and `DIV` does integer division. This is one of the most common "my average is wrong" bugs when moving between databases.

Operators work on columns row by row — for example the value of each order line:

```sql
SELECT order_id, product_id, quantity, unit_price, discount,
       quantity * unit_price * (1 - discount) AS line_total
FROM order_items WHERE order_id IN (102, 104) ORDER BY order_id, product_id;
```

Output:

```
 order_id | product_id | quantity | unit_price | discount | line_total
----------+------------+----------+------------+----------+------------
      102 |          7 |        2 |     199.95 |     0.10 |   359.9100
      104 |          5 |        1 |     479.00 |     0.00 |   479.0000
      104 |          6 |        1 |     349.99 |     0.05 |   332.4905
(3 rows)
```

Comparison operators

Operator	Meaning	Example
`=`	equal	`unit_price = 49.50`
`<>` or `!=`	not equal (`<>` is standard)	`status <> 'cancelled'`
`<`, `>`, `<=`, `>=`	ordering	`salary >= 100000`
`BETWEEN a AND b`	inclusive range	2.4
`IN (…)`	membership in a list	2.4
`LIKE`, `ILIKE`, `~`	pattern match	2.4
`IS [NOT] NULL`, `IS [NOT] DISTINCT FROM`	NULL-aware tests	2.5

Logical operators

`AND`, `OR`, `NOT` — see 2.2 for precedence and 2.5 for how they behave with `NULL`.

String concatenation

`||` joins strings in PostgreSQL, SQLite and Oracle:

```
SELECT first_name || ' ' || last_name AS full_name FROM employees WHERE emp_id <= 3;
```

Output:

```
  full_name
--------------
 Asha Rao
 Vikram Menon
 Neha Gupta
(3 rows)
```

In MySQL `||` means logical OR by default; use `CONCAT()` there. SQL Server uses `+` or `CONCAT()`. Chapter 3 covers `concat()` and how each treats `NULL`.

Compound assignment operators

Some SQL dialects have shorthand assignment operators, e.g. SQL Server's T-SQL: `SET @x += 3` (add and assign), also `-=`, `*=`, `/=`, `%=`. They apply to **procedural variables** (named values inside step-by-step code such as a stored procedure, rather than columns — plus T-SQL's `UPDATE … SET col += 1`). They are **not** part of standard SQL and PostgreSQL does not support them in SQL statements — write `SET col = col + 1` in an `UPDATE`, or `x := x + 3` inside a PL/pgSQL function (PL/pgSQL is PostgreSQL's built-in programming language for functions; chapter 14).

Dialect	Add-and-assign
PostgreSQL	`UPDATE t SET n = n + 3`; PL/pgSQL `x := x + 3`
SQL Server	`SET @x += 3`; `UPDATE t SET n += 3`
MySQL	`SET @x = @x + 3` (no `+=`)

Operator precedence (highest first, simplified)

Level	Operators
1	`::` (cast)
2	unary `-` / `+`
3	`^`
4	`*`, `/`, `%`
5	binary `+`, `-`
6	other operators such as `||`
7	`BETWEEN`, `IN`, `LIKE`, `ILIKE`
8	`<`, `>`, `=`, `<=`, `>=`, `<>`
9	`IS NULL`, `IS DISTINCT FROM` …
10	`NOT`
11	`AND`
12	`OR`

(*Unary* operators take one operand, like the `-` in `-5`; *binary* ones take two, like `a - b`.) Because unary minus binds tighter than `^`, `SELECT -2 ^ 2` returns `4` in PostgreSQL, not `-4`. When in doubt, add parentheses — they cost nothing and document intent.

Self-check

1. `SELECT 9 / 4` *in PostgreSQL?* → `2` (integer division). `9 / 4.0` → `2.25`.
2. *How do you write "not equal" in standard SQL?* → `<>` (PostgreSQL also accepts `!=`).
3. *Why might* `first_name || ' ' || last_name` *return nothing useful in MySQL?* → `||` is logical OR there by default.

TL;DR

`+ - * / %` (and PostgreSQL's `^`), with integer division truncating. `<>` for not-equal. `||` concatenates in PostgreSQL. Compound `+=` is a T-SQL/procedural feature, not standard SQL.

2.4 Ranges, lists and patterns: BETWEEN, IN, LIKE/ILIKE, regex

BETWEEN — inclusive ranges

`x BETWEEN a AND b` means `x >= a AND x <= b`. **Both ends are included.**

```
-- code/03_between_in_like.sql
SELECT name, unit_price FROM products WHERE unit_price BETWEEN 49.50 AND 199.95 ORDER BY unit_price;
```

Output:

```
            name             | unit_price
-----------------------------+------------
 USB-C Hub                   |      49.50
 Gift Card                   |      50.00
 Mechanical Keyboard         |      89.00
 Noise-Cancelling Headphones |     199.95
(4 rows)
```

`NOT BETWEEN a AND b` is the complement. `a` must be the smaller value; `BETWEEN 200 AND 50` matches nothing (PostgreSQL's `BETWEEN SYMMETRIC` accepts either order).

The timestamp trap. `placed_at` is a timestamp (date *and* time). The string `'2025-01-31'` becomes midnight at the *start* of 31 January, so an order at 23:55 that day is excluded:

```
SELECT order_id, placed_at FROM orders
WHERE placed_at BETWEEN '2025-01-01' AND '2025-01-31'
ORDER BY order_id;
```

Output:

```
 order_id |      placed_at
----------+---------------------
      101 | 2025-01-03 10:15:00
      102 | 2025-01-17 22:40:00
(2 rows)
```

Order 103 (31 Jan, 23:55) is missing. The robust pattern for any time range is **half-open**: `>=` the start and `<` the next period's start.

```sql
SELECT order_id, placed_at FROM orders
WHERE placed_at >= '2025-01-01' AND placed_at < '2025-02-01'
ORDER BY order_id;
```

Output:

```
 order_id |      placed_at
----------+---------------------
      101 | 2025-01-03 10:15:00
      102 | 2025-01-17 22:40:00
      103 | 2025-01-31 23:55:00
(3 rows)
```

IN — membership in a list

`x IN (a, b, c)` is shorthand for `x = a OR x = b OR x = c`; `NOT IN` is its negation.

```sql
SELECT order_id, status FROM orders WHERE status IN ('pending', 'cancelled') ORDER BY order_id;
SELECT first_name, job_title FROM employees WHERE job_title NOT IN ('Engineer', 'Sales Rep', 'Intern')
ORDER BY emp_id;
```

Output:

```
 order_id |  status
----------+-----------
      105 | cancelled
      111 | cancelled
      113 | pending
      114 | pending
(4 rows)

 first_name |      job_title
------------+---------------------
 Asha       | CEO
 Vikram     | Engineering Manager
 Neha       | Senior Engineer
 Daniel     | Sales Manager
 Lucas      | Marketing Lead
 Ananya     | Content Writer
 Kiran      | Finance Manager
 Omar       | Accountant
 Li         | Support Engineer
(9 rows)
```

`IN` can also take a subquery (`WHERE customer_id IN (SELECT …)`) — chapter 6. **Warning:** `NOT IN` with a `NULL` in the list returns no rows at all; see 2.5.

LIKE and ILIKE — simple patterns

`LIKE` matches text against a pattern with two wildcards:

Wildcard	Matches	Example	Matches…
`%`	any sequence of zero or more characters	`'%Desk%'`	"Standing Desk", "Desk Lamp"
`_`	exactly one character	`'_a%'`	"Daniel", "Rahul", "Sara" (second letter *a*)

```sql
SELECT name FROM products WHERE name LIKE '%Desk%' ORDER BY name;
SELECT first_name FROM employees WHERE first_name LIKE '_a%' ORDER BY first_name;
```

Output:

```
     name
---------------
 Desk Lamp
 Standing Desk
(2 rows)

 first_name
------------
 Daniel
 Rahul
 Sara
(3 rows)
```

`LIKE` is **case-sensitive** in PostgreSQL. `ILIKE` (PostgreSQL-only) ignores case:

```
SELECT email FROM employees WHERE email LIKE 'neha%';
SELECT email FROM employees WHERE email ILIKE 'neha%';
```

Output:

```
 email
-------
(0 rows)

           email
----------------------------
 Neha.Gupta@Company.example
(1 row)
```

Portable alternative: `lower(email) LIKE 'neha%'`.

Matching a literal `%` or `_` — escape it. An **escape character** is a marker that means "treat the next character as plain text, not as a wildcard". PostgreSQL's default escape character is backslash; `ESCAPE` chooses another:

```
SELECT label FROM (VALUES ('10% off'), ('100 off'), ('10_off')) AS t(label)
WHERE label LIKE '10\%%' ORDER BY label;
SELECT label FROM (VALUES ('10% off'), ('100 off'), ('10_off')) AS t(label)
WHERE label LIKE '10!_%' ESCAPE '!' ORDER BY label;
```

Output:

```
  label
---------
 10% off
(1 row)

 label
--------
 10_off
(1 row)
```

(`VALUES (…), (…) AS t(label)` builds a tiny inline table — handy for experiments.)

Regular expressions (PostgreSQL)

For richer patterns PostgreSQL offers POSIX regular expressions. A **regular expression** (regex) is a mini-language for describing text patterns — `^` means "start of text", `[A-Z]` "one capital letter", `+` "one or more of the previous"; POSIX is the Unix standard that fixes this syntax. The operators are `~` (case-sensitive match), `~*` (case-insensitive), `!~` and `!~*` (no match). Names made of a capitalised word, a space, then another capital:

```
SELECT name FROM products WHERE name ~ '^[A-Z][a-z]+ [A-Z]' ORDER BY name;
```

Output:

```
        name
---------------------
 Desk Lamp
 Ergonomic Chair
 Gift Card
 Laptop Pro 14
 Mechanical Keyboard
 Standing Desk
 Wireless Mouse
(7 rows)
```

`USB-C Hub` fails (`S` isn't lower case) and `Noise-Cancelling…` fails (hyphen, not space). The standard's `SIMILAR TO` also exists but is rarely used.

Performance note

`LIKE 'abc%'` (a fixed prefix) can use a B-tree index — an **index** is a sorted lookup structure kept beside the table so the database can jump to matching rows instead of reading them all, and a **B-tree** is the default, ordered kind (in PostgreSQL the index needs the `text_pattern_ops` operator class — a setting that tells the index how to compare values — unless the database uses the `C` collation, i.e. plain byte-by-byte comparison); `LIKE '%abc'` with a leading wildcard cannot, and scans every row. Chapter 12 shows trigram (`pg_trgm`) indexes for "contains" searches (a *trigram* is every 3-character slice of the text).

Dialect notes

Feature	PostgreSQL	MySQL	SQLite	SQL Server
`LIKE` case-sensitivity	sensitive	depends on collation (default insensitive)	insensitive for ASCII (plain English letters) by default	depends on collation (default insensitive)
Case-insensitive LIKE	`ILIKE`	`LIKE` (default)	`LIKE`	`LIKE` (default)
Regex	`~`, `~*`	`REGEXP` / `RLIKE`	`REGEXP` only when a `regexp()` function is loaded (the `sqlite3` shell includes one)	none built in (SQL Server 2025 adds `REGEXP_LIKE`)

Common mistakes

- `BETWEEN` on timestamps with date-only upper bounds (use half-open ranges).
- Forgetting that `_` is a wildcard: `LIKE 'user_%'` also matches `userA…`.
- Expecting `LIKE` to be case-insensitive in PostgreSQL.

Self-check

1. *Is `BETWEEN 10 AND 20` inclusive of 20?* → Yes, both ends.

2. *Pattern for "exactly 4 characters ending in `n`"?* → `LIKE '___n'`.
3. *How do you match names containing `%`?* → `LIKE '%\%%'` (or `ESCAPE`).

TL;DR

`BETWEEN` is inclusive (use `>=` / `<` for timestamps), `IN` is shorthand for ORs, `LIKE` uses `%` and `_` and is case-sensitive in PostgreSQL (`ILIKE` isn't), regex via `~`.

2.5 NULL and three-valued logic

What it is

`NULL` is a **marker for a missing value** — unknown ("we don't know Meera's email") or not applicable ("interns earn no commission"). It is not zero, not an empty string, and not a value at all, so it cannot be *equal* to anything — not even another `NULL`.

Why it matters

Real data is full of gaps, and `NULL` changes how every comparison, filter and calculation behaves. Most "missing rows" bugs in SQL come from it.

How it works — three-valued logic

Any comparison involving `NULL` yields the third truth value **UNKNOWN** (displayed as `NULL`). Arithmetic and concatenation with `NULL` yield `NULL`.

```sql
-- code/04_nulls.sql
SELECT NULL = NULL AS null_eq_null, NULL <> 1 AS null_ne_1, NULL IS NULL AS is_null,
       1 + NULL AS plus, 'a' || NULL AS concat;
```

Output:

```
 null_eq_null | null_ne_1 | is_null |  plus  | concat
--------------+-----------+---------+--------+--------
 (null)       | (null)    | t       | (null) | (null)
(1 row)
```

`AND`, `OR` and `NOT` follow these truth tables (generated by the database itself):

```sql
SELECT a, b, a AND b AS "a AND b", a OR b AS "a OR b", NOT a AS "NOT a"
FROM (VALUES (true), (false), (NULL::boolean)) AS x(a)
CROSS JOIN (VALUES (true), (false), (NULL::boolean)) AS y(b);
```

Output:

```
   a    |   b    | a AND b | a OR b | NOT a
--------+--------+---------+--------+--------
 t      | t      | t       | t      | f
 t      | f      | f       | t      | f
 t      | (null) | (null)  | t      | f
 f      | t      | f       | t      | t
 f      | f      | f       | f      | t
 f      | (null) | f       | (null) | t
 (null) | t      | (null)  | t      | (null)
 (null) | f      | f       | (null) | (null)
 (null) | (null) | (null)  | (null) | (null)
(9 rows)
```

Read it as "UNKNOWN might be either true or false": `FALSE AND UNKNOWN` is `FALSE` whatever the unknown is; `TRUE OR UNKNOWN` is `TRUE`; anything else involving UNKNOWN stays UNKNOWN. And `NOT UNKNOWN` is still UNKNOWN.

`WHERE` **keeps only TRUE** — both FALSE and UNKNOWN rows are dropped.

Testing for NULL: IS NULL / IS NOT NULL

```
SELECT count(*) AS eq_null  FROM employees WHERE email = NULL;
SELECT count(*) AS is_null  FROM employees WHERE email IS NULL;
SELECT first_name, last_name FROM employees WHERE email IS NULL ORDER BY emp_id;
```

Output:

```
 eq_null
---------
       0
(1 row)

 is_null
---------
       2
(1 row)

 first_name | last_name
------------+-----------
 Meera      | Iyer
 Ananya     | Das
(2 rows)
```

`= NULL` is always UNKNOWN, so it never matches. (SQL Server has a legacy `ANSI_NULLS OFF` mode where `= NULL` works; don't rely on it.)

The "missing rows" trap

Sales reps have commission `0.10`, `0.10` and `NULL`. Asking for `= 0.10` and then `<> 0.10` should cover everyone — but Sara (NULL) appears in neither:

```
SELECT first_name, commission_pct FROM employees WHERE job_title = 'Sales Rep' AND commission_pct =  0.10
ORDER BY emp_id;
SELECT first_name, commission_pct FROM employees WHERE job_title = 'Sales Rep' AND commission_pct <> 0.10
ORDER BY emp_id;
```

Output:

```
 first_name | commission_pct
------------+----------------
 Priya      |           0.10
 Arjun      |           0.10
(2 rows)

 first_name | commission_pct
------------+----------------
(0 rows)
```

`IS DISTINCT FROM` is the NULL-safe "not equal": it treats two NULLs as equal and NULL vs a value as different, and always returns TRUE or FALSE.

```
SELECT first_name, commission_pct FROM employees
WHERE job_title = 'Sales Rep' AND commission_pct IS DISTINCT FROM 0.10 ORDER BY emp_id;
```

Output:

```
 first_name | commission_pct
------------+----------------
 Sara       |         (null)
(1 row)
```

(MySQL spells NULL-safe equality `<=>`; SQL Server supports `IS [NOT] DISTINCT FROM` from version 2022.)

The NOT IN trap

`x NOT IN (1, 2, NULL)` expands to `x <> 1 AND x <> 2 AND x <> NULL`. The last part is always UNKNOWN, so the whole condition can never be TRUE:

```
SELECT count(*) AS not_in_with_null FROM departments WHERE dept_id NOT IN (1, 2, NULL);
SELECT count(*) AS not_in_no_null   FROM departments WHERE dept_id NOT IN (1, 2);
```

Output:

```
 not_in_with_null
------------------
                0
(1 row)

 not_in_no_null
----------------
              4
(1 row)
```

This bites hardest with subqueries — `WHERE id NOT IN (SELECT manager_id FROM employees)` returns nothing because the CEO's `manager_id` is NULL. Use `NOT EXISTS` instead (chapter 6).

NULL is not zero or ''

```
SELECT count(*) AS rows, count(commission_pct) AS non_null_commissions FROM employees;
SELECT '' IS NULL AS empty_string_is_null;
```

Output:

```
 rows | non_null_commissions
------+----------------------
   15 |                    3
(1 row)

 empty_string_is_null
----------------------
 f
(1 row)
```

`count(*)` counts rows; `count(column)` counts non-NULL values (chapter 4). And in PostgreSQL an empty string is a real value, not NULL. (**Oracle** is the exception: it stores `''` as NULL.)

Allowing or forbidding NULL in a column

Columns accept NULL **unless** declared `NOT NULL`. Writing `NULL` in a column definition just states the default explicitly.

```
CREATE TABLE students (
    id    integer PRIMARY KEY,
    email text NULL,          -- explicit "NULL allowed" (the default anyway)
    name  text NOT NULL       -- a value is required
);
INSERT INTO students VALUES (1, NULL, 'Ravi');
INSERT INTO students VALUES (2, 'kim@school.example', NULL);
SELECT * FROM students WHERE email IS NULL;
```

Output:

```
CREATE TABLE
INSERT 0 1
ERROR:  null value in column "name" of relation "students" violates not-null constraint
DETAIL:  Failing row contains (2, kim@school.example, null).
 id | email  | name
----+--------+------
  1 | (null) | Ravi
(1 row)
```

Replacing NULLs in output (`COALESCE`, `NULLIF`) is in chapter 3; NULLs in sorting are in 2.7, in `DISTINCT` in 2.6, in aggregates in chapter 4, in joins in chapter 5.

In the interview

> **30-second answer:** "NULL means unknown, so comparisons with it give UNKNOWN, and WHERE only keeps TRUE rows. Use IS NULL / IS NOT NULL, or IS DISTINCT FROM for NULL-safe comparison. Watch NOT IN with a NULL in the list or subquery — it returns nothing — so prefer NOT EXISTS. Aggregates like COUNT(col) and SUM skip NULLs."

- *What does* `NULL = NULL` *return?* → NULL (UNKNOWN), not TRUE.
- *Why can* `WHERE x <> 5` *miss rows?* → Rows where `x` is NULL give UNKNOWN and are filtered out.
- *Is* `''` *NULL?* → Not in PostgreSQL, MySQL, SQL Server or SQLite; yes in Oracle.

Self-check

1. `TRUE OR NULL`*?* → TRUE.
2. `FALSE AND NULL`*?* → FALSE.
3. `NOT NULL` *(as a boolean expression)?* → NULL.
4. *How many rows does* `WHERE email = NULL` *return?* → Zero, always.

TL;DR

NULL = unknown. Comparisons with NULL are UNKNOWN; `WHERE` keeps only TRUE. Test with `IS [NOT] NULL`; compare NULL-safely with `IS [NOT] DISTINCT FROM`; never put a nullable list inside `NOT IN`.

2.6 DISTINCT: removing duplicate rows

What it is

`SELECT DISTINCT` removes duplicate rows from the **result** (the table itself is untouched). It answers questions like "which categories exist?" or "which cities do our customers live in?".

```sql
SELECT DISTINCT column1, column2
FROM table_name
WHERE condition;
```

How it works

```sql
-- code/05_distinct.sql
SELECT category FROM products ORDER BY category;
SELECT DISTINCT category FROM products ORDER BY category;
```

Output:

```
  category
-------------
 Accessories
 Accessories
 Electronics
 Electronics
 Electronics
 Furniture
 Furniture
 Furniture
 (null)
(9 rows)

  category
-------------
 Accessories
 Electronics
 Furniture
 (null)
(4 rows)
```

Two rules to notice:

1. **NULLs count as duplicates of each other** for `DISTINCT` (unlike `=`) — all NULL categories collapse into one row.
2. `DISTINCT` **applies to the whole selected row**, not to the first column. With two columns, you get distinct *pairs*:

```sql
SELECT DISTINCT country, city FROM customers ORDER BY country, city;
```

Output:

```
 country |   city
---------+-----------
 India   | Bengaluru
 India   | Delhi
 India   | Mumbai
 India   | mumbai
 India   | (null)
 UK      | London
 USA     | New York
(7 rows)
```

`Mumbai` and `mumbai` are different strings, so both survive — `DISTINCT` removes only **exact** duplicates. Dirty data needs cleaning first (`trim` removes spaces at the start and end of a string; `lower` turns it into lower case). Our customer Anita Shah was entered twice with differently formatted emails:

```
SELECT DISTINCT first_name, last_name, email FROM customers WHERE last_name = 'Shah';
SELECT DISTINCT first_name, last_name, lower(trim(email)) AS email FROM customers WHERE last_name =
'Shah';
```

Output:

```
 first_name | last_name |           email
------------+-----------+---------------------------
 Anita      | Shah      |   Anita.Shah@Mail.example
 Anita      | Shah      | anita.shah@mail.example
(2 rows)

 first_name | last_name |          email
------------+-----------+-------------------------
 Anita      | Shah      | anita.shah@mail.example
(1 row)
```

Counting distinct values uses `count(DISTINCT expr)`, which ignores NULLs (chapter 4 covers aggregates — functions such as `count` and `sum` that combine many rows into one value):

```
SELECT count(*) AS rows, count(city) AS with_city, count(DISTINCT city) AS distinct_cities,
       count(DISTINCT lower(city)) AS distinct_cities_ci
FROM customers;
```

Output:

```
 rows | with_city | distinct_cities | distinct_cities_ci
------+-----------+-----------------+--------------------
   10 |         9 |               6 |                  5
(1 row)
```

DISTINCT ON (PostgreSQL)

`DISTINCT ON (expr)` keeps the **first row of each group** of equal `expr` values, where "first" is decided by `ORDER BY` (which must start with the same expressions). A classic use is "the top earner in each department":

```
SELECT DISTINCT ON (dept_id) dept_id, first_name, salary
FROM employees
WHERE dept_id IS NOT NULL
ORDER BY dept_id, salary DESC, emp_id;
```

Output:

```
 dept_id | first_name |  salary
---------+------------+-----------
       1 | Vikram     | 185000.00
       2 | Daniel     | 150000.00
       3 | Lucas      | 120000.00
       4 | Kiran      | 135000.00
       5 | Li         |  64000.00
(5 rows)
```

Other databases solve "first row per group" with window functions (functions that look at related rows without merging them into one, such as `ROW_NUMBER()` — chapter 8).

Trade-offs and when not to use it

- `DISTINCT` needs a sort or hash of the whole result (hashing turns each row into a number used to bucket equal rows together) — cheap here, expensive on millions of rows.
- **Don't use `DISTINCT` to hide a bad join.** If a join produces duplicates you didn't expect, the join condition is probably wrong (chapter 5 calls this *fan-out*); `DISTINCT` masks the symptom and can hide real duplicates too.
- If you need counts per value, you want `GROUP BY` (which gathers rows with equal values into groups, chapter 4), not `DISTINCT`.

Self-check

1. *Does `SELECT DISTINCT a, b` make `a` unique?* → No; it makes each `(a, b)` pair unique.
2. *Do two NULLs count as duplicates under DISTINCT?* → Yes.
3. *Why did `count(DISTINCT city)` give 6 but the case-insensitive version 5?* → `Mumbai` and `mumbai` differ as strings but match after `lower()`.

TL;DR

`DISTINCT` removes exact duplicate result rows (NULLs grouped together); it applies to the whole select list. Clean data (`lower`, `trim`) before de-duplicating; use PostgreSQL's `DISTINCT ON` for "first row per group".

2.7 ORDER BY: sorting results

What it is

Without `ORDER BY`, SQL returns rows in **no guaranteed order** — whatever is fastest for the database at that moment. `ORDER BY` is the only way to get a defined order.

```
SELECT column1, column2
FROM table_name
WHERE condition
ORDER BY column1 [ASC | DESC], column2 [ASC | DESC];
```

`ASC` (ascending, smallest first) is the default; `DESC` is descending.

How it works

```
-- code/06_order_by.sql
SELECT first_name, salary FROM employees ORDER BY salary DESC;
```

Output:

```
 first_name |  salary
------------+-----------
 Asha       | 250000.00
 Vikram     | 185000.00
 Daniel     | 150000.00
 Neha       | 142000.00
 Kiran      | 135000.00
 Lucas      | 120000.00
 Meera      |  98000.00
 Rahul      |  98000.00
 Omar       |  76000.00
 Priya      |  72000.00
 Arjun      |  68000.00
 Sara       |  68000.00
 Li         |  64000.00
 Ananya     |  61000.00
 Rohan      |  30000.00
(15 rows)
```

Meera and Rahul tie; their relative order is **not guaranteed** and may differ on another run. Add a **tie-breaker** column whenever order matters (especially for paging), so the order is **deterministic** — the same every run.

Several sort keys — the second key only matters within ties of the first, and each key has its own direction:

```
SELECT dept_id, first_name, salary FROM employees
ORDER BY dept_id, salary DESC, first_name;
```

Output:

```
 dept_id | first_name |  salary
---------+------------+-----------
       1 | Vikram     | 185000.00
       1 | Neha       | 142000.00
       1 | Meera      |  98000.00
       1 | Rahul      |  98000.00
       2 | Daniel     | 150000.00
       2 | Priya      |  72000.00
       2 | Arjun      |  68000.00
       2 | Sara       |  68000.00
       3 | Lucas      | 120000.00
       3 | Ananya     |  61000.00
       4 | Kiran      | 135000.00
       4 | Omar       |  76000.00
       5 | Li         |  64000.00
  (null) | Asha       | 250000.00
  (null) | Rohan      |  30000.00
(15 rows)
```

Where do NULLs go?

PostgreSQL treats NULL as **larger than any value**: NULLs come **last in ASC** and **first in DESC**. Override with `NULLS FIRST` / `NULLS LAST`:

```
SELECT name, location FROM departments ORDER BY location;
SELECT name, location FROM departments ORDER BY location NULLS FIRST;
SELECT name, location FROM departments ORDER BY location DESC NULLS LAST;
```

Output:

```
    name     | location
-------------+-----------
 Engineering | Bengaluru
 Sales       | Mumbai
 Marketing   | Mumbai
 Finance     | Pune
 Research    | Pune
 Support     | (null)
(6 rows)

    name     | location
-------------+-----------
 Support     | (null)
 Engineering | Bengaluru
 Sales       | Mumbai
 Marketing   | Mumbai
 Finance     | Pune
 Research    | Pune
(6 rows)

    name     | location
-------------+-----------
 Finance     | Pune
 Research    | Pune
 Sales       | Mumbai
 Marketing   | Mumbai
 Engineering | Bengaluru
 Support     | (null)
(6 rows)
```

Database	NULLs in ASC	NULLS FIRST/LAST supported?
PostgreSQL, Oracle	last	yes
MySQL, SQL Server, SQLite	first	SQLite yes (3.30+); MySQL/SQL Server no — use `ORDER BY col IS NULL, col` (MySQL) or a `CASE` expression

Sorting by alias, expression or position

```sql
SELECT name, unit_price * 1.18 AS gross FROM products ORDER BY gross DESC LIMIT 3;
SELECT first_name, last_name FROM employees ORDER BY length(last_name) DESC, last_name LIMIT 3;
SELECT first_name, hire_date FROM employees ORDER BY 2 LIMIT 3;
```

Output:

```
      name       |   gross
-----------------+-----------
 Laptop Pro 14   | 1532.8200
 Standing Desk   |  588.8200
 Ergonomic Chair |  412.9882
(3 rows)

 first_name | last_name
------------+-----------
 Daniel     | Brooks
 Omar       | Farouk
 Lucas      | Martin
(3 rows)

 first_name | hire_date
------------+------------
 Asha       | 2015-01-05
 Kiran      | 2016-12-01
 Daniel     | 2017-09-25
(3 rows)
```

`ORDER BY 2` means "the second column of the select list". It's quick in psql but fragile in code — reorder the select list and the sort silently changes.

Custom order with CASE

Status values should sort by workflow, not alphabetically:

```
SELECT order_id, status FROM orders
ORDER BY CASE status WHEN 'pending' THEN 1 WHEN 'shipped' THEN 2
                    WHEN 'delivered' THEN 3 ELSE 4 END, order_id;
```

Output (abridged):

```
 order_id |  status
----------+-----------
      113 | pending
      114 | pending
      107 | shipped
      110 | shipped
      112 | shipped
      101 | delivered
      ...
      109 | delivered
      105 | cancelled
      111 | cancelled
(14 rows)
```

(`CASE` is explained in chapter 3.)

Text order depends on collation

A **collation** is the rule set for comparing and sorting text in a language. The book's server uses the simple `C` collation, which sorts by byte value — so all capital letters come before all lower-case ones:

```
SELECT city FROM customers WHERE city IS NOT NULL ORDER BY city;
SELECT city FROM customers WHERE city IS NOT NULL ORDER BY lower(city), city;
```

Output:

```
   city
-----------
 Bengaluru
 Delhi
 Delhi
 London
 London
 Mumbai
 New York
 New York
 mumbai
(9 rows)

   city
-----------
 Bengaluru
 Delhi
 Delhi
 London
 London
 Mumbai
 mumbai
 New York
 New York
(9 rows)
```

A database created with a language collation such as `en_US.UTF-8` would sort `mumbai` next to `Mumbai` without help. You can also choose per query: `ORDER BY city COLLATE "en-x-icu"` (needs ICU support — ICU, International Components for Unicode, is a library of language-aware text rules).

Common mistakes

- Relying on insertion order or "it always came back sorted" — without `ORDER BY` there is no order.
- Forgetting a unique tie-breaker, then paging (2.8) shows some rows twice and others never.
- Sorting numbers stored as text: `'10' < '9'` alphabetically. Store numbers in numeric columns.

Self-check

1. *Default direction?* → ASC.
2. *Where does PostgreSQL put NULLs with `ORDER BY x DESC`?* → First.
3. *Why add `emp_id` to `ORDER BY salary DESC`?* → As a unique tie-breaker so the order is deterministic.

TL;DR

Only `ORDER BY` guarantees order. Multiple keys, each `ASC`/`DESC`; control NULLs with `NULLS FIRST/LAST`; always add a unique tie-breaker when order matters; text sorting follows the collation.

2.8 LIMIT, OFFSET and FETCH: returning a page

What it is

`LIMIT n` returns at most `n` rows. `OFFSET m` skips the first `m` rows. Together they return "page *k*" of a result. The SQL-standard spelling is `OFFSET m ROWS FETCH FIRST n ROWS ONLY`. **Always pair them with `ORDER BY`** — "the first 3 rows" means nothing without an order.

How it works

```sql
-- code/07_limit_offset.sql
-- Top 3 earners
SELECT first_name, salary FROM employees ORDER BY salary DESC LIMIT 3;

-- Page 2 with 5 rows per page (skip 5)
SELECT emp_id, first_name FROM employees ORDER BY emp_id LIMIT 5 OFFSET 5;

-- Standard SQL spelling of the same thing
SELECT emp_id, first_name FROM employees ORDER BY emp_id
OFFSET 5 ROWS FETCH FIRST 5 ROWS ONLY;
```

Output:

```
 first_name |  salary
------------+-----------
 Asha       | 250000.00
 Vikram     | 185000.00
 Daniel     | 150000.00
(3 rows)

 emp_id | first_name
--------+------------
      6 | Daniel
      7 | Priya
      8 | Arjun
      9 | Sara
     10 | Lucas
(5 rows)

 emp_id | first_name
--------+------------
      6 | Daniel
      7 | Priya
      8 | Arjun
      9 | Sara
     10 | Lucas
(5 rows)
```

For page *p* (starting at 1) with page size *s*: `LIMIT s OFFSET (p - 1) * s`.

Ties: FETCH … WITH TIES

`LIMIT 1` on a tie picks one row arbitrarily. `FETCH FIRST n ROWS WITH TIES` (PostgreSQL 13+, requires `ORDER BY`) also returns every row that ties with the last one:

```sql
SELECT first_name, salary FROM employees WHERE dept_id IN (1, 2)
ORDER BY salary LIMIT 1;
SELECT first_name, salary FROM employees WHERE dept_id IN (1, 2)
ORDER BY salary FETCH FIRST 1 ROWS WITH TIES;
```

Output:

```
 first_name |  salary
------------+----------
 Arjun      | 68000.00
(1 row)

 first_name |  salary
------------+----------
 Arjun      | 68000.00
 Sara       | 68000.00
(2 rows)
```

"Lowest-paid person" has two correct answers; `WITH TIES` returns both. (Ranking functions in chapter 8 generalise this.)

Why OFFSET gets slow — keyset pagination

`OFFSET 100000` still makes the database produce and throw away 100,000 rows, so deep pages get slower and slower. Rows inserted or deleted between page loads also shift the pages, so users see duplicates or miss rows. **Keyset** (or "seek") pagination remembers the last key shown and asks for rows after it:

```sql
-- page after the one that ended at emp_id 10
SELECT emp_id, first_name FROM employees WHERE emp_id > 10 ORDER BY emp_id LIMIT 5;
```

Output:

```
 emp_id | first_name
--------+------------
     11 | Ananya
     12 | Kiran
     13 | Omar
     14 | Li
     15 | Rohan
(5 rows)
```

With an index on the key, every page costs the same no matter how deep. The trade-off: you can go "next/previous" but not jump to page 500. Details in chapter 12.

Dialect differences

Database	First 10 rows	Skip 20, take 10
PostgreSQL	`LIMIT 10` or `FETCH FIRST 10 ROWS ONLY`	`LIMIT 10 OFFSET 20` or `OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY`
MySQL	`LIMIT 10`	`LIMIT 10 OFFSET 20` or `LIMIT 20, 10` (offset first!)
SQLite	`LIMIT 10`	`LIMIT 10 OFFSET 20`
SQL Server	`SELECT TOP (10) …`	`ORDER BY … OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY`
Oracle 12c+	`FETCH FIRST 10 ROWS ONLY`	`OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY`

Common mistakes

- `LIMIT` without `ORDER BY` — "random" rows that can change between runs.
- MySQL's `LIMIT 20, 10` read as "20 rows starting at 10" — it's the reverse.
- Paging on a non-unique sort key — rows at page boundaries repeat or vanish. Add a unique tie-breaker.

In the interview

> **30-second answer:** "`ORDER BY` plus `LIMIT n OFFSET m`, or the standard `OFFSET … FETCH`. For deep pages OFFSET is O(offset) ("Big-O" notation: its work grows in proportion to the offset) and unstable under inserts, so I'd use keyset pagination — `WHERE (sort_key, id) > (:last_key, :last_id) ORDER BY sort_key, id LIMIT n` — backed by an index." (`:last_key` and `:last_id` are placeholders the application fills with the last row it showed.)

- *How do you get the top 3 including ties?* → `FETCH FIRST 3 ROWS WITH TIES`, or `DENSE_RANK()` (chapter 8).
- *SQL Server equivalent of LIMIT?* → `TOP (n)` or `OFFSET … FETCH`.

Self-check

1. *Rows returned by `LIMIT 5 OFFSET 12` over 15 rows?* → 3 (rows 13–15).
2. *Page 4, size 20?* → `LIMIT 20 OFFSET 60`.
3. *What does keyset pagination need?* → A stable, unique ordering key (and ideally an index on it).

TL;DR

`ORDER BY … LIMIT n OFFSET m` (or `OFFSET … FETCH`). `WITH TIES` keeps ties. Deep `OFFSET` is slow and unstable; prefer keyset pagination.

2.9 How a SELECT is evaluated (logical processing order)

What it is

You *write* a query in the order `SELECT … FROM … WHERE … ORDER BY … LIMIT`, but the database *evaluates it logically* in a different order. Knowing that order explains many error messages.

(Chapter 4 inserts `GROUP BY` and `HAVING` between `WHERE` and `SELECT`, and window functions after them.)

Consequences:

- **`WHERE` cannot use a column alias** defined in the `SELECT` list, because `SELECT` hasn't happened yet.
- **`ORDER BY` can use aliases**, because it runs after `SELECT` — but in PostgreSQL only a bare alias: `ORDER BY gross` works, `ORDER BY gross + 0` fails with *`column "gross" does not exist`* (inside an expression, names refer to input columns).
- `LIMIT` is applied last, to the already sorted result.

```sql
-- code/01_select_columns.sql (end)
SELECT name, unit_price * 1.18 AS gross FROM products WHERE gross > 500;
SELECT name, unit_price * 1.18 AS gross FROM products WHERE unit_price * 1.18 > 500 ORDER BY gross DESC;
```

Output:

```
ERROR:  column "gross" does not exist
LINE 1: ...e, unit_price * 1.18 AS gross FROM products WHERE gross > 50...
                                                             ^
     name      |   gross
---------------+-----------
 Laptop Pro 14 | 1532.8200
 Standing Desk |  588.8200
(2 rows)
```

This is the *logical* order that defines the meaning; the optimizer (the database component that chooses how to run a query) may physically do things differently (e.g. use an index to read rows already sorted) as long as the result is the same.

Self-check

1. *Why does* `WHERE gross > 500` *fail?* → `WHERE` is evaluated before the `SELECT` list defines `gross`.
2. *Is* `ORDER BY gross` *allowed?* → Yes; `ORDER BY` comes after `SELECT`.

TL;DR

Logical order: FROM → WHERE → SELECT → DISTINCT → ORDER BY → LIMIT. Aliases exist from SELECT onward.

Putting it together

A query using every clause from this chapter — "the three most recent non-cancelled 2025 orders placed through a sales rep":

```sql
-- code/09_putting_it_together.sql
SELECT order_id, customer_id, sales_rep_id, placed_at, status
FROM orders
WHERE status <> 'cancelled'
  AND sales_rep_id IS NOT NULL
  AND placed_at >= '2025-01-01' AND placed_at < '2026-01-01'
ORDER BY placed_at DESC, order_id DESC
LIMIT 3;
```

Output:

```
 order_id | customer_id | sales_rep_id |      placed_at      | status
----------+-------------+--------------+---------------------+---------
      113 |           1 |            7 | 2025-06-02 10:30:00 | pending
      112 |           5 |            9 | 2025-05-20 21:05:00 | shipped
      110 |           7 |            8 | 2025-04-25 11:10:00 | shipped
(3 rows)
```

Chapter 2 TL;DR

- `SELECT cols FROM t` — name your columns; expressions and `AS` aliases are free; table aliases shorten references.
- `WHERE` keeps TRUE rows only. Precedence `NOT` > `AND` > `OR`: parenthesise.
- Integer division truncates in PostgreSQL (`7/2 = 3`); `||` concatenates; compound `+=` is not standard SQL.
- `BETWEEN` is inclusive (use half-open ranges for timestamps); `IN` = ORs; `LIKE` (`%`, `_`) is case-sensitive, `ILIKE` isn't; `~` for regex.

- NULL means unknown → three-valued logic. Use `IS [NOT] NULL`, `IS DISTINCT FROM`; beware `NOT IN` with NULLs.
- `DISTINCT` removes exact duplicate rows (NULLs grouped); `DISTINCT ON` keeps first row per group.
- Only `ORDER BY` gives order; add tie-breakers; `NULLS FIRST/LAST`; collation affects text.
- `LIMIT/OFFSET` or `OFFSET … FETCH`; `WITH TIES`; keyset pagination for deep pages.
- Logical order FROM → WHERE → SELECT → DISTINCT → ORDER BY → LIMIT.

Next: Chapter 3 — Functions and expressions.

Chapter 3 — Functions and Expressions

Chapter 2 selected and filtered stored values. Real reports need to *transform* them: round a price, tidy a messy email, work out how many days an order took to ship, label a product "premium", or show "no email on file" instead of a blank. This chapter covers the building blocks for all of that: SQL's **type system** and conversions, **number**, **string** and **date/time** functions, and the **conditional expressions** `CASE`, `COALESCE`, `NULLIF`, `GREATEST` and `LEAST`.

The SQL course these notes draw on was written for MySQL. Function names differ more between databases than anything else in SQL, so every section ends with a **dialect map** from MySQL (and SQL Server, SQLite) to PostgreSQL.

What you'll learn

#	Topic	You will be able to…
3.1	Types and casts	name PostgreSQL's core types, convert with `CAST` / `::`, avoid float surprises
3.2	Number functions	round, truncate, take powers/logs/roots, generate random numbers, format numbers
3.3	String functions	join, cut, measure, clean, search, pad, split and pattern-replace text
3.4	Date and time functions	get "now", do date arithmetic, extract/truncate parts, format/parse, handle time zones
3.5	Conditional expressions	use `CASE`, `COALESCE`, `NULLIF`, `GREATEST`/`LEAST`
3.6	Worked exercises	combine functions into realistic report columns

Setup. Examples use the `company` sample database (chapter 1, 1.11); code is in `code/`. Outputs come from PostgreSQL 14.

Functions in one paragraph

A **function** takes zero or more **arguments** and returns one value: `round(9.876, 1)` → `9.9`. Functions that work on one row at a time (everything in this chapter) are **scalar functions**; functions that squash many rows into one (`sum`, `count`, `avg`) are **aggregate functions**, covered in chapter 4. Functions nest — `lower(trim(email))` — and can appear anywhere an expression can: the `SELECT` list, `WHERE`, `ORDER BY`, `GROUP BY`, `UPDATE … SET`.

3.1 Data types and conversions

What it is

Every value in SQL has a **data type** that decides what it can hold and which operators and functions apply. `'42'` (text) and `42` (integer) look alike but behave differently: `'42' || '1'` is `'421'`, `42 + 1` is `43`. PostgreSQL is **strict**: it rarely converts types silently, which catches bugs early.

The core PostgreSQL types

Family	Type	Holds	Notes
Integer	`smallint`, `integer` (`int`), `bigint`	2-, 4-, 8-byte whole numbers	`integer` max is 2,147,483,647
Exact decimal	`numeric(p, s)` / `decimal(p, s)`	exact numbers, *p* total digits, *s* after the point	use for **money**; slower than floats
Floating point	`real`, `double precision`	approximate binary fractions	science/measurements, never money
Text	`text`, `varchar(n)`, `char(n)`	strings	in PostgreSQL `text` and `varchar` perform the same; `char(n)` pads with spaces
Boolean	`boolean`	`true`, `false`, `NULL`	
Date/time	`date`, `time`, `timestamp`, `timestamptz`, `interval`	calendar dates, clock times, instants, durations	3.4
Other	`uuid`, `json`/`jsonb`, arrays (`int[]`), `bytea`, enums	uuid = random 128-bit ID; jsonb = stored JSON; array = list in one cell; bytea = raw bytes; enum = fixed set of labels	JSON and arrays in chapter 14

Concept	PostgreSQL	MySQL	SQL Server	SQLite
Exact decimal	`numeric(10,2)`	`DECIMAL(10,2)`	`DECIMAL(10,2)`	stored as REAL/INTEGER (type *affinity*)
Variable text	`text` / `varchar(n)`	`VARCHAR(n)`, `TEXT`	`VARCHAR(n)` / `NVARCHAR(n)`	`TEXT`
Boolean	`boolean`	`TINYINT(1)` alias `BOOLEAN`	`BIT`	integer 0/1
Timestamp with zone	`timestamptz`	`TIMESTAMP` (converted to UTC, the world reference time)	`DATETIMEOFFSET`	text or number, by convention

(SQLite is the outlier: a column's type is only a *preference* — it will store text in an integer column. The full DDL (Data Definition Language — the `CREATE`/`ALTER` statements that define tables) story is in chapter 9.)

How it works — asking for and changing types

`pg_typeof(x)` reports a value's type:

```
-- code/01_types_and_casts.sql
SELECT pg_typeof(42) AS int_lit, pg_typeof(42.0) AS dec_lit, pg_typeof(4.2e1) AS sci_lit,
       pg_typeof('hi') AS str_lit, pg_typeof(true) AS bool_lit, pg_typeof(now()) AS now_type;
SELECT pg_typeof(salary) AS salary, pg_typeof(hire_date) AS hire_date, pg_typeof(email) AS email
FROM employees LIMIT 1;
```

Output:

```
 int_lit | dec_lit | sci_lit | str_lit | bool_lit |         now_type
---------+---------+---------+---------+----------+-------------------------
 integer | numeric | numeric | unknown | boolean  | timestamp with time zone
(1 row)

 salary  | hire_date | email
---------+-----------+-------
 numeric | date      | text
(1 row)
```

Notice: `42.0` is `numeric` (exact), not a float, and a quoted literal starts out as type `unknown` — PostgreSQL decides its type from context. That's why `WHERE hire_date >= '2023-01-01'` works: the string is read as a `date` because it's compared with one.

Casting (explicit conversion) has three spellings:

Form	Standard?	Example
`CAST(expr AS type)`	yes — works everywhere	`CAST('2025-03-18' AS date)`
`expr::type`	PostgreSQL shorthand	`'2025-03-18'::date`
`type 'literal'`	standard, literals only	`date '2025-03-18'`

```sql
SELECT CAST('2025-03-18' AS date) AS std_cast, '2025-03-18'::date AS pg_cast, date '2025-03-18' AS
typed_literal;
SELECT CAST(100 AS numeric(5,2)) AS to_numeric, 3.99::integer AS rounds_not_truncates,
       '42'::integer + 1 AS text_to_int, 123::text || 'abc' AS int_to_text;
```

Output:

```
  std_cast  |  pg_cast   | typed_literal
------------+------------+---------------
 2025-03-18 | 2025-03-18 | 2025-03-18
(1 row)

 to_numeric | rounds_not_truncates | text_to_int | int_to_text
------------+----------------------+-------------+-------------
     100.00 |                    4 |          43 | 123abc
(1 row)
```

Casting a decimal to `integer` **rounds** (3.99 → 4); use `trunc()` or `floor()` if you want to cut off the fraction.

Exact vs approximate numbers

`double precision` stores binary fractions, and most decimal fractions (like 0.1) have no exact binary form:

```sql
SELECT 0.1 + 0.2 = 0.3 AS numeric_equal,
       0.1::double precision + 0.2::double precision = 0.3 AS float_equal,
       0.1::double precision + 0.2::double precision AS float_sum;
SELECT 12.345::numeric(5,2) AS rounded_to_scale;
```

Output:

```
 numeric_equal | float_equal |      float_sum
---------------+-------------+---------------------
 t             | f           | 0.30000000000000004
(1 row)

 rounded_to_scale
------------------
            12.35
(1 row)
```

Rule: **money and anything that must add up exactly** → `numeric`. A `numeric(p,s)` value is rounded to *s* decimal places when stored.

Errors a strict type system gives you

```
SELECT 2147483647::integer + 1;
SELECT 'abc'::integer;
SELECT 1000::numeric(5,2);
SELECT first_name + 1 FROM employees LIMIT 1;
```

Output:

```
ERROR:  integer out of range
ERROR:  invalid input syntax for type integer: "abc"
LINE 1: SELECT 'abc'::integer;
               ^
ERROR:  numeric field overflow
DETAIL:  A field with precision 5, scale 2 must round to an absolute value less than 10^3.
ERROR:  operator does not exist: text + integer
LINE 1: SELECT first_name + 1 FROM employees LIMIT 1;
                          ^
HINT:  No operator matches the given name and argument types. You might need to add explicit type casts.
```

MySQL would quietly turn `'abc' + 1` into `1` (with a warning) and wrap or clip some overflows (values too big for their type) depending on its SQL mode (a server setting that controls how strict MySQL is). PostgreSQL refuses — annoying for a minute, valuable for years.

Booleans accept several spellings on input:

```
SELECT 'yes'::boolean AS yes, 'f'::boolean AS f, 'on'::boolean AS on_;
```

```
 yes | f | on_
-----+---+-----
 t   | f | t
(1 row)
```

Common mistakes

- Storing money in `real` / `double precision`.
- Storing numbers or dates in text columns — sorting and comparisons become alphabetical (`'10' < '9'`).
- Expecting `::integer` to truncate; it rounds.

Self-check

1. *Type of the literal `42.0` in PostgreSQL?* → `numeric`.
2. *Portable way to convert text to a date?* → `CAST('2025-03-18' AS date)`.

3. *Why is `0.1::float8 + 0.2::float8 = 0.3` false?* → Binary floating point can't represent 0.1 or 0.2 exactly.

TL;DR

Everything has a type. Use `numeric` for money, `text` for strings, `timestamptz` for instants. Convert with `CAST(x AS t)` or `x::t`. PostgreSQL won't silently coerce (automatically convert) mismatched types — that's a feature.

3.2 Number functions

What it is

Numeric functions do arithmetic you'd otherwise hand-code: rounding for display, absolute differences, powers and logarithms for growth (a **logarithm** answers "to what power must I raise the base to get this number?" — log base 10 of 1000 is 3), modulo for "every n-th", random numbers for sampling.

The functions

Function (PostgreSQL)	Does	Example → result
`abs(x)`	absolute value	`abs(-10)` → 10
`round(x)` / `round(x, d)`	round to *d* decimals (negative *d* rounds left of the point)	`round(9.876, 1)` → 9.9; `round(1234.567, -2)` → 1200
`trunc(x, d)`	cut to *d* decimals, no rounding	`trunc(9.876, 1)` → 9.8
`ceil(x)` / `ceiling(x)`	smallest integer ≥ x (round **up**)	`ceil(4.3)` → 5; `ceil(-4.3)` → -4
`floor(x)`	largest integer ≤ x (round **down**)	`floor(4.8)` → 4; `floor(-4.3)` → -5
`sqrt(x)`, `cbrt(x)`	square / cube root	`sqrt(25)` → 5
`power(a, b)`, `a ^ b`	a to the power b	`power(2, 3)` → 8
`mod(a, b)`, `a % b`	remainder	`mod(17, 5)` → 2
`div(a, b)`	integer quotient	`div(17, 5)` → 3
`exp(x)`	e^x (e ≈ 2.718, the natural growth constant)	`exp(2)` → 7.389…
`ln(x)`	natural log (base e)	`ln(10)` → 2.302585…
`log(x)`, `log10(x)`	**base-10** log in PostgreSQL	`log(100)` → 2
`log(b, x)`	log of x in base b	`log(2, 8)` → 3
`sign(x)`	-1, 0 or 1	`sign(-15)` → -1
`pi()`, `degrees(r)`, `radians(d)`	π and angle conversion	`radians(60)` → 1.047…
`greatest(…)`, `least(…)`	largest / smallest argument	`greatest(3, 9, 4)` → 9
`gcd(a, b)`, `lcm(a, b)`	greatest common divisor / least common multiple (PostgreSQL 13+)	`gcd(12, 18)` → 6
`random()`, `setseed(s)`	random `double precision` in [0, 1) (0 included, 1 excluded); seed it (give a fixed starting value) for repeatable output	
`to_char(x, fmt)`	format a number as text	`to_char(1234567, 'FM9,999,999.00')` → `1,234,567.00`

How it works — run them

```sql
-- code/02_number_functions.sql
SELECT abs(-10) AS abs, round(9.876, 1) AS round_1, round(9.876) AS round_0,
       round(1234.567, -2) AS round_neg, trunc(9.876, 1) AS trunc_1,
       ceil(4.3) AS ceil, floor(4.8) AS floor, ceil(-4.3) AS ceil_neg, floor(-4.3) AS floor_neg;
```

Output:

```
 abs | round_1 | round_0 | round_neg | trunc_1 | ceil | floor | ceil_neg | floor_neg
-----+---------+---------+-----------+---------+------+-------+----------+-----------
  10 |     9.9 |      10 |      1200 |     9.8 |    5 |     4 |       -4 |        -5
(1 row)
```

With negative numbers `ceil` moves toward zero and `floor` away from it — `floor(-4.3)` is -5, not -4. `trunc` always moves toward zero.

Halfway cases depend on the type:

```
SELECT round(2.5) AS num_2_5, round(-2.5) AS num_neg_2_5,
       round(2.5::double precision) AS dbl_2_5, round(3.5::double precision) AS dbl_3_5;
```

Output:

```
 num_2_5 | num_neg_2_5 | dbl_2_5 | dbl_3_5
---------+-------------+---------+---------
       3 |          -3 |       2 |       4
(1 row)
```

`numeric` rounds halves **away from zero** (2.5 → 3); `double precision` typically rounds halves **to the nearest even** number ("banker's rounding", 2.5 → 2, 3.5 → 4) because it uses the platform's floating-point rounding. Another reason to keep money in `numeric`.

```
SELECT sqrt(25) AS sqrt, power(2, 3) AS power, 2 ^ 3 AS caret, mod(17, 5) AS mod,
       17 % 5 AS pct, mod(-17, 5) AS mod_neg, div(17, 5) AS int_div;
SELECT exp(2) AS exp_2, ln(10) AS ln_10, log(100) AS log_100, log10(100) AS log10_100,
       log(2, 8) AS log_base2_of_8;
SELECT sign(-15) AS sign, pi() AS pi, degrees(1.047) AS degrees, radians(60) AS radians;
SELECT greatest(3, 9, 4) AS greatest, least(3, 9, 4) AS least, gcd(12, 18) AS gcd, lcm(4, 6) AS lcm;
```

Output:

```
 sqrt | power | caret | mod | pct | mod_neg | int_div
------+-------+-------+-----+-----+---------+---------
    5 |     8 |     8 |   2 |   2 |      -2 |       3
(1 row)

      exp_2       |       ln_10       | log_100 | log10_100 |   log_base2_of_8
------------------+-------------------+---------+-----------+--------------------
 7.38905609893065 | 2.302585092994046 |       2 |         2 | 3.0000000000000000
(1 row)

 sign |        pi         |      degrees      |      radians
------+-------------------+-------------------+--------------------
   -1 | 3.141592653589793 | 59.98868115019719 | 1.0471975511965976
(1 row)

 greatest | least | gcd | lcm
----------+-------+-----+-----
        9 |     3 |   6 |  12
(1 row)
```

Two things to remember: the remainder keeps the sign of the **dividend** (the number being divided; `mod(-17, 5)` = -2), and `log(x)` means **base 10** in PostgreSQL but **natural log** in MySQL and SQL Server. Use `ln()` when you mean natural log — it's unambiguous everywhere that has it.

Random numbers

`random()` returns a new value each call. `setseed()` makes the sequence repeatable for the rest of the session (your current connection to the database) — useful for tests and reproducible samples.

```
SELECT setseed(0.42);
SELECT round(random()::numeric, 4) AS r1, round(random()::numeric, 4) AS r2;
SELECT floor(random() * 6)::int + 1 AS dice_roll;
```

Output:

```
   r1   |   r2
--------+--------
 0.5699 | 0.2954
(1 row)

 dice_roll
-----------
         6
(1 row)
```

`floor(random() * n)::int + 1` gives a whole number from 1 to *n*. (Sampling rows randomly — `ORDER BY random() LIMIT k` or `TABLESAMPLE` — comes up in chapter 12.)

Formatting numbers for people

`to_char(number, pattern)` turns numbers into display strings. `9` is a digit position, `0` a digit that's always shown, `,` a group separator, `.` the decimal point, and `FM` ("fill mode") strips the padding spaces `to_char` otherwise adds.

```
SELECT to_char(1234567, 'FM9,999,999.00') AS with_commas,
       to_char(0.0725, 'FM990.00%') AS naive_pct,
       to_char(0.0725 * 100, 'FM990.00') || '%' AS pct,
       to_char(42, '00000') AS zero_padded;
```

Output:

```
 with_commas  | naive_pct |  pct  | zero_padded
--------------+-----------+-------+-------------
 1,234,567.00 | 0.07%     | 7.25% |  00042
(1 row)
```

`%` in the pattern is printed literally — it does *not* multiply by 100, so do that yourself. Without `FM`, `to_char` reserves a leading space for the sign (see `' 00042'`). Format only at the very end: the result is text and no longer sorts or sums as a number.

Real use on the sample data

```
SELECT order_id, product_id,
       quantity * unit_price * (1 - discount)            AS exact,
       round(quantity * unit_price * (1 - discount), 2)  AS rounded
FROM order_items WHERE order_id IN (104, 107, 109) ORDER BY order_id, product_id;

-- Salary per month, rounded up to the next hundred
SELECT first_name, salary, ceil(salary / 12 / 100) * 100 AS monthly_rounded_up
FROM employees WHERE emp_id <= 4 ORDER BY emp_id;
```

Output:

```
 order_id | product_id |  exact   | rounded
----------+------------+----------+---------
      104 |          5 | 479.0000 |  479.00
      104 |          6 | 332.4905 |  332.49
      107 |          6 | 629.9820 |  629.98
      109 |          4 | 151.3000 |  151.30
(4 rows)

 first_name |  salary   | monthly_rounded_up
------------+-----------+--------------------
 Asha       | 250000.00 |              20900
 Vikram     | 185000.00 |              15500
 Neha       | 142000.00 |              11900
 Rahul      |  98000.00 |               8200
(4 rows)
```

The "divide, round to a unit, multiply back" trick (`ceil(x / 100) * 100`) works for any step size — to the nearest 5 minutes, 1,000 rupees, and so on.

Dialect map (numbers)

Task	MySQL	PostgreSQL	SQL Server	SQLite
Truncate decimals	`TRUNCATE(x, d)`	`trunc(x, d)`	`ROUND(x, d, 1)`	— (cast tricks)
Random 0–1	`RAND()`	`random()`	`RAND()`	`random()` returns a 64-bit integer
Natural log	`LOG(x)` or `LN(x)`	`ln(x)`	`LOG(x)`	`ln(x)` (3.35+ math functions)
Base-10 log	`LOG10(x)`	`log(x)` / `log10(x)`	`LOG10(x)`	`log(x)` / `log10(x)`
Power	`POWER(a,b)` / `POW`	`power(a,b)` / `a ^ b`	`POWER(a,b)` (`^` is XOR, a bit-flipping operator!)	`pow(a,b)`
Number with commas	`FORMAT(x, 2)`	`to_char(x, 'FM999,999,990.00')`	`FORMAT(x, 'N2')`	no built-in (`printf('%,d', x)` groups integers only)
Convert type	`CONVERT(x, DECIMAL(5,2))` / `CAST`	`CAST(x AS numeric(5,2))` / `x::numeric(5,2)`	`CONVERT(DECIMAL(5,2), x)` / `CAST`	`CAST(x AS REAL)`
Integer division of ints	`DIV`	`/` or `div()`	`/`	`/`

Common mistakes

- Using `log()` expecting natural log in PostgreSQL.
- Formatting (`to_char`) inside calculations or before sorting.
- `round()` on `double precision` expecting "half up".

Self-check

1. *`floor(-2.5)`, `ceil(-2.5)`, `trunc(-2.5)`?* → -3, -2, -2.
2. *`round(123.456, -1)`?* → 120.
3. *PostgreSQL function for MySQL's `LOG(x)`?* → `ln(x)`.

TL;DR

`round/trunc/ceil/floor` for rounding (watch negatives and halves), `power/sqrt/exp/ln/log` for maths (PostgreSQL `log` is base 10), `mod` / `%` for remainders, `random` + `setseed` for repeatable randomness, `to_char` for display only.

3.3 String functions

What it is

String (text) functions join, cut, measure, clean and search text. They are the tools for turning messy input — stray spaces, mixed case, combined fields — into consistent values.

The functions

Function (PostgreSQL)	Does	Example → result
`concat(a, b, …)` / `a || b`	join strings	`concat('Hello', ' ', 'World')` → `Hello World`
`concat_ws(sep, a, b, …)`	join with a separator, skipping NULLs	`concat_ws('-', '2024', '01', '01')` → `2024-01-01`
`substring(s FROM start FOR len)` / `substr(s, start, len)`	part of a string (positions start at 1)	`substring('SQL Functions' FROM 1 FOR 3)` → `SQL`
`left(s, n)`, `right(s, n)`	first / last *n* characters (negative *n* = all but)	`left('Database', 3)` → `Dat`
`length(s)` / `char_length(s)`	number of characters	`length('Data')` → 4
`octet_length(s)`	number of bytes	`octet_length('Data')` → 4
`upper(s)`, `lower(s)`, `initcap(s)`	change case	`initcap('desk lamp')` → `Desk Lamp`
`replace(s, from, to)`	replace every occurrence of a substring	`replace('I love SQL', 'SQL', 'Postgres')`
`translate(s, from, to)`	replace character by character	`translate('2025/01/03', '/', '-')`
`trim(s)`, `ltrim`, `rtrim`, `trim(BOTH 'x' FROM s)`	remove spaces (or given characters) from ends	`trim(' SQL ')` → `SQL`
`position(sub IN s)` / `strpos(s, sub)`	1-based position of first match, 0 if absent	`strpos('Hello World', 'World')` → 7
`lpad(s, n, fill)`, `rpad(s, n, fill)`	pad (or cut) to length *n*	`lpad('7', 3, '0')` → `007`
`repeat(s, n)`, `reverse(s)`	repeat / reverse	`repeat('La', 3)` → `LaLaLa`
`split_part(s, delim, n)`	*n*-th field of a delimited string (fields separated by a marker such as `.` or `,`) (negative *n* from the end, PostgreSQL 14+)	`split_part('a@b.com', '@', 2)` → `b.com`
`format(fmt, …)`	printf-style building (fill `%s` slots in a template); `%I` quotes identifiers (table/column names), `%L` literals	
`regexp_replace`, `regexp_match`	regular-expression replace / capture	

How it works — concatenation and NULL

```sql
-- code/03_string_functions.sql
SELECT concat('Hello', ' ', 'World') AS concat, 'Hello' || ' ' || 'World' AS pipes,
       concat_ws('-', '2024', '01', '01') AS concat_ws;
SELECT concat('a', NULL, 'b') AS concat_null, 'a' || NULL || 'b' AS pipes_null,
       concat_ws(', ', 'Pune', NULL, 'India') AS ws_null;
```

Output:

```
   concat    |    pipes    | concat_ws
-------------+-------------+------------
 Hello World | Hello World | 2024-01-01
(1 row)

 concat_null | pipes_null |   ws_null
-------------+------------+-------------
 ab          | (null)     | Pune, India
(1 row)
```

In PostgreSQL `concat()` **ignores** NULL arguments, while `||` returns NULL if any part is NULL. `concat_ws` skips NULLs, so it's ideal for addresses with optional parts. **MySQL differs:** its `CONCAT()` returns NULL if any argument is NULL (its `CONCAT_WS` skips them).

Extracting parts

```
SELECT substring('SQL Functions' FROM 1 FOR 3) AS std_form, substring('SQL Functions', 5) AS from_5,
       substr('SQL Functions', 5, 4) AS substr, left('Database', 3) AS left_3,
       right('Postgres', 3) AS right_3, left('Database', -2) AS left_neg;
```

Output:

```
 std_form |  from_5   | substr | left_3 | right_3 | left_neg
----------+-----------+--------+--------+---------+----------
 SQL      | Functions | Func   | Dat    | res     | Databa
(1 row)
```

String positions in SQL start at **1**, not 0.

Measuring

```
SELECT length('Data') AS length, char_length('Data') AS char_length, octet_length('Data') AS bytes,
       length('  padded  ') AS with_spaces;
```

Output:

```
 length | char_length | bytes | with_spaces
--------+-------------+-------+-------------
      4 |           4 |     4 |          10
(1 row)
```

For plain ASCII (the 128 basic English letters, digits and symbols, one byte each), characters and bytes are the same. In a UTF-8 database (UTF-8 is the universal character encoding — the rule for storing any language's characters as bytes, using 1 to 4 bytes each) — which is what the book's runner creates (`ENCODING 'UTF8'`, `C` collation) — `é` counts as one character yet occupies two bytes:

```
SELECT current_setting('server_encoding') AS encoding, char_length('café') AS chars,
       octet_length('café') AS bytes, upper('café') AS upper_c_collation,
       upper('café' COLLATE "en-x-icu") AS upper_icu;
```

Output:

```
 encoding | chars | bytes | upper_c_collation | upper_icu
----------+-------+-------+-------------------+-----------
 UTF8     |     4 |     5 | CAFé              | CAFÉ
(1 row)
```

Notice the last two columns: case conversion follows the **collation**, and the plain `C` collation only knows the ASCII letters, so `é` stays lower case. A language-aware collation (here the ICU collation `en-x-icu`, available when PostgreSQL is built with ICU — International Components for Unicode, a widely used library of language rules) converts it properly. (A database created with the legacy `SQL_ASCII` encoding would not understand multibyte characters (characters stored in more than one byte) at all and would report 5 characters — always create databases as UTF-8.)

Portability trap: MySQL's `LENGTH()` counts **bytes** (use `CHAR_LENGTH()` for characters); SQL Server's `LEN()` counts characters but ignores trailing spaces (`DATALENGTH()` gives bytes).

Changing case, replacing, repeating

```
SELECT upper('hello') AS upper, lower('WORLD') AS lower, initcap('noise-cancelling headphones') AS
initcap;
SELECT replace('I love SQL', 'SQL', 'Postgres') AS replace, translate('2025/01/03', '/', '-') AS
translate,
       reverse('stressed') AS reverse, repeat('La', 3) AS repeat;
```

Output:

```
 upper | lower |           initcap
-------+-------+---------------------------
 HELLO | world | Noise-Cancelling Headphones
(1 row)

     replace     | translate  | reverse  | repeat
-----------------+------------+----------+--------
 I love Postgres | 2025-01-03 | desserts | LaLaLa
(1 row)
```

`upper` / `lower` are the standard way to compare case-insensitively: `WHERE lower(email) = lower(:input)` (`:input` stands for a value the application supplies). `replace` swaps whole substrings; `translate` maps single characters one-to-one.

Trimming

```
SELECT '[' || trim('   SQL   ') || ']' AS trim, '[' || ltrim('   SQL   ') || ']' AS ltrim,
       '[' || rtrim('   SQL   ') || ']' AS rtrim, trim(BOTH '*' FROM '**SQL**') AS trim_chars;
```

Output:

```
 trim  |  ltrim   |  rtrim   | trim_chars
-------+----------+----------+------------
 [SQL] | [SQL   ] | [   SQL] | SQL
(1 row)
```

(The brackets make the invisible spaces visible.) `trim` removes only leading and trailing characters, never those in the middle.

Searching and padding

```
SELECT position('abc' IN 'xxabcdef') AS position, strpos('Hello World', 'World') AS strpos,
       position('zzz' IN 'Hello') AS not_found;
SELECT lpad('SQL', 5, '*') AS lpad, rpad('SQL', 5, '*') AS rpad, lpad('7', 3, '0') AS zero_pad,
       lpad('Postgres', 3) AS lpad_truncates;
```

Output:

```
 position | strpos | not_found
----------+--------+-----------
        3 |      7 |         0
(1 row)

 lpad  | rpad  | zero_pad | lpad_truncates
-------+-------+----------+----------------
 **SQL | SQL** | 007      | Pos
(1 row)
```

`lpad` both pads and **truncates** to the requested length — careful with long values.

Splitting delimited text

```
SELECT split_part('www.example.com', '.', 1) AS first, split_part('www.example.com', '.', -1) AS last,
       array_to_string((string_to_array('www.example.com', '.'))[1:2], '.') AS first_two;
```

Output:

```
 first | last |  first_two
-------+------+-------------
 www   | com  | www.example
(1 row)
```

`split_part` replaces MySQL's `SUBSTRING_INDEX`: `SUBSTRING_INDEX(s, '.', -1)` (text after the last dot) is `split_part(s, '.', -1)`; "everything before the second dot" (`SUBSTRING_INDEX(s, '.', 2)`) needs the array form shown (an **array** is an ordered list of values held in one cell; `[1:2]` takes its first two elements). (Splitting into *rows* uses `string_to_table` / `unnest`, chapter 14.)

format() and regular expressions

```
SELECT format('%s earns %s', first_name, salary) AS sentence,
       format('SELECT * FROM %I WHERE name = %L', 'my table', 'O''Brien') AS safe_sql
FROM employees WHERE emp_id = 7;
SELECT regexp_replace('(022) 555-0199', '[^0-9]', '', 'g') AS digits_only,
       (regexp_match('order #A-1042 shipped', '#([A-Z])-(\d+)'))[2] AS order_number;
```

Output:

```
       sentence       |                    safe_sql
----------------------+-------------------------------------------------
 Priya earns 72000.00 | SELECT * FROM "my table" WHERE name = 'O''Brien'
(1 row)

 digits_only | order_number
-------------+--------------
 0225550199  | 1042
(1 row)
```

`%I` and `%L` quote an identifier and a literal safely — the basis of safe dynamic SQL (SQL text built as a string while the program runs, then executed; quoting stops *SQL injection*, where crafted input changes the meaning of the query — chapter 14). `regexp_replace(…, 'g')` replaces all matches (`g` = global); `regexp_match` returns an array of captured groups (the pieces of text matched by each bracketed part of the pattern), and `[2]` picks the second.

Cleaning real data

Our two Anita Shah rows differ only in formatting. Normalising them shows they're the same person:

```sql
SELECT customer_id, email AS raw, lower(trim(email)) AS clean, initcap(city) AS city_fixed
FROM customers WHERE last_name = 'Shah' ORDER BY customer_id;
```

Output:

```
 customer_id |            raw             |          clean          | city_fixed
-------------+----------------------------+-------------------------+------------
           1 | anita.shah@mail.example    | anita.shah@mail.example | Mumbai
           8 |   Anita.Shah@Mail.example  | anita.shah@mail.example | Mumbai
(2 rows)
```

Better still, clean on the way *in* (application validation, a `CHECK` constraint — a rule the database tests on every insert — or a unique index on `lower(email)`, which refuses a second row with the same value — chapter 12).

Full names and email domains:

```sql
SELECT first_name || ' ' || last_name AS full_name,
       split_part(email, '@', 2) AS domain
FROM employees WHERE emp_id IN (1, 3, 5) ORDER BY emp_id;
```

Output:

```
 full_name  |     domain
------------+-----------------
 Asha Rao   | company.example
 Neha Gupta | Company.example
 Meera Iyer | (null)
(3 rows)
```

Meera has no email, so the domain is NULL — functions given NULL generally return NULL.

Dialect map (strings)

Task	MySQL	PostgreSQL	SQL Server	SQLite
Concatenate	`CONCAT(a,b)` (NULL if any NULL)	`a || b`, `concat()` (skips NULL)	`a + b`, `CONCAT()` (NULL → '')	`a || b`, `concat()` (3.44+)
Characters	`CHAR_LENGTH(s)`	`length(s)` / `char_length(s)`	`LEN(s)` (no trailing spaces)	`length(s)`
Bytes	`LENGTH(s)`	`octet_length(s)`	`DATALENGTH(s)`	`length(CAST(s AS BLOB))` (BLOB = raw binary data)
Find position	`LOCATE(sub, s)`, `INSTR(s, sub)`, `POSITION(sub IN s)`	`position(sub IN s)`, `strpos(s, sub)`	`CHARINDEX(sub, s)`	`instr(s, sub)`
Split field	`SUBSTRING_INDEX(s, d, n)`	`split_part(s, d, n)`	`STRING_SPLIT` (rows) / `PARSENAME`	— (use `instr` + `substr`)
Left/right	`LEFT`, `RIGHT`	`left`, `right`	`LEFT`, `RIGHT`	`substr(s, 1, n)`, `substr(s, -n)`
Pad	`LPAD`, `RPAD`	`lpad`, `rpad`	`RIGHT(REPLICATE('0', n) + s, n)`	`printf('%05d', n)`
Repeat	`REPEAT`	`repeat`	`REPLICATE`	—
Regex replace	`REGEXP_REPLACE` (8.0+)	`regexp_replace`	`REGEXP_REPLACE` (SQL Server 2025)	extension

Common mistakes

- Assuming positions start at 0.
- Using `||` when a part may be NULL and getting a NULL result; use `concat`/`concat_ws` or `COALESCE`.
- Counting bytes instead of characters (MySQL `LENGTH`).
- Comparing case-insensitively without `lower()`/`ILIKE` in PostgreSQL.

Self-check

1. *Get the text after the `@` in an email in PostgreSQL?* → `split_part(email, '@', 2)`.
2. *`'a' || NULL` vs `concat('a', NULL)`?* → NULL vs `'a'`.
3. *Pad an order number 42 to `00042`?* → `lpad(42::text, 5, '0')` (or `to_char(42, 'FM00000')`).

TL;DR

`||`/`concat`/`concat_ws` join (mind NULLs), `substring`/`left`/`right`/`split_part` cut, `length`/`octet_length` measure, `lower`/`upper`/`initcap`/`trim`/`replace` clean, `position`/`strpos` search, `lpad`/`rpad` pad, `format` and `regexp_*` for power tasks.

3.4 Date and time functions

What it is

Time values cause more subtle SQL bugs than any other kind of data: months have different lengths, time zones shift, and "now" depends on when you ask. PostgreSQL has a rich, precise set of types and functions for them.

The types

Type	Holds	Example
`date`	a calendar date	`2025-03-18`
`time`	a time of day	`19:30:00`
`timestamp` (without time zone)	date + time, *no* zone — a wall-clock reading	`2025-03-18 19:30:00`
`timestamptz` (with time zone)	an absolute **instant** (one exact moment, the same everywhere on Earth); stored in UTC, shown in the session's time zone	`2025-03-18 14:00:00+00`
`interval`	a duration	`3 days`, `1 mon`, `02:30:00`

Guideline: use `timestamptz` for "when did it happen" (events, orders, logins) and `date` for calendar facts (birthdays, hire dates). Our sample `orders.placed_at` is a plain `timestamp` to keep outputs identical on every machine.

"Now"

A **transaction** is a group of statements that succeed or fail together, started with `BEGIN` and ended with `COMMIT` (a single statement on its own is also a transaction). Several "now" functions freeze at the transaction's start:

Function	Returns	Changes within a transaction?
`current_date`	today's `date`	no
`current_time`	time of day with zone (`timetz`)	no
`localtime`	time of day without zone	no
`now()` = `current_timestamp` = `transaction_timestamp()`	`timestamptz` at transaction start	**no**
`localtimestamp`	`timestamp` at transaction start	no
`statement_timestamp()`	start of the current statement	per statement
`clock_timestamp()`	the actual current time	**yes, every call**

```
-- code/04_date_functions.sql (the file sets SET TIME ZONE 'UTC' first)
SELECT current_date AS today, current_time AS time_with_tz, localtime AS local_time,
       now() AS now_tstz, localtimestamp AS local_ts;
```

Output (from the run that produced this book; yours will show your current time):

```
   today    |    time_with_tz    |   local_time    |           now_tstz            |          local_ts
------------+--------------------+-----------------+-------------------------------+----------------------
-------
 2026-09-28 | 19:15:38.702642+00 | 19:15:38.702642 | 2026-09-28 19:15:38.702642+00 | 2026-09-28
19:15:38.702642
(1 row)
```

`now()` is frozen for the whole transaction, so every row inserted in one transaction gets the same timestamp — usually what you want. `clock_timestamp()` keeps moving:

```
BEGIN;
SELECT now() AS tx_start, clock_timestamp() AS wall_clock;
SELECT pg_sleep(0.2);
SELECT now() AS tx_start_again, clock_timestamp() AS wall_clock_later;
COMMIT;
```

Output:

```
           tx_start            |          wall_clock
-------------------------------+-------------------------------
 2026-09-28 19:15:38.702968+00 | 2026-09-28 19:15:38.703095+00
(1 row)

        tx_start_again         |       wall_clock_later
-------------------------------+-------------------------------
 2026-09-28 19:15:38.702968+00 | 2026-09-28 19:15:38.905532+00
(1 row)
```

Taking the date part of a timestamp

MySQL's `DATE(ts)` is a cast in PostgreSQL:

```
SELECT placed_at, placed_at::date AS just_date, date_trunc('day', placed_at) AS truncated
FROM orders WHERE order_id = 103;
```

Output:

```
      placed_at      | just_date  |      truncated
---------------------+------------+---------------------
 2025-01-31 23:55:00 | 2025-01-31 | 2025-01-31 00:00:00
(1 row)
```

`::date` returns a `date`; `date_trunc('day', …)` returns a timestamp at midnight.

Date arithmetic

Expression	Result type	Meaning
`date + integer`	date	add days
`date - date`	integer	days between
`date/timestamp ± interval`	timestamp	add/subtract a duration
`timestamp - timestamp`	interval	elapsed time
`age(later, earlier)`	interval	"years mons days" difference

```
SELECT date '2024-01-01' + 3 AS plus_3_days, date '2024-01-01' + interval '3 days' AS plus_interval,
       date '2024-01-01' - interval '1 month' AS minus_month, date '2024-01-05' - date '2024-01-01' AS
days_between;
```

Output:

```
 plus_3_days |    plus_interval    |     minus_month     | days_between
-------------+---------------------+---------------------+--------------
 2024-01-04  | 2024-01-04 00:00:00 | 2023-12-01 00:00:00 |            4
(1 row)
```

Month arithmetic clamps to the last valid day, which makes it non-reversible:

```
SELECT date '2025-01-31' + interval '1 month' AS jan31_plus_month,
       date '2024-02-29' + interval '1 year' AS leap_plus_year,
       date '2025-01-31' + interval '1 month' - interval '1 month' AS round_trip;
```

Output:

```
  jan31_plus_month   |   leap_plus_year    |     round_trip
---------------------+---------------------+---------------------
 2025-02-28 00:00:00 | 2025-02-28 00:00:00 | 2025-01-28 00:00:00
(1 row)
```

31 January + 1 month = 28 February, and going back a month lands on 28 January, not the 31st. Billing systems must decide explicitly what "one month later" means.

Extracting fields: EXTRACT and date_part

`extract(field FROM value)` is standard SQL; `date_part('field', value)` is PostgreSQL's function form. Since PostgreSQL 14, `extract` returns `numeric`.

```
SELECT extract(YEAR FROM date '2024-01-01') AS year, extract(MONTH FROM date '2024-01-01') AS month,
       extract(DAY FROM date '2024-01-01') AS day, extract(DOW FROM date '2024-01-01') AS dow_0_sun,
       extract(ISODOW FROM date '2024-01-01') AS isodow_1_mon, extract(QUARTER FROM date '2024-08-15') AS
quarter,
       date_part('week', date '2024-01-01') AS iso_week;
```

Output:

```
 year | month | day | dow_0_sun | isodow_1_mon | quarter | iso_week
------+-------+-----+-----------+--------------+---------+----------
 2024 |     1 |   1 |         1 |            1 |       3 |        1
(1 row)
```

1 January 2024 was a **Monday**: `DOW` numbers Sunday 0 … Saturday 6, `ISODOW` numbers Monday 1 … Sunday 7. Other useful fields: `HOUR`, `MINUTE`, `SECOND`, `DOY` (day of year), `EPOCH` (seconds since 1970-01-01 UTC).

Truncating to a period: date_trunc

`date_trunc(unit, ts)` rounds *down* to the start of the unit — the workhorse of time-series reports (reports of values over time, such as "sales per month"):

```
SELECT placed_at, date_trunc('month', placed_at) AS month_start, date_trunc('week', placed_at) AS
week_start,
       date_trunc('hour', placed_at) AS hour_start
FROM orders WHERE order_id = 107;
```

Output:

```
      placed_at      |     month_start     |     week_start      |     hour_start
---------------------+---------------------+---------------------+---------------------
 2025-03-18 19:30:00 | 2025-03-01 00:00:00 | 2025-03-17 00:00:00 | 2025-03-18 19:00:00
(1 row)
```

Weeks start on **Monday** (ISO 8601, the international standard for writing dates and times). Grouping by `date_trunc('month', …)` gives one sortable key per month — see exercise E5 in 3.6.

Formatting and parsing

`to_char(value, pattern)` formats; `to_date` / `to_timestamp` parse; `make_date` / `make_timestamp` build from numbers.

Pattern	Meaning	Example
`YYYY`, `MM`, `DD`	year, month number, day	`2025`, `03`, `18`
`Month`, `Mon`	month name, abbreviation	`March`, `Mar`
`Day`, `Dy`	weekday name, abbreviation	`Tuesday`, `Tue`
`HH24`, `HH12`, `MI`, `SS`, `AM`	hour, minute, second, meridiem	`19`, `07`, `30`, `00`, `PM`
`Q`	quarter	`1`
`FM` prefix	drop padding spaces/zeros	`FMDay` → `Monday`
`"text"`	literal text	`"Q"`

```
SELECT to_char(date '2024-01-01', 'FMDay, FMMonth FMDD, YYYY') AS long_form,
       to_char(timestamp '2025-03-18 19:30', 'DD Mon YYYY HH12:MI AM') AS short_form,
       to_char(timestamp '2025-03-18 19:30', 'YYYY-"Q"Q') AS quarter_label;
SELECT to_date('18/03/2025', 'DD/MM/YYYY') AS parsed_date,
       to_timestamp('2025-03-18 19:30', 'YYYY-MM-DD HH24:MI') AS parsed_ts,
       make_date(2025, 3, 18) AS made_date;
```

Output:

```
        long_form        |      short_form      | quarter_label
-------------------------+----------------------+---------------
 Monday, January 1, 2024 | 18 Mar 2025 07:30 PM | 2025-Q1
(1 row)

 parsed_date |       parsed_ts        | made_date
-------------+------------------------+------------
 2025-03-18  | 2025-03-18 19:30:00+00 | 2025-03-18
(1 row)
```

`to_timestamp` returns a `timestamptz` interpreted in the session time zone (UTC here). For ISO strings (`'2025-03-18'`) a plain cast is simpler; use `to_date` for other layouts like `18/03/2025`, which is ambiguous otherwise.

Differences between moments

```
SELECT order_id, placed_at, shipped_at,
       shipped_at - placed_at AS time_to_ship,
       round(extract(EPOCH FROM shipped_at - placed_at) / 3600, 1) AS hours_to_ship,
       shipped_at::date - placed_at::date AS calendar_days
FROM orders WHERE shipped_at IS NOT NULL AND order_id <= 104 ORDER BY order_id;
```

Output:

```
 order_id |      placed_at      |     shipped_at      |  time_to_ship  | hours_to_ship | calendar_days
----------+---------------------+---------------------+----------------+---------------+---------------
      101 | 2025-01-03 10:15:00 | 2025-01-04 09:00:00 | 22:45:00       |          22.8 |             1
      102 | 2025-01-17 22:40:00 | 2025-01-19 11:30:00 | 1 day 12:50:00 |          36.8 |             2
      103 | 2025-01-31 23:55:00 | 2025-02-02 08:10:00 | 1 day 08:15:00 |          32.3 |             2
      104 | 2025-02-11 09:05:00 | 2025-02-12 16:45:00 | 1 day 07:40:00 |          31.7 |             1
(4 rows)
```

Three different answers to "how long did shipping take?": an `interval`, a number of hours (via `EPOCH` seconds ÷ 3600), and the number of calendar-date boundaries crossed. Order 101 took under a day but crossed one midnight. Decide which one the business means.

`age()` gives a human-style difference:

```
SELECT first_name, hire_date, age(date '2025-07-01', hire_date) AS tenure_on_2025_07_01
FROM employees WHERE emp_id IN (1, 9, 15) ORDER BY emp_id;
```

Output:

```
 first_name | hire_date  |  tenure_on_2025_07_01
------------+------------+-------------------------
 Asha       | 2015-01-05 | 10 years 5 mons 27 days
 Sara       | 2024-04-01 | 1 year 3 mons
 Rohan      | 2025-06-02 | 29 days
(3 rows)
```

`age(x)` with one argument measures from `current_date` at midnight. Whole years of service: `extract(YEAR FROM age(current_date, hire_date))`.

Expected delivery ten days after ordering:

```
SELECT order_id, placed_at::date AS order_date, (placed_at + interval '10 days')::date AS expected_by
FROM orders WHERE order_id IN (101, 105, 113) ORDER BY order_id;
```

Output:

```
 order_id | order_date | expected_by
----------+------------+-------------
      101 | 2025-01-03 | 2025-01-13
      105 | 2025-02-28 | 2025-03-10
      113 | 2025-06-02 | 2025-06-12
(3 rows)
```

Time zones

A `timestamptz` is one instant; how it *prints* depends on the session's `TimeZone` setting. `AT TIME ZONE` converts between instants and wall-clock readings:

Expression	Gives
`timestamptz AT TIME ZONE 'Zone'`	the wall-clock `timestamp` in that zone at that instant
`timestamp AT TIME ZONE 'Zone'`	the `timestamptz` instant when that zone's clock showed that reading

```
SET TIME ZONE 'UTC';
SELECT timestamptz '2025-03-18 19:30:00+05:30' AS shown_in_utc;
SET TIME ZONE 'Asia/Kolkata';
SELECT timestamptz '2025-03-18 19:30:00+05:30' AS shown_in_ist,
       timestamptz '2025-03-18 19:30:00+05:30' AT TIME ZONE 'America/New_York' AS wall_clock_ny,
       timestamp '2025-03-18 19:30' AT TIME ZONE 'Europe/London' AS london_wall_clock_as_instant;
```

Output:

```
      shown_in_utc
-----------------------
 2025-03-18 14:00:00+00
(1 row)

       shown_in_ist        |    wall_clock_ny    | london_wall_clock_as_instant
---------------------------+---------------------+------------------------------
 2025-03-18 19:30:00+05:30 | 2025-03-18 10:00:00 | 2025-03-19 01:00:00+05:30
(1 row)
```

7:30 pm in India is 14:00 UTC and 10:00 in New York (already on daylight-saving time on 18 March). 7:30 pm in London (still on GMT, Greenwich Mean Time — the UK's winter clock, equal to UTC) is 01:00 the next day in India. Use named zones (`'Asia/Kolkata'`), not fixed offsets, so daylight-saving rules are applied.

A calendar of dates

`generate_series` produces rows — here, one per day — which is how you build calendars or fill gaps in reports (chapter 7):

```
SELECT d::date AS day, to_char(d, 'Dy') AS weekday
FROM generate_series(date '2025-03-01', date '2025-03-05', interval '1 day') AS d;
```

Output:

```
    day     | weekday
------------+---------
 2025-03-01 | Sat
 2025-03-02 | Sun
 2025-03-03 | Mon
 2025-03-04 | Tue
 2025-03-05 | Wed
(5 rows)
```

Dialect map (dates)

Task	MySQL	PostgreSQL	SQL Server	SQLite
Now	`NOW()`	`now()`	`SYSDATETIME()` / `GETDATE()`	`datetime('now')`
Today	`CURRENT_DATE` / `CURDATE()`	`current_date`	`CAST(GETDATE() AS date)`	`date('now')`
Date part of datetime	`DATE(ts)`	`ts::date`	`CAST(ts AS date)`	`date(ts)`
Add 3 days	`DATE_ADD(d, INTERVAL 3 DAY)`	`d + interval '3 days'` (or `d + 3` for a date)	`DATEADD(day, 3, d)`	`date(d, '+3 days')`
Subtract a month	`DATE_SUB(d, INTERVAL 1 MONTH)`	`d - interval '1 month'`	`DATEADD(month, -1, d)`	`date(d, '-1 month')`
Year / month / day	`YEAR(d)`, `MONTH(d)`, `DAY(d)`, `EXTRACT`	`extract(YEAR FROM d)` …	`YEAR(d)`, `DATEPART(month, d)`	`strftime('%Y', d)`
Format	`DATE_FORMAT(d, '%W, %M %e, %Y')`	`to_char(d, 'FMDay, FMMonth FMDD, YYYY')`	`FORMAT(d, 'dddd, MMMM d, yyyy')`	`strftime(…)`
Days between	`DATEDIFF(end, start)`	`end::date - start::date`	`DATEDIFF(day, start, end)`	`julianday(end) - julianday(start)`
Difference in a unit	`TIMESTAMPDIFF(HOUR, start, end)`	`extract(EPOCH FROM end - start) / 3600`	`DATEDIFF(hour, start, end)`	`(julianday(end) - julianday(start)) * 24`
Truncate to month	`DATE_FORMAT(d, '%Y-%m-01')`	`date_trunc('month', d)`	`DATETRUNC(month, d)` (2022+)	`date(d, 'start of month')`

Note the argument orders: MySQL `DATEDIFF(end, start)` vs SQL Server `DATEDIFF(unit, start, end)`. SQL Server's `DATEDIFF` counts *boundaries crossed* (like `calendar_days` above), not elapsed time.

Common mistakes

- Comparing a timestamp column to a date-only upper bound (see chapter 2, 2.4) — use half-open ranges.
- Applying a function to an indexed column (one with an **index** — a sorted lookup structure that lets the database find rows without reading them all) in `WHERE` (`WHERE date_trunc('month', placed_at) = …`) — it can stop the index being used; rewrite as a range (chapter 12).
- Storing local times in `timestamp` for a multi-time-zone app — use `timestamptz`.
- Assuming "+1 month" can be undone with "−1 month".

In the interview

30-second answer: "I store instants as timestamptz, truncate with date_trunc for per-day or per-month reports, filter with half-open ranges so indexes still work, and compute durations with timestamp subtraction or EXTRACT(EPOCH …). I'm careful that month arithmetic clamps and that now() is fixed per transaction."

- *Orders per month including months with zero orders?* → Generate the months with `generate_series` and left-join the counts (a *left join* keeps every row of the first table even when the second has no match; chapters 5 and 7).
- *Difference between `timestamp` and `timestamptz`?* → `timestamptz` is an absolute instant converted to/from the session zone; `timestamp` is a zone-less wall-clock reading.

Self-check

1. *`date '2025-03-31' - interval '1 month'`?* → `2025-02-28 00:00:00`.
2. *First day of the month of `placed_at`?* → `date_trunc('month', placed_at)`.
3. *Why might two `now()` calls in one transaction be equal?* → `now()` returns the transaction start time.
4. *Hours between two timestamps as a number?* → `extract(EPOCH FROM b - a) / 3600`.

TL;DR

`date`, `timestamp`, `timestamptz`, `interval`. `now()` (transaction start) vs `clock_timestamp()`. Add intervals; subtract dates for days; `extract` fields; `date_trunc` to periods; `to_char`/`to_date` to format/parse; `AT TIME ZONE` converts; month maths clamps.

3.5 Conditional expressions: CASE, COALESCE, NULLIF, GREATEST, LEAST

What it is

Conditional expressions put *if-then-else* logic inside a query. They are expressions, so they return a value and can be used wherever a column can.

CASE

Searched CASE tests conditions in order and returns the result of the first true one:

```sql
-- code/05_case_coalesce_nullif.sql
SELECT name, unit_price,
       CASE WHEN unit_price >= 300 THEN 'premium'
            WHEN unit_price >= 50  THEN 'standard'
            ELSE 'budget' END AS price_band
FROM products ORDER BY unit_price DESC;
```

Output:

```
            name             | unit_price | price_band
-----------------------------+------------+------------
 Laptop Pro 14               |    1299.00 | premium
 Standing Desk               |     499.00 | premium
 Ergonomic Chair             |     349.99 | premium
 Noise-Cancelling Headphones |     199.95 | standard
 Mechanical Keyboard         |      89.00 | standard
 Gift Card                   |      50.00 | standard
 USB-C Hub                   |      49.50 | budget
 Desk Lamp                   |      35.00 | budget
 Wireless Mouse              |      24.99 | budget
(9 rows)
```

Order matters: a 1,299 laptop is also `>= 50`, but the first matching branch wins. Write branches from most to least specific.

Simple CASE compares one expression with a list of values:

```sql
SELECT order_id, status,
       CASE status WHEN 'pending' THEN 'Awaiting dispatch'
                   WHEN 'shipped' THEN 'On the way'
                   WHEN 'delivered' THEN 'Done'
       END AS customer_message
FROM orders WHERE order_id IN (101, 105, 107, 113) ORDER BY order_id;
```

Output:

```
 order_id |  status   | customer_message
----------+-----------+------------------
      101 | delivered | Done
      105 | cancelled | (null)
      107 | shipped   | On the way
      113 | pending   | Awaiting dispatch
(4 rows)
```

No `ELSE` means `ELSE NULL` — the cancelled order gets NULL. Always write an `ELSE` unless NULL is really what you want. Also, simple `CASE x WHEN NULL` never matches (it uses `=`); test NULLs with a searched `CASE WHEN x IS NULL`.

`CASE` appears everywhere: in `ORDER BY` for custom sort orders (chapter 2, 2.7), inside aggregates for conditional counts (`count(*) FILTER (…)` / `sum(CASE …)`, chapter 4), in `UPDATE … SET` for conditional changes (chapter 9), and for pivots (turning row values into columns, e.g. one column per month; chapter 14).

COALESCE — first non-NULL

`coalesce(a, b, c, …)` returns the first argument that isn't NULL (or NULL if all are). It's the standard way to supply defaults.

```sql
SELECT first_name, email, coalesce(email, 'no email on file') AS contact,
       commission_pct, coalesce(commission_pct, 0) AS commission_or_zero
FROM employees WHERE emp_id IN (5, 6, 9, 11) ORDER BY emp_id;
```

Output:

```
 first_name |             email             |            contact            | commission_pct |
commission_or_zero
------------+-------------------------------+-------------------------------+----------------+-----------
---------
 Meera      | (null)                        | no email on file              |         (null) |
0
 Daniel     | daniel.brooks@company.example | daniel.brooks@company.example |           0.05 |
0.05
 Sara       | sara.khan@company.example     | sara.khan@company.example     |         (null) |
0
 Ananya     | (null)                        | no email on file              |         (null) |
0
(4 rows)
```

Why it matters — NULL "poisons" arithmetic:

```sql
SELECT first_name, salary, commission_pct,
       salary * (1 + commission_pct)              AS naive_total,
       salary * (1 + coalesce(commission_pct, 0)) AS total
FROM employees WHERE dept_id = 2 ORDER BY emp_id;
```

Output:

```
 first_name |  salary   | commission_pct | naive_total |    total
------------+-----------+----------------+-------------+-------------
 Daniel     | 150000.00 |           0.05 | 157500.0000 | 157500.0000
 Priya      |  72000.00 |           0.10 |  79200.0000 |  79200.0000
 Arjun      |  68000.00 |           0.10 |  74800.0000 |  74800.0000
 Sara       |  68000.00 |         (null) |      (null) |    68000.00
(4 rows)
```

Without `coalesce`, Sara's total pay is unknown. All arguments must share a compatible type — `coalesce(salary, 'n/a')` fails; convert first (`coalesce(salary::text, 'n/a')`).

NULLIF — turn a value into NULL

`nullif(a, b)` returns NULL if `a = b`, otherwise `a`. Two classic uses:

```sql
-- 1. Avoid division by zero
SELECT a, b, a / NULLIF(b, 0) AS safe_ratio
FROM (VALUES (10, 2), (10, 0)) AS t(a, b);

-- 2. Treat blank strings as missing
SELECT email, NULLIF(trim(email), '') AS email_or_null
FROM (VALUES ('x@y.example'), ('   '), (NULL)) AS t(email);
```

Output:

```
 a  | b | safe_ratio
----+---+------------
 10 | 2 |          5
 10 | 0 |     (null)
(2 rows)

    email    | email_or_null
-------------+---------------
 x@y.example | x@y.example
             | (null)
 (null)      | (null)
(3 rows)
```

`x / 0` raises *division by zero* and aborts the whole query; `x / NULLIF(0, 0)` is `x / NULL` = NULL. Combine them: `coalesce(a / nullif(b, 0), 0)` if a zero is more useful than NULL.

CASE with dates and NULLs together

```sql
SELECT name, start_date, end_date,
       coalesce(end_date, date '2025-12-31') AS planned_or_open_end,
       CASE WHEN end_date IS NULL THEN 'running' ELSE 'finished' END AS state
FROM projects ORDER BY project_id;
```

Output:

```
       name       | start_date |  end_date  | planned_or_open_end |  state
------------------+------------+------------+---------------------+----------
 Checkout Revamp  | 2024-09-01 | 2025-03-31 | 2025-03-31          | finished
 Mobile App       | 2025-01-15 | (null)     | 2025-12-31          | running
 Spring Campaign  | 2025-02-01 | 2025-04-30 | 2025-04-30          | finished
 Year-End Audit   | 2024-11-01 | 2025-01-31 | 2025-01-31          | finished
 Helpdesk Chatbot | 2025-04-01 | (null)     | 2025-12-31          | running
(5 rows)
```

GREATEST and LEAST

Return the largest / smallest of their arguments (not across rows — that's `max`/`min`, chapter 4). In PostgreSQL they **ignore NULLs**:

```
SELECT greatest(1, NULL, 3) AS greatest, least(1, NULL, 3) AS least;
```

```
 greatest | least
----------+-------
        3 |     1
(1 row)
```

MySQL and Oracle return NULL if any argument is NULL — wrap arguments in `coalesce` for portable code. Typical use: `greatest(price - discount, 0)` to floor a value at zero.

Dialect map (conditionals)

Task	Standard / PostgreSQL	MySQL	SQL Server	Oracle
If-then-else	`CASE WHEN … THEN … ELSE … END`	also `IF(cond, a, b)`	also `IIF(cond, a, b)`	also `DECODE(…)`
First non-NULL	`COALESCE(a, b, …)`	also `IFNULL(a, b)`	also `ISNULL(a, b)`	also `NVL(a, b)`
Value → NULL	`NULLIF(a, b)`	same	same	same
Greatest/least	`GREATEST`, `LEAST` (ignore NULLs)	same names (NULL if any NULL)	`GREATEST` / `LEAST` (2022+, ignore NULLs)	same names (NULL if any NULL)

Prefer the standard forms (`CASE`, `COALESCE`, `NULLIF`) — they work everywhere.

Common mistakes

- Forgetting `ELSE` and getting unexpected NULLs.
- Overlapping `WHEN` conditions in the wrong order.
- `CASE x WHEN NULL …` — never true.
- Mixing result types in branches (`THEN 1 ELSE 'none'`) — PostgreSQL errors; make them the same type.

In the interview

30-second answer: "CASE is SQL's if-else and works anywhere an expression does — SELECT, ORDER BY, inside SUM for conditional aggregation. COALESCE returns the first non-null, so I use it for defaults and to stop NULL wiping out arithmetic. NULLIF turns a value into NULL, mainly to avoid division by zero."

- *Count delivered and cancelled orders in one row?* → `sum(CASE WHEN status='delivered' THEN 1 ELSE 0 END)`, or `count(*) FILTER (WHERE status='delivered')` in PostgreSQL (chapter 4).
- *`ISNULL` vs `COALESCE` in SQL Server?* → `ISNULL` takes two arguments and uses the first argument's type; `COALESCE` is standard, takes many, and uses type precedence (the rule for which type wins when arguments differ).

Self-check

1. *What does a CASE return when no branch matches and there's no ELSE?* → NULL.
2. *Safe average price per item when quantity may be 0?* → `total / NULLIF(quantity, 0)`.
3. *`coalesce(NULL, NULL, 'x', 'y')`?* → `'x'`.

TL;DR

`CASE` = if/else (first true branch wins; no `ELSE` → NULL). `COALESCE` = first non-NULL (defaults). `NULLIF` = value → NULL (safe division, blank → NULL). `GREATEST`/`LEAST` compare arguments (PostgreSQL skips NULLs).

3.6 Worked exercises

Try each before reading the solution. They combine this chapter with a preview of aggregates (`sum`, `count`, `GROUP BY` — chapter 4) and a simple join (`JOIN … USING` — a **join** pairs rows of two tables whose shared column, here `order_id`, matches; chapter 5).

E1. Total value of all delivered orders, rounded to 2 decimals and shown with thousands separators.

```sql
-- code/06_function_exercises.sql
SELECT to_char(round(sum(quantity * unit_price * (1 - discount)), 2), 'FM999,999,990.00') AS total_sales
FROM order_items oi
JOIN orders o USING (order_id)
WHERE o.status = 'delivered';
```

```
 total_sales
-------------
 4,508.11
(1 row)
```

Compute with numbers, round, and only then format — `to_char` is the last step.

E2. The square root of the number of order lines, and the natural log of total revenue.

```sql
SELECT round(sqrt(count(*))::numeric, 3) AS sqrt_lines,
       round(ln(sum(quantity * unit_price * (1 - discount))), 4) AS ln_revenue
FROM order_items;
```

```
 sqrt_lines | ln_revenue
------------+------------
      4.690 |     9.2130
(1 row)
```

`sqrt` of an integer returns `double precision`, and `round(x, d)` with a decimal-places argument needs `numeric` — hence the cast. Note `ln`, not `log`, for the natural logarithm.

E3. Customer display names like "Shah, A." and lower-case email domains.

```sql
SELECT customer_id,
       last_name || ', ' || left(first_name, 1) || '.' AS display_name,
       lower(split_part(trim(email), '@', 2))          AS domain
FROM customers ORDER BY customer_id LIMIT 5;
```

```
 customer_id | display_name |    domain
-------------+--------------+--------------
           1 | Shah, A.     | mail.example
           2 | Carter, B.   | mail.example
           3 | Li, C.       | mail.example
           4 | Menon, D.    | (null)
           5 | Walker, E.   | mail.example
(5 rows)
```

E4. Revenue per year and month, excluding cancelled orders.

```
SELECT extract(YEAR FROM o.placed_at)  AS year,
       extract(MONTH FROM o.placed_at) AS month,
       round(sum(oi.quantity * oi.unit_price * (1 - oi.discount)), 2) AS revenue
FROM orders o
JOIN order_items oi USING (order_id)
WHERE o.status <> 'cancelled'
GROUP BY 1, 2
ORDER BY 1, 2;
```

```
 year | month | revenue
------+-------+---------
 2025 |     1 | 1871.90
 2025 |     2 |  811.49
 2025 |     3 |  804.95
 2025 |     4 | 2148.75
 2025 |     5 |  249.93
 2025 |     6 | 1516.60
(6 rows)
```

E5. The same idea with one sortable month key (orders per month):

```
SELECT to_char(date_trunc('month', o.placed_at), 'YYYY-MM') AS month,
       count(DISTINCT o.order_id) AS orders
FROM orders o
WHERE o.status <> 'cancelled'
GROUP BY date_trunc('month', o.placed_at)
ORDER BY date_trunc('month', o.placed_at);
```

```
  month  | orders
---------+--------
 2025-01 |      3
 2025-02 |      1
 2025-03 |      2
 2025-04 |      3
 2025-05 |      1
 2025-06 |      2
(6 rows)
```

Grouping by `date_trunc` keeps a real timestamp for sorting; `to_char` only dresses it up for display. The expected-delivery exercise (order date + 10 days) is in 3.4.

Chapter 3 TL;DR

- **Types:** everything has one; `numeric` for money, `text` for strings, `date`/`timestamptz` for time; convert with `CAST(x AS t)` or `x::t`; PostgreSQL refuses silent coercions.
- **Numbers:** `round`/`trunc`/`ceil`/`floor` (numeric rounds half away from zero, floats to even), `power`, `sqrt`, `exp`, `ln`, `log` (base 10 in PostgreSQL!), `mod`/`%`, `random` + `setseed`, `to_char` for display.
- **Strings:** `||` vs `concat` (NULL handling), `substring`/`left`/`right`/`split_part`, `length` vs `octet_length`, `lower`/`upper`/`initcap`, `trim`, `replace`/`translate`, `position`/`strpos`, `lpad`/`rpad`, `format`, `regexp_replace`.
- **Dates:** `now()` vs `clock_timestamp()`, `+ interval`, `date - date`, `age`, `extract`, `date_trunc`, `to_char`/`to_date`, `AT TIME ZONE`; month arithmetic clamps.

- **Conditionals:** `CASE` (no ELSE → NULL), `COALESCE` (defaults), `NULLIF` (safe division), `GREATEST`/`LEAST`.
- MySQL names differ a lot (`DATE_ADD`, `DATE_FORMAT`, `SUBSTRING_INDEX`, `IFNULL`, `LOG` = ln, `LENGTH` = bytes) — use the dialect maps.

Next: Chapter 4 — Aggregation.

Chapter 4 — Aggregation: Summarising Many Rows

Chapters 2 and 3 worked **row by row**: `WHERE` kept or dropped each row, and functions such as `upper()` or `round()` turned one row's value into another value for the same row. Most business questions are not about single rows, though. "How many orders did we ship last month?", "What is the average salary per department?", "Which customers spent more than 1,000?" — each of these squeezes **many rows into one answer**. That squeezing is called **aggregation**, and it is the subject of this chapter.

What you'll learn

- The aggregate functions (`COUNT`, `SUM`, `AVG`, `MIN`, `MAX` and PostgreSQL's useful extras) and exactly how each treats `NULL` and empty input.
- `GROUP BY` — splitting rows into groups and getting one result row per group — and the rule about which columns you may select.
- `HAVING` — filtering whole groups after they have been summarised — and how it differs from `WHERE`.
- Conditional aggregation with `FILTER (WHERE …)` and `CASE`, the trick behind most "count several things in one pass" and pivot-style reports (reports that turn row values into columns, §4.4).
- The **logical query processing order** — the order in which the database *conceptually* evaluates the clauses of a `SELECT` — and why it explains almost every "column does not exist" error.
- Subtotals and grand totals in a single query with `GROUPING SETS`, `ROLLUP` and `CUBE`.

How to run the examples. Every query in this chapter is in `code/` and was run on the book's private PostgreSQL 14 server; the output under each query is the real `psql` output. `(null)` in the output is how we asked `psql` to display `NULL` (`\pset null '(null)'`).

```
bash tools_run_sql.sh sql '04-*'                          # run this chapter's files (fresh DB per file)
bash tools_pg.sh psql -d <db> -f sql/chapters/04-aggregation/code/02_group_by.sql
```

The sample data. We keep using the book's `company` database from Chapter 1 (`code/00_schema.sql` is an exact copy). The tables that matter most here:

Table	Rows	What is in it (the quirks we will meet)
`employees`	15	`emp_id`, `first_name`, `dept_id`, `salary`, `commission_pct`, `hire_date`, `manager_id`. The CEO (Asha) and the intern (Rohan) have **no department** (`dept_id` is `NULL`); only 3 people have a commission; two salaries are shared by two people each.
`departments`	6	Engineering, Sales, Marketing, Finance, Support, Research (Research has **no** employees).
`orders`	14	`status` is one of pending / shipped / delivered / cancelled; `sales_rep_id` is `NULL` for online orders.
`order_items`	22	One row per product in an order: `quantity`, `unit_price` (price paid), `discount` (0.10 = 10 % off).
`products`, `customers`	9, 10	One product has no category; two customers have never ordered.

4.1 Aggregate functions

What it is

An **aggregate function** takes a *set of rows* and returns *one value* that summarises them. Think of a shop's till at closing time: it does not print every sale again — it prints the number of sales, the total taken, the largest single sale. Those are aggregates.

Function	Returns	Works on	Notes
`COUNT(*)`	number of **rows**	any rows	Counts rows, even rows that are all `NULL`.
`COUNT(expr)`	number of rows where `expr` is **not NULL**	any type	The usual way to count "how many have a value".
`COUNT(DISTINCT expr)`	number of **different** non-NULL values	any type	Duplicates and `NULL`s are dropped first.
`SUM(expr)`	total	numbers, intervals (durations)	`NULL` if there are no non-NULL inputs.
`AVG(expr)`	arithmetic mean = `SUM / COUNT(expr)`	numbers, intervals	Divides by the count of **non-NULL** values.
`MIN(expr)` / `MAX(expr)`	smallest / largest	anything that can be sorted: numbers, text, dates	Text uses the column's collation (alphabetical order).

Two words you will see a lot: a **scalar function** (Chapter 3) works on one row's values and returns one value *per row*; an **aggregate function** works on many rows and returns one value *per group*. Without `GROUP BY`, the whole table (after `WHERE`) is one group, so the query returns exactly one row.

Why it matters

Reports, dashboards, KPIs (key performance indicators — the headline numbers a business tracks), billing, fraud checks and most SQL interview questions are aggregations. Doing the summary inside the database is also far cheaper than shipping every row to your application and adding them up there: the database reads the rows where they live and sends back one small result.

How it works

1. The database builds the input rows (`FROM` and any joins — a **join** combines rows of several tables by matching values, Chapter 5), then removes rows that fail `WHERE`.
2. For each aggregate in the `SELECT` list, it walks the remaining rows, keeping a small running **state** (a counter, a running total, the smallest value so far …).
3. Before feeding a value into the state, it **skips** `NULL`**s** — every aggregate except `COUNT(*)` does this.
4. At the end it turns the state into the result (for `AVG`, total ÷ count).

```
SELECT count(*)              AS employees,
       sum(salary)           AS payroll,
       round(avg(salary), 2) AS avg_salary,
       min(salary)           AS lowest,
       max(salary)           AS highest
FROM employees;
```

Output:

```
 employees |  payroll   | avg_salary |  lowest  |  highest
-----------+------------+------------+----------+-----------
        15 | 1617000.00 |  107800.00 | 30000.00 | 250000.00
(1 row)
```

The three flavours of COUNT

```sql
SELECT count(*)                AS all_rows,
       count(email)            AS with_email,
       count(dept_id)          AS with_dept,
       count(DISTINCT dept_id) AS distinct_depts,
       count(DISTINCT job_title) AS distinct_titles
FROM employees;
```

Output:

```
 all_rows | with_email | with_dept | distinct_depts | distinct_titles
----------+------------+-----------+----------------+-----------------
       15 |         13 |        13 |              5 |              12
(1 row)
```

- `count(*)` = 15: every row.
- `count(email)` = 13: two employees have no email.
- `count(dept_id)` = 13 but `count(DISTINCT dept_id)` = 5: 13 people have a department, and they are spread over 5 different departments. The two `NULL`s are not counted as a sixth "value".

NULLs are skipped — and that changes averages

```sql
SELECT count(commission_pct)            AS non_null,
       avg(commission_pct)              AS avg_ignoring_nulls,
       avg(coalesce(commission_pct, 0)) AS avg_nulls_as_zero
FROM employees;
```

Output:

```
 non_null |   avg_ignoring_nulls   |   avg_nulls_as_zero
----------+------------------------+------------------------
        3 | 0.08333333333333333333 | 0.01666666666666666667
(1 row)
```

`AVG` divided by **3** (the people who have a commission), not by 15. Whether that is right depends on the question. "What is the average commission rate *among people who earn commission*?" → 0.083 is correct. "What is the average commission rate *across all staff*, treating no commission as zero?" → you must say so with `COALESCE(commission_pct, 0)`, giving 0.017. Neither is wrong; the mistake is not knowing which one you asked for.

Empty input

```sql
SELECT count(*) AS n, sum(salary) AS total, avg(salary) AS mean, max(salary) AS top
FROM employees WHERE salary > 1000000;
```

Output:

```
 n | total  |  mean  |  top
---+--------+--------+--------
 0 | (null) | (null) | (null)
(1 row)
```

Nobody earns over a million, so the aggregates see zero rows. `COUNT` answers `0`; `SUM`, `AVG`, `MIN` and `MAX` answer `NULL` ("there is no total of nothing"). You still get **one row** back — an aggregate query

without `GROUP BY` always returns exactly one row. If your application expects `0` for an empty sum, write `COALESCE(SUM(x), 0)`.

Other useful points

`MIN` / `MAX` work on any sortable type, and an aggregate can take an **expression**, not just a column:

```
SELECT min(hire_date) AS first_hire, max(hire_date) AS latest_hire,
       min(last_name) AS first_alpha, max(last_name) AS last_alpha
FROM employees;

SELECT round(sum(quantity * unit_price * (1 - discount)), 2) AS gross_revenue
FROM order_items;
```

Output:

```
 first_hire | latest_hire | first_alpha | last_alpha
------------+-------------+-------------+------------
 2015-01-05 | 2025-06-02  | Brooks      | Wei
(1 row)

 gross_revenue
---------------
      10026.61
(1 row)
```

A `WHERE` clause simply shrinks the set being aggregated:

```
SELECT count(*) AS delivered, min(placed_at) AS first, max(placed_at) AS last
FROM orders WHERE status = 'delivered';
```

Output:

```
 delivered |        first        |        last
-----------+---------------------+---------------------
         7 | 2025-01-03 10:15:00 | 2025-04-09 16:45:00
(1 row)
```

Beyond the big five (PostgreSQL)

PostgreSQL ships many more aggregates. These are the ones you will actually use:

```
SELECT string_agg(first_name, ', ' ORDER BY first_name) AS engineers,
       array_agg(emp_id ORDER BY emp_id)                AS ids,
       bool_and(salary > 90000)                         AS all_above_90k,
       bool_or(email IS NULL)                           AS any_missing_email
FROM employees WHERE dept_id = 1;

SELECT round(stddev_samp(salary), 2) AS stddev,
       percentile_cont(0.5) WITHIN GROUP (ORDER BY salary) AS median,
       mode() WITHIN GROUP (ORDER BY salary)               AS most_common
FROM employees;
```

Output:

```
         engineers          |    ids    | all_above_90k | any_missing_email
----------------------------+-----------+---------------+-------------------
 Meera, Neha, Rahul, Vikram | {2,3,4,5} | t             | t
(1 row)

  stddev  | median | most_common
----------+--------+-------------
 57217.88 |  98000 |    68000.00
(1 row)
```

Aggregate	What it gives	Typical use
`string_agg(x, sep ORDER BY …)`	all values joined into one string	"list the products in each order"
`array_agg(x ORDER BY …)`	all values as an array (an ordered list stored as one value)	feed an app, or `unnest` (expand back into rows) later
`bool_and` / `bool_or`	TRUE if all / any inputs are true	"did every item ship?"
`stddev_samp`, `variance`	spread of the values (standard deviation ≈ how far values typically sit from the average)	analytics
`percentile_cont(f) WITHIN GROUP (ORDER BY x)`	interpolated percentile (0.5 = median)	median salary, p95 latency (the time 95 % of requests finish within)
`percentile_disc(f) …`	percentile that is an actual input value	"the median order", not an average of two
`mode() WITHIN GROUP (ORDER BY x)`	most frequent value	most common salary / status

> **Key term — percentile and median:** The *f*-th percentile is the value below which a fraction *f* of the sorted values fall; the **median** is the 50th percentile, the middle value, which (unlike the average) is not dragged by a few huge values. *Interpolated* means that when the cut falls between two values, the result is blended from both, so it may be a number that never occurs in the data. **Latency** is how long one request takes; "p95 latency" is its 95th percentile — see System Design Fundamentals.

`percentile_cont`, `percentile_disc` and `mode` are **ordered-set aggregates**: the `WITHIN GROUP (ORDER BY …)` tells them which order to read the values in.

Dialect differences

Feature	PostgreSQL	MySQL 8	SQLite	SQL Server
Join values into a string	`string_agg(x, ',')`	`GROUP_CONCAT(x SEPARATOR ',')`	`group_concat(x, ',')`	`STRING_AGG(x, ',')` (2017+)
`AVG` of an integer column	exact `numeric`	decimal	floating point	**integer** (truncated!) — cast first: `AVG(1.0 * x)`
Median	`percentile_cont(0.5) WITHIN GROUP (…)`	no built-in	no built-in	`PERCENTILE_CONT(0.5) WITHIN GROUP (…) OVER ()` (window form, Chapter 8)
Boolean aggregates	`bool_and`, `bool_or`	`MIN` / `MAX` of 0/1	`min` / `max` of 0/1	`MIN` / `MAX` of a `CASE`

Common mistakes

- **`COUNT(column)` when you meant `COUNT(*)`** (or the other way round). They differ exactly by the number of `NULL`s in that column.
- **Averaging with NULLs without thinking** — see the commission example above.
- **`SUM` of an empty set used as a number** in application code — it is `NULL`, not `0`.
- **Integer averages in SQL Server** (`AVG` of `int` returns an `int`).
- **`COUNT(DISTINCT a, b)`** — PostgreSQL does not accept two arguments; count distinct *rows* instead: `COUNT(DISTINCT (a, b))` or a subquery with `SELECT DISTINCT a, b`.

In the interview

> **30-second answer.** "Aggregate functions reduce a set of rows to one value: `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`. All of them ignore `NULL` inputs except `COUNT(*)`, which counts rows. On empty input `COUNT` returns 0 and the others return `NULL`. Without `GROUP BY` the whole filtered table is one group, so you get one row."

- *Q: What is the difference between `COUNT(*)`, `COUNT(1)` and `COUNT(col)`?* — `COUNT(*)` and `COUNT(1)` both count rows (1 is never NULL), and PostgreSQL treats them the same way; `COUNT(col)` counts non-NULL values of `col`.
- *Q: `AVG(x)` vs `SUM(x)/COUNT(*)`?* — They differ when `x` has NULLs: `AVG` divides by `COUNT(x)`. Also `SUM(int)/COUNT(*)` is integer division in PostgreSQL.
- *Q: How do you compute a median in PostgreSQL?* — `percentile_cont(0.5) WITHIN GROUP (ORDER BY x)`.

Self-check

1. A column has values 10, NULL, 20. What do `COUNT(*)`, `COUNT(col)`, `SUM(col)`, `AVG(col)` return? → 3, 2, 30, 15.
2. What does `SELECT MAX(salary) FROM employees WHERE 1 = 0` return — zero rows or one row? → One row containing `NULL`.
3. How would you count employees who have *no* email? → `COUNT(*) - COUNT(email)`, or `COUNT(*) FILTER (WHERE email IS NULL)` (§4.4).
4. Can `MIN` be used on a text column? → Yes; it returns the alphabetically first value (by the column's collation).

TL;DR

Aggregates turn many rows into one value. `COUNT(*)` counts rows; every other aggregate skips `NULL`s. Empty input → `COUNT` = 0, the rest `NULL`. No `GROUP BY` → exactly one result row.

4.2 GROUP BY

What it is

`GROUP BY` splits the rows into **groups** — one group per distinct value (or combination of values) of the listed expressions — and then runs the aggregates **once per group**. The result has **one row per group**.

Analogy: a teacher sorting exam papers into piles by class, then writing the number of papers and the average mark on top of each pile. The piles are the groups; the numbers on top are the aggregates.

How it works

1. Build and filter the rows (`FROM`, `WHERE`).
2. Put rows with the **same values** in all `GROUP BY` expressions into the same group. For grouping, all `NULL`s count as equal, so every row with a `NULL` key lands in **one** `NULL` group.
3. For each group, compute each aggregate over just that group's rows.

4. Output one row per group. The `SELECT` list may contain only the grouping expressions, aggregates, constants, or expressions built from those.

```
SELECT dept_id, count(*) AS headcount, sum(salary) AS payroll
FROM employees
GROUP BY dept_id
ORDER BY dept_id;
```

Output:

```
 dept_id | headcount |  payroll
---------+-----------+-----------
       1 |         4 | 523000.00
       2 |         4 | 358000.00
       3 |         2 | 181000.00
       4 |         2 | 211000.00
       5 |         1 |  64000.00
  (null) |         2 | 280000.00
(6 rows)
```

Six groups: five departments plus a `NULL` **group** holding Asha and Rohan, who have no department. Notice that Research (department 6) does not appear: `GROUP BY` can only make groups from rows that exist in its input. To list departments with zero employees you need an outer join (a join that also keeps rows with no match; Chapter 5).

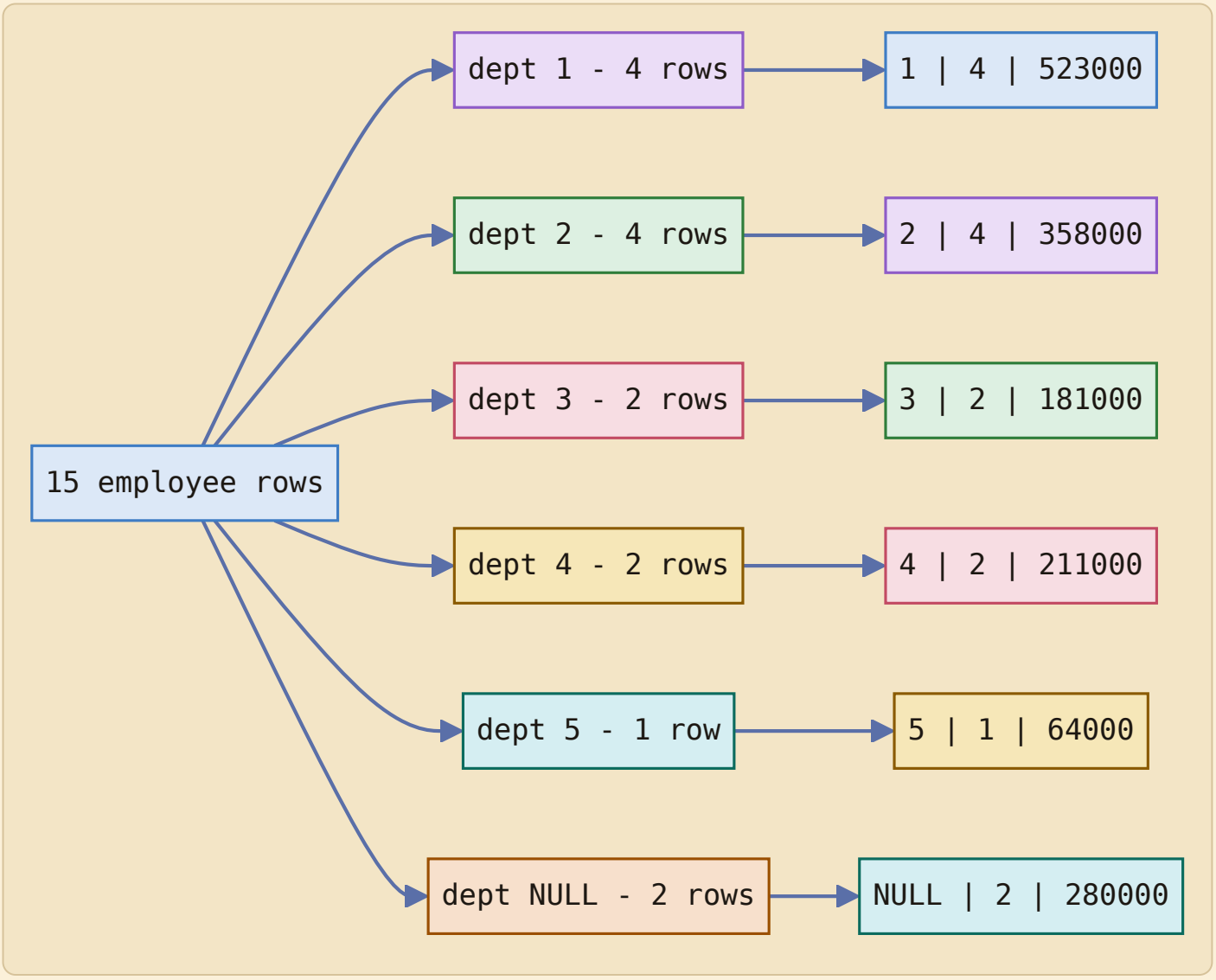


Figure 4.1 — GROUP BY dept_id: split the rows into piles, then summarise each pile into one output row.

Grouping by several columns

With two grouping columns, each **distinct pair** is a group:

```
SELECT dept_id, job_title, count(*) AS n, round(avg(salary)) AS avg_salary
FROM employees
WHERE dept_id IN (1, 2)
GROUP BY dept_id, job_title
ORDER BY dept_id, job_title;
```

Output:

```
 dept_id |      job_title      | n | avg_salary
---------+---------------------+---+------------
       1 | Engineer            | 2 |      98000
       1 | Engineering Manager | 1 |     185000
       1 | Senior Engineer     | 1 |     142000
       2 | Sales Manager       | 1 |     150000
       2 | Sales Rep           | 3 |      69333
(5 rows)
```

The order of columns in `GROUP BY` does not change *which* groups you get — `GROUP BY a, b` and `GROUP BY b, a` produce the same groups. Row order in the output is controlled only by `ORDER BY`.

Grouping by an expression

You can group by a computed value, such as the month of a timestamp:

```
SELECT date_trunc('month', placed_at)::date AS month, count(*) AS orders
FROM orders
GROUP BY date_trunc('month', placed_at)
ORDER BY month;
```

Output:

```
   month    | orders
------------+--------
 2025-01-01 |      3
 2025-02-01 |      2
 2025-03-01 |      2
 2025-04-01 |      3
 2025-05-01 |      2
 2025-06-01 |      2
(6 rows)
```

PostgreSQL also lets you refer to a `SELECT` alias or a column position in `GROUP BY` (a non-standard convenience; if an alias clashes with a real column name, the real column wins):

```
SELECT extract(year FROM hire_date) AS hire_year, count(*) AS hires
FROM employees
GROUP BY hire_year
ORDER BY 1;
```

Output:

```
 hire_year | hires
-----------+-------
      2015 |     1
      2016 |     1
      2017 |     1
      2018 |     1
      2019 |     2
      2020 |     1
      2021 |     2
      2022 |     2
      2023 |     2
      2024 |     1
      2025 |     1
(11 rows)
```

Grouping usually follows a join. Revenue per product category joins `order_items` to `products` first (joins are Chapter 5):

```sql
SELECT p.category, sum(oi.quantity) AS units,
       round(sum(oi.quantity * oi.unit_price * (1 - oi.discount)), 2) AS revenue
FROM order_items oi JOIN products p ON p.product_id = oi.product_id
GROUP BY p.category
ORDER BY revenue DESC;
```

Output:

```
  category   | units | revenue
-------------+-------+---------
 Electronics |    16 | 7314.79
 Furniture   |     5 | 1940.47
 Accessories |    11 |  671.35
 (null)      |     2 |  100.00
(4 rows)
```

A `GROUP BY` with no aggregates returns each distinct group once — the same result as `SELECT DISTINCT status FROM orders`:

```sql
SELECT status FROM orders GROUP BY status ORDER BY status;
```

Output:

```
  status
-----------
 cancelled
 delivered
 pending
 shipped
(4 rows)
```

The single-value rule

After grouping, each output row stands for a whole group. A column that is neither grouped nor aggregated could have **several different values** inside one group — which one should be shown? SQL refuses to guess:

```sql
SELECT dept_id, first_name, count(*) FROM employees GROUP BY dept_id;
```

Output:

```
ERROR:  column "employees.first_name" must appear in the GROUP BY clause or be used in an aggregate
function
LINE 1: SELECT dept_id, first_name, count(*) FROM employees GROUP BY...
                        ^
```

There is one exception. If you group by a table's **primary key** (the column whose value uniquely identifies each row), every other column of that table has exactly one value per group (it is *functionally dependent* on the key — see Chapter 11), so PostgreSQL (9.1+) allows it:

```
SELECT d.dept_id, d.name, count(e.emp_id) AS headcount
FROM departments d JOIN employees e ON e.dept_id = d.dept_id
GROUP BY d.dept_id
ORDER BY d.dept_id;
```

Output:

```
 dept_id |    name     | headcount
---------+-------------+-----------
       1 | Engineering |         4
       2 | Sales       |         4
       3 | Marketing   |         2
       4 | Finance     |         2
       5 | Support     |         1
(5 rows)
```

Database	Non-grouped, non-aggregated column in SELECT
PostgreSQL	Error, unless the column depends on a grouped primary key.
MySQL 8	Error by default (`ONLY_FULL_GROUP_BY` mode, on since 5.7.5); also detects primary-key dependency. With the mode off it returns an arbitrary value.
SQLite	Allowed ("bare column"); the value comes from an arbitrary row of the group (or from the row holding the `min`/`max` if the query has exactly one of those).
SQL Server, Oracle	Always an error.

Trade-offs and performance

The database groups rows in one of two ways (you can see which with `EXPLAIN`, the command that prints how the database plans to run a query, Chapter 12): **hash aggregation** builds an in-memory hash table (a lookup structure that turns each key into a slot number, so finding a key takes about one step) keyed by the group values (fast, no sorting, needs memory for all groups), or **sort + group aggregation** sorts the rows by the grouping key and summarises each run of equal keys (can use an index — a sorted lookup structure kept beside the table — that is already in that order, and streams rows through with little memory). Grouping by many high-cardinality columns (columns with many distinct values) creates many groups and costs more memory.

When not to use it

- If you need each **row** *and* a group total next to it (e.g. each salary and its department's average), `GROUP BY` collapses the rows you want to keep. Use a **window function** (an aggregate-like function that computes over a set of related rows but keeps every row in the output) instead: `avg(salary) OVER (PARTITION BY dept_id)` — Chapter 8.
- If you only want distinct values, `SELECT DISTINCT` states the intent more clearly.

Common mistakes

- Selecting a column that is not grouped (error above) — and "fixing" it by adding the column to `GROUP BY`, which silently creates *smaller* groups than you meant. Ask yourself: "one row per *what*?" and group by exactly that.
- Expecting groups for values that have no rows (Research). Groups come only from existing rows.
- Forgetting that `NULL` keys form their own group — often labelled with `COALESCE(dept_id::text, 'none')` in reports.
- Relying on `GROUP BY` to sort. PostgreSQL's hash aggregation returns groups in no particular order; always add `ORDER BY`.

In the interview

30-second answer. "`GROUP BY` partitions the filtered rows by the grouping columns and computes the aggregates once per partition, returning one row per group. Every selected column must be grouped or aggregated — unless it depends on a grouped primary key. `NULL`s form one group. For per-row output with group totals, use window functions instead."

- *Q: Does `GROUP BY` guarantee sorted output?* — No. Only `ORDER BY` does.
- *Q: `GROUP BY` vs `DISTINCT`?* — Without aggregates they give the same rows; `GROUP BY` is needed when you want to compute something per group.
- *Q: How do you find duplicate emails?* — `SELECT lower(email), count(*) FROM customers GROUP BY lower(email) HAVING count(*) > 1;` (the `HAVING` part is next).

Self-check

1. `GROUP BY dept_id` on our data gives 6 rows but there are 6 departments. Is the Research department one of them? → No; the sixth group is the `NULL` group. Research has no rows to group.
2. Why is `SELECT dept_id, first_name, count(*) … GROUP BY dept_id` an error? → `first_name` has several values per department; SQL will not pick one.
3. Does `GROUP BY a, b` produce more, fewer or the same number of groups as `GROUP BY a`? → The same or more — each `a` group is split by `b`.

TL;DR

`GROUP BY` = split rows into groups by key, aggregate each group, one output row per group. Select only keys and aggregates. `NULL` keys form one group. Add `ORDER BY` for order.

4.3 HAVING

What it is

`HAVING` is a filter on **groups**. It runs **after** grouping and aggregation, so its condition can use aggregates such as `count(*) > 2`. `WHERE` is a filter on **rows** and runs *before* grouping, so it cannot.

	WHERE	HAVING
Filters	individual rows	whole groups
Runs	before `GROUP BY`	after `GROUP BY` and aggregation
Can use aggregates?	**No**	Yes
Can use plain columns?	Yes	Only grouping columns (or inside aggregates)
Effect on work	fewer rows to group → cheaper	groups are already computed

How it works

```
SELECT dept_id, count(*) AS headcount
FROM employees
GROUP BY dept_id
HAVING count(*) > 2
ORDER BY dept_id;
```

Output:

```
 dept_id | headcount
---------+-----------
       1 |         4
       2 |         4
(2 rows)
```

1. Group the 15 employees by `dept_id` (6 groups).
2. Compute `count(*)` for each group.
3. Keep only groups where the count is greater than 2 — Engineering and Sales.

Using `WHERE` and `HAVING` together is common: `WHERE` removes unwanted *rows*, then `HAVING` removes unwanted *groups*. "Among orders that were not cancelled, which customers ordered at least twice?"

```
SELECT customer_id, count(*) AS orders
FROM orders
WHERE status <> 'cancelled'
GROUP BY customer_id
HAVING count(*) >= 2
ORDER BY customer_id;
```

Output:

```
 customer_id | orders
-------------+--------
           1 |      3
           2 |      2
           5 |      2
           6 |      2
(4 rows)
```

Customer 3 (Chen) has two orders in total, but one is cancelled; `WHERE` drops it first, the group then has 1 row, and `HAVING` drops the group.

`HAVING` may use an aggregate that is not in the `SELECT` list, and may combine conditions with `AND` / `OR`:

```
SELECT dept_id
FROM employees
GROUP BY dept_id
HAVING max(salary) > 140000
ORDER BY dept_id;

SELECT dept_id, count(*) AS n, round(avg(salary)) AS avg_salary
FROM employees
WHERE dept_id IS NOT NULL
GROUP BY dept_id
HAVING avg(salary) > 90000 AND count(*) >= 2
ORDER BY avg_salary DESC;
```

Output:

```
 dept_id
---------
       1
       2
  (null)
(3 rows)

 dept_id | n | avg_salary
---------+---+------------
       1 | 4 |     130750
       4 | 2 |     105500
       3 | 2 |      90500
(3 rows)
```

In the first result the `NULL` group qualifies because it contains the CEO's salary.

An aggregate in WHERE is an error

```
SELECT dept_id, count(*) FROM employees WHERE count(*) > 2 GROUP BY dept_id;
```

Output:

```
ERROR:  aggregate functions are not allowed in WHERE
LINE 1: SELECT dept_id, count(*) FROM employees WHERE count(*) > 2 G...
                                                      ^
```

When `WHERE` runs, groups do not exist yet, so there is nothing to count (§4.5 shows the full order).

Row conditions belong in WHERE

A condition on a grouping column works in either place and gives the same answer:

```
SELECT dept_id, count(*) FROM employees GROUP BY dept_id HAVING dept_id = 2;
SELECT dept_id, count(*) FROM employees WHERE dept_id = 2 GROUP BY dept_id;
```

Output:

```
 dept_id | count
---------+-------
       2 |     4
(1 row)

 dept_id | count
---------+-------
       2 |     4
(1 row)
```

Prefer `WHERE`: it removes rows *before* the expensive grouping, can use an index, and tells the reader "this is a row filter". (PostgreSQL's planner — the component that decides how to execute a query — often moves such a `HAVING` condition down into `WHERE` for you, but don't rely on every database doing so.)

HAVING without GROUP BY

If a query has `HAVING` but no `GROUP BY`, the whole input is treated as **one group**. The query returns that one row if the condition is true, and no rows otherwise:

```sql
SELECT count(*) AS products FROM products HAVING count(*) > 5;
SELECT count(*) AS products FROM products HAVING count(*) > 50;
```

Output:

```
 products
----------
        9
(1 row)

 products
----------
(0 rows)
```

Common mistakes

- Putting an aggregate condition in `WHERE` (error) or a row condition in `HAVING` (works but slower and confusing).
- Referring to a `SELECT` alias in `HAVING`: `HAVING headcount > 2` fails in PostgreSQL (`column "headcount" does not exist`), because `HAVING` is evaluated before `SELECT` names its outputs. Repeat the aggregate. (MySQL allows the alias as an extension.)
- Forgetting that `WHERE` changes what `HAVING` sees (the Chen example).

In the interview

30-second answer. "`WHERE` filters rows before grouping and can't use aggregates; `HAVING` filters groups after aggregation and can. Use `WHERE` for anything that doesn't need an aggregate, because it shrinks the input early."

- *Q: Find departments with more than 3 employees.* — `SELECT dept_id FROM employees GROUP BY dept_id HAVING COUNT(*) > 3;`
- *Q: Can you use `HAVING` without `GROUP BY`?* — Yes; the whole table is one group, so you get one row or none.
- *Q: Find duplicate rows.* — `GROUP BY` all the columns that define "duplicate" and `HAVING COUNT(*) > 1`.

Self-check

1. Which runs first, `WHERE` or `HAVING`? → `WHERE` (on rows), then grouping, then `HAVING` (on groups).
2. Rewrite `HAVING dept_id <> 5` in the better place. → `WHERE dept_id <> 5` (before `GROUP BY`) — careful: `WHERE dept_id <> 5` also drops `NULL` rows, just as the `HAVING` version drops the `NULL` group.
3. What does `SELECT count(*) FROM t HAVING count(*) > 0` return on an empty table? → No rows (the count is 0, so the one group is filtered out).

TL;DR

`HAVING` = `WHERE` for groups. Aggregates go in `HAVING`; row filters go in `WHERE`. No `GROUP BY` → one big group.

4.4 Conditional aggregation: FILTER and CASE

What it is

Conditional aggregation means aggregating only the rows that meet a condition, *inside* a single query, so you can compute several differently-filtered numbers side by side. PostgreSQL has a dedicated clause for

it — `aggregate(...) FILTER (WHERE condition)` (SQL standard; PostgreSQL 9.4+, SQLite 3.30+). Every database supports the older trick of putting a `CASE` expression inside the aggregate.

The problem it solves

"For each customer: total orders, delivered orders, cancelled orders, online orders." With plain `WHERE` you would need four queries (or four subqueries — queries nested inside another), each scanning (reading through every row of) `orders` again. Conditional aggregation does it in **one pass** over the table.

How it works

`FILTER (WHERE cond)` feeds only the rows where `cond` is true into that one aggregate. Other aggregates in the same query still see all rows of the group.

```
SELECT customer_id,
       count(*)                                        AS total,
       count(*) FILTER (WHERE status = 'delivered')    AS delivered,
       count(*) FILTER (WHERE status = 'cancelled')    AS cancelled,
       count(*) FILTER (WHERE sales_rep_id IS NULL)    AS online
FROM orders
GROUP BY customer_id
ORDER BY customer_id;
```

Output:

```
 customer_id | total | delivered | cancelled | online
-------------+-------+-----------+-----------+--------
           1 |     3 |         2 |         0 |      1
           2 |     2 |         2 |         0 |      1
           3 |     2 |         1 |         1 |      1
           4 |     1 |         0 |         1 |      0
           5 |     2 |         1 |         0 |      1
           6 |     2 |         0 |         0 |      1
           7 |     1 |         0 |         0 |      0
           8 |     1 |         1 |         0 |      1
(8 rows)
```

The portable version uses `CASE`. Two equivalent styles:

- `SUM(CASE WHEN cond THEN 1 ELSE 0 END)` — adds 1 for each matching row.
- `COUNT(CASE WHEN cond THEN 1 END)` — the `CASE` gives `NULL` for non-matching rows, and `COUNT` skips `NULL`s.

```
SELECT customer_id,
       count(*)                                          AS total,
       sum(CASE WHEN status = 'delivered' THEN 1 ELSE 0 END) AS delivered,
       count(CASE WHEN status = 'cancelled' THEN 1 END)      AS cancelled
FROM orders
GROUP BY customer_id
ORDER BY customer_id;
```

Output:

```
 customer_id | total | delivered | cancelled
-------------+-------+-----------+-----------
           1 |     3 |         2 |         0
           2 |     2 |         2 |         0
           3 |     2 |         1 |         1
           4 |     1 |         0 |         1
           5 |     2 |         1 |         0
           6 |     2 |         0 |         0
           7 |     1 |         0 |         0
           8 |     1 |         1 |         0
(8 rows)
```

Ratios and percentages

Conditional aggregates make "what share of X is Y" questions easy — divide a filtered aggregate by the unfiltered one. Multiply by `100.0` (not `100`) so integer inputs don't cause integer division.

```sql
SELECT dept_id,
       sum(salary) AS payroll,
       round(100.0 * sum(salary) FILTER (WHERE hire_date < '2020-01-01') / sum(salary), 1) AS
pct_veterans
FROM employees
WHERE dept_id IS NOT NULL
GROUP BY dept_id
ORDER BY dept_id;
```

Output:

```
 dept_id |  payroll  | pct_veterans
---------+-----------+--------------
       1 | 523000.00 |         62.5
       2 | 358000.00 |         41.9
       3 | 181000.00 |         66.3
       4 | 211000.00 |         64.0
       5 |  64000.00 |       (null)
(5 rows)
```

Support shows `NULL`, not 0: its only employee was hired in 2023, so the filtered `SUM` saw no rows and returned `NULL`. Wrap it in `COALESCE(…, 0)` if the report should show 0.

Pivot-style reports

One column per category of something is called a **pivot** (rows turned into columns). Conditional aggregation is the standard way to pivot in SQL when the column list is known in advance:

```sql
SELECT p.category,
       sum(oi.quantity) FILTER (WHERE o.status = 'delivered') AS delivered_units,
       sum(oi.quantity) FILTER (WHERE o.status = 'shipped')   AS shipped_units,
       sum(oi.quantity) FILTER (WHERE o.status = 'pending')   AS pending_units
FROM order_items oi
JOIN orders o   ON o.order_id = oi.order_id
JOIN products p ON p.product_id = oi.product_id
GROUP BY p.category
ORDER BY p.category NULLS LAST;
```

Output:

```
  category   | delivered_units | shipped_units | pending_units
-------------+-----------------+---------------+--------------
 Accessories |               6 |        (null) |             5
 Electronics |               9 |             3 |             1
 Furniture   |               2 |             3 |        (null)
 (null)      |               2 |        (null) |        (null)
(4 rows)
```

Dynamic pivots (where the columns are not known until you look at the data) and `crosstab` (a PostgreSQL extension function that builds pivot tables) are covered in Chapter 14.

Trade-offs

Approach	Portable?	Readability	Notes
`agg(x) FILTER (WHERE c)`	PostgreSQL, SQLite 3.30+ (not MySQL, SQL Server)	Best — the condition is separate from the value	Works with every aggregate, including `string_agg`, `array_agg`.
`SUM(CASE WHEN c THEN 1 ELSE 0 END)`	Everywhere	Good	Returns 0, not `NULL`, when nothing matches.
`COUNT(CASE WHEN c THEN 1 END)`	Everywhere	Good	Relies on `COUNT` skipping `NULL`.
Separate queries / subqueries	Everywhere	Verbose	Several scans of the table.

In MySQL a boolean expression is 0 or 1, so `SUM(status = 'delivered')` is a common MySQL shortcut (not valid in PostgreSQL, where `boolean` can't be summed; you would write `SUM((status = 'delivered')::int)`).

Common mistakes

- `COUNT(CASE WHEN c THEN 1 ELSE 0 END)` — counts *every* row, because 0 is not `NULL`. Use `SUM` with `ELSE 0`, or `COUNT` with no `ELSE`.
- Integer division in ratios (`100 * a / b` with integer `a`, `b`).
- Putting the condition in `WHERE` when other columns need the unfiltered rows — that filters the rows for *all* aggregates.

In the interview

30-second answer. "Conditional aggregation computes several filtered aggregates in one pass: `COUNT(*) FILTER (WHERE status='x')` in PostgreSQL, or `SUM(CASE WHEN status='x' THEN 1 ELSE 0 END)` anywhere. It's how you do per-group breakdowns, ratios and fixed pivots."

- *Q: Share of cancelled orders per month?* — `GROUP BY` month, `ROUND(100.0 * COUNT(*) FILTER (WHERE status='cancelled') / COUNT(*), 1)`.
- *Q: Why does `COUNT(CASE … ELSE 0 END)` give the wrong number?* — Because `COUNT` counts non-NULL values and 0 is non-NULL.

Self-check

1. Write "number of employees with no email per department" two ways. → `COUNT(*) FILTER (WHERE email IS NULL)` and `SUM(CASE WHEN email IS NULL THEN 1 ELSE 0 END)`.
2. What does a `FILTER`ed `SUM` return for a group where no row matches? → `NULL`.
3. Why is `100.0` used instead of `100` in percentages? → To force numeric (decimal) arithmetic instead of integer division.

TL;DR

Put the condition *inside* the aggregate (`FILTER (WHERE …)` or `CASE`) to compute many filtered numbers in one scan. It powers breakdowns, ratios and fixed pivots.

4.5 The logical order of query processing

What it is

You **write** a `SELECT` in this order: `SELECT … FROM … WHERE … GROUP BY … HAVING … ORDER BY … LIMIT`. The database **conceptually evaluates** it in a different order, starting with `FROM`. This *logical query processing order* is a mental model defined by the SQL standard's semantics: each step takes the table produced by the previous step and hands a new table to the next one.

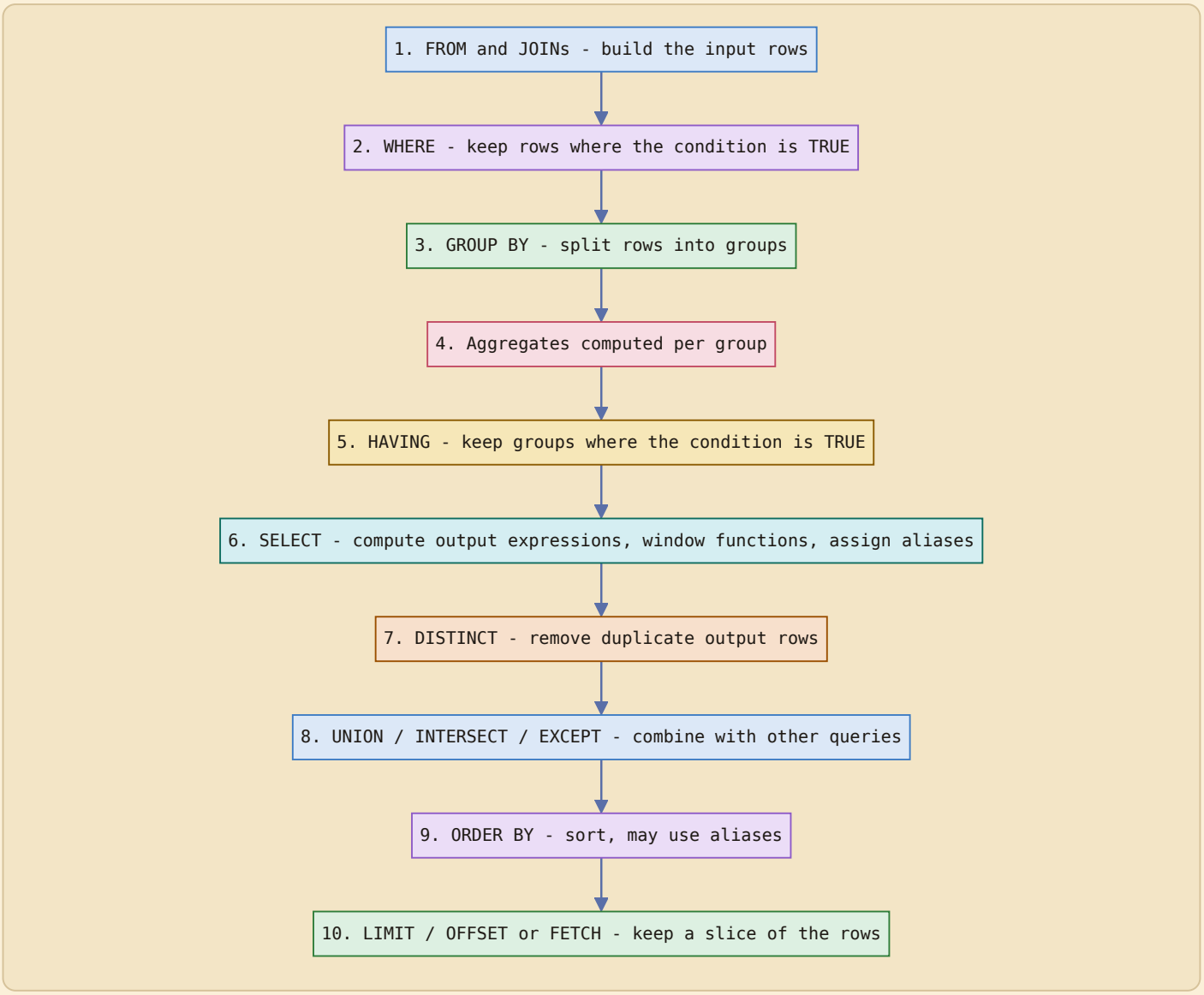


Figure 4.2 — Logical query processing order. Each step only sees what earlier steps produced.

Step	Clause	Input → output	Can refer to
1	`FROM`, `JOIN … ON`	tables → one combined row set	table columns
2	`WHERE`	rows → fewer rows	table columns (no aggregates, no `SELECT` aliases)
3–4	`GROUP BY` + aggregates	rows → one row per group	table columns / expressions
5	`HAVING`	groups → fewer groups	grouping columns, aggregates
6	`SELECT`	groups/rows → output columns	grouping columns, aggregates; **window functions run here**
7	`DISTINCT`	output rows → unique output rows	output columns
8	set operators (`UNION`, `INTERSECT`, `EXCEPT`; Chapter 6)	two results → one	output columns
9	`ORDER BY`	rows → sorted rows	output aliases, output positions, and (without `DISTINCT`/set ops) input columns
10	`LIMIT`/`OFFSET`	sorted rows → a slice	—

Why it matters

Almost every "column does not exist" or "aggregate not allowed" error is explained by this order:

- **`WHERE` can't use a `SELECT` alias** — the alias is created in step 6, `WHERE` is step 2.
- **`WHERE` can't use aggregates** — groups don't exist until step 3.
- **`ORDER BY` *can* use aliases** — it runs after `SELECT`.
- **`LIMIT` applies to the sorted, final rows** — so `ORDER BY … LIMIT 3` really means "top 3".
- **Window functions can't be used in `WHERE`** — they are computed in step 6 (Chapter 8).

It also guides efficient writing and debugging. Filtering rows as early as possible (in `WHERE`, step 2) means fewer rows to group, aggregate and sort later; and when a query returns surprising results, walking through the steps in this order — what rows does `FROM` produce, which survive `WHERE`, what groups form — usually shows where the logic went wrong.

```
SELECT emp_id, salary * 12 AS yearly FROM employees WHERE yearly > 2000000;
```

Output:

```
ERROR:  column "yearly" does not exist
LINE 1: ...mp_id, salary * 12 AS yearly FROM employees WHERE yearly > 2...
                                                             ^
```

```
SELECT emp_id, salary * 12 AS yearly FROM employees ORDER BY yearly DESC LIMIT 3;
```

Output:

```
 emp_id |   yearly
--------+------------
      1 | 3000000.00
      2 | 2220000.00
      6 | 1800000.00
(3 rows)
```

Two fixes for the `WHERE` case: repeat the expression (`WHERE salary * 12 > 2000000`), or compute it in a **derived table** (a subquery in `FROM`, Chapter 6) so that it becomes an input column of the outer query:

```sql
SELECT * FROM (SELECT emp_id, salary * 12 AS yearly FROM employees) t
WHERE yearly > 2000000;
```

Output:

```
 emp_id |   yearly
--------+------------
      1 | 3000000.00
      2 | 2220000.00
(2 rows)
```

A query that uses every clause

```sql
SELECT   dept_id, count(*) AS headcount, round(avg(salary)) AS avg_salary
FROM     employees
WHERE    hire_date >= '2018-01-01'
GROUP BY dept_id
HAVING   count(*) >= 2
ORDER BY avg_salary DESC
LIMIT    3;
```

Output:

```
 dept_id | headcount | avg_salary
---------+-----------+------------
       1 |         4 |     130750
       3 |         2 |      90500
       2 |         3 |      69333
(3 rows)
```

Tracing it step by step:

1. **FROM** `employees` → 15 rows.
2. **WHERE** hired in 2018 or later → drops Asha (2015), Kiran (2016) and Daniel (2017) → 12 rows.
3. **GROUP BY** `dept_id` → groups 1 (4 rows), 2 (3 rows), 3 (2), 4 (1: Omar), 5 (1: Li), `NULL` (1: Rohan).
4. **Aggregates** → count and average per group.
5. **HAVING** `count(*) >= 2` → keeps groups 1, 2, 3.
6. **SELECT** → computes `headcount`, `avg_salary` and names them.
7. **ORDER BY** `avg_salary DESC` (an alias — allowed here).
8. **LIMIT 3** → all three survive.

`DISTINCT` happens after `SELECT` and before `ORDER BY` and `LIMIT`, and `ORDER BY` may even sort by an aggregate that is not selected:

```sql
SELECT DISTINCT job_title FROM employees ORDER BY job_title LIMIT 4;

SELECT dept_id FROM employees WHERE dept_id IS NOT NULL
GROUP BY dept_id ORDER BY sum(salary) DESC;
```

Output:

```
   job_title
---------------
 Accountant
 CEO
 Content Writer
 Engineer
(4 rows)

 dept_id
---------
       1
       2
       4
       3
       5
(5 rows)
```

Logical order is not physical order

The logical order defines **what the answer is**, not **how the database computes it**. The query planner is free to do things in any order that gives the same answer: it may push a `WHERE` condition into a join, use an index so rows arrive already sorted and skip the sort step, stop reading once `LIMIT` rows are found, or move a `HAVING` condition on a grouping column down into the scan. `EXPLAIN` shows the physical plan (Chapter 12).

Dialect notes

Behaviour	PostgreSQL	MySQL	SQLite	SQL Server
`SELECT` alias in `GROUP BY`	allowed (extension)	allowed	allowed	**not** allowed
`SELECT` alias in `HAVING`	not allowed	allowed	allowed	not allowed
`SELECT` alias in `WHERE`	not allowed	not allowed	allowed (quirk)	not allowed
Row limit	`LIMIT n OFFSET m` / `FETCH FIRST n ROWS ONLY`	`LIMIT`	`LIMIT`	`TOP n` / `OFFSET … FETCH`

The extensions work because those databases look the name up among the output columns; they do not change the logical order.

In the interview

30-second answer. "Logically, SQL runs `FROM`/joins, then `WHERE`, `GROUP BY`, aggregates, `HAVING`, `SELECT` (including window functions), `DISTINCT`, set operators, `ORDER BY`, and finally `LIMIT`/`OFFSET`. That's why `WHERE` can't see aliases or aggregates but `ORDER BY` can. The optimizer may physically reorder work as long as the result is the same."

- *Q: Why can't I filter on `ROW_NUMBER()` in `WHERE`?* — Window functions are computed in the `SELECT` step, after `WHERE`. Wrap the query in a subquery or CTE (Common Table Expression — a named subquery

written with `WITH`, Chapter 7) and filter outside.

- *Q: Is `LIMIT` applied before or after `ORDER BY`?* — After. Without `ORDER BY`, which rows you get is undefined.
- *Q: Does `WHERE` run before the join?* — Logically after `FROM` builds the joined rows; physically the planner often filters each table before joining.

Self-check

1. Put in logical order: `HAVING`, `FROM`, `ORDER BY`, `SELECT`, `WHERE`, `GROUP BY`, `LIMIT`. → `FROM`, `WHERE`, `GROUP BY`, `HAVING`, `SELECT`, `ORDER BY`, `LIMIT`.
2. Why does `ORDER BY yearly` work but `WHERE yearly > …` fail? → `ORDER BY` runs after `SELECT` creates the alias; `WHERE` runs before.
3. Where do window functions run? → In the `SELECT` step — after `HAVING`, before `DISTINCT`/`ORDER BY`.

TL;DR

From → **W**here → **G**roup by → **H**aving → **S**elect → **D**istinct → **O**rder by → **L**imit. A clause can only use what earlier steps produced. Logical order ≠ execution plan.

4.6 Subtotals: GROUPING SETS, ROLLUP and CUBE

What it is

A normal `GROUP BY` produces one level of detail. Reports often need **several levels at once**: revenue per country *and* category, a subtotal per country, and a grand total. `GROUPING SETS`, `ROLLUP` and `CUBE` (SQL:1999; PostgreSQL 9.5+) produce all of those levels in one query, as if several `GROUP BY` queries had been glued together with `UNION ALL` (which stacks the rows of several queries into one result).

Syntax	Groupings produced	Example with `(country, category)`
`GROUPING SETS ((a, b), (a), ())`	exactly the ones you list	whatever you list
`ROLLUP (a, b)`	`(a, b)`, `(a)`, `()` — a hierarchy, dropping columns from the right	detail, per-country subtotal, grand total
`CUBE (a, b)`	every subset: `(a, b)`, `(a)`, `(b)`, `()` — 2^n groupings	detail, per-country, per-category, grand total

In a subtotal row, a column that was "rolled up" is shown as `NULL`. The function `GROUPING(col)` returns 1 when `col` is `NULL` *because it was rolled up*, and 0 when it is a real value — which is how you tell a subtotal `NULL` from a genuine `NULL` in the data.

How it works

We first define a small view (a saved query, Chapter 7) with one row per non-cancelled order line:

```sql
CREATE VIEW sales AS
SELECT c.country, coalesce(p.category, 'Other') AS category,
       oi.quantity * oi.unit_price * (1 - oi.discount) AS revenue
FROM order_items oi
JOIN orders o    ON o.order_id = oi.order_id
JOIN customers c ON c.customer_id = o.customer_id
JOIN products p  ON p.product_id = oi.product_id
WHERE o.status <> 'cancelled';
```

ROLLUP — detail, then a subtotal per country, then the grand total:

```sql
SELECT country, category, round(sum(revenue), 2) AS revenue
FROM sales
GROUP BY ROLLUP (country, category)
ORDER BY country NULLS LAST, category NULLS LAST;
```

Output:

```
 country |  category   | revenue
---------+-------------+---------
 India   | Accessories |  621.85
 India   | Electronics | 2558.04
 India   | Furniture   | 1441.47
 India   | (null)      | 4621.36
 UK      | Accessories |   49.50
 UK      | Electronics | 1808.86
 UK      | Furniture   |  499.00
 UK      | (null)      | 2357.36
 USA     | Electronics |  324.90
 USA     | Other       |  100.00
 USA     | (null)      |  424.90
 (null)  | (null)      | 7403.62
(12 rows)
```

The rows with `category = (null)` are the per-country subtotals; the last row (both `NULL`) is the grand total. Replacing those `NULL`s with labels using `GROUPING()`:

```sql
SELECT CASE WHEN grouping(country) = 1 THEN 'ALL countries' ELSE country END AS country,
       CASE WHEN grouping(category) = 1 THEN 'ALL categories' ELSE category END AS category,
       round(sum(revenue), 2) AS revenue
FROM sales
GROUP BY ROLLUP (country, category)
ORDER BY grouping(country), country, grouping(category), category;
```

Output:

```
    country    |    category    | revenue
---------------+----------------+---------
 India         | Accessories    |  621.85
 India         | Electronics    | 2558.04
 India         | Furniture      | 1441.47
 India         | ALL categories | 4621.36
 UK            | Accessories    |   49.50
 UK            | Electronics    | 1808.86
 UK            | Furniture      |  499.00
 UK            | ALL categories | 2357.36
 USA           | Electronics    |  324.90
 USA           | Other          |  100.00
 USA           | ALL categories |  424.90
 ALL countries | ALL categories | 7403.62
(12 rows)
```

CUBE adds the per-category totals across all countries. `grouping(country, category)` returns a bit mask (a number whose binary digits each flag one column: here the left bit is `country`, the right bit `category`) — 0 for detail rows, 1 when `category` was rolled up, 2 when `country` was, 3 for the grand total:

```sql
SELECT country, category, round(sum(revenue), 2) AS revenue,
       grouping(country, category) AS level_bits
FROM sales
GROUP BY CUBE (country, category)
ORDER BY level_bits, country, category;
```

Output:

```
 country |  category   | revenue | level_bits
---------+-------------+---------+------------
 India   | Accessories |  621.85 |          0
 India   | Electronics | 2558.04 |          0
 India   | Furniture   | 1441.47 |          0
 UK      | Accessories |   49.50 |          0
 UK      | Electronics | 1808.86 |          0
 UK      | Furniture   |  499.00 |          0
 USA     | Electronics |  324.90 |          0
 USA     | Other       |  100.00 |          0
 India   | (null)      | 4621.36 |          1
 UK      | (null)      | 2357.36 |          1
 USA     | (null)      |  424.90 |          1
 (null)  | Accessories |  671.35 |          2
 (null)  | Electronics | 4691.80 |          2
 (null)  | Furniture   | 1940.47 |          2
 (null)  | Other       |  100.00 |          2
 (null)  | (null)      | 7403.62 |          3
(16 rows)
```

GROUPING SETS lets you pick exactly the levels you want — here one total per country, one per category, and the grand total, but no country × category detail:

```sql
SELECT country, category, count(*) AS lines
FROM sales
GROUP BY GROUPING SETS ((country), (category), ())
ORDER BY country NULLS LAST, category NULLS LAST;
```

Output:

```
 country |  category   | lines
---------+-------------+-------
 India   | (null)      |    11
 UK      | (null)      |     5
 USA     | (null)      |     4
 (null)  | Accessories |     6
 (null)  | Electronics |     9
 (null)  | Furniture   |     4
 (null)  | Other       |     1
 (null)  | (null)      |    20
(8 rows)
```

Trade-offs

Option	Pros	Cons
`ROLLUP` / `CUBE` / `GROUPING SETS`	One statement, one scan of the input, totals always consistent	Output mixes levels — readers must understand the `NULL`s; `CUBE` grows as 2^n
Several `GROUP BY` queries + `UNION ALL`	Works in every database	Repeats the query; scans the input once per level
Totals in the application / BI tool (business-intelligence reporting software)	Flexible formatting	Moves more data out of the database

Database	Support
PostgreSQL 9.5+	`GROUPING SETS`, `ROLLUP`, `CUBE`, `GROUPING()`
SQL Server, Oracle	all of the above
MySQL 8	`GROUP BY a, b WITH ROLLUP` and `GROUPING()`; no `CUBE` or `GROUPING SETS`
SQLite	none — use `UNION ALL`

Common mistakes

- Confusing a subtotal `NULL` with a real `NULL` in the data (use `GROUPING()`; or, as the view does, `COALESCE` real `NULL`s to a label first).
- Expecting subtotals in a particular order — add an `ORDER BY` (often on `GROUPING(...)`).
- Using `CUBE` on many columns: 5 columns = 32 groupings.

In the interview

> **30-second answer.** "`ROLLUP(a, b)` gives `(a, b)`, `(a)` and the grand total — hierarchical subtotals. `CUBE(a, b)` gives every combination including `(b)` alone. `GROUPING SETS` lets you list the exact groupings. Rolled-up columns show as `NULL`, and `GROUPING(col)` tells those apart from real `NULL`s."

- *Q: How many groupings does `CUBE(a, b, c)` produce?* — $2^3 = 8$.
- *Q: How would you add a grand-total row without `ROLLUP`?* — `UNION ALL` a second query with no `GROUP BY`.

Self-check

1. Which groupings does `ROLLUP (year, month, day)` produce? → `(year, month, day)`, `(year, month)`, `(year)`, `()`.
2. What does `GROUPING(country)` return on the grand-total row? → 1.
3. Rewrite `GROUP BY ROLLUP (a)` with `GROUPING SETS`. → `GROUP BY GROUPING SETS ((a), ())`.

TL;DR

`ROLLUP` = hierarchy of subtotals + grand total; `CUBE` = all combinations; `GROUPING SETS` = exactly the ones you list. Use `GROUPING()` to label subtotal rows.

Chapter 4 TL;DR

Idea	Remember
Aggregates	`COUNT(*)` counts rows; every other aggregate (and `COUNT(col)`) skips `NULL`s. Empty input → 0 for `COUNT`, `NULL` for the rest.
`GROUP BY`	One output row per distinct key; select only keys and aggregates (or columns of a grouped primary key). `NULL`s form one group. No automatic ordering.
`HAVING`	Filters groups after aggregation; `WHERE` filters rows before. Aggregates only in `HAVING`.
Conditional aggregation	`agg(...) FILTER (WHERE …)` or `SUM(CASE …)` — many filtered numbers in one scan; ratios with `100.0 *`.
Logical order	FROM → WHERE → GROUP BY → aggregates → HAVING → SELECT (windows) → DISTINCT → set ops → ORDER BY → LIMIT.
Subtotals	`ROLLUP`, `CUBE`, `GROUPING SETS`; label rolled-up rows with `GROUPING()`.

Next: Chapter 5 — Joins combines rows from several tables, which is what most real aggregations are built on.

Chapter 5 — Joins: Combining Tables

A well-designed relational database splits information into many narrow tables: employees in one, departments in another, orders in a third. That design avoids duplication (Chapter 11 explains why), but it means the answer to almost any real question — "which department does Neha work in?", "what did Anita buy?" — is spread across tables. A **join** puts the pieces back together at query time. Joins are the single most important skill in SQL and the most common topic in SQL interviews.

What you'll learn

- What a join really is: a **Cartesian product** (every row paired with every row) filtered by a **join condition**, and `CROSS JOIN`.
- `INNER JOIN` and the three **outer joins** — `LEFT`, `RIGHT` and `FULL` — with Venn-style pictures (overlapping-circle diagrams) *and* row-by-row result tables that show exactly which rows come back and where `NULL` padding appears.
- **Self joins** (a table joined to itself: managers, pairs, sequences).
- **Semi-joins** and **anti-joins** ("customers who ordered" / "customers who never ordered") and the three ways to write them.
- Join pitfalls: **fan-out** (rows multiplying), `ON` vs `WHERE` in outer joins, `NULL` keys, `COUNT(*)` after an outer join.
- `USING`, `NATURAL JOIN`, multi-table joins, and how databases physically execute joins.

How to run the examples. All queries are in `code/` and ran on the book's private PostgreSQL 14; outputs are real `psql` output, with `NULL` displayed as `(null)`.

```
bash tools_run_sql.sh sql '05-*'
bash tools_pg.sh psql -d <db> -f sql/chapters/05-joins/code/04_outer_joins.sql
```

We use the book's `company` database (`code/00_schema.sql`, same as Chapter 1). The relationships that the joins in this chapter follow:

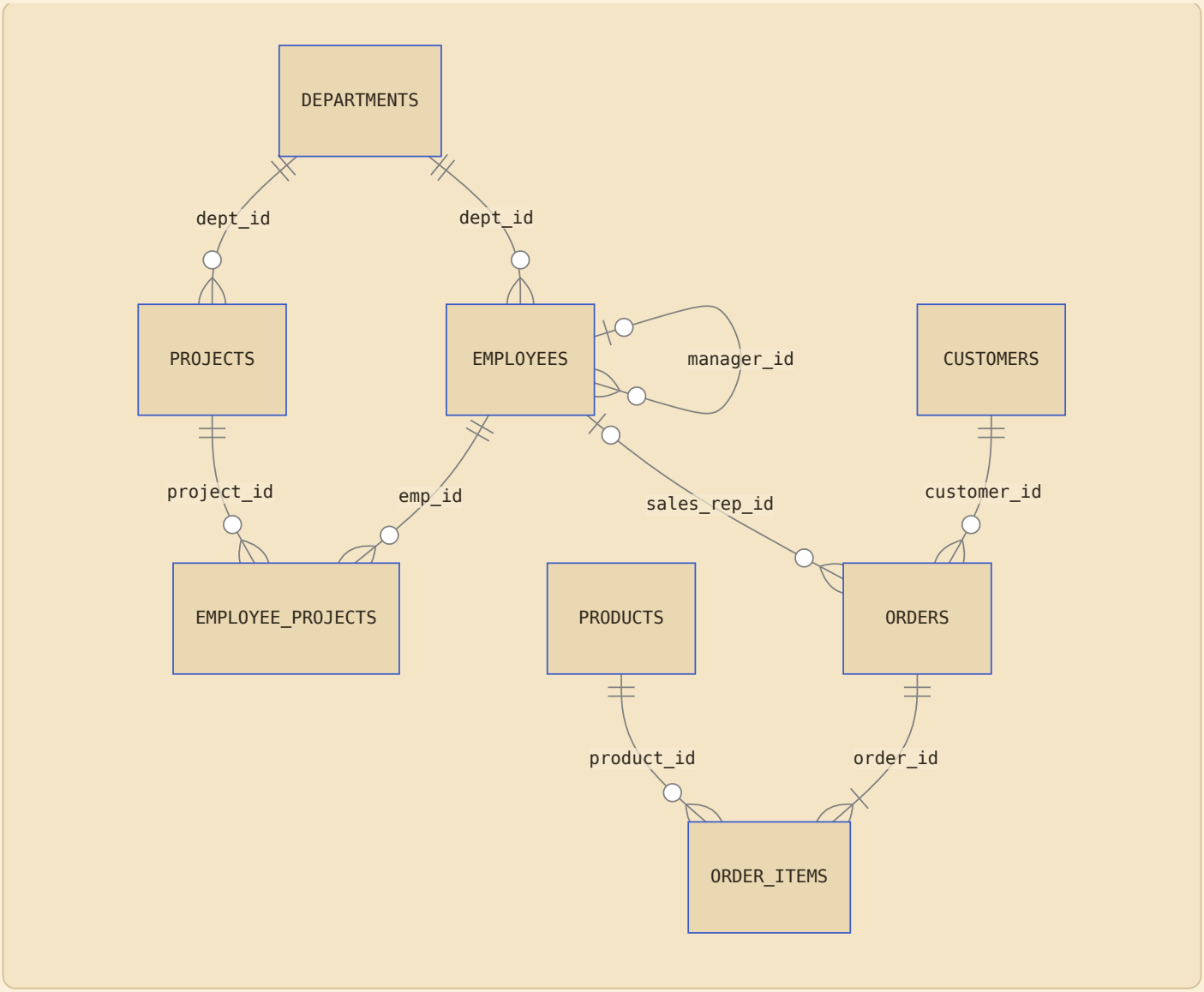


Figure 5.1 — The sample schema's foreign keys: every line is a join path.

A **foreign key** is a column whose values must match the **primary key** (the column that uniquely identifies each row, never `NULL`) of another table — `employees.dept_id` must be an existing `departments.dept_id`. Foreign keys are the natural paths along which tables are joined.

The quirks that make joins interesting: **Asha (CEO) and Rohan (intern) have no department**, the **Research** department has **no employees**, two customers (Hana, Ivan) **never ordered**, one product (Desk Lamp) was **never sold**, and online orders have **no sales rep**.

5.1 Join theory: the Cartesian product and CROSS JOIN

What it is

The **Cartesian product** of two tables (named after René Descartes' coordinate pairs) is the set of **every row of the first table paired with every row of the second**. If `A` has *m* rows and `B` has *n* rows, the product has *m* × *n* rows. SQL writes it as `CROSS JOIN` (or, in the old style, a comma between tables).

Analogy: a clothes shop with 3 sizes and 2 colours can print 3 × 2 = 6 different labels — every size with every colour.

```
CREATE TABLE sizes  (size text);
CREATE TABLE colors (color text);
INSERT INTO sizes  VALUES ('S'), ('M'), ('L');
INSERT INTO colors VALUES ('red'), ('blue');

SELECT size, color FROM sizes CROSS JOIN colors ORDER BY size, color;
SELECT count(*) AS pairs FROM sizes, colors;
```

Output:

```
 size | color
------+-------
 L    | blue
 L    | red
 M    | blue
 M    | red
 S    | blue
 S    | red
(6 rows)

 pairs
-------
     6
(1 row)
```

Every join is a filtered Cartesian product

This is the key idea of the whole chapter. Conceptually, `A JOIN B ON condition`:

1. Forms every pair (a, b) — the Cartesian product.
2. Keeps only the pairs for which the **join condition** (the `ON` clause) is `TRUE`.
3. For outer joins only: adds back the rows that found **no partner**, padding the missing side with `NULL`s.

(Real databases never build the full product when they can avoid it — §5.10 — but the *result* is defined this way.)

```
SELECT count(*) AS all_pairs FROM employees, departments;
SELECT count(*) AS matching_pairs FROM employees e, departments d WHERE e.dept_id = d.dept_id;
```

Output:

```
 all_pairs
-----------
        90
(1 row)

 matching_pairs
----------------
             13
(1 row)
```

15 employees × 6 departments = 90 pairs, of which only 13 make sense (the employee really belongs to that department). The 77 others are noise. Forgetting the join condition — an **accidental Cartesian product** — is a classic bug: the query "works" but returns far too many rows, and aggregates over it are wildly wrong.

When CROSS JOIN is what you want

A deliberate cross join is useful for building **every combination**: a report grid (every department × every month, so that months with no data still appear), test data, or a calendar.

```
SELECT d.name, m.month
FROM departments d
CROSS JOIN (VALUES ('2025-01'), ('2025-02')) AS m(month)
WHERE d.dept_id <= 2
ORDER BY d.name, m.month;
```

Output:

```
    name     |  month
-------------+---------
 Engineering | 2025-01
 Engineering | 2025-02
 Sales       | 2025-01
 Sales       | 2025-02
(4 rows)
```

Join syntax: explicit vs implicit

Style	Example	Notes
Explicit (ANSI SQL-92, the 1992 SQL standard)	`FROM a JOIN b ON a.k = b.k`	Join condition sits next to the join; required for outer joins. **Use this.**
Implicit (comma)	`FROM a, b WHERE a.k = b.k`	Older style; easy to forget a condition in a long `WHERE`; can't express outer joins in standard SQL.

A few vocabulary words used below: an **equi-join** uses only `=` in its condition; a **non-equi join** uses other operators (`<`, `BETWEEN` …); the **join key** is the column(s) compared; **left** and **right** table simply mean "written before" and "written after" the `JOIN` keyword.

5.2 The four main join types at a glance

Before looking at each join in detail, here they are side by side on two tiny tables, so you can see **every row** each one returns. There is no foreign key between them on purpose, so we can have a student in a club that doesn't exist.

```
CREATE TABLE clubs    (club_id int PRIMARY KEY, club text);
CREATE TABLE students (id int PRIMARY KEY, name text, club_id int);   -- no FK on purpose
INSERT INTO clubs    VALUES (10, 'Chess'), (20, 'Drama'), (30, 'Robotics');
INSERT INTO students VALUES (1, 'Ana', 10), (2, 'Ben', 20), (3, 'Cai', 20),
                            (4, 'Dev', NULL), (5, 'Eli', 40);
```

Output (the two inputs):

```
 id | name | club_id
----+------+---------
  1 | Ana  |      10
  2 | Ben  |      20
  3 | Cai  |      20
  4 | Dev  |  (null)
  5 | Eli  |      40
(5 rows)

 club_id |   club
---------+----------
      10 | Chess
      20 | Drama
      30 | Robotics
(3 rows)
```

Row by row, which pairs match on `c.club_id = s.club_id`?

Student	s.club_id	Matching club	Why
Ana	10	Chess	10 = 10
Ben	20	Drama	20 = 20
Cai	20	Drama	20 = 20 (Drama matches **two** students)
Dev	NULL	none	`NULL = anything` is UNKNOWN, never TRUE
Eli	40	none	no club 40 (an "orphan" row)
—	—	Robotics (30)	no student has 30

The Venn-style picture

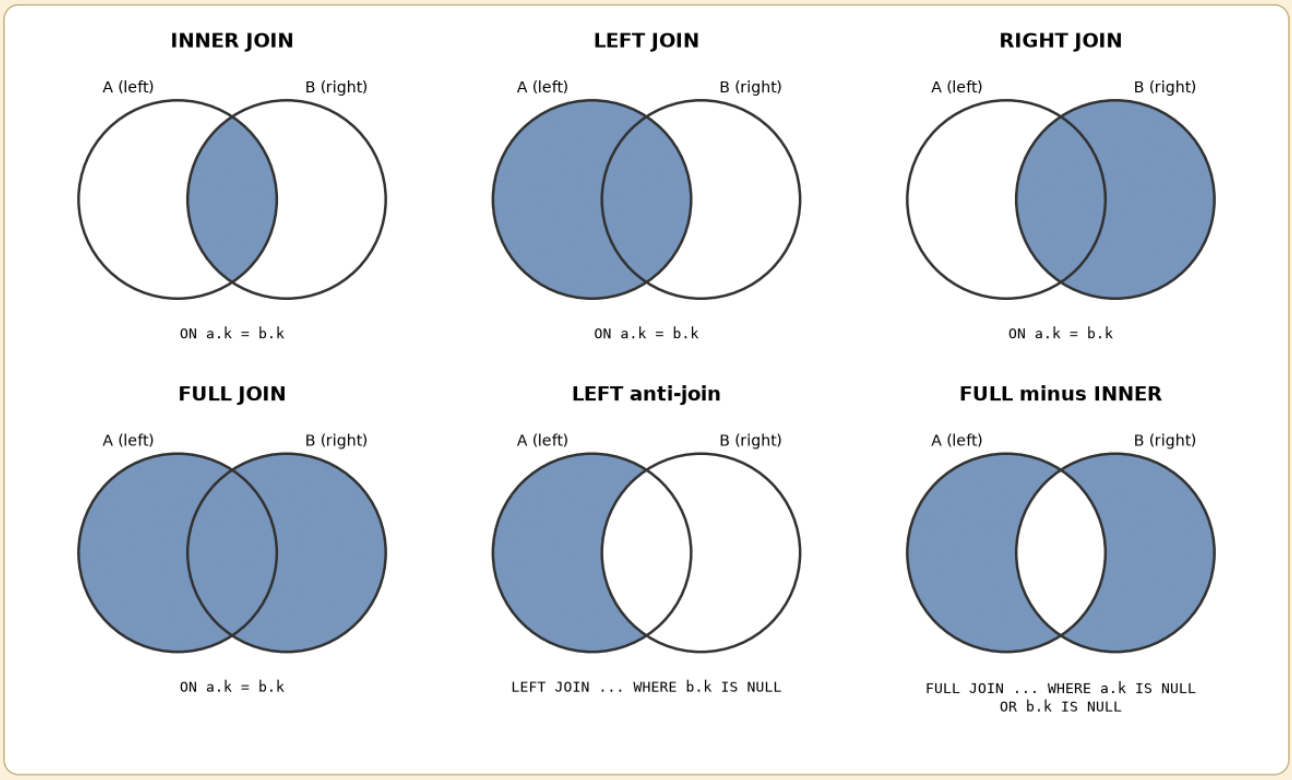


Figure 5.2 — Join "Venn" cheat sheet (drawn by `code/make_join_venn.py`). Left circle = rows of the left table, right circle = rows of the right table, overlap = pairs that satisfy the `ON` condition. Shaded = what the query returns.

A caution about this picture: a Venn diagram describes **sets of rows**, but a join produces **pairs**. One left row can pair with several right rows (Drama pairs with Ben *and* Cai), so the "overlap" can contain more rows than either table has. Use the diagram to remember *which unmatched rows are kept*, and the result tables below to see *how many rows* you get.

The result of each join, row by row

`INNER JOIN` — only the matched pairs (3 rows):

```
SELECT s.id, s.name, s.club_id AS s_club, c.club_id AS c_club, c.club
FROM students s INNER JOIN clubs c ON c.club_id = s.club_id
ORDER BY s.id;
```

```
 id | name | s_club | c_club | club
----+------+--------+--------+-------
  1 | Ana  |     10 |     10 | Chess
  2 | Ben  |     20 |     20 | Drama
  3 | Cai  |     20 |     20 | Drama
(3 rows)
```

`LEFT JOIN` — the matched pairs **plus** every unmatched *left* row, padded with `NULL`s on the right (5 rows):

```
SELECT s.id, s.name, s.club_id AS s_club, c.club_id AS c_club, c.club
FROM students s LEFT JOIN clubs c ON c.club_id = s.club_id
ORDER BY s.id;
```

```
 id | name | s_club | c_club |  club
----+------+--------+--------+--------
  1 | Ana  |     10 |     10 | Chess
  2 | Ben  |     20 |     20 | Drama
  3 | Cai  |     20 |     20 | Drama
  4 | Dev  | (null) | (null) | (null)
  5 | Eli  |     40 | (null) | (null)
(5 rows)
```

`RIGHT JOIN` — the matched pairs **plus** every unmatched *right* row, padded with `NULL`s on the left (4 rows):

```
SELECT s.id, s.name, s.club_id AS s_club, c.club_id AS c_club, c.club
FROM students s RIGHT JOIN clubs c ON c.club_id = s.club_id
ORDER BY c.club_id, s.id;
```

```
   id   |  name  | s_club | c_club |   club
--------+--------+--------+--------+----------
      1 | Ana    |     10 |     10 | Chess
      2 | Ben    |     20 |     20 | Drama
      3 | Cai    |     20 |     20 | Drama
 (null) | (null) | (null) |     30 | Robotics
(4 rows)
```

`FULL JOIN` — matched pairs, plus unmatched rows from **both** sides (6 rows):

```
SELECT s.id, s.name, s.club_id AS s_club, c.club_id AS c_club, c.club
FROM students s FULL JOIN clubs c ON c.club_id = s.club_id
ORDER BY s.id NULLS LAST, c.club_id;
```

```
   id   |  name  | s_club | c_club |   club
--------+--------+--------+--------+----------
      1 | Ana    |     10 |     10 | Chess
      2 | Ben    |     20 |     20 | Drama
      3 | Cai    |     20 |     20 | Drama
      4 | Dev    | (null) | (null) | (null)
      5 | Eli    |     40 | (null) | (null)
 (null) | (null) | (null) |     30 | Robotics
(6 rows)
```

Summary of the same example:

Join	Matched pairs	+ unmatched left (Dev, Eli)	+ unmatched right (Robotics)	Rows
`INNER`	✓			3
`LEFT`	✓	✓		5
`RIGHT`	✓		✓	4
`FULL`	✓	✓	✓	6
`CROSS`	every pair, no condition			5 × 3 = 15

Notice two things in the `LEFT` result. Dev's row has `s_club = (null)` because Dev's own value is `NULL`; Eli's has `s_club = 40` but `c_club = (null)` because the `NULL`s were **added by the join**. A `NULL` in a right-side column after a `LEFT JOIN` therefore means "no partner was found" — unless that column can also be `NULL` in the data, which is why tests for "no match" should use a column that is never `NULL` in the right table, typically its primary key.

The edges of the Venn diagram: "only in one side"

Adding `WHERE <right key> IS NULL` after a `LEFT JOIN` keeps just the rows that found no partner — the left-only crescent (an **anti-join**, §5.8). The same idea works for the right side and for "only in one of them":

```sql
SELECT s.name FROM students s LEFT JOIN clubs c ON c.club_id = s.club_id
WHERE c.club_id IS NULL ORDER BY s.id;

SELECT c.club FROM students s RIGHT JOIN clubs c ON c.club_id = s.club_id
WHERE s.id IS NULL;

SELECT s.name, c.club FROM students s FULL JOIN clubs c ON c.club_id = s.club_id
WHERE s.id IS NULL OR c.club_id IS NULL ORDER BY s.id NULLS LAST;
```

Output:

```
 name
------
 Dev
 Eli
(2 rows)

   club
----------
 Robotics
(1 row)

  name  |   club
--------+----------
 Dev    | (null)
 Eli    | (null)
 (null) | Robotics
(3 rows)
```

The rest of the chapter goes through each join type in depth on the company data.

5.3 INNER JOIN

What it is

An **inner join** returns only the pairs of rows for which the join condition is true. Rows with no partner on the other side disappear from the result. `JOIN` on its own means `INNER JOIN`.

Analogy: matching guests to reserved seats — a guest without a reservation and a seat nobody booked are both left out of the seating list.

How it works

1. For each row of the left table, find the rows of the right table where the `ON` condition is `TRUE`.
2. Output one combined row per matching pair — so a left row with 3 matches appears 3 times, and a left row with 0 matches not at all.
3. Rows where the key is `NULL` never match (the comparison is UNKNOWN).

```sql
SELECT e.emp_id, e.first_name, d.name AS department
FROM employees e
INNER JOIN departments d ON d.dept_id = e.dept_id
ORDER BY e.emp_id;
```

Output:

```
 emp_id | first_name | department
--------+------------+-------------
      2 | Vikram     | Engineering
      3 | Neha       | Engineering
      4 | Rahul      | Engineering
      5 | Meera      | Engineering
      6 | Daniel     | Sales
      7 | Priya      | Sales
      8 | Arjun      | Sales
      9 | Sara       | Sales
     10 | Lucas      | Marketing
     11 | Ananya     | Marketing
     12 | Kiran      | Finance
     13 | Omar       | Finance
     14 | Li         | Support
(13 rows)
```

13 rows: Asha (1) and Rohan (15) are missing because their `dept_id` is `NULL`, and Research is missing because no employee points to it.

Table aliases (`e`, `d`) are short names for the tables in this query. They are needed when a column name exists in both tables (`dept_id`), and make long queries readable. Always qualify columns with the alias in joins — `e.dept_id`, not just `dept_id` — or you will get `column reference "dept_id" is ambiguous`.

Filters with joins

For an inner join, a condition gives the same result whether you put it in `ON` or in `WHERE`. Convention: the relationship goes in `ON`, other filters go in `WHERE`. (For outer joins the placement **changes the result** — §5.9.)

```sql
SELECT e.first_name, e.salary, d.name AS department, d.location
FROM employees e
JOIN departments d ON d.dept_id = e.dept_id
WHERE d.location = 'Mumbai' AND e.salary > 70000
ORDER BY e.salary DESC;
```

Output:

```
 first_name |  salary   | department | location
------------+-----------+------------+----------
 Daniel     | 150000.00 | Sales      | Mumbai
 Lucas      | 120000.00 | Marketing  | Mumbai
 Priya      |  72000.00 | Sales      | Mumbai
(3 rows)
```

Joining three or more tables

Each additional `JOIN` joins the result so far with one more table. An order joined to its customer and its sales rep:

```sql
SELECT o.order_id, c.first_name AS customer, e.first_name AS rep, o.status
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
JOIN employees e ON e.emp_id = o.sales_rep_id
ORDER BY o.order_id;
```

Output:

```
 order_id | customer |  rep  |  status
----------+----------+-------+-----------
      101 | Anita    | Priya | delivered
      103 | Chen     | Arjun | delivered
      105 | Divya    | Priya | cancelled
      107 | Fatima   | Sara  | shipped
      108 | Ben      | Priya | delivered
      110 | George   | Arjun | shipped
      112 | Ethan    | Sara  | shipped
      113 | Anita    | Priya | pending
(8 rows)
```

Only 8 of the 14 orders appear — the 6 online orders have `sales_rep_id = NULL` and the inner join to `employees` dropped them. That may or may not be what you want; §5.10 fixes it with a `LEFT JOIN`.

A **many-to-many** relationship (employees ↔ projects: one employee works on many projects, one project has many employees) goes through a **junction table** (`employee_projects`, whose rows each link one employee to one project), so you join through it:

```sql
SELECT p.name AS project, d.name AS dept, e.first_name, ep.role, ep.hours
FROM employee_projects ep
JOIN employees   e ON e.emp_id     = ep.emp_id
JOIN projects    p ON p.project_id = ep.project_id
JOIN departments d ON d.dept_id    = p.dept_id
WHERE p.project_id IN (1, 2)
ORDER BY p.project_id, ep.hours DESC;
```

Output:

```
     project     |    dept     | first_name |   role    | hours
-----------------+-------------+------------+-----------+-------
 Checkout Revamp | Engineering | Neha       | Developer |   400
 Checkout Revamp | Engineering | Rahul      | Developer |   350
 Checkout Revamp | Engineering | Vikram     | Lead      |   120
 Mobile App      | Engineering | Neha       | Lead      |   300
 Mobile App      | Engineering | Meera      | Developer |   280
 Mobile App      | Engineering | Rohan      | Tester    |    90
(6 rows)
```

Joins and aggregation work together — join first, then group (Chapter 4):

```sql
SELECT e.first_name AS rep, count(DISTINCT o.order_id) AS orders,
       round(sum(oi.quantity * oi.unit_price * (1 - oi.discount)), 2) AS revenue
FROM orders o
JOIN employees e    ON e.emp_id = o.sales_rep_id
JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.status <> 'cancelled'
GROUP BY e.emp_id, e.first_name
ORDER BY revenue DESC;
```

Output:

```
  rep  | orders | revenue
-------+--------+---------
 Priya |      3 | 4141.04
 Sara  |      2 |  879.91
 Arjun |      2 |  687.00
(3 rows)
```

`count(DISTINCT o.order_id)` rather than `count(*)`: after joining to `order_items`, each order appears once *per line*, so `count(*)` would count lines, not orders (the fan-out effect, §5.9).

Non-equi joins

The join condition can be any boolean expression. Here we pair each order line with its product only when the price paid was below the current list price:

```
SELECT oi.order_id, p.name, p.unit_price AS list_price, oi.unit_price AS paid
FROM order_items oi
JOIN products p ON p.product_id = oi.product_id
               AND oi.unit_price < p.unit_price
ORDER BY oi.order_id;
```

Output:

```
 order_id |     name      | list_price |  paid
----------+---------------+------------+---------
      104 | Standing Desk |     499.00 |  479.00
      108 | Laptop Pro 14 |    1299.00 | 1249.00
(2 rows)
```

Other common non-equi joins: matching a value to a range (`ON s.amount BETWEEN b.low AND b.high` for tax brackets or grade bands), and time windows (`ON b.t > a.t AND b.t <= a.t + interval '1 day'`).

In the interview

30-second answer. "An inner join returns the combinations of rows from both tables that satisfy the join condition; unmatched rows on either side are dropped, and `NULL` keys never match. If a row has several matches it appears several times."

- *Q: Is `JOIN` the same as `INNER JOIN`?* — Yes.
- *Q: `ON` vs `WHERE` for an inner join?* — Same result; put the relationship in `ON` for readability. For outer joins they differ.
- *Q: Why did my row count go up after a join?* — One-to-many (one left row relates to many right rows): each left row repeats once per matching right row.

Self-check

1. `employees` has 15 rows. Can `employees JOIN departments` return more than 15 rows? → Not here, because each employee has at most one department (the join key is the department's primary key). Joining in the other direction to a many-side (e.g. `orders JOIN order_items`) can.
2. Why are the CEO and the intern missing from the inner join? → Their `dept_id` is `NULL`, and `NULL = x` is never true.
3. What error do you get if you write `SELECT dept_id FROM employees e JOIN departments d ON …`? → `column reference "dept_id" is ambiguous`.

TL;DR

`INNER JOIN` = matched pairs only. Unmatched and `NULL`-keyed rows vanish; multiple matches multiply rows. Qualify columns with aliases.

5.4 LEFT OUTER JOIN

What it is

A **left outer join** (`LEFT JOIN` or `LEFT OUTER JOIN` — `OUTER` is optional) returns **every row of the left table**. Rows with a partner are combined with it, exactly as in an inner join; rows **without** a partner are still returned once, with every column from the right table set to `NULL` (called **NULL padding** or **NULL extension**).

Analogy: a class register. Every student is listed; next to each is their exam mark — or a blank if they missed the exam. The blank doesn't remove the student from the register.

Why it matters

It answers the questions that inner joins silently get wrong: "list **all** customers with their order count — **including zero**", "all employees with their department — even if unassigned", "every product and how much it sold".

How it works (rows of employees LEFT JOIN departments)

The two inputs around the edge cases:

```
SELECT emp_id, first_name, dept_id FROM employees WHERE emp_id IN (1, 2, 14, 15) ORDER BY emp_id;
SELECT dept_id, name FROM departments ORDER BY dept_id;
```

```
 emp_id | first_name | dept_id
--------+------------+---------
      1 | Asha       |  (null)
      2 | Vikram     |       1
     14 | Li         |       5
     15 | Rohan      |  (null)
(4 rows)

 dept_id |    name
---------+-------------
       1 | Engineering
       2 | Sales
       3 | Marketing
       4 | Finance
       5 | Support
       6 | Research
(6 rows)
```

```
SELECT e.first_name, e.dept_id AS e_dept, d.dept_id AS d_dept, d.name
FROM employees e LEFT JOIN departments d ON d.dept_id = e.dept_id
ORDER BY d.dept_id NULLS LAST, e.emp_id;
```

Output:

```
 first_name | e_dept | d_dept |    name
------------+--------+--------+-------------
 Vikram     |      1 |      1 | Engineering
 Neha       |      1 |      1 | Engineering
 Rahul      |      1 |      1 | Engineering
 Meera      |      1 |      1 | Engineering
 Daniel     |      2 |      2 | Sales
 Priya      |      2 |      2 | Sales
 Arjun      |      2 |      2 | Sales
 Sara       |      2 |      2 | Sales
 Lucas      |      3 |      3 | Marketing
 Ananya     |      3 |      3 | Marketing
 Kiran      |      4 |      4 | Finance
 Omar       |      4 |      4 | Finance
 Li         |      5 |      5 | Support
 Asha       | (null) | (null) | (null)
 Rohan      | (null) | (null) | (null)
(15 rows)
```

1. The 13 employees who have a department are matched exactly as in the inner join.
2. Asha and Rohan find no partner. Instead of being dropped, each is output **once** with `d_dept` and `name` padded with `NULL`.
3. Research is not in the result: a left join keeps unmatched rows of the **left** table only.

Rule of thumb: **`LEFT JOIN` never returns fewer rows than the left table has.** It returns more if some left rows match several right rows.

The everyday use: counts that include zero

```sql
SELECT c.customer_id, c.first_name, count(o.order_id) AS orders
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.first_name
ORDER BY orders, c.customer_id;
```

Output:

```
 customer_id | first_name | orders
-------------+------------+--------
           9 | Hana       |      0
          10 | Ivan       |      0
           4 | Divya      |      1
           7 | George     |      1
           8 | Anita      |      1
           2 | Ben        |      2
           3 | Chen       |      2
           5 | Ethan      |      2
           6 | Fatima     |      2
           1 | Anita      |      3
(10 rows)
```

Hana and Ivan have 0 orders. It must be `count(o.order_id)` — a column from the right table — not `count(*)`: their padded row still counts as one row for `count(*)` (§5.9).

When not to use it

- If you only need **left rows that have a match** (and don't need right-side columns), a semi-join with `EXISTS` is clearer and never duplicates rows (§5.8).
- If you actually require a match, use `INNER JOIN` — a `LEFT JOIN` followed by a `WHERE` on a right column silently becomes an inner join anyway (§5.9).

Common mistakes

- Filtering the right table in `WHERE` (turns it back into an inner join) — put the condition in `ON`.
- Following a `LEFT JOIN` with an `INNER JOIN` to a table hanging off the right side: `a LEFT JOIN b … JOIN c ON c.x = b.x` drops the padded rows again, because `b.x` is `NULL`. Make the second join a `LEFT JOIN` too.
- Using `count(*)` for "how many matches".

In the interview

> **30-second answer.** "A left join returns every row of the left table; where there is a matching right row the columns are filled in, otherwise they're `NULL`. It's the standard way to keep rows with no related data — like customers with zero orders — and, with `WHERE right.pk IS NULL`, to find rows with no match."

- *Q: How many rows can a left join return?* — At least as many as the left table; more if there are one-to-many matches.
- *Q: Find employees without a department.* — `LEFT JOIN departments … WHERE d.dept_id IS NULL`, or simply `WHERE dept_id IS NULL` on employees — plus rows whose `dept_id` points to a missing department, if there is no foreign key.

Self-check

1. After `employees LEFT JOIN departments`, which employees have `d.name = NULL`? → Asha and Rohan.
2. Would the result change if we swapped the tables? → Yes, it becomes "all departments" (see §5.5).
3. Why use `count(o.order_id)` rather than `count(*)` in the customer query? → `count(*)` counts the `NULL`-padded row as 1.

TL;DR

`LEFT JOIN` keeps all left rows and pads missing right-side data with `NULL`. It's how you include "zero" and find "missing".

5.5 RIGHT OUTER JOIN

What it is

A **right outer join** is the mirror image: it keeps **every row of the right table**, padding the left side with `NULL`s where there's no match. `A RIGHT JOIN B` returns the same rows as `B LEFT JOIN A` (only the column order differs).

```
SELECT e.first_name, e.dept_id AS e_dept, d.dept_id AS d_dept, d.name
FROM employees e RIGHT JOIN departments d ON d.dept_id = e.dept_id
ORDER BY d.dept_id, e.emp_id;
```

Output:

```
 first_name | e_dept | d_dept |    name
------------+--------+--------+-------------
 Vikram     |      1 |      1 | Engineering
 Neha       |      1 |      1 | Engineering
 Rahul      |      1 |      1 | Engineering
 Meera      |      1 |      1 | Engineering
 Daniel     |      2 |      2 | Sales
 Priya      |      2 |      2 | Sales
 Arjun      |      2 |      2 | Sales
 Sara       |      2 |      2 | Sales
 Lucas      |      3 |      3 | Marketing
 Ananya     |      3 |      3 | Marketing
 Kiran      |      4 |      4 | Finance
 Omar       |      4 |      4 | Finance
 Li         |      5 |      5 | Support
 (null)     | (null) |      6 | Research
(14 rows)
```

Now Research appears with the **employee** columns padded with `NULL`, and Asha and Rohan are gone (they're unmatched *left* rows). Swapping the tables and using `LEFT JOIN` gives the same 14 rows:

```
SELECT count(*) AS left_swapped FROM departments d LEFT JOIN employees e ON d.dept_id = e.dept_id;
```

```
 left_swapped
--------------
           14
(1 row)
```

When to use it

Rarely. Most teams write every outer join as a `LEFT JOIN` and put the "keep all rows of this" table first, because a query that reads "start from departments, attach employees where possible" is easier to follow. `RIGHT JOIN` is handy when you are appending one more table to a long chain of joins and want to keep all rows of *that* new table without restructuring the query. SQLite supported `RIGHT` and `FULL` joins only from version 3.39.0 (2022).

Self-check

1. Rewrite `orders o RIGHT JOIN customers c ON …` as a left join. → `customers c LEFT JOIN orders o ON …`.
2. Which department does the right join add compared with the inner join? → Research, with `NULL` employee columns.

TL;DR

`RIGHT JOIN` = `LEFT JOIN` with the tables swapped. Prefer writing it as a `LEFT JOIN`.

5.6 FULL OUTER JOIN

What it is

A **full outer join** keeps **all rows from both tables**: matched pairs are combined; unmatched left rows are padded on the right; unmatched right rows are padded on the left. It is the union of the left-join and right-join results (without repeating the matched pairs).

```sql
SELECT e.first_name, e.dept_id AS e_dept, d.dept_id AS d_dept, d.name
FROM employees e FULL JOIN departments d ON d.dept_id = e.dept_id
ORDER BY d.dept_id NULLS LAST, e.emp_id;
```

Output:

```
 first_name | e_dept | d_dept |    name
------------+--------+--------+-------------
 Vikram     |      1 |      1 | Engineering
 Neha       |      1 |      1 | Engineering
 Rahul      |      1 |      1 | Engineering
 Meera      |      1 |      1 | Engineering
 Daniel     |      2 |      2 | Sales
 Priya      |      2 |      2 | Sales
 Arjun      |      2 |      2 | Sales
 Sara       |      2 |      2 | Sales
 Lucas      |      3 |      3 | Marketing
 Ananya     |      3 |      3 | Marketing
 Kiran      |      4 |      4 | Finance
 Omar       |      4 |      4 | Finance
 Li         |      5 |      5 | Support
 (null)     | (null) |      6 | Research
 Asha       | (null) | (null) | (null)
 Rohan      | (null) | (null) | (null)
(16 rows)
```

The row counts of the four joins on the same data line up exactly with the edge cases — 13 matched pairs, 2 unmatched employees, 1 unmatched department:

```sql
SELECT (SELECT count(*) FROM employees e JOIN departments d ON d.dept_id = e.dept_id)       AS
inner_rows,
       (SELECT count(*) FROM employees e LEFT JOIN departments d ON d.dept_id = e.dept_id)  AS left_rows,
       (SELECT count(*) FROM employees e RIGHT JOIN departments d ON d.dept_id = e.dept_id) AS
right_rows,
       (SELECT count(*) FROM employees e FULL JOIN departments d ON d.dept_id = e.dept_id)  AS full_rows;
```

```
 inner_rows | left_rows | right_rows | full_rows
------------+-----------+------------+-----------
         13 |        15 |         14 |        16
(1 row)
```

In general: `full = inner + unmatched left + unmatched right`, `left = inner + unmatched left`, `right = inner + unmatched right`.

Real-world use: reconciliation

`FULL JOIN` shines in **reconciliation** — checking that two records of the same facts agree — i.e. **comparing two lists** where each side may have entries the other lacks: planned vs actual, yesterday's snapshot vs today's, the bank statement vs the ledger.

```sql
CREATE TABLE planned (emp_id int, hours int);
CREATE TABLE actual  (emp_id int, hours int);
INSERT INTO planned VALUES (3, 300), (4, 200), (5, 250);
INSERT INTO actual  VALUES (3, 310), (5, 250), (7, 40);
SELECT coalesce(p.emp_id, a.emp_id) AS emp_id, p.hours AS planned, a.hours AS actual,
       CASE WHEN p.emp_id IS NULL THEN 'unplanned work'
            WHEN a.emp_id IS NULL THEN 'no work logged'
            WHEN p.hours <> a.hours THEN 'different'
            ELSE 'ok' END AS verdict
FROM planned p FULL JOIN actual a ON a.emp_id = p.emp_id
ORDER BY 1;
```

Output:

```
 emp_id | planned | actual |    verdict
--------+---------+--------+----------------
      3 |     300 |    310 | different
      4 |     200 | (null) | no work logged
      5 |     250 |    250 | ok
      7 |  (null) |     40 | unplanned work
(4 rows)
```

`COALESCE(p.emp_id, a.emp_id)` picks whichever side's key is present, giving one clean key column.

Dialects

Database	FULL OUTER JOIN
PostgreSQL, SQL Server, Oracle	supported
SQLite	supported since 3.39.0
MySQL / MariaDB	**not supported** — emulate: `SELECT … FROM a LEFT JOIN b ON … UNION SELECT … FROM a RIGHT JOIN b ON …` (or `UNION ALL` with the right-join half filtered to `WHERE a.key IS NULL` to avoid removing genuine duplicates)

PostgreSQL restricts `FULL JOIN` conditions to ones it can run as a merge or hash join (two of the physical join algorithms described in §5.10) (in practice: at least one equality between the two sides); an arbitrary condition such as `ON a.x < b.y` alone gives an error.

In the interview

30-second answer. "A full outer join returns every row from both tables, matching where it can and padding the other side with `NULL`s where it can't. It's used to reconcile two datasets. MySQL lacks it, so you emulate it with a `LEFT JOIN` `UNION` a `RIGHT JOIN`."

- *Q: How do you find rows that are in exactly one of two tables?* — `FULL JOIN … WHERE a.pk IS NULL OR b.pk IS NULL` (or two anti-joins, or `EXCEPT` — "rows in the first result but not the second" — both ways; Chapter 6).

Self-check

1. Inner join = 13 rows, 2 unmatched left, 1 unmatched right. How many rows in the full join? → 16.
2. Why use `COALESCE(p.emp_id, a.emp_id)`? → On unmatched rows one of the two keys is `NULL`; `COALESCE` returns the one that exists.

TL;DR

`FULL JOIN` = everything from both sides, `NULL`-padded where unmatched. Great for reconciliation; emulate with `UNION` in MySQL.

5.7 SELF JOIN

What it is

A **self join** joins a table **to itself**. It is not a new keyword — it's an ordinary inner or outer join in which the same table appears twice under two different aliases, so SQL can tell "the employee row" from "the manager row". You need it whenever rows of a table refer to other rows of the **same** table (a hierarchy through `manager_id`), or when you want to compare rows of a table with each other (pairs, sequences).

Analogy: a class photo with two copies. On one copy you circle a student; on the other you find the student sitting next to them. Same photo, two roles.

How it works

```sql
SELECT e.emp_id, e.first_name AS employee, m.first_name AS manager
FROM employees e
LEFT JOIN employees m ON m.emp_id = e.manager_id
ORDER BY e.emp_id;
```

Output:

```
 emp_id | employee | manager
--------+----------+---------
      1 | Asha     | (null)
      2 | Vikram   | Asha
      3 | Neha     | Vikram
      4 | Rahul    | Vikram
      5 | Meera    | Vikram
      6 | Daniel   | Asha
      7 | Priya    | Daniel
      8 | Arjun    | Daniel
      9 | Sara     | Daniel
     10 | Lucas    | Asha
     11 | Ananya   | Lucas
     12 | Kiran    | Asha
     13 | Omar     | Kiran
     14 | Li       | Vikram
     15 | Rohan    | Vikram
(15 rows)
```

1. `e` is "the employee" copy and `m` is "the manager" copy of the same table.
2. The condition `m.emp_id = e.manager_id` pairs each employee with the row whose id equals their `manager_id`.
3. `LEFT JOIN` keeps Asha, who has no manager (`manager_id` is `NULL`); an inner join would drop her.

More self-join patterns

Compare each employee with their manager:

```sql
SELECT e.first_name, e.salary, m.first_name AS manager, m.salary AS mgr_salary
FROM employees e JOIN employees m ON m.emp_id = e.manager_id
WHERE e.salary >= 0.7 * m.salary
ORDER BY e.emp_id;
```

```
 first_name |  salary   | manager | mgr_salary
------------+-----------+---------+------------
 Vikram     | 185000.00 | Asha    |  250000.00
 Neha       | 142000.00 | Vikram  |  185000.00
(2 rows)
```

Count direct reports (join, then group by the manager):

```sql
SELECT m.first_name AS manager, count(*) AS direct_reports
FROM employees e JOIN employees m ON m.emp_id = e.manager_id
GROUP BY m.emp_id, m.first_name
ORDER BY direct_reports DESC, manager;
```

```
 manager | direct_reports
---------+----------------
 Vikram  |              5
 Asha    |              4
 Daniel  |              3
 Kiran   |              1
 Lucas   |              1
(5 rows)
```

Find **pairs** of rows that share a value. The condition `a.emp_id < b.emp_id` removes pairing a row with itself (Arjun–Arjun) and mirror duplicates (Arjun–Sara *and* Sara–Arjun):

```sql
SELECT a.first_name AS emp_a, b.first_name AS emp_b, a.salary
FROM employees a JOIN employees b ON a.salary = b.salary AND a.emp_id < b.emp_id
ORDER BY a.salary;
```

```
 emp_a | emp_b |  salary
-------+-------+----------
 Arjun | Sara  | 68000.00
 Rahul | Meera | 98000.00
(2 rows)
```

Go up **two** levels with a second self join (for arbitrary depth, use a recursive CTE — a CTE, Common Table Expression, is a named subquery written with `WITH`, and a *recursive* one keeps joining to its own previous output until nothing new is found; Chapter 7):

```sql
SELECT e.first_name AS employee, m.first_name AS manager, g.first_name AS grand_manager
FROM employees e
JOIN employees m ON m.emp_id = e.manager_id
LEFT JOIN employees g ON g.emp_id = m.manager_id
WHERE e.dept_id = 1
ORDER BY e.emp_id;
```

```
 employee | manager | grand_manager
----------+---------+---------------
 Vikram   | Asha    | (null)
 Neha     | Vikram  | Asha
 Rahul    | Vikram  | Asha
 Meera    | Vikram  | Asha
(4 rows)
```

Compare rows **in time** — here, customers who ordered again within 60 days of an earlier order:

```sql
SELECT a.customer_id, a.order_id AS first_order, b.order_id AS next_order,
       b.placed_at::date - a.placed_at::date AS days_between
FROM orders a JOIN orders b
  ON b.customer_id = a.customer_id AND b.placed_at > a.placed_at
 AND b.placed_at <= a.placed_at + interval '60 days'
ORDER BY a.customer_id, a.order_id;
```

```
 customer_id | first_order | next_order | days_between
-------------+-------------+------------+--------------
           1 |         101 |        104 |           39
(1 row)
```

Many "compare with the previous row" problems (consecutive days, streaks) are solved either with a self join like this or, more efficiently, with `LAG` / `LEAD` window functions (functions that read a value from the previous / next row without collapsing rows; Chapter 8).

Common mistakes

- Forgetting aliases (`employees JOIN employees` is an error: `table name "employees" specified more than once`).
- Using an inner join for hierarchies and losing the root (the CEO).
- Getting every pair twice (and each row paired with itself) — use `a.id < b.id`.

In the interview

30-second answer. "A self join joins a table to itself using two aliases — for hierarchies like employee → manager, for finding pairs of rows with something in common, and for comparing a row with a related row of the same table. Use a left join to keep rows without a parent, and `a.id < b.id` to avoid duplicate pairs."

- *Q: Employees who earn more than their manager?* — `FROM employees e JOIN employees m ON m.emp_id = e.manager_id WHERE e.salary > m.salary`. (In our data: nobody.)
- *Q: How do you get the whole chain of managers?* — A recursive CTE; self joins only reach a fixed depth.

Self-check

1. In the manager query, why is Asha's manager `NULL`? → She has no `manager_id`, and the `LEFT JOIN` padded the manager columns.
2. What does `a.emp_id < b.emp_id` prevent? → Self-pairs and mirrored pairs.

TL;DR

Self join = the same table twice, two aliases, two roles. Hierarchies, pairs, and row-to-row comparisons.

5.8 Semi-joins and anti-joins

What they are

Two very common questions aren't really about *combining* columns at all:

- **Semi-join** — "rows of A that **have at least one** match in B" (customers who have ordered). Each A row is returned **at most once**, and no columns of B are needed.
- **Anti-join** — "rows of A that have **no** match in B" (customers who never ordered).

SQL has no `SEMI JOIN` keyword; you write them with `EXISTS` / `IN` (semi) and `NOT EXISTS` / `LEFT JOIN … IS NULL` / `NOT IN` (anti). The query planner (the database component that decides how to execute a query) recognises these shapes and runs them with special *semi-join* and *anti-join* algorithms.

Semi-join: EXISTS vs a plain JOIN

```
SELECT c.customer_id, c.first_name
FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id)
ORDER BY c.customer_id;
```

```
 customer_id | first_name
-------------+------------
           1 | Anita
           2 | Ben
           3 | Chen
           4 | Divya
           5 | Ethan
           6 | Fatima
           7 | George
           8 | Anita
(8 rows)
```

A plain inner join answers a different question — it returns one row **per order**:

```
SELECT c.customer_id, c.first_name
FROM customers c JOIN orders o ON o.customer_id = c.customer_id
ORDER BY c.customer_id;
```

```
 customer_id | first_name
-------------+------------
           1 | Anita
           1 | Anita
           1 | Anita
           2 | Ben
           2 | Ben
           3 | Chen
           3 | Chen
           4 | Divya
           5 | Ethan
           5 | Ethan
           6 | Fatima
           6 | Fatima
           7 | George
           8 | Anita
(14 rows)
```

Fixing that with `SELECT DISTINCT` works but does extra work (build all pairs, then remove duplicates) and hides the intent. `EXISTS` stops at the first match for each customer.

Anti-join: three ways

```sql
-- 1. NOT EXISTS
SELECT c.customer_id, c.first_name FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id)
ORDER BY 1;

-- 2. LEFT JOIN ... IS NULL
SELECT c.customer_id, c.first_name FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL
ORDER BY 1;

-- 3. NOT IN
SELECT c.customer_id, c.first_name FROM customers c
WHERE c.customer_id NOT IN (SELECT customer_id FROM orders)
ORDER BY 1;
```

Output (identical for all three here):

```
 customer_id | first_name
-------------+------------
           9 | Hana
          10 | Ivan
(2 rows)
```

Form	NULL-safe?	Notes
`NOT EXISTS (correlated subquery)` (a subquery that refers to the outer row)	**Yes**	Clearest intent; PostgreSQL plans it as an anti join. **Recommended.**
`LEFT JOIN … WHERE b.pk IS NULL`	Yes, if you test a never-NULL column (the PK)	Also planned as an anti join; handy when you want other columns from the join.
`NOT IN (subquery)`	**No**	If the subquery returns even one `NULL`, the result is **empty** — see Chapter 6. Also often planned less efficiently.

The plan (printed by `EXPLAIN`, which shows the strategy the database chose; `Seq Scan` means reading the whole table) shows PostgreSQL executing `NOT EXISTS` as a **Hash Anti Join**:

```sql
EXPLAIN (COSTS OFF)
SELECT c.customer_id FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);
```

```
                  QUERY PLAN
-----------------------------------------------
 Hash Anti Join
   Hash Cond: (c.customer_id = o.customer_id)
   ->  Seq Scan on customers c
   ->  Hash
         ->  Seq Scan on orders o
(5 rows)
```

More anti-joins on our data — the product never sold and the department with no staff:

```sql
SELECT p.name FROM products p
WHERE NOT EXISTS (SELECT 1 FROM order_items oi WHERE oi.product_id = p.product_id);

SELECT d.name FROM departments d
LEFT JOIN employees e ON e.dept_id = d.dept_id
WHERE e.emp_id IS NULL;
```

```
   name
-----------
 Desk Lamp
(1 row)

   name
----------
 Research
(1 row)
```

In the interview

30-second answer. "A semi-join returns rows of A that have a match in B, once each — write it with `EXISTS` or `IN`. An anti-join returns rows of A with no match — write it with `NOT EXISTS` or `LEFT JOIN … WHERE b.pk IS NULL`. Avoid `NOT IN` with a subquery on a nullable column: one `NULL` makes it return nothing."

- *Q: Customers who never placed an order?* — Any of the three forms; say why you prefer `NOT EXISTS`.
- *Q: Why not `JOIN` + `DISTINCT` for a semi-join?* — It multiplies then de-duplicates; `EXISTS` stops at the first match and states the intent.

Self-check

1. Which is NULL-safe: `NOT IN` or `NOT EXISTS`? → `NOT EXISTS`.
2. In the `LEFT JOIN … IS NULL` pattern, why test `o.order_id` rather than, say, `o.sales_rep_id`? → `sales_rep_id` can be `NULL` on real rows (online orders); the primary key is never `NULL`, so a `NULL` there must be padding.

TL;DR

Semi-join = `EXISTS` ("has a match", no duplicates). Anti-join = `NOT EXISTS` or `LEFT JOIN … pk IS NULL` ("has no match"). Beware `NOT IN` with `NULL`s.

5.9 Join pitfalls

Pitfall 1 — fan-out (row multiplication)

When you join a parent to **two different child tables** at once, every child row of one kind pairs with every child row of the other kind. Engineering has 4 employees and 2 projects, so joining both gives 4 × 2 = 8 rows, and every salary is counted twice:

```sql
SELECT d.name, count(*) AS joined_rows, sum(e.salary) AS wrong_payroll
FROM departments d
JOIN employees e ON e.dept_id = d.dept_id
JOIN projects  p ON p.dept_id = d.dept_id
WHERE d.dept_id = 1
GROUP BY d.name;
```

```
    name     | joined_rows | wrong_payroll
-------------+-------------+---------------
 Engineering |           8 |    1046000.00
(1 row)
```

The real payroll is 523,000. **Fix:** aggregate each child table separately (in a derived table — a subquery used in `FROM` — or a CTE), so each has one row per department, *then* join:

```sql
SELECT d.name, e.payroll, p.projects
FROM departments d
JOIN (SELECT dept_id, sum(salary) AS payroll FROM employees GROUP BY dept_id) e ON e.dept_id = d.dept_id
JOIN (SELECT dept_id, count(*) AS projects FROM projects GROUP BY dept_id)     p ON p.dept_id = d.dept_id
ORDER BY d.dept_id;
```

```
    name     |  payroll  | projects
-------------+-----------+----------
 Engineering | 523000.00 |        2
 Marketing   | 181000.00 |        1
 Finance     | 211000.00 |        1
 Support     |  64000.00 |        1
(4 rows)
```

Signs of fan-out: totals that are suspiciously exact multiples of the right answer, and `count(*)` larger than the number of parent rows. `count(DISTINCT …)` hides it for counts but **not** for sums.

Pitfall 2 — a WHERE on the right table undoes a LEFT JOIN

Goal: every customer with their number of **delivered** orders, 0 if none.

```sql
SELECT c.customer_id, count(o.order_id) AS delivered
FROM customers c LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.status = 'delivered'
GROUP BY c.customer_id ORDER BY c.customer_id;          -- wrong: customers with 0 vanish
```

```
 customer_id | delivered
-------------+-----------
           1 |         2
           2 |         2
           3 |         1
           5 |         1
           8 |         1
(5 rows)
```

Only 5 customers. Why: the `LEFT JOIN` produced padded rows for Hana and Ivan (with `o.status = NULL`), and rows for customers whose only orders were not delivered — then `WHERE o.status = 'delivered'` ran **after** the join (logical order, Chapter 4) and threw all of those away. Put the right-table condition **in the `ON` clause**, so it limits which rows *match*, not which rows *survive*:

```sql
SELECT c.customer_id, count(o.order_id) AS delivered
FROM customers c LEFT JOIN orders o ON o.customer_id = c.customer_id
                                   AND o.status = 'delivered'
GROUP BY c.customer_id ORDER BY c.customer_id;          -- right: condition in ON
```

```
 customer_id | delivered
-------------+-----------
           1 |         2
           2 |         2
           3 |         1
           4 |         0
           5 |         1
           6 |         0
           7 |         0
           8 |         1
           9 |         0
          10 |         0
(10 rows)
```

Condition on the right table of a LEFT JOIN placed in…	Effect
`ON`	Decides which right rows count as a match; unmatched left rows are still kept (padded).
`WHERE`	Filters the joined result; padded rows have `NULL` there and are removed → behaves like an `INNER JOIN`.

(A `WHERE right.pk IS NULL` test is the deliberate exception — that's the anti-join.) Conditions on the **left** table belong in `WHERE`; put in `ON` they don't remove left rows, they just stop them matching.

Pitfall 3 — NULL keys never match

```sql
CREATE TABLE a (k int, label text);
CREATE TABLE b (k int, label text);
INSERT INTO a VALUES (1, 'a1'), (2, 'a2'), (NULL, 'a-null');
INSERT INTO b VALUES (1, 'b1'), (3, 'b3'), (NULL, 'b-null');
SELECT a.label, b.label FROM a JOIN b ON a.k = b.k;
SELECT a.label, b.label FROM a JOIN b ON a.k IS NOT DISTINCT FROM b.k ORDER BY 1;
```

```
 label | label
-------+-------
 a1    | b1
(1 row)

 label  | label
--------+--------
 a-null | b-null
 a1     | b1
(2 rows)
```

`NULL = NULL` is UNKNOWN, so the two `NULL`-keyed rows don't pair. If you really want `NULL` to match `NULL` (for example when comparing two snapshots of a nullable column), use `IS NOT DISTINCT FROM` — but know that PostgreSQL can't use a hash or merge join for it, so it can be slow on large tables.

Pitfall 4 — COUNT(*) after an outer join

```sql
SELECT d.name, count(*) AS count_star, count(e.emp_id) AS count_col
FROM departments d LEFT JOIN employees e ON e.dept_id = d.dept_id
WHERE d.dept_id IN (5, 6)
GROUP BY d.name ORDER BY d.name;
```

```
   name   | count_star | count_col
----------+------------+-----------
 Research |          1 |         0
 Support  |          1 |         1
(2 rows)
```

Research has no employees, but `count(*)` says 1 — it counted the padded row. Count a right-side, never-NULL column instead.

Other pitfalls, briefly

- **Joining on the wrong columns** — e.g. `ON e.emp_id = d.dept_id` "works" (both are integers) and returns nonsense. Foreign keys document the correct paths.
- **Type mismatches** — joining `text` to `integer` fails in PostgreSQL; casting inside the condition (`ON a.code::int = b.id`) prevents index use on `a.code` (an **index** is a sorted lookup structure kept beside a table so rows can be found by value without reading everything).
- **Duplicated keys in a lookup table** — if the "one" side isn't actually unique, every match fans out. Check with `GROUP BY key HAVING count(*) > 1`.

Self-check

1. A query sums order amounts and joins both `order_items` and `payments` to `orders`. What goes wrong? → Fan-out: each item pairs with each payment, inflating the sums. Aggregate each child first.
2. `a LEFT JOIN b ON a.id = b.a_id WHERE b.active` — what kind of join is this really? → An inner join (padded rows have `b.active = NULL`). Move `b.active` to `ON`.

TL;DR

Aggregate children separately to avoid fan-out; right-table filters of a `LEFT JOIN` go in `ON`; `NULL` keys never match; count a right-side key, not `*`.

5.10 USING, NATURAL JOIN, multi-table joins, and how joins are executed

USING

When the join columns have the **same name** in both tables, `USING (col)` is a shorthand for `ON a.col = b.col`. The shared column appears **once** in the output (and in `SELECT *`), and you refer to it without a table prefix:

```sql
SELECT order_id, customer_id, first_name, status
FROM orders JOIN customers USING (customer_id)
WHERE order_id <= 103
ORDER BY order_id;
```

```
 order_id | customer_id | first_name |  status
----------+-------------+------------+-----------
      101 |           1 | Anita      | delivered
      102 |           2 | Ben        | delivered
      103 |           3 | Chen       | delivered
(3 rows)
```

In a `FULL JOIN … USING`, the merged column is `COALESCE(left.col, right.col)`, so it is filled in for unmatched rows on either side:

```
SELECT dept_id, first_name, name
FROM employees FULL JOIN departments USING (dept_id)
WHERE first_name IS NULL OR name IS NULL
ORDER BY dept_id NULLS LAST;
```

```
 dept_id | first_name |   name
---------+------------+----------
       6 | (null)     | Research
  (null) | Asha       | (null)
  (null) | Rohan      | (null)
(3 rows)
```

NATURAL JOIN — and why to avoid it

`NATURAL JOIN` joins on **every** pair of columns that happen to share a name, without listing them. It looks convenient until a table gains a column:

```
SELECT count(*) AS natural_rows FROM projects NATURAL JOIN departments;
SELECT count(*) AS using_rows   FROM projects JOIN departments USING (dept_id);
```

```
 natural_rows
--------------
            0
(1 row)

 using_rows
------------
          5
(1 row)
```

`projects` and `departments` share `dept_id` — but also `name` and `budget`. `NATURAL JOIN` required all three to be equal, so no row matched. Schemas (the table definitions) change over time (someone adds `created_at` to both tables), and a `NATURAL JOIN` changes meaning silently. **Name your join columns.**

	ON	USING	NATURAL
Columns compared	any expression	listed same-named columns	*all* same-named columns
Duplicated key column in `SELECT *`	yes (both copies)	no (one merged column)	no
Safe against schema changes	yes	yes	**no**
SQL Server support	yes	no	no

Multi-table joins

A query can join many tables; each `JOIN` adds one table to the result built so far. A realistic report — every order line with customer, product and sales rep, keeping online orders:

```
SELECT o.order_id, c.first_name AS customer, p.name AS product, oi.quantity,
       coalesce(e.first_name, 'online') AS rep
FROM orders o
JOIN customers c    ON c.customer_id = o.customer_id
JOIN order_items oi ON oi.order_id   = o.order_id
JOIN products p     ON p.product_id  = oi.product_id
LEFT JOIN employees e ON e.emp_id    = o.sales_rep_id
WHERE o.order_id BETWEEN 101 AND 104
ORDER BY o.order_id, p.name;
```

```
 order_id | customer |           product           | quantity |  rep
----------+----------+-----------------------------+----------+--------
      101 | Anita    | Laptop Pro 14               |        1 | Priya
      101 | Anita    | Wireless Mouse              |        1 | Priya
      102 | Ben      | Noise-Cancelling Headphones |        2 | online
      103 | Chen     | Mechanical Keyboard         |        1 | Arjun
      103 | Chen     | USB-C Hub                   |        2 | Arjun
      104 | Anita    | Ergonomic Chair             |        1 | online
      104 | Anita    | Standing Desk               |        1 | online
(7 rows)
```

And a department overview built entirely from outer joins, so that Sales (no projects) and Research (nothing at all) still appear:

```sql
SELECT d.name AS department,
       count(DISTINCT e.emp_id)     AS employees,
       count(DISTINCT p.project_id) AS projects,
       coalesce(sum(ep.hours), 0)   AS hours_on_own_projects
FROM departments d
LEFT JOIN employees e          ON e.dept_id = d.dept_id
LEFT JOIN projects p           ON p.dept_id = d.dept_id
LEFT JOIN employee_projects ep ON ep.project_id = p.project_id AND ep.emp_id = e.emp_id
GROUP BY d.dept_id, d.name
ORDER BY d.dept_id;
```

```
 department  | employees | projects | hours_on_own_projects
-------------+-----------+----------+-----------------------
 Engineering |         4 |        2 |                  1450
 Sales       |         4 |        0 |                     0
 Marketing   |         2 |        1 |                   350
 Finance     |         2 |        1 |                   240
 Support     |         1 |        1 |                    60
 Research    |         0 |        0 |                     0
(6 rows)
```

This query deliberately fans out (employees × projects per department) — it is only correct because the counts use `DISTINCT` and the hours come from `employee_projects` joined on **both** keys, so each (employee, project) assignment is counted once. When in doubt, pre-aggregate as in §5.9.

Rules for multi-table joins:

1. Inner joins can be written in any order — the result is the same, and the planner picks the physical order.
2. **Outer joins are not freely reorderable**: `(a LEFT JOIN b) JOIN c` can differ from `a LEFT JOIN (b JOIN c)`. Write them in the order you mean, and keep the "keep everything from here" table first.
3. Every table after the first needs its own join condition, or you get a Cartesian product with it.

How databases execute joins

The logical definition (filtered Cartesian product) is not how joins run. The planner picks one of three physical algorithms per join, based on table sizes, indexes and statistics (stored summaries of how many rows a table has and how its values are spread) — you'll see them in `EXPLAIN` (Chapter 12):

Algorithm	How it works	Good when	Cost (roughly)
Nested loop	For each outer row, look up matching inner rows (ideally through an index)	Outer side small, index on the inner join key; also the only option for non-equi joins	*outer rows × cost of one lookup*
Hash join	Build a hash table (a lookup structure that finds a key in about one step) on the smaller input's key, then probe it with each row of the other	Equi-joins of large unsorted inputs	*O(n + m)* (work grows with the total size of both inputs), needs memory for the hash table
Merge join	Sort both inputs by the key (or read them in index order), then walk them together	Inputs already sorted / indexed; very large inputs	*O(n + m)* after sorting

Practical consequence: **index your foreign-key columns** (`orders.customer_id`, `order_items.product_id` …). PostgreSQL indexes primary keys automatically but **not** foreign keys, and nested-loop joins from a parent to its children depend on those indexes.

Dialect summary for joins

Feature	PostgreSQL	MySQL 8	SQLite	SQL Server
`INNER`, `LEFT`, `CROSS`	yes	yes	yes	yes
`RIGHT JOIN`	yes	yes	3.39+	yes
`FULL JOIN`	yes	**no** (emulate with `UNION`)	3.39+	yes
`USING`, `NATURAL`	yes	yes	yes	**no**
`LATERAL` (Chapter 6)	yes	8.0.14+	no	`CROSS APPLY` / `OUTER APPLY`
`IS NOT DISTINCT FROM`	yes	`<=>`	`IS`	2022+

In the interview

30-second answer (whole chapter). "A join pairs rows from two tables that satisfy a condition — conceptually a filtered Cartesian product. Inner join keeps matches only; left/right keep all rows of one side with `NULL` padding; full keeps both; cross gives every combination. Self joins use two aliases on one table. For 'has a match / has no match' use `EXISTS` / `NOT EXISTS`. Watch for fan-out, filters on the outer side in `WHERE`, and `NULL` keys."

- *Q: What's the difference between `WHERE` and `ON` in a left join?* — `ON` decides matches; `WHERE` filters the result and removes padded rows.
- *Q: What join algorithms do you know?* — Nested loop, hash, merge; the optimizer chooses based on sizes, indexes and sort order.
- *Q: How do you do a full outer join in MySQL?* — `LEFT JOIN` `UNION` `RIGHT JOIN`.

Self-check

1. What columns does `SELECT *` return for `orders JOIN customers USING (customer_id)`? → `customer_id` once, then the other columns of `orders` and `customers`.
2. Why is `NATURAL JOIN` risky? → It joins on every same-named column, which changes when schemas change.
3. Which physical join works for `ON a.x < b.y`? → Nested loop (hash and merge need equality).

TL;DR

Prefer explicit `ON` (or `USING`), never `NATURAL`. Inner joins reorder freely; outer joins don't. The planner chooses nested loop, hash or merge — index your foreign keys.

Chapter 5 TL;DR

Join	Returns	Typical use
`CROSS JOIN`	every pair (m × n)	grids, combinations
`INNER JOIN`	matched pairs only	"orders with their customers"
`LEFT JOIN`	all left rows + matches, `NULL`-padded	"all customers, with orders if any"
`RIGHT JOIN`	all right rows + matches	same, mirrored — write as `LEFT`
`FULL JOIN`	all rows of both sides	reconciliation
Self join	table joined to itself (two aliases)	hierarchies, pairs, previous/next
Semi-join (`EXISTS`)	left rows with ≥ 1 match, once each	"customers who ordered"
Anti-join (`NOT EXISTS`, `LEFT … IS NULL`)	left rows with no match	"customers who never ordered"

Pitfalls: fan-out, right-table filters in `WHERE` after a `LEFT JOIN`, `NULL` keys, `count(*)` on padded rows, `NOT IN` with `NULL`s, `NATURAL JOIN`.

Next: Chapter 6 — Subqueries and Set Operations.

Chapter 6 — Subqueries and Set Operations

Some questions need **the answer to another question first**. "Who earns more than the average?" needs the average before it can compare. "Which customers never ordered?" needs the list of customers who *did*. SQL lets you put one query **inside** another — a **subquery** — and use its result as a value, a list, or a whole table. The second half of the chapter covers the other way of combining queries: stacking or comparing their results with the **set operators** `UNION`, `INTERSECT` and `EXCEPT`.

What you'll learn

- What a subquery (nested query) is, where it can appear, and the three shapes of result it can return: **scalar**, **row**, **table**.
- **Non-correlated** subqueries (run once) vs **correlated** subqueries (conceptually re-run for every outer row), and how PostgreSQL actually executes them.
- Multi-row subqueries with `IN`, `EXISTS`, `ANY` / `SOME` / `ALL` — and the famous `NOT IN` + `NULL` trap.
- **Derived tables** (subqueries in `FROM`), row subqueries, and `LATERAL` for "top-N per group" (the first *N* rows inside each group, e.g. the two best-paid people in every department).
- The set operations `UNION`, `UNION ALL`, `INTERSECT`, `EXCEPT` (and their `ALL` forms): rules, `NULL` handling, precedence, sorting, and when to prefer a join or `EXISTS` instead.

How to run the examples. All queries are in `code/` and ran on the book's private PostgreSQL 14; outputs are real `psql` output, `NULL` shown as `(null)`. Data: the book's `company` database (`code/00_schema.sql`; see Chapter 1).

```
bash tools_run_sql.sh sql '06-*'
bash tools_pg.sh psql -d <db> -f sql/chapters/06-subqueries-and-set-operations/code/04_exists.sql
```

6.1 Subqueries: queries inside queries

What it is

A **subquery** (also called a **nested query** or **inner query**) is a complete `SELECT` statement written in parentheses inside another SQL statement (the **outer query**). The database evaluates the subquery and uses its result in the outer query — as a single value, as a list of values to compare against, or as a temporary table.

Analogy: asking a colleague "what is the team's average salary?" before you can answer your manager's question "who earns more than average?". The colleague's answer is the subquery.

Where a subquery can appear

Position	Example	What the subquery must return
`WHERE` / `HAVING` with `=`, `<`, `>` …	`WHERE salary > (SELECT avg(salary) …)`	**one value** (scalar)
`WHERE` with `IN`, `ANY`, `ALL`	`WHERE dept_id IN (SELECT dept_id …)`	**one column, many rows**
`WHERE` with `EXISTS`	`WHERE EXISTS (SELECT 1 …)`	anything — only "any rows?" matters
`SELECT` list	`SELECT name, (SELECT count(*) …) AS n`	one value per outer row
`FROM` / `JOIN`	`FROM (SELECT … ) AS t`	a whole table (**derived table**)
Row comparison	`WHERE (a, b) = (SELECT x, y …)`	one row with several columns
`INSERT … SELECT`, `UPDATE … SET col = (…)`, `DELETE … WHERE … IN (…)`	see §6.3	depends on position

Types of subqueries

Subqueries are classified in two independent ways:

By the shape of their result

Type	Returns	Typical operators
Scalar subquery	exactly one row, one column (or no row → `NULL`)	`=`, `<>`, `<`, `>`, arithmetic, `SELECT` list
Row subquery	one row, several columns	`(a, b) = (…)`
Column / multi-row subquery	one column, many rows	`IN`, `NOT IN`, `ANY`, `ALL`
Table subquery	many rows and columns	`FROM`, `EXISTS`, `IN` with row values

By dependence on the outer query

Type	Meaning	Evaluated
Non-correlated (simple, nested)	The inner query does not mention the outer query's tables. It can run on its own.	Conceptually **once**; the result is reused.
Correlated	The inner query refers to a column of the current outer row (e.g. `WHERE x.dept_id = e.dept_id`). It can't run on its own.	Conceptually **once per outer row** (§6.3).

Textbooks and courses name the same shapes in slightly different ways. How the common names map onto the two tables above:

Name you may meet	What it means	Example in this chapter
Nested scalar subquery	returns one value, used in a comparison	`salary > (SELECT avg(salary) …)` (below)
Single-row subquery	returns at most one row (one or several columns); used with `=`, `<`, `>` or a row constructor	`(dept_id, job_title) = (SELECT … WHERE emp_id = 4)` (§6.6)
Multiple-row subquery	returns several rows; needs `IN`, `ANY`, `ALL` or `EXISTS`	`dept_id IN (SELECT dept_id … 'Mumbai')` (§6.2)
Single-column subquery	returns one column (one or many rows)	every `IN (SELECT one_column …)`
Correlated subquery	refers to the outer row	"above *their own* department's average" (§6.3)

How a non-correlated scalar subquery works

```sql
SELECT first_name, salary
FROM employees
WHERE salary > (SELECT avg(salary) FROM employees)
ORDER BY salary DESC;
```

Output:

```
 first_name |  salary
------------+-----------
 Asha       | 250000.00
 Vikram     | 185000.00
 Daniel     | 150000.00
 Neha       | 142000.00
 Kiran      | 135000.00
 Lucas      | 120000.00
(6 rows)
```

1. The database runs the inner query once: the average salary is 107,800.
2. It substitutes that value, so the outer query becomes `WHERE salary > 107800`.
3. It runs the outer query.

Without a subquery you would need two round trips (a request sent to the database plus its reply) from your application (fetch the average, then send a second query) — and the value could change between them.

A scalar subquery can be used anywhere a single value can, including the `SELECT` list and arithmetic:

```sql
SELECT first_name, salary,
       round((SELECT avg(salary) FROM employees)) AS company_avg,
       round(salary - (SELECT avg(salary) FROM employees)) AS diff
FROM employees
WHERE dept_id = 1
ORDER BY emp_id;
```

Output:

```
 first_name |  salary   | company_avg | diff
------------+-----------+-------------+-------
 Vikram     | 185000.00 |      107800 | 77200
 Neha       | 142000.00 |      107800 | 34200
 Rahul      |  98000.00 |      107800 | -9800
 Meera      |  98000.00 |      107800 | -9800
(4 rows)
```

Two classic uses — "the row with the extreme value" and "the second-highest value":

```
SELECT first_name, hire_date FROM employees
WHERE hire_date = (SELECT max(hire_date) FROM employees);

SELECT max(salary) AS second_highest
FROM employees
WHERE salary < (SELECT max(salary) FROM employees);
```

Output:

```
 first_name | hire_date
------------+------------
 Rohan      | 2025-06-02
(1 row)

 second_highest
----------------
      185000.00
(1 row)
```

The second query reads "the largest salary that is smaller than the largest salary". (For the *N*-th highest in general, window functions — functions that compute over related rows while keeping every row — such as `DENSE_RANK()` are cleaner — Chapter 8 — and interview versions are in Chapter 19.)

Scalar subqueries: the two edge cases

If a scalar subquery returns **no rows**, its value is `NULL`:

```
SELECT (SELECT salary FROM employees WHERE emp_id = 999) AS missing;
```

```
 missing
---------
  (null)
(1 row)
```

If it returns **more than one row**, the query fails at run time:

```
SELECT first_name FROM employees
WHERE salary = (SELECT salary FROM employees WHERE dept_id = 1);
```

```
ERROR:  more than one row returned by a subquery used as an expression
```

This error depends on the **data**, not just the query — it can work in testing and fail in production (the live system real users depend on) once a second matching row appears. If more than one row is possible, use `IN` (§6.2), or make the subquery provably single-row (aggregate it, or filter on a unique key).

Common mistakes

- Using `=` with a subquery that can return several rows (use `IN`).
- Forgetting the parentheses, or putting `ORDER BY` inside a subquery and expecting the outer result to be ordered (only the outermost `ORDER BY` orders the final result).
- Writing a subquery where a join is simpler — e.g. fetching a column from a related table for every row (see §6.3 for when each is better).

Self-check

1. What does a scalar subquery return when it finds no row? → `NULL`.
2. Is `(SELECT avg(salary) FROM employees)` correlated? → No — it doesn't refer to any outer table, so it runs once.
3. Name the four places a subquery can appear in a `SELECT`. → `SELECT` list, `FROM`/`JOIN`, `WHERE`, `HAVING`.

TL;DR

A subquery is a `SELECT` inside parentheses. By result: scalar, row, column, table. By dependence: non-correlated (runs once) or correlated (per outer row). Scalar with 0 rows → `NULL`; with 2+ rows → error.

6.2 Multi-row subqueries: IN and NOT IN

What it is

When the subquery returns **one column with many rows**, you compare against the whole list. `x IN (subquery)` is true if `x` equals **any** value in the list; `x NOT IN (subquery)` is true if `x` equals **none** of them.

```
SELECT first_name, dept_id FROM employees
WHERE dept_id IN (SELECT dept_id FROM departments WHERE location = 'Mumbai')
ORDER BY emp_id;
```

Output:

```
 first_name | dept_id
------------+---------
 Daniel     |       2
 Priya      |       2
 Arjun      |       2
 Sara       |       2
 Lucas      |       3
 Ananya     |       3
(6 rows)
```

The subquery returns `{2, 3}` (Sales and Marketing are in Mumbai), and the outer query keeps employees whose `dept_id` is in that set. `IN` never produces duplicates in the outer result, even if the list contains a value many times — it's a semi-join (each outer row is returned at most once if it has any match; Chapter 5).

The subquery can have its own joins and filters:

```
SELECT name FROM products
WHERE product_id IN (SELECT oi.product_id
                     FROM order_items oi JOIN orders o ON o.order_id = oi.order_id
                     WHERE o.status = 'delivered')
ORDER BY name;
```

Output:

```
           name
----------------------------
 Ergonomic Chair
 Gift Card
 Laptop Pro 14
 Mechanical Keyboard
 Noise-Cancelling Headphones
 Standing Desk
 USB-C Hub
 Wireless Mouse
(8 rows)
```

Nesting several levels

Subqueries can be nested inside subqueries. Read them **from the innermost outwards**: Furniture products → orders containing them → the customers of those orders.

```
SELECT first_name, last_name FROM customers
WHERE customer_id IN (
      SELECT customer_id FROM orders
      WHERE order_id IN (
            SELECT order_id FROM order_items
            WHERE product_id IN (SELECT product_id FROM products WHERE category = 'Furniture')))
ORDER BY customer_id;
```

Output:

```
 first_name | last_name
------------+-----------
 Anita      | Shah
 Fatima     | Qureshi
 George     | Hill
(3 rows)
```

Deep nesting is legal but hard to read; the same question is usually clearer as joins plus `DISTINCT` / `EXISTS`, or as a chain of CTEs (Common Table Expressions — named subqueries written with `WITH`; Chapter 7).

The NOT IN trap

Question: which employees are **not** anybody's manager?

```
SELECT count(*) AS not_in_rows FROM employees
WHERE emp_id NOT IN (SELECT manager_id FROM employees);
```

Output:

```
 not_in_rows
-------------
           0
(1 row)
```

Zero — obviously wrong. The subquery's list contains a `NULL` (the CEO has no manager). `x NOT IN (a, b, NULL)` means `x <> a AND x <> b AND x <> NULL`. The last comparison is UNKNOWN for every `x`, so the whole condition can never be TRUE — at best UNKNOWN — and `WHERE` keeps only TRUE rows (three-valued logic, Chapter 2). One `NULL` in the list silently empties the result.

Condition	`x = 3` , list `(1, 2)`	`x = 3` , list `(1, 2, NULL)`	`x = 1` , list `(1, 2, NULL)`
`x IN (list)`	FALSE	**UNKNOWN**	TRUE
`x NOT IN (list)`	TRUE	**UNKNOWN** (row dropped)	FALSE

Two fixes:

```
-- Fix 1: filter the NULLs out of the subquery
SELECT first_name FROM employees
WHERE emp_id NOT IN (SELECT manager_id FROM employees WHERE manager_id IS NOT NULL)
ORDER BY emp_id;

-- Fix 2 (preferred): NOT EXISTS is NULL-safe
SELECT first_name FROM employees e
WHERE NOT EXISTS (SELECT 1 FROM employees r WHERE r.manager_id = e.emp_id)
ORDER BY emp_id;
```

Output (both):

```
 first_name
------------
 Neha
 Rahul
 Meera
 Priya
 Arjun
 Sara
 Ananya
 Omar
 Li
 Rohan
(10 rows)
```

Prefer `NOT EXISTS`: it is correct whatever the data, states the intent ("there is no row that…"), and PostgreSQL plans it as an efficient anti join (a join that returns only the outer rows with *no* match), whereas `NOT IN (subquery)` must stay a filter that checks the whole list (hashed — loaded into an in-memory hash table, a structure that finds a value in about one step — if it fits in memory, otherwise re-scanned per row).

In the interview

30-second answer. "`IN (subquery)` checks membership in the list the subquery returns — a semi-join, no duplicates. `NOT IN` is dangerous: if the list contains a `NULL`, `NOT IN` is never true and returns nothing. Use `NOT EXISTS` (or filter out `NULL`s) for anti-joins."

- *Q: Why does `NOT IN` return no rows?* — The list has a `NULL`; `x <> NULL` is UNKNOWN, so the `AND` chain is never TRUE.
- *Q: Does `IN` have the same problem?* — No: a `NULL` in the list can't make a match TRUE into something else; it can only turn FALSE into UNKNOWN, and both are dropped by `WHERE`.

Self-check

1. `SELECT 5 NOT IN (1, 2, NULL)` returns? → `NULL` (UNKNOWN).
2. Which three columns in our schema would make a `NOT IN (SELECT col …)` risky? → Any nullable one, e.g. `employees.manager_id`, `employees.dept_id`, `orders.sales_rep_id`.

TL;DR

`IN` = "is in the list"; `NOT IN` + a `NULL` in the list = empty result. Anti-joins: use `NOT EXISTS`.

6.3 Correlated subqueries

What it is

A **correlated subquery** refers to a column of the **current row of the outer query**. It therefore can't be run once up front: conceptually, the database runs it again for every outer row, with that row's values plugged in.

Analogy: a teacher checking each student's mark against *that student's own class average*. The average to compare with changes as the teacher moves from student to student.

How it works

"Employees who earn more than the average **of their own department**":

```
SELECT e.first_name, e.dept_id, e.salary
FROM employees e
WHERE e.salary > (SELECT avg(x.salary) FROM employees x WHERE x.dept_id = e.dept_id)
ORDER BY e.dept_id, e.salary DESC;
```

Output:

```
 first_name | dept_id |  salary
------------+---------+-----------
 Vikram     |       1 | 185000.00
 Neha       |       1 | 142000.00
 Daniel     |       2 | 150000.00
 Lucas      |       3 | 120000.00
 Kiran      |       4 | 135000.00
(5 rows)
```

1. Take the first outer row, e.g. Vikram with `e.dept_id = 1`.
2. Run the inner query with `e.dept_id` replaced by 1 → average of Engineering = 130,750.
3. Keep Vikram if 185,000 > 130,750. Move to the next outer row and repeat.
4. Asha and Rohan (`dept_id` `NULL`) are never kept: `x.dept_id = NULL` matches nothing, so the average is `NULL` and the comparison is UNKNOWN.

The aliases `e` (outer) and `x` (inner) are essential — both refer to `employees`, and the alias tells SQL which copy each column belongs to. An unqualified column in a subquery is resolved to the **nearest** table that has it (the inner one first), which can silently turn a correlated subquery into a non-correlated one or vice versa.

"The top earner in each department" (ties would all be returned):

```
SELECT e.dept_id, e.first_name, e.salary
FROM employees e
WHERE e.salary = (SELECT max(x.salary) FROM employees x WHERE x.dept_id = e.dept_id)
ORDER BY e.dept_id;
```

Output:

```
 dept_id | first_name |  salary
---------+------------+-----------
       1 | Vikram     | 185000.00
       2 | Daniel     | 150000.00
       3 | Lucas      | 120000.00
       4 | Kiran      | 135000.00
       5 | Li         |  64000.00
(5 rows)
```

Correlated subqueries in SELECT, and the join alternative

A correlated scalar subquery in the `SELECT` list works like a per-row lookup:

```
SELECT c.customer_id, c.first_name,
       (SELECT count(*)       FROM orders o WHERE o.customer_id = c.customer_id) AS orders,
       (SELECT max(placed_at)::date FROM orders o WHERE o.customer_id = c.customer_id) AS last_order
FROM customers c
ORDER BY c.customer_id;
```

Output:

```
 customer_id | first_name | orders | last_order
-------------+------------+--------+------------
           1 | Anita      |      3 | 2025-06-02
           2 | Ben        |      2 | 2025-04-01
           3 | Chen       |      2 | 2025-05-06
           4 | Divya      |      1 | 2025-02-28
           5 | Ethan      |      2 | 2025-05-20
           6 | Fatima     |      2 | 2025-06-15
           7 | George     |      1 | 2025-04-25
           8 | Anita      |      1 | 2025-04-09
           9 | Hana       |      0 | (null)
          10 | Ivan       |      0 | (null)
(10 rows)
```

The same result with a `LEFT JOIN` and `GROUP BY`:

```
SELECT c.customer_id, c.first_name, count(o.order_id) AS orders, max(o.placed_at)::date AS last_order
FROM customers c LEFT JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.first_name
ORDER BY c.customer_id;
```

(Identical output — omitted.) How PostgreSQL's optimizer (the component that decides how to run a query) executes the subquery version, as shown by `EXPLAIN` (which prints the chosen plan; `Seq Scan` = read the whole table):

```
EXPLAIN (COSTS OFF)
SELECT c.customer_id, (SELECT count(*) FROM orders o WHERE o.customer_id = c.customer_id) AS orders
FROM customers c;
```

```
                      QUERY PLAN
------------------------------------------------------
 Seq Scan on customers c
   SubPlan 1
     ->  Aggregate
           ->  Seq Scan on orders o
                 Filter: (customer_id = c.customer_id)
(5 rows)
```

`SubPlan 1` under the customers scan means the inner aggregate really **runs once per customer**. With an index on `orders.customer_id` (an **index** is a sorted lookup structure that finds rows by a column's value without reading the whole table) each run is a quick index lookup, and for a page of 20 rows that is perfectly fine. Over a million customers it's a million lookups, where the join version reads `orders` once and aggregates everything with a single hash join (build a hash table from one input, then probe it with each row of the other).

	Correlated subquery	**Join + `GROUP BY` (or derived table)**
Readability	Very clear for "per row, look up X"	Clear once you know joins
Execution (PostgreSQL)	`EXISTS` / `IN` forms become semi/anti joins; scalar forms in `SELECT` / `WHERE` run per outer row	Set-based (whole tables at once): one pass, hash/merge join
Multiple values from the same lookup	One subquery **per value** (repeated work)	One join gives all columns
Risk	Slow on large outer tables without an index	Fan-out if you join several children (Chapter 5 §5.9)

Rule of thumb: `EXISTS` / `NOT EXISTS` correlated subqueries are excellent and idiomatic. For scalar lookups over **many** rows, prefer a join, a derived table, or `LATERAL` (§6.7).

Correlated subqueries in UPDATE and DELETE

Correlated subqueries are also the standard way to update a table from related data:

```sql
ALTER TABLE projects ADD COLUMN total_hours int;
UPDATE projects p
SET total_hours = (SELECT coalesce(sum(hours), 0) FROM employee_projects ep WHERE ep.project_id =
p.project_id);
SELECT project_id, name, total_hours FROM projects ORDER BY project_id;
```

Output:

```
 project_id |       name       | total_hours
------------+------------------+-------------
          1 | Checkout Revamp  |         870
          2 | Mobile App       |         670
          3 | Spring Campaign  |         370
          4 | Year-End Audit   |         240
          5 | Helpdesk Chatbot |          75
(5 rows)
```

(PostgreSQL also offers `UPDATE … FROM` for join-style updates — Chapter 9.)

Common mistakes

- Forgetting to correlate (`WHERE x.dept_id = dept_id` without the outer alias resolves to `x.dept_id = x.dept_id`, which is always true for non-NULL values).

- A correlated scalar subquery that can return two rows (error at run time).
- Using per-row subqueries for large reports without an index on the correlated column.

In the interview

> **30-second answer.** "A correlated subquery references the outer row, so logically it's evaluated once per outer row — like 'salary above *their department's* average'. The optimizer turns `EXISTS` / `IN` forms into semi-joins, but scalar ones often really run per row, so for large sets I'd rewrite with a join or window function."

- *Q: Rewrite 'above department average' without a correlated subquery.* — Join to a derived table of department averages, or use `avg(salary) OVER (PARTITION BY dept_id)` in a subquery and filter outside.
- *Q: Can a correlated subquery appear in `SELECT`?* — Yes, as a scalar subquery; it must return at most one row.

Self-check

1. Why are Asha and Rohan never returned by the "above department average" query? → Their `dept_id` is `NULL`, so the inner average is `NULL` and the comparison is UNKNOWN.
2. What does `SubPlan` in an `EXPLAIN` tell you? → A subquery evaluated (potentially) once per outer row.
3. Is `WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id)` correlated? → Yes (it references `c`).

TL;DR

Correlated = refers to the outer row, evaluated per outer row. Great with `EXISTS`; for big scalar lookups prefer joins, derived tables or window functions.

6.4 EXISTS and NOT EXISTS

What it is

`EXISTS (subquery)` is TRUE if the subquery returns **at least one row**, FALSE if it returns none. It never returns UNKNOWN, and it doesn't care what the subquery's `SELECT` list contains. `NOT EXISTS` is the opposite. They are almost always correlated.

```
SELECT d.name FROM departments d
WHERE EXISTS (SELECT 1 FROM projects p WHERE p.dept_id = d.dept_id AND p.end_date IS NULL)
ORDER BY d.dept_id;
```

Output:

```
    name
-------------
 Engineering
 Support
(2 rows)
```

"Departments that have at least one project still running." The database can stop searching as soon as it finds one matching project for a department.

Because only the existence of a row matters, `SELECT 1`, `SELECT *` or even `SELECT NULL` inside `EXISTS` all behave identically; `SELECT 1` is a common convention that signals "I don't need the columns".

```
SELECT count(*) FROM departments d
WHERE EXISTS (SELECT NULL FROM employees e WHERE e.dept_id = d.dept_id);
```

```
 count
-------
     5
(1 row)
```

NOT EXISTS — the anti-join

"Employees who are on no project":

```
SELECT e.first_name, e.job_title FROM employees e
WHERE NOT EXISTS (SELECT 1 FROM employee_projects ep WHERE ep.emp_id = e.emp_id)
ORDER BY e.emp_id;
```

Output:

```
 first_name |   job_title
------------+---------------
 Asha       | CEO
 Daniel     | Sales Manager
 Arjun      | Sales Rep
 Sara       | Sales Rep
(4 rows)
```

"For all" questions: double NOT EXISTS

SQL has no "for all" operator, but "X for **all** Y" is the same as "there is **no** Y for which X is missing". Which customers bought **every** Accessories product?

```
SELECT c.customer_id, c.first_name FROM customers c
WHERE NOT EXISTS (
    SELECT 1 FROM products p
    WHERE p.category = 'Accessories'
      AND NOT EXISTS (SELECT 1 FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
                      WHERE o.customer_id = c.customer_id AND oi.product_id = p.product_id));
```

Output:

```
 customer_id | first_name
-------------+------------
           3 | Chen
(1 row)
```

Read it as: "customers for whom there is no Accessories product that they have not bought". This pattern is called **relational division**. An often simpler alternative is counting: group each customer's distinct Accessories purchases and keep those whose count equals the number of Accessories products (`HAVING count(DISTINCT product_id) = (SELECT count(*) FROM products WHERE category = 'Accessories')`).

IN vs EXISTS vs NOT IN vs NOT EXISTS with a NULL

A block list that contains a `NULL` shows the difference clearly:

```sql
CREATE TABLE blocked (customer_id int);
INSERT INTO blocked VALUES (4), (NULL);
SELECT (SELECT count(*) FROM customers WHERE customer_id IN (SELECT customer_id FROM blocked))
AS in_rows,
       (SELECT count(*) FROM customers c WHERE EXISTS (SELECT 1 FROM blocked b WHERE b.customer_id =
c.customer_id)) AS exists_rows,
       (SELECT count(*) FROM customers WHERE customer_id NOT IN (SELECT customer_id FROM blocked))
AS not_in_rows,
       (SELECT count(*) FROM customers c WHERE NOT EXISTS (SELECT 1 FROM blocked b WHERE b.customer_id =
c.customer_id)) AS not_exists_rows;
```

Output:

```
 in_rows | exists_rows | not_in_rows | not_exists_rows
---------+-------------+-------------+-----------------
       1 |           1 |           0 |               9
(1 row)
```

`IN` and `EXISTS` agree (1 blocked customer). `NOT IN` returns 0 because of the `NULL`; `NOT EXISTS` correctly returns the 9 other customers.

	IN (subquery)	EXISTS (subquery)
Question asked	"is this value in the list?"	"is there any matching row?"
Correlated?	usually not	usually yes
Multiple columns	`(a, b) IN (SELECT x, y …)`	any condition in the subquery's `WHERE`
Negation with `NULL`s	`NOT IN` breaks	`NOT EXISTS` is safe
Performance in PostgreSQL	both become semi joins	both become semi joins

Old advice that "`EXISTS` is always faster than `IN`" came from older optimizers; modern optimizers (PostgreSQL included) usually turn both `IN (subquery)` and `EXISTS` into the same semi-join. Choose by clarity — but always prefer `NOT EXISTS` over `NOT IN`.

Self-check

1. What does `EXISTS` return if the subquery's single row is all `NULL`s? → TRUE — a row exists.
2. Rewrite `WHERE dept_id IN (SELECT dept_id FROM departments WHERE location = 'Pune')` with `EXISTS`. → `WHERE EXISTS (SELECT 1 FROM departments d WHERE d.dept_id = e.dept_id AND d.location = 'Pune')`.

TL;DR

`EXISTS` = "at least one row". Never UNKNOWN, ignores the select list, stops at the first match. `NOT EXISTS` is the safe anti-join; double `NOT EXISTS` expresses "for all".

6.5 ANY, SOME and ALL

What it is

`ANY` and `ALL` let you use any comparison operator (`=`, `<>`, `<`, `>`, `<=`, `>=`) against a list:

- `x > ANY (subquery)` — true if `x` is greater than **at least one** value (i.e. greater than the minimum). `SOME` is an exact synonym for `ANY`.
- `x > ALL (subquery)` — true if `x` is greater than **every** value (i.e. greater than the maximum).

```sql
SELECT first_name, salary FROM employees
WHERE salary > ANY (SELECT salary FROM employees WHERE dept_id = 2)
  AND dept_id <> 2
ORDER BY salary;

SELECT first_name, salary FROM employees
WHERE salary > ALL (SELECT salary FROM employees WHERE dept_id = 2)
ORDER BY salary;
```

Output:

```
 first_name |  salary
------------+-----------
 Omar       |  76000.00
 Rahul      |  98000.00
 Meera      |  98000.00
 Lucas      | 120000.00
 Kiran      | 135000.00
 Neha       | 142000.00
 Vikram     | 185000.00
(7 rows)

 first_name |  salary
------------+-----------
 Vikram     | 185000.00
 Asha       | 250000.00
(2 rows)
```

Sales salaries range from 68,000 to 150,000. `> ANY` means "above 68,000": Ananya (61,000) and Li (64,000) fail it, while Asha and Rohan drop out for another reason — `dept_id <> 2` is UNKNOWN for their `NULL` department. `> ALL` means "above 150,000" (Vikram and Asha).

Expression	Same as
`x = ANY (list)`	`x IN (list)`
`x <> ALL (list)`	`x NOT IN (list)` — with the same `NULL` trap
`x > ANY (list)`	`x > (SELECT min(…))` (if the list is non-empty and has no `NULL`s)
`x > ALL (list)`	`x > (SELECT max(…))` (same caveats)
`x <> ANY (list)`	"differs from at least one value" — rarely what people mean

```sql
SELECT count(*) AS eq_any FROM products WHERE category = ANY (SELECT DISTINCT category FROM products
WHERE unit_price > 300);
SELECT count(*) AS ne_all_with_null FROM products
WHERE unit_price <> ALL (SELECT unit_price FROM order_items UNION ALL SELECT NULL);
```

Output:

```
 eq_any
--------
      6
(1 row)

 ne_all_with_null
------------------
                0
(1 row)
```

Empty lists

Over an **empty** subquery, `ALL` is TRUE (vacuously: there is no value that fails) and `ANY` is FALSE (there is no value that passes). This differs from the `min`/`max` rewrite, where `max` of nothing is `NULL`:

```
SELECT (5 > ALL (SELECT salary FROM employees WHERE dept_id = 6)) AS all_empty,
       (5 > ANY (SELECT salary FROM employees WHERE dept_id = 6)) AS any_empty;
```

```
 all_empty | any_empty
-----------+-----------
 t         | f
(1 row)
```

PostgreSQL also accepts an **array** (an ordered list of values held in one value) on the right-hand side, which is how application code usually passes a list as one parameter (a value sent separately from the SQL text, filled into a placeholder):

```
SELECT name FROM products WHERE product_id = ANY (ARRAY[1, 3, 5]) ORDER BY product_id;
```

```
     name
---------------
 Laptop Pro 14
 USB-C Hub
 Standing Desk
(3 rows)
```

Database	`ANY` / `SOME` / `ALL` with subquery	`= ANY(array)`
PostgreSQL	yes	yes
MySQL, SQL Server, Oracle	yes	no arrays
SQLite	**no** — use `IN`, `EXISTS`, `min`/`max`	no

Self-check

1. `salary > ALL (SELECT salary FROM employees WHERE dept_id = 99)` — with no department 99, which rows pass? → All rows (ALL over an empty set is TRUE).
2. What is `= ANY` equivalent to? → `IN`.

TL;DR

`ANY`/`SOME` = compare with at least one; `ALL` = compare with every one. `= ANY` ≡ `IN`; `<> ALL` ≡ `NOT IN` (NULL trap). `ALL` of empty = TRUE.

6.6 Table subqueries: derived tables and row subqueries

What it is

A subquery in the `FROM` clause is a **derived table** (PostgreSQL's docs call it a *subquery in FROM*; some people say *inline view*). Its result is treated as a temporary, unnamed table that exists only for this query. You must give it an alias (optional from PostgreSQL 16; our server is 14):

```
SELECT * FROM (SELECT 1 AS x);
```

```
ERROR:  subquery in FROM must have an alias
LINE 1: SELECT * FROM (SELECT 1 AS x);
                      ^
HINT:  For example, FROM (SELECT ...) [AS] foo.
```

Why it matters

Derived tables let you **build a query in stages**: compute something, then filter, join or aggregate the result. They solve the "can't use an alias in `WHERE`" and "can't aggregate an aggregate" problems from Chapter 4.

```
SELECT dept_id, headcount
FROM (SELECT dept_id, count(*) AS headcount FROM employees GROUP BY dept_id) AS t
WHERE headcount >= 2
ORDER BY dept_id;
```

Output:

```
 dept_id | headcount
---------+-----------
       1 |         4
       2 |         4
       3 |         2
       4 |         2
  (null) |         2
(5 rows)
```

(The same filter could be written with `HAVING`; the derived-table form becomes necessary when the next step is something `HAVING` can't do.) An **aggregate of an aggregate** — "the average department payroll" — needs two levels, because `avg(sum(salary))` is not allowed:

```
SELECT round(avg(payroll)) AS avg_department_payroll
FROM (SELECT dept_id, sum(salary) AS payroll
      FROM employees WHERE dept_id IS NOT NULL GROUP BY dept_id) AS per_dept;
```

Output:

```
 avg_department_payroll
------------------------
                 267400
(1 row)
```

Derived tables can also be **joined** like any table — the standard cure for join fan-out (Chapter 5 §5.9). Each order's total next to that customer's average order total:

```
SELECT o.order_id, o.customer_id, round(t.order_total, 2) AS order_total, round(ca.avg_total, 2) AS
customer_avg
FROM orders o
JOIN (SELECT order_id, sum(quantity * unit_price * (1 - discount)) AS order_total
      FROM order_items GROUP BY order_id) t ON t.order_id = o.order_id
JOIN (SELECT o2.customer_id, avg(t2.total) AS avg_total
      FROM orders o2
      JOIN (SELECT order_id, sum(quantity * unit_price * (1 - discount)) AS total
            FROM order_items GROUP BY order_id) t2 ON t2.order_id = o2.order_id
      GROUP BY o2.customer_id) ca ON ca.customer_id = o.customer_id
WHERE o.customer_id IN (1, 5)
ORDER BY o.order_id;
```

Output:

```
 order_id | customer_id | order_total | customer_avg
----------+-------------+-------------+--------------
      101 |           1 |     1323.99 |      1151.36
      104 |           1 |      811.49 |      1151.36
      106 |           5 |      174.97 |       212.45
      112 |           5 |      249.93 |       212.45
      113 |           1 |     1318.60 |      1151.36
(5 rows)
```

Notice the order-total subquery written **twice**. That repetition is exactly what a CTE (`WITH order_totals AS (…)`) removes — Chapter 7. Since PostgreSQL 12 a simple CTE that is referenced once is inlined (pasted into the main query), i.e. executed just like the equivalent derived table; CTEs mainly win on readability and reuse.

Row subqueries

A **row subquery** returns a single row with several columns and is compared with a **row constructor** `(a, b)`. "Employees with the same department *and* job title as employee 4":

```
SELECT first_name, job_title, dept_id FROM employees
WHERE (dept_id, job_title) = (SELECT dept_id, job_title FROM employees WHERE emp_id = 4)
ORDER BY emp_id;
```

Output:

```
 first_name | job_title | dept_id
------------+-----------+---------
 Rahul      | Engineer  |       1
 Meera      | Engineer  |       1
(2 rows)
```

With `IN`, a multi-column subquery can return many rows. "Each customer's latest order" as "(customer, time) pairs that are some customer's maximum":

```
SELECT customer_id, order_id, placed_at::date
FROM orders
WHERE (customer_id, placed_at) IN (SELECT customer_id, max(placed_at) FROM orders GROUP BY customer_id)
ORDER BY customer_id;
```

Output:

```
 customer_id | order_id | placed_at
-------------+----------+------------
           1 |      113 | 2025-06-02
           2 |      108 | 2025-04-01
           3 |      111 | 2025-05-06
           4 |      105 | 2025-02-28
           5 |      112 | 2025-05-20
           6 |      114 | 2025-06-15
           7 |      110 | 2025-04-25
           8 |      109 | 2025-04-09
(8 rows)
```

(Row constructors with `IN` work in PostgreSQL, MySQL and Oracle, but **not** in SQL Server — use `EXISTS` there.)

Self-check

1. Why can't you write `SELECT avg(sum(salary)) FROM employees GROUP BY dept_id`? → Aggregates can't be nested; compute the sums in a derived table and average them outside.
2. What must every derived table have in PostgreSQL 14? → An alias.

TL;DR

Subquery in `FROM` = derived table: build queries in stages, aggregate aggregates, pre-aggregate before joins. Row subqueries compare several columns at once.

6.7 LATERAL subqueries

What it is

A normal derived table can't refer to other tables in the same `FROM` clause — it's evaluated independently. A `LATERAL` subquery **can** refer to columns of tables listed *before* it, so it is re-evaluated for each row of those tables. Think of it as a correlated subquery that may return **many rows and many columns**, placed in `FROM`.

```
SELECT d.name, t.n FROM departments d,
     (SELECT count(*) AS n FROM employees e WHERE e.dept_id = d.dept_id) t;
```

```
ERROR:  invalid reference to FROM-clause entry for table "d"
LINE 2: ... count(*) AS n FROM employees e WHERE e.dept_id = d.dept_id)...
                                                             ^
HINT:  There is an entry for table "d", but it cannot be referenced from this part of the query.
```

The problem it solves: top-N per group

"The two highest-paid employees in each department" is hard with plain subqueries (you can't `LIMIT` per group). With `LATERAL`, for each department run a small query that sorts that department's employees and takes 2:

```
SELECT d.name, t.first_name, t.salary
FROM departments d
CROSS JOIN LATERAL (
    SELECT e.first_name, e.salary FROM employees e
    WHERE e.dept_id = d.dept_id
    ORDER BY e.salary DESC, e.emp_id
    LIMIT 2) AS t
ORDER BY d.dept_id, t.salary DESC;
```

Output:

```
    name     | first_name |  salary
-------------+------------+----------
 Engineering | Vikram     | 185000.00
 Engineering | Neha       | 142000.00
 Sales       | Daniel     | 150000.00
 Sales       | Priya      |  72000.00
 Marketing   | Lucas      | 120000.00
 Marketing   | Ananya     |  61000.00
 Finance     | Kiran      | 135000.00
 Finance     | Omar       |  76000.00
 Support     | Li         |  64000.00
(9 rows)
```

`CROSS JOIN LATERAL` drops outer rows for which the subquery returns nothing (Research is missing). To keep them, use `LEFT JOIN LATERAL … ON true` — `ON true` because the correlation is already inside the subquery:

```sql
SELECT d.name, t.first_name, t.salary
FROM departments d
LEFT JOIN LATERAL (
    SELECT e.first_name, e.salary FROM employees e
    WHERE e.dept_id = d.dept_id
    ORDER BY e.salary DESC LIMIT 1) AS t ON true
ORDER BY d.dept_id;
```

Output:

```
    name     | first_name |  salary
-------------+------------+----------
 Engineering | Vikram     | 185000.00
 Sales       | Daniel     | 150000.00
 Marketing   | Lucas      | 120000.00
 Finance     | Kiran      | 135000.00
 Support     | Li         |  64000.00
 Research    | (null)     |    (null)
(6 rows)
```

"Each customer's most recent order, with several of its columns" — something a scalar subquery can only do one column at a time:

```sql
SELECT c.first_name, last.order_id, last.status, last.placed_at::date
FROM customers c
LEFT JOIN LATERAL (
    SELECT o.order_id, o.status, o.placed_at FROM orders o
    WHERE o.customer_id = c.customer_id
    ORDER BY o.placed_at DESC LIMIT 1) AS last ON true
ORDER BY c.customer_id;
```

Output:

```
 first_name | order_id |  status   | placed_at
------------+----------+-----------+------------
 Anita      |      113 | pending   | 2025-06-02
 Ben        |      108 | delivered | 2025-04-01
 Chen       |      111 | cancelled | 2025-05-06
 Divya      |      105 | cancelled | 2025-02-28
 Ethan      |      112 | shipped   | 2025-05-20
 Fatima     |      114 | pending   | 2025-06-15
 George     |      110 | shipped   | 2025-04-25
 Anita      |      109 | delivered | 2025-04-09
 Hana       |   (null) | (null)    | (null)
 Ivan       |   (null) | (null)    | (null)
(10 rows)
```

With an index on `orders (customer_id, placed_at)`, each lateral lookup reads just one index entry — often the fastest way to get "latest row per group" when there are few outer rows. Window functions (`ROW_NUMBER() OVER (PARTITION BY …)`, Chapter 8) are the other standard approach and are usually better when you need the answer for *every* group of a large table.

Database	Syntax
PostgreSQL 9.3+	`CROSS JOIN LATERAL (…)`, `LEFT JOIN LATERAL (…) ON true`
MySQL 8.0.14+	`LATERAL`
SQL Server, Oracle 12c+	`CROSS APPLY` / `OUTER APPLY` (Oracle also has `LATERAL`)
SQLite	not supported

TL;DR

`LATERAL` = a derived table that can see earlier `FROM` items, evaluated per outer row; can return many rows/columns. Top-N per group, latest row per group. `LEFT JOIN LATERAL … ON true` keeps outer rows with no result.

6.8 Set operations and UNION

What it is

Set operations combine the **results of two queries** into one result, by rows, like the operations on sets in mathematics (a *set* is a collection of distinct items; a *Venn picture* draws two sets as overlapping circles):

Operator	Result	Venn picture
`UNION`	rows in the first **or** the second result (duplicates removed)	both circles
`UNION ALL`	all rows of both results, duplicates kept	both circles, overlap counted twice
`INTERSECT`	rows in **both** results	only the overlap
`EXCEPT`	rows in the first result that are **not** in the second	left circle minus overlap

Joins combine tables **side by side** (more columns); set operations stack results **on top of each other** (more or fewer rows, same columns).

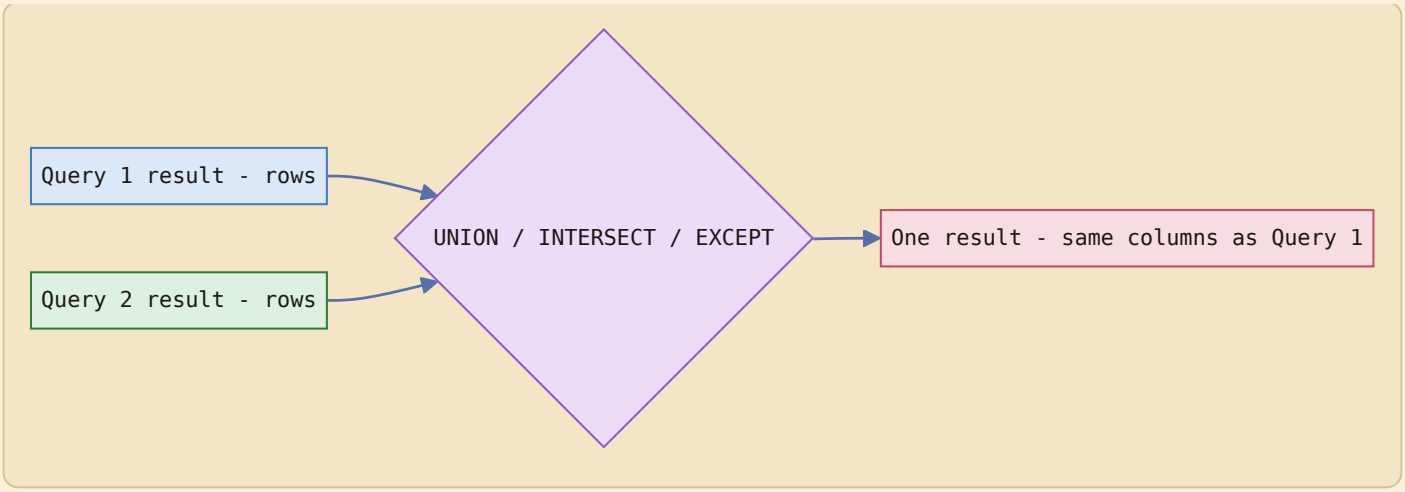


Figure 6.1 — A set operation takes two results with the same columns and produces one.

The rules

1. Both queries must return the **same number of columns**.
2. Corresponding columns must have **compatible types** (PostgreSQL resolves them to a common type, like `CASE` does).
3. Column **names** of the result come from the **first** query.
4. Rows are compared **as whole rows**, and — unlike `=` — two `NULL`s are treated as **equal** (not distinct).
5. `ORDER BY` / `LIMIT` at the end apply to the **combined** result (§6.11).

Breaking rule 1 or 2:

```
SELECT first_name, last_name FROM employees UNION SELECT first_name FROM customers;
SELECT emp_id FROM employees UNION SELECT first_name FROM customers;
```

```
ERROR:  each UNION query must have the same number of columns
LINE 1: ...first_name, last_name FROM employees UNION SELECT first_name...
                                                             ^

ERROR:  UNION types integer and text cannot be matched
LINE 1: SELECT emp_id FROM employees UNION SELECT first_name FROM cu...
                                                 ^
```

UNION

One contact list from two tables, with a literal column (a fixed value typed into the query, here `'employee'`) to show where each row came from:

```
SELECT first_name, last_name, 'employee' AS kind FROM employees WHERE dept_id = 3
UNION
SELECT first_name, last_name, 'customer' FROM customers WHERE country = 'UK'
ORDER BY kind, last_name;
```

Output:

```
 first_name | last_name |   kind
------------+-----------+----------
 Ben        | Carter    | customer
 George     | Hill      | customer
 Ananya     | Das       | employee
 Lucas      | Martin    | employee
(4 rows)
```

Every city where we have customers or offices — `UNION` removes the duplicates (Mumbai, Bengaluru, Delhi …):

```
SELECT city FROM customers WHERE city IS NOT NULL
UNION
SELECT location FROM departments WHERE location IS NOT NULL
ORDER BY city;
```

Output:

```
   city
-----------
 Bengaluru
 Delhi
 London
 Mumbai
 New York
 Pune
 mumbai
(7 rows)
```

Note that `mumbai` and `Mumbai` are different strings, so both survive — clean data first (`initcap(city)`, Chapter 3) if case shouldn't matter.

UNION vs UNION ALL

```
SELECT count(*) AS union_rows     FROM (SELECT country FROM customers UNION     SELECT 'India') t;
SELECT count(*) AS union_all_rows FROM (SELECT country FROM customers UNION ALL SELECT 'India') t;
```

Output:

```
 union_rows
------------
          3
(1 row)

 union_all_rows
----------------
             11
(1 row)
```

`UNION` returned 3 (India, UK, USA — duplicates inside **each** input are removed too, not just between them). `UNION ALL` returned all 10 + 1 = 11 rows.

	UNION	UNION ALL
Duplicates	removed (whole-row comparison)	kept
Extra work	a sort or hash over the whole result	none — rows are just appended
Use when	you need distinct rows	the inputs can't overlap, or duplicates are meaningful (counting, event logs)

Default to **`UNION ALL`** unless you actually need de-duplication; `UNION` on large results costs a sort or hash and hides data problems.

A typical `UNION ALL`: stacking two kinds of events into one timeline:

```
SELECT placed_at AS at, 'order placed'  AS event, order_id FROM orders   WHERE customer_id = 1
UNION ALL
SELECT shipped_at,      'order shipped',          order_id FROM orders   WHERE customer_id = 1 AND
shipped_at IS NOT NULL
ORDER BY at;
```

Output:

```
         at          |     event     | order_id
---------------------+---------------+----------
 2025-01-03 10:15:00 | order placed  |      101
 2025-01-04 09:00:00 | order shipped |      101
 2025-02-11 09:05:00 | order placed  |      104
 2025-02-12 16:45:00 | order shipped |      104
 2025-06-02 10:30:00 | order placed  |      113
(5 rows)
```

And rule 4 — set operations treat `NULL`s as duplicates of each other:

```
SELECT NULL::text AS v UNION SELECT NULL::text;
```

```
   v
--------
 (null)
(1 row)
```

Self-check

1. `SELECT 1 UNION SELECT 1 UNION ALL SELECT 1` returns how many rows? → 2 (operators apply left to right: `(1 UNION 1)` = 1 row, then `UNION ALL 1` = 2 rows).
2. Where do the result's column names come from? → The first query.
3. Why prefer `UNION ALL` by default? → No de-duplication cost and no silently lost rows.

TL;DR

Set operations stack results with the same column count and compatible types; names come from the first query. `UNION` de-duplicates (cost!), `UNION ALL` doesn't. `NULL`s count as equal.

6.9 INTERSECT

What it is

`INTERSECT` returns the rows that appear in **both** results (the overlap of the Venn diagram), without duplicates.

"Cities where we have both customers and an office":

```
SELECT city FROM customers
INTERSECT
SELECT location FROM departments
ORDER BY city;
```

Output:

```
   city
-----------
 Bengaluru
 Mumbai
 (null)
(3 rows)
```

The `(null)` row is a lesson in rule 4: one customer has no city and one department has no location, and set operations treat the two `NULL`s as equal. An inner join on `c.city = d.location` would not match them. Add `WHERE city IS NOT NULL` if that row is unwanted.

"Employees who are on a project **and** manage someone":

```
SELECT emp_id FROM employee_projects
INTERSECT
SELECT manager_id FROM employees
ORDER BY emp_id;
```

Output:

```
 emp_id
--------
      2
     10
     12
(3 rows)
```

`INTERSECT` is equivalent to a semi-join with `EXISTS` on all columns (using `IS NOT DISTINCT FROM` for `NULL`s) plus `DISTINCT`. Use `INTERSECT` when you compare whole rows of two similar results; use `EXISTS`/`JOIN` when you need other columns from either side.

6.10 EXCEPT

What it is

`EXCEPT` returns the rows of the first result that do **not** appear in the second (the left crescent), without duplicates. Oracle calls it `MINUS` (Oracle 21c also accepts `EXCEPT`).

"Products that were never ordered":

```
SELECT product_id FROM products
EXCEPT
SELECT product_id FROM order_items;
```

Output:

```
 product_id
------------
          8
(1 row)
```

Unlike `UNION` and `INTERSECT`, **order matters**: `A EXCEPT B` ≠ `B EXCEPT A`.

```
SELECT city FROM customers EXCEPT SELECT location FROM departments ORDER BY 1;
SELECT location FROM departments EXCEPT SELECT city FROM customers ORDER BY 1;
```

Output:

```
   city
----------
 Delhi
 London
 New York
 mumbai
(4 rows)

 location
----------
 Pune
(1 row)
```

Customer cities with no office; office cities with no customers (only Pune).

Because `NULL`s compare equal in set operations, `EXCEPT` is also a handy way to **diff two queries that contain `NULL`s** — for example, check that a refactored query (rewritten for clarity or speed, meant to behave identically) returns exactly the same rows as the old one: `(old EXCEPT new)` and `(new EXCEPT old)` should both be empty. Here Meera's row is removed even though her email is `NULL`:

```sql
SELECT first_name, email FROM employees WHERE dept_id = 1
EXCEPT
SELECT first_name, email FROM employees WHERE emp_id IN (2, 5)
ORDER BY 1;
```

Output:

```
 first_name |            email
------------+-----------------------------
 Neha       | Neha.Gupta@Company.example
 Rahul      | rahul.verma@company.example
(2 rows)
```

Without INTERSECT and EXCEPT

Some databases lacked these operators for a long time — MySQL only gained `INTERSECT` and `EXCEPT` in 8.0.31 (2022) — so you will often see them written with semi- and anti-joins instead (Chapter 5 §5.8):

```sql
-- INTERSECT-like: customer cities that also have an office
SELECT DISTINCT c.city FROM customers c
WHERE EXISTS (SELECT 1 FROM departments d WHERE d.location IS NOT DISTINCT FROM c.city)
ORDER BY c.city;

-- EXCEPT-like: customers whose city has no office (whole rows)
SELECT c.customer_id, c.first_name, c.city FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM departments d WHERE d.location = c.city)
ORDER BY c.customer_id;
```

Output:

```
   city
-----------
 Bengaluru
 Mumbai
 (null)
(3 rows)

 customer_id | first_name |   city
-------------+------------+----------
           2 | Ben        | London
           4 | Divya      | Delhi
           5 | Ethan      | New York
           6 | Fatima     | (null)
           7 | George     | London
           8 | Anita      | mumbai
           9 | Hana       | New York
          10 | Ivan       | Delhi
(8 rows)
```

(A `LEFT JOIN (SELECT DISTINCT location FROM departments) d ON d.location = c.city WHERE d.location IS NULL` returns the same 8 rows — see the code file.) Two differences from the real operators matter: you must add `DISTINCT` yourself to get set semantics (no duplicate rows), and `=` does **not** treat `NULL`s as equal. That is why the first query uses `IS NOT DISTINCT FROM` to reproduce `INTERSECT`'s `(null)` row, and why Fatima (no city) appears in the `NOT EXISTS` result although `EXCEPT` would have removed her `NULL` city. In return, the `EXISTS` forms can return any columns of the outer table, not just the compared ones.

The ALL variants: bag semantics

SQL tables are really **bags** (multisets): the same row can appear several times. The `ALL` forms keep track of **how many times** each row appears:

Operator	Row appearing *m* times in A and *n* times in B appears …
`UNION ALL`	*m* + *n* times
`UNION`	once
`INTERSECT ALL`	*min(m, n)* times
`INTERSECT`	once, if *m* > 0 and *n* > 0
`EXCEPT ALL`	*max(m − n, 0)* times
`EXCEPT`	once, if *m* > 0 and *n* = 0

```sql
CREATE TABLE a (x int);
CREATE TABLE b (x int);
INSERT INTO a VALUES (1), (1), (1), (2), (3);
INSERT INTO b VALUES (1), (1), (2), (2), (4);
SELECT x FROM a INTERSECT     SELECT x FROM b ORDER BY x;
SELECT x FROM a INTERSECT ALL SELECT x FROM b ORDER BY x;
SELECT x FROM a EXCEPT        SELECT x FROM b ORDER BY x;
SELECT x FROM a EXCEPT ALL    SELECT x FROM b ORDER BY x;
```

Output (in order):

```
 x
---
 1
 2
(2 rows)

 x
---
 1
 1
 2
(3 rows)

 x
---
 3
(1 row)

 x
---
 1
 3
(2 rows)
```

Check against the table: value 1 is 3 times in `a` and 2 times in `b` → `INTERSECT ALL` gives min(3, 2) = 2 copies, `EXCEPT ALL` gives 3 − 2 = 1 copy. Value 2 occurs once in `a` and twice in `b` → 1 copy in `INTERSECT ALL`, 0 in `EXCEPT ALL`. Value 3 is only in `a` → only in the `EXCEPT` results. For completeness, `UNION` gives `1, 2, 3, 4` and `UNION ALL` gives all 10 values.

Dialect support

Operator	PostgreSQL	MySQL	SQLite	SQL Server	Oracle
`UNION`, `UNION ALL`	yes	yes	yes	yes	yes
`INTERSECT`, `EXCEPT`	yes	8.0.31+	yes	yes	`INTERSECT`, `MINUS` (+ `EXCEPT` in 21c)
`INTERSECT ALL`, `EXCEPT ALL`	yes	8.0.31+	no	no	21c+

In the interview

30-second answer. "`UNION` combines two results and removes duplicates; `UNION ALL` keeps them and is cheaper. `INTERSECT` keeps rows in both; `EXCEPT` keeps rows in the first but not the second. Both inputs need the same number of columns with compatible types, and set operators treat `NULL`s as equal."

- *Q: `UNION` vs `UNION ALL`?* — De-duplication vs none; use `UNION ALL` unless you need distinct rows.
- *Q: Find IDs present in table A but not in B.* — `SELECT id FROM a EXCEPT SELECT id FROM b`, or `NOT EXISTS`. `EXCEPT` compares only the selected columns and returns distinct rows; `NOT EXISTS` returns full rows of `a` (with duplicates if `a` has them).
- *Q: How do you check two queries return identical results?* — `(q1 EXCEPT ALL q2) UNION ALL (q2 EXCEPT ALL q1)` must be empty.

Self-check

1. `A = {1,1,2}`, `B = {1}`. What are `A EXCEPT B` and `A EXCEPT ALL B`? → `{2}` and `{1, 2}`.

2. Which set operator is not commutative (gives a different answer when its two sides are swapped)? → `EXCEPT`.
3. Why did `INTERSECT` return a `(null)` city? → Set operations treat two `NULL`s as the same value.

> **TL;DR**
>
> `INTERSECT` = in both; `EXCEPT` = in the first only (order matters). `ALL` variants count duplicates (min, difference). Great for diffing results.

6.11 Precedence, sorting, and choosing the right tool

Precedence

`INTERSECT` binds **tighter** than `UNION` and `EXCEPT` (like multiplication before addition); `UNION` and `EXCEPT` are evaluated left to right. Parentheses override:

```
SELECT 1 AS v UNION SELECT 2 INTERSECT SELECT 3;          -- = 1 UNION (2 INTERSECT 3)
(SELECT 1 AS v UNION SELECT 2) INTERSECT SELECT 3;        -- parentheses change it
```

Output:

```
 v
---
 1
(1 row)

 v
---
(0 rows)
```

When mixing operators, **always use parentheses** — readers shouldn't need to remember precedence.

ORDER BY and LIMIT

A single `ORDER BY`/`LIMIT` at the end applies to the **whole combined result**:

```
SELECT first_name, salary FROM employees WHERE dept_id = 1
UNION ALL
SELECT first_name, salary FROM employees WHERE dept_id = 2
ORDER BY salary DESC
LIMIT 3;
```

Output:

```
 first_name |  salary
------------+-----------
 Vikram     | 185000.00
 Daniel     | 150000.00
 Neha       | 142000.00
(3 rows)
```

To sort or limit **each branch**, wrap the branches in parentheses:

```
(SELECT first_name, salary FROM employees WHERE dept_id = 1 ORDER BY salary DESC LIMIT 1)
UNION ALL
(SELECT first_name, salary FROM employees WHERE dept_id = 2 ORDER BY salary DESC LIMIT 1);
```

Output:

```
 first_name |  salary
------------+-----------
 Vikram     | 185000.00
 Daniel     | 150000.00
(2 rows)
```

The final `ORDER BY` may only use **output column names or positions** — not expressions. Move the set operation into a derived table to sort by an expression:

```
SELECT first_name FROM employees UNION SELECT first_name FROM customers ORDER BY upper(first_name);
```

```
ERROR:  invalid UNION/INTERSECT/EXCEPT ORDER BY clause
LINE 1: ...s UNION SELECT first_name FROM customers ORDER BY upper(firs...
                                                             ^
DETAIL:  Only result column names can be used, not expressions or functions.
HINT:  Add the expression/function to every SELECT, or move the UNION into a FROM clause.
```

```
SELECT * FROM (SELECT first_name FROM employees UNION SELECT first_name FROM customers) t
ORDER BY upper(first_name) LIMIT 3;
```

```
 first_name
------------
 Ananya
 Anita
 Arjun
(3 rows)
```

(Without any `ORDER BY`, the row order of a set operation — like any query — is not guaranteed.)

EXCEPT vs NOT EXISTS

```
SELECT customer_id FROM customers EXCEPT SELECT customer_id FROM orders ORDER BY 1;

SELECT c.customer_id, c.first_name, c.city FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id)
ORDER BY 1;
```

Output:

```
 customer_id
-------------
           9
          10
(2 rows)

 customer_id | first_name |   city
-------------+------------+----------
           9 | Hana       | New York
          10 | Ivan       | Delhi
(2 rows)
```

Both find the customers who never ordered, but `EXCEPT` can only return the compared columns, while `NOT EXISTS` returns any columns of the outer table.

Choosing between joins, subqueries and set operations

You want…	Best tool
Columns from several related tables in one row	`JOIN` (Chapter 5)
Rows of A that have / don't have a match in B	`EXISTS` / `NOT EXISTS` (semi / anti-join)
Compare with one computed value (average, max)	scalar subquery
Compare with a list	`IN` (never `NOT IN` on nullable columns)
Build in stages; aggregate an aggregate; pre-aggregate before a join	derived table or CTE (Chapter 7)
Top-N / latest row per group	`LATERAL` or a window function (Chapter 8)
Stack similar results from different sources	`UNION ALL` (`UNION` if de-duplication is needed)
Rows common to / missing from two similar results, whole-row comparison, `NULL`-safe	`INTERSECT` / `EXCEPT`

In the interview

30-second answer (whole chapter). "A subquery is a query nested in another; it can return a scalar, a row, a column or a table. Non-correlated subqueries run once; correlated ones depend on the outer row. I use `EXISTS` / `NOT EXISTS` for semi- and anti-joins and avoid `NOT IN` because of `NULL`s. Derived tables and `LATERAL` let me build queries in stages. Set operators combine whole results: `UNION ALL` by default, `UNION` to de-duplicate, `INTERSECT` for common rows, `EXCEPT` for differences — same column count, compatible types, `NULL`s treated as equal."

- *Q: When is a subquery better than a join?* — When you only test existence (`EXISTS`), compare with an aggregate, or need to aggregate before joining. A join is better when you need columns from both sides.
- *Q: Can you `ORDER BY` inside a `UNION` branch?* — Only inside parentheses and normally with `LIMIT`; otherwise the final `ORDER BY` sorts the whole result.

Self-check

1. What does `SELECT 1 UNION SELECT 2 INTERSECT SELECT 2` return? → `{1, 2}` — `INTERSECT` first gives `{2}`, then `UNION` with `{1}`.
2. How do you sort a `UNION` result by `lower(name)`? → Wrap it in a derived table, or add `lower(name)` as an output column of every branch.

TL;DR

`INTERSECT` before `UNION`/`EXCEPT`; use parentheses. Final `ORDER BY`/`LIMIT` apply to the whole result and accept only output columns. Pick joins for columns, `EXISTS` for existence, set operators for whole-result comparisons.

Chapter 6 TL;DR

Tool	Returns / does	Watch out for
Scalar subquery	one value	0 rows → `NULL`; 2+ rows → run-time error
`IN (subquery)`	membership (semi-join)	—
`NOT IN (subquery)`	non-membership	any `NULL` in the list → no rows
Correlated subquery	per-outer-row evaluation	slow for big scalar lookups; qualify columns
`EXISTS` / `NOT EXISTS`	"is there a row?"	the safe semi/anti-join
`ANY` / `ALL`	compare with at least one / every value	`ALL` over empty = TRUE; `<> ALL` = `NOT IN`
Derived table	subquery in `FROM`	needs an alias (before PostgreSQL 16)
`LATERAL`	per-row subquery in `FROM`	`LEFT JOIN LATERAL … ON true` to keep all rows
`UNION` / `UNION ALL`	stack results	`UNION` costs a de-duplication
`INTERSECT` / `EXCEPT`	common / missing rows	`EXCEPT` isn't commutative; `NULL`s compare equal

Next: Chapter 7 — CTEs, Views and Recursion gives names to subqueries so that long queries read top to bottom.

Chapter 7 — CTEs, Views and Recursion: Naming Queries and Walking Trees

Real queries grow. A report that needs "order totals, then customer totals, then only the customers above average" quickly turns into three levels of nested subqueries that nobody can read. This chapter gives you the tools SQL has for **naming** a piece of a query so you can build big queries out of small, readable steps:

- a **common table expression (CTE)** names a query for the length of *one* statement;
- a **recursive CTE** calls itself, which lets plain SQL walk hierarchies (org charts, category trees, bills of materials — parts lists where each part can itself be made of parts) and graphs (networks of connected items: flight routes, friend networks) and produce sequences;
- a **view** names a query *permanently* in the database, so everyone can reuse it like a table;
- a **materialized view** also stores the query's *result*, trading freshness for speed.

It also covers `generate_series`, PostgreSQL's quick way to produce rows of numbers or dates, which you need for "fill in the missing days" reports.

What you'll learn

#	Topic	You will be able to…
7.1	CTE basics	write `WITH` queries, chain several CTEs, know when PostgreSQL inlines them
7.2	Data-modifying CTEs	archive/move rows in one atomic (all-or-nothing) statement with `RETURNING`
7.3	Recursive CTE mechanics	explain anchor, recursive term, working table and termination
7.4	Hierarchies	list a tree top-down with depth and path, walk up to the root, total a subtree
7.5	Graphs and cycles	traverse a graph without looping forever; use `CYCLE` and `SEARCH` (PostgreSQL 14)
7.6	`generate_series`	make number/date rows, fill gaps in time series, create test data
7.7	Views	create, replace, update through, and protect views (`WITH CHECK OPTION`)
7.8	Materialized views	store results, refresh them (also `CONCURRENTLY`), index them
7.9	Choosing	pick between subquery, CTE, view, materialized view and temporary table

Setup. Examples use the `company` sample database from chapter 1, section 1.11 plus one extra table, `routes` (a small flight network, see 7.5). The files are in `code/`; `code/00_schema.sql` builds everything. Subqueries and derived tables, which CTEs replace, are explained in chapter 6.

7.1 Common table expressions (WITH)

What it is

A **common table expression** is a named, temporary result set (the rows a query returns) that you define at the start of a statement with `WITH name AS (query)` and then use in the rest of that statement as if it were a table. It disappears when the statement ends.

Analogy: when you solve a maths problem you write intermediate results on scrap paper ("let x = total per customer") and then use x in the final answer. A CTE is that scrap paper, with a name.

```
WITH cte_name [(col1, col2, ...)] AS (
    SELECT ...                -- any query
)
[, another_cte AS (SELECT ... FROM cte_name ...)]   -- later CTEs may use earlier ones
SELECT ... FROM cte_name ...;  -- the main statement
```

The problem it solves

Without CTEs you nest derived tables (subqueries in `FROM`). Nesting reads *inside-out*: the first thing that runs is buried deepest. CTEs let you write the same logic *top-down*, in the order you think about it, and give every step a meaningful name. They also let you use one intermediate result **several times** without copying it.

How it works

1. Each `name AS (query)` defines one CTE. Separate several with commas; write `WITH` only once.
2. A CTE can reference any CTE defined **before** it (and, with `RECURSIVE`, itself — see 7.3).
3. The final statement (a `SELECT`, `INSERT`, `UPDATE` or `DELETE`) uses the CTEs by name.
4. The CTE exists only while this one statement runs. Nothing is stored in the catalog (the database's internal list of its tables, views and other objects).

One CTE — employees paid above their department average:

```
-- code/01_cte_basics.sql
WITH dept_avg AS (
    SELECT dept_id, round(avg(salary), 0) AS avg_salary
    FROM employees
    WHERE dept_id IS NOT NULL
    GROUP BY dept_id
)
SELECT e.first_name, e.dept_id, e.salary, d.avg_salary
FROM employees e
JOIN dept_avg d ON d.dept_id = e.dept_id
WHERE e.salary > d.avg_salary
ORDER BY e.dept_id, e.salary DESC;
```

Output:

```
 first_name | dept_id |  salary   | avg_salary
------------+---------+-----------+------------
 Vikram     |       1 | 185000.00 |     130750
 Neha       |       1 | 142000.00 |     130750
 Daniel     |       2 | 150000.00 |      89500
 Lucas      |       3 | 120000.00 |      90500
 Kiran      |       4 | 135000.00 |     105500
(5 rows)
```

`dept_avg` behaves like a four-row table (one row per department that has staff). The main query joins to it like any other table.

Several CTEs as a pipeline. Each step builds on the previous one — order value → customer lifetime value → the overall average — and the final `SELECT` reads almost like English:

```sql
WITH order_totals AS (          -- step 1: value of each non-cancelled order
    SELECT o.order_id, o.customer_id,
           sum(oi.quantity * oi.unit_price * (1 - oi.discount)) AS order_value
    FROM orders o
    JOIN order_items oi USING (order_id)
    WHERE o.status <> 'cancelled'
    GROUP BY o.order_id, o.customer_id
),
customer_totals AS (            -- step 2: lifetime value per customer
    SELECT customer_id, count(*) AS orders, round(sum(order_value), 2) AS lifetime_value
    FROM order_totals
    GROUP BY customer_id
),
overall AS (                    -- step 3: one number to compare against
    SELECT round(avg(lifetime_value), 2) AS avg_ltv FROM customer_totals
)
SELECT c.first_name, c.last_name, ct.orders, ct.lifetime_value, o.avg_ltv
FROM customer_totals ct
JOIN customers c USING (customer_id)
CROSS JOIN overall o
WHERE ct.lifetime_value > o.avg_ltv
ORDER BY ct.lifetime_value DESC;
```

Output:

```
 first_name | last_name | orders | lifetime_value | avg_ltv
------------+-----------+--------+----------------+---------
 Anita      | Shah      |      3 |        3454.08 | 1057.66
 Ben        | Carter    |      2 |        1858.36 | 1057.66
(2 rows)
```

`overall` is a one-row CTE, so `CROSS JOIN` simply attaches the average to every row. Written with nested subqueries this would be three levels deep.

Using a CTE twice. Here `monthly` is read by the main query *and* by a scalar subquery:

```sql
WITH monthly AS (
    SELECT date_trunc('month', placed_at)::date AS month, count(*) AS orders
    FROM orders
    GROUP BY 1
)
SELECT month, orders,
       orders = (SELECT max(orders) FROM monthly) AS is_best_month
FROM monthly
ORDER BY month;
```

Output:

```
   month    | orders | is_best_month
------------+--------+---------------
 2025-01-01 |      3 | t
 2025-02-01 |      2 | f
 ...
 2025-04-01 |      3 | t
 ...
(6 rows)
```

Naming the columns in the header. `WITH dept_size(dept, headcount) AS (...)` renames the CTE's output columns, handy when the inner query has unnamed expressions such as `count(e.emp_id)`:

```
WITH dept_size(dept, headcount) AS (
    SELECT d.name, count(e.emp_id)
    FROM departments d LEFT JOIN employees e USING (dept_id)
    GROUP BY d.name
)
SELECT * FROM dept_size WHERE headcount >= 2 ORDER BY headcount DESC, dept;
```

Output:

```
    dept     | headcount
-------------+-----------
 Engineering |         4
 Sales       |         4
 Finance     |         2
 Marketing   |         2
(4 rows)
```

How PostgreSQL executes a CTE: inlined or materialized

A CTE can be run in two ways:

- **Inlined** — the optimizer (the database component that chooses how to run a query) pastes the CTE's query into the main query, exactly like a subquery, and optimizes the whole thing together (for example pushing a `WHERE` filter down into it so an index — a sorted lookup structure that finds rows without reading the whole table — can be used).
- **Materialized** — the CTE runs once, its rows are stored in a temporary work area, and the main query scans that stored copy. Filters from outside cannot reach inside.

Since **PostgreSQL 12**, a non-recursive, side-effect-free (it only reads; it changes no data) CTE that is referenced **once** is inlined automatically; one referenced two or more times is materialized. You can override the choice with `AS MATERIALIZED` or `AS NOT MATERIALIZED`. (Before version 12 every CTE was materialized — an "optimization fence" that old blog posts still warn about.)

```
EXPLAIN (COSTS OFF)
WITH big AS (SELECT * FROM employees)
SELECT first_name FROM big WHERE emp_id = 3;

EXPLAIN (COSTS OFF)
WITH big AS MATERIALIZED (SELECT * FROM employees)
SELECT first_name FROM big WHERE emp_id = 3;
```

Output:

```
                  QUERY PLAN
-----------------------------------------------
 Index Scan using employees_pkey on employees
   Index Cond: (emp_id = 3)
(2 rows)

          QUERY PLAN
------------------------------
 CTE Scan on big
   Filter: (emp_id = 3)
   CTE big
     ->  Seq Scan on employees
(4 rows)
```

The first plan never mentions the CTE: the filter reached the primary-key index. The second copies the whole table into the CTE, then filters. (`EXPLAIN` prints the plan — the steps the database chose; `Seq`

`Scan` means reading the whole table. It is covered properly in chapter 12.)

`MATERIALIZED` is useful when the CTE is expensive and referenced several times, or when you deliberately want to compute something once (for example a call to a slow function). `NOT MATERIALIZED` is useful when a multiply-referenced CTE is cheap and each use filters it differently.

Dialect notes

Feature	PostgreSQL	MySQL	SQLite	SQL Server
`WITH` (non-recursive)	yes	8.0+	3.8.3+	2005+
`WITH RECURSIVE` keyword	required for recursion	required	required	*not used* — plain `WITH` can recurse
`MATERIALIZED` hint	12+	no	3.35+	no
CTE with `INSERT/UPDATE/DELETE` inside	yes (7.2)	no	no	no (but a CTE can be the *target* of an `UPDATE`/`DELETE`)

In SQL Server the statement *before* a `WITH` must end with a semicolon, which is why you often see `;WITH` in T-SQL (SQL Server's SQL dialect) code (SQL Server — not run).

Trade-offs

	Derived table (subquery in `FROM`)	CTE
Readability	inside-out	top-down, named steps
Reuse in the same statement	copy-paste	reference by name
Recursion	no	yes
Performance (PostgreSQL 12+)	same when inlined	same when inlined; can force materialization
Scope	the one place it's written	the whole statement

When not to use it

- When the query is simple enough to read without it — one level of subquery is fine.
- When many different statements need the same logic: that is a **view** (7.7).
- When you need the intermediate result across several statements (for example to index it or look at it twice in a session — one connection's lifetime): use a **temporary table** (a real table that only your session can see and that is dropped automatically when it ends; 7.9).

Common mistakes

- **Writing `WITH` twice** (`WITH a AS (...) WITH b AS (...)`). Use one `WITH` and commas.
- **Referencing a CTE defined later** in the list. Order matters: a CTE sees only those above it (unless it is recursive and refers to itself).
- **Assuming a CTE "caches" work.** In PostgreSQL 12+ a CTE used once is inlined, so it is re-optimized, not cached. Use `MATERIALIZED` if you truly want one evaluation.
- **A missing comma between CTEs**, or an extra one after the last CTE.

Real-world examples

Analytics teams write reports as a chain of CTEs (`raw → cleaned → joined → aggregated → final`); dbt (a popular tool for building data-transformation pipelines out of SQL files) encourages exactly this style.

In the interview

> **30-second answer:** "A CTE is a named subquery declared with `WITH` that exists for one statement. It makes complex queries readable as a sequence of steps, lets you reference the same intermediate

result several times, and — with `WITH RECURSIVE` — lets a query refer to itself to walk hierarchies. In PostgreSQL 12 and later a CTE referenced once is inlined, so it costs the same as a subquery; you can force materialization with `AS MATERIALIZED`."

- *Is a CTE faster than a subquery?* → Usually identical, because modern optimizers inline it. It can be slower if it is materialized and a selective outer filter can't be pushed into it; it can be faster if an expensive CTE is materialized once and reused.
- *CTE vs temporary table?* → A CTE lasts one statement and isn't indexed; a temp table lasts the session (or transaction), can be indexed and analyzed (have statistics gathered for the optimizer), and can be reused by many statements.
- *Can you update through a CTE?* → In PostgreSQL the CTE itself can contain `UPDATE/DELETE/INSERT ... RETURNING` (next section).

Self-check

1. *How long does a CTE exist?* → Only for the single statement it belongs to.
2. *Can CTE `b` use CTE `a` if `a` is written after `b`?* → No; a CTE can only refer to CTEs defined before it (or itself when recursive).
3. *In PostgreSQL 14, is a CTE referenced once materialized?* → No, it is inlined unless you write `AS MATERIALIZED` (or it is recursive or has side effects).

TL;DR

`WITH name AS (query)` names a step for one statement. Chain steps with commas, reference them as tables, reuse them freely. PostgreSQL 12+ inlines single-use CTEs; `MATERIALIZED` / `NOT MATERIALIZED` override.

7.2 Data-modifying CTEs (PostgreSQL)

What it is

In PostgreSQL a CTE may contain an `INSERT`, `UPDATE` or `DELETE` with a `RETURNING` clause. The rows it returns can feed the rest of the statement. This lets you do several changes as **one statement** — for example "delete these rows and copy them into an archive" — which is automatically atomic: either all of it happens or none of it.

How it works

1. `DELETE ... RETURNING *` (or `UPDATE`/`INSERT ... RETURNING`) inside a CTE produces the affected rows.
2. Later CTEs or the main statement read those rows by the CTE's name.
3. Every data-modifying CTE runs **to completion exactly once**, even if nothing reads its output.
4. All parts of the statement see the **same snapshot** (a frozen picture of the data at one moment) — the data as it was *before* the statement started. They do not see each other's changes, except through `RETURNING`.

Example — archive cancelled orders and their items:

```sql
-- code/02_data_modifying_cte.sql
CREATE TABLE orders_archive (LIKE orders INCLUDING ALL);
CREATE TABLE order_items_archive (LIKE order_items INCLUDING ALL);

WITH gone_items AS (
    DELETE FROM order_items
    WHERE order_id IN (SELECT order_id FROM orders WHERE status = 'cancelled')
    RETURNING *
),
saved_items AS (
    INSERT INTO order_items_archive SELECT * FROM gone_items RETURNING order_id
),
gone_orders AS (
    DELETE FROM orders
    WHERE status = 'cancelled'
    RETURNING *
)
INSERT INTO orders_archive
SELECT * FROM gone_orders
RETURNING order_id, status;
```

Output:

```
 order_id |  status
----------+-----------
      105 | cancelled
      111 | cancelled
(2 rows)
```

```sql
SELECT (SELECT count(*) FROM orders)              AS live_orders,
       (SELECT count(*) FROM orders_archive)      AS archived_orders,
       (SELECT count(*) FROM order_items_archive) AS archived_items;
```

Output:

```
 live_orders | archived_orders | archived_items
-------------+-----------------+----------------
          12 |               2 |              2
(1 row)
```

`CREATE TABLE ... (LIKE t INCLUDING ALL)` copies the column definitions, defaults and constraints of `t` (see chapter 9).

The snapshot rule, shown:

```sql
WITH bump AS (
    UPDATE products SET unit_price = unit_price + 1 WHERE product_id = 2
    RETURNING unit_price
)
SELECT (SELECT unit_price FROM products WHERE product_id = 2) AS seen_by_select,
       (SELECT unit_price FROM bump)                          AS new_value;
```

Output:

```
 seen_by_select | new_value
----------------+-----------
          24.99 |     25.99
(1 row)
```

The outer `SELECT` still reads the old price; only `RETURNING` shows the new one. After the statement finishes, the table holds 25.99.

Common mistakes

- **Modifying the same row twice in one statement** (two CTEs updating the same row). The result is unpredictable; don't do it.
- **Expecting a later CTE to "see" an earlier change by querying the table.** It won't — read from the earlier CTE's `RETURNING` instead.
- **Porting to MySQL/SQL Server.** They don't support this; there you use a transaction (a group of statements that are committed or undone together) with separate statements (chapter 13).

TL;DR

PostgreSQL CTEs can `INSERT/UPDATE/DELETE ... RETURNING`; the statement is atomic and every part sees the pre-statement snapshot. Great for "move rows" and "update and log" in one go.

7.3 Recursive CTEs: the mechanics

What it is

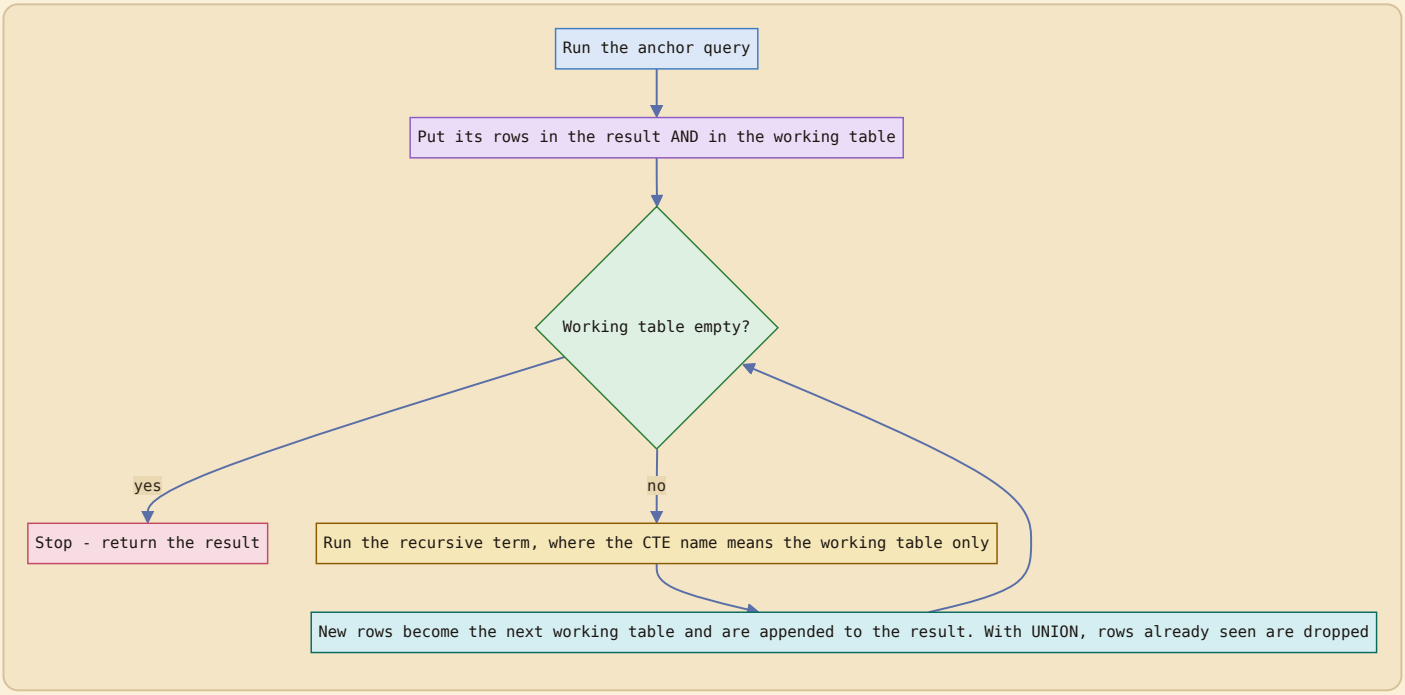
A **recursive CTE** is a CTE that refers to itself. It is written `WITH RECURSIVE name AS (anchor UNION ALL recursive_term)`. It is how standard SQL expresses *repetition*: "start with these rows, then keep deriving new rows from the last batch until no new rows appear."

Analogy: finding everyone in a family tree descended from one grandparent. You start with the grandparent (the anchor), then find their children, then the children's children, and stop when a generation has no children.

The problem it solves

Some questions have no fixed depth: "all reports under this manager, at every level", "every category inside Electronics", "every city reachable from Mumbai". A join handles exactly one level; you would need an unknown number of self-joins (chapter 5 shows the one-level self-join). A recursive CTE handles any depth.

How it works



1. **Anchor** (the non-recursive part, before `UNION [ALL]`): runs once and produces the starting rows.
2. **Recursive term** (after `UNION [ALL]`): references the CTE name. On each pass, that name means *only the rows produced by the previous pass* (the "working table"), not the whole result so far.
3. Rows produced by each pass are added to the result and become the input of the next pass.
4. **Termination:** the process stops when a pass produces no rows. Your `WHERE` or join condition must eventually make that happen.
5. `UNION ALL` keeps every row; `UNION` discards rows that are duplicates of rows already produced — which also stops loops whose state repeats.

Rules for the recursive term in PostgreSQL: it may reference the CTE only **once**, and not inside an aggregate function, a subquery, or on the nullable side of an outer join. The anchor and recursive term must produce the same number of columns with compatible types — cast in the anchor (`0::bigint`, `name::text`) if the recursive term would produce a wider type.

Counting 1 to 5:

```sql
-- code/03_recursive_series.sql
WITH RECURSIVE counter(n) AS (
    SELECT 1                -- anchor: the starting row(s)
    UNION ALL
    SELECT n + 1            -- recursive term: runs on the rows produced last time
    FROM counter
    WHERE n < 5             -- stop condition
)
SELECT n FROM counter;
```

Output:

```
 n
---
 1
 2
 3
 4
 5
(5 rows)
```

Trace: anchor → {1}. Pass 1 reads {1} → {2}. Pass 2 reads {2} → {3} … pass 4 reads {4} → {5}. Pass 5 reads {5}; `5 < 5` is false → no rows → stop.

Carrying state in several columns. Each row can hold more than a counter. Here each pass computes the next Fibonacci pair (each Fibonacci number is the sum of the two before it) and a running factorial ($n! = 1 \times 2 \times \ldots \times n$):

```sql
WITH RECURSIVE fib(i, a, b, fact) AS (
    SELECT 1, 0::bigint, 1::bigint, 1::numeric
    UNION ALL
    SELECT i + 1, b, a + b, fact * (i + 1)
    FROM fib
    WHERE i < 10
)
SELECT i, a AS fibonacci, fact AS factorial FROM fib;
```

Output:

```
 i  | fibonacci | factorial
----+-----------+-----------
  1 |         0 |         1
  2 |         1 |         2
  3 |         1 |         6
  4 |         2 |        24
 ...
 10 |        34 |   3628800
(10 rows)
```

Walking dates. The same pattern steps through time (here one week at a time):

```
WITH RECURSIVE weeks(week_start) AS (
    SELECT DATE '2025-01-06'
    UNION ALL
    SELECT (week_start + 7)::date FROM weeks WHERE week_start < DATE '2025-02-03'
)
SELECT week_start, to_char(week_start, 'Dy') AS weekday FROM weeks;
```

Output:

```
 week_start | weekday
------------+---------
 2025-01-06 | Mon
 2025-01-13 | Mon
 2025-01-20 | Mon
 2025-01-27 | Mon
 2025-02-03 | Mon
(5 rows)
```

In PostgreSQL `generate_series` (7.6) is simpler for this; the recursive form is what you use in MySQL, which has no `generate_series`.

A runaway query and two safety nets. A recursive CTE without a stop condition never ends. While experimenting, a `LIMIT` on the outer query stops it (PostgreSQL evaluates the recursion lazily, only as far as the outer query asks):

```
WITH RECURSIVE forever(n) AS (
    SELECT 1 UNION ALL SELECT n + 1 FROM forever      -- no stop condition!
)
SELECT n FROM forever LIMIT 3;
```

Output:

```
 n
---
 1
 2
 3
(3 rows)
```

And `UNION` instead of `UNION ALL` stops a computation whose state starts repeating. Doubling modulo 7 (keeping only the remainder after dividing by 7) goes 1 → 2 → 4 → 1 …; the second 1 is a duplicate, so it is discarded and the working table becomes empty:

```sql
WITH RECURSIVE mod_cycle(n) AS (
    SELECT 1
    UNION
    SELECT (n * 2) % 7 FROM mod_cycle
)
SELECT n FROM mod_cycle;
```

Output:

```
 n
---
 1
 2
 4
(3 rows)
```

`UNION` only helps when the *whole row* repeats. If a row carries a growing column (a depth, a path, a running total) it never repeats exactly — you need explicit cycle detection (7.5).

Dialect notes

	PostgreSQL	MySQL 8	SQLite	SQL Server
Syntax	`WITH RECURSIVE`	`WITH RECURSIVE`	`WITH RECURSIVE`	`WITH` (no keyword)
Built-in depth limit	none	`cte_max_recursion_depth` = 1000 by default	none	100 by default; `OPTION (MAXRECURSION n)`, 0 = unlimited
`UNION` (distinct) in recursion	yes	yes	yes	no, `UNION ALL` only
`SEARCH` / `CYCLE` clauses	14+	no	no	no

Oracle also supports recursive `WITH` (since 11g R2) and its older proprietary `CONNECT BY` syntax.

Common mistakes

- **No stop condition** → the query runs until it runs out of memory or disk (or, in MySQL/SQL Server, hits the depth limit with an error).
- **Type mismatch between anchor and recursive term.** The anchor fixes each column's type. A `varchar(40)` column in the anchor followed by `path || ' > ' || name` (which yields `text`) is rejected with *"column has type character varying(40) in non-recursive term but type text overall"*; and `SELECT 1` makes a column `integer`, which a growing factorial soon overflows (exceeds the largest value the type can hold). Cast in the anchor (`first_name::text`, `1::numeric`).
- **Thinking the recursive term sees the whole result**: it sees only the previous pass's rows.
- **Relying on output order.** Without `ORDER BY` the order of a recursive result is not guaranteed. Build a sort key (depth, path, or `SEARCH`, 7.5) and order by it.

Self-check

1. *What are the two parts of a recursive CTE?* → The anchor (runs once) and the recursive term (runs repeatedly on the previous pass's rows), joined by `UNION ALL` or `UNION`.
2. *When does recursion stop?* → When a pass produces no new rows.
3. *Why cast `0::bigint` in the anchor?* → The anchor fixes the column types; without the cast the column is `integer` and larger values in later passes would overflow.

4. *What does SQL Server need instead of* `WITH RECURSIVE`*?* → Just `WITH`; and it limits recursion to 100 levels unless `OPTION (MAXRECURSION n)`.

TL;DR

Anchor rows → repeatedly apply the recursive term to the last batch → stop when a batch is empty. Always have a stop condition, cast types in the anchor, and `ORDER BY` explicitly.

7.4 Hierarchies: trees stored as parent pointers

What it is

The most common way to store a tree in a relational table is the **adjacency list**: each row stores the key of its parent (`employees.manager_id` points to another `employees.emp_id`; `NULL` marks the root). Recursive CTEs are the standard way to query such trees.

How it works — three classic queries

Question	Anchor	Recursive step
Top-down: the whole tree (or a subtree)	the root(s): `manager_id IS NULL` (or `emp_id = X`)	children: `e.manager_id = parent.emp_id`
Bottom-up: the chain of ancestors	the starting node	parent: `m.emp_id = child.manager_id`
Subtree aggregates	every (boss, direct report) pair	extend each pair one level down, keeping the boss

Top-down with depth and path. The path column doubles as a sort key that keeps each manager's team directly under them:

```sql
-- code/04_hierarchy.sql
WITH RECURSIVE org AS (
    SELECT emp_id, first_name, manager_id, 1 AS depth,
           first_name::text AS path
    FROM employees
    WHERE manager_id IS NULL                        -- anchor: the root
    UNION ALL
    SELECT e.emp_id, e.first_name, e.manager_id, o.depth + 1,
           o.path || ' > ' || e.first_name
    FROM employees e
    JOIN org o ON e.manager_id = o.emp_id           -- children of rows found so far
)
SELECT repeat('    ', depth - 1) || first_name AS org_chart, depth, path
FROM org
ORDER BY path;
```

Output:

```
   org_chart    | depth |         path
----------------+-------+----------------------
 Asha           |     1 | Asha
     Daniel     |     2 | Asha > Daniel
         Arjun  |     3 | Asha > Daniel > Arjun
         Priya  |     3 | Asha > Daniel > Priya
         Sara   |     3 | Asha > Daniel > Sara
     Kiran      |     2 | Asha > Kiran
         Omar   |     3 | Asha > Kiran > Omar
 ...
     Vikram     |     2 | Asha > Vikram
         Li     |     3 | Asha > Vikram > Li
 ...
         Rohan  |     3 | Asha > Vikram > Rohan
(15 rows)
```

Pass 1 found Asha's four direct reports (depth 2), pass 2 found their reports (depth 3), pass 3 found nobody, so it stopped. Sorting by a text path works here because names don't contain `>`; for a robust sort key use an array of ids (`ARRAY[emp_id]`, then `path || e.emp_id`) — arrays compare element by element. To list only one subtree, change the anchor to `WHERE emp_id = 2`.

Bottom-up: who is above the intern?

```sql
WITH RECURSIVE chain AS (
    SELECT emp_id, first_name, job_title, manager_id, 0 AS steps_up
    FROM employees WHERE emp_id = 15
    UNION ALL
    SELECT m.emp_id, m.first_name, m.job_title, m.manager_id, c.steps_up + 1
    FROM employees m
    JOIN chain c ON m.emp_id = c.manager_id
)
SELECT steps_up, first_name, job_title FROM chain ORDER BY steps_up;
```

Output:

```
 steps_up | first_name |      job_title
----------+------------+---------------------
        0 | Rohan      | Intern
        1 | Vikram     | Engineering Manager
        2 | Asha       | CEO
(3 rows)
```

The only difference from top-down is the direction of the join: `m.emp_id = c.manager_id` ("find my parent") instead of `e.manager_id = o.emp_id` ("find my children"). Real uses: breadcrumbs (`Home > Electronics > Laptops`), approval chains, permission inheritance.

Subtree totals: team size and payroll under each manager. The trick is to carry the *boss* unchanged while walking down, producing one row per (boss, anyone below them):

```sql
WITH RECURSIVE reports AS (
    SELECT manager_id AS boss, emp_id, salary        -- direct reports
    FROM employees WHERE manager_id IS NOT NULL
    UNION ALL
    SELECT r.boss, e.emp_id, e.salary                -- their reports, and so on
    FROM reports r
    JOIN employees e ON e.manager_id = r.emp_id
)
SELECT b.first_name AS manager, count(*) AS team_size, sum(r.salary) AS team_salary
FROM reports r JOIN employees b ON b.emp_id = r.boss
GROUP BY b.first_name
ORDER BY team_size DESC, manager;
```

Output:

```
 manager | team_size | team_salary
---------+-----------+-------------
 Asha    |        14 |  1367000.00
 Vikram  |         5 |   432000.00
 Daniel  |         3 |   208000.00
 Kiran   |         1 |    76000.00
 Lucas   |         1 |    61000.00
(5 rows)
```

Asha has all 14 other employees below her; Vikram has Neha, Rahul, Meera, Li and Rohan. The aggregate runs in the *outer* query — aggregates aren't allowed inside the recursive term.

Other ways to store trees (for comparison)

Model	How	Reads	Writes	Notes
Adjacency list (this chapter)	`parent_id` column	recursive CTE	trivial (change one pointer)	the default choice
Materialized path	store `'1/2/15'` (or PostgreSQL `ltree`, an extension type for dotted label paths)	`LIKE '1/2/%'`, indexable	moving a subtree rewrites all descendants' paths	great for breadcrumbs
Nested sets	store `lft`/`rgt` numbers	subtree = range query	inserts renumber many rows	read-heavy, rarely-changing trees
Closure table	separate table of every (ancestor, descendant, depth) pair	simple joins	many rows per insert	*SQL Antipatterns* recommends it for heavy tree querying

Common mistakes

- **Anchoring on the wrong side**: using `manager_id = X` in the anchor lists X's reports but *not X*; use `emp_id = X` if X should be included.
- **Bad data creating cycles** (A manages B, B manages A). A top-down walk from a real root never reaches such a loop, but a walk that starts inside it runs forever. Protect with a path check (7.5) or a `CHECK`/trigger (a function the database runs automatically when rows change) that prevents cycles.
- **Sorting by depth only**, which lists all level-2 rows, then all level-3 rows, losing the "team under manager" layout. Sort by the path.

In the interview

30-second answer: "Store the tree as an adjacency list — each row points to its parent. To read a subtree, anchor on the root and recursively join children whose parent is in the previous level,

carrying depth and a path for ordering. For ancestors, reverse the join. For totals per node, carry the top node through the recursion and aggregate outside."

- *How would you stop infinite loops if the data could contain a cycle?* → Carry an array of visited ids and add `WHERE NOT e.emp_id = ANY(path)`, or use PostgreSQL 14's `CYCLE` clause.
- *What if the tree is huge and queried constantly?* → Index `manager_id`; consider a closure table or materialized path (`ltree` with a GiST index — a PostgreSQL index type for data that isn't simply sortable, such as paths or shapes) for fast subtree reads.

Self-check

1. *Which join condition walks down, which walks up?* → Down: `child.manager_id = found.emp_id`; up: `parent.emp_id = found.manager_id`.
2. *Why carry `path`?* → To show the route from the root and to sort each subtree under its parent.
3. *Why is the `count(*)` outside the recursive CTE in the team-size query?* → Aggregates are not allowed in the recursive term; you generate all (boss, report) pairs first, then group.

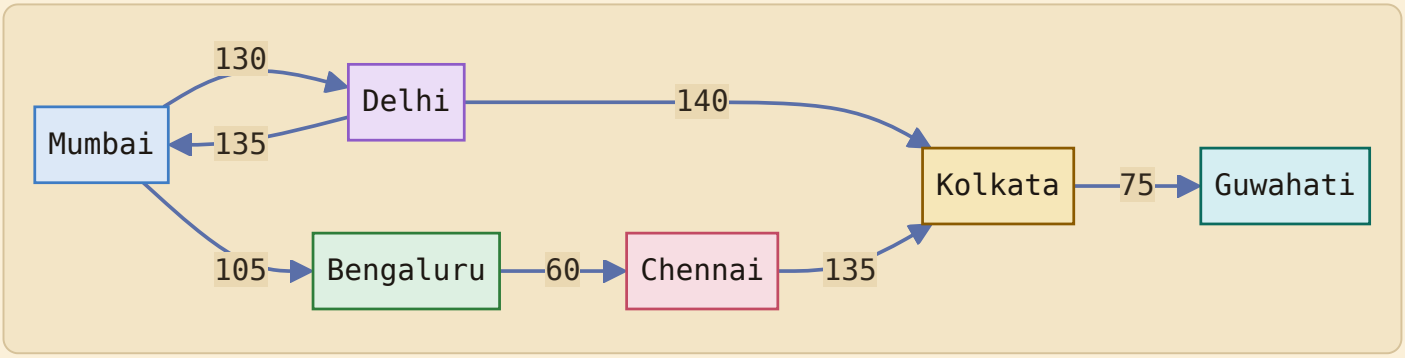
TL;DR

Adjacency list + recursive CTE = any-depth tree queries. Down: join children; up: join parent; totals: carry the ancestor, aggregate outside. Carry depth/path for display and ordering.

7.5 Graphs and cycles

What it is

A **graph** is like a tree but nodes can have several parents and paths can loop back (a **cycle**). Flight routes, road networks, "follows" relationships and package dependencies are graphs. Our sample `routes` table stores one-way flights; Mumbai → Delhi → Mumbai is a cycle.



The problem it solves

A naive recursive walk over a graph with a cycle never ends: Mumbai → Delhi → Mumbai → Delhi … and each row differs (the total time grows), so `UNION` does not stop it. You must remember where you have been.

How it works (manual cycle detection)

1. Carry an **array of visited nodes** (`path`) in every row.
2. In the recursive term, only follow an edge if its destination is not already in the path: `WHERE r.to_city <> ALL (t.path)`.
3. Every row therefore represents a *simple path* (no repeated city); the number of simple paths is finite, so recursion ends.

```sql
-- code/05_graph_cycles.sql
WITH RECURSIVE trips AS (
    SELECT to_city AS city, ARRAY['Mumbai', to_city] AS path, minutes AS total_min
    FROM routes WHERE from_city = 'Mumbai'
    UNION ALL
    SELECT r.to_city, t.path || r.to_city, t.total_min + r.minutes
    FROM trips t
    JOIN routes r ON r.from_city = t.city
    WHERE r.to_city <> ALL (t.path)              -- never revisit a city
)
SELECT city, array_to_string(path, ' -> ') AS route, total_min
FROM trips
ORDER BY city, total_min;
```

Output:

```
   city    |                        route                         | total_min
-----------+------------------------------------------------------+-----------
 Bengaluru | Mumbai -> Bengaluru                                  |       105
 Chennai   | Mumbai -> Bengaluru -> Chennai                       |       165
 Delhi     | Mumbai -> Delhi                                      |       130
 Guwahati  | Mumbai -> Delhi -> Kolkata -> Guwahati               |       345
 Guwahati  | Mumbai -> Bengaluru -> Chennai -> Kolkata -> Guwahati |       375
 Kolkata   | Mumbai -> Delhi -> Kolkata                           |       270
 Kolkata   | Mumbai -> Bengaluru -> Chennai -> Kolkata            |       300
(7 rows)
```

`ARRAY[...]` builds a PostgreSQL array (chapter 14), `||` appends to it, and `x <> ALL (array)` is true when `x` differs from every element (chapter 6 covers `ALL`). Delhi → Mumbai was never followed because Mumbai is already in every path.

Shortest (fastest) route to each city — keep the best row per city with `DISTINCT ON` (chapter 2):

```sql
-- same trips CTE as above, then:
SELECT DISTINCT ON (city) city, total_min, array_to_string(path, ' -> ') AS route
FROM trips
WHERE city <> 'Mumbai'
ORDER BY city, total_min;
```

Output:

```
   city    | total_min |                route
-----------+-----------+---------------------------------------
 Bengaluru |       105 | Mumbai -> Bengaluru
 Chennai   |       165 | Mumbai -> Bengaluru -> Chennai
 Delhi     |       130 | Mumbai -> Delhi
 Guwahati  |       345 | Mumbai -> Delhi -> Kolkata -> Guwahati
 Kolkata   |       270 | Mumbai -> Delhi -> Kolkata
(5 rows)
```

This enumerates *every* simple path, which explodes on big, dense graphs. It is fine for small graphs and interview questions; real route planners use graph algorithms (Dijkstra, A* — classic shortest-path methods that never enumerate every path) in application code or extensions such as pgRouting, or graph databases (databases that store nodes and edges directly) (chapter 16).

The CYCLE and SEARCH clauses (PostgreSQL 14+)

SQL:1999 defines two helper clauses that PostgreSQL implemented in version 14 (the version this book runs):

- `CYCLE col SET is_cycle USING path_col` — PostgreSQL tracks the visited values of `col` for you, marks a row whose `col` value was already seen on its path with `is_cycle = true`, and does not recurse further from it.
- `SEARCH DEPTH FIRST BY col SET ord` or `SEARCH BREADTH FIRST BY col SET ord` — adds a column `ord` that you `ORDER BY` to get depth-first (follow one branch all the way down before the next) or breadth-first (finish each level before going deeper) order.

```
WITH RECURSIVE trips(city, total_min) AS (
    SELECT to_city, minutes FROM routes WHERE from_city = 'Mumbai'
    UNION ALL
    SELECT r.to_city, t.total_min + r.minutes
    FROM trips t JOIN routes r ON r.from_city = t.city
) CYCLE city SET is_cycle USING visited
SELECT city, total_min, is_cycle FROM trips
WHERE city IN ('Mumbai', 'Delhi')
ORDER BY total_min;
```

Output:

```
  city  | total_min | is_cycle
--------+-----------+----------
 Delhi  |       130 | f
 Mumbai |       265 | f
 Delhi  |       395 | t
(3 rows)
```

Why is Mumbai (265) *not* a cycle? The anchor starts from the routes' *destinations*, so "Mumbai" was not yet in the tracked path (Delhi → Mumbai). The next hop back to Delhi repeats Delhi, so that row is flagged and recursion stops there. To treat the start city as visited, begin the anchor with the start city itself (as the manual version did with `ARRAY['Mumbai', to_city]`).

```
WITH RECURSIVE org(emp_id, first_name, manager_id) AS (
    SELECT emp_id, first_name, manager_id FROM employees WHERE manager_id IS NULL
    UNION ALL
    SELECT e.emp_id, e.first_name, e.manager_id
    FROM employees e JOIN org o ON e.manager_id = o.emp_id
) SEARCH BREADTH FIRST BY emp_id SET ord
SELECT first_name, ord FROM org ORDER BY ord LIMIT 6;
```

Output:

```
 first_name |  ord
------------+--------
 Asha       | (0,1)
 Vikram     | (1,2)
 Daniel     | (1,6)
 Lucas      | (1,10)
 Kiran      | (1,12)
 Neha       | (2,3)
(6 rows)
```

For breadth-first, `ord` is (level, id): all of level 1 comes before level 2. `DEPTH FIRST` would instead produce an array path, like our manual `path` column.

Self-check

1. *Why doesn't `UNION` stop the Mumbai–Delhi loop?* → Each lap has a different `total_min`/path, so rows never repeat exactly.

2. *What does `r.to_city <> ALL (t.path)` do?* → Follows an edge only if its destination hasn't been visited on this path.
3. *Which PostgreSQL version added `CYCLE` and `SEARCH`?* → 14.

TL;DR

Graphs can loop, so carry a visited-path array and refuse to revisit, or use PostgreSQL 14's `CYCLE` clause. Path enumeration is exponential (the work can double with every extra hop) — fine for small graphs, not a route planner.

7.6 generate_series: making rows (PostgreSQL)

What it is

`generate_series(start, stop [, step])` is a PostgreSQL **set-returning function**: used in `FROM`, it returns one row per value from `start` to `stop`. It works for integers, numerics, and timestamps/dates (step given as an `interval`).

```sql
-- code/06_generate_series.sql
SELECT * FROM generate_series(1, 5) AS n;
SELECT * FROM generate_series(10, 0, -5) AS n;
SELECT d::date AS day FROM generate_series(DATE '2025-03-01', DATE '2025-03-04', INTERVAL '1 day') AS d;
```

Output (the three results, one after another):

```
 n
---
 1
 2
 ...
 5
(5 rows)

 n
----
 10
  5
  0
(3 rows)

    day
------------
 2025-03-01
 2025-03-02
 2025-03-03
 2025-03-04
(4 rows)
```

With `date` arguments PostgreSQL picks the `timestamp with time zone` version of the function, so cast the result back to `date` for display.

The problem it solves: missing rows in reports

`GROUP BY` can only produce groups for values that **exist** in the data. A month with no orders simply doesn't appear, and a chart drawn from the result silently skips it. The fix: generate the full calendar, then `LEFT JOIN` the data to it and count.

```sql
WITH months AS (
    SELECT m::date AS month
    FROM generate_series(DATE '2025-01-01', DATE '2025-07-01', INTERVAL '1 month') AS m
)
SELECT m.month, count(o.order_id) AS orders
FROM months m
LEFT JOIN orders o ON date_trunc('month', o.placed_at) = m.month
GROUP BY m.month
ORDER BY m.month;
```

Output:

```
   month    | orders
------------+--------
 2025-01-01 |      3
 2025-02-01 |      2
 ...
 2025-06-01 |      2
 2025-07-01 |      0
(7 rows)
```

July shows `0`. Note `count(o.order_id)`, not `count(*)`: for July the `LEFT JOIN` produces one row with `NULL` order columns, and `count(*)` would count it as 1 (chapter 4).

Test data in one statement:

```sql
CREATE TABLE fake_events AS
SELECT g AS id, DATE '2025-01-01' + (g % 90) AS day, (g * 37) % 100 AS score
FROM generate_series(1, 1000) AS g;
SELECT count(*), min(day), max(day), round(avg(score), 1) AS avg_score FROM fake_events;
```

Output:

```
 count |    min     |    max     | avg_score
-------+------------+------------+-----------
  1000 | 2025-01-01 | 2025-03-31 |      49.5
(1 row)
```

The arithmetic keeps the data deterministic (identical on every run; no `random()`), which is handy for reproducible performance tests (chapter 12 uses this idea with millions of rows).

Dialect notes

	Numbers / dates series
PostgreSQL	`generate_series(...)`
SQL Server 2022+	`GENERATE_SERIES(start, stop [, step])` — numbers only (SQL Server — not run)
SQLite	`generate_series` table-valued function from the `series` extension (built into the `sqlite3` shell)
MySQL 8	none — use a recursive CTE (7.3) or a permanent "numbers"/"calendar" table

Many data warehouses (databases built for analysing large amounts of historical data) keep a permanent **calendar table** (one row per date with flags like `is_weekend`, `fiscal_quarter`) for the same purpose.

TL;DR

`generate_series` produces rows of numbers or timestamps. Generate the full range, `LEFT JOIN` the facts, `count(col)` — and empty days/months appear as 0.

7.7 Views

What it is

A **view** is a named query stored in the database. You query it like a table (`SELECT * FROM staff_directory`), but it stores **no data**: every time you use it, the database substitutes the view's query into yours and runs it against the current contents of the underlying (**base**) tables.

Analogy: a view is a saved search in your email client. It doesn't copy emails into a new folder; it re-runs the search each time you open it, so it's always up to date.

Why it matters

Benefit	Example
Simplicity — hide a complex join once	analysts query `staff_directory` instead of re-writing the join
Security — expose only some columns/rows	grant access to a view without `salary`, not to `employees`
Stable interface — change tables, keep the view	rename/split a table and redefine the view so old queries still work
Consistency — one definition of a metric	"active customer" is defined once, not in 20 reports

How it works

1. `CREATE VIEW name AS query;` stores the query text (parsed) in the catalog.
2. When a query mentions the view, PostgreSQL's rewriter (a stage that runs before planning and expands views) replaces the name with the stored query — just like an inlined CTE — and the optimizer plans the combined query.
3. Because nothing is stored, a view is always current and costs exactly as much as its query.

```sql
-- code/07_views.sql
CREATE VIEW staff_directory AS
SELECT e.emp_id, e.first_name || ' ' || e.last_name AS full_name,
       e.job_title, d.name AS department, e.email
FROM employees e
LEFT JOIN departments d USING (dept_id);

SELECT full_name, job_title, department FROM staff_directory
WHERE department = 'Sales' ORDER BY full_name;
```

Output:

```
   full_name   |   job_title   | department
---------------+---------------+------------
 Arjun Singh   | Sales Rep     | Sales
 Daniel Brooks | Sales Manager | Sales
 Priya Nair    | Sales Rep     | Sales
 Sara Khan     | Sales Rep     | Sales
(4 rows)
```

Always current — change a base table and the view reflects it immediately:

```sql
UPDATE employees SET job_title = 'Senior Sales Rep' WHERE emp_id = 7;
SELECT full_name, job_title FROM staff_directory WHERE emp_id = 7;
```

Output:

```
 full_name  |    job_title
------------+------------------
 Priya Nair | Senior Sales Rep
(1 row)
```

Changing and dropping views

- `CREATE OR REPLACE VIEW` swaps the definition in place (keeping permissions), but in PostgreSQL the new query must return the **same columns, same names, same types, in the same order**; you may only **add** columns at the end.
- To remove or reorder columns: `DROP VIEW` and `CREATE VIEW` again (re-granting privileges — the permissions given with `GRANT`).
- Objects that depend on each other are tracked: you can't drop a table (or a view) that a view uses unless you add `CASCADE`, which drops the dependent views too.

```sql
-- adding hire_date at the end: allowed
CREATE OR REPLACE VIEW staff_directory AS
SELECT e.emp_id, e.first_name || ' ' || e.last_name AS full_name,
       e.job_title, d.name AS department, e.email, e.hire_date
FROM employees e
LEFT JOIN departments d USING (dept_id);

-- removing columns: refused
CREATE OR REPLACE VIEW staff_directory AS
SELECT e.emp_id, e.job_title FROM employees e;

-- dropping a table the view depends on: refused
DROP TABLE departments;
```

Output:

```
ERROR:  cannot drop columns from view
ERROR:  cannot drop table departments because other objects depend on it
DETAIL:  constraint employees_dept_id_fkey on table employees depends on table departments
constraint projects_dept_id_fkey on table projects depends on table departments
view staff_directory depends on table departments
HINT:  Use DROP ... CASCADE to drop the dependent objects too.
```

(Two foreign keys also depend on `departments`; `CASCADE` would drop those constraints and the view — rarely what you want.)

Updatable views

You can `INSERT`, `UPDATE` and `DELETE` through a view if PostgreSQL can map each view row to exactly one base-table row. Since PostgreSQL 9.3 a view is **automatically updatable** when its query:

- reads from exactly **one** table or updatable view in `FROM`;
- has no `GROUP BY`, `HAVING`, `DISTINCT`, `LIMIT`/`OFFSET`, window functions, aggregates, `UNION`/`INTERSECT`/`EXCEPT`, or `WITH` at top level.

Computed columns in such a view are simply read-only; plain columns can be written. For anything more complex you write an `INSTEAD OF` trigger (a trigger is a function the database runs automatically on a change; `INSTEAD OF` runs it in place of the write; chapter 14).

```sql
CREATE VIEW furniture AS
SELECT product_id, name, unit_price, category
FROM products
WHERE category = 'Furniture';

UPDATE furniture SET unit_price = 329.99 WHERE name = 'Ergonomic Chair';
INSERT INTO furniture VALUES (10, 'Bookshelf', 120.00, 'Furniture');
-- Nothing stops a row that falls OUTSIDE the view's WHERE...
INSERT INTO furniture VALUES (11, 'Webcam', 59.00, 'Electronics');
SELECT product_id, name, category FROM products WHERE product_id >= 10;
SELECT count(*) AS visible_in_view FROM furniture WHERE product_id >= 10;
```

Output:

```
 product_id |   name    |  category
------------+-----------+-------------
         10 | Bookshelf | Furniture
         11 | Webcam    | Electronics
(2 rows)

 visible_in_view
-----------------
               1
(1 row)
```

The Webcam went into `products` through the `furniture` view but is invisible in that view — a confusing "disappearing row". The `discontinued` column, not in the view, got its default (`false`).

`WITH CHECK OPTION` makes the view reject inserts and updates whose resulting row would not satisfy the view's `WHERE`:

```sql
CREATE OR REPLACE VIEW furniture AS
SELECT product_id, name, unit_price, category
FROM products
WHERE category = 'Furniture'
WITH CHECK OPTION;

INSERT INTO furniture VALUES (12, 'Monitor Arm', 79.00, 'Accessories');
```

Output:

```
ERROR:  new row violates check option for view "furniture"
DETAIL:  Failing row contains (12, Monitor Arm, Accessories, 79.00, f).
```

(`LOCAL` checks only this view's condition; `CASCADED`, the default, also checks the conditions of views it is built on.)

A view with a join or `GROUP BY` is read-only:

```sql
CREATE VIEW dept_payroll AS
SELECT d.name AS department, count(e.emp_id) AS headcount, sum(e.salary) AS payroll
FROM departments d LEFT JOIN employees e USING (dept_id)
GROUP BY d.name;
SELECT * FROM dept_payroll ORDER BY payroll DESC NULLS LAST;
UPDATE dept_payroll SET headcount = 0 WHERE department = 'Sales';
```

Output:

```
 department  | headcount |  payroll
-------------+-----------+-----------
 Engineering |         4 | 523000.00
 Sales       |         4 | 358000.00
 ...
 Research    |         0 |    (null)
(6 rows)

ERROR:  cannot update view "dept_payroll"
DETAIL:  Views containing GROUP BY are not automatically updatable.
HINT:  To enable updating the view, provide an INSTEAD OF UPDATE trigger or an unconditional ON UPDATE DO INSTEAD rule.
```

The catalog knows which views are updatable:

```sql
SELECT table_name, is_updatable
FROM information_schema.views
WHERE table_schema = 'public'
ORDER BY table_name;
```

Output:

```
   table_name    | is_updatable
-----------------+--------------
 dept_payroll    | NO
 furniture       | YES
 staff_directory | NO
(3 rows)
```

Views and security

In PostgreSQL a view accesses its base tables with the privileges of the **view's owner**, so you can `GRANT SELECT ON staff_directory TO analyst` without granting anything on `employees`; the analyst sees names and titles but never salaries. Two refinements:

- `WITH (security_barrier)` stops a cleverly written user function in the caller's `WHERE` from peeking at rows the view's own `WHERE` is meant to hide (the optimizer otherwise may evaluate the cheap user function first).
- PostgreSQL 15 added `WITH (security_invoker = true)` to make a view use the *caller's* privileges instead (PostgreSQL 15 — not run here on 14).

Roles, `GRANT` and row-level security (rules that limit which rows each user may see) are in chapter 14.

Dialect notes

	PostgreSQL	MySQL	SQLite	SQL Server
`CREATE OR REPLACE VIEW`	yes (add columns only)	yes (any change)	no — drop & create	`CREATE OR ALTER VIEW` (2016 SP1+)
Auto-updatable simple views	yes	yes	no — read-only, use `INSTEAD OF` triggers	yes
`WITH CHECK OPTION`	yes	yes	no	yes
Materialized views	yes (7.8)	no	no	"indexed views" (kept up to date automatically)

When not to use it

- **Stacking views on views** many levels deep: each looks simple, the combined query is huge and hard to tune. Keep layers shallow.
- **As a performance tool**: a plain view is exactly as fast as its query. If the query is slow, the view is slow — consider a materialized view or better indexes.
- **`SELECT *` in a view definition**: PostgreSQL expands `*` when the view is *created*, so columns added to the table later do **not** appear in the view. List columns explicitly.

Common mistakes

- Expecting a view to store data (that's a materialized view).
- Being surprised by "disappearing rows" when inserting through a filtered view — add `WITH CHECK OPTION`.
- Trying to reorder or drop columns with `CREATE OR REPLACE` in PostgreSQL.
- Adding `ORDER BY` inside a view and relying on it: order is only guaranteed by the **outer** query's `ORDER BY`.

Real-world examples

- Reporting layers: BI (business-intelligence) tools (Metabase, Tableau, Power BI) are pointed at views rather than raw tables.
- Backward compatibility: after splitting `users` into `accounts` and `profiles`, a `users` view joins them so old code keeps working.
- PostgreSQL's own `information_schema` is a set of views over the system catalogs.

In the interview

> **30-second answer:** "A view is a stored query that acts like a virtual table. It stores no data, so it's always current and costs the same as running its query. Views simplify complex joins, provide a stable interface when tables change, and enforce security by exposing only certain columns or rows. Simple single-table views are updatable; `WITH CHECK OPTION` stops writes that would fall outside the view."

- *Does a view improve performance?* → Not by itself; it is expanded into the query. A materialized view (or indexed view in SQL Server) can.
- *Can you insert into a view?* → Yes if it's a simple single-table view (auto-updatable) or has `INSTEAD OF` triggers.
- *What happens to a view if you add a column to its table?* → Nothing; the view keeps the column list captured when it was created.

Self-check

1. *Where is a view's data stored?* → Nowhere; only its query is stored.
2. *Why did the Webcam row vanish from the `furniture` view?* → It was inserted through the view but doesn't satisfy `category = 'Furniture'`.
3. *What prevents that?* → `WITH CHECK OPTION`.
4. *Can `CREATE OR REPLACE VIEW` remove a column in PostgreSQL?* → No; drop and recreate.
5. *Why is `staff_directory` not updatable?* → It joins two tables.

TL;DR

A view is a named, stored query — virtual table, no data, always current. Use for simplicity, security and stable interfaces. Simple single-table views are writable; add `WITH CHECK OPTION`. Avoid deep view stacks and `SELECT *` definitions.

7.8 Materialized views

What it is

A **materialized view** stores the *result* of its query on disk, like a table, and only recomputes it when you run `REFRESH MATERIALIZED VIEW`. It trades **freshness** for **speed**: reads are as fast as reading a small table, but the data is only as recent as the last refresh.

Analogy: a printed report on your desk. Reading it is instant, but it doesn't change when the numbers in the system change — you have to print a new one.

How it works

1. `CREATE MATERIALIZED VIEW name AS query;` runs the query once and stores the rows (add `WITH NO DATA` to create it empty; it can't be queried until refreshed).
2. Queries read the stored rows. They can use indexes you create on the materialized view.
3. `REFRESH MATERIALIZED VIEW name;` re-runs the query and replaces the contents. This takes a strong lock (a hold on the object that makes other sessions wait): **reads are blocked** during the refresh.
4. `REFRESH MATERIALIZED VIEW CONCURRENTLY name;` builds the new result separately and applies only the differences, so readers are not blocked. It requires a **unique index** on the materialized view (plain columns, no `WHERE`) and is slower overall.

```sql
-- code/08_materialized_views.sql
CREATE MATERIALIZED VIEW product_sales AS
SELECT p.product_id, p.name,
       coalesce(sum(oi.quantity), 0) AS units_sold,
       coalesce(round(sum(oi.quantity * oi.unit_price * (1 - oi.discount)), 2), 0) AS revenue
FROM products p
LEFT JOIN order_items oi USING (product_id)          -- every order line
GROUP BY p.product_id, p.name;

SELECT name, units_sold, revenue FROM product_sales ORDER BY revenue DESC LIMIT 3;
```

Output:

```
      name       | units_sold | revenue
-----------------+------------+---------
 Laptop Pro 14   |          5 | 6380.05
 Standing Desk   |          2 |  978.00
 Ergonomic Chair |          3 |  962.47
(3 rows)
```

Stale until refreshed:

```sql
INSERT INTO orders VALUES (115, 9, NULL, '2025-06-20 09:00', 'pending', NULL);
INSERT INTO order_items VALUES (115, 5, 3, 499.00, 0.00);

SELECT name, units_sold, revenue FROM product_sales WHERE name = 'Standing Desk';
REFRESH MATERIALIZED VIEW product_sales;
SELECT name, units_sold, revenue FROM product_sales WHERE name = 'Standing Desk';
```

Output:

```
     name      | units_sold | revenue
---------------+------------+---------
 Standing Desk |          2 |  978.00
(1 row)

     name      | units_sold | revenue
---------------+------------+---------
 Standing Desk |          5 | 2475.00
(1 row)
```

Concurrent refresh needs a unique index:

```
REFRESH MATERIALIZED VIEW CONCURRENTLY product_sales;          -- fails
CREATE UNIQUE INDEX product_sales_pk ON product_sales (product_id);
REFRESH MATERIALIZED VIEW CONCURRENTLY product_sales;          -- works
CREATE INDEX product_sales_revenue ON product_sales (revenue DESC);
SELECT matviewname, ispopulated FROM pg_matviews;
```

Output:

```
ERROR:  cannot refresh materialized view "public.product_sales" concurrently
HINT:  Create a unique index with no WHERE clause on one or more columns of the materialized view.
  matviewname  | ispopulated
---------------+-------------
 product_sales | t
(1 row)
```

Trade-offs

	View	Materialized view	Summary table maintained by triggers/jobs
Stores data	no	yes	yes
Freshness	always current	as of last refresh	as current as your code keeps it
Read speed	cost of the query	fast (indexable)	fast
Write overhead	none	none at write time; refresh cost later	on every write
Effort	trivial	trivial + schedule refreshes	most code, most bugs

PostgreSQL has **no automatic or incremental refresh**: each refresh recomputes the whole query. You schedule it (cron, the classic Unix scheduler that runs commands on a timetable; the `pg_cron` extension; your job scheduler) or trigger it after batch loads (bulk imports of new data). Oracle supports incremental "fast refresh"; SQL Server's indexed views are updated automatically on every write (with many restrictions); MySQL has no materialized views.

When to use / not use

- **Use** for dashboards and reports over large data that can be minutes or hours old, expensive aggregations read far more often than the data changes, and precomputed search/lookup data.
- **Don't use** when readers need up-to-the-second numbers, or when the base data changes constantly and refreshing costs more than it saves.

Common mistakes

- Forgetting to schedule refreshes — the dashboard shows last month's numbers.
- Using plain `REFRESH` on a busy materialized view and blocking every reader for the duration.

- Treating it as a cache that updates itself (in PostgreSQL it doesn't).

In the interview

> **30-second answer:** "A view stores a query; a materialized view stores the query's result. Materialized views make expensive aggregations fast to read and can be indexed, but they're stale until refreshed. In PostgreSQL refresh is manual and full; `CONCURRENTLY` avoids blocking readers but needs a unique index."

- *How would you keep a materialized view fresh?* → Scheduled refresh (e.g. every 5 minutes with pg_cron), or refresh after each ETL batch (Extract-Transform-Load: a job that copies and reshapes data into the database); if you need real-time, maintain a summary table incrementally instead.
- *Materialized view vs cache like Redis?* → The materialized view lives in the database, is queryable with SQL and consistent with a point in time; an external cache (a fast copy of hot data kept outside the database — Redis is an in-memory key-value store) is faster per key but needs invalidation logic (removing or updating cached copies when the real data changes; see System Design Fundamentals and chapter 15).

Self-check

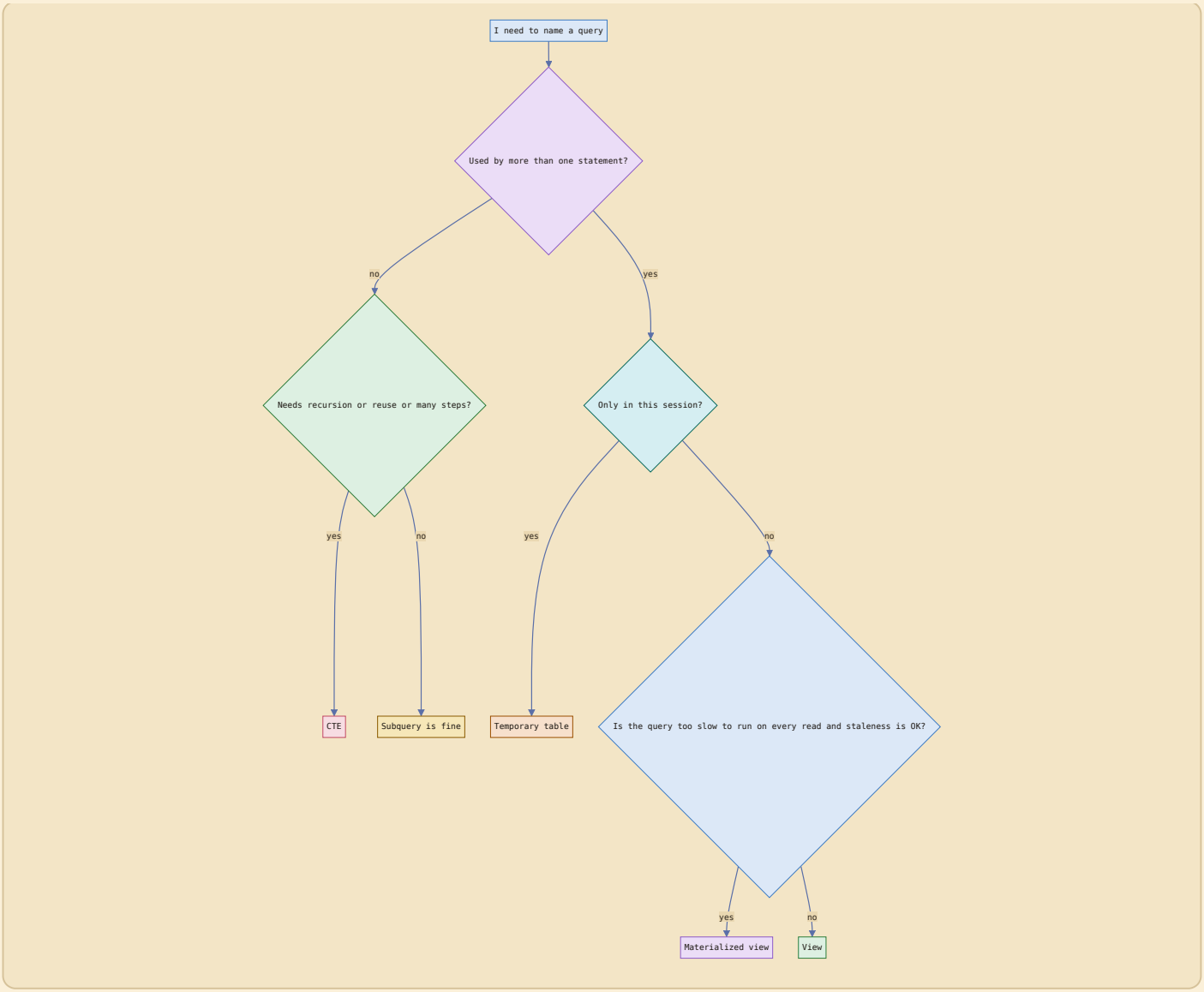
1. *Does a materialized view reflect an insert automatically in PostgreSQL?* → No; only after `REFRESH`.
2. *What does `CONCURRENTLY` need?* → A unique index on the materialized view with no `WHERE` clause.
3. *Can you index a materialized view?* → Yes, like a table.

TL;DR

Materialized view = stored result + manual refresh. Fast reads, stale data. Use `CONCURRENTLY` (with a unique index) to avoid blocking readers; schedule refreshes yourself.

7.9 Choosing the right tool

Need	Use	Lifetime	Stores rows?	Indexable?
One-off intermediate result inside one query, used once	subquery / derived table (ch. 6)	one statement	no	no
Readable multi-step query, reuse within the statement, recursion, move rows atomically	CTE	one statement	only if materialized, temporarily	no
Reusable definition for many queries and users, security, stable interface	view	permanent	no	no (index the base tables)
Fast reads of expensive results that may be slightly stale	materialized view	permanent	yes	yes
Intermediate data reused by several statements in a session, needs indexes/statistics	`CREATE TEMP TABLE` (ch. 9)	session or transaction	yes	yes



Chapter 7 TL;DR

- **CTE** (`WITH name AS (...)`) names a step for one statement; chain with commas; reuse by name. PostgreSQL 12+ inlines single-use CTEs; `MATERIALIZED` / `NOT MATERIALIZED` override.
- **Data-modifying CTEs** (PostgreSQL) run `INSERT/UPDATE/DELETE ... RETURNING` inside `WITH` — atomic multi-step changes; all parts see the pre-statement snapshot.
- **Recursive CTE** = anchor `UNION [ALL]` recursive term; the recursive term sees only the previous pass's rows; stops when a pass returns nothing. Cast types in the anchor, always have a stop condition, `ORDER BY` a path/depth.
- **Trees**: adjacency list + recursion — join children to go down, join the parent to go up, carry the top node and aggregate outside for subtree totals.
- **Graphs**: carry a visited array and refuse revisits, or use `CYCLE` (PostgreSQL 14+); `SEARCH BREADTH/DEPTH FIRST` gives an ordering column.
- `generate_series` makes number/date rows: full calendar + `LEFT JOIN` + `count(col)` fills gaps with zeros. MySQL uses a recursive CTE instead.
- **View** = stored query, no data, always current; for simplicity, security, stable interfaces. Simple single-table views are updatable; `WITH CHECK OPTION` blocks out-of-view writes; `CREATE OR REPLACE` can only append columns in PostgreSQL.
- **Materialized view** = stored result; fast, indexable, stale until `REFRESH` (`CONCURRENTLY` needs a unique index). No auto-refresh in PostgreSQL.

Next: chapter 8 — window functions, which compute running totals, rankings and comparisons with neighbouring rows without collapsing rows the way `GROUP BY` does.

Chapter 8 — Window Functions: Rankings, Running Totals and Row-to-Row Comparisons

`GROUP BY` answers "what is the total per department?" but throws away the individual rows to do it. Very often you need both at once: *each* employee **and** their department's average; *each* day's sales **and** the running total so far; *each* order **and** the previous order by the same customer. **Window functions** do exactly that. They compute a value across a set of related rows — the *window* — while keeping every row in the result.

They replace awkward self-joins (a table joined to itself) and correlated subqueries (subqueries re-run for every outer row), and they sit behind many SQL interview questions (top-N per group, running totals, second-highest salary, streaks, month-over-month growth).

What you'll learn

#	Topic	You will be able to…
8.1	`OVER()`	explain how a window function differs from `GROUP BY`
8.2	`PARTITION BY`	compute per-group values next to each row (share of total, diff from average)
8.3	`ORDER BY` in a window	build running totals and understand the default frame and its tie trap
8.4	Frames: `ROWS` / `RANGE` / `GROUPS`, `EXCLUDE`	choose exactly which rows each calculation sees
8.5	Ranking	use `ROW_NUMBER`, `RANK`, `DENSE_RANK`, `NTILE`, `PERCENT_RANK`, `CUME_DIST` and handle ties
8.6	Value functions	compare rows with `LAG` / `LEAD`, `FIRST_VALUE` / `LAST_VALUE` / `NTH_VALUE`
8.7	Running & moving calculations	running totals, moving averages, cumulative share, growth %
8.8	Top-N per group & dedup (de-duplication: removing extra copies)	the most-asked window pattern, plus "keep the latest row"
8.9	Gaps and islands	find streaks, missing ranges and sessions (bursts of activity separated by idle time)
8.10	Named windows, `FILTER`, evaluation order, performance	write tidy window queries and know how they execute

Setup. The `company` sample database from chapter 1 plus two small tables added in `code/00_schema.sql`:

Table	Rows	Why
`daily_sales(store, day, amount)`	Mumbai 1–7 March 2025; Pune the same week **except 4 March** (closed)	running totals, moving averages, frames, a gap
`logins(user_id, login_date)`	two users' login days in May 2025 with breaks	streaks (gaps and islands)

Mumbai's sales for 3 and 4 March are both 1500, and two Pune days are both 650 — deliberate ties. In `employees`, Rahul and Meera (Engineering) both earn 98 000 and Arjun and Sara (Sales) both earn 68 000.

8.1 What a window function is

What it is

A **window function** is a function that, for each row, looks at a set of rows related to it (its **window**) and returns one value — **without merging the rows**. You recognise it by the `OVER (...)` clause after the function call:

```
function_name(args) OVER (
    [PARTITION BY ...]      -- split rows into independent groups
    [ORDER BY ...]          -- order rows inside each group
    [frame clause]          -- which rows around the current one to use
)
```

Analogy: in a class photo, `GROUP BY` replaces each row of students with one sign ("Row 2: average height 150 cm"). A window function keeps every student and pins a badge on each: "your row's average is 150 cm", "you are 3rd tallest in your row", "the student to your left is 2 cm shorter".

The problem it solves

Compare `GROUP BY` with a window:

```
-- code/01_window_basics.sql
SELECT dept_id, round(avg(salary)) AS dept_avg
FROM employees WHERE dept_id IN (1, 2)
GROUP BY dept_id ORDER BY dept_id;
```

Output:

```
 dept_id | dept_avg
---------+----------
       1 |   130750
       2 |    89500
(2 rows)
```

Two rows: the individual employees are gone. With windows, every employee stays and gets the aggregates beside them:

```
SELECT first_name, dept_id, salary,
       round(avg(salary) OVER ())                    AS company_avg,
       round(avg(salary) OVER (PARTITION BY dept_id)) AS dept_avg,
       count(*)          OVER (PARTITION BY dept_id)  AS dept_size
FROM employees
WHERE dept_id IN (1, 2)
ORDER BY dept_id, salary DESC;
```

Output:

```
 first_name | dept_id |  salary   | company_avg | dept_avg | dept_size
------------+---------+-----------+-------------+----------+-----------
 Vikram     |       1 | 185000.00 |      110125 |   130750 |         4
 Neha       |       1 | 142000.00 |      110125 |   130750 |         4
 ...
 Daniel     |       2 | 150000.00 |      110125 |    89500 |         4
 Priya      |       2 |  72000.00 |      110125 |    89500 |         4
 ...
(8 rows)
```

- `OVER ()` — an empty window means "all rows of the result" (here the 8 rows left after `WHERE`), so `company_avg` is the same on every row.
- `OVER (PARTITION BY dept_id)` — the window is "all rows with my `dept_id`".

Without window functions you'd need a join to a grouped subquery (or a correlated subquery per row) for each extra column.

How it works

1. The query's `FROM`, `WHERE`, `GROUP BY` and `HAVING` run first and produce a set of rows.
2. For each window function, rows are split into **partitions** (`PARTITION BY`) and sorted inside each partition (`ORDER BY`).
3. For each row, the function looks at the rows of its window (the whole partition, or a **frame** within it — 8.3/8.4) and computes one value.
4. That value becomes a new column. Row count is unchanged.
5. Then `DISTINCT`, the final `ORDER BY` and `LIMIT` run.

Two families of functions can be used with `OVER`:

Family	Functions	Notes
Aggregate functions used as windows	`sum`, `avg`, `count`, `min`, `max`, `string_agg`, `array_agg`, `bool_and`, …	any normal aggregate becomes a window function when you add `OVER`
Dedicated window functions	`row_number`, `rank`, `dense_rank`, `percent_rank`, `cume_dist`, `ntile`, `lag`, `lead`, `first_value`, `last_value`, `nth_value`	only valid with `OVER`; the ranking ones take no arguments

Dialect notes

	Window functions available since	Frame support
PostgreSQL	8.4 (2009)	`ROWS`, `RANGE` with offsets, `GROUPS`, `EXCLUDE` (offsets/`GROUPS`/`EXCLUDE` since 11)
MySQL	8.0 (2018)	`ROWS`, `RANGE`; no `GROUPS`, no `EXCLUDE`
SQLite	3.25 (2018)	`ROWS`, `RANGE`; `GROUPS` and `EXCLUDE` since 3.28
SQL Server	ranking 2005; aggregate `ORDER BY` + frames 2012	`ROWS`; `RANGE` only with `UNBOUNDED`/`CURRENT ROW`; no `GROUPS`/`EXCLUDE`
Oracle	8i (1999)	full, plus extensions

TL;DR

`f(...) OVER (...)` computes across related rows but keeps every row. `OVER ()` = all rows; add `PARTITION BY`, `ORDER BY` and a frame to narrow it.

8.2 PARTITION BY: per-group values next to each row

What it is

`PARTITION BY expr` splits the rows into independent groups, like `GROUP BY` does, but without collapsing them. Each row's window is its own partition; the function "restarts" in every partition.

Typical uses

```
SELECT first_name, dept_id, salary,
       round(100.0 * salary / sum(salary) OVER (PARTITION BY dept_id), 1) AS pct_of_dept_payroll,
       salary - round(avg(salary) OVER (PARTITION BY dept_id))            AS vs_dept_avg
FROM employees
WHERE dept_id IN (1, 2)
ORDER BY dept_id, salary DESC;
```

Output:

```
 first_name | dept_id |  salary   | pct_of_dept_payroll | vs_dept_avg
------------+---------+-----------+---------------------+-------------
 Vikram     |       1 | 185000.00 |                35.4 |    54250.00
 Neha       |       1 | 142000.00 |                27.2 |    11250.00
 Rahul      |       1 |  98000.00 |                18.7 |   -32750.00
 Meera      |       1 |  98000.00 |                18.7 |   -32750.00
 ...
(8 rows)
```

"Percent of group total" and "difference from group average" are the two most common `PARTITION BY` calculations. `100.0 *` forces decimal division (chapter 3).

Partitioning rules

- You can partition by several expressions: `PARTITION BY store, date_trunc('month', day)`.
- `NULL` values form **one** partition together (just like `GROUP BY`):

```
SELECT first_name, dept_id, count(*) OVER (PARTITION BY dept_id) AS same_dept
FROM employees
WHERE dept_id IS NULL OR dept_id = 4
ORDER BY dept_id NULLS LAST;
```

Output:

```
 first_name | dept_id | same_dept
------------+---------+-----------
 Kiran      |       4 |         2
 Omar       |       4 |         2
 Asha       |  (null) |         2
 Rohan      |  (null) |         2
(4 rows)
```

- Different window functions in one `SELECT` can use different partitions (the first example mixed `OVER ()` with `OVER (PARTITION BY dept_id)`).

Common mistakes

- **Expecting fewer rows.** A window never removes rows; if you want one row per group, you want `GROUP BY` (or filter afterwards, 8.8).
- **Mixing a window and a non-grouped column in a `GROUP BY` query**: in a grouped query, window functions see the *grouped* rows, so their arguments must be grouped columns or aggregates (8.7 shows `sum(sum(x)) OVER ()`).

Self-check

1. *How many rows does `SELECT name, avg(salary) OVER (PARTITION BY dept_id) FROM employees` return?* → One per employee (15).
2. *Where do rows with `dept_id` NULL go?* → Together in one partition.

TL;DR

`PARTITION BY` = `GROUP BY` without collapsing. Perfect for "share of group total" and "difference from group average".

8.3 ORDER BY inside OVER: running calculations and the default frame

What it is

Adding `ORDER BY` inside `OVER` does two things:

1. It defines the order that ranking and offset functions (`row_number`, `lag`, …) use.
2. For **aggregate** window functions it changes the default window from "the whole partition" to "from the start of the partition **up to the current row**" — so `sum` becomes a **running total**, `count` a running count, `max` a running maximum.

```sql
-- code/02_order_and_default_frame.sql
SELECT store, day, amount,
       sum(amount) OVER (PARTITION BY store ORDER BY day) AS running_total,
       count(*)    OVER (PARTITION BY store ORDER BY day) AS days_so_far
FROM daily_sales
WHERE store = 'Mumbai'
ORDER BY day;
```

Output:

```
 store  |    day     | amount  | running_total | days_so_far
--------+------------+---------+---------------+-------------
 Mumbai | 2025-03-01 | 1200.00 |       1200.00 |           1
 Mumbai | 2025-03-02 |  900.00 |       2100.00 |           2
 Mumbai | 2025-03-03 | 1500.00 |       3600.00 |           3
 Mumbai | 2025-03-04 | 1500.00 |       5100.00 |           4
 Mumbai | 2025-03-05 |  700.00 |       5800.00 |           5
 Mumbai | 2025-03-06 | 1100.00 |       6900.00 |           6
 Mumbai | 2025-03-07 | 1800.00 |       8700.00 |           7
(7 rows)
```

The default frame and the tie trap

The exact default, when `ORDER BY` is present and no frame is written, is:

```sql
RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
```

`RANGE` works on *values*, not physical rows: "current row" means the current row **and all its peers** — rows with the same `ORDER BY` value. So when the ordering column has ties, tied rows are added together and all get the same "running" total:

```sql
SELECT first_name, salary,
       sum(salary) OVER (ORDER BY salary)                    AS default_range,
       sum(salary) OVER (ORDER BY salary
                         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS rows_frame,
       sum(salary) OVER (ORDER BY salary, emp_id)            AS unique_order
FROM employees
WHERE dept_id = 2
ORDER BY salary, emp_id;
```

Output:

```
 first_name |  salary   | default_range | rows_frame | unique_order
------------+-----------+---------------+------------+--------------
 Arjun      |  68000.00 |     136000.00 |   68000.00 |     68000.00
 Sara       |  68000.00 |     136000.00 |  136000.00 |    136000.00
 Priya      |  72000.00 |     208000.00 |  208000.00 |    208000.00
 Daniel     | 150000.00 |     358000.00 |  358000.00 |    358000.00
(4 rows)
```

- `default_range`: Arjun and Sara are peers, so both see 68 000 + 68 000.
- `rows_frame`: `ROWS` counts physical rows, so the total grows one row at a time — but which tied row comes "first" is **not defined**; next time Sara could get 68 000.
- `unique_order`: adding a tie-breaker (`emp_id`) makes the order unique, so there are no peers and every frame type gives the same, **deterministic** (identical on every run) result. This is the best fix.

`OVER (...)` contains	Default window for aggregates
nothing / only `PARTITION BY`	the whole partition
`ORDER BY`	`RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` (start → current row + its peers)

Common mistakes

- **Running totals that "jump"** on duplicate dates or amounts — the peer effect. Order by a unique key, or write `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`.
- **Confusing the window's `ORDER BY` with the query's `ORDER BY`.** The one inside `OVER` only controls the calculation; the output order is set by the final `ORDER BY` (without it, output order is not guaranteed).

Self-check

1. *Why can a running total show the same value on two rows?* → The default `RANGE` frame includes peers — rows with the same `ORDER BY` value.
2. *Two ways to avoid it?* → Make the `ORDER BY` unique (add a key) or use a `ROWS` frame (plus a tie-breaker for determinism).

TL;DR

`ORDER BY` in `OVER` turns aggregates into running aggregates. The default frame is `RANGE … CURRENT ROW`, which lumps ties together — add a unique tie-breaker.

8.4 Frames: ROWS, RANGE, GROUPS and EXCLUDE

What it is

The **frame** is the subset of the partition that an aggregate (and `first_value`/`last_value`/`nth_value`) looks at for the current row. It "slides" as the current row moves. Ranking functions and `lag`/`lead` ignore the frame.

```
{ ROWS | RANGE | GROUPS } BETWEEN frame_start AND frame_end [EXCLUDE ...]

frame_start / frame_end:
    UNBOUNDED PRECEDING      -- first row of the partition
    n PRECEDING              -- n rows / n units of value / n peer groups before
    CURRENT ROW
    n FOLLOWING
    UNBOUNDED FOLLOWING      -- last row of the partition
```

Short form: `ROWS 2 PRECEDING` means `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW`.

Mode	`n PRECEDING` means	Peers (ties)	Needs
`ROWS`	*n physical rows* before	treated as separate rows	any `ORDER BY`
`RANGE`	rows whose **value** is within *n* of the current value (`n` can be a number or an `interval`)	always included together	exactly one `ORDER BY` column of a numeric/date/time type when an offset is used
`GROUPS` (PostgreSQL 11+)	*n peer groups* before (a group = rows with equal `ORDER BY` values)	included together	an `ORDER BY`

Frame diagrams

In the tables below, each line is one "current row" (**X**), and `x` marks the other rows inside its frame. The SQL uses `string_agg(...) OVER (frame)` to list the days in each frame — a handy trick to *see* a frame.

`ROWS BETWEEN 2 PRECEDING AND CURRENT ROW` — a sliding window of up to 3 rows:

```sql
-- code/03_frames.sql
SELECT day, amount,
       string_agg(to_char(day, 'DD'), ',') OVER w AS frame_days,
       sum(amount) OVER w                         AS frame_sum
FROM daily_sales
WHERE store = 'Mumbai'
WINDOW w AS (ORDER BY day ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
ORDER BY day;
```

Output:

```
    day     | amount  | frame_days | frame_sum
------------+---------+------------+-----------
 2025-03-01 | 1200.00 | 01         |   1200.00
 2025-03-02 |  900.00 | 01,02      |   2100.00
 2025-03-03 | 1500.00 | 01,02,03   |   3600.00
 2025-03-04 | 1500.00 | 02,03,04   |   3900.00
 2025-03-05 |  700.00 | 03,04,05   |   3700.00
 2025-03-06 | 1100.00 | 04,05,06   |   3300.00
 2025-03-07 | 1800.00 | 05,06,07   |   3600.00
(7 rows)
```

current row	01	02	03	04	05	06	07
01	**X**	·	·	·	·	·	·
02	x	**X**	·	·	·	·	·
03	x	x	**X**	·	·	·	·
04	·	x	x	**X**	·	·	·
05	·	·	x	x	**X**	·	·
06	·	·	·	x	x	**X**	·
07	·	·	·	·	x	x	**X**

The first two frames are *shorter* — there are no earlier rows. (The `WINDOW w AS (...)` clause names a window so it can be reused; see 8.10.)

Centered: `ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING` — smoothing that looks both ways:

```
SELECT day, amount,
       string_agg(to_char(day, 'DD'), ',') OVER w AS frame_days,
       round(avg(amount) OVER w, 1)                AS centered_avg
FROM daily_sales
WHERE store = 'Mumbai'
WINDOW w AS (ORDER BY day ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING)
ORDER BY day;
```

Output:

```
    day     | amount  | frame_days | centered_avg
------------+---------+------------+--------------
 2025-03-01 | 1200.00 | 01,02      |       1050.0
 2025-03-02 |  900.00 | 01,02,03   |       1200.0
 ...
 2025-03-07 | 1800.00 | 06,07      |       1450.0
(7 rows)
```

`ROWS` vs `RANGE` when a day is missing. Pune has no row for 4 March. "The previous 2 rows" and "the previous 2 days" now differ:

```
SELECT day, amount,
       string_agg(to_char(day, 'DD'), ',') OVER (ORDER BY day
            ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)                    AS rows_2,
       string_agg(to_char(day, 'DD'), ',') OVER (ORDER BY day
            RANGE BETWEEN INTERVAL '2 days' PRECEDING AND CURRENT ROW)  AS range_2_days
FROM daily_sales
WHERE store = 'Pune'
ORDER BY day;
```

Output:

```
    day     | amount |  rows_2  | range_2_days
------------+--------+----------+--------------
 2025-03-01 | 400.00 | 01       | 01
 2025-03-02 | 650.00 | 01,02    | 01,02
 2025-03-03 | 500.00 | 01,02,03 | 01,02,03
 2025-03-05 | 800.00 | 02,03,05 | 03,05
 2025-03-06 | 650.00 | 03,05,06 | 05,06
 2025-03-07 | 300.00 | 05,06,07 | 05,06,07
(6 rows)
```

`ROWS 2 PRECEDING` (three rows, whatever their dates):

current row	01	02	03	05	06	07
01	**X**	·	·	·	·	·
02	x	**X**	·	·	·	·
03	x	x	**X**	·	·	·
05	·	x	x	**X**	·	·
06	·	·	x	x	**X**	·
07	·	·	·	x	x	**X**

`RANGE INTERVAL '2 days' PRECEDING` (dates from current − 2 days to current):

current row	01	02	03	05	06	07
01	**X**	·	·	·	·	·
02	x	**X**	·	·	·	·
03	x	x	**X**	·	·	·
05	·	·	x	**X**	·	·
06	·	·	·	x	**X**	·
07	·	·	·	x	x	**X**

For "average of the last 3 *calendar* days", `RANGE` with an interval is correct and `ROWS` silently reaches back to 2 March. (Alternatively, fill the missing days with `generate_series` first — chapter 7.)

`ROWS` vs `RANGE` vs `GROUPS` with ties. Ordering Pune by `amount` (two days have 650):

```
SELECT day, amount,
       string_agg(amount::int::text, ',') OVER (ORDER BY amount
            ROWS   BETWEEN 1 PRECEDING AND CURRENT ROW) AS rows_1,
       string_agg(amount::int::text, ',') OVER (ORDER BY amount
            RANGE  BETWEEN 150 PRECEDING AND CURRENT ROW) AS range_150,
       string_agg(amount::int::text, ',') OVER (ORDER BY amount
            GROUPS BETWEEN 1 PRECEDING AND CURRENT ROW) AS groups_1
FROM daily_sales
WHERE store = 'Pune'
ORDER BY amount, day;
```

Output:

```
    day     | amount | rows_1  |  range_150  |  groups_1
------------+--------+---------+-------------+-------------
 2025-03-07 | 300.00 | 300     | 300         | 300
 2025-03-01 | 400.00 | 300,400 | 300,400     | 300,400
 2025-03-03 | 500.00 | 400,500 | 400,500     | 400,500
 2025-03-02 | 650.00 | 500,650 | 500,650,650 | 500,650,650
 2025-03-06 | 650.00 | 650,650 | 500,650,650 | 500,650,650
 2025-03-05 | 800.00 | 650,800 | 650,650,800 | 650,650,800
(6 rows)
```

- `ROWS 1 PRECEDING`: the previous physical row, so the two 650 rows get *different* frames.
- `RANGE 150 PRECEDING`: every row whose amount is between (current − 150) and current, **including all peers**. For 650: amounts 500…650 → 500, 650, 650.

- `GROUPS 1 PRECEDING`: the current peer group plus the one group before it. For 650: groups {500} and {650, 650}. For 800: groups {650, 650} and {800}.

`EXCLUDE` (PostgreSQL 11+) removes rows from the frame: `EXCLUDE CURRENT ROW`, `EXCLUDE GROUP` (current row and its peers), `EXCLUDE TIES` (peers but not the current row), `EXCLUDE NO OTHERS` (default). Example — each day compared with the average of all *other* days:

```
SELECT day, amount,
       round(avg(amount) OVER (ORDER BY day ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
                               EXCLUDE CURRENT ROW), 1) AS avg_of_other_days
FROM daily_sales
WHERE store = 'Mumbai'
ORDER BY day;
```

Output:

```
    day     | amount  | avg_of_other_days
------------+---------+-------------------
 2025-03-01 | 1200.00 |            1250.0
 2025-03-02 |  900.00 |            1300.0
 ...
 2025-03-07 | 1800.00 |            1150.0
(7 rows)
```

(Week total 8700; for 1 March: (8700 − 1200) / 6 = 1250.)

Choosing a frame

You want	Frame
Running total (deterministic)	`ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` + unique `ORDER BY`
Last *N* rows (e.g. last 7 records)	`ROWS BETWEEN N-1 PRECEDING AND CURRENT ROW`
Last *N* days of time, with gaps in data	`RANGE BETWEEN INTERVAL 'N-1 days' PRECEDING AND CURRENT ROW`
Whole partition (e.g. for `last_value`)	`ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING` (or no `ORDER BY`)
Current value and the next distinct value(s)	`GROUPS BETWEEN CURRENT ROW AND 1 FOLLOWING`

Common mistakes

- **Writing `ROWS 6 PRECEDING` for "last 7 days" when dates can be missing** — use `RANGE` with an interval.
- **Forgetting the frame applies only to some functions.** `row_number`, `rank`, `lag`, `lead`, `ntile` ignore it.
- **Frame start after frame end** (e.g. `BETWEEN CURRENT ROW AND 1 PRECEDING`) — an error.
- **Porting `GROUPS` / `EXCLUDE` / `RANGE n PRECEDING` to SQL Server or MySQL** — not supported there (see the dialect table in 8.1).

In the interview

30-second answer: "The frame says which rows around the current one an aggregate sees. `ROWS` counts physical rows, `RANGE` uses the ordering value — so ties are always included and you can say 'within 7 days' — and `GROUPS` counts groups of tied rows. The default with `ORDER BY` is `RANGE UNBOUNDED PRECEDING TO CURRENT ROW`, which is why running totals can repeat on ties."

- *Moving 7-day average with missing days?* → `avg(x) OVER (ORDER BY day RANGE BETWEEN INTERVAL '6 days' PRECEDING AND CURRENT ROW)`, or fill the calendar and use `ROWS 6 PRECEDING`.

Self-check

1. *Pune, current row 5 March: which days are in `ROWS 2 PRECEDING`? In `RANGE INTERVAL '2 days' PRECEDING`?* → 2, 3, 5 March; 3, 5 March.
2. *Which frame mode treats tied rows as separate?* → `ROWS`.
3. *What does `EXCLUDE CURRENT ROW` do?* → Removes the current row from its own frame.
4. *What is `ROWS 3 PRECEDING` short for?* → `ROWS BETWEEN 3 PRECEDING AND CURRENT ROW`.

TL;DR

Frame = which rows an aggregate sees. `ROWS` = physical rows, `RANGE` = value distance (ties included, intervals allowed), `GROUPS` = peer groups. `EXCLUDE` removes rows. Write the frame explicitly whenever it matters.

8.5 Ranking functions

What it is

Ranking functions give each row a number based on its position in the window's `ORDER BY`. They differ in **how they treat ties**.

Function	Returns	Ties
`row_number()`	1, 2, 3, … unique per row	tied rows get different numbers — **in no guaranteed order**
`rank()`	position with gaps ("Olympic" ranking)	same rank; next rank skips (1, 2, 2, 4)
`dense_rank()`	position without gaps	same rank; next rank is +1 (1, 2, 2, 3)
`percent_rank()`	(rank − 1) / (rows in partition − 1), from 0 to 1	same value
`cume_dist()`	(rows ordered before the current row + its peers) / rows in partition, from >0 to 1	same value
`ntile(n)`	bucket number 1…n, buckets as equal as possible	ties can be split across buckets

```sql
-- code/04_ranking.sql
SELECT first_name, salary,
       row_number()  OVER w AS row_number,
       rank()        OVER w AS rank,
       dense_rank()  OVER w AS dense_rank,
       round(percent_rank() OVER w :: numeric, 2) AS pct_rank,
       round(cume_dist()    OVER w :: numeric, 2) AS cume_dist,
       ntile(3)      OVER w AS ntile_3
FROM employees
WHERE dept_id IN (1, 2)
WINDOW w AS (ORDER BY salary DESC)
ORDER BY salary DESC, row_number;
```

Output:

```
 first_name |  salary   | row_number | rank | dense_rank | pct_rank | cume_dist | ntile_3
------------+-----------+------------+------+------------+----------+-----------+---------
 Vikram     | 185000.00 |          1 |    1 |          1 |     0.00 |      0.13 |       1
 Daniel     | 150000.00 |          2 |    2 |          2 |     0.14 |      0.25 |       1
 Neha       | 142000.00 |          3 |    3 |          3 |     0.29 |      0.38 |       1
 Rahul      |  98000.00 |          4 |    4 |          4 |     0.43 |      0.63 |       2
 Meera      |  98000.00 |          5 |    4 |          4 |     0.43 |      0.63 |       2
 Priya      |  72000.00 |          6 |    6 |          5 |     0.71 |      0.75 |       2
 Arjun      |  68000.00 |          7 |    7 |          6 |     0.86 |      1.00 |       3
 Sara       |  68000.00 |          8 |    7 |          6 |     0.86 |      1.00 |       3
(8 rows)
```

Reading the ties:

- Rahul and Meera tie at 98 000: `rank` 4 and 4, then Priya gets **6** (position 5 is "used up"); `dense_rank` gives 4, 4, then **5**.
- `row_number` gave Rahul 4 and Meera 5 — but nothing in `ORDER BY salary DESC` says so; it could flip. Add a tie-breaker (`ORDER BY salary DESC, emp_id`) whenever you use `row_number`.
- `percent_rank` for Priya: (6 − 1) / (8 − 1) = 0.71. `cume_dist` for Rahul/Meera: 5 of 8 rows have salary ≥ 98 000 in this descending order → 0.63.
- `ntile(3)` over 8 rows: 8 / 3 = 2 remainder 2, so the first two buckets get 3 rows and the last gets 2.

Ranking within groups — add `PARTITION BY`:

```sql
SELECT dept_id, first_name, salary,
       dense_rank() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rank_in_dept
FROM employees
WHERE dept_id IS NOT NULL
ORDER BY dept_id, rank_in_dept, first_name;
```

Output:

```
 dept_id | first_name |  salary   | rank_in_dept
---------+------------+-----------+--------------
       1 | Vikram     | 185000.00 |            1
       1 | Neha       | 142000.00 |            2
       1 | Meera      |  98000.00 |            3
       1 | Rahul      |  98000.00 |            3
 ...
       5 | Li         |  64000.00 |            1
(13 rows)
```

`ntile` for quartiles. Bucket everyone into four salary quartiles (the four equal-sized quarters of the sorted values), then summarise each bucket (window in a subquery, `GROUP BY` outside):

```sql
SELECT quartile, count(*) AS people, min(salary), max(salary)
FROM (SELECT salary, ntile(4) OVER (ORDER BY salary) AS quartile FROM employees) t
GROUP BY quartile
ORDER BY quartile;
```

Output:

```
 quartile | people |    min    |    max
----------+--------+-----------+-----------
        1 |      4 |  30000.00 |  68000.00
        2 |      4 |  68000.00 |  98000.00
        3 |      4 |  98000.00 | 142000.00
        4 |      3 | 150000.00 | 250000.00
(4 rows)
```

Note that 68 000 and 98 000 appear in two buckets each: `ntile` fills buckets by row count and **splits ties**. If tied values must stay together, compute buckets from `percent_rank` or `cume_dist` instead (or use `percentile_cont`, an ordered-set aggregate — an aggregate that reads its input in sorted order, like a median — for exact quartile boundaries — chapter 4).

Which ranking function? (the question interviewers love)

Question	Use
"Exactly one row per …" / pagination (splitting results into pages) / dedup	`row_number()` with a unique tie-breaker
"Top 3 places, ties share a place, next place skips" (sports)	`rank()`
"The 2nd highest *distinct* salary" / "top 3 salary levels"	`dense_rank()`
"Top 10 %"	`percent_rank() <= 0.1` or `cume_dist() <= 0.1` (depending on how ties should count)
"Split into 4 equal groups"	`ntile(4)`

Nth highest value — the classic: `dense_rank` makes ties count once.

```sql
-- 2nd highest distinct salary (NULL if there isn't one)
SELECT max(salary) FROM (
    SELECT salary, dense_rank() OVER (ORDER BY salary DESC) AS dr FROM employees
) t WHERE dr = 2;
```

Output:

```
    max
-----------
 185000.00
(1 row)
```

Asha, the CEO, earns 250 000, so Vikram's 185 000 is the second-highest distinct value. Wrapping in `max()` returns one row even when several people share that salary, and `NULL` rather than no rows when there is no 2nd value.

Self-check

1. *Salaries 100, 90, 90, 80: rank of 80? dense_rank?* → 4; 3.
2. *Why add `emp_id` to `ORDER BY` for `row_number`?* → To make the numbering deterministic among ties.
3. *`ntile(4)` over 10 rows — bucket sizes?* → 3, 3, 2, 2.
4. *Which function answers "second highest distinct salary"?* → `dense_rank() = 2`.

TL;DR

`row_number` (unique, arbitrary among ties), `rank` (ties share, gaps after), `dense_rank` (ties share, no gaps), `ntile(n)` (n near-equal buckets), `percent_rank`/`cume_dist` (relative position 0–1). Always think

about ties.

8.6 Value (offset) functions: LAG, LEAD, FIRST_VALUE, LAST_VALUE, NTH_VALUE

What it is

Value functions return a value **from another row** of the window.

Function	Returns	Uses the frame?
`lag(expr [, offset [, default]])`	`expr` from `offset` rows **before** (default 1)	no — relative to the partition's order
`lead(expr [, offset [, default]])`	`expr` from `offset` rows **after**	no
`first_value(expr)`	`expr` from the frame's first row	yes
`last_value(expr)`	`expr` from the frame's last row	yes
`nth_value(expr, n)`	`expr` from the frame's *n*-th row (1-based), NULL if the frame is shorter	yes

When there is no such row (the first row has no "previous"), `lag`/`lead` return `NULL`, or the `default` you pass.

Day-over-day change:

```sql
-- code/05_value_functions.sql
SELECT store, day, amount,
       lag(amount)  OVER w          AS prev_day,
       amount - lag(amount) OVER w  AS change,
       lead(amount) OVER w          AS next_day,
       lag(amount, 2, 0) OVER w     AS two_days_ago_or_0
FROM daily_sales
WHERE day <= '2025-03-04'
WINDOW w AS (PARTITION BY store ORDER BY day)
ORDER BY store, day;
```

Output:

```
 store  |    day     | amount  | prev_day | change  | next_day | two_days_ago_or_0
--------+------------+---------+----------+---------+----------+-------------------
 Mumbai | 2025-03-01 | 1200.00 |   (null) |  (null) |   900.00 |                 0
 Mumbai | 2025-03-02 |  900.00 |  1200.00 | -300.00 |  1500.00 |                 0
 ...
 Mumbai | 2025-03-04 | 1500.00 |  1500.00 |    0.00 |   (null) |            900.00
 Pune   | 2025-03-01 |  400.00 |   (null) |  (null) |   650.00 |                 0
 Pune   | 2025-03-02 |  650.00 |   400.00 |  250.00 |   500.00 |                 0
 Pune   | 2025-03-03 |  500.00 |   650.00 | -150.00 |   (null) |            400.00
(7 rows)
```

`PARTITION BY store` stops `lag` from reaching into the other store's rows: Pune's first day has no previous day even though Mumbai's last row came before it. The `WHERE` runs before the window, so `lead` on 4 March sees nothing (5 March was filtered out).

Note that `lag` means "previous **row**", not "previous **day**": in Pune, the row before 5 March is 3 March. If you need the previous *calendar* day, fill gaps first or check the date difference (8.9).

The `last_value` trap. With `ORDER BY` in the window, the default frame ends at the current row, so `last_value` just returns the current row:

```
SELECT day, amount,
       first_value(amount) OVER (ORDER BY day)    AS first_amt,
       last_value(amount)  OVER (ORDER BY day)    AS last_default_frame,
       last_value(amount)  OVER (ORDER BY day
            ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS last_whole_partition,
       nth_value(amount, 2) OVER (ORDER BY day
            ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS second_amt
FROM daily_sales
WHERE store = 'Pune'
ORDER BY day;
```

Output:

```
    day     | amount | first_amt | last_default_frame | last_whole_partition | second_amt
------------+--------+-----------+--------------------+----------------------+------------
 2025-03-01 | 400.00 |    400.00 |             400.00 |               300.00 |     650.00
 2025-03-02 | 650.00 |    400.00 |             650.00 |               300.00 |     650.00
 2025-03-03 | 500.00 |    400.00 |             500.00 |               300.00 |     650.00
 ...
 2025-03-07 | 300.00 |    400.00 |             300.00 |               300.00 |     650.00
(6 rows)
```

Fix: extend the frame to `UNBOUNDED FOLLOWING`, or use `first_value` with the order reversed (`first_value(amount) OVER (ORDER BY day DESC)`), which needs no frame.

`first_value(first_name) OVER (PARTITION BY dept_id ORDER BY salary DESC)` puts each department's top earner beside every employee — handy for "gap to the leader" columns (the file shows it).

Time between events — days between a customer's consecutive orders (a standard retention metric, i.e. a measure of how often customers come back):

```
SELECT customer_id, order_id, placed_at::date AS placed,
       placed_at::date - lag(placed_at::date) OVER (PARTITION BY customer_id ORDER BY placed_at) AS
days_since_prev
FROM orders
WHERE customer_id IN (1, 2, 5)
ORDER BY customer_id, placed_at;
```

Output:

```
 customer_id | order_id |   placed   | days_since_prev
-------------+----------+------------+-----------------
           1 |      101 | 2025-01-03 |          (null)
           1 |      104 | 2025-02-11 |              39
           1 |      113 | 2025-06-02 |             111
           2 |      102 | 2025-01-17 |          (null)
           2 |      108 | 2025-04-01 |              74
 ...
(7 rows)
```

Subtracting two `date`s gives an integer number of days in PostgreSQL.

Dialect notes

- `IGNORE NULLS` (e.g. `lag(x) IGNORE NULLS OVER (...)`, "the last non-null value") is in the SQL standard and supported by Oracle, SQL Server 2022, BigQuery and Snowflake (two cloud data-warehouse products). PostgreSQL gains it in **version 19** (in beta as of September 2026); PostgreSQL 14–18 and

MySQL lack it. There, emulate it with a running `count(x)` as a group key and `first_value` within each group, or a correlated subquery.

- `nth_value(...) FROM LAST` (counting from the end of the frame) exists in Oracle and a few warehouses, not in PostgreSQL, MySQL or SQL Server.

Common mistakes

- `last_value` without a full frame; `lag` without `PARTITION BY` (crosses groups); `lag` treated as "previous period" when periods can be missing.
- The first row's `amount - lag(amount)` is `NULL`; pass a `default` or use `coalesce` if you want 0.

In the interview

> **30-second answer:** "`LAG` and `LEAD` read a column from a previous or next row in the window's order, per partition, which replaces a self-join on 'row n−1'. `FIRST_VALUE`, `LAST_VALUE` and `NTH_VALUE` read from positions in the frame — and because the default frame ends at the current row, `LAST_VALUE` needs `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`."

- *Year-over-year growth per product?* → `lag(revenue) OVER (PARTITION BY product ORDER BY year)`, then `(revenue - prev) / prev`.
- *How did people do this before window functions?* → A self-join on `t2.day = t1.day - 1` (breaks with gaps) or a correlated subquery `(SELECT ... WHERE t2.day < t1.day ORDER BY day DESC LIMIT 1)`.

Self-check

1. *What does `lag(x, 1, 0)` return on the first row?* → 0.
2. *Why does `last_value(x) OVER (ORDER BY d)` equal `x`?* → The default frame ends at the current row (plus peers).
3. *Which of these ignore the frame: `lag`, `first_value`, `lead`, `nth_value`?* → `lag` and `lead`.

TL;DR

`lag` / `lead` = previous/next row per partition (with optional offset and default).
`first_value` / `last_value` / `nth_value` read the frame — give `last_value` a full frame.

8.7 Running totals, moving averages and growth

What it is

The everyday analytics calculations are just aggregate window functions with the right frame:

Metric	Window
Running total	`sum(x) OVER (PARTITION BY g ORDER BY t ROWS UNBOUNDED PRECEDING)`
Moving average of last 3 rows	`avg(x) OVER (PARTITION BY g ORDER BY t ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)`
Cumulative share	running total / `sum(x) OVER (PARTITION BY g)`
Share of total	`x / sum(x) OVER (PARTITION BY g)`
Growth vs previous period	`x / lag(x) OVER (ORDER BY t) - 1`

```sql
-- code/06_running_and_moving.sql
SELECT store, day, amount,
       sum(amount) OVER (PARTITION BY store ORDER BY day
                         ROWS UNBOUNDED PRECEDING)                      AS running_total,
       round(avg(amount) OVER (PARTITION BY store ORDER BY day
                         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 1)  AS moving_avg_3,
       count(*) OVER (PARTITION BY store ORDER BY day
                         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)      AS rows_in_avg,
       round(100.0 * sum(amount) OVER (PARTITION BY store ORDER BY day ROWS UNBOUNDED PRECEDING)
                   / sum(amount) OVER (PARTITION BY store), 1)          AS pct_of_week_so_far
FROM daily_sales
ORDER BY store, day;
```

Output:

```
 store  |    day     | amount  | running_total | moving_avg_3 | rows_in_avg | pct_of_week_so_far
--------+------------+---------+---------------+--------------+-------------+--------------------
 Mumbai | 2025-03-01 | 1200.00 |       1200.00 |       1200.0 |           1 |               13.8
 Mumbai | 2025-03-02 |  900.00 |       2100.00 |       1050.0 |           2 |               24.1
 Mumbai | 2025-03-03 | 1500.00 |       3600.00 |       1200.0 |           3 |               41.4
 ...
 Mumbai | 2025-03-07 | 1800.00 |       8700.00 |       1200.0 |           3 |              100.0
 ...
 Pune   | 2025-03-03 |  500.00 |       1550.00 |        516.7 |           3 |               47.0
 Pune   | 2025-03-05 |  800.00 |       2350.00 |        650.0 |           3 |               71.2
 ...
 Pune   | 2025-03-07 |  300.00 |       3300.00 |        583.3 |           3 |              100.0
(13 rows)
```

Two things to notice:

1. The first two moving averages use only 1 and 2 rows (`rows_in_avg`) — a "warm-up" period. Averages over fewer points than intended can mislead on a chart.
2. Pune's 5 March average (650.0) uses 2, 3 and 5 March because the frame is `ROWS` — see 8.4 for the `RANGE` alternative.

To hide warm-up rows, show the average only when the frame is full: `CASE WHEN count(*) OVER w = 3 THEN avg(amount) OVER w END` (in `code/06_running_and_moving.sql`).

Windows on top of GROUP BY

Window functions run **after** `GROUP BY`, so they can work on the grouped rows. That's how you get "monthly revenue and the running total of monthly revenue" in one query. The inner `sum` is the normal aggregate (per month); the outer `sum(...) OVER` adds up those monthly sums:

```sql
SELECT date_trunc('month', o.placed_at)::date AS month,
       sum(oi.quantity * oi.unit_price * (1 - oi.discount))::numeric(10,2) AS revenue,
       sum(sum(oi.quantity * oi.unit_price * (1 - oi.discount)))
           OVER (ORDER BY date_trunc('month', o.placed_at)::date)::numeric(10,2) AS running_revenue
FROM orders o JOIN order_items oi USING (order_id)
WHERE o.status <> 'cancelled'
GROUP BY 1
ORDER BY 1;
```

Output:

```
   month    | revenue | running_revenue
------------+---------+-----------------
 2025-01-01 | 1871.90 |         1871.90
 2025-02-01 |  811.49 |         2683.39
 ...
 2025-06-01 | 1516.60 |         7403.62
(6 rows)
```

`sum(sum(...))` looks odd but is correct. It's often clearer to aggregate in a CTE (Common Table Expression — a named subquery written with `WITH`, chapter 7) first and apply windows in the outer query — as here for **month-over-month growth**:

```sql
WITH monthly AS (
    SELECT date_trunc('month', o.placed_at)::date AS month,
           sum(oi.quantity * oi.unit_price * (1 - oi.discount)) AS revenue
    FROM orders o JOIN order_items oi USING (order_id)
    WHERE o.status <> 'cancelled'
    GROUP BY 1
)
SELECT month, round(revenue, 2) AS revenue,
       round(lag(revenue) OVER (ORDER BY month), 2) AS prev_month,
       round(100.0 * (revenue / lag(revenue) OVER (ORDER BY month) - 1), 1) AS mom_growth_pct
FROM monthly
ORDER BY month;
```

Output:

```
   month    | revenue | prev_month | mom_growth_pct
------------+---------+------------+----------------
 2025-01-01 | 1871.90 |     (null) |         (null)
 2025-02-01 |  811.49 |    1871.90 |          -56.6
 2025-03-01 |  804.95 |     811.49 |           -0.8
 ...
 2025-06-01 | 1516.60 |     249.93 |          506.8
(6 rows)
```

If a month could have *no* orders, generate the months first (chapter 7) — otherwise `lag` would compare with two months back. Guard divisions with `NULLIF(prev, 0)` if the previous value can be zero.

Real-world examples

Account balance after each transaction (running `sum` ordered by time and id), 7-day moving average of daily active users, stock level as a running sum of `+received −shipped`.

TL;DR

Running total = `sum` + `ROWS UNBOUNDED PRECEDING`; moving average = `avg` + `ROWS n PRECEDING` (watch warm-up rows and gaps); growth = value / `lag(value)` − 1. Windows run after `GROUP BY`, so `sum(sum(x)) OVER (...)` works — a CTE is often clearer.

8.8 Top-N per group and de-duplication

The problem

"Top 2 earners in each department", "latest order per customer", "best-selling product per category" — these **top-N-per-group** questions are the most common window-function interview problem. `LIMIT` can't do

it (it limits the whole result, not each group), and `GROUP BY` returns only the aggregated value, not the rest of the row.

Why you need a subquery or CTE

Window functions are computed *after* `WHERE`, so you cannot filter on them in the same query level:

```sql
-- code/07_top_n_per_group.sql
SELECT first_name, salary FROM employees
WHERE row_number() OVER (ORDER BY salary DESC) <= 3;
```

Output:

```
ERROR:  window functions are not allowed in WHERE
LINE 2: WHERE row_number() OVER (ORDER BY salary DESC) <= 3;
              ^
```

(They're not allowed in `GROUP BY` or `HAVING` either.) The pattern is always: **compute the rank in a CTE or subquery, filter in the outer query.**

How it works

1. In a CTE, add `row_number()` / `rank()` / `dense_rank()` `OVER (PARTITION BY group ORDER BY measure DESC)`.
2. In the outer query, `WHERE rank_col <= N`.
3. Pick the ranking function according to how ties should be treated.

```sql
WITH ranked AS (
    SELECT d.name AS dept, e.first_name, e.salary,
           row_number() OVER (PARTITION BY e.dept_id ORDER BY e.salary DESC, e.emp_id) AS rn,
           rank()       OVER (PARTITION BY e.dept_id ORDER BY e.salary DESC)            AS rnk,
           dense_rank() OVER (PARTITION BY e.dept_id ORDER BY e.salary DESC)            AS drnk
    FROM employees e JOIN departments d USING (dept_id)
)
SELECT dept, first_name, salary, rn, rnk, drnk
FROM ranked
WHERE dept IN ('Engineering', 'Sales') AND rnk <= 3
ORDER BY dept, rn;
```

Output:

```
    dept     | first_name |  salary   | rn | rnk | drnk
-------------+------------+-----------+----+-----+------
 Engineering | Vikram     | 185000.00 |  1 |   1 |    1
 ...
 Engineering | Rahul      |  98000.00 |  3 |   3 |    3
 Engineering | Meera      |  98000.00 |  4 |   3 |    3
 Sales       | Daniel     | 150000.00 |  1 |   1 |    1
 Sales       | Priya      |  72000.00 |  2 |   2 |    2
 Sales       | Arjun      |  68000.00 |  3 |   3 |    3
 Sales       | Sara       |  68000.00 |  4 |   3 |    3
(8 rows)
```

The three filters give different answers when there are ties — here for Sales:

```sql
-- ranked = Sales only; then:
SELECT 'row_number <= 3' AS rule, string_agg(first_name, ', ' ORDER BY rn) AS kept FROM ranked WHERE rn
<= 3
UNION ALL
SELECT 'rank <= 3',       string_agg(first_name, ', ' ORDER BY rn) FROM ranked WHERE rnk <= 3
UNION ALL
SELECT 'dense_rank <= 2', string_agg(first_name, ', ' ORDER BY rn) FROM ranked WHERE drnk <= 2;
```

Output:

```
      rule       |            kept
-----------------+----------------------------
 row_number <= 3 | Daniel, Priya, Arjun
 rank <= 3       | Daniel, Priya, Arjun, Sara
 dense_rank <= 2 | Daniel, Priya
(3 rows)
```

Filter	Meaning	Rows per group
`row_number() <= N`	exactly N rows; ties broken by your tie-breaker	exactly N (or fewer)
`rank() <= N`	everyone whose position is within the top N; ties included	can exceed N
`dense_rank() <= N`	everyone having one of the top N *distinct* values	can far exceed N

Ask in an interview: "If two people tie for 3rd, should both be returned?"

Latest row per key (de-duplication). Same pattern with `N = 1`, ordered newest first:

```sql
SELECT customer_id, order_id, placed_at, status
FROM (
    SELECT o.*, row_number() OVER (PARTITION BY customer_id ORDER BY placed_at DESC) AS rn
    FROM orders o
) t
WHERE rn = 1
ORDER BY customer_id;
```

Output:

```
 customer_id | order_id |      placed_at      |  status
-------------+----------+---------------------+-----------
           1 |      113 | 2025-06-02 10:30:00 | pending
           2 |      108 | 2025-04-01 12:00:00 | delivered
           3 |      111 | 2025-05-06 07:55:00 | cancelled
 ...
           8 |      109 | 2025-04-09 16:45:00 | delivered
(8 rows)
```

Find duplicates to delete — number the rows within each duplicate group, and everything with `rn > 1` is an extra copy. The duplicate Anita Shah (chapter 1's messy email) is found after normalising the email:

```sql
WITH dups AS (
    SELECT customer_id,
           row_number() OVER (PARTITION BY lower(trim(email)) ORDER BY customer_id) AS rn
    FROM customers
    WHERE email IS NOT NULL
)
SELECT customer_id AS would_delete FROM dups WHERE rn > 1;
```

Output:

```
 would_delete
--------------
            8
(1 row)
```

To actually delete: `DELETE FROM customers WHERE customer_id IN (SELECT customer_id FROM dups WHERE rn > 1)` (after re-pointing that customer's orders — chapter 9 covers `DELETE` and foreign keys — columns whose values must match a key in another table).

Alternatives

Approach	Top-1	Top-N	Notes
Window + filter (above)	yes	yes	portable, flexible about ties
`DISTINCT ON (group) ... ORDER BY group, measure DESC`	yes	no	PostgreSQL only, concise
`LATERAL (... ORDER BY ... LIMIT N)` per group (ch. 6)	yes	yes	fastest when an index (sorted lookup structure) on (group, measure) exists and groups are few
Correlated subquery `WHERE salary = (SELECT max(...) ...)`	yes (with ties)	awkward	classic pre-window approach
`QUALIFY rn <= N`	yes	yes	filter on windows without a subquery — Snowflake, BigQuery, DuckDB, Teradata; **not** PostgreSQL / MySQL / SQL Server

For top-1, `SELECT DISTINCT ON (dept_id) dept_id, first_name, salary FROM employees WHERE dept_id IS NOT NULL ORDER BY dept_id, salary DESC, emp_id` returns each department's best-paid person in one short query (the file runs it).

In the interview

30-second answer: "Rank rows inside each group with a window function in a CTE — `PARTITION BY` the group, `ORDER BY` the measure descending — then filter `rank <= N` in the outer query, because window functions can't appear in `WHERE`. I'd choose `row_number`, `rank` or `dense_rank` depending on whether ties should be included, and add a tie-breaker for determinism."

- *Top 3 salaries per department with ties?* → `dense_rank() <= 3` (three distinct salary levels) or `rank() <= 3` (three places).
- *How to make it fast on a big table?* → Index `(dept_id, salary DESC)`; PostgreSQL can then read each partition pre-sorted; for few groups a `LATERAL ... LIMIT N` uses the index directly.

Self-check

1. *Why can't you write `WHERE rank() OVER (...) = 1`?* → Window functions are evaluated after `WHERE`.
2. *Top 2 per group, ties allowed to exceed 2 — which function?* → `rank()` (or `dense_rank()` for distinct values).
3. *How do you keep only the newest row per `user_id`?* → `row_number() OVER (PARTITION BY user_id ORDER BY created_at DESC, id DESC) = 1` in a subquery.

TL;DR

Rank in a CTE, filter outside. `row_number` = exactly N, `rank` = N places with ties, `dense_rank` = N distinct values. `rn = 1` newest-first is the standard dedup. PostgreSQL shortcut for top-1: `DISTINCT ON`.

8.9 Gaps and islands

What it is

An **island** is a run of consecutive values (consecutive days a user logged in, consecutive seat numbers that are free, consecutive readings above a threshold). A **gap** is a missing stretch between islands. "Longest login streak", "find missing invoice numbers" and "group events into sessions" are all gaps-and-islands problems.

How it works — the row-number trick

For a run of consecutive dates, the date goes up by 1 each row and so does `row_number()`. Their **difference is therefore constant within a run** and changes when a day is skipped. That constant identifies the island.

1. Number the rows per user in date order: `rn = row_number() OVER (PARTITION BY user_id ORDER BY login_date)`.
2. Compute `grp = login_date - rn` (a date minus an integer is a date).
3. `GROUP BY user_id, grp`: each group is one island; `min` / `max` / `count` describe it.

```sql
-- code/08_gaps_and_islands.sql
SELECT user_id, login_date,
       row_number() OVER (PARTITION BY user_id ORDER BY login_date) AS rn,
       login_date - row_number() OVER (PARTITION BY user_id ORDER BY login_date)::int AS grp
FROM logins
WHERE user_id = 1
ORDER BY login_date;
```

Output:

```
 user_id | login_date | rn |    grp
---------+------------+----+------------
       1 | 2025-05-01 |  1 | 2025-04-30
       1 | 2025-05-02 |  2 | 2025-04-30
       1 | 2025-05-03 |  3 | 2025-04-30
       1 | 2025-05-06 |  4 | 2025-05-02
       1 | 2025-05-07 |  5 | 2025-05-02
       1 | 2025-05-10 |  6 | 2025-05-04
(6 rows)
```

The value of `grp` has no meaning of its own — it's just a label shared by the rows of one run. (`row_number()` returns `bigint`; the `::int` cast is needed because PostgreSQL defines `date - integer`, not `date - bigint`.)

Each island (streak):

```sql
WITH marked AS (
    SELECT user_id, login_date,
           login_date - row_number() OVER (PARTITION BY user_id ORDER BY login_date)::int AS grp
    FROM logins
)
SELECT user_id, min(login_date) AS streak_start, max(login_date) AS streak_end,
       count(*) AS days
FROM marked
GROUP BY user_id, grp
ORDER BY user_id, streak_start;
```

Output:

```
 user_id | streak_start | streak_end | days
---------+--------------+------------+------
       1 | 2025-05-01   | 2025-05-03 |    3
       1 | 2025-05-06   | 2025-05-07 |    2
       1 | 2025-05-10   | 2025-05-10 |    1
       2 | 2025-05-02   | 2025-05-05 |    4
       2 | 2025-05-09   | 2025-05-10 |    2
(5 rows)
```

Longest streak per user — one more step (`DISTINCT ON` keeps the top row per user):

```
WITH marked AS (...same as above...),
streaks AS (
    SELECT user_id, grp, count(*) AS days, min(login_date) AS from_day
    FROM marked GROUP BY user_id, grp
)
SELECT DISTINCT ON (user_id) user_id, days AS longest_streak, from_day
FROM streaks
ORDER BY user_id, days DESC, from_day;
```

Output:

```
 user_id | longest_streak |  from_day
---------+----------------+------------
       1 |              3 | 2025-05-01
       2 |              4 | 2025-05-02
(2 rows)
```

If the data can contain the same date twice for a user, de-duplicate first (or use `dense_rank` instead of `row_number`), otherwise the trick breaks.

Gaps with `lead`. Compare each row with the next one; a difference greater than 1 is a gap:

```
SELECT user_id,
       login_date + 1              AS gap_start,
       next_login - 1              AS gap_end,
       next_login - login_date - 1 AS missing_days
FROM (
    SELECT user_id, login_date,
           lead(login_date) OVER (PARTITION BY user_id ORDER BY login_date) AS next_login
    FROM logins
) t
WHERE next_login - login_date > 1
ORDER BY user_id, gap_start;
```

Output:

```
 user_id | gap_start  |  gap_end   | missing_days
---------+------------+------------+--------------
       1 | 2025-05-04 | 2025-05-05 |            2
       1 | 2025-05-08 | 2025-05-09 |            2
       2 | 2025-05-06 | 2025-05-08 |            3
(3 rows)
```

The same query with integer ids finds missing invoice or ticket numbers.

The general method: flag and running sum

The row-number trick needs "consecutive" to mean "+1". For any other rule — events less than 30 minutes apart, temperature above a threshold, same status as the previous row — use two steps:

1. **Flag** the first row of each island: `CASE WHEN <this row does NOT continue the previous one> THEN 1 ELSE 0 END`, using `lag`.
2. **Running sum** of the flags: `sum(flag) OVER (ORDER BY ...)` numbers the islands 1, 2, 3, …

Here: **sessionization** — a new session starts when more than 30 minutes have passed since the previous event:

```
WITH ev(user_id, at) AS (VALUES
    (7, TIMESTAMP '2025-05-01 09:00'), (7, '2025-05-01 09:10'), (7, '2025-05-01 09:25'),
    (7, '2025-05-01 11:00'), (7, '2025-05-01 11:05'),
    (7, '2025-05-01 15:30')
), flagged AS (
    SELECT user_id, at,
           CASE WHEN at - lag(at) OVER (PARTITION BY user_id ORDER BY at) <= INTERVAL '30 minutes'
                THEN 0 ELSE 1 END AS new_session
    FROM ev
)
SELECT user_id, at, new_session,
       sum(new_session) OVER (PARTITION BY user_id ORDER BY at) AS session_no
FROM flagged
ORDER BY at;
```

Output:

```
 user_id |         at          | new_session | session_no
---------+---------------------+-------------+------------
       7 | 2025-05-01 09:00:00 |           1 |          1
       7 | 2025-05-01 09:10:00 |           0 |          1
       7 | 2025-05-01 09:25:00 |           0 |          1
       7 | 2025-05-01 11:00:00 |           1 |          2
       7 | 2025-05-01 11:05:00 |           0 |          2
       7 | 2025-05-01 15:30:00 |           1 |          3
(6 rows)
```

The first event has no previous one: `lag` is `NULL`, the comparison is `NULL` (not true), so the `CASE` falls to `ELSE 1` — a new session, as intended. Group by `(user_id, session_no)` to get session start, end and length. (`VALUES` as a CTE creates inline sample rows; the first row's `TIMESTAMP` literal sets the column type.)

Real-world examples

App streaks and contribution graphs, web-analytics sessions (30 minutes of inactivity is a common cut-off), periods when CPU (the processor) load stayed above 90 %, missing cheque or invoice numbers.

In the interview

30-second answer: "For consecutive dates or integers, subtract `row_number()` from the value; the difference is constant within each run, so grouping by it gives the islands with `min`, `max`, `count`. For general rules, flag each row that starts a new island using `lag`, then a running `sum` of the flags gives an island id. Gaps come from `lead(value) - value > 1`."

- *Users who logged in 3+ consecutive days?* → Islands query, then `HAVING count(*) >= 3`, select distinct users.
- *What if a user logs in twice on the same day?* → De-duplicate to one row per day first, or use `dense_rank`.

Self-check

1. *Why is* `login_date - row_number()` *constant inside a streak?* → Both increase by exactly 1 per row while the days are consecutive.
2. *What does the running sum of "new island" flags give?* → A sequential island number per row.
3. *How do you list missing numbers between consecutive ids?* → `lead(id) - id > 1` → gap from `id + 1` to `lead(id) - 1`.

TL;DR

Islands: `value - row_number()` is a run label → `GROUP BY` it. General rule: flag starts with `lag`, running `sum` of flags = island id. Gaps: `lead(v) - v > 1`.

8.10 Named windows, FILTER, evaluation order and performance

The WINDOW clause

When several functions share a window, define it once with `WINDOW name AS (...)` (after `HAVING`, before `ORDER BY`) and write `OVER name`. A window can also be based on another: `OVER (w ROWS 2 PRECEDING)` adds a frame to named window `w` (the base window must not have its own frame).

```sql
-- code/09_named_windows_filter_plan.sql
SELECT store, day, amount,
       sum(amount) OVER w_run    AS running_total,
       max(amount) OVER w_run    AS best_so_far,
       rank()      OVER w_amount AS amount_rank
FROM daily_sales
WHERE store = 'Pune'
WINDOW w_run    AS (PARTITION BY store ORDER BY day),
       w_amount AS (PARTITION BY store ORDER BY amount DESC)
ORDER BY day;
```

Output:

```
 store |    day     | amount | running_total | best_so_far | amount_rank
-------+------------+--------+---------------+-------------+-------------
 Pune  | 2025-03-01 | 400.00 |        400.00 |      400.00 |           5
 Pune  | 2025-03-02 | 650.00 |       1050.00 |      650.00 |           2
 Pune  | 2025-03-03 | 500.00 |       1550.00 |      650.00 |           4
 Pune  | 2025-03-05 | 800.00 |       2350.00 |      800.00 |           1
 Pune  | 2025-03-06 | 650.00 |       3000.00 |      800.00 |           2
 Pune  | 2025-03-07 | 300.00 |       3300.00 |      800.00 |           6
(6 rows)
```

`max(...) OVER w_run` is a running maximum ("best day so far"). The `WINDOW` clause is supported by PostgreSQL, MySQL 8, SQLite and SQL Server 2022 (not earlier SQL Server).

FILTER on window aggregates

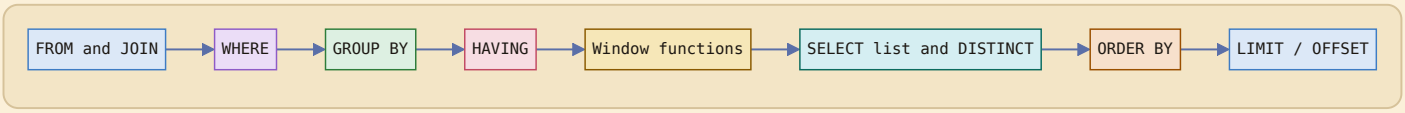
The `FILTER (WHERE ...)` clause from chapter 4 also works on aggregate window functions (PostgreSQL and SQLite; elsewhere use `count(CASE WHEN cond THEN 1 END) OVER (...)`):

```sql
SELECT day, amount,
       count(*) FILTER (WHERE amount >= 1500) OVER (ORDER BY day) AS big_days_so_far
FROM daily_sales
WHERE store = 'Mumbai'
ORDER BY day;
```

Output:

```
    day     | amount  | big_days_so_far
------------+---------+-----------------
 2025-03-01 | 1200.00 |               0
 2025-03-02 |  900.00 |               0
 2025-03-03 | 1500.00 |               1
 2025-03-04 | 1500.00 |               2
 2025-03-05 |  700.00 |               2
 2025-03-06 | 1100.00 |               2
 2025-03-07 | 1800.00 |               3
(7 rows)
```

Where windows fit in the logical order



Consequences:

- **`WHERE` changes the window.** `rank() OVER (ORDER BY salary DESC)` with `WHERE dept_id = 1` ranks only Engineering (Vikram 1, Neha 2, Rahul and Meera 3). For company-wide ranks shown only for Engineering, rank in a subquery **without** the filter and filter outside.
- Windows **can't** be used in `WHERE`, `GROUP BY` or `HAVING` (they don't exist yet) → wrap in a subquery/CTE (8.8).
- Windows **can** be used in `ORDER BY` (`ORDER BY rank() OVER (...)`) and can take aggregates as arguments in grouped queries (8.7).
- `DISTINCT` runs after windows, so `SELECT DISTINCT dept_id, count(*) OVER (PARTITION BY dept_id)` works (one row per department).
- `LIMIT` runs last, so `row_number() ... LIMIT 5` numbers *all* rows first.

Performance

PostgreSQL evaluates windows with a **WindowAgg** node (one step in the execution plan that `EXPLAIN` prints) that reads rows sorted by the `PARTITION BY` + `ORDER BY` keys:

```
EXPLAIN (COSTS OFF)
SELECT store, day, sum(amount) OVER (PARTITION BY store ORDER BY day)
FROM daily_sales;

EXPLAIN (COSTS OFF)
SELECT first_name, rank() OVER (PARTITION BY dept_id ORDER BY salary DESC)
FROM employees;
```

Output:

```
                        QUERY PLAN
-----------------------------------------------------
 WindowAgg
   ->  Index Scan using daily_sales_pkey on daily_sales
(2 rows)

                QUERY PLAN
---------------------------------------
 WindowAgg
   ->  Sort
         Sort Key: dept_id, salary DESC
         ->  Seq Scan on employees
(4 rows)
```

- In the first plan the primary key `(store, day)` (the columns that uniquely identify each row, which PostgreSQL backs with an index) already delivers rows in the needed order, so there's **no sort**. In the second, PostgreSQL must sort by `dept_id, salary DESC`.
- **Index tip:** an index on `(partition columns, order columns)` can remove the sort — e.g. `CREATE INDEX ON employees (dept_id, salary DESC)` (chapter 12).
- Windows with **different** `PARTITION BY`/`ORDER BY` need separate sorts; windows sharing the same definition are computed in one pass — another reason to reuse a named window.
- Sliding frames with `sum`/`count`/`avg` are computed incrementally (by adjusting the previous row's result rather than starting over), but `min`/`max` over a moving frame (and frames with `EXCLUDE`) may need re-scanning the frame, which gets expensive for large frames.
- A window over millions of rows sorts all of them; filter in `WHERE` first when the logic allows.

Chapter 8 TL;DR

- **Window function** = `f(...) OVER (PARTITION BY … ORDER BY … frame)`: computed per row over related rows; **row count never changes**.
- `PARTITION BY` = groups without collapsing → share of total, difference from group average.
- `ORDER BY` **in** `OVER` makes aggregates running; the default frame `RANGE UNBOUNDED PRECEDING … CURRENT ROW` includes ties — use a unique order or `ROWS`.
- **Frames**: `ROWS` (physical rows), `RANGE` (value distance, intervals, ties included), `GROUPS` (peer groups), `EXCLUDE` — PostgreSQL 11+ for offsets/`GROUPS`/`EXCLUDE`.
- **Ranking**: `row_number` (unique), `rank` (gaps), `dense_rank` (no gaps), `ntile`, `percent_rank`, `cume_dist`. Always decide how ties behave.
- **Value functions**: `lag`/`lead` (ignore frame), `first_value`/`last_value`/`nth_value` (use frame — give `last_value` a full frame). `IGNORE NULLS` only from PostgreSQL 19.
- **Patterns**: running total, moving average, growth vs `lag`, top-N per group (rank in CTE, filter outside), dedup (`row_number() = 1`), gaps & islands (`value − row_number()`, or flag + running sum).
- **Order of evaluation**: after `WHERE`/`GROUP BY`/`HAVING`, before `DISTINCT`/`ORDER BY`/`LIMIT`. No `QUALIFY` in PostgreSQL.
- **Performance**: one sort per distinct window; an index on (partition, order) columns can remove it.

Next: chapter 9 — DDL, DML and constraints: creating and changing tables, keys and constraints, and inserting, updating and deleting data safely.

Chapter 9 — DDL, DML and Constraints: Building Tables and Changing Data Safely

So far the book has only *read* data. This chapter is about creating the structures that hold data and changing what is in them:

- **DDL — Data Definition Language**: statements that define structure: `CREATE`, `ALTER`, `DROP`, `TRUNCATE` (and `CREATE INDEX`).
- **Keys and constraints**: rules attached to tables — primary and foreign keys, `UNIQUE`, `CHECK`, `NOT NULL`, `DEFAULT` — enforced on every change, whichever application writes.
- **DML — Data Manipulation Language**: statements that change rows: `INSERT`, `UPDATE`, `DELETE`, plus PostgreSQL's "insert or update" (`ON CONFLICT`) and `RETURNING`.
- **Backup and restore**: copying a database and getting data back after a mistake — run for real with `pg_dump` and `pg_restore` (PostgreSQL's command-line programs that write a database out to a file and load it back in).

Designing tables (entities, relationships, normal forms — sets of rules for keeping each fact in only one place) is chapter 10 and chapter 11; here we focus on the statements.

What you'll learn

#	Topic	You will be able to…
9.1	Data types	pick the right type for numbers, text, dates, booleans, and know what each rejects
9.2	`CREATE`	create databases, schemas, tables (also from queries and as copies), temporary tables
9.3	Keys and constraints	explain super/candidate/primary/alternate/foreign/composite/unique keys and use every constraint, including `ON DELETE` actions
9.4	Generated keys	use identity columns, sequences, `serial`, UUIDs and generated columns
9.5	`ALTER`	add/rename/drop columns, change types, add constraints safely on live tables
9.6	`DROP` and `TRUNCATE`	remove tables or empty them, and compare with `DELETE`
9.7	Indexes (brief)	create/drop indexes; clustered vs non-clustered
9.8	`INSERT`	insert single/multiple rows, defaults, `INSERT … SELECT`, `RETURNING`
9.9	`UPDATE`	update with conditions, expressions, `CASE`, other tables
9.10	`DELETE`	delete with conditions, other tables, soft deletes
9.11	Upsert and `MERGE`	`INSERT … ON CONFLICT`, and what `MERGE` (PostgreSQL 15+) looks like
9.12	Backup and restore	logical vs physical backups; `pg_dump` / `pg_restore`; other databases' commands

Setup. Examples run on the `company` sample database (chapter 1, section 1.11); each file in `code/` starts from a fresh copy and creates any extra tables it needs. Errors are shown on purpose — each `ERROR:` in an output block is the database refusing something.

9.1 Data types

What it is

Every column has a **data type** saying what kind of value it holds (whole number, decimal, text, date…). The type decides storage, which operations work, how values sort and — just as important — which values are **rejected**. A type is the first and cheapest constraint.

The common types

Kind	PostgreSQL type	Holds	Use for	MySQL / SQL Server notes
Integer	`smallint` (2 bytes)	−32 768 … 32 767	small ranges: ranks, ratings	same names
	`integer` / `int` (4 bytes)	about ±2.1 billion	counts, most ids	same
	`bigint` (8 bytes)	about ±9.2 × 10^{18}	large ids, populations, byte counts	same
Exact decimal	`numeric(p, s)` / `decimal(p, s)`	*p* digits in total, *s* after the point	**money**, anything that must add up exactly	same
Approximate	`real`, `double precision`	binary floating point (fast but approximate)	measurements, science, ML (machine-learning) features	`FLOAT` / `DOUBLE`; SQL Server `float`
Fixed text	`char(n)`	exactly *n* characters, padded with spaces	fixed codes (rarely worth it)	same
Variable text	`varchar(n)`	up to *n* characters	text with a real maximum length	SQL Server: `nvarchar(n)` for Unicode (text in any alphabet)
	`text`	any length	most text in PostgreSQL	MySQL `TEXT` has limits and index restrictions
Date / time	`date`	a calendar day	birthdays, due dates	same
	`time`	time of day	opening hours	same
	`timestamp`	date + time, **no** time zone	local "wall clock" times	MySQL `DATETIME`
	`timestamptz`	an exact instant (stored in UTC — the world's reference clock — shown in the session's time zone; a *session* is your current connection)	event times, created_at	MySQL `TIMESTAMP` (converted to UTC); SQL Server `datetimeoffset`
Year only	(none) — use `smallint` + `CHECK`, or `date`			MySQL has `YEAR`
Boolean	`boolean`	true / false / NULL	flags	MySQL `BOOLEAN` = `TINYINT(1)`; SQL Server `bit`
Other useful	`uuid`, `jsonb`, arrays, `inet`, ranges		see chapter 14	

More on type conversion, casts and functions per type is in chapter 3.

Seeing the rules in action:

```sql
-- code/01_data_types.sql
SET TimeZone = 'UTC';   -- make timestamptz output the same on every machine
CREATE TABLE type_demo (
    qty        integer,
    views      bigint,
    rank_no    smallint,
    price      numeric(10,2),
    ratio      double precision,
    code       char(5),
    name       varchar(10),
    notes      text,
    born_on    date,
    starts_at  time,
    created    timestamp,
    created_tz timestamptz,
    active     boolean,
    birth_year smallint CHECK (birth_year BETWEEN 1900 AND 2100)  -- no YEAR type in PostgreSQL
);
INSERT INTO type_demo VALUES
 (3, 9000000000, 7, 19.999, 0.1::float8 + 0.2::float8, 'AB', 'Tara', 'long text ...',
  '1994-09-22', '09:30', '2025-06-01 10:00', '2025-06-01 10:00+05:30', true, 1994);

SELECT price, ratio, '[' || code || ']' AS code_padded, length(code) AS len_code,
       octet_length(code) AS bytes_code, created, created_tz
FROM type_demo;
```

Output:

```
 price |        ratio        | code_padded | len_code | bytes_code |       created       |
created_tz
-------+---------------------+-------------+----------+------------+---------------------+---------------
---------
 20.00 | 0.30000000000000004 | [AB]        |        2 |          5 | 2025-06-01 10:00:00 | 2025-06-01
04:30:00+00
(1 row)
```

What happened:

- `price` 19.999 was **rounded** to 2 decimal places → 20.00.
- `ratio`: 0.1 + 0.2 in binary floating point is not exactly 0.3 (`0.1::float8` is PostgreSQL's shorthand for a **cast** — converting a value to another type, here an 8-byte float) (the file also shows `0.1::numeric + 0.2::numeric = 0.3` is true, the `float8` version false). That's why money is never stored in `float`.
- `code` is stored padded to 5 characters (5 bytes), but PostgreSQL ignores a `char`'s trailing spaces, so `length` says 2 — confusing; prefer `text`/`varchar`.
- `created_tz`: 10:00 in India (+05:30) was stored as an instant and shown in the session time zone (UTC) as 04:30. **What the types reject:**

```sql
INSERT INTO type_demo (name) VALUES ('Christopher Columbus');
INSERT INTO type_demo (rank_no) VALUES (40000);
INSERT INTO type_demo (price) VALUES (123456789.00);
INSERT INTO type_demo (born_on) VALUES ('2025-02-30');
INSERT INTO type_demo (birth_year) VALUES (1850);
```

Output:

```
ERROR:  value too long for type character varying(10)
ERROR:  smallint out of range
ERROR:  numeric field overflow
DETAIL:  A field with precision 10, scale 2 must round to an absolute value less than 10^8.
ERROR:  date/time field value out of range: "2025-02-30"
ERROR:  new row for relation "type_demo" violates check constraint "type_demo_birth_year_check"
...
```

`numeric(10,2)` allows 10 digits of which 2 are after the point, so at most 8 before it ($< 10^8$). MySQL in non-strict mode (a lenient setting that turns many errors into warnings) would silently truncate some of these values instead of failing; PostgreSQL always errors.

How to choose

1. **Money** → `numeric(p, s)` (or integer cents), never `float`. **Ids/counts** → `integer`, or `bigint` if it could ever pass 2 billion.
2. **Text** → `text` in PostgreSQL; `varchar(n)` only when the maximum is a real business rule (no speed difference). SQL Server: `nvarchar` for Unicode.
3. **Instants** → `timestamptz`; `timestamp` only for wall-clock times that must not shift (a shop's opening time). **Days** → `date`.
4. Pick the smallest type that safely fits — changing it later on a big table is expensive (9.5).

Common mistakes

Money in `float` (rounding errors); dates as `text` (`'03/04/2025'` — March or April? can't sort); `varchar(255)` everywhere by habit (a meaningless limit); `timestamp` for events across time zones (ambiguous instants).

9.2 CREATE: databases, schemas and tables

What it is

`CREATE` makes new database objects. The ones you'll use most:

Statement	Creates
`CREATE DATABASE name;`	a new, separate database on the server (the running database program)
`CREATE SCHEMA name;`	a namespace **inside** a database (like a folder); tables are then `schema.table`
`CREATE TABLE name (...);`	a table: columns, types, constraints
`CREATE TABLE name AS SELECT ...;`	a table filled with a query's result
`CREATE TABLE name (LIKE other INCLUDING ALL);`	an empty copy of another table's structure
`CREATE TEMP TABLE name (...);`	a table visible only to your session, dropped automatically at the end
`CREATE VIEW`, `CREATE INDEX`, `CREATE SEQUENCE`, `CREATE FUNCTION` …	covered in chapters 7, 9.7, 9.4, 14

How CREATE TABLE works

```
CREATE TABLE [IF NOT EXISTS] [schema.]table_name (
    column_name  data_type  [column constraints],   -- one line per column
    ...,
    [table constraints]                             -- e.g. PRIMARY KEY (a, b)
);
```

Square brackets in a syntax outline like this mark optional parts — you don't type the brackets.

1. The name must be unique within its schema (default `public`); each column gets a name, a type and optionally constraints (9.3) and a `DEFAULT`.
2. Multi-column constraints go after the columns.
3. `IF NOT EXISTS` turns "already exists" into a notice (an informational message, not an error) — handy in setup scripts, but it does **not** check that the existing table has the same columns.

```
-- code/02_create.sql
CREATE DATABASE school_demo;          -- a new database (then dropped again in the script)
CREATE SCHEMA school;

CREATE TABLE school.students (
    student_id   integer      PRIMARY KEY,
    first_name   varchar(50)  NOT NULL,
    last_name    varchar(50)  NOT NULL,
    roll_number  integer      UNIQUE,
    email        text,
    enrolled_on  date         NOT NULL DEFAULT current_date
);

CREATE TABLE IF NOT EXISTS school.students (student_id integer);

CREATE TABLE school.bad (
    id   integer PRIMARY KEY,
    name text,
);
```

Output:

```
NOTICE:  relation "students" already exists, skipping
ERROR:  syntax error at or near ")"
LINE 4: );
```

A comma after the last column is a syntax error (PostgreSQL, MySQL, SQL Server) — a common typo after deleting the last line.

Tables from queries and copies:

```
CREATE TABLE school.sales_staff AS
SELECT emp_id, first_name, salary FROM employees WHERE dept_id = 2;
SELECT * FROM school.sales_staff ORDER BY emp_id;
```

Output:

```
 emp_id | first_name |  salary
--------+------------+-----------
      6 | Daniel     | 150000.00
      7 | Priya      |  72000.00
 ...
(4 rows)
```

`CREATE TABLE … AS` copies column names, types and data, but **no constraints, defaults or indexes**. (`SELECT … INTO new_table FROM …` does the same in PostgreSQL and SQL Server.) To copy only the structure, with constraints and defaults, use `LIKE`:

```
CREATE TABLE school.products_copy (LIKE products INCLUDING ALL);   -- 0 rows, same columns/constraints
```

Temporary tables (`CREATE TEMP TABLE scratch (id int, note text);`) live in a private per-session schema (the file's catalog query — the **catalog** is PostgreSQL's set of internal tables describing every object — shows a `pg_temp_N` schema and `relpersistence = 't'` instead of `'p'`) and vanish when the session ends — or at transaction end with `ON COMMIT DROP`. Use them for multi-step processing inside one session (7.9).

> **Key term — transaction:** a group of statements the database treats as one unit: either all of them take effect or none do. You start one with `BEGIN`, make it permanent with `COMMIT`, or undo everything since `BEGIN` with `ROLLBACK`. Without `BEGIN`, PostgreSQL wraps each statement in its own tiny transaction. Transactions are covered in depth in chapter 13.

Dialect notes

	PostgreSQL	MySQL	SQL Server	SQLite
Database vs schema	database contains schemas	"schema" = "database" (synonyms)	database contains schemas (`dbo` default)	one file = one database
Switch database	reconnect (`\c db` in psql, PostgreSQL's command-line client)	`USE db;`	`USE db;`	open another file / `ATTACH`
`CREATE TABLE IF NOT EXISTS`	yes	yes	no (check `OBJECT_ID` first)	yes
Temporary table	`CREATE TEMP TABLE`	`CREATE TEMPORARY TABLE`	`CREATE TABLE #name`	`CREATE TEMP TABLE`
`CREATE DATABASE` inside a transaction	not allowed	implicit commit (silently ends the open transaction first)	not allowed	n/a

---

9.3 Keys and constraints

What they are

A **key** is a column (or set of columns) whose values identify rows. A **constraint** is a rule the database checks on every write; a statement that would break it fails completely. Applications have bugs and there is usually more than one of them, but every write passes through the database — so that is where the rules belong.

The kinds of keys

Think of a `members` table with `member_id`, `email`, `phone`, `full_name`:

Key	Definition	In `members`
Super key	any column set that is unique for every row (may include unnecessary columns)	`{member_id}`, `{member_id, full_name}`, `{email, phone}`, …
Candidate key	a *minimal* super key — drop any column and it stops being unique; a table can have several	`{member_id}`, `{email}`
Primary key (PK)	the candidate key chosen as main identifier: unique **and** `NOT NULL`, one per table	`member_id`
Alternate key	a candidate key that wasn't chosen as primary; enforced with `UNIQUE`	`email`
Unique key	columns declared `UNIQUE`: no duplicates, NULLs allowed (below); many per table	`email`, `phone`
Composite key	two or more columns; only the *combination* must be unique	`(club_id, member_id)` in `club_memberships`
Foreign key (FK)	column(s) whose values must match a key (usually the PK) of another (or the same) table	`club_memberships.member_id → members.member_id`
Natural vs surrogate	natural = a real-world value (email, ISBN); surrogate = a generated number/UUID (9.4)	`email` is natural, `member_id` is surrogate

Keys and their theory (functional dependencies) are developed further in chapter 10 and chapter 11.

The constraints

Constraint	Rule	Written as
`NOT NULL`	a value is required	`col type NOT NULL`
`DEFAULT`	value used when an `INSERT` omits the column (not a check, but declared alongside)	`col type DEFAULT expr`
`UNIQUE`	no two rows share the value(s); NULLs don't count as equal	`col type UNIQUE` / `UNIQUE (a, b)`
`PRIMARY KEY`	`UNIQUE` + `NOT NULL`; one per table; creates a unique index (a lookup structure, 9.7)	`col type PRIMARY KEY` / `PRIMARY KEY (a, b)`
`CHECK`	a boolean expression must not be false	`CHECK (price >= 0)` / `CHECK (end_date >= start_date)`
`FOREIGN KEY`	value must exist in the referenced key (or be NULL)	`col type REFERENCES other(col)` / `FOREIGN KEY (a) REFERENCES other(x)`
`EXCLUDE` (PostgreSQL)	no two rows may "conflict" by an operator (e.g. overlapping bookings of one room)	see chapter 14

A **column constraint** is written on the column; a **table constraint** after the columns (required for multi-column rules). Name important ones with `CONSTRAINT name …` — the name appears in errors and is needed to drop it; otherwise PostgreSQL generates names like `members_email_key`.

```sql
-- code/03_keys_and_constraints.sql
CREATE TABLE members (
    member_id  integer PRIMARY KEY,                       -- primary key
    email      text    NOT NULL UNIQUE,                   -- candidate key -> alternate key
    phone      text    UNIQUE,                            -- unique key, NULL allowed
    full_name  text    NOT NULL,
    plan       text    NOT NULL DEFAULT 'free'
                       CONSTRAINT plan_is_known CHECK (plan IN ('free', 'pro', 'team')),
    joined_on  date    NOT NULL DEFAULT current_date,
    left_on    date,
    CONSTRAINT left_after_joined CHECK (left_on IS NULL OR left_on >= joined_on)   -- table-level check
);

INSERT INTO members (member_id, email, phone, full_name, joined_on)
VALUES (1, 'ravi@mail.example', NULL, 'Ravi Kumar', '2025-01-10'),
       (2, 'lena@mail.example', NULL, 'Lena Ortiz', '2025-02-01');     -- two NULL phones: fine
SELECT member_id, email, phone, plan FROM members ORDER BY member_id;
```

Output:

```
 member_id |       email       | phone  | plan
-----------+-------------------+--------+------
         1 | ravi@mail.example | (null) | free
         2 | lena@mail.example | (null) | free
(2 rows)
```

`plan` was filled from its `DEFAULT`. Now each rule refusing bad data:

```sql
INSERT INTO members (member_id, email, full_name) VALUES (1, 'x@mail.example', 'Dup Id');
INSERT INTO members (member_id, email, full_name, joined_on) VALUES (NULL, 'y@mail.example', 'No Id',
'2025-03-01');
INSERT INTO members (member_id, email, full_name) VALUES (3, 'ravi@mail.example', 'Dup');
INSERT INTO members (member_id, email, full_name, plan, joined_on) VALUES (4, 'z@mail.example', 'Z',
'gold', '2025-03-01');
INSERT INTO members (member_id, email, full_name, joined_on, left_on)
VALUES (5, 'w@mail.example', 'W', '2025-03-01', '2025-01-01');
```

Output:

```
ERROR:  duplicate key value violates unique constraint "members_pkey"
DETAIL:  Key (member_id)=(1) already exists.
ERROR:  null value in column "member_id" of relation "members" violates not-null constraint
...
ERROR:  duplicate key value violates unique constraint "members_email_key"
...
ERROR:  new row for relation "members" violates check constraint "plan_is_known"
...
ERROR:  new row for relation "members" violates check constraint "left_after_joined"
...
```

Two subtle rules:

- **`CHECK` passes when its expression is `NULL`.** A check fails only when the expression is *false*; `NULL` ("unknown") is accepted. That's why `left_after_joined` explicitly allows `left_on IS NULL`, and why a column that must be present also needs `NOT NULL`. (Three-valued logic — every condition is true, false or unknown: chapter 2.)
- **`UNIQUE` allows several `NULL`s** in PostgreSQL, MySQL, SQLite and Oracle, because NULL is not equal to NULL. SQL Server allows only **one** NULL in a unique constraint (use a filtered unique index — an index

over only the rows matching a `WHERE` — for "unique when present"). PostgreSQL 15 added `UNIQUE NULLS NOT DISTINCT` to treat NULLs as equal (PostgreSQL 15 — not run).

Foreign keys and referential actions

A foreign key keeps **referential integrity**: a child row can't point to a parent that doesn't exist. The interesting question is what happens when the *parent* is deleted or its key changes. You choose with `ON DELETE` / `ON UPDATE`:

Action	When the referenced parent row is deleted (or its key updated)
`NO ACTION` (default)	error if children still reference it (checked at the end of the statement; can be deferred)
`RESTRICT`	error, checked immediately (can't be deferred)
`CASCADE`	delete (or update) the child rows too
`SET NULL`	set the child's FK column(s) to NULL
`SET DEFAULT`	set the child's FK column(s) to their default

```
CREATE TABLE clubs (
    club_id  integer PRIMARY KEY,
    name     text NOT NULL UNIQUE
);
CREATE TABLE club_memberships (
    club_id    integer REFERENCES clubs(club_id)     ON DELETE CASCADE,
    member_id  integer REFERENCES members(member_id) ON DELETE RESTRICT,
    role       text NOT NULL DEFAULT 'member',
    PRIMARY KEY (club_id, member_id)                          -- composite key
);
CREATE TABLE club_events (
    event_id   integer PRIMARY KEY,
    club_id    integer REFERENCES clubs ON DELETE SET NULL,           -- "REFERENCES clubs" = its PK
    title      text NOT NULL
);
INSERT INTO clubs VALUES (10, 'Chess'), (20, 'Hiking');
INSERT INTO club_memberships (club_id, member_id) VALUES (10, 1), (10, 2), (20, 1);
INSERT INTO club_events VALUES (100, 10, 'Blitz night'), (101, 20, 'Sunrise trek');

INSERT INTO club_memberships (club_id, member_id) VALUES (10, 1);       -- composite PK duplicate
INSERT INTO club_memberships (club_id, member_id) VALUES (30, 1);       -- no club 30
DELETE FROM members WHERE member_id = 2;                                  -- RESTRICT
```

Output:

```
ERROR:  duplicate key value violates unique constraint "club_memberships_pkey"
...
ERROR:  insert or update on table "club_memberships" violates foreign key constraint
"club_memberships_club_id_fkey"
DETAIL:  Key (club_id)=(30) is not present in table "clubs".
ERROR:  update or delete on table "members" violates foreign key constraint
"club_memberships_member_id_fkey" on table "club_memberships"
DETAIL:  Key (member_id)=(2) is still referenced from table "club_memberships".
```

Deleting the Chess club triggers both `CASCADE` (its memberships go) and `SET NULL` (its event stays, orphaned on purpose):

```
DELETE FROM clubs WHERE club_id = 10;
SELECT * FROM club_memberships ORDER BY club_id, member_id;
SELECT * FROM club_events ORDER BY event_id;
```

Output:

```
 club_id | member_id |  role
---------+-----------+--------
      20 |         1 | member
(1 row)

 event_id | club_id |    title
----------+---------+--------------
      100 |  (null) | Blitz night
      101 |      20 | Sunrise trek
(2 rows)
```

Choosing an action: `CASCADE` for rows that are *part of* the parent (order lines, memberships); `RESTRICT` / `NO ACTION` for things that must not vanish silently (a customer with orders); `SET NULL` when the child should survive without the link.

Every constraint is recorded in the `pg_constraint` catalog (`contype` `p` = primary key, `u` = unique, `f` = foreign key, `c` = check); the file lists the eight created here.

Deferrable constraints. Constraints are normally checked after each statement; `DEFERRABLE INITIALLY DEFERRED` postpones the check to `COMMIT`, allowing temporarily inconsistent states — e.g. two rows that reference each other:

```
CREATE TABLE people (
    person_id  integer PRIMARY KEY,
    partner_id integer REFERENCES people DEFERRABLE INITIALLY DEFERRED
);
BEGIN;
INSERT INTO people VALUES (1, 2);   -- person 2 doesn't exist yet: allowed for now
INSERT INTO people VALUES (2, 1);
COMMIT;                             -- checked here: both exist
```

Both rows are committed. (`UNIQUE`, `PRIMARY KEY`, `REFERENCES` and `EXCLUDE` can be deferrable in PostgreSQL; `CHECK` and `NOT NULL` cannot.)

Dialect notes

InnoDB is MySQL's default **storage engine** (the component that actually stores and reads rows); SQLite's `PRAGMA` is its command for changing settings.

	PostgreSQL	MySQL (InnoDB)	SQL Server	SQLite
`CHECK` enforced	yes	since 8.0.16 (parsed but ignored before)	yes	yes
FK enforced	yes	yes	yes	only after `PRAGMA foreign_keys = ON`
Index auto-created on FK column	**no** — create one yourself	yes	no	no
Multiple NULLs in `UNIQUE`	yes	yes	no (one)	yes
Deferrable constraints	yes	no	no	FKs only

When not to rely on constraints alone

Constraints check single rows (and FK existence). Multi-row rules ("at most 5 members per club") need triggers (functions the database runs automatically when rows are inserted, updated or deleted), `EXCLUDE` constraints or careful transactions (chapter 13, chapter 14).

Common mistakes

- **No index on the FK column**: in PostgreSQL each parent delete then scans the whole child table.
- **`ON DELETE CASCADE` everywhere**: one delete can wipe out large parts of the database.
- **Forgetting `NOT NULL`** next to `CHECK` or `UNIQUE` when a value is required.
- **A changeable natural key** (email, phone) as the PK — every FK must change with it. Use a surrogate PK plus `UNIQUE` on the natural key.

In the interview

> **30-second answer:** "A candidate key is a minimal set of columns that identifies a row; the primary key is the chosen one — unique, not null, one per table; the rest are alternate keys enforced with `UNIQUE` (NULLs allowed). A composite key combines columns. A foreign key references another table's key, and `ON DELETE CASCADE / SET NULL / RESTRICT` decides what happens to children. `CHECK`, `NOT NULL` and `DEFAULT` complete the set."

- *Primary key vs unique key?* → PK: one per table, never NULL, the row's identity (often the clustered index — the one that stores the rows themselves in key order, 9.7 — in SQL Server/MySQL). UNIQUE: many per table, NULLs allowed.
- *Natural or surrogate PK?* → Usually a surrogate (`bigint` identity or UUID) for stability, plus `UNIQUE` on the natural key to keep the business rule.

Self-check

1. *Can a table have two primary keys?* → No; one primary key (possibly composite), any number of unique keys.
2. *Does `CHECK (qty > 0)` reject NULL?* → No; add `NOT NULL`.
3. *Does PostgreSQL index a foreign-key column automatically?* → No.
4. *Is `{member_id, email}` a candidate key?* → No — it's a super key but not minimal (`member_id` alone is unique).

9.4 Generated keys: identity, sequences, serial, UUID, generated columns

What it is

Most tables use a generated **surrogate key**. PostgreSQL generates numbers with a **sequence** — an object that hands out increasing values (`nextval`); an **identity column** (SQL standard, PostgreSQL 10+) is a column tied to its own sequence.

Syntax	Meaning
`GENERATED ALWAYS AS IDENTITY`	database always generates the value; supplying one is an error (unless `OVERRIDING SYSTEM VALUE`)
`GENERATED BY DEFAULT AS IDENTITY`	generated when you don't supply one; you may supply your own
`serial` / `bigserial`	older PostgreSQL shorthand: integer column + sequence + `DEFAULT nextval(...)`
`uuid DEFAULT gen_random_uuid()`	random 128-bit id — a UUID, *universally unique identifier* (built in since PostgreSQL 13)
`GENERATED ALWAYS AS (expr) STORED`	a **generated column**: computed from other columns on every write (PostgreSQL 12+)

```sql
-- code/04_identity_and_sequences.sql
CREATE TABLE tickets (
    ticket_id  bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    subject    text NOT NULL,
    price      numeric(8,2) NOT NULL,
    qty        integer NOT NULL DEFAULT 1,
    total      numeric(10,2) GENERATED ALWAYS AS (price * qty) STORED
);
INSERT INTO tickets (subject, price) VALUES ('Printer jam', 10);
INSERT INTO tickets (subject, price, qty) VALUES ('VPN access', 5, 3), ('New laptop', 1200, 1)
RETURNING ticket_id, total;
```

Output:

```
 ticket_id |  total
-----------+---------
         2 |   15.00
         3 | 1200.00
(2 rows)
```

```sql
INSERT INTO tickets (ticket_id, subject, price) VALUES (99, 'Manual id', 1);
INSERT INTO tickets (subject, price, total) VALUES ('Fake total', 1, 500);
```

Output:

```
ERROR:  cannot insert a non-DEFAULT value into column "ticket_id"
DETAIL:  Column "ticket_id" is an identity column defined as GENERATED ALWAYS.
HINT:  Use OVERRIDING SYSTEM VALUE to override.
ERROR:  cannot insert a non-DEFAULT value into column "total"
DETAIL:  Column "total" is a generated column.
```

Sequences never go backwards — gaps are normal:

```sql
INSERT INTO tickets (ticket_id, subject, price) OVERRIDING SYSTEM VALUE VALUES (99, 'Manual id', 1);
BEGIN;
INSERT INTO tickets (subject, price) VALUES ('Will be rolled back', 1);
ROLLBACK;
INSERT INTO tickets (subject, price) VALUES ('After rollback', 2);
SELECT ticket_id, subject, total FROM tickets ORDER BY ticket_id;
```

Output:

```
 ticket_id |    subject     |  total
-----------+----------------+---------
         1 | Printer jam    |   10.00
 ...
         3 | New laptop     | 1200.00
         5 | After rollback |    2.00
        99 | Manual id      |    1.00
(5 rows)
```

Number 4 went to the rolled-back insert and is never reused: sequences are deliberately **not** transactional (a rollback doesn't undo them), so concurrent inserts (inserts running at the same moment in different sessions) never wait for each other. Never assume ids are gap-free (for legal invoice numbers, generate them separately under a lock — a **lock** is a claim on a row or table that makes other sessions wait until you finish).

Why identity beats `serial`: `serial` is just a default, so a manual id doesn't advance the sequence and a later generated one collides:

```sql
CREATE TABLE legacy (id serial PRIMARY KEY, label text);
INSERT INTO legacy (label) VALUES ('a'), ('b');       -- ids 1, 2
INSERT INTO legacy (id, label) VALUES (3, 'manual');  -- allowed
INSERT INTO legacy (label) VALUES ('c');              -- sequence gives 3 -> collision
```

Output:

```
ERROR:  duplicate key value violates unique constraint "legacy_pkey"
DETAIL:  Key (id)=(3) already exists.
```

`GENERATED ALWAYS` refuses the manual insert up front. (After loading your own ids, resync: `SELECT setval('legacy_id_seq', (SELECT max(id) FROM legacy));`.)

A standalone sequence, e.g. for invoice numbers shared by several tables:

```sql
CREATE SEQUENCE invoice_no START 1000 INCREMENT 10;
SELECT nextval('invoice_no') AS first, nextval('invoice_no') AS second, currval('invoice_no') AS current;
-- 1000 | 1010 | 1010
```

`currval` returns the last value *this session* got from `nextval`.

UUIDs: `key_id uuid PRIMARY KEY DEFAULT gen_random_uuid()` gives 128-bit ids (36 characters as text). They can be generated anywhere (even in the app) without coordination and don't reveal row counts, but take 16 bytes (vs 8 for `bigint`) and random values scatter index inserts. Time-ordered UUIDv7 (a UUID whose first bits are a timestamp, so new ids sort roughly by creation time) reduces that; PostgreSQL 18 adds a built-in `uuidv7()` (not run here).

Dialect notes

	Auto-increment syntax
PostgreSQL	`GENERATED {ALWAYS | BY DEFAULT} AS IDENTITY` (10+), `serial` (legacy)
MySQL	`AUTO_INCREMENT`
SQL Server	`IDENTITY(1,1)`; `SEQUENCE` objects since 2012
SQLite	`INTEGER PRIMARY KEY` (alias of the rowid, SQLite's hidden row number); `AUTOINCREMENT` prevents reuse of deleted ids
Oracle	identity columns since 12c; sequences

Generated columns: PostgreSQL 12–17 `STORED` only; PostgreSQL 18 adds `VIRTUAL` (computed on read) and makes it the default when neither keyword is given (not run); MySQL 5.7+ supports both; SQL Server has "computed columns" (optionally `PERSISTED`).

9.5 ALTER: changing existing tables

What it is

`ALTER TABLE` changes an existing table's structure — usually with data in it — without recreating it.

Task	PostgreSQL
Add a column	`ALTER TABLE t ADD COLUMN c type [DEFAULT …] [NOT NULL];`
Drop a column	`ALTER TABLE t DROP COLUMN c;`
Rename a column / table	`ALTER TABLE t RENAME COLUMN a TO b;` / `ALTER TABLE t RENAME TO u;`
Change a type	`ALTER TABLE t ALTER COLUMN c TYPE newtype [USING expr];`
Set / drop a default	`ALTER TABLE t ALTER COLUMN c SET DEFAULT …;` / `DROP DEFAULT`
Set / drop NOT NULL	`ALTER TABLE t ALTER COLUMN c SET NOT NULL;` / `DROP NOT NULL`
Add / drop a constraint	`ALTER TABLE t ADD CONSTRAINT name …;` / `DROP CONSTRAINT name;`

How it works (with real data)

```sql
-- code/05_alter.sql
CREATE TABLE staff (id integer PRIMARY KEY, first_name text, last_name text);
INSERT INTO staff VALUES (1, 'Tara', 'Mendes'), (2, 'Omar', 'Mendes'), (3, 'Yuki', 'Tanaka');

ALTER TABLE staff ADD COLUMN salary integer;
ALTER TABLE staff ADD COLUMN country text NOT NULL DEFAULT 'US';
SELECT * FROM staff ORDER BY id;
```

Output:

```
 id | first_name | last_name | salary | country
----+------------+-----------+--------+---------
  1 | Tara       | Mendes    | (null) | US
  2 | Omar       | Mendes    | (null) | US
  3 | Yuki       | Tanaka    | (null) | US
(3 rows)
```

Existing rows get NULL, or the default if there is one. Since PostgreSQL 11, adding a column with a constant default is instant (no table rewrite — copying every row into a new file).

```sql
ALTER TABLE staff RENAME COLUMN first_name TO given_name;
ALTER TABLE staff RENAME TO team;

UPDATE team SET salary = 50000 + id * 1000;
ALTER TABLE team ALTER COLUMN salary TYPE numeric(10,2) USING salary::numeric(10,2);
ALTER TABLE team ALTER COLUMN last_name TYPE integer;                -- text -> integer?
```

Output:

```
ERROR:  column "last_name" cannot be cast automatically to type integer
HINT:  You might need to specify "USING last_name::integer".
```

`USING` tells PostgreSQL how to compute the new value from the old one; without it only automatic (implicit/assignment) casts — the conversions PostgreSQL considers safe to apply on its own — are allowed.

Adding `NOT NULL` to a column that has NULLs fails — `ALTER TABLE team ADD COLUMN email text NOT NULL;` gives *ERROR: column "email" of relation "team" contains null values*. Add, backfill (fill in values for the existing rows), then constrain:

```
ALTER TABLE team ADD COLUMN email text;
UPDATE team SET email = lower(given_name) || '@corp.example';
ALTER TABLE team ALTER COLUMN email SET NOT NULL;
```

Constraints on live tables: `NOT VALID` then `VALIDATE`. Adding a `CHECK` or FK normally scans the whole table under a strong lock. On a big table, add it `NOT VALID` (enforced for new and changed rows only), fix old data, then `VALIDATE CONSTRAINT` (scans under a lighter lock):

```
ALTER TABLE team ADD CONSTRAINT team_email_uq UNIQUE (email);
INSERT INTO team (id, given_name, last_name, salary, email) VALUES (4, 'Leo', 'Brandt', -5,
'leo@corp.example');
ALTER TABLE team ADD CONSTRAINT salary_positive CHECK (salary > 0);            -- fails: Leo's row
ALTER TABLE team ADD CONSTRAINT salary_positive CHECK (salary > 0) NOT VALID;  -- accepted
ALTER TABLE team VALIDATE CONSTRAINT salary_positive;                          -- fails until fixed
UPDATE team SET salary = 48000 WHERE id = 4;
ALTER TABLE team VALIDATE CONSTRAINT salary_positive;                          -- succeeds
```

Output:

```
ERROR:  check constraint "salary_positive" of relation "team" is violated by some row
ERROR:  check constraint "salary_positive" of relation "team" is violated by some row
```

(`UNIQUE`/`PRIMARY KEY` can't be `NOT VALID`; build a unique index `CONCURRENTLY` (without blocking writes, 9.7) and then `ADD CONSTRAINT … UNIQUE USING INDEX` instead.)

Several changes in one statement (one lock, one pass):

```
ALTER TABLE team
    DROP COLUMN country,
    ADD COLUMN hired_on date,
    ALTER COLUMN last_name SET NOT NULL;
SELECT conname, pg_get_constraintdef(oid) AS definition
FROM pg_constraint WHERE conrelid = 'team'::regclass ORDER BY conname;
```

Output:

```
     conname     |           definition
-----------------+--------------------------------
 salary_positive | CHECK ((salary > (0)::numeric))
 staff_pkey      | PRIMARY KEY (id)
 team_email_uq   | UNIQUE (email)
(3 rows)
```

Note the primary key is still called `staff_pkey`: renaming a table doesn't rename its constraints or indexes.

Locks and production safety

Most `ALTER TABLE` forms take an **ACCESS EXCLUSIVE** lock — no reads or writes until it finishes. Metadata-only changes (edits to the table's description in the catalog that don't touch its rows: nullable column, constant default, rename, drop column — which only hides it) hold it briefly; **type changes that rewrite the table** can hold it for minutes. Production (the live system real users depend on) practice: set `lock_timeout` (so a statement gives up instead of waiting indefinitely for a lock), use `NOT VALID` + `VALIDATE` and `CREATE INDEX CONCURRENTLY`, and "expand → migrate → contract" (add the new column, backfill in batches — chunks of a few thousand rows at a time — switch the app, drop the old one). Locks: chapter 13.

Dialect notes

Task	PostgreSQL	MySQL	SQL Server
Change type	`ALTER COLUMN c TYPE t`	`MODIFY COLUMN c t`	`ALTER COLUMN c t`
Rename column	`RENAME COLUMN a TO b`	`RENAME COLUMN a TO b` (8.0) / `CHANGE a b t`	`EXEC sp_rename 't.a', 'b', 'COLUMN'`
Rename table	`ALTER TABLE t RENAME TO u`	`RENAME TABLE t TO u`	`EXEC sp_rename 't', 'u'`
DDL inside a transaction (rollback-able)	yes	no — implicit commit	yes (mostly)

Common mistakes

`NOT NULL` without a default or backfill (fails); a type change on a huge table in business hours (long exclusive lock); renaming columns that application SQL still uses (views follow renames automatically in PostgreSQL, application code does not).

9.6 DROP and TRUNCATE (vs DELETE)

What they are

- `DROP` removes an object entirely: the table's definition, data, indexes, constraints, triggers and permissions. `DROP TABLE t;`, `DROP DATABASE d;`, `DROP SCHEMA s;`, `DROP VIEW v;`, `DROP INDEX i;`.
- `TRUNCATE` removes **all rows** from a table but keeps the table itself (structure, indexes, grants). `TRUNCATE TABLE t;` (the word `TABLE` is optional in PostgreSQL and MySQL, required in SQL Server and Oracle).
- `DELETE` (DML, 9.10) removes rows one by one, optionally only those matching a `WHERE`.

Analogy: `DELETE` erases lines from a notebook one by one; `TRUNCATE` swaps in a fresh empty notebook; `DROP` throws the notebook away.

How TRUNCATE works

```sql
-- code/06_drop_and_truncate.sql
CREATE TABLE audit_log (log_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY, msg text NOT NULL);
INSERT INTO audit_log (msg) SELECT 'event ' || g FROM generate_series(1, 5) g;

TRUNCATE audit_log;
SELECT count(*) AS rows_after_truncate FROM audit_log;                    -- 0
INSERT INTO audit_log (msg) VALUES ('after truncate') RETURNING log_id;   -- 6: identity continues
TRUNCATE audit_log RESTART IDENTITY;
INSERT INTO audit_log (msg) VALUES ('after restart') RETURNING log_id;    -- 1
```

Output (the three results): `0`, then `log_id = 6`, then `log_id = 1`.

In PostgreSQL, `TRUNCATE` is **transactional** — you can roll it back:

```
BEGIN;
TRUNCATE audit_log;
SELECT count(*) AS inside_tx FROM audit_log;       -- 0
ROLLBACK;
SELECT count(*) AS after_rollback FROM audit_log;  -- 1
```

A table referenced by a foreign key can't be truncated alone, because that would orphan the child rows; `CASCADE` truncates the referencing tables too:

```
TRUNCATE customers;
BEGIN;
TRUNCATE customers CASCADE;
SELECT (SELECT count(*) FROM customers) AS customers,
       (SELECT count(*) FROM orders) AS orders,
       (SELECT count(*) FROM order_items) AS items;
ROLLBACK;
```

Output:

```
ERROR:  cannot truncate a table referenced in a foreign key constraint
DETAIL:  Table "orders" references "customers".
HINT:  Truncate table "orders" at the same time, or use TRUNCATE ... CASCADE.
NOTICE:  truncate cascades to table "orders"
NOTICE:  truncate cascades to table "order_items"
 customers | orders | items
-----------+--------+-------
         0 |      0 |     0
(1 row)
```

How DROP works

`DROP TABLE audit_log;` removes the table; querying it afterwards gives *relation "audit_log" does not exist*, a second `DROP` errors, and `DROP TABLE IF EXISTS` only prints a notice — the form to use in scripts. `DROP` refuses while other objects depend on the table (default `RESTRICT`); `CASCADE` drops the dependents too — views are dropped, and FKs *pointing at* the table are removed (the child table and its rows stay):

```
CREATE VIEW big_spenders AS SELECT customer_id FROM orders GROUP BY customer_id HAVING count(*) > 2;
DROP TABLE orders;
BEGIN;
DROP TABLE orders CASCADE;
SELECT count(*) AS order_items_still_there FROM order_items;
ROLLBACK;
```

Output:

```
ERROR:  cannot drop table orders because other objects depend on it
DETAIL:  constraint order_items_order_id_fkey on table order_items depends on table orders
view big_spenders depends on table orders
HINT:  Use DROP ... CASCADE to drop the dependent objects too.
NOTICE:  drop cascades to 2 other objects
...
 order_items_still_there
-------------------------
                      22
(1 row)
```

`DROP DATABASE name;` removes a whole database (you can't be connected to it, and in PostgreSQL it can't run inside a transaction). `DROP SCHEMA s CASCADE;` removes a schema and everything in it.

DELETE vs TRUNCATE vs DROP

	`DELETE FROM t [WHERE …]`	`TRUNCATE t`	`DROP TABLE t`
Kind	DML	DDL	DDL
Removes	chosen rows (or all)	all rows	rows **and** the table
`WHERE` possible	yes	no	no
Speed on big tables	slow: row by row, all logged (each change written to the transaction log)	fast: releases the data files	fast
Space returned to the OS	usually not: dead rows (old, deleted versions) are reused after `VACUUM` (PostgreSQL's cleanup process), the file rarely shrinks	yes	yes
Row-level `ON DELETE` triggers	fire	don't fire (PostgreSQL has `ON TRUNCATE` triggers)	n/a
FK checks	per row (or cascades)	refused if referenced, unless `CASCADE`	refused if referenced, unless `CASCADE`
Identity / auto-increment	continues	PostgreSQL: continues unless `RESTART IDENTITY`; MySQL & SQL Server: reset	gone
Rollback	yes	inside a transaction in PostgreSQL & SQL Server; **not** in MySQL & Oracle (implicit commit)	same as `TRUNCATE`
Lock	row locks	exclusive table lock	exclusive table lock

"Can `TRUNCATE`/`DROP` be undone?" therefore depends on the database — in MySQL the only way back is a backup (9.12); Oracle can undo a `DROP TABLE` from its recycle bin (`FLASHBACK TABLE t TO BEFORE DROP`, if enabled) but not a `TRUNCATE`. SQLite has no `TRUNCATE`; `DELETE FROM t` without `WHERE` is optimized to do the same job.

Common mistakes

- Running `DROP`/`TRUNCATE` on the wrong connection (production instead of dev) — make production visually distinct and use reviewed migrations (versioned, checked-in scripts that change the schema step by step).

- `… CASCADE` without reading what it cascades to — try it inside `BEGIN … ROLLBACK` first, as above.
- `DELETE` to empty a 100-million-row table (hours, huge log) when `TRUNCATE` takes a second.
- Expecting `TRUNCATE` to be invisible to concurrent transactions: it isn't MVCC-safe (MVCC, *multi-version concurrency control*, lets each transaction read a **snapshot** — a frozen, consistent view of the data as of one moment; chapter 13) — a transaction with an older snapshot that reads the table afterwards sees it empty.

In the interview

30-second answer: "`DELETE` is DML: selected rows, one at a time, fires triggers, keeps the table. `TRUNCATE` is DDL: all rows at once by deallocating storage — much faster, no `WHERE`, no delete triggers, often resets identity; rollback-able in PostgreSQL and SQL Server, not in MySQL/Oracle. `DROP` removes the table itself."

Self-check

1. *Empty a staging table (a scratch table where incoming data lands before processing) quickly and reset its ids in PostgreSQL?* → `TRUNCATE staging RESTART IDENTITY;`
2. *Can you `TRUNCATE` a table that other tables reference by FK?* → Only with `CASCADE` (or truncating them in the same statement).
3. *What does `DROP TABLE orders CASCADE` do to `order_items`?* → Drops the FK constraint on it; the table and rows remain.

9.7 Indexes (in brief)

An **index** is a separate data structure (by default a B-tree — a balanced, always-sorted tree of keys) that finds rows by column values without reading the whole table — like a book's index sending you straight to the page. This section shows only the DDL; how indexes work, their kinds, `EXPLAIN` (the command that shows how the database plans to run a query), and when they help or hurt are in chapter 12.

```sql
-- code/07_indexes_brief.sql
CREATE INDEX idx_employees_last_name ON employees (last_name);
CREATE INDEX idx_orders_customer_placed ON orders (customer_id, placed_at DESC);   -- composite

-- A UNIQUE index can enforce rules a UNIQUE constraint can't, e.g. case-insensitive uniqueness:
CREATE UNIQUE INDEX idx_customers_email_ci ON customers (lower(trim(email)));
```

Output:

```
ERROR:  could not create unique index "idx_customers_email_ci"
DETAIL:  Key (lower(TRIM(BOTH FROM email)))=(anita.shah@mail.example) is duplicated.
```

The sample data really has a duplicate customer (chapter 1's Anita Shah, entered twice with different capitalisation) — the index found it. After cleaning (`UPDATE customers SET email = NULL WHERE customer_id = 8`) the same statement succeeds.

Primary keys and unique constraints create their indexes automatically — `pg_indexes` (a catalog view listing every index) lists `employees_pkey` and `employees_email_key` next to the ones created above. Other index DDL: `DROP INDEX [IF EXISTS] name;`, and `CREATE INDEX CONCURRENTLY …` / `DROP INDEX CONCURRENTLY …`, which avoid blocking writes on a live table (slower, and not allowed inside a transaction block).

Clustered vs non-clustered

	Clustered index	Non-clustered (secondary) index
What it is	the table's rows are **stored in the index's order** — the index *is* the table	a separate structure of (key → pointer to the row)
How many	at most one per table	many
SQL Server	`CREATE CLUSTERED INDEX ix ON t (col);` (the PK is clustered by default)	`CREATE [NONCLUSTERED] INDEX ix ON t (col);`
MySQL InnoDB	always the primary key (no PK → the first `UNIQUE NOT NULL` index, else a hidden row id)	every other index; it stores the PK value as the pointer
PostgreSQL	**none** — tables are unordered "heaps"; `CLUSTER t USING ix;` re-sorts the table once, but new rows aren't kept in order	every index is secondary

```
CLUSTER orders USING idx_orders_customer_placed;   -- one-time physical re-ordering in PostgreSQL
```

A clustered index makes range scans (reading every key between two values) on its key fast (neighbouring keys are neighbouring rows) but makes the choice of key important: a random key (like a random UUID) causes page splits on insert (a full storage page — the fixed-size block, 8 KB in PostgreSQL, that data is kept in — must be split in two to make room).

Cost of indexes: each index must be maintained on every insert and on updates of its columns (and cleaned up after deletes), and takes disk space. Index what queries filter, join and sort on — not every column.

9.8 INSERT

What it is

`INSERT` adds new rows to a table.

```
INSERT INTO table_name [(column1, column2, ...)]
VALUES (value1, value2, ...) [, (...), ...]
[ON CONFLICT ...]            -- 9.11
[RETURNING ...];             -- PostgreSQL: give back the inserted rows
-- or
INSERT INTO table_name [(columns)] SELECT ...;
```

How it works

1. The **column list** names the columns you supply, in order. Omitted columns get their `DEFAULT`, else NULL (an error if `NOT NULL` without a default).
2. Without a column list you must supply **every** column in table order — fragile: adding a column later breaks the statement.
3. Every row is checked against types and constraints; if any row fails, the **whole statement** fails.

```sql
-- code/08_insert.sql
CREATE TABLE contacts (
    contact_id  integer GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    first_name  text NOT NULL,
    last_name   text NOT NULL,
    age         integer CHECK (age >= 0),
    city        text DEFAULT 'Pune',
    created_at  timestamp NOT NULL DEFAULT '2025-06-01 09:00'
);

INSERT INTO contacts (first_name, last_name, age) VALUES ('Tara', 'Mendes', 28);

INSERT INTO contacts (first_name, last_name, age, city) VALUES      -- several rows at once
    ('Omar', 'Mendes', 31, 'Mumbai'),
    ('Yuki', 'Tanaka', NULL, DEFAULT);                              -- DEFAULT keyword

INSERT INTO contacts VALUES (DEFAULT, 'Leo', 'Brandt', 40, 'Delhi', '2025-06-02 10:00');  -- all columns

INSERT INTO contacts (first_name, last_name) VALUES ('Ines', 'Silva')
RETURNING contact_id, city, created_at;
```

Output (of the `RETURNING`):

```
 contact_id | city |     created_at
------------+------+---------------------
          5 | Pune | 2025-06-01 09:00:00
(1 row)
```

`RETURNING` hands back generated values (ids, defaults) in the same round trip (one request to the server and its reply) — no second query, no race (a **race condition** is a bug whose outcome depends on the timing of concurrent sessions).

`INSERT … SELECT` copies rows produced by any query:

```sql
INSERT INTO contacts (first_name, last_name, city)
SELECT first_name, last_name, city FROM customers WHERE country = 'UK'
RETURNING contact_id, first_name, city;
```

Output:

```
 contact_id | first_name |  city
------------+------------+--------
          6 | Ben        | London
          7 | George     | London
(2 rows)
```

Errors worth recognising:

```sql
INSERT INTO contacts (first_name, last_name) VALUES ("Ana", "Lima");
INSERT INTO contacts (first_name, last_name) VALUES ('Ana', 'Lima', 22);
INSERT INTO contacts (first_name, last_name, age) VALUES ('Ok', 'Row', 20), ('Bad', 'Row', -1);
SELECT count(*) AS still_7 FROM contacts;
```

Output:

```
ERROR:  column "Ana" does not exist
ERROR:  INSERT has more expressions than target columns
ERROR:  new row for relation "contacts" violates check constraint "contacts_age_check"
DETAIL:  Failing row contains (9, Bad, Row, -1, Pune, 2025-06-01 09:00:00).
 still_7
---------
       7
(1 row)
```

- In standard SQL (PostgreSQL, SQLite, SQL Server by default, Oracle) **double quotes name identifiers** (names of tables, columns and other objects); strings use single quotes. MySQL accepts double-quoted strings by default, so MySQL-style code often breaks elsewhere.
- The multi-row insert failed as a whole — the valid 'Ok' row was not kept (but it consumed identity values 8 and 9: more gaps).

Bulk loading: multi-row `VALUES` or `INSERT … SELECT` beats one statement per row; for millions of rows use `COPY table FROM file` (psql: `\copy` for a client file), MySQL `LOAD DATA INFILE`, SQL Server `BULK INSERT` / `bcp`.

Dialect notes

	PostgreSQL	MySQL	SQL Server	SQLite
Multi-row `VALUES`	yes	yes	yes (max 1000 rows per statement)	yes
Get generated values back	`RETURNING`	`LAST_INSERT_ID()`	`OUTPUT inserted.*` / `SCOPE_IDENTITY()`	`RETURNING` (3.35+) / `last_insert_rowid()`
`INSERT … DEFAULT VALUES`	yes	`INSERT INTO t () VALUES ()`	yes	yes

9.9 UPDATE

What it is

`UPDATE` changes column values in existing rows.

```
UPDATE table_name
SET column1 = expr1, column2 = expr2, ...
[FROM other_tables]          -- PostgreSQL: use other tables to compute / filter
[WHERE condition]            -- which rows; WITHOUT it, EVERY row changes
[RETURNING ...];
```

How it works

1. `WHERE` picks the rows, as in `SELECT`. No `WHERE` = all rows.
2. Every `SET` expression is evaluated from the row's **old** values, then all assignments apply together.
3. New rows must satisfy all constraints, or the statement fails and nothing changes.
4. PostgreSQL writes a new version of each updated row (MVCC — chapter 13); `VACUUM` later removes the old one.

Raise Sales salaries by 10 %:

```sql
-- code/09_update.sql
UPDATE employees
SET salary = salary * 1.10
WHERE dept_id = 2
RETURNING emp_id, first_name, salary;
```

Output:

```
 emp_id | first_name |  salary
--------+------------+-----------
      6 | Daniel     | 165000.00
      7 | Priya      |  79200.00
 ...
(4 rows)
```

Percentages are multiplications: +10 % is `* 1.10`, +50 % is `* 1.5`, doubling is `* 2` — check the factor against the requirement.

All `SET` expressions see the old row:

```sql
UPDATE products
SET unit_price   = unit_price * 0.9,
    discontinued = unit_price * 0.9 < 40          -- uses the OLD unit_price
WHERE category = 'Furniture'
RETURNING name, unit_price, discontinued;
```

Output:

```
      name       | unit_price | discontinued
-----------------+------------+--------------
 Standing Desk   |     449.10 | f
 Ergonomic Chair |     314.99 | f
 Desk Lamp       |      31.50 | t
(3 rows)
```

(In MySQL, assignments are applied left to right, so a later expression sees the *new* value of an earlier column — a real portability trap. PostgreSQL and SQL Server follow the standard.)

Different changes for different rows with `CASE`:

```sql
UPDATE orders
SET status = CASE WHEN shipped_at IS NULL THEN 'cancelled' ELSE 'delivered' END
WHERE status IN ('pending', 'shipped') AND placed_at < '2025-06-10'
RETURNING order_id, status;
```

Output:

```
 order_id |  status
----------+-----------
      107 | delivered
 ...
      113 | cancelled
(4 rows)
```

Using another table — PostgreSQL's `UPDATE … FROM` joins the target to other tables:

```
ALTER TABLE customers ADD COLUMN lifetime_orders integer NOT NULL DEFAULT 0;
UPDATE customers c
SET lifetime_orders = t.n
FROM (SELECT customer_id, count(*) AS n FROM orders GROUP BY customer_id) t
WHERE t.customer_id = c.customer_id;
SELECT customer_id, first_name, lifetime_orders FROM customers ORDER BY customer_id LIMIT 4;
```

Output:

```
 customer_id | first_name | lifetime_orders
-------------+------------+-----------------
           1 | Anita      |               3
           2 | Ben        |               2
 ...
(4 rows)
```

Make sure each target row matches **at most one** row of the `FROM` source; if it matches several, PostgreSQL uses one of them unpredictably. That's why the example aggregates first. The portable alternative is a correlated scalar subquery (a subquery that returns one value and is re-run for each outer row) in `SET`:

```
UPDATE departments d
SET budget = (SELECT coalesce(sum(salary), 0) * 1.5 FROM employees e WHERE e.dept_id = d.dept_id)
WHERE d.dept_id = 5
RETURNING name, budget;                          -- Support | 96000.00
```

The classic accident — and the habit that prevents it. `UPDATE` without `WHERE` updates every row. Practise risky statements inside a transaction, check the effect, then `COMMIT` or `ROLLBACK`:

```
BEGIN;
UPDATE employees SET salary = 1;
SELECT count(*) AS rows_now_paid_1 FROM employees WHERE salary = 1;
ROLLBACK;
SELECT count(*) AS rows_paid_1_after_rollback FROM employees WHERE salary = 1;
```

Output:

```
 rows_now_paid_1
-----------------
              15
(1 row)

 rows_paid_1_after_rollback
----------------------------
                          0
(1 row)
```

Other safety habits: run the `WHERE` as a `SELECT count(*)` first; check psql's `UPDATE n` row count; use `RETURNING`; in MySQL clients, `--safe-updates` refuses key-less updates.

Constraints still apply: setting `dept_id = 99` fails the foreign key, `unit_price = -1` fails the `CHECK` (both shown in the file).

9.10 DELETE

What it is

`DELETE` removes rows from a table.

```
DELETE FROM table_name
[USING other_tables]         -- PostgreSQL: join to other tables
[WHERE condition]            -- WITHOUT it, EVERY row is deleted
[RETURNING ...];
```

How it works

1. `WHERE` picks the rows; without it all rows go (the table remains).
2. Deleting a parent that still has children fails or cascades, as its foreign keys declare (9.3).
3. In PostgreSQL the rows are only marked dead; `VACUUM` reclaims the space later.

```
-- code/10_delete.sql
DELETE FROM order_items
WHERE order_id = 105
RETURNING order_id, product_id, quantity;
```

Output:

```
 order_id | product_id | quantity
----------+------------+----------
      105 |          1 |        2
(1 row)
```

```
DELETE FROM customers WHERE customer_id = 1;
```

Output:

```
ERROR:  update or delete on table "customers" violates foreign key constraint "orders_customer_id_fkey"
on table "orders"
DETAIL:  Key (customer_id)=(1) is still referenced from table "orders".
```

Deleting based on another table — PostgreSQL's `DELETE … USING`, or the portable subquery form:

```
DELETE FROM order_items oi
USING orders o
WHERE o.order_id = oi.order_id
  AND o.status = 'cancelled'
RETURNING oi.order_id, oi.product_id;          -- (111, 2)

DELETE FROM orders
WHERE status = 'cancelled'
  AND order_id NOT IN (SELECT order_id FROM order_items)
RETURNING order_id;                            -- 105, 111

DELETE FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id)
RETURNING customer_id, first_name;
```

Output of the last statement:

```
 customer_id | first_name
-------------+------------
           4 | Divya
           9 | Hana
          10 | Ivan
(3 rows)
```

(Divya's only order was a cancelled one, removed just before.) Children first, then parents — the order foreign keys demand. (`NOT IN` with a subquery is safe here only because `order_items.order_id` can't be NULL; prefer `NOT EXISTS` — see chapter 6.)

Delete everything — safely practised:

```sql
BEGIN;
DELETE FROM order_items;
SELECT count(*) AS items_inside_tx FROM order_items;       -- 0
ROLLBACK;
SELECT count(*) AS items_after_rollback FROM order_items;  -- 20
```

Soft delete. Many applications never physically delete business data; they mark it:

```sql
ALTER TABLE products ADD COLUMN deleted_at timestamp;
UPDATE products SET deleted_at = '2025-06-30 12:00' WHERE name = 'Desk Lamp';
CREATE VIEW active_products AS SELECT * FROM products WHERE deleted_at IS NULL;
SELECT count(*) AS all_products, (SELECT count(*) FROM active_products) AS active FROM products;
```

Output:

```
 all_products | active
--------------+--------
            9 |      8
(1 row)
```

	Hard delete	Soft delete
Undo / audit	only from backups or audit tables	trivial: clear `deleted_at`
Queries	simple	every query must filter (use a view, or row-level security — per-user rules that hide rows)
Unique constraints	simple	a deleted row still blocks its email — use a partial unique index (PostgreSQL's filtered index) `WHERE deleted_at IS NULL`
Privacy law ("right to be forgotten")	satisfied	must still purge eventually

Deleting millions of rows: do it in batches (e.g. `DELETE … WHERE id IN (SELECT id … LIMIT 10000)` repeated) to keep transactions and locks short; for time-based data, partitioned tables (one logical table stored as several physical pieces, e.g. one per month) let you `DROP` a whole old partition instantly (chapter 15).

Dialect notes: UPDATE / DELETE using another table

	UPDATE	DELETE
PostgreSQL	`UPDATE t SET … FROM s WHERE t.id = s.id`	`DELETE FROM t USING s WHERE t.id = s.id`
SQL Server	`UPDATE t SET … FROM t JOIN s ON …`	`DELETE t FROM t JOIN s ON …`
MySQL	`UPDATE t JOIN s ON … SET t.col = …`	`DELETE t FROM t JOIN s ON …`
SQLite	`UPDATE t SET … FROM s WHERE …` (3.33+)	subquery only (`WHERE id IN (SELECT …)`)

9.11 Upsert: INSERT … ON CONFLICT, and MERGE

What it is

An **upsert** means "insert this row, or if its key already exists, update (or skip) it". `SELECT`-then-`INSERT`-or-`UPDATE` in application code has a race: two sessions both see "not there" and both insert. PostgreSQL (9.5+) does it atomically (as one indivisible step that no other session can slip in between) with `INSERT … ON CONFLICT`.

How it works

```
INSERT INTO t (cols) VALUES (...)
ON CONFLICT (conflict_target)          -- the unique/PK column(s), or ON CONSTRAINT name
DO NOTHING                             -- skip rows that would conflict
-- or
DO UPDATE SET col = EXCLUDED.col, ...  -- EXCLUDED = the row you tried to insert
          [WHERE condition];           -- update only when this is true
```

1. PostgreSQL tries to insert each row; if it would violate the unique index named by the conflict target, it runs the `DO` action instead of raising an error.
2. In `DO UPDATE`, `t.col` (or an alias) is the **existing** row, `EXCLUDED.col` the **proposed** one.
3. Each row ends up inserted, updated or skipped — atomically, even under concurrency.

```
-- code/11_upsert.sql
CREATE TABLE inventory (
    sku         text PRIMARY KEY,
    qty         integer NOT NULL CHECK (qty >= 0),
    updated_at  timestamp NOT NULL
);
INSERT INTO inventory VALUES ('MOUSE-1', 10, '2025-06-01 08:00'), ('HUB-2', 4, '2025-06-01 08:00');

INSERT INTO inventory VALUES ('MOUSE-1', 99, '2025-06-02 08:00'), ('LAMP-3', 7, '2025-06-02 08:00')
ON CONFLICT (sku) DO NOTHING
RETURNING sku, qty;
```

Output:

```
  sku   | qty
--------+-----
 LAMP-3 |   7
(1 row)
```

MOUSE-1 already existed, so it was skipped silently (and isn't in `RETURNING`).

Insert or add to the existing stock:

```
INSERT INTO inventory AS i VALUES
    ('MOUSE-1', 5, '2025-06-03 08:00'),
    ('KEYB-4',  2, '2025-06-03 08:00')
ON CONFLICT (sku) DO UPDATE
SET qty        = i.qty + EXCLUDED.qty,
    updated_at = EXCLUDED.updated_at
RETURNING sku, qty, updated_at, (xmax <> 0) AS was_update;
```

Output:

```
   sku   | qty |     updated_at      | was_update
---------+-----+---------------------+------------
 MOUSE-1 |  15 | 2025-06-03 08:00:00 | t
 KEYB-4  |   2 | 2025-06-03 08:00:00 | f
(2 rows)
```

(`xmax` is a hidden system column on every PostgreSQL row; `xmax <> 0` is a common trick to tell updated from inserted rows; it relies on an internal detail, so don't build business logic on it.)

Conditional update — only if the incoming data is newer (a common rule when syncing from several sources):

```
INSERT INTO inventory AS i VALUES ('HUB-2', 1, '2025-05-01 00:00')    -- older than what we have
ON CONFLICT (sku) DO UPDATE
SET qty = EXCLUDED.qty, updated_at = EXCLUDED.updated_at
WHERE EXCLUDED.updated_at > i.updated_at
RETURNING sku, qty;
```

Output:

```
 sku | qty
-----+-----
(0 rows)
```

Nothing changed: HUB-2 keeps qty 4 from 1 June (the file lists the table afterwards). **Rules and errors:**

- The conflict target must match a unique index or constraint (`ON CONFLICT (sku)` or `ON CONFLICT ON CONSTRAINT inventory_pkey`). `DO NOTHING` may omit the target (then any unique violation is skipped).
- One statement can't affect the same row twice:

```
INSERT INTO inventory VALUES ('PEN-5', 1, '2025-06-04'), ('PEN-5', 2, '2025-06-04')
ON CONFLICT (sku) DO UPDATE SET qty = EXCLUDED.qty;
```

Output:

```
ERROR:  ON CONFLICT DO UPDATE command cannot affect row a second time
HINT:  Ensure that no rows proposed for insertion within the same command have duplicate constrained
values.
```

De-duplicate the input first (e.g. `SELECT DISTINCT ON (sku) …`).

- Other constraint violations (a `CHECK`, a different unique index) still raise errors.

MERGE (PostgreSQL 15+, SQL Server, Oracle)

`MERGE` is the SQL-standard statement for "compare a source with a target and insert, update or delete accordingly". PostgreSQL added it in **version 15** (SQL Server has had it since 2008, Oracle since 9i);

this book's server is 14, where `MERGE` is a syntax error. The general shape (PostgreSQL 15+ / SQL Server — not run here):

```
MERGE INTO inventory AS t
USING incoming AS s ON t.sku = s.sku
WHEN MATCHED AND s.qty = 0 THEN DELETE
WHEN MATCHED THEN UPDATE SET qty = s.qty, updated_at = s.updated_at
WHEN NOT MATCHED THEN INSERT (sku, qty, updated_at) VALUES (s.sku, s.qty, s.updated_at);
```

	`INSERT … ON CONFLICT`	`MERGE`
Can delete	no	yes (`WHEN MATCHED … THEN DELETE`)
Needs a unique index	yes	no — any join condition
Safe against concurrent inserts of the same key	yes (that's its purpose)	not by itself: two sessions can both take the `NOT MATCHED` branch → unique violation; retry or lock
`RETURNING`	yes	PostgreSQL 17+ (which also added `WHEN NOT MATCHED BY SOURCE` for deleting target rows missing from the source)

Rule of thumb in PostgreSQL: **upserts of single rows/batches → `ON CONFLICT`; synchronising a table from a staging table with deletes → `MERGE`** (15+).

Dialect notes

	Upsert syntax
PostgreSQL	`INSERT … ON CONFLICT … DO NOTHING / DO UPDATE SET … EXCLUDED.col` (9.5+); `MERGE` (15+)
MySQL	`INSERT … ON DUPLICATE KEY UPDATE col = VALUES(col)` (or row alias `AS new … new.col`, 8.0.19+); `INSERT IGNORE` ≈ do nothing
SQLite	`INSERT … ON CONFLICT(col) DO UPDATE SET col = excluded.col` (3.24+); `INSERT OR IGNORE` / `INSERT OR REPLACE`
SQL Server	`MERGE`, or `UPDATE` + `INSERT … WHERE NOT EXISTS` with locking hints
Oracle	`MERGE`

In the interview

30-second answer: "In PostgreSQL, `INSERT … ON CONFLICT (key) DO UPDATE SET col = EXCLUDED.col` — atomic and concurrency-safe because it relies on the unique index; `DO NOTHING` skips duplicates. `MERGE` (PostgreSQL 15+, SQL Server, Oracle) is the standard multi-branch statement that can also delete, but doesn't by itself protect against concurrent inserts."

Self-check

1. *What does `EXCLUDED` refer to?* → The row that was proposed for insertion.
2. *What does `ON CONFLICT` need?* → A unique index/constraint matching the conflict target.

9.12 Backup and restore

What it is

A **backup** is a copy you can rebuild the database from; a **restore** does the rebuilding. Backups protect against what replication (keeping live copies — **replicas** — of the database on other servers; see System

Design Fundamentals) can't: a mistaken `DELETE`/`DROP` (replicas copy it instantly), data-corrupting bugs, ransomware (malware that encrypts your files and demands payment), losing a data centre.

Two terms decide a backup strategy:

- **RPO — recovery point objective**: how much recent data you can afford to lose (e.g. 5 minutes).
- **RTO — recovery time objective**: how long you can afford to be down while restoring (e.g. 1 hour).

Kinds of backups

Kind	What is copied	Tools (PostgreSQL)	Good for	Limits
Logical	SQL / an archive of schema + data	`pg_dump`, `pg_dumpall`	small/medium DBs, single tables, moving across versions	slow for huge DBs; restore rebuilds indexes
Physical	the database files themselves	`pg_basebackup`, file-system snapshots, pgBackRest, Barman	large DBs, fast full restore	whole cluster (in PostgreSQL: one server instance with all its databases) only; same major version (e.g. 16.x)
Continuous (PITR)	a base backup **plus** the write-ahead log (WAL) stream	WAL archiving + base backup	restore to *any moment* (just before the bad `DELETE`)	more setup and storage

> **Key term — WAL and PITR:** the **write-ahead log (WAL)** is a file of every change, written *before* the change is applied to the tables, so the database can replay it after a crash. **Point-in-time recovery (PITR)** restores a **base backup** (a full physical copy) and then replays archived WAL up to any chosen moment — for example one second before a bad `DELETE`.

Other terms: **full** (everything), **differential** (changes since the last full), **incremental** (changes since the last backup of any kind; PostgreSQL 17 added incremental `pg_basebackup`), **hot/online** (taken while the database is in use — all of the above are).

pg_dump and pg_restore, for real

`pg_dump` takes a **consistent** snapshot of one database while others keep reading and writing (it runs inside a single repeatable-read transaction — an isolation level where the transaction sees the data exactly as it was when it started; chapter 13). Formats:

Format flag	Output	Restore with	Notes
`-F p` (plain, default)	a `.sql` script	`psql -f`	readable, editable; replayed statement by statement (`--single-transaction` for all-or-nothing)
`-F c` (custom)	one compressed archive	`pg_restore`	pick objects to restore, reorder, parallel restore (`-j`)
`-F d` (directory)	a folder, one file per table	`pg_restore`	the only format that can **dump** in parallel (`-j`)
`-F t` (tar)	a tar archive (a standard Unix file bundle)	`pg_restore`	like custom, but no compression or parallel restore

The script does a full round trip on the book's private server, in a temporary directory deleted afterwards:

```
# code/12_backup_restore.sh (key lines; CONN="-h /tmp -p 5544 -U postgres")
pg_dump $CONN -d shop -F p -f "$WORK/shop.sql"          # plain SQL script
pg_dump $CONN -d shop -F c -f "$WORK/shop.dump"         # custom archive
pg_dump $CONN -d shop -F d -j 2 -f "$WORK/shop_dir"     # directory format, 2 parallel jobs
pg_restore -l "$WORK/shop.dump"                         # list the archive's contents
psql $CONN -d shop -c "DROP TABLE order_items"          # the "disaster"
pg_restore $CONN -d shop -t order_items "$WORK/shop.dump"      # bring back one table
pg_restore $CONN -d shop_copy -j 2 "$WORK/shop.dump"    # full restore into a new, empty DB
psql $CONN -d shop_plain -f "$WORK/shop.sql"            # plain dumps are replayed by psql
pg_dump $CONN -d shop -t products --schema-only         # structure only (--data-only: rows only)
```

Output (run with `bash chapters/09-ddl-dml-and-constraints/code/12_backup_restore.sh` from the book root):

```
== 1. a database to protect
shop: 10 customers, 14 orders, 22 order_items
== 2. logical backups in three formats
...
-- first CREATE TABLE in the plain dump:
CREATE TABLE public.customers (
    customer_id integer NOT NULL,
    first_name text NOT NULL,
...
-- plain dump contains 8 COPY blocks (one per table with data)
== 3. what is inside the custom-format archive (pg_restore -l = table of contents)
TABLE public customers postgres
TABLE public orders postgres
TABLE DATA public customers postgres
...
== 4. disaster: someone drops a table
ERROR:  relation "order_items" does not exist
== 5. restore just that table (definition + data) from the archive
shop: 10 customers, 14 orders, 22 order_items
-- note: -t restores the table and its rows, not its PK/FK constraints:
0 constraints on order_items
== 6. full restore into a NEW database, 2 parallel jobs
shop_copy: 10 customers, 14 orders, 22 order_items
3 constraints on order_items
== 7. a plain dump is restored with psql
shop_plain: 10 customers, 14 orders, 22 order_items
== 8. schema-only and data-only dumps of one table
CREATE TABLE statements in schema-only dump: 1, in data-only dump: 0
...
```

What it shows:

1. A plain dump is just SQL: `CREATE TABLE`s, a `COPY … FROM stdin` block of rows per table, then constraints and indexes (added *after* the data, which is faster). `stdin` means *standard input*: the rows follow inline in the same script.
2. A custom archive has a **table of contents** (`pg_restore -l`); `-t`, `-n` (schema) and `-L` (edited list) choose what to restore.
3. `-t` restores a table's definition and rows **but not its constraints or indexes** (separate archive entries) — re-add them with `-L` or by re-running the DDL.

Useful options: `--clean --if-exists` (drop objects before recreating), `--create` (create the database itself), `--no-owner` (when restoring as a different user), `-j N` (parallelism). `pg_dump` dumps one database; **roles and tablespaces** (database users/groups, and named disk locations for data) are cluster-wide, so also run `pg_dumpall --globals-only`.

Backup and restore in other databases

	Full backup	Restore	Notes
PostgreSQL	`pg_dump` (logical), `pg_basebackup` + WAL (physical, PITR)	`pg_restore` / `psql`; restore base backup + replay WAL to a target time	command-line tools, not SQL statements
MySQL	`mysqldump`, MySQL Shell dump utilities (logical); Percona XtraBackup / MySQL Enterprise Backup (physical)	`mysql < dump.sql`; binary logs (MySQL's log of every change) for point-in-time	
SQL Server	`BACKUP DATABASE db TO DISK = '…\db.bak'` (also `WITH DIFFERENTIAL`, `BACKUP LOG`, `TO URL` for cloud storage)	`RESTORE DATABASE db FROM DISK = '…' [WITH NORECOVERY / RECOVERY, STOPAT = …]`	backup and restore are T-SQL (SQL Server's SQL dialect) statements; the others mostly use separate tools
SQLite	`.backup file` in the shell, `VACUUM INTO 'file'`	copy the file back / `.restore`	the database is one file

Two corrections to things some tutorials show (SQL Server — not run):

- `BACKUP DATABASE` has no `WITH SCHEDULE = …` option. Backups are scheduled outside the statement — SQL Server Agent jobs or maintenance plans (elsewhere: cron or systemd timers — the standard Linux job schedulers — or a backup tool's scheduler).
- There is no `RESTORE TABLE`. To recover one table, restore the backup under **another database name** (`RESTORE DATABASE db_old FROM DISK = … WITH MOVE …`) and copy the table across — like our `pg_restore -t` or `shop_copy`.

Best practices

- **3-2-1 rule**: 3 copies, on 2 kinds of storage, 1 off-site.
- **Test restores regularly** — an untested backup is a hope. Automate a restore into a scratch server and run sanity queries (row counts, as the script does).
- **Encrypt** backups and restrict who can read them.
- **Match RPO/RTO**: nightly `pg_dump` alone risks 24 h of loss; PITR gets it to seconds.
- **Replication is not a backup** — replicas faithfully copy `DROP TABLE` (chapter 15).
- Managed databases (run for you by a cloud provider: Amazon RDS, Cloud SQL, Azure) provide automated snapshots + PITR within a retention window (how long backups are kept).

In the interview

> **30-second answer:** "Logical backups (`pg_dump`) export schema and data — flexible, single tables, portable across versions, slow for huge databases. Physical backups copy the data files; with archived WAL they allow point-in-time recovery. Choose by RPO/RTO, keep 3-2-1 copies, encrypt, test restores — and replication doesn't protect against a bad `DELETE` ."

- *Someone ran `DELETE` without `WHERE` at 14:03 — how do you recover?* → PITR: restore the last base backup to a new server and replay WAL up to 14:02:59, then copy the lost rows back (or fail over to it — switch the application to use it).
- *pg_dump on a busy database — does it block?* → No; it reads a consistent snapshot. It does take light locks that block DDL like `DROP` or `ALTER` on dumped tables while it runs.

Self-check

1. *Which `pg_dump` formats can `pg_restore` read?* → custom, directory and tar (plain is replayed with `psql`).
2. *Which format can dump in parallel?* → Directory (`-F d -j N`).

3. *What does `pg_dump` not include that `pg_dumpall --globals-only` does?* → Roles and tablespaces.

Chapter 9 TL;DR

- **Types** validate as well as store: `numeric` for money, `integer`/`bigint` for ids, `text`, `timestamptz`, `date`, `boolean`.
- `CREATE` databases, schemas, tables (`IF NOT EXISTS`, `AS SELECT` = data without constraints, `LIKE … INCLUDING ALL` = structure, `TEMP`).
- **Keys**: super ⊇ candidate → one primary (unique + not null) + alternate keys (`UNIQUE`); composite keys; foreign keys with `ON DELETE CASCADE | SET NULL | RESTRICT | NO ACTION`. `CHECK` rejects only false; `UNIQUE` allows several NULLs (one in SQL Server). Name constraints; index FK columns.
- **Generated keys**: `GENERATED ALWAYS AS IDENTITY` (not `serial`), sequences have gaps, UUIDs for distributed ids, `GENERATED … STORED` for derived columns.
- `ALTER`: add/drop/rename columns, `TYPE … USING`, `SET NOT NULL` after backfilling, `NOT VALID` + `VALIDATE` on big tables; watch exclusive locks.
- `DROP` removes the object, `TRUNCATE` empties it fast (transactional in PostgreSQL, `RESTART IDENTITY`, `CASCADE`), `DELETE` removes chosen rows.
- **Indexes**: `CREATE [UNIQUE] INDEX [CONCURRENTLY]`; clustered vs non-clustered; details in chapter 12.
- **DML**: `INSERT` (list columns, multi-row, `INSERT … SELECT`), `UPDATE` (`SET` sees old values, `FROM`), `DELETE` (`USING`, children first), all with `RETURNING`. Practise destructive statements in `BEGIN … ROLLBACK`.
- **Upsert**: `INSERT … ON CONFLICT … DO NOTHING / DO UPDATE SET … = EXCLUDED.…`; `MERGE` needs PostgreSQL 15+.
- **Backups**: `pg_dump` (plain/custom/directory) + `pg_restore` (selective, parallel); physical + WAL for PITR; RPO/RTO, 3-2-1, test restores.

Next: chapter 10 — data modeling and ER diagrams, which decides *which* tables and keys to create in the first place.

Chapter 10 — Data Modeling and ER Diagrams

Up to now the tables in this book were handed to you. In real work nobody hands you tables: somebody describes a business in a few sentences ("we rent bikes to people"), and you must decide what to store, how the pieces connect, and which rules the database should enforce. That decision is **data modeling**, and it matters more than any single query you will write, because any fact left out of the model is a question no SQL, however clever, can ever answer.

This chapter walks the whole path: from a plain-language brief, to a drawing that business people can check (an **ER diagram**), to precise tables with keys and constraints (the **relational model** — the theory of storing all data as tables of rows and columns, §10.11), to `CREATE TABLE` statements running on PostgreSQL. Chapter 11 then tests those tables for repeated data, and Chapter 17 applies everything to nine full case studies.

What you'll learn

1. What a data model is, who reads it, and the three levels it is drawn at (conceptual, logical, physical).
2. A repeatable seven-step modeling process, and the best practices that separate easy-to-change models from painful ones.
3. The ER model: entities, the five kinds of attribute, relationships, degree, cardinality, participation, relationship attributes and weak entities — in **Chen notation** and in **crow's-foot notation** (the one `mermaid erDiagram` draws — Mermaid is a tool that turns a short text description into a diagram).
4. A complete worked example: an employee-management ER diagram built step by step.
5. The relational model: relations, tuples, attributes, domains, `NULL`, the seven kinds of key and the integrity constraints.
6. The mechanical procedure that turns any ER diagram into tables, and the judgement calls it leaves to you.

All runnable code is in `code/`: `00_schema.sql` builds a small bike-rental database; `01`–`04` run on a fresh database each (see Chapter 1 for the private PostgreSQL 14 server). Every "Output:" block below is real psql output (psql is PostgreSQL's command-line client).

10.1 What data modeling is

What it is

Data modeling is deciding three things before you write any SQL:

Part	Plain meaning	Usually becomes
Entity	a *thing* you keep data about (Customer, Bike, Rental) — almost always a noun	a table
Attribute	one *fact* about an entity (a customer's email, a bike's frame number)	a column
Relationship	a *link* between entities (a customer *rents* a bike)	a foreign key (a column holding another table's key), or a table of its own

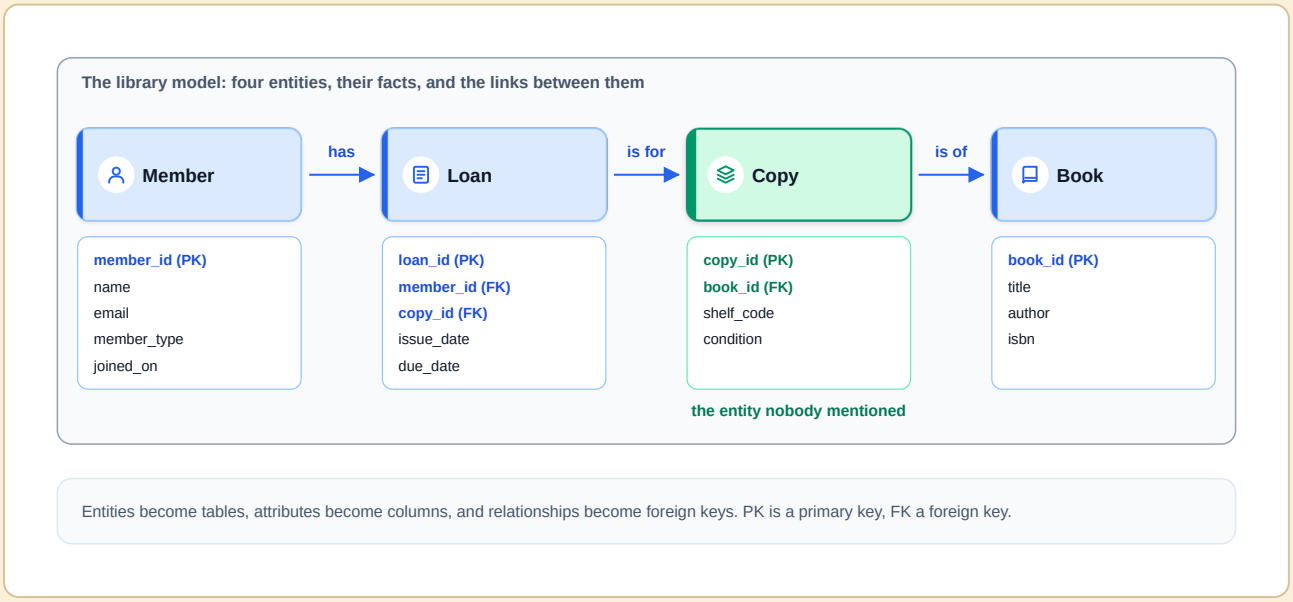
The result — a **data model** — is a picture of those three things and nothing else. It says nothing about indexes (lookup structures that speed up searches), `VARCHAR` lengths or which database product you will use. Analogy: it is the architect's floor plan, drawn before anyone picks brick suppliers.

The problem it solves — the question you cannot answer

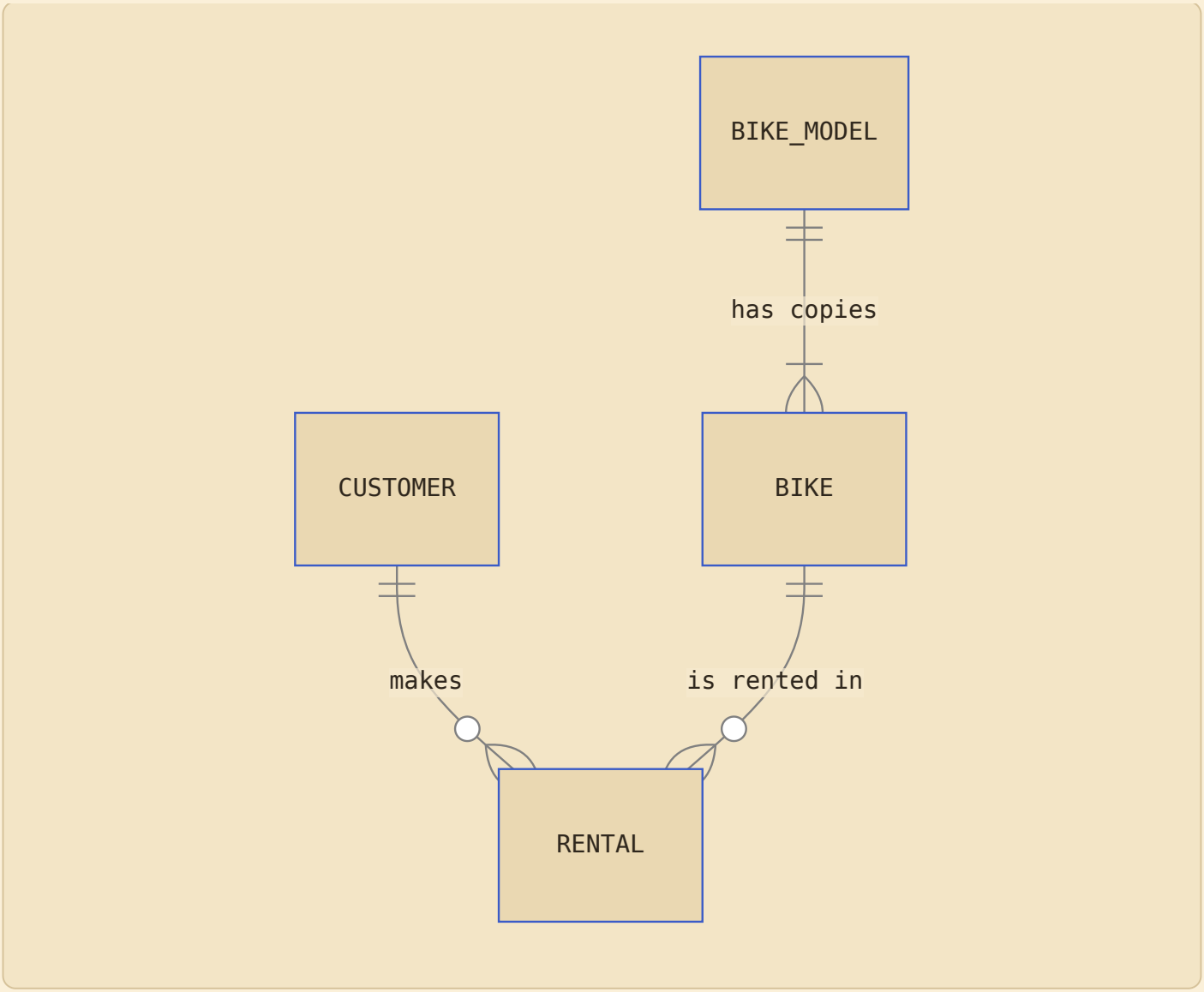
A small bike-rental shop says: *"We rent bikes to customers."* That sounds like two tables, `customer` and `bike`. Then the owner asks: *"We own three 'Urban 3' bikes. One came back with a bent wheel — which one, and who had it?"*

If your `bike` table actually describes a **model** of bike (name, type, hourly rate), it cannot answer. The physical object with a frame number is a different thing from the product it is an example of. Add a separate entity for the physical bike and the question becomes a short join. The code got no smarter; the database simply began recording one more fact.

Figure — the same idea in the course's library example: a Book (a title) is not a Copy (the object on the shelf) (course diagram)



The bike shop's model therefore has four entities: `customer`, `bike_model` (the "title"), `bike` (the physical object) and `rental` (who had which bike, when).



Who reads a data model

Reader	What they check
Business people (shop owner, registrar, librarian)	"Is anything missing? Are the rules right?" — they spot a forgotten entity in seconds
Developers	which tables exist and what each one means
Database administrators	storage, indexes, backups (mostly at the physical level)
Analysts	where each fact lives, so reports are correct

Since non-technical readers must follow it, you describe the model in everyday language first and add technical detail afterwards — which is why §10.2 has three levels.

What data modeling is *not*

- **Not writing SQL.** SQL comes last and is nearly mechanical once the model is right (§10.15).
- **Not performance tuning.** Indexes and query plans (the step-by-step strategy the database picks for running a query) come later (Chapter 12). Quickly answering the wrong question helps nobody.
- **Not application code.** Two very different apps can share one model.

Why it matters

- **Fewer wrong answers** — rules in the model become constraints that every program must obey.

- **Less repeated data** — each fact stored once cannot contradict itself.
- **Cheaper change** — a model that matches reality absorbs new requirements; one that doesn't fights each of them.
- **Shared vocabulary** — when the team says "rental", everyone means the same thing.

TL;DR

A data model = entities (things) + attributes (facts) + relationships (links). Entities → tables, attributes → columns, relationships → foreign keys. If the model does not hold a fact, no query can recover it.

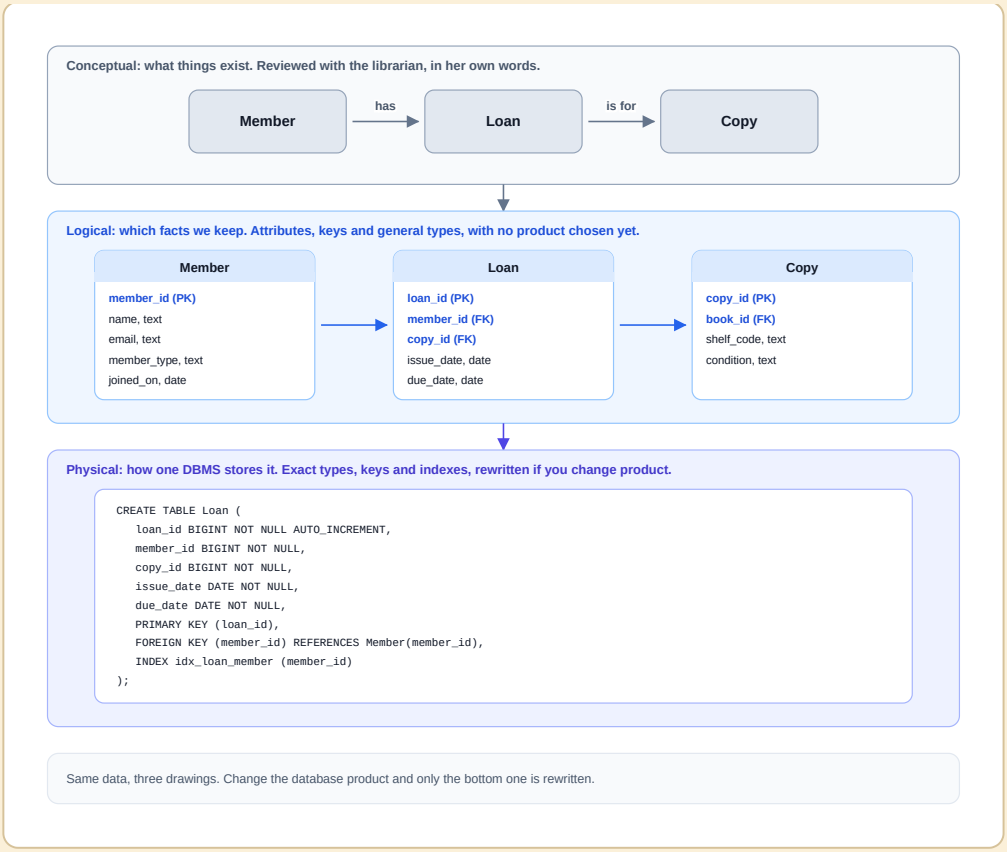
10.2 The three levels of a data model

What it is

The same model is drawn three times, each time with more detail, for a different audience:

	Conceptual	Logical	Physical
Shows	entities and relationships only	+ attributes, keys, general types (text/number/date)	+ exact types, indexes, partitioning (splitting a big table into pieces), storage options
Detail	low	medium	high
Main audience	business people	designers, developers	DBAs (database administrators), developers
Tied to a DBMS (database management system, e.g. PostgreSQL) product?	no	no	**yes**
Question it answers	"Is every thing present, and is each link correct?"	"Which facts are recorded, and which keys join the tables?"	"How do we store this well on *this* database?"
Bike-shop example	Customer *rents* Bike; Bike *is of* BikeModel	`rental(rental_id, customer_id → customer, bike_id → bike, out_at, due_at, back_at)`	`CREATE TABLE rental (... timestamp ..., INDEX on customer_id)`

Figure — the same data at the three levels (course diagram)



How it works

1. **Conceptual** — boxes and lines, in the users' own words. Technology-independent: it would be equally valid for a relational database, a document store (a database that keeps JSON-like documents instead of tables) or a paper ledger.
2. **Logical** — add attributes, primary and foreign keys, and general types. This is where **normalization** happens (removing repeated data — Chapter 11). Still product-neutral.
3. **Physical** — rewrite for one DBMS: `bigint` vs `int`, `numeric(6,2)`, `timestamp`, indexes for the known queries, identity columns (keys the database numbers automatically), partitioning.

The physical version of the rental table from `code/00_schema.sql`:

```
CREATE TABLE rental (
  rental_id   bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  customer_id bigint      NOT NULL REFERENCES customer(customer_id),
  bike_id     bigint      NOT NULL REFERENCES bike(bike_id),
  out_at      timestamp   NOT NULL,
  due_at      timestamp   NOT NULL,
  back_at     timestamp,                          -- NULL = bike still out (meaningful NULL)
  CHECK (due_at > out_at),
  CHECK (back_at IS NULL OR back_at >= out_at)
);
CREATE INDEX rental_customer_idx ON rental(customer_id);   -- "rentals of this customer"
CREATE INDEX rental_bike_idx     ON rental(bike_id);       -- "who has this bike"
```

The entity came from the conceptual model, the columns and keys from the logical model; only the exact types, the identity column and the two indexes are physical decisions. Identity syntax differs by product, which is exactly why this level is product-specific:

Auto-numbered key	Syntax
PostgreSQL 10+	`bigint GENERATED ALWAYS AS IDENTITY` (older: `bigserial`)
MySQL	`BIGINT AUTO_INCREMENT`
SQLite	`INTEGER PRIMARY KEY` (becomes the rowid, SQLite's built-in row number)
SQL Server	`BIGINT IDENTITY(1,1)`

Why bother with three levels

- **Every reader gets a version pitched at them.** A shop owner reviewing three boxes finds a missing entity; the same person shown `CREATE TABLE` finds nothing.
- **Mistakes are caught while cheap.** A missing entity costs a conversation at the conceptual level and a migration (a script that changes an existing schema and its data) once two million rows exist.
- **Technology is isolated.** Switch from MySQL to PostgreSQL and only the physical model is rewritten. This is **data independence** (changing how data is stored without rewriting the programs that use it; defined in Chapter 1) applied at design time.

The levels are not a one-way street: working out the logical model often shows the conceptual one was wrong, and you go back. That is cheap precisely because the conceptual drawing is small.

Common mistakes

- Showing column types to business reviewers (they stop reading).
- Calling a diagram with attributes and keys "conceptual" — once keys appear it is logical.
- Treating logical and physical as the same: *logical = what the business needs; physical = how one product stores it.*

Self-check

1. Which level first shows primary keys? → **Logical.**
2. You migrate from SQL Server to PostgreSQL. Which levels change? → **Only the physical.**
3. Where does normalization happen? → **Logical level.**

TL;DR

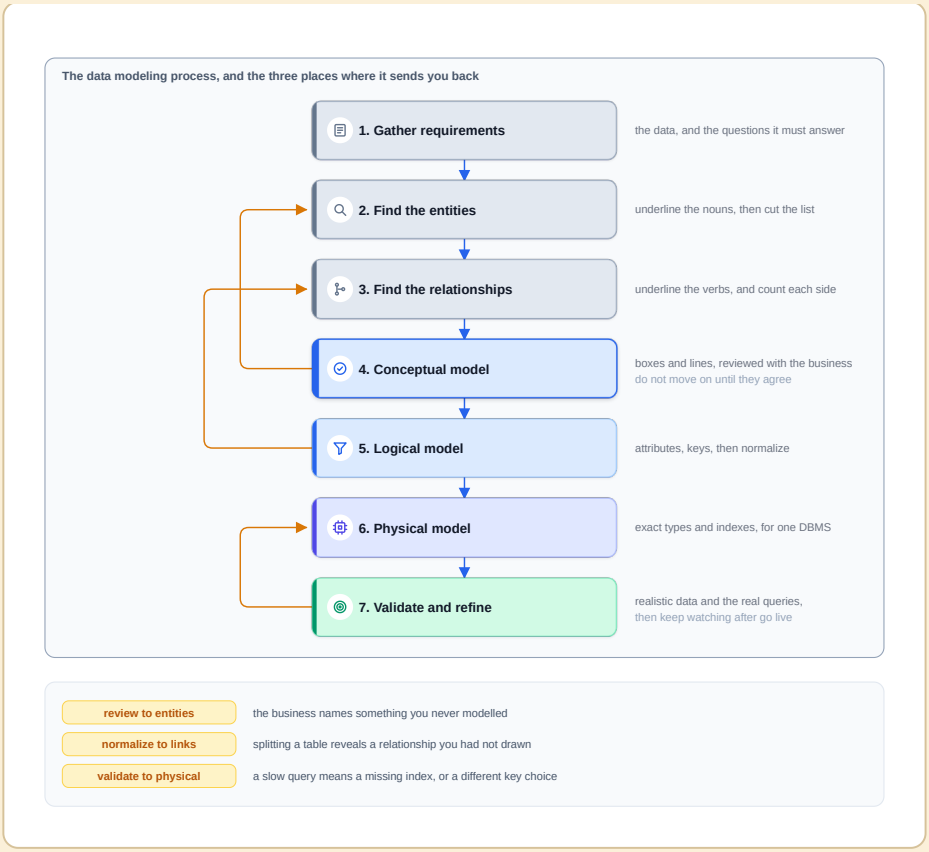
Conceptual (things + links, for the business) → logical (attributes, keys, general types, normalized) → physical (exact types, indexes, one DBMS). Only the physical level depends on the product.

10.3 The data modeling process

What it is

A fixed order of work where every step produces something you can show someone. Starting with tables is the classic mistake — it fixes the answer before you understand the question.

Figure — seven steps and the three places the work loops back (course diagram)



How it works — the bike shop, step by step

1. **Gather requirements — including the questions.** Talk to the people doing the work. Write down what must be recorded *and* what the system must answer:
   - Customers are casual or members; members may keep a bike longer.
   - The shop owns several physical bikes of each model.
   - A customer takes a bike at a time and must return it by a due time.
   - Late returns cost 2.00 per started hour.
   - Staff must see which bikes are out and who has them.

   An unanswerable question is a bug in the design, and right now it costs nothing to fix.
2. **Find the entities: underline the nouns.** customer, member, model, bike, time, fee, staff. Keep a noun as an entity only if you store *several facts* about it and there are *many instances* of it:
   - customer, bike_model, bike, rental → entities.
   - time → one value → an attribute.
   - casual/member → two kinds of customer → start with a `customer_type` attribute.
   - late fee → calculable from times → a **derived** value (§10.6), not stored.
   - staff → users of the system, not necessarily data in it.
3. **Find the relationships: underline the verbs between nouns.** customer *rents* bike; bike *is of* bike_model. For each, ask "how many on each side?" (cardinality, §10.7) and write it in words: one customer has many rentals; each rental is for one bike.
4. **Draw and review the conceptual model** with the owner, reading it back as sentences. She will correct you ("customers can also *reserve* a bike for tomorrow") — a new entity for the price of one box. Do not continue until the business agrees.
5. **Design the logical model.** Attributes, primary keys, foreign keys, general types; then normalize.
6. **Build the physical model.** Exact types, and an index for every question from step 1 (`rental.customer_id`, `rental.bike_id`).

7. **Validate, implement, refine.** Load realistic volumes (not ten rows — enough rows for slowness to show up), run the step-1 questions, check correctness and speed. After go-live (the launch to real users) keep watching: a model right for 10 thousand rentals may need a new index at 10 million.

The loops back are part of the process

Trigger	Sends you back to
Stakeholder (anyone with a say in the system) review reveals something new (reservations)	step 2 (entities)
Normalization splits a table and exposes a hidden link	step 3 (relationships)
Validation finds a slow query	step 6 (index or key choice)

Looping back is the method working, not failing: every loop costs less than releasing the error.

Trade-offs to keep in mind throughout

- **Normalize vs denormalize** — normalize first (split tables so each fact is stored once); add redundancy (deliberate duplicate copies) back only for a measured slow query (Chapter 11).
- **Growth** — rows in three years, not today; it changes key and index choices.
- **Access control** — mark sensitive data early (emails, payments).
- **Integrity** — each business rule you collected in step 1 belongs in a constraint, not in an `if` inside the application.

The model answering the step-1 questions (`code/01_bike_questions.sql`)

```sql
-- Q1: which bikes are out right now, and who has them?
SELECT b.frame_no, m.model_name, c.full_name, r.due_at
FROM rental r
JOIN bike b       ON b.bike_id = r.bike_id
JOIN bike_model m ON m.model_id = b.model_id
JOIN customer c   ON c.customer_id = r.customer_id
WHERE r.back_at IS NULL
ORDER BY b.frame_no;
-- Q2: bikes never rented (NOT EXISTS anti-join)   Q4: late fee, derived at read time
SELECT r.rental_id, c.full_name,
       ceil(extract(epoch FROM (r.back_at - r.due_at)) / 3600) * 2.00 AS late_fee
FROM rental r JOIN customer c USING (customer_id)
WHERE r.back_at > r.due_at;
```

Output:

```
--- Q1: which bikes are out right now, and who has them?
 frame_no | model_name |  full_name   |       due_at
----------+------------+--------------+---------------------
 FR-2001  | Ridge 29   | Meera Nair   | 2026-09-27 18:00:00
 FR-3001  | Volt City  | Lena Fischer | 2026-09-27 10:30:00
(2 rows)

--- Q2: which physical bikes have never been rented?
 frame_no | model_name
----------+------------
 FR-1002  | Urban 3
 FR-1003  | Urban 3
(2 rows)

--- Q3: how many bikes of each model exist, and how many are out?
 model_name | bikes | out_now
------------+-------+---------
 Ridge 29   |     1 |       1
 Urban 3    |     3 |       0
 Volt City  |     1 |       1
(3 rows)

--- Q4: late returns, with a late fee of 2.00 per started hour (fee is derived, not stored)
 rental_id | full_name  | late_fee
-----------+------------+----------
         2 | Tom Okafor |     2.00
(1 row)
```

Q2 and Q3 are only possible because `bike` (the physical object) is separate from `bike_model`.

In the interview

30-second answer: "I start from requirements *and the questions the system must answer*. Nouns give candidate entities, verbs give relationships. I draw a conceptual model and review it with the business, then add attributes and keys and normalize (logical), then choose types and indexes for the target database (physical), and validate with realistic data and the real queries. I expect to loop back."

- *How do you decide whether a noun is an entity or an attribute?* → Entity if it carries several facts and has many instances; a single value (a date, a colour) is an attribute.
- *When do you add indexes?* → At the physical level, for the queries written down in requirements — not "on every column".

TL;DR

Requirements (+ questions) → nouns = entities → verbs = relationships → conceptual review → logical (keys, normalize) → physical (types, indexes) → validate with real data; loop back when reality disagrees.

10.4 Best practices in data modeling

Two models can both work on day one and differ enormously a year later. The difference comes from small early decisions. Here are the rules, each with the reason.

Practice	Rule	Why
Questions first	Write the list of questions before drawing.	A model is only "correct" relative to the questions it must answer.
Rules in the database	Every stated rule → a `PRIMARY KEY`, `FOREIGN KEY`, `NOT NULL`, `UNIQUE` or `CHECK`.	A constraint protects every program that ever connects, including tomorrow's ad-hoc (one-off, written for a single task) script; an `if` protects only the program it is in.
One naming convention	Decide once (`snake_case` = lower case with underscores, singular *or* plural tables) and never mix. Same thing = same name everywhere (`customer_id`, never `cust_no` elsewhere). Name FKs after their target.	Joins read naturally; nobody has to guess.
Types on purpose	Dates as `date`/`timestamp`, never text; money as `numeric`, never `float`/`double`; text sized for real data. Prefer the narrowest type that fits.	Text dates accept "31 February" and sort wrongly; floating point cannot represent most decimals exactly, so totals drift; narrower rows = more rows per page (the fixed-size block the database reads from disk) = fewer disk reads.
Decide what NULL means	`NOT NULL` whenever the fact must exist; allow `NULL` only when "missing" is meaningful (`back_at IS NULL` = bike still out).	`NULL` means unknown/not applicable, is not 0 or '' and is not equal to anything (§10.12).
Keys deliberately	Surrogate primary key (a generated id with no business meaning) + `UNIQUE` on the natural key (a real-world identifier: `email`, `frame_no`). Keep keys narrow.	Natural values change; a PK is copied into every FK and index.
Normalize, then denormalize with evidence	Default to 3NF (third normal form, the usual target of normalization); denormalize only after a measured slow query that an index could not fix — and document why.	Redundant copies can disagree (Chapter 11).
Index for real queries	Index what the requirement questions filter/join on; nothing else.	Every index slows every write and costs disk.
Design for predictable growth	Estimate rows in 3 years; don't build tables for imagined requirements.	Speculative tables become the parts nobody understands.
Document meaning	For each entity/column: what it means, allowed values, units.	Anyone can see `status` is text; only docs say which 4 values it holds.

The CHECK constraints in the bike schema are this practice in action: a rental due before it starts, or returned before it left, is refused for every client.

```
CHECK (due_at > out_at),
CHECK (back_at IS NULL OR back_at >= out_at)
```

Common mistakes

- **A list in one column** — `phones = '555-1, 555-2'` cannot be searched, counted or constrained → separate table.
- **Numbered columns** — `phone1, phone2, phone3` stores a list across columns and breaks the day someone owns a fourth phone.
- **One huge table** — 60 columns, mostly empty per row, is several entities that were never separated.
- **Over-normalizing** — seven joins for an everyday question is a real cost; 3NF is the usual stopping point.

- **Rules only in code** — some other program will break them.

Real-world examples

- **Stripe's API** (the programming interface of the payments company Stripe) represents amounts as integers in the smallest currency unit (cents) — the same idea as "never float for money".
- **ISO country codes** (the international standard two-letter codes such as `IN`, `GB`) are `char(2)` — "size text for the data" in the wild.

TL;DR

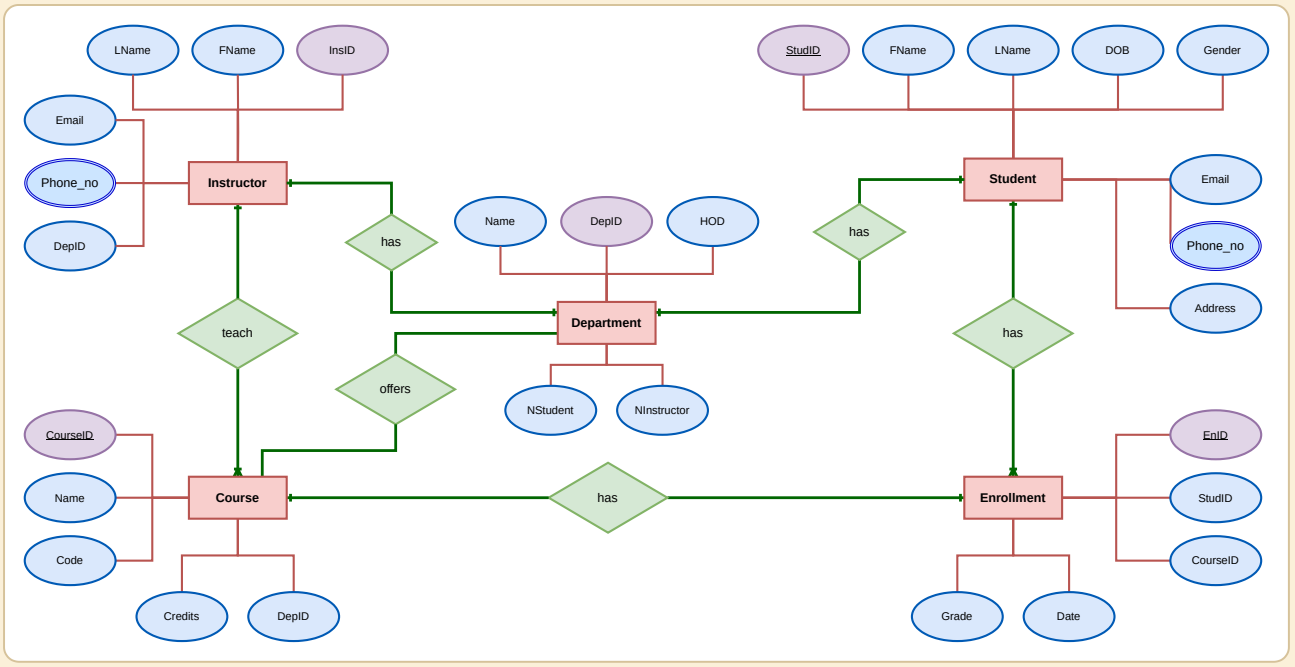
Questions first; constraints in the schema; one naming style; correct types (dates as dates, money as `numeric`); explicit `NOT NULL`; surrogate PK + unique natural key; normalize to 3NF, denormalize with measurements; index only real queries; document meaning.

10.5 The Entity-Relationship (ER) model and its notations

What it is

The **Entity-Relationship model** (ER model), proposed by Peter Chen in 1976, lets you sketch entities, attributes and relationships on paper long before you create a table. An **ER diagram (ERD)** is one such drawing. It is how you draw the conceptual model (entities + relationships only) and then the logical model (the same diagram with attributes, keys and cardinalities added). It never names a DBMS.

Figure — a complete Chen-notation ER diagram for a student management system; by §10.10 you can read every symbol (course diagram)



Two notations for the same model

	Chen notation	Crow's-foot notation (IE / Martin)
Entity	rectangle	rectangle with a title bar
Attribute	ellipse attached by a line	a row listed *inside* the rectangle
Relationship	diamond labelled with a verb	the line itself, labelled with a verb
Cardinality	`1`, `N`, `M` written near the diamond (or `(min,max)` pairs)	the *shape* of each line end (see table below)
Participation	double line = total, single = partial	inner symbol: bar = at least one, circle = may be zero
Used by	textbooks, exams, teaching	nearly every design tool (dbdiagram, MySQL Workbench, ERwin, Lucidchart), and `mermaid erDiagram`
Strength	shows the concepts explicitly	compact; fits 40 tables on a screen

Neither is "more correct". Learn Chen to understand the concepts; you will read crow's foot immediately afterwards. In this book the course's Chen figures show the concepts, and every schema is also drawn in crow's foot with mermaid.

Reading crow's-foot line ends — each end has an *outer* symbol (maximum) and an *inner* symbol (minimum):

Mermaid syntax (left end / right end)	Picture	Meaning (min, max)
`|o` / `o|`	circle + bar	zero or one `(0,1)`
`||` / `||`	bar + bar	exactly one `(1,1)`
`}o` / `o{`	circle + crow's foot	zero or more `(0,N)`
`}|` / `|{`	bar + crow's foot	one or more `(1,N)`

So `CUSTOMER ||--o{ RENTAL : makes` reads: *each rental belongs to exactly one customer; a customer makes zero or more rentals*. Strictly, mermaid draws a solid line `--` for an **identifying** relationship (the child's key includes the parent's — a weak entity, §10.9) and a dashed line `..` for a non-identifying one. This book draws every line solid for readability; a weak entity is recognised by a PK that contains the owner's key.

Strengths and limits of an ERD

- **Good at:** making the things, their links and the counts on each link visible, so even a non-technical reviewer notices what is missing.
- **Not a table diagram:** a single M:N diamond turns into *three* tables (§10.7).
- **Not a performance document:** no indexes or storage.
- **Not a process diagram:** it says a student enrols in a course, not which screens or approvals happen.

> **TL;DR**
>
> The ER model (Chen, 1976) draws entities, attributes and relationships. Chen = rectangles/ellipses/diamonds (teaching); crow's foot = attributes inside boxes and cardinality in line ends (industry, mermaid). Same model, two pictures.

10.6 Entities and the five kinds of attribute

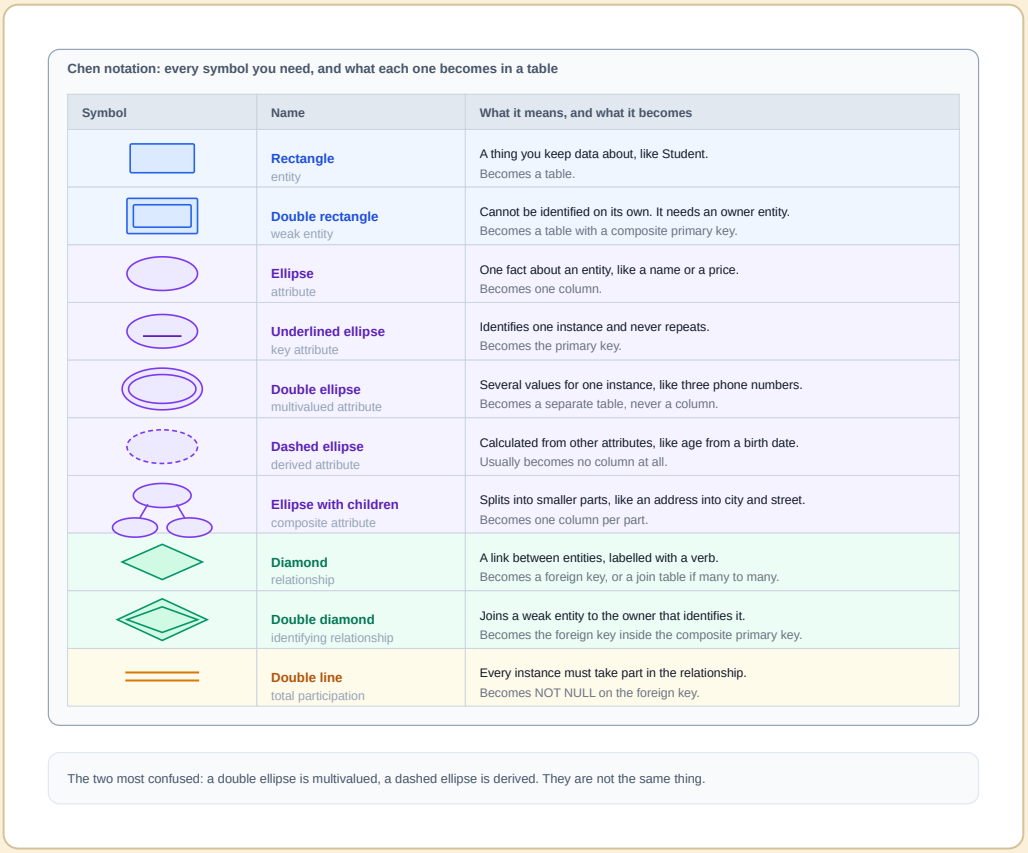
Entities: type, instance, set

An **entity** is any object or concept in the business that you record data about and can distinguish from others like it. Three words are used loosely but mean different things:

Term	Meaning	Example
Entity type	the general kind; what the rectangle is labelled	`Employee`
Entity instance	one particular member	employee #2, Ravi Menon
Entity set	all instances that exist right now	every row in the `employee` table today

You *draw* types; the database *stores* sets.

Figure — every Chen symbol and its meaning (course diagram)

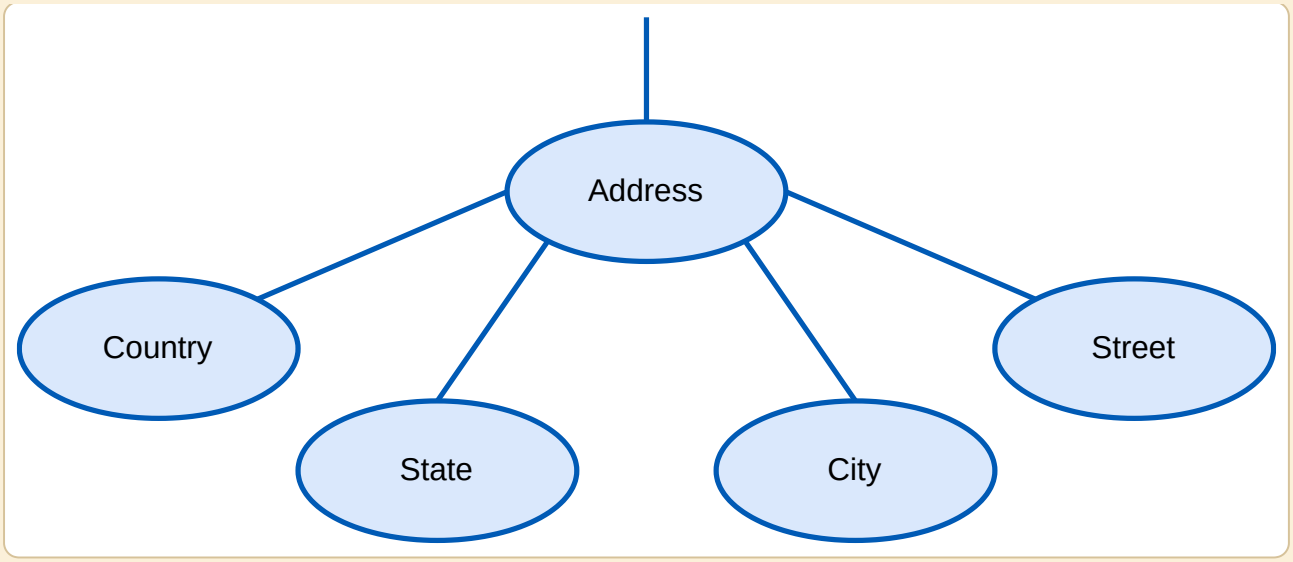


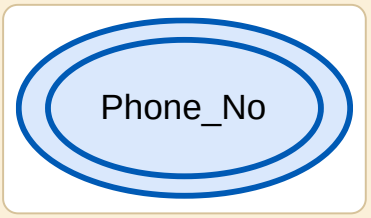
The five kinds of attribute

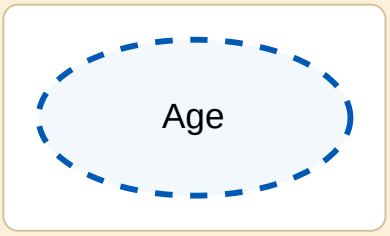
Kind	Chen symbol	Example	Becomes in tables
Key	ellipse, name **underlined**	`emp_id`	the primary key
Simple (atomic)	plain ellipse	`birth_date`	one column
Composite	ellipse with child ellipses	`full_name` = first + middle initial + last	one column **per part**
Multivalued	**double** ellipse	`phone` (an employee may have several)	a **separate table** (owner key + value)
Derived	**dashed** ellipse	`age` (from `birth_date`)	usually **no column** — computed on read

The two most confused: **double ellipse = multivalued**, **dashed ellipse = derived**.

Figure — composite, multivalued and derived attributes in Chen notation (course diagrams)



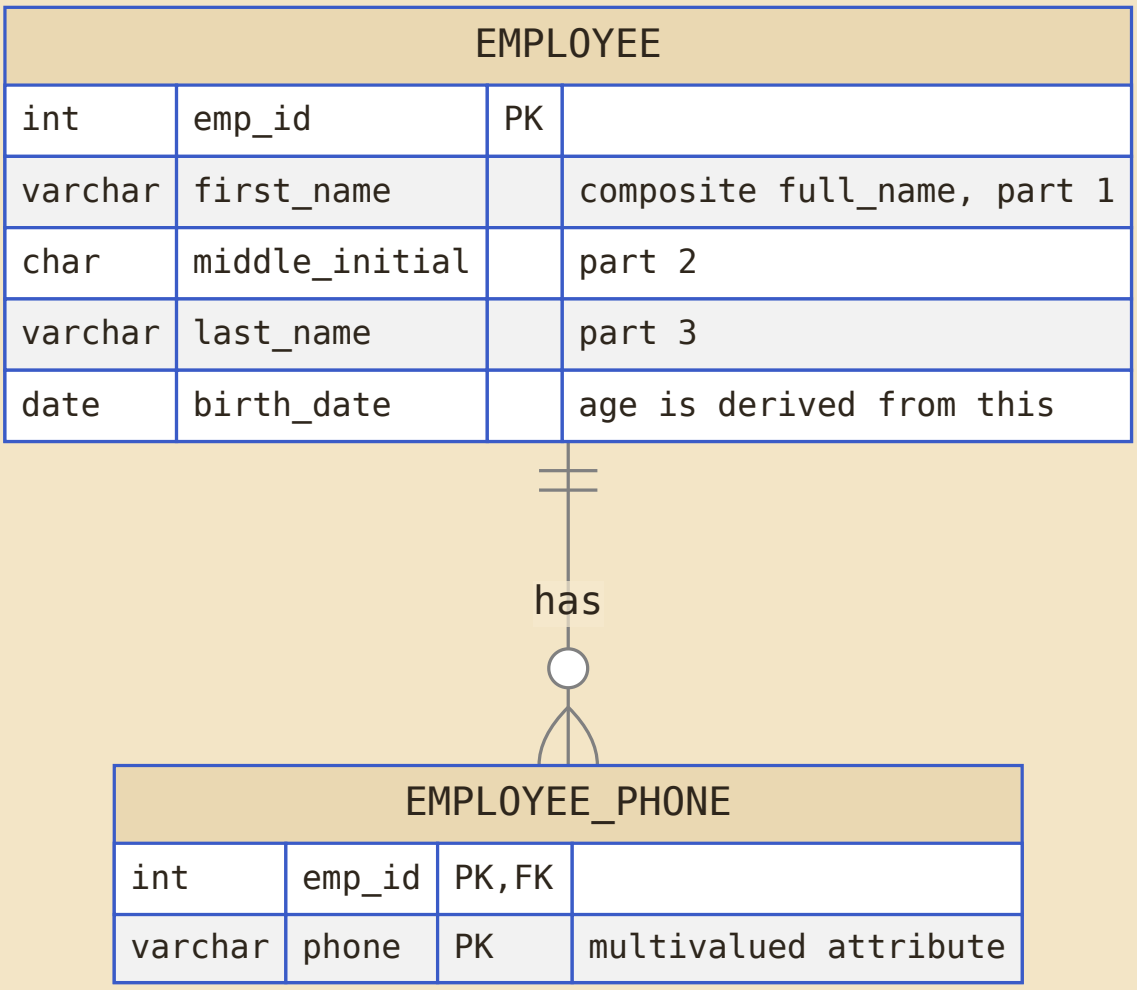




Why each "becomes" rule holds:

- **Composite → split.** If the whole address sits in one column you cannot search by city. (If you *never* query the parts, one column is fine — split the day you need to.)
- **Multivalued → own table.** A relational cell holds exactly one value (§10.12), so a list cannot be a column. `phone1, phone2, phone3` is the mistake the double ellipse exists to prevent.
- **Derived → compute.** A stored `age` is wrong a year later. Store a derived value only if it is expensive to compute *and* you accept keeping it fresh — or if it must be **frozen in time** (an order total at the prices of the day of purchase is not really derivable from *today's* data).

In crow's foot there are no special shapes: the composite is already split into columns, the multivalued attribute is already a separate box, and a derived attribute is usually omitted or marked with a comment:



Self-check

1. A double ellipse means…? → **Multivalued** (e.g. phone numbers).
2. `age` next to `birth_date` — which kind, and what column? → **Derived**; no column, compute it.
3. `address` split into street/city/postcode is…? → **Composite**; one column per part.

TL;DR

Key (underlined) → PK; simple → column; composite → one column per part; multivalued (double ellipse) → separate table; derived (dashed) → computed, not stored.

10.7 Relationships: degree, cardinality and participation

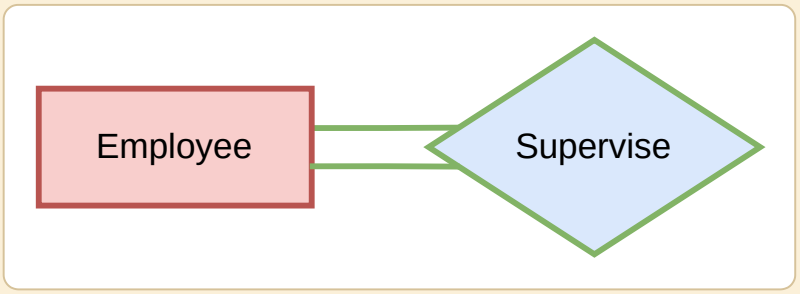
A **relationship** is an association between entities, drawn in Chen as a diamond labelled with a verb that reads well aloud ("Employee *works in* Department", not "Employee *has* Department"). As with entities, the **relationship type** is the general association you draw, and the **relationship set** is the actual pairs stored (employee 2 works in department 10, …).

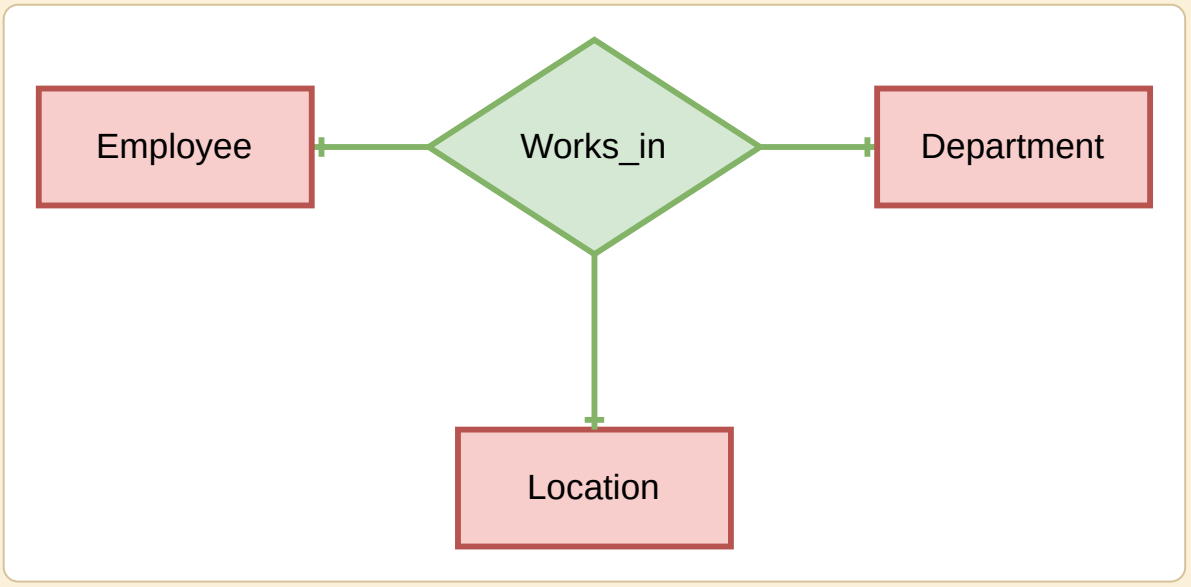
Three separate questions describe every relationship: **degree** (how many entity *types*), **cardinality** (how many *instances* on each side) and **participation** (is taking part *required*).

Degree

Degree	Name	Example	Tables
1	**Unary** (recursive)	an employee *supervises* employees	self-referencing FK: `employee.supervisor_id → employee.emp_id`
2	**Binary**	employee *works in* department	the usual case
3	**Ternary**	supplier *supplies* part *to* site	one table with three FKs, usually all three in the PK

Figure — unary and ternary relationships (course diagrams)





A ternary relationship records a fact that three binary relationships cannot: *which* combination is true. The demo in `code/04_ternary_and_weak_entity.sql` stores three supplier-part-site facts, then keeps only the binary pairs and joins them back:

```
WITH sp AS (SELECT DISTINCT supplier_id, part_id FROM supplies),
     ss AS (SELECT DISTINCT supplier_id, site_id FROM supplies)
SELECT ... FROM sp JOIN ss USING (supplier_id) ...   -- rebuild from pairs
```

Output:

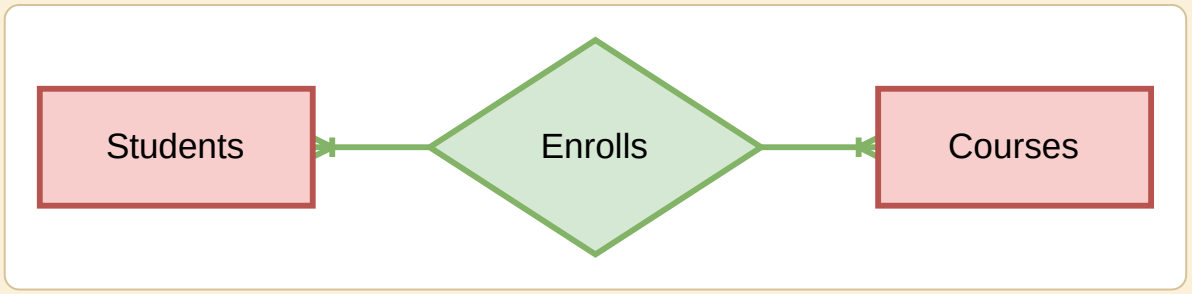
```
--- the ternary fact: who supplies what, where
 supplier | part  | city
----------+-------+-------
 Acme     | bolt  | Leeds
 Acme     | panel | Pune
 Bolt Co  | bolt  | Pune
(3 rows)

--- keep only binary pairs (supplier-part, supplier-site) and join them back:
 supplier | part  | city  | really_true
----------+-------+-------+-------------
 Acme     | bolt  | Leeds | t
 Acme     | bolt  | Pune  | f
 Acme     | panel | Leeds | f
 Acme     | panel | Pune  | t
 Bolt Co  | bolt  | Pune  | t
(5 rows)
```

From the pairs alone you cannot tell whether Acme sends bolts to Pune. Ternary relationships are rare, though — before drawing one, check the fact truly needs all three at once (Chapter 11's 5NF, §11.13, is the formal version of this question).

Cardinality (mapping cardinality)

Cardinality	Meaning	Example
1:1	each side links to at most one of the other	employee ↔ payroll record
1:N	one on the left, many on the right	a department employs many employees
N:1	1:N viewed from the opposite side	many employees work in one department
M:N	many on both sides	employees work on many projects; a project has many employees

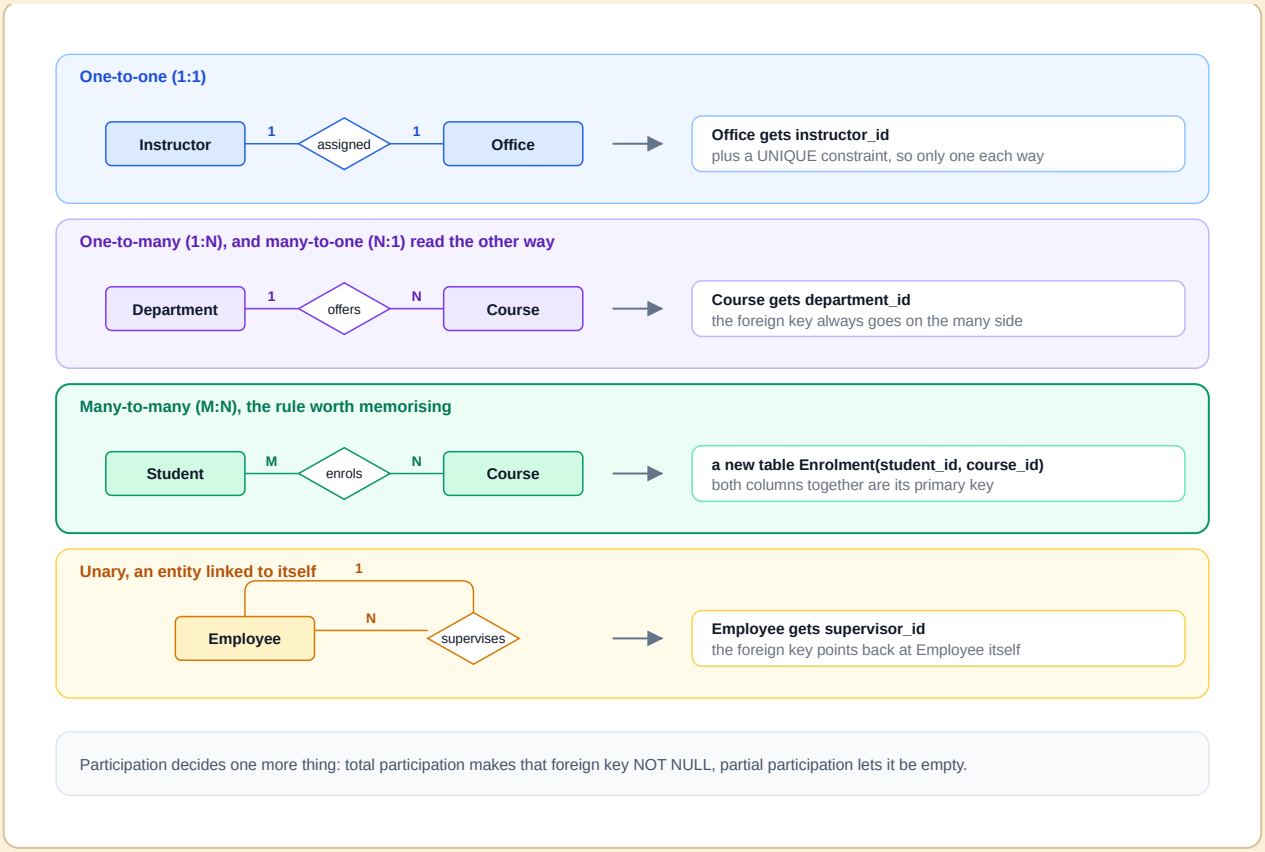
Figure — many-to-many in Chen notation (course diagram)



1:N and N:1 are one relationship described from opposite ends; both names are correct.

How each cardinality becomes tables

Figure — cardinality to foreign keys (course diagram)



Cardinality	Rule	Why
1:N / N:1	FK on the **many** side (`employee.dept_id`)	the "one" side would need a list in a cell
M:N	a new **junction table** (also *join*, *bridge* or *associative* table) holding both FKs, which together form its PK (`works_on(emp_id, project_id)`)	there is no single side to put it on
1:1	FK on **either** side + `UNIQUE`; prefer the side with total participation so it can be `NOT NULL`	without `UNIQUE` it silently becomes 1:N
Unary	FK to the same table	

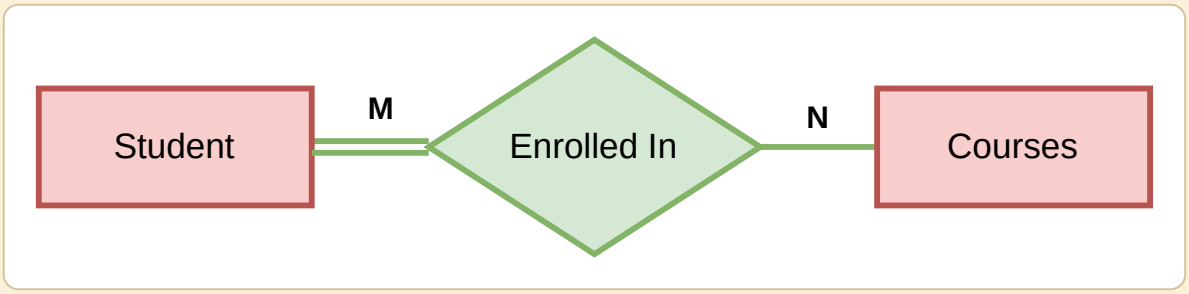
Memorise this one: **every M:N link in a diagram costs three tables in SQL.**

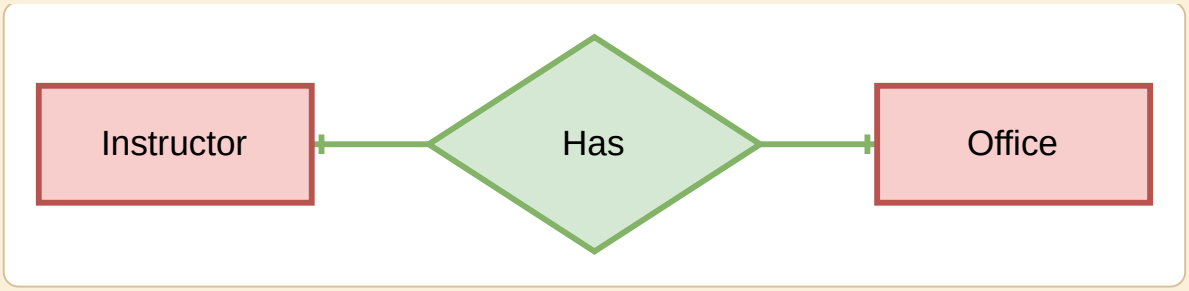
Participation

Participation says whether every instance *must* take part.

	Chen drawing	Meaning	Becomes
Total (mandatory)	double line	every employee must work in a department	FK `NOT NULL`
Partial (optional)	single line	an employee may have no supervisor	FK allows `NULL`

Figure — total (double line) and partial (single line) participation (course diagrams)





Skipping participation is easy and expensive: it decides whether a column can be empty, and that decision affects every query (outer vs inner joins, `IS NULL` checks).

The (min, max) notation

Design tools often label each end with a pair **(min, max)** — the fewest and most times one instance can take part:

Pair	Meaning	Participation	Crow's foot
`(0,1)`	at most once, maybe never	partial	`o|`
`(1,1)`	exactly once	total	`||`
`(0,N)`	any number, including none	partial	`o{`
`(1,N)`	at least once	total	`|{`

The minimum is the participation; the maximum is the cardinality. That is why tools prefer the pair: one label says both.

Careful: in Chen's `(min,max)` style the pair is usually written next to the entity it describes ("each employee works in (1,1) departments"), which is the *opposite* end from where crow's foot draws its symbol. Always check a diagram's legend.

In the interview

30-second answer: "Degree counts the entity types involved — one (unary), two (binary) or three (ternary). Cardinality is how many instances link: 1:1, 1:N or M:N. Participation is whether taking part is mandatory. 1:N puts the FK on the many side; M:N needs a junction table; 1:1 is an FK plus UNIQUE; total participation makes the FK NOT NULL."

- *Where does the FK go in a 1:N?* → Many side. *Why not the one side?* → It would need a list in one cell, which violates 1NF (first normal form: one value per cell).
- *How do you enforce 1:1 in SQL?* → FK + `UNIQUE` on it (or share the same PK in both tables).
- *Can you enforce "every department has at least one employee" with a plain FK?* → No; an FK constrains the child side. You need a deferred constraint trigger (a trigger — code the database runs automatically on a change — whose check waits until commit) or application logic.

TL;DR

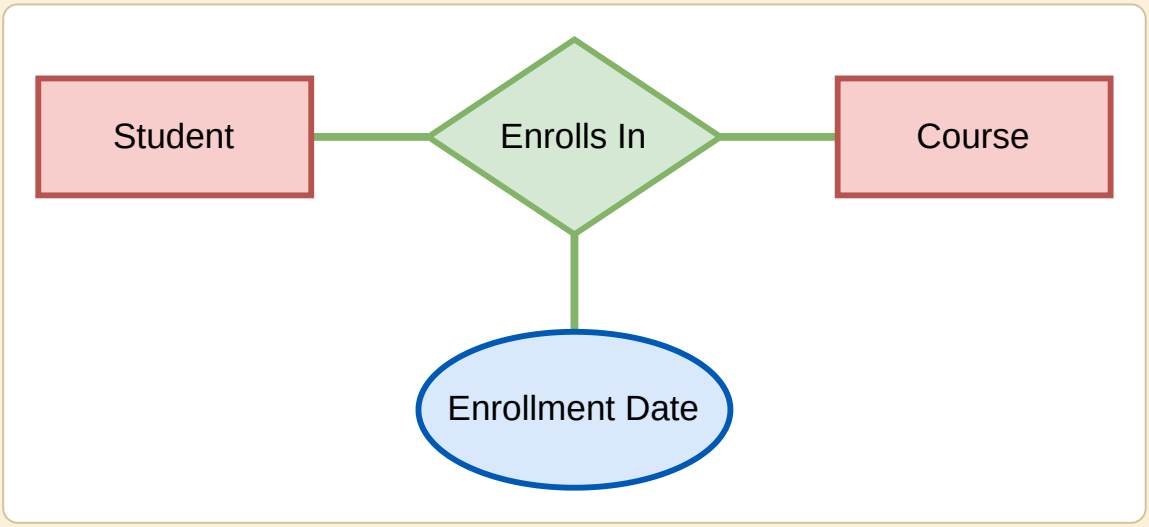
Degree = number of entity types; cardinality = max instances per side; participation = min (mandatory or not). 1:N → FK on many side; M:N → junction table; 1:1 → FK + UNIQUE; unary → self-FK; total → NOT NULL.

10.8 Attributes of relationships

What it is

Some facts describe the **link**, not either entity. The hours an employee spends on a project depend on *both* the employee and the project. In Chen notation such an attribute hangs off the diamond.

Figure — an attribute on a relationship (course diagram)



The test: *would you need to know both entities to know the value?*

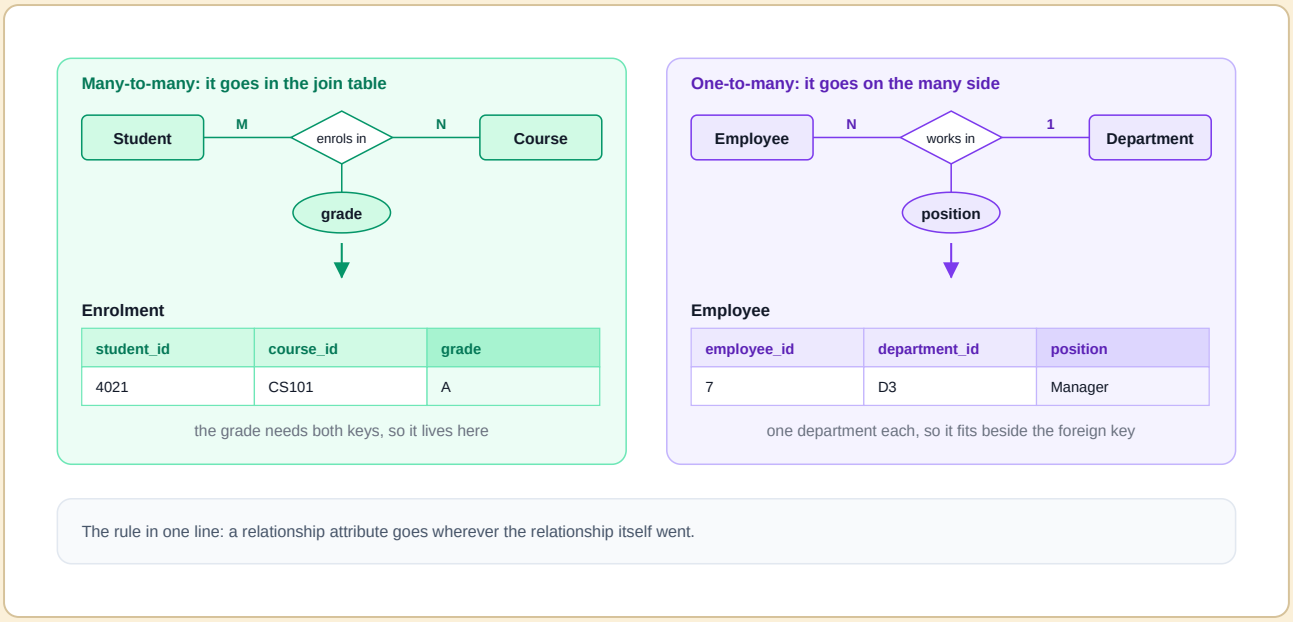
Attribute	Needs	Belongs on
`weekly_hours`	employee **and** project	the relationship *works on*
`start_date` of an assignment	employee **and** project	the relationship
`title`	project only	Project
`last_name`	employee only	Employee

Where it lands in tables

Put the attribute wherever the relationship itself ended up:

- **M:N** → a column in the junction table: `works_on(emp_id, project_id, weekly_hours)`.
- **1:N** → the many side, beside the FK (if an employee works in exactly one department, `role_in_dept` sits on `employee` next to `dept_id`).
- **1:1** → wherever the FK went.

Figure — placement depends on cardinality (course diagram)



Putting it on an entity forces a false rule: `weekly_hours` on `employee` allows only one number per employee, however many projects they work on.

When a relationship should become an entity

When a link collects many facts (a rental has out time, due time, return time, fee), promote it to an entity. The tables come out identical; the diagram just gains a box for a noun people already use. **Test:** if the business has a noun for the link ("rental", "loan", "enrolment", "booking"), make it an entity. That is why `rental` in the bike shop is an entity, not a diamond between Customer and Bike.

> **TL;DR**
>
> If a fact needs both ends of a link, it is a relationship attribute; it lands in the junction table (M:N) or the many side (1:N). A link with its own name and many facts becomes an entity.

10.9 Weak entities

What it is

A **weak entity** has nothing of its own that singles it out. "Room 101" is ambiguous — many buildings have one. Only *(Main building, room 101)* identifies it. The entity it depends on is the **owner** (or identifying / strong entity).

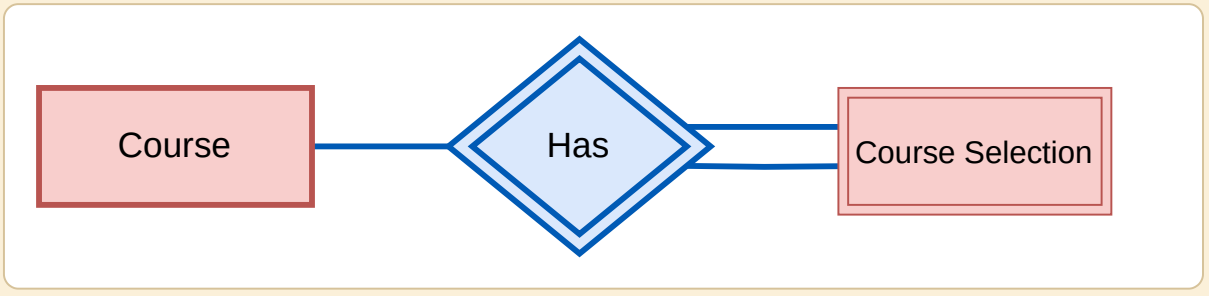
Four properties:

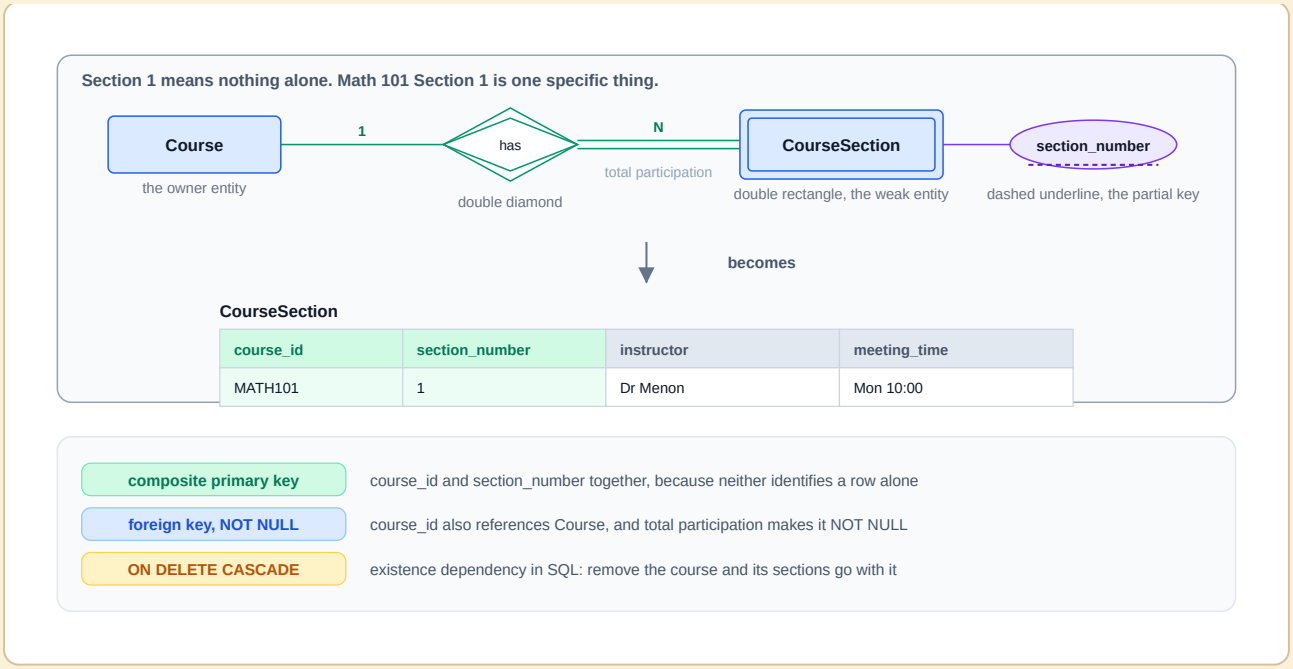
1. **No key of its own** — nothing inside it is unique across all instances.
2. **A partial key** (also called a **discriminator**) — unique *within one owner* (`room_no`).
3. **Existence dependency** — delete the building and its rooms mean nothing.
4. **Total participation in the identifying relationship** — no room belongs to no building.

How it is drawn

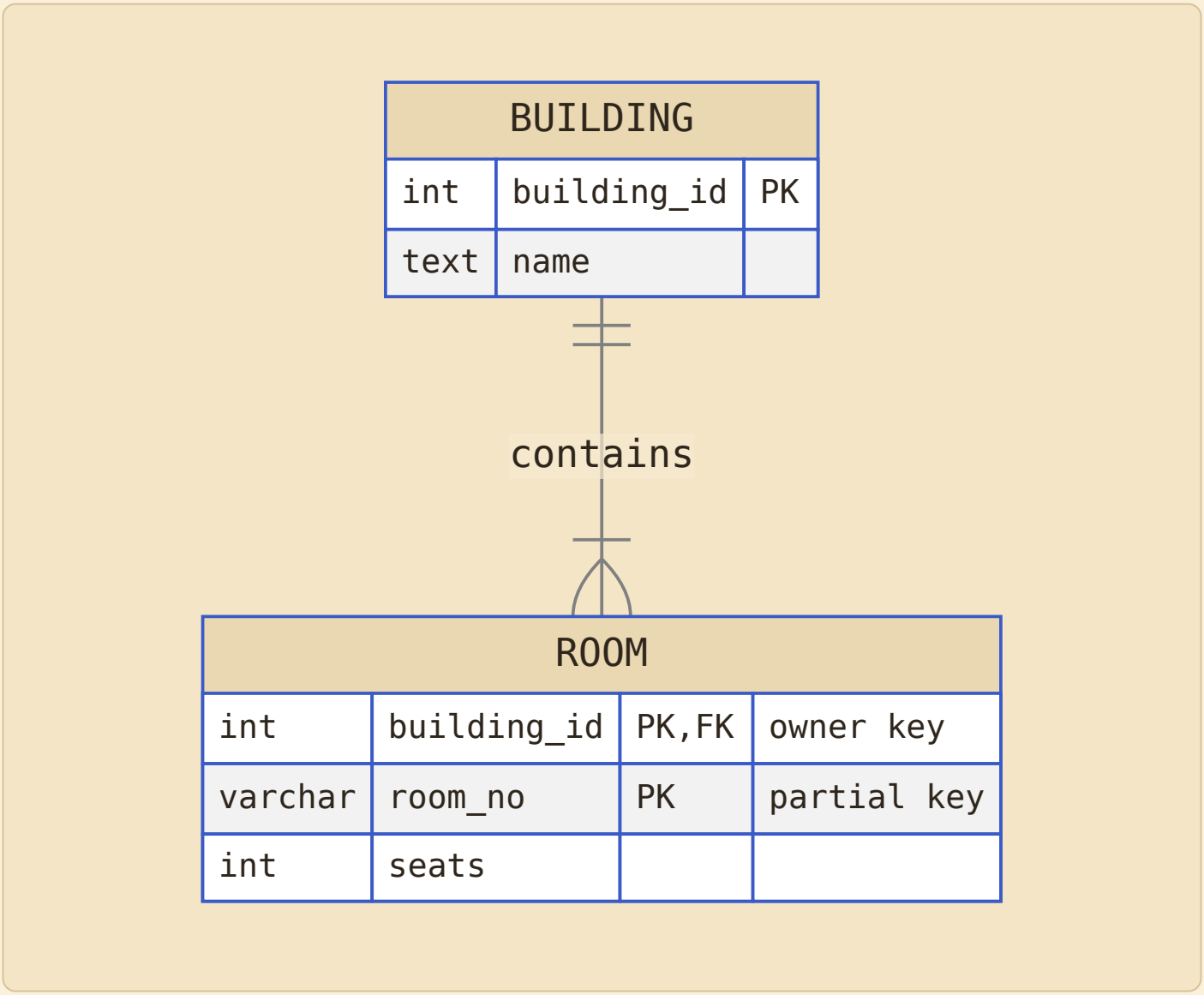
Chen symbol	Meaning
double rectangle	the weak entity
double diamond	the **identifying relationship**
dashed underline	the partial key
double line (weak side)	total participation

Figure — a weak entity in Chen notation, and its table (course diagrams)





In crow's foot (mermaid), a weak entity is shown by an **identifying (solid) line** and a PK that includes the owner's key:



What it becomes in SQL — two valid options

```
-- Option A: composite key = owner key + partial key
CREATE TABLE room_a (
  building_id int NOT NULL REFERENCES building ON DELETE CASCADE,
  room_no     varchar(6) NOT NULL,                -- partial key (discriminator)
  seats       int,
  PRIMARY KEY (building_id, room_no)
);
-- Option B: surrogate key + UNIQUE on (owner key, partial key)
CREATE TABLE room_b (
  room_id     int GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  building_id int NOT NULL REFERENCES building ON DELETE CASCADE,
  room_no     varchar(6) NOT NULL,
  seats       int,
  UNIQUE (building_id, room_no)                   -- without this, duplicates slip in
);
```

Output (both options refuse a duplicate room, and cascade when the owner goes):

```
--- both options refuse a second room 101 in building 1
NOTICE:  room_a: duplicate key value violates unique constraint "room_a_pkey"
NOTICE:  room_b: duplicate key value violates unique constraint "room_b_building_id_room_no_key"
--- existence dependency: delete the Annex, its rooms go too
 room_a_rows | room_b_rows
-------------+-------------
           1 |           1
(1 row)
```

	Composite PK (A)	Surrogate PK (B)
Enforces "no duplicate partial key per owner"	automatically	only with the extra `UNIQUE` (easy to forget)
Referencing it from other tables	two columns	one column
Key width copied into child FKs/indexes	wider	narrow

Common examples: order → order line (`(order_id, line_no)`), building → room, invoice → invoice line. **Weak or not is a design decision:** bank transactions numbered per account are weak; with a global transaction id they are strong. A reasonable default is option B *with* the unique constraint.

Self-check

1. What is a partial key? → An attribute unique only within one owner (e.g. `line_no` within an order).
2. Which SQL clause expresses existence dependency? → `ON DELETE CASCADE` on the owner FK.
3. Why is the owner FK `NOT NULL`? → Participation in the identifying relationship is total.

TL;DR

Weak entity = identified only together with its owner. PK = owner key + partial key (or surrogate + `UNIQUE(owner, partial)`), owner FK `NOT NULL` and `ON DELETE CASCADE`. Chen: double rectangle, double diamond, dashed underline.

10.10 Worked example — an employee-management ER diagram

The brief

A company wants to:

1. Record each employee: name, date of birth, one or more phone numbers, and the department they belong to (every employee has exactly one department).
2. Keep one payroll record per employee (monthly salary, cash advance, tax); no payroll record is shared by two employees. Net pay is salary − advance − tax.
3. Record who supervises whom (the CEO has no supervisor).
4. Record which projects employees work on and for how many hours a week (many-to-many).
5. Record each employee's dependants (children, spouse) for insurance; a dependant is identified by name *within* an employee.

Step 1 — entities (nouns)

Employee, Department, Payroll, Project, Dependant. ("Record", "information" and "number" are words about data, not things.)

Step 2 — attributes, classified

Entity	Attribute	Kind	Reason
Employee	`emp_id`	key	issued once, never repeats
	`full_name`	composite	first name, middle initial, last name
	`birth_date`	simple	one value
	`phone`	**multivalued**	"one or more"
	`age`	**derived**	from `birth_date`
Payroll	`payroll_id`	key	
	`monthly_salary`, `cash_advance`, `tax`	simple, **stored**	typed in by a person — nothing computes them
	`net_pay`	**derived**	salary − advance − tax
Department	`dept_id` (key), `dept_name`		
Project	`project_id` (key), `title`		
Dependant	`dep_name`	**partial key**	unique only within one employee
	`relationship`	simple	

The recurring trap is salary vs net pay: **if someone types the value in, it is stored; if you can compute it from stored attributes, it is derived.**

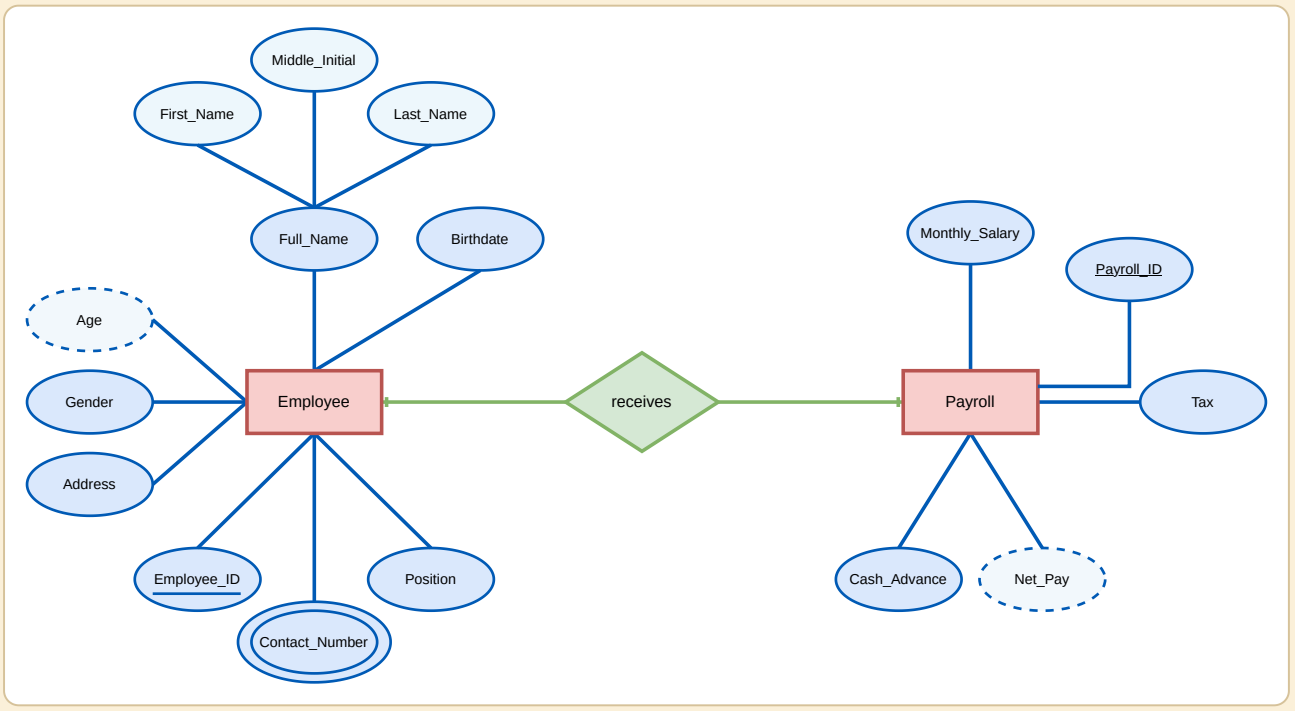
Steps 3–4 — relationships with degree, cardinality and participation

Relationship	Degree	Cardinality	Participation
Employee *works in* Department	binary	N:1	employee total (brief says "exactly one")
Employee *is paid through* Payroll	binary	1:1	both total
Employee *supervises* Employee	unary	1:N	partial (CEO has none)
Employee *works on* Project, attribute `weekly_hours`	binary	M:N	partial
Employee *has* Dependant (identifying)	binary	1:N	dependant total

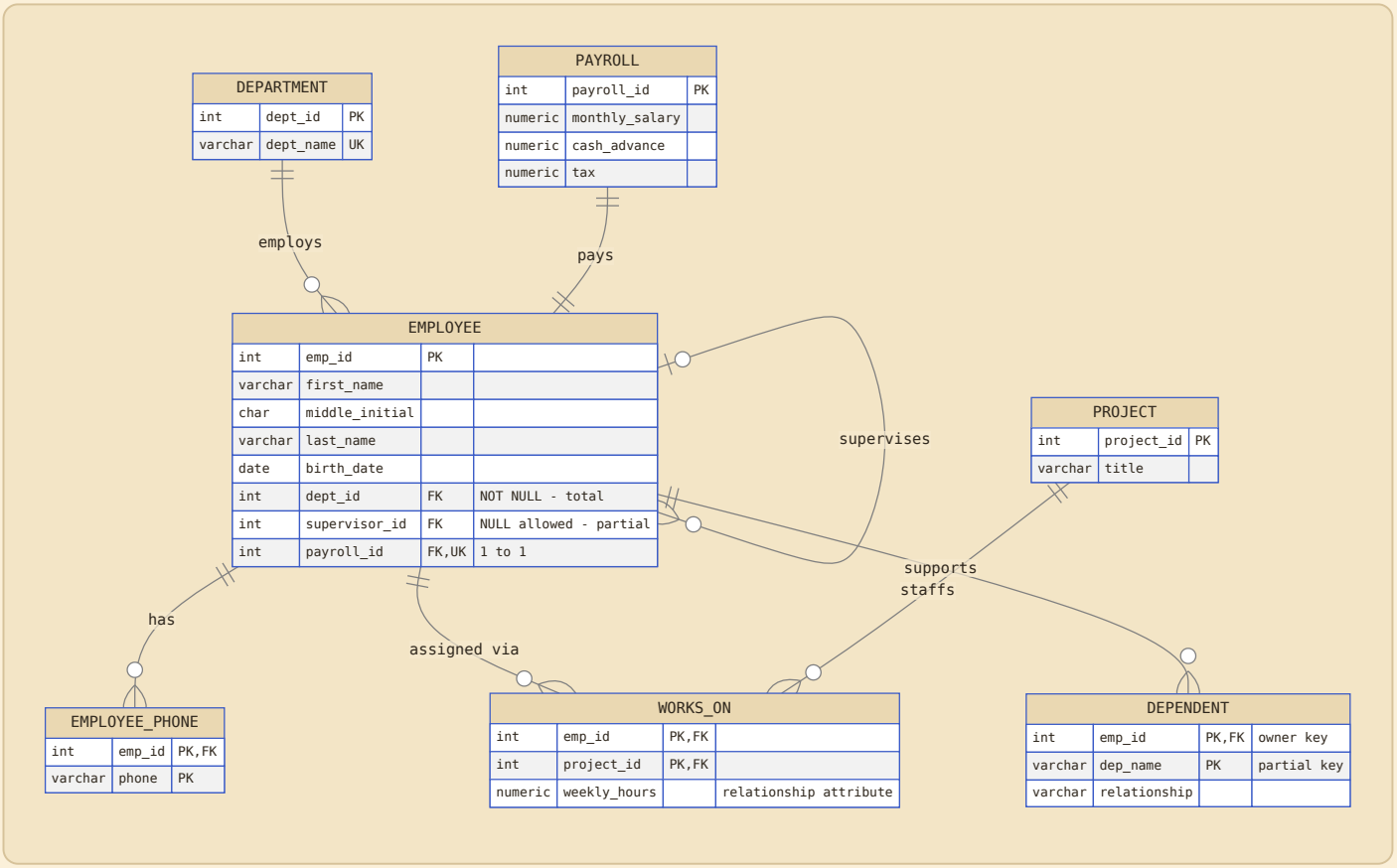
Step 5 — the diagram

The course's Chen diagram for the core of this system (Employee + Payroll, with composite, multivalued and derived attributes) shows every special symbol at once:

Figure — Employee and Payroll in Chen notation: underlined keys, composite Full_Name, double-ellipse Contact_Number, dashed Age and Net_Pay (course diagram)



The full design in crow's-foot notation:



Read it back as sentences — if one sounds wrong, the diagram is wrong: "Each employee works in exactly one department; a department employs zero or more employees (the brief does not require every department

to have staff). Each employee has exactly one payroll record and vice versa. An employee may supervise several employees and has at most one supervisor…"

Design check: does Payroll deserve its own table?

A 1:1 with total participation on **both** sides usually describes one thing stored in two places. Reasons to keep two tables anyway:

- **Access control** — salaries can be granted to HR only (`GRANT` gives a user or role a permission: `GRANT SELECT ON payroll TO hr`; see Chapter 14).
- **Row width** — the frequently read `employee` row stays narrow.
- **Optional data** — if many employees had no payroll row, a separate table stores nothing for them instead of several `NULL`s.

Reasons to merge: no join, and no way to have an employee without payroll. Either answer is acceptable in an interview **if you notice the question and justify your choice.** Here we keep them separate for access control. §10.15 converts this diagram to tables and runs it.

TL;DR

Nouns → entities; classify every attribute (key/simple/composite/multivalued/derived); settle degree, cardinality and participation separately for each verb; read the diagram back as sentences; a 1:1 with total participation both ways should always prompt "one table or two?".

10.11 The relational model

What it is

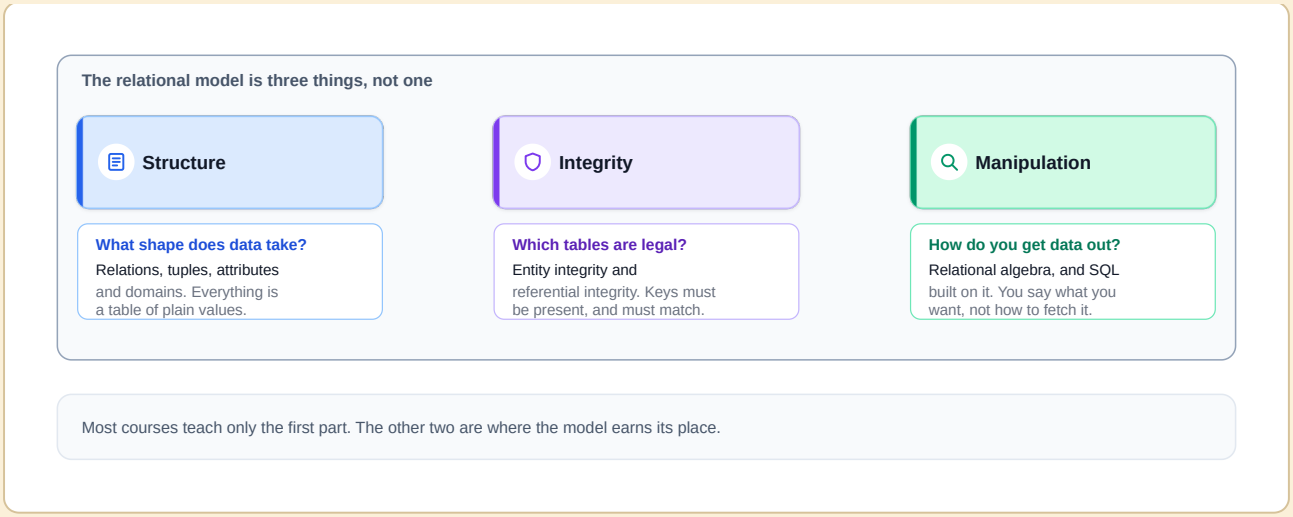
The ER diagram is a drawing; a database stores **relations** (tables). The **relational model**, published by **Edgar F. Codd in 1970**, says: store all data as tables of values, and let programs *describe* the data they want while the system works out how to fetch it.

Before it, hierarchical and network databases (older designs where records were linked in trees or webs) made programs walk stored paths of pointers (stored disk addresses of other records) (employee record → pointer → department record). Reorganize the file and every program walking that path broke.

The three parts of the model

Part	Question it answers	Content
Structure	What shape is the data?	Only relations: a name, named columns, rows of single values. No nested records, arrays or pointers.
Integrity	Which tables are legal?	Keys and constraints declared once on the table; the DBMS refuses any change that breaks them.
Manipulation	How do I get data out?	Declarative queries (SQL: you state *what* you want, not how to fetch it), built on **relational algebra**: operations (select, project, join, union, difference …) that take relations and return a relation.

Figure — structure, integrity, manipulation (course diagram)



Links are values, not pointers

`employee.dept_id = 10` links to the department whose `dept_id` is 10 — the shared *value* is the whole link. Consequences:

- The link survives copying the table to another server, reloading in a different order, or rebuilding indexes.
- The DBMS can **check** the link (a foreign key refuses `dept_id = 99` if no department 99 exists). A raw pointer cannot be checked like that.

What you get

- **One rule reaches every program** — constraints live with the data.
- **Physical data independence** — queries name *what*, not *how*, so indexes and storage can change without editing queries.
- **Some logical data independence** — adding a column or hiding a split behind a view (a saved query that behaves like a table) does not break existing queries. It is *not* total: renaming a column or shrinking its type still breaks the programs that use it.

What it does not promise

- **Nothing about storage** — files, pages and indexes belong to the storage engine (the part of the DBMS that reads and writes the files) (Chapter 12).
- **Joins are not free** — matching values costs work that grows with table size.
- **A table is not always an entity** — tables also come from M:N relationships (`works_on`) and multivalued attributes (`employee_phone`).

> **TL;DR**
>
> Codd (1970): everything is a relation of single values; integrity rules are declared on tables; queries are declarative. Rows link by shared values, which can be checked and survive reorganization.

10.12 Relations, tuples, attributes — the precise vocabulary

The core words

Formal term	SQL term	Older file term	Meaning
relation	table	file	named set of rows with fixed named columns
tuple	row	record	one value for each attribute
attribute	column	field	a named column
domain	data type + constraints	field type	the set of allowed values for an attribute
degree (arity)	number of columns	number of fields	a property of the schema
cardinality	number of rows	number of records	a property of the current data

Figure — anatomy of a relation (course diagram)

- **Domain** is what you *mean*, which is more than a type: `int` is a type; "integers 1–500" is the domain you meant for room capacity, enforced with `CHECK`.
- **Schema vs instance.** The **relation schema** is the design — `room(room_id, code, capacity)` — and rarely changes. The **relation instance** is the set of rows at this moment and changes with every insert. A key belongs to the *schema*, so it has to be true for any data the table could ever legally contain — the current rows prove nothing.
- **Two meanings of "cardinality".** In §10.7 it meant how many instances link (1:N); here it means a row count. Both are standard; context tells you which.

Five properties of every relation

1. **One value per cell** (atomic) — no lists, no nested records. This is why multivalued attributes need their own table, and it is the heart of 1NF (§11.7).
2. **Rows have no inherent order** — a relation is a set, so if you need sorted output you must ask for it with `ORDER BY`.
3. **Columns have no inherent order either** — you refer to them by name. (Real SQL tables do remember the order columns were declared in, so `SELECT *` always lists them the same way; still, list the columns you want explicitly.)

4. **Attribute names are unique within a relation.**
5. **No duplicate tuples** — a set has no duplicates. *Here SQL differs:* a SQL table accepts identical rows unless a `PRIMARY KEY` or `UNIQUE` forbids them. Declaring a key closes the gap.

NULL

`NULL` marks a **missing** value: either *unknown* (grade not yet computed) or *not applicable* (no middle name). It is not zero and not the empty string. SQL uses **three-valued logic**: comparing anything with `NULL` gives *unknown*, and `WHERE` keeps only rows where the condition is *true* (Chapter 2 covers this in depth). From `code/03_keys_and_integrity.sql`:

```
SELECT (SELECT count(*) FROM lecturer WHERE room_id = NULL)  AS eq_null,
       (SELECT count(*) FROM lecturer WHERE room_id IS NULL) AS is_null,
       count(room_id) AS count_col, count(*) AS count_star
FROM lecturer;
```

Output:

```
 eq_null | is_null | count_col | count_star
---------+---------+-----------+------------
       0 |       2 |         1 |          3
(1 row)
```

`= NULL` matches nothing, `IS NULL` works, `count(col)` skips nulls while `count(*)` counts rows. So whenever a column is nullable, each query touching it must decide how missing values are treated.

Self-check

1. `student(roll_no, name, cgpa, dept)` holds 8,000 rows. Degree? Cardinality? → **4** and **8,000**.
2. Does a SQL table forbid duplicate rows by default? → **No** — only a key or `UNIQUE` does.
3. Why can't you prove `{name}` is a key by looking at the rows? → Keys are schema properties; rows can only *disprove* one (two equal names).

TL;DR

Relation = table, tuple = row, attribute = column, domain = allowed values; degree = #columns (schema), cardinality = #rows (instance). Cells are atomic, order is meaningless, formal relations have no duplicates (SQL tables do unless keyed). `NULL` = missing; test with `IS NULL`.

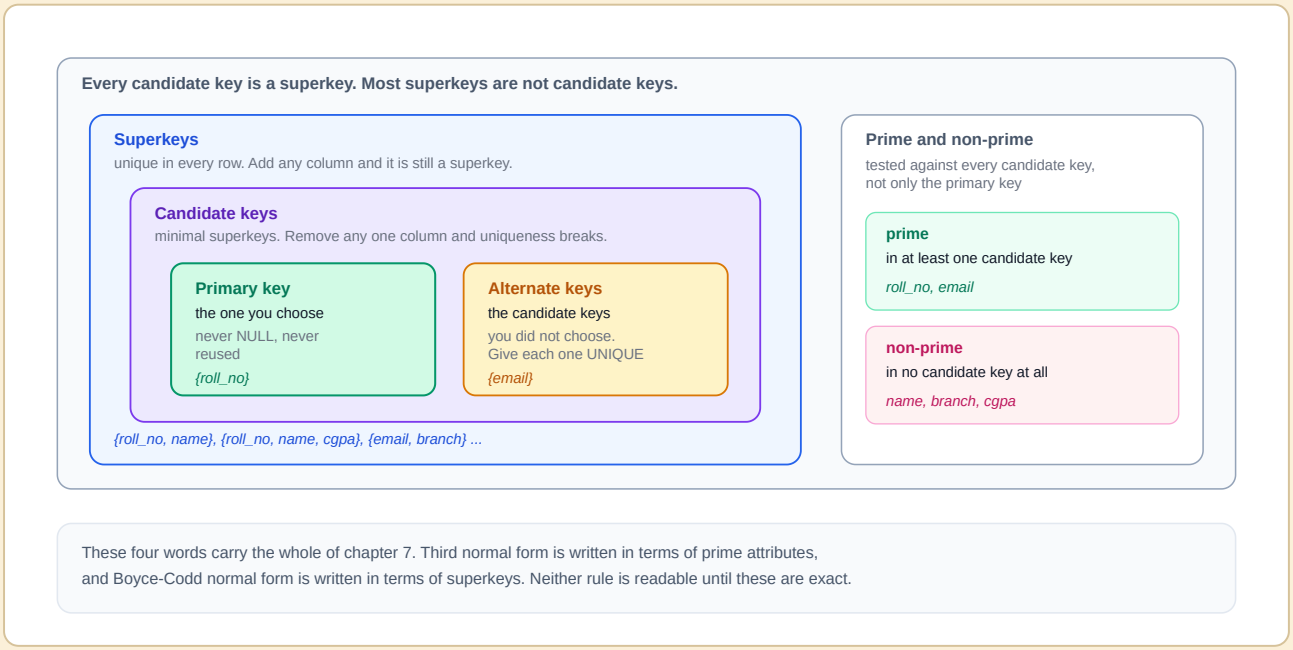
10.13 Keys in relational databases

Two identical-looking rows — same name, department and year — and a support ticket asking to change "that student". Without something that tells rows apart, the request has no answer. **Keys** make identity exist. Chapter 11 states its rules with exactly these definitions, so precision matters.

Running example: `lecturer(lecturer_id, email, name, room_id)`, where ids are issued once and emails are unique.

Key	Definition	Example
Superkey	a set of attributes that can **never** repeat in two rows (in any legal instance)	`{lecturer_id}`, `{email}`, `{lecturer_id, name}`, all four columns …
Candidate key	a **minimal** superkey — remove any attribute and it stops being unique	`{lecturer_id}`, `{email}`
Primary key	the one candidate key you *choose*; unique and never `NULL`	`lecturer_id`
Alternate key	any candidate key other than the chosen primary one — declare it `UNIQUE`	`email`
Composite key	any key with 2+ attributes (a shape, not a separate kind)	`booking(room_id, day, slot)`
Foreign key	attributes whose values must match a key in a (possibly the same) table, or be `NULL`	`lecturer.room_id → room.room_id`
Natural key	made of real-world data	email, ISBN, frame number
Surrogate key	system-generated, no business meaning	identity integer, UUID

Figure — superkeys ⊇ candidate keys ⊇ primary key, with prime and non-prime attributes (course diagram)



Key facts to be exact about:

- **A superkey stays a superkey when you add columns to it**, so there are many of them; candidate keys are the ones with nothing spare. Minimality is about *removability*, not size — a 3-column candidate key is fine if none can go.
- **All candidate keys are superkeys, but the reverse is rarely true.**
- **Prime attribute** = appears in *some* candidate key (any of them, not only the chosen primary key). **Non-prime** = belongs to none. In `lecturer`, `lecturer_id` and `email` are prime; `name` and `room_id` are non-prime. 3NF is defined with these words (§11.9).
- **Composite example.** `enrolment(student_id, course_id, term, grade)`: students may retake a course another term, so only `{student_id, course_id, term}` identifies a row, and dropping any of the three breaks it.
- **Foreign keys:** may be `NULL` (no link yet — that is not a violation); may reference any `PRIMARY KEY` **or** `UNIQUE` column set; may reference their own table (`employee.supervisor_id`), which is how a unary relationship becomes a column.

- **Surrogate vs natural:** surrogates are narrow and never change; natural keys are readable and sometimes save a join, but change when the business changes (a new roll-number format means updating every FK). **Default: surrogate PK + `UNIQUE` natural key** — the `customer` table in `00_schema.sql` does exactly that.

In the interview

30-second answer: "A superkey uniquely identifies rows; a candidate key is a minimal superkey; the primary key is the candidate I pick — unique and not null; the other candidates are alternate keys and get UNIQUE constraints. A foreign key references a primary or unique key and may be null. My default is a generated surrogate PK, with the natural key kept unique."

- *Is `{id, name}` a candidate key?* → A superkey, not a candidate key: `name` can be dropped.
- *UUID vs bigint surrogate?* → UUIDs can be generated anywhere without coordination but are 16 bytes vs 8 and random UUIDs scatter B-tree (the default, always-sorted index structure) inserts; time-ordered UUIDs (v7) reduce that.
- *Is an attribute of an alternate key prime?* → Yes — prime means "in any candidate key".

TL;DR

Superkey (unique) ⊇ candidate key (minimal) ⊇ primary key (chosen, NOT NULL); other candidates = alternate keys (UNIQUE). Prime = in some candidate key. FKs match a PK/UNIQUE or are NULL. Surrogate PK + unique natural key is the safe default.

10.14 Relational integrity constraints

What it is

An **integrity constraint** is a condition stored data is never allowed to break. Whatever program connects, the DBMS tests it during each `INSERT`, `UPDATE` and `DELETE` and refuses any change that would violate it.

Figure — the integrity constraints and the SQL that declares each (course diagram)

Five rules the database enforces for every application that connects

Constraint	What it refuses	How you declare it
Domain the allowed set of values	Marks of 210 out of 100. The text "twenty" in a number column.	`marks INT` `CHECK (marks BETWEEN 0 AND 100)`
Entity integrity every row is identifiable	A row whose primary key is empty. Half of a composite key left blank.	`PRIMARY KEY (loan_id)` NOT NULL comes with it automatically

Referential integrity
a foreign key points at a real row, or at nothing
	A loan for a member id that does not exist.
	Deleting a member who still has loans, unless you chose CASCADE or SET NULL.
		`FOREIGN KEY (member_id)`
		`REFERENCES Member(member_id)`
		`ON DELETE RESTRICT`

Key constraint no duplicate values	Two members sharing one email address. Several NULLs are usually still allowed.	`UNIQUE KEY uq_member_email (email)` add NOT NULL if the value is required
Check constraint any condition you can state	A due date earlier than the issue date. A negative price.	`CHECK (due_date > issue_date)` one rule, held for every application

The same rule written as an if statement in one application holds only for that application. In the schema it holds for all of them.

Constraint	Rule	SQL
Domain	a value must be in the attribute's domain	the data type, plus `CHECK (capacity BETWEEN 1 AND 500)`
Entity integrity	no part of a primary key may be `NULL`	`PRIMARY KEY` (implies `NOT NULL`)
Key	candidate keys are unique	`PRIMARY KEY`, `UNIQUE`
Referential integrity	an FK value matches an existing referenced row, or is `NULL` — no **orphan rows**	`FOREIGN KEY … REFERENCES …`
(also declared the same way)	must always have a value / default when omitted	`NOT NULL`, `DEFAULT` (DEFAULT never refuses anything, so it is not strictly a constraint)

Why entity integrity: a primary key identifies a row, and `NULL` means "unknown" — a row with unknown identity cannot be distinguished, so the model rejects it. For composite keys *every* column must be present.

What happens when a referenced (parent) row is deleted or its key updated

Action (`ON DELETE` / `ON UPDATE`)	Effect	Use when
`NO ACTION` (default)	error if children exist; in PostgreSQL the check runs at the end of the statement and can be `DEFERRABLE` (postponed) to the end of the transaction — the group of statements committed together	general default
`RESTRICT`	error immediately, cannot be deferred	you want an immediate refusal
`CASCADE`	delete/update the children too	weak entities, pure child data (phones, order lines) — dangerous elsewhere
`SET NULL`	children's FK becomes `NULL` (column must be nullable)	optional relationships
`SET DEFAULT`	children's FK becomes its default	rare

Dialect notes: MySQL/InnoDB (MySQL's default storage engine) treats `NO ACTION` as `RESTRICT` and rejects `SET DEFAULT`; SQL Server has no `RESTRICT` keyword (use `NO ACTION`); SQLite enforces FKs only after `PRAGMA foreign_keys = ON` (a SQLite settings command). MySQL enforces `CHECK` only from 8.0.16 (earlier versions parsed and ignored it).

Running it (`code/03_keys_and_integrity.sql`)

Each failing statement is wrapped in a `DO` block (a one-off piece of PostgreSQL procedural code) that catches the error and prints it as a `NOTICE` (an informational message), so the script keeps going.

```
CREATE TABLE room (
  room_id  int PRIMARY KEY,                                  -- entity integrity: unique + NOT NULL
  code     varchar(10) NOT NULL UNIQUE,                      -- alternate (candidate) key
  capacity int NOT NULL CHECK (capacity BETWEEN 1 AND 500)   -- domain constraint
);
CREATE TABLE lecturer (
  lecturer_id int PRIMARY KEY,
  name        varchar(60) NOT NULL,
  email       varchar(120) UNIQUE,                           -- UNIQUE but nullable
  room_id     int REFERENCES room(room_id) ON DELETE SET NULL  -- optional link
);
CREATE TABLE booking (
  room_id  int  NOT NULL REFERENCES room(room_id) ON DELETE RESTRICT,
  day      date NOT NULL,
  slot     int  NOT NULL CHECK (slot BETWEEN 1 AND 8),
  PRIMARY KEY (room_id, day, slot)                           -- composite key
);
-- each violation is attempted inside:  DO $$ BEGIN ... EXCEPTION WHEN ... THEN RAISE NOTICE ... END $$;
```

Output:

```
--- 1. domain: capacity 900 is outside the allowed values
NOTICE:  new row for relation "room" violates check constraint "room_capacity_check"
--- 2. entity integrity: a primary key may not be NULL
NOTICE:  null value in column "room_id" of relation "room" violates not-null constraint
--- 3. key constraint: the alternate key code must be unique
NOTICE:  duplicate key value violates unique constraint "room_code_key"
--- 4. referential integrity: no orphan rows
NOTICE:  insert or update on table "booking" violates foreign key constraint "booking_room_id_fkey"
--- 5. UNIQUE lets several rows have NULL (Dr Silva and Dr Wu both have no email)
   name   | email
----------+-------
 Dr Silva |
 Dr Wu    |
(2 rows)

--- 6. ON DELETE RESTRICT: room 1 still has bookings
NOTICE:  update or delete on table "room" violates foreign key constraint "booking_room_id_fkey" on table
"booking"
--- 7. ON DELETE SET NULL: deleting room 2 empties Dr Silva's room_id
   name   | room_id
----------+---------
 Dr Park  |       1
 Dr Silva |
 Dr Wu    |
(3 rows)
```

UNIQUE and NULL — a classic surprise

Because `NULL` is never equal to `NULL`, most databases let a `UNIQUE` column hold **many** nulls (output 5). If "unique when present" is not what you want, add `NOT NULL`.

DBMS	Multiple NULLs in a UNIQUE column?
PostgreSQL	yes by default; PostgreSQL 15+ can say `UNIQUE NULLS NOT DISTINCT` to allow only one (not run here — our server is 14)
MySQL (InnoDB)	yes
SQLite	yes
SQL Server	**no — only one NULL**; use a filtered unique index (one covering only rows that match its `WHERE`) `WHERE col IS NOT NULL` for "many"

Why rules belong in the schema

1. **They hold for everybody** — web app, mobile backend (the server code behind a phone app), nightly import, a hand-typed fix at midnight.
2. **They cannot drift** — one copy, not one per application.
3. **They document meaning** — the `CREATE TABLE` tells a reader what the data is.

Costs: constraints add a little work to each write, and a thoughtless `CASCADE` can wipe out far more rows than someone meant. Both are small next to the benefit of a single, central enforcement point.

Common mistakes

- Forgetting to declare alternate keys → two customers with the same email.
- `ON DELETE CASCADE` on a "real" relationship (deleting a customer silently deletes their invoices).
- Assuming a nullable `UNIQUE` column means "at most one missing".
- Leaving FKs off "for speed" — orphans appear and joins silently lose rows.

TL;DR

Domain (type + CHECK), entity integrity (PK never NULL), key (UNIQUE), referential (FK matches or NULL; choose RESTRICT/NO ACTION/CASCADE/SET NULL on delete). UNIQUE usually allows many NULLs. Put rules in the schema so they hold for every client.

10.15 Converting an ER diagram to tables

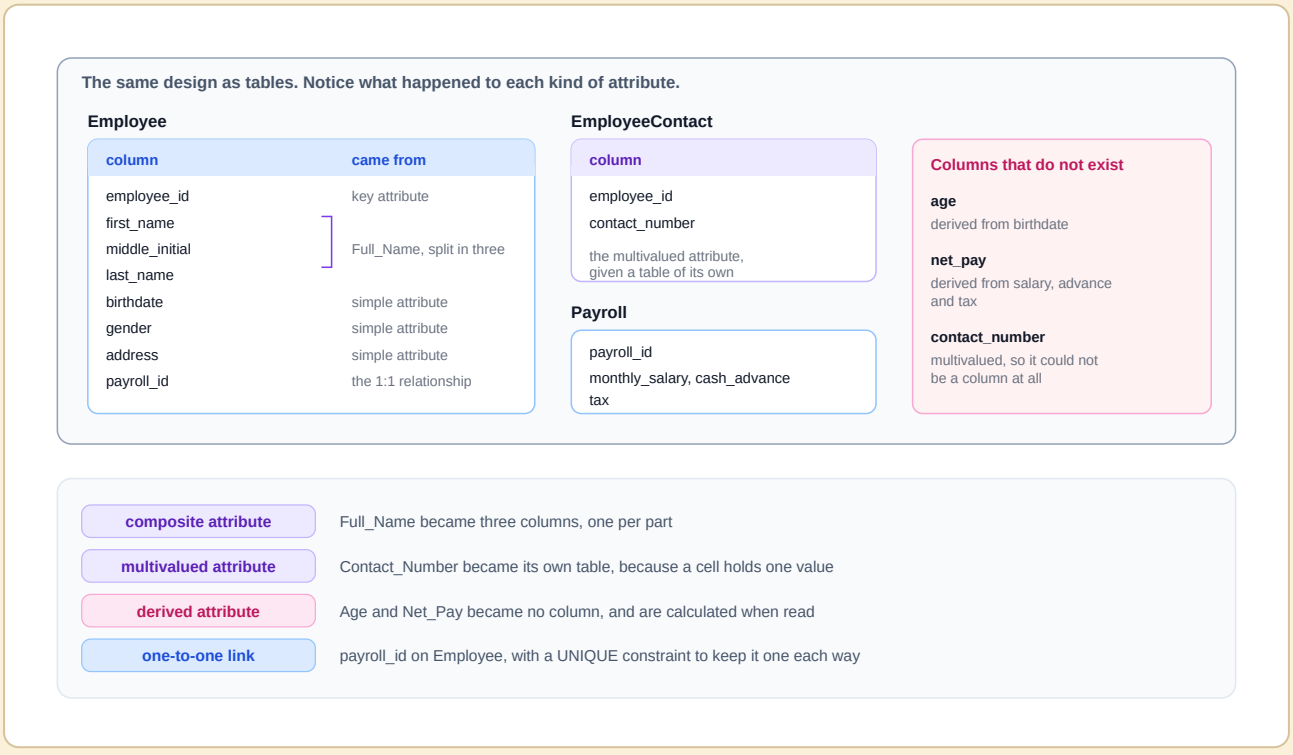
The procedure

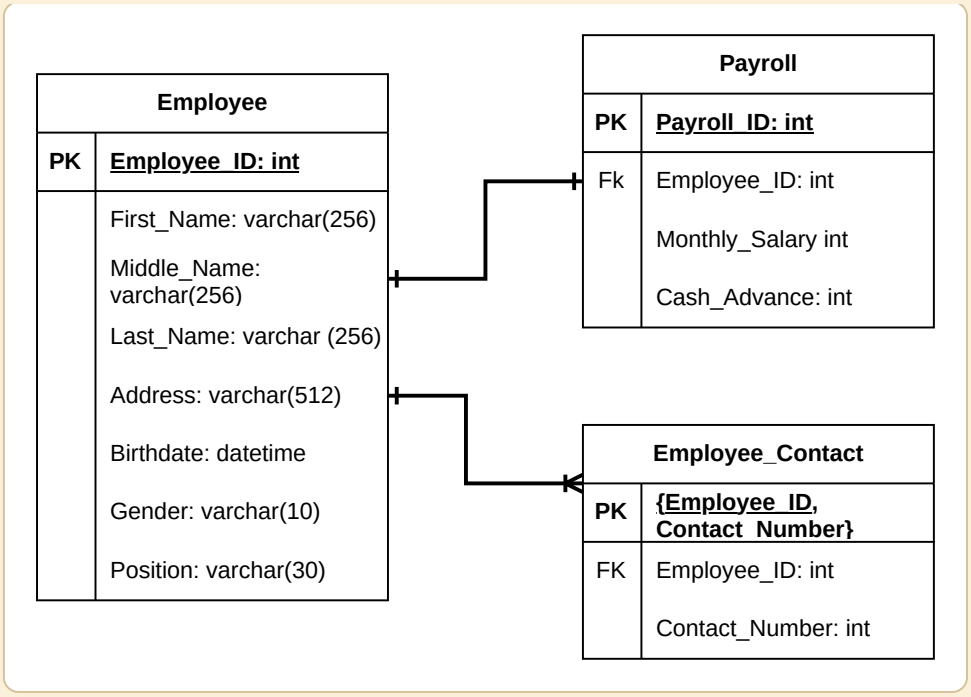
Once the diagram is right, conversion is mechanical — one decision per construct:

#	ER construct	Relational result
1	Strong entity	a table; key attribute → primary key
2	Simple attribute	a column
3	Composite attribute	one column per part (no column for the whole)
4	Derived attribute	no column (compute in the query or a view)
5	Multivalued attribute	own table: owner key + value, both in the PK, `ON DELETE CASCADE`
6a	1:N / N:1 relationship	FK on the many side
6b	M:N relationship	junction table: both FKs, together the PK
6c	1:1 relationship	FK + `UNIQUE` on either side, prefer the total-participation side
6d	Unary relationship	FK to the same table
6e	Ternary relationship	table with three FKs (PK usually all three)
7	Relationship attribute	follows the relationship: junction table (M:N) or many side (1:N)
8	Weak entity	table with PK = owner key + partial key; owner FK `NOT NULL`, usually `ON DELETE CASCADE`
9	Participation	total → FK `NOT NULL`; partial → FK nullable

Create tables in **dependency order** — parents before children, since a foreign key can only point at a table that already exists.

Figure — the course's Employee/Payroll example as tables: the composite splits, the multivalued attribute becomes a table, the derived attributes vanish (course diagrams)




Employee
PK
Employee_ID: int
First_Name: varchar(256)
Middle_Name: varchar(256)
Last_Name: varchar (256)
Address: varchar(512)
Birthdate: datetime
Gender: varchar(10)
Position: varchar(30)
Payroll
PK
Payroll_ID: int
Fk
Employee_ID: int
Monthly_Salary int
Cash_Advance: int
Employee_Contact
PK
{Employee_ID, Contact_Number}
FK
Employee_ID: int
Contact_Number: int

Running the procedure on §10.10 (`code/02_employee_er_to_tables.sql`)

```
CREATE TABLE department (                    -- strong entity -> table
  dept_id   int PRIMARY KEY,
  dept_name varchar(60) NOT NULL UNIQUE
);
CREATE TABLE payroll (                       -- strong entity -> table
  payroll_id     int PRIMARY KEY,
  monthly_salary numeric(10,2) NOT NULL CHECK (monthly_salary >= 0),  -- stored (typed in)
  cash_advance   numeric(10,2) NOT NULL DEFAULT 0,
  tax            numeric(10,2) NOT NULL DEFAULT 0
  -- net_pay is DERIVED -> no column (computed in the view below)
);
CREATE TABLE employee (
  emp_id         int PRIMARY KEY,                          -- key attribute
  first_name     varchar(40) NOT NULL,                     -- composite attribute full_name
  middle_initial char(1),                                  --   split into
  last_name      varchar(40) NOT NULL,                     --   one column per part
  birth_date     date NOT NULL,                            -- simple; age is DERIVED
  dept_id        int  NOT NULL REFERENCES department(dept_id),  -- 1:N, total participation -> NOT NULL
  supervisor_id  int  REFERENCES employee(emp_id),         -- unary 1:N, partial -> NULL allowed
  payroll_id     int  NOT NULL UNIQUE REFERENCES payroll(payroll_id)  -- 1:1 -> FK + UNIQUE
);
CREATE TABLE employee_phone (                -- multivalued attribute -> own table
  emp_id int         NOT NULL REFERENCES employee(emp_id) ON DELETE CASCADE,
  phone  varchar(20) NOT NULL,
  PRIMARY KEY (emp_id, phone)
);
CREATE TABLE project (project_id int PRIMARY KEY, title varchar(80) NOT NULL);
CREATE TABLE works_on (                      -- M:N -> junction table
  emp_id     int NOT NULL REFERENCES employee(emp_id),
  project_id int NOT NULL REFERENCES project(project_id),
  weekly_hours numeric(4,1) NOT NULL CHECK (weekly_hours > 0),  -- relationship attribute
  PRIMARY KEY (emp_id, project_id)
);
CREATE TABLE dependent (                     -- weak entity
  emp_id       int NOT NULL REFERENCES employee(emp_id) ON DELETE CASCADE,  -- owner key
  dep_name     varchar(40) NOT NULL,                                       -- partial key
  relationship varchar(20) NOT NULL,
  PRIMARY KEY (emp_id, dep_name)                                           -- owner key + partial key
);
-- Derived attributes live in a view (fixed "as of" date so the output is repeatable):
CREATE VIEW employee_card AS
SELECT e.emp_id,
       concat_ws(' ', e.first_name, e.middle_initial, e.last_name)   AS full_name,
       date_part('year', age(date '2026-09-28', e.birth_date))::int  AS age,
       p.monthly_salary - p.cash_advance - p.tax                     AS net_pay,
       (SELECT string_agg(phone, ', ' ORDER BY phone) FROM employee_phone ph
         WHERE ph.emp_id = e.emp_id)                                 AS phones
FROM employee e JOIN payroll p USING (payroll_id);
```

Output:

```
--- employee_card (derived: full_name, age, net_pay; multivalued: phones)
 emp_id |  full_name   | age | net_pay |              phones
--------+--------------+-----+---------+---------------------------------
      1 | Aisha Khan   |  41 | 4900.00 | +44 20 7946 0001
      2 | Ravi K Menon |  33 | 4250.00 | +91 80 4000 0003, +91 98450 00002
      3 | Sofia Rossi  |  28 | 3100.00 |
(3 rows)

--- the unary relationship: who reports to whom
 employee | supervisor
----------+------------
 Aisha    |
 Ravi     | Aisha
 Sofia    | Aisha
(3 rows)

--- 1:1 is enforced by UNIQUE: a second employee on payroll 501 is refused
NOTICE:  refused: duplicate key value violates unique constraint "employee_payroll_id_key"
--- weak entity: deleting the owner removes its dependents (ON DELETE CASCADE)
 dependents_left | phones_left
-----------------+-------------
               0 |           0
(1 row)
```

(Before deleting employee 2 the script first removes their `works_on` rows and clears the `supervisor_id` pointing at them, because those FKs use the default `NO ACTION` — the database would otherwise refuse, which is exactly referential integrity at work.)

Checking a conversion

1. **Count:** 5 entities + 1 multivalued attribute + 1 M:N relationship = 7 tables. ✓
2. **Every relationship produced something** (an FK, a UNIQUE FK or a table). A relationship that produced nothing was missed.
3. **No derived attribute survived** as a column — or, if one did, a comment says why.
4. **No multivalued attribute left in its parent** — a list column is a 1NF violation (Chapter 11).

TL;DR

Entities → tables; simple → columns; composite → parts; derived → nothing; multivalued → own table; 1:N → FK on many side; M:N → junction table; 1:1 → FK + UNIQUE; weak → owner key + partial key with CASCADE; total participation → NOT NULL. Parents first.

10.16 Best practices for ER → relational mapping

The procedure is mechanical for small diagrams; large ones need judgement in a few places.

Situation	Recommendation	Why / exception
Composite attribute	Store the parts; do **not** also store the whole. Build the whole in a query or view.	Two copies disagree after the first partial update. Exception: never query the parts → one column is fine.
Multivalued attribute	Always a separate table with a composite PK (owner key + value). Never numbered columns.	The composite PK stops the same phone being stored twice for one person.
Derived attribute	Compute on read. Store only if expensive *or* it must be frozen in time (order total at purchase prices) — and comment the column.	Otherwise the next engineer deletes it as a "mistake".
1:1 with total participation both sides	Probably one table. Keep two only for access control, row width, or mostly-absent optional data.	Two tables always joined 1:1 are cost without benefit.
M:N	The junction table is mandatory; relationship attributes live there; give it a business name (`enrolment`, `booking`) not `student_course`.	Add a surrogate key only if other tables must reference a single row — and keep `UNIQUE` on the pair so nobody enrols twice.
Weak entity with a surrogate key	Keep `UNIQUE (owner_key, partial_key)`.	Otherwise the rule the weak entity expressed is enforced nowhere.
Naming	One style for tables and columns; FKs named after their target; **role names** when two FKs hit the same table (`origin_airport_id`, `destination_airport_id`, not `airport_id_1/2`).	Readable joins.
Normalization	Run the normal forms over the result (Chapter 11); expect one or two further splits.	A careful ERD lands *near* 3NF, rarely exactly on it.
Large diagrams	Create parent tables before children and finish one area (say orders) before starting the next (inventory); record each choice you made that the rules left open (merged 1:1s, stored derived columns, surrogate keys on weak entities).	Those are the choices someone will question later.

Common mistakes

- Converting an M:N with an FK on one side (only works if it was really 1:N).
- Keeping `full_name` *and* `first_name`/`last_name`.
- Surrogate key on a junction table without the pair `UNIQUE`.
- Creating child tables before parents in a migration script.

In the interview

30-second answer (for "turn this ERD into tables"): "Each strong entity becomes a table with its key as PK. Composite attributes split into columns, derived ones are computed, multivalued ones get their own table. 1:N puts the FK on the many side, M:N gets a junction table holding both keys plus any relationship attributes, 1:1 is an FK with UNIQUE. A weak entity's PK combines its owner's key with its partial key, with cascade delete, and total participation means NOT NULL. Then I check the result against the normal forms."

- *Where does `grade` go in student–course?* → In the enrolment junction table.
- *Would you merge a 1:1?* → Yes unless access control, row width or sparse optional data justify splitting.

Self-check

1. Two FKs from `flight` to `airport` — how do you name them? → By role: `origin_airport_id`, `destination_airport_id`.
2. A junction table has a surrogate PK. What else must it have? → `UNIQUE` on the two FK columns.
3. Is the converted schema automatically in 3NF? → No; check it with the normal forms.

TL;DR

Split composites without keeping the whole; multivalued → table; derived → computed (unless frozen); question every total 1:1; junction tables hold relationship attributes and a unique pair; name by role; convert in dependency order; then normalize.

Key terms

Term	Meaning
Data model	picture of entities, attributes and relationships
Conceptual / logical / physical model	things + links / + attributes, keys, general types / + exact types, indexes for one DBMS
Data independence	changing storage (physical) or some schema details (logical) without changing applications
Entity type / instance / set	the kind / one member / all current members
Key, simple, composite, multivalued, derived attribute	identifies / one value / has parts / many values / computed
Relationship type / set	the association / the actual pairs
Degree	number of entity types in a relationship (unary, binary, ternary)
Cardinality (ER)	max instances per side: 1:1, 1:N, M:N
Participation	total (mandatory, double line) or partial (optional)
(min, max)	min = participation, max = cardinality
Junction table	table implementing an M:N relationship
Weak entity, owner, partial key (discriminator), identifying relationship	entity identified only with its owner, via owner key + partial key
Chen vs crow's-foot	ellipse/diamond notation vs attributes-in-box + line-end notation
Relation, tuple, attribute, domain	table, row, column, allowed values
Degree / cardinality (relation)	number of columns / number of rows
Schema vs instance	design vs current data
Superkey, candidate, primary, alternate, composite, foreign, natural, surrogate key	see §10.13
Prime / non-prime attribute	in some candidate key / in none
Domain, entity, key, referential integrity	allowed values / PK not NULL / uniqueness / FK matches or NULL
Orphan row	child row whose FK points at nothing
RESTRICT / NO ACTION / CASCADE / SET NULL / SET DEFAULT	what happens to children when a parent is deleted/updated

---

Chapter 10 TL;DR

- **Model before tables.** Entities (nouns) → tables, attributes → columns, relationships (verbs) → FKs or junction tables. A fact you do not model is a question you cannot answer.
- **Three levels:** conceptual (for the business), logical (attributes, keys, normalization), physical (types, indexes, one DBMS).
- **Process:** requirements + questions → entities → relationships → conceptual review → logical → physical → validate; loop back freely.

- **Best practices:** constraints in the schema, one naming style, dates as dates, money as `numeric`, explicit `NOT NULL`, surrogate PK + unique natural key, 3NF by default, index real queries.
- **ER notation:** Chen (rectangle, ellipse, diamond; double ellipse = multivalued, dashed = derived, double rectangle/diamond = weak) vs crow's foot (`||`, `o|`, `o{`, `|{`), which mermaid draws.
- **Relationships:** degree, cardinality, participation are three separate questions; (min,max) = (participation, cardinality).
- **Relational model (Codd 1970):** relations of atomic values, links by value, declarative queries; SQL tables allow duplicates unless keyed; `NULL` needs `IS NULL`.
- **Keys:** superkey ⊇ candidate (minimal) ⊇ primary; alternates get `UNIQUE`; prime = in any candidate key.
- **Integrity:** domain, entity, key, referential (+ ON DELETE choices); UNIQUE usually allows many NULLs.
- **Mapping:** composite → parts, multivalued → table, derived → view, 1:N → FK on many, M:N → junction, 1:1 → FK + UNIQUE, weak → owner key + partial key + CASCADE, total → NOT NULL. Then normalize — Chapter 11.

Next: Chapter 11 — Normalization, which checks a design for redundancy and splits tables safely.

Chapter 11 — Normalization

Chapter 10 turned a business description into tables. This chapter asks the next question: **does each fact live in exactly one place?** When it doesn't, the table can contradict itself — a customer with two cities, a doctor with two specialties — and some facts cannot be stored or are lost by accident. **Normalization** is the precise, rule-based way to find such tables and split them safely.

Normalization is often taught as vague advice ("avoid redundancy"). Here it is built on one exact tool, the **functional dependency**, so that "is this table in 3NF?" (third normal form, §11.9) has a checkable yes/no answer. A small Python program (`code/fd_tools.py`) computes closures (everything a set of columns determines, §11.4), candidate keys (minimal column sets that identify a row) and normal-form violations, and every worked answer in the chapter was verified with it. The SQL files show the anomalies (the insert/update/delete failures of §11.6) happening on PostgreSQL and disappearing after decomposition (splitting one table into several).

What you'll learn

1. Functional dependencies: the definition, where they come from, and their types (trivial, full, partial, transitive, multivalued).
2. Armstrong's inference rules, attribute closure, finding every candidate key, and minimal covers.
3. The three anomalies (insert, update, delete) — demonstrated live.
4. 1NF, 2NF, 3NF, BCNF, 4NF and 5NF: the rule, a violating table, the fix, and the proof that nothing was lost.
5. Lossless decomposition and dependency preservation — and why 3NF, not BCNF, is the usual stopping point.
6. Four exercises with full solutions, and when to deliberately **denormalize**.

Files in `code/`: `00_schema.sql` (one wide order table), `01`–`07` (SQL demos, each on a fresh database) and `fd_tools.py` (run with Python 3: `python fd_tools.py`). All "Output:" blocks are real.

11.1 Functional dependencies

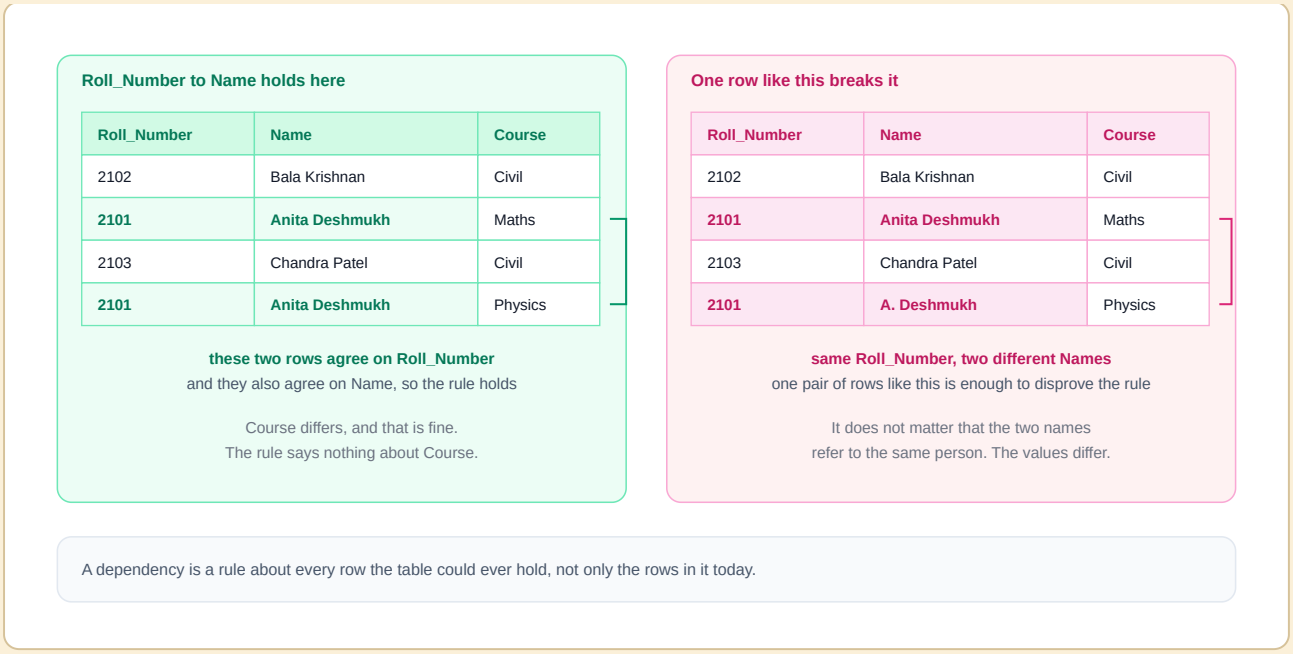
What it is

A **functional dependency (FD)** says that the values of one set of columns *fix* the values of another. Written `X -> Y`, read "X determines Y" (or "Y is functionally dependent on X").

The test, in terms of rows: take any two rows. If they agree on every column of `X`, they must agree on every column of `Y`. A single pair of rows that agrees on `X` but differs on `Y` proves `X -> Y` false.

Analogy: a national ID number determines a person's date of birth — two records with the same ID and different birthdays mean one record is wrong. But a birthday does not determine an ID: millions of people share one.

Figure — the FD rule in rows: two rows that agree on the left side must agree on the right side (course diagram)



The running example

This chapter uses one deliberately bad table from a small online shop (`code/00_schema.sql`):

order_id	product_id	order_date	customer_id	customer_name	customer_city	product_name	list_price	qty
O1	P1	2026-09-01	C1	Nadia Lopez	Lisbon	Kettle	29.00	1
O1	P2	2026-09-01	C1	Nadia Lopez	Lisbon	Mug	6.50	4
O2	P1	2026-09-03	C2	Jonas Berg	Oslo	Kettle	29.00	1
O3	P3	2026-09-05	C1	Nadia Lopez	Lisbon	Teapot	18.00	2
O4	P4	2026-09-06	C3	Priya Das	Kochi	Tea tin	9.00	3

The shop owner confirms four rules — these are the FDs:

#	FD	In words
F1	`order_id -> order_date, customer_id`	an order is placed on one date by one customer
F2	`customer_id -> customer_name, customer_city`	a customer has one name and one city
F3	`product_id -> product_name, list_price`	a product has one name and one current list price
F4	`{order_id, product_id} -> qty`	a product appears once per order, with one quantity

> **Vocabulary**

- **Determinant** — the left side (`customer_id` in F2). Several normal forms are phrased in terms of determinants.
- **Dependent** — the right side.
- A side with several columns is **composite** (`{order_id, product_id}` in F4).

An FD is a rule, not a summary of today's rows

Look at the table: every `customer_name` is different, so `customer_name -> customer_city` *happens* to hold in these five rows. It is **not** a real dependency — two customers called Nadia Lopez can live in different cities. Rows can **disprove** an FD (by showing a counter-pair) but can **never prove** one. FDs are constraints on every legal state of the table, like `NOT NULL`.

Where FDs come from: ask the people who own the data, one question per candidate rule — *"If I tell you the X, do you always know the Y?"* Customer id → city? "Yes, always." City → customer? "No." Write every

answer down before designing tables; that list is the input to everything in this chapter.

Direction matters: `X -> Y` says nothing about `Y -> X`. Both can hold (an id and a unique email determine each other), but each needs its own check.

FDs and keys are the same idea

From §10.13: a **superkey** is a set that never repeats. In FD terms: **`X` is a superkey precisely when `X` functionally determines all columns** (two rows equal on `X` would be equal everywhere, i.e. the same row). A **candidate key** is then a superkey from which nothing can be removed. So the tool that tests FDs (closure, §11.4) is also the tool that finds keys.

Why the rest of the chapter needs this

Each normal form is a statement about which FDs may exist in a table: 2NF bans one shape, 3NF another, BCNF a third. Once the FDs are written down, "is this table normalized?" stops being opinion.

> **TL;DR**
>
> `X -> Y`: rows equal on X are equal on Y. One counter-pair disproves it; data never proves it — FDs come from business rules. X is a superkey iff X determines all columns.

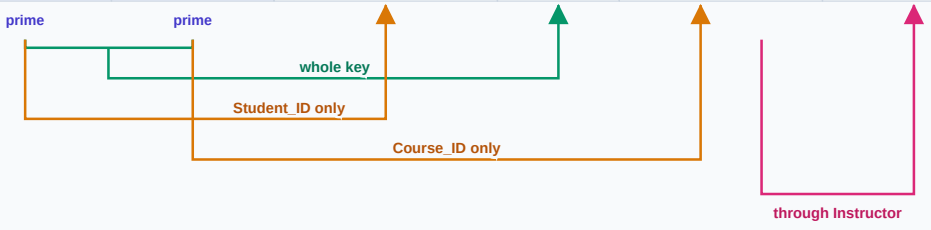
11.2 Types of functional dependencies

In the running example the only candidate key is `{order_id, product_id}`, so **prime attributes** (members of some candidate key — §10.13) are `order_id` and `product_id`; all others are **non-prime**.

Figure — full, partial and transitive dependencies on one table (course diagram, using the course's enrolment example)

Enrollment. The candidate key is shaded, and every dependency is drawn below the table.

Student_ID	Course_ID	Student_Name	Grade	Instructor	Dept
S101	C101	Alice Smith	A	Dr. Smith	Science
S101	C102	Alice Smith	B	Dr. Brown	Arts
S102	C101	Bob Johnson	B	Dr. Smith	Science
S103	C103	Carol White	C	Dr. Lee	Commerce
S103	C104	Carol White	A	Dr. Smith	Science



full dependency	Grade needs the whole key. Drop either half and it is no longer fixed. This shape is fine.
partial dependency	Student_Name needs only Student_ID. Instructor needs only Course_ID. Second normal form bans this.
transitive dependency	the key reaches Dept only through Instructor, which is not a key. Third normal form bans this.

Type	Definition	Example (running table)	Banned by
Trivial	the right side is contained in the left: `{order_id, product_id} -> product_id`	always true; says nothing	never a problem — every rule says "non-trivial"
Non-trivial	the right side adds a column the left side lacks	`customer_id -> customer_city`	—
Completely non-trivial	left and right share no column	`product_id -> list_price`	—
Full	needs its *whole* (composite) determinant: remove any column and it fails	`{order_id, product_id} -> qty` (O1 has quantities 1 and 4; P1 appears in O1 and O2)	—
Partial	a *proper part* of a candidate key already determines a non-prime column	`order_id -> order_date`, `product_id -> product_name`	**2NF**
Transitive	from `X -> Y` plus `Y -> Z` you get `X -> Z`, where `Y` is not a superkey and `Z` is non-prime	`order_id -> customer_id -> customer_city`	**3NF**
Determinant not a superkey	any non-trivial `X -> Y` with X not a superkey	all of F1–F3 here	**BCNF**
Multivalued (`X ->> Y`)	for one X, the *set* of Y values is independent of the other columns	a restaurant's dishes vs its delivery zones (§11.12)	**4NF**

Notes worth being precise about:

- **Full vs partial only makes sense for composite determinants.** With a one-column key there is no "part", so a partial dependency is impossible.
- **The two conditions on "transitive" matter.** Without "Y is not a superkey" every table would be full of transitive dependencies (the key determines itself determines everything).
- **Partial and transitive can overlap:** `order_id -> customer_city` is partial (order_id is part of the key) *and* transitive (via customer_id).
- **Multivalued dependencies** are a different kind of constraint (sets of values, not single values); every FD is also an MVD (multivalued dependency), not vice versa. §11.12 treats them.

Self-check

1. `{order_id, product_id} -> order_id` — which type? → Trivial.
2. Why is `product_id -> product_name` partial? → `product_id` is a proper part of the only candidate key and `product_name` is non-prime.
3. In a table with a single-column key, which of partial/transitive can exist? → Only transitive.

TL;DR

Trivial (always true) · full (needs whole key) · partial (part of a key → non-prime; kills 2NF) · transitive (key → non-key → non-prime; kills 3NF) · non-superkey determinant (kills BCNF) · multivalued (kills 4NF).

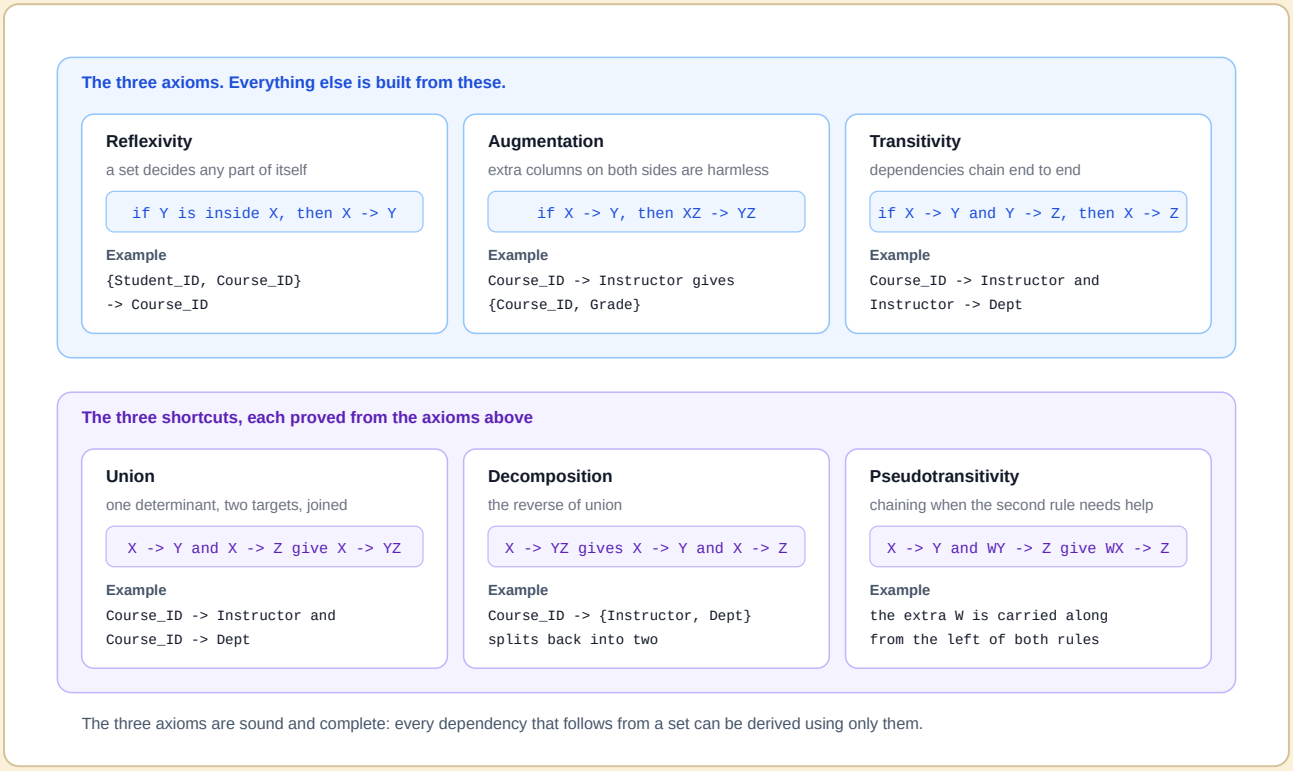
11.3 Armstrong's inference rules

What it is

A set of FDs implies more FDs than it lists. Nobody wrote `order_id -> customer_city`, yet F1 (`order_id -> customer_id`) and F2 (`customer_id -> customer_city`) force it. The given FDs `F` plus everything they

imply is the **closure of F**, written **F⁺**. The rules for deriving implied FDs are **Armstrong's axioms** (W. W. Armstrong, 1974).

Figure — the three axioms and the three derived rules (course diagram)



The three axioms

Axiom	Rule	Example	Why it is true
Reflexivity	if `Y ⊆ X` then `X -> Y`	`{order_id, product_id} -> product_id`	rows equal on X are equal on any part of X (these are the trivial FDs)
Augmentation	if `X -> Y` then `XZ -> YZ`	from F2: `{customer_id, product_id} -> {customer_city, product_id}`	rows equal on X and Z are equal on Y (by the FD) and on Z (by assumption)
Transitivity	if `X -> Y` and `Y -> Z` then `X -> Z`	F1 + F2: `order_id -> customer_city`	chain the two guarantees

The axioms are:

- **Sound** — they never derive an FD that is false in some table satisfying F.
- **Complete** — every FD that holds in *all* tables satisfying F can be derived with them.

So these three are enough; the next three add convenience, not power.

Three derived rules

Rule	Pattern	Example
Union	`X -> Y` and `X -> Z` ⟹ `X -> YZ`	`order_id -> order_date` and `order_id -> customer_city` ⟹ `order_id -> {order_date, customer_city}`
Decomposition (split)	`X -> YZ` ⟹ `X -> Y` and `X -> Z`	F3 ⟹ `product_id -> list_price`
Pseudotransitivity	`X -> Y` and `WY -> Z` ⟹ `WX -> Z`	suppose a customer leaves one review per product: `{customer_id, product_id} -> review_stars`. With F1 (`order_id -> customer_id`), taking W = product_id: `{order_id, product_id} -> review_stars`

Decomposition is why worked examples first rewrite every FD with a single column on the right — the list becomes easier to scan.

Deriving union from the axioms (a common exam question):

1. Augment `X -> Y` with X: `X -> XY` (since XX = X).
2. Augment `X -> Z` with Y: `XY -> YZ`.
3. Transitivity on 1 and 2: `X -> YZ`. ∎

Decomposition follows from reflexivity (`YZ -> Y`) plus transitivity; pseudotransitivity from augmenting `X -> Y` with W and then transitivity.

What the rules are for

Finding candidate keys (most "X determines everything" facts are derived, not given) and checking claims like "this table is in 3NF", which depend on implied FDs too. Applying the rules by hand is slow for 6+ columns, so in practice you use attribute closure (next), which the axioms justify.

`fd_tools.py` confirms both derivations above:

```
order_id -> customer_city follows? True
customer_id -> order_id follows?   False
pseudotransitivity: order_id -> customer_id and {customer_id, product_id} -> review_stars
  so {order_id, product_id} -> review_stars? True
```

TL;DR

F⁺ = all FDs implied by F. Axioms: reflexivity, augmentation, transitivity — sound and complete. Shortcuts: union, decomposition, pseudotransitivity.

11.4 Attribute closure and finding candidate keys

What it is

The **closure of an attribute set X**, written **X⁺**, is every attribute X determines, directly or through chains. It always contains X itself.

The algorithm

1. Start with `result = X`.
2. Scan the FDs. For any FD whose **entire** left side is inside `result`, add its right side.
3. Repeat full passes until a pass adds nothing. `result` is X⁺.

The only classic mistake is firing an FD whose left side is only *partly* present. Repeated passes matter because an FD can become usable halfway through (once an earlier FD adds the column it needs).

The core of `fd_tools.py`:

```
def closure(attrs, fds, trace=False):
    """X+ : keep adding the right side of any FD whose WHOLE left side is already in the set."""
    result = set(attrs)
    changed = True
    while changed:
        changed = False
        for lhs, rhs in fds:
            if lhs <= result and not rhs <= result:   # lhs fully present, something new to add
                result |= rhs
                changed = True
    return frozenset(result)
```

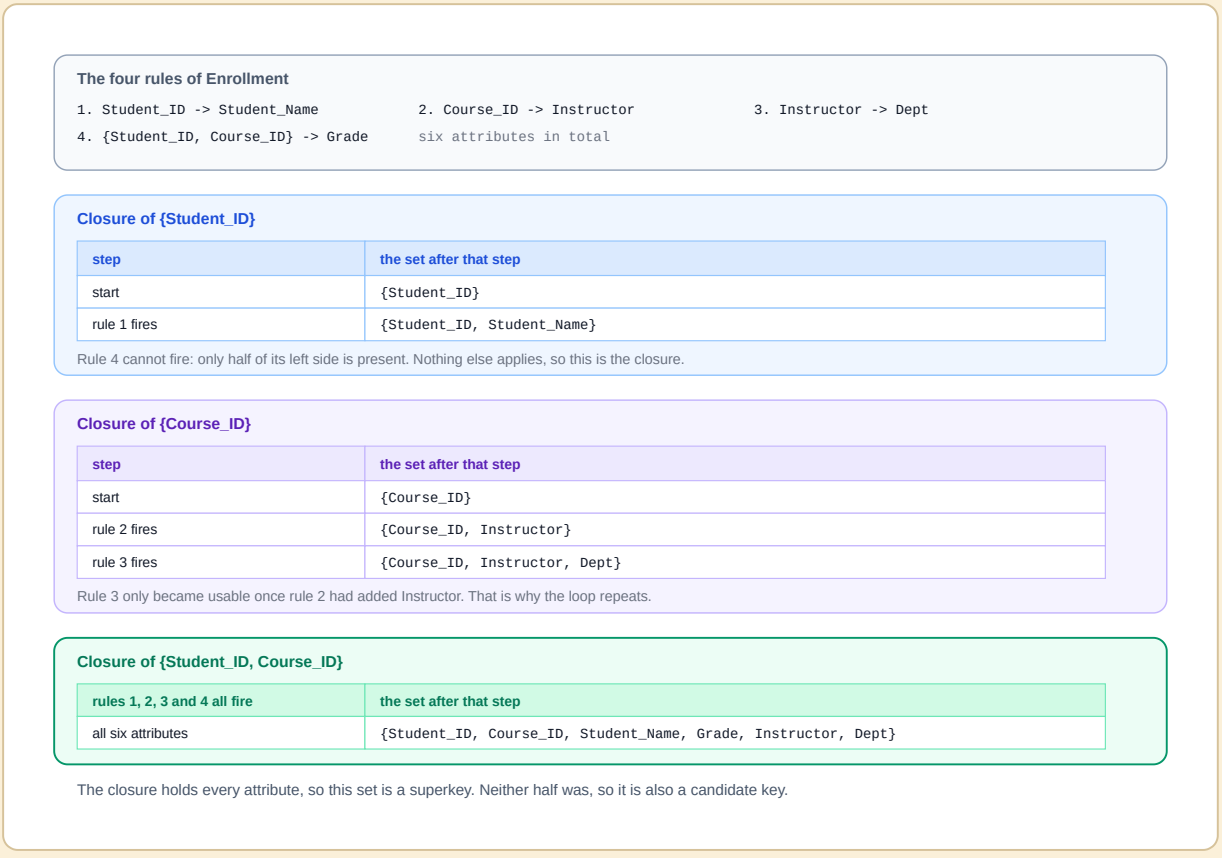
Worked closures on the running example

{order_id}⁺ — start `{order_id}`. F1 fires (`order_id` present) → add `order_date, customer_id`. Now F2's left side `customer_id` is present → add `customer_name, customer_city`. F3 needs `product_id` — absent. F4 needs *both* `order_id` and `product_id` — only one present, **skip**. Nothing more fires.

Output of `fd_tools.py`:

```
closure trace for {order_id}:
    step 1: {order_id} -> {customer_id, order_date} fires, set = {customer_id, order_date, order_id}
    step 2: {customer_id} -> {customer_city, customer_name} fires, set = {customer_city, customer_id,
customer_name, order_date, order_id}
{order_id}+ = {customer_city, customer_id, customer_name, order_date, order_id}
{product_id}+ = {list_price, product_id, product_name}
{order_id, product_id}+ = {customer_city, customer_id, customer_name, list_price, order_date, order_id,
product_id, product_name, qty}
```

Figure — the closure loop step by step on the course's enrolment example (course diagram)



Two uses of closure

Question	Test
Is X a **superkey**?	X^+ contains **all** attributes of the table
Is X a **candidate key**?	X is a superkey **and** no proper subset is (it is enough to test the subsets with one attribute removed)
Does **X -> Y** follow from F?	$Y \subseteq X^+$

On the running example: `{order_id, product_id}⁺` is all nine columns → superkey. `{order_id}⁺` (5 columns) and `{product_id}⁺` (3 columns) are not → neither half is a superkey → **`{order_id, product_id}` is a candidate key**. And `customer_id -> order_id` does **not** follow (`order_id ∉ {customer_id}⁺`) — matching reality: a customer has many orders.

Finding *all* candidate keys systematically

1. **Classify attributes** by where they appear in the FDs:
   - never on any right side → **must be in every key** (nothing can derive them);
   - only on right sides (never on a left) → **in no key** (they add nothing);
   - on both sides → maybe.
2. Compute the closure of the "must" set. If it is everything, it is the *only* candidate key.
3. Otherwise add "maybe" attributes one at a time, then two at a time, …, skipping any set that contains a key already found (it would not be minimal).

Running example: `order_id` and `product_id` never appear on a right side → both in every key, and their closure is everything → the unique candidate key. That shortcut is why `fd_tools.candidate_keys` starts from that "core".

A harder example (Exercise 3 below): R(A, B, C, D, E) with `A -> B`, `BC -> D`, `D -> A`, `C -> E`. `C` is never on a right side → in every key; `C⁺ = {C, E}` is not everything. Try adding one attribute:

```
{C}+ = {C, E}
{A, C}+ = {A, B, C, D, E}
{B, C}+ = {A, B, C, D, E}
{C, D}+ = {A, B, C, D, E}
{C, E}+ = {C, E}
```

So there are **three** candidate keys: `{A,C}`, `{B,C}`, `{C,D}`. Prime attributes: A, B, C, D; non-prime: E. Tables with several overlapping keys like this are exactly where BCNF differs from 3NF (§11.11).

In the interview

30-second answer: "To test whether X is a key, compute X^+: start with X and repeatedly add the right side of any FD whose left side is fully contained, until nothing changes. If X^+ is the whole relation, X is a superkey; if no smaller subset is, it's a candidate key. Attributes that never appear on a right-hand side must be in every key, which is where I start."

- *How do you check whether an FD is implied?* → Compute the closure of its left side and see if the right side is inside.
- *Complexity?* → In big-O notation (how the work grows as the input grows), a naive loop is $O(|F|^2 \cdot |attrs|)$ per closure; finding all keys can be exponential (doubling with every extra attribute) in the worst case (a relation can have exponentially many keys), but class-sized problems are tiny.

TL;DR

X⁺ = everything X determines (fire only FDs whose whole LHS is present; loop until stable). Superkey ⟺ X⁺ = all attributes; candidate key = minimal superkey; X -> Y implied ⟺ Y ⊆ X⁺. Never-on-the-right attributes are in every key.

11.5 Minimal (canonical) cover

What it is

Different FD lists can say the same thing. A **minimal cover** (also called a **canonical cover**, F_c) is an equivalent list with no waste:

1. every right side is a **single** attribute (use decomposition);
2. no left side has an **extraneous** attribute (one you can drop without changing F⁺);
3. no FD is **redundant** (derivable from the others).

It is the input to the 3NF synthesis algorithm (§11.14) and a common interview/exam exercise.

Worked example

F = { `A -> BC`, `B -> C`, `AB -> D` }

1. Split: `A -> B`, `A -> C`, `B -> C`, `AB -> D`.
2. Extraneous LHS: in `AB -> D`, is B extraneous? `A⁺` (using the current list) = {A, B, C, D} contains D → yes, replace with `A -> D`.
3. Redundant FDs: `A -> C` follows from `A -> B` and `B -> C` → remove it.

Result `{A -> B, B -> C, A -> D}` — the same as `fd_tools.py` prints:

```
=== minimal cover of {A->BC, B->C, AB->D}: {A}->{B}, {B}->{C}, {A}->{D}
```

A minimal cover is not always unique (the order you remove things can matter), but every minimal cover implies exactly F⁺.

TL;DR

Minimal cover = singleton right sides, no extraneous left attributes, no redundant FDs; equivalent to F. Needed for 3NF synthesis.

11.6 Anomalies — why normalization exists

The problem it solves

The wide order table stores "Nadia lives in Lisbon" three times and "P1 is a Kettle at 29.00" twice. That repetition causes three named failures:

Anomaly	What goes wrong	In the running table
Insertion	you cannot record one fact without inventing an unrelated one	a new product nobody has ordered yet has no row to live in — the key needs an `order_id`
Update	one real-world change must be repeated on many rows; miss one and the table contradicts itself	Nadia moves: 3 rows must change
Deletion	deleting one fact silently deletes another	cancel Priya's only order and the shop forgets Priya *and* the tea tin

Figure — the three anomalies on the course's enrolment table (course diagram)

Seeing them happen (`code/01_anomalies.sql`)

```
-- INSERT: a new product with no order
INSERT INTO order_line_wide (order_id, product_id, ..., product_name, list_price, qty)
VALUES (NULL, 'P5', ..., 'Milk jug', 12.00, 1);              -- inside a DO block that catches the error
-- UPDATE: Nadia moves, but a careless WHERE edits only one of her rows
UPDATE order_line_wide SET customer_city = 'Porto' WHERE customer_id = 'C1' AND order_id = 'O3';
-- DELETE: cancel O4
DELETE FROM order_line_wide WHERE order_id = 'O4';
```

Output:

```
--- the same facts are stored many times
 customer_id | customer_name | customer_city | rows_holding_it
-------------+---------------+---------------+-----------------
 C1          | Nadia Lopez   | Lisbon        |               3
 C2          | Jonas Berg    | Oslo          |               1
 C3          | Priya Das     | Kochi         |               1
(3 rows)

--- INSERT anomaly: a new product nobody has ordered yet has nowhere to go
NOTICE:  refused: null value in column "order_id" of relation "order_line_wide" violates not-null
constraint
--- UPDATE anomaly: Nadia moves to Porto, but the update misses one of her rows
 customer_id | cities_on_file
-------------+----------------
 C1          | {Lisbon,Porto}
(1 row)

--- DELETE anomaly: cancel order O4 and customer C3 and product P4 vanish with it
 rows_about_priya | rows_about_tea_tin
------------------+--------------------
                0 |                  0
(1 row)
```

What normalization does

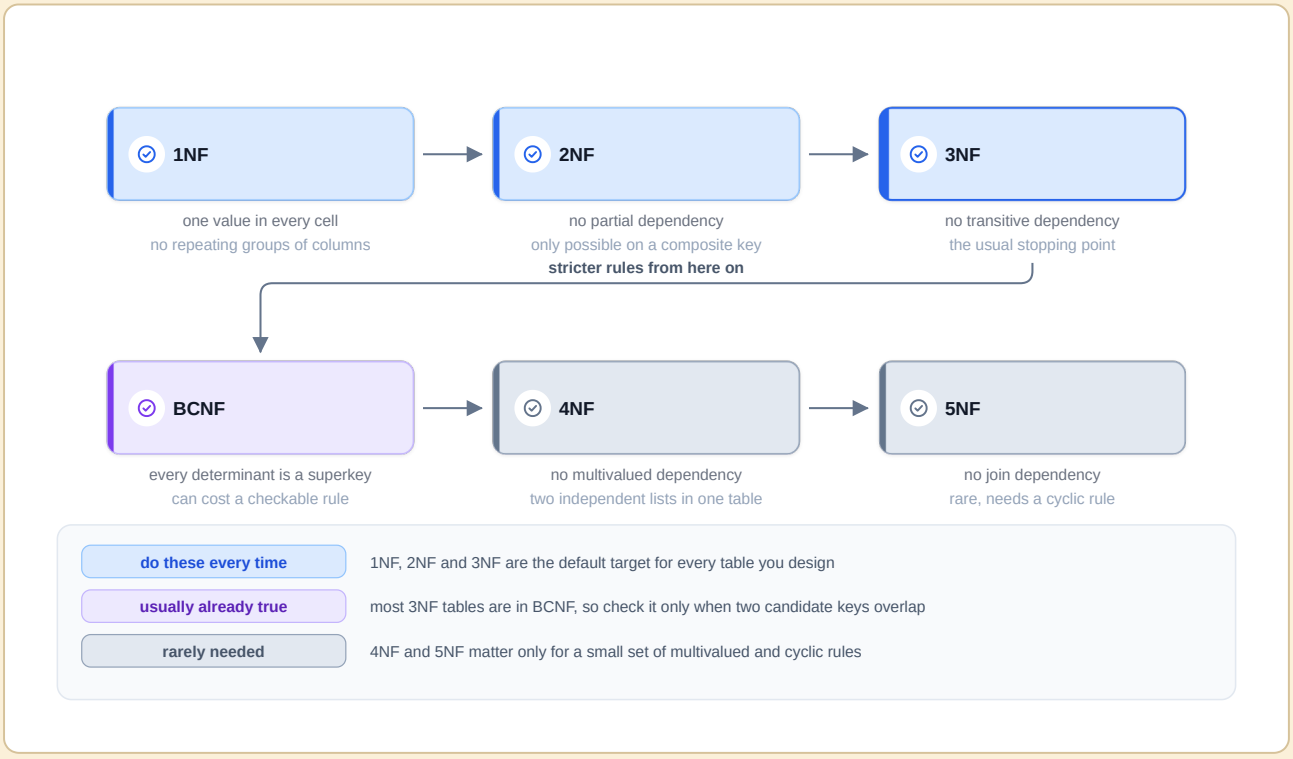
Normalization splits a table along its functional dependencies so that **every fact has exactly one home** — "Nadia lives in Lisbon" in one row of a `customer` table. One row to update, nothing unrelated to delete, and every fact can be inserted on its own. It does this with **decomposition** (replacing a table by projections of it — a *projection* is a table made of some of its columns, duplicates removed) that must be **lossless** (§11.10).

The ladder of normal forms

Each **normal form** is a named test a table passes or fails. Each assumes the previous one and is stricter:

Form	Removes	One-line rule	Origin
1NF	lists and repeating groups in cells	every cell holds one atomic value	Codd 1970
2NF	partial dependencies	no non-prime attribute depends on part of a candidate key	Codd 1971
3NF	transitive dependencies	no non-prime attribute depends on a non-superkey	Codd 1971
BCNF	remaining FD redundancy	every non-trivial FD's determinant is a superkey	Boyce & Codd 1974
4NF	independent multivalued facts	every non-trivial MVD's determinant is a superkey	Fagin 1977
5NF	join dependencies not implied by keys	the table cannot be split losslessly except via its keys	Fagin 1979

Figure — the ladder of normal forms (course diagram)



(There are also domain-key NF and 6NF — used in temporal databases (which record how values change over time) and some data-warehouse (a database built for analysis and reporting rather than day-to-day transactions) styles — which are beyond everyday design.)

You do not always climb to the top. Each step removes redundancy and usually adds a join. **3NF is where most practical designs stop**: it removes the redundancy behind the worst anomalies and always has a safe decomposition (§11.14). Climbing higher is something you choose on purpose when a particular problem demands it. Putting redundancy back on purpose is **denormalization** (§11.16).

TL;DR

Redundancy → insertion, update and deletion anomalies. Normalization decomposes tables along FDs so each fact is stored once. 1NF → 2NF → 3NF → BCNF → 4NF → 5NF, each stricter; most designs stop at 3NF.

11.7 First normal form (1NF)

What it is

A table is in **1NF** when:

1. every cell holds a **single, indivisible (atomic)** value — no lists, sets or nested records;
2. there are **no repeating groups** — no `phone1, phone2, phone3` columns;
3. rows are distinguishable — the table has a key (which, in the relational model, every relation has).

It is purely about the **shape of a cell**.

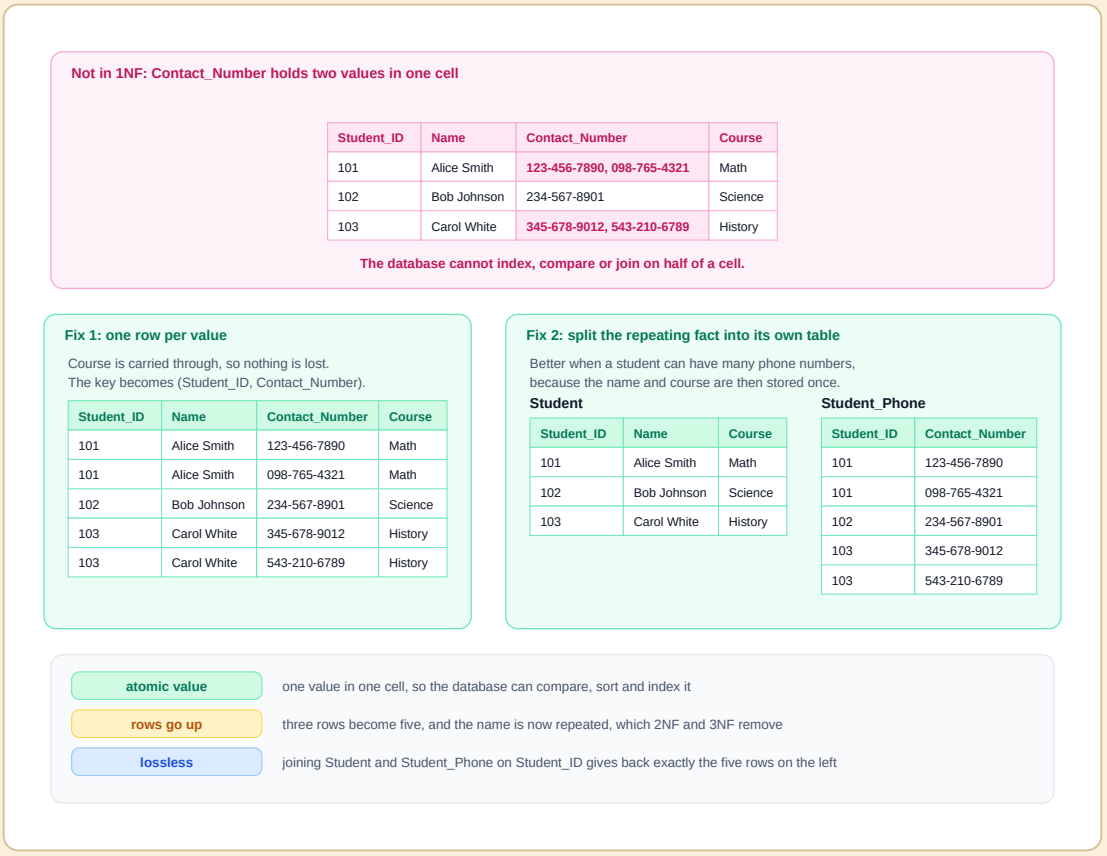
The problem it solves

```
vol_id | name  | languages
     1 | Hana  | Japanese, English
     3 | Ama   | Twi, English, French
```

"Who speaks English?" needs a text search (`LIKE '%English%'`), which also matches "Old English" or a typo; you cannot count languages, index them usefully, or constrain their spelling.

How to fix it — two routes

Figure — a list in a cell, fixed by one-row-per-value or by a separate table (course diagram)



1. **One row per value** in the same table (the key becomes `(vol_id, language)`). This reaches 1NF — but copies the other columns onto every new row.

2. **A separate table** for the repeating fact — `volunteer(vol_id, name)` + `volunteer_language(vol_id, language)`. This is what 2NF would produce from route 1 anyway (the name depends only on `vol_id`), and it is what §10.6 predicted for a multivalued attribute.

`code/03_first_normal_form.sql` repairs both kinds of violation in SQL (*unpivoting* turns several columns into rows; `CROSS JOIN LATERAL` lets the `VALUES` list read the current row's columns):

```sql
-- a list in one cell -> one row per value (string_to_table: PostgreSQL 14+)
CREATE TABLE volunteer_language AS
SELECT vol_id, trim(lang) AS language
FROM volunteer_packed, string_to_table(languages, ',') AS lang;
ALTER TABLE volunteer_language ADD PRIMARY KEY (vol_id, language);

-- a repeating group phone1..phone3 -> rows (unpivot with LATERAL VALUES)
SELECT g.guest_id, g.name, p.phone
FROM guest_sideways g
CROSS JOIN LATERAL (VALUES (g.phone1), (g.phone2), (g.phone3)) AS p(phone)
WHERE p.phone IS NOT NULL
ORDER BY g.guest_id, p.phone;
```

Output:

```
--- repair: one row per value (string_to_table needs PostgreSQL 14+)
 vol_id | language
--------+----------
      1 | English
      1 | Japanese
      2 | Spanish
      3 | English
      3 | French
      3 | Twi
(6 rows)

--- now counting and exact matching are ordinary SQL
 language | speakers
----------+----------
 English  |        2
 French   |        1
 Japanese |        1
 Spanish  |        1
 Twi      |        1
(5 rows)

--- unpivot the phone columns into rows; guests without a phone produce no rows
 guest_id | name  |  phone
----------+-------+----------
       10 | Ivo   | 555-0101
       10 | Ivo   | 555-0102
       12 | Malik | 555-0301
       12 | Malik | 555-0303
(4 rows)
```

Dialect note: on PostgreSQL < 14 use `unnest(string_to_array(languages, ','))`; MySQL 8 has `JSON_TABLE` or a recursive CTE; SQL Server has `STRING_SPLIT` and `CROSS APPLY (VALUES …)` for the unpivot. (Not run here.)

What 1NF does *not* say — the two classic misconceptions

- **1NF does not require column values to be unique.** Only keys are unique; `name` may repeat freely.
- **1NF does not reduce redundancy — route 1 increases it.** Splitting one row into three copies every other value three times. Removing that repetition is 2NF's and 3NF's job.

Also: a normalization step splits tables or rows but **never drops a column**. If a column disappears, information was lost and the design is wrong.

Borderline cases

- **Structured text** (a whole address in one column) — a 1NF problem only if your application queries the parts.
- **JSON / arrays** — JSON is a text format for nested data (`{"key": [1, 2]}`); both are strictly non-atomic. PostgreSQL has first-class `jsonb` (JSON stored in a parsed binary form) and arrays with indexes (Chapter 14); storing a document *on purpose* (sparse attributes — ones most rows don't have — or payloads you never filter on) is a legitimate design. The question is: do you search, join or constrain on what is inside? If yes, it belongs in rows.

Common mistakes

- Comma-separated ids in a column ("tag_ids = '3,7,9'") — the *SQL Antipatterns* book calls this **Jaywalking**; you cannot join on it or enforce FKs.
- Numbered columns (`item1…item5`).
- Believing 1NF means "no duplicate values".

TL;DR

1NF = atomic cells, no repeating groups, a key. Fix lists with one row per value or (better) a child table. 1NF does not remove redundancy and does not require unique values.

11.8 Second normal form (2NF)

What it is

A table is in **2NF** when it is in 1NF **and no non-prime attribute is partially dependent on any candidate key** — every non-key column depends on the *whole* key.

A table whose candidate keys all have **one column** is automatically in 2NF (there is no "part" of a single column).

Finding the violation in the running table

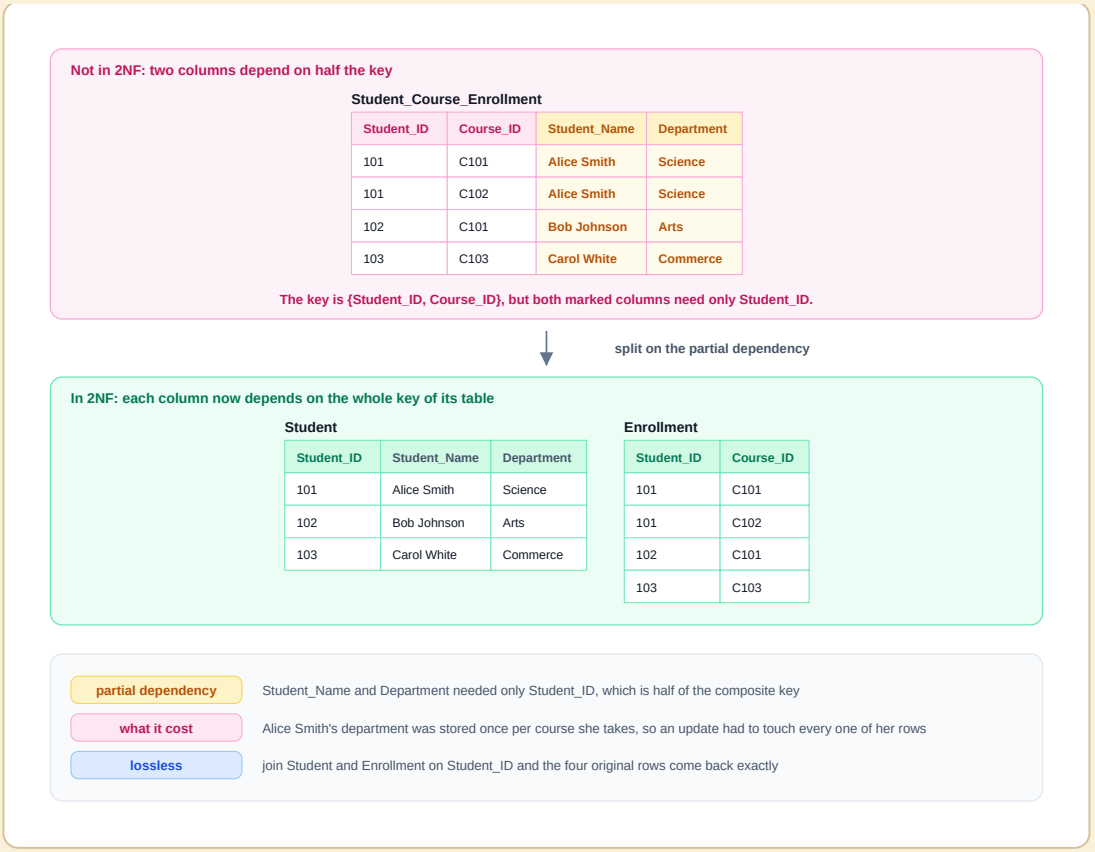
Candidate key `{order_id, product_id}`. Test each non-prime column: does it need the whole key?

Column	Needs	Verdict
`qty`	order **and** product	full ✓
`order_date`, `customer_id`	`order_id` only	partial ✗
`customer_name`, `customer_city`	`order_id` only (via customer_id)	partial ✗
`product_name`, `list_price`	`product_id` only	partial ✗

`fd_tools.py` agrees:

```
candidate keys: {order_id, product_id}
2NF violations: ['{order_id} -> {customer_city, customer_id, customer_name, order_date}', '{product_id} -
> {list_price, product_name}']
```

Figure — a composite-key table, its partial dependencies and the 2NF split (course diagram)



How to fix it

1. For each part of the key that has dependents, create a table keyed by that part and move its dependents there.
2. Leave the full-key columns in a table keyed by the whole key.
3. Group the moved columns **by the key part that determines them** — one new table per part.

Result:

- `order_header(order_id, order_date, customer_id, customer_name, customer_city)`
- `product(product_id, product_name, list_price)`
- `order_item(order_id, product_id, qty)`

`product` and `order_item` are now clean. `order_header` still repeats Nadia's city once per order — that is a *transitive* dependency, which 2NF cannot see. Next section.

Common mistakes

- Moving only one part's dependents (e.g. splitting off `product` but leaving order facts in the composite-key table) — still not 2NF.
- Checking only the primary key: 2NF is about **every candidate key** and **non-prime** attributes (Exercise 1 has a second candidate key).

TL;DR

2NF = 1NF + no non-prime column determined by part of a candidate key. Only composite keys can violate it. Fix: one table per key part with its dependents, plus the full-key table.

11.9 Third normal form (3NF)

What it is

3NF requires 2NF **plus: no non-prime attribute is transitively dependent on a candidate key** — non-key columns depend on the key, the whole key, and nothing but the key. (A popular mnemonic, usually credited to Bill Kent.)

The precise textbook form (Zaniolo's), which you should know for interviews: **for every non-trivial FD `X -> A`, either X is a superkey, or A is a prime attribute.** It also covers 2NF, so a table meeting it is automatically in 2NF.

Finding the violation

`order_header(order_id, order_date, customer_id, customer_name, customer_city)` has a single-column key, so it is in 2NF. But:

`order_id -> customer_id` and `customer_id -> customer_city`, and `customer_id` is **not a superkey** here (Nadia has three orders), and `customer_city` is **non-prime** → transitive dependency → **not 3NF**. Check with the precise form: FD `customer_id -> customer_city` — is `customer_id` a superkey? No. Is `customer_city` prime? No. Both escapes fail.

Figure — a transitive dependency through a non-key column and the 3NF split (course diagram)

How to fix it

Move the columns at the **far end of the chain** into a new table whose key is the **middle** column:

- `customer(customer_id, customer_name, customer_city)`
- `orders(order_id, order_date, customer_id → customer)`

The middle column stays in `orders` as an FK — without it there would be nothing to join on.

2NF vs 3NF side by side

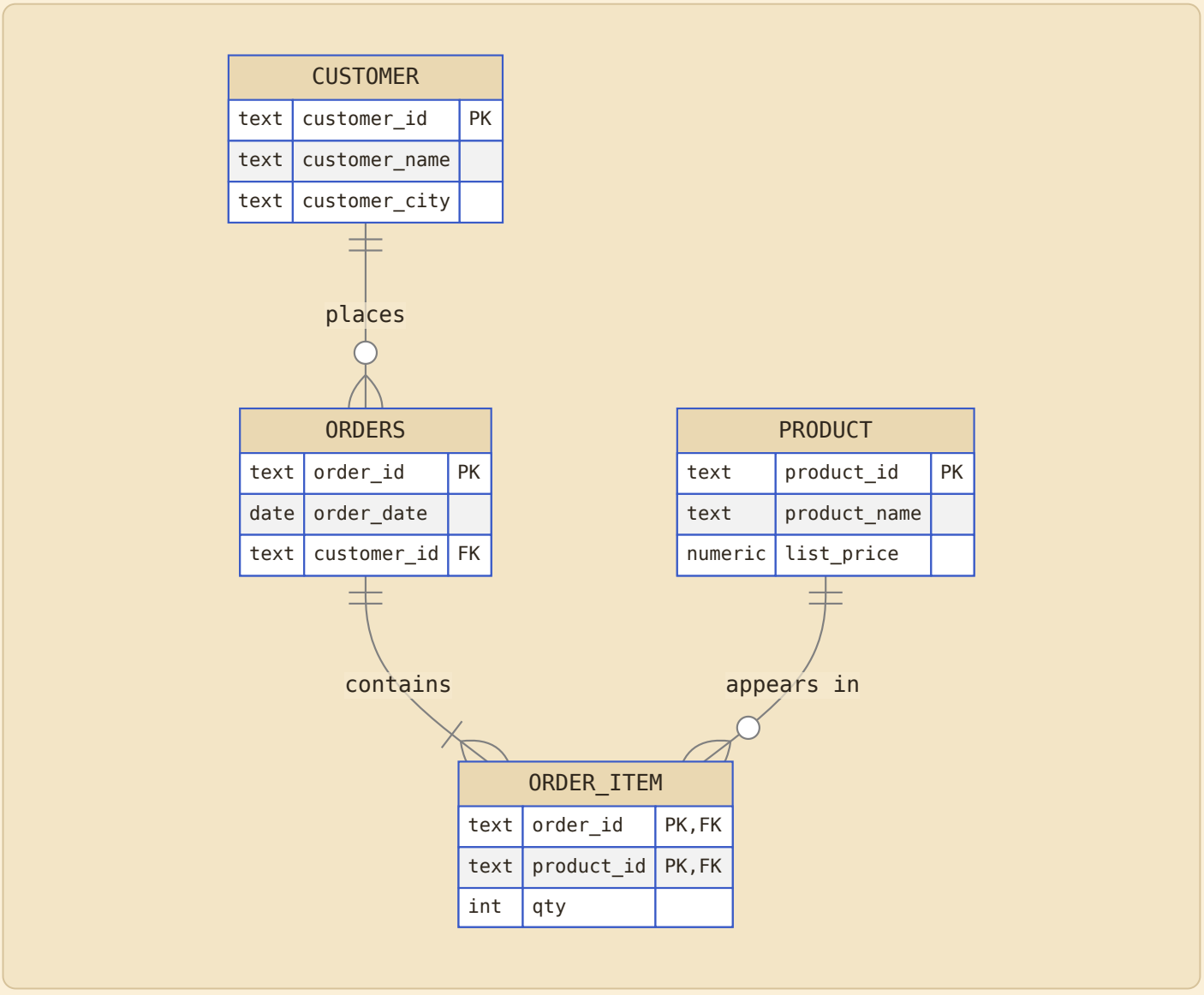
	Dependency runs through	Possible when
2NF violation	**part** of a candidate key	the key is composite
3NF violation	a **non-key** column	any key shape, including one column

So 3NF must be checked on every table; 2NF only on tables with composite keys.

The complete running example in 3NF

The 3NF synthesis algorithm (§11.14) in `fd_tools.py` produces exactly our four tables:

```
3NF synthesis: ['{customer_id, order_date, order_id}', '{customer_city, customer_id, customer_name}',
'{list_price, product_id, product_name}', '{order_id, product_id, qty}']
```



§11.10 builds these tables in PostgreSQL and re-runs the anomalies.

In the interview

30-second answer: "1NF: atomic values. 2NF: no non-key attribute depends on part of a composite key. 3NF: no non-key attribute depends on another non-key attribute — formally, for every non-trivial FD X → A, X is a superkey or A is prime. I normalize to 3NF by default because it removes the update anomalies and there's always a lossless, dependency-preserving 3NF decomposition."

- *Give a 3NF violation with a single-column key.* → `employee(emp_id, dept_id, dept_name)`: `dept_id -> dept_name`.
- *Is a table with only key columns in 3NF?* → Yes (no non-prime attributes at all); check BCNF separately.

> **TL;DR**

3NF = 2NF + no non-key → non-key dependency (formally: every FD's LHS is a superkey or RHS is prime). Fix: split the chain's tail into a table keyed by the middle column, which stays behind as an FK.

11.10 Decomposition done right: lossless joins and dependency preservation

Every split must satisfy two properties — one mandatory, one highly desirable.

Lossless (non-additive) join — mandatory

Joining the pieces must return **exactly** the original rows — nothing lost, **nothing invented**. (A lossy split usually *adds* rows, "spurious tuples"; the information lost is which combinations were true.)

Test for a two-way split of R into R1 and R2: the split is lossless iff the shared columns `R1 ∩ R2` are a **superkey of R1 or of R2** (i.e. `R1 ∩ R2 -> R1` or `R1 ∩ R2 -> R2` is in F^+). Every split in this chapter shares a column that is the key of one piece.

`code/04_lossy_join.sql` shows what happens otherwise. `staff(emp, dept, city)` with `emp -> dept, city`, split two ways:

```
-- bad: (emp, city) + (dept, city); city is a key of neither piece
WITH ec AS (SELECT DISTINCT emp, city FROM staff),
     dc AS (SELECT DISTINCT dept, city FROM staff)
SELECT ec.emp, dc.dept, ec.city,
       (ec.emp, dc.dept, ec.city) IN (SELECT emp, dept, city FROM staff) AS real_row
FROM ec JOIN dc USING (city) ORDER BY 1, 2;
```

Output:

```
--- bad split: (emp, city) and (dept, city). The shared column city is a key of neither piece.
 emp |  dept   | city  | real_row
-----+---------+-------+----------
 Ana | Sales   | Leeds | t
 Ana | Support | Leeds | f
 Ben | Sales   | Leeds | f
 Ben | Support | Leeds | t
 Cai | Sales   | York  | t
(5 rows)

--- good split: (emp, dept) and (emp, city). The shared column emp is a key of both.
 invented_rows
---------------
             0
(1 row)
```

Dependency preservation — strongly desirable

Every original FD should still be checkable **inside a single table** (so a `PRIMARY KEY`/`UNIQUE` constraint enforces it), or be implied by FDs that are. If not, enforcing the rule needs a join — a trigger (code the database runs automatically on every change) or application code. §11.11 shows a split that is lossless but not dependency-preserving.

The running example decomposed in PostgreSQL (`code/02_decompose_and_fix.sql`)

```
CREATE TABLE customer AS SELECT DISTINCT customer_id, customer_name, customer_city FROM order_line_wide;
CREATE TABLE product  AS SELECT DISTINCT product_id, product_name, list_price     FROM order_line_wide;
CREATE TABLE orders   AS SELECT DISTINCT order_id, order_date, customer_id        FROM order_line_wide;
CREATE TABLE order_item AS SELECT order_id, product_id, qty                       FROM order_line_wide;
ALTER TABLE customer   ADD PRIMARY KEY (customer_id);
ALTER TABLE product    ADD PRIMARY KEY (product_id);
ALTER TABLE orders     ADD PRIMARY KEY (order_id),
                       ADD FOREIGN KEY (customer_id) REFERENCES customer;
ALTER TABLE order_item ADD PRIMARY KEY (order_id, product_id),
                       ADD FOREIGN KEY (order_id)   REFERENCES orders ON DELETE CASCADE,
                       ADD FOREIGN KEY (product_id) REFERENCES product;
-- lossless check: (original EXCEPT rejoined) and (rejoined EXCEPT original) must both be empty
```

Output:

```
--- row counts after the split
 customers | products | orders | order_items
-----------+----------+--------+-------------
         3 |        4 |      4 |           5
(1 row)

--- lossless? join the pieces back and compare with the original, both directions
 missing_rows | invented_rows
--------------+---------------
            0 |             0
(1 row)

--- INSERT: a product with no orders is now fine
--- UPDATE: Nadia moves once, in one row
 customer_city
---------------
 Porto
(1 row)

--- DELETE: cancel O4; Priya and the tea tin survive
 priya_rows | tea_tin_rows | o4_items
------------+--------------+----------
          1 |            1 |        0
(1 row)
```

All three anomalies are gone, and `SELECT DISTINCT` + `ADD PRIMARY KEY` is the standard way to perform such a split on live data (if `ADD PRIMARY KEY` fails, the data already contradicts the FD you assumed — a useful check in itself). The pieces also preserve every FD: each of F1–F4 lives inside one table.

TL;DR

Lossless: shared columns must be a key of one side (else the join invents rows). Dependency-preserving: each FD checkable in one table. Verify a split with `EXCEPT` both ways.

11.11 Boyce–Codd normal form (BCNF)

What it is

BCNF holds when the left side of **every non-trivial FD `X -> Y` is a superkey** — "every determinant is a (super)key". Compared with 3NF's precise form, BCNF simply removes the second escape ("…or Y is prime"). Hence every BCNF table is in 3NF; not vice versa. It is sometimes called "3.5NF".

When does 3NF pass but BCNF fail? Only if the table has **several composite candidate keys sharing at least one attribute**. With a single candidate key, 3NF and BCNF coincide — which is why reaching 3NF usually means reaching BCNF too.

A 3NF table that is not in BCNF

A clinic's rules: each doctor has exactly one specialty (`doctor -> specialty`); a patient sees exactly one doctor per specialty (`{patient, specialty} -> doctor`).

patient	specialty	doctor
Ali	Cardiology	Dr Osei
Ali	Dermatology	Dr Lind
Beth	Cardiology	Dr Osei
Beth	Neurology	Dr Tan
Chen	Cardiology	Dr Moreau

`fd_tools.py`:

```
=== visit (3NF, not BCNF)
candidate keys: {doctor, patient}, {patient, specialty}
prime: {doctor, patient, specialty}  non-prime: {}
2NF violations: none
3NF violations: none
BCNF violations: ['{doctor} -> {specialty}']
highest normal form: 3NF
BCNF decomposition: ['{doctor, specialty}', '{doctor, patient}']
FDs no longer checkable in one piece: ['{patient, specialty} -> {doctor}']
```

Why: both candidate keys are composite and overlap on `patient`, so every attribute is prime. `doctor -> specialty` passes 3NF because `specialty` is prime, but fails BCNF because `doctor` is not a superkey. The redundancy is real: "Dr Osei is a cardiologist" is stored once per patient, with the usual anomalies — from `code/05_bcnf.sql`:

```
--- the fact "Dr Osei is a cardiologist" is stored once per patient
  doctor   |  specialty  | copies
-----------+-------------+--------
 Dr Osei   | Cardiology  |      2
 ...
--- nothing stops the copies disagreeing (update anomaly)
 doctor  |      specialties
---------+-----------------------
 Dr Osei | {Cardiology,Oncology}
(1 row)
```

Decomposing

Split on the offending FD: `doctor_specialty(doctor PK, specialty)` and `patient_doctor(patient, doctor)` (PK both columns). The shared column `doctor` is the key of `doctor_specialty`, so the join is lossless (the script prints `invented_rows 0, missing_rows 0`).

The honest cost: a lost dependency

`{patient, specialty} -> doctor` now spans two tables, so no single `PRIMARY KEY`/`UNIQUE` can enforce it:

```
INSERT INTO patient_doctor VALUES ('Ali', 'Dr Moreau');     -- accepted!
```

Output:

```
--- the cost: {patient, specialty} -> doctor is no longer checked by any one table
 patient | specialty  |         doctors
---------+------------+-------------------------
 Ali     | Cardiology | {"Dr Moreau","Dr Osei"}
(1 row)
```

Ali now has two cardiologists, which the business forbids. **General result:** you can always split a table into **3NF** pieces that are both lossless and dependency-preserving; for **BCNF** that is not guaranteed. This is why practitioners usually stop at 3NF and move to BCNF only when the redundancy actually hurts.

A practical middle way (also in `05_bcnf.sql`): keep the 3NF table (so `{patient, specialty}` stays its PK) and add a master (single authoritative) `doctor_specialty` table plus a **composite foreign key** `(doctor, specialty) → doctor_specialty(doctor, specialty) ON UPDATE CASCADE`. The copies can no longer disagree, and one update propagates:

```
NOTICE:  refused: insert or update on table "visit_3nf" violates foreign key constraint
"visit_3nf_doctor_specialty_fkey"
 patient |    specialty    | doctor
---------+-----------------+---------
 Ali     | Cardiac surgery | Dr Osei
 Beth    | Cardiac surgery | Dr Osei
(2 rows)
```

The trap: "looks like BCNF, is really 2NF"

`enrol(student, course, instructor)` with `course -> instructor`, key `{student, course}`. It is tempting to call this a BCNF example, but `instructor` is **non-prime** and hangs off **half** of the key → a **2NF** violation; the table is not even in 2NF. `fd_tools.py`: `2NF violations: ['{course} -> {instructor}'] … highest normal form: 1NF`. **Rule of thumb:** a non-prime dependent whose determinant is a proper subset of a key signals a 2NF problem; only when every attribute involved is prime and a determinant still is not a superkey is it a genuine BCNF problem.

In the interview

30-second answer: "BCNF says every determinant must be a superkey. It's stricter than 3NF, which also allows X → A when A is prime. The two only differ for tables with several composite candidate keys that overlap. BCNF decompositions are always lossless but not always dependency-preserving, so sometimes I stay at 3NF and enforce the rule with a composite foreign key."

- *Example of 3NF-not-BCNF?* → Patient/specialty/doctor with doctor → specialty.
- *What's lost in the BCNF split?* → The FD {patient, specialty} → doctor can't be enforced by one table's key.

TL;DR

BCNF: every non-trivial FD's LHS is a superkey. Differs from 3NF only with overlapping composite candidate keys. Split on the violating FD (lossless), but you may lose dependency preservation.

11.12 Fourth normal form (4NF) — independent multivalued facts

What it is

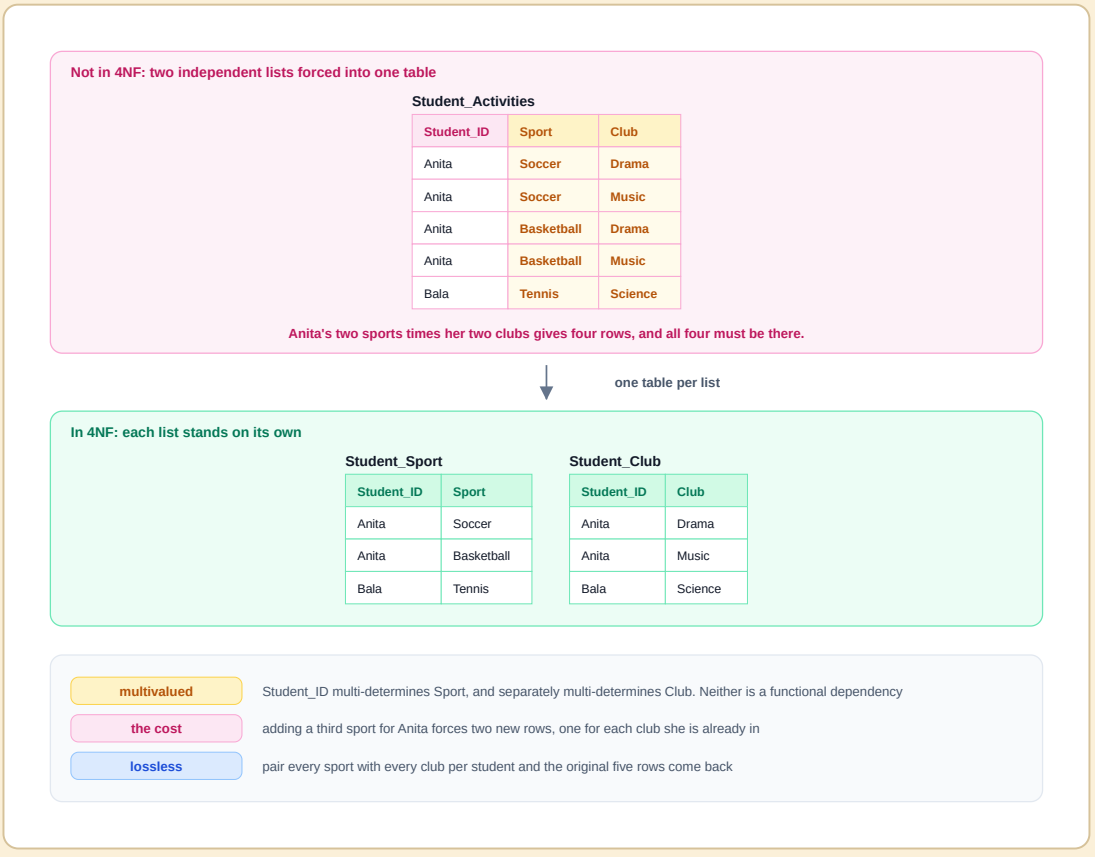
3NF and BCNF are about FDs (one value determines one value). **4NF** handles a different redundancy: two **independent lists** about the same thing stored in one table.

A restaurant has a set of dishes and a separately chosen set of delivery zones. Stored together, every dish must be paired with every zone:

restaurant	dish	zone
Saffron	Biryani	North
Saffron	Biryani	Centre
Saffron	Dal	North
Saffron	Dal	Centre
Nori	Ramen	East

Here `restaurant` does not functionally determine `dish` (Saffron has two). What holds is a **multivalued dependency (MVD)**, written `restaurant ->> dish`: for a given restaurant, the set of dishes is the same whatever zone you look at (and symmetrically `restaurant ->> zone` — MVDs come in complementary pairs). The table's only key is all three columns, so it is in BCNF — and still redundant.

Figure — two independent lists stored as a full cross product, and the 4NF split (course diagram, using sports and clubs)



The rule

A table is in **4NF** when it is in BCNF and **for every non-trivial MVD `X ->> Y`, X is a superkey**. In practice: **one table per independent list**.

The signature and the damage (`code/06_4nf_5nf.sql`)

The tell-tale sign is a **full cross product** per key value: 2 dishes × 2 zones = 4 rows. Adding one dish needs one new row *per zone*; forget one and the table claims that dish is not delivered somewhere:

```
--- restaurant ->> dish | zone: every dish is paired with every zone (a full cross product)
 restaurant | dishes | zones | rows
------------+--------+-------+------
 Nori       |      1 |     1 |    1
 Saffron    |      2 |     2 |    4
(2 rows)

--- adding one dish to Saffron needs one row PER ZONE; forget one and the table lies
 restaurant |     dish     | zones_listed
------------+--------------+--------------
 Saffron    | Biryani      |            2
 Saffron    | Dal          |            2
 Saffron    | Paneer tikka |            1
(3 rows)

--- 4NF split: one table per independent list; the join rebuilds the original exactly
 dish_rows | zone_rows | invented_rows
-----------+-----------+---------------
         3 |         3 |             0
(1 row)
```

Five rows became 3 + 3, and every new fact is now a single row. The split is lossless precisely because the lists were independent (an MVD `X ->> Y` is exactly the condition for splitting R into XY and X(R−Y) losslessly).

Caution: if the lists are *not* independent (a dish is only delivered to some zones), then the three-column table is the correct design and there is no MVD. As with FDs, the business rule decides, not the rows.

TL;DR

MVD `X ->> Y`: for each X, the set of Y values is independent of the other columns (visible as a full cross product). 4NF: every non-trivial MVD's LHS is a superkey → store each independent list in its own table.

11.13 Fifth normal form (5NF) — facts that need three tables

What it is

5NF (also **project-join normal form, PJ/NF**) handles a rarer case: a table with no FD or MVD problems that nonetheless can be split into **three (or more)** projections that join back exactly — while **no two** of them can. The constraint behind it is a **join dependency (JD)**.

Classic shape — a **cyclic business rule**: *if a supplier supplies a part, and the part is used in a project, and the supplier supplies that project, then the supplier supplies that part to that project.*

supplier	part	project
Acme	bolt	Bridge
Acme	bolt	Tower
Acme	nut	Tower
Brix	bolt	Tower

Why two tables are not enough

Take the three pairwise projections — supplier-part (`sp`), part-project (`pj`), supplier-project (`sj`). **Every** two-way join invents a false row; the three-way join does not (`code/06_4nf_5nf.sql`):

```
--- rows INVENTED by each two-way join (none of these is true)
 pieces | supplier | part | project
--------+----------+------+---------
 pj+sj  | Brix     | nut  | Tower
 sp+pj  | Brix     | bolt | Bridge
 sp+sj  | Acme     | nut  | Bridge
(3 rows)

--- the three-way join invents nothing: 5NF stores sp, pj and sj instead of spj
 invented_rows
---------------
             0
(1 row)
```

For example `sp+pj` claims Brix supplies bolts to Bridge; the third table `sj` rules it out, because Brix does not supply the Bridge project at all.

Figure — why two pairwise tables invent a row that the third rules out (course diagram)

The rule

A table is in **5NF** when every non-trivial join dependency on it is implied by its candidate keys. Practically: when a real cyclic rule links three things, keep the **three two-column tables** (here `sp`, `pj`, `sj`) and derive the combined view by a three-way join. Then inserting one pairwise fact automatically implies the right combinations.

How often 4NF and 5NF come up

Rarely. Independent lists usually get separate tables from the start (they are separate multivalued attributes in the ER diagram), and three-way rules are often simpler to enforce with a constraint or in code. Know them to **name** the problem precisely and to recognise the cross-product and invented-row signatures. Proof of correctness is the same as always: join the pieces and get exactly the original rows.

TL;DR

5NF (PJ/NF): no join dependency except those implied by keys. Signature: every two-way join of the projections invents rows but the three-way join doesn't → store the three pairwise tables.

11.14 Putting it together — a normalization procedure

Step by step for any table

1. **Write down the FDs** from business rules (not from sample rows). Split right sides to single attributes.
2. **Find all candidate keys** with closure (§11.4); mark prime / non-prime.
3. **1NF:** atomic cells, no repeating groups.
4. **2NF:** any non-prime attribute determined by part of a candidate key? → split per key part.
5. **3NF:** any FD `X -> A` with X not a superkey and A non-prime? → split the chain.
6. **BCNF (only if overlapping composite keys exist):** any FD with a non-superkey LHS? → split, and check what you lose.
7. **4NF/5NF (only if you see cross products / three-way rules).**
8. **Verify every split:** lossless (shared columns are a key of one side; `EXCEPT` both ways is empty) and ideally dependency-preserving; no column dropped.

Two textbook algorithms (both in `fd_tools.py`)

	3NF synthesis (Bernstein)	**BCNF decomposition**
Steps	1. compute a minimal cover · 2. one table per group of FDs with the same LHS · 3. if no table contains a candidate key, add one · 4. drop tables contained in others	while some table has an FD X → Y with X not a superkey *of that table*: replace it by X^+ (within the table) and (table − X^+) ∪ X
Lossless?	always	always
Dependency-preserving?	**always**	not always
Redundancy left?	possibly (only the 3NF-not-BCNF kind)	none from FDs

Run on Exercise 3's relation (R(A,B,C,D,E), `A -> B, BC -> D, D -> A, C -> E`):

```
BCNF decomposition: ['{A, B}', '{C, E}', '{A, D}', '{C, D}']
FDs lost by it: ['{B, C} -> {D}']
3NF synthesis: ['{A, B}', '{B, C, D}', '{A, D}', '{C, E}']
```

The BCNF result loses `BC -> D`; the 3NF result keeps it (in `{B, C, D}`) at the cost of some redundancy. This trade-off is the whole BCNF-vs-3NF story in one line.

TL;DR

FDs → keys → check 1NF, 2NF, 3NF (BCNF/4NF/5NF when their signatures appear) → split → verify lossless + dependency-preserving. 3NF synthesis always preserves dependencies; BCNF decomposition may not.

11.15 Exercises (with solutions)

Try each one on paper before looking at the solution — the real skill is noticing the dependency in a table you have never seen. Every answer below was checked with `fd_tools.py` (its output is quoted).

Exercise 1 — workshop sign-ups (target: 2NF, and beyond)

A conference records sign-ups in one table:

attendee_id	workshop_id	attendee_name	attendee_email	workshop_title	room	seat_no
A1	W1	Kofi	kofi@mail.example	Intro to SQL	R12	4
A1	W2	Kofi	kofi@mail.example	Indexing	R07	9
A2	W1	Mei	mei@mail.example	Intro to SQL	R12	5
A3	W3	Lars	lars@mail.example	Data modeling	R12	1

Rules: an attendee has one name and one email; **emails are unique per attendee**; a workshop has one title and one room; each attendee has one seat in each workshop they attend. Is the table in 2NF? Normalize it to 3NF.

Solution

FDs: `attendee_id -> attendee_name, attendee_email`; `attendee_email -> attendee_id`; `workshop_id -> workshop_title, room`; `{attendee_id, workshop_id} -> seat_no`.

Keys — the twist: because email identifies an attendee, there are **two** candidate keys:

```
candidate keys: {attendee_email, workshop_id}, {attendee_id, workshop_id}
prime: {attendee_email, attendee_id, workshop_id}  non-prime: {attendee_name, room, seat_no,
workshop_title}
2NF violations: ['{attendee_email} -> {attendee_name}', '{attendee_id} -> {attendee_name}',
'{workshop_id} -> {room, workshop_title}']
highest normal form: 1NF
```

`attendee_email` is **prime** (it is in a candidate key), so `attendee_id -> attendee_email` is not itself a 2NF problem — but `attendee_name`, `workshop_title` and `room` are non-prime and depend on halves of keys. **Not in 2NF.**

Decomposition (the 3NF synthesis agrees):

```
3NF synthesis: ['{attendee_email, attendee_id, attendee_name}', '{room, workshop_id, workshop_title}',
'{attendee_id, seat_no, workshop_id}']
```

- `attendee(attendee_id PK, attendee_name, attendee_email UNIQUE)` — both keys enforced.
- `workshop(workshop_id PK, workshop_title, room)`
- `signup(attendee_id FK, workshop_id FK, seat_no, PK (attendee_id, workshop_id))`

All three are in BCNF: every determinant in each is a key of that table. Lossless: `signup` shares `attendee_id` (key of `attendee`) and `workshop_id` (key of `workshop`). **Common wrong answer:** splitting off only `attendee` and leaving the workshop columns in the composite-key table — still a partial dependency on `workshop_id`. **Beyond the question:** is `(workshop_id, seat_no)` unique too (one person per seat)? If so, declare `UNIQUE (workshop_id, seat_no)` on `signup` — a rule no normal form would have told you to add.

Exercise 2 — flights (target: 3NF, and question the assumption)

flight_id	flight_date	tail_number	aircraft_type	seat_capacity
9001	2026-10-01	G-EZAA	A320	186
9002	2026-10-01	G-EZBB	A320	180
9003	2026-10-02	G-EZAA	A320	186
9004	2026-10-02	EI-FJK	B737-800	189

`flight_id` is the key. A flight is operated by one aircraft (identified by its tail number); each aircraft has one type and one seat capacity. Is the table in 2NF? In 3NF? Normalize it.

Solution

2NF: yes, automatically — the only candidate key `{flight_id}` has one column, so nothing can depend on "part" of it (and cells are atomic).

FDs: `flight_id -> flight_date, tail_number`; `tail_number -> aircraft_type, seat_capacity`. Chain: `flight_id -> tail_number -> aircraft_type`, and `tail_number` is not a key here (G-EZAA flies 9001 and 9003) → **transitive dependency → not 3NF**.

```
2NF violations: none
3NF violations: ['{tail_number} -> {aircraft_type}', '{tail_number} -> {seat_capacity}']
highest normal form: 2NF
BCNF decomposition: ['{aircraft_type, seat_capacity, tail_number}', '{flight_date, flight_id,
tail_number}']
```

Tables: `aircraft(tail_number PK, aircraft_type, seat_capacity)` and `flight(flight_id PK, flight_date, tail_number FK)`. The middle column `tail_number` stays in `flight` as the FK. Now a cabin refit on G-EZAA is one update, and a newly delivered aircraft can be recorded before its first flight.

Question the assumption. A tempting "further" step is `aircraft_type -> seat_capacity` and a third table `aircraft_type(type, capacity)`. The data refutes it: G-EZAA and G-EZBB are both A320s with 186 and 180 seats — capacity depends on each airframe's cabin layout, not on the type. One counter-pair kills the FD. Had the business told you "every A320 has 186 seats", the third table would be right; **a dependency is a claim about the world**, and when you state your assumptions in an interview, say which ones the answer depends on.

Exercise 3 — abstract relation (keys, closure, BCNF vs 3NF)

R(A, B, C, D, E) with F = { `A -> B`, `BC -> D`, `D -> A`, `C -> E` }. (a) Find all candidate keys. (b) Highest normal form? (c) Give a BCNF decomposition and say what it loses. (d) Give a 3NF decomposition.

Solution

(a) `C` never appears on a right side → in every key; `C⁺ = {C, E}`. Adding one attribute: `{A,C}⁺`, `{B,C}⁺`, `{C,D}⁺` are all of R; `{C,E}⁺ = {C,E}`. **Keys: AC, BC, CD** (any other superkey contains one of these).

(b) Prime = {A, B, C, D}; non-prime = {E}. `C -> E`: C is part of a key, E non-prime → partial dependency → **only 1NF**.

```
candidate keys: {A, C}, {B, C}, {C, D}
2NF violations: ['{C} -> {E}']
3NF violations: ['{C} -> {E}']
BCNF violations: ['{A} -> {B}', '{D} -> {A}', '{C} -> {E}']
highest normal form: 1NF
```

Note `A -> B` and `D -> A` are fine for 3NF (B and A are prime) but violate BCNF.

(c) BCNF: `{A,B}`, `{C,E}`, `{A,D}`, `{C,D}` — lossless, but `BC -> D` **is lost** (B and C, D are never together in one table).

(d) 3NF synthesis: `{A,B}`, `{B,C,D}`, `{A,D}`, `{C,E}` — `{B,C,D}` contains the key BC, so it is lossless, and every FD lives in one table.

Exercise 4 — quick-fire: keys and highest normal form

#	Relation and FDs
Q1	R(A,B,C): `A -> B`, `B -> C`
Q2	R(A,B,C): `AB -> C`, `C -> B`
Q3	R(A,B,C,D): `AB -> C`, `B -> D`
Q4	R(A,B,C): `A -> B`, `B -> A`, `A -> C`
Q5	Tutor(tutor, subject, language): a tutor's subjects and the languages they teach in are independent

Solution

```
Q1 R(A,B,C)  A->B, B->C          keys: {A}               highest NF: 2NF
Q2 R(A,B,C)  AB->C, C->B         keys: {A, B}, {A, C}    highest NF: 3NF
Q3 R(A,B,C,D)  AB->C, B->D       keys: {A, B}            highest NF: 1NF
Q4 R(A,B,C)  A->B, B->A, A->C    keys: {A}, {B}          highest NF: BCNF
```

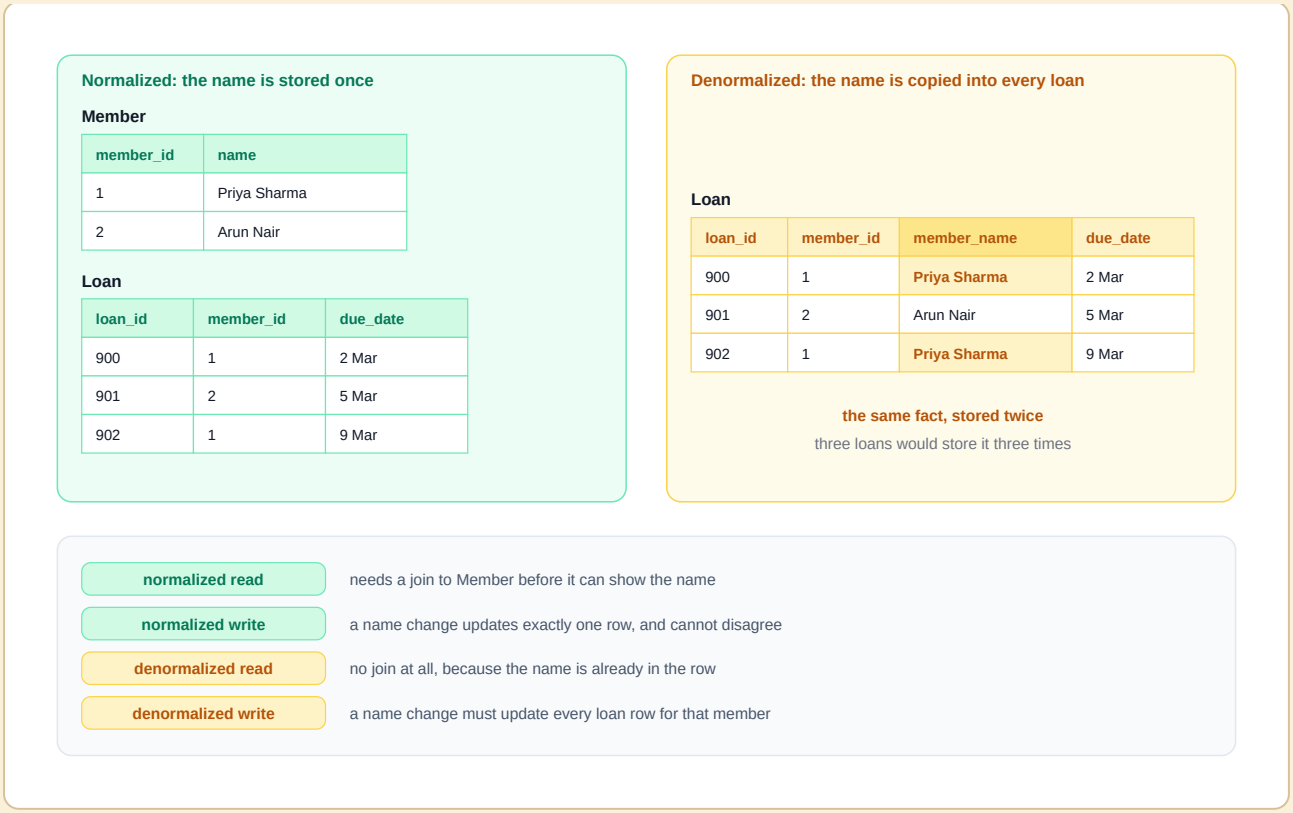
- Q1: transitive A → B → C (single key, so 2NF holds).
- Q2: the textbook 3NF-not-BCNF shape — `C -> B` with B prime, C not a superkey.
- Q3: `B -> D` is partial (B is part of key AB, D non-prime).
- Q4: A and B are both keys; every determinant is a key.
- Q5: no non-trivial FDs, key = all three columns → BCNF; but `tutor ->> subject` (and `->> language`) is a non-trivial MVD with a non-superkey LHS → **BCNF, not 4NF**. Split into `tutor_subject` and `tutor_language`.

11.16 Denormalization — when and how to break the rules

What it is

Denormalization is deliberately adding redundancy back into a normalized design to make important reads faster or simpler — accepting extra write work and a consistency risk. It is a conscious, measured trade, not "skipping normalization": you must know exactly which redundancy you add and how it stays correct.

Figure — the same data normalized and denormalized, with the cost of each (course diagram)



Why it matters — the trade

	Normalized	Denormalized
Each fact stored	once	more than once
Writes	touch one row; cannot disagree	touch several places; can drift if not maintained
Reads	may need joins / aggregation	fewer joins; precomputed answers
Storage	smaller	larger
Best for	OLTP (online transaction processing: many small writes)	read-heavy, reporting, analytics (OLAP, online analytical processing)

Common techniques

Technique	Example	Keeping it correct
Stored summary / counter column	`orders.item_count`, a post's `like_count`	trigger in the same transaction, or batch recompute
Materialized view (a query result saved as a table)	nightly sales-by-product report	`REFRESH MATERIALIZED VIEW` (stale — out of date — in between; `CONCURRENTLY`, which refreshes without blocking readers, needs a unique index)
Duplicated column to avoid a join	copying `customer_name` onto `orders` for a list page	trigger or application code on every change — the riskiest option
Pre-joined / wide tables	star-schema dimensions in a data warehouse (Kimball-style dimensional modeling)	rebuilt by ETL, not updated in place
Caches outside the DB	Redis copy of a product page	expiry / invalidation (see the system-design notes)

Key terms — star schema, ETL, cache: a **star schema** is the classic data-warehouse layout: one long, narrow *fact* table of events (sales) surrounded by wide *dimension* tables describing them (product, store, date) — Ralph Kimball popularised this "dimensional modeling". **ETL** (extract, transform, load) is the batch job that copies data out of the operational database, reshapes it and loads it into the warehouse. A **cache** is a fast copy of data kept closer to the reader (Redis is a popular in-memory one); *expiry* and *invalidation* are how stale copies get thrown away. More in System Design Fundamentals.

Running it (`code/07_denormalization.sql`)

```
CREATE TABLE orders (order_id text PRIMARY KEY, customer text NOT NULL,
                     item_count int NOT NULL DEFAULT 0);   -- DENORMALIZED: derivable from order_item
CREATE TABLE order_item (
  order_id   text REFERENCES orders ON DELETE CASCADE,
  product_id text REFERENCES product,
  qty        int  NOT NULL CHECK (qty > 0),
  price_paid numeric(8,2) NOT NULL,   -- NOT redundancy: the price at order time is its own fact
  PRIMARY KEY (order_id, product_id)
);
-- AFTER INSERT/UPDATE OF qty/DELETE trigger adjusts orders.item_count in the same transaction
CREATE MATERIALIZED VIEW sales_by_product AS
SELECT p.name, sum(oi.qty) AS units, sum(oi.qty * oi.price_paid) AS revenue
FROM order_item oi JOIN product p USING (product_id) GROUP BY p.name;
```

Output:

```
--- stored item_count vs the value recomputed from order_item
 order_id | stored | recomputed
----------+--------+------------
 O1       |      4 |          4
 O2       |      2 |          2
(2 rows)

--- the list price changes later; price_paid on old orders must not
 order_id |  name  | price_paid | todays_price
----------+--------+------------+--------------
 O1       | Kettle |      29.00 |        31.00
 O2       | Kettle |      27.00 |        31.00
(2 rows)

--- after a new sale, before REFRESH (stale copy)
  name  | units | revenue
--------+-------+---------
 Kettle |     3 |   83.00
 Mug    |     3 |   19.50
(2 rows)

--- after REFRESH
  name  | units | revenue
--------+-------+---------
 Kettle |     3 |   83.00
 Mug    |    13 |   84.50
(2 rows)
```

Two lessons: the trigger keeps the counter exact (but adds a write to every item change and a row lock on the order — other transactions changing that order must wait); the materialized view is cheap to read but **stale until refreshed**.

Not everything that looks redundant is. `price_paid` looks like a copy of `list_price`, but the FD is `{order_id, product_id} -> price_paid` — the price *on that day* — a different fact from today's list price. In the running example we assumed `product_id -> list_price`; in a real shop you would store both. Always ask: *is this the same fact, or a snapshot of it?*

When to denormalize — the rule

1. Normalize first (3NF).
2. Find a specific, important query that is too slow **on realistic data**, and check that indexes, query rewrites and caching did not fix it.
3. Add the smallest redundancy that fixes it, **with an enforced maintenance path** (trigger, materialized view refresh, ETL), and document which query it serves.

When not to denormalize

- Write-heavy tables (every write pays the maintenance).
- When the data must be exactly consistent at all times and you have no reliable maintenance mechanism.
- "Because joins are slow" in general — joins on indexed keys are what relational databases are built for (Chapter 12).

Real-world examples

- **Data warehouses** (Snowflake, BigQuery, Redshift — cloud warehouse services) routinely use denormalized star schemas: facts in a narrow table, wide dimension tables — because data is loaded in batches and read by many analytical queries.
- **Social feeds** keep counters (likes, followers) as stored columns rather than `COUNT(*)` on every page view.
- Wide-column and document stores (non-relational "NoSQL" databases; Chapter 16) are designed around query-shaped, denormalized data.

In the interview

> **30-second answer:** "I normalize to 3NF by default so every fact has one home. I denormalize only for a measured read bottleneck that indexing didn't solve — typically a counter column maintained by a trigger, a materialized view for reports, or a star schema for analytics — and I make sure the redundant copy has a single, enforced maintenance path."

- *Is storing the order total denormalization?* → If it is recomputable from current data, yes; if it freezes historical prices, it is its own fact.
- *Materialized view vs trigger-maintained table?* → MV: simple, periodically stale; trigger: always current, adds write cost and locking.

TL;DR

Denormalize = add controlled redundancy for reads, after normalizing and measuring. Use counters with triggers, materialized views or warehouse star schemas; every copy needs an enforced maintenance path. Snapshots (price paid) are facts, not redundancy.

Key terms

Term	Meaning
Functional dependency `X -> Y`	rows equal on X are equal on Y
Determinant / dependent	left / right side of an FD
Trivial / non-trivial FD	RHS ⊆ LHS / RHS adds a new attribute
Full / partial dependency	needs the whole composite determinant / a proper part of a key suffices
Transitive dependency	key → non-superkey → non-prime attribute
Multivalued dependency `X ->> Y`	for each X, the set of Y values is independent of the rest
Join dependency	the table equals the join of certain projections
F^+ / X^+	closure of an FD set / of an attribute set
Armstrong's axioms	reflexivity, augmentation, transitivity (sound and complete)
Union, decomposition, pseudotransitivity	derived inference rules
Minimal (canonical) cover	equivalent FD set with single RHS, no extraneous attributes, no redundant FDs
Prime / non-prime attribute	in some candidate key / in none
Insertion / update / deletion anomaly	can't add a fact alone / must change many rows / deleting loses an unrelated fact
1NF, 2NF, 3NF, BCNF, 4NF, 5NF	see §11.6 table
Lossless (non-additive) decomposition	joining pieces returns exactly the original rows
Spurious tuple	row invented by a lossy join
Dependency preservation	every FD enforceable within one piece
3NF synthesis / BCNF decomposition	the two standard normalization algorithms
Denormalization	deliberate, maintained redundancy for read performance
Materialized view	stored query result refreshed on demand

Chapter 11 TL;DR

- **FDs are the foundation.** `X -> Y` is a business rule; data can disprove it, never prove it. X is a superkey iff X^+ is everything.
- **Tools:** Armstrong's axioms (sound, complete), attribute closure (fire only fully-present LHS; loop), candidate keys (start with attributes never on a right side), minimal covers.
- **Anomalies** (insert/update/delete) come from storing a fact more than once; normalization decomposes tables so each fact lives in one place.
- **1NF** atomic cells · **2NF** no non-prime on part of a key · **3NF** every FD's LHS is a superkey or RHS is prime · **BCNF** every determinant is a superkey · **4NF** no independent lists (MVDs) · **5NF** no join dependencies beyond keys.
- **Every split must be lossless** (shared columns = key of one side; check with `EXCEPT` both ways) and ideally dependency-preserving. 3NF can always have both; BCNF sometimes loses an FD — which is why 3NF is the default stopping point.

- **Denormalize** only for a measured read problem, with an enforced way to keep copies correct; snapshots such as `price_paid` are separate facts, not redundancy.
- Next: Chapter 12 makes the joins that normalization creates fast; Chapter 17 applies Chapters 10–11 to full designs.

Chapter 12 — Indexes, Storage and Query Performance

Up to now we have asked *what* a query returns. This chapter asks *how fast*, and why. The answer always comes down to one question: **how many pages of data does the database have to touch?** To answer it you need to know how rows sit on disk, what an index really is, how the query planner (the part of the database that decides *how* to run a query) chooses between reading everything and jumping straight to the rows it needs, and which query habits quietly defeat an index. Every claim below is measured on a real PostgreSQL 14 server with a one-million-row table.

What you'll learn

- How a database stores rows: the storage hierarchy, 8 KB pages, slotted pages, heap files, TOAST, row vs column layout, and compression/encoding tricks.
- File organizations (heap, sorted, hashed, clustered) and why physical order matters.
- What an index is; primary, clustering and secondary indexes; dense vs sparse; single-level and multilevel indexes with block-count arithmetic.
- B-trees and B+ trees, hash indexes (static, extendible, linear hashing), Bloom filters (with a simulation), and PostgreSQL's GIN, GiST, BRIN and bloom indexes.
- Reading `EXPLAIN (ANALYZE, BUFFERS)` (PostgreSQL's command for showing how a query ran): seq scan, index scan, index-only scan, bitmap scan.
- Composite indexes and the leftmost-prefix rule, covering (`INCLUDE`), partial and expression indexes.
- Anti-patterns: functions on columns, leading wildcards (and the `pg_trgm` fix), `OR`, OFFSET paging (and keyset paging), N+1 queries.
- The three join algorithms and the statistics the planner relies on.

How to run the examples. SQL files are in `code/`; the runner (a helper script) builds a fresh database per file, loading `00_schema.sql` first (about 5 seconds, it generates one million orders):

```
bash tools_run_sql.sh sql '12-*'                                  # all files of this chapter
bash tools_pg.sh psql -d ch_12 -f sql/chapters/12-indexes-storage-and-query-performance/code/02_scans.sql
cd sql/chapters/12-indexes-storage-and-query-performance/code
python3 bloom_filter_sim.py; python3 compression_demo.py
../../../../.venv-books/bin/python n_plus_one.py                  # needs psycopg
```

> **About the numbers.** Timings (`actual time`, `Execution Time`, `exec_ms`) depend on the machine and on what is already cached; yours will differ. **Buffer counts** (pages touched) and **plan shapes** are the stable things to compare. Long plan lines are trimmed with `...`.

The data is a small shop. Every value is computed from the row number, so every run builds the same tables:

```
CREATE TABLE customers (id int PRIMARY KEY, name text, email text, country text, signup_on date); --
100,000 rows
CREATE TABLE orders (id bigint PRIMARY KEY, customer_id int REFERENCES customers(id),
                     status text, amount numeric(10,2), created_at timestamp, note text);
INSERT INTO orders
SELECT g, (g::bigint * 7919) % 100000 + 1,               -- exactly 10 orders per customer, scattered
       CASE WHEN g % 100 < 2 THEN 'pending' WHEN g % 100 < 5 THEN 'refunded'
            ELSE 'shipped' END,                          -- 2 % / 3 % / 95 %
       ((g::bigint * 37) % 50000) / 100.0 + 1,
       TIMESTAMP '2024-01-01' + g * INTERVAL '31 seconds', -- grows with id
       CASE WHEN g % 10 = 0 THEN 'gift wrap please' END   -- 90 % NULL
FROM generate_series(1, 1000000) AS g;
VACUUM ANALYZE orders;
```

`VACUUM ANALYZE` at the end cleans up the table and gathers the statistics the planner relies on (12.16). `psycopg`, needed by the last command, is the Python driver (library) for talking to PostgreSQL. The schema file also defines `plan_stats(query)`, which returns just the execution time and pages touched of a query, for side-by-side tables.

12.1 How a database stores data

What it is

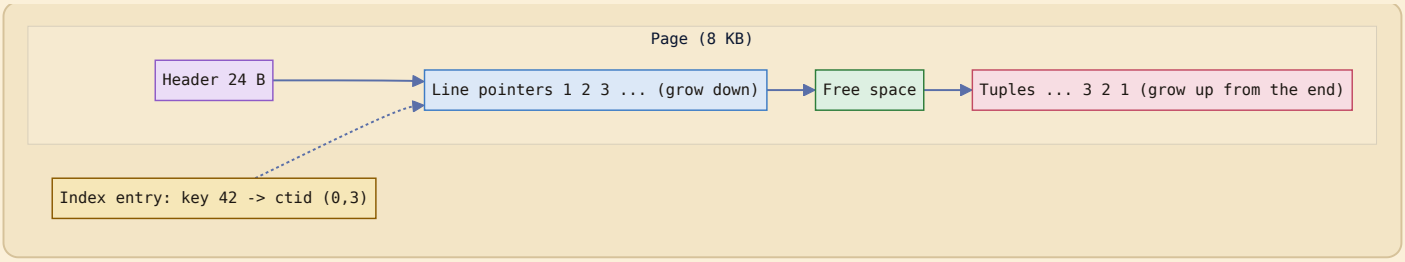
A database keeps data on **persistent storage** and works on it in **memory**. The layers differ by a factor of thousands in speed:

Level	Examples	Survives power loss?	Rough access time	Role in a DBMS
CPU cache / RAM (main memory; primary storage)	L1–L3 cache, DRAM	No (volatile — lost when power goes)	~1–100 ns	Buffer pool: cached pages, sort and hash memory
SSD / NVMe (secondary storage)	flash drives	Yes	~20–100 µs	Data files, indexes, write-ahead log (the change log written before the data files)
Hard disk (secondary)	spinning disks	Yes	~5–10 ms per random seek	Same, on older or cheaper servers
Tertiary / archive	tape, object storage (cloud file storage such as S3 Glacier)	Yes	seconds to hours	Backups, cold archives

Because the slow levels are so slow, a DBMS never fetches single rows from disk; it moves data in fixed-size **blocks** (also called **pages**): 8 KB in PostgreSQL and SQL Server, 16 KB in MySQL InnoDB, 4 KB by default in SQLite. The part of RAM that holds recently used pages is the **buffer pool** (PostgreSQL: `shared_buffers`). A read that finds its page there is a *hit*; one that must go to the operating system is a *read*. In `EXPLAIN (BUFFERS)` output you will see exactly those two words.

How a table is laid out

1. Each table is stored as one or more **files** (PostgreSQL: one file per 1 GB segment under `base/<database-oid>/`, where an *oid* is PostgreSQL's internal object number). A file is a sequence of pages numbered 0, 1, 2, …
2. Each page is a **slotted page**: a small header, an array of **line pointers** (slots) growing from the front, and the row data (**tuples**) growing from the back. Free space sits in the middle.
3. A row's physical address is (page number, slot number). PostgreSQL exposes it as the hidden column **`ctid`**. Indexes store these addresses. Because an index points at a *slot*, not a byte offset, the page can shuffle tuples around internally without breaking any index.
4. Large values are handled by **TOAST** (The Oversized-Attribute Storage Technique): when a row grows beyond about 2 KB, big values are compressed and, if still too big, cut into chunks stored in a separate TOAST table.



Seeing it on the real server

```
SELECT relname, pg_size_pretty(pg_relation_size(oid)) AS main_fork, ... FROM pg_class WHERE ...;
SELECT ctid, id, status, amount FROM orders WHERE id IN (1, 2, 3, 124, 125) ORDER BY id;
SELECT lower, upper, special, pagesize FROM page_header(get_raw_page('orders', 0));
SELECT lp, lp_off, lp_len, t_ctid FROM heap_page_items(get_raw_page('orders', 0)) LIMIT 4;
```

Output (from `01_storage_pages.sql`, which uses the `pageinspect` extension — an add-on module for looking at raw pages):

```
   relname   | main_fork | pages | est_rows
-------------+-----------+-------+----------
 customers   | 7344 kB   |   918 |   100000
 orders      | 68 MB     |  8662 |  1000000
 orders_pkey | 21 MB     |  2745 |  1000000

 ctid  | id  |  status  | amount
-------+-----+----------+--------
 (0,1) |   1 | pending  |   1.37
 (0,2) |   2 | refunded |   1.74
 (0,3) |   3 | refunded |   2.11
 (1,8) | 124 | shipped  |  46.88
 (1,9) | 125 | shipped  |  47.25

 lower | upper | special | pagesize
-------+-------+---------+----------
   488 |   504 |    8192 |     8192

 lp | lp_off | lp_len | t_ctid
----+--------+--------+--------
  1 |   8128 |     64 | (0,1)
  2 |   8064 |     64 | (0,2)
  3 |   8000 |     64 | (0,3)
  4 |   7936 |     64 | (0,4)
```

What this shows:

- One million orders occupy **8,662 pages**; a full scan must touch all of them. That number is the yardstick for everything that follows.
- Page 0 holds 116 rows (row 124 is the 8th row of page 1). Each tuple is 64 bytes: a 23-byte tuple header (transaction ids used by MVCC — *multi-version concurrency control*, PostgreSQL's way of keeping old row versions so readers and writers don't block each other; see chapter 13) plus the column values, aligned.
- `lower = 488` is where the slot array ends, `upper = 504` where tuple data begins: only 16 bytes free, so the page is full. Tuple 1 sits at the very end (offset 8128 + 64 = 8192) and later tuples are placed in front of it.

TOAST in action — three values of very different shape:

```
 id | logical_bytes | stored_bytes
----+---------------+--------------
  1 |            30 |           31      -- tiny: stored inline, 1-byte length header
  2 |         40000 |          477      -- 'abcdefgh' x 5000: compressed ~84x
  3 |         48000 |        48000      -- hex digests: incompressible, moved out of line as is
```

(Hex digests are hash outputs written in hexadecimal — effectively random text, so they don't compress.)

`pg_column_size` reports the bytes actually stored. PostgreSQL keeps a value uncompressed when compression would not save enough.

From memory to bytes: serialization, endianness, alignment

- **Serialization** turns an in-memory value (a row, a JSON document) into a byte sequence that can be written to a page or sent over the network; **deserialization** rebuilds it. Each engine has its own format: PostgreSQL's tuple layout, MongoDB's BSON (a binary form of JSON), Parquet's (a column-oriented file format) column chunks. Compact formats mean fewer pages.
- **Endianness** is the byte order of multi-byte numbers: *big-endian* writes the most significant byte first, *little-endian* the least significant first (x86 and ARM machines are little-endian). The integer 258 is `01 02` big-endian and `02 01` little-endian. A file format or network protocol must fix one order, or data written on one machine is misread on another — why PostgreSQL's binary files are not portable across architectures while `pg_dump` output is.
- **Alignment**: CPUs read a 4- or 8-byte value fastest at an address that is a multiple of its size, so engines insert padding. In PostgreSQL, column order therefore affects row size: `(bool, bigint, bool, bigint)` pads each `bool` to 8 bytes, while `(bigint, bigint, bool, bool)` does not. Index pages are laid out on aligned boundaries for the same reason.

Row stores vs column stores

Layout	How a page is filled	Good at	Examples
Row-oriented	All columns of a row together	OLTP (online transaction processing — many small reads and writes of single rows): fetch or change whole rows by key	PostgreSQL, MySQL InnoDB, SQL Server (rowstore), SQLite
Column-oriented	All values of *one column* together	Analytics: scan a few columns of billions of rows; compresses very well	ClickHouse, Amazon Redshift, Snowflake, DuckDB, Parquet files, SQL Server columnstore

Summing `amount` in a row store reads every whole row; a column store reads only the `amount` column, which also compresses well. More in DDIA (the book *Designing Data-Intensive Applications*): storage and retrieval.

Self-check

1. Why do databases read whole pages instead of single rows? → Storage transfers blocks anyway, and a random access costs far more than the extra bytes; grouping rows per page amortizes the cost (spreads it over many rows).
2. What is a `ctid`? → The (page, slot) address of a row version in PostgreSQL; indexes store it.

12.2 Data compression and encoding

What it is

Compression stores the same information in fewer bytes. It is **lossless** when the original can be rebuilt exactly (always the case for database data) and **lossy** when some detail is thrown away (JPEG, MP3 — never

used for table values). **Encoding** is compression that exploits the *type and shape* of a column rather than treating it as a byte stream.

Why it matters

Fewer bytes means fewer pages to read, more rows fitting in the buffer pool, and cheaper storage and network transfer. The price is CPU time to compress and decompress, and sometimes slower in-place updates.

The common techniques

Technique	Idea	Works best on	Used by
Run-length encoding (RLE)	Store (value, how many times in a row)	Sorted or slowly changing columns	Column stores, Parquet, bitmap indexes
Dictionary encoding	Store each distinct value once; rows hold a small integer code	Few distinct values (country, status)	Parquet, ClickHouse `LowCardinality`, Redshift
Delta encoding	Store the first value, then differences	Timestamps, ids, sorted numbers	Time-series DBs (databases of timestamped measurements: Gorilla/Prometheus), Parquet
Bit-packing	Use only as many bits as the largest value needs	Small integers (codes, deltas)	Combined with the two above
Lempel–Ziv (LZ77/LZ78 family)	Replace a repeated byte sequence with a reference ("copy 8 bytes from 40 bytes back")	Text, repeated strings; no type knowledge	PostgreSQL TOAST (`pglz`, `lz4` since PG 14), zlib/zstd (widely used compression libraries) page compression
Huffman coding	Give frequent symbols short bit codes and rare ones long codes	Skewed symbol frequencies	Inside zlib/DEFLATE and zstd

`compression_demo.py` applies each technique to a column of 100,000 values:

```
status  = ["shipped"] * 60_000 + ["pending"] * 25_000 + ["refunded"] * 15_000   # sorted
country = [["IN", "US", "DE", "BR", "JP"][(i * 7) % 5] for i in range(N)]      # 5 distinct
ts      = [1_700_000_000 + i * 31 + (i % 7) for i in range(N)]                  # increasing
# rle(), dictionary(), delta() are ~6 lines each; see the file
```

Output:

```
status   raw text               :   715,000 bytes
status   RLE (3 runs)           :        34 bytes  -> [['shipped', 60000], ['pending', 25000],
['refunded', 15000]]
country  raw text               :   200,000 bytes
country  dictionary + 3-bit codes:    37,510 bytes  (dictionary ['IN', 'DE', 'JP', 'US', 'BR'], codes [0,
1, 2, 3, 4, 0]...)
ts       raw 8-byte ints        :   800,000 bytes
ts       delta + 6-bit packing  :    75,007 bytes  (first 1700000000, deltas [32, 32, 32, 32, 32, 32]...)
ts       zlib on the text       :   273,868 bytes  (no knowledge of the data type)
```

Type-aware encodings beat the general-purpose compressor by a wide margin and can often be queried without decompressing (counting `'pending'` rows is reading one run length) — why column stores sort data and choose an encoding per column.

Trade-offs and when not to use it

- Compression costs CPU on every write and read; for hot (frequently accessed), small OLTP rows it usually is not worth it (PostgreSQL compresses only when a row exceeds ~2 KB).

12.3 File organization

What it is

File organization is the rule that decides *where* in the data file a new record goes, and therefore how records can be found later.

Organization	Where records go	Fast	Slow	Real systems
Heap (unordered)	Any page with free space (usually the end)	Inserts; full scans	Finding one record without an index (scan everything)	PostgreSQL tables, SQL Server heaps, Oracle default
Sequential (sorted)	Kept in order of a key	Range reads and ordered output on that key; binary search (repeatedly halving the search range)	Inserts/deletes in the middle (shift records or use overflow pages, reorganize periodically)	Old ISAM files, sorted SSTables in LSM engines
Hashed	Page = hash(key) mod number of buckets	Equality lookups on the key: ~1 page read	Ranges and ordering; growing beyond the bucket count	Hash clusters (Oracle), hash-partitioned stores
ISAM (indexed sequential)	Sorted data file plus a static multilevel index built at load time; later inserts go to overflow pages	Reads and ranges on a stable file	Overflow chains grow until the file is reorganized	IBM mainframe ISAM files
B+ tree file organization	Records live in the leaves of a B+ tree on the key (12.6); the tree rebalances itself on insert	Point and range access that stays fast under inserts	Some space kept free in every leaf; splits	Same as the next row
Clustered / index-organized	The table *is* a B+ tree ordered by a key; leaf pages hold the rows	Primary-key lookups and ranges	Inserts in random key order cause page splits; secondary indexes must store the key	MySQL InnoDB, SQL Server clustered index, Oracle IOT (index-organized table), SQLite rowid / `WITHOUT ROWID` tables

Key terms — hashing and LSM engines: a **hash function** turns a key into a seemingly random number; `mod N` (the remainder after dividing by N) then picks one of N **buckets**. **ISAM** (indexed sequential access method) is IBM's classic sorted-file-plus-index design. An **LSM** (log-structured merge) engine buffers writes in memory and flushes them as immutable sorted files called **SSTables** (sorted string tables), merging them in the background — fast writes, at the cost of reads checking several files.

A hashed file with `bucket = key mod 5` puts keys 21, 32 and 43 in buckets 1, 2 and 3; key 36 also lands in bucket 1 — a **collision**, resolved by chaining it next to 21 (12.7). A clustered file with a department key stores all employees of department 7 in the same few blocks, so "list department 7" reads only those blocks.

PostgreSQL has only heap tables; `CLUSTER` sorts a heap **once** by an index, but new rows are not kept in order afterwards.

Why physical order matters — measured

Each customer's 10 orders are scattered through the table (the generator jumps by 7,919 customers per row). Summing the orders of 1,000 customers (10,000 rows) — before and after `CLUSTER` (`09_clustering.sql`):

```
SELECT correlation FROM pg_stats WHERE tablename = 'orders' AND attname = 'customer_id';
EXPLAIN (ANALYZE, BUFFERS, COSTS OFF)
SELECT sum(amount) FROM orders WHERE customer_id BETWEEN 1000 AND 1999;
CLUSTER orders USING orders_customer_idx;  ANALYZE orders;
-- ... same two queries again
```

```
 correlation = 0.0034                                        -- before: random order (sampled; varies
per run)
 Aggregate (actual time=5.814..5.815 rows=1 loops=1)
   Buffers: shared hit=8454 read=14
   ->  Bitmap Heap Scan on orders (... rows=10000 loops=1)
         Heap Blocks: exact=8454                             -- 10,000 rows on 8,454 pages!
 ...
 correlation = 1                                             -- after CLUSTER
 Aggregate (actual time=0.895..0.895 rows=1 loops=1)
   Buffers: shared hit=2 read=100
   ->  Index Scan using orders_customer_idx on orders (... rows=10000 loops=1)
 Execution Time: 0.901 ms
```

Same rows, same index: **8,468 pages → 102 pages**, because after clustering the 10,000 rows sit next to each other. The statistic **correlation** (−1 … 1) tells the planner how well a column's order matches the physical order.

A related knob is **fillfactor**: how full to pack each page when inserting, leaving room so that updated row versions can stay on the same page (PostgreSQL's heap-only-tuple — HOT — updates, which skip index maintenance, chapter 13). The same 100,000 rows take 443 pages at the default `fillfactor = 100` and 633 pages at `fillfactor = 70`.

Trade-offs

- Only **one** physical order per table — pick the key of your most important range queries (often time or tenant id — the customer organisation a row belongs to in a multi-tenant app).
- In InnoDB and SQL Server the clustered key is copied into every secondary index, so a wide or random key (a random UUID — a 128-bit randomly generated id) bloats every index and causes page splits; prefer a short, increasing key.

12.4 What an index is

What it is

An **index** is a separate, smaller data structure that maps **search-key values to row addresses**, kept sorted (or hashed) so that the database can find matching rows without reading the whole table. Think of the index at the back of a textbook: sorted terms with page numbers. You look up "B+ tree", jump to page 412, and never read pages 1–411.

- **Search key**: the column(s) the index is built on (not necessarily a primary or unique key).
- **Index entry**: (key value, pointer). The pointer is a `ctid` in PostgreSQL, the primary-key value in InnoDB secondary indexes, a row id in Oracle.

The problem it solves

Without an index, `WHERE customer_id = 4242` must check every row. With one, the database walks a handful of index pages and fetches only the matching rows:

```
-- no index
 Seq Scan on orders (actual time=2.944..30.041 rows=10 loops=1)
   Filter: (customer_id = 4242)
   Rows Removed by Filter: 999990
   Buffers: shared hit=8662
 Execution Time: 30.068 ms
-- CREATE INDEX orders_customer_idx ON orders (customer_id);
 Bitmap Heap Scan on orders (actual time=0.015..0.023 rows=10 loops=1)
   Buffers: shared hit=10 read=3
   ->  Bitmap Index Scan on orders_customer_idx (... rows=10 loops=1)
 Execution Time: 0.036 ms
```

8,662 pages → 13 pages; ~30 ms → ~0.04 ms. That is the whole reason indexes exist.

Kinds of index (classic terminology)

Term	Meaning
Primary index	Built on the ordering key of a *sorted* file, usually the primary key; one entry per data block is enough (sparse).
Clustering index	Built on a sorting field that is *not* unique (e.g. file sorted by department); one entry per distinct value, pointing to its first block.
Secondary index	Built on any other column; the file is not sorted by it, so it needs one entry per record (dense). A table can have many.
Dense index	An entry for **every** search-key value (every record).
Sparse index	Entries for only **some** values — typically the first key of each block; works only if the file is sorted on that key.
Clustered vs non-clustered (vendor usage)	*Clustered*: the table rows are stored in index order (at most one per table — SQL Server, InnoDB PK). *Non-clustered*: a separate structure pointing to rows. PostgreSQL indexes are all non-clustered.
Unique index	Rejects duplicate keys; how `PRIMARY KEY` and `UNIQUE` constraints are enforced.

The SQL

```
CREATE INDEX orders_customer_idx ON orders (customer_id);              -- B-tree by default
CREATE UNIQUE INDEX customers_email_uq ON customers (email);          -- enforces uniqueness
CREATE INDEX CONCURRENTLY orders_note_idx ON orders (note);           -- PG: build without blocking
writes
DROP INDEX orders_customer_idx;                          -- MySQL / SQL Server: DROP INDEX name ON table
ALTER TABLE t ADD CONSTRAINT t_pk PRIMARY KEY (id);      -- creates a unique index implicitly
```

SQL Server adds `CREATE [CLUSTERED|NONCLUSTERED] INDEX`. `CONCURRENTLY` (PostgreSQL) and `ONLINE = ON` (SQL Server Enterprise) / `ALGORITHM=INPLACE, LOCK=NONE` (MySQL) avoid long write locks (locks that stop other sessions from changing the table) on big tables.

Trade-offs

Benefit	Cost
Fast lookups, ranges, joins, `ORDER BY`, `MIN/MAX`	Every `INSERT`, `DELETE` and many `UPDATE`s must also change every index
Enforces uniqueness	Disk and buffer-pool space (our 1M-row PK index is 21 MB)
Enables index-only scans	Too many indexes slow writes and make the planner's job harder

When not to index

- Small tables (a few pages): a sequential scan is already cheap.
- Columns that most rows share (low selectivity): `status = 'shipped'` matches 95 % of rows — see 12.10.
- Write-heavy tables whose columns are rarely searched.
- Columns only ever used inside functions (index the expression instead — 12.12).

Common mistakes

- "Index every column." Each one costs writes; unused indexes are pure overhead (find them with `pg_stat_user_indexes.idx_scan = 0` — a statistics view that counts how often each index is used).
- Forgetting that **foreign-key columns are not indexed automatically** in PostgreSQL (MySQL InnoDB does create one). Without an index, joins on the FK and `ON DELETE` checks on the parent scan the child table.

In the interview

"An index is a sorted (or hashed) side structure mapping key values to row locations, so a lookup reads a few pages instead of the whole table; it speeds up filters, joins and sorts at the cost of slower writes and extra space. I index selective predicates (`WHERE` conditions that match few rows) and join keys and confirm with EXPLAIN."

- *Why not index everything?* → Write amplification (one logical change causing many physical writes), space, and cache pressure (indexes competing for limited memory); low-selectivity indexes are ignored anyway.
- *Clustered vs non-clustered?* → Clustered stores the rows themselves in key order (one per table); non-clustered is a separate structure with pointers.

12.5 Single-level indexing, with numbers

How it works

An index file is much smaller than the data file because each entry holds only a key and a pointer. Searching it with binary search costs about $\log_2$(index blocks) reads, plus one read for the data block. The arithmetic (from *Database System Concepts* and *Fundamentals of Database Systems*):

- **Blocking factor** `bfr = ⌊block size / record size⌋` — records per block. (`⌊x⌋` rounds down, `⌈x⌉` rounds up.)
- **Blocks** `b = ⌈records / bfr⌉`.
- **Binary search** on `b` sorted blocks: `⌈log₂ b⌉` block reads.

Worked example (own numbers)

A file of **r = 1,000,000** records, each **200 bytes**, block size **4,096 bytes**. Index entries are 8-byte key + 8-byte pointer = **16 bytes**.

Access path	Calculation	Block reads
Data file	bfr = ⌊4096/200⌋ = 20; b = 1,000,000/20 = **50,000 blocks**	—
Linear scan, unordered file	average b/2, worst b	25,000 avg / 50,000 worst
Binary search, file sorted on the key	$\lceil \log_2 50{,}000 \rceil = 16$	**16**
Primary index (sparse): one entry per data block	50,000 entries; index bfr = ⌊4096/16⌋ = 256; 50,000/256 → **196 blocks**; $\lceil \log_2 196 \rceil = 8$, +1 data block	**9**
Secondary index (dense) on an unsorted column: one entry per record	1,000,000/256 → **3,907 blocks**; $\lceil \log_2 3907 \rceil = 12$, +1	**13**
Multilevel (index on the index, fan-out — entries per index block — 256)	level 1: 3,907 blocks → level 2: ⌈3907/256⌉ = 16 → level 3: 1 block; 3 levels + 1 data block	**4**

Two lessons: a sparse index is much smaller than a dense one but requires a sorted file; and building an index **on top of the index** — each level 256× smaller — turns a $\log_2$ search into a $\log_{256}$ search. The top level is small enough to stay in RAM, so in practice only the lower levels and the data block cost disk reads. That idea, made dynamic, is the B+ tree. (Courses often call primary, clustering and secondary indexes *single-level* and B-/B+ trees *multi-level* indexes.)

Trade-offs of single-level indexes

- Sparse: small, but only possible for the file's sort key, and inserts must keep the file sorted.
- Dense: works for any column, but large; still needs binary search over thousands of blocks.
- Both are **static**: inserting into a sorted index means shifting entries. Multilevel trees with free space in each node fix that.

Self-check

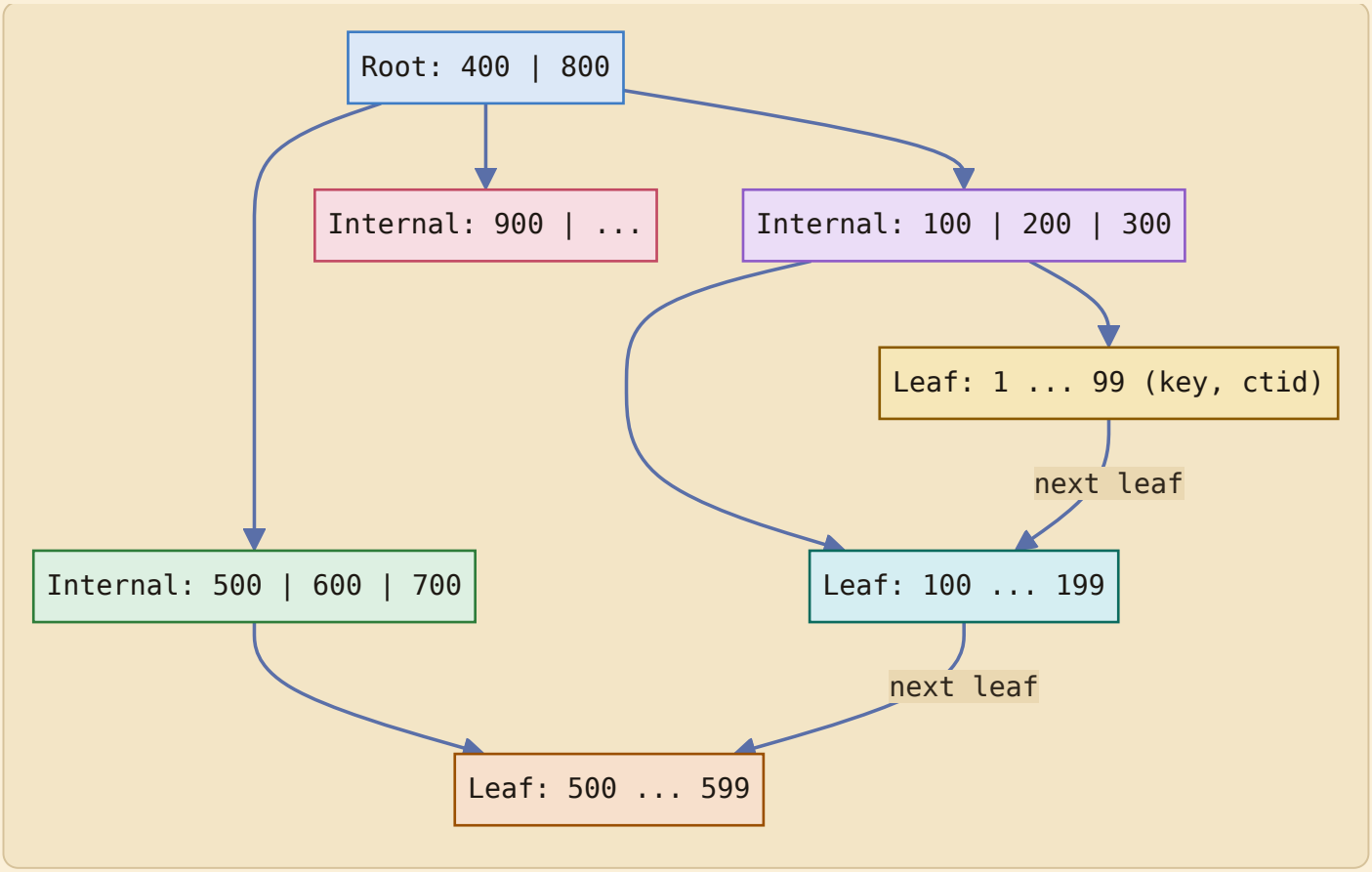
1. Why can a secondary index not be sparse? → The file is not sorted on that column, so the first key of a block says nothing about the others.
2. With fan-out 256, how many index levels for 16 million entries? → $256^3 \approx 16.7$ million, so 3 levels.

12.6 Tree-based indexes: B-trees and B+ trees

What it is

A **B-tree** (Bayer & McCreight, 1972) is a balanced search tree whose nodes are whole disk pages, each holding hundreds of sorted keys. "Balanced" means every leaf is at the same depth, so every lookup costs the same number of page reads. The **B+ tree** — what nearly every database actually uses (PostgreSQL's "btree", InnoDB, SQL Server, SQLite, Oracle) — adds two rules:

1. **All data pointers live in the leaves.** Internal nodes hold only separator keys and child pointers, so more of them fit per page (higher **fan-out**).
2. **Leaves are linked** left-to-right (PostgreSQL: both ways), so a range scan finds the first key and then just walks sideways.



Why a tree at all? In big-O notation (how work grows with the number of entries n), an unsorted array needs O(n) comparisons per search; a sorted array allows O(log n) binary search but each insert must shift up to n entries. A B-tree keeps keys sorted *and* absorbs inserts locally, with O(log n) search, insert and delete.

How it works

- **Search** (e.g. `id = 500000`): read the root, pick the child whose key range contains 500,000, repeat until a leaf, then follow the pointer to the heap (the table's unordered data file). Cost = tree height + 1. In the figure, looking for 650 goes root (400 < 650 < 800 → middle child) → internal (600 < 650 < 700) → the leaf holding 600–699.
- **Range** (`BETWEEN 1000 AND 1999`): search for 1000, then read leaves left to right until a key > 1999.
- **Insert**: find the leaf; if it has room, insert in sorted position. If it is full, **split** it into two half-full leaves and push the middle key up into the parent; if the parent overflows it splits too. Only a root split makes the tree taller — which is why the tree stays balanced.
- **Delete**: remove the entry; if a node becomes too empty, borrow from a sibling or merge. (PostgreSQL does not merge eagerly; it reclaims empty pages via `VACUUM`.)
- **Order** `n` of a tree: each node holds up to `n − 1` keys and `n` children, and every non-root node stays at least half full.

Seeing it on the server

```
SELECT level AS root_level FROM bt_metap('orders_pkey');
SELECT type, live_items, avg_item_size, free_size FROM bt_page_stats('orders_pkey', 1);
```

```
 root_level = 2                  -- levels 2 (root), 1 (internal), 0 (leaves): height 3
 type | live_items | avg_item_size | free_size
------+------------+---------------+-----------
 l    |        367 |            16 |       808
```

A leaf page holds 367 entries of 16 bytes and keeps ~10 % free for future inserts (the B-tree default fillfactor is 90). One million keys need about 2,700 leaves, and just one internal level to point to them. That is why the primary-key lookup in 12.10 touched only **7 buffers** — the three tree levels, the heap page and a few index-metadata accesses — against 8,662 for a full scan. A billion keys would need only one more level.

B-tree vs B+ tree vs others

Structure	Data pointers in	Range scans	Used by
B-tree (classic)	Every node	Needs in-order tree walk	Rare today; some file systems
B+ tree	Leaves only; leaves linked	Walk the leaf chain	PostgreSQL, MySQL, SQL Server, Oracle, SQLite
LSM tree	Sorted immutable files merged in the background	Merge several files	RocksDB, Cassandra, ScyllaDB, LevelDB (write-optimized storage engines and NoSQL databases; see ch. 16)
R-tree	Bounding boxes	Spatial "overlaps/near"	PostGIS (PostgreSQL's maps-and-geometry extension, via GiST), SQLite R*Tree

Trade-offs

- Random-key inserts (random UUIDs) hit random leaves: more page splits and more cache misses (needed pages not already in memory) than an increasing key.
- PostgreSQL 13+ **deduplicates** repeated keys in leaves, so a B-tree on a low-uniqueness column is much smaller (our `customer_id` index: 9 MB vs 21 MB for unique `created_at`).

In the interview

> **30-second answer:** "Databases use B+ trees: balanced trees whose nodes are pages with hundreds of keys, so even a billion rows need only 3–4 levels. Values sit in linked leaves, so range scans walk the leaves. Inserts split full nodes and push keys up, keeping every leaf at the same depth."

- *B-tree vs LSM?* → B+ trees update pages in place (read-optimized); LSM trees append and merge (write-optimized, need Bloom filters for reads).

Self-check

1. Why do B+ tree internal nodes hold no row pointers? → To fit more separator keys per page, raising fan-out and lowering height.
2. When does a B+ tree get taller? → Only when the root splits.

12.7 Hash-based indexing

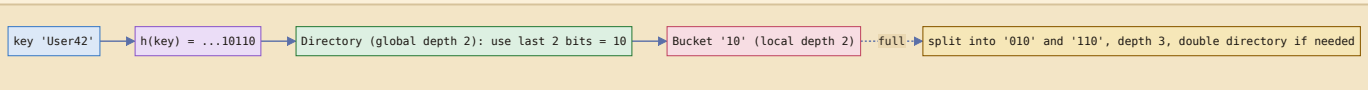
What it is

A **hash index** computes `hash(key)` and uses it to pick a **bucket** (a page or chain of pages) that holds the entries for that key. A lookup reads one bucket — typically one or two pages — no matter how large the table.

How it works

1. **Hash function**: maps a key to an integer spread evenly across buckets (`bucket = h(key) mod N`). A good one scatters similar keys (`User1`, `User2`) far apart.

2. **Collisions**: different keys can land in the same bucket. Databases put them in the same bucket page; when it fills, an **overflow page** is chained on. (In-memory hash tables also use *open addressing*: probe the next slots.)
3. **Static hashing** fixes `N` up front. Example with `h(k) = k mod 7`: keys 15, 23, 40 go to buckets 1, 2 and 5; inserting 29 (29 mod 7 = 1) collides with 15 and is **chained** in bucket 1, so a search for 29 hashes to bucket 1 and walks its short chain. If the data grows, chains get long; if it shrinks, space is wasted. Rehashing everything into a bigger table is expensive.
4. **Dynamic hashing** grows gradually:
   - **Extendible hashing** keeps a **directory** of `2^d` pointers indexed by the last `d` bits of the hash (**global depth**). Each bucket records how many bits it really uses (**local depth**). When a bucket overflows it splits in two using one more bit; only if its local depth already equals the global depth does the directory double (pointers copied, no data moved).
   - **Linear hashing** has no directory: buckets are split one at a time in a fixed round-robin order (a "next to split" pointer), using `h mod 2N` for already-split buckets. Growth is smooth; overflow chains are temporary. PostgreSQL's hash index (based on Seltzer and Yigit's 1991 hashing package) likewise has no directory: it splits one bucket at a time and allocates bucket pages in power-of-2 groups whose location is computed from the bucket number.



On the server

```
CREATE INDEX o_cust_hash ON orders USING hash (customer_id);
EXPLAIN (COSTS OFF) SELECT * FROM orders WHERE customer_id = 42;   -- Bitmap Index Scan on o_cust_hash
EXPLAIN (COSTS OFF) SELECT * FROM orders WHERE customer_id < 42;   -- Seq Scan: hash can't do ranges
```

```
   index_name    | type  |  size
-----------------+-------+---------
 o_cust_hash     | hash  | 32 MB
 o_created_btree | btree | 21 MB
 o_cust_btree    | btree | 9256 kB
 o_created_brin  | brin  | 24 kB
```

Trade-offs

	Hash index	B-tree
`=` lookups	O(1) buckets	O(log n) pages (3–4 in practice)
Ranges, `ORDER BY`, prefix `LIKE`	No	Yes
Unique constraints	No (PostgreSQL)	Yes
Size	Can be larger (no deduplication — 32 MB vs 9 MB above)	Compact with dedup

In practice the B-tree's extra 2–3 page reads are usually cached, so hash indexes win only for very large, equality-only keys (long strings, URLs). PostgreSQL hash indexes are crash-safe (intact after a server crash) only since version 10 — old advice to avoid them dates from before that. MySQL InnoDB has no user-created hash indexes (its *adaptive hash index* is built automatically in memory); the MEMORY engine does. Hashing is also the heart of **hash joins** (12.15) and **hash partitioning** (ch. 15).

Self-check

1. Why can't a hash index answer `ORDER BY customer_id`? → Hashing destroys order: neighbouring keys land in unrelated buckets.

12.8 Bloom filters

What it is

A **Bloom filter** (Burton Bloom, 1970) is a compact, probabilistic set: it answers "is x in the set?" with either **"definitely not"** or **"probably yes"**. It never gives a false negative; it gives false positives at a rate you choose by spending more bits. Analogy: a bouncer with a smudged guest list who never turns away a real guest but occasionally lets in a stranger.

Why it matters

Checking a filter in RAM costs nanoseconds; checking disk costs micro- to milliseconds. If most lookups are for keys that are *not* there, the filter skips almost all of that disk work.

How it works

1. Start with an array of **m bits**, all 0, and choose **k** independent hash functions.
2. **Add(x)**: compute the k hashes, set those k bits to 1.
3. **Query(x)**: compute the k hashes; if **any** bit is 0, x was never added (definitely absent). If all are 1, x is *probably* present — those bits may have been set by other keys.
4. With n keys inserted, the false-positive probability is approximately **$p \approx (1 - e^{-kn/m})^k$**, minimized at **$k = (m/n)\cdot \ln 2$**; then about **$1.44 \cdot \log_2(1/p)$ bits per key** are needed (≈ 9.6 bits for 1 %).
5. You cannot delete (clearing a bit could erase other keys). **Counting Bloom filters** keep a small counter per position instead of a bit to allow deletes; **scalable Bloom filters** add new, larger filters as n grows.

A tiny example (10 bits, k = 3; hash outputs made up for illustration). Insert `"sql"` → positions 2, 5, 8 set. Insert `"java"` → positions 1, 5, 9 set (5 was already 1). Bits now: `0110010011` (positions 1, 2, 5, 8, 9). Query `"css"` → positions 2, 6, 9: bit 6 is 0, so **definitely absent**. Query `"go"` → positions 1, 2, 8: all 1 though `"go"` was never added — a **false positive**. A *false positive* is "maybe present" for an absent key; a *false negative* ("absent" for a present key) cannot happen. The work per operation depends on k, not on m.

Simulation vs theory

`bloom_filter_sim.py` builds real filters (SHA-256 — a standard cryptographic hash function — with double hashing), inserts 10,000 keys and probes 50,000 keys that were never added:

```
class BloomFilter:
    def __init__(self, m_bits, k_hashes):
        self.m, self.k = m_bits, k_hashes
        self.bits = bytearray((m_bits + 7) // 8)
    def _positions(self, key):                          # h_i = h1 + i*h2 (mod m)
        d = hashlib.sha256(key.encode()).digest()
        h1, h2 = int.from_bytes(d[:8], "big"), int.from_bytes(d[8:16], "big") | 1
        return [(h1 + i * h2) % self.m for i in range(self.k)]
    def add(self, key):
        for p in self._positions(key): self.bits[p // 8] |= 1 << (p % 8)
    def might_contain(self, key):
        return all(self.bits[p // 8] >> (p % 8) & 1 for p in self._positions(key))
```

Output:

```
bits/key   m bits   k  measured FP  theory FP    memory
       4    40000   3     14.6340%   14.6892%     4.9KB
       8    80000   6      2.1700%    2.1577%     9.8KB
      10   100000   7      0.8720%    0.8194%    12.2KB
      16   160000  11      0.0520%    0.0459%    19.5KB

Effect of k at a fixed 8 bits/key:
  k= 1  measured 11.604%   theory 11.750%
  k= 2  measured  4.770%   theory  4.893%
  k= 4  measured  2.324%   theory  2.397%
  k= 6  measured  2.170%   theory  2.158%
  k= 8  measured  2.654%   theory  2.549%
  k=12  measured  4.780%   theory  4.833%
```

The measured rates track the formula closely; ~10 bits per key buys < 1 % false positives regardless of how long the keys are; and too many hash functions hurts (k = 12 fills the array with ones).

Real-world examples

- **LSM-tree databases** (RocksDB, Cassandra, HBase, LevelDB) keep one Bloom filter per sorted file so a read skips files that cannot hold the key.
- **"Is this username taken?"** checks and duplicate-submission checks in web apps: a filter in memory answers most "no" cases without touching the database.
- **Web caches and CDNs** (content delivery networks: servers around the world that keep copies of web content near users; see System Design Fundamentals) use them to avoid caching objects seen only once; routers and distributed caches use them to summarize which keys a peer holds.
- **Join filters**: many engines build a Bloom filter from the smaller join side and push it to the scan of the larger side (Spark — a distributed data-processing engine — SQL Server bitmap filters, Oracle).
- **PostgreSQL `bloom` extension**: an index that stores a small signature per row covering many columns, so one index answers equality on *any* subset of them:

```
CREATE INDEX wide_bloom ON wide USING bloom (a, b, c, d, e) WITH (length = 80);
EXPLAIN (ANALYZE, COSTS OFF, TIMING OFF, SUMMARY OFF) SELECT * FROM wide WHERE c = 5 AND e = 9;
```

```
 bloom_index_size | table_size
------------------+------------
 4720 kB          | 15 MB
 Bitmap Heap Scan on wide (actual rows=50 loops=1)
   Recheck Cond: ((c = 5) AND (e = 9))
   Rows Removed by Index Recheck: 97          <- the false positives, removed by rechecking
   ->  Bitmap Index Scan on wide_bloom (actual rows=147 loops=1)
```

Self-check

1. Can a Bloom filter say "no" to a key that was added? → Never; added keys set all their bits.
2. What happens to p if you double m at the same n? → Drops sharply (roughly from 2 % to 0.05 % between 8 and 16 bits/key).
3. Why can't you delete from a plain Bloom filter? → Bits are shared among keys; use a counting Bloom filter.

TL;DR

Bit array + k hashes; "no" is certain, "yes" is probable; ~10 bits/key ≈ 1 % false positives; used to skip disk reads in LSM stores and joins.

12.9 PostgreSQL's other index types: GIN, GiST, BRIN

B-trees answer "equal / less / greater". Other questions — *does this array contain X*, *do these ranges overlap*, *which rows fall in this month of a huge log table* — need other structures. All examples are in `10_index_types.sql`.

Type	Structure in one line	Answers	Typical use
B-tree (default)	Balanced sorted tree (12.6)	`= < > BETWEEN`, prefix `LIKE`, `ORDER BY`	Almost everything
Hash	Buckets by hash (12.7)	`=` only	Long keys, equality only
GIN (Generalized Inverted Index)	For each *element* (array item, JSON key/value, word, trigram — a 3-character chunk of text) a list of rows containing it	`@>` contains, `?` key exists, full-text `@@`, `LIKE '%x%'` with `pg_trgm`	Tags, JSONB documents, search
GiST (Generalized Search Tree)	Balanced tree of "bounding" predicates (boxes, ranges) that may overlap	Overlap `&&`, containment, nearest-neighbour `<->`	Geometry (PostGIS), ranges, exclusion constraints
SP-GiST	Space-partitioned trees (quadtrees, tries)	Points, IP ranges, text prefixes	Unevenly distributed data
BRIN (Block Range INdex)	min/max summary per range of 128 pages	Ranges on columns that follow physical order	Append-only (rows only ever added, never changed) logs, time-series
bloom (extension)	Bloom signature per row (12.8)	`=` on any subset of many columns	Wide tables with ad-hoc filters

BRIN — our `created_at` rises with insertion order (correlation 1), so a 24 kB BRIN index replaces a 21 MB B-tree:

```
Aggregate (actual time=0.716..0.717 rows=1 loops=1)
  Buffers: shared hit=130 read=3
  ->  Bitmap Heap Scan on orders (actual time=0.222..0.656 rows=2787 loops=1)
        Rows Removed by Index Recheck: 11992
        Heap Blocks: lossy=128                <- one 128-page block range read
        ->  Bitmap Index Scan on o_created_brin (... rows=1280 loops=1)
```

It is *lossy*: it only knows which 128-page ranges *might* hold matching timestamps, so it reads the whole range and rechecks every row. On a column in random order every range would span everything and BRIN would be useless.

GIN on arrays and JSONB:

```
CREATE INDEX posts_tags_gin ON posts USING gin (tags);
CREATE INDEX posts_meta_gin ON posts USING gin (meta jsonb_path_ops);
SELECT count(*) FROM posts WHERE tags @> ARRAY['go', 'advanced'];     -- 20000, via Bitmap Index Scan on posts_tags_gin
EXPLAIN (COSTS OFF) SELECT count(*) FROM posts WHERE meta @> '{"lang": "hi", "likes": 7}';
--    ->  Bitmap Index Scan on posts_meta_gin
```

GIN is fast to read but slower to update (one row touches many posting lists — the per-element lists of rows; PostgreSQL buffers new entries in a "pending list").

GiST enforcing "no double booking" — an **exclusion constraint** (no two rows may conflict under the given operators; `tsrange` is a range of timestamps and `&&` means "overlaps"; needs `btree_gist` for the `room`

`WITH =` part):

```
CREATE TABLE bookings (room int, during tsrange, EXCLUDE USING gist (room WITH =, during WITH &&));
INSERT INTO bookings VALUES (101, '[2024-05-01 14:00, 2024-05-03 11:00)');
INSERT INTO bookings VALUES (102, '[2024-05-01 14:00, 2024-05-03 11:00)');  -- other room: fine
INSERT INTO bookings VALUES (101, '[2024-05-02 14:00, 2024-05-04 11:00)');  -- overlaps!
```

```
ERROR:  conflicting key value violates exclusion constraint "bookings_room_during_excl"
DETAIL:  Key (room, during)=(101, ["2024-05-02 14:00:00","2024-05-04 11:00:00")) conflicts with existing key (room, during)=(101, ["2024-05-01 14:00:00","2024-05-03 11:00:00")).
```

Elsewhere: MySQL offers B-tree, full-text, spatial and multi-valued JSON indexes (8.0.17+); SQL Server rowstore, columnstore, full-text and spatial; SQLite B-tree plus FTS5 and R*Tree.

12.10 Reading EXPLAIN: the four ways to read a table

What it is

`EXPLAIN` shows the **plan** the optimizer (another name for the planner) chose: a tree of operators, read from the innermost (most indented) node outwards. `EXPLAIN ANALYZE` also **runs** the query and adds real row counts and times; `BUFFERS` adds pages hit in cache / read from the OS. Key fields:

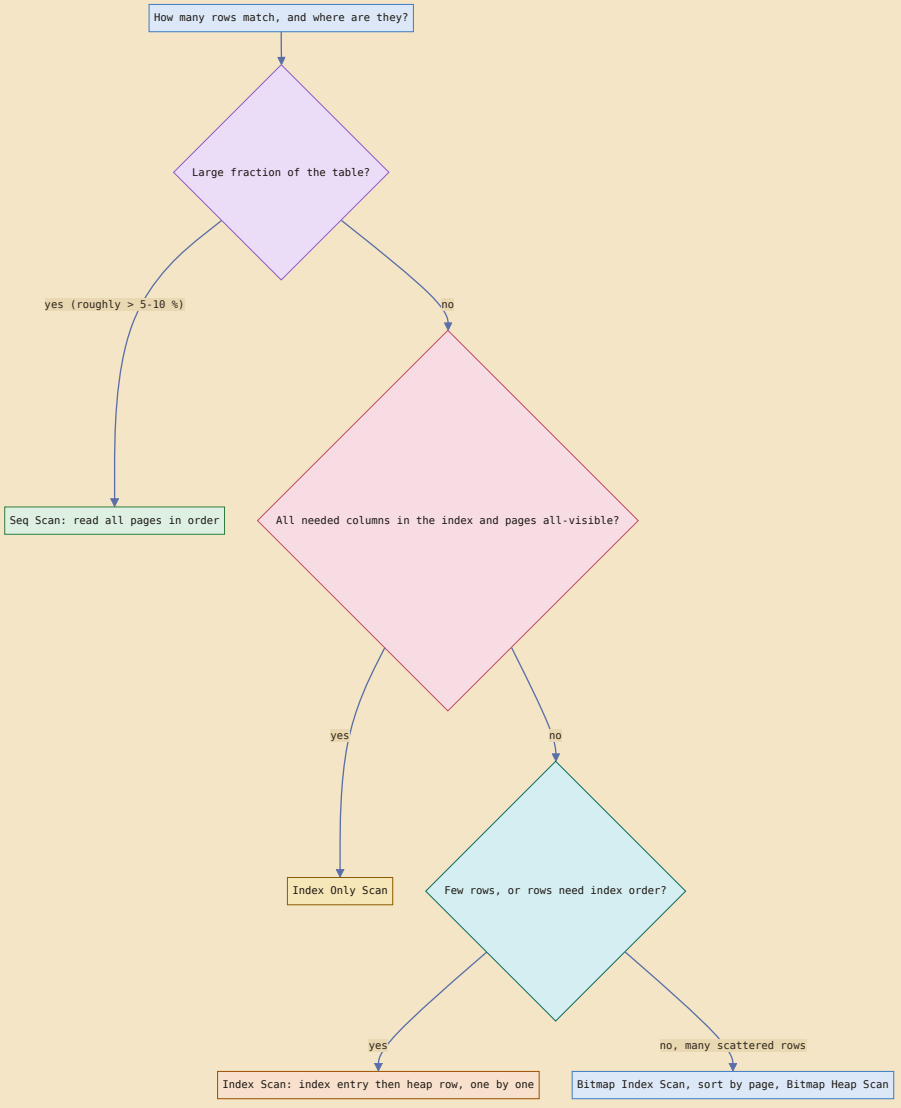
Field	Meaning
`cost=0.42..8.44`	Planner's estimate in abstract units: startup cost .. total cost (a sequential page read = 1.0, random = `random_page_cost` = 4.0)
`rows=` (estimate) vs `actual ... rows=`	Estimated vs real row count — a big gap means bad statistics (12.16)
`loops=`	How many times the node ran (inner side of a nested loop); times and rows are per loop
`Buffers: shared hit=/read=`	Pages found in the buffer pool / fetched from the OS
`Filter` + `Rows Removed by Filter`	Rows read then thrown away — wasted work
`Index Cond`	Condition applied *inside* the index — the useful work

`EXPLAIN ANALYZE` executes the statement — wrap `INSERT/UPDATE/DELETE` in `BEGIN … ROLLBACK`. MySQL has `EXPLAIN` and `EXPLAIN ANALYZE` (8.0.18+); SQL Server shows plans via `SET SHOWPLAN_XML ON` / "actual execution plan"; SQLite uses `EXPLAIN QUERY PLAN`.

The four access methods, measured

From `02_scans.sql` (parallel query — splitting one query across several worker processes — switched off to keep plans small):

Query	Plan	Pages touched	Time
`customer_id = 4242`, no index	**Seq Scan** — read every page, filter	8,662	~30 ms
same, with `orders_customer_idx`	**Bitmap Index + Bitmap Heap Scan**	13	~0.04 ms
`id = 500000`	**Index Scan** on `orders_pkey`	7	~0.01 ms
`count(*) … customer_id BETWEEN 1000 AND 1999`	**Index Only Scan**, `Heap Fetches: 0`	15	~0.5 ms
`sum(amount) … status = 'shipped'` (95 %)	**Seq Scan** despite the index	8,662	~76 ms
`sum(amount) … customer_id BETWEEN 1000 AND 1999`	**Bitmap Heap Scan**, `exact=8454`	8,468	~5 ms
`status = 'refunded' AND customer_id < 2000`	**BitmapAnd** of two indexes	—	—



How to read each:

1. **Seq Scan** reads pages in physical order (storage read-ahead — fetching the next pages before they are requested — makes this fast). For a predicate keeping 95 % of rows it is the right choice.

2. **Index Scan** walks the B-tree, then fetches each heap row; best for a few rows, and rows come out **in index order**, so `ORDER BY … LIMIT` can stop early.
3. **Index Only Scan** answers from the index alone, visiting the heap only for pages not marked all-visible in the **visibility map** (maintained by `VACUUM`; shown as `Heap Fetches`, ch. 13).
4. **Bitmap scan** collects matching `ctid`s into a bitmap (a compact array of bits, one per row or page), sorts them by page and reads each needed page once; it can combine indexes (`BitmapAnd/Or`). Short of memory, the bitmap turns *lossy* (whole pages) and rows are rechecked.

Note the 1 % case: 10,000 rows lived on **8,454 of 8,662 pages**. The index saved almost nothing because the rows were scattered — the reason the planner cares about correlation (12.3) and why "selective" means *few pages*, not just few rows.

Self-check

1. Why did `status = 'shipped'` ignore its index? → 95 % of rows match; reading index + nearly every heap page randomly costs more than one sequential pass.
2. What does `Heap Fetches: 0` mean? → Every page was all-visible, so the index-only scan never touched the table.
3. What is the difference between `Filter` and `Index Cond`? → `Index Cond` narrows the search inside the index; `Filter` discards rows after they were read.

12.11 Composite indexes and column order

What it is

A **composite** (multi-column) index sorts by the first column, then by the second within equal first values, and so on — like a phone book sorted by last name, then first name. From `03_composite_indexes.sql`, with `CREATE INDEX orders_cust_created_idx ON orders (customer_id, created_at)`:

Query	Uses the index?	Why
`WHERE customer_id = 777`	Yes (3 index pages)	Leading column
`WHERE customer_id = 777 ORDER BY created_at DESC LIMIT 3`	Yes, **Index Scan Backward**, no Sort node, 6 pages	Within one customer, entries are already in time order
`WHERE created_at >= '2024-06-01' AND created_at < '2024-06-01 00:10'`	**No** → Seq Scan, 8,662 pages, ~30 ms	Second column alone: one day's rows are spread across all customers' sections
same, after `CREATE INDEX … (created_at)`	Index Only Scan, 4 pages, ~0.04 ms	Now the range is contiguous

```
Limit (actual time=0.002..0.003 rows=3 loops=1)
  Buffers: shared hit=6
  ->  Index Scan Backward using orders_cust_created_idx on orders (... rows=3 loops=1)
        Index Cond: (customer_id = 777)
```

This is the **leftmost-prefix rule**: an index on `(a, b, c)` serves filters on `a`, `a,b` and `a,b,c` efficiently, but not on `b` or `c` alone. (Oracle, MySQL 8.0.13+ and PostgreSQL 18+ can sometimes "skip scan" a composite index when the leading column has few distinct values; don't design around it.)

Equality first, range last

Same query, `customer_id = 777 AND created_at >= '2024-06-01'`, two column orders:

```
-- (customer_id, created_at)
   ->  Bitmap Index Scan on orders_cust_created_idx (... rows=5 loops=1)
         Buffers: shared hit=3
-- (created_at, customer_id)
 Index Scan using orders_created_cust_idx on orders (actual time=1.541..8.275 rows=5 loops=1)
   Index Cond: ((created_at >= '2024-06-01 ...') AND (customer_id = 777))
   Buffers: shared hit=5 read=2211
```

8 pages (3 of them index) vs 2,216 pages. With the equality column first, all matches sit in one narrow slice. With the range column first, the index must walk every entry after June 1 (≈ 55 % of the index) and test `customer_id` on each. Rule of thumb: **equality columns first, then the range or sort column; among equality columns, the most-used-alone first.**

Common mistakes

- Separate `(a)`, `(a, b)`, `(a, b, c)` indexes — the last covers the others.
- Putting a low-selectivity column first "because it's in every query" when queries also filter on a range — check the range test above.
- Assuming `(a, b)` helps `ORDER BY b` — only when `a` is fixed by equality.

12.12 Covering, partial and expression indexes

From `04_covering_partial_expression.sql`.

Covering index (`INCLUDE`, PostgreSQL 11+, SQL Server). Store extra *payload* columns in the leaf entries so the query never visits the table. They are not part of the key (not sorted, not unique-checked).

```
CREATE INDEX orders_cust_incl_idx ON orders (customer_id) INCLUDE (amount);
SELECT sum(amount) FROM orders WHERE customer_id BETWEEN 5000 AND 5999;
```

Index	Plan	Pages	Time
`(customer_id)`	Bitmap Heap Scan, 8,450 heap pages	8,464	~5.3 ms
`(customer_id) INCLUDE (amount)`	**Index Only Scan**, Heap Fetches: 0	42	~0.75 ms

In MySQL you get the same effect by adding the column to the key (`(customer_id, amount)`); InnoDB secondary indexes also always contain the primary key.

Partial index (PostgreSQL, SQLite, SQL Server "filtered index"; not MySQL). Index only the rows a query cares about:

```
CREATE INDEX orders_pending_idx ON orders (created_at) WHERE status = 'pending';
SELECT id, created_at FROM orders WHERE status = 'pending' ORDER BY created_at LIMIT 5;
```

```
        index_name         |  size
---------------------------+--------
 orders_pending_idx        | 456 kB       -- 2 % of rows
 orders_status_created_idx | 30 MB        -- full (status, created_at) rival
 Limit (actual time=0.009..0.011 rows=5 loops=1)
   ->  Index Scan using orders_pending_idx on orders (... rows=5 loops=1)
         Buffers: shared hit=2 read=2
```

The plan has no `Index Cond` at all: every entry in the index is already a pending order. Perfect for work queues (tables of jobs waiting to be processed: "oldest unprocessed job"), soft deletes (`WHERE deleted_at IS NULL`) and **conditional uniqueness** — `CREATE UNIQUE INDEX ON orders (customer_id) WHERE status =`

`'pending'` allows at most one pending order per customer. The planner uses a partial index only when the query's `WHERE` provably implies the index predicate; `status = 'refunded'` gets a Seq Scan + Sort.

Expression (functional) index. Index the result of an expression; queries must use the *same* expression:

```
CREATE INDEX customers_lower_email_idx ON customers (lower(email));
SELECT id FROM customers WHERE lower(email) = 'user4242@mail.com';   -- Seq Scan 918 pages -> Index Scan 4 pages
SELECT id FROM customers WHERE upper(email) = 'USER4242@MAIL.COM';   -- Seq Scan: different expression
```

Only `IMMUTABLE` functions are allowed (same input, same output forever). MySQL 8.0.13+ supports functional key parts `((lower(email)))`; SQL Server indexes a computed column; alternatives are a `citext` (case-insensitive text) column or a case-insensitive collation (the rules a database uses to compare and sort text).

12.13 Anti-patterns that disable indexes

From `05_antipatterns.sql`. The index only helps when the **bare column** is compared with something the planner can compute up front — a property called **sargable** (Search ARGument ABLE).

Anti-pattern	Plan	Fix	Plan after fix
`WHERE created_at::date = DATE '2024-06-01'`	Seq Scan, 8,662 pages, ~39 ms	`created_at >= '2024-06-01' AND created_at < '2024-06-02'`	Index Only Scan, 15 pages, ~0.2 ms
`WHERE id + 1 = 500001`	Seq Scan	`WHERE id = 500001 - 1`	Index Scan on `orders_pkey`
`WHERE email LIKE '%4242@corp%'` (leading wildcard)	Seq Scan, 918 pages	trigram GIN index	Bitmap scan, 74 pages
`WHERE email = … OR name = …` (name unindexed)	Seq Scan	index both sides (`BitmapOr`) or rewrite as `UNION`	BitmapOr
`WHERE lower(email) = …` without an expression index	Seq Scan	expression index (12.12)	Index Scan

Other classics: comparing a column to a value of a different type (in MySQL `WHERE varchar_col = 123` casts the column and skips the index), `NOT IN` / `<>` (rarely selective), and `SELECT *` (rules out index-only scans).

Prefix vs leading wildcard. A B-tree is sorted by the *start* of the string, so `LIKE 'User4242%'` becomes a range (`email >= 'User4242' AND email < 'User4243'`, visible in the plan's Index Cond). This database uses the C collation; under other collations create the index with `text_pattern_ops`. For `'%…%'` the sort order is useless; **pg_trgm** splits text into 3-character pieces and a GIN index finds rows sharing the pattern's trigrams:

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;
SELECT show_trgm('User42');      -- {"  u","  us","42 ",er4,r42,ser,use}
CREATE INDEX customers_email_trgm_idx ON customers USING gin (email gin_trgm_ops);
```

```
Bitmap Heap Scan on customers (actual time=0.563..0.565 rows=3 loops=1)
  Recheck Cond: (email ~~ '%4242@corp%'::text)
  ->  Bitmap Index Scan on customers_email_trgm_idx (... rows=3 loops=1)
        Buffers: shared hit=71
```

For word search in prose, use full-text search (word-based search that understands word forms and ranks matches) instead (`tsvector` + GIN, ch. 14).

The N+1 query problem

Not an index problem but the most common performance bug in application code: load a list (1 query), then run one more query **per item** (N queries). ORMs do this silently when you touch a lazy relationship inside a loop.

> **Key term — ORM:** an *object-relational mapper* is a library that lets application code work with database rows as ordinary objects (`customer.orders`) and writes the SQL for you. A **lazy** relationship is loaded only when first touched — convenient, but touching it once per item in a loop fires one query per item. **Eager loading** fetches the related rows up front in one or two queries. `n_plus_one.py` totals the orders of 500 customers three ways:

```
for cid, _name in customers:                                    # N+1
    conn.execute("SELECT coalesce(sum(amount), 0) FROM orders WHERE customer_id = %s", (cid,))
conn.execute("SELECT customer_id, sum(amount) FROM orders WHERE customer_id = ANY(%s) "
             "GROUP BY customer_id", (ids,))                     # batched: 2 queries
conn.execute("SELECT c.id, coalesce(sum(o.amount), 0) FROM (...) c "
             "LEFT JOIN orders o ON o.customer_id = c.id GROUP BY c.id")   # 1 query
```

```
n_plus_one       queries= 501  local time=   14.5 ms  + network @1ms RTT ~    501 ms
batched_in_list  queries=   2  local time=    1.4 ms  + network @1ms RTT ~      2 ms
single_join      queries=   1  local time=    3.1 ms  + network @1ms RTT ~      1 ms
all three return the same totals: True
```

Over a local Unix socket (a same-machine connection with no network) the loop costs 10×; across a real network each round trip (a request out and its reply back; RTT = round-trip time) adds latency (waiting time), so 501 queries at 1 ms each is half a second. Fixes: a join, an `IN` / `= ANY` batch, or the ORM's eager loading (`select_related` / `prefetch_related` in Django, `JOIN FETCH` / `@EntityGraph` in JPA, `includes` in Rails — popular Python, Java and Ruby frameworks).

12.14 OFFSET vs keyset pagination

`LIMIT 20 OFFSET 999960` does not jump to row 999,960: the database must produce and **discard** all earlier rows. **Keyset** (seek) pagination instead remembers the last row shown and asks for rows *after* it, which an index answers directly. `06_pagination.sql`, with `CREATE INDEX orders_created_id_idx ON orders (created_at, id)`, newest first, 20 per page:

```
-- OFFSET: page N
SELECT id, created_at, amount FROM orders
ORDER BY created_at DESC, id DESC LIMIT 20 OFFSET (N - 1) * 20;
-- Keyset: pass the last (created_at, id) of the previous page
SELECT id, created_at, amount FROM orders
WHERE (created_at, id) < (:'last_ts', :last_id)
ORDER BY created_at DESC, id DESC LIMIT 20;
```

```
      method       | exec_ms | buffers
-------------------+---------+---------
 OFFSET, page 1     |    0.02 |       4
 OFFSET, page 100   |    0.15 |      39
 OFFSET, page 10000 |   15.47 |    3425
 OFFSET, page 49999 |   79.05 |   17114
 keyset, page 49999 |    0.02 |       4
 rows_that_differ = 0
 Limit (actual time=0.004..0.007 rows=20 loops=1)
   ->  Index Scan Backward using orders_created_id_idx on orders (... rows=20 loops=1)
         Index Cond: (ROW(created_at, id) < ROW('2024-01-01 00:21:11'::timestamp ..., 41))
```

OFFSET cost grows linearly with the page number; keyset stays at 4 pages. Rules: make the sort key **unique** by adding the primary key as a tie-breaker (else rows with equal timestamps are skipped or repeated); the row comparison `(a, b) < (x, y)` matches a composite index in PostgreSQL (elsewhere write `a < x OR (a = x AND b < y)`); keyset cannot jump to "page 537" but never skips or duplicates rows when data changes between requests. APIs (the interfaces programs call over the web) hide the last key in an opaque **cursor** — a token the client passes back without reading it (Stripe, Slack, GitHub GraphQL).

12.15 Join algorithms

The logical join (ch. 5) says *what* to match; the planner picks *how*. From `07_join_algorithms.sql`:

Algorithm	How it works	Cost	Best when
Nested loop	For each outer row, look up matching inner rows (ideally via an index)	outer rows × one lookup	Outer side small, inner side indexed on the join key
Hash join	Build a hash table (an in-memory lookup table keyed by hash) on the smaller input's join key, then stream the larger input and probe it	read both once; needs memory (`work_mem`, PostgreSQL's per-operation memory budget)	Large unsorted inputs, equality joins only
Merge join	Both inputs sorted on the key (by index or by a Sort); walk them together like merging two sorted lists	read both once (+ sorts)	Inputs already sorted, or output needs that order; also works for range-ish joins

Three customers (30 orders):

```
Nested Loop (actual time=0.015..0.047 rows=30 loops=1)
  ->  Index Scan using customers_pkey on customers c (... rows=3 loops=1)
  ->  Bitmap Heap Scan on orders o (... rows=10 loops=3)     <- inner side runs 3 times
        ->  Bitmap Index Scan on orders_customer_idx (... loops=3)
```

All customers × all orders, grouped by country:

```
HashAggregate (actual time=362.870..362.906 rows=10 loops=1)
  ->  Hash Join (actual time=11.526..288.224 rows=1000000 loops=1)
        Hash Cond: (o.customer_id = c.id)
        ->  Seq Scan on orders o (... rows=1000000 loops=1)
        ->  Hash (actual time=11.401..11.402 rows=100000 loops=1)
              Buckets: 131072  Batches: 2  Memory Usage: 2976kB
```

`Batches: 2` means the 100,000-row hash table did not fit in `work_mem` (4 MB), so it was split and partly written to temporary files (`temp read/written` in the buffers line). Raising `work_mem` for such a query

would make it one batch. With `ORDER BY c.id LIMIT 5` the planner avoids hashing everything: it walks `customers_pkey` in `id` order in a **Nested Loop**, looks up each customer's orders by index, and stops after five rows. A **Merge Join** needs both inputs sorted on the key; here it appears only when forced (below).

Forcing each algorithm with the `enable_*` settings (for experiments only):

```
     join_size        |  forced  |  join_node  | exec_ms
----------------------+----------+-------------+---------
 small: 3 customers   | nestloop | Nested Loop |     0.1
 small: 3 customers   | hash     | Hash Join   |    33.9
 small: 3 customers   | merge    | Merge Join  |     0.2
 large: all 1M orders | nestloop | Nested Loop |   525.8
 large: all 1M orders | hash     | Hash Join   |   236.7
 large: all 1M orders | merge    | Merge Join  |   487.5
```

No algorithm always wins: hash join is 300× slower than a nested loop for the small join (it scans all 1M orders to build/probe) and 2× faster for the large one. The planner chooses using **row estimates** — which is why statistics matter. MySQL used only nested-loop variants until 8.0.18 added hash joins; SQL Server and Oracle have all three.

12.16 Statistics: how the planner estimates

What it is

The planner cannot run every candidate plan, so it **estimates** how many rows each step produces using statistics that `ANALYZE` (the command that gathers table statistics) collects from a random sample (30,000 rows at the default `default_statistics_target = 100`) and stores in `pg_statistic` (readable through the `pg_stats` view). Autovacuum (PostgreSQL's background maintenance process) runs `ANALYZE` automatically after enough rows change. Because sampling is random, your frequencies may differ slightly. From `08_statistics.sql`:

```
   attname   | null_frac | n_distinct | correlation
-------------+-----------+------------+-------------
 amount      |         0 |      50066 |       0.002
 created_at  |         0 |         -1 |       1.000
 customer_id |         0 |  -0.100437 |       0.003
 note        |    0.9009 |          1 |       1.000
 status      |         0 |          3 |       0.907
     most_common_vals      |        most_common_freqs
---------------------------+---------------------------------
 {shipped,refunded,pending} | {0.95103335,0.0293,0.019666666}
```

Statistic	Meaning	Used for
`null_frac`	Fraction of NULLs	`IS NULL` estimates
`n_distinct`	Distinct values; negative = fraction of rows (−1 = unique, −0.1 = 10 rows per value)	`=` on non-common values, `GROUP BY` sizes
`most_common_vals/freqs`	The most frequent values and their shares	`status = 'refunded'` → 0.0293 × 1M ≈ 29,300 rows
`histogram_bounds`	Equal-population buckets (a histogram: how values are spread across ranges) over the other values	Range predicates
`correlation`	Physical order vs column order	Index-scan cost (12.3)

With fresh statistics, estimate and reality agree: `rows=29300` estimated vs `actual rows=30000`.

When statistics go wrong

Stale statistics. A table where `'error'` was 0.1 % of rows when analyzed; then 200,000 error rows arrive and nobody re-analyzes:

```
 Index Scan using events_kind_idx on events  (cost=0.42..24.78 rows=440 ...) (actual rows=200200 loops=1)
 -- after ANALYZE:
 Index Scan using events_kind_idx on events  (cost=0.42..5249.09 rows=200000 ...) (actual rows=200200
loops=1)
```

The estimate was off by roughly 450× (the exact figure varies with ANALYZE's random sample). Here the plan survived (the new rows happen to be stored together), but in a join, such an error is how a planner picks a nested loop that runs 200,000 times instead of one hash join. Run `ANALYZE` after bulk loads; don't wait for autovacuum.

Correlated columns. The planner multiplies selectivities as if columns were independent. `city = 42 AND state = 4` (city determines state): estimated about 100 rows (106 in the run shown; it varies with sampling), actual 1,000. **Extended statistics** teach it the dependency:

```
CREATE STATISTICS addr_city_state (dependencies) ON city, state FROM addresses;
ANALYZE addresses;      -- estimate becomes about 1,000 (1040 in this run), actual 1000
```

Other levers: `ALTER TABLE … ALTER COLUMN … SET STATISTICS 1000` for a finer histogram on a skewed column (one where a few values are far more common than others); `CREATE STATISTICS … (ndistinct, mcv)` for multi-column `GROUP BY` and value combinations.

In the interview

30-second answer: "The optimizer is cost-based (it scores candidate plans by estimated work and picks the lowest): it estimates rows per step from sampled statistics — null fraction, distinct counts, most common values, histograms, correlation — and picks the cheapest plan. When a query is slow, I compare estimated and actual rows in EXPLAIN ANALYZE; a big gap points to stale or missing statistics, fixed with ANALYZE, a higher statistics target or extended statistics."

- *Why would the planner ignore my index?* → Low selectivity, poor correlation, tiny table, stale stats, a function or type mismatch on the column, or the query needs most rows anyway.

12.17 A tuning workflow

1. **Find** the expensive queries: `pg_stat_statements` (an extension that keeps timing statistics for every query), slow-query logs (`log_min_duration_statement`, MySQL slow log), `auto_explain`.
2. **Read** `EXPLAIN (ANALYZE, BUFFERS)`: big `Rows Removed by Filter`, estimate/actual gaps, `Batches > 1` or on-disk sorts, huge `loops=`.
3. **Fix the query first** (sargable predicates, fewer columns, keyset, no N+1), **then add indexes** (equality → range/sort, `INCLUDE`, partial, expression; `CONCURRENTLY` in production).
4. **Verify** with `EXPLAIN`; drop indexes whose `pg_stat_user_indexes.idx_scan` stays 0; keep statistics fresh.
5. **Still too big?** Partitioning (splitting a table into pieces), replicas (read-only copies on other servers), caching, sharding (spreading data across several servers) — see System Design Fundamentals and chapter 15.

Dialect notes

Feature	PostgreSQL	MySQL (InnoDB)	SQL Server	SQLite
Table storage	Heap + separate indexes	Clustered on PK	Heap or clustered index	B-tree by rowid (or `WITHOUT ROWID`)
Page size	8 KB	16 KB	8 KB	4 KB default
Covering columns	`INCLUDE` (11+)	add to key; PK always included	`INCLUDE`	add to key
Partial index	Yes	No	Filtered index	Yes
Expression index	Yes	Functional key parts (8.0.13+)	Index on computed column	Yes
Hash index	Yes	MEMORY engine; adaptive hash (auto)	Memory-optimized tables	No
Plan output	`EXPLAIN (ANALYZE, BUFFERS)`	`EXPLAIN ANALYZE` (8.0.18+)	Actual execution plan	`EXPLAIN QUERY PLAN`

Chapter TL;DR

- Performance = **pages touched**. A table is a file of 8 KB slotted pages; 1M orders = 8,662 pages, and a Seq Scan reads them all (~30 ms here).
- Rows are addressed by `ctid` (page, slot); large values are compressed/moved by TOAST. Column stores + RLE/dictionary/delta encoding make analytics cheap.
- File organizations: heap, sorted, hashed, clustered. Physical order (correlation) decides whether an index read touches 100 pages or 8,400.
- Indexes: primary/clustering/secondary, dense/sparse; multilevel indexes → **B+ trees** (3 levels for 1M keys). Hash indexes: equality only. Bloom filters: "definitely not / probably yes", ~10 bits/key ≈ 1 % false positives.
- GIN for contains/JSON/trigrams, GiST for overlaps and exclusion constraints, BRIN for huge append-only time columns.
- Scans: Seq (many rows) · Index (few rows, ordered) · Index Only (all columns in the index, vacuumed) · Bitmap (many scattered rows, combinable).
- Composite indexes: **leftmost prefix**; **equality first, range last** (8 vs 2,216 pages). `INCLUDE` for index-only scans; partial indexes for hot subsets; expression indexes for `lower(email)`.
- Keep predicates **sargable**; no functions on indexed columns; `pg_trgm` for `'%x%'`; kill N+1 loops; **keyset** instead of deep OFFSET (79 ms → 0.02 ms).
- Joins: nested loop (small + index), hash (big, equality), merge (sorted). The planner chooses from **statistics** — compare estimated vs actual rows and `ANALYZE` when they diverge.
- Next: chapter 13 (MVCC, vacuum — why dead rows and visibility maps affect all of the above) and chapter 15 (partitioning and scaling beyond one server).

Chapter 13 — Transactions and Concurrency

A database is rarely used by one person at a time: a shop's stock table takes hundreds of checkouts per second while reports read it. Two things can go wrong — a failure can stop a change **halfway**, and two users working **at the same time** can trip over each other. This chapter covers the two tools against those problems, **transactions** and **concurrency control**, and shows how PostgreSQL really behaves with live sessions (separate open connections to the database) running side by side.

What you'll learn

- Transactions, their life cycle, and `BEGIN` / `COMMIT` / `ROLLBACK` / `SAVEPOINT`.
- The four ACID guarantees and how databases implement each.
- Every classic anomaly — dirty read/write, non-repeatable read, phantom, lost update, write skew — **reproduced on a real PostgreSQL server** with two interleaved sessions.
- The four isolation levels in PostgreSQL vs MySQL, SQL Server, Oracle and SQLite.
- Concurrency control: locks and two-phase locking, optimistic validation, timestamp ordering, and MVCC (PostgreSQL's `xmin` / `xmax` from real queries).
- Deadlocks, `SELECT … FOR UPDATE`, `NOWAIT`, and the `SKIP LOCKED` job queue.

How to run the examples. All code lives in `code/`. The single-session files run on the book's private PostgreSQL 14 server (port 5544):

```
bash tools_run_sql.sh sql                                    # runs every chapter's .sql files
bash tools_pg.sh psql -d <db> -f sql/chapters/13-transactions-and-
concurrency/code/01_commit_and_rollback.sql
```

The two-session demos are in `code/concurrency_demo.py`: connections **A** and **B** send statements in a fixed order; a statement expected to block runs in a background thread (a second line of execution inside the same program) while a third *monitor* connection checks `pg_stat_activity` (a system view listing every connection and what it is doing) to prove it is waiting on a lock:

```
cd sql/chapters/13-transactions-and-concurrency/code
../../../../.venv-books/bin/python concurrency_demo.py            # all scenarios (~2 s)
../../../../.venv-books/bin/python concurrency_demo.py writeskew  # just one group
```

In the output, `A| …` is a statement from session A, `->` the server's answer, `..` a monitor observation. Transaction IDs differ on every run. Each scenario starts from fresh sample data (`00_schema.sql`):

```
CREATE TABLE accounts (id int PRIMARY KEY, owner text NOT NULL,
                       balance numeric(10,2) NOT NULL CHECK (balance >= 0));
INSERT INTO accounts VALUES (1, 'asha', 500), (2, 'ben', 300), (3, 'chen', 50);

CREATE TABLE products (id int PRIMARY KEY, name text NOT NULL,
                       stock int NOT NULL CHECK (stock >= 0), version int NOT NULL DEFAULT 1);
INSERT INTO products VALUES (1, 'concert ticket', 10, 1), (2, 'poster', 3, 1);

CREATE TABLE on_call (doctor text PRIMARY KEY, shift text NOT NULL, on_duty boolean NOT NULL);
INSERT INTO on_call VALUES ('dr_rao', 'night', true), ('dr_kim', 'night', true), ('dr_lee', 'day', true);

CREATE TABLE jobs (id int PRIMARY KEY, payload text NOT NULL,
                   status text NOT NULL DEFAULT 'queued', claimed_by text);
INSERT INTO jobs (id, payload) SELECT g, 'email #' || g FROM generate_series(1, 6) g;
```

13.1 What a transaction is

What it is

A **transaction** is a group of database operations treated as **one logical unit of work**: either every operation takes effect, or none does. It ends with a **commit** (all changes become permanent and visible to others) or an **abort**/rollback (every change is undone, as if it never ran).

Analogy: a transaction is like a shopping cart at a self-checkout. You can scan ten items, but nothing is paid for until you press "pay". If the card is declined, the whole cart is cancelled — you never end up paying for seven of the ten items.

The problem it solves

Sending ₹100 from Asha to Ben needs two writes: subtract 100 from Asha, add 100 to Ben. If the server crashes or a rule is broken **between** them, Asha has lost 100 and Ben never received it. A transaction makes the pair all-or-nothing, which also makes **retrying safe**: after an abort there are no half-applied leftovers, so the application simply runs it again.

ACID in one table

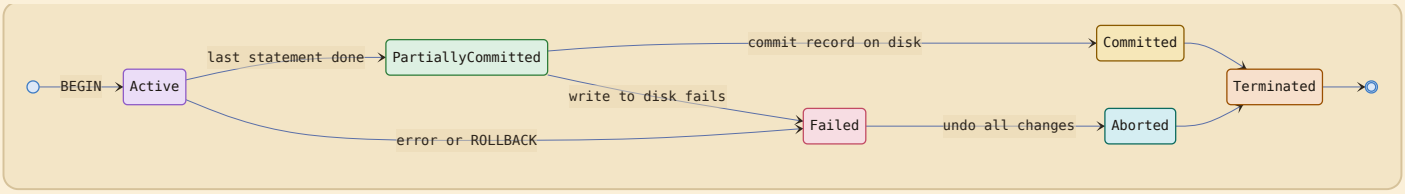
The guarantees a transaction gives are summarised by the acronym **ACID** (coined by Härder and Reuter in 1983). Each letter gets its own section below.

Letter	Guarantee	Main mechanism
Atomicity	All or nothing — no half-done transfers	Undo information / WAL (write-ahead log, §13.5), shadow copies (§13.2)
Consistency	Valid state → valid state (constraints, invariants — rules that must always hold — are true after commit)	Constraints + correct application logic
Isolation	Concurrent transactions don't see each other's in-progress work	Locks, MVCC (multi-version concurrency control, §13.14), SSI (serializable snapshot isolation, §13.8)
Durability	Committed means kept, even after a crash or power loss	WAL flushed to disk, replication (live copies on other servers)

Life cycle of a transaction

A transaction moves through a small set of **states**:

State	What is happening
Active	Statements are running; changes are not yet final.
Partially committed	Last statement done; results not yet safely on permanent storage.
Committed	The commit record is durable; changes are permanent.
Failed	An error (constraint, crash, deadlock — two transactions each waiting for the other, §13.11 — cancel) stops normal progress.
Aborted	All changes rolled back; the database is as before the transaction.
Terminated	Finished (by commit or abort); locks and snapshot (the frozen view of the data it was reading) released.



After an abort the application either **restarts** the transaction (for temporary problems such as a deadlock) or **gives up** and reports the error (for logical problems such as "insufficient funds").

How to write one in SQL

Statement	Effect
`BEGIN` (or `START TRANSACTION`)	Start a transaction block.
`COMMIT`	Make all changes permanent.
`ROLLBACK`	Undo all changes since `BEGIN`.
`SAVEPOINT name`	Mark a point inside the transaction.
`ROLLBACK TO SAVEPOINT name`	Undo only the work done after that mark; the transaction stays open.
`RELEASE SAVEPOINT name`	Forget the mark (keep the work).

Autocommit. If you do *not* write `BEGIN`, PostgreSQL (like psql, JDBC — Java's standard database API — and most drivers, the libraries programs use to talk to a database, by default) wraps **each single statement** in its own transaction and commits it immediately. So a lone `UPDATE` is already atomic; you need `BEGIN … COMMIT` only when several statements must succeed or fail together. (Python's `psycopg`, the usual PostgreSQL driver for Python, is the odd one out: by default it opens a transaction for you and waits for `conn.commit()`.)

`code/01_commit_and_rollback.sql`:

```sql
-- 1) Money transfer that commits: both rows change together.
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;   -- debit asha
UPDATE accounts SET balance = balance + 100 WHERE id = 2;   -- credit ben
COMMIT;
SELECT id, owner, balance, (SELECT sum(balance) FROM accounts) AS bank_total
FROM accounts ORDER BY id;

-- 2) A transfer we change our mind about: ROLLBACK restores the old state.
BEGIN;
UPDATE accounts SET balance = balance - 200 WHERE id = 1;
SELECT 'inside txn' AS moment, balance FROM accounts WHERE id = 1;
ROLLBACK;
SELECT 'after rollback' AS moment, balance FROM accounts WHERE id = 1;

-- 3) Savepoints: undo only part of a transaction.
BEGIN;
UPDATE accounts SET balance = balance + 10 WHERE id = 3;    -- keep this
SAVEPOINT bonus;
UPDATE accounts SET balance = balance + 999 WHERE id = 3;   -- oops
ROLLBACK TO SAVEPOINT bonus;                                -- undo just the oops
COMMIT;
SELECT id, owner, balance FROM accounts WHERE id = 3;
```

Output:

```
BEGIN
UPDATE 1
UPDATE 1
COMMIT
 id | owner | balance | bank_total
----+-------+---------+------------
  1 | asha  |  400.00 |     850.00
  2 | ben   |  400.00 |     850.00
  3 | chen  |   50.00 |     850.00
(3 rows)

BEGIN
UPDATE 1
   moment   | balance
------------+---------
 inside txn |  200.00
(1 row)

ROLLBACK
     moment     | balance
----------------+---------
 after rollback |  400.00
(1 row)

BEGIN
UPDATE 1
SAVEPOINT
UPDATE 1
ROLLBACK
COMMIT
 id | owner | balance
----+-------+---------
  3 | chen  |   60.00
(1 row)
```

The bank total stays 850 (money moved, none created or lost); the session saw its own uncommitted 200, but after `ROLLBACK` the balance is 400 again; and only the +10 survived the savepoint.

Dialect differences

	PostgreSQL	MySQL (InnoDB)	SQL Server	SQLite
Start	`BEGIN` / `START TRANSACTION`	`START TRANSACTION` / `BEGIN`	`BEGIN TRAN`	`BEGIN [DEFERRED|IMMEDIATE|EXCLUSIVE]`
DDL (structure changes like `CREATE TABLE`) inside a transaction	Transactional (`CREATE TABLE` can be rolled back)	Causes an implicit commit (silently commits the open transaction)	Mostly transactional	Transactional
After an error mid-transaction	Whole transaction is "aborted" until `ROLLBACK`	Only the failing statement is undone; transaction continues	Depends on `XACT_ABORT`	Only the failing statement is undone

Weaker guarantees some systems offer

Many NoSQL (non-relational) and caching systems trade multi-statement transactions away for speed or scaling and offer smaller building blocks:

Guarantee	What it means	Example
Single-object atomicity	A write to *one* record/document is all-or-nothing, but nothing spans two records.	A single MongoDB document update; a Cassandra row write.
Atomic increment	The server does read-add-write in one indivisible step, so concurrent `+1`s never lose each other.	Redis (a popular in-memory key-value store) `INCR`; SQL `UPDATE … SET n = n + 1`.
Compare-and-set (CAS)	"Write the new value only if the current value is still what I read." If someone changed it, the write fails and you retry.	Redis `WATCH`/`MULTI`, DynamoDB (Amazon's managed key-value database) conditional writes, a SQL `UPDATE … WHERE version = 7` (§13.7).

They protect **one** object only; keeping several records in step (a transfer, an order plus its stock decrement) needs a real transaction — which is why MongoDB (4.0+) and NewSQL systems (distributed databases that keep SQL and full transactions) added multi-document transactions.

In the interview

> **30-second answer:** "A transaction is a unit of work that fully commits or fully rolls back, with the ACID guarantees. In SQL: `BEGIN … COMMIT/ROLLBACK`, savepoints for partial rollback; without `BEGIN` each statement autocommits."

- *Why not undo step 1 myself on error?* The crash may kill your program too; the database's log handles it even after a power failure.
- *Is a single `UPDATE` atomic?* Yes — in autocommit mode it is its own transaction, and even a multi-row `UPDATE` changes all matching rows or none.

Self-check

1. Name the six transaction states. → Active, partially committed, committed, failed, aborted, terminated.
2. What does autocommit mean? → Each statement is its own transaction, committed as soon as it succeeds.
3. What does `ROLLBACK TO SAVEPOINT s` keep? → Everything done before `SAVEPOINT s`; the transaction stays open.

13.2 Atomicity

What it is

Atomicity means a transaction is **indivisible**: all of its changes are applied (commit), or none are (rollback). There is no visible "half-done" state.

The problem it solves

Take accounts X = 30 and Y = 100 and a transfer of 10 from X to Y. The debit succeeds (X = 20), then the credit fails. Without atomicity the database is left with X = 20, Y = 100: ten units have disappeared. With atomicity the failure rolls back the debit too, leaving X = 30, Y = 100 — exactly as before.

How it works — three implementations

1. Shadow copy (the simplest idea, used by early systems and some editors).

1. When a transaction begins, copy the whole database file; the copy is the **shadow**.
2. Make every change in the shadow only; the original is untouched.

3. To commit, atomically switch a **db-pointer** (a small file naming the current database file) to the shadow, then delete the old file.
4. To abort, delete the shadow; the pointer never moved.

So the only commit point is the pointer switch: if every write succeeds but the crash comes just before the switch, or a failure happens midway, the transaction is aborted and the untouched original remains current.

It is impractical for real databases: copying the **entire** database per transaction is slow and wastes storage, and it allows only **one** transaction at a time, so it can't scale. (Its descendant *shadow paging* — copy only modified pages — was used in System R and lives on in copy-on-write stores — changes always go to fresh copies, never over the original — such as LMDB.)

2. Logging (how PostgreSQL, MySQL, SQL Server and Oracle do it). Changes go into in-memory pages, and every change is also described in the **write-ahead log (WAL)** (§13.5). If the transaction aborts or the system crashes before commit, the engine undoes the changes from its log — or, in PostgreSQL, simply marks the transaction ID *aborted* so its row versions are ignored (§13.14).

3. Across machines. When one transaction touches several databases, **two-phase commit (2PC)** makes them agree: a coordinator asks every participant to *prepare* (make the change durable but not final) and, only if all say yes, tells them to *commit* (chapter 15; PostgreSQL's `PREPARE TRANSACTION`). Optimistic concurrency control (§13.9) also applies a transaction's writes all at once, at commit, only if no conflict was found.

Code: an error aborts the whole transaction

`code/02_atomicity_on_error.sql` — Chen has 50 and tries to send 80. The credit to Asha runs first and succeeds; then the debit breaks the `CHECK (balance >= 0)` rule (`\set ON_ERROR_STOP off` is a psql setting that lets the script continue after an error):

```
\set ON_ERROR_STOP off
BEGIN;
UPDATE accounts SET balance = balance + 80 WHERE id = 1;   -- credit asha: OK
UPDATE accounts SET balance = balance - 80 WHERE id = 3;   -- debit chen: violates CHECK
SELECT balance FROM accounts WHERE id = 1;                  -- refused: txn is aborted
COMMIT;                                                     -- turns into ROLLBACK
SELECT id, owner, balance FROM accounts ORDER BY id;        -- nothing changed

BEGIN;
UPDATE accounts SET balance = balance + 5 WHERE id = 2;
SAVEPOINT before_risky;
INSERT INTO accounts VALUES (1, 'dup', 1);                  -- duplicate primary key
ROLLBACK TO SAVEPOINT before_risky;
COMMIT;
SELECT id, owner, balance FROM accounts WHERE id = 2;
```

Output (psql file path shortened):

```
BEGIN
UPDATE 1
psql:02_atomicity_on_error.sql:10: ERROR:  new row for relation "accounts" violates check constraint
"accounts_balance_check"
psql:02_atomicity_on_error.sql:11: ERROR:  current transaction is aborted, commands ignored until end of
transaction block
ROLLBACK
 id | owner | balance
----+-------+---------
  1 | asha  |  500.00
  2 | ben   |  300.00
  3 | chen  |   50.00
(3 rows)

BEGIN
UPDATE 1
SAVEPOINT
psql:02_atomicity_on_error.sql:19: ERROR:  duplicate key value violates unique constraint "accounts_pkey"
ROLLBACK
COMMIT
 id | owner | balance
----+-------+---------
  2 | ben   |  305.00
(1 row)
```

After an error PostgreSQL marks the transaction **aborted** and refuses further commands; `COMMIT` is answered with `ROLLBACK`, undoing Asha's successful +80 too. Rolling back to a savepoint clears the error and keeps Ben's +5.

Common mistakes

- Running the two halves of a transfer as two autocommitted statements.
- Catching the error in application code and carrying on to `COMMIT` — PostgreSQL silently rolls back; MySQL may commit the half that worked.
- Assuming rollback undoes an email sent or card charged inside the transaction — it undoes only **database** changes; record the action in an *outbox* table (a table of pending messages, written in the same transaction and sent by a separate process) and send after commit.

In the interview

30-second answer: "All-or-nothing. Shadow copy (work on a copy, flip a pointer at commit) explains the idea but copies too much and serialises everything; real engines log changes and roll back from the log; across databases you need two-phase commit."

- *Does atomicity say anything about concurrency?* No — ACID's "atomic" is about failures; other users are isolation's job (unlike "atomic" in multithreaded code).

Self-check

1. In shadow copy, what happens if the crash occurs after all writes but before the pointer update? → The transaction is aborted; the original database is still current.
2. Why doesn't shadow copy scale? → It copies the whole database per transaction and allows one writer at a time.
3. What does PostgreSQL answer to `COMMIT` in an aborted transaction? → `ROLLBACK`.

13.3 Consistency

What it is

Consistency (the C in ACID) means every committed transaction leaves the database **valid if it started valid**. "Valid" is defined by rules: primary and unique keys, foreign keys, `CHECK` and `NOT NULL` constraints, and business **invariants** such as "money is neither created nor destroyed by a transfer". A transaction that would break a rule must fail and leave the previous valid state untouched; rules may be briefly false only *inside* a transaction.

Worked example — a transfer keeps the total

Asha has 300, Ben has 400: the total is **700**. Transfer 100 from Asha to Ben:

Moment	Step	Asha	Ben	Total	Valid?
t1	initial state	300	400	700	yes
t2	`BEGIN`	300	400	700	yes
t3	debit Asha 100	200	400	**600**	no (intermediate)
t4	credit Ben 100	200	500	700	yes
t5	`COMMIT`	200	500	700	yes
t6	final state	200	500	700	yes

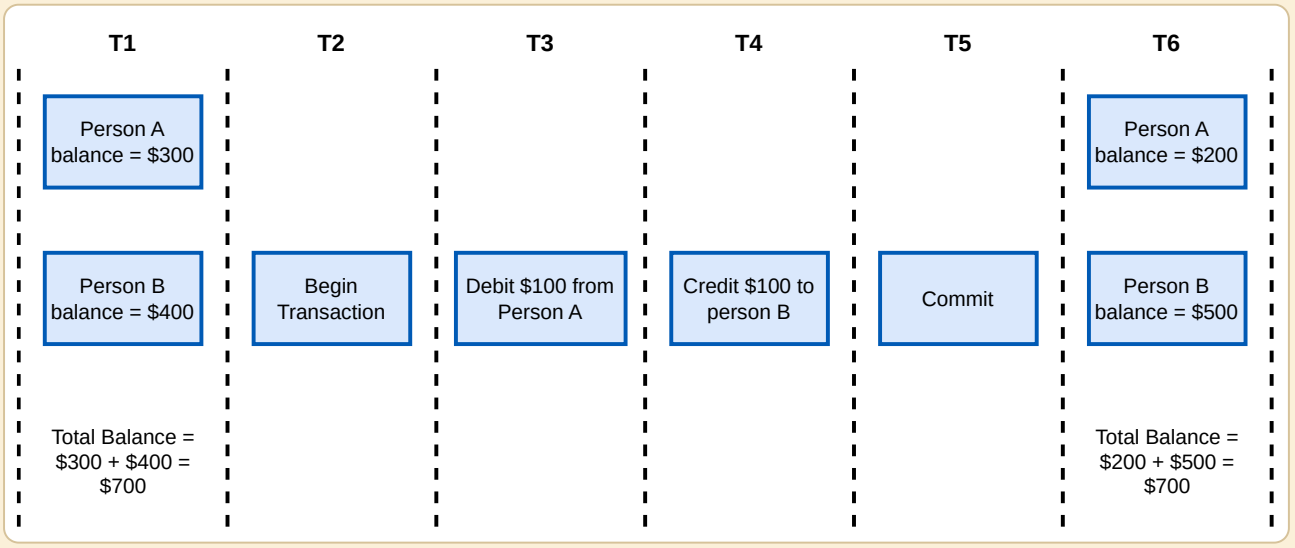


Figure — the total balance is correct before and after the transaction; only the invisible middle is inconsistent (course diagram).

The temporary 600 at t3 is harmless: nobody else can see it (isolation) and it can never be the final state (atomicity).

How it is achieved

1. **Declarative constraints** checked by the database: `PRIMARY KEY`, `UNIQUE`, `FOREIGN KEY`, `CHECK (balance >= 0)`, `NOT NULL` (chapter 9). §13.2's demo showed the `CHECK` rejecting an overdraft.
2. **Pre-transaction checks** in the application or in SQL: e.g., "does Asha have enough money?" before debiting (best done *in the same statement*: `UPDATE … SET balance = balance - 100 WHERE id = 1 AND balance >= 100`).
3. **Post-transaction verification**: deferred constraints (`DEFERRABLE INITIALLY DEFERRED` checks a foreign key only at commit) or audits confirm the invariant, e.g., the ledger still sums to the same total.
4. **Atomicity** and **isolation** stop failures and concurrent transactions from leaving a broken state.

When consistency can be violated

Cause	Example	Who stops it
Constraint violation	Inserting a duplicate primary key	The database rejects the statement
Crash mid-transaction	Crash after debit, before credit	Atomicity + recovery roll it back
Logic bug	Code transfers more than the balance because it forgot the check	Only a constraint (`CHECK`) or correct code
Concurrency without enough isolation	Two withdrawals both see enough money and both proceed	Isolation level / locking (§13.4–13.8)

Trade-offs and a note on the word "consistency"

The database enforces only rules it knows about. Cross-row invariants ("at least one doctor on call") can't be a simple `CHECK`; the application enforces them and needs strong enough isolation (§13.8). Hence Martin Kleppmann (author of DDIA, *Designing Data-Intensive Applications*) calls C a property of the *application*, with A, I and D as the database's tools for it.

Also, **ACID consistency is not CAP consistency.** In CAP (the theorem about trade-offs in distributed databases; see System Design Fundamentals), "consistency" means *linearizability* — every read sees the latest write across replicas (copies of the data on other servers; chapter 15). Same word, different idea.

In the interview

> **30-second answer:** "Every committed state obeys the rules — keys, checks, business invariants. The database enforces declared constraints, the application the rest; atomicity and isolation keep failures and concurrency from breaking them."

- *Give an invariant a CHECK can't express.* "At least one doctor is on call per shift" — it spans rows; needs SERIALIZABLE or explicit locking.

Self-check

1. Is a temporarily wrong total inside a transaction a violation? → No; only committed states must be valid.
2. Name three enforcement mechanisms. → Constraints, pre-checks in the statement/app, atomic rollback (plus isolation).
3. Why can concurrency break consistency? → Two transactions each valid alone can together break an invariant if isolation is too weak.

13.4 Isolation — anomalies and isolation levels

What it is

Isolation controls what one running transaction can see of another's in-progress work. Ideally each behaves **as if it were alone**, but running transactions truly one at a time is too slow, so databases let them overlap and offer **isolation levels**: weaker levels allow more overlap but permit certain **anomalies** (wrong results caused by interleaving); stronger levels forbid more of them at the cost of blocking or aborts.

Analogy: cooks sharing one kitchen. "One cook at a time" is full isolation; real kitchens let everyone work with rules (don't take from someone's half-finished dish). The isolation level is how strict the

rules are.

The anomalies

Timelines below: **T1** and **T2** are concurrent; the account starts at 500.

1. Dirty read — reading data another transaction has written but **not committed**.

Time	T1	T2
1	`BEGIN`; balance = 500 − 100 → **400** (uncommitted)	
2		reads balance → **400**
3	`ROLLBACK` → balance is 500 again	
4		has acted on 400, a value that never officially existed

2. Dirty write — overwriting a value another transaction has written but **not committed**. T1 adds 100 (500 → 600, uncommitted); T2 adds 100 on top (→ 700); then T1 aborts. Restoring "500" erases T2's deposit; keeping 700 keeps T1's cancelled deposit; the correct 600 can't be reconstructed. Practically every database, at every level, prevents dirty writes by making the second writer **wait** for the first to finish.

3. Non-repeatable read (also called read skew) — the same row read twice in one transaction gives **different committed values**, because another transaction committed a change in between.

Time	T1	T2
1	reads balance → 500	
2		balance = 400; `COMMIT`
3	reads balance again → **400**	

"Read skew" is the broader name for seeing *different moments in time* in one transaction — e.g., reading account 1 before a transfer and account 2 after it, so the total looks wrong. §13.6 shows exactly this on PostgreSQL.

4. Phantom read — the same **query** run twice returns a **different set of rows**, because another transaction inserted or deleted rows that match the condition.

Time	T1	T2
1	`SELECT count(*) … WHERE balance >= 100` → 2	
2		`INSERT` a matching row; `COMMIT`
3	same query → **3** (a "phantom" row appeared)	

5. Lost update — two transactions each **read** a value, **compute** a new value, and **write** it; the second write silently overwrites the first.

Time	T1	T2
1	reads stock → 10	
2		reads stock → 10
3	writes 10 − 1 = 9	
4		writes 10 − 1 = 9
	Two items sold, stock only fell by one.	

6. Write skew — two transactions read the **same rows**, each decides based on what it read, and each writes **a different row**. Neither overwrote the other, yet together they break a rule each checked: two on-call

doctors each see "2 on call", both go off duty, nobody is left (§13.8). A lost update is the special case where both write the *same* row.

The isolation levels

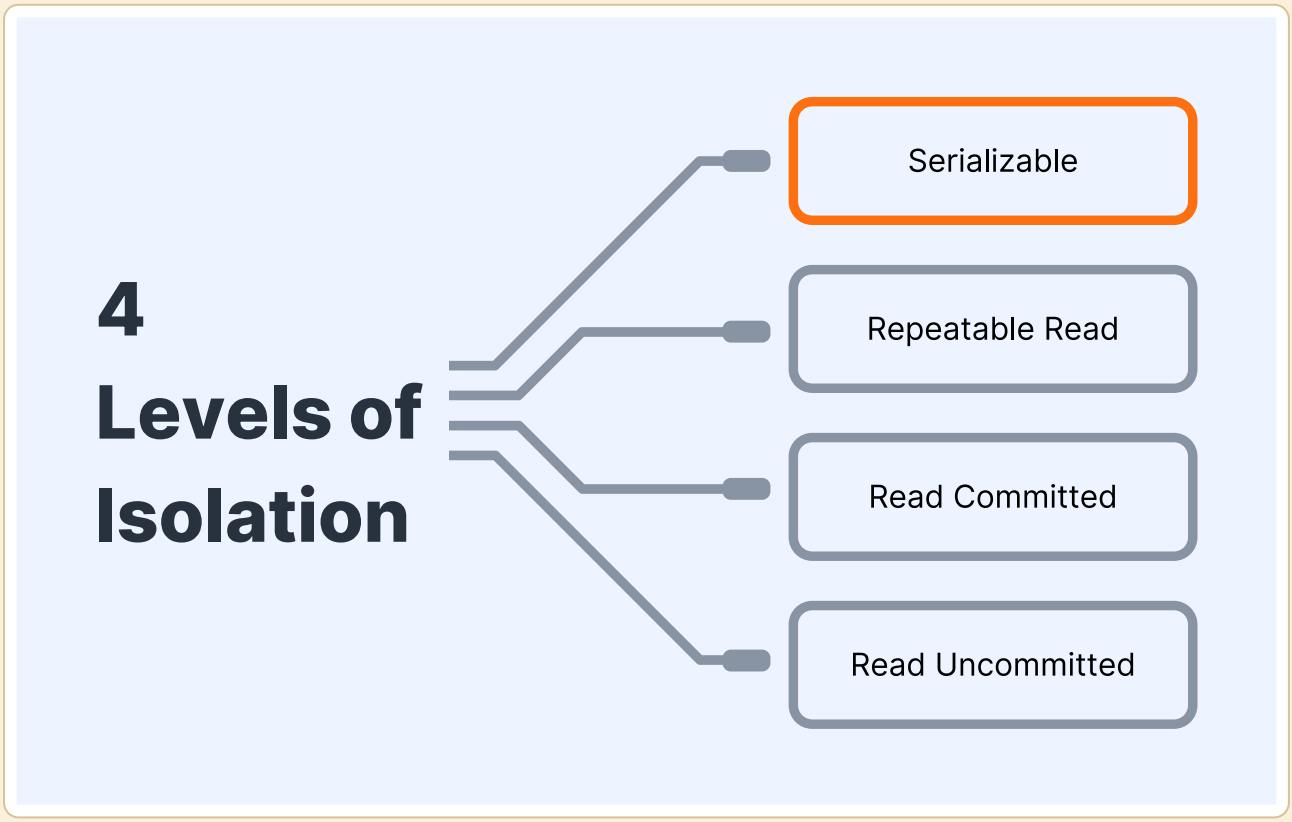


Figure — the four SQL-standard isolation levels (course diagram).

The SQL standard defines four levels by which of the first three read anomalies they must prevent:

Level	Dirty read	Non-repeatable read	Phantom
Read Uncommitted	possible	possible	possible
Read Committed	prevented	possible	possible
Repeatable Read	prevented	prevented	possible
Serializable	prevented	prevented	prevented

That is the *minimum*; real engines often give more, and the standard ignores lost updates and write skew. What **PostgreSQL** actually provides (verified in §13.6–13.8):

Level in PostgreSQL	Dirty read	Dirty write	Non-repeatable / read skew	Phantom	Lost update	Write skew	How it's done
Read Uncommitted	**no** (treated as Read Committed)	no	yes	yes	yes	yes	new snapshot per statement
Read Committed (default)	no	no	yes	yes	yes (read-then-write in app)	yes	new snapshot per statement + row locks for writes
Repeatable Read	no	no	no	**no** (stronger than the standard)	no — aborts with a serialization error	**yes**	one snapshot per transaction (snapshot isolation)
Serializable	no	no	no	no	no	no — aborts one	snapshot isolation + SSI conflict tracking

Two mechanisms matter. At **Read Committed**, MVCC engines hand readers the last committed version so they never wait, while lock-based engines (SQL Server unless `READ_COMMITTED_SNAPSHOT` is on) make readers wait for the writer. **Serializable** (outcome equals *some* one-at-a-time order) is built with **two-phase locking** (take locks as you go and release none until the end, §13.10; MySQL, SQL Server) or **Serializable Snapshot Isolation (SSI)**, which tracks read/write dependencies and aborts a transaction when needed (PostgreSQL 9.1+) — so always be ready to retry. Read Uncommitted (SQL Server's `NOLOCK` hint) is almost never used.

Other engines at a glance

Engine	Default level	Notes
PostgreSQL	Read Committed	RR = snapshot isolation (no phantoms); Serializable = SSI.
MySQL InnoDB	Repeatable Read	Plain `SELECT` reads a snapshot; locking reads (`FOR UPDATE`, `UPDATE`) read the *latest* row with next-key locks (locks on index entries plus the gaps between them, §13.10) against phantoms. Does **not** abort lost updates at RR — use `FOR UPDATE` or atomic updates.
SQL Server	Read Committed (locking)	Optional `READ_COMMITTED_SNAPSHOT` and `SNAPSHOT` isolation use row versions in tempdb (SQL Server's shared scratch database).
Oracle	Read Committed	Its "SERIALIZABLE" is actually snapshot isolation (write skew possible).
SQLite	Serializable	Only one writer at a time for the whole database file; WAL mode lets readers run during a write.

How to choose and set a level

```
BEGIN ISOLATION LEVEL REPEATABLE READ;                  -- for one transaction
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;           -- first statement inside BEGIN
SET default_transaction_isolation = 'serializable';     -- for the session
SHOW transaction_isolation;
```

If you need…	Use
Normal OLTP (online transaction processing: many short reads and writes), single-statement updates (`SET x = x + 1`)	Read Committed
A consistent report or export across many queries	Repeatable Read (often `READ ONLY`)
Read-then-write logic on the same row	`SELECT … FOR UPDATE` at Read Committed, or Repeatable Read + retry
Invariants that span rows (no double booking, at least one on call)	Serializable + retry, or explicit locks

In the interview

> **30-second answer:** "Isolation levels trade correctness for concurrency; the standard defines four by which of dirty reads, non-repeatable reads and phantoms they allow. Postgres never allows dirty reads, its Repeatable Read is snapshot isolation (no phantoms or lost updates, but write skew), and Serializable uses SSI to abort one transaction of a write skew."

- *What's the difference between a non-repeatable read and a phantom?* Non-repeatable: the *same row* changed value. Phantom: the *set of rows* matching a condition changed (insert/delete).
- *Why isn't everyone on Serializable?* More aborts (or blocking with 2PL) and the application must retry; most workloads are fine at Read Committed with a few targeted locks.

Self-check

1. Which anomaly involves an insert? → Phantom read.
2. What is PostgreSQL's default level? MySQL's? → Read Committed; Repeatable Read.
3. Two transactions update *different* rows and break a rule together — which anomaly? → Write skew.

13.5 Durability

What it is

Durability promises that once the database says "committed", the change **stays** — even if the power fails or the process is killed a millisecond later. People act on "committed" (ship the order, release the ticket), so recovery must restore every committed transaction and nothing uncommitted.

Analogy: a shop that writes every sale in an ink ledger *before* handing over the receipt; if the till loses power, the ledger still says what was sold.

How it works

1. **Write-ahead logging (WAL).** A record describing each change must reach the log before the changed data page may be written. At `COMMIT` the database appends a commit record and **flushes** (`fsync` — the operating-system call that forces data out of memory caches onto the physical disk) the log to disk; only then does it answer "COMMIT". Data pages are written later, lazily; after a crash, recovery **replays** the log, redoing committed changes that never reached the data files. (One sequential append is much faster than updating scattered pages.)
2. **Checkpoints.** Periodically all dirty pages (pages changed in memory but not yet written to the data files) are written to the data files and "everything up to log position X is on disk" is recorded, so recovery replays only the log *after* the latest checkpoint (PostgreSQL: `checkpoint_timeout`, default 5 min, and `max_wal_size`).
3. **Synchronous disk writes.** A commit is acknowledged only after the log is on stable storage, not just in RAM or the OS cache. PostgreSQL's `synchronous_commit = on` and `fsync = on` (both defaults) guarantee this; `full_page_writes` protects against *torn pages* (a page only partly written when power fails).

4. **Replication.** Disks die, so the WAL is shipped to other machines (PostgreSQL streaming replication — sending WAL records to replicas continuously as they are written); with *synchronous* replication a commit also waits for a replica (chapter 15).

Code: watching the WAL grow

`code/05_durability_wal.sql` inserts 1,000 rows and measures how far the WAL position (**LSN**, log sequence number) moved:

```
SELECT name, setting FROM pg_settings
WHERE name IN ('fsync', 'synchronous_commit', 'wal_level', 'full_page_writes', 'checkpoint_timeout')
ORDER BY name;

CREATE TEMP TABLE lsn_marks (label text, lsn pg_lsn);
INSERT INTO lsn_marks VALUES ('before', pg_current_wal_lsn());
INSERT INTO accounts SELECT g, 'user' || g, 100 FROM generate_series(10, 1009) g;   -- 1,000 rows
INSERT INTO lsn_marks VALUES ('after', pg_current_wal_lsn());

SELECT pg_size_pretty(pg_wal_lsn_diff(
         (SELECT lsn FROM lsn_marks WHERE label = 'after'),
         (SELECT lsn FROM lsn_marks WHERE label = 'before'))) AS wal_written_for_1000_rows;

BEGIN;
SET LOCAL synchronous_commit = off;     -- faster commit; a crash may lose this txn, never corrupts
UPDATE accounts SET balance = balance + 1 WHERE id = 10;
COMMIT;
```

Output:

```
        name        | setting
--------------------+---------
 checkpoint_timeout | 300
 fsync              | on
 full_page_writes   | on
 synchronous_commit | on
 wal_level          | replica
(5 rows)
...
 wal_written_for_1000_rows
---------------------------
 135 kB
(1 row)
...
```

The 1,000 rows produced about 135 kB of WAL (the exact size can vary slightly by version and settings). The `synchronous_commit = off` transaction returns before its WAL is flushed: a crash within about three times `wal_writer_delay` could lose that one commit, but never corrupt the database — fine for click logging, never for payments. Conversely, a normal committed transfer or order survives a power cut a moment later, because WAL replay restores it.

Common mistakes

- Turning off `fsync` for speed in production — a crash can then **corrupt** the database, not just lose recent commits.
- Assuming a replica makes a commit durable when replication is asynchronous (the primary — the main server that accepts writes — doesn't wait for replicas) — the primary can die before shipping it.
- Disks or cloud volumes that lie about flushes (a write cache — fast memory on the drive — without battery backup, so its contents vanish on power loss).

In the interview

> **30-second answer:** "Committed data survives crashes: the WAL record is fsynced before the commit is acknowledged, data pages are written later, and restart replays the log from the last checkpoint. Replication protects against losing the machine."

- *Why not write the data pages at commit?* Scattered page writes cost far more than a single sequential append to the log, and the log record alone is enough to redo the change.

Self-check

1. What must be on disk before `COMMIT` returns? → The transaction's WAL records including the commit record.
2. What does a checkpoint shorten? → Crash recovery (replay starts at the last checkpoint).
3. Does asynchronous replication strengthen durability of the latest commit? → No; only synchronous replication does.

13.6 Watching isolation anomalies on a real server

This section runs the anomalies from §13.4 on PostgreSQL 14 with two live sessions (A and B) using `code/concurrency_demo.py`. The core of the driver:

```
class Session:
    def __init__(self, name):
        self.name = name
        self.conn = psycopg.connect(f"{BASE} dbname={DB}", autocommit=True)  # we send BEGIN/COMMIT
ourselves
        self.pid = self.conn.info.backend_pid

    def run(self, sql):                         # send one statement, print the answer
        print(f"  {self.name}| {sql}  -> {self._exec(sql)}")

    def start(self, sql):                       # a statement we expect to BLOCK: run it in a thread
        print(f"  {self.name}| {sql}")
        box = {}
        t = threading.Thread(target=lambda: box.setdefault("r", self._exec(sql)))
        t.start()
        return self, t, box

class Monitor:                                  # a third connection that watches the other two
    def wait_until_blocked(self, s, timeout=5.0):
        # poll pg_stat_activity until session s shows wait_event_type = 'Lock'
        ...
        # SELECT wait_event_type, wait_event, pg_blocking_pids(pid)
        # FROM pg_stat_activity WHERE pid = %s
```

`_exec` returns the rows, the command tag (`UPDATE 1`), or `ERROR <sqlstate>: <message>` (SQLSTATE is the standard five-character error code, e.g. `40001`). In deadlock messages, it replaces the process IDs with `<A>` / `<B>` so the output is readable.

Dirty read and dirty write — impossible in PostgreSQL

Session A changes Asha's balance to 0 but doesn't commit. Session B even asks for `READ UNCOMMITTED`:

```
=== Dirty read and dirty write: impossible in PostgreSQL ===
  A| BEGIN  -> BEGIN
  A| UPDATE accounts SET balance = 0 WHERE id = 1  -> UPDATE 1
  B| BEGIN ISOLATION LEVEL READ UNCOMMITTED  -> BEGIN
  B| SHOW transaction_isolation  -> read uncommitted
  B| SELECT balance FROM accounts WHERE id = 1  -> 500.00
  B| UPDATE accounts SET balance = balance + 1 WHERE id = 1
     .. B is WAITING (wait_event=transactionid, blocked by A)
  A| ROLLBACK  -> ROLLBACK
     -> (B resumes) UPDATE 1
  B| COMMIT  -> COMMIT
  M| SELECT balance FROM accounts WHERE id = 1  -> 501.00
```

- **No dirty read:** B really is at `read uncommitted`, yet reads the committed **500**, not A's 0 — PostgreSQL implements Read Uncommitted as Read Committed. The read also didn't block: MVCC hands B the older committed version.
- **No dirty write:** B's `UPDATE` of the same row **waits** (wait event `transactionid`: B waits for A's transaction to end). After A's rollback B updates the committed 500 → 501.

Non-repeatable read / read skew: Read Committed vs Repeatable Read

A reads Asha's balance. Meanwhile B transfers 100 from Asha (account 1) to Ben (account 2) and commits. A then reads both accounts.

```
=== Non-repeatable read / read skew: READ COMMITTED ===
  A| BEGIN ISOLATION LEVEL READ COMMITTED  -> BEGIN
  A| SELECT balance FROM accounts WHERE id = 1  -> 500.00
  B| BEGIN  -> BEGIN
  B| UPDATE accounts SET balance = balance - 100 WHERE id = 1  -> UPDATE 1
  B| UPDATE accounts SET balance = balance + 100 WHERE id = 2  -> UPDATE 1
  B| COMMIT  -> COMMIT
  A| SELECT balance FROM accounts WHERE id = 1  -> 400.00
  A| SELECT balance FROM accounts WHERE id = 2  -> 400.00
  A| COMMIT  -> COMMIT
```

At Read Committed the same row gave **500 then 400** — a non-repeatable read. An auditor adding its first read of account 1 (500) to its read of account 2 (400) would report **900**, though the true total was 800 at every instant: **read skew**.

```
=== Non-repeatable read / read skew: REPEATABLE READ ===
  A| BEGIN ISOLATION LEVEL REPEATABLE READ  -> BEGIN
  A| SELECT balance FROM accounts WHERE id = 1  -> 500.00
  ... (B runs the same transfer as above and commits)
  A| SELECT balance FROM accounts WHERE id = 1  -> 500.00
  A| SELECT balance FROM accounts WHERE id = 2  -> 300.00
  A| COMMIT  -> COMMIT
```

At Repeatable Read A keeps seeing the world as of its **first statement** (not `BEGIN`): 500 and 300, total 800 — consistent, just slightly old.

Phantom read: Read Committed vs Repeatable Read

```
=== Phantom read: READ COMMITTED ===
  A| BEGIN ISOLATION LEVEL READ COMMITTED  -> BEGIN
  A| SELECT count(*) FROM accounts WHERE balance >= 100  -> 2
  B| INSERT INTO accounts VALUES (4, 'dina', 900)  -> INSERT 0 1
  A| SELECT count(*) FROM accounts WHERE balance >= 100  -> 3
  A| COMMIT  -> COMMIT

=== Phantom read: REPEATABLE READ (PostgreSQL prevents it) ===
  A| BEGIN ISOLATION LEVEL REPEATABLE READ  -> BEGIN
  A| SELECT count(*) FROM accounts WHERE balance >= 100  -> 2
  B| INSERT INTO accounts VALUES (4, 'dina', 900)  -> INSERT 0 1
  A| SELECT count(*) FROM accounts WHERE balance >= 100  -> 2
  A| COMMIT  -> COMMIT
```

At Read Committed a phantom appears (2 → 3). At Repeatable Read the count stays 2 — the standard would *allow* a phantom, but PostgreSQL's snapshot simply doesn't contain Dina's row.

Common mistakes

- Believing Repeatable Read shows "the latest data" — a long Repeatable Read transaction sees an increasingly stale world (and holds back cleanup, §13.14).
- Running a multi-query report at Read Committed and getting totals that never existed. Use `BEGIN ISOLATION LEVEL REPEATABLE READ READ ONLY`.

13.7 Lost updates and how to prevent them

The problem

A **lost update** comes from the *read–modify–write* pattern: the application reads a value, computes a new one in its own code, and writes it back; if two copies overlap, the later write wipes out the earlier one. It is the most common concurrency bug in web apps (stock counters, likes, wallet balances, wiki edits).

Seeing it happen

Two customers buy a concert ticket at the same moment. Each app instance reads the stock, subtracts one in Python/Java, and writes the result:

```
=== Lost update 1: read-modify-write in the application (READ COMMITTED) ===
  A| BEGIN  -> BEGIN
  A| SELECT stock FROM products WHERE id = 1  -> 10
  B| BEGIN  -> BEGIN
  B| SELECT stock FROM products WHERE id = 1  -> 10
  A| UPDATE products SET stock = 9 WHERE id = 1  -> UPDATE 1
  A| COMMIT  -> COMMIT
  B| UPDATE products SET stock = 9 WHERE id = 1  -> UPDATE 1
  B| COMMIT  -> COMMIT
  M| SELECT stock FROM products WHERE id = 1  -> 9
```

Two tickets sold, stock fell only to **9**, and nothing errored. The fix must make read and write one protected step; there are four standard ways.

Fix 1 — let the database do the arithmetic (atomic update)

```
=== Lost update fix 1: atomic UPDATE ... SET stock = stock - 1 ===
  A| BEGIN  -> BEGIN
  A| UPDATE products SET stock = stock - 1 WHERE id = 1 RETURNING stock  -> 9
  B| BEGIN  -> BEGIN
  B| UPDATE products SET stock = stock - 1 WHERE id = 1 RETURNING stock
     .. B is WAITING (wait_event=transactionid, blocked by A)
     .. pg_locks for B: (transactionid, ExclusiveLock, True), (tuple, ExclusiveLock, True),
(transactionid, ShareLock, False)
  A| COMMIT  -> COMMIT
     -> (B resumes) 8
  B| COMMIT  -> COMMIT
  M| SELECT stock FROM products WHERE id = 1  -> 8
```

B's `UPDATE` waits for A's row lock; when A commits, PostgreSQL (at Read Committed) **re-reads the newest row version** and applies `stock - 1` to it: 9 → 8. The `pg_locks` line (a system view of locks held and awaited) shows the mechanism: B holds its own transaction-ID lock and a *tuple* lock (first in line for the row) and waits (`granted = False`) for a `ShareLock` on **A's transaction ID** — "wake me when A ends". The row lock itself is recorded in the row (its `xmax`), not in a lock table, so locking a million rows needs no million lock entries.

Use when the new value can be computed in SQL (`n = n + 1`, `balance = balance - 100 WHERE balance >= 100`) — the simplest, fastest fix.

Fix 2 — pessimistic lock: `SELECT … FOR UPDATE`

When the new value needs application logic (call a pricing service, validate a complex rule), lock the row while reading it:

```
=== Lost update fix 2: pessimistic SELECT ... FOR UPDATE ===
  A| BEGIN  -> BEGIN
  A| SELECT stock FROM products WHERE id = 1 FOR UPDATE  -> 10
  B| BEGIN  -> BEGIN
  B| SELECT stock FROM products WHERE id = 1 FOR UPDATE
     .. B is WAITING (wait_event=transactionid, blocked by A)
  A| UPDATE products SET stock = 9 WHERE id = 1  -> UPDATE 1
  A| COMMIT  -> COMMIT
     -> (B resumes) 9
  B| UPDATE products SET stock = 8 WHERE id = 1  -> UPDATE 1
  B| COMMIT  -> COMMIT
  M| SELECT stock FROM products WHERE id = 1  -> 8
```

B's read waits until A commits and then returns **9**, not 10, so B computes 8. The lock lasts until the end of the transaction — keep such transactions short and never wait for a human while holding one.

Fix 3 — Repeatable Read detects it (optimistic, by the database)

```
=== Lost update fix 3: REPEATABLE READ detects the conflict ===
  A| BEGIN ISOLATION LEVEL REPEATABLE READ  -> BEGIN
  A| SELECT stock FROM products WHERE id = 1  -> 10
  B| BEGIN ISOLATION LEVEL REPEATABLE READ  -> BEGIN
  B| SELECT stock FROM products WHERE id = 1  -> 10
  A| UPDATE products SET stock = 9 WHERE id = 1  -> UPDATE 1
  A| COMMIT  -> COMMIT
  B| UPDATE products SET stock = 9 WHERE id = 1  -> ERROR 40001: could not serialize access due to
concurrent update
  B| ROLLBACK  -> ROLLBACK
  (B retries the whole transaction)
  ... (B re-reads 9, writes 8, commits)
  M| SELECT stock FROM products WHERE id = 1  -> 8
```

B's snapshot says 10, but the row was changed by a transaction that committed after that snapshot, so PostgreSQL refuses with SQLSTATE **40001** (`serialization_failure`). The application **retries the whole transaction** and writes 8. (MySQL/InnoDB at Repeatable Read raises no such error — its `UPDATE` reads the latest row version — so there use Fix 1 or 2.)

Fix 4 — optimistic locking with a version column (compare-and-set)

Compare-and-set (CAS) means "change the value only if it is still what I read". It holds no lock, so it suits long gaps between read and write (a user editing a form for minutes, several HTTP requests — separate web requests, each in its own transaction). Add a `version` column and include it in the `WHERE`:

```
=== Lost update fix 4: optimistic locking with a version column (compare-and-set) ===
  A| SELECT stock, version FROM products WHERE id = 1  -> (10, 1)
  B| SELECT stock, version FROM products WHERE id = 1  -> (10, 1)
  A| UPDATE products SET stock = 9, version = version + 1 WHERE id = 1 AND version = 1  -> UPDATE 1
  B| UPDATE products SET stock = 9, version = version + 1 WHERE id = 1 AND version = 1  -> UPDATE 0
  (B sees UPDATE 0 -> someone changed the row; re-read and retry)
  B| SELECT stock, version FROM products WHERE id = 1  -> (9, 2)
  B| UPDATE products SET stock = 8, version = version + 1 WHERE id = 1 AND version = 2  -> UPDATE 1
  M| SELECT stock, version FROM products WHERE id = 1  -> (8, 3)
```

B's first update matched **0 rows** — the conflict signal (the version is no longer the *expected* 1). B re-reads (9, version 2) and succeeds. The database checks and writes in one atomic step; `UPDATE 0` means "someone else won — reload and retry, or tell the user the record changed". JPA/Hibernate's (Java) `@Version`, Rails' (Ruby) `lock_version`, DynamoDB conditional writes and Redis `WATCH`/`MULTI`/`EXEC` are the same idea: no locks (so no deadlocks), great when conflicts are rare, wasteful retries when they are frequent.

Comparison

Fix	Blocking?	Retry needed?	Best for
Atomic `UPDATE … SET x = x - 1`	Briefly (row lock)	No	Counters, balances — anything expressible in SQL
`SELECT … FOR UPDATE`	Yes, until commit	No	Complex logic between read and write, short transactions
Repeatable Read / Serializable	No (aborts instead)	Yes (40001)	Code that already has a retry loop
Version column (CAS)	No	Yes (0 rows)	Long "think time", web forms, APIs, distributed caches

In the interview

30-second answer: "Two read-modify-write cycles interleave and one write overwrites the other. Fixes: atomic SQL arithmetic; SELECT FOR UPDATE; Repeatable Read, where Postgres aborts the second writer (40001); or a version column where zero updated rows means retry."

- *Which would you use for a "like" counter?* Atomic `UPDATE posts SET likes = likes + 1` (or a separate likes table and count).
- *Which for a user editing a product page for minutes?* Version column — never hold a DB lock across user think time.
- *Does Read Committed prevent lost updates?* Only if the write is expressed as one atomic statement; not for app-side read-modify-write.

Self-check

1. Why did the naive version end at 9? → Both read 10 and both wrote 10 − 1.
2. What does `UPDATE 0` mean in optimistic locking? → The version changed; someone else updated first.
3. What SQLSTATE does PostgreSQL use for a serialization failure? → 40001.

13.8 Write skew and Serializable isolation

The problem

Hospital rule: **at least one doctor on call for the night shift**. Dr Rao and Dr Kim, both on call, each click "go off duty"; the app checks "is anyone else on call?" and, if so, updates *that doctor's own row*. Different rows, so no row-lock conflict — yet the rule can break.

Repeatable Read (snapshot isolation) allows it

```
=== Write skew: REPEATABLE READ lets both doctors go off duty ===
  A| BEGIN ISOLATION LEVEL REPEATABLE READ  -> BEGIN
  B| BEGIN ISOLATION LEVEL REPEATABLE READ  -> BEGIN
  A| SELECT count(*) FROM on_call WHERE shift = 'night' AND on_duty  -> 2
  B| SELECT count(*) FROM on_call WHERE shift = 'night' AND on_duty  -> 2
  A| UPDATE on_call SET on_duty = false WHERE doctor = 'dr_rao'  -> UPDATE 1
  B| UPDATE on_call SET on_duty = false WHERE doctor = 'dr_kim'  -> UPDATE 1
  A| COMMIT  -> COMMIT
  B| COMMIT  -> COMMIT
  M| SELECT count(*) FROM on_call WHERE shift = 'night' AND on_duty  -> 0
```

Nobody is on call. Each transaction was correct against its snapshot, but run one after the other the second would have seen 1 and refused. The same shape appears in double-booking a room, claiming a username checked by `SELECT` first, or overspending a shared budget.

Serializable (SSI) aborts one

```
=== Write skew: SERIALIZABLE aborts one of them ===
  A| BEGIN ISOLATION LEVEL SERIALIZABLE  -> BEGIN
  B| BEGIN ISOLATION LEVEL SERIALIZABLE  -> BEGIN
  A| SELECT count(*) FROM on_call WHERE shift = 'night' AND on_duty  -> 2
  B| SELECT count(*) FROM on_call WHERE shift = 'night' AND on_duty  -> 2
  A| UPDATE on_call SET on_duty = false WHERE doctor = 'dr_rao'  -> UPDATE 1
  B| UPDATE on_call SET on_duty = false WHERE doctor = 'dr_kim'  -> UPDATE 1
  A| COMMIT  -> COMMIT
  B| COMMIT  -> ERROR 40001: could not serialize access due to read/write dependencies among transactions
     DETAIL: Reason code: Canceled on identification as a pivot, during commit attempt.
  M| SELECT count(*) FROM on_call WHERE shift = 'night' AND on_duty  -> 1
```

How **Serializable Snapshot Isolation (SSI)** caught it, in steps:

1. Each Serializable transaction still reads from a snapshot (readers and writers don't block).
2. It records what it **read** as *SIREAD* "locks" — non-blocking notes such as "B read the night-shift rows".
3. When a transaction writes something a concurrent one read, a **read-write dependency** (rw-conflict) is recorded: here "A wrote what B read" and "B wrote what A read".
4. Such a cycle means no serial order could produce the result. On the dangerous pattern (a "pivot" transaction with a conflict in and a conflict out) PostgreSQL aborts one transaction with **40001**; the retried B sees only 1 doctor on call and refuses.

The error can appear at any statement, including `COMMIT`, so **every Serializable transaction needs a retry loop**:

```python
for attempt in range(5):
    try:
        with conn.transaction():                    # BEGIN ... COMMIT
            conn.execute("SET TRANSACTION ISOLATION LEVEL SERIALIZABLE")
            go_off_duty(conn, doctor)
        break
    except psycopg.errors.SerializationFailure:     # SQLSTATE 40001
        continue                                    # rerun the whole transaction
```

SSI can also abort harmless transactions (false positives) — its bookkeeping is approximate and coarsens from rows to pages or whole tables when a transaction reads a lot. Keep Serializable transactions short and mark read-only ones `READ ONLY`.

Fix without Serializable: lock what you based the decision on

If you know exactly which rows the decision depends on, lock them. Here both transactions run at the default Read Committed:

```
=== Write skew fix without SERIALIZABLE: lock the rows you base the decision on ===
  A| BEGIN  -> BEGIN
  B| BEGIN  -> BEGIN
  A| SELECT doctor FROM on_call WHERE shift = 'night' AND on_duty FOR UPDATE  -> (dr_rao), (dr_kim)
  B| SELECT doctor FROM on_call WHERE shift = 'night' AND on_duty FOR UPDATE
     .. B is WAITING (wait_event=transactionid, blocked by A)
  A| UPDATE on_call SET on_duty = false WHERE doctor = 'dr_rao'  -> UPDATE 1
  A| COMMIT  -> COMMIT
     -> (B resumes) dr_kim
  (B's app sees only one doctor left, so it refuses to go off duty)
  B| ROLLBACK  -> ROLLBACK
  M| SELECT count(*) FROM on_call WHERE shift = 'night' AND on_duty  -> 1
```

After waiting, B's `FOR UPDATE` re-checks the rows and returns only Dr Kim. Limitation: `FOR UPDATE` locks only rows that **exist**. If the rule is about the *absence* of rows ("no booking overlaps 10:00–11:00"), there is nothing to lock. Then use Serializable; a constraint (`UNIQUE`, or PostgreSQL's exclusion constraint — which forbids any two rows that conflict, here overlapping times for the same room — `EXCLUDE USING gist (room WITH =, during WITH &&)`, which needs the `btree_gist` extension for the `=` part); or *materialize the conflict* — pre-create lockable rows, e.g., one per room per time slot.

In the interview

30-second answer: "Two transactions read the same data, decide on it, and write different rows — no write conflict, but an invariant breaks. Snapshot isolation allows it. Fixes: Serializable (SSI aborts one with 40001 — retry), FOR UPDATE on the rows the decision depends on, or a unique/exclusion constraint."

- *Why doesn't FOR UPDATE fix double-booking of an empty slot?* The conflicting row doesn't exist yet (a phantom); use a constraint, Serializable, or materialized slot rows.

Self-check

1. At Repeatable Read, why was no conflict detected? → The two transactions updated different rows.
2. What are SIREAD locks? → Non-blocking records of what a Serializable transaction read, used to find rw-dependencies.
3. Name two non-Serializable fixes. → `FOR UPDATE` on the rows read; a unique/exclusion constraint.

13.9 Concurrency control — the big picture

What it is

Concurrency control is the part of a database that decides how overlapping transactions may interleave so the result stays correct: no anomalies from §13.4 (data integrity), each transaction isolated to the degree its level promises, and as much parallelism as possible (performance).

The formal correctness goal is **serializability**: a concurrent execution (a *schedule*) is correct if its effect equals that of *some* serial order of the same transactions. Two operations **conflict** if they belong to different transactions, touch the same item, and at least one is a write; a schedule is *conflict-serializable* if its conflict graph (an arrow T1 → T2 whenever an operation of T1 conflicts with, and comes before, one of T2) has no cycle.

The four families

Technique	Core idea	Waits or aborts?	Used by
Pessimistic / lock-based (e.g., 2PL)	Lock before you touch; others wait	Waits (deadlocks possible)	SQL Server, MySQL locking reads, DB2; `FOR UPDATE` everywhere
Optimistic (OCC)	Work freely, validate at commit, abort on conflict	Aborts	App-level version columns, some in-memory and distributed DBs
Timestamp ordering	Each txn gets a timestamp; operations must respect timestamp order	Aborts	Mostly textbook; ideas used in distributed DBs
MVCC	Keep several versions; readers read an old version, never block writers	Readers never wait; writers wait for writers	PostgreSQL, Oracle, MySQL InnoDB, SQL Server snapshot modes

These are combined in practice: PostgreSQL = MVCC for reads + row locks for writes + SSI (optimistic validation) for Serializable.

Optimistic vs pessimistic

Optimistic concurrency control (OCC) assumes conflicts are **rare**. A transaction runs in three phases: (1) **read/execute** in a private workspace, taking no locks; (2) **validate** at commit — did a transaction that committed meanwhile change anything it read? (3) **write** if not, otherwise **abort and retry**.

Example: users A and B open the same product; A saves first; B's save fails validation and retries — the version-column pattern of §13.7.

Pessimistic concurrency control (PCC) assumes conflicts are **likely**: **lock** data when first touching it, make conflicting transactions **wait**, and **release** at commit or rollback.

Example: A locks the product row; B waits until A finishes (§13.7 Fix 2).

Aspect	Optimistic (OCC)	Pessimistic (PCC)
Assumes	Conflicts are rare	Conflicts are common
Locks	None during execution	Taken on access, held to the end
Cost of a conflict	Abort and redo the work	Waiting
Overhead with no conflicts	Very low	Lock management on every access
Deadlocks	None (no waits)	Possible — need detection
Fits	Read-heavy, low-contention (few transactions competing for the same rows), distributed systems, long user think time	Write-heavy, hot rows (a popular product's stock), critical updates where retrying is expensive

Real-world examples. **Git** (the version-control tool) is optimistic: nobody locks a file, and conflicts surface only at push/merge time. **Google Docs** lets everyone type and reconciles edits as they sync. **Banking** and **ticketing** lean pessimistic on hot rows so a seat is never sold twice; **e-commerce** locks (or atomically decrements) the stock row at purchase.

In the interview

> **30-second answer:** "Keep interleaved transactions serializable. Pessimistic: lock and wait (high contention; deadlocks possible). Optimistic: validate at commit and abort on conflict (rare

conflicts). Timestamp ordering: follow start order. MVCC: versions so readers never block writers. Postgres combines MVCC, row locks and SSI."

Self-check

1. Three phases of OCC? → Read/execute, validate, write (or abort).
2. Why does OCC have no deadlocks? → Transactions never wait for each other's locks.
3. Which is better for a flash sale on one product? → Pessimistic (or an atomic update): contention is very high, OCC would abort constantly.

13.10 Lock-based concurrency control

What it is

With **lock-based concurrency control**, a transaction must obtain a **lock** on data before using it; a conflicting request waits until the lock is released — so others can't read or overwrite your in-progress changes.

Types of locks

Lock	Purpose	Others may…
Shared (S)	Reading	also take S (read together); may **not** write
Exclusive (X)	Writing	do nothing with the item (no S, no X) until released
Intention Shared (IS)	On a table: "I will take S locks on some rows inside"	—
Intention Exclusive (IX)	On a table: "I will take X locks on some rows inside"	—

Lock granularity

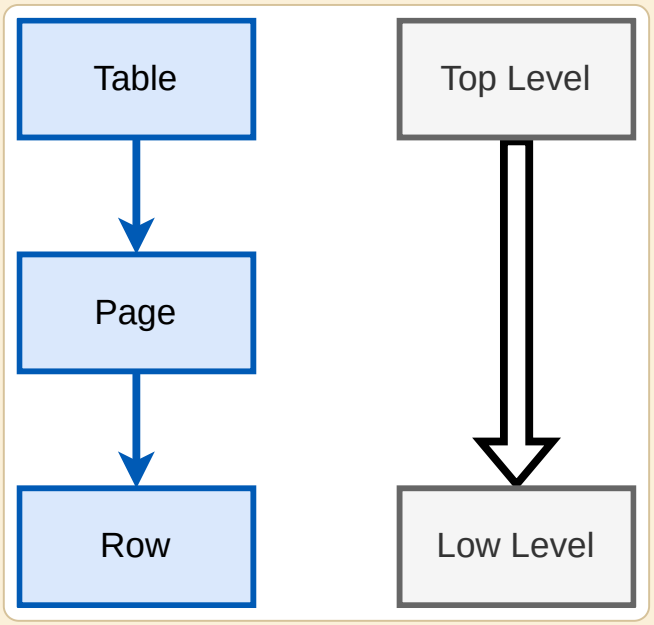


Figure — locks can be taken on a whole table, a page or a single row (course diagram).

Granularity	Advantage	Disadvantage	Example
Row	Maximum concurrency — other rows stay free	Many locks to manage (memory, CPU)	Update one customer's row
Page	Middle ground	Locks unrelated rows that share the page	Some storage engines (older SQL Server, DB2)
Table	Simple, very low overhead	Blocks everyone else	Batch update of the whole `orders` table

Some engines **escalate**: when a transaction holds thousands of row locks, they are swapped for one table lock (SQL Server does this). PostgreSQL never escalates row locks because it stores them in the rows themselves.

Intention locks — why they exist

T1 holds an X lock on row 5 of `employees`; T2 asks for an S lock on the **whole table**. Without extra information the lock manager would have to scan every row lock for conflicts. **Intention locks** make locking *hierarchical*:

1. T1 first places **IX** on the `employees` table, then **X** on row 5.
2. T2 asks for **S** on the table. The manager only checks table-level locks: S conflicts with IX, so T2 waits — no row scan needed.
3. A T3 wanting X on row 9 places IX on the table (IX is compatible with IX) and proceeds in parallel with T1.

Compatibility (can the requested lock be granted while another transaction holds…?):

Held → / Requested ↓	IS	IX	S	X
IS	yes	yes	yes	no
IX	yes	yes	no	no
S	yes	no	yes	no
X	no	no	no	no

Two-phase locking (2PL)

2PL is the classic protocol that guarantees conflict-serializability, in two phases:

1. **Growing (expanding) phase** — it may acquire locks but not release any.
2. **Shrinking phase** — once it releases its first lock, it may not acquire any new one.

The moment it holds the most locks is the **lock point**; transactions are serialized in the order of their lock points.

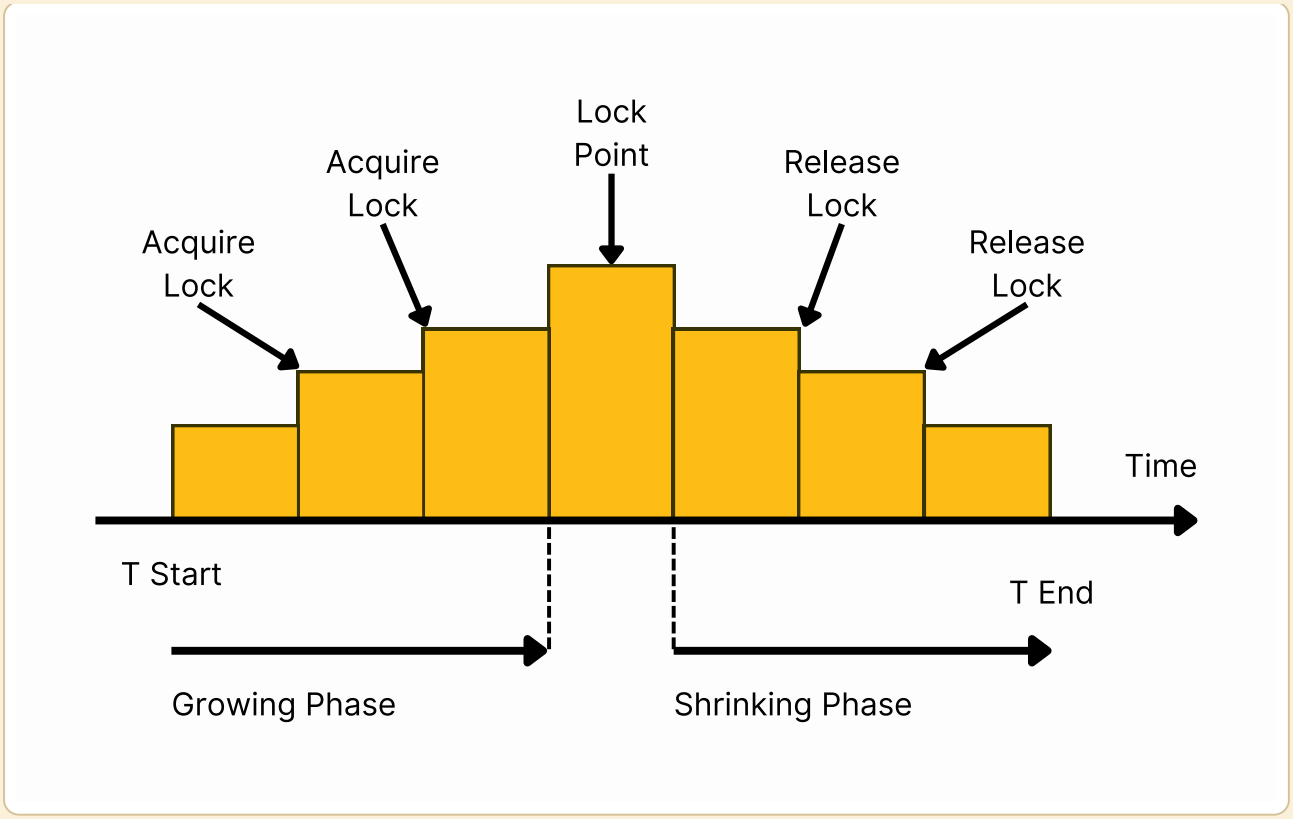


Figure — the number of locks held rises, peaks at the lock point, then falls (course diagram).

Variants you should know:

Variant	Rule	Why
Basic 2PL	grow, then shrink	Serializable, but a transaction could read data from another that later aborts (*cascading aborts*)
Strict 2PL	hold all **X** locks until commit/abort	No one reads uncommitted data; what most lock-based databases do
Rigorous 2PL	hold **all** locks until commit/abort	Simplest to reason about; serial order = commit order
Conservative 2PL	take every lock before starting	No deadlocks, but you must know the data set in advance

Pros: guaranteed serializability. *Cons:* **deadlocks** (§13.11) and lower throughput (work completed per second) under contention — a long reader blocks writers and vice versa. MySQL and SQL Server implement Serializable with (strict) 2PL; it suits strong-isolation domains such as finance.

Predicate locking and index-range locking

2PL on existing rows can't stop **phantoms**: a new row that *would* match your query didn't exist when you locked. Two answers:

Predicate locking extends 2PL by locking the **condition** itself — every row, existing or future, that satisfies `WHERE …`. Example: table `seats` holds seat numbers 10 and 30; transaction T runs `UPDATE seats SET held = true WHERE seat_no BETWEEN 10 AND 30`.

Seat	Exists now?	Locked by predicate lock?
10	yes	yes
20	no	yes — an insert of seat 20 must wait
30	yes	yes

It prevents phantoms and write skew, but checking every lock request against every active predicate is expensive, and it over-locks.

Index-range locking (*next-key* or *gap* locking) approximates the predicate with ranges of an **index**: it locks the matching index entries *and the gaps between them*, so an insert of seat 20 must wait. (Locking only the existing keys 10 and 30 would let seat 20 in — phantoms would remain; covering the gap is the whole point.) It is cheaper than predicate locking because it reuses the index, at the price of sometimes locking a slightly wider range; without a suitable index the engine locks far more (up to the whole table). MySQL InnoDB's next-key locks at Repeatable Read work this way.

Aspect	2PL on rows	Predicate locking	Index-range (next-key) locking
What is locked	Rows actually touched	Every row matching the condition, even future ones	Matching index entries plus gaps between them
Phantoms prevented?	No	Yes	Yes (needs an index on the condition)
Seen in	All lock-based engines	Theory; PostgreSQL SSI's SIREAD locks are a non-blocking cousin	MySQL InnoDB, SQL Server key-range locks

PostgreSQL's locks in practice

PostgreSQL has eight **table-level lock modes** (taken automatically by statements) and four **row-level** modes. `code/04_lock_modes.sql` shows what a transaction holds:

```
BEGIN;
SELECT * FROM accounts WHERE id = 1;                          -- plain read
UPDATE products SET stock = stock - 1 WHERE id = 1;           -- write
SELECT * FROM on_call WHERE doctor = 'dr_rao' FOR SHARE;      -- shared row lock
LOCK TABLE jobs IN SHARE MODE;                                -- explicit table lock
SELECT c.relname, l.mode, l.granted
FROM pg_locks l JOIN pg_class c ON c.oid = l.relation
WHERE l.pid = pg_backend_pid() AND c.relnamespace = 'public'::regnamespace
ORDER BY c.relname, l.mode;
COMMIT;

SELECT pg_try_advisory_lock(42) AS got_it_first_time;
SELECT count(*) AS advisory_locks_held FROM pg_locks WHERE locktype = 'advisory' AND pid =
pg_backend_pid();
SELECT pg_advisory_unlock(42) AS released;
```

Output (row echoes trimmed; the three advisory-lock results condensed onto one line):

```
    relname    |       mode       | granted
---------------+------------------+---------
 accounts      | AccessShareLock  | t
 accounts_pkey | AccessShareLock  | t
 jobs          | ShareLock        | t
 on_call       | RowShareLock     | t
 on_call_pkey  | RowShareLock     | t
 products      | RowExclusiveLock | t
 products_pkey | RowExclusiveLock | t
(7 rows)
...
 got_it_first_time → t     advisory_locks_held → 1     released → t
```

These table modes play the role of intention locks: `RowExclusiveLock` (taken by `UPDATE/INSERT/DELETE`) means "I'm changing rows in here"; it doesn't conflict with other writers but does conflict with `SHARE` and stronger modes:

Common statement	Table lock taken	Blocks
`SELECT`	ACCESS SHARE	only ACCESS EXCLUSIVE (`DROP`, most `ALTER TABLE`)
`SELECT … FOR UPDATE/SHARE`	ROW SHARE	EXCLUSIVE, ACCESS EXCLUSIVE
`INSERT/UPDATE/DELETE`	ROW EXCLUSIVE	SHARE and stronger
`CREATE INDEX`	SHARE	writers
`CREATE INDEX CONCURRENTLY`, `VACUUM`	SHARE UPDATE EXCLUSIVE	other schema changes / vacuums
most `ALTER TABLE`, `DROP`, `TRUNCATE`, `VACUUM FULL`	ACCESS EXCLUSIVE	everything, even `SELECT`

Row-level lock modes, strongest first: `FOR UPDATE` (taken by `DELETE` or an update of key columns), `FOR NO KEY UPDATE` (a normal `UPDATE`), `FOR SHARE` (read, block changes), `FOR KEY SHARE` (foreign-key checks; blocks only key changes/deletes). **Advisory locks** (`pg_advisory_lock(n)`) lock a number that only your application gives meaning to — e.g., "only one nightly job may run".

Common mistakes

- An `ALTER TABLE` on a busy table waits for ACCESS EXCLUSIVE — and **every** later query queues behind it. Use `SET lock_timeout = '2s'` for migrations (scripted schema changes).
- Holding locks during slow work (HTTP calls, user input).
- Relying on the absence of a row without a unique constraint or Serializable (phantoms).

In the interview

> **30-second answer:** "S locks for reads, X for writes; only S is compatible with S. Finer granularity means more concurrency and more overhead; intention locks let the manager check table-versus-row conflicts without scanning rows. 2PL (grow then shrink) guarantees serializability, strict 2PL keeps write locks to commit, and next-key locks stop phantoms."

- *Why does strict 2PL hold X locks until commit?* So nobody reads uncommitted data and cascading aborts can't happen.
- *How does Postgres avoid lock-table explosion for row locks?* Row locks are stored in the tuple header (`xmax` + infomask, a field of status bits); waiters wait on the holder's transaction ID.

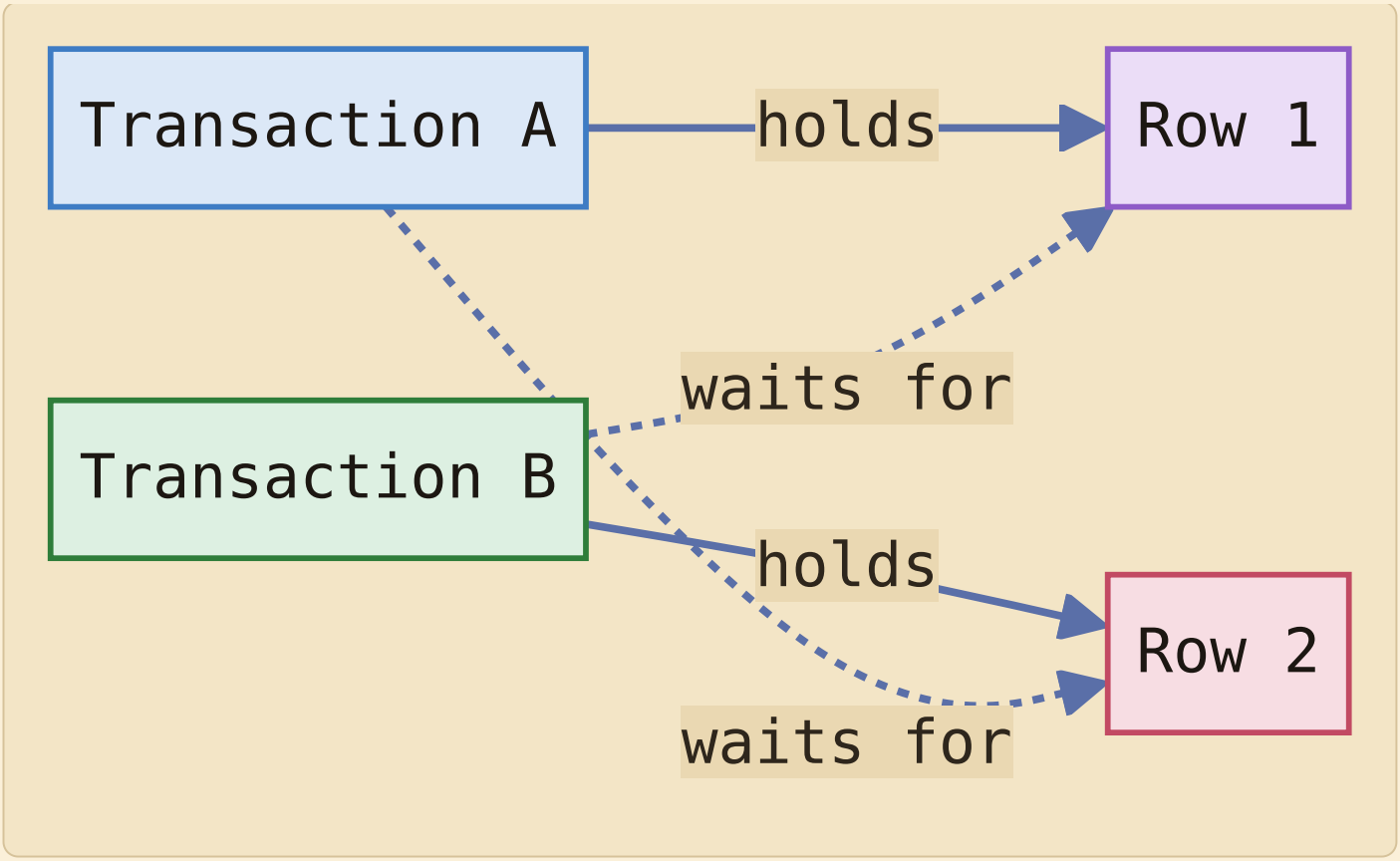
Self-check

1. Can two transactions hold S on the same row? X? → S yes; X no.
2. What does IX on a table announce? → The transaction will take X locks on some rows in it.
3. How does index-range locking stop phantoms? → It locks the gaps between index keys, so matching inserts wait.

13.11 Deadlocks

What it is

A **deadlock** is a cycle of waiting: T1 holds resource 1 and waits for resource 2, while T2 holds 2 and waits for 1. Neither can ever proceed unless something intervenes.



How databases handle it

Strategy	How	Trade-off
Detection	Build a *wait-for graph* (edge T1 → T2 when T1 waits for T2); a cycle is a deadlock; abort one victim	Some work lost; used by PostgreSQL, MySQL, SQL Server, Oracle
Timeout	Abort any transaction that waits too long	Simple, but may abort non-deadlocked waiters or react slowly (`lock_timeout`, InnoDB `innodb_lock_wait_timeout`)
Prevention by ordering	Always lock resources in the same global order (e.g., lower `id` first)	Needs discipline in application code — the most practical fix
Wait-die / wound-wait	By start timestamp: in *wait-die* an older requester waits, a younger one aborts ("dies"); in *wound-wait* an older requester aborts ("wounds") the younger holder, a younger one waits	No cycles; more aborts; some distributed databases
Conservative 2PL	Take all locks up front	Must know everything in advance

Seeing PostgreSQL detect one

Two transfers in opposite directions: A moves 10 from account 1 to 2; B moves 10 from 2 to 1. Each locks its "from" row first.

```
=== Deadlock: PostgreSQL detects it and aborts one transaction ===
  A| SET deadlock_timeout = '10s'  -> SET
  B| SET deadlock_timeout = '200ms'  -> SET
  A| BEGIN  -> BEGIN
  B| BEGIN  -> BEGIN
  A| UPDATE accounts SET balance = balance - 10 WHERE id = 1  -> UPDATE 1
  B| UPDATE accounts SET balance = balance - 10 WHERE id = 2  -> UPDATE 1
  A| UPDATE accounts SET balance = balance + 10 WHERE id = 2
     .. A is WAITING (wait_event=transactionid, blocked by B)
  B| UPDATE accounts SET balance = balance + 10 WHERE id = 1  -> ERROR 40P01: deadlock detected
     DETAIL: Process <B> waits for ShareLock on transaction <xid>; blocked by process <A>.
             Process <A> waits for ShareLock on transaction <xid>; blocked by process <B>.
  B| ROLLBACK  -> ROLLBACK
     -> (A resumes) UPDATE 1
  A| COMMIT  -> COMMIT
  M| SELECT id, balance FROM accounts WHERE id IN (1, 2) ORDER BY id  -> (1, 490.00), (2, 310.00)
```

How PostgreSQL does it:

1. A waiting backend (the server process serving one connection) first sleeps for `deadlock_timeout` (default **1 s**) — most waits end by themselves, and the check is costly.
2. Then it inspects the wait-for graph. B's timer fired first (the demo set 200 ms for B, 10 s for A to make the victim predictable); it found the cycle A → B → A and aborted **B** with SQLSTATE **40P01**.
3. B's locks are released, A's update completes and commits; the application retries B.

The fix for this pattern is ordering: always lock the lower account id first (`SELECT … FROM accounts WHERE id IN (1, 2) ORDER BY id FOR UPDATE`), so both transfers queue for row 1 and no cycle can form.

Common mistakes

- Not retrying on 40P01 (or 40001) — both mean "try the whole transaction again".
- Locking rows in input order (an unsorted `for item in cart` loop) — sort by primary key first.

In the interview

30-second answer: "A cycle in the wait-for graph. Databases detect it (Postgres after `deadlock_timeout`, 1 s by default) and abort a victim, which the app retries. Prevent by locking in a consistent order and keeping transactions short; textbook alternatives are wait-die and wound-wait."

Self-check

1. What graph is used for detection? → The wait-for graph between transactions.
2. PostgreSQL's deadlock SQLSTATE? → 40P01.
3. Simplest prevention in application code? → Acquire locks in a fixed global order.

13.12 Row locks for work queues: `FOR UPDATE`, `NOWAIT`, `SKIP LOCKED`

The problem it solves

Many apps keep a **job table** (emails to send, images to resize) served by several workers (processes that pick up and run jobs), each of which must grab jobs **no other worker** holds. With plain `FOR UPDATE` all workers lock the *same* first rows and queue behind each other.

The options

Clause	If the row is locked by someone else…
`FOR UPDATE`	wait until they finish
`FOR UPDATE NOWAIT`	fail immediately with error 55P03
`FOR UPDATE SKIP LOCKED`	silently skip that row and take the next unlocked one

`SKIP LOCKED` exists in PostgreSQL (9.5+), MySQL (8.0+) and Oracle; SQL Server's `READPAST` hint does the same.

Seeing it work

```
=== Job queue: FOR UPDATE SKIP LOCKED (and NOWAIT) ===
  A| BEGIN  -> BEGIN
  A| SELECT id, payload FROM jobs WHERE status = 'queued' ORDER BY id LIMIT 2 FOR UPDATE SKIP LOCKED  ->
(1, email #1), (2, email #2)
  B| BEGIN  -> BEGIN
  B| SELECT id, payload FROM jobs WHERE status = 'queued' ORDER BY id LIMIT 2 FOR UPDATE SKIP LOCKED  ->
(3, email #3), (4, email #4)
  B| SELECT id FROM jobs WHERE id = 1 FOR UPDATE NOWAIT  -> ERROR 55P03: could not obtain lock on row in
relation "jobs"
  B| ROLLBACK  -> ROLLBACK
  B| UPDATE jobs SET status = 'running', claimed_by = 'B' WHERE id IN (SELECT id FROM jobs WHERE status =
'queued' ORDER BY id LIMIT 2 FOR UPDATE SKIP LOCKED) RETURNING id  -> (3), (4)
  A| UPDATE jobs SET status = 'done', claimed_by = 'A' WHERE id IN (1, 2)  -> UPDATE 2
  A| COMMIT  -> COMMIT
  M| SELECT id, status, claimed_by FROM jobs ORDER BY id  -> (1, done, A), (2, done, A), (3, running, B),
(4, running, B), (5, queued, None), (6, queued, None)
```

Worker A locked jobs 1–2; worker B, running the identical query, got 3–4 **without waiting**, and `NOWAIT` on job 1 failed at once. The usual form **claims and marks in one statement**:

```sql
UPDATE jobs SET status = 'running', claimed_by = $1
WHERE id IN (SELECT id FROM jobs
             WHERE status = 'queued'
             ORDER BY id
             LIMIT 10
             FOR UPDATE SKIP LOCKED)
RETURNING id, payload;
```

Two designs: (a) keep the transaction open while processing — a crashed worker's rollback releases its jobs automatically; (b) mark `running` and commit at once, plus a `claimed_at` column and a sweeper (a periodic clean-up job) that re-queues stuck jobs. A partial index `CREATE INDEX ON jobs (id) WHERE status = 'queued'` keeps the claim fast (chapter 12).

When not to use it

`SKIP LOCKED` deliberately hides locked rows, so never use it for reports or business-rule checks. For very high throughput, fan-out (delivering one message to many consumers) or replay (re-reading old messages), use a message broker — a dedicated queueing service such as Kafka, RabbitMQ or SQS (System Design Fundamentals); a Postgres queue shines when jobs must be created in the same transaction as the business data.

Real-world examples

Job-queue libraries such as GoodJob (Ruby), Oban (Elixir), graphile-worker (Node) and Procrastinate (Python) use this pattern; ticketing systems use it to hand concurrent buyers different free seats.

13.13 Timestamp-based concurrency control

What it is

Timestamp ordering (TO) gives every transaction a unique, increasing **timestamp** TS(T) at start (clock or counter). The result must equal running the transactions **in timestamp order**, older first; instead of waiting for locks, a transaction that would break the order is **aborted and restarted** with a new timestamp. The result is always serializable.

Key bookkeeping

Symbol	Kept for	Meaning
TS(T)	each transaction	its start timestamp — its place in the serial order
RTS(X)	each data item X	the largest timestamp of any transaction that has **read** X
WTS(X)	each data item X	the largest timestamp of any transaction that has **written** X

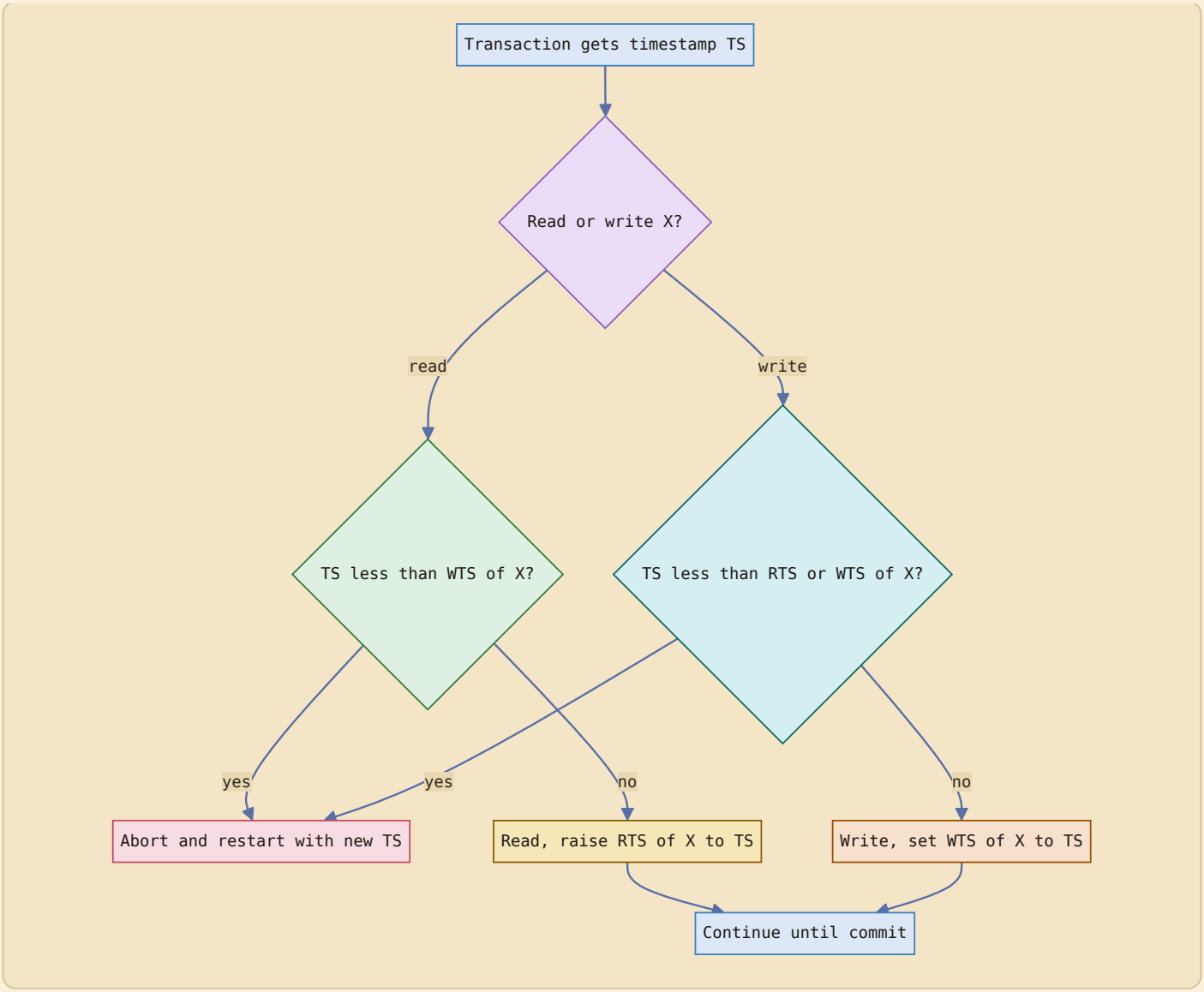
How it works — the rules

When T wants to read X:

1. If TS(T) < WTS(X): a *younger* transaction already overwrote X — T would read a value "from its future". **Abort T** (restart with a new timestamp).
2. Otherwise read X and set RTS(X) = max(RTS(X), TS(T)).

When T wants to write X:

1. If TS(T) < RTS(X): a younger transaction already read the old value. **Abort T.**
2. If TS(T) < WTS(X): a younger transaction already wrote X, so T's write is outdated. **Abort T** — or, under the **Thomas write rule**, **skip** the obsolete write (timestamp order would have overwritten it anyway).
3. Otherwise write X and set WTS(X) = TS(T).



Worked example — booking the last seats

Item **S** = seats left for a show (initially RTS = 0, WTS = 0). Priya's booking transaction starts first and gets TS = 5; Omar's starts a moment later with TS = 7.

Step	Operation	Check	Result	RTS(S)	WTS(S)
1	T5 reads S	5 ≥ WTS 0	allowed	5	0
2	T7 reads S	7 ≥ WTS 0	allowed	7	0
3	T7 writes S (books a seat)	7 ≥ RTS 7 and ≥ WTS 0	allowed	7	7
4	T5 writes S (books a seat)	5 < RTS 7	**T5 aborted**	7	7
5	T5 restarts as T9, reads S	9 ≥ WTS 7	allowed, sees Omar's booking	9	7

Priya's first attempt relied on a count that a "later" transaction had already read and changed, so it was rolled back instead of double-selling; the retry sees the current count.

Trade-offs

Advantages	Disadvantages
No deadlocks — nobody waits for a lock	**Many rollbacks** under contention; work is thrown away
Guarantees **serializability** by construction	**Starvation** — a long transaction may be restarted again and again because younger ones keep touching its data
Simple conflict rule: compare two numbers	**Overhead** — every read updates RTS (reads become metadata writes — writes of bookkeeping data); extra care is needed to avoid reading uncommitted values

Where it is used

Pure TO is mostly a textbook protocol. Its descendant **multiversion timestamp ordering** (versions tagged with timestamps; each read picks the right one) underlies many MVCC systems, and distributed databases rely on timestamps: Google Spanner uses TrueTime (GPS- and atomic-clock-backed time with a known error bound), CockroachDB hybrid logical clocks (physical time combined with a counter) plus MVCC.

In the interview

> **30-second answer:** "Transactions must behave as if run in start-timestamp order. Each item keeps its largest read and write timestamps; a read is rejected if a younger transaction already wrote the item, a write if a younger one already read or wrote it, and the loser restarts with a new timestamp. No deadlocks, but many aborts and possible starvation; the Thomas write rule skips obsolete writes."

Self-check

1. T with TS 4 wants to write X where RTS(X) = 6. → Abort T (a younger transaction read X).
2. What does the Thomas write rule change? → An outdated write (TS < WTS, but ≥ RTS) is ignored instead of causing an abort.
3. Why can't TO deadlock? → Transactions never wait; conflicts cause aborts.

13.14 Multiversion concurrency control (MVCC)

What it is

MVCC keeps **several versions** of each row. A write creates a **new version** and marks the old one as replaced; each transaction reads from a **snapshot** — the versions committed as of a certain moment. So **readers and writers never block each other** (writers still block writers on the same row), and every transaction sees a consistent picture.

Analogy: a shared document with full version history. Editing creates a new revision; someone reading "as of 10:00" keeps reading that revision undisturbed.

How it works

1. **Versions carry metadata:** the transaction that **created** it and the one that **deleted/replaced** it (empty if current).
2. **Snapshots:** when a transaction (Repeatable Read) or statement (Read Committed) starts, the database notes which transactions had committed.
3. **Visibility:** a version is visible if its creator committed before the snapshot and its deleter is empty, aborted, or not committed as of the snapshot.
4. **Writes** create new versions; concurrent writers of one row are serialized by a row lock (§13.7), and at Repeatable Read/Serializable a concurrent committed change raises a serialization error.

5. **Cleanup:** versions no snapshot can see are garbage (PostgreSQL's `VACUUM` cleanup command, InnoDB's purge thread — its background cleaner).

Worked example — versions of an account

Account A starts at 500 (created by transaction 1); account B at 300. Transaction 2 adds 200 to A; later, transaction 3 subtracts 100 from A.

After	Versions of account A (created_by, deleted_by, balance)	Account B
txn 1	(1, –, **500**)	(1, –, 300)
txn 2	(1, **2**, 500) · (2, –, **700**)	unchanged
txn 3	(1, 2, 500) · (2, **3**, 700) · (3, –, **600**)	unchanged

A reader whose snapshot predates txn 2's commit still sees 500; one taken between txn 2 and txn 3 sees 700; new readers see 600 — simultaneously, without locks.

MVCC in PostgreSQL: `xmin` and `xmax` for real

PostgreSQL stores the "created by" and "deleted by" fields in every row version as hidden system columns: `xmin` (inserting transaction ID) and `xmax` (deleting/locking transaction ID, 0 if none). `ctid` is the version's physical address (page, slot).

One session — `code/03_mvcc_xmin_xmax.sql` repeats the worked example (500 → +200 → −100) and then uses the `pageinspect` extension to look *inside* the table's first 8 KB page:

```
CREATE EXTENSION IF NOT EXISTS pageinspect;
CREATE TABLE wallet (id int PRIMARY KEY, balance int);
INSERT INTO wallet VALUES (1, 500), (2, 300);
SELECT ctid, xmin, xmax, id, balance FROM wallet ORDER BY id;

UPDATE wallet SET balance = balance + 200 WHERE id = 1;
UPDATE wallet SET balance = balance - 100 WHERE id = 1;
SELECT ctid, xmin, xmax, id, balance FROM wallet ORDER BY id;

SELECT lp AS slot, t_xmin, t_xmax, t_ctid
FROM heap_page_items(get_raw_page('wallet', 0)) ORDER BY lp;

BEGIN;
UPDATE wallet SET balance = 0 WHERE id = 2;
SELECT txid_current() AS my_txid;
ROLLBACK;
SELECT ctid, xmin, xmax, id, balance FROM wallet WHERE id = 2;

VACUUM wallet;
SELECT lp AS slot, lp_flags, t_xmin, t_xmax, t_ctid
FROM heap_page_items(get_raw_page('wallet', 0)) ORDER BY lp;
```

Output (transaction IDs differ on every run; command tags trimmed):

```
 ctid  | xmin | xmax | id | balance
-------+------+------+----+---------
 (0,1) |  802 |    0 |  1 |     500
 (0,2) |  802 |    0 |  2 |     300
(2 rows)

UPDATE 1
UPDATE 1
 ctid  | xmin | xmax | id | balance
-------+------+------+----+---------
 (0,4) |  804 |    0 |  1 |     600
 (0,2) |  802 |    0 |  2 |     300
(2 rows)

 slot | t_xmin | t_xmax | t_ctid
------+--------+--------+--------
    1 |    802 |    803 | (0,3)
    2 |    802 |      0 | (0,2)
    3 |    803 |    804 | (0,4)
    4 |    804 |      0 | (0,4)
(4 rows)

BEGIN
UPDATE 1
 my_txid
---------
     805
(1 row)

ROLLBACK
 ctid  | xmin | xmax | id | balance
-------+------+------+----+---------
 (0,2) |  802 |  805 |  2 |     300
(1 row)

VACUUM
 slot | lp_flags | t_xmin | t_xmax | t_ctid
------+----------+--------+--------+--------
    1 |        2 |        |        |
    2 |        1 |    802 |    805 | (0,5)
    3 |        0 |        |        |
    4 |        1 |    804 |      0 | (0,4)
    5 |        0 |        |        |
(5 rows)
```

Reading it:

1. Transaction 802 inserted both rows (`xmax = 0`: not replaced).
2. After the updates, `SELECT` shows only the current version of row 1, at a **new address** `(0,4)` with `xmin = 804`: an update is "insert a new version + mark the old one".
3. The raw page still holds **all three versions** of row 1 — the worked example exactly: slot 1 (500, created by 802, replaced by 803), slot 3 (700, 803 → 804), slot 4 (600, current). Each old version's `t_ctid` points to its successor.
4. The rolled-back update left `xmax = 805` on row 2, yet the row is visible with 300: transaction 805 is marked **aborted** in the commit log (`pg_xact`), so its `xmax` is ignored. PostgreSQL rolls back instantly by marking, not undoing.
5. `VACUUM` removed the dead versions: slots 3 and 5 (the aborted version) are unused (`lp_flags = 0`) and slot 1 became a **redirect** (`lp_flags = 2`) to slot 4 — a *HOT* (heap-only tuple) chain, so the index can keep pointing at slot 1.

Two sessions — the `mvcc` scenario of the driver shows what *another* transaction sees while an update is uncommitted:

```
=== MVCC: xmin / xmax seen from two sessions ===
  A| SELECT xmin, xmax, balance FROM accounts WHERE id = 1  -> (1697, 0, 500.00)
  A| BEGIN  -> BEGIN
  A| UPDATE accounts SET balance = 450 WHERE id = 1  -> UPDATE 1
  A| SELECT txid_current()  -> 1698
  A| SELECT xmin, xmax, balance FROM accounts WHERE id = 1  -> (1698, 0, 450.00)
  B| SELECT xmin, xmax, balance FROM accounts WHERE id = 1  -> (1697, 1698, 500.00)
  A| COMMIT  -> COMMIT
  B| SELECT xmin, xmax, balance FROM accounts WHERE id = 1  -> (1698, 0, 450.00)
```

While A's transaction 1698 is open, **two versions** exist: A sees its new one (450); B sees the **old** one (500) with `xmax = 1698` — "being replaced by an uncommitted transaction, so still visible to me". After A commits, B's next statement sees 450. No one waited.

Trade-offs

Advantages	Costs
Readers and writers don't block each other	Extra storage for old versions; tables and indexes can **bloat** (grow with wasted space from dead versions)
Consistent snapshots for reports and backups (`pg_dump` uses one)	Needs garbage collection (autovacuum — the background process that runs `VACUUM` automatically — must keep up)
Cheap rollback in PostgreSQL (mark aborted)	Long-running transactions keep old versions alive for everyone
Basis for snapshot isolation and SSI	Snapshot isolation alone still allows write skew (§13.8)

Two designs. PostgreSQL keeps every version **in the table** (vacuum later). MySQL InnoDB and Oracle update **in place** and rebuild older versions from an **undo log** — less bloat, more work for long readers ("snapshot too old" in Oracle).

Common mistakes

- Leaving a session "idle in transaction" for hours: vacuum can't remove versions newer than its snapshot, so tables bloat. Set `idle_in_transaction_session_timeout`.
- Disabling autovacuum: besides bloat, PostgreSQL's 32-bit transaction IDs must be "frozen" (marked as old enough to be visible to everyone) by vacuum before they wrap around — run out and restart from zero (~2 billion transactions) — or the server stops accepting writes.
- Thinking MVCC means "no locks at all" — writes to the same row still lock and wait.

Real-world examples

Shoppers browse while prices change, customers view balances during batch posting, and `pg_dump` takes a coherent backup while the database keeps running — each of these works because readers get snapshots.

In the interview

30-second answer: "Keep multiple row versions tagged with the writing transaction; readers pick the version visible to their snapshot, so readers and writers don't block each other. In Postgres an update inserts a tuple and sets xmax on the old one; VACUUM removes dead tuples. Costs: bloat, garbage collection, and long transactions hurt."

- *Non-zero `xmax` on a visible row?* Its deleter/updater hasn't committed or aborted — or it's only a row lock.

Self-check

1. What do `xmin` and `xmax` record? → The creating transaction and the deleting/replacing (or locking) transaction.
2. Why did the updated row get a new `ctid`? → An UPDATE writes a new tuple version elsewhere in the page.
3. What makes a rolled-back change invisible in PostgreSQL? → The transaction is marked aborted in the commit log; its versions are ignored.

13.15 Putting it together

A practical checklist

1. Keep transactions **short** — never open across user think time or slow network calls.
2. Prefer **atomic statements** (`SET n = n - 1 WHERE n > 0`) over read-then-write in code; otherwise `FOR UPDATE`, or a **version column** for long gaps.
3. Declare every invariant you can as a **constraint** (unique, check, foreign key, exclusion); for cross-row rules use **Serializable + retry** or lock the rows the decision depends on.
4. **Retry** the whole transaction on SQLSTATE 40001 and 40P01, with backoff (waiting a little longer before each new attempt) and a limit.
5. Lock rows in a **consistent order**; set `lock_timeout` for migrations and `idle_in_transaction_session_timeout` to protect vacuum.
6. For consistent multi-query reports use `REPEATABLE READ READ ONLY`.

Deeper theory: DDIA ch. 7 notes. Thread-level locking in application code: LLD (low-level design) handbook concurrency concepts and concurrency problems. Distributed transactions and replication: chapter 15; transactions from Python/Java: chapter 14. A one-page summary of isolation levels, anomalies and lock choices is in chapter 20, §20.12.

Chapter TL;DR

- A **transaction** is all-or-nothing; write it with `BEGIN … COMMIT/ROLLBACK`, using savepoints for partial undo.
- **ACID:** atomicity (logs/shadow copies), consistency (constraints + code), isolation (locks/MVCC/SSI), durability (WAL + fsync + checkpoints + replication).
- **Anomalies:** dirty read, dirty write, non-repeatable read/read skew, phantom, lost update, write skew. PostgreSQL never allows dirty reads/writes; Read Committed allows the next four; Repeatable Read (snapshot isolation) stops all but write skew; Serializable (SSI) stops everything by aborting with 40001.
- **Lost updates:** prefer atomic SQL updates; otherwise `FOR UPDATE`, Repeatable Read + retry, or a version column (CAS) for long think times.
- **Concurrency control:** pessimistic (S/X/intention locks, 2PL, predicate/next-key locks) vs optimistic (validate at commit) vs timestamp ordering vs **MVCC** (PostgreSQL's `xmin`/`xmax`, cleaned by VACUUM).
- **Deadlocks** are detected (40P01) and prevented by consistent lock ordering; **retry** 40001/40P01.
- `FOR UPDATE SKIP LOCKED` turns a table into a safe multi-worker job queue.

Next: Chapter 14 — Advanced SQL and using SQL from code, which runs these transactions from Python and Java. For quick revision, see the transactions sheet in Chapter 20.

Chapter 14 — Advanced SQL and Using SQL from Code

So far every query in this book has been typed into `psql` (PostgreSQL's command-line client) by a person. Real systems are different: the database runs **code of its own** (functions, procedures, triggers), stores **semi-structured data** — data whose shape varies from row to row (JSON, a text format for nested data; arrays; searchable text) — protects data **per user and per row**, and — most of all — is driven by **application programs** written in Python, Java and other languages. This chapter covers those "last mile" skills.

What you'll learn

- Writing functions and stored procedures in SQL and PL/pgSQL, and when logic belongs in the database.
- Triggers: automatic `updated_at` columns, validation rules, a generic audit log, statement-level triggers.
- JSON/JSONB and arrays: storing, querying, updating and indexing them.
- Full-text search with `tsvector` / `tsquery`, ranking and highlighting.
- Pivoting rows into columns (and back) with `FILTER`, `CASE`, `crosstab` and `LATERAL`.
- Security: roles, `GRANT` / `REVOKE`, column privileges and row-level security (RLS).
- SQL injection — seeing it succeed, then fixing it with parameters.
- Calling the database from Python (`psycopg`, `sqlite3`) and Java (JDBC): parameters, transactions, batches, streaming.
- Connection pooling, and the trade-offs between ORMs and hand-written SQL.

How to run the examples. Everything is in `code/` and runs on the book's private PostgreSQL 14 server (port 5544). The shared sample database from Chapter 1 — a small office-gear company with `departments`, `employees`, `customers`, `products`, `orders` and `order_items` — is loaded first from `code/00_schema.sql`; each numbered file then runs in a fresh database.

```
bash tools_run_sql.sh sql '14-*'                                    # all SQL files of this chapter
.venv-books/bin/python sql/chapters/14-advanced-sql-and-using-sql-from-code/code/psycopg_demo.py
python3 sql/chapters/14-advanced-sql-and-using-sql-from-code/code/sqlite3_demo.py
java --enable-native-access=ALL-UNNAMED -cp '.java-books/lib/*' \
     sql/chapters/14-advanced-sql-and-using-sql-from-code/code/JdbcDemo.java
```

14.1 User-defined functions

What it is

A **user-defined function** (UDF) is a named piece of logic stored *inside* the database that takes arguments and returns a value (or a set of rows). You call it from SQL exactly like a built-in function such as `upper()` or `round()`. Think of it as a spreadsheet formula you give a name, so everyone uses the same one.

PostgreSQL lets you write the body in several **procedural languages**:

Language	What the body looks like	Use it for
`sql`	One or more plain SQL statements; the last one's result is returned	Short calculations and reusable queries; the planner (the part of the database that decides how to run a query) can often *inline* them — paste their body into the calling query
`plpgsql`	PL/pgSQL: SQL plus variables, `IF`, loops, error handling	Logic with branches, loops, or several steps
`plpython3u`, `plperl`, …	Other languages (extensions, often "untrusted" = superuser — an all-powerful admin role — only)	Rare; needs the extension installed

Why it matters

- **One definition of a rule.** "Line total = quantity × price × (1 − discount)" written once, used by every report and every application.
- **Fewer round trips** (a request to the server and its reply). A multi-step operation runs next to the data instead of sending many statements over the network.
- **Reusable query building blocks.** A function that returns a table can be joined like a table.

How it works

1. `CREATE FUNCTION name(args) RETURNS type LANGUAGE … AS $$ body $$` stores the function in the catalog (PostgreSQL's internal tables describing every object; here `pg_proc`).
2. `$$ … $$` is **dollar quoting**: a way to write a string literal without escaping (marking as "literal, not special") the single quotes inside the body.
3. Each argument can have a `DEFAULT`; callers may pass arguments by position or by name (`p_limit => 1`).
4. A **volatility** label tells the planner how the function behaves — it affects optimisation and whether the function may be used in an index:

Label	Promise	Example
`IMMUTABLE`	Same inputs → same result forever; reads no tables	`line_total(2, 10, 0)`
`STABLE`	Same result within one statement; may read tables	"tier of customer 3"
`VOLATILE` (default)	Anything may change; may modify data	`random()`, a function that inserts rows

Only `IMMUTABLE` functions can be used in an index expression. Lying (marking a table-reading function `IMMUTABLE`) gives wrong, cached-looking results.

Example — SQL and PL/pgSQL functions

From `code/01_functions.sql`:

```sql
-- (a) A plain SQL function, SQL-standard body (PostgreSQL 14+)
CREATE FUNCTION line_total(qty integer, price numeric, disc numeric DEFAULT 0)
RETURNS numeric
LANGUAGE sql IMMUTABLE
RETURN round(qty * price * (1 - disc), 2);

-- (b) PL/pgSQL: variables, IF, SELECT ... INTO, RAISE
CREATE FUNCTION customer_tier(p_customer_id integer)
RETURNS text
LANGUAGE plpgsql STABLE
AS $$
DECLARE
    v_spent numeric;
BEGIN
    IF NOT EXISTS (SELECT 1 FROM customers WHERE customer_id = p_customer_id) THEN
        RAISE EXCEPTION 'customer % does not exist', p_customer_id
              USING ERRCODE = 'no_data_found';
    END IF;

    SELECT coalesce(sum(line_total(oi.quantity, oi.unit_price, oi.discount)), 0)
    INTO   v_spent
    FROM   orders o JOIN order_items oi USING (order_id)
    WHERE  o.customer_id = p_customer_id AND o.status <> 'cancelled';

    RETURN CASE WHEN v_spent >= 1500 THEN 'gold'
                WHEN v_spent >= 300  THEN 'silver'
                ELSE 'bronze' END;
END;
$$;

SELECT customer_id, first_name, customer_tier(customer_id) AS tier
FROM   customers ORDER BY customer_id LIMIT 5;
```

Output:

```
 customer_id | first_name |  tier
-------------+------------+--------
           1 | Anita      | gold
           2 | Ben        | gold
           3 | Chen       | bronze
           4 | Divya      | bronze
           5 | Ethan      | silver
```

What each part does: `DECLARE` lists local variables; `SELECT … INTO v_spent` stores a query result in a variable; `RAISE EXCEPTION … USING ERRCODE` aborts with a chosen SQLSTATE code (the standard five-character error code); `%` in the message is replaced by the next argument. `RAISE NOTICE` (used below) only prints a message.

Catching errors. A `BEGIN … EXCEPTION WHEN no_data_found THEN … END` block catches errors raised inside it (`SQLERRM` holds the message): the file's `safe_tier(999)` returns `unknown (customer 999 does not exist)` instead of failing. An `EXCEPTION` block silently creates a **savepoint** (a marker inside a transaction that you can roll back to) on entry, so it costs a little — don't wrap every statement in one.

Set-returning functions. `RETURNS TABLE (...)` makes a function usable in `FROM`, like a parameterised view:

```
CREATE FUNCTION top_products(p_limit integer DEFAULT 3)
RETURNS TABLE (product text, units bigint, revenue numeric)
LANGUAGE sql STABLE AS $$
    SELECT p.name, sum(oi.quantity), sum(line_total(oi.quantity, oi.unit_price, oi.discount))
    FROM   order_items oi JOIN orders o USING (order_id) JOIN products p USING (product_id)
    WHERE  o.status <> 'cancelled'
    GROUP  BY p.name ORDER BY 3 DESC LIMIT p_limit
$$;
SELECT * FROM top_products(p_limit => 1);  -- named argument; top_products() uses the default 3
```

```
    product    | units | revenue
---------------+-------+---------
 Laptop Pro 14 |     3 | 3782.05
```

In PL/pgSQL, a set-returning function can instead loop (`FOR r IN SELECT … LOOP`) and emit one row at a time with `RETURN NEXT` — the file's `salary_bands(50000)` does this to count employees per 50,000 salary band.

One-off code without a function: `DO $$ … $$;` runs an anonymous PL/pgSQL block once. The file ends with one (it prints `NOTICE: pending orders: 2`) and with a look at the catalog, where `prokind` is `f` for function and `provolatile` is `i`/`s`/`v`.

Dialect differences

	PostgreSQL	MySQL	SQL Server	SQLite
Stored functions	`CREATE FUNCTION`, many languages	`CREATE FUNCTION` (SQL/PSM style)	`CREATE FUNCTION` (T-SQL, SQL Server's dialect), scalar and table-valued	None in SQL; register functions from the host language (the language of the program using SQLite) (e.g. Python's `conn.create_function`)
Replace	`CREATE OR REPLACE FUNCTION`	`DROP` then `CREATE`	`CREATE OR ALTER`	—
Error raising	`RAISE EXCEPTION`	`SIGNAL SQLSTATE`	`THROW`	`RAISE()` only inside triggers

Common mistakes

- **Wrong volatility**: a function that reads tables marked `IMMUTABLE` can return stale results; a pure calculation left `VOLATILE` blocks optimisations.
- **Hiding row-by-row loops in functions.** Calling a query-running function once per row of a million-row table runs a million queries; a set-based (one statement over all rows at once) `JOIN` is usually far faster.
- **Name clashes** between PL/pgSQL variables and column names (`WHERE customer_id = customer_id`). Prefix parameters (`p_`) and variables (`v_`).

In the interview

30-second answer: "A UDF packages logic in the database so it's reused and runs next to the data. In Postgres I'd use a `LANGUAGE sql` function for simple expressions — it can be inlined — and PL/pgSQL for branching or loops. I'd set the right volatility, keep heavy business rules in the application unless they guard data integrity, and prefer set-based SQL over per-row function calls."

- *Function vs view?* → A view has no parameters; a set-returning function does. For a fixed query, a view is simpler and fully optimisable.

- *Why can't I index `lower(email)` with my own function?* → You can, but only if the function is `IMMUTABLE`.

Self-check

1. What does `$$` do? → It quotes the function body without needing to double every `'` inside it.
2. Can a `STABLE` function update a table? → It shouldn't; it promises not to change the database. Use `VOLATILE`.
3. How do you return several rows? → `RETURNS TABLE(...)` or `RETURNS SETOF type`, with a query body or `RETURN NEXT` / `RETURN QUERY`.

TL;DR

Functions are named, reusable logic stored in the database: `sql` for simple, `plpgsql` for procedural. Label volatility honestly, prefix variable names, and don't replace set-based SQL with row-by-row calls.

14.2 Stored procedures

What it is

A **stored procedure** is like a function that *does* something rather than *computes* something. You run it with `CALL`, it returns no value (except through `INOUT`/`OUT` parameters), and — the key difference in PostgreSQL (procedures arrived in version 11) — it may **commit or roll back transactions inside its body**.

	Function	Procedure
Invoked by	`SELECT f(...)`, inside any expression	`CALL p(...)` only
Returns	A value or a set of rows	Nothing; results via `INOUT` / `OUT` parameters
`COMMIT` / `ROLLBACK` inside	Not allowed — always runs inside the caller's transaction	Allowed when called outside an explicit transaction block
Typical use	Calculations, reusable queries, trigger bodies	Batch jobs, maintenance, multi-step operations

Why it matters

Long batch jobs (archive a year of orders, back-fill a column) should commit in **chunks**: a single giant transaction holds locks for a long time, keeps old row versions alive (blocking vacuum, PostgreSQL's clean-up of dead row versions — see Chapter 13), and loses all its work if it fails at the end.

Example — archiving in committed batches

From `code/02_procedures.sql`:

```
CREATE TABLE orders_archive (LIKE orders INCLUDING ALL);

CREATE PROCEDURE archive_delivered(p_before date, p_batch int, INOUT moved int DEFAULT 0)
LANGUAGE plpgsql AS $$
DECLARE n int;
BEGIN
    LOOP
        WITH picked AS (
            SELECT order_id FROM orders
            WHERE  status = 'delivered' AND placed_at < p_before
              AND  order_id NOT IN (SELECT order_id FROM orders_archive)
            ORDER  BY order_id LIMIT p_batch
        )
        INSERT INTO orders_archive
        SELECT o.* FROM orders o JOIN picked USING (order_id);
        GET DIAGNOSTICS n = ROW_COUNT;     -- rows affected by the last statement
        EXIT WHEN n = 0;
        moved := moved + n;
        RAISE NOTICE 'batch of % committed', n;
        COMMIT;
    END LOOP;
END;
$$;

CALL archive_delivered('2025-04-01', 2);
```

Output:

```
NOTICE:  batch of 2 committed
NOTICE:  batch of 2 committed
NOTICE:  batch of 1 committed
 moved
-------
     5
```

The five delivered orders placed before April 2025 (101, 102, 103, 104, 106) were copied in three separately committed batches, and the `INOUT` parameter came back as a one-row result.

The catch. If the caller already opened a transaction (`BEGIN; CALL …`), the procedure is inside it and cannot end it:

```
BEGIN;
DO $$ DECLARE m int := 0; BEGIN
    CALL archive_delivered('2025-12-31', 2, m);
EXCEPTION WHEN invalid_transaction_termination THEN
    RAISE NOTICE 'caught: %', SQLERRM;
END $$;
ROLLBACK;
```

```
NOTICE:  batch of 2 committed
NOTICE:  caught: invalid transaction termination
```

The first `NOTICE` prints before the `COMMIT` fails — the message is misleading, which is why the file rolls everything back. Many drivers (the libraries programs use to talk to a database) open a transaction automatically (see 14.10), so a procedure that commits must be called in **auto-commit** mode (each statement committed on its own).

Dialect differences

	Syntax	Transaction control inside
PostgreSQL 11+	`CREATE PROCEDURE … ; CALL p()`	Yes, if not called inside a transaction block
MySQL	`CREATE PROCEDURE` (the `mysql` client needs `DELIMITER //` to change the statement terminator — the character that ends a statement, normally `;`); `CALL p()`	Yes
SQL Server	`CREATE PROCEDURE`; `EXEC p`	Yes; can also return result sets directly
SQLite	No stored procedures	—

When not to use it

Stored procedures tie logic to one database product, are harder to unit-test (check with small automated tests), version and deploy (release to servers) than application code, and scale only as far as the database server. A common, balanced rule: **integrity rules and data-heavy batch work in the database; business workflows in the application.**

Self-check

1. How do you get a value back from a procedure? → An `INOUT` (or, from PG 14, `OUT`) parameter.
2. Why does `COMMIT` in a procedure sometimes fail? → The caller is already inside a transaction block.
3. Why commit in batches? → Shorter locks, less bloat (wasted space from dead row versions), and completed work survives a later failure.

TL;DR

Procedures are `CALL`ed, return through `INOUT` parameters, and can `COMMIT` inside — ideal for batch jobs, but only when called in auto-commit mode.

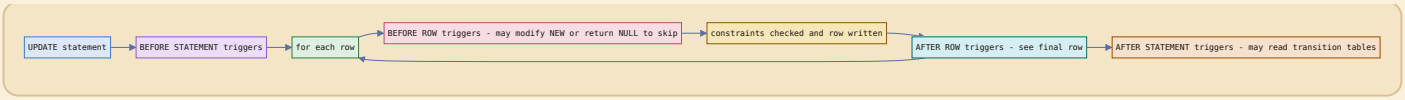
14.3 Triggers

What it is

A **trigger** tells the database: "whenever *this event* happens on *this table*, run *this function* automatically". Events are `INSERT`, `UPDATE`, `DELETE` and `TRUNCATE`. It's like a motion-sensor light: nobody presses a switch, the event itself does.

Choice	Options	Meaning
Timing	`BEFORE` / `AFTER` / `INSTEAD OF` (views only)	Before the row is written (can change or skip it), after it's written (can react), or replacing the write on a view
Level	`FOR EACH ROW` / `FOR EACH STATEMENT`	Once per affected row, or once per SQL statement (even if 0 rows)
Condition	`WHEN (…)`	Fire only if the condition on `OLD` / `NEW` is true
Column filter	`UPDATE OF col`	Fire only when those columns are named in the `UPDATE`

Inside a row trigger, `NEW` is the new version of the row (INSERT/UPDATE), `OLD` the previous one (UPDATE/DELETE); `TG_OP`, `TG_TABLE_NAME` and friends describe the event. A trigger function is declared `RETURNS trigger`.



Why it matters

Triggers enforce behaviour **no matter which application, script or person** changes the data: an `updated_at` stamp, an audit trail (a record of who changed what, when), a denormalised counter (a stored copy of a count, kept for fast reads) kept in sync, a rule a `CHECK` constraint cannot express because it needs another table.

Example 1 — `updated_at`

From `code/03_triggers.sql`:

```
ALTER TABLE products ADD COLUMN updated_at timestamptz NOT NULL DEFAULT '2025-01-01 00:00+00';

CREATE FUNCTION set_updated_at() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
    NEW.updated_at := now();      -- change the row before it is written
    RETURN NEW;                   -- BEFORE row trigger: return the row to store (NULL = skip it)
END;
$$;

CREATE TRIGGER products_updated_at
BEFORE UPDATE ON products
FOR EACH ROW
WHEN (OLD.* IS DISTINCT FROM NEW.*)          -- skip no-op updates
EXECUTE FUNCTION set_updated_at();

UPDATE products SET unit_price = 22.99 WHERE product_id = 2;
UPDATE products SET unit_price = 35.00 WHERE product_id = 8;   -- same value as before
SELECT product_id, unit_price, updated_at > '2025-01-02' AS stamped_now
FROM   products WHERE product_id IN (2, 8) ORDER BY product_id;
```

```
 product_id | unit_price | stamped_now
------------+------------+-------------
          2 |      22.99 | t
          8 |      35.00 | f
```

`EXECUTE FUNCTION` is the PostgreSQL 11+ spelling (older versions write `EXECUTE PROCEDURE`, still accepted).

Example 2 — a validation rule that needs another table

```
CREATE FUNCTION forbid_discontinued() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
    IF (SELECT discontinued FROM products WHERE product_id = NEW.product_id) THEN
        RAISE EXCEPTION 'product % is discontinued', NEW.product_id;
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER order_items_check_product
BEFORE INSERT OR UPDATE OF product_id ON order_items
FOR EACH ROW EXECUTE FUNCTION forbid_discontinued();
-- inserting the discontinued Desk Lamp (id 8):
-- NOTICE:  rejected: product 8 is discontinued
```

Example 3 — a generic audit log

One function works for any table because `to_jsonb(OLD)` / `to_jsonb(NEW)` turn a whole row into JSON:

```
CREATE TABLE audit_log (
    id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    table_name  text        NOT NULL,
    op          text        NOT NULL,
    changed_by  text        NOT NULL DEFAULT current_user,
    changed_at  timestamptz NOT NULL DEFAULT now(),
    old_row     jsonb,
    new_row     jsonb
);

CREATE FUNCTION audit_row() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO audit_log (table_name, op, old_row, new_row)
    VALUES (TG_TABLE_NAME, TG_OP,
            CASE WHEN TG_OP IN ('UPDATE','DELETE') THEN to_jsonb(OLD) END,
            CASE WHEN TG_OP IN ('INSERT','UPDATE') THEN to_jsonb(NEW) END);
    RETURN NULL;                  -- the return value of an AFTER trigger is ignored
END;
$$;

CREATE TRIGGER customers_audit
AFTER INSERT OR UPDATE OR DELETE ON customers
FOR EACH ROW EXECUTE FUNCTION audit_row();

INSERT INTO customers VALUES (11, 'Jia', 'Wong', 'jia.wong@mail.example', 'Singapore', 'Singapore',
'2025-06-20');
UPDATE customers SET city = 'Mumbai' WHERE customer_id = 8;
DELETE FROM customers WHERE customer_id = 11;

SELECT id, table_name, op, changed_by, old_row ->> 'city' AS old_city, new_row ->> 'city' AS new_city
FROM   audit_log ORDER BY id;
```

```
 id | table_name |   op   | changed_by | old_city  | new_city
----+------------+--------+------------+-----------+-----------
  1 | customers  | INSERT | postgres   |           | Singapore
  2 | customers  | UPDATE | postgres   | mumbai    | Mumbai
  3 | customers  | DELETE | postgres   | Singapore |
```

Which columns changed? Compare the two JSON objects key by key:

```
SELECT n.key, o.value AS before, n.value AS after
FROM   audit_log a, jsonb_each(a.new_row) n
JOIN   jsonb_each(a.old_row) o USING (key)
WHERE  a.op = 'UPDATE' AND n.value IS DISTINCT FROM o.value;
```

```
 key  |  before  |  after
------+----------+----------
 city | "mumbai" | "Mumbai"
```

Because the audit insert runs **in the same transaction** as the change, a rolled-back change leaves no audit row — the log never lies.

Example 4 — statement-level trigger with transition tables

A row trigger on a 1-million-row `UPDATE` fires a million times. A **statement-level** trigger fires once, and (PostgreSQL 10+) can see all changed rows at once through **transition tables**:

```
CREATE TRIGGER products_bulk_price
AFTER UPDATE ON products
REFERENCING OLD TABLE AS old_rows NEW TABLE AS new_rows
FOR EACH STATEMENT EXECUTE FUNCTION log_bulk_price_change();
-- the function reads new_rows / old_rows like tables (count, avg of price differences)

UPDATE products SET unit_price = unit_price * 1.10 WHERE category = 'Furniture';
-- NOTICE:  3 rows changed price, average change 29.47
```

Trade-offs and dialect differences

	PostgreSQL	MySQL	SQL Server	SQLite
Row-level	Yes	Yes (only kind)	No — statement-level only, with `inserted` / `deleted` pseudo-tables	Yes (only kind)
Statement-level	Yes (+ transition tables)	No	Yes (the only kind)	No
`INSTEAD OF`	On views	No	On tables and views	On views
Body	A separate trigger function	Inline SQL body	Inline T-SQL	Inline SQL body

When not to use it

- For **business workflows** (send an email, call a payment API — the interface another service exposes to programs) — triggers run inside the transaction and must be fast; put a row in an *outbox* table (a table of pending messages) instead and let a worker (a background process) act on it.
- When a **constraint** can do the job (`CHECK`, `FOREIGN KEY`, `UNIQUE`, generated column) — constraints are declarative, faster and visible in the schema.
- When the logic surprises people: "invisible" side effects are the most common complaint about triggers.

Common mistakes

- Forgetting `RETURN NEW` in a `BEFORE` row trigger — returning `NULL` silently **skips** the row.
- A trigger that updates its own table, re-firing itself in a loop (guard with `WHEN` or `pg_trigger_depth()`).

- Expecting triggers on `TRUNCATE` from a row-level trigger — `TRUNCATE` only fires statement-level `ON TRUNCATE` triggers.

Real-world examples

Compliance audit trails (who changed a salary, when), `updated_at` columns used by sync jobs, and the **transactional outbox** pattern for reliable event publishing (telling other systems that something changed).

In the interview

> **30-second answer:** "A trigger runs a function automatically on INSERT/UPDATE/DELETE, before or after, per row or per statement. I use them for cross-cutting (applying everywhere, regardless of which code writes) data rules — audit logs, timestamps, integrity checks a constraint can't express — and avoid them for business workflows because they're invisible and slow down writes."

- *BEFORE vs AFTER?* → BEFORE can modify or reject the row; AFTER sees the final row (after constraints) and is for reactions like auditing.
- *Does a trigger's work roll back with the statement?* → Yes, it runs in the same transaction.
- *How to audit a bulk update efficiently?* → A statement-level trigger with transition tables.

Self-check

1. Which record is available in a DELETE trigger? → `OLD` only.
2. What happens if a BEFORE row trigger returns NULL? → That row's operation is skipped.
3. Why is `WHEN (OLD.* IS DISTINCT FROM NEW.*)` useful? → It skips no-op updates, so `updated_at` changes only when data really changed.

TL;DR

Triggers = automatic functions on data events. Great for audit logs, `updated_at` and cross-table checks; keep them small, prefer constraints when possible, and never put slow external work in them.

14.4 JSON and JSONB

What it is

JSON (JavaScript Object Notation) is the text format of web APIs: objects `{"key": value}`, arrays `[…]`, strings, numbers, booleans and `null`. PostgreSQL has two column types for it:

	`json`	`jsonb` ("JSON binary")
Storage	The exact text you sent	Parsed into a binary tree
Duplicate keys, whitespace, key order	Kept as written	Last duplicate wins; whitespace removed; keys reordered
Speed	Fast to store, re-parsed on every read	Slightly slower to store, fast to query
Operators & indexing	Few	Containment `@>`, key-exists `?`, GIN (inverted) indexes, SQL/JSON path

Use `jsonb` unless you must keep the original text byte-for-byte.

```
SELECT '{"b":1, "a":2, "a":3}'::json AS as_json, '{"b":1, "a":2, "a":3}'::jsonb AS as_jsonb;
```

```
       as_json        |     as_jsonb
----------------------+------------------
 {"b":1, "a":2, "a":3} | {"a": 3, "b": 1}
```

Why it matters

Some data really is **semi-structured**: product specifications differ by category (a laptop has RAM, a desk has a height range), webhook payloads (data another service pushes to your app over HTTP) change shape, user preferences grow new keys. A `jsonb` column stores that without a new column (or a new table) for every attribute — while the stable, important fields stay in normal, typed, constrained columns. For whole-document databases see Chapter 16.

How it works — operators

Operator / function	Returns	Example → result
`->` key or index	`jsonb`	`specs -> 'brand'` → `"Nimbus"`
`->>` key or index	`text`	`specs ->> 'brand'` → `Nimbus`
`#>` / `#>>` path	`jsonb` / `text`	`specs #>> '{screen,type}'` → `OLED`
`@>` contains	boolean	`specs @> '{"brand":"Nimbus"}'`
`?`, `?&`	boolean	
`@?`, `@@`	boolean	SQL/JSON path exists / path predicate (PG 12+)
`-`, `#-`		
`jsonb_set(doc, path, value)`	`jsonb`	replace or add a value at a path

Example — product specs

From `code/04_jsonb.sql` (the file first adds `specs jsonb` and fills it for five products):

```
SELECT name,
       specs -> 'brand'                AS brand_json,
       specs ->> 'brand'               AS brand_text,
       specs #>> '{screen,type}'       AS screen_type,
       (specs ->> 'ram_gb')::int       AS ram_gb,
       specs -> 'ports' -> 0           AS first_port,
       jsonb_array_length(specs -> 'ports') AS n_ports
FROM   products WHERE specs ? 'ports' ORDER BY product_id;
```

```
      name      | brand_json | brand_text | screen_type | ram_gb | first_port | n_ports
----------------+------------+------------+-------------+--------+------------+---------
 Laptop Pro 14  | "Nimbus"   | Nimbus     | OLED        |     16 | "usb-c"    |       2
 Wireless Mouse | "Clicko"   | Clicko     |             |        | "usb-a"    |       1
 USB-C Hub      | "Nimbus"   | Nimbus     |             |        | "usb-c"    |       4
```

Note the quotes: `->` gives JSON (`"Nimbus"`), `->>` gives plain text. Missing keys give `NULL`, not an error. To compare numbers, cast the text: `(specs ->> 'ram_gb')::int`.

Filtering:

```
SELECT name FROM products WHERE specs @> '{"brand":"Nimbus"}';          -- Laptop Pro 14, USB-C Hub
SELECT name FROM products WHERE specs -> 'ports' ? 'hdmi';              -- Laptop Pro 14, USB-C Hub
SELECT name, jsonb_path_query_first(specs, '$.height_cm.max') AS max_height
FROM   products WHERE specs @@ '$.height_cm.max > 100';                 -- Standing Desk | 120
```

(`?` on an array checks whether a string **element** is present.)

Updating — `jsonb` values are immutable, so each operation builds a new document:

```
UPDATE products SET specs = jsonb_set(specs, '{ram_gb}', '32')  WHERE product_id = 1;
UPDATE products SET specs = specs || '{"color":"black"}'         WHERE product_id = 2;
UPDATE products SET specs = specs - 'wireless'                   WHERE product_id = 7;
UPDATE products SET specs = specs #- '{screen,type}'             WHERE product_id = 1;
```

```
 product_id |                                          specs
------------+-------------------------------------------------------------------------------------------
          1 | {"brand": "Nimbus", "ports": ["usb-c", "hdmi"], "ram_gb": 32, "screen": {"inches": 14}}
          2 | {"brand": "Clicko", "color": "black", "ports": ["usb-a"], "wireless": true}
          7 | {"brand": "Hushh", "battery_h": 30}
```

Rows → JSON and back. `jsonb_build_object` + `jsonb_agg` build a nested document — e.g. an API response for order 108 in one query: `{"items": [{"qty": 1, "product": "Laptop Pro 14"}, …], "status": "delivered", "order_id": 108}`. In the other direction, `jsonb_to_recordset('[{"sku":"A1","qty":2},{"sku":"B7","qty":5}]') AS t(sku text, qty int)` turns an incoming payload into rows `(A1, 2)`, `(B7, 5)`. `jsonb_each(doc)` similarly turns an object into `(key, value)` rows.

Indexing

A **GIN** (Generalized Inverted Index — an index from each key/value to the rows containing it) serves `@>`, `?`, `?|`, `?&` and the path operators; a normal B-tree **expression index** serves equality and ranges on one extracted key. (The file disables sequential scans only because the demo table is tiny, and runs `VACUUM products` first: an index built over rows that still carry old versions from recent `UPDATE`s may be ignored by transactions whose snapshots are older than the index. See Chapter 12 for reading plans.)

```
CREATE INDEX products_specs_gin ON products USING gin (specs);
EXPLAIN (COSTS OFF) SELECT name FROM products WHERE specs @> '{"brand":"Nimbus"}';
CREATE INDEX products_brand_idx ON products ((specs ->> 'brand'));
EXPLAIN (COSTS OFF) SELECT name FROM products WHERE specs ->> 'brand' = 'Nimbus';
```

```
Bitmap Heap Scan on products
  Recheck Cond: (specs @> '{"brand": "Nimbus"}'::jsonb)
  ->  Bitmap Index Scan on products_specs_gin
        Index Cond: (specs @> '{"brand": "Nimbus"}'::jsonb)

Index Scan using products_brand_idx on products
  Index Cond: ((specs ->> 'brand'::text) = 'Nimbus'::text)
```

The GIN index is *not* used for `specs ->> 'brand' = 'Nimbus'` — write the filter as `@>` or add the expression index.

Dialect differences

	Type	Extract text	Notes
PostgreSQL	`json`, `jsonb`	`->>`, `#>>`, `jsonb_path_query`	GIN indexes; `JSON_TABLE` and SQL/JSON query functions added in PG 17
MySQL 5.7+	`JSON` (binary)	`->>`, `JSON_EXTRACT`, `JSON_UNQUOTE`	Index via generated columns or multi-valued indexes (8.0.17+)
SQL Server	`nvarchar` + `JSON_VALUE` / `OPENJSON`; native `json` type in SQL Server 2025 and Azure SQL	`JSON_VALUE(doc, '$.brand')`	Index a computed column
SQLite	`TEXT` with built-in JSON functions; binary JSONB since 3.45	`->>` (3.38+), `json_extract`	Expression indexes

When not to use it

When fields are known, always present, filtered or joined on, or need constraints (`NOT NULL`, foreign keys) — make them **real columns**. Stuffing everything into one JSON blob is the "EAV / schemaless" antipattern (EAV = entity–attribute–value, storing every attribute as a generic key/value pair): no types, no constraints, poor statistics, awkward queries.

Common mistakes

- Comparing `->` output with text (`specs -> 'brand' = 'Nimbus'` fails: jsonb vs text). Use `->>` or `@>`.
- Sorting or comparing numbers as text (`'9' > '10'`). Cast first.
- Updating one key of a large document still rewrites the whole row.

In the interview

> **30-second answer:** "I use `jsonb` for genuinely variable attributes alongside normal columns. I query with `->>` / `@>`, index with GIN for containment or a B-tree expression index for one hot key, and promote any field that becomes important to a real column."

- *json vs jsonb?* → jsonb is parsed, indexable and fast to query; json keeps exact text.

Self-check

1. `'{"a":1}'::jsonb -> 'a'` vs `->> 'a'`? → jsonb `1` vs text `'1'`.
2. Which index helps `WHERE specs @> '{"wireless":true}'`? → A GIN index on `specs`.
3. How to append a key? → `specs || '{"k":"v"}'` or `jsonb_set`.

TL;DR

`jsonb` + `->>`/`@>` + GIN = flexible attributes inside a relational table. Keep core fields as columns.

14.5 Arrays

What it is

PostgreSQL columns can hold an **array** — an ordered list of values of one type (`text[]`, `int[]`). Arrays are **1-based**, and reading past the end returns `NULL` instead of failing.

Example — product tags

From `code/05_arrays.sql`:

```sql
ALTER TABLE products ADD COLUMN tags text[] NOT NULL DEFAULT '{}';
UPDATE products SET tags = CASE product_id
    WHEN 1 THEN ARRAY['laptop','work','premium']
    WHEN 2 THEN '{mouse,wireless,budget}'          -- text literal form
    WHEN 4 THEN ARRAY['keyboard','work']
    WHEN 5 THEN ARRAY['desk','work','premium']
    WHEN 7 THEN ARRAY['audio','wireless','premium']
    ELSE '{}' END;

SELECT name, tags, tags[1] AS first_tag, tags[2:3] AS slice, cardinality(tags) AS n, tags[9] AS no_such
FROM   products WHERE product_id IN (1, 2, 9) ORDER BY product_id;
```

```
      name      |          tags           | first_tag |       slice       | n | no_such
----------------+-------------------------+-----------+-------------------+---+---------
 Laptop Pro 14  | {laptop,work,premium}   | laptop    | {work,premium}    | 3 |
 Wireless Mouse | {mouse,wireless,budget} | mouse     | {wireless,budget} | 3 |
 Gift Card      | {}                      |           | {}                | 0 |
```

Operation	Syntax	Result in the demo
Element in array	`'premium' = ANY (tags)`	Laptop, Headphones, Desk
Contains all	`tags @> ARRAY['work','premium']`	Laptop, Desk
Overlaps (any in common)	`tags && ARRAY['wireless','keyboard']`	Keyboard, Headphones, Mouse
Append / remove	`array_append(a, x)`, `array_remove(a, x)`	`{a,b,c}`, `{b}`
Concatenate / find	`a	
Array ↔ rows	`unnest(a) WITH ORDINALITY`, `array_agg(x ORDER BY …)`	see below
Array ↔ string	`string_to_array(s, ',')`, `array_to_string(a, '	')`

`unnest` and `array_agg` are opposites; together they answer "which tags are popular?":

```sql
SELECT tag, count(*) AS products, array_agg(name ORDER BY name) AS which
FROM   products, unnest(tags) AS tag
GROUP  BY tag HAVING count(*) > 1 ORDER BY products DESC, tag;
```

```
   tag    | products |                            which
----------+----------+-------------------------------------------------------------------
 premium  |        3 | {"Laptop Pro 14","Noise-Cancelling Headphones","Standing Desk"}
 work     |        3 | {"Laptop Pro 14","Mechanical Keyboard","Standing Desk"}
 wireless |        2 | {"Noise-Cancelling Headphones","Wireless Mouse"}
```

Arrays as a parameter. Application code often needs "rows whose id is in this list". Instead of building `IN (3, 5, 7)` as a string (injection risk — see 14.9 — a new statement per list length), pass **one array parameter**: `WHERE product_id = ANY ($1)`. The file shows it with `PREPARE by_ids(int[])` / `EXECUTE by_ids('{3,5,7}')` (a server-side prepared statement: parsed once, run with different values), which returns USB-C Hub, Standing Desk and Noise-Cancelling Headphones. A GIN index on `tags` serves `@>`, `&&` and `<@` (the plan in the file shows `Bitmap Index Scan on products_tags_gin`).

When not to use it

If you need to **join** on the elements, enforce **foreign keys** on them, or update single elements often, use a child table (`product_tags(product_id, tag)`) — that's what first normal form (Chapter 11) recommends. Arrays suit small, read-mostly lists like tags. MySQL, SQL Server and SQLite have no array column type (use JSON arrays or a child table).

Self-check

1. What is `ARRAY[10,20][0]`? → `NULL` — arrays start at 1.
2. `= ANY(arr)` vs `@>`? → `x = ANY(arr)` tests one value; `arr @> other` tests that every element of `other` is present.
3. Why pass `= ANY($1)` from code? → One statement for any list length, and values are bound safely.

TL;DR

Arrays: 1-based lists in one column; query with `ANY`, `@>`, `&&`, explode with `unnest`, build with `array_agg`, index with GIN; use a child table when elements need relationships.

14.6 Full-text search

What it is

Full-text search (FTS) finds documents that match *words* — ignoring case, word endings and filler words — and ranks them by relevance, like a small search engine inside the database.

Key terms:

Term	Meaning
Lexeme	A normalised word: lower-cased and **stemmed** ("shipping", "shipped" → `ship`)
Stop word	A very common word ("the", "and", "was") that is dropped
`tsvector`	A document turned into sorted lexemes with positions: `'chair':2 'ship':4,8`
`tsquery`	A search: lexemes joined by `&` (and), `
`@@`	"Does this vector match this query?"
Text search configuration	Language rules, e.g. `'english'` (stemmer + stop-word list)

Why it matters

`LIKE '%word%'` has three problems: no stemming (searching "returning" misses "returned"), no ranking, and a leading `%` cannot use a B-tree index, so every row is scanned. FTS solves all three and is often enough to postpone running a separate search engine such as Elasticsearch/OpenSearch (dedicated search servers).

How it works

1. `to_tsvector('english', text)` parses the text, drops stop words and stems the rest.
2. A query function builds a `tsquery` from user input:
   - `to_tsquery` — expects operator syntax (`'ship & chair'`); errors on bad syntax.
   - `plainto_tsquery` — all words ANDed.
   - `websearch_to_tsquery` (PG 11+) — Google-like: quotes for phrases, `or`, `-` to exclude; never errors on user input.
3. `vector @@ query` tests a match; `ts_rank` scores it; `ts_headline` highlights matches.
4. A GIN index on the `tsvector` makes matching fast.

From `code/06_full_text_search.sql`:

```
SELECT to_tsvector('english', 'The chairs were shipped quickly and the shipping was free') AS doc;
SELECT to_tsquery('english', 'shipping & chair')                          AS q1,
       plainto_tsquery('english', 'fast shipping chairs')                 AS q2,
       websearch_to_tsquery('english', '"standing desk" -cheap or lamp')  AS q3;
```

```
                  doc
-----------------------------------------
 'chair':2 'free':10 'quick':5 'ship':4,8

       q1        |           q2             |                  q3
-----------------+--------------------------+---------------------------------------
 'ship' & 'chair' | 'fast' & 'ship' & 'chair' | 'stand' <-> 'desk' & !'cheap' | 'lamp'
```

Example — a help-centre search

The search vector is a **generated column** (computed automatically from other columns on every write; PostgreSQL 12+) so it can never go stale; `setweight` marks title words `A` (most important) and body words `B`, which `ts_rank` rewards:

```
CREATE TABLE help_articles (
    id     int PRIMARY KEY,
    title  text NOT NULL,
    body   text NOT NULL,
    search tsvector GENERATED ALWAYS AS (
             setweight(to_tsvector('english', title), 'A') ||
             setweight(to_tsvector('english', body),  'B')) STORED
);
-- 5 articles: 'Returning a chair', 'Standing desk assembly', 'Shipping times',
--             'Keyboard not connecting', 'Cheap desk lamps discontinued'

SELECT id, title, round(ts_rank(search, q)::numeric, 3) AS rank
FROM   help_articles, websearch_to_tsquery('english', 'chair shipping') AS q
WHERE  search @@ q
ORDER  BY rank DESC, id;
```

```
 id |       title       | rank
----+-------------------+-------
  1 | Returning a chair | 0.588
  3 | Shipping times    | 0.529
```

More query shapes from the same file:

Query	Matches	Why
`to_tsquery('english', 'assem:*')`	2 Standing desk assembly	`:*` = prefix match (autocomplete)
`websearch_to_tsquery('english', '"standing desk"')`	2	Phrase: `stand <-> desk`
`websearch_to_tsquery('english', 'desk -lamp')`	2, 3	Article 5 excluded by `-lamp`

Highlighting shows the user *why* a result matched:

```
SELECT id, ts_headline('english', body, websearch_to_tsquery('english', 'desk chair'),
                       'StartSel=[, StopSel=], MaxWords=12, MinWords=5') AS snippet
FROM   help_articles WHERE search @@ websearch_to_tsquery('english', 'desk chair');
```

```
 id |              snippet
----+----------------------------------
  3 | [desk] or [chair] ships in a week
```

And stemming in action — no body contains the word "returning", but article 1's body has "returned" and "returns":

```
 like_hits | fts_hits          -- body ILIKE '%returning%'  vs  to_tsvector(body) @@ 'returning'
-----------+----------
         0 |        1
```

With `CREATE INDEX … USING gin (search)` the plan becomes `Bitmap Index Scan on help_articles_search_gin`.

Trade-offs

	PostgreSQL FTS	Dedicated engine (Elasticsearch / OpenSearch / Solr)
Setup	None — same database, same transaction	Separate cluster (group of servers), sync pipeline (a job copying data across)
Freshness	Immediately consistent	Near-real-time, eventually consistent (copies catch up after a short delay)
Features	Stemming, ranking, phrases, prefixes, weights	Fuzzy/typo tolerance, facets (match counts per category, shown as filters), synonyms at scale, relevance tuning, distributed scoring
Best for	Up to millions of documents, app-level search	Very large corpora, search-centric products

Typo tolerance in Postgres needs the `pg_trgm` extension (trigram similarity — comparing 3-character chunks of text), which is not installed on the book's server. Other databases: MySQL `FULLTEXT` indexes with `MATCH … AGAINST`, SQL Server full-text catalogs with `CONTAINS` / `FREETEXT`, SQLite's `FTS5` virtual tables.

Common mistakes

- Mixing configurations: indexing with `to_tsvector('english', …)` but querying with `to_tsvector(body)` (default config) — the index won't be used. Always name the configuration.
- Passing raw user input to `to_tsquery` — a stray `&` or `(` raises a syntax error. Use `websearch_to_tsquery`.
- Computing `to_tsvector` in every query instead of storing it (generated column) and indexing it.

In the interview

30-second answer: "Postgres FTS turns text into a `tsvector` of stemmed lexemes and matches a `tsquery` with `@@`. I store the vector as a weighted generated column, index it with GIN, parse user input with `websearch_to_tsquery`, and rank with `ts_rank`. I'd move to Elasticsearch only when I need fuzzy matching, facets or scale beyond one database."

Self-check

1. Why does "shipping" match "shipped"? → Both stem to the lexeme `ship`.
2. Which index type? → GIN on the `tsvector`.
3. How to make titles count more? → `setweight(…, 'A')` for title lexemes; `ts_rank` weights A highest.

TL;DR

FTS = `tsvector @@ tsquery` + GIN + `ts_rank` + `ts_headline`; beats `LIKE` on stemming, ranking and speed.

14.7 Pivot and unpivot

What it is

Pivoting turns *values in rows* into *columns* — like an Excel pivot table: one row per category, one column per month. **Unpivoting** is the reverse: a wide table becomes tall again. Databases prefer tall (long)

data; reports and spreadsheets prefer wide data. Standard SQL has no `PIVOT` keyword, so we build pivots from conditional aggregation (introduced in Chapter 4).

Example — revenue by category and month

From `code/07_pivot.sql`. A view first produces the tall data (`category, month, revenue`):

```
SELECT category,
       round(sum(revenue) FILTER (WHERE month = '2025-01'), 2) AS jan,
       round(sum(revenue) FILTER (WHERE month = '2025-02'), 2) AS feb,
       round(sum(revenue) FILTER (WHERE month = '2025-03'), 2) AS mar,
       round(sum(revenue) FILTER (WHERE month = '2025-04'), 2) AS apr,
       round(sum(revenue), 2)                                  AS all_months
FROM   monthly_category_revenue
GROUP  BY category ORDER BY category;
```

```
  category   |   jan   |  feb   |  mar   |   apr   | all_months
-------------+---------+--------+--------+---------+------------
 (none)      |         |        | 100.00 |         |     100.00
 Accessories |  188.00 |        |        |  200.80 |     671.35
 Electronics | 1683.90 |        |  74.97 | 1448.95 |    4691.80
 Furniture   |         | 811.49 | 629.98 |  499.00 |    1940.47
```

(`all_months` also includes May and June, which have no column here.) The portable form, which works in every database, puts `CASE` inside the aggregate: `sum(CASE WHEN month = '2025-01' THEN revenue ELSE 0 END) AS jan` — with `ELSE 0`, empty cells show `0.00` instead of `NULL`. The same technique counts order statuses per customer (`count(*) FILTER (WHERE o.status = 'delivered') AS delivered`, …).

`crosstab` (tablefunc extension)

PostgreSQL's `tablefunc` extension provides `crosstab(source_sql, category_sql)`, which pivots a three-column query (row id, category, value). It is **not installed on the book's server** (Homebrew's — a macOS package manager — build here ships without it), so this is not run:

```
-- (PostgreSQL with contrib — not run)
CREATE EXTENSION IF NOT EXISTS tablefunc;
SELECT *
FROM crosstab(
  $$ SELECT category, month, round(revenue, 2) FROM monthly_category_revenue ORDER BY 1, 2 $$,
  $$ VALUES ('2025-01'), ('2025-02'), ('2025-03'), ('2025-04') $$
) AS ct(category text, jan numeric, feb numeric, mar numeric, apr numeric);
```

You must still list the output columns and their types in `AS ct(...)`; the two-argument form places each value in the right column even when some categories are missing. `FILTER` / `CASE` does the same job without an extension, so most teams use them.

Unpivot with `LATERAL (VALUES …)`

```
CREATE TABLE quarterly_targets (region text PRIMARY KEY, q1 int, q2 int, q3 int, q4 int);
INSERT INTO quarterly_targets VALUES ('India', 120, 150, 170, 200), ('UK', 60, 65, NULL, 80);

SELECT t.region, v.quarter, v.target
FROM   quarterly_targets t
CROSS  JOIN LATERAL (VALUES ('Q1', t.q1), ('Q2', t.q2), ('Q3', t.q3), ('Q4', t.q4)) AS v(quarter, target)
WHERE  v.target IS NOT NULL
ORDER  BY t.region, v.quarter;
```

This returns 7 rows — four for India and three for the UK (its `NULL` Q3 target is filtered out).

`LATERAL` (Chapter 6) lets the `VALUES` list refer to the current row's columns. The portable alternative is a `UNION ALL` of one `SELECT` per column.

Dynamic pivots

SQL needs the column list **when the query is planned**, so a pivot whose columns depend on the data (every month that exists) requires either building the SQL text in code (with identifiers quoted — see 14.9) or returning a JSON object per row and letting the application lay it out:

```
SELECT category, jsonb_object_agg(month, round(revenue, 2) ORDER BY month) AS by_month
FROM   monthly_category_revenue GROUP BY category ORDER BY category;
```

```
  category   |                          by_month
-------------+-------------------------------------------------------------
 (none)      | {"2025-03": 100.00}
 Accessories | {"2025-01": 188.00, "2025-04": 200.80, "2025-06": 282.55}
 ...           (Electronics and Furniture rows similar)
```

Dialect differences

	Pivot	Unpivot
PostgreSQL	`FILTER`, `CASE`, `crosstab` (tablefunc)	`LATERAL (VALUES …)`, `unnest`
SQL Server / Oracle	`PIVOT (SUM(x) FOR col IN (…))` operator	`UNPIVOT` operator (skips NULLs)
MySQL	`CASE` (or `SUM(cond)` since booleans are 0/1)	`UNION ALL`; `LATERAL` from 8.0.14
SQLite	`FILTER` (3.30+) or `CASE`	`UNION ALL`

In the interview

30-second answer: "Pivot = group by the row key and write one conditional aggregate per output column — `SUM(x) FILTER (WHERE col = 'v')` or `SUM(CASE WHEN …)`. Unpivot = `CROSS JOIN LATERAL (VALUES …)` or `UNION ALL`. If the columns aren't known in advance, generate the SQL in code or return JSON."

- *`count(CASE WHEN … THEN 1 END)` vs `sum(CASE WHEN … THEN 1 ELSE 0 END)`?* → Same counts; `count` ignores the NULLs from the missing `ELSE`.

TL;DR

Pivot with conditional aggregates, unpivot with `LATERAL (VALUES …)`; dynamic column lists need generated SQL or JSON.

14.8 Security: roles, privileges and row-level security

What it is

- A **role** is PostgreSQL's single concept for both *users* and *groups*. A role with `LOGIN` can connect (a user); a role without it is a group that other roles are members of. Roles are **cluster-wide** (shared by all databases on one server).
- A **privilege** is permission for one action on one object: `SELECT`, `INSERT`, `UPDATE`, `DELETE`, `TRUNCATE`, `REFERENCES`, `TRIGGER` on tables; `USAGE`/`CREATE` on schemas; `EXECUTE` on functions; `CONNECT` on databases.
- `GRANT` gives privileges; `REVOKE` takes them away. These are the **DCL** (data control language) statements from Chapter 1.

- **Row-level security (RLS)** adds *per-row* filters: two users may read the same table but see different rows.

Why it matters

The **principle of least privilege**: every account gets only what it needs. If the reporting tool's password leaks, the attacker can read reports — not delete orders. If the web app has an injection bug (14.9), a read-only role limits the damage. RLS pushes tenant isolation (a *tenant* is one customer organisation sharing the app: "customer A never sees customer B's data") into the database, so one forgotten `WHERE tenant_id = …` in the app can't leak data.

Example — groups, grants, column privileges

From `code/08_security.sql`. (Roles use a `ch14_` prefix and are dropped at the end, because they are shared server-wide.)

```
CREATE ROLE ch14_reporting NOLOGIN;                          -- a group
CREATE ROLE ch14_analyst  LOGIN IN ROLE ch14_reporting;      -- a user in that group
CREATE ROLE ch14_app      LOGIN;                             -- the application's account

GRANT SELECT ON orders, order_items TO ch14_reporting;
GRANT SELECT, INSERT, UPDATE ON orders TO ch14_app;
GRANT SELECT (customer_id, first_name, city, country) ON customers TO ch14_reporting;  -- column-level
```

Querying `information_schema.role_table_grants` (`information_schema` is the SQL-standard set of catalog views) confirms: `ch14_app` has INSERT, SELECT, UPDATE on `orders`; `ch14_reporting` has SELECT on `orders` and `order_items`. (Column-level grants appear in `information_schema.column_privileges`, not in this table-level view.)

`SET ROLE` lets a superuser act as another role for testing — the same checks apply as if that role had logged in:

```
SET ROLE ch14_analyst;
SELECT count(*) FROM orders;                                  -- 14
SELECT first_name, city FROM customers WHERE customer_id = 1;      -- Anita | Mumbai
PERFORM email FROM customers;             -- NOTICE: analyst: permission denied for table customers
DELETE FROM orders WHERE order_id = 114;  -- NOTICE: analyst: permission denied for table orders
RESET ROLE;
REVOKE SELECT ON order_items FROM ch14_reporting;   -- analyst now denied on order_items too
```

(The failing statements are wrapped in `DO` blocks — one-off anonymous PL/pgSQL code — in the file so the script continues; `PERFORM` is PL/pgSQL's way to run a query and discard its result.) The analyst had no direct grants: privileges were **inherited** from the group, so revoking from the group affected every member at once — grant to groups, not to individuals.

Example — row-level security: each sales rep sees only their orders

```
CREATE ROLE ch14_priya LOGIN;      -- employee 7
CREATE ROLE ch14_arjun LOGIN;      -- employee 8
CREATE TABLE rep_logins (role_name text PRIMARY KEY, emp_id int NOT NULL REFERENCES employees);
INSERT INTO rep_logins VALUES ('ch14_priya', 7), ('ch14_arjun', 8);
GRANT SELECT ON rep_logins TO ch14_priya, ch14_arjun;
GRANT SELECT, UPDATE ON orders TO ch14_priya, ch14_arjun;

ALTER TABLE orders ENABLE ROW LEVEL SECURITY;          -- from now on: no matching policy = no rows

CREATE POLICY rep_sees_own_orders ON orders
    FOR ALL TO ch14_priya, ch14_arjun
    USING      (sales_rep_id = (SELECT emp_id FROM rep_logins WHERE role_name = current_user))
    WITH CHECK (sales_rep_id = (SELECT emp_id FROM rep_logins WHERE role_name = current_user));

CREATE POLICY reporting_reads_all ON orders FOR SELECT TO ch14_reporting USING (true);

SET ROLE ch14_priya;
SELECT current_user, order_id, sales_rep_id, status FROM orders ORDER BY order_id;
UPDATE orders SET status = 'shipped' WHERE order_id = 103;     -- Arjun's order
UPDATE orders SET sales_rep_id = 8 WHERE order_id = 101;       -- hand her order to Arjun (in a DO block)
```

```
 current_user | order_id | sales_rep_id |  status
--------------+----------+--------------+-----------
 ch14_priya   |      101 |            7 | delivered
 ch14_priya   |      105 |            7 | cancelled
 ch14_priya   |      108 |            7 | delivered
 ch14_priya   |      113 |            7 | pending

UPDATE 0
NOTICE:  priya: new row violates row-level security policy for table "orders"
```

Role	Policy that applies	count(*) from orders
`ch14_priya`	`rep_sees_own_orders`	4
`ch14_arjun`	`rep_sees_own_orders`	2
`ch14_analyst` (member of reporting)	`reporting_reads_all`	14
`ch14_app`	none — has `SELECT` but no policy	**0**
`postgres` (superuser / owner)	bypasses RLS	14

- `USING` filters which *existing* rows are visible (to `SELECT`, `UPDATE`, `DELETE`) — so the `UPDATE` of order 103 found 0 rows rather than failing.
- `WITH CHECK` validates rows being *written* (`INSERT`, `UPDATE`) — so Priya can't move a row out of her own view.
- Policies are **permissive** by default (combined with `OR`); `AS RESTRICTIVE` policies are combined with `AND`.

Multi-tenant isolation with a session setting

SaaS (software as a service: one hosted app shared by many customer organisations) apps usually connect as **one** application role and tell the database which tenant the current request belongs to:

```sql
ALTER TABLE notes ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON notes
    USING (tenant = current_setting('app.tenant', true));    -- true: unset -> NULL, not an error

SET ROLE ch14_app;
SET app.tenant = 'acme';    SELECT * FROM notes;          -- only acme's 2 notes
SET app.tenant = 'globex';  SELECT * FROM notes;          -- only globex's note
RESET app.tenant;           SELECT count(*) FROM notes;   -- 0: no tenant, no rows
```

The app runs `SET LOCAL app.tenant = …` at the start of each transaction (`SET LOCAL` resets at commit — important with connection pools (sets of reusable open connections), 14.12, where the next request reuses the connection).

Common mistakes

- **Testing RLS as a superuser or the table owner** — both bypass it, so "it works" proves nothing. Test with `SET ROLE`, and use `ALTER TABLE … FORCE ROW LEVEL SECURITY` if the owner must be filtered too.
- Letting the application connect as a superuser or the table owner "because it's easier".
- Forgetting that new tables aren't covered by old grants — use `ALTER DEFAULT PRIVILEGES … GRANT …` for future tables.
- Leaving the `PUBLIC` pseudo-role (an implicit role every user belongs to) with rights you didn't intend (in PG ≤ 14, everyone can `CREATE` in the `public` schema; PG 15 removed that default).

Dialect differences

	Roles / users	Row-level security
PostgreSQL	Roles (users = roles with `LOGIN`)	`CREATE POLICY` (9.5+)
MySQL 8.0	Users + roles	None built in — use views or app logic
SQL Server 2016+	Logins, users, roles	Security policies with inline predicate functions (functions returning a true/false filter)
Oracle	Users, roles	Virtual Private Database (VPD)
SQLite	No users — file-system permissions only	None

In the interview

30-second answer: "I apply least privilege: group roles that hold grants, login roles as members, the app gets only the DML it needs, reporting gets read-only, and nobody uses the superuser from code. For multi-tenant data I add row-level security keyed on a session variable, test it with `SET ROLE` — never as owner — and keep a tenant_id index so the policy filter is cheap."

- *USING vs WITH CHECK?* → Visibility of existing rows vs validity of new/changed rows.

Self-check

1. Why did `ch14_app` see 0 orders? → RLS is enabled and no policy applies to it — default deny.
2. What does `REVOKE` on a group do to its members? → They lose the inherited privilege.
3. Why `SET LOCAL` with pooled connections? → The setting ends with the transaction, so it can't leak to the next request.

TL;DR

Roles + `GRANT`/`REVOKE` for what each account may do; RLS policies for which rows it may see or write; test as the real role, never as owner.

14.9 SQL injection and parameterized queries

What it is

SQL injection happens when user input is pasted into the *text* of a SQL statement, so specially crafted input changes the statement's meaning. It has been in the OWASP (Open Worldwide Application Security Project) Top 10 list of web risks since the list began, and has caused many real breaches (for example the 2015 TalkTalk breach in the UK and the 2023 MOVEit Transfer attacks).

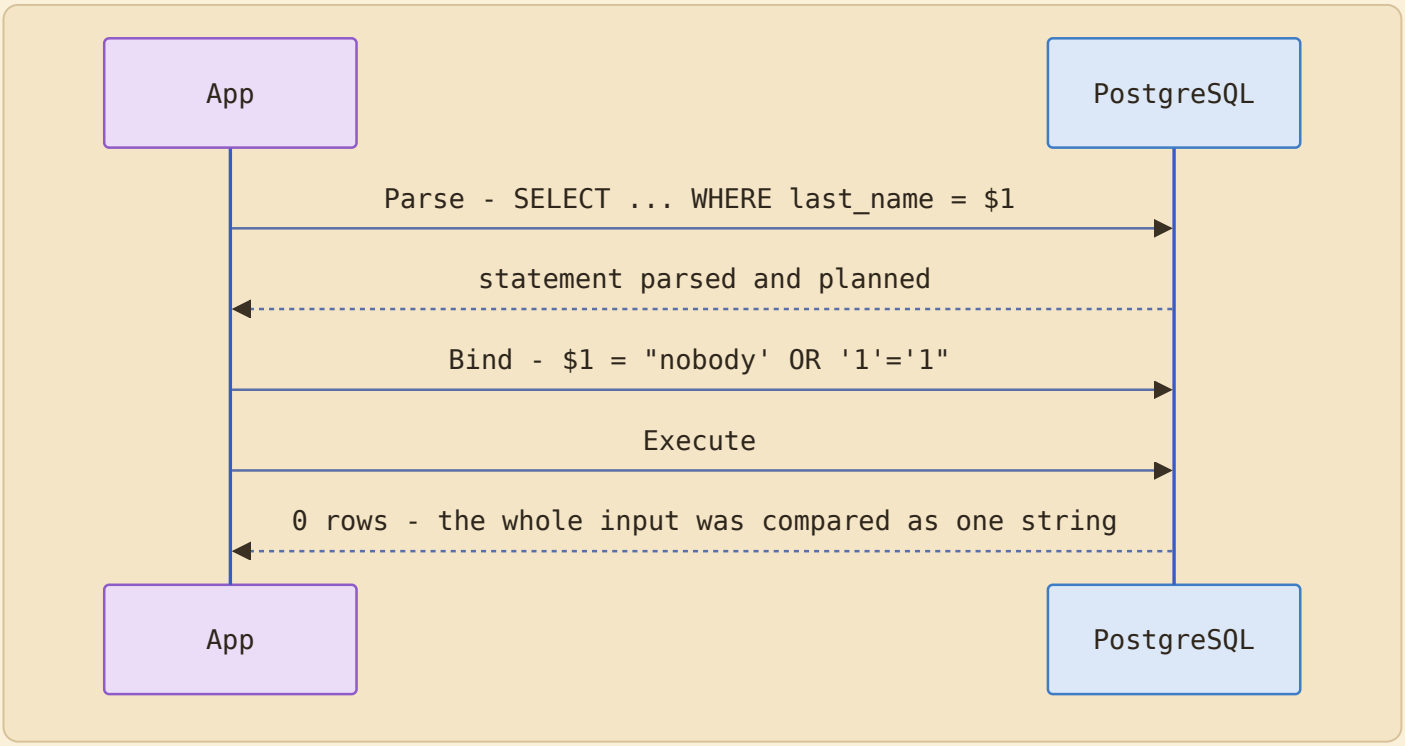
Suppose the code builds:

```
SELECT … FROM customers WHERE last_name = '<input>'
```

With the input `nobody' OR '1'='1`, the text becomes `… WHERE last_name = 'nobody' OR '1'='1'` — a condition that is always true.

How it's fixed — parameters

A **parameterized query** (also *prepared statement* or *bind variables*) sends the SQL text with placeholders (`$1`, `?`, `%s`) and the values **separately**. The server parses the SQL first, then plugs the values in as data. Input can never become SQL syntax, however many quotes it contains.



Example — injection inside the database (dynamic SQL)

The same bug appears in PL/pgSQL when `EXECUTE` runs a string. From `code/09_sql_injection.sql`:

```
-- VULNERABLE
CREATE FUNCTION find_customers_bad(p_last_name text)
RETURNS TABLE (customer_id int, first_name text, email text)
LANGUAGE plpgsql AS $$
BEGIN
    RETURN QUERY EXECUTE
        'SELECT customer_id, first_name, email FROM customers WHERE last_name = '''
        || p_last_name || '''';
END;
$$;
SELECT * FROM find_customers_bad('Carter');                -- 1 row: Ben
SELECT * FROM find_customers_bad($$nobody' OR '1'='1$$);     -- 10 rows: every customer leaks

-- SAFE: the value is passed with USING, referenced as $1
CREATE FUNCTION find_customers_good(p_last_name text) ... AS $$
BEGIN
    RETURN QUERY EXECUTE
        'SELECT customer_id, first_name, email FROM customers WHERE last_name = $1'
        USING p_last_name;
END;
$$;
SELECT count(*) AS rows_leaked FROM find_customers_good($$nobody' OR '1'='1$$);   -- 0
```

Identifiers can't be parameters. Placeholders stand for *values*, never for table or column names. When a name must be dynamic, check it against an **allow-list** (a fixed list of permitted names) and quote it with `format('%I')` (identifier) — use `%L` for a literal:

```
SELECT format('SELECT * FROM %I WHERE name = %L', 'Order Items', $$O'Brien$$);
--  SELECT * FROM "Order Items" WHERE name = 'O''Brien'
```

`count_rows(p_table)` in the file raises an error for any table outside `('orders','customers','products')` before building its query. SQL-level prepared statements work the same way: `PREPARE orders_of(int) AS … WHERE customer_id = $1; EXECUTE orders_of(2);`.

The client-side versions of this demo (Python f-string vs `%s`, Java string concatenation vs `PreparedStatement`) are in 14.10 and 14.11 — all show **10 rows vs 0 rows**.

Defence in depth

Layer	Defence
Query	Parameters for every value; allow-list + `%I` / `quote_ident` for identifiers
Account	Least-privilege database role (14.8) — injection can't drop tables the role can't touch
Data	RLS for tenant isolation
Code	Linters/static analysis (tools that inspect code without running it) that flag string-built SQL; code review
Errors	Don't show raw database errors to users (they help attackers map the schema)

Escaping by hand (doubling quotes yourself) is fragile — encodings (the byte representation of characters) and edge cases have broken it repeatedly. Use parameters.

In the interview

30-second answer: "Injection is user input interpreted as SQL because it was concatenated into the query. The fix is parameterized queries — values travel separately and can't change the parse tree (the server's structured reading of the statement). For dynamic identifiers use an allow-list and identifier quoting. Add least privilege so a bug has limited blast radius (the amount of damage it

can do). ORMs (object-relational mappers — libraries that generate SQL from code objects, 14.13) parameterize by default, but raw-SQL escape hatches (helpers for writing your own SQL) in ORMs are still injectable."

- *Do prepared statements help performance too?* → Yes — the server can reuse the parsed/planned statement across executions.
- *Is `LIKE ?` safe?* → From injection yes, but `%` and `_` in the input still act as wildcards; escape them if they should be literal.

TL;DR

Never build SQL from input strings; bind values as parameters, allow-list identifiers, and limit what the database account can do.

14.10 Using SQL from Python

Drivers and the DB-API

Python talks to databases through **drivers** that follow the **DB-API 2.0** standard (PEP 249 — a PEP is a Python Enhancement Proposal, an official design document): `connect()` → a **connection** (a session with the server) → **cursors** (objects that run statements and hold results) → `execute()`, `fetchone()`, `fetchall()`, `commit()`, `rollback()`. Two drivers are shown here:

	`psycopg` (version 3)	**`sqlite3`**
Database	PostgreSQL server	SQLite file or in-memory DB, in-process (runs inside your program, no separate server)
Install	`pip install "psycopg[binary]"` (pip is Python's package installer; already in `.venv-books`, the book's virtual environment)	Built into Python
Placeholder	`%s` or `%(name)s` (*never* Python `%` formatting)	`?` or `:name`
Transactions	Implicit `BEGIN` unless `autocommit=True`; `with conn.transaction():` blocks	Implicit transaction before DML (data-changing statements: `INSERT`, `UPDATE`, `DELETE`); `with conn:` commits/rolls back

See also the Python book's standard-library tour for `sqlite3` basics (Python ch. 14).

Example — psycopg against the private server

From `code/psycopg_demo.py` (it creates a fresh database `ch_14_code` and loads the shared schema). Key parts:

```
SERVER = "host=/tmp port=5544 user=postgres"       # Unix socket directory + port
DSN = SERVER + " dbname=ch_14_code"

# autocommit=True: each statement commits on its own, and conn.transaction() blocks are
# real BEGIN ... COMMIT/ROLLBACK units
with psycopg.connect(DSN, autocommit=True) as conn:
    # 1. parameters: values are sent separately from the SQL text
    cur = conn.execute(
        "SELECT first_name, city FROM customers WHERE country = %s AND signup_date >= %s ORDER BY
customer_id",
        ("India", "2025-01-01"))
    for row in cur.fetchall():
        print("  ", row)
    with conn.cursor(row_factory=dict_row) as cur:              # rows as dicts
        cur.execute("SELECT name, unit_price FROM products WHERE unit_price > %(min)s ORDER BY unit_price
DESC",
                    {"min": 300})

    # 2. injection: f-string vs parameter; identifiers via psycopg.sql
    evil = "nobody' OR '1'='1"
    conn.execute(f"SELECT count(*) FROM customers WHERE last_name = '{evil}'")      # NEVER
    conn.execute("SELECT count(*) FROM customers WHERE last_name = %s", (evil,))
    q = sql.SQL("SELECT {} FROM customers WHERE customer_id = %s").format(sql.Identifier("city"))

    # 3. a transaction: both updates or neither
    try:
        with conn.transaction():
            conn.execute("UPDATE products SET unit_price = unit_price * 0.9 WHERE category =
'Furniture'")
            conn.execute("UPDATE products SET unit_price = -1 WHERE product_id = 2")   # CHECK fails
    except psycopg.errors.CheckViolation as e:
        print("rolled back:", e.diag.message_primary)

    # 4. batch (executemany), bulk load (cur.copy("COPY ... FROM STDIN") + write_row),
    #    and a named = server-side cursor that streams rows in chunks
    with conn.transaction(), conn.cursor(name="big_scan") as cur:
        cur.itersize = 4                                        # rows per round trip
        cur.execute("SELECT g FROM generate_series(1, 10) g")
        print("streamed:", [r[0] for r in cur])
```

Output:

```
connected to ch_14_code on port 5544 | server 14

== 1. query, fetch, parameters ==
customers: 10
   ('Chen', 'Bengaluru')
   ('Divya', 'Delhi')
   ('Fatima', None)
   ('Anita', 'mumbai')
   ('Ivan', 'Delhi')
dict rows: [{'name': 'Laptop Pro 14', 'unit_price': Decimal('1299.00')}, ...]   (3 dicts, shortened here)

== 2. SQL injection: string building vs parameters ==
built SQL : SELECT count(*) FROM customers WHERE last_name = 'nobody' OR '1'='1'
f-string  : 10 rows
parameter : 0 rows
composed  : SELECT "city" FROM customers WHERE customer_id = %s -> Bengaluru

== 3. transactions ==
rolled back: new row for relation "products" violates check constraint "products_unit_price_check"
desk price after rollback: 499.00
desk price after commit: 479.00

== 4. batch insert, COPY, streaming ==
executemany rowcount: 3
customers now: 16
streamed: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

== 5. connection pooling ==
no pool : 20 requests used 20 server processes
pool(2) : 20 requests used 2 server processes
```

Notes on what the output shows:

- **Type mapping**: SQL `NULL` → `None`, `numeric` → `Decimal` (exact, never `float` for money), `date` → `datetime.date`.
- **Transactions**: the 10 % furniture discount was rolled back together with the failing update — the desk still costs 499.00. Exceptions are typed by SQLSTATE (`psycopg.errors.CheckViolation`, `UniqueViolation`, …).
- **Bulk loading speed**, fastest last: one `execute` per row → `executemany` (psycopg 3 pipelines it — sends many statements without waiting for each reply) → `COPY` (PostgreSQL's bulk-load command).
- **Server-side cursor** (`conn.cursor(name=…)`): the result stays on the server and is fetched in chunks of `itersize`, so a 10-million-row export doesn't need 10 million rows in memory.
- **Default transaction trap**: without `autocommit=True`, psycopg sends `BEGIN` before your first statement and nothing is saved until `conn.commit()` (or leaving `with psycopg.connect(...)` normally). A forgotten commit, or a connection left "idle in transaction" (connected, with a transaction open, doing nothing), is a classic bug.

Example — sqlite3

From `code/sqlite3_demo.py` — same ideas, `?` placeholders, no server:

```
conn = sqlite3.connect(":memory:")
conn.row_factory = sqlite3.Row
conn.execute("PRAGMA foreign_keys = ON")    # SQLite leaves FK checks off unless asked
...
with conn:                                  # commit on success, rollback on exception
    conn.executemany("INSERT INTO products (name, price, specs) VALUES (?, ?, ?)", rows)
for r in conn.execute("SELECT product_id, name, price FROM products WHERE price < :max ORDER BY price",
{"max": 40}):
    print(dict(r))
```

```
{'product_id': 1, 'name': 'Wireless Mouse', 'price': 24.99}
{'product_id': 3, 'name': 'Desk Lamp', 'price': 35.0}
rolled back: UNIQUE constraint failed: products.name
hub price: 49.5
f-string  rows: 3
parameter rows: 0
sqlite 3.53.4 | brands: ['Clicko', 'Nimbus']
```

(The version line shows the SQLite library bundled with this Python; yours may differ.) Note `with conn:` in `sqlite3` manages the **transaction**, not closing — call `conn.close()` yourself.

Self-check

1. Why is `cur.execute("… = %s" % value)` wrong? → That's Python string formatting — injection again. Pass values as the second argument.
2. How do you stream a huge result in psycopg? → A named (server-side) cursor with `itersize`.
3. What type does `numeric` become? → `decimal.Decimal`.

14.11 Using SQL from Java (JDBC)

What it is

JDBC (Java Database Connectivity) is Java's standard database API in `java.sql`. A vendor **driver** jar (a packaged Java library file) implements it; for PostgreSQL that's `org.postgresql:postgresql` (version 42.7.13 here, from Maven Central, the main public repository of Java libraries). The core types:

Type	Role
`DriverManager` / `DataSource`	Hands out connections (a `DataSource` is how pools plug in)
`Connection`	A session; controls auto-commit, commit/rollback, isolation level
`PreparedStatement`	SQL with `?` placeholders, typed setters (`setString`, `setObject`, `setBigDecimal`)
`ResultSet`	A cursor over rows; `next()` + typed getters
`SQLException`	Errors, with `getSQLState()` (e.g. `23514` = check violation)

Resources are closed with **try-with-resources** (Java syntax that closes objects automatically at the end of a block) so connections are never leaked. For the Java language side see the Java book (ch. 8 exceptions, ch. 18 tooling & JDBC).

Running it. The jar lives in `.java-books/lib/`. Because the book's private server listens only on a Unix socket (`/tmp/.s.PGSQL.5544`), the URL adds the `junixsocket` socket factory (a plug-in that lets Java connect through a Unix socket — a same-machine connection without networking; two more jars in the same folder); against a normal server the URL is just `jdbc:postgresql://localhost:5544/ch_14_jdbc`. The `java` launcher can compile and run a single source file in one step (source-file mode, Java 11+; the book targets Java 25):

```
java --enable-native-access=ALL-UNNAMED -cp '.java-books/lib/*' \
     sql/chapters/14-advanced-sql-and-using-sql-from-code/code/JdbcDemo.java
```

(The file starts with `// tools_run: skip` so the Java book's generic runner doesn't try to run it without the driver.)

Example — key parts of `code/JdbcDemo.java`

```
// PreparedStatement: '?' placeholders, typed setters and getters
String sql = """
        SELECT first_name, city, signup_date
        FROM customers WHERE country = ? AND signup_date >= ? ORDER BY customer_id""";
try (PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setString(1, "India");
    ps.setObject(2, LocalDate.of(2025, 1, 1));              // java.time maps to DATE
    try (ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
            String city = rs.getString("city");              // SQL NULL -> Java null
            LocalDate d = rs.getObject("signup_date", LocalDate.class);
            ...
        }
    }
}

// Transaction: turn auto-commit off, commit or roll back as one unit
conn.setAutoCommit(false);
conn.setTransactionIsolation(Connection.TRANSACTION_REPEATABLE_READ);
try (PreparedStatement up = conn.prepareStatement(
        "UPDATE products SET unit_price = unit_price + ? WHERE product_id = ?")) {
    up.setBigDecimal(1, new BigDecimal("-10.00")); up.setInt(2, 5); up.executeUpdate();
    up.setBigDecimal(1, new BigDecimal("-99.00")); up.setInt(2, 2); up.executeUpdate(); // CHECK fails
    conn.commit();
} catch (SQLException e) {
    conn.rollback();                                         // undo BOTH updates
} finally {
    conn.setAutoCommit(true);
}

// Batch: setXxx(...) + ps.addBatch() per row, then int[] counts = ps.executeBatch(); conn.commit();
```

Output:

```
connected: PostgreSQL 14.15 (Homebrew) via PostgreSQL JDBC Driver 42.7.13

== PreparedStatement ==
  Chen   Bengaluru  2025-01-08
  Divya  Delhi      2025-01-20
  Fatima (null)     2025-02-14
  Anita  mumbai     2025-03-09
  Ivan   Delhi      2025-05-30

== SQL injection ==
concatenated : 10 rows
parameterized: 0 rows

== transaction ==
rolled back, SQLState 23514: ERROR: new row for relation "products" violates check constraint
"products_unit_price_check"
desk price unchanged: 499.00

== batch ==
batch update counts: [1, 1, 1]
  Monitor Arm -> 61.95
  Webcam HD -> 83.90
  Cable Tray -> 20.48
```

The last block used `UPDATE … RETURNING name, unit_price` with `executeQuery()` — PostgreSQL's `RETURNING` gives back computed values without a second query (JDBC's `getGeneratedKeys()` is the portable

alternative for new ids).

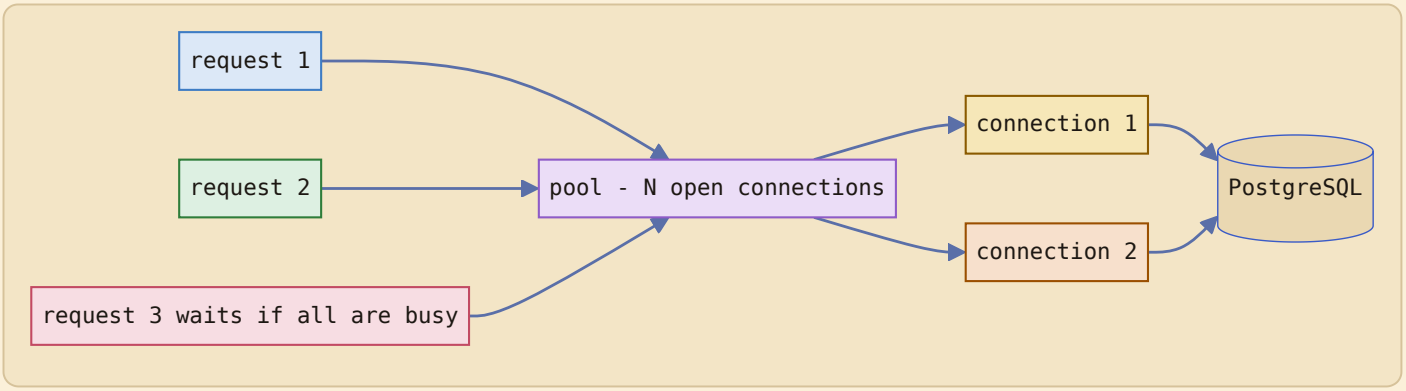
Common mistakes (Python and Java alike)

- **Leaking connections** — not closing them (use `with` / try-with-resources); the server runs out of connection slots.
- **Using `float`/`double` for money** — use `Decimal` / `BigDecimal`.
- **Forgetting `rs.wasNull()`**: `getInt` returns `0` for SQL `NULL`; use `getObject(col, Integer.class)` or check `wasNull()`.
- **Auto-commit surprises** — JDBC starts in auto-commit *on* (every statement its own transaction); psycopg starts with it *off*. Know which one your code is in.
- **One query per row** in a loop (the N+1 pattern, 14.13) instead of a join or `= ANY(?)` with an array (`conn.createArrayOf("int", ids)`).

14.12 Connection pooling

What it is

Opening a PostgreSQL connection is expensive: a network handshake (the opening back-and-forth that sets up a connection), TLS (the encryption layer for network traffic; see System Design Fundamentals), authentication (checking who you are), and — PostgreSQL uses a **process per connection** — forking (the operating system copying a process to create) a new server process that allocates its own memory. That costs milliseconds, while a simple indexed query takes well under one. A **connection pool** opens a few connections once and **lends** them to requests, like a library lending books instead of printing a new copy for each reader.



The last section of `psycopg_demo.py` shows the difference with a 15-line `TinyPool` (a `queue.Queue` of open connections): 20 requests without a pool created **20 server processes**; with a pool of 2 they used **2**.

How it works

1. At start-up the pool opens *min* connections.
2. A request **borrows** a connection (`getConnection()` / `pool.connection()`), waiting up to a timeout if all are busy.
3. When done, it **returns** it; the pool resets session state (rolls back an open transaction, may run a validation query) and keeps it open.
4. The pool closes connections that are idle too long or reach a maximum lifetime.

Kinds of pools

Where	Examples	Notes
In the application	HikariCP (Java; Spring Boot's — a popular Java web framework — default), `psycopg_pool`, SQLAlchemy's pool, node-postgres `Pool`	Per process; total = processes × pool size
External proxy (a separate program sitting between the apps and the database)	PgBouncer, pgcat, Amazon RDS Proxy	Shares a small set of server connections among many app processes

PgBouncer's **transaction mode** gives a server connection to a client only for the length of one transaction — very efficient, but session features (plain `SET`, session advisory locks — application-defined locks on a number — and `LISTEN`, PostgreSQL's notification subscription) don't carry across transactions; use `SET LOCAL` (as in 14.8).

Sizing

More connections is **not** faster: past a point, processes compete for CPU cores and disk, and each costs memory (`max_connections` defaults to 100). A widely quoted starting point from the HikariCP project is roughly *(CPU cores × 2) + effective disk spindles* (the number of physical disks working in parallel), then measure. Watch the pool's **wait time** — if requests queue, look for slow queries before raising the size.

Common mistakes

- Holding a connection during slow non-database work (calling an external API mid-transaction).
- Session state leaking between borrowers (`SET` without `LOCAL`, temp tables, open transactions).
- Every microservice replica (one running copy of a small, separately deployed service) with a pool of 50 → thousands of connections; use an external pooler.

Self-check

1. Why are PostgreSQL connections costly? → Handshake + auth + a forked server process per connection.
2. What breaks in PgBouncer transaction mode? → Session-level state such as plain `SET` and session advisory locks.

14.13 ORMs vs raw SQL

What it is

An **ORM** (Object-Relational Mapper) maps tables to classes and rows to objects, and generates SQL for you: Hibernate/JPA (Java), SQLAlchemy and the Django ORM (Python), Entity Framework (.NET), ActiveRecord (Ruby), Prisma (TypeScript). In between sit **query builders** and **mappers** — jOOQ, MyBatis, Knex, SQLAlchemy Core — which help write SQL without hiding it. The mismatch between object graphs and tables is called the **object-relational impedance mismatch**.

Trade-offs

	ORM	Raw SQL / query builder
CRUD (create, read, update, delete) code	Very little	More boilerplate (repetitive setup code)
Safety	Parameterized by default	Parameterized if you do it (always do)
Complex queries (windows, CTEs, pivots, JSONB, FTS)	Awkward or impossible → raw-SQL escape hatch	Full power of the database
Performance visibility	SQL hidden; easy to generate N+1 or huge joins	You see and tune every query
Portability	Easier to switch databases	Tied to a dialect (usually fine)
Learning curve	ORM concepts (sessions, lazy loading, dirty checking — the ORM tracking which objects changed) *plus* SQL	SQL
Schema changes	Often paired with migrations (versioned schema-change scripts: Alembic, Flyway, Liquibase — these work without an ORM too)	Same migration tools

The N+1 query problem

The classic ORM trap: load 100 orders (1 query), then touch `order.customer` for each one, and **lazy loading** fires one more query per order (100 queries). In SQL terms:

```sql
-- N+1: what lazy loading does
SELECT * FROM orders WHERE status = 'shipped';           -- 1 query
SELECT * FROM customers WHERE customer_id = 6;           -- + 1 per order ...
SELECT * FROM customers WHERE customer_id = 7;
-- fix: one join (ORM: eager loading, e.g. JOIN FETCH / joinedload / select_related)
SELECT o.*, c.first_name FROM orders o JOIN customers c USING (customer_id) WHERE o.status = 'shipped';
-- or one batched lookup: WHERE customer_id = ANY($1)   (ORM: batch fetching / selectinload / prefetch_related)
```

Spot it by logging SQL in development and counting queries per request. (Indexing side: Chapter 12.)

When to use which

- **ORM**: CRUD-heavy business apps, many simple entities, teams that value productivity — while still reading the SQL it generates.
- **Raw SQL / builder**: reporting, analytics, bulk jobs, anything using window functions, CTEs, JSONB or FTS, and hot paths (the most frequently run code) that need tuning.
- **Most real systems mix both**: ORM for everyday writes, hand-written SQL for the hard 10 %.

In the interview

30-second answer: "ORMs speed up CRUD and parameterize by default, but hide the SQL, which invites N+1 queries and inefficient joins. I use an ORM for simple entity work, drop to SQL or a query builder for reporting and performance-critical paths, log generated SQL, and fix N+1 with eager or batched loading."

- *How do you detect N+1?* → Count queries per request in logs/APM (application performance monitoring tools); a request running the same query shape 100 times is the tell.
- *Are ORMs immune to injection?* → Their generated queries are, but raw-SQL helpers with string concatenation are not.

Self-check

1. What is lazy loading? → Related objects are fetched only when first accessed — a hidden query each time.
2. Two fixes for N+1? → A join (eager loading) or one `IN` / `= ANY` batch query.

Chapter TL;DR

Topic	Remember
Functions	`sql` for simple, `plpgsql` for procedural; honest volatility; set-based beats per-row calls
Procedures	`CALL`, `INOUT`, can `COMMIT` in batches — only from auto-commit mode
Triggers	BEFORE modifies/rejects, AFTER reacts; audit with `to_jsonb(OLD/NEW)`; statement-level + transition tables for bulk; no slow external work
JSONB	`->` json, `->>` text, `@>` contains, GIN index; core fields stay columns
Arrays	1-based; `ANY`, `@>`, `&&`, `unnest`, `array_agg`; `= ANY($1)` for id lists from code
Full-text search	`tsvector @@ tsquery`, stemming, `websearch_to_tsquery`, `ts_rank`, weighted generated column + GIN
Pivot	`FILTER` / `CASE` aggregates; `crosstab` needs tablefunc; unpivot with `LATERAL (VALUES …)`
Security	Group roles, least privilege, `GRANT` / `REVOKE`; RLS `USING` (read) + `WITH CHECK` (write); owners/superusers bypass
Injection	Never concatenate input; bind parameters; allow-list + `%I` for identifiers
From code	psycopg `%s`, sqlite3 `?`, JDBC `?`; explicit transactions; `Decimal` / `BigDecimal`; batch/COPY; stream big results
Pooling	Reuse connections; small pools; `SET LOCAL`; PgBouncer for many clients
ORMs	Productive for CRUD, watch for N+1; raw SQL for complex/hot queries

Next: Chapter 15 — Scaling databases, which takes one server to many: partitioning, replication, sharding and failover.

Chapter 15 — Scaling Databases: Partitioning, Sharding, Replication and Recovery

A single PostgreSQL server goes a long way: one machine with plenty of **RAM** (the fast working memory a computer loses when powered off), fast **SSDs** (solid-state drives — disks with no moving parts, much faster than spinning hard drives) and good indexes (Chapter 12) can serve thousands of queries per second. Sooner or later, though, the data outgrows one disk, the traffic outgrows one **CPU** (the processor that executes instructions), or the business can't accept that one broken machine takes the whole product offline. This chapter is about what databases do when **one box is no longer enough**.

Two ideas carry most of the weight:

- **Partitioning / sharding**: split the data so each piece is smaller (more capacity, faster scans).
- **Replication**: copy the data so there is more than one of each piece (availability, read scaling, disaster recovery).

Then come the problems they create: copies disagree (consistency models, CAP), machines must agree (consensus, leader election), and failures must not lose committed data (logging, checkpoints, two-phase commit, failover, backups).

What you'll learn

Section	Topic	Runnable part
15.1	What a distributed database is, scale up vs scale out	—
15.2	Partitioning: horizontal/vertical, range/list/hash/composite/round-robin	Postgres declarative partitioning + pruning with `EXPLAIN`
15.3	Sharding, shard keys, consistent hashing	Python: keys moved when a shard is added
15.4	Distributed query processing	`postgres_fdw`: predicate/aggregate push-down
15.5	Replication: types, sync vs async, single/multi/leaderless	Python: quorum reads/writes
15.6	Consistency models	—
15.7	CAP and PACELC	—
15.8	Consensus (Paxos, Raft, PBFT, PoW) and leader election	Python: Bully vs Ring message counts
15.9	Fault tolerance: WAL, redo/undo, LSNs, checkpoints, 2PC/3PC	Postgres WAL/LSN demo, Python recovery + 2PC
15.10	High availability and failover	—
15.11	Data resilience: redundancy, mirroring, backups	—

> **Going deeper.** This chapter explains each idea once, at interview depth. For book-level depth see your DDIA notes — Replication, Partitioning, Trouble with distributed systems, Consistency and consensus — and for the design-interview pattern view the system-design-patterns notes: Storing data (primary-replica, sharding, consistent hashing, WAL), Consistency (2PC, saga, quorum, vector clocks), Scaling, Reliability.

All SQL files are in `code/` and run on the book's private PostgreSQL 14 server (`bash tools_run_sql.sh sql '15-*'`). `00_schema.sql` creates 12 customers in three regions (`NA`, `EU`, `APAC`) and 3,000 orders spread over 2022–2024. Python simulations run with the books' venv (`.venv-books/bin/python code/<file>.py`).

15.1 Distributed databases: the big picture

What it is

A **distributed database** stores its data on several machines (**nodes**), possibly in different **data centres** (buildings full of servers, power and network gear), but presents itself to applications as *one* database. Think of a library with branches across a city: you search one catalogue and borrow from whichever branch has the book, without knowing which building holds which shelf.

Term / characteristic	Meaning
Node	One server (physical or virtual) running a database process. **Cluster** = the group of nodes forming the database.
Transparency	The client doesn't need to know *where* a row is stored (**location transparency**) or *how many copies* exist (**replication transparency**).
Fault tolerance	The system keeps working when individual nodes fail, because data is copied elsewhere.
Concurrency	Many transactions on many nodes run at once without corrupting shared data (the concurrency control of Chapter 13, now across machines).
Scalability	Capacity grows by adding nodes.
Network partition	Some nodes can't talk to others because the network between them failed. Don't confuse it with *data* partitioning (15.2).

Scale up vs scale out

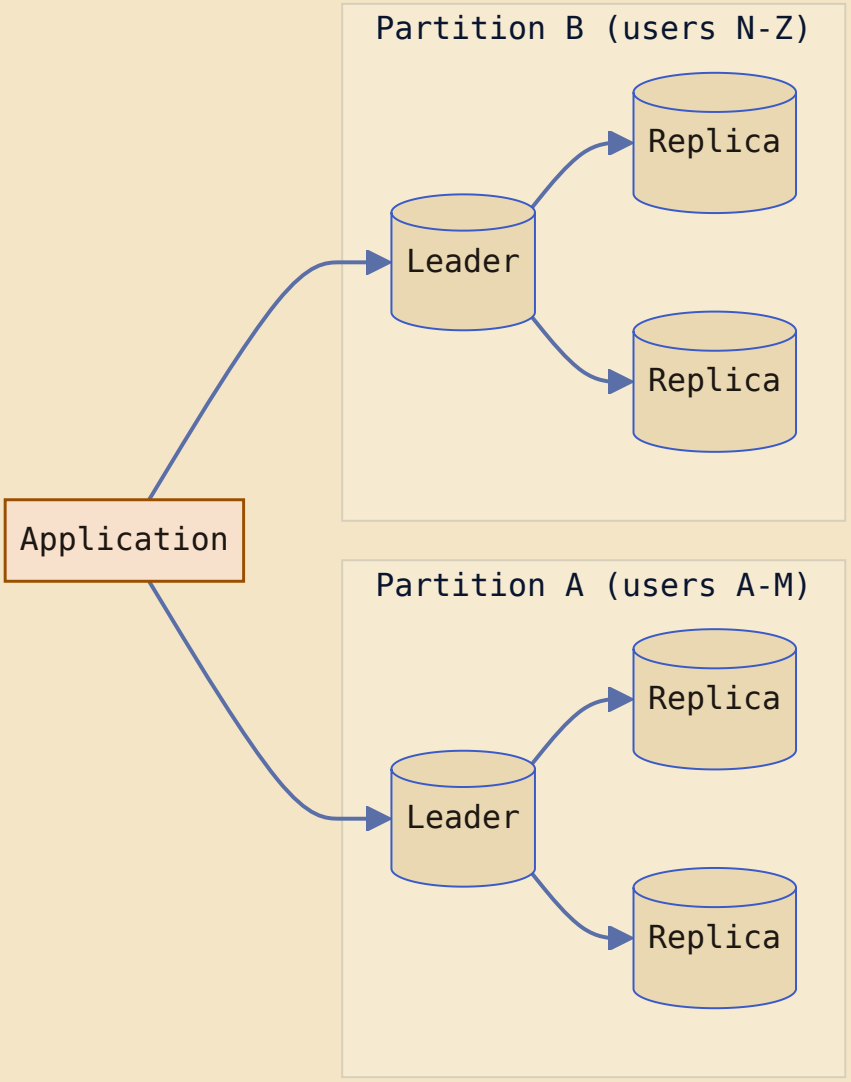
	Vertical scaling (scale up)	Horizontal scaling (scale out)
How	Bigger machine: more CPU, RAM, faster disks	More machines, each holding part of the data/load
Example	Move Postgres from 8 to 64 cores for Black Friday	Add nodes to a Cassandra or CockroachDB cluster as users grow
Limit	The biggest machine you can buy (and its price)	Coordination cost between nodes
Failure	Still one machine = one point of failure	Survives node loss if data is replicated
App changes	None	Often needed (shard keys, cross-node queries)

Distributed databases are built for **scale-out**. Scale-up is simpler and usually comes first — many companies run a large Postgres or MySQL **primary** (the one server that accepts writes) plus **read replicas** (copies that serve only reads) for years before they shard. For a gentle introduction to these system-design building blocks see System Design Fundamentals.

Benefits and challenges

Benefits	Challenges
High availability (the service stays usable almost all the time) — other copies serve if a node or region dies	**Consistency** of copies written in different places (15.5–15.7)
Lower latency (latency = the time one request takes, start to finish) — data near its users (Singapore reads a Singapore node)	**Network latency** — milliseconds per cross-region hop
Load spreading — no node takes every request	**Failure detection** — silent may mean dead or slow (15.8–15.10)
Capacity grows with node count	**Operational complexity** — harder debugging and upgrades

How scale-out works: partition *and* replicate



Partitioning gives each node a *subset* of the data (capacity, parallel work); **replication** stores each subset on *several* nodes (fault tolerance, read capacity). Most real systems do both. *Example:* a food-delivery app in 20 countries partitions menus and orders by country (queries are local, and data-residency laws such as **GDPR** (the EU's General Data Protection Regulation, which restricts where and how personal data is stored) are easier to meet), replicates each partition to three **availability zones** (separate data centres in one cloud region, with independent power and networking), and adds nodes to the busiest cities on New Year's Eve.

Self-check

1. *Which two techniques does almost every distributed database combine?* Partitioning (split) and replication (copy).
2. *Why try scale-up first?* No application changes and no distributed-systems problems; its limits are cost and a **single point of failure** (one component whose failure takes everything down).

15.2 Partitioning

What it is

Partitioning splits one large table into smaller pieces (**partitions**) that are stored and managed separately while queries still see one logical table — like a filing cabinet with one drawer per year. Its goals: **faster queries** (a query for March 2023 reads only the 2023 partition — **partition pruning**), **easier maintenance** (archive a year by detaching one partition), **scalability** (partitions can live on different nodes — then it's *sharding*, 15.3), and **fault tolerance** (each partition can be replicated independently).

Horizontal vs vertical

Aspect	Horizontal (split rows)	Vertical (split columns)
Each piece has	All columns, some rows (2023 orders, 2024 orders)	Some columns, all rows; pieces share the primary key (1:1)
Speeds up	Queries that filter on the partition key	Queries that need only a few columns
Cost	Routing; queries without the key hit every piece	Joins to reassemble a full row
Typical use	Huge event/order/log tables, distributed DBs	Wide tables with big text/JSON/blob columns

A vertical split (`code/05_vertical_partitioning.sql`) keeps listing-page columns (`product_id`, `name`, `price`) in `product_core` and the long `description` / `spec_sheet` text in `product_details`, whose primary key also references `product_core`. With 1,000 products, `pg_total_relation_size` shows:

```
 core_size | details_size
-----------+--------------
 128 kB    | 1672 kB
```

The listing page now reads 128 kB instead of about 1.8 MB; a view (`product_full`) joins the halves back when needed. Postgres already moves very large values out of line by itself (**TOAST**, Chapter 12), so a manual split pays off most for medium-sized, rarely read columns.

Partitioning strategies

Strategy	Rule	Good for	Watch out for
Range	Key in an interval, e.g. `2023-01-01 ≤ date < 2024-01-01`	Time data, "last N days", archiving	**Hot partition**: all new writes hit the newest range
List	Key in a set of values, e.g. `region IN ('EU')`	Regions, tenants, data-residency rules	Uneven sizes; new values need a partition (or DEFAULT)
Hash	`hash(key) mod N`	Even spread with no natural range (`order_id`, `user_id`)	Range queries can't be pruned; changing N moves data
Composite	Two levels, e.g. range by month, then hash by server	High-volume time series	More partitions to manage
Round-robin	Row 1 → P1, row 2 → P2, row 3 → P3, row 4 → P1…	Perfectly even bulk loads and parallel scans	Can't find a row by key — every lookup scans all partitions

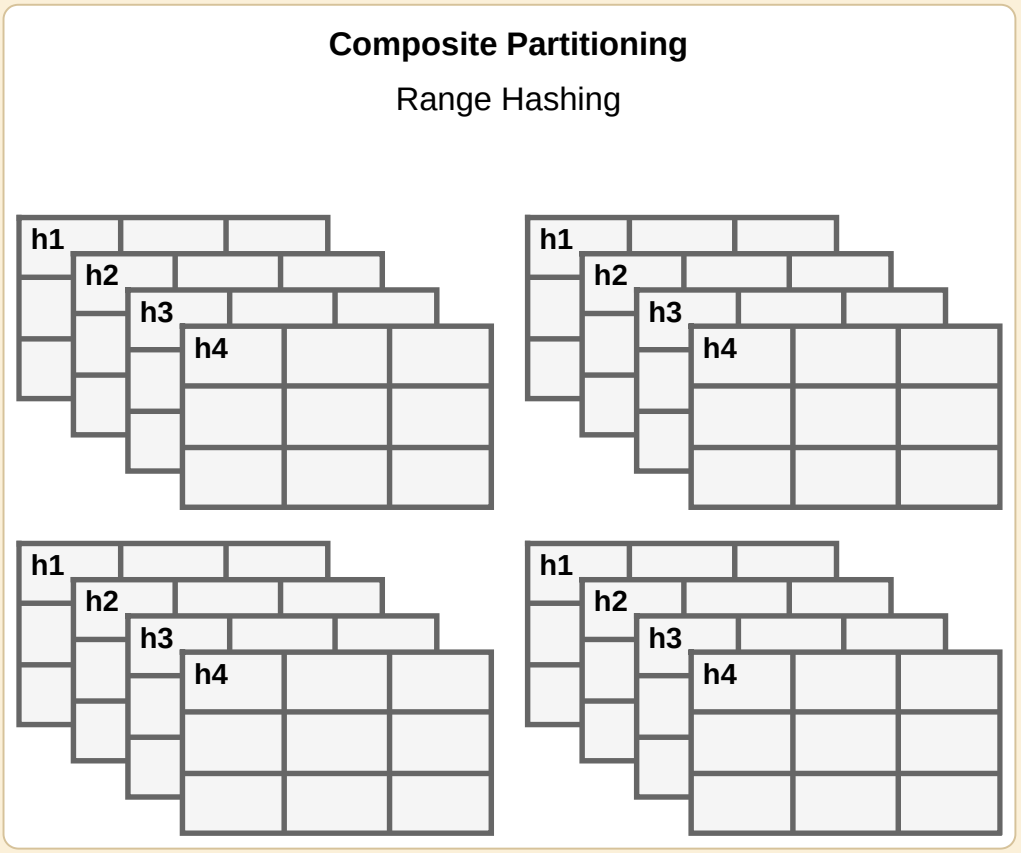


Figure — Composite (range + hash) partitioning: each range partition is itself split into hash buckets h1–h4 (course diagram).

Postgres supports range, list and hash natively, and composite by nesting them; it has no round-robin (that appears in warehouse systems, e.g. Azure Synapse's `ROUND_ROBIN` distribution). **To choose**, look at your most common and most expensive queries: the partition key should be in their `WHERE` clause so pruning works, it should spread data reasonably evenly, and adding/dropping partitions should stay simple.

How it works in PostgreSQL (declarative partitioning)

Declarative partitioning arrived in PostgreSQL 10 (hash partitioning, default partitions and primary keys on partitioned tables in 11). You create a **parent** with `PARTITION BY RANGE | LIST | HASH (columns)` — it stores no rows — and each **partition** with `CREATE TABLE … PARTITION OF parent FOR VALUES …`. Inserts into the parent are routed to the right partition (a row that fits none is rejected unless a `DEFAULT` partition exists), and a `WHERE` on the partition key lets the **planner** (the part of Postgres that decides how to execute a query) **prune** partitions that can't match.

Range partitioning, with pruning

`code/01_range_partitioning.sql`:

```
CREATE TABLE orders_by_year (
  order_id    int  NOT NULL,
  customer_id int  NOT NULL,
  order_date  date NOT NULL,
  amount      numeric(10,2) NOT NULL,
  PRIMARY KEY (order_id, order_date)          -- a PK must include the partition key
) PARTITION BY RANGE (order_date);

CREATE TABLE orders_2022 PARTITION OF orders_by_year FOR VALUES FROM ('2022-01-01') TO ('2023-01-01');
CREATE TABLE orders_2023 PARTITION OF orders_by_year FOR VALUES FROM ('2023-01-01') TO ('2024-01-01');
CREATE TABLE orders_2024 PARTITION OF orders_by_year FOR VALUES FROM ('2024-01-01') TO ('2025-01-01');
CREATE TABLE orders_other PARTITION OF orders_by_year DEFAULT;   -- anything outside the ranges

INSERT INTO orders_by_year SELECT * FROM orders;
INSERT INTO orders_by_year VALUES (9001, 1, '2025-03-15', 99);   -- no 2025 partition -> DEFAULT

SELECT tableoid::regclass AS partition, count(*) AS rows,
       min(order_date) AS first_day, max(order_date) AS last_day
FROM orders_by_year GROUP BY 1 ORDER BY 1;
```

```
  partition   | rows | first_day  |  last_day
--------------+------+------------+------------
 orders_2022  | 1016 | 2022-01-01 | 2022-12-31
 orders_2023  |  990 | 2023-01-01 | 2023-12-31
 orders_2024  |  994 | 2024-01-01 | 2024-12-31
 orders_other |    1 | 2025-03-15 | 2025-03-15
```

`FROM … TO …` has an **inclusive** lower and **exclusive** upper bound, so `2023-01-01` belongs to `orders_2023`; `tableoid::regclass` is a hidden column naming the physical table holding each row; and the PK includes `order_date` because each partition enforces uniqueness only on its own rows.

Pruning at plan time, versus a query that doesn't filter on the key:

```
EXPLAIN (COSTS OFF) SELECT sum(amount) FROM orders_by_year
WHERE order_date >= '2023-03-01' AND order_date < '2023-04-01';
EXPLAIN (COSTS OFF) SELECT sum(amount) FROM orders_by_year WHERE customer_id = 7;
```

```
 Aggregate
   ->  Seq Scan on orders_2023 orders_by_year
         Filter: ((order_date >= '2023-03-01'::date) AND (order_date < '2023-04-01'::date))

 Aggregate
   ->  Append
         ->  Seq Scan on orders_2022 orders_by_year_1
               Filter: (customer_id = 7)
         ...   (same for orders_2023, orders_2024 and orders_other)
```

`Append` means "scan each child and concatenate" — all four partitions are read. **Partitioning only helps queries that filter on the partition key.** When the value is known only at run time (a **prepared-statement** parameter — a prepared statement is a query parsed and planned once, then run many times with different values plugged into `$1`, `$2` …), Postgres 11+ prunes during execution:

```
SET plan_cache_mode = force_generic_plan;
PREPARE by_day(date) AS SELECT count(*) FROM orders_by_year WHERE order_date = $1;
EXPLAIN (COSTS OFF) EXECUTE by_day('2023-05-05');
```

```
Aggregate
  ->  Append
        Subplans Removed: 3
        ->  Seq Scan on orders_2023 orders_by_year_1
              Filter: (order_date = $1)
```

Maintenance is where range partitioning shines — archiving a year is a **metadata** change (an edit to the database's own bookkeeping about which tables exist, not to the rows), not a giant `DELETE`:

```
ALTER TABLE orders_by_year DETACH PARTITION orders_2022;         -- instant; now a plain table
DELETE FROM orders_other WHERE order_date >= '2025-01-01' AND order_date < '2026-01-01';
CREATE TABLE orders_2025 PARTITION OF orders_by_year FOR VALUES FROM ('2025-01-01') TO ('2026-01-01');
INSERT INTO orders_by_year VALUES (9001, 1, '2025-03-15', 99);
```

Afterwards the parent holds `orders_2023` (990), `orders_2024` (994) and `orders_2025` (1), and the detached `orders_2022` still has its 1,016 rows as an ordinary table. The `DELETE` is needed because Postgres refuses to create `orders_2025` while the DEFAULT partition still holds 2025 rows — better to create partitions ahead of time (a scheduled job or the `pg_partman` **extension** — an add-on package installed into Postgres with `CREATE EXTENSION`). `DETACH PARTITION … CONCURRENTLY` (Postgres 14+) avoids blocking queries on the parent.

List, hash and composite

`code/02_list_partitioning.sql` splits customers by region (`FOR VALUES IN ('NA')`, `('EU')`, `('APAC')`):

```
   partition    |         customers
----------------+---------------------------
 customers_na   | Ben, Elena, Grace, Jack
 customers_eu   | Chloe, Dmitri, Ines, Liam
 customers_apac | Asha, Farid, Hiro, Kavya

 Seq Scan on customers_eu customers_by_region
   Filter: (region = 'EU'::text)

NOTICE:  rejected: no partition of relation "customers_by_region" found for row
```

The last line comes from inserting a `'LATAM'` customer with no matching partition and no `DEFAULT` — a useful safety net that stops data landing in the wrong jurisdiction.

`code/03_hash_partitioning.sql` spreads orders over four partitions (`FOR VALUES WITH (MODULUS 4, REMAINDER 0..3)`): 730 / 752 / 774 / 744 rows — close to even. `WHERE order_id = 2024` prunes to one partition (`Seq Scan on orders_h2`), but `order_id BETWEEN 100 AND 110` produces an `Append` over all four, because hashing scatters neighbouring values.

`code/04_composite_partitioning.sql` partitions server logs by month, and each month is itself declared `PARTITION BY HASH (server_id)` with two leaves (`MODULUS 2, REMAINDER 0/1`):

```
 leaf_partition  | rows | servers
-----------------+------+---------
 logs_2024_01_h0 |  403 | 1,2
 logs_2024_01_h1 |  804 | 3,4,5,6
 logs_2024_02_h0 |  265 | 1,2
 logs_2024_02_h1 |  529 | 3,4,5,6

 Aggregate
   ->  Seq Scan on logs_2024_02_h1 server_logs
         Filter: ((logged_at >= '2024-02-10 00:00:00+00'::timestamp with time zone) AND
                  (logged_at < '2024-02-11 00:00:00+00'::timestamp with time zone) AND (server_id = 3))
```

A filter on both keys prunes to **one leaf of four**. The 2-vs-4 server split also teaches that **hashing evens things out only with many distinct key values** (six servers into two buckets is lumpy). `pg_partition_tree('server_logs')` lists the tree: parent (level 0), months (level 1), leaves (level 2).

Trade-offs and dialects

Postgres limits to know: unique/PK constraints must include every partition-key column; planning time grows with partition count (hundreds are fine, many thousands are heavy); an index on the parent becomes a *local* index on each partition — there is no global index; an `UPDATE` that changes the key moves the row (PG 11+); and `enable_partitionwise_join` / `_aggregate` (off by default) process matching partitions pairwise.

Other dialects *(not run)*: **MySQL** declares `PARTITION BY RANGE/LIST/HASH/KEY (...) (PARTITION p0 VALUES LESS THAN (...), ...)` inside `CREATE TABLE`, and every unique key must include the partitioning columns; **SQL Server** uses a partition function plus a partition scheme (range only); **Oracle** adds interval partitioning (auto-created ranges); **SQLite** has none.

Common mistakes

- A key the hot queries don't filter on (every query scans every partition); partitioning tables under a few million rows at all — a good index is simpler.
- Forgetting next month's partition, so inserts fail or pile into DEFAULT; thousands of tiny partitions; expecting hash partitioning to speed up range scans.

In the interview

30-second answer: "Partitioning splits a big table into physical pieces by a key: range for time, list for categories, hash for even spread, composite to combine. The win is pruning — the planner skips partitions the `WHERE` clause excludes — plus cheap maintenance like dropping a month instantly. It only helps queries that filter on the key, and in Postgres unique keys must include it."

- *Range vs hash for an `events` table?* Range by time if queries are "recent events" and old data is archived; hash by `user_id` if queries are per user and write load must be spread.
- *Hot-partition problem?* Range-by-time sends all current writes to the newest partition; mitigate with time + hash composite.

Self-check

1. *Why is `2023-01-01` in `orders_2023`?* Range upper bounds are exclusive.
2. *What does `Subplans Removed` mean?* Partitions pruned at execution time because the value was a parameter.
3. *Which strategy can't find a row by key without scanning everything?* Round-robin.

TL;DR

Partition by the column your big queries filter on: range for time, list for categories, hash for even spread, composite to combine. Check pruning with `EXPLAIN`; unique keys must include the partition key.

15.3 Sharding

What it is

Sharding is horizontal partitioning where each partition — a **shard** — lives in a **separate database server**, a complete, independent database holding a subset of the rows. A **shard key** decides which shard owns a row, and a **router** sends each query to the right shard(s). Example: `order_id` 1–1,000,000 on shard A and 1,000,001–2,000,000 on shard B; `WHERE order_id = 42` goes only to A.

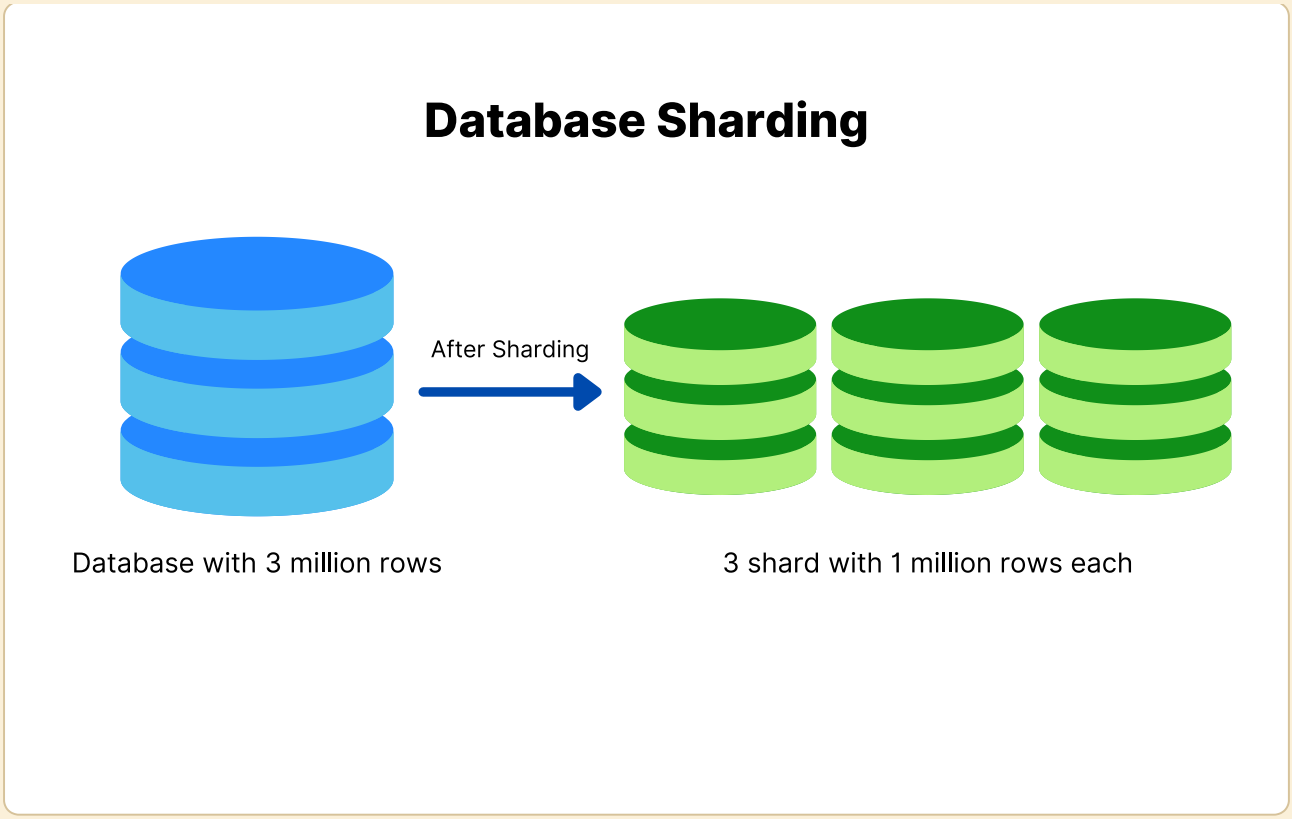


Figure — One database with millions of rows split into three shards, each an independent database (course diagram).

How it works

1. Pick a **shard key** (`user_id`, `tenant_id`).
2. Choose a **mapping** from key to shard: a range table, a hash, or a lookup directory.
3. A **router** applies it: the application, a **proxy** (a middle server that receives client connections and forwards them to the right database — Vitess for MySQL; pgcat/pgdog for Postgres), or the database itself (Citus, MongoDB `mongos`, CockroachDB).
4. Single-shard queries go straight to one shard; queries without the key are **scattered** to all shards and **gathered** — slower.
5. Each shard is **replicated** (15.5) — sharding alone doesn't remove single points of failure; a shard without replicas loses its slice of data when it dies.

Choosing the shard key (the most important decision)

Property	Why	Example
Spreads data and load evenly	Avoids a **hotspot** — one overloaded shard sets the speed of the whole system	`user_id` (millions of values), not `country`
Matches the main query pattern	Most queries hit exactly one shard	B2B SaaS sharded by `tenant_id`
Keeps related data together	Joins and transactions stay on one shard (**co-location**)	A user's orders on the user's shard
Rarely changes / minimises rebalancing	Changing the key or shard count means moving data	Immutable ids, not email addresses

Typical choices: `user_id` for social apps, `tenant_id` for SaaS, time ranges for time-series (but then all current writes hit the newest shard).

Compared with partitioning inside one server, sharding needs a router instead of the planner, turns cross-piece queries into network scatter-gather and cross-piece transactions into distributed commits

(2PC, 15.9), and lets one shard fail while the others keep serving (**fault isolation**). Its benefits: **write scaling**, **smaller working sets** (the **working set** is the data a system touches often; here each shard's hot data fits in RAM), fault isolation and more total capacity.

Rebalancing: why `hash % N` hurts and consistent hashing helps

With naive `hash(key) % N`, changing N from 4 to 5 changes the answer for most keys. (A **hash** function turns any key into a fixed-size, random-looking number; the same key always gives the same number.) **Consistent hashing** puts shards and keys on a ring (0 to 2^{32}); each key belongs to the next shard clockwise, so a new shard only takes keys from its neighbour. **Virtual nodes** give each shard many small points on the ring to even out the load (Cassandra, Riak and Dynamo-style systems). `code/consistent_hashing.py` places 100,000 keys, then adds a fifth shard:

The ring is a sorted list of `(hash, shard)` points; `lookup` uses `bisect` (Python's binary search over a sorted list) to find the first point clockwise of `h(key)`:

```
def lookup(self, key):
    i = bisect.bisect(self.hashes, h(key)) % len(self.points)  # next point clockwise
    return self.points[i][1]
```

```
hash % N                    moved  79.7%   max/min shard load 1.01
ring, 1 point per shard     moved   2.8%   max/min shard load 10.44
ring, 200 vnodes per shard  moved  17.1%   max/min shard load 1.28
ideal fraction to move when going 4 -> 5 shards: 1/5 = 20.0%
```

- `hash % N` balances perfectly but moves **80%** of keys — a huge migration.
- One ring point per shard moves little only because the new shard landed on a tiny arc; the biggest shard holds 10× the smallest.
- **200 vnodes** move about 17% (near the ideal 20%) and stay balanced — why real systems use them.

Another approach: a **fixed number of logical partitions** (say 1,024) mapped to servers by a directory; adding a server moves whole partitions and nothing is re-hashed (Citus shards, Vitess keyspace ranges and Kafka partitions work roughly this way). More in DDIA ch.6 — rebalancing.

Sharding Postgres in practice. Core Postgres has no built-in sharding: use application-level routing, the **Citus** extension (`create_distributed_table('orders','customer_id')` co-locates tables sharing the key), partitions whose leaves are `postgres_fdw` foreign tables (15.4), or a Postgres-compatible distributed SQL database (CockroachDB, YugabyteDB — Chapter 16).

Common mistakes

- **Monotonic key + range sharding** (auto-increment id, timestamp): every new row hits the last shard.
- **Low-cardinality keys** (cardinality = the number of distinct values; country, status): too few shards, some huge.
- **Sharding too early**: read replicas, caching (Caching patterns) and a bigger machine are cheaper first steps.
- **Forgetting cross-shard needs**: global unique constraints, cross-shard joins and all-user reports become hard.

In the interview

30-second answer: "Sharding puts horizontal partitions on separate servers so writes and storage scale out. The shard key should spread load evenly, match the main access pattern so most queries hit one shard, and keep related rows together. Queries without the key become scatter-gather and cross-shard transactions need 2PC. For rebalancing I'd use consistent hashing with virtual nodes, or fixed logical partitions mapped to nodes, never `hash mod N`."

- *A celebrity user overloads one shard?* Split that key (`user123#0..9`) to spread writes and merge on read, or cache their reads.
- *Unique ids across shards?* UUIDv7 (a **UUID** is a 128-bit random-looking id that can be generated anywhere without coordination; v7 starts with a timestamp so ids sort by time), Snowflake-style ids (timestamp + machine id + sequence), or per-shard sequences with disjoint ranges.

Self-check

1. *Why is `hash % N` bad for resharding?* Changing N remaps most keys (≈80% going from 4 to 5).
2. *What do virtual nodes fix?* Uneven arcs on the ring, hence uneven load.

15.4 Distributed query processing

What it is

When a query's rows are spread over several nodes, the database must decide **which nodes to ask, what to ask each one, and how to combine the answers** — moving as little data as possible, using nodes in parallel, and still returning a correct answer.

How it works

Step 1 includes **data localisation** (finding which nodes hold each table); step 3 minimises network traffic and time given data location, link speeds and node load; step 5 merges partial results (union, final sort, final sum).

Technique	Idea
Query fragmentation	Independent fragments run simultaneously on different nodes.
Predicate push-down	Apply `WHERE` on the node storing the data, so only matching rows travel.
Aggregate push-down	Each node computes partial `count` / `sum`; only small results travel.
Local (co-located) joins	Both tables sharded on the join key → each node joins its own pieces.
Distributed joins	Otherwise **broadcast** the small table, **repartition (shuffle)** both sides by the join key, or **semi-join**: send only join keys first, then fetch matching rows.
Smart sharding, caching	Keep common queries on one node; reuse frequent intermediate results.

Challenges: data localisation, network latency, a far larger plan space, nodes failing mid-query, and consistency when pieces come from replicas at slightly different moments.

Seeing it in Postgres with `postgres_fdw`

`postgres_fdw` (a **foreign data wrapper**) lets one Postgres query tables on another; the remote tables appear locally as **foreign tables**. `code/06_distributed_query_fdw.sql` points it back at the same private server: schema `shard_b` plays the "remote node", and `remote.orders` is the foreign table:

```
CREATE EXTENSION IF NOT EXISTS postgres_fdw;
CREATE SERVER node_b FOREIGN DATA WRAPPER postgres_fdw
  OPTIONS (host '/tmp', port '5544', dbname 'ch_15');          -- in real life: another host
-- ... user mapping; shard_b.orders created as a copy of orders ...
IMPORT FOREIGN SCHEMA shard_b LIMIT TO (orders) FROM SERVER node_b INTO remote;

EXPLAIN (VERBOSE, COSTS OFF) SELECT order_id, amount FROM remote.orders WHERE amount > 500;
EXPLAIN (VERBOSE, COSTS OFF) SELECT customer_id, count(*), sum(amount) FROM remote.orders GROUP BY 1;
```

```
 Foreign Scan on remote.orders
   Output: order_id, amount
   Remote SQL: SELECT order_id, amount FROM shard_b.orders WHERE ((amount > 500::numeric))

 Foreign Scan
   Output: customer_id, (count(*)), (sum(amount))
   Relations: Aggregate on (remote.orders)
   Remote SQL: SELECT customer_id, count(*), sum(amount) FROM shard_b.orders GROUP BY 1
```

The `Remote SQL` line shows what was pushed down: the filter (only two columns and matching rows travel) and the whole `GROUP BY` (only 12 rows come back). A **cross-node join** — local `customers` in the USA joined to remote `orders` — returns the right answer (Ben 250 orders / 64,750.00, Grace 250 / 65,000.00, Jack 250 / 64,750.00), but its plan reveals a weakness (a **Hash Join** builds an in-memory lookup table from one side, then probes it with each row of the other):

```
Hash Join
  Hash Cond: (o.customer_id = c.customer_id)
  ->  Foreign Scan on remote.orders o
        Remote SQL: SELECT order_id, customer_id FROM shard_b.orders
  ->  Hash
        ->  Seq Scan on public.customers c
              Filter: (c.country = 'USA'::text)
```

The remote query has **no WHERE**: all 3,000 orders cross the network and only 750 survive. The **semi-join** fix finds the USA customer ids locally first and ships only those:

```
EXPLAIN (VERBOSE, COSTS OFF)
SELECT o.order_id, o.customer_id FROM remote.orders o
WHERE o.customer_id = ANY (ARRAY(SELECT customer_id FROM customers WHERE country = 'USA'));
```

```
 Foreign Scan on remote.orders o
   Output: o.order_id, o.customer_id
   Remote SQL: SELECT order_id, customer_id FROM shard_b.orders WHERE ((customer_id = ANY
($1::integer[])))
   InitPlan 1 (returns $0)
     ->  Seq Scan on public.customers
           Filter: (customers.country = 'USA'::text)
```

The `InitPlan` (a sub-query Postgres runs once before the main query) runs locally; its three ids travel as `$1`, and only the 750 matching orders come back. Distributed **optimisers** (the components that choose the cheapest execution plan — Citus, CockroachDB, Spark, Trino) make such choices automatically from table sizes and data location.

Common mistakes

- Wrapping remote columns in functions the remote side can't evaluate, which blocks push-down — check `Remote SQL` in `EXPLAIN VERBOSE`.
- Joining two large tables sharded on *different* keys in the hot path — co-locate or denormalise.

- `ORDER BY … LIMIT` across shards without per-shard limits — each shard should return its own top N first.

In the interview

> **30-second answer:** "The **coordinator** (the node that receives the query and orchestrates the others) localises the data, splits the query into per-node fragments, pushes filters and partial aggregates to the data, runs fragments in parallel and merges results. Network transfer dominates, so the optimiser prefers co-located joins, broadcasts small tables and uses semi-joins to ship keys instead of rows."

- *Why can't `AVG` be pushed down as `AVG`?* An average of averages is wrong when groups differ in size; push down `SUM` and `COUNT`, divide at the end.

Self-check

1. *Where do you see what `postgres_fdw` pushed down?* The `Remote SQL:` line of `EXPLAIN VERBOSE`.
2. *What does a semi-join ship first?* Only the join keys from the smaller side.

15.5 Replication

What it is

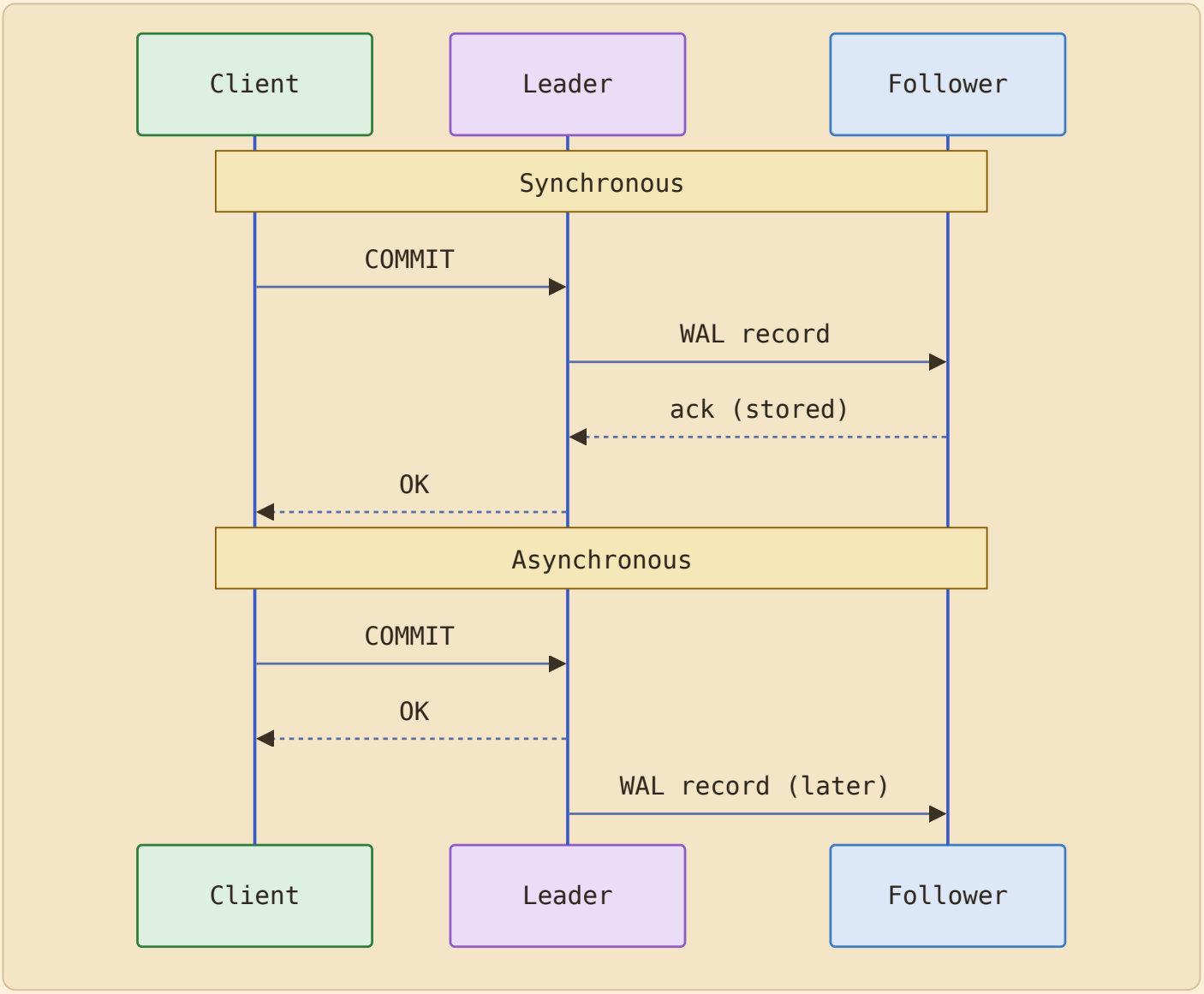
Replication keeps **copies of the same data on several nodes** (**replicas**) — like keeping photos on your laptop, an external drive and the cloud. Benefits: **high availability** (a replica takes over when the primary dies), **read scaling** (reports read from replicas), **lower latency** (a Frankfurt replica serves Europe), **disaster recovery** (**DR** — getting back to work after losing a whole site; here, a copy in another region). Costs: keeping replicas in sync, extra storage and network traffic, and **conflicts** when several copies accept writes.

Kinds of replication (what gets copied, and when)

Type	How it works	Trade-off	Typical use
Full	Every replica holds the whole database	Any replica answers anything / storage × replicas	DR copies
Transactional	Each committed transaction streams to replicas in commit order, near real time	Close and ordered / continuous overhead	OLTP (day-to-day transactional workload) read replicas, Postgres logical replication
Snapshot	Periodically copy a full snapshot	Very simple / only as fresh as the last copy	Nightly reporting copies
Merge	Copies change independently (even offline), then merge by rules	Works disconnected / conflicts	Field and mobile apps
Key-based incremental	Copy rows whose `updated_at`/version is newer than the last sync	Moves little data / **misses hard deletes** (rows physically removed) unless tombstoned (marked deleted instead)	ETL (extract-transform-load pipelines) into warehouses, IoT (internet-connected device) sync

A retailer might mix them: orders and stock transactionally, the catalogue as nightly snapshots, a full replica in another region.

Synchronous vs asynchronous



Synchronous replication returns a commit only when the sync replica(s) also have it: no acknowledged commit is lost, but every write pays a network **round trip** (a message there plus the reply back) and commits **block** if that replica is down (use it for money and orders). **Asynchronous** returns once the leader has it: fast, but the newest commits may be lost if the leader dies, and replicas lag (**replication lag**) — fine for read replicas, cross-region copies and analytics. Most systems are **semi-synchronous**: wait for *one* (or a quorum of) replicas, the rest async. **In PostgreSQL**, physical **streaming replication** ships the **WAL** (write-ahead log — the database's append-only record of every change, 15.9) to **standbys** (replica servers ready to take over), async by default; `synchronous_standby_names` (e.g. `'ANY 1 (s1, s2)'`) makes commits wait, and `synchronous_commit` sets how far: `remote_write` (received), `on` (flushed), `remote_apply` (applied, so standby reads see it). **Logical replication** (`CREATE PUBLICATION` / `SUBSCRIPTION`, PG 10+) streams row changes per table.

Replication topologies

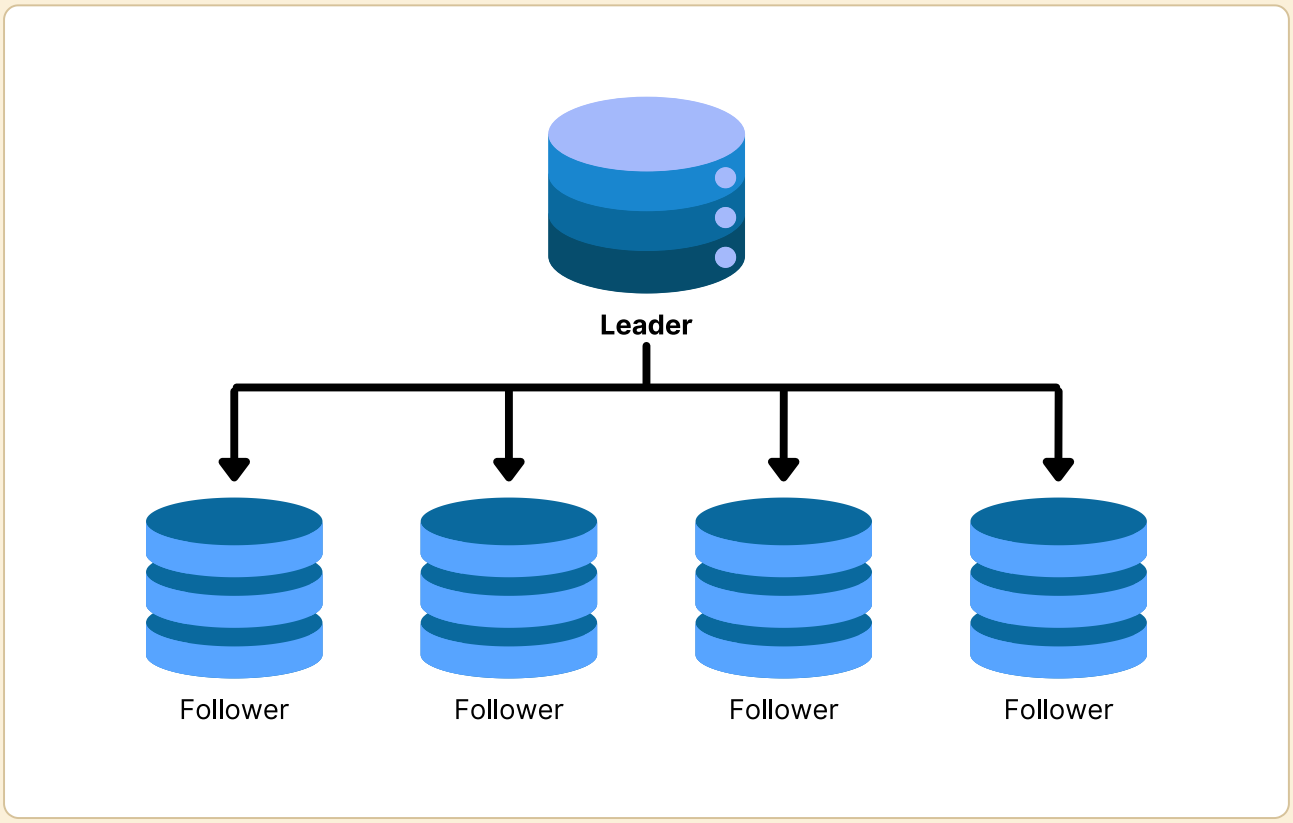


Figure — Single-leader: one leader takes writes and streams them to followers, which serve reads (course diagram).

1. Single-leader (primary/replica; formerly master/slave). All writes go to the leader, which records them in its **replication log / WAL** and ships the log to **followers** (sync or async). Reads may use any node. If the leader fails, **failover** promotes a follower (15.10). Simple, no write conflicts, strong consistency possible; but write throughput is one machine's and async followers can be stale. Examples: PostgreSQL, MySQL, SQL Server Always On, MongoDB replica sets.

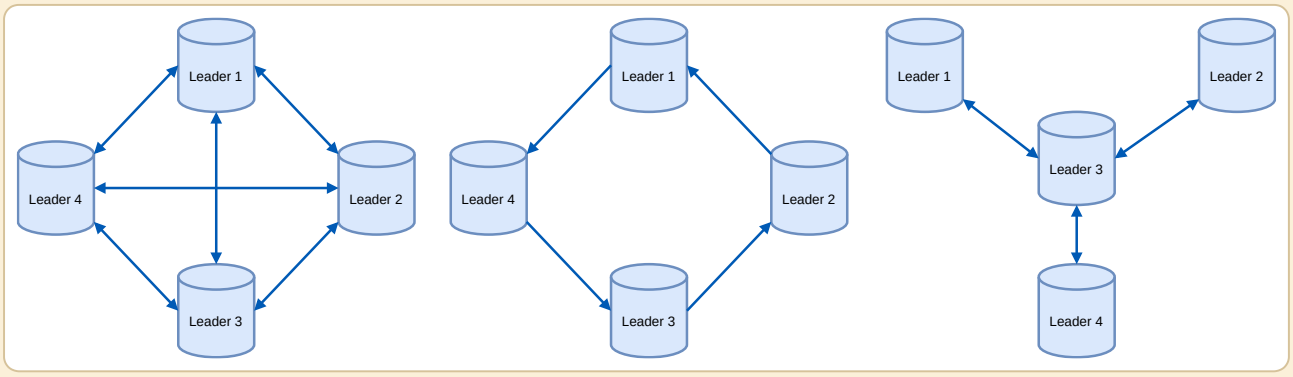


Figure — Multi-leader: several leaders accept writes and replicate to each other (course diagram).

2. Multi-leader (active-active). Several nodes (often one per region) accept writes and replicate to each other. The same row changed at two leaders at once is a **conflict**, resolved by **last-write-wins** (by timestamp — silently drops one write), custom merge code, or **CRDTs** (conflict-free replicated data types, which merge automatically). If one leader fails, clients switch to another. Used for multi-region writes, offline apps (each device is a leader) and collaborative editing. Examples: MySQL Group Replication (multi-primary), Postgres with BDR/pgEdge, CouchDB.

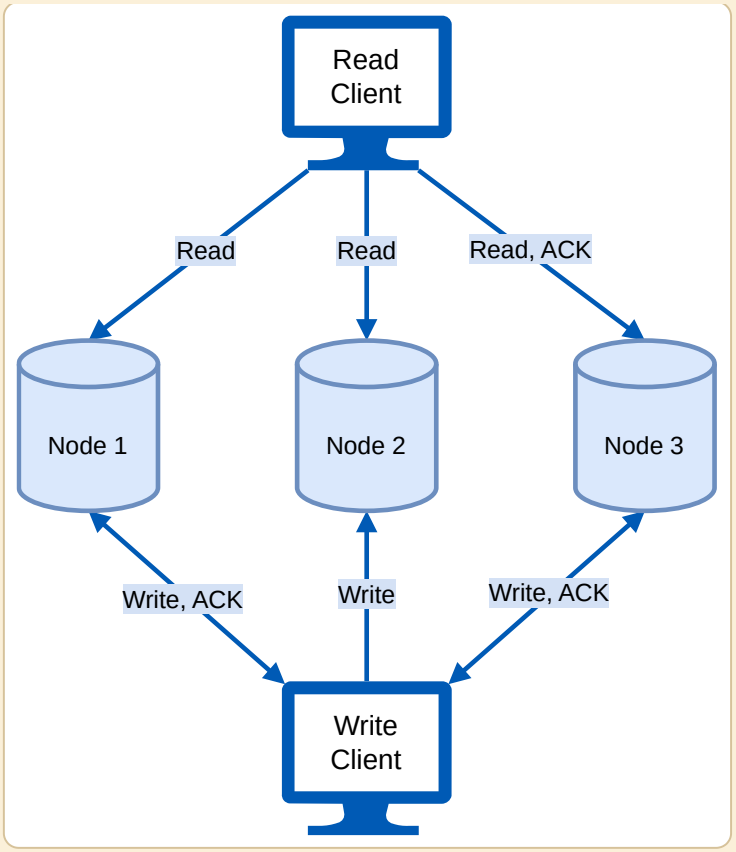


Figure — Leaderless: the client writes to and reads from several nodes directly (course diagram).

3. Leaderless (Dynamo-style). The client (or a coordinator for it) sends each write to all **N** replicas and succeeds once **W** acknowledge; reads ask **R** replicas and keep the newest version. If **R + W > N**, every read set overlaps every write set (the **quorum condition**). Stale replicas are fixed by **read repair** (the reader writes the newest value back), background **anti-entropy** (Merkle-tree comparison — a **Merkle tree** is a tree of hashes where each parent hashes its children, so two replicas can find differing data by comparing a few hashes) and **hinted handoff** (writes for a down node are held and delivered later). Concurrent writes are detected with **version vectors / vector clocks** (a per-node list of counters attached to each value; comparing two lists tells whether one write happened after the other or they were concurrent). Examples: Cassandra, ScyllaDB, Riak, Amazon's original Dynamo (the DynamoDB *service* actually uses a leader per partition). Leaderless systems use **quorums, not consensus** — no Paxos/Raft per write — so they stay available but guarantee less.

`code/quorum.py` simulates N = 3: in each trial a new version reaches only W random replicas, then a read asks R random replicas, keeps the highest version, and counts as stale if that is still the old value:

```
N = 3 replicas, 10,000 trials per setting
 W  R  R+W>N  stale reads
 1  1  False    66.4%
 1  2  False    34.0%
 1  3  True      0.0%
 2  1  False    33.8%
 2  2  True      0.0%
 ...  (every remaining row has R+W>N and 0.0%)

N=3, W=2, R=2: writes and reads still succeed with 1 replica down (2 of 3 answer); with 2 down they fail
-> the system chooses consistency.
```

Every setting with R + W > N had **zero** stale reads. N=3, W=2, R=2 survives one failed replica — Cassandra's `QUORUM` level; `ONE` is fast but possibly stale. Quorums still aren't full **linearizability** (the strongest guarantee, defined in 15.6): sloppy quorums (writes accepted by stand-in nodes when the usual ones are down), concurrent writes and clock-based last-write-wins can produce anomalies (DDIA ch.5).

Choosing a topology: correctness first → single-leader with a sync replica; writes in several regions → multi-leader with a conflict plan; must stay writable through failures and eventual is acceptable → leaderless with quorums.

Replication lag in practice. A user updates their profile, the next page is served by a lagging replica, and the old version appears. Fixes: read your own recent writes from the leader, pin a session to one replica, or wait until the replica's `pg_last_wal_replay_lsn()` reaches the write's LSN — the session guarantees of 15.6.

In the interview

> **30-second answer:** "Replication keeps copies for availability, read scaling and DR. Sync loses no acknowledged commits but slows writes; async is fast but can lose recent commits and serve stale reads, so most systems are semi-sync. Topologies: single-leader (simple, no conflicts), multi-leader (multi-region writes, conflicts), leaderless (quorums with R + W > N, read repair)."

- *A user doesn't see their own comment?* A lagging async replica served the read; use read-your-writes routing.
- *Last-write-wins downside?* Concurrent updates are silently lost, and **clock skew** (machines' clocks disagreeing slightly) can discard the newer one.

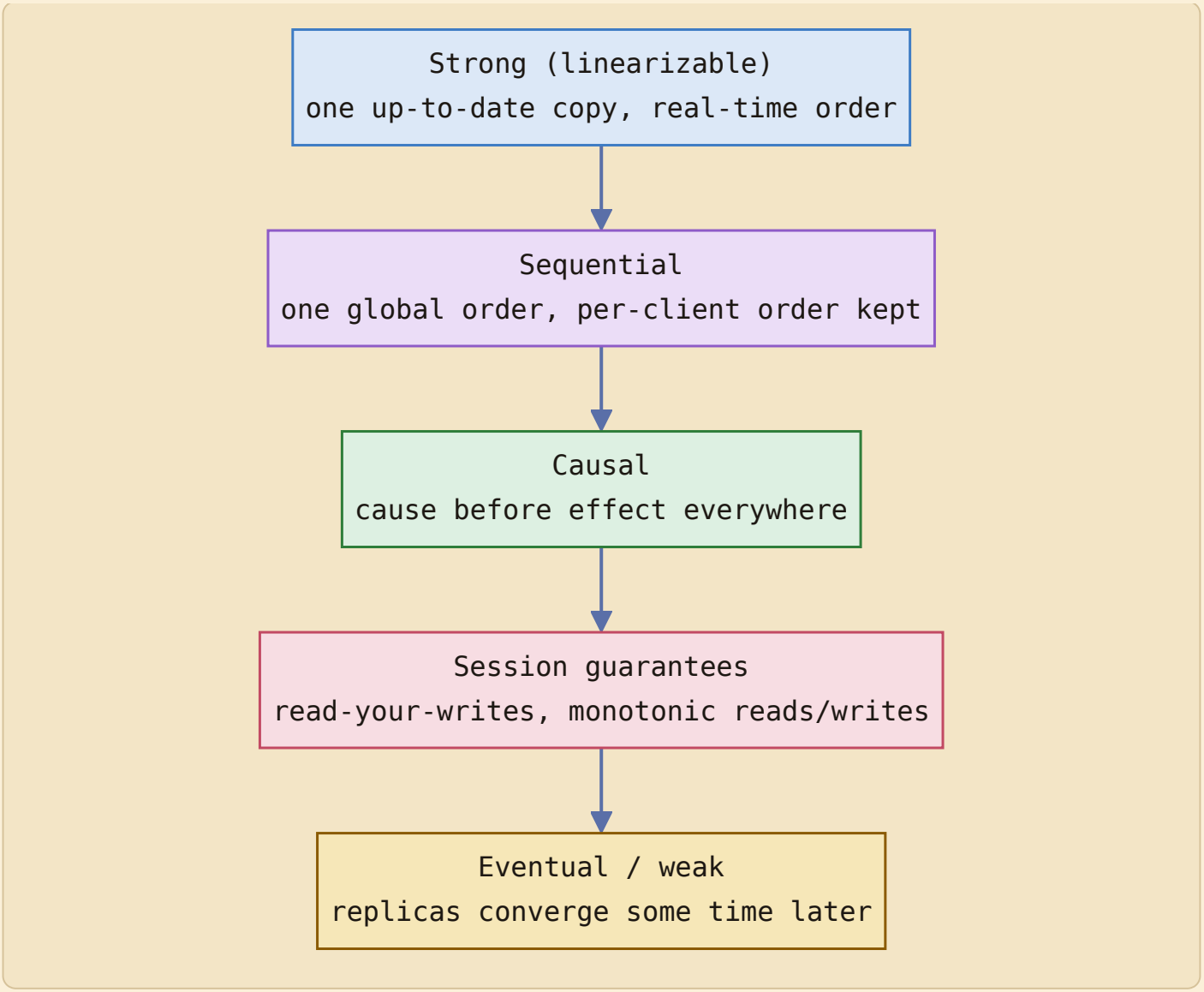
Self-check

1. *Which replication type can miss hard deletes?* Key-based incremental.
2. *N=5: which W and R guarantee overlap?* Any pair with R + W > 5, e.g. W=3, R=3.

15.6 Consistency models

What it is

With several copies, "what will a read return?" has more than one answer. A **consistency model** is the contract: which writes a read is guaranteed to see, and in what order clients observe operations. Stronger models are easier to program against but need more coordination (slower, less available); weaker ones are faster but push work onto the application.


Strong (linearizable)
one up-to-date copy, real-time order
Sequential
one global order, per-client order kept
Causal
cause before effect everywhere
Session guarantees
read-your-writes, monotonic reads/writes
Eventual / weak
replicas converge some time later

The models

Model	Guarantee	How it's built	Pros / cons	Use
Strong (linearizable)	Every read sees the **latest completed write**; the system acts like one copy with operations in **real-time** order (if A finishes before B starts, everyone sees A first)	Synchronous replication or consensus (15.8), atomic operations, one global order	Simplest to reason about, no stale reads / coordination latency, scales less, may be **unavailable** during partitions	Balances, stock at checkout, "is this username taken?", locks. etcd, ZooKeeper writes, Spanner, CockroachDB, single-node Postgres
Sequential	All clients see **the same order**, and each client's operations in its own program order — but not necessarily real-time order	A global sequencer	One agreed history / still needs coordination, no recency promise	CPU memory models, distributed shared memory; rare by name in databases
Causal	If A could have influenced B (same client did A then B, or B read A), **everyone sees A before B**; unrelated (**concurrent**) operations may appear in different orders	Track dependencies with **vector clocks**; delay an update until its dependencies are visible	Little coordination, works across regions, matches intuition / complex tracking, brief divergence	Replies after questions, chat, collaborative editing; MongoDB causally consistent sessions
Weak / eventual	Few promises; **eventual consistency**: if writes stop, replicas converge	Async propagation, anti-entropy, conflict resolution	Lowest latency, highest availability / stale and out-of-order reads, confusing UX (a like count that jumps)	Caches/CDNs (content delivery networks — servers worldwide holding copies of content), counters, analytics, DNS (the internet's name-to-address lookup)

Sequential example: Alice posts P1 then P2, Bob posts Q1; everyone sees one interleaving (say P1, Q1, P2), never P2 before P1 — but a reader just after Q1 finished may not see it yet.

Session (client-centric) guarantees

Weaker than the global models, but they fix the anomalies users notice, **within one session**:

Guarantee	Promise	Anomaly prevented	Example
Read-your-writes	You see your own earlier writes	"My new profile photo reverted"	Profile edits, cart
Monotonic reads	Never an older version after a newer one	A comment *disappears* on refresh	Feeds, notifications, config values
Monotonic writes	Your writes apply in the order issued	"price = 12" applied before the earlier "price = 10"	Version-control commits, debit then credit
Writes-follow-reads	A write after reading X is ordered after X	Reply stored before the post it answers	Comment threads

Session consistency bundles them (Azure Cosmos DB's default level): track the last version/LSN each session saw and route its reads to replicas at least that fresh — coherent for the user and cheap, though sessions may see different states and routing loses some **load-balancing** (spreading requests across servers) freedom.

How to choose: strong for money and stock; session guarantees where users mostly need their own actions to stick; causal for cause-and-effect content; eventual for high-volume, tolerant data. Many systems let you **choose per operation** — Cassandra consistency levels, DynamoDB `ConsistentRead`, MongoDB read/write concerns, Cosmos DB's five levels.

> **Don't confuse** this "C" with the **C in ACID** (Chapter 13), which means "transactions keep constraints and invariants true". Consistency *models* are about **replicas and ordering**.

In the interview

> **30-second answer:** "A consistency model says what reads can return under replication. Linearizable: one up-to-date copy in real-time order. Sequential: one global order without the real-time promise. Causal: cause before effect. Eventual: convergence only. Session guarantees — read-your-writes, monotonic reads — fix what users see. Stronger costs latency and availability, so I pick per feature."

- *Model for a comment thread?* Causal — replies must never precede their parent.
- *Linearizable vs serializable?* Linearizability is about single objects and real-time recency; serializability is about multi-object transactions behaving as if run one at a time. Both together = "strict serializability" (Spanner).

Self-check

1. *A refresh shows an older comment list. Which guarantee broke?* Monotonic reads.
2. *Sequential vs strong?* Strong also respects real-time order.

15.7 The CAP theorem (and PACELC)

What it is

The **CAP theorem** (Eric Brewer's conjecture, 2000; proved by Gilbert and Lynch, 2002) concerns three properties of a replicated store:

- **C — Consistency** (here *linearizability*; see also System Design Fundamentals for CAP in context): every read sees the most recent successful write. If Priya's balance drops from ₹500 to ₹300, no node may still report ₹500.
- **A — Availability**: every request to a **non-failed** node gets a non-error response, without waiting indefinitely.
- **P — Partition tolerance**: the system keeps working when the network drops or delays messages between groups of nodes.

The theorem: *during a network partition, a system must choose between consistency and availability.*

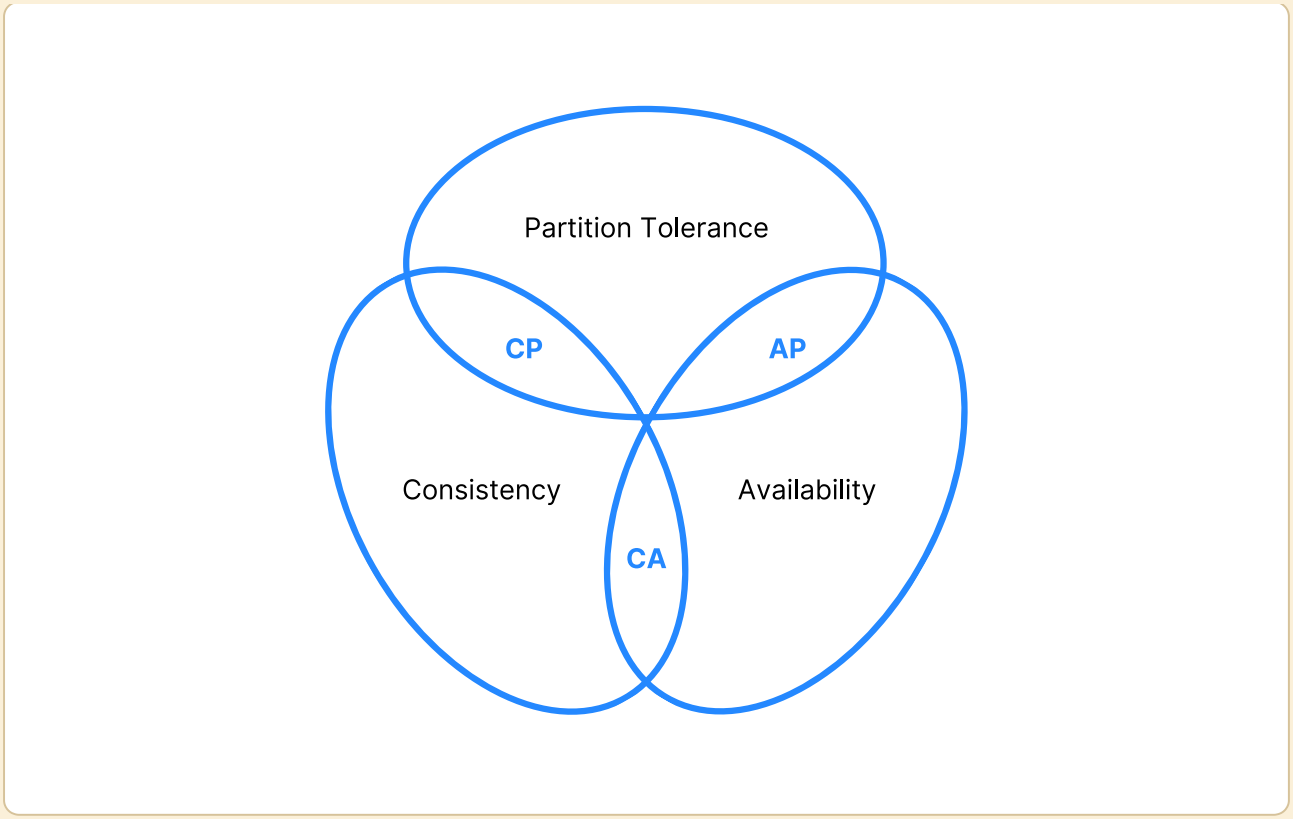


Figure — The three CAP properties and the CP, AP and CA combinations (course diagram).

Why "pick two of three" is misleading

In anything spanning more than one machine, **partitions will happen** (switches fail, cables get cut), so P can't be given up. The real choice is what to do *during* a partition:

Choice	During a partition	Pros	Cons	Examples
CP	Refuse or delay some requests (usually on the minority side) rather than return stale data	Correct, no divergence	Errors/timeouts for some users, coordination latency	etcd, ZooKeeper (small, strongly consistent coordination stores), HBase, Spanner, CockroachDB
AP	Keep answering on both sides, reconcile later	Always responsive	Temporarily inconsistent, needs conflict resolution	Cassandra/ScyllaDB (at `ONE`), Riak, CouchDB, DNS
CA	Only if partitions never happen — effectively one node	Simple and strong	No tolerance of network failure	A single Postgres server (not distributed, so CAP doesn't really apply)

Example: a subscriber counter is kept in two data centres and the link breaks. **AP** lets both sides keep accepting clicks and merges the counts later; **CP** makes the minority side refuse clicks until the link returns.

Nuances: the choice is often **per operation** (CP for "place order and decrement stock", AP for "show reviews"); with **no** partition a system can be both C and A; and CAP's A is strict — many "CP" systems are highly available in practice.

PACELC

Daniel Abadi's **PACELC** (2010): **if Partition, choose A or C; Else, choose Latency or Consistency** — even on a good day, synchronous coordination costs latency.

System	Partition	Else
Cassandra, Riak, DynamoDB (default reads)	PA	EL (fast, possibly stale)
Spanner, CockroachDB, etcd	PC	EC (pay latency for consistency)

In the interview

30-second answer: "CAP says that during a partition a replicated store gives up either linearizable consistency or availability. Partitions are unavoidable, so the real choice is CP or AP — banking and inventory lean CP, feeds and carts lean AP with reconciliation. PACELC adds that without partitions you still trade latency for consistency."

- *Is a single Postgres server CA?* It isn't distributed, so CAP isn't about it; with async replicas plus failover it's neither fully C (can lose recent commits) nor fully A.

Self-check

1. *What does C mean in CAP, and what does PACELC add?* Linearizability; the latency-vs-consistency trade-off in normal operation.

15.8 Consensus and leader election

What consensus is

Consensus gets several nodes to **agree on one value** (or one ordered log) even when some crash or messages are delayed. Everything that needs "exactly one" rests on it — one leader, one commit decision, one log order, one lock owner — so it underpins consistency, fault tolerance, event ordering and coordination. A correct protocol guarantees **agreement** (all non-faulty nodes decide the same value), **validity** (the decision was proposed by someone; if all propose v, it's v), **termination** (non-faulty nodes eventually decide) and **fault tolerance** (it survives a minority of **crash faults**, or fewer than a third of **Byzantine faults** — nodes that lie). It's hard because nodes crash, networks partition and reorder messages, some nodes may be malicious, and every round trip adds latency. The **FLP result** (Fischer, Lynch, Paterson, 1985) proves no **deterministic** (same inputs, always same steps) protocol can *guarantee* termination in a fully asynchronous network if even one node may crash — so real protocols are always *safe* and use timeouts to *usually* make progress.

The algorithms in brief

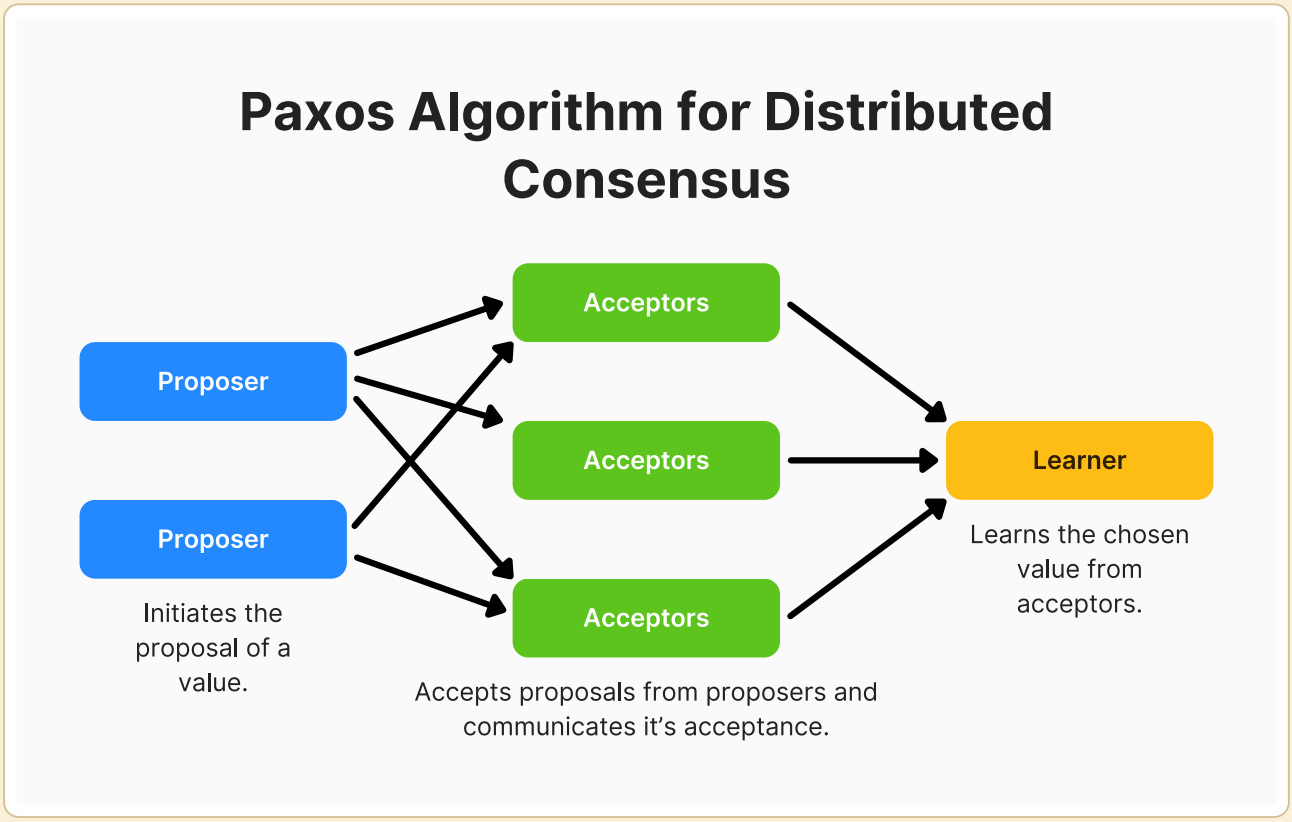


Figure — Paxos roles: proposers send numbered proposals to a set of acceptors, which accept and communicate the chosen value (course diagram).

Paxos (Leslie Lamport; written 1989, published 1998) has **proposers**, **acceptors** and **learners**. (1) **Prepare/Promise:** a proposer sends *prepare(n)* with a unique, higher number to a majority; each acceptor promises to ignore anything below *n* and reports the highest-numbered proposal it already accepted. (2) **Accept:** with a majority of promises the proposer sends *accept(n, v)*, where *v* must be the highest-numbered value reported (its own only if none). (3) Once a **majority accepts**, *v* is chosen and learners are told. It tolerates ⌊(N−1)/2⌋ crashes, needs no synchronised clocks for safety and is formally proven, but it's hard to implement, costs two round trips per decision (**Multi-Paxos** keeps a stable leader to skip phase 1), and duelling proposers can stall it. Used in Google's Chubby and Spanner.

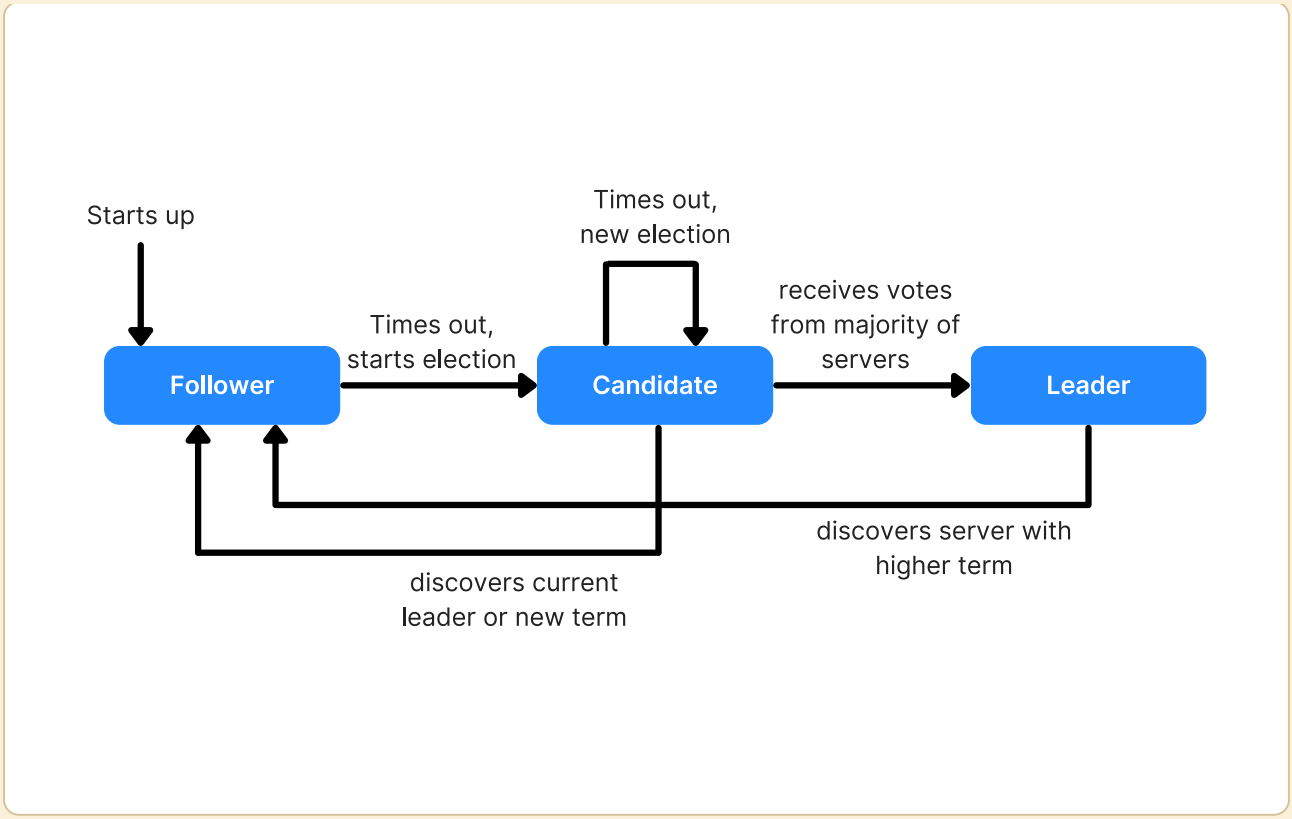


Figure — Raft node states: follower → candidate (on timeout) → leader (on majority vote), and back to follower on discovering a newer term (course diagram).

Raft (Ongaro and Ousterhout, 2014) gives the same guarantees but was designed to be **understandable**. (1) **Election:** time is split into numbered **terms**; a follower that hears no **heartbeat** (a small periodic "I'm alive" message) within its **randomised election timeout** (150–300 ms in the paper) becomes a **candidate**, increments the term and requests votes; each node votes once per term, only for a candidate whose log is at least as up to date as its own; a majority makes a **leader**, and the random timeouts make split votes rare. (2) **Log replication:** the leader sends **AppendEntries** (also the heartbeat); an entry is **committed** once a majority stores it. (3) **Safety:** only a node holding every committed entry can win. Its weaknesses: the leader is a throughput ceiling, unstable networks cause repeated elections, and each commit waits for a majority. Used in etcd (Kubernetes' store), Consul, CockroachDB, TiKV, YugabyteDB and Kafka's KRaft; MongoDB's replica-set protocol is Raft-inspired.

PBFT (Practical Byzantine Fault Tolerance; Castro and Liskov, 1999) tolerates **f lying nodes out of N ≥ 3f + 1**. In each **view** a **primary** runs *pre-prepare* (assign a sequence number) → *prepare* (replicas broadcast; prepared after 2f matching messages from others) → *commit* (after 2f + 1 matching commits) → *reply* (the client waits for f + 1 identical replies, so one is honest). It costs **O(N²) messages**, so clusters stay at tens of nodes; used in **permissioned blockchains** (shared ledgers where only approved participants may join; Hyperledger family) and critical control systems.

Proof of Work (Bitcoin) serves **open** networks, where fake identities make vote-counting useless: miners race to find a **nonce** (a throwaway number tried over and over) that puts the block hash under a target, others verify cheaply, and the chain with the most accumulated work wins — no central authority, but heavy energy use, slow finality (a block every ~10 minutes) and exposure to a **51% attack** (whoever controls most of the computing power can rewrite recent history). **Proof of Stake** (Ethereum since the 2022 "Merge") weights votes by locked-up currency instead.

Algorithm	Fault model, tolerance	Cost	Fits
Paxos / Multi-Paxos	Crash, ⌊(N−1)/2⌋	Majority round trips; hard to implement	Databases, lock services
Raft	Crash, ⌊(N−1)/2⌋	Majority round trips, leader-bound	Config stores, distributed SQL
PBFT	Byzantine, ⌊(N−1)/3⌋	$O(N^2)$ messages	Permissioned ledgers
PoW / PoS	Byzantine, open membership (honest majority of hash power / stake)	Slow, energy-hungry / better throughput	Public blockchains

Known members that may crash → Raft or Paxos (what you'll meet inside databases); known members that may lie → PBFT family; unknown members → PoW/PoS. For the full reasoning see DDIA ch.9.

Leader election

A **leader** is the single decider for writes, task assignment or a lock; election picks one and re-picks when it fails. Hard parts: a slow leader looks dead, partitions can create **two leaders** (**split brain**), and clocks differ between nodes.

Strategy	How it works	Strengths	Weaknesses
Bully	The detecting node sends ELECTION to all **higher** ids; if none answers it wins, else the higher node takes over; the winner broadcasts COORDINATOR	Simple, deterministic (highest live id)	Up to **$O(n^2)$** messages; the top node always leads, even if weak
Ring	ELECTION travels round a logical ring, each node substituting its id if higher; whoever gets its **own** id back wins and circulates COORDINATOR	About **2n** messages, equal participation	Slow on big rings; a dead node breaks the ring unless skipped
Randomised	Candidates wait a **random delay**, the first to announce wins, collisions retry	Simple, collisions rare	Non-deterministic (Raft adds majority votes)
ZooKeeper	Each candidate creates an **ephemeral sequential** znode (gone when its session dies); the **lowest** leads; each watches only the znode just **before** its own	Reliable, simple API; ZooKeeper runs the consensus protocol **ZAB**	Extra system to run (same pattern: etcd, Consul, Kubernetes **Lease**)

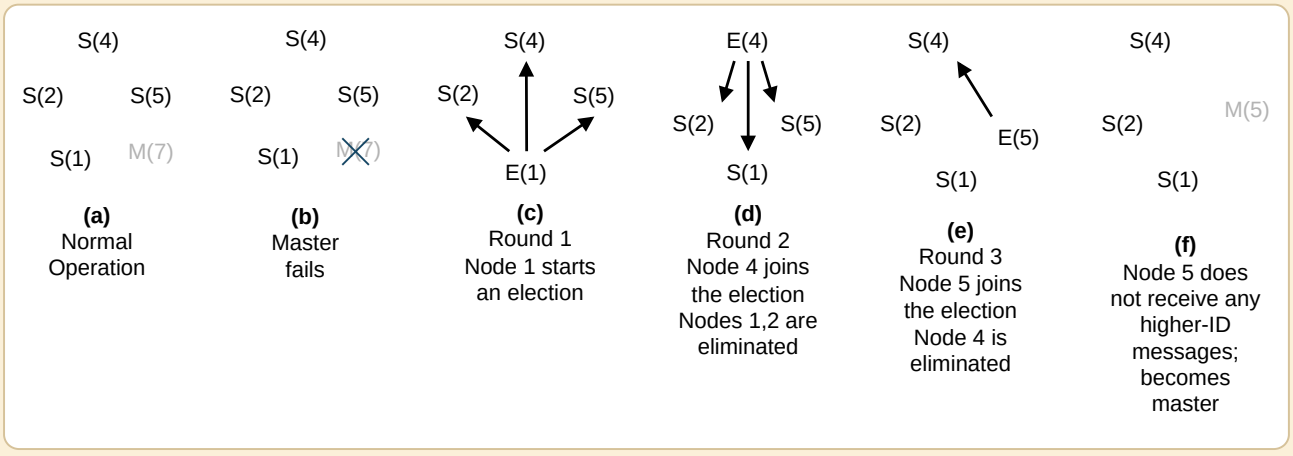


Figure — Bully election: after the leader fails, election messages flow to higher ids until the highest live node announces itself (course diagram).

`code/leader_election.py` counts messages when the highest-id leader has crashed and the *lowest* id notices first (Bully's worst case):

nodes	bully leader	bully msgs	ring leader	ring msgs
5	4	19	4	8
10	9	89	9	18
50	49	2449	49	98

Same leader either way; Bully grows roughly with n^2, Ring with $2n$.

Fencing. A correctly elected leader can be *paused* (long **GC** — garbage-collection pause, when a runtime stops the program to free unused memory — or a **VM** (virtual machine) freeze), replaced, then wake up still believing it leads. Give each new leader a higher term/epoch number (a **fencing token**) and have storage reject writes carrying an older one — Raft terms do exactly this.

In the interview

30-second answer: "Consensus lets nodes agree on a value or log despite failures. Raft elects a leader with randomised timeouts and majority votes and commits an entry once a majority stores it, tolerating $\lfloor (N-1)/2 \rfloor$ crashes; Paxos is the older equivalent. PBFT tolerates liars with $3f+1$ nodes at $O(N^2)$ messages. For leader election I'd use etcd or ZooKeeper and fence stale leaders with epoch numbers."

- *Why 3 or 5 nodes, not 4?* A majority of 4 is 3, so 4 nodes tolerate one failure — no better than 3.
- *What is split brain?* Two nodes both acting as leader after a partition; prevent it with majority quorums and fencing.

Self-check

1. *A 5-node Raft cluster tolerates how many failures?* 2.
2. *PBFT with f = 2 needs at least?* 3f + 1 = 7 nodes.
3. *Which znode does a ZooKeeper candidate watch?* The one just before its own.
4. *What can't be guaranteed in a fully asynchronous system (FLP)?* Termination.

TL;DR

Consensus = majority agreement that survives crashes (Raft/Paxos) or lies (PBFT, 3f+1). Leader election needs consensus underneath and fencing on top — use etcd, ZooKeeper or Consul rather than inventing your own.

15.9 Fault tolerance and recovery

What it is

Fault tolerance is the ability to **keep working correctly while parts are failing**. A **fault** is one component going wrong (dead disk, crashed process, dropped packet); a **failure** is the whole system stopping or answering wrongly — fault tolerance stops the first becoming the second, like a twin-engine plane landing on one engine. Its objectives are **continuity of service**, **data integrity**, **graceful degradation** (recommendations vanish but checkout works) and **recovery**, in support of **availability**, **reliability**, **safety** (failures stay contained — aviation, cars) and **maintainability** (easy repair and upgrade).

Handling a fault has five phases: **detection** (heartbeats, health checks, monitoring) → **diagnosis** (logs, error codes, tools) → **evidence generation** (detailed logs, snapshots, audit trails) → **assessment** (impact analysis, dependency maps) → **recovery** (fail over, restart, reconfigure, restore). Techniques exist at every level: **hardware** (RAID — several disks combined so one can fail without data loss, see 15.11 — hot-swappable drives, dual power supplies), **software** (exception handling, retries with **backoff** (waiting longer between each retry), timeouts, graceful degradation, **circuit breakers** (stop calling a failing service for a while so it can recover) — see Reliability patterns) and **system** (checkpointing,

mirroring, load balancing, replication and failover). The rest of this section covers the database machinery: WAL, checkpoints and distributed commit.

Write-ahead logging (WAL)

The rule: *a change's log record must be safely on disk before the changed data page is written.* The log is an append-only sequential file (fast to write); data pages are written back lazily. After a crash the data files may be stale or half-written, but **the log has the truth**. A transaction changes **pages** (fixed-size blocks, 8 kB in Postgres, in which tables are stored) in memory (the **buffer pool** — the database's in-RAM cache of pages) and creates a log record per change; at **COMMIT** the log up to the commit record is **flushed (fsync'd** — `fsync` is the system call that forces data out of OS caches onto the physical disk), and only then is the commit acknowledged — now **durable** (the D of ACID, Chapter 13). *Example:* Ravi moves ₹5,000 from A to B; "A −5,000", "B +5,000" and a commit record are flushed and he sees "Transfer complete". If the server crashes a millisecond later, before the data pages were written, recovery replays those records.

Redo, undo and LSNs

- **Redo** makes **committed** work survive: recovery re-applies logged changes (**after-images** — the new values; **before-images** are the old values) that may not have reached the data files.
- **Undo** makes **uncommitted** work vanish: before-images restore any in-flight change that reached disk.
- Most databases do both. The classic design, **ARIES** (analysis → redo, "repeat history" → undo the losers), is used by DB2 and SQL Server; InnoDB uses the same redo + undo idea.
- **PostgreSQL's WAL is redo-only.** Thanks to **MVCC** (multi-version concurrency control, Chapter 13 — readers see an older version of a row while a writer creates a new one) an update writes a *new* row version; an aborted transaction is just marked not-committed in `pg_xact`, so its versions are invisible and **VACUUM** (Postgres's background clean-up of dead row versions) removes them later. InnoDB keeps separate **undo logs**, which also serve its MVCC snapshots.

Every WAL record has a **log sequence number (LSN)** — its byte position in the log, always increasing. LSNs **order** redo/undo, tell recovery **what's already applied** (each data page stores the LSN of its last change, so older records are skipped), **mark checkpoints** (replay starts from the checkpoint's redo LSN) and **measure replication lag** (primary LSN vs replica replay LSN).

`code/07_wal_lsn_checkpoint.sql` first prints the server's defaults (`wal_level` = replica, `synchronous_commit` = on — commit waits for the WAL flush, `checkpoint_timeout` = 5min, `max_wal_size` = 1GB, `full_page_writes` = on), then measures WAL growth:

```
INSERT INTO lsn_marks VALUES ('before', pg_current_wal_insert_lsn());
INSERT INTO ledger SELECT i, 'acct-' || (i % 5), (i % 7) - 3 FROM generate_series(1, 5000) i;
INSERT INTO lsn_marks VALUES ('after 5,000 inserts', pg_current_wal_insert_lsn());

SELECT a.label AS from_mark, b.label AS to_mark,
       pg_wal_lsn_diff(b.lsn, a.lsn) > 0         AS wal_grew,
       pg_wal_lsn_diff(b.lsn, a.lsn) > 5000 * 50 AS more_than_50_bytes_per_row
FROM lsn_marks a, lsn_marks b WHERE a.label = 'before' AND b.label = 'after 5,000 inserts';

CHECKPOINT;
SELECT redo_lsn >= (SELECT lsn FROM lsn_marks WHERE label = 'after 5,000 inserts')
       AS redo_point_moved_past_our_inserts
FROM pg_control_checkpoint();
```

```
 from_mark |       to_mark       | wal_grew | more_than_50_bytes_per_row
-----------+---------------------+----------+---------------------------
 before    | after 5,000 inserts | t        | t

 redo_point_moved_past_our_inserts
-----------------------------------
 t
```

`pg_current_wal_insert_lsn()` returns the current end of WAL (like `0/1A2B3C4`; booleans are printed because exact LSNs differ per run). Each row cost over 50 bytes of WAL — row data, index entries and headers. With `full_page_writes = on`, the first change to a page after a checkpoint logs the **whole page**, guarding against **torn pages** (pages only partly written when power was lost, mixing old and new bytes). After `CHECKPOINT` the **redo LSN** is past our inserts, so a crash now would start replay there.

Checkpointing

A **checkpoint** flushes all **dirty pages** (pages changed in memory but not yet written to disk) and logs *"everything before LSN X is on disk"*, so recovery replays only from there and older WAL (not needed by replicas or archiving) can be recycled.

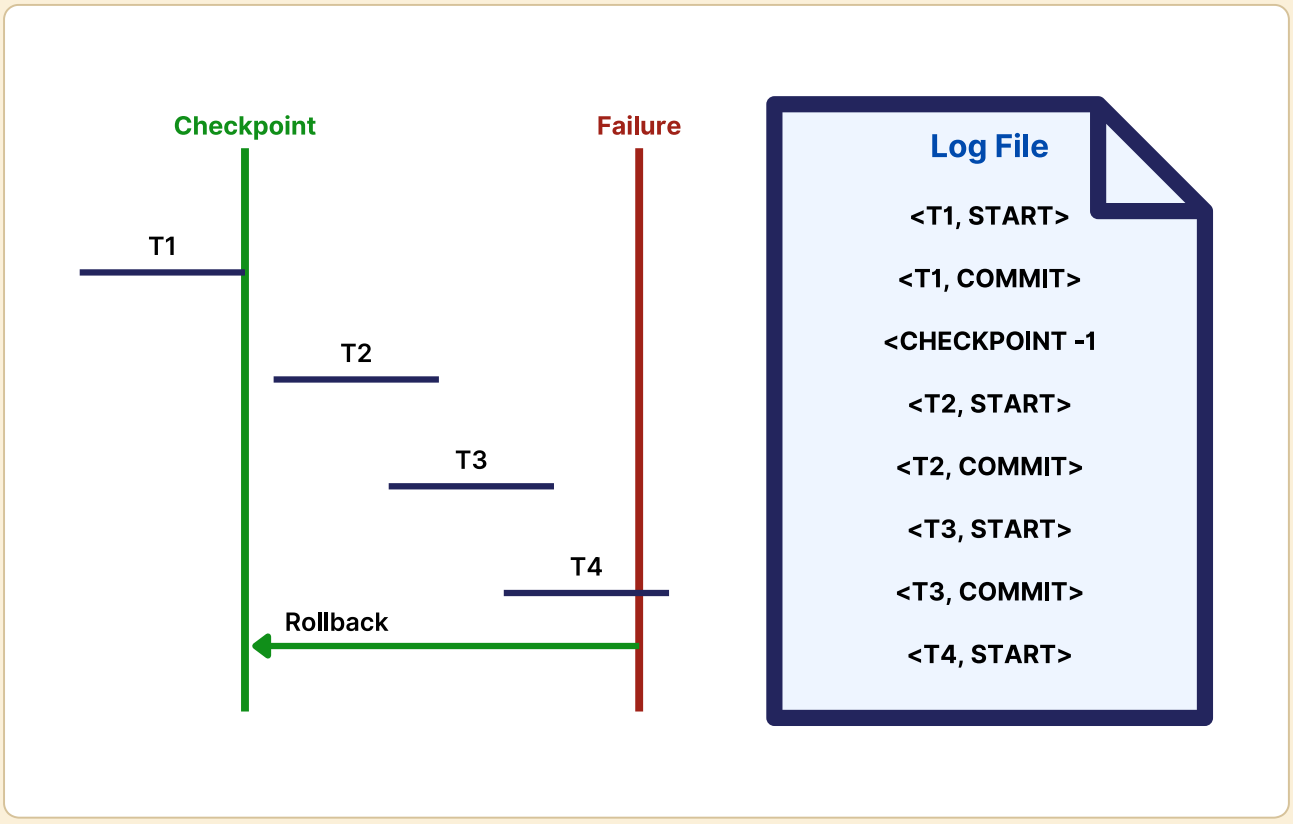


Figure — T1 finishes before the checkpoint, T2 and T3 commit after it, T4 is still running at the failure and is rolled back (course diagram).

At the failure: **T1** is already in the data files (ignored); **T2 and T3** committed after the checkpoint and may exist only in the log (**redone**); **T4** never committed (**undone** — in Postgres, simply invisible). `code/wal_recovery.py` runs this on a toy log — T1 (A→B 300) commits before the checkpoint, T2 (C→D 50) after it, and T3 (A→C 20) is interrupted after one of its pages was flushed — with a forward **redo** pass over updates after the checkpoint and a backward **undo** pass using before-images of transactions that never committed:

```
on disk after crash : {'A': 580, 'B': 400, 'C': 500, 'D': 0}
redo  LSN 206 T2: C = 450
redo  LSN 207 T2: D = 50
redo  LSN 210 T3: A = 580
undo  LSN 210 T3: A = 600
recovered state     : {'A': 600, 'B': 400, 'C': 450, 'D': 50}
money conserved     : True
```

T1 needed nothing, T2 was replayed, T3's early-flushed change was rolled back, and the 1,500 total is intact.

Checkpoints come in two types. A **sharp (consistent)** checkpoint pauses new work, flushes every dirty page and resumes — simple and fully consistent, but a visible stall. A **fuzzy** checkpoint logs a start point, flushes pages *while work continues*, then logs the end; recovery starts from the start point — no downtime, more bookkeeping. PostgreSQL uses the fuzzy style: every `checkpoint_timeout` (5 min default) or when `max_wal_size` of WAL accumulates, spreading writes over `checkpoint_completion_target` (0.9 of the interval by default since PG 14).

Distributed checkpointing. The nodes' saved states must form a **consistent global state**: if node 1's checkpoint records *sending* a message, node 2's must record *receiving* it. **Coordinated** checkpointing (a coordinator, or markers along channels as in the Chandy–Lamport snapshot) is always consistent at the cost of a short pause; **uncoordinated** checkpointing is cheaper but recovery must search for a consistent combination and can cascade rollbacks (the **domino effect**). *Example:* payment, stock and email nodes restore from one global checkpoint, so you never get "stock decremented but payment never recorded" (Apache Flink — a stream-processing engine — uses barrier checkpoints that work this way).

Coordinated commit: two-phase commit (2PC)

Distributed recovery is hard because networks partition, nodes fail and recover at different times, many transactions run concurrently, and every node must agree to commit or abort. When one transaction changes data on **several nodes** (a transfer from an account on node 1 to one on node 2), atomicity must span machines. **2PC** uses a **coordinator**:

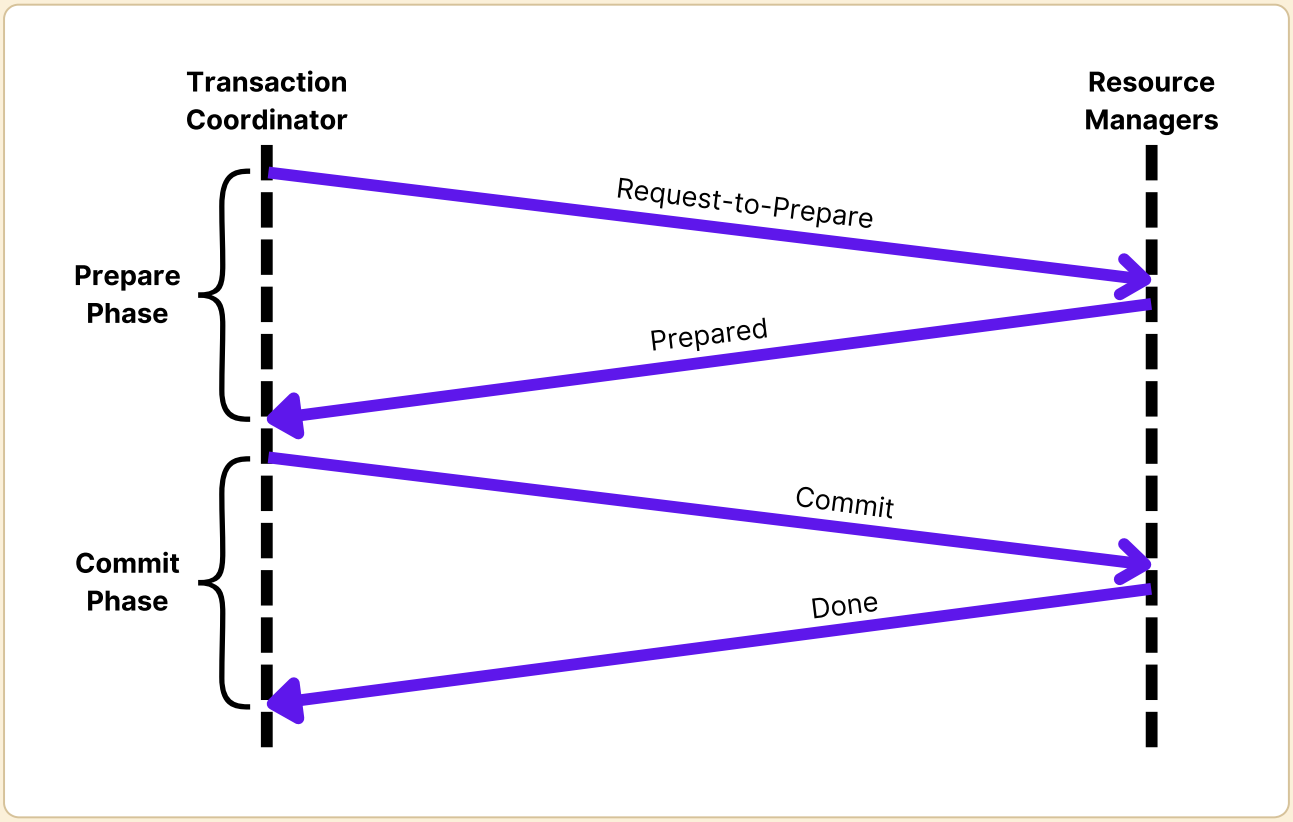


Figure — 2PC message flow: phase 1 asks every participant to prepare and collects their votes; phase 2 sends the commit and waits for acknowledgements (course diagram).

1. **Phase 1 — prepare (vote).** "Can you commit?" Each participant makes its changes durable, logs **PREPARED** and votes **YES** — from then on it may not decide alone — or votes **NO** (constraint failure, disk full).
2. **Phase 2 — decide.** All YES → the coordinator logs **COMMIT** and tells everyone to commit; any NO or timeout → it logs **ABORT** and everyone rolls back.

Recovery: a participant restarting without a PREPARED record aborts alone; one *with* PREPARED must **ask the coordinator**. If the coordinator crashes, participants that haven't voted can abort, but those that **voted YES must wait** — guessing could break atomicity. That wait, with **locks** held (locks stop other transactions touching the same rows), is **2PC's blocking problem**; network delays and partitions also stall decisions. `code/two_phase_commit.py` shows all three cases:

```
--- happy path: debit on node 1, credit on node 2
  ...
  phase 2 final states: {'node1': 'committed', 'node2': 'committed'}
--- one participant votes no (e.g. constraint check failed)
  ...
  phase 2 final states: {'node1': 'aborted', 'node2': 'aborted'}
--- coordinator failure after the votes
  phase 1 votes: {'node1': True, 'node2': True}
  coordinator crashes before sending a decision
  node1: prepared, holding locks, cannot commit or abort alone -> BLOCKED
  node2: prepared, holding locks, cannot commit or abort alone -> BLOCKED
```

2PC in PostgreSQL — the participant side, driven by an external transaction manager *(not run: it needs `max_prepared_transactions > 0`, and the book server keeps the default 0)*:

```
BEGIN;
UPDATE accounts SET balance = balance - 5000 WHERE id = 'A';
PREPARE TRANSACTION 'transfer-42';   -- phase 1: durable, survives a crash, locks kept
COMMIT PREPARED 'transfer-42';       -- later, on the coordinator's decision (or ROLLBACK PREPARED)
```

Java XA/JTA transaction managers (XA is the standard interface for distributed transactions; JTA is Java's API for it) and MySQL's `XA START … XA PREPARE … XA COMMIT` work the same way. Forgotten prepared transactions hold locks and block VACUUM — monitor `pg_prepared_xacts`.

Three-phase commit (3PC)

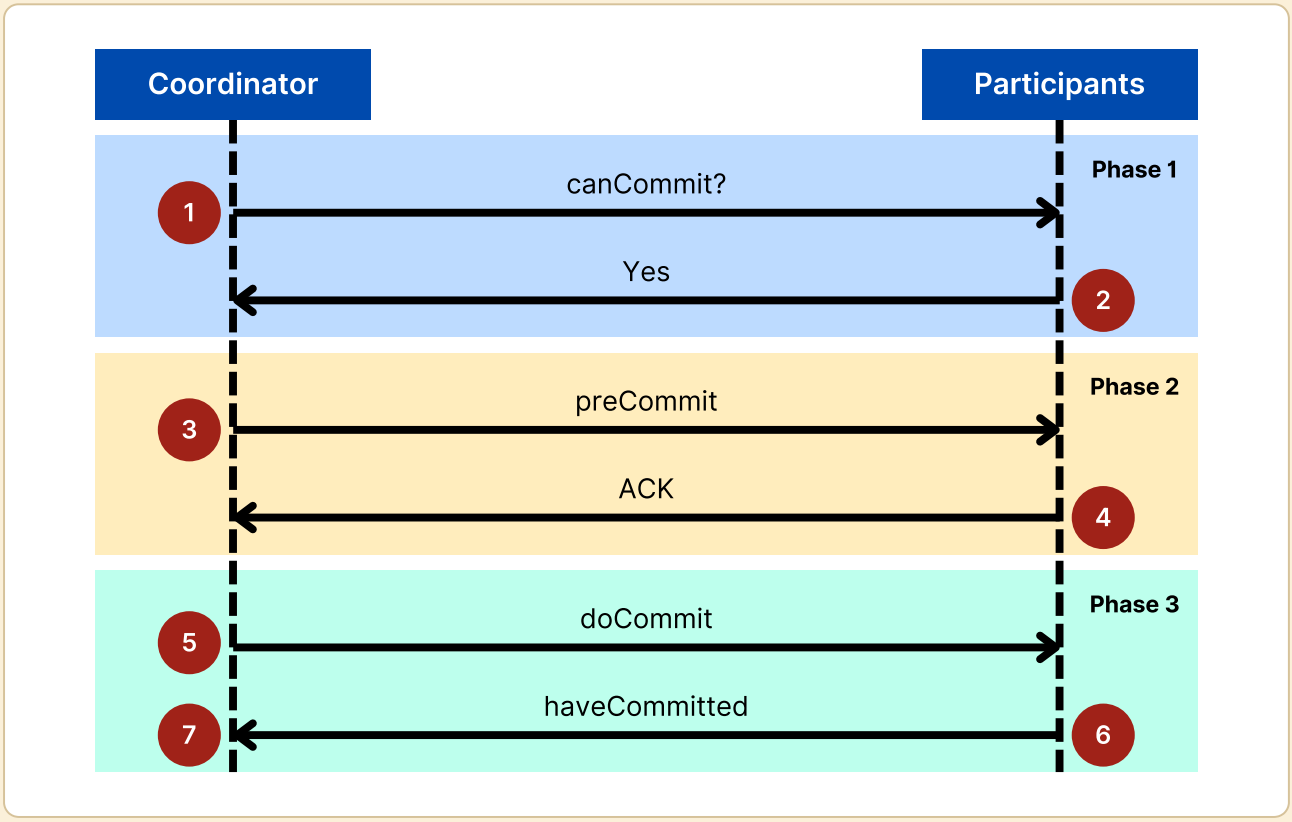


Figure — 3PC: canCommit?/Yes, then preCommit/ACK, then doCommit/haveCommitted (course diagram).

3PC inserts a **PreCommit** step between voting and committing: (1) **CanCommit** — vote as in 2PC; (2) **PreCommit** — if all voted YES, participants acknowledge and log that the outcome will be commit; (3) **DoCommit** — the final commit. If the coordinator dies, participants that logged pre-commit know everyone voted YES and can commit after a timeout; if no live participant has seen pre-commit, nobody can have committed, so they abort. This is **non-blocking** — *provided* message delays are bounded and nodes only crash. Under a real **partition** both sides can time out and decide differently, so 3PC is rarely used. Modern systems make the coordinator fault-tolerant by **running the commit decision through consensus** (Spanner: 2PC over Paxos groups; CockroachDB: a transaction record replicated by Raft), and cross-service workflows use **sagas** (compensating steps) — see 2PC and Saga patterns.

In the interview

30-second answer: "Databases survive crashes with write-ahead logging: the log is flushed before commit is acknowledged and data pages are written later. Recovery starts at the last checkpoint, redoes committed changes and undoes uncommitted ones — Postgres needs no undo thanks to MVCC. Across nodes, 2PC gives atomic commit, but prepared participants block if the coordinator dies, so real systems replicate the decision with Paxos/Raft or use sagas."

- *What does* `synchronous_commit = off` *trade?* Commits return before the WAL flush: a crash can lose the last fraction of a second of commits, never corrupt data.

Self-check

1. *T2 committed after the last checkpoint; T4 hadn't committed. At a crash?* T2 is redone, T4 undone (invisible in Postgres).
2. *A 2PC participant voted YES and lost the coordinator. Can it abort?* No — it must wait.
3. *Sharp vs fuzzy checkpoint?* Sharp pauses work to flush; fuzzy flushes while work continues.

TL;DR

Log first, write data later; checkpoints bound recovery; recovery = redo committed + undo uncommitted. 2PC blocks on coordinator failure and 3PC isn't partition-safe, so modern systems put the commit decision on a consensus group.

15.10 High availability: replication plus failover

What it is

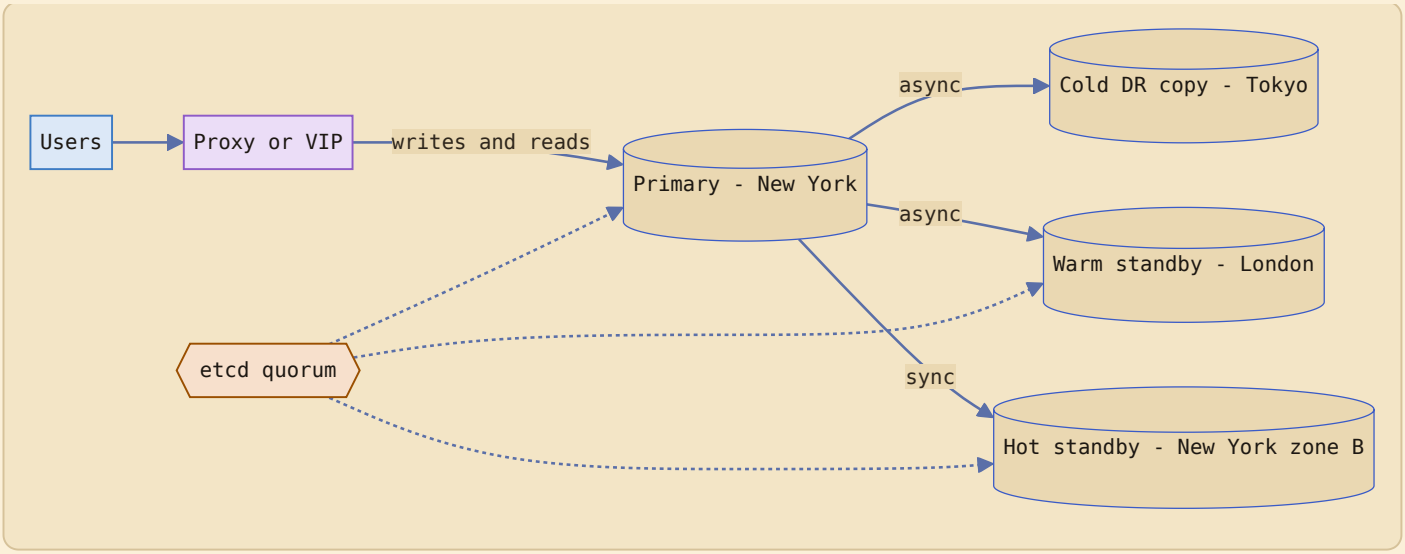
High availability (HA) keeps the service up despite failures, measured in "nines" (the percentage of time the service is up): 99.9% ≈ 8.8 hours of downtime a year, 99.99% ≈ 53 minutes. Two targets make it concrete: **RPO** (recovery point objective — how much *data* you may lose; synchronous replication gives RPO = 0) and **RTO** (recovery time objective — how long you may be *down*; hot standbys and automatic failover make it short). HA = **replication** (15.5; a copy exists) + **failover** (switch to it). A sync standby has every acknowledged commit at failover (RPO 0, slower commits — typical for a local standby in financial systems); an async one may miss the last commits (RPO > 0, fast commits — remote regions, read replicas).

Failover

Type	Standby state	Recovery time	Cost
Cold	Not running; must start and restore/catch up	Minutes to hours	Cheapest
Warm	Running and receiving updates, but needs some catch-up or config	Seconds to minutes	Medium
Hot	Running, in sync, ready to serve immediately	Seconds	Highest

1. **Heartbeats** — several missed "I'm alive" signals raise suspicion.
2. **Decision** — a **quorum** (majority of nodes or witnesses) must agree the primary is dead, preventing **split brain**; **priority rules** pick the standby (highest replayed LSN, preferred zone).
3. **Promotion** — Postgres: `pg_ctl promote` or `SELECT pg_promote()` (PG 12+).
4. **Fencing** the old primary — STONITH ("shoot the other node in the head"), lease revocation, or blocking at the proxy.
5. **Redirecting clients** — DNS, a **virtual IP** (an address that can be moved from one machine to another), a proxy (HAProxy, PgBouncer) or **service discovery** (a registry clients ask "where is the primary right now?").

Core PostgreSQL has **no automatic failover** (switching to a standby without a human — see the steps above); use **Patroni** (leader state in etcd/Consul/ZooKeeper — consensus-backed), **repmgr** or **pg_auto_failover**, or a managed service (Amazon RDS/Aurora, Cloud SQL, Azure Database). **Best practices:** redundancy across failure domains (racks, zones, regions), load-balanced reads with writes routed to the current primary, monitoring (replication lag, heartbeats, disk), **backups too** (a bad `DELETE` replicates in milliseconds — HA is not a backup), and **regular failover tests** (game days, **chaos experiments** — deliberately breaking things in a controlled way to prove recovery works).



A zone failure in New York fails over to the sync hot standby in seconds with no data loss. Losing all of New York promotes London (RPO seconds, RTO about a minute; it also serves European reads). Tokyo is a cold copy for when both are gone. The etcd quorum spans three sites, so a partitioned minority can never promote itself.

In the interview

30-second answer: "HA = replicas plus automatic failover: heartbeats detect failure, a quorum confirms it to avoid split brain, the most up-to-date standby is promoted, the old primary is fenced, clients are redirected. Sync standbys give zero data loss, async lower latency with some loss. I'd state RPO/RTO, choose hot/warm/cold to match, and test failover regularly."

Self-check

1. *RPO with async replication?* Greater than zero — the lag at the moment of failure.

15.11 Data resilience: redundancy, mirroring, backups

HA keeps the service **running**; **data resilience** makes sure the **data itself** survives disk crashes, corruption, ransomware, operator mistakes and site disasters.

Redundancy

Technique	How	Overhead	Survives
Replication	Full copies on other nodes (15.5)	N×	Node or site loss
RAID 1 / 10	Mirror each disk (10 = mirrors, striped; popular for databases)	2×	1 disk per mirror pair
RAID 5 / 6	Stripe data (spread it across disks) with one / two parity blocks (extra blocks computed from the data, used to rebuild a lost disk)	1 / 2 disks	Any 1 / any 2 disks
Erasure coding	k data + m parity fragments; any k rebuild the data	(k+m)/k: 6+3 → 1.5×	Any m fragments

Erasure coding (e.g. Reed–Solomon) gives replication-level safety at 1.5× instead of 3× storage, paid for with CPU and slower rebuilds — **object stores** (services like Amazon S3 that store whole files by key) and HDFS (Hadoop's distributed file system) use it for large, cooler (rarely read) data.

Mirroring

Mirroring keeps an **exact, real-time copy**: every write goes to primary and mirror together, and the mirror takes over immediately. Near-instant failover and always current, but double storage and extra write latency. *Example:* a bank mirrors its transaction database synchronously between two nearby data centres. (SQL Server literally called it "Database Mirroring"; Postgres's equivalent is synchronous streaming replication.)

Media failure recovery

A **media failure** is lost or damaged storage — **physical** (dead drive, bad sectors) or **logical** (corruption from software/firmware bugs or malware). Defences: **hot swapping** failed RAID drives while running; **health monitoring** (**SMART** drive telemetry, cloud volume metrics); **data scrubbing** (periodically read every block, verify **checksums** (small fingerprints computed from data; a mismatch means corruption), repair from redundancy — ZFS, Ceph; Postgres detects corrupted pages with `initdb --data-checksums`); and **replication** to other nodes and regions.

Backup and restoration

A **backup** is an independent copy that can take you **back in time** — replicas can't, they follow the primary instantly. **Restoration** gets data back from it.

Type	Contains	Backup cost	Restore needs
Full	Everything	Slowest, largest	Just that backup (simplest restore)
Incremental	Changes since the **last backup of any kind**	Fastest, smallest	Last full + **every** incremental since, in order
Differential	Changes since the **last full**	Grows through the week	Last full + the **latest** differential

PostgreSQL (commands in Chapter 9 — backup and restore): `pg_dump` / `pg_restore` make **logical** backups of one database, portable across versions. `pg_basebackup` plus **continuous WAL archiving** (`archive_mode`, `archive_command`) gives a **physical** backup you can replay to *any moment* — **point-in-time recovery (PITR)** with `recovery_target_time`, the cure for "someone ran `DELETE` without `WHERE` at 14:02". PostgreSQL 17 added incremental base backups (`pg_basebackup --incremental`); pgBackRest and Barman manage all of this.

Best practices: backup frequency to match RPO (daily plus continuous WAL for critical data); **off-site** copies following the **3-2-1 rule** (3 copies, 2 media, 1 off-site); **test restores** regularly; **versioning and immutability** (object lock — stored copies can't be changed or deleted for a set period) so **ransomware** (malware that encrypts or deletes data and demands payment) can't destroy every copy. *Example:* a hospital takes a full backup every Sunday, incrementals Monday–Saturday and continuous WAL, stores copies locally and in another cloud region, and each month restores "last Wednesday 15:00" into a test environment and checks row counts.

In the interview

30-second answer: "Replication and RAID protect against hardware loss but not bad writes, which replicate too. So I add backups — full plus incremental or differential, off-site, versioned, test-restored. In Postgres that's base backups plus WAL archiving for PITR. Erasure coding gives durable storage at about 1.5× instead of 3× for large, cold data."

- *Accidental `DROP TABLE` at 14:02?* Restore the base backup into a separate instance, replay WAL to 14:01:59, copy the table back.

Self-check

1. *Which RAID level survives any two disk failures?* RAID 6.

2. *Why isn't a synchronous replica a backup?* It copies mistakes instantly; there's no going back in time.

> **TL;DR**
>
> Redundancy (replicas, RAID, erasure coding) survives hardware loss; backups (full/incremental/differential + WAL archiving) survive mistakes and disasters. Keep copies off-site and versioned, and test restores.

Chapter 15 TL;DR

Topic	Remember
Scale	Scale up first, then scale out with **partitioning + replication** together.
Partitioning	Range (time), list (category), hash (even), composite. Verify pruning with `EXPLAIN`. Unique keys include the partition key.
Sharding	Partitions on separate servers; a high-cardinality shard key matching the main query; consistent hashing with vnodes.
Distributed queries	Push filters and partial aggregates to the data, co-locate joins, semi-join to ship keys not rows.
Replication	Sync = no loss but slower; async = fast, lag, possible loss. Single-leader by default. Leaderless quorums need R + W > N.
Consistency	Strong > sequential > causal > session > eventual. Pick per feature.
CAP / PACELC	Under partition choose C or A; otherwise latency or consistency.
Consensus	Raft/Paxos: 2f+1 nodes survive f crashes. PBFT: 3f+1 for f liars. Elect leaders via etcd/ZooKeeper and fence stale ones.
Recovery	WAL first, checkpoints bound replay, redo committed + undo uncommitted (Postgres: redo-only + MVCC). 2PC blocks, 3PC isn't partition-safe; consensus-backed commit is the modern answer.
HA	Heartbeat → quorum decision → promote → fence → redirect. State RPO/RTO.
Resilience	Replicas are not backups. Full/incremental/differential + PITR, off-site, tested.

Next: Chapter 16 — Beyond relational looks at the NoSQL, NewSQL and specialised databases that make these trade-offs in different ways.

Chapter 16 — Beyond Relational: NoSQL, NewSQL and Specialised Databases

Relational databases are the right default for most applications. They offer a flexible query language, joins, constraints and **ACID** transactions (Atomic, Consistent, Isolated, Durable — a transaction either fully happens or not at all, and once committed it survives crashes; Chapter 13). But some workloads fit tables awkwardly. Examples include a product catalogue where every item has different attributes, a **session cache** (fast storage for logged-in users' temporary state) that needs sub-millisecond reads, billions of sensor readings per day, "friends of friends who like jazz", or "every restaurant within 2 km". A whole family of databases grew up around those shapes of data. This chapter tours them, shows how much of each you can already do **inside PostgreSQL**, and ends with a decision table for choosing.

What you'll learn

Section	Topic	Runnable part
16.1	Why NoSQL appeared, CAP and NoSQL, BASE vs ACID, the four families	—
16.2	Key-value stores (Redis, Riak)	KV + TTL + leaderboard in Postgres
16.3	Document databases (MongoDB, CouchDB)	**JSONB as a document store** with GIN indexes
16.4	Column-family and wide-column stores (Cassandra, HBase)	Wide-row time-series table
16.5	Graph databases (Neo4j, Neptune)	Friends-of-friends and shortest path with recursive CTEs
16.6	In-memory databases (SAP HANA, SingleStore, Redis)	—
16.7	NewSQL / distributed SQL (Spanner, CockroachDB)	—
16.8	Time-series and spatial databases	Partitions, BRIN, `date_bin`, geometric KNN with GiST
16.9	Big data frameworks: Hadoop, Spark, SQL-on-Hadoop, Kafka, Flink	Python MapReduce in miniature
16.10	Choosing a database	Decision table
16.11	Inside Elasticsearch/ELK (inverted index, analyzers, shards, refresh) and Redis (event loop, encodings, RDB/AOF, Cluster slots, eviction)	**Python inverted index + BM25 over two shards**

Prerequisites: Chapter 15 (partitioning, replication, CAP, consensus) and Chapter 12 (indexes). For the data-model theory behind this chapter, see your DDIA notes: ch.2 — data models, ch.3 — storage engines (LSM trees, column storage), ch.10 — batch and ch.11 — stream processing. For the pattern view, see Caching, Batch and streaming and Storing data.

All SQL runs on the book's private PostgreSQL 14 (`bash tools_run_sql.sh sql`); the two Python toys (MapReduce, inverted index) run with `bash tools_run_python.sh sql`. Snippets for other products (MongoDB, Redis, Cassandra, Neo4j, PostGIS) are marked **(not run)**.

16.1 Why NoSQL?

What it is

NoSQL ("Not only SQL") is an umbrella term for databases that **don't use the relational table model** as their primary model. The term caught on around 2009. They trade some relational features, usually joins,

a fixed **schema** (the declared list of tables, columns and types), and sometimes multi-row transactions, for **flexible data models, easy horizontal scaling, and speed on specific access patterns**.

The problem it solves

In the 2000s, web companies hit limits with the relational setups of the time:

Pain with relational (as typically deployed then)	NoSQL answer
Rigid schema: every row has the same columns, and changing them means **migrations** (scripted schema changes applied to a live database)	**Flexible/dynamic schema**: each record carries its own fields (JSON — a text format of nested key/value objects and lists — or BSON documents, dynamic columns)
Scale-up only: one bigger server, which gets expensive and eventually hits a ceiling	**Scale-out built in**: automatic sharding and replication across cheap nodes
Joins and transactions get slow at huge volume	Data **stored the way it's read** (denormalised — deliberately duplicated — or pre-aggregated), so there are no joins in the **hot path** (the code run on every request)

Example. An online marketplace sells laptops (RAM, CPU, warranty), T-shirts (size, material) and books (authors, pages). In a relational schema that means **sparse columns** (many columns that are empty for most rows), an **entity-attribute-value** table (one row per `(item, attribute name, value)` triple), or one table per category. In a document store, each product is simply a document with whatever fields it has (16.3).

Modern relational systems have closed much of this gap: Postgres has JSONB (a binary, indexable JSON column type, 16.3), partitioning, logical replication, and **Citus** (an extension that spreads Postgres tables across many servers) for scale-out. That's why this chapter shows the Postgres equivalent of each model.

CAP and NoSQL

Recall CAP (15.7; also summarised in System Design Fundamentals). During a **network partition** (some nodes can't reach others) a distributed store must choose **consistency** or **availability**:

- **CP** stores refuse or delay requests on the minority side to stay consistent. Examples: **HBase**, MongoDB (with majority write/read concern — the setting for how many replicas must confirm a write or agree on a read), etcd.
- **AP** stores keep answering and reconcile later. Examples: **Cassandra**, Riak, CouchDB, DynamoDB reads (eventually consistent unless you request a strongly consistent read).

Many NoSQL systems make this **tunable per request**, such as Cassandra's `ONE` vs `QUORUM`.

BASE: the other end of the spectrum from ACID

Many AP-leaning NoSQL systems describe their guarantees as **BASE**:

- **Basically Available**: the system answers almost every request, even during partial failures, though the answer may not be the newest. *Example:* a Cassandra read at consistency `ONE` succeeds as long as one replica is up.
- **Soft state**: replicas may be temporarily out of sync, and the state can change without new input as updates propagate. *Example:* asynchronous replication in a MongoDB **replica set** (a group of servers holding the same data: one **primary** takes writes, **secondaries** copy it), where a secondary can lag the primary.
- **Eventual consistency**: if writes stop, all replicas converge (15.6). *Example:* a Cassandra write that reached only some replicas is spread by hinted handoff (another node holds the write until the missing replica returns), read repair (a reader fixes stale copies it notices) and anti-entropy (background comparison and syncing of replicas).

BASE is essentially the **AP** choice from CAP, packaged as a design philosophy. It suits social feeds, activity streams and metrics, where being up matters more than every reader seeing the very latest value.

Property	ACID (typical relational)	BASE (typical AP NoSQL)
Atomicity	All-or-nothing, including multi-row transactions	Often per single record/document only
Consistency	Constraints hold after every commit, and reads see committed data	Replicas may briefly disagree and converge later
Isolation	Concurrent transactions don't interfere (per isolation level)	Weaker, and concurrent writes may conflict and need resolution
Durability	Committed = on disk (WAL)	Configurable (write concern, replication factor = number of copies, fsync policy = how often data is forced to disk)
Availability focus	Consistency first during failures	Availability first
Scaling	Traditionally vertical plus read replicas	Horizontal by design

Correction to a common myth: the line has blurred. MongoDB has supported multi-document ACID transactions since 4.0 (2018). DynamoDB has had `TransactWriteItems` since 2018. Cassandra has lightweight transactions (Paxos-based **compare-and-set** — "set the value to X only if it is still Y", done atomically; Paxos is a consensus protocol, 15.8). Meanwhile Postgres stores schemaless JSON. Treat ACID vs BASE as a **spectrum of defaults**, not a hard divide.

The four NoSQL families

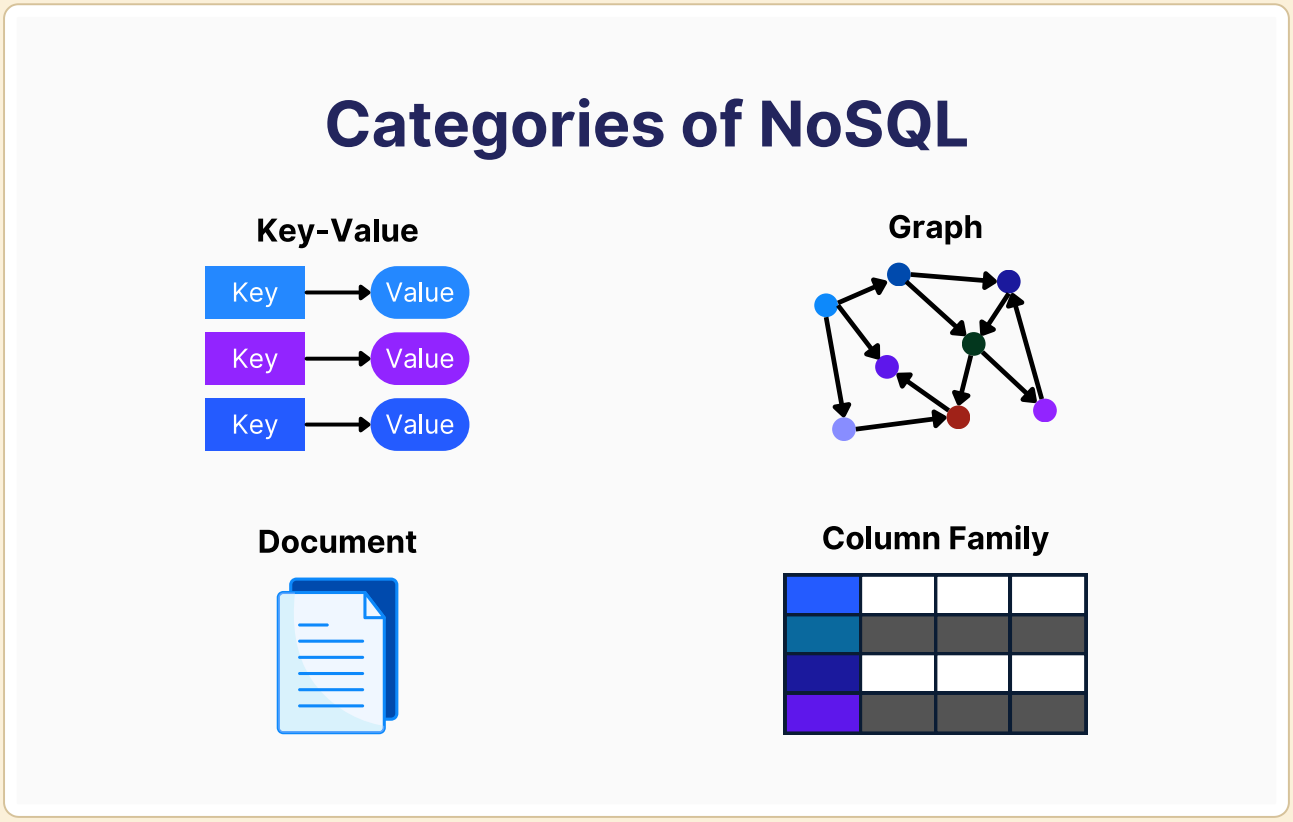


Figure — The main NoSQL families: key-value, document, column-family and graph (course diagram).

Family	Data model	Great at	Examples
Key-value	Opaque (not interpreted by the database) value looked up by a unique key	Caching, sessions, counters, leaderboards	Redis, Riak KV, Memcached, DynamoDB (also a document store)
Document	JSON-like documents grouped into collections	Catalogues, CMS (content management system) content, user profiles	MongoDB, CouchDB, Firestore
Column-family / wide-column	Rows with a partition key, columns grouped into families, sorted within a partition	Huge write volumes, time-series, logs	Cassandra, HBase, ScyllaDB, Bigtable
Graph	Nodes and edges, both with properties	Relationships: social, fraud rings, recommendations	Neo4j, Amazon Neptune, ArangoDB

Features they share: **dynamic schema** (no migration to add a field), **horizontal scaling** (add nodes), **performance tuned to one access pattern**, and **fault tolerance** through built-in replication.

In the interview

30-second answer: "NoSQL databases drop the relational model to get flexible schemas, easy scale-out and speed on specific access patterns. There are four families: key-value, document, wide-column and graph. Many lean AP in CAP terms and follow BASE (basically available, soft state, eventually consistent) instead of strict ACID, though modern ones offer transactions and tunable consistency. You model data around queries, not around normalisation."

- *Is MongoDB ACID?* Single-document operations always were. Multi-document transactions exist since 4.0 (replica sets) and 4.2 (sharded clusters).
- *When is Postgres JSONB enough?* When you need flexible attributes *plus* joins, constraints and transactions, at a scale one Postgres (or Citus) can handle.

Self-check

1. *What does the S in BASE mean?* Soft state: replica state may change as updates propagate, even with no new writes.
2. *Name a CP and an AP NoSQL store.* HBase (CP) and Cassandra (AP by default).
3. *Which family fits "who are my friends' friends"?* Graph.

TL;DR

NoSQL = flexible models + scale-out + access-pattern-specific speed, often with BASE semantics. The ACID/BASE divide is now a spectrum.

16.2 Key-value stores

What it is

The simplest database: a giant, distributed **dictionary**. Each **key** is unique (a string or number), and the **value** is whatever you store: text, a number, JSON, or a binary **blob** (raw bytes such as an image). The database doesn't interpret the value. You `GET key`, `SET key value` and `DELETE key`, and there are no joins and usually no queries by value.

```
session:9f2c      -> {"user_id": 42, "cart": ["LAP-13"], "expires": "..."}
rate:10.0.0.7     -> 17
weather:paris     -> {"temp_c": 18}
```

Designing keys is the whole data model. Use prefixes like `user:42:profile`, because the key is the only way in.

Key-value stores fit when you need very fast access by a known identifier, when relationships between items are minimal or handled in the application, and when each pair can be treated independently.

Popular systems

Redis ("Remote Dictionary Server") keeps everything **in memory** and answers in well under a millisecond. Values can be rich **data structures**: strings, hashes (small field→value maps), lists, sets, **sorted sets** (members kept ordered by a numeric score), streams, bitmaps and HyperLogLogs (a tiny structure that estimates how many distinct items it has seen). It also offers **pub/sub** messaging (publishers send to a channel; every subscriber to it receives the message), **Lua** scripting (small scripts run inside the server), key expiry (**TTL**, time to live — the key deletes itself after a set time) and **eviction policies** (rules for which keys to throw out) such as **LRU** (least recently used) / **LFU** (least frequently used) when memory is full. **Persistence** (saving data to disk so it survives a restart) is optional, via RDB snapshots and/or an append-only file (AOF). Redis Cluster shards keys over 16,384 hash slots (fixed buckets each key is hashed into; 16.11).

```
# Redis (not run)
SET session:42 '{"user":42}' EX 1800     # expires in 30 min
GET session:42
INCR page:home:views                      # atomic counter
ZADD steps 12050 ana 8300 bo 15010 cy     # sorted set = leaderboard
ZREVRANGE steps 0 2 WITHSCORES            # top 3
```

Riak KV is a Dynamo-style (modelled on Amazon's Dynamo paper) **distributed** key-value store: leaderless replication, consistent hashing, tunable N/R/W quorums (N copies, W must confirm a write, R are asked on a read — 15.5), and **eventual consistency**, with sibling values for conflicts. It was built for high availability and survives node failures without failover. Riak's original company (Basho) closed in 2017, and the code lives on as open source.

Use cases

Use	Example	Why KV fits
Caching	A weather app caches the latest forecast per city. A shop caches product pages in Redis to spare the database.	Read by key, expiry built in
Session storage	Logged-in user state for a web app	Tiny, per-user, high-churn data
Real-time data	Live stock prices or ride locations updated many times a second	In-memory speed
Counters and rate limits (caps on how often a client may call)	"100 requests per minute per API key"	Atomic `INCR` with a TTL
Leaderboards	A fitness app ranks users by daily steps	Sorted sets keep order as scores change

Performance: strengths and limits

Strengths	Limits
O(1) lookups by key (constant time, however big the data), very high **throughput** (operations per second)	**No ad-hoc queries** (one-off questions not planned in advance): filtering, joins and secondary lookups must be built in the app or with extra keys
Easy to shard (hash the key) and replicate	**Poor fit for relationships** between items
Values can be anything	In-memory stores are **bounded by RAM** (and RAM is expensive)

The same pattern in Postgres

`code/02_key_value_style.sql` builds a KV table with TTL, **UPSERT** (insert, or update if the key already exists) "put", and a leaderboard:

```
CREATE UNLOGGED TABLE kv (            -- UNLOGGED: no WAL, fast, emptied after a crash (cache-like)
  k          text PRIMARY KEY,
  v          jsonb NOT NULL,
  expires_at timestamptz
);
-- PUT = insert or overwrite
INSERT INTO kv VALUES ('session:42', '{"user":42,"cart":["LAP-13","TSH-01"]}', now() + interval '30
minutes')
ON CONFLICT (k) DO UPDATE SET v = EXCLUDED.v, expires_at = EXCLUDED.expires_at;
INSERT INTO kv VALUES ('weather:paris', '{"temp_c":18}', now() - interval '1 minute');  -- already
expired
-- GET only unexpired keys
SELECT k, v FROM kv WHERE k = 'session:42' AND (expires_at IS NULL OR expires_at > now());
```

Output:

```
     k      |                    v
------------+-------------------------------------------
 session:42 | {"cart": ["LAP-13", "TSH-01"], "user": 42}

 weather_hits
--------------
            0
```

The expired `weather:paris` key isn't returned. The leaderboard is a window function (Chapter 8), and ties share a rank:

```
 rank | user_name | steps
------+-----------+-------
    1 | cy        | 15010
    2 | ana       | 12050
    3 | bo        |  8300
    3 | di        |  8300
```

Cleanup is `DELETE FROM kv WHERE expires_at <= now() RETURNING k`, which evicts `weather:paris`. Redis does this expiry automatically. And `pg_class.relpersistence = 'u'` confirms the table is unlogged. This is fine for moderate loads. Redis still wins on latency, on built-in expiry and on data structures.

Common mistakes

- Using a cache as the **source of truth** (the authoritative copy every other copy derives from) without persistence, then losing data on restart.
- Huge values (MBs) or "hot keys" (single keys getting a large share of all traffic) that one shard must serve alone.

- Forgetting TTLs, so memory fills and eviction starts dropping useful keys.

In the interview

> **30-second answer:** "A key-value store is a distributed hash map: O(1) get/put by key, easy to shard and replicate, no queries by value. Redis adds in-memory speed and data structures like sorted sets. I use it for caching, sessions, counters, rate limiting and leaderboards, with a TTL on keys, and keep the durable truth in a primary database."

- *How would you build a rate limiter?* `INCR key` with `EXPIRE` for a fixed window, or a sorted set of timestamps for a sliding window.
- *Cache invalidation strategy?* **Cache-aside** (the app checks the cache, and on a miss reads the database and fills the cache) with TTLs, and delete or refresh the key on writes. See the caching patterns.

Self-check

1. *Why can't a pure KV store answer "all sessions of user 42"?* It only looks up by key. You'd need a secondary key such as `user:42:sessions` maintained by the app.
2. *What does Redis's sorted set give a leaderboard?* Members kept ordered by score, with O(log n) updates and fast range queries.
3. *What does `UNLOGGED` trade in Postgres?* Speed (no WAL) against durability. The table is truncated after a crash and is not replicated.

TL;DR

Key-value = dictionary at scale. It's the fastest way to get by key, with nothing beyond that. Perfect for caches, sessions, counters and leaderboards.

16.3 Document databases

What it is

A **document database** stores data as self-contained **documents**, usually JSON or a binary form of it (MongoDB's **BSON**). Documents are grouped into **collections** (roughly, tables), and collections into a **database**. Every document has a unique id (`_id` in MongoDB). Two documents in one collection may carry **different fields** and can **nest** objects and arrays.

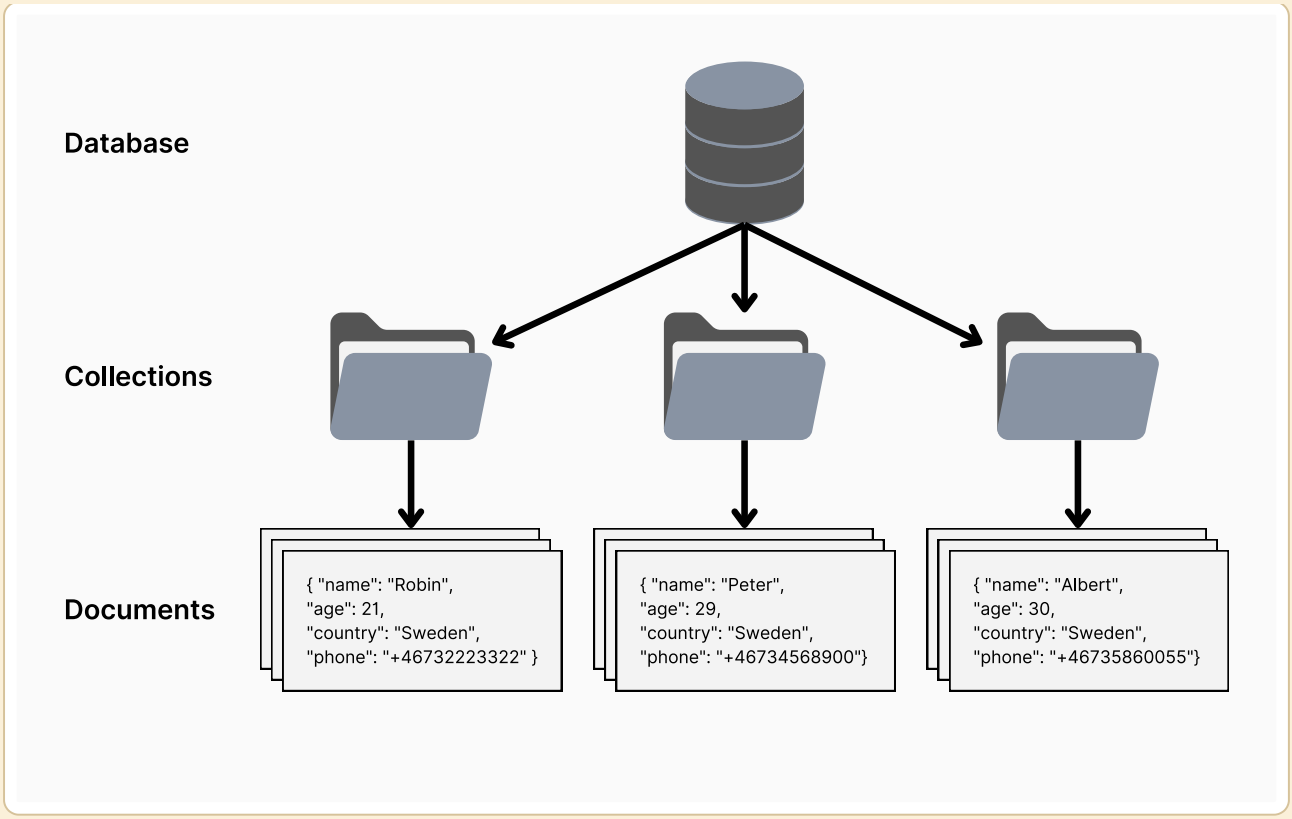


Figure — A document database: a database holds collections, and each collection holds JSON documents (course diagram).

```
{ "_id": 1, "name": "Meera", "city": "Pune",  "phone": "+91 98xxxxxx01" }
{ "_id": 2, "name": "Tomás", "city": "Porto", "languages": ["pt", "en"] }
```

The second document has a field the first lacks, and that's allowed.

Key features

1. **Flexible schema**: add fields without migrations. A product catalogue can hold electronics with `warranty` and `brand` next to clothing with `size` and `material`.
2. **Nested structures**: a customer document can **embed** their addresses and recent orders, so one read returns everything a page needs.
3. **Performance without joins**: data that is read together is stored together.
4. **Horizontal scaling**: sharding by a shard key is built in (MongoDB).

Popular systems

- **MongoDB**: BSON documents, a rich query language and **secondary indexes** (indexes on fields other than the primary id), **aggregation pipelines** (`$match` → `$group` → `$sort`, similar to SQL's `WHERE` / `GROUP BY`), replica sets (single-leader, 15.5), sharding, and multi-document transactions since 4.0.
- **CouchDB**: JSON over a **RESTful HTTP API** (resources addressed by URLs and manipulated with standard web verbs such as GET, PUT and DELETE), multi-leader **replication and sync** designed for occasionally connected devices (PouchDB in the browser syncs with it), and **MVCC** (multi-version concurrency control — keeping several versions of a record) document revisions for conflict detection.

```
// MongoDB shell (not run)
db.products.insertOne({ sku: "LAP-15", type: "laptop", price: 1399, specs: { ram_gb: 32 } })
db.products.find({ type: "laptop", "specs.ram_gb": { $gte: 16 } }).sort({ price: 1 })
db.products.aggregate([{ $group: { _id: "$type", avgPrice: { $avg: "$price" } } }])
db.products.createIndex({ type: 1, price: 1 })
```

Use cases

Use	Example
Content management	Blog posts with title, body, tags, author and a varying set of metadata
Product catalogues	Each category has different attributes, all in one collection
User profiles and settings	Optional, evolving fields per user
Event and activity logging	A game records player actions as documents for later analysis

Modelling: embed or reference?

Embed (nest inside the parent)	Reference (store the id, look it up separately)
Data is read together and owned by the parent (a customer's addresses)	Data is shared by many parents (a product referenced by many orders)
Bounded size (a few addresses)	Unbounded growth (all of a user's comments ever)
One read, atomic single-document update	Needs a second query or `$lookup` (the document world's join)

Strengths and limits

Strengths	Limits
Schema flexibility for evolving data	**Many-to-many relationships** and cross-document constraints are awkward. Relational handles them better.
Fast reads when documents match the page	**Duplicated data** (denormalised) must be kept in sync by the app
Built-in scale-out	Some configurations favour availability over strong consistency (e.g. reading from secondaries)

A document store inside Postgres: JSONB

PostgreSQL's `jsonb` type stores parsed, binary JSON. It supports operators, indexing and **SQL/JSON path** queries (a mini-language like `$.authors[*]` for navigating inside a JSON value). `code/01_jsonb_document_store.sql` builds a mini product catalogue:

```
CREATE TABLE products (
  id  bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  doc jsonb NOT NULL
);
INSERT INTO products (doc) VALUES
 ('{"sku":"LAP-13","type":"laptop","name":"Aero 13","price":999,
    "brand":"Nimbus","warranty_years":2,"specs":{"ram_gb":16,"cpu":"8-core"}}'),
 ('{"sku":"TSH-01","type":"tshirt","name":"Plain Tee","price":15,
    "sizes":["S","M","L"],"material":"cotton"}'),
 ('{"sku":"LAP-15","type":"laptop","name":"Aero 15","price":1399,
    "brand":"Nimbus","warranty_years":3,"specs":{"ram_gb":32,"cpu":"12-core"}}'),
 ('{"sku":"BK-042","type":"book","name":"Data at Scale","price":45,
    "authors":["R. Rao","M. Chen"],"pages":412}');

SELECT doc->>'sku' AS sku, (SELECT string_agg(k, ',' ORDER BY k) FROM jsonb_object_keys(doc) k) AS keys
FROM products ORDER BY id;
```

```
  sku   |                      keys
--------+-------------------------------------------------
 LAP-13 | brand,name,price,sku,specs,type,warranty_years
 TSH-01 | material,name,price,sizes,sku,type
 LAP-15 | brand,name,price,sku,specs,type,warranty_years
 BK-042 | authors,name,pages,price,sku,type
```

Every row has its own shape, just like documents in a collection.

Querying. `->` returns JSON, `->>` returns text, and `#>>` follows a path:

```
SELECT doc->>'name' AS name, (doc->>'price')::numeric AS price, doc#>>'{specs,ram_gb}' AS ram_gb
FROM products WHERE doc->>'type' = 'laptop' ORDER BY price;
```

```
  name   | price | ram_gb
---------+-------+--------
 Aero 13 |   999 | 16
 Aero 15 |  1399 | 32
```

Containment `@>` means "does the document contain this sub-document?". Arrays match if they contain the listed elements:

```
SELECT doc->>'name' AS name FROM products WHERE doc @> '{"sizes":["M"]}';                    -- Plain Tee
SELECT doc->>'name' AS name FROM products WHERE doc @> '{"brand":"Nimbus","specs":{"ram_gb":32}}'; -- Aero 15
SELECT doc->>'name' AS name FROM products WHERE doc ? 'authors';                             -- Data at Scale
SELECT jsonb_path_query(doc, '$.authors[*]') AS author FROM products WHERE doc->>'sku' = 'BK-042';
```

```
  author
-----------
 "R. Rao"
 "M. Chen"
```

Partial updates use `jsonb_set` (set a path), `||` (merge keys) and `-` (remove a key):

```
UPDATE products SET doc = jsonb_set(doc, '{price}', '1299') WHERE doc->>'sku' = 'LAP-15';
UPDATE products SET doc = doc || '{"on_sale":true}'         WHERE doc->>'sku' = 'TSH-01';
UPDATE products SET doc = doc - 'warranty_years'            WHERE doc->>'sku' = 'LAP-13';
```

```
  sku   | price | on_sale | warranty
--------+-------+---------+----------
 LAP-13 | 999   |         |
 TSH-01 | 15    | true    |
 LAP-15 | 1299  |         | 3
 BK-042 | 45    |         |
```

(Under the hood Postgres rewrites the whole row version. JSONB is not updated in place, which is why huge, frequently updated documents are a poor fit.)

Indexing. After adding 20,000 generated products:

```sql
CREATE INDEX products_doc_gin ON products USING gin (doc jsonb_path_ops);   -- serves @> on ANY key
EXPLAIN (COSTS OFF) SELECT count(*) FROM products WHERE doc @> '{"sku":"GEN-777"}';

CREATE INDEX products_price_idx ON products (((doc->>'price')::numeric));  -- B-tree on ONE field
EXPLAIN (COSTS OFF) SELECT doc->>'sku' FROM products WHERE (doc->>'price')::numeric > 503;
```

```
Aggregate
  ->  Bitmap Heap Scan on products
        Recheck Cond: (doc @> '{"sku": "GEN-777"}'::jsonb)
        ->  Bitmap Index Scan on products_doc_gin
              Index Cond: (doc @> '{"sku": "GEN-777"}'::jsonb)

Bitmap Heap Scan on products
  Recheck Cond: (((doc ->> 'price'::text))::numeric > '503'::numeric)
  ->  Bitmap Index Scan on products_price_idx
        Index Cond: (((doc ->> 'price'::text))::numeric > '503'::numeric)
```

The **GIN** index (generalised inverted index, Chapter 12 — it maps each key/value inside the documents to the rows containing it) covers containment queries on any key. (A **Bitmap Index Scan** collects matching row locations from the index; the **Bitmap Heap Scan** then fetches those rows from the table.) `jsonb_path_ops` is smaller and faster for `@>` but doesn't support `?`, while the default `jsonb_ops` supports both. An **expression B-tree** (a normal sorted index built on the result of an expression rather than a plain column) serves range and sort queries on one field.

Guard rails that document databases often lack. A `CHECK` constraint can enforce required keys and types:

```sql
ALTER TABLE products ADD CONSTRAINT doc_has_sku_and_price
  CHECK (doc ? 'sku' AND jsonb_typeof(doc->'price') = 'number');
-- inserting '{"name":"No SKU","price":"cheap"}' now fails:
-- NOTICE:  rejected: new row for relation "products" violates check constraint "doc_has_sku_and_price"
```

A common hybrid design is **regular columns for the stable, queried fields** (sku, price, type) plus a **JSONB column for the variable attributes**. That gives relational integrity where it matters and flexibility where it helps. More JSON functions are in Chapter 14.

In the interview

30-second answer: "A document store keeps self-contained JSON documents in collections, with flexible schemas and nesting, so a page can be served by one read. I embed data that's owned and read together, and reference data that's shared or unbounded. It's weaker at many-to-many relationships and cross-document constraints. If I also need joins and transactions, Postgres JSONB with a GIN index gives most of the benefit."

- *`json` vs `jsonb` in Postgres?* `json` stores the exact text (keeps whitespace, duplicate keys and key order). `jsonb` stores a parsed binary form that is indexable and faster to query. Use `jsonb`.
- *How do you index a JSONB field for range queries?* A B-tree expression index on `((doc->>'field')::type)`.

Self-check

1. *Which operator asks "does this JSON contain that sub-JSON"?* `@>`.
2. *When should you reference instead of embed?* When the child data is shared or grows without bound.
3. *Which index type serves `@>` on any key?* GIN.

TL;DR

Documents = flexible, nested records read in one go. Model around your queries. In Postgres, `jsonb` + GIN + CHECK constraints gives you a document store with SQL around it.

16.4 Column-family and wide-column stores

What it is

A **column-family** database stores rows identified by a **row key**. Each row's columns are grouped into **column families**, which are named groups of related columns stored together on disk. Rows don't need the same columns, and a row can have thousands of them, hence **"wide"**.

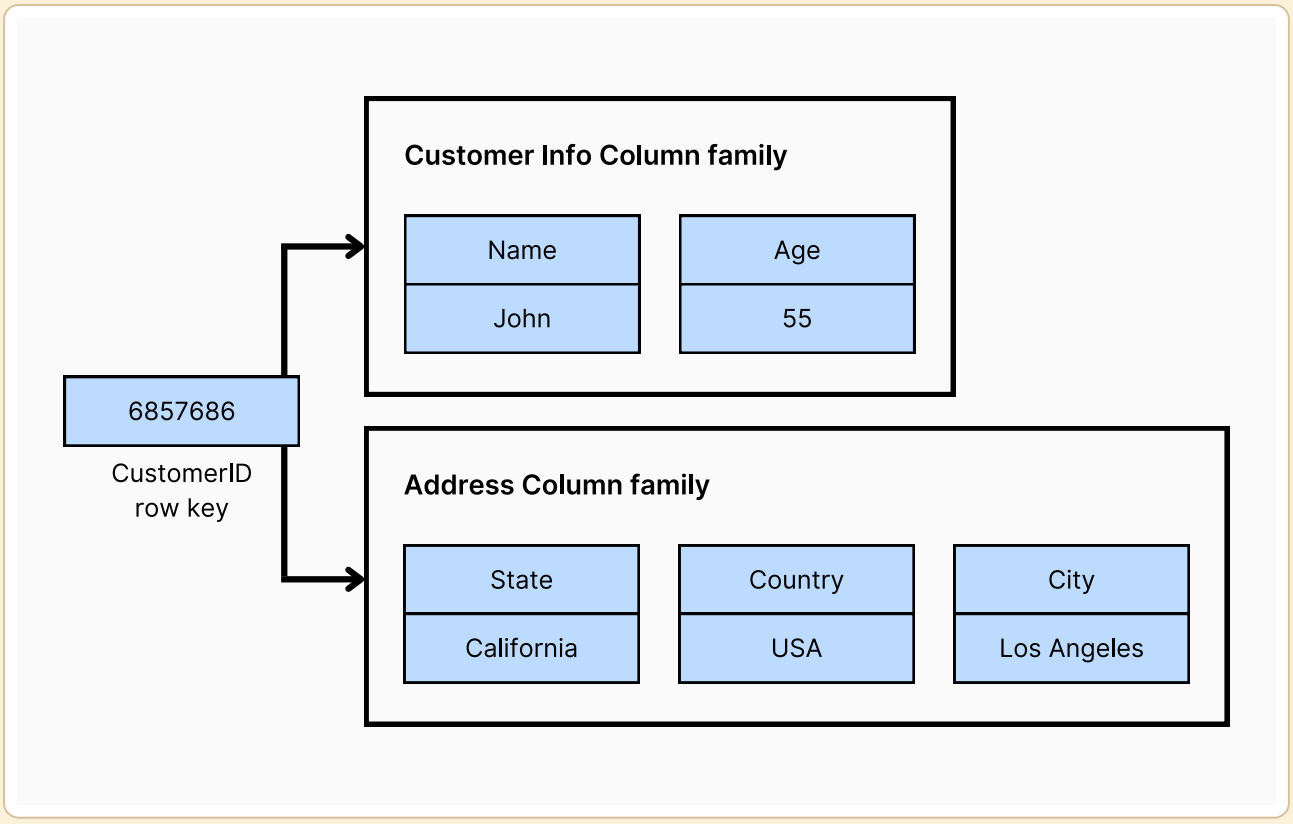


Figure — Row key `CustomerID` with a "Customer Info" family (Name, Age) and an "Address" family (State, Country, City) (course diagram).

For example, row key `cust#6857686` might have:

- family **info**: `name = "John"`, `age = 55`
- family **address**: `state = "California"`, `country = "US"`, `city = "..."`

A query for the address reads only the `address` family's storage. This is the **column-family model** of Google's Bigtable (2006), which HBase copied.

The **wide-column model**, as used by Cassandra and ScyllaDB, builds on it for distribution:

- A **partition key** decides which node stores the row (consistent hashing, 15.3). All rows with the same partition key live together.
- **Clustering columns** sort the rows *within* a partition, which makes range scans within a partition cheap.
- Replication is typically **eventually consistent** with **tunable** consistency.

```cql
-- Cassandra CQL (not run): one partition per server per day, rows sorted by time
CREATE TABLE metrics (
  server_id text, day date, ts timestamp, cpu_pct float, mem_mb int,
  PRIMARY KEY ((server_id, day), ts)          -- (partition key), clustering column
) WITH CLUSTERING ORDER BY (ts DESC);
SELECT ts, cpu_pct FROM metrics WHERE server_id = 'S1' AND day = '2024-12-06' LIMIT 60;
```

> **Don't confuse** *wide-column* stores (Cassandra, HBase) with **columnar** analytics databases (ClickHouse, Redshift, BigQuery, Parquet files). Columnar stores keep each *column* of a table contiguous on disk so that `SUM(amount)` over billions of rows reads only one column. Wide-column stores keep each *row partition* together, organised by key. The names are similar but the purposes are different (DDIA ch.3 — column-oriented storage).

Why they fit time-series and write-heavy data

- **Sequential writes**: most use **LSM trees** (log-structured merge trees). Writes go to an in-memory table plus a commit log, then are flushed as **immutable** (never modified after writing) sorted files (**SSTables**, sorted string tables) that are merged in the background (**compaction**). Writes are appends, so throughput is very high.
- **Partitioning by time range**: a partition key such as `(sensor, day)` spreads load and keeps each day's data together.
- **Flexible columns**: a new metric is just a new column, with no schema migration for the existing rows.

server_id	ts	cpu_pct	mem_mb
web-1	2024-12-06 10:00	41	1980
web-1	2024-12-06 10:01	44	2012
db-1	2024-12-06 10:00	63	1705

A server-monitoring feed: partitioned by server, clustered by time, so "last hour of web-1" is one sequential read.

Cassandra vs HBase

	Cassandra (wide-column)	HBase (column-family)
Origin	Facebook (2008), combining Dynamo's distribution with Bigtable's data model	Apache clone of Google Bigtable, on Hadoop
Architecture	Leaderless ring, every node equal	Master + RegionServers, data in **HDFS** (Hadoop's distributed file system, 16.9)
Scaling unit	Token ranges on the ring	**Regions** (key ranges) that split automatically as they grow
Consistency	**Tunable** per query (`ONE`, `QUORUM`, `ALL`), AP by default	**Strong** per-row consistency (CP)
Query	CQL (Cassandra Query Language: SQL-like, no joins)	Java API, get/put/scan by row key, or SQL through Apache Phoenix
Typical use	User activity feeds, messaging, IoT at global scale	Big-data random read/write next to Hadoop and Spark jobs, e.g. transaction histories for fraud analysis

Use cases

- **Time-series and IoT**: temperature and humidity from millions of smart-home sensors.
- **Logs**: application and server logs for fast troubleshooting.

- **Activity and recommendation data**: every view and purchase per user, fed into recommenders.
- **Real-time analytics**: counters and recent windows for dashboards and alerts, such as price movements.

Limits: you must design tables **per query** (the same data is often written to several tables), there are no joins, secondary indexes are limited and ad-hoc queries are hard, and deletes create **tombstones** (markers saying "this was deleted", since immutable files can't be edited) that slow reads until compaction.

The same idea in Postgres

`code/04_time_series_and_wide_rows.sql` mirrors the wide-column layout. A composite primary key `(server_id, ts)` acts like partition key + clustering column, and range partitions by day provide retention:

```
CREATE TABLE metrics (
  server_id text        NOT NULL,       -- like a wide-column "partition key"
  ts        timestamptz NOT NULL,       -- like a "clustering key": rows sorted by time
  cpu_pct   real,
  mem_mb    int,
  PRIMARY KEY (server_id, ts)
) PARTITION BY RANGE (ts);
-- one partition per day; 3 servers x 2 days x 1,440 minutes = 8,640 readings
EXPLAIN (COSTS OFF)
SELECT ts, cpu_pct FROM metrics
WHERE server_id = 'web-1' AND ts >= '2024-12-06 23:30' AND ts < '2024-12-07 00:00';
```

```
 Bitmap Heap Scan on metrics_2024_12_06 metrics
   Recheck Cond: ((server_id = 'web-1'::text) AND (ts >= '2024-12-06 23:30:00+00'::timestamp with time
zone) AND ...)
   ->  Bitmap Index Scan on metrics_2024_12_06_pkey
         Index Cond: ((server_id = 'web-1'::text) AND (ts >= '2024-12-06 23:30:00+00'::timestamp with
time zone) AND ...)
```

One partition is scanned, and inside it the primary key's `(server_id, ts)` order gives a narrow range scan, which is exactly the wide-column access pattern. (More in 16.8.)

In the interview

30-second answer: "Wide-column stores like Cassandra distribute rows by a partition key and sort them within the partition by clustering columns. Writes go to LSM trees, so they're append-fast. You design one table per query and live with no joins. Consistency is tunable per request. They're great for time-series, logs and activity feeds at huge write rates. They aren't the same as columnar analytics stores."

- *How would you model "latest 50 messages per chat" in Cassandra?* Partition key `chat_id` (plus a time bucket if chats are huge), clustering column `sent_at DESC`, then `LIMIT 50`.
- *Why bucket the partition key by day?* To keep partitions bounded in size (Cassandra's guidance is roughly 100 MB or less per partition) and to spread hot writes.

Self-check

1. *What decides which node stores a Cassandra row?* The hash of its partition key.
2. *What orders rows inside a partition?* The clustering columns.
3. *HBase consistency vs Cassandra default?* HBase is strongly consistent per row. Cassandra is tunable, eventual by default.

TL;DR

Wide-column = partition key for placement + clustering key for order + LSM for fast writes. One table per query. Built for massive write-heavy, time-ordered data.

16.5 Graph databases

What it is

A **graph database** stores **nodes** (entities such as people, products or places), **edges** (relationships such as `KNOWS`, `BOUGHT`, `MEMBER_OF`) and **properties** on both (`name`, `since`, `weight`). Edges are first-class, stored data, not something computed at query time from foreign keys.

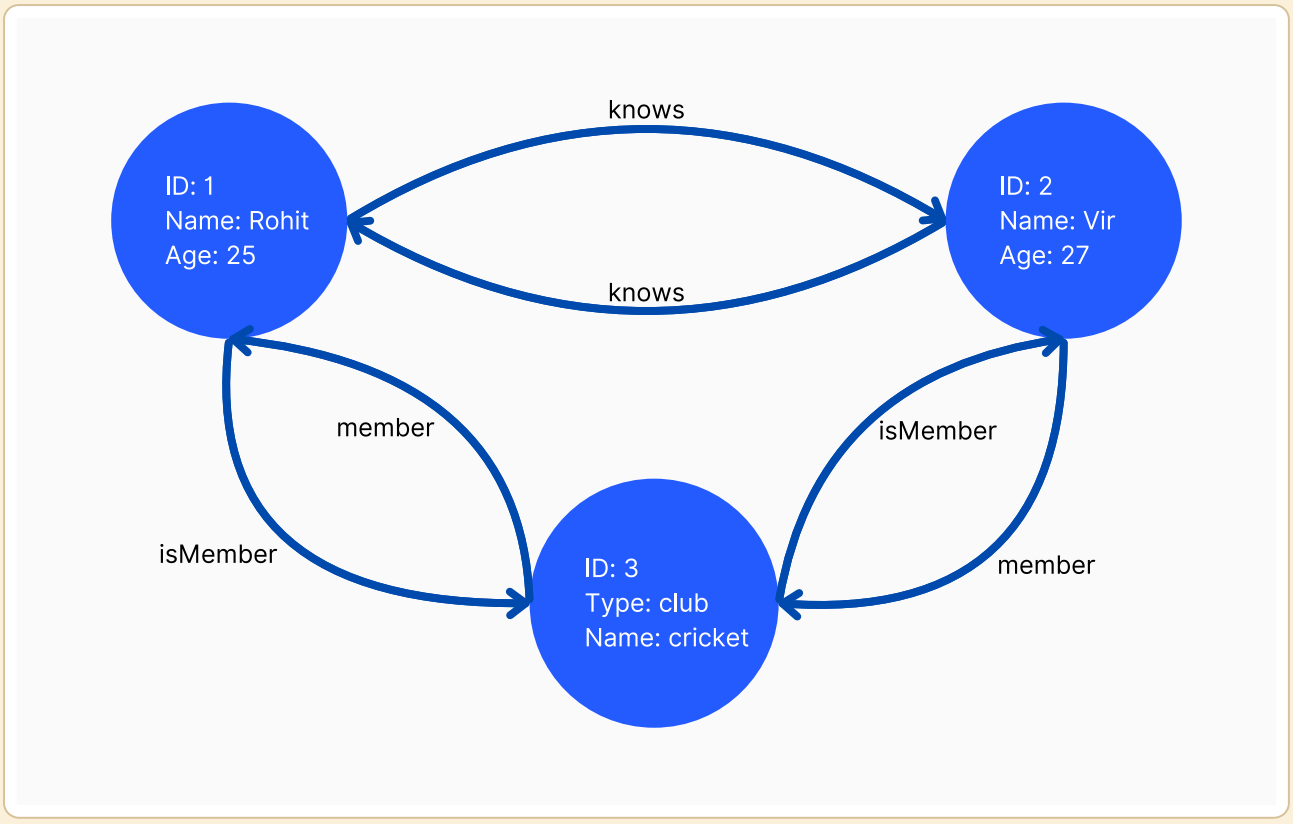


Figure — Two people connected by `knows` edges, each a member of a cricket club (`isMember`) (course diagram).

Choose a graph database when the **relationships matter as much as, or more than, the records**: social connections, recommendations, fraud rings, network topology, knowledge graphs, access-control hierarchies.

Relational vs graph for the same data

Relational: a `person` table plus a `knows(src, dst)` link table. For friendships, store both directions or query both columns:

src	dst
1 (Ava)	2 (Bilal)
1 (Ava)	3 (Chen)
2 (Bilal)	4 (Dana)
…	…

Finding one person's friends means looking up the person's id, scanning or index-seeking the link table for matching `src`, and joining back to `person` for names. Each extra hop ("friends of friends of friends") adds **another join**. With B-tree lookups, each hop costs O(log n) per edge, and intermediate results can explode.

Graph: each node stores direct pointers to its edges (Neo4j calls this **index-free adjacency**). Finding friends means going to the node and following its `KNOWS` edges. The cost depends on the number of edges traversed, **not the total size of the database**, so deep, variable-length traversals stay fast.

```
// Neo4j Cypher (not run)
CREATE (a:Person {name:'Ava'})-[:KNOWS]->(b:Person {name:'Bilal'});
MATCH (me:Person {name:'Ava'})-[:KNOWS]->(f)-[:KNOWS]->(fof)
WHERE NOT (me)-[:KNOWS]->(fof) AND fof <> me
RETURN DISTINCT fof.name;                                  // friend suggestions
MATCH p = shortestPath((:Person {name:'Ava'})-[:KNOWS*]-(:Person {name:'Goran'}))
RETURN [n IN nodes(p) | n.name];
```

Advantages: relationship queries are **simpler to write** (patterns instead of join chains), **fast at depth** ("who are Chen's friends?" or six hops away), and the model is **flexible** (add new node or edge types freely).

Query languages: **Cypher/openCypher** (Neo4j, Neptune, Memgraph), **Gremlin** (Apache TinkerPop), **SPARQL** (RDF triple stores), and the new ISO standard **GQL** (ISO/IEC 39075, 2024). SQL:2023 also added **SQL/PGQ** for property-graph queries over relational tables.

Use cases

1. **Social networks**: mutual friends, degrees of separation, "people you may know".
2. **Recommendations**: nodes for users, products and categories, and edges for bought, liked and similar-to. "Customers who bought this also bought…" becomes a two-hop traversal.
3. **Fraud detection**: accounts sharing devices, addresses or cards form rings that are easy to spot as **connected components** (groups of nodes all reachable from one another).
4. **Network/IT dependency maps and knowledge graphs.**

Scaling limits

- **Horizontal sharding is hard**: cutting a highly connected graph across machines means traversals cross the network on almost every hop. Many graph databases scale reads with replicas but keep the graph on one machine (or require careful partitioning).
- **Bulk updates** of every node matching a property are slower than in column or document stores.
- **Aggregations over everything** (totals, reports) are not their strength. Export to a warehouse for that.

Graph queries in Postgres

`code/03_graph_in_sql.sql` models seven people and symmetric `knows` edges, then walks the graph with recursive **CTEs** (common table expressions — named sub-queries in a `WITH` clause; a recursive one repeatedly feeds its output back in) (Chapter 7):

```sql
-- 2 hops: friend-of-friend suggestions for Ava
SELECT DISTINCT p.name AS suggestion
FROM knows k1 JOIN knows k2 ON k2.src = k1.dst JOIN person p ON p.id = k2.dst
WHERE k1.src = 1 AND k2.dst <> 1
  AND k2.dst NOT IN (SELECT dst FROM knows WHERE src = 1);

-- N hops: degrees of separation (path array prevents cycles)
WITH RECURSIVE walk(id, depth, path) AS (
  SELECT 1, 0, ARRAY[1]
  UNION ALL
  SELECT k.dst, w.depth + 1, w.path || k.dst
  FROM walk w JOIN knows k ON k.src = w.id
  WHERE k.dst <> ALL (w.path) AND w.depth < 6
)
SELECT p.name, min(depth) AS degrees
FROM walk JOIN person p USING (id) GROUP BY p.name ORDER BY degrees, p.name;
```

Output (Ava's friends are Bilal and Chen, the suggestion, degrees, and the shortest path to Goran):

```
 suggestion
------------
 Dana

  name  | degrees
--------+---------
 Ava    |       0
 Bilal  |       1
 Chen   |       1
 Dana   |       2
 Eli    |       3
 Fatima |       4
 Goran  |       5

                 shortest_path
-----------------------------------------------
 Ava -> Bilal -> Dana -> Eli -> Fatima -> Goran
```

This works well for small graphs and shallow depths. Recursive CTEs enumerate **every path**, so on large, dense graphs the work grows exponentially with depth. That's the point where a graph engine (or the Apache AGE extension, which adds Cypher to Postgres) pays off.

In the interview

30-second answer: "A graph database stores nodes, edges and properties, and follows edges directly, so multi-hop queries cost in proportion to the edges touched rather than the table sizes. It's ideal for social graphs, recommendations and fraud rings. It's harder to shard and weaker at bulk aggregates. For a few hops on modest data, SQL with a link table and recursive CTEs is fine."

- *Why are deep joins slow in SQL?* Each hop is another join over the whole link table (through an index), and intermediate results multiply.
- *How would you detect a fraud ring?* Link accounts to shared devices, cards and addresses, then find connected components or short cycles.

Self-check

1. *What's index-free adjacency?* Each node points directly to its neighbours, so traversal needs no index lookup.
2. *Why store friendships in both directions in the SQL model?* So a single `src = ?` lookup finds all friends of a person.

3. *Name two graph query languages.* Cypher and Gremlin (also SPARQL and GQL).

TL;DR

Graphs make relationships first-class. Use one when the questions are "how is X connected to Y". Recursive CTEs cover the small cases.

16.6 In-memory databases

What it is

An **in-memory database (IMDB)** keeps its primary copy of the data in **RAM** rather than on disk. RAM access takes about 100 **nanoseconds** (billionths of a second), while an SSD read takes tens to hundreds of **microseconds** (millionths of a second). The engine can use data structures designed for memory (**hash tables** — arrays indexed by a key's hash for near-instant lookup; **skip lists** — sorted linked lists with "express lanes" for fast search; compact column vectors) instead of disk pages and buffer management.

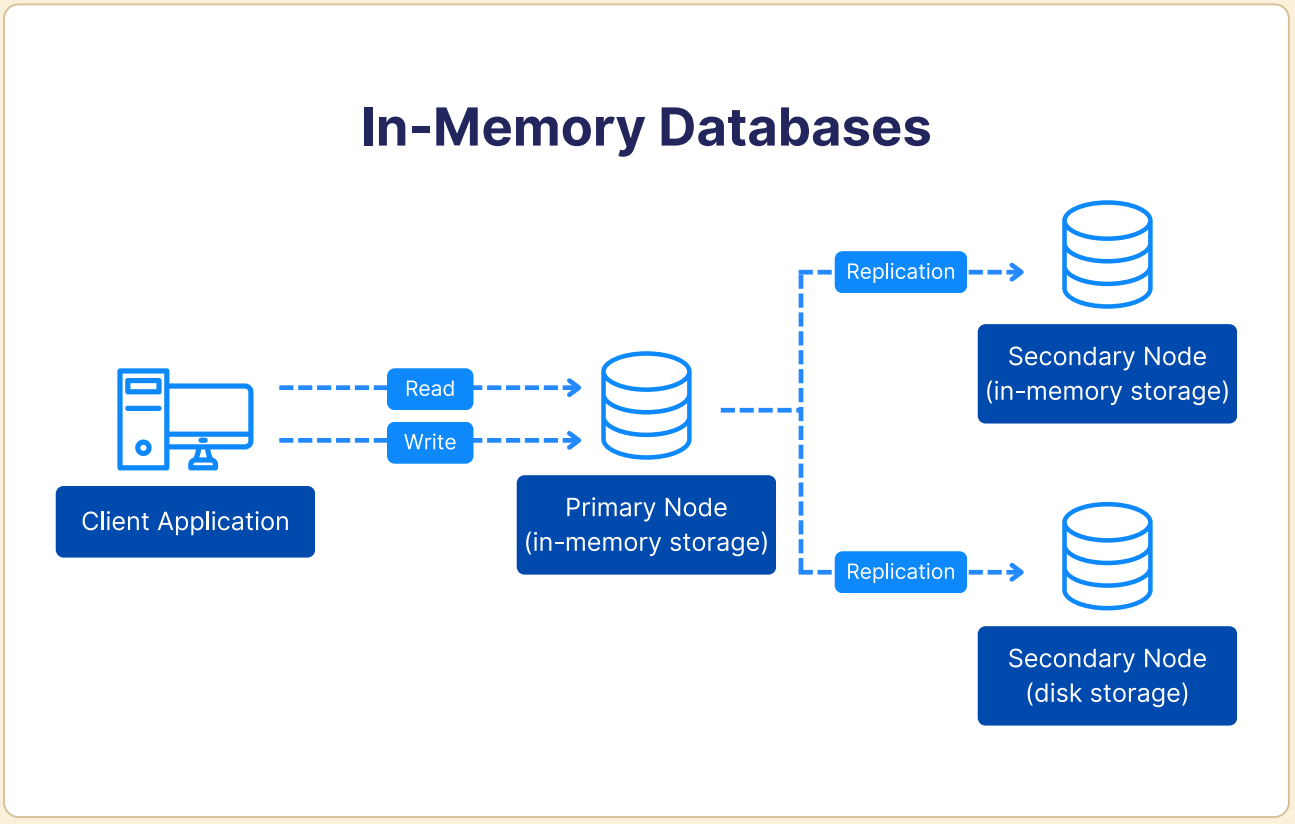


Figure — The client reads and writes the primary node's in-memory store, and data is replicated to secondaries, some of which persist to disk (course diagram).

Architecture

1. **Primary node**: holds the working data in memory and serves reads and writes.
2. **Replication**: changes stream to **secondary nodes** (in memory, or disk-backed) for availability.
3. **Persistence**: because RAM is **volatile** (lost on power-off or crash), IMDBs write **snapshots** and/or an **append-only / write-ahead log** to disk. On restart they reload the snapshot and replay the log. This is the same WAL idea as in 15.9.

Benefits

Benefit	Example
Speed	A user's recent purchase history in well under a millisecond
Real-time processing	Live game scores and leaderboards, fraud checks during card authorisation, IoT alerts
Simpler, memory-optimised structures	Redis's native hashes and sorted sets
High concurrency	Hundreds of thousands of operations per second per node

Limitations

- **Volatility**: without persistence you lose data on restart, and with it you add **I/O** (input/output — reading and writing disk or network) and recovery time.
- **Cost**: RAM costs far more per GB than disk, so large datasets are expensive.
- **Scaling**: adding capacity means adding RAM on more nodes, and data size is bounded by total cluster memory.

Real systems

- **SAP HANA**: an in-memory, **column-oriented** database that runs **OLTP and OLAP on the same data** (OLTP = online transaction processing, many small reads/writes like placing orders; OLAP = online analytical processing, big scans and aggregates for reports) (sometimes called HTAP, hybrid transactional/analytical processing). *Example:* a retailer analyses today's sales in real time and adjusts stock orders the same afternoon.
- **SingleStore** (called **MemSQL** until 2020): distributed SQL with in-memory row storage plus disk-based columnstore tables, and MySQL-wire-compatible (speaks MySQL's network protocol, so MySQL drivers work) **ANSI SQL** (the standard SQL language). *Example:* a bank ingests millions of card transactions a day and scores each one for fraud in real time.
- **Redis** (16.2), **Memcached** (a cache only, no persistence), and **VoltDB** (in-memory NewSQL).

Postgres note: Postgres is not an IMDB, but with enough `shared_buffers` (Postgres's own in-memory page cache) and OS cache, its hot data is served from memory anyway. `UNLOGGED` tables (16.2) skip the WAL for cache-like data. The on-disk WAL is what lets it survive a crash.

In the interview

> **30-second answer:** "In-memory databases keep data in RAM for microsecond access and use snapshots plus an append-only log, plus replicas, for durability. They're ideal for caches, sessions, leaderboards and real-time analytics. The trade-offs are cost per GB and dataset size limited by memory."

- *How does Redis survive a restart?* RDB snapshots and/or AOF (append-only file) replay, and replicas.

Self-check

1. *What makes RAM data durable?* Snapshots and logs on disk, plus replication.
2. *What's HTAP?* Running transactional and analytical workloads on the same live data (SAP HANA, SingleStore).

TL;DR

RAM-first storage for speed. Durability comes from logs, snapshots and replicas. The price is cost and a size ceiling.

16.7 NewSQL / distributed SQL

What it is

NewSQL (today usually called **distributed SQL**) databases keep the **relational model, SQL and ACID transactions** but **scale out horizontally** like NoSQL, with automatic sharding, replication and failover.

The gap they fill

- **Traditional SQL** (MySQL, PostgreSQL): strong consistency and ACID, but designed around one primary node. Scaling writes means scaling up or sharding by hand, and cross-shard transactions are your problem.
- **NoSQL** (MongoDB, Cassandra): scale-out and flexibility, but historically weaker guarantees (eventual consistency, limited transactions). That's risky for balances, inventory and bookings.

NewSQL aims to offer **both**: SQL and ACID *at* horizontal scale.

How it works

1. **Automatic sharding.** Tables are split into ranges (CockroachDB calls them *ranges*, TiDB *regions*, Spanner *splits*) that move between nodes automatically as they grow or get hot. *Example:* a ride-hailing app's trips spread across nodes, so one busy city doesn't slow the others.
2. **Consensus-based replication.** Each shard is a **Raft** (CockroachDB, TiDB, YugabyteDB) or **Paxos** (Spanner) group of usually 3 or 5 replicas. A write commits once a majority stores it, so the loss of a minority loses nothing and needs no manual failover (15.8).
3. **Distributed transactions.** Transactions spanning shards use **2PC whose coordinator state is itself replicated by consensus**, which avoids 2PC's blocking problem (15.9). They provide **serializable** isolation (concurrent transactions behave as if run one after another), the default in CockroachDB and Spanner.
4. **Clock handling for global consistency.** **Google Spanner** uses **TrueTime**, GPS- and atomic-clock-backed time with a known uncertainty bound. It waits out that uncertainty before acknowledging a commit ("commit wait"), which gives externally consistent (**linearizable** — every reader sees commits in real-time order) transactions worldwide. CockroachDB uses **hybrid logical clocks** (timestamps combining the machine's physical clock with a counter that keeps causally related events ordered) on ordinary servers, with an uncertainty interval and retries.
5. **SQL front end**: familiar SQL, and often a wire-compatible protocol (CockroachDB and YugabyteDB speak PostgreSQL's protocol, TiDB speaks MySQL's).

Examples

System	Notes
Google Spanner (paper 2012, Cloud Spanner service 2017)	Global, Paxos + TrueTime, strict serializability (serializable + linearizable). *Use:* a global shop keeping stock counts correct across regions during a flash sale.
CockroachDB	Postgres-compatible, Raft per range, serializable by default, survives zone or region loss. *Use:* a fintech processing payments across several data centres.
TiDB	MySQL-compatible, TiKV storage (Raft), with an optional columnar replica (TiFlash) for HTAP.
YugabyteDB	Reuses PostgreSQL's query layer on a Raft-replicated distributed store.
VoltDB	In-memory, partitioned, serial execution per partition.

Where it fits: global apps that need correct transactions (payments, inventory, bookings), multi-region active-active with strong consistency, and real-time queries over distributed data. The costs are higher write latency (consensus round trips, and cross-region round trips for global writes), more complex operations, and a SQL dialect that is compatible but not identical.

SQL vs NoSQL vs NewSQL

Feature	Traditional SQL	NoSQL	NewSQL
Data model	Relational tables	Key-value, document, wide-column, graph	Relational tables
Schema	Defined up front (JSON columns add flexibility)	Dynamic	Defined up front, with JSON support
Scaling	Vertical + read replicas, manual sharding	Horizontal, built in	Horizontal, built in
Consistency	Strong, ACID	Often eventual/tunable (BASE)	Strong, ACID (serializable)
Typical use	ERP (enterprise resource planning: finance, stock, HR systems), banking, most business apps	Caches, feeds, IoT, catalogues	Global transactional apps, high-scale OLTP
Examples	PostgreSQL, MySQL, SQL Server	Redis, MongoDB, Cassandra, Neo4j	Spanner, CockroachDB, TiDB, YugabyteDB

In the interview

30-second answer: "NewSQL keeps SQL and ACID but scales out. Data is auto-sharded into ranges, each replicated with Raft or Paxos, and cross-shard transactions use 2PC on top of those consensus groups, so there's no single blocking coordinator. Spanner uses TrueTime for global external consistency, and CockroachDB uses hybrid logical clocks. The cost is write latency from consensus."

- *Why is a global write slower in Spanner/CockroachDB?* A majority of replicas, possibly in other regions, must acknowledge it, and Spanner adds the commit-wait.

Self-check

1. *Which consensus protocol does CockroachDB use?* Raft.
2. *What problem does TrueTime solve?* Ordering transactions globally without a central clock, by waiting out a bounded clock uncertainty.

TL;DR

NewSQL = relational + ACID + automatic scale-out through consensus-replicated shards. You pay for it with latency.

16.8 Time-series and spatial databases

Time-series databases

What it is

A **time-series database (TSDB)** is optimised for **sequences of timestamped measurements**: metrics, sensor readings, prices. Data arrives in time order, is mostly appended and rarely updated, and is queried by **time range**, often **downsampled** (averages per minute or hour).

How they index and store temporal data

- **Time-partitioned storage**: data lives in chunks per hour, day or week, so "last 7 days" touches only 7 chunks, and dropping old data drops whole chunks. *Example:* smart-home sensor readings stored one partition per day.
- **B-trees and LSM trees**: B-trees serve range scans. **LSM trees** make high-rate appends cheap (InfluxDB's engine, Cassandra and others; see 16.4).

- **Compression**: consecutive timestamps and values are similar, so encodings like **delta** (store the difference from the previous value), delta-of-delta and XOR floats (Facebook's **Gorilla** paper, 2015) shrink data by around 10×. Columnar layouts help too.

Features

Feature	Meaning
Write optimisation	Millions of points per second
Retention policies	Delete (or downsample) data older than N days automatically
Continuous aggregates / downsampling	Keep per-minute data for a week and per-hour data for a year
Time functions	Buckets, gap filling, interpolation, "last value"

Applications

- **IoT**: temperature, humidity and power usage in homes and factories.
- **Server monitoring**: CPU, memory and network metrics, as in **Prometheus** (its own TSDB) and the Datadog platform.
- **Finance**: tick-by-tick prices and volumes for trend analysis.

Products: **InfluxDB**, **TimescaleDB** (a PostgreSQL extension with automatic time partitions called *hypertables*, compression and continuous aggregates), **Prometheus**, **QuestDB**, kdb+.

Time-series in plain Postgres

The rest of `code/04_time_series_and_wide_rows.sql` shows the TSDB toolkit using built-in features:

```
-- Downsample: 6-hour buckets per server (date_bin: PostgreSQL 14+)
SELECT server_id, date_bin('6 hours', ts, timestamptz '2024-12-06') AS bucket,
       round(avg(cpu_pct)::numeric, 1) AS avg_cpu, max(mem_mb) AS peak_mem
FROM metrics WHERE ts >= '2024-12-06'
GROUP BY 1, 2 ORDER BY 1, 2 LIMIT 6;
```

```
 server_id |         bucket         | avg_cpu | peak_mem
-----------+------------------------+---------+----------
 db-1      | 2024-12-06 00:00:00+00 |    42.0 |     1923
 db-1      | 2024-12-06 06:00:00+00 |    54.0 |     1915
 db-1      | 2024-12-06 12:00:00+00 |    66.0 |     1922
 db-1      | 2024-12-06 18:00:00+00 |    78.0 |     1923
 web-1     | 2024-12-06 00:00:00+00 |    22.0 |     1923
 web-1     | 2024-12-06 06:00:00+00 |    34.0 |     1917
```

```
CREATE INDEX metrics_ts_brin ON metrics USING brin (ts);
-- index sizes on one day's partition (4,320 rows)
```

```
          index           |  size
--------------------------+--------
 metrics_2024_12_06_pkey   | 152 kB
 metrics_2024_12_06_ts_idx | 24 kB
```

A **BRIN** index (block range index, Chapter 12) stores only min/max per block range. That's tiny, and it's ideal when rows arrive in time order. (24 kB is BRIN's minimum footprint. On millions of rows it stays small while the B-tree grows in proportion to the rows.)

```
-- Retention policy: dropping a day is instant (no giant DELETE, no vacuum debt).
DROP TABLE metrics_2024_12_05;
SELECT min(ts)::date AS oldest_day_kept, count(*) AS readings FROM metrics;
```

```
 oldest_day_kept | readings
-----------------+----------
 2024-12-06      |     4320
```

Spatial databases

> **What it is**

A **spatial database** stores and queries **geometry and geography**: where things are and how shapes relate.

Type	Represents	Example
Point	A location	A shop's GPS coordinates
Line (LineString)	A path	A road, a delivery route
Polygon	An area	A city boundary, a delivery zone

Spatial indexing

Ordinary B-trees order values along one dimension and can't answer "near this point" in 2-D. Spatial indexes can:

- **R-tree**: groups nearby objects into nested **bounding rectangles**. A query descends only into the rectangles that overlap the search area. *Example:* "stores within 10 km" checks a few boxes instead of every store. PostgreSQL's **GiST** index (generalised search tree — a framework for tree indexes over non-sortable data) implements R-tree-style indexing for geometric types and PostGIS.
- **Quadtree**: recursively splits 2-D space into **four quadrants** until each cell holds few points. It is good for points and uneven density (Postgres's SP-GiST supports quadtrees).
- **Geohash**: encodes latitude/longitude as a short **string** whose shared prefix means nearby. That lets spatial lookups run on a plain key-value store or B-tree (e.g. `u09tv` is a cell covering central Paris). Edge cases at cell borders mean you also check neighbouring cells. Related grid systems are Uber's **H3** (hexagons) and Google's **S2**.

Spatial queries

- **Proximity**: "restaurants within 5 km", "the 3 nearest drivers" (k-nearest-neighbour, KNN).
- **Intersection / containment**: "which delivery zones overlap?", "is this address inside the service area?"
- **Routing**: "shortest route from A to B" (pgRouting, graph algorithms over road networks).

Applications

- **Location-based services**: nearby restaurants and fuel stations, as in map apps.
- **Smart cities and logistics**: traffic, bus routes, rubbish-collection routes.
- **Geo-analytics**: where customers cluster, and where to open the next store.

Products: **PostGIS** (the standard geospatial extension for PostgreSQL), **BigQuery GIS**, Oracle Spatial, SQL Server spatial types, MongoDB 2dsphere indexes, Redis GEO commands.

Spatial queries with Postgres built-ins

PostGIS isn't installed on the book server, so `code/05_spatial_builtin.sql` uses Postgres's **built-in geometric types** (flat x/y plane, units here are km) with a **GiST** index:

```
CREATE TABLE store (id int PRIMARY KEY, name text, loc point NOT NULL);
-- 6 named stores + 5,000 kiosks scattered on the grid
CREATE INDEX store_loc_gist ON store USING gist (loc);    -- R-tree-like index

SELECT name, round((loc <-> point(2, 2))::numeric, 2) AS km          -- <-> = distance
FROM store WHERE id < 100 ORDER BY loc <-> point(2, 2) LIMIT 3;
SELECT name FROM store WHERE id < 100 AND loc <@ circle(point(0, 0), 5) ORDER BY name;   -- radius
SELECT name FROM store WHERE id < 100 AND loc <@ box(point(2, 3), point(7, 9)) ORDER BY name; -- zone
EXPLAIN (COSTS OFF) SELECT id FROM store ORDER BY loc <-> point(2, 2) LIMIT 3;
```

```
   name    |  km
-----------+------
 Riverside | 2.24
 Central   | 2.83
 Mall      | 3.16

   name       (within 5 km of (0,0))
-----------
 Central
 Mall
 Old Town
 Riverside

   name       (inside the delivery-zone box)
-----------
 Harbour
 Riverside

 Limit
   ->  Index Scan using store_loc_gist on store
         Order By: (loc <-> '(2,2)'::point)
```

The last plan is **KNN search through the GiST index**: the index returns points in distance order, so `LIMIT 3` stops after three, with no full sort.

For real latitude/longitude on the curved Earth, use PostGIS (`4326` below is the id of **WGS 84**, the coordinate system GPS uses):

```
-- PostGIS (not run)
CREATE EXTENSION postgis;
CREATE TABLE shop (id int PRIMARY KEY, name text, geom geography(Point, 4326));  -- WGS 84 lon/lat
CREATE INDEX ON shop USING gist (geom);
SELECT name FROM shop
WHERE ST_DWithin(geom, ST_MakePoint(2.3522, 48.8566)::geography, 2000);        -- within 2 km of Paris
centre
SELECT z.name FROM zone z JOIN shop s ON ST_Intersects(z.area, s.geom);
```

Time-series vs spatial

Feature	Time-series	Spatial
Data focus	Timestamped measurements	Coordinates, lines, areas
Indexing	Time partitions/chunks, B-tree/LSM, BRIN, compression	R-tree (GiST), quadtree, geohash/H3/S2
Typical query	"Average CPU per hour, last 24 h"	"Shops within 10 km", "which zone contains this point"
Applications	IoT, monitoring, finance	Maps, logistics, smart cities

In the interview

> **30-second answer:** "Time-series stores partition by time, append with LSM or B-trees, compress heavily and apply retention and downsampling. In Postgres that's time partitions, BRIN, `date_bin`, or TimescaleDB. Spatial stores use R-trees, quadtrees or geohashes for near/within/intersects queries. In Postgres that's GiST, with PostGIS for real geography."

- *Design "drivers near me" at scale?* Put driver locations in a geohash/H3 cell index (Redis GEO or an in-memory grid) updated every few seconds, query the neighbouring cells, then rank by exact distance.

Self-check

1. *Why is BRIN a good fit for time-series?* Rows arrive in time order, so per-block min/max summaries are selective and tiny.
2. *What does an R-tree store in internal nodes?* Bounding rectangles of its children.
3. *Why check neighbouring geohash cells?* Two nearby points can sit on opposite sides of a cell border.

TL;DR

Time-series: partition by time, compress, retain, downsample. Spatial: R-tree/GiST, quadtrees and geohashes for near/within/intersects. PostGIS and TimescaleDB bring both to Postgres.

16.9 Big data and distributed processing frameworks

What "big data" means

Big data is data too large, too fast or too varied for one traditional database server to store and process in reasonable time. The classic **3 Vs**:

- **Volume**: terabytes to petabytes (1 TB = 1,000 GB; 1 PB = 1,000 TB) and beyond.
- **Velocity**: data arrives continuously and fast (**clickstreams** — logs of every user click — and sensors) and must be processed quickly.
- **Variety**: structured tables, semi-structured JSON and logs, unstructured text, images and video.

(Veracity and value are often added as extra Vs.) The challenges are **storage** at that size, **processing** in acceptable time, and **scaling** as it grows. The answer is the same as in Chapter 15: **spread storage and computation over many machines and run in parallel**, moving computation to where the data is.

Hadoop: HDFS + MapReduce

- **HDFS (Hadoop Distributed File System)** splits files into large **blocks** (128 MB by default) spread across nodes, with each block **replicated** (3 copies by default) for fault tolerance.
- **MapReduce** is a programming model with a **map** phase (each node transforms its local blocks into key/value pairs), a **shuffle** (pairs are grouped by key across the network), and a **reduce** phase (each group is combined into a result). Intermediate results are written to disk between stages, which is robust but slow for multi-step jobs.

Example: a retailer stores years of sales files in HDFS and runs MapReduce jobs to total sales per product per month.

`code/mapreduce_word_count.py` runs the phases in miniature over three "nodes" of order-event logs, including a **combiner** that pre-aggregates on each node before the shuffle:

```python
def map_phase(lines):                # runs locally on each node
    for line in lines:
        action, item = line.split()
        yield (f"{action}:{item}", 1)
# ... combine per node, then send each key to reducer  hash(key) % 2 ...
```

```
map+combine on node-1: [('buy:laptop', 1), ('view:laptop', 2), ('view:tshirt', 1)]
map+combine on node-2: [('buy:book', 1), ('view:book', 1), ('view:laptop', 1)]
map+combine on node-3: [('buy:laptop', 1), ('view:laptop', 1), ('view:tshirt', 1)]
reducer 0: {'buy:laptop': 2, 'view:book': 1}
reducer 1: {'buy:book': 1, 'view:laptop': 4, 'view:tshirt': 2}
```

Each key ends up at exactly one reducer, which is why the totals are complete. It's the same as `SELECT action, item, count(*) … GROUP BY action, item`, but executed across machines.

Apache Spark

Spark is a general-purpose distributed engine built to fix MapReduce's slowness:

- **In-memory processing**: intermediate results stay in RAM from one step to the next (originally as **RDDs**, resilient distributed datasets that can be recomputed from their **lineage** (the recorded chain of steps that produced them) if a node fails, and today mostly as **DataFrames** — distributed tables with named columns). That makes iterative jobs such as machine learning and multi-stage queries much faster.
- **Many workloads in one engine**: batch processing, **Structured Streaming**, machine learning (MLlib) and graph processing (GraphX).
- **APIs** in Python (PySpark), Scala, Java, R, and SQL.

Example: a social platform uses Spark to crunch behaviour logs into features for personalised recommendations.

SQL-on-Hadoop: SQL over big data

Tool	What it is
Apache Hive	A **data warehouse** (a database built for analytics over large historical data) over HDFS. HiveQL queries are compiled into distributed jobs (originally MapReduce, later Tez or Spark). Good for large batch queries on structured files.
Presto / Trino	Distributed SQL query engines (Presto from Facebook, 2012, and Trino is its fork) for **fast, interactive** queries that can **federate** (query several separate systems as if one) many sources: HDFS/S3 (Amazon's object storage) files, MySQL, Postgres, Kafka…
Spark SQL	SQL and DataFrames on Spark, sharing one optimiser with the other Spark APIs

Example: a retailer uses Trino to join in-store sales (in Postgres) with online clickstream data (**Parquet** — a compressed, column-oriented file format — on S3) in one query.

Real-time: Kafka and Flink

- **Apache Kafka** is a distributed, replicated, append-only **log**, used as a **message broker** (middleware that passes messages from senders to receivers). **Producers** append events to **topics** (named streams, split into partitions), and **consumers** read at their own pace and can replay. *Example:* a logistics company streams GPS pings from delivery vans through Kafka to live dashboards.
- **Apache Flink** is a **stream processor**: low-latency, **event-driven**, **stateful** computations (windows, joins, pattern detection) with **event-time** handling (using when an event actually happened, not when it arrived) and **exactly-once** state (each event affects results once, even after failures and retries) through distributed checkpoints (15.9). *Example:* a trading platform flags unusual trade patterns within milliseconds.

A common architecture streams events into Kafka, processes them in Flink or Spark Streaming for real-time views, and lands them in a **data lake** (cheap storage of raw files in any format, usually on object storage) or warehouse for **batch** analysis (processing large accumulated chunks at once) with Spark, Hive or Trino. **Change data capture (CDC)** — reading the database's change log and publishing every insert/update/delete as an event — from Postgres into Kafka (for example with Debezium, an open-source CDC tool) feeds the same pipeline from your OLTP database (Batch and streaming patterns).

Applications

Area	Example
IoT / sensors	Smart factories analyse machine telemetry with Spark to predict failures
Recommendations	Streaming services update suggestions from live viewing events with Flink
Fraud detection	Banks score transactions from Kafka streams with Spark/Flink in real time
Data integration	Retailers combine store and web sales with Presto/Trino

In the interview

30-second answer: "Big data is volume, velocity and variety beyond one server. Hadoop stores blocks replicated in HDFS and processes them with MapReduce (map, shuffle, reduce, via disk). Spark keeps data in memory and covers batch, streaming, ML and SQL. Hive, Trino and Spark SQL put SQL on top. For real time, Kafka is the durable event log and Flink does stateful, exactly-once stream processing."

- *Why is Spark faster than MapReduce for iterative jobs?* It keeps intermediate data in memory instead of writing it to HDFS after every step.
- *What is a combiner?* A local reduce before the shuffle that shrinks network traffic. It only works for associative operations like sum and count.

Self-check

1. *What are the 3 Vs?* Volume, velocity, variety.
2. *What does the shuffle guarantee?* All values for a key reach the same reducer.
3. *Kafka vs Flink?* Kafka stores and transports event streams, and Flink computes on them.

TL;DR

Split data into replicated blocks, move code to data, map → shuffle → reduce. Spark does it in memory. SQL engines sit on top. Kafka + Flink handle real time.

16.10 Choosing a database

Start from the **access patterns and guarantees you need**, not from fashion. For most new applications the right first answer is still **PostgreSQL**, adding JSONB, partitioning, PostGIS or TimescaleDB as needed. Add a specialised store when a requirement clearly exceeds what it does well.

If you need…	Consider	Why	Postgres can go far with
General business data, joins, constraints, transactions	**PostgreSQL / MySQL**	ACID, SQL, mature tooling	— (this is its home)
Sub-millisecond cache, sessions, counters, rate limits, leaderboards	**Redis** (Memcached for a pure cache)	In memory, TTL, data structures	`UNLOGGED` tables for moderate load
Flexible, nested, per-item attributes, read as whole objects	**MongoDB**, Firestore, CouchDB (offline sync)	Document model, easy scale-out	`jsonb` + GIN + CHECK
Massive write throughput, time-ordered per key, multi-region always-on	**Cassandra / ScyllaDB**, HBase/Bigtable	LSM writes, partitioned wide rows, tunable consistency	Partitioning + BRIN up to a point
Deep relationship queries (paths, communities, fraud rings)	**Neo4j**, Neptune	Traversals cost per edge touched	Recursive CTEs for shallow/small graphs
Metrics and IoT with retention and downsampling	**TimescaleDB**, InfluxDB, Prometheus	Time chunks, compression, continuous aggregates	Range partitions + `date_bin` + BRIN
Maps, "near me", zones	**PostGIS**, BigQuery GIS, Redis GEO	Spatial types + R-tree indexes	Built-in `point` / `box` + GiST (planar only)
SQL + ACID beyond one node, multi-region strong consistency	**CockroachDB, Spanner, YugabyteDB, TiDB**	Consensus-replicated shards	Citus for scale-out Postgres
Ad-hoc analytics over billions of rows	**Columnar warehouse**: BigQuery, Snowflake, Redshift, ClickHouse	Column storage, vectorised scans (processing batches of values per CPU instruction)	Read replicas and summary tables
Petabyte batch and stream processing	**Spark, Trino, Kafka + Flink**	Distributed compute over a data lake	CDC out of Postgres into these
Full-text search with relevance ranking and facets	**Elasticsearch / OpenSearch**	Inverted indexes, scoring	`tsvector` + GIN full-text search (Chapter 14)

Questions to ask:

1. What are the top queries, and do they need joins, ranges, aggregations or traversals?
2. What consistency does each piece of data need (15.6)? Money needs strong consistency, and like counts can be eventual.
3. What's the read/write ratio and the peak throughput? How big will the data be in 3 years?
4. Single region or multi-region? What RPO/RTO (15.10)?
5. What can the team operate? Each extra database adds backups, monitoring, upgrades and **on-call** load (engineers on rota to respond to outages).

Polyglot persistence. Large systems commonly use several stores, each for what it does best. For example: Postgres for orders and payments, Redis for sessions and caching, Elasticsearch for product search, Kafka + a warehouse for analytics, and all of it fed by CDC from the primary database so there's **one source of truth**. See the database-design case studies in Chapter 17.

In the interview

> **30-second answer:** "I start from access patterns and consistency needs. Relational Postgres is the default for transactional data. I add Redis for caching and sessions, a document store if data is truly schemaless and read as whole objects, Cassandra for huge write-heavy time-ordered data, a graph DB for deep relationship queries, a TSDB for metrics, NewSQL if I need SQL + ACID across regions, and a columnar warehouse for analytics. Every extra store is operational cost, so each must earn its place."

Self-check

1. *Where should like counts vs account balances live, consistency-wise?* Like counts can be eventually consistent. Balances need strong consistency and ACID.
2. *What keeps several stores in sync in polyglot persistence?* One source of truth plus CDC/event streams feeding the others.

16.11 Inside two workhorses: Elasticsearch (and the ELK stack) and Redis

Two non-relational systems sit next to almost every production relational database: a **search engine** for text search and log analytics, and **Redis** for caching and fast data structures. §16.2 and §16.10 said *when* to use them; this section opens them up so you can explain *how* they work and where they fail.

Search engines: what it is

Elasticsearch (and its open-source **fork** — an independent copy of a project's code that is developed separately — **OpenSearch**, created by AWS in 2021 after Elastic changed its licence) is a distributed search and analytics engine built on the **Apache Lucene** library. You send it JSON documents; it makes every field searchable within about a second and answers relevance-ranked full-text queries, filters and aggregations over billions of documents. Solr is an older Lucene-based alternative.

The problem it solves

`WHERE description LIKE '%running shoe%'` cannot use a B-tree index (the pattern starts with `%`), so it scans every row, does not match "shoes" or "Running", and returns rows in no useful order. Users expect typo tolerance, **stemming** (reducing words to a root, so "running" finds "run"), **relevance ranking** (best matches first) and **facets** (per-category match counts such as "Brand: Nike (42)"). PostgreSQL's `tsvector` + GIN (14.6) covers the basics inside the database; a dedicated engine takes over when search is a core feature or the data is huge.

How it works — the inverted index

A B-tree maps *row → values*; an **inverted index** maps each **term → the list of documents containing it** (the **posting list**), like the index at the back of a book.

1. **Analysis.** At index time an **analyzer** turns text into terms: *character filters* (strip HTML), a *tokenizer* (split on word boundaries into **tokens**, i.e. individual words), then *token filters* (lowercase, remove **stop words** — very common words that carry little meaning — such as "the", **stem** "running" → "run", add synonyms). The same analyzer runs on the query, so "Running SHOES" and "running shoe" meet on the same terms. A `text` field is analysed; a `keyword` field is stored whole for exact filters, sorting and aggregations.
2. **Matching.** A query for `run AND trail` intersects two sorted posting lists — fast even for millions of entries.
3. **Scoring.** Matches are ranked with **BM25** (the default since Elasticsearch 5): a term counts more if it is frequent in *this* document (term frequency, with diminishing returns), rare across *all* documents (inverse document frequency), and the document is short (length normalisation).

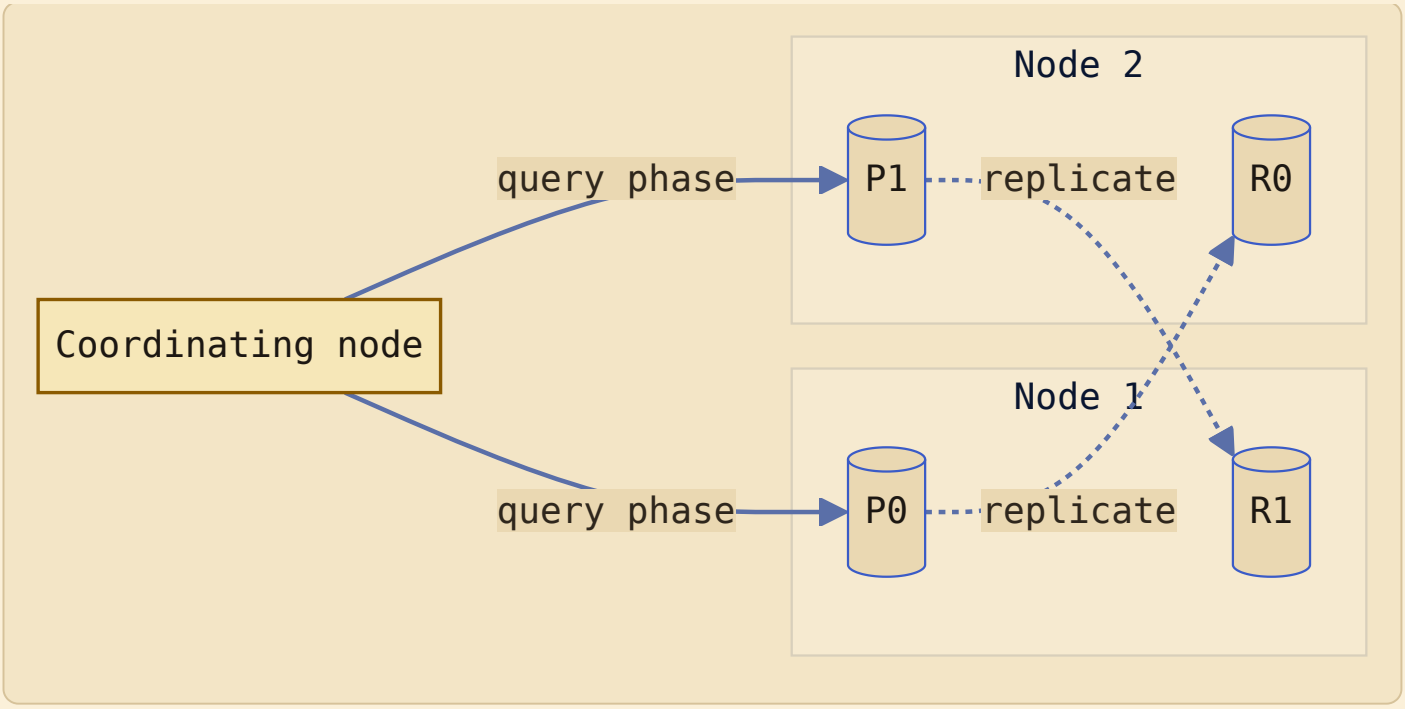
`code/inverted_index.py` builds all three in miniature, splits five products over two shards and merges the per-shard results:

```
analyze('Running shoes for the TRAIL') -> ['run', 'shoe', 'trail']
shard 0: docs [2, 4]
shard 1: docs [1, 3, 5]
postings['run'         ] -> {1: 2, 3: 1, 5: 1}
postings['trail'       ] -> {1: 1, 2: 1, 4: 1}
postings['shoe'        ] -> {1: 1, 3: 1}
postings['hik'         ] -> {2: 1, 5: 1}
postings['lightweight' ] -> {1: 1, 5: 1}
docs matching run AND trail: [1]
query terms: ['trail', 'run', 'shoe']
shard 0 top hits: [(2, 0.578), (4, 0.53)]
shard 1 top hits: [(1, 2.123), (3, 1.391), (5, 0.53)]
merged top 3: [(1, 2.123), (3, 1.391), (2, 0.578)]
  doc 1: Running shoes for trail running, lightweight
  doc 3: Road running shoes with extra cushioning
  doc 2: Waterproof hiking boots for the trail
```

The toy stemmer turns "hiking" into `hik`; real stemmers (Porter, Snowball) and dictionary-based lemmatisers do better, but the idea is the same. Notice doc 4 ("Trail mix snack") matches `trail` yet loses to documents that match more, rarer terms.

How it works — shards, replicas and near-real-time search

- An **index** (like a table) is split into **primary shards**, each a complete Lucene index. A document goes to `shard = hash(_routing or _id) % number_of_primary_shards` (`%` is the remainder after division), so the primary shard count is fixed at creation (changing it means `_split`, `_shrink` or reindexing).
- Each primary has **replica shards** on other nodes for failover and extra read capacity. Writes go to the primary, which forwards them to the in-sync replicas.
- A search is **scatter-gather**: the coordinating node sends the query to one copy of every shard (*query phase*: each returns its top-N ids and scores), merges them, then fetches the winning documents (*fetch phase*). Deep **pagination** (fetching results page by page, here starting at hit 10,000) is expensive because every shard must return 10,000+ hits; use `search_after` instead.
- Lucene writes new documents into small, **immutable segments**. A **refresh** (every 1 second by default) opens the latest segment for search — hence **near-real-time**: a document is searchable about a second after indexing, not instantly. A **translog** (write-ahead log) makes acknowledged writes durable before the next Lucene commit (**flush**), and background **merges** combine small segments and purge deleted documents.

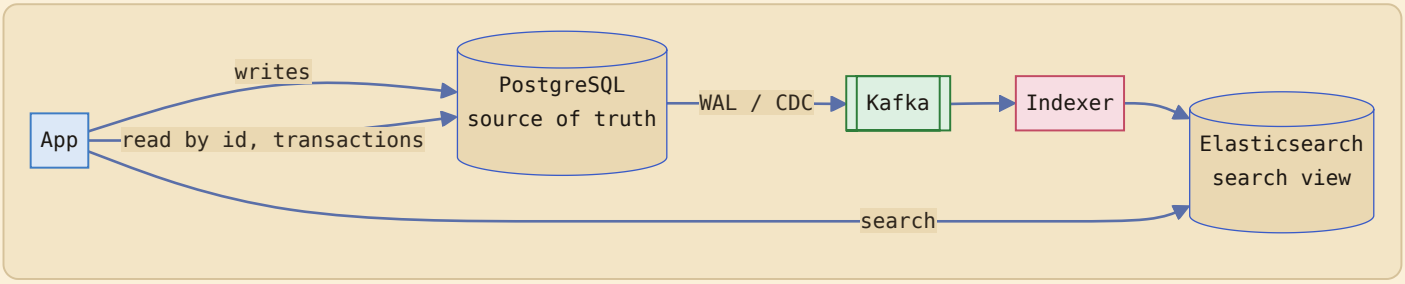


The ELK stack

ELK = **Elasticsearch** (store and search) + **Logstash** (ingest pipelines: parse, enrich, route) + **Kibana** (dashboards and search UI). Lightweight shippers (**Beats**, now largely **Elastic Agent**) were added, and Elastic calls the whole set the **Elastic Stack**. The typical log pipeline: applications write JSON logs → an agent ships them (often via Kafka as a buffer) → ingest parsing → time-based indices (`logs-2026.09.28`) with **index lifecycle management** moving old indices to cheaper nodes and deleting them after the retention period. OpenSearch offers the same shape (OpenSearch Dashboards instead of Kibana). Loki and ClickHouse are popular cheaper alternatives for logs.

When to pair it with a database — and when not

Keep the **relational database as the source of truth** and treat the search index as a derived, rebuildable view. Feed it by **change data capture** (e.g. Debezium reading the Postgres WAL — its write-ahead log of changes — → Kafka → an indexer) or an **outbox table** (the app writes the event into a table in the same transaction as the data, and a separate process forwards it), not by **dual writes** (the app writing to both systems itself) from the application, which drift apart when one write fails.



Do **not** use Elasticsearch as the primary store for money or orders: no multi-document transactions, no joins, eventual visibility (refresh), and mapping changes need reindexing. Stay with Postgres full-text search while the data fits one database and relevance needs are simple.

Redis internals: what it is

§16.2 introduced **Redis** as an in-memory key-value store with rich data structures. (Redis moved to **source-available** licences (code you can read but with restrictions on use) in 2024 and added the AGPLv3 open-source option in 2025; the Linux Foundation's **Valkey** fork keeps the BSD licence. The internals below apply to both.)

How it works — one thread, an event loop

Redis executes all commands on **one main thread** (a thread is one independent sequence of execution inside a program). It uses **I/O multiplexing** (`epoll` on Linux, `kqueue` on macOS/BSD — operating-system calls that report which of many connections have data ready) to watch thousands of client **sockets** (open network connections) at once and runs each ready command to completion, one at a time.

- **Why it is fast anyway:** data is in RAM, commands are O(1) or O(log n) on compact structures, and there are no **locks** (mechanisms that stop two threads touching the same data at once) or **context switches** (the CPU pausing one thread to run another). A single instance handles on the order of 100,000+ simple operations per second.
- **Why every command is atomic:** nothing else runs in the middle of `INCR` or a Lua script / `MULTI`–`EXEC` block.
- **The catch:** one slow command blocks *everyone*. `KEYS *`, `HGETALL` on a hash with millions of fields, `DEL` of a huge key or a long Lua script stall all clients. Use `SCAN` (iterates keys in small batches), `UNLINK` (frees memory in a background thread) and keep keys small. Since Redis 6, optional **I/O threads** parallelise reading and writing sockets, but command execution stays single-threaded.
- **Expiry** is lazy (checked when a key is touched) plus an active background sampler that deletes a portion of expired keys each cycle.

How it works — data structures and encodings

Each type switches internal encoding by size to save memory: small hashes, lists and sorted sets use a compact **listpack** (one contiguous block), then convert to a **hash table**, **quicklist** (linked list — items chained by pointers — of listpacks) or **skip list + hash table** as they grow. The sorted set's skip list gives O(log n) insert, rank and range — why it is the classic leaderboard structure (§16.2). Sets of small integers use an **intset**. `OBJECT ENCODING key` shows which one a key uses.

How it works — persistence: RDB and AOF

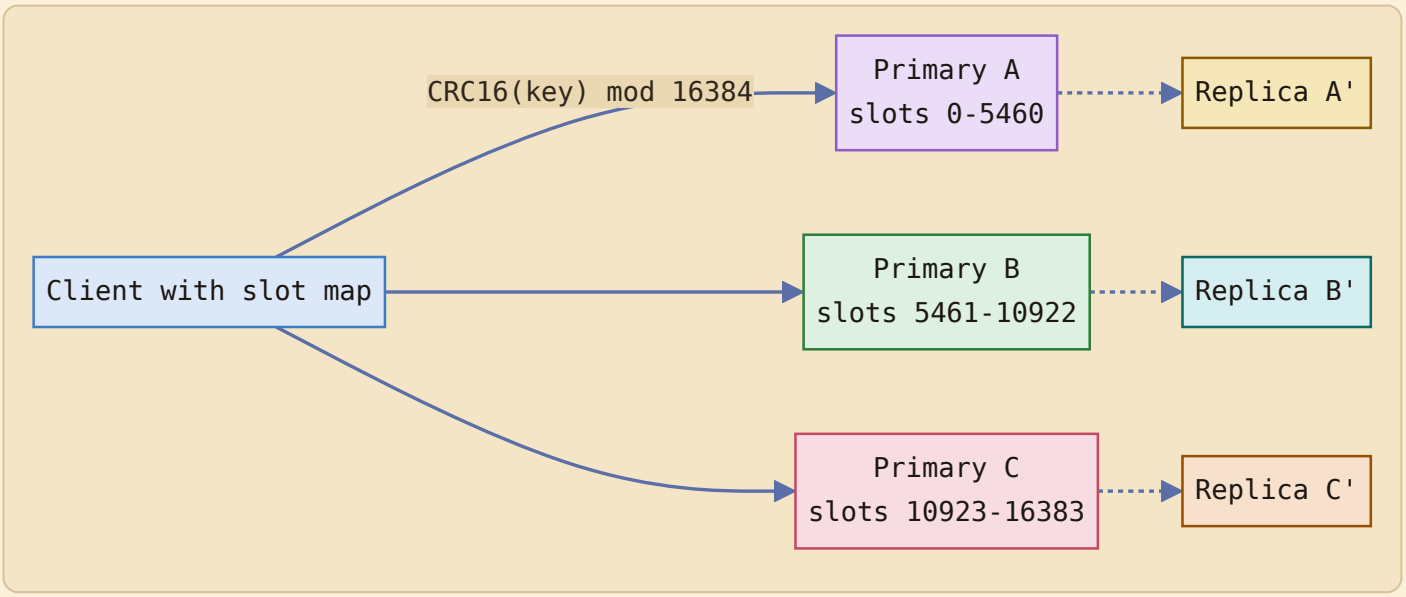
	RDB snapshot	AOF (append-only file)
What	point-in-time binary dump of the whole dataset	log of every write command
How	`fork()` (the OS call that clones a running process); the child writes the snapshot while the parent keeps serving; the OS copies memory pages only when they change (**copy-on-write**)	appended on each write; `fsync` policy `always`, `everysec` (default) or `no`; rewritten in the background to stay compact
Data loss on crash	everything since the last snapshot (minutes)	about 1 second with `everysec`
Restart speed	fast load	slower replay (hybrid AOF with an RDB preamble helps)

Most deployments enable both, or run Redis as a pure cache with persistence off. Watch memory during `fork()`: a write-heavy instance can briefly need much more RAM as pages are copied.

How it works — replication, Sentinel and Cluster

- **Replication** is asynchronous primary → replicas. A replica does a full sync (RDB transfer) once, then streams commands; after a short disconnect it resumes from the primary's **replication backlog** (`PSYNC`). Asynchronous means an acknowledged write can be lost if the primary dies before replicating it (`WAIT` reduces, not removes, that window).
- **Sentinel** monitors primaries and promotes a replica on failure (high availability without sharding).
- **Redis Cluster** shards data over **16,384 hash slots**: `slot = CRC16(key) mod 16384` (CRC16 is a fast 16-bit checksum function used here as a hash), and each primary owns a range of slots. Clients cache the slot map; a request to the wrong node gets a `MOVED` redirect (or `ASK` during slot migration). Multi-key commands work only when all keys are in one slot — force that with a **hash tag**: `{user:42}:cart` and `{user:42}:profile` hash only the part in braces. Nodes **gossip** (each periodically

swaps status with a few random peers, so news spreads through the cluster) to detect failures and promote replicas.



How it works — eviction when memory is full

With `maxmemory` set, `maxmemory-policy` decides what to drop: `noeviction` (the default: writes fail with an error), `allkeys-lru` / `allkeys-lfu` (the usual cache choice), `volatile-lru` / `volatile-lfu` / `volatile-ttl` (only keys with a TTL), or `allkeys-random` / `volatile-random`. LRU and LFU are **approximated** by sampling a few keys (`maxmemory-samples`, default 5) and evicting the best candidate, which avoids the memory cost of a full LRU list. Memcached, by contrast, is multi-threaded, strings-only, has no persistence or replication, and uses a **slab allocator** (memory pre-split into fixed-size chunks grouped by size class) with per-slab LRU — simpler and good for a pure cache, but without Redis's data structures.

Trade-offs

	Elasticsearch / OpenSearch	Redis
Best at	relevance search, filters + facets, log analytics	sub-millisecond reads/writes, counters, sessions, leaderboards, rate limits
Consistency	near-real-time (≈1 s refresh), no multi-doc transactions	atomic single commands/scripts; async replication can lose recent writes
Scaling	shards (fixed count per index) + replicas	Cluster hash slots + replicas; data bounded by RAM
Role	derived view fed from the source of truth	cache or fast secondary store; primary only if losing ~1 s of writes is acceptable

Common mistakes

- **Dual writes** to Postgres and Elasticsearch from the app → use CDC or an outbox so the index can always be rebuilt.
- **Too many small shards** (thousands of tiny indices) → each shard costs **heap** (the memory Elasticsearch's Java runtime keeps objects in) and **file handles** (OS slots for open files); aim for shards of roughly 10–50 GB.
- **Expecting read-your-writes from search** → a new document appears after the next refresh; read it back by id from the database, or use `refresh=wait_for` sparingly.
- **`KEYS *` or giant keys in production Redis** → blocks the single thread; use `SCAN`, split big keys, `UNLINK`.

- **No `maxmemory` / `noeviction` on a cache** → writes start failing when memory fills; pick an eviction policy.
- **Multi-key operations across Cluster slots** → `CROSSSLOT` error; design keys with hash tags.

In the interview

> **30-second answer:** "Elasticsearch stores an inverted index per shard: analysers turn text into terms, posting lists map terms to documents, and BM25 ranks matches. An index is split into a fixed number of primary shards with replicas; queries scatter to every shard and the results are merged. New documents become searchable after a one-second refresh, so it is near-real-time. I keep Postgres as the source of truth and feed Elasticsearch via CDC. Redis runs commands on a single thread with an epoll event loop, so each command is atomic but a slow one blocks everyone; it persists with RDB snapshots and/or an AOF, replicates asynchronously, shards over 16,384 hash slots in Cluster mode, and evicts with approximated LRU or LFU when it hits `maxmemory`."

- *Why is Redis single-threaded yet fast?* → In-memory data, no locks, cheap commands, and I/O multiplexing; the bottleneck is usually the network, not the CPU.
- *RDB vs AOF?* → Snapshots are compact and fast to load but lose minutes; AOF with `everysec` loses about one second but replays slower. Many use both.
- *Why can't you change Elasticsearch's primary shard count in place?* → Routing is `hash(id) % shards`; changing the divisor moves most documents, so you split, shrink or reindex.

Self-check

1. What does an analyzer do at query time? → Applies the same tokenising and filtering as at index time so query and document terms match.
2. Why is Elasticsearch "near" real-time? → New segments become searchable only at the next refresh (1 s default).
3. Which Redis structure backs a leaderboard and why? → A sorted set (skip list + hash): O(log n) score updates and rank queries.
4. Two keys must be updated in one `MULTI` on Redis Cluster — how? → Give them the same hash tag so they land in the same slot.

TL;DR

Search engines = analysers + inverted index + BM25, split into fixed primary shards with replicas, near-real-time via 1 s refresh; ELK = Elasticsearch + Logstash + Kibana for logs; pair with a relational source of truth through CDC. Redis = single-threaded event loop (atomic, but beware slow commands), size-adaptive encodings, RDB + AOF persistence, async replication, 16,384-slot Cluster with hash tags, approximated LRU/LFU eviction.

Sources: Elasticsearch Guide (near real-time search, shards and replicas, text analysis, similarity/BM25, index lifecycle management), Apache Lucene documentation, Robertson & Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond" (2009), Redis documentation (persistence, replication, cluster specification, eviction, `OBJECT ENCODING`, I/O threads), Memcached wiki, Debezium documentation.

Chapter 16 TL;DR

Model	One-line summary	Examples	In Postgres
Key-value	Get/put by key, fastest possible	Redis, Riak	PK table, `UNLOGGED`, UPSERT
Document	Flexible nested JSON per record	MongoDB, CouchDB	`jsonb` + GIN
Wide-column	Partition key + clustering order + LSM writes	Cassandra, HBase	Composite PK + partitions
Graph	Nodes and edges, fast multi-hop traversal	Neo4j, Neptune	Link table + recursive CTE
In-memory	RAM-first, persisted by snapshot + log	SAP HANA, SingleStore, Redis	Buffer cache, unlogged tables
NewSQL	SQL + ACID + auto-sharding via Raft/Paxos	Spanner, CockroachDB, TiDB	Citus (scale-out)
Time-series	Time chunks, compression, retention	TimescaleDB, InfluxDB	Range partitions + BRIN + `date_bin`
Spatial	Geometry types + R-tree style indexes	PostGIS	`point` / `box` + GiST, PostGIS
Big data	Distributed batch and stream processing	Hadoop, Spark, Trino, Kafka, Flink	CDC out to them
Search engine	Analyzer + inverted index + BM25; shards + replicas; ~1 s refresh	Elasticsearch, OpenSearch (ELK for logs)	`tsvector` + GIN

Redis internals (16.11): one command thread + epoll event loop, RDB/AOF persistence, async replication, 16,384 Cluster slots, approximated LRU/LFU eviction.

BASE vs ACID is a spectrum. Choose by access pattern and consistency need, default to Postgres, and add specialised stores only when they clearly earn their operational cost.

Next: Chapter 17 — Schema design case studies.

Chapter 17 — Schema Design Case Studies

Chapters 10–16 gave you the tools one at a time: modeling, normalization, indexes, transactions, scaling, and the non-relational options. Interviews and real projects never ask for one tool at a time. They say "design the database for a hotel booking site" and expect you to go from a vague brief to tables, keys, the queries that matter, and a view on what breaks at scale. This chapter does exactly that thirteen times, always with the same template, so the method becomes a habit rather than a memory exercise.

What you'll learn

1. A repeatable eight-step template for any schema-design question, plus the mapping rules and judgement calls every case reuses.
2. Nine classic business systems — hospital, hotel, bank, photo-sharing (Instagram), e-commerce, food delivery, library, ticket booking, warehouse inventory.
3. Four large-scale product designs — Quora, TikTok, Airbnb, Snapchat — where the relational core meets caches (fast in-memory copies of hot data), **object storage** (services such as Amazon S3 that store whole files by key) and NoSQL (non-relational databases, Chapter 16).
4. Four of the designs built for real on PostgreSQL (e-commerce, ticket booking with seat-hold concurrency, library, Instagram feed) with seed data and real output.
5. The modeling patterns that keep coming back — junction tables, status enums, soft deletes, audit columns, polymorphic associations, denormalized counters (each is defined where it first appears below) — compared in one table at the end.

How to run the examples. All code is in `code/`. `00_schema.sql` creates four PostgreSQL schemas — `shop`, `tix`, `lib`, `insta` — with small invented data; files `01`–`04` each run on a fresh copy:

```
bash tools_run_sql.sh sql '17-*'                          # run this chapter's files
bash tools_pg.sh psql -d ch_17 -f sql/chapters/17-schema-design-case-studies/code/02_ticket_booking.sql
```

Every "Output:" block below is real psql output (command tags like `UPDATE 2` trimmed). This chapter **applies** earlier material rather than re-teaching it: modeling and ER notation are in Chapter 10, normal forms in Chapter 11, index types and `EXPLAIN` in Chapter 12, locking and isolation in Chapter 13, partitioning/sharding/replication in Chapter 15, and NoSQL families in Chapter 16.

17.0 The template

What it is

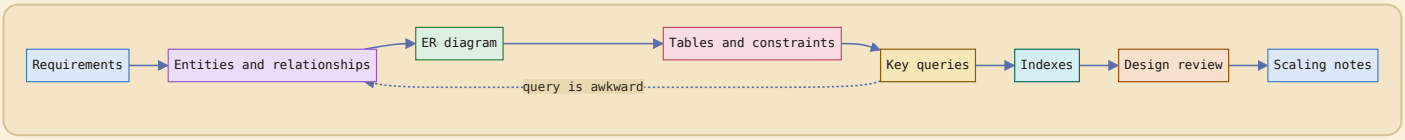
A **schema-design case study** is a short, structured answer to "what tables would you build for X?". The template below is the modeling process of Chapter 10 §10.3 compressed into interview-sized steps.

How it works

1. **Requirements.** List what users do (functional) and the numbers and guarantees that matter (non-functional: reads vs writes, consistency, **retention** — how long data must be kept). Then **write down the 5–6 questions the system must answer** ("which rooms are free between two dates?") — they drive the tables *and* the indexes. Ask; don't assume.
2. **Entities and relationships.** Nouns that you store several facts about, with several instances, become entities; verbs become relationships. For each relationship decide **cardinality** (how many rows on each side can be linked: 1:1 one-to-one, 1:N one-to-many, M:N many-to-many) **and participation** (total = every row must take part; partial = optional). Say each one aloud as a sentence.
3. **ER diagram.** Draw it — an **entity-relationship diagram** shows tables as boxes and relationships as lines (here with `erDiagram` **crow's-foot notation**, where a three-pronged "foot" on a line end means

"many" and a bar means "one"). The diagram is where a reviewer spots a missing entity fastest.

4. **Key tables (DDL).** **DDL** (data definition language) is the `CREATE`/`ALTER` part of SQL. Map entities to tables: **primary keys** (the column(s) that uniquely identify a row), **foreign keys** (columns that must match a primary key in another table), `NOT NULL`, `UNIQUE`, `CHECK`. An M:N relationship becomes a **junction table** (a table whose primary key is the pair of foreign keys). Aim for **3NF** (third normal form: every non-key column depends on the key, the whole key and nothing but the key — Chapter 11), then **denormalize** (deliberately store a duplicate or derived value for speed) on purpose.
5. **Key queries.** Write the 3–5 queries the product runs most. They validate the schema: if a core question needs six joins and a `DISTINCT`, the model is probably wrong.
6. **Indexes.** One index per hot access path, usually a **composite index** (an index on several columns, in a chosen order) that matches the query's `WHERE` + `ORDER BY` (see Chapter 12).
7. **Design review.** Check the tables against 1NF → BCNF (the ladder of **normal forms**, rules that remove redundant storage of facts — Chapter 11), name every deliberate redundancy, and list what the design *cannot* do yet (no history, an unenforced rule). Volunteering the limits reads as judgement, not as a mistake.
8. **Scaling notes.** Which table grows fastest, what to **partition** (split into pieces inside one database) or **shard** (split across several database servers) it by, what to cache, what to move to another store, and where strict consistency is required.



Rules every case reuses

These mapping rules (Chapter 10 §10.15) and judgement calls come up in all thirteen designs, so the cases below only point at them.

Situation	Rule
Total participation (every order has a customer)	The foreign key is `NOT NULL`; partial participation stays nullable
1:1 relationship	Foreign key **plus** `UNIQUE` (or a shared primary key) — without it the relationship is silently 1:N
M:N relationship	Junction table; an attribute of the *pairing* (rate agreed, price of a seat at a show, dosage per medicine) lives there
Composite attribute (name = first + last)	Split into columns
Multivalued attribute (several phone numbers)	Its own table keyed by (owner, value)
Money	`numeric` / `DECIMAL`, never `float` (0.10 has no exact binary form) or `int` (no cents)
Times that order events	`timestamptz`, not `date` — two postings on one day still need an order
Codes with letters or leading zeros (policy, passport, room "204B")	`text` / `varchar`, never `int`
Table named `user`, `order`, `group`, `like`, `transaction`, `show`	Reserved (or keyword) in some dialects — rename (`app_user`, `customer_order`, `screening`) rather than quote forever
Passwords	Store a slow salted hash (a one-way scramble of the password plus a random per-user value, `password_hash`), never the password
One entity doing two jobs	Split **type** from **instance**: `titles` / `copies`, `room_types` / `rooms`, `seats` / `show_seats`
Only current state stored	Keep history rows (loans, bookings, status events, ledger) — "when did…?" needs them
A value that must not change later	Snapshot it (price paid, address shipped to, fine charged)
Designing for Instagram scale on day one	State the single-node design first, then say what changes at 100×

In the interview

30-second answer: "I'll clarify the main use cases and scale, pick the entities and relationships, sketch the ER diagram, turn it into tables with keys and constraints, check it against the top queries, add indexes for those queries, and finish with what I'd partition, cache or move to another store as it grows."

- *How do you know the design is right?* — Every core query is simple and index-friendly, every business rule that must never be broken is a **constraint** (a rule the database itself enforces) or runs inside one **transaction** (a group of statements that succeed or fail together), and no fact is stored twice without a reason.
- *When do you denormalize?* — When a measured hot read needs it (a feed, a counter), and only with a plan to keep the copy correct (same transaction, **trigger** — code the database runs automatically on insert/update/delete — or a periodic **reconciliation** job that recomputes the value and flags differences).

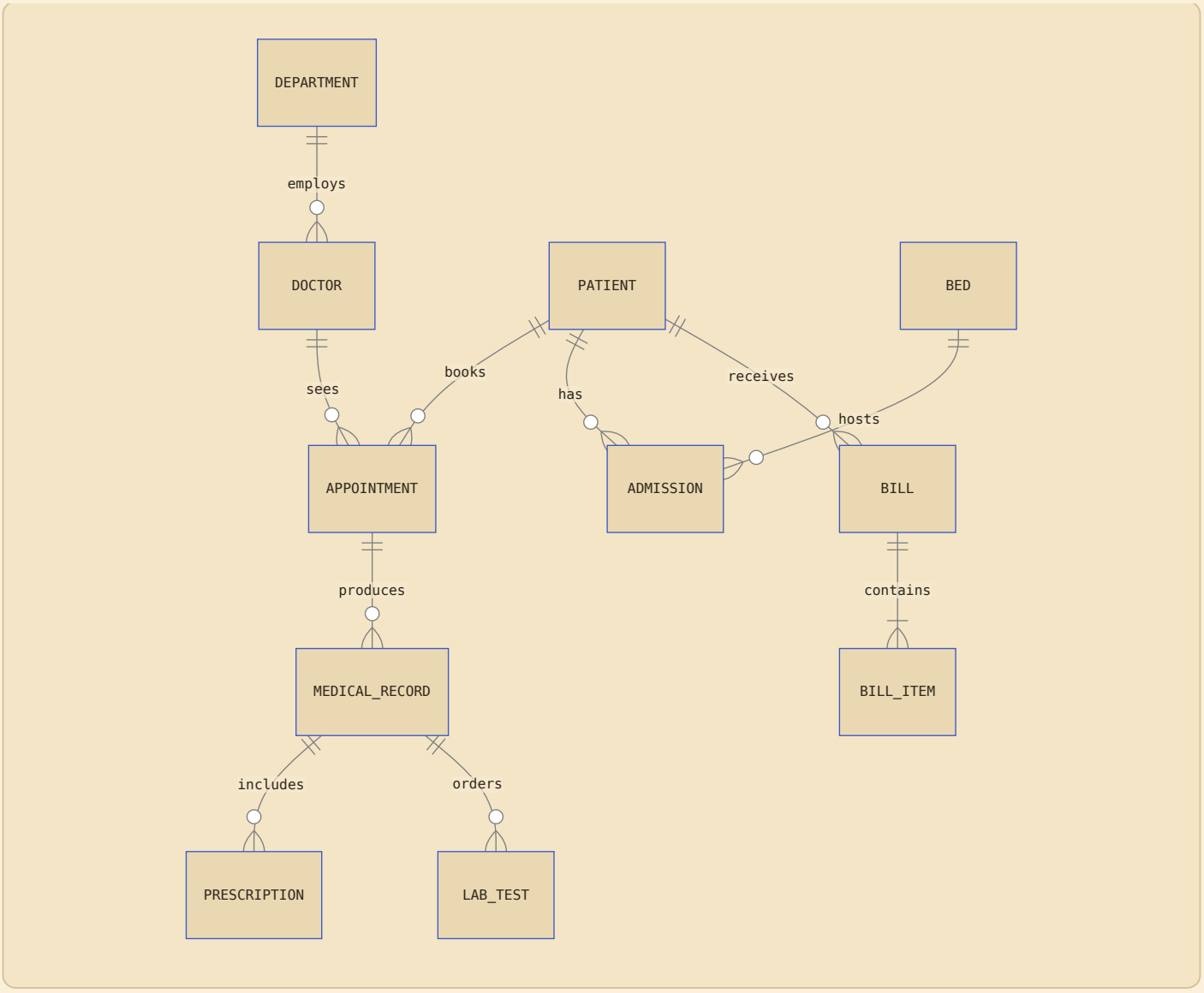
17.1 Hospital management system

Requirements

- Register **patients** (name, birth date, contact, one or more phone numbers, insurance policies) and **doctors** (specialty) grouped into **departments**.
- Schedule **appointments** (each pairs one patient with one doctor at a time slot); a doctor is never in two overlapping slots.
- Record **admissions** to a **room/bed** (in-patients), with admit and discharge dates.
- Keep **medical records** (diagnosis, treatment, written by a doctor), **prescriptions** listing one or more **medicines** from a catalogue, and **lab tests** with results.
- Produce **bills** per visit/admission with line items and payments.
- Non-functional: records are legally retained for years and must be **auditable** (every change traceable to who and when); access is strictly **role-based** (permissions depend on the user's role, e.g. doctor or clerk).
- Questions to answer: a doctor's appointments tomorrow, a patient's full history, free rooms now, what a patient still owes, the medicines on a prescription.

Entities and relationships

Relationship	Cardinality	Note
Department — Doctor	1:N	a doctor belongs to one department
Patient — Appointment — Doctor	M:N via `appointments`	the appointment is the junction row with its own attributes (time, status)
Patient — Admission — Bed	1:N and N:1	a bed has many admissions over time, one at a time
Appointment / Admission — Medical record	1:N	a record belongs to one encounter
Doctor — Medical record	1:N	the author; total participation
Record — Prescription, Record — Lab test	1:N	
Prescription — Medicine	M:N	junction `prescription_items`; dose and frequency per medicine live there
Patient — Insurance policy	1:N	partial: a patient may have none; `(provider, policy_no)` is `UNIQUE`
Patient — Phone	1:N	multivalued attribute → `patient_phones (patient_id, phone)`
Patient — Bill — Bill items	1:N, 1:N	


DEPARTMENT
employs
DOCTOR
PATIENT
BED
sees
books
has
receives
hosts
APPOINTMENT
ADMISSION
BILL
produces
contains
MEDICAL_RECORD
BILL_ITEM
includes
orders
PRESCRIPTION
LAB_TEST

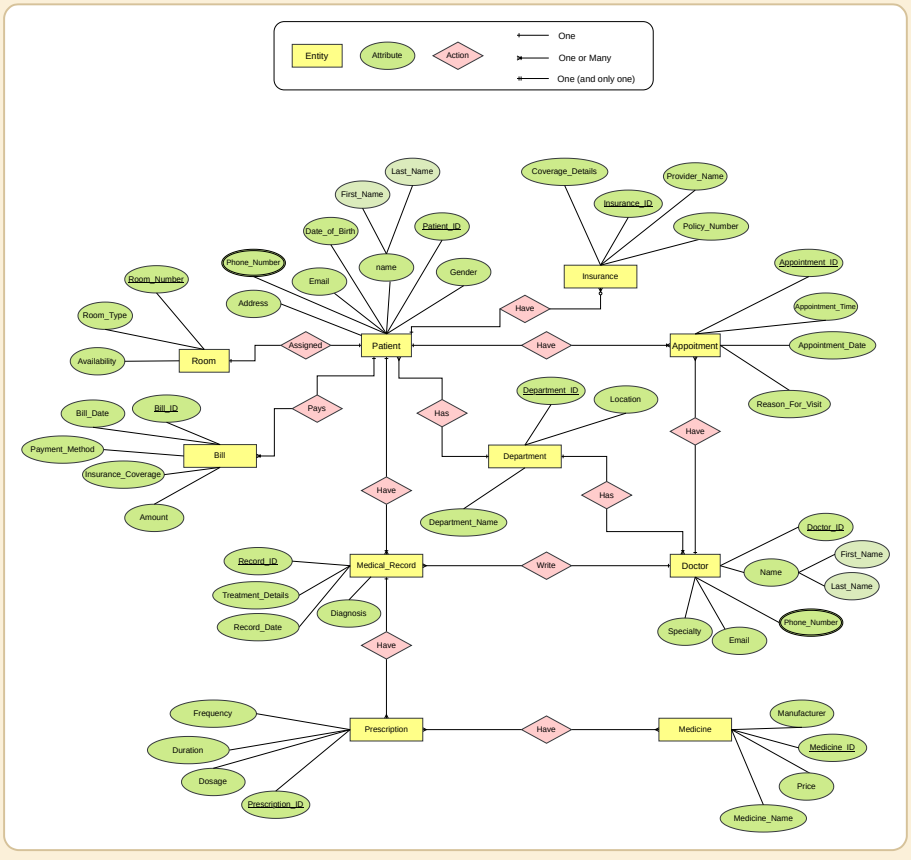

Entity
Attribute
Action
One
One or Many
One (and only one)
Last_Name
First_Name
Date_of_Birth
Patient_ID
Phone_Number
name
Email
Gender
Address
Room_Number
Room_Type
Availability
Room
Assigned
Patient
Have
Insurance
Coverage_Details
Provider_Name
Insurance_ID
Policy_Number
Appointment_ID
Appointment_Time
Appointment_Date
Appoitment
Reason_For_Visit
Pays
Has
Bill_Date
Bill_ID
Payment_Method
Insurance_Coverage
Amount
Bill
Department_ID
Location
Department
Department_Name
Record_ID
Treatment_Details
Record_Date
Diagnosis
Medical_Record
Write
Doctor_ID
First_Name
Name
Last_Name
Phone_Number
Specialty
Email
Doctor
Frequency
Duration
Dosage
Prescription_ID
Prescription
Manufacturer
Medicine_ID
Price
Medicine_Name
Medicine

Figure — the course's hospital ER diagram: ten entities with cardinality and participation marked (course diagram)

Key tables

```
CREATE TABLE appointments (
  appointment_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  patient_id bigint NOT NULL REFERENCES patients,
  doctor_id  bigint NOT NULL REFERENCES doctors,
  slot       tstzrange NOT NULL,                    -- [start, end)
  status     text NOT NULL CHECK (status IN ('booked','checked_in','completed','cancelled','no_show')),
  -- no two active appointments of the same doctor may overlap (needs: CREATE EXTENSION btree_gist)
  EXCLUDE USING gist (doctor_id WITH =, slot WITH &&) WHERE (status <> 'cancelled')
);
CREATE TABLE admissions (
  admission_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  patient_id bigint NOT NULL REFERENCES patients,
  bed_id     int    NOT NULL REFERENCES beds,
  admitted_at   timestamptz NOT NULL,
  discharged_at timestamptz CHECK (discharged_at > admitted_at)
);
CREATE UNIQUE INDEX one_patient_per_bed ON admissions (bed_id) WHERE discharged_at IS NULL;
```

(`tstzrange` is a time-range type — a start and end timestamp in one value; `&&` means "overlaps".) The **exclusion constraint** (`EXCLUDE USING gist … WITH &&`) is PostgreSQL's way of saying "no two rows may have overlapping ranges for the same doctor" — a rule a plain `UNIQUE` cannot express. MySQL and SQL Server have no equivalent; there you **lock** the doctor's row (`SELECT … FOR UPDATE` — read rows and lock them so other transactions must wait before changing them until you finish) and check for overlaps inside the booking transaction (Chapter 13).

Key queries

1. A doctor's schedule for a day: `WHERE doctor_id = $1 AND slot && tstzrange($day, $day + interval '1 day')`.
2. Free beds per ward: beds with no open admission (`NOT EXISTS … discharged_at IS NULL`).
3. A patient's full history: records, prescriptions and lab results across all encounters, newest first.
4. Unpaid bills older than 30 days.

Indexes

The exclusion constraint's **GiST** index (a Postgres index type that handles ranges and geometry, Chapter 12) serves schedule lookups; `admissions (patient_id)`, `medical_records (patient_id, created_at DESC)`, and the partial unique index above for bed occupancy.

Design review

- **Normal forms.** The phone and junction tables have no non-key columns, so 2NF holds; `department_name` stays in `departments` (copying it onto patients would be a **transitive dependency** — a column depending on another non-key column rather than on the key — breaking 3NF); the extra **candidate keys** (other columns that could also uniquely identify a row) (`email`, `(provider, policy_no)`) keep BCNF.
- **Room occupancy.** The simplest model is `rooms.patient_id UNIQUE` (a 1:1 "occupies"), but it only knows who is in the room *now*. The `admissions` table with a partial unique index keeps the same rule *and* the history.
- **Dose per medicine.** If dose sat on `prescriptions`, every medicine on one prescription would share it; it belongs to the pairing.
- **Still missing:** who cancelled an appointment and when (status history), and a patient's department over time.

Scaling notes

A hospital's volume is modest — one well-tuned PostgreSQL **primary** (the one server that accepts writes) with a **replica** (a read-only copy kept in sync) for reports is plenty. The hard requirements are **integrity and audit**: append-only record tables (corrections are new rows that reference the old one), `created_by`/`created_at` on every clinical row, **row-level security** (database rules that filter which rows each user may see) for role-based access (Chapter 14), and encrypted backups with **point-in-time recovery** (restoring the database as it was at any chosen moment) (Chapter 15 §15.9). Large files (scans, X-rays) go to object storage; the table keeps only the key.

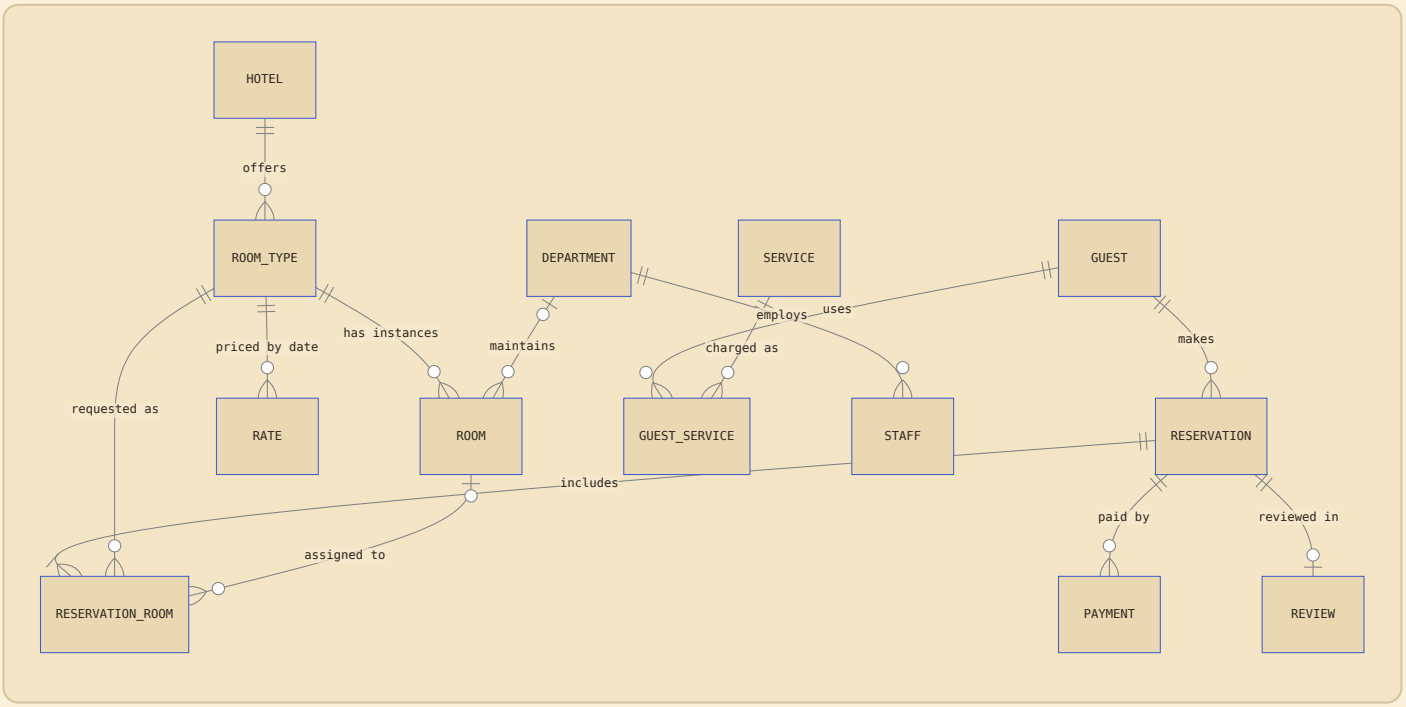
17.2 Hotel reservation system

Requirements

- **Hotels** have **room types** (single, double, suite) with a base price, and physical **rooms** of each type.
- **Guests** search availability by city and dates, then create a **reservation** for one or more rooms; no room may be double-booked.
- Reservation life cycle: pending → confirmed → checked-in → checked-out, or cancelled. **Payments** and **reviews** attach to reservations.
- Prices vary by date (seasonal rates).
- Guests are identified by a document (passport, national ID) and may have several phone numbers; they also use **extra services** (dining, spa, laundry) charged to the stay.
- **Staff** belong to **departments** (housekeeping, front desk); a department looks after rooms.
- Questions: free rooms between two dates, what a guest paid and owes, today's check-ins, services a guest used, revenue per room type last month, who works in housekeeping.

Entities and relationships

The central decision is **type versus instance**: guests book a *room type* for a date range; a specific *room* is assigned at or before check-in. Modeling only rooms makes search slow; modeling only types makes check-in impossible.



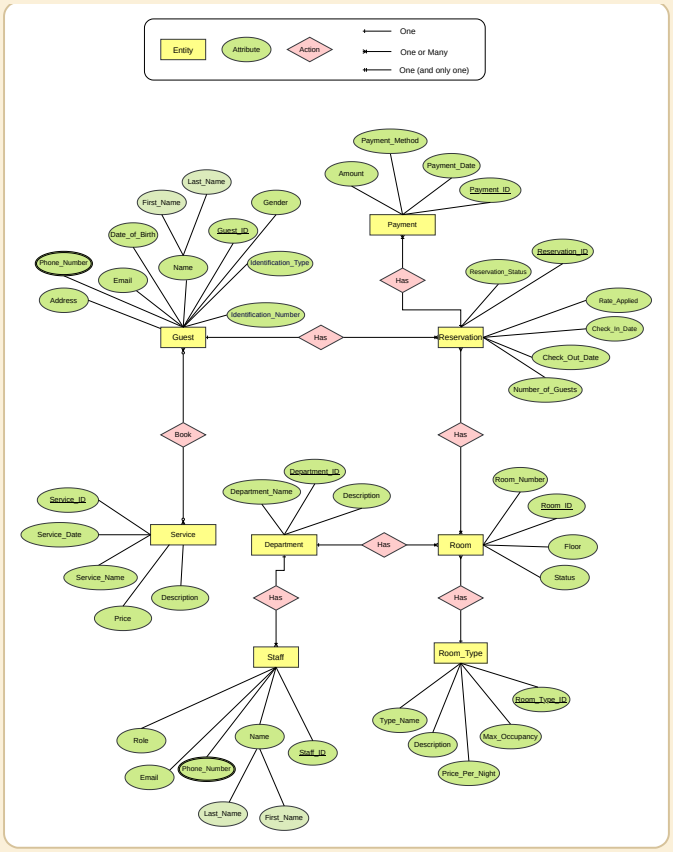


Figure — the course's hotel ER diagram (eight entities); reservation-to-room and guest-to-service are M:N (course diagram)

Two details from the full model: the price and maximum occupancy belong to the **room type**, not to each room (ten identical doubles share one price); and the usage of a service is a junction row `guest_services (guest_services_id, guest_id, service_id, reservation_id, used_on, quantity, price_charged)` with **its own surrogate key**, because the same guest orders dinner on several nights — a `(guest_id, service_id)` key would allow only one use ever. `guests` carries `UNIQUE (id_type, id_number)` so the same identity document can never appear on two guest rows.

Key tables

```
-- reservations (guest_id NOT NULL, hotel_id, check_in, check_out CHECK (check_out > check_in), status CHECK (...))
CREATE TABLE reservation_rooms (
  reservation_id bigint REFERENCES reservations ON DELETE CASCADE,
  line_no        smallint,
  room_type_id   int NOT NULL REFERENCES room_types,
  room_id        int REFERENCES rooms,            -- NULL until assigned
  stay           daterange NOT NULL,              -- copied from the reservation for the constraint
  nightly_price  numeric(10,2) NOT NULL,          -- snapshot
  PRIMARY KEY (reservation_id, line_no),
  EXCLUDE USING gist (room_id WITH =, stay WITH &&)   -- a room is never in two overlapping stays
);
-- Inventory per type per night: the row that search reads and booking decrements.
CREATE TABLE room_inventory (
  room_type_id int REFERENCES room_types,
  night        date,
  total        smallint NOT NULL,
  reserved     smallint NOT NULL DEFAULT 0 CHECK (reserved <= total),
  PRIMARY KEY (room_type_id, night)
);
```

The exclusion constraint needs `CREATE EXTENSION btree_gist` (for the `=` on an integer inside a GiST index); rows with `room_id` still `NULL` never conflict, and cancelling a reservation must clear or

delete its lines so the room is released.

`room_inventory` is a deliberate **denormalization**: counting overlapping reservations for every search is too slow, so each night keeps a counter. Booking increments `reserved` for every night of the stay in one transaction; the `CHECK` makes overbooking fail. (Hotels that *allow* a small overbooking set the check to `reserved <= total * 1.1`.)

Key queries

```sql
-- Room types in a hotel with at least one room free on every night of 10–13 June
SELECT rt.room_type_id, rt.name
FROM room_types rt JOIN room_inventory ri USING (room_type_id)
WHERE rt.hotel_id = $1 AND ri.night >= '2026-06-10' AND ri.night < '2026-06-13'
GROUP BY rt.room_type_id
HAVING count(*) = 3 AND min(ri.total - ri.reserved) > 0;
```

Other hot queries: today's arrivals and departures per hotel, a guest's upcoming stays, average rating per hotel.

Indexes

`reservations (hotel_id, check_in)` for arrival lists, `reservations (guest_id, check_in DESC)`, the inventory primary key for search, and the GiST index behind the exclusion constraint.

Design review

- **Normal forms.** Price lives on `room_types`, not `rooms` (3NF); `nightly_price` on the booking line is a *snapshot*, a different fact from today's list price.
- **Double booking.** `UNIQUE` only detects identical values, never two date ranges that overlap — hence the exclusion constraint (PostgreSQL) or a check inside the booking transaction elsewhere.
- **A room status column** (free/occupied) repeats what reservations already say and can disagree with them; keep it only for things bookings cannot express, such as "under maintenance".
- **Still missing:** mid-stay room changes (the line would need its own date range, which `stay` provides here), and who cancelled a booking and when.

Scaling notes

Search traffic is 100–1,000× booking traffic. Serve search from replicas or a search index (Elasticsearch/OpenSearch, search engines covered in Chapter 16) and a cache of inventory, but make the **booking** transaction hit the primary and re-check the inventory row under lock — search results may be a few seconds stale, bookings may not. Partition `reservations` by `check_in` month; shard by `hotel_id` if one cluster is not enough, since nearly every transaction touches a single hotel.

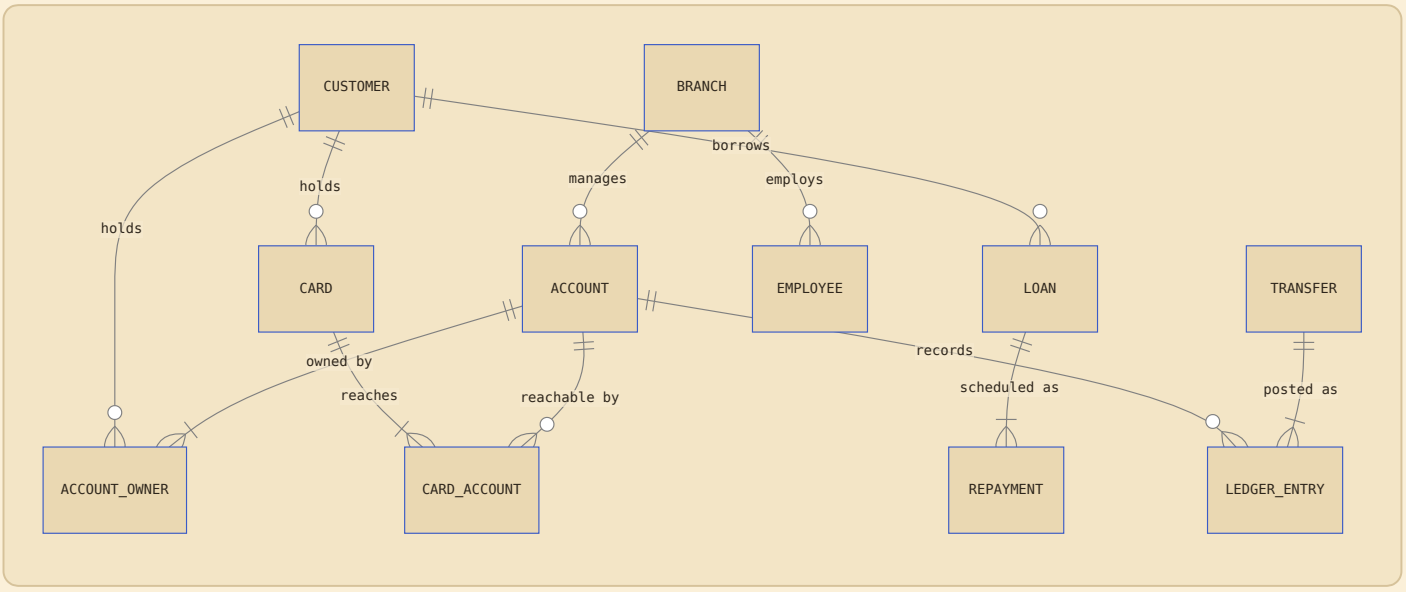
17.3 Banking system

Requirements

- **Customers** own one or more **accounts** (checking, savings, loan); an account may have joint owners.
- Deposits, withdrawals and **transfers** between accounts; balances may never go below an allowed limit.
- A full, immutable history of every money movement; statements per month.
- **Branches** (each publishing several phone numbers) with their **employees**; **cards**, where one card can reach several accounts; **loans** with repayments.
- Questions: the balance now, last month's activity, a customer's accounts, what is still owed on a loan, which accounts a card reaches, who works at a branch.
- Non-functional: strict consistency (**ACID** — atomic, consistent, isolated, durable transactions, Chapter 13), auditability, regulatory retention.

Entities and relationships

The key modeling idea is **double-entry bookkeeping**: money is never "updated", it is *moved*. A **transaction** (the business event) has two or more **entries** (ledger lines), each a signed amount on one account, and the entries of one transaction sum to zero.



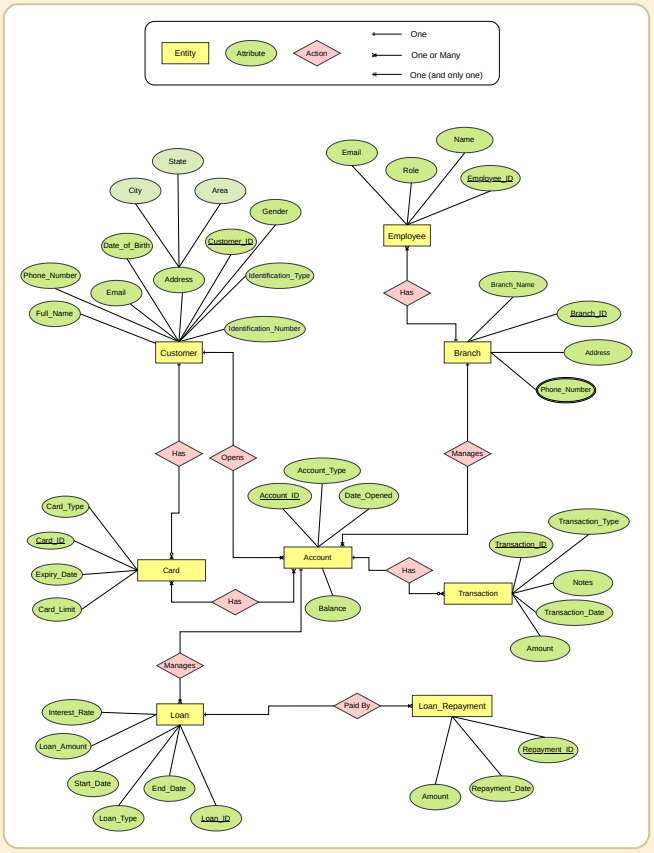


Figure — the course's bank ER diagram (eight entities); the card-to-account link is M:N (course diagram)

Key tables

```
CREATE TABLE accounts (
  account_id  bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  iban        text NOT NULL UNIQUE,
  kind        text NOT NULL CHECK (kind IN ('checking','savings','loan')),
  currency    char(3) NOT NULL,
  balance     numeric(18,2) NOT NULL DEFAULT 0,   -- cached; the ledger is the truth
  overdraft_limit numeric(18,2) NOT NULL DEFAULT 0,
  CHECK (balance >= -overdraft_limit),
  status      text NOT NULL DEFAULT 'open' CHECK (status IN ('open','frozen','closed'))
);
CREATE TABLE transfers (
  transfer_id     bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  idempotency_key uuid NOT NULL UNIQUE,           -- a retried request cannot post twice
  kind            text NOT NULL,
  created_at      timestamptz NOT NULL DEFAULT now()
);
CREATE TABLE ledger_entries (
  entry_id    bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  transfer_id bigint NOT NULL REFERENCES transfers,
  account_id  bigint NOT NULL REFERENCES accounts,
  amount      numeric(18,2) NOT NULL CHECK (amount <> 0),   -- + credit, - debit
  balance_after numeric(18,2) NOT NULL
);
```

Money is `numeric`, never `float` (binary floating point cannot represent 0.10 exactly). `idempotency_key` makes the transfer **idempotent** — repeating the same request (e.g. after a network timeout) has the same effect as sending it once; a `uuid` is a 128-bit random id the client generates. The ledger is **append-only** (rows are only ever added, never changed or deleted): revoke `UPDATE` and `DELETE` on it; a mistake is fixed with a reversing transfer.

Key queries

A transfer is the textbook transaction (Chapter 13 §13.1):

```
BEGIN;
INSERT INTO transfers (idempotency_key, kind) VALUES ($key, 'transfer') RETURNING transfer_id;
-- lock both rows in a fixed order (lower id first) so two opposite transfers cannot deadlock
SELECT account_id FROM accounts WHERE account_id IN ($from, $to) ORDER BY account_id FOR UPDATE;
UPDATE accounts SET balance = balance - $amt WHERE account_id = $from;   -- CHECK stops overdraft
UPDATE accounts SET balance = balance + $amt WHERE account_id = $to;
INSERT INTO ledger_entries (transfer_id, account_id, amount, balance_after) VALUES (...), (...);
COMMIT;
```

Others: a monthly statement (`WHERE account_id = $1 AND created_at` in the month, ordered), a reconciliation check that every transfer's entries sum to zero (`GROUP BY transfer_id HAVING sum(amount) <> 0` must return nothing), and "balance = sum of entries" per account.

Indexes

(A **deadlock**, mentioned in the code comment above, is two transactions each waiting for a lock the other holds, so neither can proceed; the database kills one.) `ledger_entries (account_id, entry_id DESC)` for statements, `account_owners (customer_id)`, and the unique `idempotency_key`. The junction `card_accounts` has PK `(card_id, account_id)`, which already answers "which accounts can this card reach"; the reverse question needs an extra index on `account_id`, because an index is usable only from its leading column.

Design review

- **Transfers need a pairing key.** A simpler model with one `transactions` row per account (type = deposit/withdrawal/transfer) cannot name both sides of a transfer; the `transfer_id` shared by the

debit and credit entries fixes that. Name the table `ledger_entries` or `account_transactions` — `TRANSACTION` is reserved in SQL Server and a keyword elsewhere.

- **Cached balance.** `accounts.balance` is a stored total of the entries. Normal forms don't forbid it (it isn't a dependency *inside* one row), but it can drift, so a nightly job recomputes and compares.
- **Rules a foreign key cannot express:** that a card only reaches accounts its owner holds; that each transfer's entries sum to zero. Check them in the transaction or with a periodic query.
- **Still missing:** which employee opened an account or approved a loan (audit columns), card number and status (blocked ≠ expired), and the width of `numeric` — pick it for the largest expected value, because widening later rewrites the table. Customer `city` and `state` together are a small 3NF compromise most teams accept knowingly.

Scaling notes

Correctness beats **throughput** (work done per second): **synchronous replication** (a commit waits until a copy on another server has it) to a **standby** (a replica ready to take over) so a **failover** (switching to the standby when the primary dies) loses no committed money (Chapter 15 §15.5), `SERIALIZABLE` (the strictest isolation level: transactions behave as if run one at a time) or explicit row locks for balance checks, and range partitioning of the ledger by month for archiving. When one node is not enough, shard by account (or customer); a transfer between shards then needs a **saga** (a chain of local steps, each with a compensating "undo" step) or **two-phase commit** (a coordinator asks every server to prepare, then tells all to commit — Chapter 15 §15.9) — which is why many banks keep the core ledger on a single, very large, **vertically scaled** (made bigger rather than multiplied) database or a distributed SQL system (Spanner, CockroachDB).

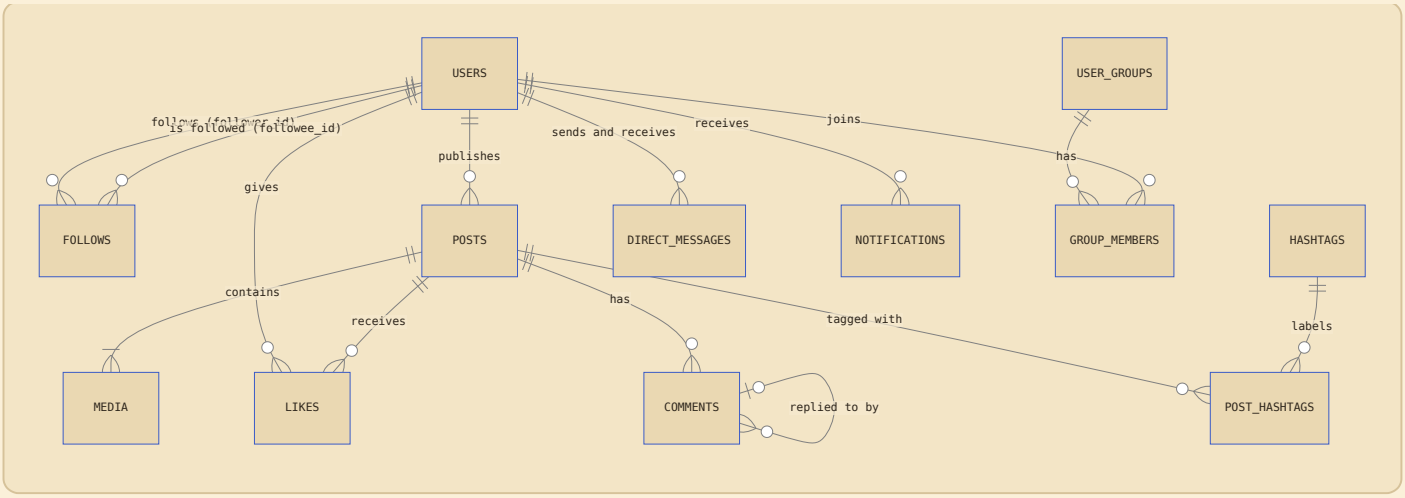
17.4 Instagram-style photo sharing (runnable)

Requirements

- **Users** have a profile, can be public or private, and **follow** other users (one-directional; "friends" = mutual follows).
- Users publish **posts** with a caption and one or more **media** items (photos/videos, a carousel keeps their order), tagged with **hashtags**.
- Other users **like** posts (at most once each) and **comment**, including replies to comments.
- Users send **direct messages** to one other user, receive **notifications** (likes, comments, follows), and create or join **groups**.
- The **home feed** shows recent posts from followed accounts, newest first, paged; profiles show like and follower counts instantly.
- Non-functional: extremely read-heavy (feeds and profiles), hundreds of millions of posts, media bytes far larger than the rows.

Entities and relationships

`follows` is a **self-referencing M:N** relationship on `users` (both columns point to `users`). `likes` and `post_hashtags` are junction tables. `comments` references itself through `parent_id` to form reply threads.



Key tables

From `00_schema.sql` (schema `insta`):

```
CREATE TABLE follows (
  follower_id bigint REFERENCES users,
  followee_id bigint REFERENCES users,
  created_at  timestamptz NOT NULL,
  PRIMARY KEY (follower_id, followee_id),       -- "whom do I follow?" + no duplicate follows
  CHECK (follower_id <> followee_id)
);
CREATE INDEX ON follows (followee_id);          -- "who follows me?"
CREATE TABLE posts (
  post_id bigint PRIMARY KEY, user_id bigint NOT NULL REFERENCES users,
  caption text, created_at timestamptz NOT NULL,
  like_count int NOT NULL DEFAULT 0, comment_count int NOT NULL DEFAULT 0
);
CREATE INDEX ON posts (user_id, created_at DESC, post_id DESC);
-- media (post_id, position, url = object-store key, kind): carousel order kept by position
CREATE TABLE likes (
  user_id bigint REFERENCES users, post_id bigint REFERENCES posts ON DELETE CASCADE,
  created_at timestamptz NOT NULL,
  PRIMARY KEY (post_id, user_id)                  -- one like per user per post
);
-- comments (comment_id, post_id, user_id, parent_id REFERENCES comments, body, created_at)
```

The same file also creates `direct_messages` (two FKs to `users`, sender and recipient), `notifications` (with `actor_id` and `post_id`, so tapping one can open the post that caused it), and `user_groups` + `group_members` — a group's multivalued "members" attribute becomes a junction table that can also record `joined_at`. `user`, `group` and `like` are reserved words in PostgreSQL or MySQL, hence the plural or prefixed names.

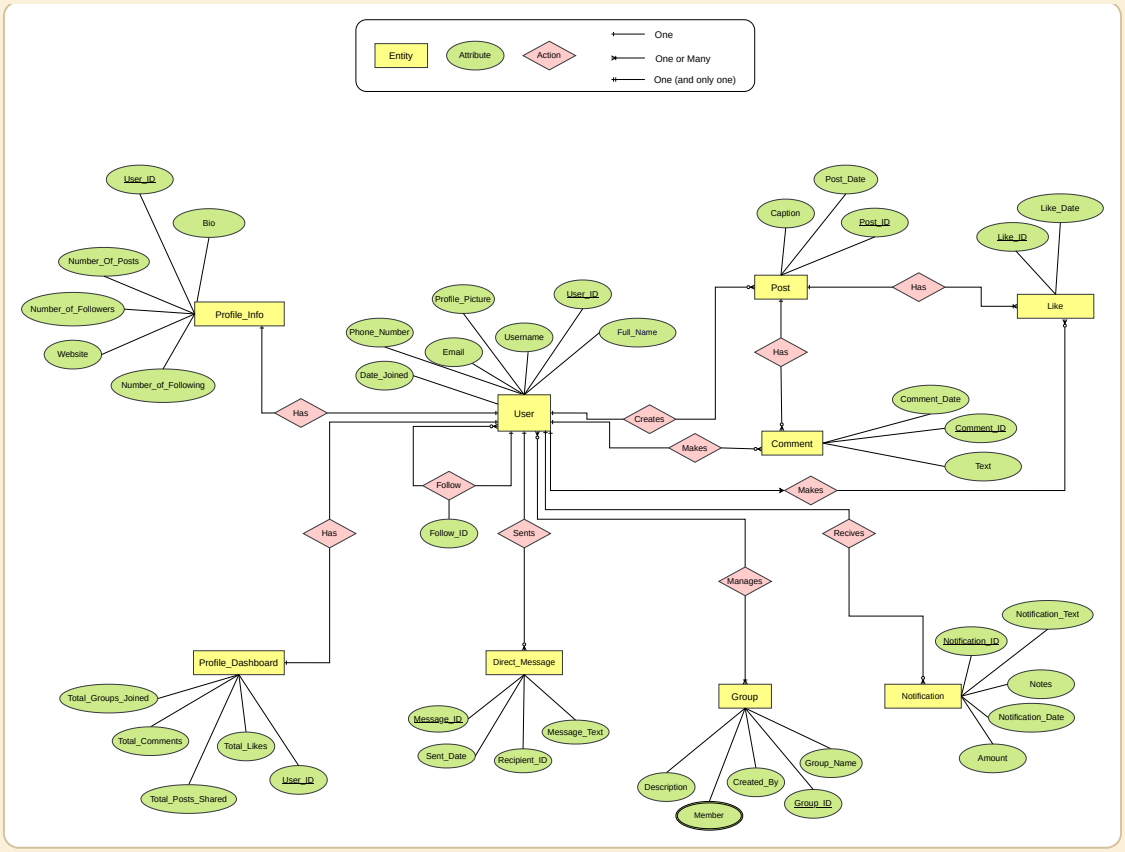


Figure — the course's Instagram ER diagram, with Follow drawn as a relationship from User back to User and profile data as 1:1 tables (course diagram)

The course version keeps profile numbers (posts, followers, following) in a separate **1:1 table** whose primary key is also the FK to the user — a *shared primary key*, the simplest way to guarantee one row per user. Here those counters sit on `users` directly; either way they are a speed decision the normal forms cannot see, because the redundancy spans two tables.

Key queries

The home feed is a join from "whom I follow" to their posts. **Fan-out on read** means computing it at request time, as here:

```
SELECT p.post_id, u.username, p.caption, p.created_at, p.like_count
FROM follows f
JOIN posts p ON p.user_id = f.followee_id
JOIN users u ON u.user_id = p.user_id
WHERE f.follower_id = 1
ORDER BY p.created_at DESC, p.post_id DESC
LIMIT 2;
-- next page: add  AND (p.created_at, p.post_id) < ('2026-05-02 07:30+00', 1003)
```

Output:

```
 post_id | username |           caption           |       created_at       | like_count
---------+----------+-----------------------------+------------------------+------------
    1005 | cy       | Sourdough attempt 2 #baking | 2026-05-03 10:15:00+00 |          1
    1003 | bo       | Trail views #running #hills | 2026-05-02 07:30:00+00 |          2

 post_id | username |       caption        |       created_at        (page 2)
---------+----------+----------------------+------------------------
    1002 | cy       | Fresh bread #baking  | 2026-05-01 08:00:00+00
    1001 | bo       | Sunrise run #running | 2026-05-01 06:10:00+00
```

Page 2 uses **keyset pagination** (`OFFSET n` would skip the first n rows by reading and discarding them): "continue after the last `(created_at, post_id)` I showed" instead of `OFFSET`, so it stays fast on page 500 and never repeats or skips a post when new ones arrive (see Chapter 12). `post_id` breaks ties between posts with the same timestamp.

A like must insert the row **and** bump the counter together, and a double tap must not count twice:

```
BEGIN;
WITH ins AS (
  INSERT INTO likes (user_id, post_id, created_at) VALUES (4, 1005, '2026-05-03 13:00+00')
  ON CONFLICT DO NOTHING RETURNING post_id
)
UPDATE posts SET like_count = like_count + 1 WHERE post_id IN (SELECT post_id FROM ins)
RETURNING post_id, like_count;
-- ...the same statement again a minute later...
COMMIT;
```

Output:

```
 post_id | like_count
---------+------------
    1005 |          2
(1 row)

 post_id | like_count          <- the repeat like changed nothing
---------+------------
(0 rows)
```

(`ON CONFLICT DO NOTHING` silently skips an insert that would violate a unique key; the **CTE** — `WITH ins AS (…)`, a named sub-query — returns a row only if the insert happened.) `04_instagram.sql` also finds mutual follows (a **self-join** — a table joined to itself — of `follows` with the columns swapped), prints the comment thread under a post indented by depth with a recursive CTE over `parent_id` (the technique of Chapter 7), and ranks hashtags by posts in the last three days. The file's last query is a **reconciliation check** that recounts likes and lists posts whose cached `like_count` disagrees (it returns 0 rows). Every denormalized counter deserves one.

Indexes

Access path	Index
Whom does X follow?	`follows` PK `(follower_id, followee_id)`
Who follows X?	`follows (followee_id)`
X's posts, newest first (profile grid, feed merge)	`posts (user_id, created_at DESC, post_id DESC)`
Has X liked this post? / who liked it	`likes` PK `(post_id, user_id)`
Posts with a tag	`post_hashtags` PK `(tag_id, post_id)`

Design review

The classic defect in a social schema is writing follow and like as 1:N: both are M:N and need a uniqueness rule on the pair (here the composite PKs), plus `CHECK (follower_id <> followee_id)`. The PK order decides which direction each index serves, so `follows (followee_id)` and `group_members (user_id)` exist for the reverse questions. Follow rows carry `created_at` ("since when?"), comments carry `parent_id` for replies, and media is a table because a post can hold several.

Scaling notes

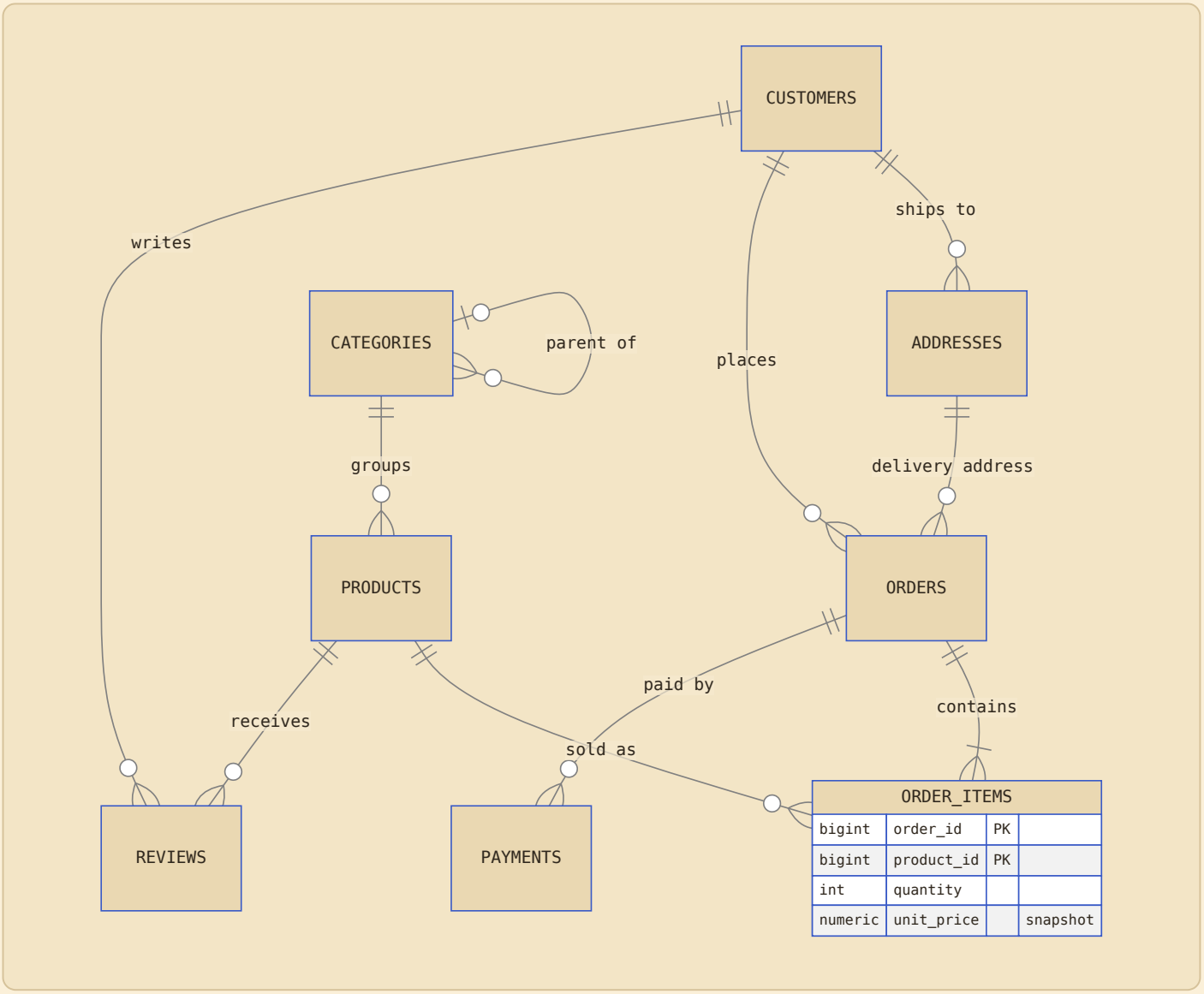
- **Media** lives in object storage (S3-like) behind a **CDN** (content delivery network — servers around the world that cache files close to users; see System Design Fundamentals); rows keep keys only.
- **Feed:** fan-out on read is fine for small follow counts. At scale, **fan-out on write** pushes each new post id into every follower's precomputed feed list (often Redis or a wide-column store); celebrities with millions of followers are handled by fan-out on read, merged at request time — the classic hybrid (Chapter 16).
- **Counters** are the hottest rows: a viral post turns `UPDATE posts SET like_count = …` into a lock queue (**lock contention**: many transactions waiting for the same row). Batch increments in a cache and flush periodically, or keep sharded counter rows and sum them.
- **Sharding** by `user_id` keeps a user's posts, likes-given and follows together; ids come from a generator that embeds time (Snowflake-style — Twitter's scheme of timestamp + machine id + counter) so they sort by creation without a central sequence (Chapter 15 §15.3).

17.5 E-commerce store (runnable)

Requirements

- **Customers** register, keep several **addresses**, browse **products** (each identified by a **SKU**, stock-keeping unit — the shop's product code) organized in a **category tree**, and read/write **reviews** (one per customer per product).
- A customer places an **order** of several products; each **order item** records quantity and the price *at purchase time*.
- Stock is decremented on purchase and may never go negative. Orders move through `pending → paid → shipped → delivered` (or `cancelled`); **payments** attach to orders.
- Discontinued products disappear from the shop but old orders still show them.
- Each order has one **delivery** record (status, date, partner) and may use one **coupon** (code, discount, expiry, usage limit).
- Non-functional: browsing is read-heavy; checkout must be strictly consistent.
- Questions: a customer's orders newest first, the lines of an order and their prices, products in a category with stock, a product's average rating, undelivered orders, whether a coupon code is valid.

Entities and relationships



`order_items` is the junction between `orders` and `products` (M:N) carrying its own attributes. `categories.parent_id` is a self-reference that forms a tree (Electronics → Audio → Headphones).

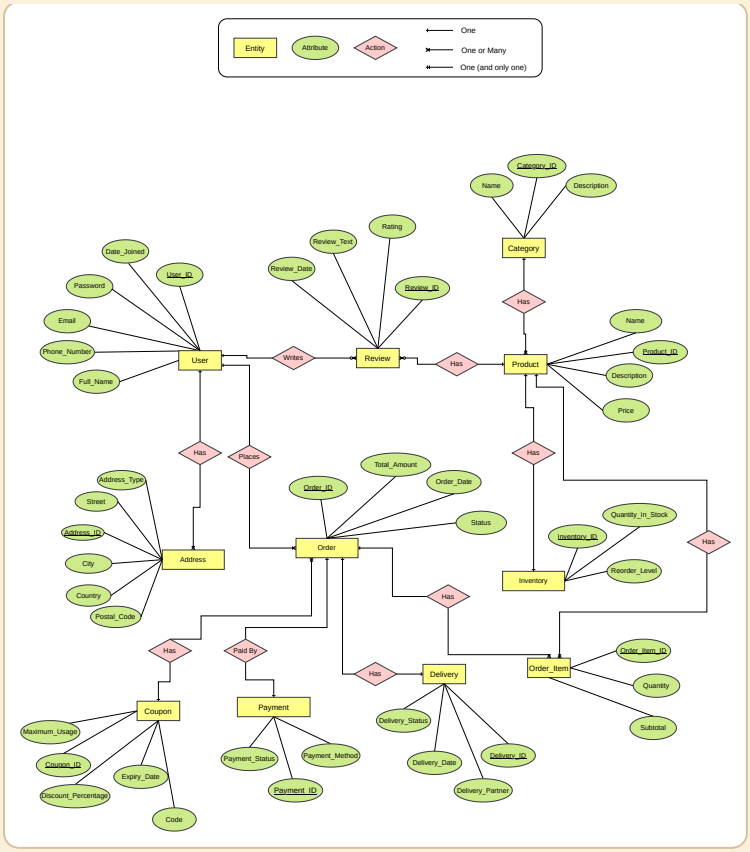


Figure — the course's e-commerce ER diagram with eleven entities, including inventory, delivery and coupon (course diagram)

Not in the runnable schema but part of the full design: `deliveries (order_id UNIQUE REFERENCES orders, status, delivered_on, partner)` and `payments` with `order_id UNIQUE` if exactly one payment per order is the rule — the `UNIQUE` is what makes each 1:1; `coupons (code UNIQUE, discount_pct, expires_on, max_uses)` with a **nullable** `orders.coupon_id` (partial participation); stock either on `products` (as here) or in a 1:1 `inventory` table keyed by `product_id` with a reorder level; and `password_hash` on customers.

Key tables

```
CREATE TYPE order_status AS ENUM ('pending', 'paid', 'shipped', 'delivered', 'cancelled');

CREATE TABLE products (
  product_id  bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  sku         text NOT NULL UNIQUE,
  name        text NOT NULL,
  category_id int NOT NULL REFERENCES categories,
  price       numeric(10,2) NOT NULL CHECK (price >= 0),
  stock       int NOT NULL CHECK (stock >= 0),     -- overselling becomes impossible
  deleted_at  timestamptz                          -- soft delete
);
CREATE TABLE orders (
  order_id    bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  customer_id bigint NOT NULL REFERENCES customers,
  ship_to     bigint NOT NULL REFERENCES addresses,
  status      order_status NOT NULL DEFAULT 'pending',
  placed_at   timestamptz NOT NULL,
  total       numeric(12,2) NOT NULL DEFAULT 0      -- denormalized sum of the items
);
CREATE TABLE order_items (
  order_id   bigint REFERENCES orders ON DELETE CASCADE,
  product_id bigint REFERENCES products,
  quantity   int NOT NULL CHECK (quantity > 0),
  unit_price numeric(10,2) NOT NULL,
  PRIMARY KEY (order_id, product_id)
);
CREATE INDEX ON orders (customer_id, placed_at DESC);
CREATE INDEX ON products (category_id) WHERE deleted_at IS NULL;   -- partial: live products only
```

Three decisions worth saying out loud in an interview:

- **Price snapshot.** `order_items.unit_price` copies the price when the order is placed. It is not a normalization violation: the price *paid* is a different fact from the price *now*.
- **Soft delete.** Deleting a product row would break the foreign keys of old orders (or **cascade** — automatically delete — the history with it). `deleted_at` hides it from the shop instead; the **partial index** (an index over only the rows matching a `WHERE`) keeps the live catalogue lookups small.
- **Status enum.** A PostgreSQL `ENUM` (a custom type allowing only a fixed list of values; or a `CHECK … IN (…)`, or a **lookup table** — a small table of allowed values referenced by a foreign key) limits `status` to known values. Enums are compact but adding a value needs `ALTER TYPE`; a lookup table is the most flexible (MySQL has inline `ENUM`; SQL Server and SQLite use `CHECK`).

Key queries

Placing an order is one transaction: create the order, decrement stock **only where enough stock exists**, copy the prices, then set the total (`01_ecommerce.sql`):

```
BEGIN;
INSERT INTO orders (customer_id, ship_to, placed_at)
VALUES (2, 2, '2026-05-05 12:00+00') RETURNING order_id \gset
WITH cart(product_id, qty) AS (VALUES (1::bigint, 2), (3::bigint, 1)),
reserved AS (
  UPDATE products p SET stock = p.stock - c.qty
  FROM cart c
  WHERE p.product_id = c.product_id AND p.stock >= c.qty AND p.deleted_at IS NULL
  RETURNING p.product_id, c.qty, p.price
)
INSERT INTO order_items (order_id, product_id, quantity, unit_price)
SELECT :order_id, product_id, qty, price FROM reserved
RETURNING order_id, product_id, quantity, unit_price;
-- the application compares the row count with the cart size and issues ROLLBACK if short
UPDATE orders SET total = (SELECT sum(quantity * unit_price) FROM order_items WHERE order_id = :order_id)
WHERE order_id = :order_id RETURNING order_id, total;
COMMIT;
```

Output:

```
 order_id | product_id | quantity | unit_price
----------+------------+----------+------------
        5 |          1 |        2 |      89.00
        5 |          3 |        1 |      29.99
(2 rows)

 order_id | total
----------+--------
        5 | 207.99
```

(`\gset` is a psql command that stores the returned `order_id` in a variable; application code would read it from the `RETURNING` result.) Because the `UPDATE` both checks and decrements stock in one statement, two concurrent checkouts for the last unit cannot both succeed: the second one waits for the first's **row lock** (released when the first transaction commits or rolls back — `ROLLBACK` undoes everything since `BEGIN`), then re-checks `stock >= qty` on the new value (Chapter 13 §13.7). The `CHECK (stock >= 0)` is the last line of defence:

```
psql:01_ecommerce.sql:35: ERROR:  new row for relation "products" violates check constraint
"products_stock_check"
DETAIL:  Failing row contains (2, SP-200, Bluetooth speaker, 2, 45.50, -1, null).
```

A best-seller report still counts the soft-deleted earbuds (2 units, 19.98), because history must not change when the catalogue does. The file also walks the category tree with a recursive CTE (all live products under Electronics at any depth) and runs a **payment reconciliation**: orders marked paid whose payments do not add up to the total. It finds order 3 (total 45.50, paid 30.00) — the kind of mismatch a nightly job should flag.

Indexes

`orders (customer_id, placed_at DESC)` serves "my orders"; `order_items (product_id)` serves sales per product (the PK already covers lookups by `order_id`); the partial index on live products serves category pages. Product search by text belongs in full-text search (Chapter 14) or a search engine.

Design review

- **Normal forms.** `order_items` and `reviews` have two-column candidate keys and every other column depends on the whole pair, so 2NF holds. A stored line `subtotal = quantity × unit_price` would break 3NF (and BCNF) on purpose; `orders.total` is the same redundancy across two tables — kept for fast order lists and checked by a reconciliation query.

- **Freeze what the order promised.** The unit price is snapshotted; the shipping address should be too (copy it onto the order), or editing an address rewrites where old orders "went".
- **Carts** look like orders (`carts` + `cart_items`) but read the *live* price every time.
- **Still missing:** counting coupon uses against `max_uses` (per customer or overall — concurrent checkouts must lock or count atomically) and an order status history.

Scaling notes

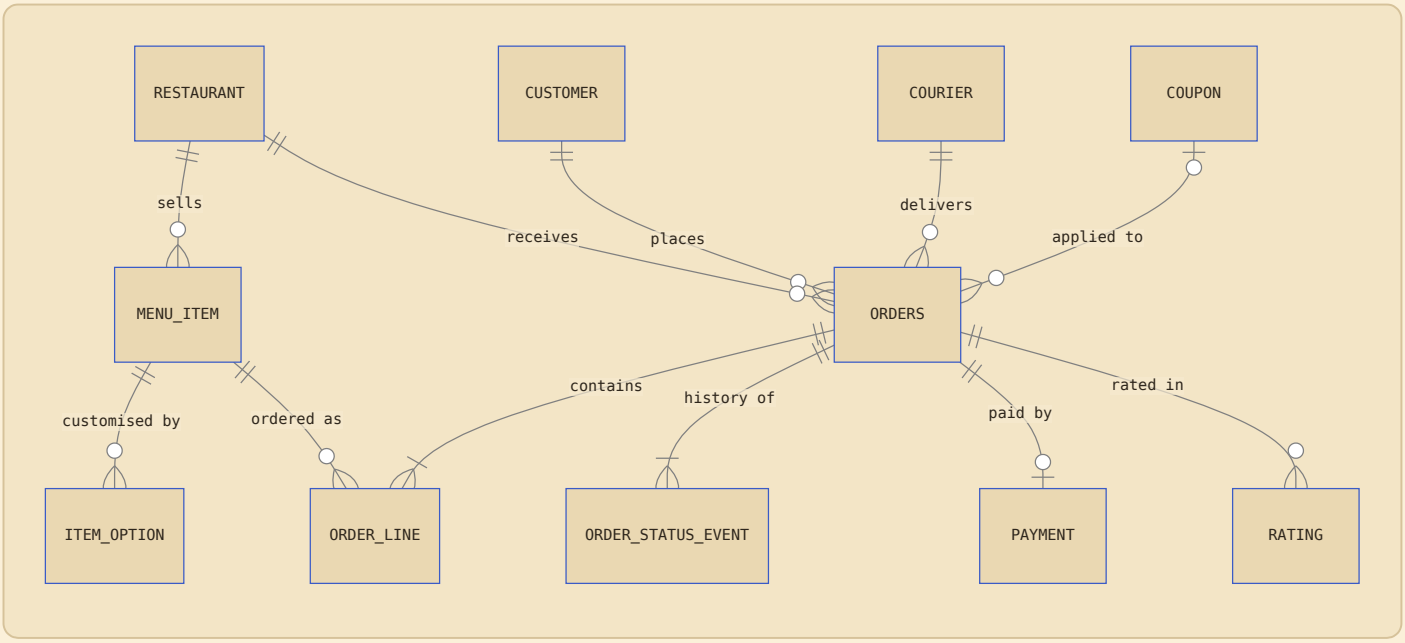
The catalogue is read-mostly: cache it and serve it from replicas or a search index. Carts are short-lived and **high-churn** (created and discarded constantly) — a **key-value store** (a database that just maps a key to a value, like Redis) with expiry fits them well. Orders and payments stay relational and ACID; partition `orders` by `placed_at` month for archiving and shard by `customer_id` when a single primary is saturated. For flash sales, a single **hot** (very frequently updated) `stock` row becomes the **bottleneck** (the one part that limits overall speed) — pre-split inventory into several rows (buckets) or reserve stock in a fast counter and settle in the database.

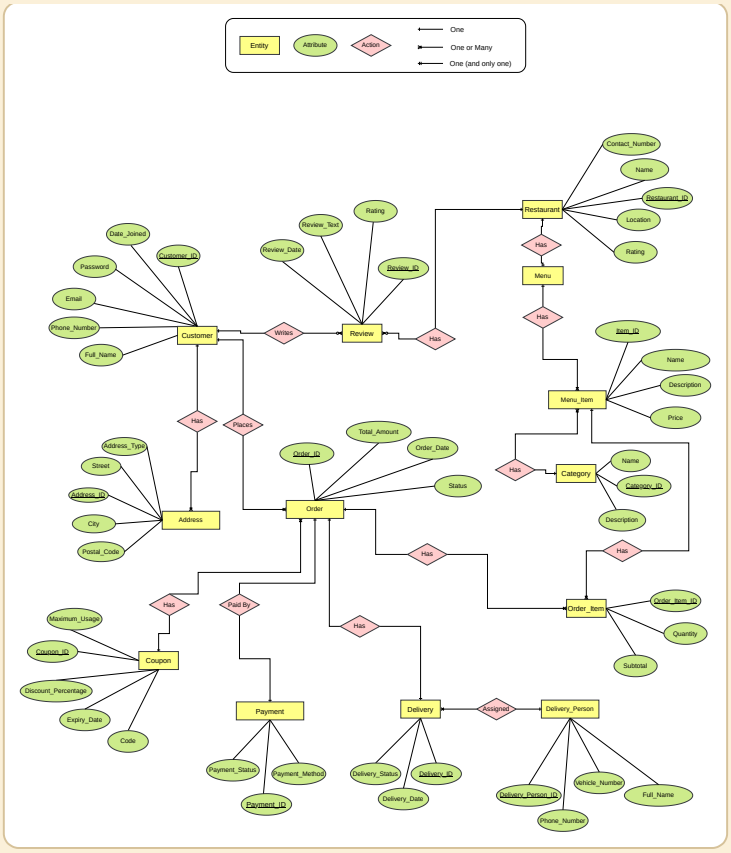
17.6 Food delivery platform

Requirements

- **Customers** order from **restaurants**; each restaurant has a **menu** of **items** (with options such as size or toppings) and opening hours.
- An **order** contains items from one restaurant; it moves `placed → accepted → preparing → picked_up → delivered` (or `cancelled`), and every change is time-stamped.
- A **delivery partner** (courier) is assigned to each order; their location updates every few seconds while active.
- **Payments**, **ratings** for restaurant and courier, and **promotions/coupons**. Customers keep several delivery **addresses**; menu items belong to **categories**.
- Non-functional: lunch and dinner peaks, live tracking, search by distance.
- Questions: today's menu of a restaurant, a customer's orders this month, the lines of an order, where an order is and when it was picked up, a courier's deliveries today, a restaurant's reviews.

Entities and relationships




Entity
Attribute
Action
One
One or Many
One (and only one)
Restaurant
Contact_Number
Name
Restaurant_ID
Location
Rating
Has
Menu
Has
Menu_Item
Item_ID
Name
Description
Price
Has
Category
Name
Category_ID
Description
Has
Has
Has

(for **SLAs** — service-level agreements, promised targets such as "delivered within 40 minutes" — disputes, and delivery-time analytics).

Key queries

1. Open restaurants within 5 km that serve "pizza" — needs a **geospatial** index (one that understands map coordinates; PostGIS, Postgres's geography extension, `geography` + GiST; see Chapter 16 §16.8).
2. A restaurant's live queue: `WHERE restaurant_id = $1 AND status IN ('placed','accepted','preparing') ORDER BY placed_at`.
3. Nearest available courier for an order (geo query over the couriers' latest positions).
4. Average prep time per restaurant: time between the `accepted` and `picked_up` events.

Indexes

`orders (restaurant_id, status)` partial `WHERE status NOT IN ('delivered','cancelled')` keeps the live-queue index tiny; `orders (customer_id, placed_at DESC)`; a GiST index on restaurant location.

Design review

A stored line subtotal and a stored `restaurants.rating` (instead of `avg()` over reviews) are the deliberate redundancies — name both. Payments should carry their own amount (partial payments, refunds, tips); a review should reference an order, or anyone can review a restaurant they never ordered from; a coupon's usage limit needs an atomic count, because two simultaneous checks in application code both pass; and the delivery address is snapshotted (`delivery_addr jsonb`) so editing an address never moves last month's order.

Scaling notes

Courier locations are high-frequency, short-lived writes — keep them in an **in-memory store** (a database that keeps data in RAM for speed) with geo support (Redis `GEOADD`/`GEOSEARCH`) and write only trip summaries to PostgreSQL. Shard orders by city/region: almost no order crosses a city. Menus are read-heavy and cacheable.

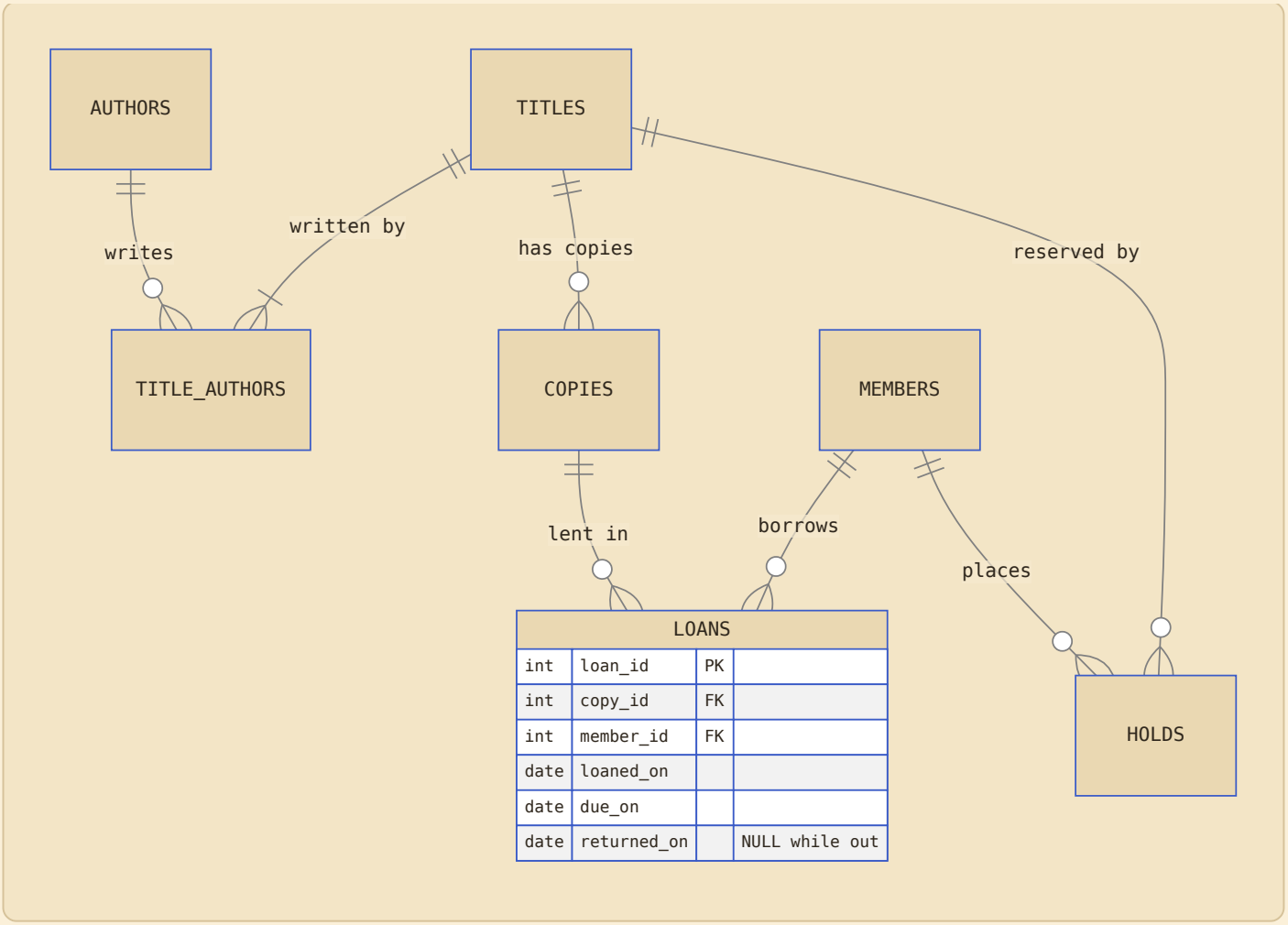
17.7 Library management system (runnable)

Requirements

- The catalogue lists **titles** (identified by **ISBN**, the international book number) with one or more **authors** in a fixed order; the library owns several physical **copies** of a title across **branches**.
- **Members** borrow copies (**loans**) for a fixed period; each member has a maximum number of open loans.
- A copy can be on at most one open loan. Late returns incur a **fine** per day.
- Members place **holds** (reservations) on a title when no copy is free; holds are served first-come, first-served.
- Lost or discarded copies leave the catalogue but their loan history stays.
- Questions: can a member get this title today; which of a member's loans are late; the lending history of one physical copy; the order of the hold queue; a member's unpaid total.

Entities and relationships

The library is the cleanest example of **type versus instance**: a *title* is the abstract book, a *copy* is the object on the shelf. Loans point at copies; holds point at titles (you want *any* copy).



Key tables

```
CREATE TABLE title_authors (                        -- M:N junction with an ordering column
  isbn char(13) REFERENCES titles, author_id int REFERENCES authors,
  position smallint NOT NULL,
  PRIMARY KEY (isbn, author_id)
);
CREATE TABLE copies (
  copy_id int PRIMARY KEY, isbn char(13) NOT NULL REFERENCES titles,
  branch text NOT NULL, condition text NOT NULL DEFAULT 'good',
  withdrawn_at date                                 -- soft delete
);
CREATE TABLE loans (
  loan_id int GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  copy_id int NOT NULL REFERENCES copies, member_id int NOT NULL REFERENCES members,
  loaned_on date NOT NULL, due_on date NOT NULL, returned_on date,
  CHECK (due_on > loaned_on),
  CHECK (returned_on IS NULL OR returned_on >= loaned_on)
);
-- At most ONE open loan per copy:
CREATE UNIQUE INDEX one_open_loan_per_copy ON loans (copy_id) WHERE returned_on IS NULL;
CREATE TABLE holds (
  isbn char(13) REFERENCES titles, member_id int REFERENCES members,
  placed_at timestamptz NOT NULL,
  PRIMARY KEY (isbn, member_id)                     -- one hold per member per title
);
```

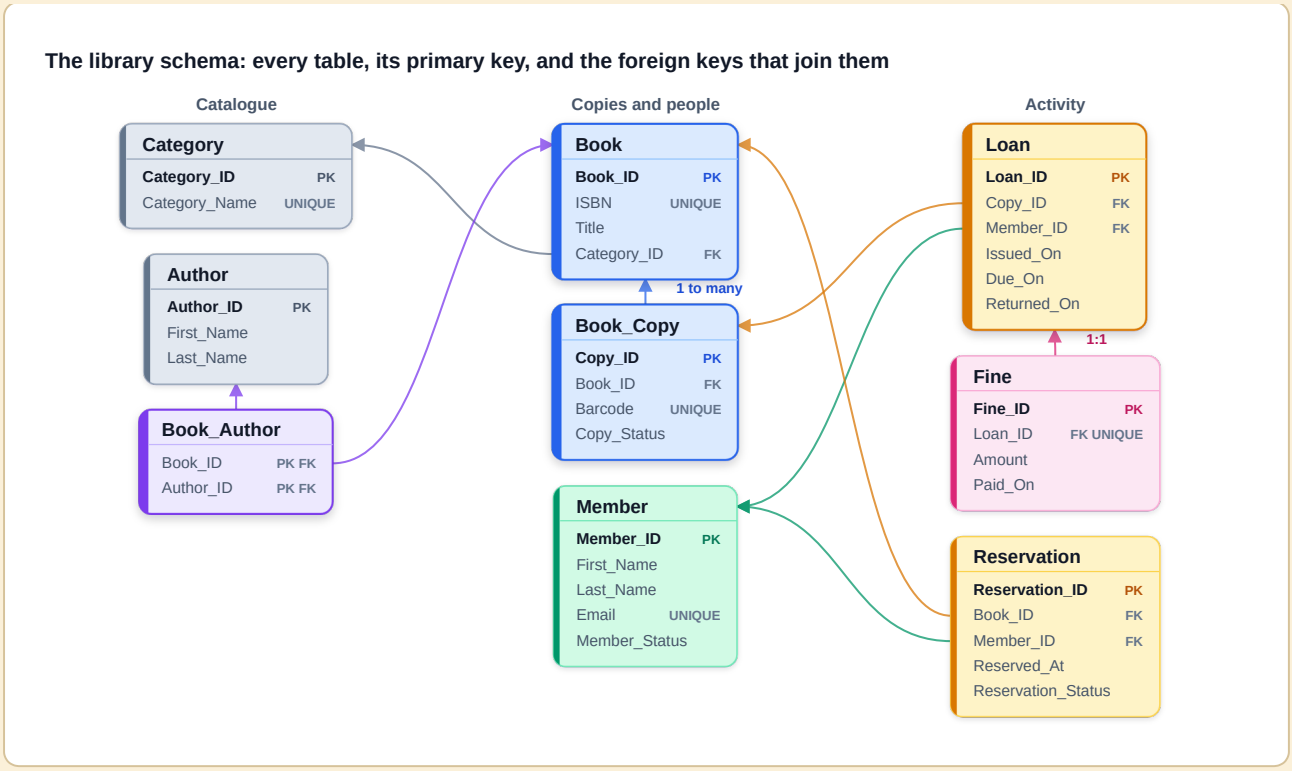


Figure — the course's library model: Book and Book_Copy at the centre, with loans pointing at copies and reservations at books (course diagram)

The tell-tale pair of foreign keys: `loans.copy_id` **but** `holds.isbn` — members borrow one physical object but reserve a title. A copy's `copy_id` plays the role of its barcode. The schema also has `fines (loan_id PRIMARY KEY REFERENCES loans, amount, paid_on)`: using the loan id as the key makes it 1:1 (at most one fine per loan), and the amount is **stored** although it looks derived, because a later change of the daily rate must not rewrite old fines. The course version gives books a **surrogate** `book_id` (an artificial id with no business meaning, as opposed to a **natural key** such as the ISBN) with `isbn UNIQUE` (shorter FKs, and some items have no ISBN) and a `category` table; authors live in a junction table rather than a comma-separated `authors` column, which would break 1NF and turn "books by this author" into a text search.

Note what is **not** stored: a copy's "available" flag. Availability is derived from "no open loan and not withdrawn", so it can never disagree with the loans table. The **partial unique index** is the star: it enforces a rule about *some* rows (open loans) that a plain `UNIQUE` cannot, and it doubles as the index for "is this copy out?". (SQLite and PostgreSQL support partial indexes; SQL Server calls them *filtered* indexes; MySQL has neither — there you would use a **generated column** (a column computed automatically from others) that is non-NULL only for open loans and put a unique index on it.)

Key queries

The catalogue view — authors in order, and live availability (`03_library.sql`, "today" fixed at 2026-05-10):

```
SELECT t.title,
       string_agg(a.name, ', ' ORDER BY ta.position) AS authors,
       (SELECT count(*) FROM copies c
         WHERE c.isbn = t.isbn AND c.withdrawn_at IS NULL
           AND NOT EXISTS (SELECT 1 FROM loans l WHERE l.copy_id = c.copy_id AND l.returned_on IS NULL)
       ) AS available
FROM titles t JOIN title_authors ta USING (isbn) JOIN authors a USING (author_id)
GROUP BY t.isbn
ORDER BY t.title;
```

Output:

```
         title         |      authors       | available
-----------------------+--------------------+-----------
 Joins at Scale        | J. Ortiz, S. Novak |         0
 Query Plans Explained | R. Kumar           |         1
 Tables and Keys       | R. Kumar           |         1
```

(`Query Plans Explained` has two copies but one is withdrawn; `Tables and Keys` has two, one of them out.) Overdue loans and fines come straight from dates — `date - date` is a whole number of days in PostgreSQL:

```
 name |      title      |   due_on   | days_late | fine
------+-----------------+------------+-----------+------
 Omar | Tables and Keys | 2026-05-04 |         6 | 1.50
```

Trying to lend copy 101 while it is still out hits the partial unique index:

```
psql:03_library.sql:32: ERROR:  duplicate key value violates unique constraint "one_open_loan_per_copy"
DETAIL:  Key (copy_id)=(101) already exists.
```

Check-out with the member's limit is a conditional insert — it inserts only if the member has fewer open loans than `max_loans`:

```
INSERT INTO loans (copy_id, member_id, loaned_on, due_on)
SELECT 102, 2, :today, date :today + 14
WHERE (SELECT count(*) FROM loans WHERE member_id = 2 AND returned_on IS NULL)
      < (SELECT max_loans FROM members WHERE member_id = 2)
RETURNING loan_id, copy_id, member_id, due_on;
```

Output: `7 | 102 | 2 | 2026-05-24`. Under concurrency two check-outs by the same member could both see "1 open loan"; lock the member row first (`SELECT … FROM members WHERE member_id = 2 FOR UPDATE`) to serialize them (make them run one after the other; Chapter 13 §13.8 explains this **write-skew** shape — two transactions each read the same state, then both write based on it). The hold queue is a **window function** (a function that computes over a set of related rows without collapsing them, Chapter 8) — `row_number() OVER (PARTITION BY isbn ORDER BY placed_at)` — giving Eva position 1 and Omar position 2 for *Joins at Scale*.

Indexes

The partial unique index (open loan per copy), `loans (member_id) WHERE returned_on IS NULL` (a member's current loans), `copies (isbn)`, and a full-text index on titles/authors for search.

Design review

Loans are a *history* table — a `current_member_id` column on copies would answer only "who has it now". `title_authors` has no non-key columns beyond `position`, which depends on the whole key, so 2NF holds; ISBN and copy id are candidate keys, so BCNF holds. Still missing: holds that expire (an `expires_at` plus a sweep job when a copy is set aside) and a `CHECK` on copy condition so a typo cannot become a new status.

Scaling notes

A library is small — a single PostgreSQL instance serves a city's whole network. Historical loans can be partitioned by year. The only real concurrency hot spot is the popular new release with a long hold queue; handling "copy returned → offer to the next hold → expire after 3 days" as a small **state machine** (a fixed set of states with allowed transitions between them) in one transaction keeps it correct.

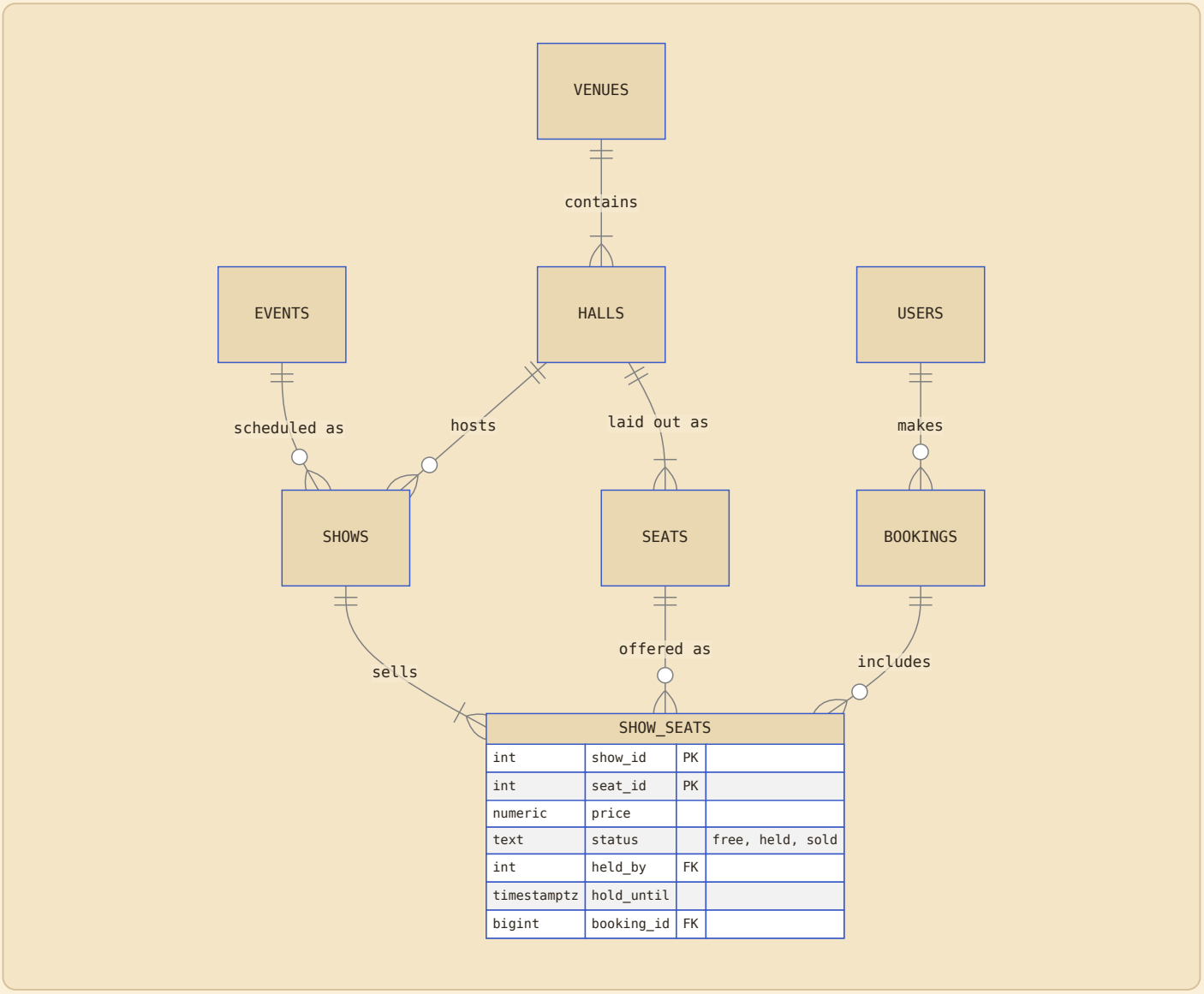
17.8 Ticket booking system (runnable, with seat-hold concurrency)

Requirements

- **Venues** have **halls** (screens/auditoriums) with a fixed **seat** layout (row, number, tier).
- **Events** (films, concerts) run as **shows** at a hall and time. Each seat of each show has a price.
- A user selects specific seats; they are **held** for a few minutes while the user pays. If payment succeeds the seats become **sold** under a **booking**; if the hold expires they return to sale.
- **No seat may ever be sold twice**, even when thousands of people click at the same moment when sales open.
- A booking has at most one **payment**; cancelling a booking releases its seats.
- Questions: free seats for a show, a customer's bookings, what is on in a city today, how full a show was, whether a booking is paid.

Entities and relationships

The key table is `show_seats` — one row per seat *per show*. The physical `seats` table never changes; `show_seats` is the inventory users compete for, and its row lock is the unit of concurrency.



Key tables

```
CREATE TABLE show_seats (
  show_id    int REFERENCES shows,
  seat_id    int REFERENCES seats,
  price      numeric(8,2) NOT NULL,
  status     text NOT NULL DEFAULT 'free' CHECK (status IN ('free', 'held', 'sold')),
  held_by    int REFERENCES users,
  hold_until timestamptz,
  booking_id bigint REFERENCES bookings,
  PRIMARY KEY (show_id, seat_id),               -- a seat appears once per show
  CONSTRAINT held_needs_owner_and_expiry
    CHECK ((status = 'held') = (hold_until IS NOT NULL AND held_by IS NOT NULL)),
  CONSTRAINT sold_needs_booking CHECK ((status = 'sold') = (booking_id IS NOT NULL))
);
CREATE INDEX ON show_seats (show_id) WHERE status = 'free';
```

The two named `CHECK`s tie the status to the columns that must accompany it: a held seat always has an owner and an expiry, a sold seat always has a booking, and a free seat has neither. Impossible states become impossible to store.

Three things all sound like "seat": the physical seat (row H, number 14), **the seat at one screening** (sold at 20:00, free at 18:00), and a claim on it. A single `is_booked` flag on `seats` fails as soon as a hall has a second show. The **price** also belongs to the seat-at-a-show pairing — neither the seat alone (weekends cost more) nor the show alone (recliners cost more) determines it.

Figure — the course's ticket model: Seat → Screening_Seat → Booking_Seat is where the correctness lives (course diagram)

The course models the claim as a separate junction `booking_seats (screening_seat_id PRIMARY KEY, booking_id)`. The trap it highlights: a composite PK `(booking_id, screening_seat_id)` only stops *one booking* claiming a seat twice — two different bookings could still both claim H14. Making `screening_seat_id` the **whole** primary key means a seat can be claimed once, ever, and the second insert fails in the database. That design has no holds and must *delete* the claim rows when a booking is cancelled (marking the booking cancelled would block the seat forever); the `status` column used here

gives the same once-only guarantee through the guarded `UPDATE` below and adds holds. The course also names the table `screening` because `SHOW` is reserved in MySQL; `shows` (plural) is safe.

Key queries

Hold seats with a *guarded* update: the `WHERE` clause only matches seats that are free (or whose hold has expired), and the rows it returns tell you exactly which seats you got (`02_ticket_booking.sql`, "now" fixed at 10:00):

```
UPDATE show_seats ss
SET status = 'held', held_by = 1, hold_until = timestamptz :now + interval '10 minutes'
FROM seats se
WHERE ss.seat_id = se.seat_id AND ss.show_id = 10
  AND se.row_label = 'B' AND se.seat_no IN (2, 3)
  AND (ss.status = 'free' OR (ss.status = 'held' AND ss.hold_until < :now))
RETURNING se.row_label || se.seat_no AS seat, ss.status, ss.hold_until;
```

Output — Asha gets B2 and B3; Ben's request a moment later for B3 and B4 returns only B4, so the app releases it and asks him to choose again:

```
 seat | status |       hold_until
------+--------+------------------------
 B2   | held   | 2026-05-20 10:10:00+00
 B3   | held   | 2026-05-20 10:10:00+00

 seat | held_by          <- Ben asked for B3 and B4
------+---------
 B4   |       2
```

Confirm after payment in one transaction — create the booking, then flip only this user's *unexpired* holds to sold:

```
BEGIN;
INSERT INTO bookings (user_id, show_id, status, created_at)
VALUES (1, 10, 'confirmed', :now) RETURNING booking_id \gset
UPDATE show_seats
SET status = 'sold', booking_id = :booking_id, held_by = NULL, hold_until = NULL
WHERE show_id = 10 AND held_by = 1 AND status = 'held' AND hold_until >= :now
RETURNING seat_id, status, booking_id;
COMMIT;
```

The file continues with "any two free premium seats" using `FOR UPDATE SKIP LOCKED` (lock matching rows but skip any another transaction already has locked) (many buyers each get *different* seats without waiting on each other — the pattern of Chapter 13 §13.12), an expiry sweep that frees stale holds, a failed attempt to mark a seat sold without a booking (`violates check constraint "sold_needs_booking"`), and occupancy per show with `count(*) FILTER (…)` (show 10: 2 sold of 12, revenue 400.00).

Proof on two live sessions

Does the guarded update really stop two *simultaneous* buyers? `05_seat_race.sql` opens two real connections, A and B, with PostgreSQL's `dblink` extension (which lets one session open and run queries on another connection), and runs the same hold on seat 9 in both, at the default `READ COMMITTED` **isolation level** (each statement sees only data committed before it started):

```
--- A: holds seat 9 (C1) but has not committed yet
 seat_id | held_by
---------+---------
       9 |       1
--- B: sends the same UPDATE for user 2; it must wait for A's row lock
 b_still_running | b_waits_on
-----------------+------------
               1 | Lock
--- A commits. B wakes up, re-checks status = 'free' on the new row version, and updates nothing
 rows_b_changed
----------------
              0
--- final state of the seat: held by user 1 only
 seat_id | status | held_by
---------+--------+---------
       9 | held   |       1
```

How it works:

1. A's `UPDATE` locks the row (seat 9) until A commits.
2. B's `UPDATE` finds the same row, sees it is locked, and waits (`wait_event_type = Lock`).
3. When A commits, PostgreSQL re-evaluates B's `WHERE` clause against the **newest** version of the row. `status = 'free'` is now false, so B updates zero rows.
4. The application sees 0 rows returned and tells user 2 "seat taken".

No `SELECT` then `UPDATE` (which *would* race), no application lock, no `SERIALIZABLE` needed. The alternatives: **optimistic locking** with a `version` column (don't lock; at write time check the version is unchanged and retry if it isn't — good when contention is low), or a unique constraint on a separate `seat_holds (show_id, seat_id)` table (insert-to-claim).

Indexes

The PK `(show_id, seat_id)` serves the seat map and every hold; the partial index on free seats serves "best available" and "how many left"; `show_seats (held_by) WHERE status = 'held'` would speed the confirm step; `shows (event_id, starts_at)` for listings.

Two rules remain outside these tables: someone must generate the `show_seats` rows for every new show (300 seats × 6 shows = 1,800 rows a day per hall — a real job), and two shows must not overlap in one hall, which needs an exclusion constraint on `(hall_id, tstzrange(starts_at, ends_at))` or a check in the scheduling transaction.

Scaling notes

The whole problem is a **burst of contention on a small set of rows** (a big concert's on-sale minute). Rows per show are few, so the database can hold them, but connections cannot: put a **virtual waiting room / queue** in front (a holding page that admits users in batches), let only N users into seat selection at a time, and keep holds short. Shows are independent, so shard by `show_id` (or event); a hot show can even get a dedicated shard. Cache the seat map (it may be a second stale) but never the hold itself.

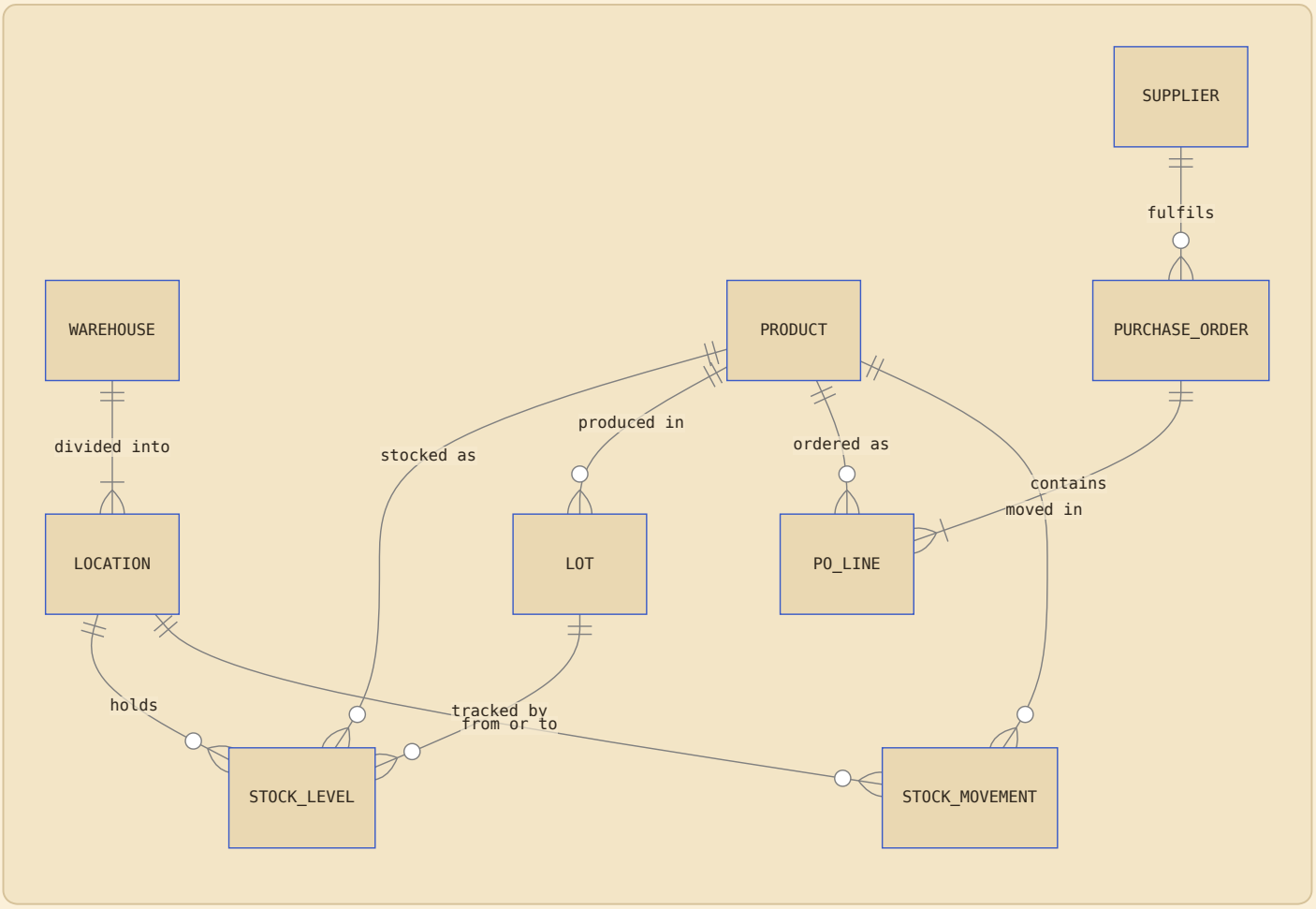
17.9 Warehouse inventory system

Requirements

- **Products** (SKUs) are stored in several **warehouses**, each divided into **bin locations** (aisle-rack-shelf).
- Track **stock on hand** per SKU per location, plus **reserved** quantity for open orders.
- Goods arrive via **purchase orders** from **suppliers** (receipts), leave via **shipments** for sales orders, and move between locations (**transfers**); every movement is recorded.

- Support **batches/lots** (groups of units made together, sharing an expiry date) with expiry dates (first-expired, first-out) and reorder alerts when stock falls below a **reorder point**.
- Each supplier has its own SKU and **lead time** (days from ordering to delivery) for a product; purchase orders have numbered lines.
- Questions: total stock of a SKU and its bin locations; a bin's movements over the past week; the cause of a count mismatch; the quickest supplier for a SKU; open purchase-order quantities.

Entities and relationships



Key tables

```sql
CREATE TABLE stock_levels (                      -- current state, one row per product-lot-location
  product_id  bigint REFERENCES products,
  location_id bigint REFERENCES locations,
  lot_id      bigint REFERENCES lots,
  on_hand     int NOT NULL CHECK (on_hand >= 0),
  reserved    int NOT NULL DEFAULT 0 CHECK (reserved BETWEEN 0 AND on_hand),
  PRIMARY KEY (product_id, location_id, lot_id)
);
CREATE TABLE stock_movements (                   -- append-only ledger of every change
  movement_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  product_id  bigint NOT NULL REFERENCES products,
  lot_id      bigint REFERENCES lots,
  from_location bigint REFERENCES locations,     -- NULL for receipts
  to_location   bigint REFERENCES locations,     -- NULL for shipments / write-offs
  quantity    int NOT NULL CHECK (quantity > 0),
  reason      text NOT NULL CHECK (reason IN ('receipt','shipment','transfer','adjustment','return')),
  reference   text,                              -- PO number, order number ...
  moved_at    timestamptz NOT NULL DEFAULT now(),
  CHECK (from_location IS NOT NULL OR to_location IS NOT NULL)
);
```

This is the same **ledger + cached balance** design as the bank (§17.3): movements are the truth, `stock_levels` is the fast answer, and both change in the same transaction. A periodic query that rebuilds levels from movements and compares them catches bugs — and explains a shelf count that disagrees with the screen by replaying one product's movements in order. Two opposite reasons to store a derivable value: the library fine is kept so it **never** changes; this balance is kept because summing every movement on each read would be too slow.

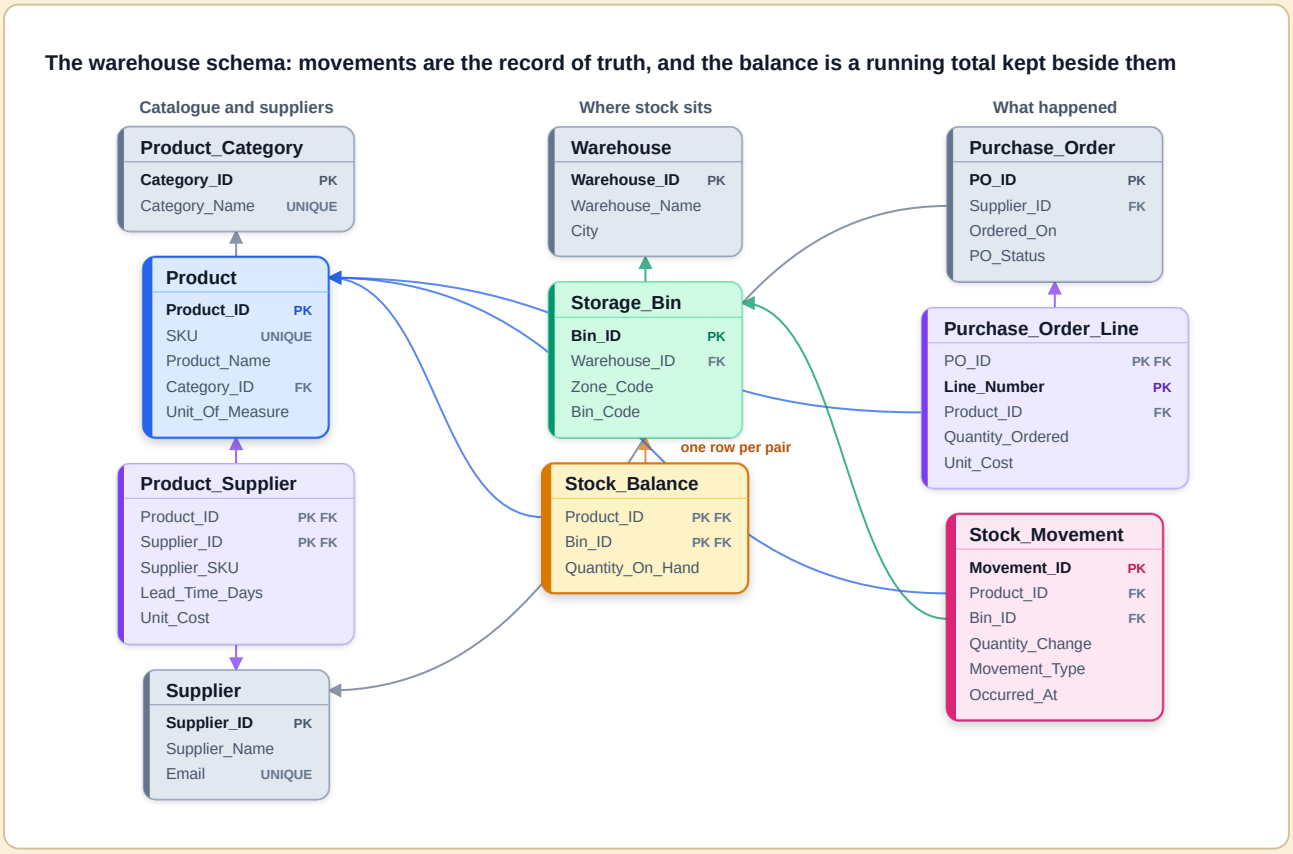


Figure — the course's warehouse model: Stock_Movement records what happened, Stock_Balance is the running total beside it (course diagram)

The course variant records one row per bin with a **signed** `quantity_change` (+50 received, −3 picked) instead of from/to columns — one table for every movement type, and a transfer becomes two rows. Its

other tables: `product_suppliers (product_id, supplier_id, supplier_sku, lead_time_days, unit_cost)` — attributes of the *pairing*, so putting `supplier_name` there would be a 2NF **partial dependency** (a column depending on only part of a composite key); and `purchase_order_lines (po_id, line_no, …)`, a **weak entity** whose key includes its parent's, with `ON DELETE CASCADE`. `unit_cost` appears on both: the agreed price now versus the price paid on that order. A movement's cause is stored as `reference_type` + `reference_id` — a polymorphic reference no FK can check (see §17.14). Whether `on_hand` may drop below zero is a business choice — a few operations like negative stock as an alarm for an unrecorded delivery — so choose deliberately and document it.

Key queries

1. Available to promise for a SKU across warehouses: `sum(on_hand - reserved)` grouped by warehouse.
2. Pick list for an order, first-expired-first-out: stock rows for the SKU joined to `lots`, ordered by `expires_on`, with a running total (window `sum() OVER (ORDER BY expires_on)`) until the ordered quantity is covered.
3. Reorder alerts: products whose total available is below `reorder_point`.
4. Movement history of a lot (for recalls).

Indexes

`stock_levels` PK for picks; `stock_levels (location_id)` for bin counts; `stock_movements (product_id, moved_at DESC)` and `(reference)`; `lots (product_id, expires_on)`.

Scaling notes

Each warehouse is mostly independent: shard or even run one database per site, with a central read model for company-wide stock. Movement tables grow forever — partition by month. Reservations for online orders are the contention point; a reservation row per order line (with expiry, like the ticket holds) is safer than decrementing a shared counter.

The last four cases are large consumer products. The relational core looks like the earlier designs; what changes is that **no single database fits every part**, so each design ends with a short "which store for which data" table (Chapter 16 §16.10 has the general decision guide).

At this scale the template gains three steps before the tables:

1. **Non-functional targets** — **high availability** (the service is almost always up), low **latency** (the time one request takes; here a few hundred ms), **horizontal scalability** (growing by adding machines), and *where **eventual consistency** (copies may briefly disagree but converge) is acceptable* (vote and view counts) versus where it is not (money, bookings, identity).
2. **A back-of-the-envelope storage estimate** — users × bytes per row, items per day × size. Multiply carefully (1 KB × 10 million = 10 GB, not 10 TB); the usual result is that rows are small and **media dominates** by orders of magnitude, which is why media goes to object storage.
3. **A high-level architecture** — clients → **load balancer** (spreads incoming requests over many servers) → **API gateway** (the single entry point that handles routing and authentication) → a few **microservices** (small, separately deployed services: user, content, search …), each owning its data store, with a **message queue** (Kafka, RabbitMQ — a buffer where one service leaves tasks for another to process later) for **asynchronous** work (done in the background, not while the user waits) such as search indexing, notifications and video processing.

After the schema come **sharding** (pick the key that keeps a request on one shard; use **consistent hashing** (a key-to-shard mapping where adding a shard moves only a small share of keys, §15.3) so adding a shard moves little data) and **replication** (primary–replica with automatic failover for the relational part, **leaderless** multi-node replication (any replica accepts writes) for Cassandra-style stores, replicas in several regions) — the mechanics are in Chapter 15.

17.10 Quora (questions and answers)

Requirements

- Users ask **questions**, write **answers**, and **comment** on both; questions carry **topics**.
- Users **upvote/downvote** answers; answers are ranked by votes and quality.
- Users **follow** people, topics and questions; a personalized feed shows new questions and answers from what they follow.
- Full-text **search** over questions; suggestions of similar questions while typing.
- Scale: hundreds of millions of monthly users, far more reads than writes. Vote counts may lag (eventual consistency is fine there).
- Estimate (100 M users at 2 KB; per year 10 M questions at 1 KB, 50 M answers at 2 KB, 500 M votes at 0.5 KB, 10 M media files at 500 KB): about 200 GB of user rows, then roughly 10 GB + 100 GB + 250 GB of text and **5 TB of media** per year — the relational data fits comfortably; media and replicas drive the storage budget.

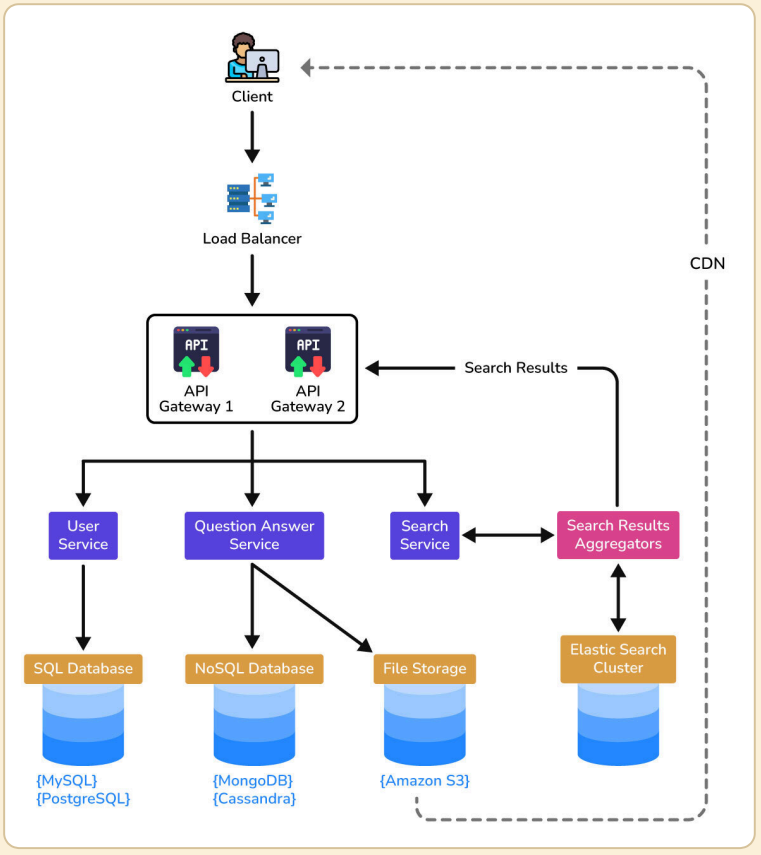
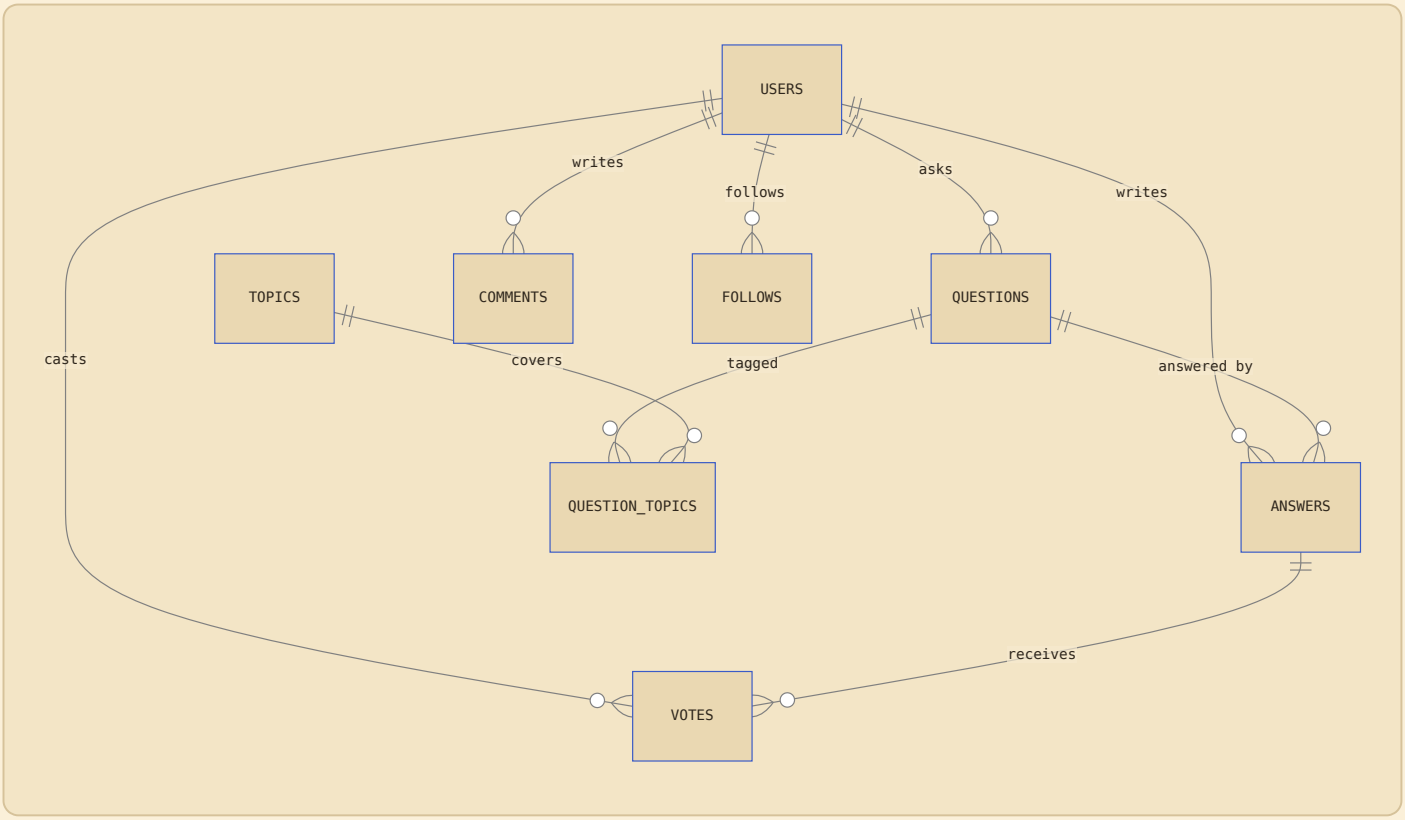


Figure — the course's Quora high-level design: clients, load balancer, API gateway, user / question-answer / search services, a relational DB, a NoSQL store, Elasticsearch, a message queue and file storage (course diagram)

Entities and relationships



Key tables

```sql
CREATE TABLE answers (
  answer_id   bigint PRIMARY KEY,
  question_id bigint NOT NULL REFERENCES questions,
  author_id   bigint NOT NULL REFERENCES users,
  body        text NOT NULL,
  upvotes     int NOT NULL DEFAULT 0, downvotes int NOT NULL DEFAULT 0,   -- denormalized
  created_at  timestamptz NOT NULL, updated_at timestamptz,
  UNIQUE (question_id, author_id)                -- one answer per user per question
);
-- votes (answer_id, user_id, value CHECK (value IN (-1, 1)), PRIMARY KEY (answer_id, user_id))
CREATE TABLE comments (                          -- polymorphic target: a question OR an answer
  comment_id  bigint PRIMARY KEY,
  question_id bigint REFERENCES questions,
  answer_id   bigint REFERENCES answers,
  author_id   bigint NOT NULL REFERENCES users,
  body text NOT NULL, created_at timestamptz NOT NULL,
  CHECK (num_nonnulls(question_id, answer_id) = 1)
);
CREATE TABLE follows (                           -- polymorphic: follow a user, topic or question
  follower_id bigint REFERENCES users,
  target_type text NOT NULL CHECK (target_type IN ('user','topic','question')),
  target_id   bigint NOT NULL,                   -- no FK possible: it points at 3 tables
  PRIMARY KEY (follower_id, target_type, target_id)
);
```

These two tables show the two ways to model a **polymorphic association** (one row that may point to different parent tables). `comments` uses **one nullable FK per possible parent** plus a `CHECK` that exactly one is set — foreign keys still work. `follows` uses a **type + id pair**, which is compact and extensible but gives up referential integrity (the database cannot check that topic 42 exists). Prefer the first when the parent types are few and fixed.

Key queries

1. Answers to a question ranked by score: `ORDER BY upvotes - downvotes DESC, created_at` (or a stored `score` that also weighs author credibility).
2. Feed: new answers in followed topics/questions/people since the last visit.
3. Casting a vote: `INSERT … ON CONFLICT (answer_id, user_id) DO UPDATE SET value = EXCLUDED.value`, adjusting the counters by the difference.

Indexes

`answers (question_id, upvotes DESC)`, `question_topics (topic_id, question_id)`, `follows (target_type, target_id)` for "who follows this topic", full-text (`tsvector` GIN) or an external search engine for questions.

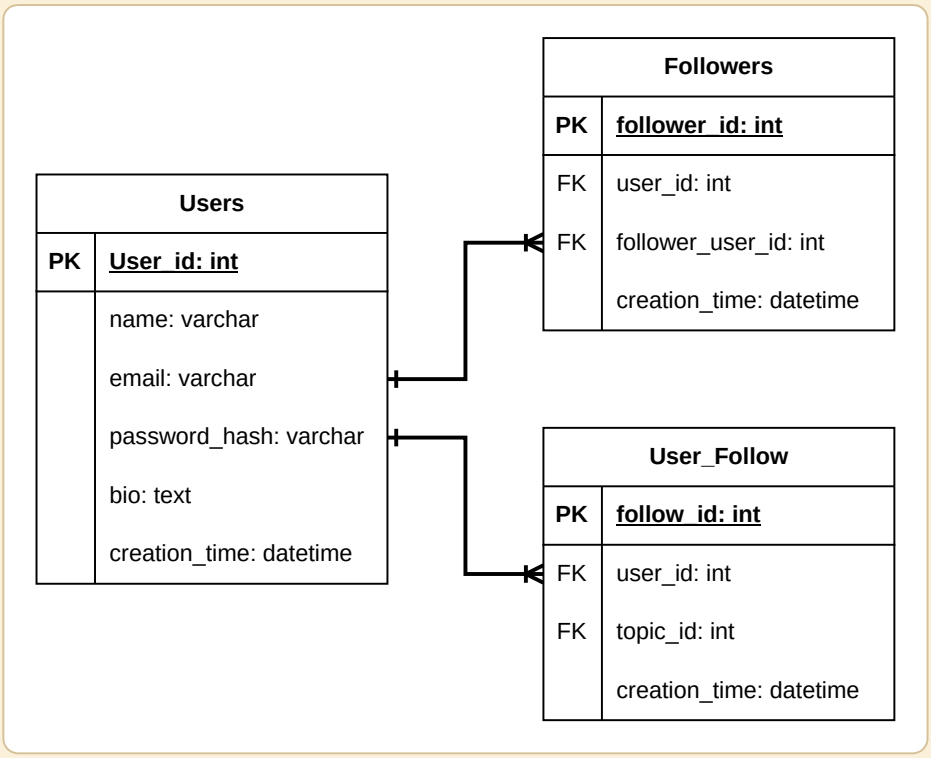


Figure — the course's split: users, follower and topic-follow tables in the relational DB; questions with embedded answers in a document store (course diagram)

The document alternative. The course stores each question as one MongoDB-style document with its answers (and their comments) **embedded**, so a question page is a single read; a mirrored user document carries follower counts and followed topics. Embedding is denormalization: great for "show this question", awkward for "all answers by this user" or for a question with 10,000 answers (MongoDB caps a document at 16 MB) — the usual compromise embeds the top answers and keeps the rest in their own collection.

Sharding. By `user_id`, a user's content is local but celebrity users become **hot spots** (shards overloaded by one popular key); by `question_id` (hashed), load spreads evenly but a user's timeline needs a **scatter-gather** (send the query to every shard and merge the answers) across shards. The recommended mix: question data sharded by a hash of `question_id`, user data (profiles, follows) by a hash of `user_id`, consistent hashing, and a shard-manager service that maps keys to shards.

Which store for which data

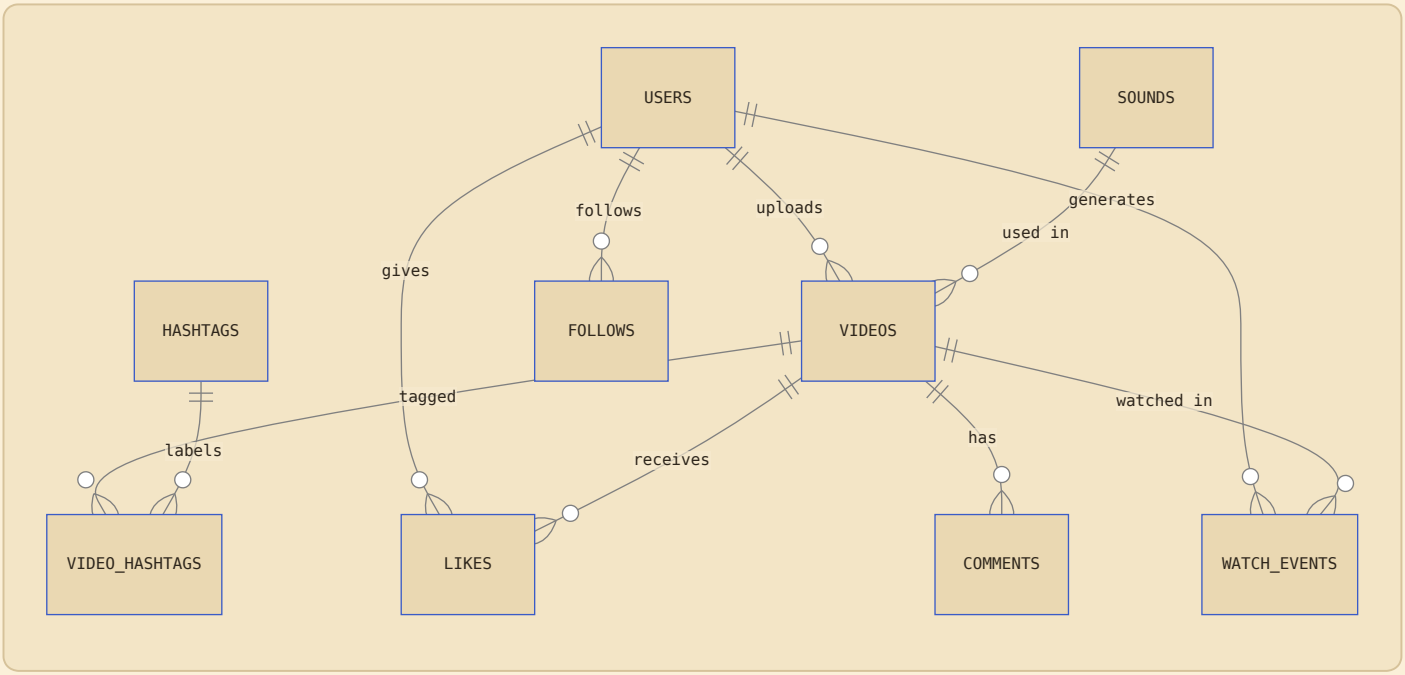
Data	Store
Users, questions, answers, votes	Sharded relational DB (Quora itself runs sharded MySQL — tens of terabytes and hundreds of thousands of queries per second, per its 2020 engineering blog) — or questions/answers in a document or wide-column store, as in the course design
Answer rankings, feed lists, counters	In-memory cache (Redis/Memcached)
Search and "similar question" suggestions	Search engine (Elasticsearch/OpenSearch)
Images in answers	Object storage + CDN

17.11 TikTok (short-video platform)

Requirements

- Users upload short **videos** with captions, **sounds** (music tracks reusable across videos) and **hashtags**.
- Viewers **like**, **comment**, **share**, and **follow** creators; view counts are shown on every video.
- The main surface is a recommendation-driven **"For You" feed**, fed by **watch events** (what was watched, for how long, skipped, replayed).
- Real-time **notifications** (new follower, like, comment); **trending** videos and hashtags; videos grouped into **categories** (dance, comedy).
- Scale: billions of video views per day; view and watch events vastly outnumber everything else.
- Estimate: with generous assumptions (1 B daily users each uploading 2 videos of 50 MB) raw video is 1 B × 2 × 50 MB = **100 PB a day**, against about 4 TB of metadata (2 B videos × 2 KB) and about 2 TB of comments (2 B × 1 KB). Video bytes outweigh the metadata and comments by more than 10,000 to 1, so they live in object storage with **lifecycle rules** (automatic policies that move old files to cheaper storage or delete them), never in a database.

Entities and relationships



Key tables

```
CREATE TABLE videos (
  video_id    bigint PRIMARY KEY,                 -- time-ordered id (Snowflake-style)
  creator_id  bigint NOT NULL REFERENCES users,
  sound_id    bigint REFERENCES sounds,
  caption     text,
  storage_key text NOT NULL,                      -- original upload in object storage
  duration_ms int NOT NULL,
  status      text NOT NULL CHECK (status IN ('processing','public','private','removed')),
  created_at  timestamptz NOT NULL
);
-- video_renditions (video_id, resolution, bitrate_kbps, storage_key): transcoded copies
CREATE TABLE video_stats (                        -- hot counters kept apart from the video row
  video_id bigint PRIMARY KEY REFERENCES videos,
  views bigint NOT NULL DEFAULT 0, likes bigint NOT NULL DEFAULT 0,
  comments bigint NOT NULL DEFAULT 0, shares bigint NOT NULL DEFAULT 0
);
```

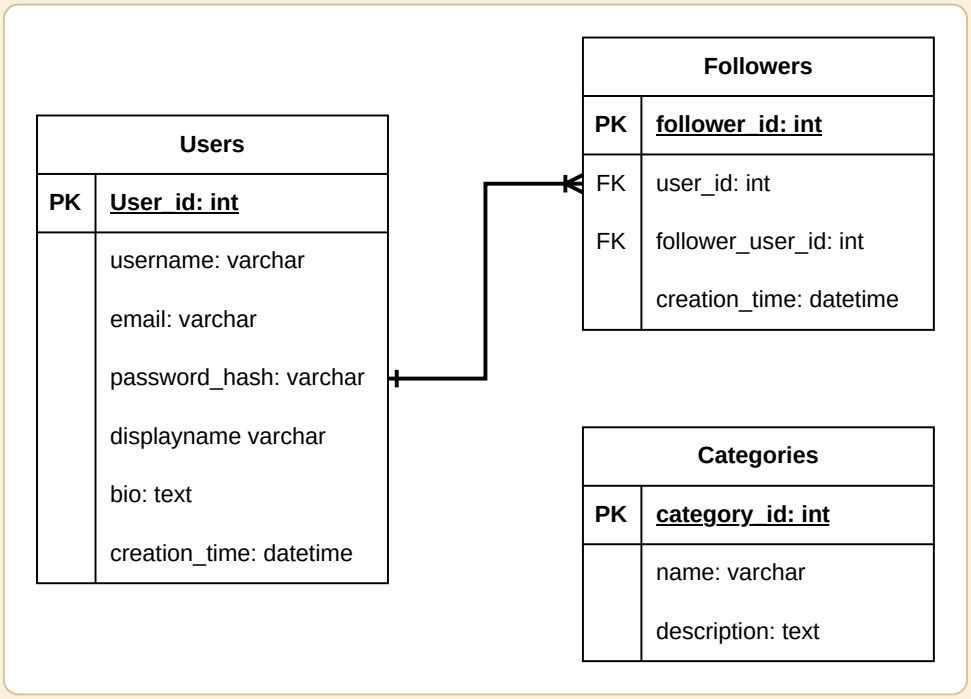


Figure — the course's TikTok schema: users, followers and categories relational; videos, interactions and user activity in Cassandra (course diagram)

In the course design the high-volume tables live in **Cassandra**, modeled around the query rather than the entity. A correct **CQL** (Cassandra Query Language) sketch (Cassandra has no `ENUM` type, and **counter** columns — special columns that can only be incremented or decremented — must sit in a table of their own; a `timeuuid` is a unique id that embeds a timestamp):

```cql
-- CQL (Cassandra) — not run
CREATE TABLE interactions_by_video (
  video_id uuid, created_at timeuuid, user_id bigint,
  kind text,                      -- 'like' | 'comment' | 'share'
  comment_text text, parent_id timeuuid,
  PRIMARY KEY ((video_id), created_at)          -- partition key, then clustering column
) WITH CLUSTERING ORDER BY (created_at DESC);
CREATE TABLE video_counters (
  video_id uuid PRIMARY KEY, views counter, likes counter, comments counter, shares counter
);
CREATE TABLE user_activity (
  user_id bigint, activity_time timestamp, activity_type text, details text,
  PRIMARY KEY ((user_id), activity_time)
) WITH CLUSTERING ORDER BY (activity_time DESC);
```

Relational tables keep what needs strong consistency (accounts, follows, categories); Cassandra keeps what needs write throughput and can be eventually consistent. The same idea appears relationally below.

(`video_renditions` holds **transcoded** copies — the same video re-encoded at several resolutions and **bitrates**, i.e. data per second, for different devices and connections.) Splitting `video_stats` from `videos` is a **vertical partition** (splitting a table's columns into two tables sharing a key): the counters change constantly, the video metadata almost never, so separating them keeps metadata rows and their cache entries stable. Watch events are **not** a table in the primary database at all — they are an **event stream** (an append-only, ordered flow of records, e.g. in Kafka, consumed by other systems).

Key queries

1. A creator's profile grid: `videos WHERE creator_id = $1 AND status = 'public' ORDER BY video_id DESC` (time-ordered ids sort by upload time).
2. Videos using a sound (the "use this sound" page).
3. Trending hashtags in the last hour — computed by the **stream processor** (a system that computes results continuously as events arrive), not by SQL over billions of rows.
4. For You feed — candidate generation and ranking by a recommendation service; the database only fetches metadata for the chosen ids.

Indexes

`videos (creator_id, video_id DESC)`, `videos (sound_id)`, `video_hashtags (hashtag_id, video_id)`, `likes (video_id, user_id)` primary key plus `likes (user_id, created_at)` for "videos I liked".

Which store for which data

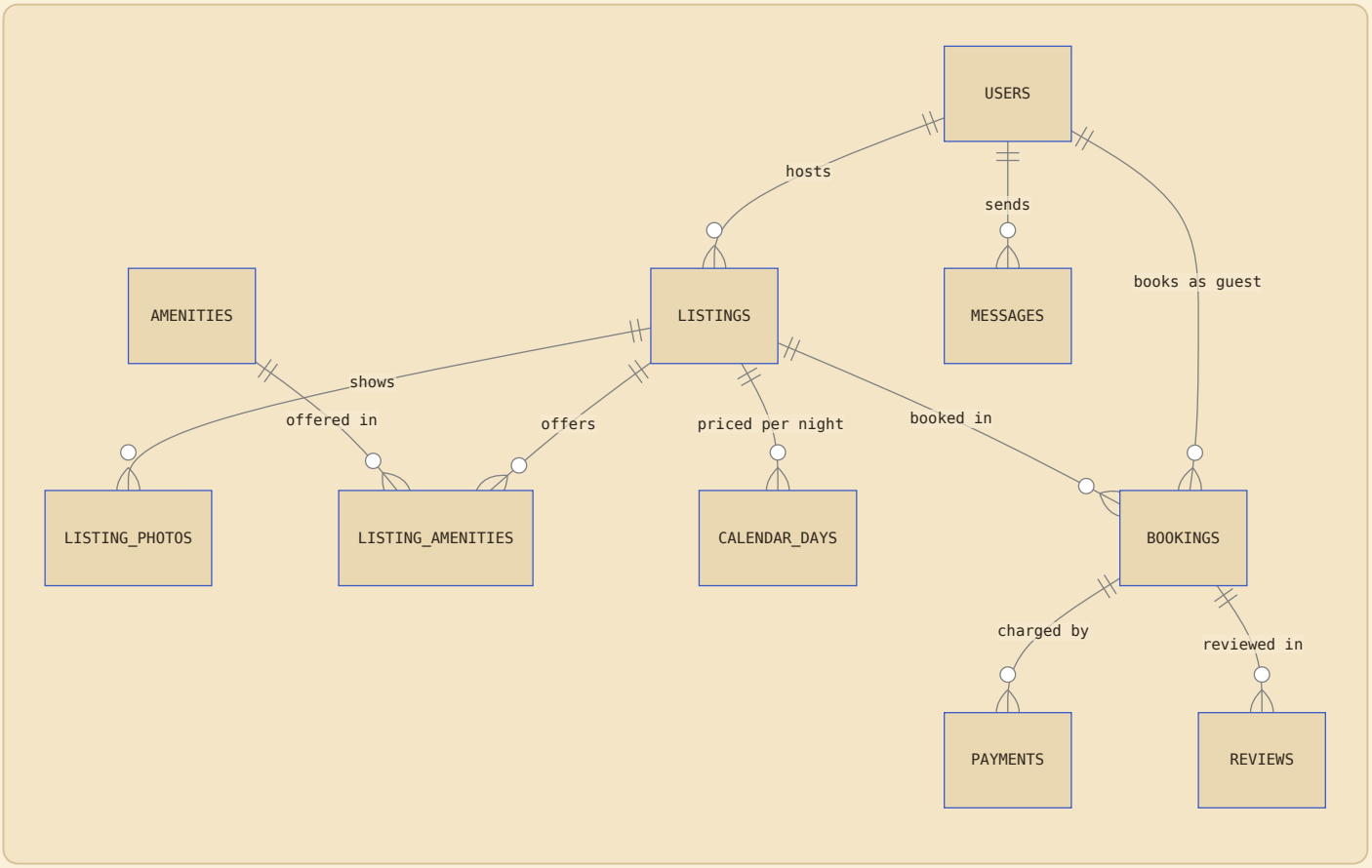
Data	Store
Users, video metadata, follows	Sharded relational or distributed SQL
Video files and renditions	Object storage + CDN (edge caches near viewers)
Views, watch time, skips	Event log (Kafka) → stream processing → analytics warehouse / wide-column store
Counters	In-memory counters, flushed periodically; displayed counts may lag a little
Recommendation features, embeddings (lists of numbers that represent an item so similar items are close)	Feature store (a database of precomputed model inputs) + vector index (finds nearest embeddings; FAISS library, Milvus) fed by training pipelines
Notifications, processing jobs	Message queue → notification and transcoding **workers** (background processes that take jobs from the queue)

17.12 Airbnb (short-term rentals)

Requirements

- **Hosts** publish **listings** (address, geo-location, capacity, amenities, photos, house rules) with a **calendar** of availability and nightly prices.
- **Guests** search by location, dates and guests; filter by price and amenities; then **book** (instant or on host approval).
- No listing may be booked twice for overlapping nights.
- **Payments** (charge the guest, pay out the host later, refunds and deposits through an external **payment gateway** — a third-party service that processes card payments), **reviews** in both directions after a stay, and **messaging** between guest and host. Hosts accept or decline booking requests.
- Estimate (per day: 1 M listings created or updated with ~5 KB of data and 10 photos of ~1 MB; 10 M reservations and 10 M payments at 1 KB; 5 M reviews at 0.5 KB): photos ≈ **10 TB/day**, while reservations, payments and reviews together are only ≈ 22 GB/day — the transactional core is small enough for a relational cluster.

Entities and relationships



A user is both a potential **host** and a **guest**: one `users` table with two roles, not two tables.

Key tables

```sql
-- needs the PostGIS extension for geography (not run here)
CREATE TABLE listings (
  listing_id bigint PRIMARY KEY,
  host_id    bigint NOT NULL REFERENCES users,
  title      text NOT NULL,
  location   geography(Point, 4326) NOT NULL,     -- PostGIS; lat/long for radius search
  max_guests smallint NOT NULL,
  base_price numeric(10,2) NOT NULL,
  status     text NOT NULL CHECK (status IN ('draft','active','snoozed','deleted'))
);
CREATE TABLE calendar_days (                      -- per-night availability and price overrides
  listing_id bigint REFERENCES listings,
  night      date,
  price      numeric(10,2),                        -- NULL = base price
  blocked    boolean NOT NULL DEFAULT false,       -- host blocked it
  PRIMARY KEY (listing_id, night)
);
CREATE TABLE bookings (
  booking_id bigint PRIMARY KEY,
  listing_id bigint NOT NULL REFERENCES listings,
  guest_id   bigint NOT NULL REFERENCES users,
  stay       daterange NOT NULL,
  guests     smallint NOT NULL,
  total      numeric(12,2) NOT NULL,
  status     text NOT NULL CHECK (status IN ('requested','confirmed','cancelled','completed')),
  EXCLUDE USING gist (listing_id WITH =, stay WITH &&) WHERE (status IN ('requested','confirmed'))
);
-- reviews (booking_id, author_role 'guest' | 'host', rating, body,
--          PRIMARY KEY (booking_id, author_role))  -> each side reviews once per stay
```

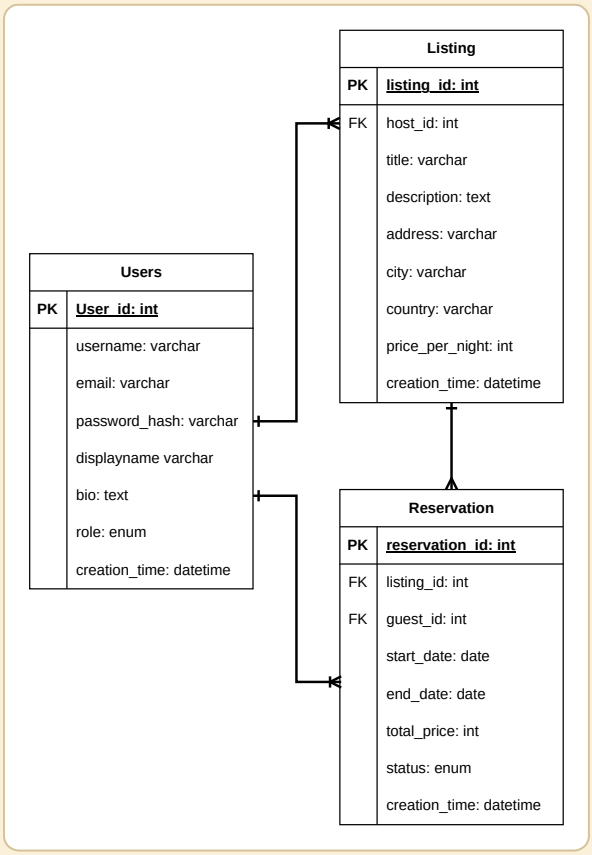


Figure — the course's Airbnb schema: users (with a host/guest role), listings and reservations relational; reviews, user activity and property images in Cassandra (course diagram)

The course splits the work into **user, listing, booking and payment services**. Users, listings, reservations and payments stay relational (atomic booking and money); reviews, activity logs and `property_images (listing_id, image_id, url)` — partitioned by `listing_id` so one read returns a listing's photos — go to Cassandra; photo bytes go to object storage. Its `users.role` column (host or guest) works only if a person is never both; a second listing-owner check (`listings.host_id`) is the more flexible way to express "host".

The **exclusion constraint** on `(listing_id, stay)` is the same tool as the hospital appointments: two active bookings of one listing cannot overlap. Unlike a hotel, each listing is a single unit, so no inventory counter is needed.

Key queries

1. Search: active listings within 10 km (`ST_DWithin(location, $point, 10000)`) with enough capacity and **no** overlapping active booking and no blocked night in the range — plus price filtering.
2. Book: insert the booking; the exclusion constraint rejects a conflicting stay, even under concurrency.
3. Host dashboard: upcoming bookings per listing; payouts due.
4. Listing rating: `avg(rating)` of guest reviews, usually cached on the listing.

Indexes

GiST on `listings.location`; the exclusion constraint's GiST index on `(listing_id, stay)`; `bookings (guest_id, lower(stay))` for "my trips"; `calendar_days` PK.

Which store for which data

Data	Store
Users, listings, bookings, payments	Relational (strong consistency for bookings and money)
Search with geo + filters + ranking	Search engine fed by change-data-capture from the DB
Photos	Object storage + CDN
Messages	Relational or wide-column, partitioned by conversation
Pricing suggestions, analytics	Data warehouse (replicated nodes plus regular backups)

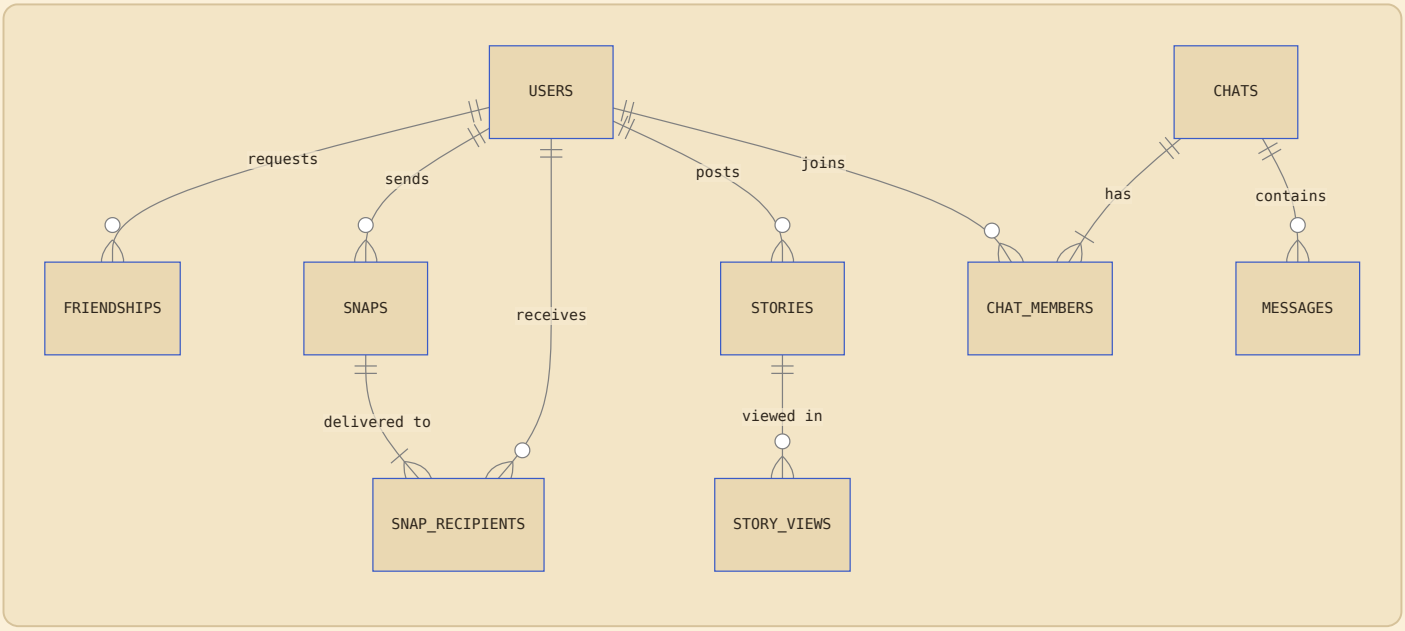
Replication follows the pattern above: a primary with read replicas in several regions and automatic failover for the relational core, multi-node replication with eventual consistency for Cassandra.

17.13 Snapchat (ephemeral messaging)

Requirements

- Users add **friends** (mutual, request/accept), send **snaps** (photo/video) to one or more friends, and **chat**.
- A snap **disappears** after every recipient has opened it, or after a maximum time (e.g. 30 days unopened); a **story** is visible to friends for 24 hours.
- Senders see delivery and "opened" status per recipient; screenshots are reported.
- Snaps and stories can carry **filters/lenses**; users can be **searched** by name.
- Scale: billions of snaps a day; almost all data is short-lived.
- Estimate (500 M daily users, 10 snaps each — 80 % photos of 1 MB, 20 % videos of 5 MB — plus one 10 MB story each): 4 PB + 5 PB of snaps and 5 PB of stories, about **14 PB of new media a day**. Because snaps vanish on viewing and stories after 24 hours, what is *held* stays bounded — deletion is as important a workload as writing.

Entities and relationships



Key tables

```
CREATE TABLE friendships (                        -- one row per pair, stored in canonical order
  user_low   bigint REFERENCES users,
  user_high  bigint REFERENCES users,
  status     text NOT NULL CHECK (status IN ('pending','accepted','blocked')),
  requested_by bigint NOT NULL,
  PRIMARY KEY (user_low, user_high),
  CHECK (user_low < user_high)
);
CREATE TABLE snaps (
  snap_id    bigint PRIMARY KEY,
  sender_id  bigint NOT NULL REFERENCES users,
  media_key  text NOT NULL,                        -- encrypted blob in object storage
  created_at timestamptz NOT NULL,
  expires_at timestamptz NOT NULL                  -- hard deadline even if never opened
);
CREATE TABLE snap_recipients (
  snap_id      bigint REFERENCES snaps ON DELETE CASCADE,
  recipient_id bigint REFERENCES users,
  delivered_at timestamptz, opened_at timestamptz, screenshot_at timestamptz,
  PRIMARY KEY (recipient_id, snap_id)              -- "my inbox" is a PK range scan
);
-- stories (story_id, user_id, media_key, created_at, expires_at = created_at + 24 h, set on insert)
```

(`expires_at` cannot be a `GENERATED … STORED` column: `timestamptz + interval` depends on the session time zone, so PostgreSQL rejects it as not **immutable** — a generated column's expression must always give the same output for the same input.)

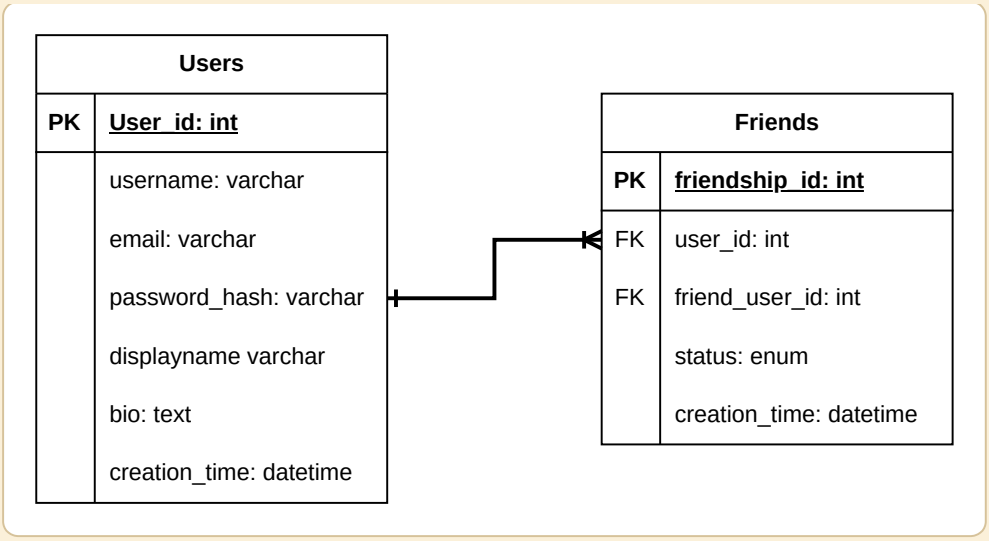


Figure — the course's Snapchat schema: users and friends relational; snaps, stories and messages in Cassandra; users, snaps and stories indexed in Elasticsearch for search (course diagram)

The course's Cassandra `snaps` table is keyed by `snap_id` with a secondary index on the receiver. Cassandra works best when the **partition key is what you query by**, so an inbox table partitioned by recipient (as `snap_recipients` above does relationally) avoids the secondary index. A search engine (Elasticsearch) holds a user index for "find friends", sharded automatically by the number of primary shards you configure.

The **canonical pair** trick (`user_low < user_high`) stores a symmetric relationship once, so "are A and B friends?" is one lookup with `(least(a,b), greatest(a,b))`. Compare Instagram's `follows`, which is directional and therefore needs one row per direction.

Key queries

1. Inbox: unopened snaps for a recipient, newest first.
2. Open a snap: set `opened_at`; when every recipient row has `opened_at`, delete the media and the snap.
3. Friends' stories: stories of accepted friends where `expires_at > now()`.
4. Expiry sweep: delete snaps and stories past `expires_at` (in batches).

Indexes

`snap_recipients` PK `(recipient_id, snap_id)`, `snaps (expires_at)` for the sweeper, `stories (user_id, created_at DESC)`, `friendships (user_high)` for the reverse lookup.

Scaling notes and stores

Ephemeral (short-lived) data is a poor fit for row-by-row `DELETE` at this volume (**dead tuples** — deleted row versions still occupying space — **vacuum**, Postgres's clean-up process for them, and **index bloat**, indexes swollen with entries for dead rows). Two better tools: **time partitions** (partition `snaps` by day and `DROP` whole partitions — instant and bloat-free, Chapter 15 §15.2) or a store with native **TTL** (time-to-live), such as Cassandra/ScyllaDB or DynamoDB, where rows expire by themselves. Snap's 2017 **IPO** (initial public offering — first sale of shares on the stock market) filing disclosed five-year cloud commitments of $2 billion with Google Cloud and $1 billion with AWS — at this scale, managed cloud databases are the usual choice rather than self-run servers.

Data	Store
Accounts, friend graph	Relational or key-value, sharded by user
Snaps, recipients, chat messages	Wide-column / key-value with TTL, partitioned by recipient or chat
Media	Encrypted object storage + CDN, lifecycle rules delete it
Presence ("online now" status), typing indicators	In-memory store
User search	Search engine index

Partitioning happens at three levels: **horizontal** (rows split by `user_id` or `snap_id` — users by range or consistent hashing, trying to keep friend lookups on one shard; Cassandra hashes its partition keys itself), **vertical** (user data and media data in separate stores), and **functional** (each microservice — user, media, messaging, search — owns its data).

17.14 Recurring modeling patterns

Across the thirteen cases the same few patterns did most of the work. This table is the chapter in one page.

Pattern	What it is	Seen in	Use it when	Watch out for
Junction table	A table whose PK is two FKs, resolving an M:N relationship, often with its own attributes	`order_items`, `likes`, `title_authors`, `post_hashtags`, `reservation_rooms`	Any M:N relationship; the PK doubles as the uniqueness rule ("like once")	Add an index on the *second* column for the reverse lookup
Type vs instance	Separate the kind of thing from the physical thing	`titles` / `copies`, `room_types` / `rooms`, `seats` / `show_seats`	Users choose a kind but you track individual items	Forgetting it forces one table to do two jobs
Status enum + state history	A constrained `status` column for the current state; optionally an append-only table of transitions	orders, bookings, reservations, food-delivery events	Every entity with a life cycle	Enforce allowed values (`ENUM`, `CHECK`, or lookup table); `CHECK`s can tie a status to required columns (ticket seats)
Soft delete	A `deleted_at` / `withdrawn_at` column instead of `DELETE`	products, library copies, listings	History or FKs must survive removal	Every query must filter it — use partial indexes and views; unique constraints need `WHERE deleted_at IS NULL`; privacy laws may still require a hard delete
Audit columns / ledger	`created_at`, `updated_at`, `created_by`; or a full append-only ledger of changes	hospital records, bank ledger, stock movements	Legal, financial or medical data; debugging "who changed this?"	Maintain with defaults/triggers, not by hand; ledgers need a cached balance for speed and a reconciliation query
Polymorphic association	One row points to one of several parent types	Quora comments (nullable FKs + `CHECK`), follows (`target_type` + `target_id`)	A comment/like/follow can target several entity kinds	The type+id form loses foreign keys; prefer one nullable FK per parent, or a shared supertype table
Denormalized counter / copy	Store a derived value (count, total, current status) to avoid recomputing	`like_count`, `orders.total`, `room_inventory`, `answers.upvotes`, `video_stats`	A measured hot read	Update it in the same transaction (or trigger), add a reconciliation check, and beware lock contention on hot rows
Snapshot of volatile data	Copy a value that may later change	`unit_price`, delivery address, `nightly_price`	The fact is "what it was at the time"	It is not a normalization bug — document it as intentional
Self-reference	A FK to the same table	category tree, comment replies, `follows`	Hierarchies and graphs	Query with recursive CTEs; guard against cycles
Overlap prevention	Exclusion constraint on a range, or partial unique index on "open" rows	appointments, Airbnb stays, hotel rooms, open library loans	"No two X at the same time"	PostgreSQL-specific; elsewhere lock and check in a transaction

Pattern	What it is	Seen in	Use it when	Watch out for
Guarded update / claim	`UPDATE … WHERE state = 'free' RETURNING …`	seat holds, stock decrement	Many users compete for the same item	Check the returned row count; hold short; `SKIP LOCKED` for "any available"
Time partitioning / TTL	Split big tables by time; drop old partitions or let rows expire	ledgers, movements, snaps, stories	Append-heavy data with a retention rule	Queries should include the partition key in `WHERE`

Judgement calls — cover the answer, say it aloud

1. *`order_items.unit_price` duplicates `products.price` — a 3NF violation?* → No: price paid on that order and today's list price are different facts.
2. *Why no stored `is_available` flag on copies or seats?* → Availability is derived from loans / seat status; a second copy of the truth drifts.
3. *A foreign key that should be 1:1 — what else does it need?* → `UNIQUE` (or make it the primary key); otherwise it is 1:N.
4. *Where does the agreed rate, a seat's price, or a medicine's dose go?* → On the junction row: it describes the pairing, not either side.
5. *Hotel search counts per night; Airbnb uses an exclusion constraint. Why?* → A room type has many interchangeable units; a listing is one unit.
6. *When is `target_type + target_id` acceptable?* → When target kinds change often and you accept checking integrity outside the database.
7. *The one-sentence recipe for "no double booking"?* → Make the conflicting state impossible to store (unique / exclusion / check constraint) and claim the item with one guarded statement in a short transaction.

Chapter 17 TL;DR

- Use one template every time: **requirements & questions → entities, cardinality & participation → ER diagram → tables & constraints → key queries → indexes → design review → scaling** (the review checks normal forms, names deliberate redundancy, and lists what the design cannot do).
- Put business rules **in the schema** where possible: `CHECK` for ranges and state/column consistency, partial unique indexes for "one open X", exclusion constraints for "no overlapping X", unique keys for "once per user".
- Separate **type from instance** (titles/copies, room types/rooms, seats/show seats) and **current state from history** (status column + events or ledger).
- **Snapshot** values that must not change later (prices paid, addresses shipped to); **soft-delete** rows that history still references.
- Denormalize **deliberately** (counters, totals, inventory per night) — update in the same transaction and keep a reconciliation query.
- Concurrency for scarce items: a **guarded** `UPDATE … WHERE status = 'free'` in a short transaction is enough — demonstrated on two live sessions; `SKIP LOCKED` for "any available".
- At scale, the relational core stays for money, bookings and identity; media goes to object storage + CDN, feeds and counters to caches, events to logs, search to a search engine, and short-lived data to TTL stores or dropped time partitions.

Next: Chapter 18 — SQL interview problems, part 1, which practises the query skills from Chapters 2–8 on interview-style problems.

Chapter 18 — SQL Interview Problems I: Method and 42 Warm-Up to Medium Problems

Chapters 2–8 taught the tools one at a time. Interviews mix them. You get a short business question and a schema, and you have to decide what to filter, what to group, what to join and what could go wrong, then say it all out loud while you type. This chapter covers the method first and then works through 42 practice problems (easy up to lower-medium). They are grouped by the **pattern** they train, so you learn to spot the pattern rather than memorise answers.

What you'll learn

#	Section	Patterns
18.1	How to approach a SQL interview question	clarify → sample → build → edge cases → verify
18.2	Filtering (P1–P8)	`WHERE`, `AND` / `OR`, `%`, `IS DISTINCT FROM`, `char_length`, regex, row-level `CASE`
18.3	Aggregation (P9–P16)	`GROUP BY`, `HAVING`, `MIN` / `MAX` / `COUNT`, `string_agg`
18.4	Joins (P17–P22)	inner, self, cross + left, "top 1 with ties"
18.5	Anti-joins (P23–P26)	`NOT EXISTS`, `LEFT JOIN … IS NULL`, the `NOT IN` + NULL trap
18.6	Conditional aggregation (P27–P28)	`FILTER`, `SUM(CASE …)`, `UNION ALL` then pivot
18.7	Date logic (P29–P33)	half-open month ranges, date arithmetic, gaps and islands (finding runs of consecutive values), "value as of a date"
18.8	Ratios and percentages (P34–P38)	integer division, `NULLIF`, weighted averages, retention (share of users who come back)
18.9	Window and ranking basics (P39–P42)	second highest, first row per group, running total, compare to group average
18.10	Pattern table	every problem → pattern → key SQL feature

The harder problems (Nth highest, top-N per group, streaks, pivots, trips and users) are in Chapter 19.

How to run the examples. Every problem has its own file in `code/`. Each file creates its own small tables, fills them with invented data, and runs the solution plus an edge case or alternative. The runner gives each file a fresh PostgreSQL 14 database. The output under each query is real `psql` (PostgreSQL's command-line client) output, and `(null)` is how we display `NULL` (SQL's marker for a missing or unknown value).

```
bash tools_run_sql.sh sql '18-*'                    # all 42 files, fresh DB each
bash tools_pg.sh psql -d postgres -c 'CREATE DATABASE scratch'
bash tools_pg.sh psql -d scratch -f sql/chapters/18-sql-interview-problems-1/code/26_reps_without_red.sql
```

Every problem statement, dataset and explanation below is written for this book. The problems are classic interview-style exercises; many also appear on public practice sites under similar names.

18.1 How to approach a SQL interview question

What it is

A repeatable routine for turning a vague question ("which sellers were idle last year?") into a correct query while the interviewer watches. Think of it as a pilot's checklist: the steps are dull, but they catch the mistakes that nerves cause.

Why it matters

Most wrong answers in SQL interviews come from **misreading the question** or **missing an edge case** (an unusual input at the boundary of what the query must handle, such as an empty table or a NULL), not from syntax: counting rows instead of distinct users, losing people with zero activity, integer division turning 0.67 into 0, `NOT IN` silently returning nothing. Interviewers also score your *process*: do you ask questions, test your idea on a few rows, and explain your trade-offs?

How it works — the five steps

1. **Clarify.** Repeat the question in your own words, then ask:
   - What is one **row** of each table (the *grain*)? What is the **primary key** (the column or columns that uniquely identify a row)? Can there be **duplicate rows**?
   - Which columns can be **NULL**? What should happen to rows with NULLs?
   - Should entities with **no activity** appear (with 0) or be left out?
   - How should **ties** be handled: return all of them, or any one?
   - Output: which columns, what names, what **order**, how much **rounding**?
   - Are dates **inclusive**? Which time zone? Is "consecutive" by calendar day or by record?
2. **Sketch sample data.** Write 3–5 rows, including one awkward row (a NULL, a duplicate, a tie), plus the output you expect. This settles misunderstandings before you write code.
3. **Build in the logical order.** Write the query in the order the database evaluates it (see Chapter 4, logical query order): `FROM`/`JOIN` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `ORDER BY` → `LIMIT`. Use **CTEs** (common table expressions: named sub-queries written as `WITH step1 AS (…)`) so each step can be checked on its own (Chapter 7).
4. **Attack it with edge cases**, using the checklist below.
5. **Verify and discuss.** Run the query against your sample in your head (or for real). Then mention what **index** (a sorted lookup structure that lets the database find rows without reading the whole table) would help and give one alternative formulation.

The edge-case checklist

Risk	Typical symptom	Usual fix
NULL in a comparison	rows vanish from `<>`, `NOT IN` returns nothing	`IS DISTINCT FROM`, `NOT EXISTS`, `COALESCE` (returns its first non-NULL argument)
Entities with no rows	people with zero orders missing	`LEFT JOIN` from the entity table, `count(col)` not `count(*)`
Duplicates	counts too high, repeated ids	`DISTINCT`, `count(DISTINCT …)`, check the join's grain
Join fan-out (one row matching many, so rows multiply)	sums doubled after joining two child tables	aggregate before joining, or `count(DISTINCT)`
Ties	"the winner" is really two people	`RANK` / `DENSE_RANK` (ranking functions; ties share a rank), `FETCH FIRST … WITH TIES`
Integer division	`1/3 = 0`	multiply by `100.0` or cast to `numeric` first
Division by zero	runtime error	`NULLIF(denominator, 0)`
Empty result vs NULL	"return NULL if none" gives zero rows	wrap in an aggregate or a scalar subquery (a sub-query used as a single value)
Date boundaries	last day of the month (or 29 Feb) lost	half-open ranges: `>= start AND < next_start`

Common mistakes

- Starting to type before knowing the grain of each table.
- `SELECT`ing a column that is neither grouped nor aggregated (PostgreSQL rejects it; MySQL with the `ONLY_FULL_GROUP_BY` setting off returns an arbitrary value).
- Filtering the right-hand table of a `LEFT JOIN` in `WHERE`, which turns it back into an inner join (Chapter 5).
- Leaving `ORDER BY` out when the question asks for an order. Without it, SQL guarantees **no** order.

In the interview

30-second answer to "how do you approach a SQL problem?" "I first pin down the grain and keys of each table, what NULLs and duplicates mean, and whether zero-activity rows and ties should appear. I sketch a few rows with the expected output, then build the query in evaluation order (joins, filters, grouping, having, select, sort) using CTEs so I can check each step. Finally I test NULLs, duplicates, ties, empty groups and date boundaries, and mention an index or an alternative."

- *"Your query returns nothing for a new customer — is that right?"* It depends on the spec. If they should appear with 0, start from the customer table with a `LEFT JOIN` and `COALESCE` the aggregate.
- *"Would this be fast on 100 M rows?"* Talk about the filter/join columns and a matching index, **sargable** predicates (conditions written so an index can be used — "Search ARGument ABLE"; e.g. no function wrapped around the indexed column), and **pre-aggregation** (summarising rows before a join or storing summaries ahead of time). See Chapter 12.

Self-check

1. Why ask about the grain first? → It tells you what one row means, so you know whether to count rows or distinct ids, and whether a join can multiply rows.
2. What's wrong with `WHERE year(d) = 2020`? → It works, but wrapping the column in a function prevents an ordinary index on `d` from being used. Use `d >= '2020-01-01' AND d < '2021-01-01'`.
3. `count(*)` vs `count(col)` after a `LEFT JOIN`? → `count(*)` counts the NULL-padded row as 1; `count(col)` counts it as 0.

TL;DR

Clarify (grain, NULLs, zeros, ties, output), sketch sample rows, build in evaluation order with CTEs, attack with the edge-case checklist, then verify and discuss performance.

18.2 Filtering problems (P1–P8)

The simplest pattern: keep or drop **individual rows** with `WHERE`. The traps are `NULL` (**three-valued logic**: a condition can be `TRUE`, `FALSE` or `UNKNOWN`, and comparisons with NULL give `UNKNOWN`), boundaries (`>` vs `>=`) and string rules. Background: Chapter 2 and Chapter 3.

P1 · Big cities — `OR` filter

Problem. A `city` table stores name, country, area in km² and population. List the cities that are *either* larger than 3,000 km² *or* home to more than 5 million people.

Data (`01_big_cities.sql`): six invented towns. Coralbay sits exactly on both limits (3000 km², 5,000,000). Eastvale and Fernhill have an unknown (NULL) area.

```
SELECT name FROM city
WHERE area_km2 > 3000 OR population > 5000000
ORDER BY name;
```

Output:

```
   name
-----------
 Aldport
 Brimstead
 Eastvale
(3 rows)
```

Why it works. Each row is tested on its own. Coralbay fails because "more than" is strict. Eastvale's area test gives `UNKNOWN`, but `UNKNOWN OR TRUE` is `TRUE`, so its population is enough. Fernhill gives `UNKNOWN OR FALSE` = `UNKNOWN`, and `WHERE` keeps only `TRUE`.

Alternative. `SELECT … WHERE area_km2 > 3000 UNION SELECT … WHERE population > 5000000` returns the same set. On some engines (notably older MySQL) this lets each branch use its own index. PostgreSQL can instead combine two indexes with a *BitmapOr* (a plan step that merges the row lists found by two index scans) for the plain `OR`.

P2 · Odd-numbered, not-boring movies — modulo and `<>`

Problem. From `movie(id, title, description, rating)`, return movies whose id is odd and whose description isn't `'boring'`, highest rating first.

Data (`02_odd_movies.sql`): ids 1–5 and 7, one of them described as boring, one with a NULL description, plus id −3.

```
SELECT * FROM movie
WHERE id % 2 = 1 AND description <> 'boring'
ORDER BY rating DESC;
```

Output:

```
 id |    title    | description | rating
----+-------------+-------------+--------
  5 | Deep Blue   | documentary |   8.90
  1 | Night Train | thriller    |   8.10
(2 rows)
```

Why it works. `%` is the remainder. Odd ids leave remainder 1. The `<>` drops the boring one.

Edge cases. Two rows went missing silently. In PostgreSQL, `-3 % 2` is `-1` (the result takes the sign of the **dividend**, the number being divided), so `= 1` misses negative odd ids. And `NULL <> 'boring'` is `UNKNOWN`. If an unknown description should count as "not boring", say so explicitly:

```sql
SELECT * FROM movie
WHERE id % 2 <> 0 AND description IS DISTINCT FROM 'boring'
ORDER BY rating DESC;
```

```
 id |    title    | description | rating
----+-------------+-------------+--------
  5 | Deep Blue   | documentary |   8.90
  1 | Night Train | thriller    |   8.10
 -3 | Rewind      | comedy      |   6.00
  7 | Untitled    | (null)      |   5.00
(4 rows)
```

MySQL has `MOD(id, 2)` as well as `%`. Its **NULL-safe** equality (one that treats two NULLs as equal and never returns `UNKNOWN`) is `<=>`, so "is distinct from" is written `NOT (a <=> b)`.

P3 · Green products — `AND` on flag columns

Problem. `inventory(product_id, organic, compostable)` holds `'Y'`/`'N'` flags. Return the products that have both flags set.

```sql
SELECT product_id FROM inventory
WHERE organic = 'Y' AND compostable = 'Y'
ORDER BY product_id;
```

```
 product_id
------------
         10
         13
(2 rows)
```

Why it works. Both conditions must be `TRUE` for the same row.

Edge case / design note. Flags stored as text invite dirty values such as `'y'` or `' Y'`. The file puts a `CHECK (organic IN ('Y','N'))` on each column, and inserting `'y'` fails with *violates check constraint "inventory_organic_check"*. In PostgreSQL a `boolean NOT NULL` column is cleaner still: `WHERE organic AND compostable`. MySQL's source table often uses `ENUM('Y','N')`.

P4 · Everyone outside department 3 — NULL-safe "not equal"

Problem. From `staff(id, name, dept_id)`, list people who are not in department 3. One person has no department yet.

```sql
SELECT name FROM staff WHERE dept_id <> 3 ORDER BY name;              -- naive
```

```
 name
------
 Ana
 Chen
(2 rows)
```

```sql
SELECT name FROM staff WHERE dept_id IS DISTINCT FROM 3 ORDER BY name; -- NULL-safe
```

```
 name
------
 Ana
 Chen
 Dara
(3 rows)
```

Why it matters. `NULL <> 3` is `UNKNOWN`, so Dara disappears from the naive version. Whether Dara *should* appear is a clarifying question. Most people would say "not in department 3" includes "in no department". `IS DISTINCT FROM` treats NULL as an ordinary comparable value. The portable spelling is `dept_id <> 3 OR dept_id IS NULL`.

P5 · Self-assigned tasks — comparing two columns, removing duplicates

Problem. `task_log(task_id, assigner_id, assignee_id, done_on)` may contain duplicate rows. Return, in ascending order, each person who completed at least one task they had given to themselves.

```sql
SELECT DISTINCT assigner_id AS employee_id
FROM task_log
WHERE assigner_id = assignee_id
ORDER BY employee_id;
```

```
 employee_id
-------------
           7
           9
(2 rows)
```

Why it works. The `WHERE` compares two columns of the same row. `DISTINCT` collapses repeats: without it the output is `7, 7, 9, 9` (one duplicate row for person 7, and two self-assigned tasks for person 9). `GROUP BY assigner_id` gives the same result. "At least one" nearly always means you need `DISTINCT`, `GROUP BY` or `EXISTS`.

P6 · Long comments — length in characters

Problem. `comment(id, body)`: return ids of comments longer than 100 characters (spaces and punctuation count).

```sql
SELECT id FROM comment WHERE char_length(body) > 100 ORDER BY id;
```

```
 id
----
  2
(1 row)
```

Why it works / edge cases. Comment 3 is exactly 100 characters, so it is not "more than" 100. Comment 4 is 60 copies of `é`: 60 **characters** but 120 **bytes** in **UTF-8** (the standard text encoding, which stores

accented and non-Latin characters in 2–4 bytes each).

```
SELECT id, char_length(body) AS chars, octet_length(body) AS bytes FROM comment ORDER BY id;
```

```
 id | chars | bytes
----+-------+-------
  1 |    10 |    10
  2 |   102 |   102
  3 |   100 |   100
  4 |    60 |   120
(4 rows)
```

Engine	Characters	Bytes
PostgreSQL	`char_length(s)`, `length(s)`	`octet_length(s)`
MySQL	`CHAR_LENGTH(s)`	`LENGTH(s)` (a classic trap)
SQL Server	`LEN(s)` (ignores trailing spaces)	`DATALENGTH(s)`
SQLite	`length(s)`	`length(CAST(s AS BLOB))`

P7 · Valid company e-mails — regular expressions

Problem. `employee(id, name, email)`. An address is valid when the part before `@` starts with a letter and continues with letters, digits, `_`, `.` or `-`, and the domain is exactly `northwind.dev`. Return the valid rows.

Data (`07_valid_emails.sql`): eight addresses. They include one starting with `_`, one starting with a digit, `northwindXdev`, `northwind.dev.evil.com`, an upper-case domain and one containing `#`.

```
SELECT id, name, email FROM employee
WHERE email ~ '^[A-Za-z][A-Za-z0-9_.-]*@northwind\.dev$'
ORDER BY id;
```

```
 id | name |        email
----+------+---------------------
  1 | Ana  | ana.k@northwind.dev
  5 | Eve  | Eve-2@northwind.dev
(2 rows)
```

Why it works. A **regular expression** (regex) is a mini-language for describing text patterns, and `~` tests whether a string matches one. `^` and `$` **anchor** the pattern to the start and end, so it must match the whole string. `[A-Za-z]` forces a letter first, and the class after it allows the other characters (inside `[...]` a `.` is literal, and a `-` placed last is literal). `\.` is a literal dot.

Edge cases. Remove the anchors and the backslash, and six rows match. That includes `dee@northwindXdev`, because an unescaped `.` matches any character, and `fay@northwind.dev.evil.com`, because the end is not anchored. `~` is case-sensitive, so `gus@NORTHWIND.DEV` fails. Use `~*` if the domain should be case-insensitive (it then matches ids 1, 5, 7). Dialects: MySQL `REGEXP` / `REGEXP_LIKE()`, Oracle `REGEXP_LIKE()`. SQL Server gained `REGEXP_LIKE()` only in SQL Server 2025 (database compatibility level 170) and Azure SQL; older versions fall back to `LIKE` patterns. In MySQL string literals the backslash itself must be doubled (`'\\.'`).

P8 · Triangle test — a `CASE` per row

Problem. `segment(a, b, c)` holds three lengths per row. Add a column saying `'Yes'` if they can form a triangle and `'No'` otherwise.

```sql
SELECT a, b, c,
       CASE WHEN a + b > c AND a + c > b AND b + c > a THEN 'Yes' ELSE 'No' END AS triangle
FROM segment;
```

```
 a  |   b    | c | triangle
----+--------+---+----------
  3 |      4 | 5 | Yes
 10 |      2 | 3 | No
  5 |      5 | 5 | Yes
  1 |      2 | 3 | No
  7 | (null) | 4 | No
(5 rows)
```

Why it works. The **triangle inequality** says every pair of sides must be longer than the third side. `(1,2,3)` is *degenerate*: it is flat, because 1 + 2 is not *greater* than 3.

Alternative / edge case. It is enough to check the longest side: `2 * greatest(a,b,c) < a + b + c`. A missing side makes the condition `UNKNOWN`, and `CASE` falls through to `ELSE 'No'`, which quietly claims something we don't know. Adding `WHEN a IS NULL OR b IS NULL OR c IS NULL THEN 'Unknown'` makes that row say `Unknown` (see the file). PostgreSQL's `greatest()` ignores NULLs, whereas MySQL's returns NULL if any argument is NULL.

18.3 Aggregation problems (P9–P16)

Collapse many rows into one per group. The traps: which rows exist before grouping, how aggregates treat NULLs, and what an **empty** input returns. Background: Chapter 4.

P9 · Largest number that appears once — `HAVING` then `MAX`

Problem. `nums(n)` has no key and contains repeats. Return the biggest value that occurs exactly once, or NULL if no value is unique.

```sql
SELECT max(n) AS num
FROM (SELECT n FROM nums GROUP BY n HAVING count(*) = 1) AS singles;
```

```
 num
-----
   5
(1 row)
```

Why it works. The inner query groups equal values and keeps groups of size 1: {1, 4, 5}. The outer `max` picks 5.

Edge case — empty input. After inserting 1, 4, 5 again, no value is unique. An aggregate over zero rows still returns **one row** (`max` → NULL), which is exactly what "report NULL" asks for:

```
  num
--------
 (null)
(1 row)
```

The tempting `… HAVING count(*) = 1 ORDER BY n DESC LIMIT 1` returns **0 rows** in the same situation. That is a different answer, and graders that expect a NULL row mark it wrong.

P10 · Units sold per product — `SUM` with `GROUP BY`

Problem. `sale(sale_id, product_id, yr, qty, unit_price)` references `product`. Report total quantity per product.

```
SELECT product_id, sum(qty) AS total_quantity
FROM sale
GROUP BY product_id
ORDER BY product_id;
```

```
 product_id | total_quantity
------------+----------------
        100 |             25
        200 |             15
(2 rows)
```

Edge case — zero sellers. Product 300 never sold, so it has no group. If the business wants it listed, start from `product`:

```
SELECT p.product_id, COALESCE(sum(s.qty), 0) AS total_quantity
FROM product p LEFT JOIN sale s USING (product_id)
GROUP BY p.product_id ORDER BY p.product_id;
```

```
 product_id | total_quantity
------------+----------------
        100 |             25
        200 |             15
        300 |              0
(3 rows)
```

`sum` over only NULLs is NULL, not 0, which is why the `COALESCE` is needed.

P11 · First login per player — `MIN` on dates

Problem. `activity(player_id, device_id, played_on, games)` has one row per player per day. Give each player's first login date.

```
SELECT player_id, min(played_on) AS first_login
FROM activity GROUP BY player_id ORDER BY player_id;
```

```
 player_id | first_login
-----------+-------------
         1 | 2024-02-27
         2 | 2024-06-25
         3 | 2024-03-02
(3 rows)
```

Why it works. `min` works on anything sortable. Dates sort chronologically, and player 1's earliest row was inserted last, which makes no difference. **Follow-up trap:** "and the device used that day?" cannot be answered by adding `device_id` to this query, because it is neither grouped nor aggregated. That is P40.

P12 · Follower count — `COUNT(*)` vs `COUNT(col)`

Problem. `follows(user_id, follower_id)` has one row per "follower follows user". Count followers per user, ordered by user id.

```sql
SELECT user_id, count(*) AS followers_count
FROM follows GROUP BY user_id ORDER BY user_id;
```

```
 user_id | followers_count
---------+-----------------
       1 |               2
       2 |               1
       3 |               3
(3 rows)
```

Edge case — users with zero followers. User 4 exists in `app_user` but nobody follows them. Joining from the user table fixes the missing row, but only if you count a column from the right-hand side:

```sql
SELECT u.user_id, count(f.follower_id) AS good, count(*) AS wrong
FROM app_user u LEFT JOIN follows f USING (user_id)
GROUP BY u.user_id ORDER BY u.user_id;
```

```
 user_id | good | wrong
---------+------+-------
       1 |    2 |     2
       2 |    1 |     1
       3 |    3 |     3
       4 |    0 |     1
(4 rows)
```

The LEFT JOIN produces one NULL-padded row for user 4. `count(*)` counts it, while `count(f.follower_id)` skips the NULL.

P13 · When was each bike last used — `MAX` and sorting by an aggregate

Problem. `ride(ride_id, bike_no, started_at, ended_at)`. Report the latest end time per bike, with the most recently used bike first.

```sql
SELECT bike_no, max(ended_at) AS last_used
FROM ride GROUP BY bike_no ORDER BY last_used DESC;
```

```
 bike_no |      last_used
---------+---------------------
 B-10    | 2024-05-03 17:45:00
 B-20    | 2024-05-01 09:50:00
 B-30    | 2024-04-28 12:20:00
(3 rows)
```

Edge case. Bike B-20 is out on a ride right now (`ended_at` is NULL). `max` skips NULLs, so B-20 reports its previous ride. Ask whether "in use now" should count as most recent. The file adds `bool_or(ended_at IS NULL) AS out_now` to flag it (`t` for B-20). Also note that in PostgreSQL `ORDER BY … DESC` puts NULLs **first**. Add `NULLS LAST` if some bike could have only unfinished rides.

P14 · Missed appointments — filter, then group

Problem. `appointment(id, patient_id, doctor_id, appt_date, status)` with status Completed / Cancelled / No-Show. Count no-shows per patient and leave out patients who never missed one.

```sql
SELECT patient_id, count(*) AS no_shows
FROM appointment
WHERE status = 'No-Show'
GROUP BY patient_id
ORDER BY patient_id;
```

```
 patient_id | no_shows
------------+----------
         11 |        2
         12 |        2
(2 rows)
```

Why it works. `WHERE` removes the other statuses *before* grouping, so patient 13 (only a cancellation) never forms a group. This matches the spec that such patients are left out.

Alternative. When you need several counts at once, keep all rows and count conditionally: `count(*) FILTER (WHERE status = 'No-Show')`, with `HAVING` that count `> 0`. In the same pass it can also report `count(*) AS booked` (patient 11: 2 of 3). That idea is developed in 18.6.

P15 · Products sold per day — `COUNT(DISTINCT)` and `string_agg`

Problem. `sales_log(sold_on, product)` has no key and contains duplicates. For each date, return how many *different* products were sold and a comma-separated, alphabetical list of them, ordered by date.

```sql
SELECT sold_on,
       count(DISTINCT product)                          AS num_sold,
       string_agg(DISTINCT product, ',' ORDER BY product) AS products
FROM sales_log
GROUP BY sold_on
ORDER BY sold_on;
```

```
  sold_on   | num_sold |   products
------------+----------+--------------
 2024-06-01 |        2 | Ink,Pen
 2024-06-02 |        1 | Stapler
 2024-06-03 |        3 | Clip,Ink,Pen
(3 rows)
```

Why it works / edge cases. `DISTINCT` inside both aggregates removes the second "Pen" on 1 June. A NULL product on 3 June is ignored by both aggregates. With `DISTINCT`, PostgreSQL requires the `ORDER BY` inside `string_agg` to use the aggregated expression itself.

Engine	Ordered, de-duplicated list
PostgreSQL	`string_agg(DISTINCT p, ',' ORDER BY p)`
MySQL	`GROUP_CONCAT(DISTINCT p ORDER BY p SEPARATOR ',')` (output capped by `group_concat_max_len`, default 1024 bytes)
SQL Server 2017+	`STRING_AGG(p, ',') WITHIN GROUP (ORDER BY p)`: no `DISTINCT`, so de-duplicate in a subquery first
SQLite	`group_concat(DISTINCT p)`: order not guaranteed before 3.44

P16 · Customer basket summary — the same pattern, per customer

Problem. `purchase(bought_on, customer_id, product)` may repeat. For each customer, show the number of distinct products and their alphabetical list.

```sql
SELECT customer_id,
       count(DISTINCT product) AS num_products,
       string_agg(DISTINCT product, ',' ORDER BY product) AS products
FROM purchase GROUP BY customer_id ORDER BY customer_id;
```

```
 customer_id | num_products |    products
-------------+--------------+----------------
           1 |            2 | Mug,Tea
           2 |            1 | Coffee
           3 |            3 | Coffee,Mug,Tea
(3 rows)
```

Alternative. If the result goes to application code, return a real array rather than a string you'll have to split again: `array_agg(DISTINCT product ORDER BY product)` → `{Mug,Tea}`. The lesson of P15 and P16 is that "one row per X with a list of Y" is a `GROUP BY X` plus a string or array aggregate.

18.4 Join problems (P17–P22)

Combine rows from two tables, or from one table with itself. The traps: rows that have no partner, rows that get multiplied, and ties. Background: Chapter 5.

P17 · Product name for every sale — basic inner join

Problem. `sale(sale_id, product_id, yr, qty, unit_price)` and `product(product_id, name)`. For every sale, show the product name, year and unit price, ordered by year.

```sql
SELECT p.name AS product_name, s.yr AS year, s.unit_price AS price
FROM sale s
JOIN product p ON p.product_id = s.product_id
ORDER BY s.yr, s.sale_id;
```

```
 product_name | year | price
--------------+------+-------
 Kettle       | 2022 |    27
 Kettle       | 2023 |    25
 Toaster      | 2023 |    40
(3 rows)
```

Why it works. Each sale finds its single matching product (`product_id` is the product table's primary key), so the join neither loses nor duplicates sales. **Edge case:** with a foreign key, every sale has a product. Without one, an orphan sale would vanish from an inner join, so switch to `LEFT JOIN` and show a NULL name. Break ties in the sort (`, s.sale_id`) so the order is **deterministic** (the same every run). An **orphan** here is a child row whose parent row doesn't exist.

P18 · Average process time per machine — self-join start to end

Problem. `machine_event(machine_id, process_id, kind, ts)` logs a `'start'` row and an `'end'` row per process. For each machine, report the average of (end − start), rounded to 3 decimals.

Data (`18_process_time.sql`): three machines with two finished processes each; machine 2 also has a third process that never ended.

```sql
SELECT s.machine_id, round(avg(e.ts - s.ts), 3) AS processing_time
FROM machine_event s
JOIN machine_event e
  ON e.machine_id = s.machine_id AND e.process_id = s.process_id
 AND s.kind = 'start' AND e.kind = 'end'
GROUP BY s.machine_id
ORDER BY s.machine_id;
```

```
 machine_id | processing_time
------------+-----------------
          0 |           0.894
          1 |           0.995
          2 |           1.456
(3 rows)
```

Why it works. This is a **self-join** (a table joined to itself). **Aliases** (short alternative names) `s` and `e` give two views of the same table. The join puts each process's start and end on one row, and `avg` then averages the differences. The unfinished process has no `end` row, so the inner join drops it.

Alternative / edge case. You can pivot each process to one row with `max(ts) FILTER (WHERE kind = 'start')` and `… 'end'`, and then average the differences. That gives the same numbers, because the unfinished process's difference is NULL and `avg` skips it. A tempting shortcut is `(sum of end ts − sum of start ts) / count(DISTINCT process_id)`, and it would be wrong here, because it adds the unfinished start into the sum. **PostgreSQL note:** `round(x, 3)` exists only for `numeric`. Here `ts` is `numeric`. If it were `double precision` (a `float` — an approximate binary number type), write `round((…)::numeric, 3)` (`::numeric` is PostgreSQL's shorthand for a **cast**, i.e. converting a value to another type).

P19 · Fiction books per author — join, filter, group, two-key sort

Problem. `book(book_id, title, genre)` and `book_author(author_id, book_id, author_name)`. Count each author's Fiction titles, leave out authors with none, and sort by count (highest first) and then by name.

```sql
SELECT a.author_id, a.author_name, count(*) AS fiction_books_count
FROM book_author a
JOIN book b ON b.book_id = a.book_id
WHERE b.genre = 'Fiction'
GROUP BY a.author_id, a.author_name
ORDER BY fiction_books_count DESC, a.author_name, a.author_id;
```

```
 author_id | author_name | fiction_books_count
-----------+-------------+---------------------
        10 | Iris Vale   |                   2
        13 | Iris Vale   |                   1
        11 | Omar Reyes  |                   1
(3 rows)
```

Why it works. The inner join plus `WHERE` keeps only fiction rows, so authors with zero fiction form no group, which is what the spec wants. **Edge case — duplicate names:** two different people are called Iris Vale. Grouping by `author_name` alone merges them into one row showing `3`, which is wrong. Always group by the **key** and add the name for display.

P20 · Exam sittings per student and subject — CROSS JOIN + LEFT JOIN

Problem. `student`, `subject`, and `exam_sitting(student_id, subject_name)` (duplicates allowed, one row per sitting). For every student × subject pair, report how many times the student sat that exam (0 included), ordered by student id and subject.

```sql
SELECT st.student_id, st.student_name, su.subject_name,
       count(e.subject_name) AS attended_exams
FROM student st
CROSS JOIN subject su
LEFT JOIN exam_sitting e
       ON e.student_id = st.student_id AND e.subject_name = su.subject_name
GROUP BY st.student_id, st.student_name, su.subject_name
ORDER BY st.student_id, su.subject_name;
```

```
 student_id | student_name | subject_name | attended_exams
------------+--------------+--------------+----------------
          1 | Noor         | Art          |              0
          1 | Noor         | Math         |              3
          1 | Noor         | Physics      |              1
          2 | Kofi         | Art          |              1
          2 | Kofi         | Math         |              1
          2 | Kofi         | Physics      |              0
          6 | Lin          | Art          |              0
          6 | Lin          | Math         |              0
          6 | Lin          | Physics      |              0
(9 rows)
```

Why it works. "Every pair, including zeros" means you must *create* the pairs, because the **fact table** (the table recording events — here, sittings) only has pairs that happened. `CROSS JOIN` (every row of one table combined with every row of the other) builds the full 3 × 3 grid. The `LEFT JOIN` attaches matching sittings (both columns in `ON`), and `count(e.subject_name)` turns missing matches into 0. This **grid + left join** pattern also fills in missing days in time series (with `generate_series`, which produces a sequence of numbers or dates).

P21 · Election winner — ORDER BY an aggregate, and ties

Problem. `candidate(id, name)` and `vote(id, candidate_id)` (one row per ballot). Return the name of the candidate with the most votes (the data guarantees a single winner).

```sql
SELECT c.name
FROM candidate c
JOIN vote v ON v.candidate_id = c.id
GROUP BY c.id, c.name
ORDER BY count(*) DESC
LIMIT 1;
```

```
 name
-------
 Tariq
(1 row)
```

Why it works. Group ballots per candidate, sort by the count, and keep the top row. Grouping by `c.id` (not just name) keeps namesakes apart.

Edge case — ties. The guarantee rarely holds in real life. After one more vote for Victor, both have 3 and `LIMIT 1` would pick one arbitrarily. The standard `FETCH FIRST 1 ROWS WITH TIES` (PostgreSQL 13+, Oracle 12c+; SQL Server uses `TOP 1 WITH TIES`) returns every row tied with the last one kept:

```
  name  | votes
--------+-------
 Tariq  |     3
 Victor |     3
(2 rows)
```

MySQL has no `WITH TIES`. Use `RANK() OVER (ORDER BY count(*) DESC)` in a subquery and keep rank 1 (Chapter 8).

P22 · "Middle" users in a follow graph — self-join or EXISTS

Problem. `follow(followee, follower)`: the follower follows the followee, and nobody follows themselves. Find users who follow at least one person *and* have at least one follower. Report each one's follower count, alphabetically.

Data: amy←bo, bo←cy, bo←dee, cy←eve, cy←bo, dee←amy (read "amy is followed by bo").

```
SELECT f.followee AS follower, count(*) AS num
FROM follow f
WHERE EXISTS (SELECT 1 FROM follow g WHERE g.follower = f.followee)
GROUP BY f.followee
ORDER BY f.followee;
```

```
 follower | num
----------+-----
 amy      |   1
 bo       |   2
 cy       |   2
 dee      |   1
(4 rows)
```

Why it works. Each row of `f` is one follower of `f.followee`. `EXISTS` keeps the row only if that same user also appears as somebody's *follower*. Eve is followed by no one, so she is excluded.

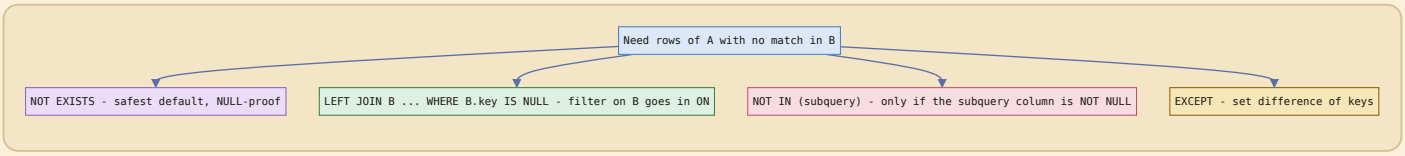
Alternative — and the fan-out trap. The self-join `follow a JOIN follow b ON b.followee = a.follower` also works. However, each of bo's followers pairs with each person bo follows, so the rows multiply:

```
 follower | num | inflated
----------+-----+----------
 amy      |   1 |        1
 bo       |   2 |        4
 cy       |   2 |        2
 dee      |   1 |        1
(4 rows)
```

Only `count(DISTINCT b.follower)` is correct there. When a join can match many rows on **both** sides, either de-duplicate or use `EXISTS` (a *semi-join* — it keeps a row if at least one match exists — which never multiplies).

18.5 Anti-join problems (P23–P26)

"Find X that has **no** Y" is an **anti-join**. There are three common spellings. They behave identically, except that `NOT IN` breaks when the subquery contains a NULL. Background: Chapter 5 §5.8 and Chapter 6 §6.4.



P23 · Sellers with no 2020 sales — NOT EXISTS with a date range

Problem. `seller(seller_id, seller_name)` and `orders(order_id, sold_on, cost, customer_id, seller_id)`. List sellers who made no sale at all in 2020, by name.

```sql
SELECT s.seller_name
FROM seller s
WHERE NOT EXISTS (
  SELECT 1 FROM orders o
  WHERE o.seller_id = s.seller_id
    AND o.sold_on >= DATE '2020-01-01' AND o.sold_on < DATE '2021-01-01')
ORDER BY s.seller_name;
```

```
 seller_name
-------------
 Reed Ltd
 Tern LLC
(2 rows)
```

Why it works. For each seller, the subquery asks "is there any 2020 order?". `NOT EXISTS` keeps sellers where the answer is no: Reed sold only in 2019, and Tern never sold. Quill's 31 December order counts, and Sable's 1 January 2021 order doesn't. The **half-open** range `>= 1 Jan AND < next 1 Jan` is safe for `date` and `timestamp` alike. `BETWEEN '2020-01-01' AND '2020-12-31'` would miss 31 December after midnight if the column were a timestamp. PostgreSQL executes `NOT EXISTS` as a hash or merge **anti-join** (two internal join algorithms: one builds an in-memory lookup table, the other walks two sorted inputs side by side), and an index on `orders(seller_id, sold_on)` makes each probe cheap.

P24 · Dormant accounts — LEFT JOIN … IS NULL

Problem. `account(account_id, account_name)` and `txn(txn_id, account_id, txn_date)`. Return accounts that made no transaction in 2020, by name.

```sql
SELECT a.account_name
FROM account a
LEFT JOIN txn t
       ON t.account_id = a.account_id
      AND t.txn_date >= DATE '2020-01-01' AND t.txn_date < DATE '2021-01-01'
WHERE t.txn_id IS NULL
ORDER BY a.account_name;
```

```
 account_name
--------------
 Juniper
 Kestrel
(2 rows)
```

Why it works. The LEFT JOIN attaches only *2020* transactions. Accounts with none get a NULL-padded row, and `WHERE t.txn_id IS NULL` keeps exactly those. Test a column that can't be NULL in a real match, such as the primary key.

The classic bug. Move the date test into `WHERE` and you ask for rows where `t` is NULL **and** `t.txn_date` is in 2020. No row can be both, so the result is `(0 rows)`. **Alternative:** a **set difference** of ids (rows in the first result that are not in the second), `SELECT account_id FROM account EXCEPT SELECT account_id FROM txn WHERE …2020…`, which returns the same Juniper and Kestrel.

P25 · Sessions with no ads — anti-join on a range condition

Problem. `playback(session_id, customer_id, start_t, end_t)` and `ad(ad_id, customer_id, shown_at)`. A session is ad-free if that customer was shown no ad between `start_t` and `end_t` (inclusive). List ad-free sessions.

Data: customer 1 has sessions [1,5] and [15,23] and ads at 5 and 9. Customer 2 has [10,12] and [17,28] and ads at 17 and 20. Customer 3 has [2,8] and an ad at 30.

```sql
SELECT p.session_id
FROM playback p
WHERE NOT EXISTS (
  SELECT 1 FROM ad a
  WHERE a.customer_id = p.customer_id
    AND a.shown_at BETWEEN p.start_t AND p.end_t)
ORDER BY p.session_id;
```

```
 session_id
------------
          2
          3
          5
(3 rows)
```

Why it works / edge cases. The match condition is not plain equality. It is "same customer **and** timestamp inside the interval". Session 1 is excluded because the ad at 5 sits exactly on its (inclusive) end, and session 4 because of the ad at 17. The ad at 9 falls between customer 1's sessions, so it affects neither. Forgetting `a.customer_id = p.customer_id` would let another viewer's ads disqualify a session.

P26 · Reps who never sold to RED — NOT EXISTS vs NOT IN

Problem. `sales_rep(rep_id, name)`, `company(com_id, name, city)`, `orders(order_id, order_date, com_id, rep_id, amount)`. Return the reps with no order from the company named `'RED'`.

```sql
SELECT r.name
FROM sales_rep r
WHERE NOT EXISTS (
  SELECT 1 FROM orders o JOIN company c ON c.com_id = o.com_id
  WHERE o.rep_id = r.rep_id AND c.name = 'RED')
ORDER BY r.name;
```

```
 name
------
 Kai
 Lu
 Ned
(3 rows)
```

Why it works. The subquery joins orders to companies so it can filter by company *name*, and it is **correlated** on the rep (it refers to the outer row `r`, so it is evaluated per rep).

Edge case — the NOT IN trap. `WHERE r.rep_id NOT IN (SELECT o.rep_id FROM … WHERE c.name = 'RED')` returns the same three names. Then we insert one RED order with no rep (`rep_id` NULL), and the `NOT IN` version returns:

```
 name
------
(0 rows)
```

`x NOT IN (1, 4, NULL)` means `x <> 1 AND x <> 4 AND x <> NULL`. The last test is `UNKNOWN` for every `x`, so no row qualifies. `NOT EXISTS` still returns Kai, Lu, Ned. Default to `NOT EXISTS`. If you must use `NOT IN`, add `WHERE o.rep_id IS NOT NULL` inside.

18.6 Conditional aggregation (P27–P28)

Conditional aggregation computes several filtered aggregates over the same groups in one pass. It is SQL's way to turn row values (statuses, types) into columns, a simple **pivot**. PostgreSQL and standard SQL spell it `agg(...) FILTER (WHERE cond)`. Every engine accepts `SUM(CASE WHEN cond THEN x ELSE 0 END)`. See Chapter 4 §4.4.

P27 · Attendance summary — one column per status

Problem. `employee(employee_id, employee_name)` and `attendance(employee_id, att_date, status)` with statuses Present / Absent / Late (duplicates possible). Show, for every employee, the number of rows of each status, ordered by id.

```
SELECT e.employee_id, e.employee_name,
       count(*) FILTER (WHERE a.status = 'Present') AS days_present,
       count(*) FILTER (WHERE a.status = 'Absent')  AS days_absent,
       count(*) FILTER (WHERE a.status = 'Late')    AS days_late
FROM employee e
LEFT JOIN attendance a ON a.employee_id = e.employee_id
GROUP BY e.employee_id, e.employee_name
ORDER BY e.employee_id;
```

```
 employee_id | employee_name | days_present | days_absent | days_late
-------------+---------------+--------------+-------------+-----------
           1 | Aiko          |            2 |           0 |         1
           2 | Bram          |            2 |           1 |         0
           3 | Cleo          |            0 |           0 |         0
(3 rows)
```

Why it works. The LEFT JOIN keeps Cleo, who has no records. Each `count(*) FILTER` counts only the rows passing its condition, and `count` never returns NULL. Bram's duplicated "Present" row counts twice, as the spec says to count *rows*. If it meant *days*, use `count(DISTINCT a.att_date) FILTER (…)`.

Portable form and a trap. The `sum(CASE …)` spelling with `THEN 1 ELSE 0` also gives Cleo 0 (column `present_else0`). Drop the `ELSE 0` and the `CASE` yields NULL for her one padded row, so `sum` returns **NULL**:

```
 employee_id | present_else0 | present_no_else
-------------+---------------+-----------------
           1 |             2 |               2
           2 |             2 |               2
           3 |             0 |          (null)
(3 rows)
```

P28 · Monthly approvals and chargebacks — UNION ALL, then pivot

Problem. `txn(id, country, state, amount, trans_date)` with state approved/declined, and `chargeback(trans_id, cb_date)`. For each month and country, report the approved count and amount, and the chargeback count and amount. A chargeback belongs to the month it **arrived** and carries the original transaction's amount. Leave out month/country pairs where everything is zero.

```sql
WITH events AS (
  SELECT to_char(trans_date, 'YYYY-MM') AS month, country, 'approved' AS kind, amount
  FROM txn WHERE state = 'approved'
  UNION ALL
  SELECT to_char(c.cb_date, 'YYYY-MM'), t.country, 'chargeback', t.amount
  FROM chargeback c JOIN txn t ON t.id = c.trans_id
)
SELECT month, country,
       count(*) FILTER (WHERE kind = 'approved')                    AS approved_count,
       COALESCE(sum(amount) FILTER (WHERE kind = 'approved'), 0)    AS approved_amount,
       count(*) FILTER (WHERE kind = 'chargeback')                  AS chargeback_count,
       COALESCE(sum(amount) FILTER (WHERE kind = 'chargeback'), 0) AS chargeback_amount
FROM events
GROUP BY month, country
ORDER BY month, country;
```

```
  month  | country | approved_count | approved_amount | chargeback_count | chargeback_amount
---------+---------+----------------+-----------------+------------------+-------------------
 2024-05 | US      |              1 |            1000 |                1 |              2000
 2024-06 | US      |              2 |            8000 |                1 |              1000
 2024-09 | US      |              0 |               0 |                1 |              5000
(3 rows)
```

Why it works. The two event kinds have **different dates**, so a single join can't bucket them. The trick is to stack them into one long list of `(month, country, kind, amount)` with `UNION ALL`, which keeps duplicates, unlike `UNION`. After that, one `GROUP BY` plus four `FILTER`s pivots the list. Notice what this data exercises. The declined May transaction still produces a May chargeback. Chargebacks land in later months (June, September). The DE row, which is only a declined transaction, produces no event, so the all-zero rows disappear with no `HAVING` needed. `sum … FILTER` over no rows is NULL, hence the `COALESCE`. The `to_char(d, 'YYYY-MM')` month label is `DATE_FORMAT(d, '%Y-%m')` in MySQL and `FORMAT(d, 'yyyy-MM')` in SQL Server.

18.7 Date-logic problems (P29–P33)

Date questions test ranges, arithmetic and "what was true on day X". Useful PostgreSQL facts: `date - date` is an **integer** number of days; `date + 1` is the next day; `date_trunc('month', ts)` gives the first instant of the month. A **half-open** range includes its start but excludes its end (`>= start AND < next_start`). Background: Chapter 3.

P29 · Weather class per country in November 2019 — month range + CASE on an average

Problem. `country(country_id, country_name)` and `weather(country_id, weather_state, day)`. Using only November 2019 readings, label each country Cold if its average is ≤ 15, Hot if ≥ 25, and Warm otherwise.

```sql
SELECT c.country_name,
       CASE WHEN avg(w.weather_state) <= 15 THEN 'Cold'
            WHEN avg(w.weather_state) >= 25 THEN 'Hot'
            ELSE 'Warm' END AS weather_type
FROM weather w
JOIN country c ON c.country_id = w.country_id
WHERE w.day >= DATE '2019-11-01' AND w.day < DATE '2019-12-01'
GROUP BY c.country_id, c.country_name
ORDER BY c.country_name;
```

```
 country_name | weather_type
--------------+--------------
 Arland       | Cold
 Borea        | Cold
 Calidra      | Warm
 Dune         | Hot
(4 rows)
```

Why it works. `WHERE` keeps November rows. Arland's October readings and Dune's December readings are ignored, so Dune's single November reading of 25 makes it Hot. `CASE` can test an aggregate because it runs after grouping. `CASE` stops at the first true branch, so order the thresholds carefully. PostgreSQL's `avg(int)` returns `numeric`, so there is no truncation. **Edge case:** a country with no November data simply doesn't appear. If it must appear, `LEFT JOIN` from `country` with the date filter in `ON`, and decide what label an unknown average gets.

P30 · Engagement level for February 2020 — leap-year boundary

Problem. `app_user(user_id, user_name)` and `engagement(user_id, score, day)`. For February 2020, classify each user's average score: below 20 Low, 20–60 inclusive Medium, above 60 High.

```sql
SELECT u.user_name,
       CASE WHEN avg(e.score) < 20  THEN 'Low'
            WHEN avg(e.score) <= 60 THEN 'Medium'
            ELSE 'High' END AS engagement_level
FROM app_user u
JOIN engagement e ON e.user_id = u.user_id
WHERE e.day >= DATE '2020-02-01' AND e.day < DATE '2020-03-01'
GROUP BY u.user_id, u.user_name
ORDER BY u.user_name;
```

```
 user_name | engagement_level
-----------+------------------
 Ravi      | Medium
 Sofia     | Medium
 Theo      | High
(3 rows)
```

Why it works / edge case. Same shape as P29, but this data hits a boundary. 2020 is a leap year, and Ravi's 90 was recorded on **29 February**. A hand-written `BETWEEN '2020-02-01' AND '2020-02-28'` drops it, and Ravi's average falls from 50 (Medium) to 10 (Low). The half-open `< DATE '2020-03-01'` never needs to know how long the month is. Sofia's 1 March reading is correctly excluded. (A MySQL-style `YEAR(d) = 2020 AND MONTH(d) = 2` is also correct but can't use a plain index on `d`.)

P31 · Absences longer than three days — date subtraction, inclusive counting

Problem. `absence(employee_id, employee_name, start_date, end_date)` with start ≤ end. Return absences that last more than three days, where both the first and the last day count as days off.

```sql
SELECT employee_name, start_date, end_date, end_date - start_date + 1 AS days_off
FROM absence
WHERE end_date - start_date + 1 > 3
ORDER BY employee_name;
```

```
 employee_name | start_date |  end_date  | days_off
---------------+------------+------------+----------
 Hana          | 2024-03-01 | 2024-03-05 |        5
 Jade          | 2024-02-27 | 2024-03-01 |        4
(2 rows)
```

Why it works / edge cases. `end - start` counts the *gaps* between days, and `+ 1` counts the *days*. Ivo (10–12 March) is exactly 3 days, so it is not "more than three". Jade's absence crosses 29 February 2024 and is still counted correctly as 4 days. **Clarify** whether "more than three days" means calendar or working days. For working days, count `generate_series(start, end, '1 day')` rows whose `extract(isodow …) < 6` (ISO day of week: Monday = 1 … Sunday = 7). Dialects: MySQL `DATEDIFF(end, start)` (days), SQL Server `DATEDIFF(day, start, end)`, SQLite `julianday(end) - julianday(start)`.

P32 · Absence streaks of 3+ days — gaps and islands (first taste)

Problem. `attendance(student_id, att_date, status)` with one row per student per day, status Present/Absent. Find every run of **three or more consecutive calendar days** of absence, and report the student and the run's first day.

Data: student 1 is absent 1–3 Jan, present 4 Jan, then absent 5–8 Jan. Student 2 is absent 1–2 and 4–5 Jan (no run of 3). Student 3 is absent 30 Jan – 1 Feb, across a month boundary.

The trick: within a run of consecutive dates, `date − row_number()` is constant. (`row_number()` is a **window function** numbering rows 1, 2, 3… within each `PARTITION BY` group in the given order; `PARTITION BY` splits rows into groups without collapsing them.)

```sql
SELECT student_id, att_date,
       row_number() OVER w AS rn,
       att_date - (row_number() OVER w)::int AS grp
FROM attendance
WHERE status = 'Absent' AND student_id = 1
WINDOW w AS (PARTITION BY student_id ORDER BY att_date)
ORDER BY att_date;
```

```
 student_id |  att_date  | rn |    grp
------------+------------+----+------------
          1 | 2024-01-01 |  1 | 2023-12-31
          1 | 2024-01-02 |  2 | 2023-12-31
          1 | 2024-01-03 |  3 | 2023-12-31
          1 | 2024-01-05 |  4 | 2024-01-01
          1 | 2024-01-06 |  5 | 2024-01-01
          1 | 2024-01-07 |  6 | 2024-01-01
          1 | 2024-01-08 |  7 | 2024-01-01
(7 rows)
```

```sql
WITH absent AS (
  SELECT student_id, att_date,
         att_date - (row_number() OVER (PARTITION BY student_id ORDER BY att_date))::int AS grp
  FROM attendance
  WHERE status = 'Absent'
)
SELECT student_id, min(att_date) AS start_date, count(*) AS days
FROM absent
GROUP BY student_id, grp
HAVING count(*) >= 3
ORDER BY student_id, start_date;
```

```
 student_id | start_date | days
------------+------------+------
          1 | 2024-01-01 |    3
          1 | 2024-01-05 |    4
          3 | 2024-01-30 |    3
(3 rows)
```

Why it works. When the date goes up by 1 and the row number also goes up by 1, their difference stays the same. A gap in the dates bumps the difference, which starts a new group (an *island*). Grouping by `(student, grp)` then yields one row per run. `min` gives its start and `count` its length.

Common mistake. Subtracting two `row_number()`s that use the **same** partition and order always gives 0, so every absence lands in one group. The "two row numbers" variant only works when the two numberings differ: one over *all* rows and one over *absent* rows. That variant is what you want when "consecutive" means consecutive **records** (for example school days, skipping weekends) rather than calendar days. Full treatment: Chapter 8 §8.9 and Chapter 19.

P33 · Price on a given date — latest row on or before a date

Problem. `price_change(product_id, new_price, effective_on)`. Each product costs 10 until its first change. Report every product's price on 15 March 2024.

```sql
WITH latest AS (
  SELECT DISTINCT ON (product_id) product_id, new_price
  FROM price_change
  WHERE effective_on <= DATE '2024-03-15'
  ORDER BY product_id, effective_on DESC
)
SELECT p.product_id, COALESCE(l.new_price, 10) AS price
FROM (SELECT DISTINCT product_id FROM price_change) AS p
LEFT JOIN latest l USING (product_id)
ORDER BY p.product_id;
```

```
 product_id | price
------------+-------
          1 |    30
          2 |    50
          3 |    10
(3 rows)
```

Why it works. The CTE throws away future changes and then keeps, per product, the row with the newest `effective_on` (`DISTINCT ON` keeps the first row of each group in the `ORDER BY`). Product 1 changed on the 14th and again **on** the 15th, and `<=` picks the 15th (30). Product 3's first change is on the 18th, so it has no row in `latest`, and the LEFT JOIN plus `COALESCE` gives the default 10.

Alternatives. A correlated scalar subquery, `COALESCE((SELECT new_price … ORDER BY effective_on DESC LIMIT 1), 10)`, gives the same result and is fast with an index on `(product_id, effective_on)`. `ROW_NUMBER() … = 1` works in every engine. In a real schema, the list of products should come from a `product` table, because products that never changed price would be missing from `price_change`.

18.8 Ratios and percentages (P34–P38)

Ratio questions are mostly about **arithmetic types**. In PostgreSQL, SQL Server and SQLite, `int / int` is integer division (`2/3 = 0`), whereas MySQL's `/` returns a decimal. Guard every denominator that can be 0 with `NULLIF(den, 0)`, and round at the end, not in the middle.

P34 · Search query quality — averaging a ratio, share of "poor" rows

Problem. `search_log(query_name, result, position, rating)` (duplicates allowed). Per query name: *quality* is the average of `rating / position`, and *poor %* is the percentage of rows with rating below 3. Round both to 2 decimals.

```
SELECT query_name,
       round(avg(rating::numeric / position), 2)                      AS quality,
       round(100.0 * count(*) FILTER (WHERE rating < 3) / count(*), 2) AS poor_query_percentage
FROM search_log
GROUP BY query_name
ORDER BY query_name;
```

```
 query_name | quality | poor_query_percentage
------------+---------+-----------------------
 Cat        |    0.66 |                 33.33
 Dog        |    2.50 |                 33.33
(2 rows)
```

Why it works. The cast makes each ratio exact (Dog: 5, 2.5, 0.005 → average 2.50). `100.0 * …` forces decimal arithmetic before the division.

The bug to mention. Without the casts, every `rating / position` truncates (5/2 → 2, 1/200 → 0) and the poor share becomes `1 / 3 * 100 = 0`:

```
 query_name |      int_quality       | int_poor_pct
------------+------------------------+--------------
 Cat        | 0.3333333333333333333 |            0
 Dog        |     2.3333333333333333 |            0
(2 rows)
```

MySQL gets these right without casts (its `/` is decimal), which is why solutions ported from MySQL often break on PostgreSQL. Also note that `round(double precision, int)` doesn't exist in PostgreSQL. Keep the arithmetic in `numeric`.

P35 · Ad click-through rate — divide by a possibly-zero count

Problem. `ad_action(ad_id, user_id, action)` with actions Clicked / Viewed / Ignored. **CTR** (click-through rate, the share of impressions that led to a click) = clicks × 100 / (clicks + views), or 0 when an ad has neither. Round to 2 decimals and sort by CTR (highest first), then by ad id.

```
SELECT ad_id,
       round(COALESCE(100.0 * count(*) FILTER (WHERE action = 'Clicked')
                      / NULLIF(count(*) FILTER (WHERE action IN ('Clicked','Viewed')), 0), 0), 2) AS ctr
FROM ad_action
GROUP BY ad_id
ORDER BY ctr DESC, ad_id;
```

```
 ad_id |  ctr
-------+-------
     1 | 66.67
     3 | 50.00
     2 | 33.33
     5 |  0.00
(4 rows)
```

Why it works. Two conditional counts give clicks and clicks + views. Ad 5 was only ignored, so its denominator is 0. `NULLIF(0, 0)` turns it into NULL, the division yields NULL instead of an error, and

`COALESCE(…, 0)` applies the spec's rule. "Ignored" rows are in the group but excluded from both counts. The secondary sort key `ad_id` makes ties deterministic.

P36 · Each book's share of all loans — ratio against a grand total

Problem. `book(book_id, title)` and `loan(loan_id, book_id, user_id, loan_date)`. A book's loan share is (its loans) / (all loans). Show every book's share rounded to 2 decimals, highest first, ties by id.

```
SELECT b.book_id,
       round(count(l.loan_id)::numeric / NULLIF((SELECT count(*) FROM loan), 0), 2) AS lf
FROM book b
LEFT JOIN loan l ON l.book_id = b.book_id
GROUP BY b.book_id
ORDER BY lf DESC, b.book_id;
```

```
 book_id |  lf
---------+------
       1 | 0.50
       2 | 0.33
       3 | 0.17
       4 | 0.00
(4 rows)
```

Why it works. The numerator is per group. The **denominator** (the bottom of the fraction; the **numerator** is the top) is the whole-table count from an uncorrelated scalar subquery, which runs once. Book 4 was never lent, so the LEFT JOIN keeps it and `count(l.loan_id)` is 0. Without `::numeric`, 3/6 would be 0 in PostgreSQL.

Alternative. A window over the grouped result avoids the second scan: `count(l.loan_id)::numeric / NULLIF(sum(count(l.loan_id)) OVER (), 0)`. Window functions run after `GROUP BY`, so they can aggregate the aggregates. This gives the same output here. Keep the `NULLIF`: with an empty `loan` table the window sum is 0 and the division would fail. The two denominators differ only if `loan` could hold book ids missing from `book`, which the foreign key rules out.

P37 · Average selling price — weighted average with a range join

Problem. `price(product_id, start_d, end_d, price)` holds non-overlapping price periods. `units_sold(product_id, bought_on, units)` holds sales. For each product, the average selling price is total revenue / total units, using the price valid on each sale's date. Products that never sold show 0. Round to 2 decimals.

```
SELECT p.product_id,
       round(COALESCE(sum(u.units * p.price)::numeric / sum(u.units), 0), 2) AS average_price
FROM price p
LEFT JOIN units_sold u
       ON u.product_id = p.product_id
      AND u.bought_on BETWEEN p.start_d AND p.end_d
GROUP BY p.product_id
ORDER BY p.product_id;
```

```
 product_id | average_price
------------+---------------
          1 |          6.96
          2 |         16.96
          3 |          0.00
(3 rows)
```

Why it works. This is a **range join** (the join condition is "falls inside an interval", not equality). The join matches each sale to exactly one price period (same product, date inside the period). Product 1 sold 100 units at 5 and 15 at 20: (500 + 300) / 115 = 6.96. Product 3 has a price but no sales. Its sums are NULL, NULL/NULL is NULL (not an error), and `COALESCE` makes it 0.

The bug. `avg(p.price)` averages the *price rows* (5 and 20 → 12.50) and ignores that most units sold at 5. Whenever rows carry a quantity, use a **weighted** average: `sum(value × weight) / sum(weight)`. If a sale could fall outside every period, it would silently vanish, so check with an anti-join.

P38 · Came back the next day — retention fraction

Problem. `activity(player_id, device_id, played_on, games)`. What share of players showed up again exactly one day after their first-ever login? Round to 2 decimals.

```
WITH first_day AS (
  SELECT player_id, min(played_on) AS first_on FROM activity GROUP BY player_id
)
SELECT round(count(a.player_id)::numeric / count(*), 2) AS fraction
FROM first_day f
LEFT JOIN activity a ON a.player_id = f.player_id AND a.played_on = f.first_on + 1;
```

```
 fraction
----------
     0.50
(1 row)
```

Why it works. The CTE gives one row per player (the denominator). The LEFT JOIN looks for a row exactly one day later (`date + 1`). `count(a.player_id)` counts only players who matched (the numerator). Players 1 and 4 returned (player 4 on 29 February 2024, which `+ 1` handles). Player 3 came back two days later, which doesn't count, and player 2 never came back. The result is 2/4. Without the cast, `count / count` is `0`. This "day-1 retention" metric is a staple of **product-analytics** interviews (questions about measuring how users behave in an app). Variants ask for the fraction *per install date* (group by `first_on`) or for day-7 retention (`+ 7`).

18.9 Window and ranking basics (P39–P42)

A first look at problems that are easiest with **window functions**: functions that compute over related rows without collapsing them. Full coverage is in Chapter 8, and the harder variants are in Chapter 19.

P39 · Second-highest salary — and "return NULL if none"

Problem. `emp(id, salary)`. Return the second-highest **distinct** salary, or NULL when there isn't one.

Data: salaries 300, 300, 200, 100 (a tie at the top).

```
SELECT (SELECT DISTINCT salary FROM emp ORDER BY salary DESC OFFSET 1 LIMIT 1)
       AS second_highest_salary;
```

```
 second_highest_salary
-----------------------
                   200
(1 row)
```

Why it works. `DISTINCT` collapses the two 300s, so `OFFSET 1` skips exactly one value. Wrapping the query in an outer `SELECT (…)` makes it a **scalar subquery**, which yields NULL when the inner query finds

nothing. After deleting all but the 300s, the wrapped query returns `(null)` in one row, while an unwrapped query (the file shows the bare `dense_rank` version) returns 0 rows.

Alternative. `dense_rank() OVER (ORDER BY salary DESC)` numbers distinct values 1, 1, 2, 3. Keep `dr = 2`, and wrap the result in `max(salary)` so an empty result still becomes one NULL row. Without `DISTINCT`, `OFFSET 1` would return the duplicate 300, the most common wrong answer. Another classic is `SELECT max(salary) FROM emp WHERE salary < (SELECT max(salary) FROM emp)`. The general N-th version is in Chapter 19.

P40 · Device used at first login — first row per group

Problem. Using `activity(player_id, device_id, played_on, games)`, report the device each player used on their earliest day.

```
SELECT DISTINCT ON (player_id) player_id, device_id
FROM activity
ORDER BY player_id, played_on;
```

```
 player_id | device_id
-----------+-----------
         1 |         2
         2 |         3
         3 |         1
(3 rows)
```

Why it works. `DISTINCT ON (player_id)` (PostgreSQL only) keeps the first row per player in `ORDER BY` order, so the earliest day wins, and `device_id` comes along with it. That's exactly what `min(played_on)` alone (P11) could not give.

Portable alternatives (same output in the file):

- `row_number() OVER (PARTITION BY player_id ORDER BY played_on) = 1` in a subquery. Use `rank()` if two rows could share the first date and you want both.
- `WHERE (player_id, played_on) IN (SELECT player_id, min(played_on) … GROUP BY player_id)`, a **row-value** `IN` (comparing several columns at once as a tuple). SQL Server doesn't support row-value `IN`, so there you'd join to the grouped subquery instead.

P41 · Games played so far — running total

Problem. For every `(player, day)` in `activity`, show the total games that player has played up to and including that day.

```
SELECT player_id, played_on,
       sum(games) OVER (PARTITION BY player_id ORDER BY played_on) AS games_played_so_far
FROM activity
ORDER BY player_id, played_on;
```

```
 player_id | played_on  | games_played_so_far
-----------+------------+---------------------
         1 | 2024-03-01 |                   5
         1 | 2024-05-02 |                  11
         1 | 2024-06-25 |                  12
         3 | 2024-03-02 |                   0
         3 | 2024-07-03 |                   5
(5 rows)
```

Why it works. `PARTITION BY` restarts the sum per player. `ORDER BY` inside `OVER` makes the default **frame** (the set of rows the window function looks at for the current row) "from the first row up to the current row, including rows that tie on the sort key" (`RANGE UNBOUNDED PRECEDING`). Here `(player_id, played_on)`

is unique, so there are no ties. If two rows could share a date and you want a strict row-by-row total, write `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`.

Alternative. Before window functions (MySQL < 8.0), you'd self-join each row to all earlier-or-same rows of the same player and `sum`. It gives the same output but does O(n²) work per player (work grows with the square of the number of rows: 10× the rows, 100× the work).

P42 · Active businesses — compare each row to its group's average

Problem. `event(business_id, event_type, occurrences)`. The average for an event type is the mean of `occurrences` over the businesses that have it. A business is **active** if, on *more than one* event type, its occurrences are strictly above that type's average. List active businesses.

First attach each type's average to every row:

```
 business_id | event_type | occurrences | type_avg
-------------+------------+-------------+----------
           1 | ads        |          11 |     8.00
           2 | ads        |           7 |     8.00
           3 | ads        |           6 |     8.00
           4 | coupons    |           9 |     9.00
           1 | page views |           3 |     7.50
           2 | page views |          12 |     7.50
           1 | reviews    |           7 |     5.00
           3 | reviews    |           3 |     5.00
(8 rows)
```

```sql
SELECT business_id
FROM (SELECT business_id, occurrences,
             avg(occurrences) OVER (PARTITION BY event_type) AS type_avg
      FROM event) AS e
WHERE occurrences > type_avg
GROUP BY business_id
HAVING count(*) > 1
ORDER BY business_id;
```

```
 business_id
-------------
           1
(1 row)
```

Why it works. The window `avg` keeps every row and adds the type average next to it. You can't write `WHERE occurrences > avg(...)`, because windows are evaluated after `WHERE`, hence the subquery. Business 1 beats the average on ads and reviews (2 types), and business 2 only on page views. Business 4 is alone in "coupons", so it equals the average and never beats it. **Alternative without windows:** join `event` to `SELECT event_type, avg(occurrences) … GROUP BY event_type` on `event_type`. **Edge case:** "more than one" means `> 1`, not `>= 1`, and "strictly greater" means `>`. Read comparison words carefully.

18.10 Pattern table

#	Problem	Pattern	Key SQL feature
P1	Big cities	Filtering	`OR`, three-valued logic with NULL
P2	Odd non-boring movies	Filtering	`%`, `<>`, `IS DISTINCT FROM`, `ORDER BY DESC`
P3	Green products	Filtering	`AND`, `CHECK` constraint
P4	Outside department 3	Filtering	NULL-safe `IS DISTINCT FROM`
P5	Self-assigned tasks	Filtering	column-to-column compare, `DISTINCT`
P6	Long comments	Filtering	`char_length` vs `octet_length`
P7	Valid e-mails	Filtering	regex `~`, anchors, escaping
P8	Triangle test	Row-level logic	`CASE`, `greatest`
P9	Largest unique number	Aggregation	`HAVING count(*) = 1`, aggregate over empty set → NULL
P10	Units per product	Aggregation	`SUM` + `GROUP BY`, `LEFT JOIN` + `COALESCE`
P11	First login	Aggregation	`MIN` on dates
P12	Follower count	Aggregation	`count(*)` vs `count(col)`
P13	Bike last used	Aggregation	`MAX`, sort by aggregate, NULL handling
P14	Missed appointments	Aggregation	`WHERE` before `GROUP BY`; `FILTER` + `HAVING`
P15	Products per day	Aggregation	`count(DISTINCT)`, `string_agg(DISTINCT … ORDER BY)`
P16	Customer basket	Aggregation	`string_agg` / `array_agg`
P17	Sale product names	Joins	`INNER JOIN` on a key
P18	Process time	Joins	self-join; `FILTER` pivot alternative
P19	Fiction per author	Joins	join + filter + group by key, two-key sort
P20	Exam sittings	Joins	`CROSS JOIN` grid + `LEFT JOIN` + `count(col)`
P21	Election winner	Joins / top-1	`ORDER BY count(*) DESC LIMIT 1`, `WITH TIES`
P22	Middle followers	Joins / semi-join	`EXISTS`; self-join needs `count(DISTINCT)`
P23	Idle sellers	Anti-join	`NOT EXISTS`, half-open date range
P24	Dormant accounts	Anti-join	`LEFT JOIN … IS NULL`, filter in `ON`; `EXCEPT`
P25	Ad-free sessions	Anti-join	`NOT EXISTS` with `BETWEEN` range condition
P26	Reps without RED	Anti-join	`NOT EXISTS` vs `NOT IN` + NULL
P27	Attendance summary	Conditional aggregation	`count(*) FILTER`, `SUM(CASE … ELSE 0)`
P28	Monthly chargebacks	Conditional aggregation	`UNION ALL` then `FILTER`, `to_char` month
P29	Weather class	Date logic	month range + `CASE` on `avg`
P30	Engagement level	Date logic	half-open range (leap day), `CASE` buckets
P31	Long absences	Date logic	`date - date`, inclusive counting

#	Problem	Pattern	Key SQL feature
P32	Absence streaks	Date logic / gaps & islands	`date - row_number()` grouping
P33	Price on a date	Date logic / as-of	`DISTINCT ON` or `LIMIT 1` subquery + `COALESCE` default
P34	Query quality	Ratios	cast before dividing, `FILTER` share
P35	Ad CTR	Ratios	`NULLIF` + `COALESCE` for zero denominators
P36	Loan share	Ratios	ratio to grand total (scalar subquery or `sum() OVER ()`)
P37	Average selling price	Ratios	range join, weighted average
P38	Next-day return	Ratios / retention	`min` CTE + `LEFT JOIN` on `date + 1`
P39	Second-highest salary	Ranking	`DISTINCT … OFFSET 1` in scalar subquery, `dense_rank`
P40	First-login device	Ranking / first per group	`DISTINCT ON`, `row_number() = 1`
P41	Games so far	Window	running `sum() OVER (PARTITION BY … ORDER BY …)`
P42	Active businesses	Window	`avg() OVER (PARTITION BY)` then `HAVING count(*) > 1`

Chapter 18 TL;DR

- **Method:** clarify grain, keys, NULLs, zero rows, ties and output. Sketch rows, build in evaluation order with CTEs, attack edge cases, then verify and discuss indexes.
- **Filtering:** `WHERE` keeps only `TRUE`. `NULL` comparisons give `UNKNOWN`, so use `IS DISTINCT FROM` when NULLs must count. Anchor and escape regexes. Count characters, not bytes.
- **Aggregation:** filter before grouping. `count(col)` skips NULLs, which is what makes zero-counts work after a LEFT JOIN. An aggregate over nothing returns one NULL row, while `LIMIT 1` over nothing returns no row.
- **Joins:** group by keys, not names. Build "every pair" grids with `CROSS JOIN` + `LEFT JOIN`. Watch for fan-out and prefer `EXISTS` for "has at least one". Use `WITH TIES` or `RANK` for fair top-1 answers.
- **Anti-joins:** `NOT EXISTS` is the safe default. With `LEFT JOIN … IS NULL`, the right-table filters belong in `ON`. `NOT IN` returns nothing if the subquery holds a NULL.
- **Conditional aggregation:** `agg(...) FILTER (WHERE …)` or `SUM(CASE … ELSE 0)` pivots values into columns. Stack differently-dated events with `UNION ALL` first.
- **Dates:** half-open ranges (`>= start AND < next_start`); `date - date` is days; `+ 1` for "inclusive" counts; `date - row_number()` finds consecutive runs; "as of a date" = latest row `<=` the date + a default.
- **Ratios:** cast to `numeric` (or `* 100.0`) before dividing, `NULLIF` the denominator, round last, and use weighted averages when rows carry quantities.
- **Windows:** `dense_rank`/`DISTINCT … OFFSET` for N-th highest (wrap for NULL), `DISTINCT ON`/`row_number` for the first row per group, `sum() OVER (… ORDER BY …)` for running totals, `avg() OVER (PARTITION BY …)` to compare a row with its group.

Next: Chapter 19 — SQL interview problems, part 2, with harder problems: top-N per group, streaks, pivots and relational division.

Chapter 19 — SQL Interview Problems 2: Medium and Hard Patterns

The previous chapter practised the basics (filters, grouping, joins, anti-joins, simple ratios). This chapter covers the medium and hard problems: ranking inside groups, running sums, streaks in ordered data, rows into columns (even when the columns are unknown), tables as small graphs, date buckets, and strings built from rows.

Each problem has a statement in our own words, a self-contained file in `code/` (tables, a tiny invented sample, the solution and often an alternative), the real `psql` (PostgreSQL's command-line client) output (`(null)` marks NULL, SQL's "missing/unknown" value), why it works, and the trap or alternative worth mentioning. All 41 files run on PostgreSQL 14 with `bash tools_run_sql.sh sql '19-*'`.

What you'll learn

- The patterns behind most medium/hard questions, with a template for each (19.11).
- `ROW_NUMBER` vs `RANK` vs `DENSE_RANK` (**window functions** — functions that compute over a set of related rows without collapsing them — that number rows), and why ties change the answer.
- The *gaps-and-islands* trick (`value - ROW_NUMBER()`) for streaks and "N in a row".
- Static pivots (rows turned into columns) with `FILTER`, dynamic pivots with **PL/pgSQL** (PostgreSQL's built-in procedural language, with variables and loops) + `format()` (builds a string from a template), and unpivot (columns turned back into rows) via **JSON** (a text format of nested key/value objects).
- PostgreSQL traps: **integer division** (`int / int` drops the fraction, so `1/3 = 0`), division by zero, `NOT IN` with NULLs, `COUNT(*)` after a `LEFT JOIN`.

Background you may want open: window functions (ranking, frames, gaps and islands), subqueries and set operations, CTEs, and the interview method in Chapter 18.

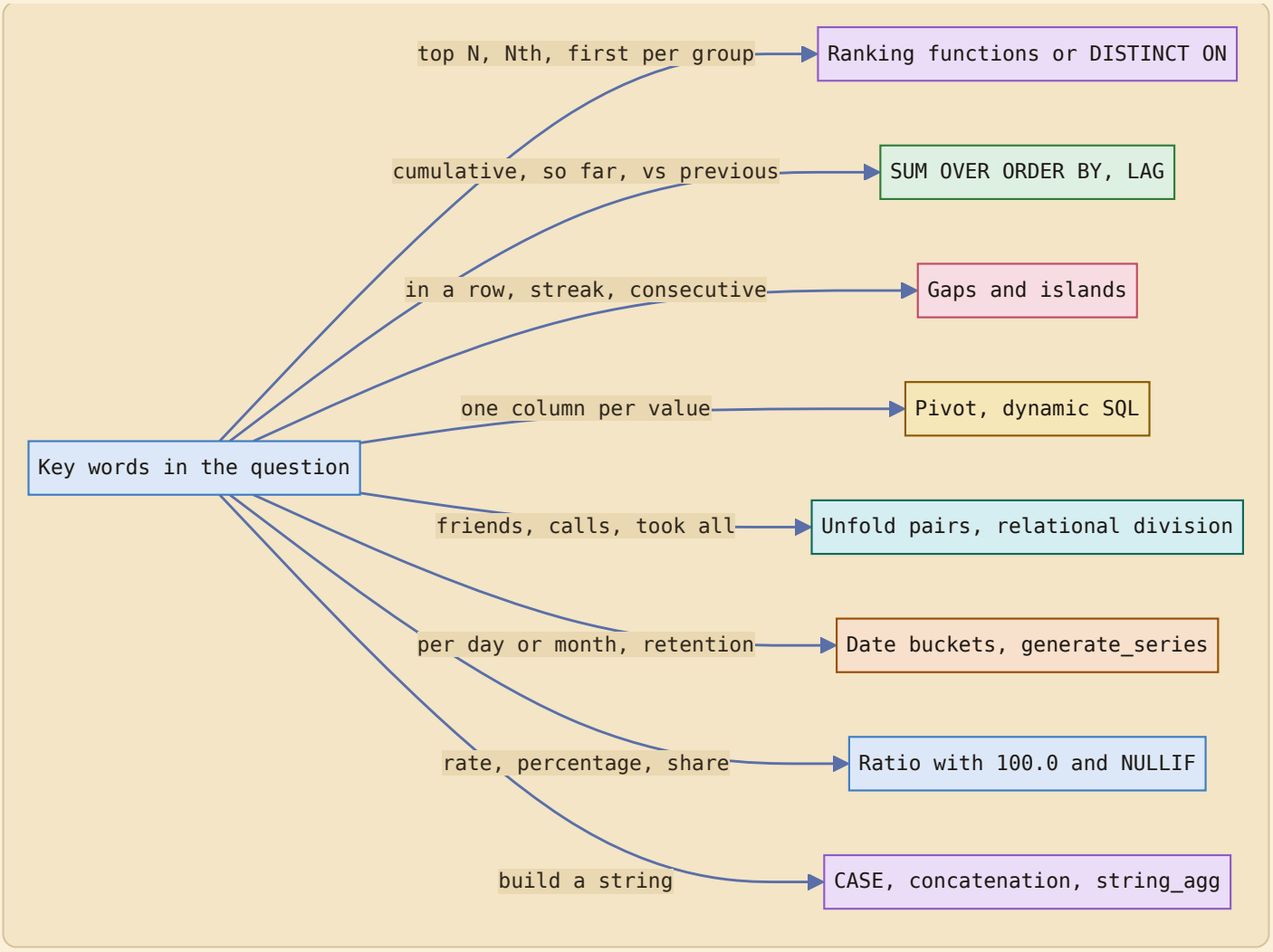


Figure — typical wording in a question and the pattern it usually points to.

19.1 How to attack a medium or hard problem

The Chapter 18 method still applies; three habits matter most here:

Habit	What it means in practice
Build in layers	One **CTE** (common table expression — a named sub-query in a `WITH` clause) per idea ("one row per user-day", then "rank within the day", then "keep rank 1"); run each layer before adding the next.
Say the tie rule out loud	"If two invoices have the same total, which one wins?" decides between `ROW_NUMBER`, `RANK` and `DENSE_RANK`.
Hunt the empty case	Players with no wins, orders with no deliveries, days with no visits: decide whether they appear with 0, with NULL, or not at all.

Run any file on its own with `bash tools_pg.sh psql -d <empty_db> -f code/NN_name.sql`.

19.2 Warm-ups: joins, counts and grids

Three short problems that hide a classic trap each: NULL in a comparison, `COUNT(*)` after an outer join, and "every combination" questions.

19.2.1 Staff rated above 8

Problem. Two tables: `staff` (id, name, team) and `reviews` (one score per staff member). List the names of people whose score is strictly greater than 8.

`code/01_high_rated_staff.sql`

```
CREATE TABLE staff   (staff_id int PRIMARY KEY, full_name text, team text);
CREATE TABLE reviews (staff_id int PRIMARY KEY REFERENCES staff, score int);
INSERT INTO staff VALUES (1,'Asha','Web'),(2,'Bruno','Web'),(3,'Chen','Data'),(4,'Dana','Data');
INSERT INTO reviews VALUES (1,9),(2,8),(3,10),(4,NULL);

SELECT s.full_name
FROM   staff s
JOIN   reviews r ON r.staff_id = s.staff_id
WHERE  r.score > 8
ORDER  BY s.full_name;

-- Edge case: NULL score is neither > 8 nor <= 8
SELECT count(*) FILTER (WHERE score > 8)      AS above,
       count(*) FILTER (WHERE NOT score > 8)  AS not_above,
       count(*)                               AS all_rows
FROM   reviews;
```

Output:

```
 full_name
-----------
 Asha
 Chen
(2 rows)

 above | not_above | all_rows
-------+-----------+----------
     2 |         1 |        4
(1 row)
```

Why it works. The **inner join** (keeps only rows that have a match on both sides) pairs each person with their review; `WHERE r.score > 8` keeps the high ones. Anyone without a review row disappears in the join, which is what "has a rating above 8" means.

Edge case. Dana's score is NULL: `NULL > 8` and `NOT (NULL > 8)` are both *unknown*, so she lands in neither bucket (2 + 1 ≠ 4). "Everyone not above 8" needs `OR score IS NULL`.

19.2.2 Likes per post

Problem. `posts` lists posts; `post_likes` has one row per (post, user who liked it). Show every post with its number of likes, including posts nobody liked, in post order.

`code/02_likes_per_post.sql`

```
CREATE TABLE posts      (post_id int PRIMARY KEY, author text);
CREATE TABLE post_likes (post_id int REFERENCES posts, liker_id int, PRIMARY KEY (post_id, liker_id));
INSERT INTO posts VALUES (10,'ana'),(11,'ben'),(12,'cy');
INSERT INTO post_likes VALUES (10,1),(10,2),(10,3),(11,1);

SELECT p.post_id, count(l.liker_id) AS likes
FROM   posts p
LEFT   JOIN post_likes l ON l.post_id = p.post_id
GROUP  BY p.post_id
ORDER  BY p.post_id;
```

Output:

```
 post_id | likes
---------+-------
      10 |     3
      11 |     1
      12 |     0
(3 rows)
```

Why it works. The `LEFT JOIN` (an **outer join**: keeps every row of the left table, matched or not) keeps post 12 even though it has no likes; its join columns come back as NULL (a **NULL-padded** row). `count(l.liker_id)` counts only non-NULL values, so the padded row contributes 0.

Common mistake. `count(*)` counts the NULL-padded row too, so post 12 would show 1 (the file runs it; the same trap as Chapter 18, P12). After an outer join, count a column from the *optional* side.

19.2.3 Average mark for every learner × module

Problem. For every learner and every module in the catalogue (even ones the learner has no marks in), show the learner's average mark in that module, rounded to 2 decimals; NULL when there are no marks. Sort by learner, then module.

`code/03_course_grid_averages.sql`

```
CREATE TABLE learners (learner_id int PRIMARY KEY, name text);
CREATE TABLE modules  (module text PRIMARY KEY);
CREATE TABLE marks    (learner_id int, module text, mark int);
INSERT INTO learners VALUES (1,'Ivy'),(2,'Omar');
INSERT INTO modules  VALUES ('SQL'),('Stats');
INSERT INTO marks    VALUES (1,'SQL',70),(1,'SQL',85),(1,'Stats',60),(2,'SQL',90);

SELECT l.learner_id, l.name, m.module,
       round(avg(k.mark), 2) AS avg_mark
FROM   learners l
CROSS  JOIN modules m
LEFT   JOIN marks k ON k.learner_id = l.learner_id AND k.module = m.module
GROUP  BY l.learner_id, l.name, m.module
ORDER  BY l.learner_id, m.module;
```

Output:

```
 learner_id | name | module | avg_mark
------------+------+--------+----------
          1 | Ivy  | SQL    |    77.50
          1 | Ivy  | Stats  |    60.00
          2 | Omar | SQL    |    90.00
          2 | Omar | Stats  |   (null)
(4 rows)
```

Why it works. "Every A with every B" is a `CROSS JOIN` (every row of one table paired with every row of the other) — it builds the full grid of 2 × 2 pairs first. The `LEFT JOIN` then attaches marks where they exist, and `avg()` of only NULLs is NULL, which is exactly the "no marks" answer (Omar/Stats).

Alternative. If the question only wants modules the learner actually took, drop the cross join and group the `marks` table directly. Always ask which one is meant.

19.3 Nth highest, top-N per group and "first row per group"

The ranking family. The three ranking functions differ only on ties:

Scores	ROW_NUMBER	RANK	DENSE_RANK
95, 95, 90	1, 2, 3	1, 1, 3	1, 1, 2

Use `ROW_NUMBER` when exactly one row must win, `RANK` when "all tied for first" should be kept, and `DENSE_RANK` when the question says "top N *distinct* values". PostgreSQL's `DISTINCT ON (cols)` is a compact shortcut for "first row per group": it keeps the first row of each group of equal `cols` values, in `ORDER BY` order.

19.3.1 Second-fastest lap time

Problem. Each runner has one lap time, in seconds. Return the second-smallest *distinct* time as one row, or a single NULL row when there is no second value.

`code/04_second_fastest.sql`

```
CREATE TABLE laps (runner_id int PRIMARY KEY, seconds int NOT NULL);
INSERT INTO laps VALUES (1,62),(2,58),(3,58),(4,65),(5,61);

-- Solution: scalar subquery -> NULL when OFFSET skips past the end
SELECT (SELECT DISTINCT seconds FROM laps
        ORDER  BY seconds
        OFFSET 1 LIMIT 1) AS second_fastest;

-- Alternative: DENSE_RANK generalises to the N-th
SELECT max(seconds) FILTER (WHERE dr = 2) AS second_fastest
FROM  (SELECT seconds, dense_rank() OVER (ORDER BY seconds) AS dr FROM laps) t;

-- Edge case: only one distinct time left
DELETE FROM laps WHERE seconds <> 58;
SELECT (SELECT DISTINCT seconds FROM laps ORDER BY seconds OFFSET 1 LIMIT 1) AS second_fastest;
```

Output (both formulations give 61; after the `DELETE` the answer is a NULL row):

```
 second_fastest
----------------
             61
(1 row)
...
 second_fastest
----------------
         (null)
(1 row)
```

Why it works. Same idea as Chapter 18, P39: `DISTINCT` collapses the two 58s, `OFFSET 1 LIMIT 1` takes the next value, and the **scalar-subquery** wrapper (a sub-query used as a single value) turns "no row" into one NULL row.

Alternatives. `min(seconds) WHERE seconds > (SELECT min(seconds) …)` (also in the file) works only for N = 2. `DENSE_RANK` handles any N (`dr = N`), and wrapping it in `max(...) FILTER (...)` again turns "no row" into NULL.

19.3.2 Top three distinct scores per subject

Problem. Pupils' scores per subject. For each subject, list every pupil whose score is one of the three highest *different* scores in that subject (ties included).

`code/05_top3_per_subject.sql`

```
CREATE TABLE subjects (subject_id int PRIMARY KEY, title text);
CREATE TABLE results  (result_id int PRIMARY KEY, pupil text, score int,
                       subject_id int REFERENCES subjects);
INSERT INTO subjects VALUES (1,'Maths'),(2,'History');
INSERT INTO results VALUES
 (1,'Ann',95,1),(2,'Bob',95,1),(3,'Cal',90,1),(4,'Dee',82,1),(5,'Eve',70,1),
 (6,'Fay',88,2),(7,'Gus',75,2);

WITH ranked AS (
  SELECT s.title, r.pupil, r.score,
         dense_rank() OVER (PARTITION BY r.subject_id ORDER BY r.score DESC) AS dr
  FROM   results r
  JOIN   subjects s USING (subject_id)
)
SELECT title AS subject, pupil, score
FROM   ranked
WHERE  dr <= 3
ORDER  BY subject, score DESC, pupil;

-- Alternative: correlated subquery "fewer than 3 distinct higher scores"
SELECT s.title AS subject, r.pupil, r.score
FROM   results r
JOIN   subjects s USING (subject_id)
WHERE  (SELECT count(DISTINCT r2.score) FROM results r2
        WHERE r2.subject_id = r.subject_id AND r2.score > r.score) < 3
ORDER  BY subject, score DESC, pupil;
```

Output (all queries return these rows):

```
 subject | pupil | score
---------+-------+-------
 History | Fay   |    88
 History | Gus   |    75
 Maths   | Ann   |    95
 Maths   | Bob   |    95
 Maths   | Cal   |    90
 Maths   | Dee   |    82
(6 rows)
```

Why it works. `DENSE_RANK` gives Ann and Bob rank 1, Cal rank 2, Dee rank 3, so four pupils appear for Maths — ties don't use up places. History has only two pupils, so both appear.

Alternative. The **correlated** version (its sub-query refers to the outer row, so it re-runs for each row) keeps a row if fewer than three distinct scores in its subject beat it: **portable** (works on any SQL database), but O(n²) per group (work grows with the square of the row count). With `RANK`, Dee would get rank 4 and vanish — a common wrong answer.

19.3.3 One home office per employee

Problem. Employees can be assigned to several offices; exactly one of them is flagged `'Y'` as the home office. Someone with only one office has that office as home, whatever the flag says. Return each employee with their home office.

`code/06_home_office.sql`

```
CREATE TABLE desk_assignments (emp_id int, office_id int, is_home char(1) NOT NULL CHECK (is_home IN
('Y','N')),
                              PRIMARY KEY (emp_id, office_id));
INSERT INTO desk_assignments VALUES
 (1,100,'N'),(1,200,'Y'),(1,300,'N'),   -- several offices, one flagged
 (2,100,'N'),                            -- single office, flag ignored
 (3,300,'Y');

-- Solution: DISTINCT ON keeps the first row of each emp_id in ORDER BY order
SELECT DISTINCT ON (emp_id) emp_id, office_id
FROM   desk_assignments
ORDER  BY emp_id, is_home DESC;      -- 'Y' sorts after 'N', so DESC puts it first

-- Alternative: the rule written out literally
SELECT emp_id, office_id FROM desk_assignments WHERE is_home = 'Y'
UNION
SELECT emp_id, min(office_id) FROM desk_assignments
GROUP  BY emp_id HAVING count(*) = 1
ORDER  BY emp_id;
```

Output (all queries return these rows):

```
 emp_id | office_id
--------+-----------
      1 |       200
      2 |       100
      3 |       300
(3 rows)
```

Why it works. `DISTINCT ON (emp_id)` keeps each employee's first row under `ORDER BY emp_id, is_home DESC`; `'Y' > 'N'`, so a flagged office sorts first, and a single office is kept as is. The flag is `NOT NULL` on purpose: PostgreSQL sorts NULLs *first* under `DESC`, so a NULL flag would beat `'Y'` (add `NULLS LAST` if NULLs are possible). Portable form: `ROW_NUMBER() OVER (PARTITION BY emp_id ORDER BY is_home DESC) = 1`.

Alternative / edge case. The `UNION` version states the rule literally. If the data breaks the rule (several offices, none flagged), the ranking version silently picks *some* office and the `UNION` version returns nothing for that person — choose your failure.

19.3.4 The most expensive invoice's lines

Problem. `items` holds unit prices; `line_items` holds (invoice, item, quantity). Find the invoice with the largest total — smallest invoice id wins a tie — and list its lines with quantity and line price.

`code/07_largest_invoice.sql`

```
CREATE TABLE items      (item_id int PRIMARY KEY, unit_price int);
CREATE TABLE line_items (invoice_id int, item_id int REFERENCES items, qty int,
                         PRIMARY KEY (invoice_id, item_id));
INSERT INTO items VALUES (1,10),(2,25),(3,4);
INSERT INTO line_items VALUES
 (501,1,3),(501,3,5),        -- 30 + 20 = 50
 (502,2,2),                  -- 50  (tie with 501)
 (503,1,1),(503,3,2);        -- 18

WITH priced AS (
  SELECT l.invoice_id, l.item_id, l.qty, l.qty * i.unit_price AS line_total,
         sum(l.qty * i.unit_price) OVER (PARTITION BY l.invoice_id) AS invoice_total
  FROM   line_items l JOIN items i USING (item_id)
), winner AS (
  SELECT invoice_id FROM priced
  ORDER  BY invoice_total DESC, invoice_id
  LIMIT  1
)
SELECT item_id, qty, line_total
FROM   priced
WHERE  invoice_id = (SELECT invoice_id FROM winner)
ORDER  BY item_id;
```

Output:

```
 item_id | qty | line_total
---------+-----+------------
       1 |   3 |         30
       3 |   5 |         20
(2 rows)
```

Why it works. A window `sum(...) OVER (PARTITION BY invoice_id)` puts each invoice's total next to each of its lines without collapsing them. The `winner` CTE sorts by total descending, then id ascending, and keeps one row. Invoices 501 and 502 both total 50, so 501 wins on id.

Alternative. Group `line_items` by invoice to get totals, pick the winner with `ORDER BY ... LIMIT 1`, then join back — the same logic with one more pass. `FETCH FIRST 1 ROW WITH TIES` is the tool if the question instead wanted *all* tied invoices.

19.3.5 Confirmed or waitlisted

Problem. Flights have a seat count; bookings have a passenger, a flight and a unique booking time. A booking is confirmed if, in booking-time order, it is within the flight's seat count; otherwise it is waitlisted.

`code/08_seat_status.sql`

```
CREATE TABLE flights  (flight_id int PRIMARY KEY, seats int);
CREATE TABLE bookings (passenger_id int PRIMARY KEY, flight_id int, booked_at timestamp UNIQUE);
INSERT INTO flights VALUES (7,2),(8,1),(9,0);
INSERT INTO bookings VALUES
 (1,7,'2026-03-01 09:00'),(2,7,'2026-03-01 08:00'),(3,7,'2026-03-02 10:00'),
 (4,8,'2026-03-03 12:00'),(5,8,'2026-03-01 12:00'),
 (6,9,'2026-03-01 07:00');

SELECT b.passenger_id,
       CASE WHEN row_number() OVER (PARTITION BY b.flight_id ORDER BY b.booked_at) <= f.seats
            THEN 'Confirmed' ELSE 'Waitlist' END AS status
FROM   bookings b
JOIN   flights f USING (flight_id)
ORDER  BY b.passenger_id;
```

Output:

```
 passenger_id |  status
--------------+-----------
            1 | Confirmed
            2 | Confirmed
            3 | Waitlist
            4 | Waitlist
            5 | Confirmed
            6 | Waitlist
(6 rows)
```

Why it works. `row_number()` numbers the bookings of each flight in time order; comparing that number with `seats` gives the status in one pass. On flight 7, passenger 2 booked first, then 1, then 3 (waitlisted). Flight 9 has zero seats, so even the first booking waits.

Edge case. Booking times are unique here, so `RANK` and `ROW_NUMBER` agree. If two bookings could share a timestamp, `RANK` would confirm both (possibly overbooking); `ROW_NUMBER` with a tie-breaker such as `passenger_id` keeps the count exact.

19.3.6 Pupils who are never top or bottom

Problem. A "middle-of-the-pack" pupil sat at least one test and never had the highest or the lowest score in any test they sat (ties for top or bottom count as top or bottom). List them by id.

`code/09_middle_of_the_pack.sql`

```
CREATE TABLE pupils (pupil_id int PRIMARY KEY, name text);
CREATE TABLE tests  (test_id int, pupil_id int, score int, PRIMARY KEY (test_id, pupil_id));
INSERT INTO pupils VALUES (1,'Ada'),(2,'Ben'),(3,'Cleo'),(4,'Dev'),(5,'Eli'),(6,'Fin');
INSERT INTO tests VALUES
 (10,1,60),(10,2,75),(10,3,90),(10,5,70),
 (20,1,55),(20,2,80),(20,4,80),(20,5,62);   -- Ben and Dev tie for top of test 20; Fin sat nothing

WITH marked AS (
  SELECT pupil_id,
         score = max(score) OVER (PARTITION BY test_id)
      OR score = min(score) OVER (PARTITION BY test_id) AS at_extreme
  FROM   tests
)
SELECT p.pupil_id, p.name
FROM   pupils p
JOIN   marked m USING (pupil_id)
GROUP  BY p.pupil_id, p.name
HAVING NOT bool_or(m.at_extreme)
ORDER  BY p.pupil_id;
```

Output:

```
 pupil_id | name
----------+------
        5 | Eli
(1 row)
```

Why it works. Two window aggregates put each test's max and min on every row, so `at_extreme` is true when a pupil matches either. Grouping per pupil and keeping `NOT bool_or(at_extreme)` (`bool_or` is true if any value in the group is true) means "no test where they were extreme". The inner join to `marked` drops Fin, who sat nothing.

Alternative. Mark extremes with `RANK() ... DESC = 1 OR RANK() ... ASC = 1` and exclude those pupils — with `NOT EXISTS`, not `NOT IN`, which returns nothing if the subquery yields a NULL.

19.3.7 Viewers who became streamers

Problem. A sessions table records each user's sessions as `viewer` or `streamer`. For users whose earliest session was as a viewer and who streamed at least once, return how many streaming sessions they have. Sort by that count, then user id, both descending.

`code/10_viewers_then_streamers.sql`

```
CREATE TABLE sessions (session_id int PRIMARY KEY, user_id int, started_at timestamp,
                       kind text CHECK (kind IN ('viewer','streamer')));
INSERT INTO sessions VALUES
 (1,101,'2026-01-01 10:00','viewer'),(2,101,'2026-01-02 10:00','streamer'),(3,101,'2026-01-05
10:00','streamer'),
 (4,102,'2026-01-01 09:00','streamer'),(5,102,'2026-01-03 09:00','viewer'),
 (6,103,'2026-01-02 08:00','viewer'),(7,103,'2026-01-04 08:00','streamer'),
 (8,104,'2026-01-02 08:00','viewer');                                    -- never streamed

WITH firsts AS (
  SELECT DISTINCT ON (user_id) user_id, kind AS first_kind
  FROM   sessions
  ORDER  BY user_id, started_at
)
SELECT s.user_id, count(*) FILTER (WHERE s.kind = 'streamer') AS streams
FROM   sessions s
JOIN   firsts f USING (user_id)
WHERE  f.first_kind = 'viewer'
GROUP  BY s.user_id
HAVING count(*) FILTER (WHERE s.kind = 'streamer') > 0
ORDER  BY streams DESC, s.user_id DESC;
```

Output:

```
 user_id | streams
---------+---------
     101 |       2
     103 |       1
(2 rows)
```

Why it works. `DISTINCT ON` finds each user's first session kind (user 102 started as a streamer, so is out). Joining back and counting with `FILTER (WHERE kind = 'streamer')` gives the streams; `HAVING ... > 0` drops user 104, who never streamed.

Alternative. `first_value(kind) OVER (PARTITION BY user_id ORDER BY started_at)` (the value from the first row of the window) puts the first kind on every row (the file prints it: viewer, streamer, viewer, viewer).

19.3.8 First year on loan (keep the ties)

Problem. A library keeps yearly loan statistics per branch and book: number of loans and fee per loan. For each book, report its first year on loan with the loans and fee from that year — if the first year has several rows, report all of them.

`code/11_first_year_loans.sql`

```
CREATE TABLE loan_stats (branch_id int, book_id int, yr int, loans int, fee int,
                         PRIMARY KEY (branch_id, book_id, yr));
INSERT INTO loan_stats VALUES
 (1,100,2023,12,2),(1,100,2024,20,2),
 (2,100,2023, 5,3),                    -- second row for book 100 in its first year
 (1,200,2025, 7,1);

-- Solution: RANK keeps every row tied for the earliest year
SELECT book_id, yr AS first_year, loans, fee
FROM  (SELECT *, rank() OVER (PARTITION BY book_id ORDER BY yr) AS rk FROM loan_stats) t
WHERE  rk = 1
ORDER  BY book_id, loans DESC;

-- Pitfall: ROW_NUMBER (or DISTINCT ON) silently drops one of the 2023 rows of book 100
SELECT DISTINCT ON (book_id) book_id, yr AS first_year, loans, fee
FROM   loan_stats
ORDER  BY book_id, yr;
```

Output:

```
 book_id | first_year | loans | fee
---------+------------+-------+-----
     100 |       2023 |    12 |   2
     100 |       2023 |     5 |   3
     200 |       2025 |     7 |   1
(3 rows)

 book_id | first_year | loans | fee
---------+------------+-------+-----
     100 |       2023 |    12 |   2
     200 |       2025 |     7 |   1
(2 rows)
```

Why it works. `rank() OVER (PARTITION BY book_id ORDER BY yr)` gives rank 1 to *every* row in the earliest year, so both 2023 rows of book 100 survive. The classic equivalent is "find `min(yr)` per book, then join back on (book, year)".

Edge case. The second query is the tempting shortcut and it is wrong for this data: `DISTINCT ON` (like `ROW_NUMBER`) keeps exactly one row per book, losing branch 2's 2023 figures. Whenever "first" can tie, prefer `RANK` or the join-back.

19.3.9 Busiest collaborators (all ties)

Problem. Each row says one person invited another to a project and it was accepted. Count, for every person, the distinct projects they were part of (as inviter *or* invitee) and return everyone tied for the highest count.

`code/12_busiest_collaborators.sql`

```sql
CREATE TABLE collabs (inviter int, invitee int, project_id int, accepted_on date,
                      PRIMARY KEY (inviter, invitee, project_id));
INSERT INTO collabs VALUES
 (1,2,900,'2026-02-01'),(1,3,900,'2026-02-02'),(2,3,901,'2026-02-03'),
 (4,1,902,'2026-02-04'),(3,4,903,'2026-02-05'),(1,4,903,'2026-02-06');

WITH involvement AS (
  SELECT inviter AS person, project_id FROM collabs
  UNION                                    -- UNION also removes duplicate (person, project)
  SELECT invitee, project_id FROM collabs
), counts AS (
  SELECT person, count(*) AS projects,
         rank() OVER (ORDER BY count(*) DESC) AS rk
  FROM   involvement
  GROUP  BY person
)
SELECT person, projects FROM counts WHERE rk = 1 ORDER BY person;
```

Output:

```
 person | projects
--------+----------
      1 |        3
      3 |        3
(2 rows)
```

Why it works. Two `SELECT`s turn each row into two "person took part in project" facts; `UNION` removes repeats (person 1 twice in project 900). A window `rank()` over the grouped counts marks every top scorer with 1 — persons 1 and 3, three projects each.

Alternative. `WHERE projects = (SELECT max(projects) FROM counts)` is equally correct and reads well; `ORDER BY ... LIMIT 1` would be wrong because it drops ties.

19.4 Running totals and row-to-row comparisons

A running total is `SUM(...) OVER (PARTITION BY group ORDER BY time)`; "compared with last time" is `LAG` (a window function returning a value from the previous row; `LEAD` returns the next row's). Chapter 8 explains **frames** (the range of rows a window function looks at for each row) in depth (§8.7).

19.4.1 Running total per team

Problem. Each row is one player's points for a team on a day ((team, day) is unique). Show, for each team and day, the team's cumulative points up to and including that day.

`code/13_running_total_by_team.sql`

```sql
CREATE TABLE team_points (player text, team char(1), day date, points int, PRIMARY KEY (team, day));
INSERT INTO team_points VALUES
 ('Lia','A','2026-05-01',10),('Max','B','2026-05-01',7),
 ('Noa','A','2026-05-03',4), ('Oli','B','2026-05-02',9),
 ('Pia','A','2026-05-06',6);

SELECT team, day,
       sum(points) OVER (PARTITION BY team ORDER BY day) AS running_total
FROM   team_points
ORDER  BY team, day;
```

Output (the file's alternative returns the same rows):

```
 team |    day     | running_total
------+------------+---------------
 A    | 2026-05-01 |            10
 A    | 2026-05-03 |            14
 A    | 2026-05-06 |            20
 B    | 2026-05-01 |             7
 B    | 2026-05-02 |            16
(5 rows)
```

Why it works. `PARTITION BY team` restarts the sum per team; `ORDER BY day` makes it cumulative. With an `ORDER BY` and no explicit frame the default is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`.

Alternative / edge case. A self-join summing rows of the same team with `day <=` this day (in the file) works on old databases but is O(n²). If days could repeat, the default `RANGE` frame adds all same-day peers at once; use `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` for row-by-row totals.

19.4.2 Last person who fits in the lift

Problem. People queue for a lift with a 500 kg limit; each has a weight and a queue position (`turn`). People board strictly in order. Return the last person who can board without the total exceeding the limit (assume the first person always fits).

`code/14_lift_capacity.sql`

```sql
CREATE TABLE lift_queue (person_id int PRIMARY KEY, name text, kg int, turn int UNIQUE);
INSERT INTO lift_queue VALUES
 (1,'Rae',90,3),(2,'Sid',120,1),(3,'Tom',200,2),(4,'Uma',60,5),(5,'Vik',85,4);

WITH boarding AS (
  SELECT name, turn, sum(kg) OVER (ORDER BY turn) AS load_kg
  FROM   lift_queue
)
SELECT name, load_kg
FROM   boarding
WHERE  load_kg <= 500
ORDER  BY turn DESC
LIMIT  1;
```

Output:

```
 name | load_kg
------+---------
 Vik  |     495
(1 row)
```

Why it works. The running sum in queue order gives the load after each person boards: 120, 320, 410, 495, 555. Keep loads ≤ 500 and take the latest in the queue — Vik. Weights are positive, so the sum only grows and "latest turn" equals "largest load".

Alternative. A correlated subquery (in the file) recomputes each prefix sum — correct, O(n²), and the answer to "how would you do this without window functions?".

19.4.3 Performance and improvement rate

Problem. Review rounds per person (round number, score 1–100). Report, per person, (a) *performance* = average of score ÷ round, and (b) *improvement rate* = percentage of that person's reviews where the score beat the previous round's score **and** is above 50. Round both to 2 decimals.

`code/15_review_trends.sql`

```sql
CREATE TABLE review_rounds (person text, rnd int, score int);
INSERT INTO review_rounds VALUES
 ('Kim',1,40),('Kim',2,55),('Kim',3,70),('Kim',4,65),
 ('Lou',1,80),('Lou',2,45);

WITH prev AS (
  SELECT person, rnd, score,
         lag(score) OVER (PARTITION BY person ORDER BY rnd) AS prev_score
  FROM   review_rounds
)
SELECT person,
       round(avg(score::numeric / rnd), 2)                                AS performance,
       round(100.0 * count(*) FILTER (WHERE score > 50 AND score > prev_score)
             / count(*), 2)                                               AS improvement_pct
FROM   prev
GROUP  BY person
ORDER  BY person;
```

Output:

```
 person | performance | improvement_pct
--------+-------------+-----------------
 Kim    |       26.77 |           50.00
 Lou    |       51.25 |            0.00
(2 rows)
```

Why it works. `lag(score)` puts the previous round's score on each row (NULL for round 1, and `score > NULL` is not true, so round 1 never counts). `count(*) FILTER (...)` counts qualifying rows; multiplying by `100.0` first keeps the maths in `numeric`.

Common mistake. In PostgreSQL `55 / 2` is `27`: integer division truncates, and `avg(score / rnd)` gives Kim 26.50 instead of 26.77 (shown in the file). Cast one side first, as in Chapter 18, P34.

19.5 Gaps and islands: consecutive rows and streaks

An *island* is a maximal run of rows that belong together in order (consecutive ids, days or wins); the *gaps* are the breaks. If values rise by exactly 1 inside a run, `value - ROW_NUMBER()` is **constant** inside it and jumps at every gap, so grouping by it gives one group per island.

id	visitors ≥ 100?	row_number among busy rows	id − row_number
1	yes	1	0
3	yes	2	1
4	yes	3	1
5	yes	4	1
7	yes	5	2

19.5.1 Runs of busy days at the gate

Problem. Daily visitor counts with an increasing id. Return every row that belongs to a run of three or more *consecutive ids* where each row has at least 100 visitors, ordered by date.

`code/16_busy_gate_runs.sql`

```sql
CREATE TABLE gate_counts (id int PRIMARY KEY, visit_day date UNIQUE, visitors int);
INSERT INTO gate_counts VALUES
 (1,'2026-07-01',150),(2,'2026-07-02', 90),(3,'2026-07-03',120),(4,'2026-07-04',130),
 (5,'2026-07-05',101),(6,'2026-07-06', 20),(7,'2026-07-07',300),(8,'2026-07-08',310);

WITH busy AS (
  SELECT id, visit_day, visitors,
         id - row_number() OVER (ORDER BY id) AS island
  FROM   gate_counts
  WHERE  visitors >= 100
), sized AS (
  SELECT *, count(*) OVER (PARTITION BY island) AS run_len FROM busy
)
SELECT id, visit_day, visitors, island, run_len
FROM   sized
WHERE  run_len >= 3
ORDER  BY visit_day;
```

Output:

```
 id | visit_day  | visitors | island | run_len
----+------------+----------+--------+---------
  3 | 2026-07-03 |      120 |      1 |       3
  4 | 2026-07-04 |      130 |      1 |       3
  5 | 2026-07-05 |      101 |      1 |       3
(3 rows)
```

Why it works. Filtering first leaves only busy rows; `id - row_number()` labels ids 3, 4, 5 with island 1, while id 1 (island 0) and ids 7–8 (island 2) are too short. `count(*) OVER (PARTITION BY island)` attaches each island's length to its rows, so no join-back is needed.

Alternative. Group by `island` with `HAVING count(*) >= 3` and join those islands back to the rows — the same idea in two steps. A `LAG`/`LEAD` solution needs three cases (row is first, middle or last of a run) and gets messy.

19.5.2 Values repeated three times in a row

Problem. A sequence of readings ordered by id (ids may have gaps). Return every value that occurs at least three times consecutively.

`code/17_triple_repeats.sql`

```sql
CREATE TABLE readings (id int PRIMARY KEY, val int);
INSERT INTO readings VALUES (1,5),(2,5),(3,5),(4,2),(5,7),(7,7),(9,7),(10,5),(11,5);

-- Solution: compare each row with its neighbours (ids have gaps, order still defines "in a row")
SELECT DISTINCT val
FROM  (SELECT val,
              lag(val)  OVER (ORDER BY id) AS before,
              lead(val) OVER (ORDER BY id) AS after
       FROM   readings) t
WHERE  val = before AND val = after
ORDER  BY val;

-- Alternative: islands via the difference of two ROW_NUMBERs, works for any run length k
SELECT DISTINCT val
FROM  (SELECT val,
              row_number() OVER (ORDER BY id)
            - row_number() OVER (PARTITION BY val ORDER BY id) AS island
       FROM   readings) t
GROUP  BY val, island
HAVING count(*) >= 3
ORDER  BY val;
```

Output (all queries return these rows):

```
 val
-----
   5
   7
(2 rows)
```

Why it works. `lag` and `lead` expose each row's neighbours in id order; a row equal to both is the middle of a run of three or more. The 7s sit at ids 5, 7, 9 — id gaps don't matter, because "consecutive" means consecutive *in the ordering*.

Alternative. Overall position minus position among equal values is constant within each run; `GROUP BY` it with `HAVING count(*) >= k` works for any k. The two versions disagree on NULLs: three NULLs in a row fail `val = before` but form one island (`PARTITION BY val` groups NULLs), so decide whether a missing reading can repeat.

19.5.3 Members active five days in a row

Problem. A login log (duplicates possible, several logins per day) and a members table. Return members who logged in on five or more consecutive calendar days, ordered by id.

`code/18_five_day_streaks.sql`

```sql
CREATE TABLE members (member_id int PRIMARY KEY, name text);
CREATE TABLE logins  (member_id int, login_day date);          -- duplicates allowed
INSERT INTO members VALUES (1,'Ana'),(2,'Bo'),(3,'Cy');
INSERT INTO logins VALUES
 (1,'2026-06-01'),(1,'2026-06-02'),(1,'2026-06-02'),(1,'2026-06-03'),(1,'2026-06-04'),(1,'2026-06-05'),
 (2,'2026-06-01'),(2,'2026-06-02'),(2,'2026-06-04'),(2,'2026-06-05'),(2,'2026-06-06'),
 (3,'2026-06-28'),(3,'2026-06-29'),(3,'2026-06-30'),(3,'2026-07-01'),(3,'2026-07-02');

WITH days AS (SELECT DISTINCT member_id, login_day FROM logins),
islands AS (
  SELECT member_id, login_day,
         login_day - (row_number() OVER (PARTITION BY member_id ORDER BY login_day))::int AS anchor
  FROM   days
)
SELECT m.member_id, m.name
FROM   islands i JOIN members m USING (member_id)
GROUP  BY m.member_id, m.name, i.anchor
HAVING count(*) >= 5
ORDER  BY m.member_id;

-- Alternative: the 5th distinct day after this one is exactly 4 days later
SELECT DISTINCT member_id
FROM  (SELECT member_id, login_day,
              lead(login_day, 4) OVER (PARTITION BY member_id ORDER BY login_day) AS day5
       FROM  (SELECT DISTINCT member_id, login_day FROM logins) d) t
WHERE  day5 = login_day + 4
ORDER  BY member_id;
```

Output:

```
 member_id | name
-----------+------
         1 | Ana
         3 | Cy
(2 rows)

 member_id
-----------
         1
         3
(2 rows)
```

Why it works. First reduce to *distinct* days — duplicates would break the numbering. Then `login_day - row_number` (a date minus an integer is a date in PostgreSQL) gives the same "anchor" date for every day in a streak. Cy's streak crosses June → July and still works, because date arithmetic knows month lengths.

Alternative. `lead(login_day, 4)` looks four distinct days ahead; if that day is exactly `login_day + 4`, the five days in between must all be present. It only handles a fixed length; islands also give each streak's length.

19.5.4 Interview candidates from contest medals

Problem. Each contest row stores the gold, silver and bronze winners; contest ids are consecutive. A coder qualifies if they reached the podium (any colour) in at least three back-to-back contests, **or** took gold at least three times overall, in any contests. Return candidates' names and emails.

`code/19_medal_candidates.sql`

```sql
CREATE TABLE contests (contest_id int PRIMARY KEY, gold int, silver int, bronze int);
CREATE TABLE coders   (coder_id int PRIMARY KEY, name text, email text);
INSERT INTO coders VALUES (1,'Mia','mia@example.com'),(2,'Ned','ned@example.com'),
                          (3,'Oz','oz@example.com'),(4,'Pat','pat@example.com');
INSERT INTO contests VALUES
 (51,1,2,3),(52,2,1,4),(53,3,4,1),   -- Mia medals in 51,52,53
 (54,4,3,2),(55,4,2,3),(56,4,3,1);   -- Pat wins gold 3 times

WITH medals AS (
  SELECT contest_id, m.coder_id, m.kind
  FROM   contests c
  CROSS  JOIN LATERAL (VALUES (c.gold,'gold'),(c.silver,'silver'),(c.bronze,'bronze')) AS m(coder_id,
kind)
), runs AS (
  SELECT coder_id
  FROM  (SELECT coder_id, contest_id - row_number() OVER (PARTITION BY coder_id ORDER BY contest_id) AS
island
         FROM medals) t
  GROUP  BY coder_id, island
  HAVING count(*) >= 3
), golds AS (
  SELECT coder_id FROM medals WHERE kind = 'gold'
  GROUP  BY coder_id HAVING count(*) >= 3
)
SELECT name, email
FROM   coders
WHERE  coder_id IN (SELECT coder_id FROM runs UNION SELECT coder_id FROM golds)
ORDER  BY name;
```

Output:

```
 name |      email
------+-----------------
 Mia  | mia@example.com
 Oz   | oz@example.com
 Pat  | pat@example.com
(3 rows)
```

Why it works. `CROSS JOIN LATERAL (VALUES ...)` (`VALUES` builds a small inline table; `LATERAL` lets it refer to columns of the current row `c`) unpivots the medal columns into one row per (contest, coder, kind). Rule 1 is an island on `contest_id - row_number()` per coder; rule 2 is `GROUP BY ... HAVING count(*) >= 3` on golds; `IN (... UNION ...)` combines them. Mia medals in 51–53, Oz in 53–56, Pat has three golds; Ned's medals (51, 52, 54, 55) break at 53.

Edge case. A coder cannot win two medals in the same contest here; if they could, deduplicate `(coder_id, contest_id)` before numbering, or the island arithmetic breaks.

19.5.5 Longest winning streak

Problem. Match results per player (`win`, `draw`, `loss`) by date. For every player, return the length of their longest run of consecutive wins — 0 for a player who never won.

`code/20_longest_win_streak.sql`

```sql
CREATE TABLE games (player_id int, played_on date, outcome text CHECK (outcome IN ('win','draw','loss')),
                    PRIMARY KEY (player_id, played_on));
INSERT INTO games VALUES
 (1,'2026-01-01','win'),(1,'2026-01-02','win'),(1,'2026-01-03','loss'),
 (1,'2026-01-04','win'),(1,'2026-01-05','win'),(1,'2026-01-06','win'),
 (2,'2026-01-01','draw'),(2,'2026-01-02','win'),
 (3,'2026-01-01','loss');

WITH tagged AS (
  SELECT player_id, outcome,
         row_number() OVER (PARTITION BY player_id ORDER BY played_on)
       - row_number() OVER (PARTITION BY player_id, outcome ORDER BY played_on) AS island
  FROM   games
), win_runs AS (
  SELECT player_id, count(*) AS run_len
  FROM   tagged
  WHERE  outcome = 'win'
  GROUP  BY player_id, island
)
SELECT p.player_id, coalesce(max(w.run_len), 0) AS longest_streak
FROM  (SELECT DISTINCT player_id FROM games) p
LEFT  JOIN win_runs w USING (player_id)
GROUP  BY p.player_id
ORDER  BY p.player_id;

-- Alternative: a running count of non-wins gives every streak its own label
SELECT player_id, max(len) AS longest_streak
FROM  (SELECT player_id, breaks, count(*) FILTER (WHERE outcome = 'win') AS len
       FROM  (SELECT player_id, outcome,
                     count(*) FILTER (WHERE outcome <> 'win')
                       OVER (PARTITION BY player_id ORDER BY played_on) AS breaks
              FROM games) g
       GROUP  BY player_id, breaks) s
GROUP  BY player_id
ORDER  BY player_id;
```

Output (all queries return these rows):

```
 player_id | longest_streak
-----------+----------------
         1 |              3
         2 |              1
         3 |              0
(3 rows)
```

Why it works. Row number over all games minus row number within the same outcome is constant inside each run of equal outcomes. Counting `win` rows per (player, island) gives each streak's length and `max` the longest; the `LEFT JOIN` from all players plus `coalesce(..., 0)` keeps player 3, who never won.

Alternative. A running count of non-wins (`count(*) FILTER (WHERE outcome <> 'win') OVER (...)`) stays the same across a winning streak and bumps at each draw or loss, so it also labels streaks. Grouping by it and counting wins returns 0 naturally for groups without wins.

19.5.6 Deposits over the limit two months running

Problem. Accounts have a monthly income limit. A movements table holds deposits (`in`) and withdrawals (`out`). Flag accounts whose total deposits in a calendar month exceed the limit in at least two consecutive months.

`code/21_income_over_limit.sql`

```
CREATE TABLE accounts  (account_id int PRIMARY KEY, monthly_limit int);
CREATE TABLE movements (movement_id int PRIMARY KEY, account_id int, direction text, amount int, at
timestamp);
INSERT INTO accounts VALUES (1,1000),(2,500),(3,800);
INSERT INTO movements VALUES
 (1,1,'in', 700,'2026-01-05'),(2,1,'in', 400,'2026-01-20'),   -- Jan 1100 > 1000
 (3,1,'in',1200,'2026-02-11'),                                -- Feb 1200 > 1000 -> flagged
 (4,2,'in', 600,'2026-01-03'),(5,2,'in', 700,'2026-03-03'),   -- Jan, Mar: not consecutive
 (6,2,'out',900,'2026-02-03'),                                -- withdrawals don't count
 (7,3,'in', 900,'2025-12-30'),(8,3,'in', 950,'2026-01-02');   -- Dec -> Jan crosses a year

WITH monthly AS (
  SELECT m.account_id, date_trunc('month', m.at)::date AS mon
  FROM   movements m JOIN accounts a USING (account_id)
  WHERE  m.direction = 'in'
  GROUP  BY m.account_id, date_trunc('month', m.at), a.monthly_limit
  HAVING sum(m.amount) > a.monthly_limit
)
SELECT DISTINCT account_id
FROM  (SELECT account_id, mon,
              lag(mon) OVER (PARTITION BY account_id ORDER BY mon) AS prev_mon
       FROM monthly) t
WHERE  prev_mon = mon - interval '1 month'
ORDER  BY account_id;
```

Output:

```
 account_id
------------
          1
          3
(2 rows)
```

Why it works. `date_trunc('month', at)` buckets movements by month and `HAVING` keeps over-limit months. `lag(mon)` brings the previous over-limit month alongside, so "consecutive" is `prev_mon = mon - interval '1 month'`. Account 3 is caught across December → January, which a naive "month number minus one" would miss.

Edge case. Account 2 is over in January and March but not February, so it is not flagged; its February withdrawal is ignored by `direction = 'in'`. For "N consecutive months" use the island trick on month numbers (`year * 12 + month`).

19.6 Pivot and unpivot (static and dynamic)

Pivot = turn distinct values of a column into separate columns; *unpivot* = the reverse. SQL needs the output columns fixed when a query is planned, so a pivot whose columns depend on the data requires **dynamic SQL**: a query that writes another query, which is then executed.

19.6.1 Names by region, side by side

Problem. A list of attendees (name, region; duplicates possible) from America, Asia and Europe. Produce three columns, `america`, `asia`, `europe`, each listing that region's names alphabetically from the top, with NULL where a column runs out.

`code/22_names_by_region.sql`

```sql
CREATE TABLE attendees (name text, region text);
INSERT INTO attendees VALUES
 ('Zoe','Europe'),('Ali','Asia'),('Ben','America'),('Kai','Asia'),
 ('Ada','Europe'),('Liv','Europe'),('Ben','America');     -- duplicate row kept

WITH numbered AS (
  SELECT name, region,
         row_number() OVER (PARTITION BY region ORDER BY name) AS rn
  FROM   attendees
)
SELECT max(name) FILTER (WHERE region = 'America') AS america,
       max(name) FILTER (WHERE region = 'Asia')    AS asia,
       max(name) FILTER (WHERE region = 'Europe')  AS europe
FROM   numbered
GROUP  BY rn
ORDER  BY rn;
```

Output:

```
 america |  asia  | europe
---------+--------+--------
 Ben     | Ali    | Ada
 Ben     | Kai    | Liv
 (null)  | (null) | Zoe
(3 rows)
```

Why it works. `row_number()` per region gives each name its line in its column. Grouping by that number puts every region's 1st name on row 1, 2nd names on row 2, and so on; `max(name) FILTER (...)` picks the single name per region on that line.

Alternative. `max(CASE WHEN region = 'Asia' THEN name END)` is the portable form of `FILTER` (MySQL and SQL Server lack `FILTER`). Duplicate rows stay duplicates (Ben twice); add `DISTINCT` in the inner query if the question wants unique names.

19.6.2 Dynamic pivot: one column per store

Problem. `shelf_prices(product_id, store, price)`. Produce one row per product with one column per store — columns sorted by store name, NULL where the product isn't sold — for whatever stores exist when the code runs.

`code/23_dynamic_pivot.sql`

```
SET client_min_messages = warning;   -- hide "view does not exist, skipping" notices
CREATE TABLE shelf_prices (product_id int, store text, price int, PRIMARY KEY (product_id, store));
INSERT INTO shelf_prices VALUES
 (1,'north',10),(1,'west',12),
 (2,'east', 7),(2,'north',8),(2,'west',9),
 (3,'west', 30);

-- Approach A: a procedure writes the SELECT with format() and (re)creates a view
CREATE PROCEDURE pivot_shelf_prices()
LANGUAGE plpgsql AS $$
DECLARE
  col_list text;
BEGIN
  SELECT string_agg(format('max(price) FILTER (WHERE store = %L) AS %I', store, store),
                    ', ' ORDER BY store)
  INTO   col_list
  FROM  (SELECT DISTINCT store FROM shelf_prices) s;

  DROP VIEW IF EXISTS shelf_prices_wide;       -- column set may change, so rebuild
  EXECUTE format('CREATE VIEW shelf_prices_wide AS
                  SELECT product_id, %s FROM shelf_prices GROUP BY product_id', col_list);
END $$;

CALL pivot_shelf_prices();
SELECT * FROM shelf_prices_wide ORDER BY product_id;

-- A new store appears: re-run the procedure and the view grows a column
INSERT INTO shelf_prices VALUES (3,'central',28);
CALL pivot_shelf_prices();
SELECT * FROM shelf_prices_wide ORDER BY product_id;

-- Approach B (psql only): build the query text, then \gexec runs it
SELECT format('SELECT product_id, %s FROM shelf_prices GROUP BY product_id ORDER BY product_id',
              string_agg(format('max(price) FILTER (WHERE store = %L) AS %I', store, store),
                         ', ' ORDER BY store))
FROM  (SELECT DISTINCT store FROM shelf_prices) s
\gexec
```

Output (first call, then after `central` appears; Approach B and the `crosstab` version in the file print the same four-store table):

```
 product_id |  east  | north  | west
------------+--------+--------+------
          1 | (null) |     10 |   12
          2 |      7 |      8 |    9
          3 | (null) | (null) |   30
(3 rows)

 product_id | central |  east  | north  | west
------------+---------+--------+--------+------
          1 |  (null) | (null) |     10 |   12
          2 |  (null) |      7 |      8 |    9
          3 |      28 | (null) | (null) |   30
(3 rows)
```

Why it works.

1. (A **procedure** is a named block of code stored in the database and run with `CALL`; a **view** is a saved query you can `SELECT` from like a table.) `string_agg(format(...), ', ' ORDER BY store)` builds the column list, e.g. `max(price) FILTER (WHERE store = 'east') AS east, ...`. `%L` quotes a **literal** (a data value such as `'east'`) and `%I` quotes an **identifier** (a name such as a column), so a store named

`O'Neil's` or `Store 1` cannot break the SQL or inject code (**SQL injection**: data that sneaks extra SQL commands into a query).

2. `EXECUTE` runs the generated `CREATE VIEW`. A PL/pgSQL function must declare its result columns in advance — exactly what we don't know — so the procedure publishes a view that a plain `SELECT *` reads.
3. When `central` appears, calling the procedure again rebuilds the view with a new column. `DROP VIEW` first, because `CREATE OR REPLACE VIEW` can only *append* columns, not insert one in the middle. On an empty table `string_agg` returns NULL and the generated `SELECT product_id, FROM …` is invalid, so real code should check for that first.

Alternatives. In `psql`, `\gexec` runs each value the previous query returns as SQL — the quickest ad-hoc dynamic pivot. `crosstab()` (from the `tablefunc` **extension** — an optional add-on package for PostgreSQL — in the file) still needs the output columns spelled out in `AS ct(...)`. In MySQL the equivalent is `GROUP_CONCAT` to build the column list plus `PREPARE`/`EXECUTE` (watch `group_concat_max_len`); SQL Server has `PIVOT` plus `sp_executesql`.

19.6.3 Dynamic unpivot: store columns back to rows

Problem. `price_grid(product_id, <one int column per store>)`, where the store columns can change and NULL means "not sold". Produce rows `(product_id, store, price)` with no rows for NULL prices.

`code/24_dynamic_unpivot.sql`

```sql
SET client_min_messages = warning;   -- hide "view does not exist, skipping" notices
CREATE TABLE price_grid (product_id int PRIMARY KEY, east int, north int, west int);
INSERT INTO price_grid VALUES (1,NULL,10,12),(2,7,8,9),(3,NULL,NULL,30);

-- Approach A: no dynamic SQL at all -- turn each row into JSON and explode its keys
SELECT g.product_id, kv.key AS store, kv.value::int AS price
FROM   price_grid g
CROSS  JOIN LATERAL jsonb_each_text(to_jsonb(g) - 'product_id') AS kv
WHERE  kv.value IS NOT NULL
ORDER  BY g.product_id, store;

-- Approach B: read the column names from the catalog and generate UNION ALL branches
CREATE PROCEDURE unpivot_price_grid()
LANGUAGE plpgsql AS $$
DECLARE
  branches text;
BEGIN
  SELECT string_agg(format('SELECT product_id, %L AS store, %I AS price FROM price_grid WHERE %I IS NOT
NULL',
                           column_name, column_name, column_name),
                    ' UNION ALL ' ORDER BY column_name)
  INTO   branches
  FROM   information_schema.columns
  WHERE  table_schema = current_schema()
    AND  table_name   = 'price_grid'
    AND  column_name <> 'product_id';

  DROP VIEW IF EXISTS price_grid_long;
  EXECUTE 'CREATE VIEW price_grid_long AS ' || branches;
END $$;

CALL unpivot_price_grid();
SELECT * FROM price_grid_long ORDER BY product_id, store;
```

Output (all queries return these rows):

```
 product_id | store | price
------------+-------+-------
          1 | north |    10
          1 | west  |    12
          2 | east  |     7
          2 | north |     8
          2 | west  |     9
          3 | west  |    30
(6 rows)
```

Why it works.

- **Approach A** needs no dynamic SQL: `to_jsonb(g)` turns the row into `{"product_id":1,"east":null,...}`, `- 'product_id'` drops that key, and `jsonb_each_text` explodes the rest into (key, value) text pairs (hence `::int`). New columns are picked up automatically.
- **Approach B** reads column names from `information_schema.columns` (the standard **catalog** views where a database describes its own tables and columns), generates one `SELECT ... WHERE col IS NOT NULL` per column, joins them with `UNION ALL`, and `EXECUTE`s the result.

Edge case. With a fixed, known set of columns, the simplest unpivot is `CROSS JOIN LATERAL (VALUES ('east', east), ('north', north), ('west', west)) AS v(store, price)` — as used for medals in 19.5.4. Use `UNION ALL` (stack results, keeping duplicates), not `UNION` (stack and remove duplicates): `UNION` would wrongly merge identical rows and costs a sort.

19.7 Self-joins, "both directions" and graph-like questions

Friendship, calls and matches store a *pair* in one row, but the question asks about *each person*. The standard move is to "unfold" the pair with `UNION ALL` (or `UNION`) so each participant gets their own row, then aggregate normally. Relational division ("took *all* courses"; defined in 19.7.6) also lives here.

19.7.1 Page suggestions from friends' likes

Problem. `friendships(a, b)` stores each friendship once, in either column order. `page_likes(user_id, page_id)`. Suggest to user 1 every page liked by at least one friend and not already liked by user 1, without duplicates.

`code/25_page_suggestions.sql`

```sql
CREATE TABLE friendships (a int, b int, PRIMARY KEY (a, b));      -- stored once, either direction
CREATE TABLE page_likes  (user_id int, page_id int, PRIMARY KEY (user_id, page_id));
INSERT INTO friendships VALUES (1,2),(3,1),(1,4),(5,6);
INSERT INTO page_likes VALUES
 (1,77),(2,77),(2,88),(3,88),(3,99),(4,55),(5,66),(6,11);

WITH my_friends AS (
  SELECT b AS friend FROM friendships WHERE a = 1
  UNION
  SELECT a FROM friendships WHERE b = 1
)
SELECT DISTINCT l.page_id
FROM   page_likes l
JOIN   my_friends f ON f.friend = l.user_id
WHERE  NOT EXISTS (SELECT 1 FROM page_likes me
                   WHERE me.user_id = 1 AND me.page_id = l.page_id)
ORDER  BY l.page_id;

-- Alternative: set difference
SELECT page_id FROM page_likes
WHERE  user_id IN (SELECT b FROM friendships WHERE a = 1 UNION SELECT a FROM friendships WHERE b = 1)
EXCEPT
SELECT page_id FROM page_likes WHERE user_id = 1
ORDER  BY page_id;
```

Output (all queries return these rows):

```
 page_id
---------
      55
      88
      99
(3 rows)
```

Why it works. A friendship may be stored as (1, x) or (x, 1), so `my_friends` looks both ways (2, 3, 4). Joining likes to friends gives candidates; `NOT EXISTS` drops page 77, which user 1 already likes, and `DISTINCT` collapses page 88 (liked by two friends).

Alternative. `EXCEPT` (set difference: rows in the first result that are not in the second) expresses "their pages minus my pages" directly and also removes duplicates. Avoid `page_id NOT IN (SELECT ...)` if the subquery column can be NULL — one NULL makes the whole condition unknown and returns nothing.

19.7.2 Popularity percentage

Problem. Friend pairs, stored once each. For every user, popularity = number of friends ÷ number of distinct users on the platform × 100, rounded to 2 decimals. Order by user.

`code/26_friend_share.sql`

```sql
CREATE TABLE friend_pairs (u1 int, u2 int, PRIMARY KEY (u1, u2));
INSERT INTO friend_pairs VALUES (1,2),(1,3),(2,3),(4,1),(5,6);

WITH both_ways AS (
  SELECT u1 AS person, u2 AS friend FROM friend_pairs
  UNION
  SELECT u2, u1 FROM friend_pairs
)
SELECT person,
       round(100.0 * count(*) / (SELECT count(DISTINCT person) FROM both_ways), 2) AS popularity_pct
FROM   both_ways
GROUP  BY person
ORDER  BY person;
```

Output:

```
 person | popularity_pct
--------+----------------
      1 |          50.00
      2 |          33.33
      3 |          33.33
      4 |          16.67
      5 |          16.67
      6 |          16.67
(6 rows)
```

Why it works. Unfolding each pair both ways makes "my friends" a simple `GROUP BY person` count. The denominator is the count of distinct people in the unfolded set (users 1–6). `100.0 *` at the front keeps everything numeric.

Common mistake. `3 / 6 * 100` is `0` in PostgreSQL (the file shows it), because `3 / 6` truncates first; put `100.0` first. If some users have no friends, take the denominator from a `users` table instead.

19.7.3 Same person first and last in the day

Problem. Calls store caller, callee and time. For each user and each day, look at the first and last call they took part in (as caller *or* callee). Return users for whom, on at least one day, both of those calls were with the same other person.

`code/27_same_first_last_contact.sql`

```sql
CREATE TABLE phone_calls (caller int, callee int, called_at timestamp, PRIMARY KEY (caller, callee,
called_at));
INSERT INTO phone_calls VALUES
 (1,2,'2026-04-01 08:00'),(3,1,'2026-04-01 12:00'),(2,1,'2026-04-01 20:00'),  -- user 1: 2 ... 2
 (4,5,'2026-04-02 09:00'),(4,6,'2026-04-02 18:00'),                          -- user 4: 5 ... 6
 (7,8,'2026-04-03 10:00');                                                    -- one call: first = last

WITH legs AS (                          -- every call seen from both participants
  SELECT caller AS person, callee AS other, called_at FROM phone_calls
  UNION ALL
  SELECT callee, caller, called_at FROM phone_calls
), ends AS (
  SELECT person, called_at::date AS day, other,
         row_number() OVER (PARTITION BY person, called_at::date ORDER BY called_at)      AS from_start,
         row_number() OVER (PARTITION BY person, called_at::date ORDER BY called_at DESC) AS from_end
  FROM   legs
)
SELECT DISTINCT person
FROM   ends
GROUP  BY person, day
HAVING max(other) FILTER (WHERE from_start = 1) = max(other) FILTER (WHERE from_end = 1)
ORDER  BY person;
```

Output:

```
 person
--------
      1
      2
      3
      5
      6
      7
      8
(7 rows)
```

Why it works. `legs` shows every call from both sides, so `other` is always the other party. Ascending and descending `row_number()`s per (person, day) tag the first and last call; the two `max(other) FILTER (...)` extract those partners and `HAVING` compares them. User 1 talked to 2 first and last on 1 April (with 3 in between), and so did user 2 (both calls with 1); user 4 did not.

Edge case. A person with a single call that day has the same call as first and last, so they qualify — that is why 3, 5, 6, 7 and 8 appear. Confirm with the interviewer whether that counts; if not, add `HAVING count(*) > 1`.

19.7.4 League table

Problem. Clubs, plus fixtures with home/away clubs and goals. Award 3 points for a win, 1 for a draw, 0 for a loss. List every club with its points, highest first, ties by club id.

`code/28_league_table.sql`

```
CREATE TABLE clubs    (club_id int PRIMARY KEY, club text);
CREATE TABLE fixtures (fixture_id int PRIMARY KEY, home int, away int, home_goals int, away_goals int);
INSERT INTO clubs VALUES (1,'Rovers'),(2,'United'),(3,'Athletic'),(4,'Wanderers');
INSERT INTO fixtures VALUES (1,1,2,2,0),(2,3,1,1,1),(3,2,3,3,1),(4,1,3,0,1);

WITH per_club AS (                    -- one row per (club, match), from that club's side
  SELECT home AS club_id, home_goals AS gf, away_goals AS ga FROM fixtures
  UNION ALL
  SELECT away, away_goals, home_goals FROM fixtures
)
SELECT c.club_id, c.club,
       coalesce(sum(CASE WHEN p.gf > p.ga THEN 3 WHEN p.gf = p.ga THEN 1 ELSE 0 END), 0) AS points
FROM   clubs c
LEFT   JOIN per_club p USING (club_id)
GROUP  BY c.club_id, c.club
ORDER  BY points DESC, c.club_id;
```

Output:

```
 club_id |   club    | points
---------+-----------+--------
       1 | Rovers    |      4
       3 | Athletic  |      4
       2 | United    |      3
       4 | Wanderers |      0
(4 rows)
```

Why it works. `per_club` rewrites each match twice, once from each club's point of view (goals for, goals against), so one `CASE` scores every row. `LEFT JOIN` from `clubs` plus `coalesce(..., 0)` keeps Wanderers, who played nothing.

Alternative. Joining with `ON club_id = home OR club_id = away` and a bigger `CASE` (which side is the club?) also works, but `OR` joins use **indexes** (sorted lookup structures for finding rows fast) poorly. Unfolding first is cleaner and extends to goal difference.

19.7.5 Departments with two above-average earners

Problem. Payroll rows (id, name, department, salary). Return each department that has two or more people paid above that department's mean salary.

`code/29_depts_with_high_earners.sql`

```
CREATE TABLE payroll (emp_id int PRIMARY KEY, name text, dept text, salary int);
INSERT INTO payroll VALUES
 (1,'A','Ops',50),(2,'B','Ops',60),(3,'C','Ops',70),(4,'D','Ops',80),    -- avg 65: C, D above
 (5,'E','Law',90),(6,'F','Law',40),(7,'G','Law',41),                      -- avg 57: only E
 (8,'H','Art',30);

SELECT dept
FROM  (SELECT dept, salary, avg(salary) OVER (PARTITION BY dept) AS dept_avg FROM payroll) t
WHERE  salary > dept_avg
GROUP  BY dept
HAVING count(*) >= 2
ORDER  BY dept;
```

Output (the file's alternative returns the same rows):

```
 dept
------
 Ops
(1 row)
```

Why it works. `avg(salary) OVER (PARTITION BY dept)` attaches each department's average to every employee without grouping them away. Filter rows above that average, then group by department and keep those with two or more. Ops (avg 65) has C and D; Law (avg 57) only has E.

Alternative. A correlated subquery (in the file) computes the department average per row; a third route joins `payroll` to itself on department, groups by employee and keeps `HAVING p.salary > avg(q.salary)`. The window version reads the table once.

19.7.6 Students enrolled in every course

Problem. A course catalogue and an enrolment log (duplicates possible). Return students enrolled in every course in the catalogue. This is *relational division*: "which X are related to **all** Y?".

`code/30_took_every_course.sql`

```sql
CREATE TABLE catalogue   (course_id int PRIMARY KEY);
CREATE TABLE enrolments  (student_id int NOT NULL, course_id int REFERENCES catalogue);   -- duplicates possible
INSERT INTO catalogue VALUES (1),(2),(3);
INSERT INTO enrolments VALUES (10,1),(10,2),(10,3),(11,1),(11,1),(11,2),(12,3),(12,2),(12,1),(12,1);

SELECT student_id
FROM   enrolments
GROUP  BY student_id
HAVING count(DISTINCT course_id) = (SELECT count(*) FROM catalogue)
ORDER  BY student_id;

-- Alternative: "no course exists that this student did not take" (double NOT EXISTS)
SELECT DISTINCT e.student_id
FROM   enrolments e
WHERE  NOT EXISTS (SELECT 1 FROM catalogue c
                   WHERE NOT EXISTS (SELECT 1 FROM enrolments x
                                     WHERE x.student_id = e.student_id AND x.course_id = c.course_id))
ORDER  BY e.student_id;
```

Output (all queries return these rows):

```
 student_id
------------
         10
         12
(2 rows)
```

Why it works. Count each student's *distinct* courses and compare with the catalogue size. `DISTINCT` matters: student 11 has three enrolment rows but only two courses. The foreign key guarantees no course outside the catalogue can inflate the count.

Alternative. The double `NOT EXISTS` is the textbook formulation: "there is no course that this student has not taken". It does not depend on counting, so it stays correct even without the foreign key (enrolments in unknown courses would break the count version).

19.7.7 Shop revenue under two uniqueness rules

Problem. Shops with 2024 revenue, 2025 revenue and coordinates. Total the 2025 revenue of shops whose 2024 revenue equals at least one *other* shop's 2024 revenue **and** whose (lat, lon) is shared with *no* other shop. Round to 2 decimals.

`code/31_shop_revenue_filter.sql`

```
CREATE TABLE shops (shop_id int PRIMARY KEY, rev_2024 numeric(10,2), rev_2025 numeric(10,2),
                    lat numeric(8,5), lon numeric(8,5));
INSERT INTO shops VALUES
 (1,100.00,120.50,51.50000,-0.12000),
 (2,100.00,140.25,48.85000, 2.35000),
 (3,200.00,210.00,40.71000,-74.00000),
 (4,100.00,  90.00,40.71000,-74.00000),   -- same place as shop 3
 (5,300.00,330.00,35.68000,139.69000);

SELECT round(sum(rev_2025), 2) AS total_2025
FROM  (SELECT rev_2025,
              count(*) OVER (PARTITION BY rev_2024) AS same_rev,
              count(*) OVER (PARTITION BY lat, lon) AS same_spot
       FROM   shops) t
WHERE  same_rev > 1 AND same_spot = 1;
```

Output:

```
 total_2025
------------
     260.75
(1 row)
```

Why it works. Two window counts label each shop at once: how many shops share its 2024 revenue, and how many share its location. Keep `same_rev > 1 AND same_spot = 1`: shops 1 and 2 (shop 4 shares revenue but also shares a location with shop 3). 120.50 + 140.25 = 260.75.

Edge case. Equality on **floating-point** columns (approximate binary number types such as `float`) is fragile (0.1 + 0.2 ≠ 0.3 in binary floats), so values compared for equality belong in `numeric`. Grouping twice with `HAVING` and joining back gives the same answer in more steps.

19.8 Date bucketing and retention

Group by a *derived* date — the first day a user appeared, the calendar month, the booking day — and measure what happens inside each bucket. `generate_series` (produces a sequence of numbers or dates) fills missing buckets. A **cohort** is a group of users who started in the same period; **retention** is the share of a cohort that comes back later.

19.8.1 Day-1 retention

Problem. Game plays per player per day. A player's *install day* is their first play day. For each install day, report the number of installs and *day-1 retention* = share of those players who also played the next day, rounded to 2 decimals.

`code/32_day1_retention.sql`

```sql
CREATE TABLE plays (player_id int, device_id int, play_day date, games int, PRIMARY KEY (player_id,
play_day));
INSERT INTO plays VALUES
 (1,9,'2026-03-01',3),(1,9,'2026-03-02',1),
 (2,8,'2026-03-01',0),(2,8,'2026-03-04',2),
 (3,7,'2026-03-02',5),(3,7,'2026-03-03',1),
 (4,6,'2026-03-02',1);

WITH firsts AS (
  SELECT player_id, min(play_day) AS install_day FROM plays GROUP BY player_id
)
SELECT f.install_day,
       count(*) AS installs,
       round(avg((EXISTS (SELECT 1 FROM plays p
                          WHERE p.player_id = f.player_id
                            AND p.play_day = f.install_day + 1))::int), 2) AS day1_retention
FROM   firsts f
GROUP  BY f.install_day
ORDER  BY f.install_day;

-- Alternative: LEFT JOIN to the next day's row
SELECT f.install_day, count(*) AS installs,
       round(count(p.player_id)::numeric / count(*), 2) AS day1_retention
FROM  (SELECT player_id, min(play_day) AS install_day FROM plays GROUP BY player_id) f
LEFT  JOIN plays p ON p.player_id = f.player_id AND p.play_day = f.install_day + 1
GROUP  BY f.install_day
ORDER  BY f.install_day;
```

Output (all queries return these rows):

```
 install_day | installs | day1_retention
-------------+----------+----------------
 2026-03-01  |        2 |           0.50
 2026-03-02  |        2 |           0.50
(2 rows)
```

Why it works. `firsts` gives each player one install day. `EXISTS` is true when a play exists on `install_day + 1`; averaging the boolean **cast** (type conversion, written `::int`) to int gives the fraction retained (true → 1, false → 0). Player 2 returned on 4 March, not the next day, so does not count.

Alternative. A `LEFT JOIN` to the next-day row and `count(p.player_id) / count(*)` does the same (the primary key on (player, day) guarantees at most one match). Note the `::numeric` cast — without it the integer division returns 0.

19.8.2 Transactions per visit histogram

Problem. Bank visits (customer, day) and transactions (customer, day, amount; duplicates allowed; every transaction happens on a visit day). Build a **histogram** (a count of how many items fall into each value or range): for each number k from 0 up to the largest per-visit count, how many visits had exactly k transactions. Include k values with zero visits.

`code/33_transactions_per_visit.sql`

```
CREATE TABLE branch_visits (customer_id int, visit_day date, PRIMARY KEY (customer_id, visit_day));
CREATE TABLE branch_txns   (customer_id int, txn_day date, amount int);   -- duplicates possible
INSERT INTO branch_visits VALUES (1,'2026-02-01'),(1,'2026-02-05'),(2,'2026-02-01'),(3,'2026-02-02'),
(4,'2026-02-03');
INSERT INTO branch_txns VALUES
 (1,'2026-02-01',50),(1,'2026-02-01',50),(1,'2026-02-01',20),   -- 3 in one visit
 (2,'2026-02-01',10),                                           -- 1
 (3,'2026-02-02',99);                                           -- 1

WITH per_visit AS (
  SELECT v.customer_id, v.visit_day, count(t.txn_day) AS txns
  FROM   branch_visits v
  LEFT   JOIN branch_txns t ON t.customer_id = v.customer_id AND t.txn_day = v.visit_day
  GROUP  BY v.customer_id, v.visit_day
)
SELECT n AS transactions_count, count(p.txns) AS visits_count
FROM   generate_series(0, (SELECT max(txns) FROM per_visit)) AS n
LEFT   JOIN per_visit p ON p.txns = n
GROUP  BY n
ORDER  BY n;
```

Output:

```
 transactions_count | visits_count
--------------------+--------------
                  0 |            2
                  1 |            2
                  2 |            0
                  3 |            1
(4 rows)
```

Why it works. `per_visit` counts each visit's transactions (`count(t.txn_day)` makes visits with none count 0 after the `LEFT JOIN`). `generate_series(0, max)` produces every k, including 2, which no visit has; a `LEFT JOIN` then counts matching visits, giving 0 for the empty bucket.

Alternative. Without `generate_series` (e.g. MySQL), use a **recursive CTE** (one that repeatedly feeds its own output back in) counting from 0 to the max, or `ROW_NUMBER()` over any big-enough table.

19.8.3 Titles borrowed per day

Problem. A borrowing log (date, title; duplicates possible). For each date, report how many distinct titles were borrowed and a comma-separated list of them in alphabetical order.

`code/34_titles_per_day.sql`

```
CREATE TABLE borrow_log (borrowed_on date, title text);          -- no key, duplicates allowed
INSERT INTO borrow_log VALUES
 ('2026-08-01','Dune'),('2026-08-01','Emma'),('2026-08-01','Dune'),
 ('2026-08-02','Ulysses'),('2026-08-02','Beloved'),('2026-08-03','Emma');

SELECT borrowed_on,
       count(DISTINCT title)                            AS titles,
       string_agg(DISTINCT title, ',' ORDER BY title)   AS title_list
FROM   borrow_log
GROUP  BY borrowed_on
ORDER  BY borrowed_on;
```

Output:

```
 borrowed_on | titles |   title_list
-------------+--------+-----------------
 2026-08-01  |      2 | Dune,Emma
 2026-08-02  |      2 | Beloved,Ulysses
 2026-08-03  |      1 | Emma
(3 rows)
```

Why it works. `count(DISTINCT title)` ignores the second "Dune". `string_agg(DISTINCT title, ',' ORDER BY title)` concatenates unique titles in sorted order; PostgreSQL allows `DISTINCT` together with `ORDER BY` only when the ordering expression is the aggregated one, which it is here.

Dialect note. MySQL: `GROUP_CONCAT(DISTINCT title ORDER BY title SEPARATOR ',')` (limited by `group_concat_max_len`, default 1024 bytes). SQL Server: `STRING_AGG(title, ',') WITHIN GROUP (ORDER BY title)`, with no `DISTINCT` — deduplicate in a subquery first.

19.8.4 Same-day first bookings

Problem. Clinic bookings record when a booking was made and the day the patient wanted. A booking is *same-day* when those dates match. Considering only each patient's first booking (earliest booking date), what percentage were same-day? Round to 2 decimals.

`code/35_first_booking_same_day.sql`

```sql
CREATE TABLE bookings_clinic (booking_id int PRIMARY KEY, patient_id int, booked_on date, wanted_on
date);
INSERT INTO bookings_clinic VALUES
 (1,1,'2026-09-01','2026-09-01'),(2,1,'2026-09-05','2026-09-09'),
 (3,2,'2026-09-02','2026-09-04'),
 (4,3,'2026-09-03','2026-09-03'),
 (5,4,'2026-09-10','2026-09-12'),(6,4,'2026-09-01','2026-09-01');

SELECT round(100.0 * avg((booked_on = wanted_on)::int), 2) AS same_day_pct
FROM  (SELECT DISTINCT ON (patient_id) booked_on, wanted_on
       FROM   bookings_clinic
       ORDER  BY patient_id, booked_on) first_bookings;
```

Output:

```
 same_day_pct
--------------
        75.00
(1 row)
```

Why it works. `DISTINCT ON (patient_id) ... ORDER BY patient_id, booked_on` keeps each patient's earliest booking (patient 4's is 1 September, although inserted last). `(booked_on = wanted_on)::int` is 1 or 0, so 100 × its average is the same-day share: 75.00.

Alternative. The "min then join back" version (`min(booked_on)` per patient, join on patient and date) works everywhere; if a patient could have two bookings on their first day it would count both, whereas `DISTINCT ON` picks one — know which the question wants.

19.9 Ratios and cancellation rates

Rates are `count(condition) / count(all)`. The details that go wrong: filtering *which rows are in the denominator*, integer division, and division by zero. `avg(boolean::int)` is a compact way to compute a share.

19.9.1 Ride cancellation rate (the "trips and users" classic)

Problem. Rides (rider, driver, status, request date) and users (with a `banned` flag). For each day from 1 to 3 October 2026, compute the share of requests that were cancelled (by rider or driver), counting only rides where **both** the rider and the driver are not banned. Round to 2 decimals.

`code/36_ride_cancellation_rate.sql`

```
CREATE TABLE app_users (user_id int PRIMARY KEY, banned boolean, role text);
CREATE TABLE rides     (ride_id int PRIMARY KEY, rider_id int, driver_id int, city_id int,
                        status text CHECK (status IN
('completed','cancelled_by_driver','cancelled_by_rider')),
                        requested_on date);
INSERT INTO app_users VALUES
 (1,false,'rider'),(2,true,'rider'),(3,false,'rider'),
 (10,false,'driver'),(11,false,'driver'),(12,true,'driver');
INSERT INTO rides VALUES
 (1,1,10,1,'completed',          '2026-10-01'),
 (2,2,10,1,'cancelled_by_driver','2026-10-01'),   -- banned rider: ignored
 (3,3,11,1,'cancelled_by_rider', '2026-10-01'),
 (4,1,12,1,'completed',          '2026-10-02'),   -- banned driver: ignored
 (5,3,10,1,'cancelled_by_driver','2026-10-02'),
 (6,1,11,1,'completed',          '2026-10-03'),
 (7,3,11,1,'cancelled_by_rider', '2026-10-04');   -- outside the window

SELECT r.requested_on AS day,
       round(avg((r.status <> 'completed')::int), 2) AS cancellation_rate
FROM   rides r
JOIN   app_users ru ON ru.user_id = r.rider_id  AND NOT ru.banned
JOIN   app_users du ON du.user_id = r.driver_id AND NOT du.banned
WHERE  r.requested_on BETWEEN '2026-10-01' AND '2026-10-03'
GROUP  BY r.requested_on
ORDER  BY day;
```

Output (the file's alternative returns the same rows):

```
    day     | cancellation_rate
------------+-------------------
 2026-10-01 |              0.50
 2026-10-02 |              1.00
 2026-10-03 |              0.00
(3 rows)
```

Why it works. Joining `app_users` once as rider and once as driver, with `NOT banned` in each `ON`, removes rides with any banned party from numerator and denominator alike. `avg((status <> 'completed')::int)` is the cancelled share. Ride 2 (banned rider) and ride 4 (banned driver) disappear, so 1 October is 1 of 2 cancelled.

Alternative / edge cases. The file's `NOT IN` + `FILTER` version is safe because `user_id` is a non-NULL primary key. Days with no eligible rides produce no row; drive the query from `generate_series` if they must appear. On a `timestamp` column `BETWEEN` would miss most of the last day — use `>= start AND < end + 1 day`.

19.9.2 Delivery fulfilment rate

Problem. Orders (order, customer) and delivery attempts (order, time, `delivered` or `failed`). An order's rate is delivered attempts ÷ all attempts; an order with no attempts has rate 0. Report each customer's average order rate, rounded to 2 decimals.

`code/37_delivery_success_rate.sql`

```sql
CREATE TABLE shop_orders (order_id int PRIMARY KEY, customer_id int, placed_at timestamp);
CREATE TABLE attempts    (order_id int REFERENCES shop_orders, tried_at timestamp,
                          outcome text CHECK (outcome IN ('delivered','failed')), PRIMARY KEY (order_id,
tried_at));
INSERT INTO shop_orders VALUES (1,100,'2026-05-01'),(2,100,'2026-05-02'),(3,200,'2026-05-03'),
(4,200,'2026-05-04');
INSERT INTO attempts VALUES
 (1,'2026-05-02 10:00','failed'),(1,'2026-05-03 10:00','delivered'),   -- order 1: 0.5
 (2,'2026-05-03 11:00','delivered'),                                   -- order 2: 1.0
 (3,'2026-05-04 09:00','failed');                                      -- order 3: 0.0, order 4: no
attempts

WITH per_order AS (
  SELECT o.order_id, o.customer_id,
         coalesce(count(*) FILTER (WHERE a.outcome = 'delivered')::numeric
                  / nullif(count(a.order_id), 0), 0) AS rate
  FROM   shop_orders o
  LEFT   JOIN attempts a USING (order_id)
  GROUP  BY o.order_id, o.customer_id
)
SELECT customer_id, round(avg(rate), 2) AS fulfilment_rate
FROM   per_order
GROUP  BY customer_id
ORDER  BY customer_id;
```

Output:

```
 customer_id | fulfilment_rate
-------------+-----------------
         100 |            0.75
         200 |            0.00
(2 rows)
```

Why it works. `LEFT JOIN` keeps order 4, which has no attempts. For it, `count(a.order_id)` is 0; `nullif(0, 0)` turns the divisor into NULL, the division yields NULL instead of an error, and `coalesce(..., 0)` applies the "no attempts = 0" rule. Customer 100 averages (0.5 + 1.0) / 2 = 0.75.

Dialect trap. In a MySQL `SELECT`, `x / 0` yields NULL (plus a warning under the default **SQL mode** — MySQL's setting for how strictly it treats questionable data; strict mode turns it into an error only inside `INSERT`/`UPDATE`), so `IFNULL(x / 0, 0)` "works" there. PostgreSQL raises `division by zero` and aborts the query — `NULLIF(divisor, 0)` is the portable guard.

19.9.3 Share of members who borrowed each book

Problem. Library members and a borrowings table ((book, member) unique). For each book, the percentage of all members who borrowed it, rounded to 2 decimals; order by percentage descending, then book id.

`code/38_borrow_share_per_book.sql`

```sql
CREATE TABLE members_lib (member_id int PRIMARY KEY, name text);
CREATE TABLE borrowings  (book_id int, member_id int, PRIMARY KEY (book_id, member_id));
INSERT INTO members_lib VALUES (1,'Al'),(2,'Bea'),(3,'Cas');
INSERT INTO borrowings VALUES (500,1),(500,2),(500,3),(501,2),(502,1),(502,3);

SELECT book_id,
       round(100.0 * count(DISTINCT member_id) / (SELECT count(*) FROM members_lib), 2) AS pct_of_members
FROM   borrowings
GROUP  BY book_id
ORDER  BY pct_of_members DESC, book_id;
```

Output:

```
 book_id | pct_of_members
---------+----------------
     500 |         100.00
     502 |          66.67
     501 |          33.33
(3 rows)
```

Why it works. The numerator counts distinct borrowers per book; the denominator is a scalar subquery over *all* members, including non-borrowers. `100.0 *` first avoids truncation, and `ORDER BY` can use the alias because it runs after `SELECT`.

Edge case. If there were no members, the denominator would be 0 and PostgreSQL would raise an error; `nullif(..., 0)` makes the result NULL instead.

19.10 String building and expression evaluation

Some questions treat SQL as a small **interpreter** (a program that reads and evaluates expressions): evaluate stored comparisons, render a formula, or validate shapes. The tools are `CASE`, `||` **concatenation** (joining strings end to end), `format()` and `string_agg` (an aggregate that joins a group's values into one string).

19.10.1 Evaluate stored comparisons

Problem. `vars(name, val)` holds integer variables; `exprs(lhs, op, rhs)` holds comparisons such as `x < y`, with `op` in `<`, `>`, `=`. Return every expression with its truth value.

`code/39_compare_variables.sql`

```
CREATE TABLE vars  (name text PRIMARY KEY, val int);
CREATE TABLE exprs (lhs text REFERENCES vars, op text CHECK (op IN ('<','>','=')), rhs text REFERENCES
vars,
                    PRIMARY KEY (lhs, op, rhs));
INSERT INTO vars  VALUES ('x',4),('y',9),('z',4);
INSERT INTO exprs VALUES ('x','<','y'),('x','>','y'),('x','=','z'),('y','=','z');

SELECT e.lhs, e.op, e.rhs,
       CASE e.op WHEN '<' THEN l.val < r.val
                 WHEN '>' THEN l.val > r.val
                 WHEN '=' THEN l.val = r.val END AS result
FROM   exprs e
JOIN   vars l ON l.name = e.lhs
JOIN   vars r ON r.name = e.rhs
ORDER  BY e.lhs, e.op, e.rhs;
```

Output:

```
 lhs | op | rhs | result
-----+----+-----+--------
 x   | <  | y   | t
 x   | =  | z   | t
 x   | >  | y   | f
 y   | =  | z   | f
(4 rows)
```

Why it works. Joining `vars` twice puts both operand values on one row, and `CASE e.op WHEN ...` maps the symbol to the real comparison. The `boolean` result prints as `t`/`f`; add `::text` if the answer must read `true`/`false`.

Alternative. Dynamic SQL (`EXECUTE format(...)`) could evaluate any operator, but it lets data inject code and is overkill for three operators.

19.10.2 Build a polynomial equation string

Problem. Terms of a polynomial: `power` (0–100, unique) and non-zero integer `coef` (−100…100). Render the left-hand side as one string, highest power first, each term as sign + absolute coefficient + `X^power` (just `X` for power 1, nothing for power 0), followed by `=0`.

`code/40_polynomial_string.sql`

```
CREATE TABLE poly_terms (power int PRIMARY KEY CHECK (power BETWEEN 0 AND 100),
                         coef int CHECK (coef BETWEEN -100 AND 100 AND coef <> 0));
INSERT INTO poly_terms VALUES (0,-6),(1,5),(3,2),(2,-1);

SELECT string_agg(
         CASE WHEN coef > 0 THEN '+' ELSE '-' END
         || abs(coef)
         || CASE WHEN power = 0 THEN ''
                 WHEN power = 1 THEN 'X'
                 ELSE 'X^' || power END,
         '' ORDER BY power DESC) || '=0' AS equation
FROM   poly_terms;
```

Output:

```
     equation
------------------
 +2X^3-1X^2+5X-6=0
(1 row)
```

Why it works. Each term is sign `||` `abs(coef)` `||` power suffix. `string_agg(..., '' ORDER BY power DESC)` joins them with no separator; the `ORDER BY` inside the aggregate sets the order, not the table.

Edge case. `||` accepts an integer when the other side is text (it is cast); a NULL piece would make the whole term NULL, whereas `concat()` treats NULL as empty. MySQL: `GROUP_CONCAT(... ORDER BY power DESC SEPARATOR '')`.

19.10.3 Can four sides form a rectangle?

Problem. Rows of four side lengths `a, b, c, d`, taken in order around the shape. Report `Yes` if opposite sides are equal (a = c and b = d), otherwise `No`.

`code/41_rectangle_check.sql`

```
CREATE TABLE side_sets (a int, b int, c int, d int, PRIMARY KEY (a, b, c, d));
INSERT INTO side_sets VALUES (3,5,3,5),(4,4,4,4),(3,3,5,5),(2,7,2,8);

SELECT a, b, c, d,
       CASE WHEN a = c AND b = d THEN 'Yes' ELSE 'No' END AS rectangle
FROM   side_sets;
```

Output:

```
 a | b | c | d | rectangle
---+---+---+---+-----------
 3 | 5 | 3 | 5 | Yes
 4 | 4 | 4 | 4 | Yes
 3 | 3 | 5 | 5 | No
 2 | 7 | 2 | 8 | No
(4 rows)
```

Why it works. A row-level `CASE` needs no grouping. Order matters: 3, 3, 5, 5 has equal sides but *adjacent* ones, so it fails (if sides could be rearranged, sort the four values and compare the two smallest and the two largest).

Edge case. If any side could be NULL, `a = c` is unknown and the `CASE` falls to `ELSE 'No'`; mention that choice to the interviewer.

19.11 Pattern catalogue

Pattern	Recognise it by	Template query (PostgreSQL)
Nth highest value	"second highest", "Nth largest", "NULL if none"	`SELECT (SELECT DISTINCT v FROM t ORDER BY v DESC OFFSET n-1 LIMIT 1)`
Top-N per group	"top 3 in each", "highest per department"	`WHERE dense_rank() OVER (PARTITION BY g ORDER BY v DESC) <= n` (in a CTE)
First/last row per group	"first session", "latest status", "earliest booking"	`SELECT DISTINCT ON (g) * FROM t ORDER BY g, ts`
All tied for the max	"most ... include ties"	`WHERE cnt = (SELECT max(cnt) FROM c)` or `rank() = 1`
Capacity cut-off	"confirmed vs waitlist", "fits in the lift"	`row_number()` or `sum(w) OVER (ORDER BY pos)` compared with the limit
Running total	"cumulative", "so far", "to date"	`sum(v) OVER (PARTITION BY g ORDER BY ts)`
Compare with previous row	"improved over last", "change since"	`v - lag(v) OVER (PARTITION BY g ORDER BY ts)`
Consecutive ids/days (islands)	"N days in a row", "3+ consecutive"	`GROUP BY g, day - row_number() OVER (PARTITION BY g ORDER BY day)::int HAVING count(*) >= n`
Runs of equal values / streaks	"longest winning streak", "repeated 3 times in a row"	`row_number() OVER (PARTITION BY g ORDER BY ts) - row_number() OVER (PARTITION BY g, val ORDER BY ts)` as island id
Static pivot	fixed list of output columns	`max(v) FILTER (WHERE k = 'a') AS a, ... GROUP BY id`
Pivot of lists	"names under each continent"	`row_number() OVER (PARTITION BY k ORDER BY v)` then `GROUP BY rn` with `FILTER`
Dynamic pivot	"one column per store", columns unknown	`string_agg(format('... %L ... %I', k, k))` then `EXECUTE` (procedure + view) or `\gexec`
Unpivot	wide columns to rows	`CROSS JOIN LATERAL (VALUES ('a', a), ('b', b))` or `jsonb_each_text(to_jsonb(t) - 'id')`
Pair stored once	friendships, calls, matches	`SELECT a AS p, b AS o FROM t UNION ALL SELECT b, a FROM t`
Relational division	"took all", "bought every"	`HAVING count(DISTINCT y) = (SELECT count(*) FROM all_y)`
Compare with group average	"above department average"	`v > avg(v) OVER (PARTITION BY g)`
Uniqueness checks	"shared with another", "unique location"	`count(*) OVER (PARTITION BY k) > 1` / `= 1`
Cohort / retention	"install date", "came back next day"	`min(day)` per user, then `avg((EXISTS (... day = first + 1))::int)`
Histogram with empty buckets	"from 0 to max", "include zero"	`generate_series(0, max) n LEFT JOIN counts ON c = n`
Ratio / rate	"rate", "percentage", "share"	`round(100.0 * count(*) FILTER (WHERE cond) / nullif(count(*), 0), 2)`

Pattern	Recognise it by	Template query (PostgreSQL)
String from rows	"list of titles", "build the equation"	`string_agg(expr, sep ORDER BY k)`

Chapter 19 TL;DR

- **Ranking**: `ROW_NUMBER` = exactly one winner, `RANK` = keep ties, `DENSE_RANK` = top N *distinct* values. `DISTINCT ON` is PostgreSQL's shortcut for "first row per group". Use a scalar subquery or aggregate when "no answer" must be a NULL row.
- **Running windows**: `SUM(...) OVER (PARTITION BY ... ORDER BY ...)` for totals so far, `LAG` for "vs previous"; mind the `RANGE` default when the order key ties.
- **Gaps and islands**: `value - ROW_NUMBER()` (or the difference of two row numbers) is constant inside a run; group by it. Deduplicate first.
- **Pivots**: `MAX(...) FILTER (WHERE ...)` per column; for data-driven columns generate SQL with `format()` (`%I`/`%L`) and `EXECUTE` it into a view, or `\gexec`. Unpivot with `LATERAL VALUES` or `jsonb_each_text`.
- **Pairs**: unfold a pair stored in one row with `UNION ALL` so each side gets a row; "all of them" is relational division.
- **Dates**: bucket with `date_trunc` or `min()` per user; fill empty buckets with `generate_series`.
- **Ratios**: `100.0` first, `NULLIF(x, 0)` on the divisor, and a deliberate choice of which rows form the denominator.
- **PostgreSQL traps**: integer division truncates, division by zero is an error, `NOT IN` with a NULL returns nothing, `COUNT(*)` after a `LEFT JOIN` counts the padding row, and `DESC` sorts NULLs first.

Next: Chapter 20 — Cheat sheets and glossary, which condenses the whole book into quick-reference tables, a pattern catalogue and an A–Z glossary.

Chapter 20 — Cheat Sheets, Interview Q&A and Glossary

This last chapter is the book on a few pages. It adds no new topics. It collects the syntax, rules, decision tables and vocabulary of Chapters 1–19 in one place so you can revise quickly before an interview, or look something up while you work. Every entry links back to the section that explains it properly, with examples and real output.

What you'll find here

- **Syntax cheat sheets** for each area of the book (20.2–20.14), most ending with the traps that matter most.
- **The logical query-processing order** as a diagram (20.1). Most "why does this error happen" questions are answered by it.
- **Decision tables**: which join (20.5), which subquery form (20.6), which window function (20.8), which index (20.11), which isolation level (20.12), which scaling tool (20.14) and which database (20.15).
- **A dialect table** for PostgreSQL, MySQL, SQL Server, SQLite and Oracle (20.16).
- **Verified behaviours** (20.17). About a hundred claims from these sheets, such as `7/2 = 3`, `NULL` logic, rank ties and frame defaults, are asserted by a runnable script on PostgreSQL 14. A second script checks the SQLite column of the dialect table.
- **The interview pattern catalogue** (20.18), linking every pattern to the solved problems in Chapters 18–19.
- **Fifty interview questions with short answers** (20.19), each linked to its section.
- **A glossary** of 595 terms, from *1NF* to *ZooKeeper* (20.20), covering every technical term, acronym and product name the book uses.

How to use it: read a sheet, cover the right-hand column of a table and say the answer out loud. Follow the link whenever an answer does not come easily. All SQL is PostgreSQL unless a row says otherwise. Other databases' syntax in the dialect table (20.16) was taken from their documentation and not run, except the SQLite column. Every technical term on these sheets, from *alias* to *ZooKeeper*, has a plain-English definition in the glossary (20.20): look a word up there the first time you meet it.

20.1 The logical query-processing order

You **write** `SELECT … FROM … WHERE … GROUP BY … HAVING … ORDER BY … LIMIT`, but the database **evaluates** it (logically) starting from `FROM`. Each step receives the table produced by the step before it. The optimizer (the part of the database that decides how to run a query) may physically reorder the work, but the result is always as if these steps ran in this order.

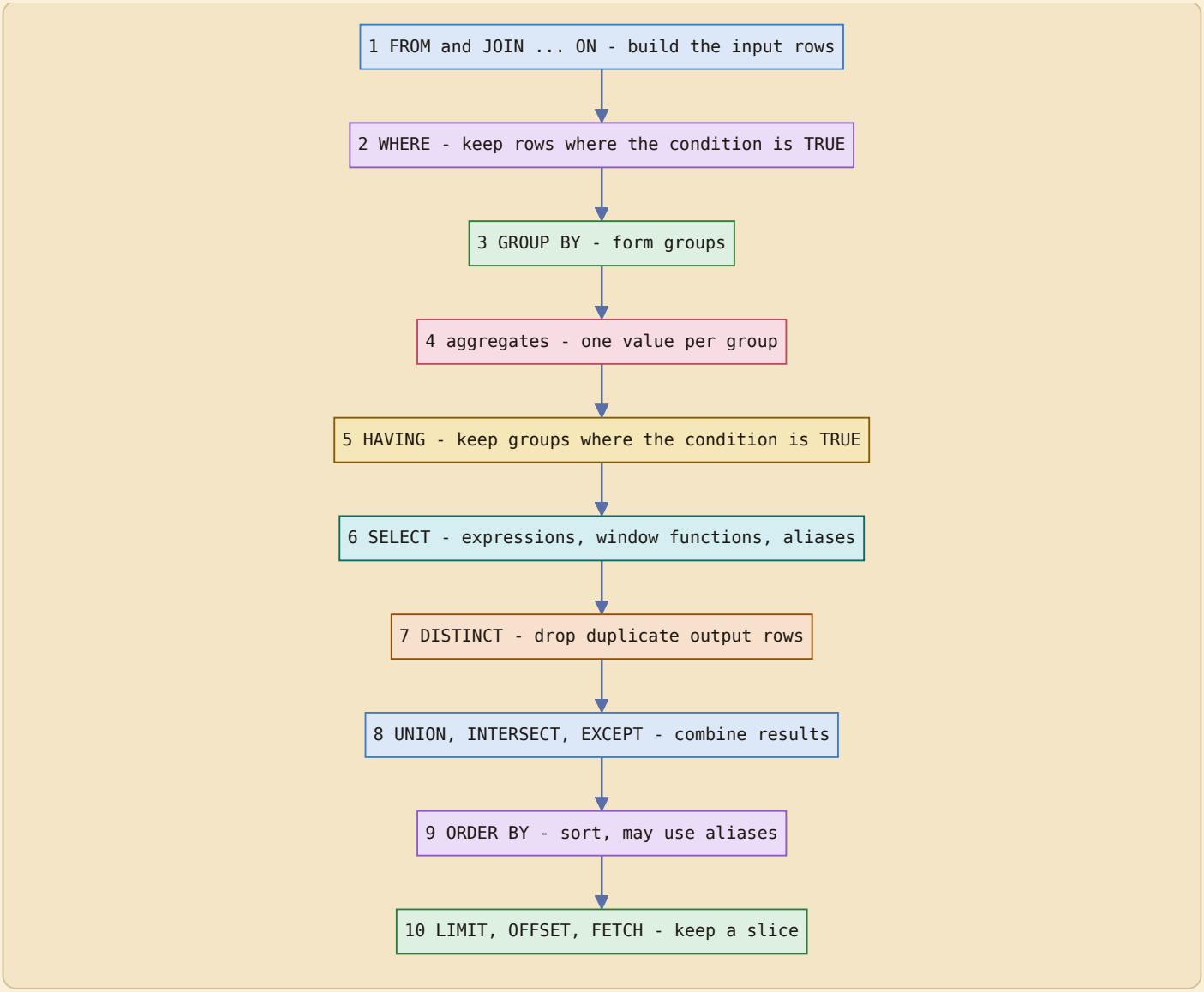


Figure 20.1 — Logical query-processing order (full explanation in §4.5 and §2.9).

Consequence	Because
`WHERE` cannot use a `SELECT` alias	aliases are created at step 6, `WHERE` is step 2
`WHERE` cannot contain `count(*)`, use `HAVING`	groups do not exist before step 3
A window function cannot appear in `WHERE` or `HAVING`	windows are computed at step 6. Filter them in an outer query or CTE (§8.10)
`ORDER BY` can use aliases and column positions	it runs after `SELECT`
`LIMIT` without `ORDER BY` returns arbitrary rows	only step 9 gives order
A filter in `WHERE` on the right table of a `LEFT JOIN` removes the padded rows	`WHERE` runs after the join has built them (§5.9)
With `SELECT DISTINCT`, `ORDER BY` may only use output columns	sorting happens after de-duplication

Two PostgreSQL relaxations: `GROUP BY` may refer to output aliases and positions (`GROUP BY 1`), and if you group by a table's primary key, the other columns of that table may be selected without being listed.

20.2 SELECT, filtering, NULL, sorting and paging

Covers Chapter 2.

```sql
SELECT DISTINCT e.dept_id, e.salary * 12 AS yearly     -- expressions + aliases
FROM   employees AS e                                   -- table alias
WHERE  e.salary BETWEEN 1000 AND 5000                   -- inclusive on both ends
  AND  e.dept_id IN (1, 2, 3)                           -- same as a chain of ORs
  AND  e.name ILIKE 'a%'                                -- LIKE is case-sensitive, ILIKE is not
  AND  e.manager_id IS NOT NULL                         -- never "= NULL"
ORDER  BY yearly DESC NULLS LAST, e.dept_id             -- add a tie-breaker
LIMIT  10 OFFSET 20;                                    -- or OFFSET 20 ROWS FETCH FIRST 10 ROWS ONLY
```

Need	Syntax	Remember
Rename a column	`expr AS name`	double-quote a name with spaces or capitals: `AS "Total"`
Pattern match	`LIKE 'a_%'`	`%` = any run of characters, `_` = exactly one. `ESCAPE` for literal `%`. `~` / `~*` for regular expressions (text patterns)
Range	`x BETWEEN a AND b`	inclusive. For timestamps use half-open `x >= a AND x < b`
NULL test	`IS NULL`, `IS NOT NULL`	`x = NULL` is never TRUE
NULL-safe compare	`a IS [NOT] DISTINCT FROM b`	treats two NULLs as equal
Default for NULL	`COALESCE(x, 0)`	returns the first non-NULL argument
Unique rows	`SELECT DISTINCT a, b`	applies to the whole output row, and NULLs count as equal
First row per group	`SELECT DISTINCT ON (g) … ORDER BY g, ts`	PostgreSQL only. The leading `ORDER BY` keys must match `g`
Sort	`ORDER BY a DESC NULLS LAST, b`	PostgreSQL puts NULLs **last** in ascending order and **first** in descending order
Page	`LIMIT n OFFSET k` / `OFFSET k ROWS FETCH FIRST n ROWS ONLY`	always with `ORDER BY`. Deep pages → keyset pagination, which continues after the last key seen (§12.14)
Keep ties at the cut-off	`FETCH FIRST n ROWS WITH TIES`	needs `ORDER BY` (PostgreSQL 13+)

Three-valued logic. A comparison with `NULL` gives `UNKNOWN` (shown as NULL). `WHERE` and `HAVING` keep only rows where the condition is TRUE. `CHECK` constraints reject only FALSE.

AND	TRUE	FALSE	UNKNOWN
TRUE	TRUE	FALSE	UNKNOWN
FALSE	FALSE	FALSE	FALSE
UNKNOWN	UNKNOWN	FALSE	UNKNOWN

OR	TRUE	FALSE	UNKNOWN
TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	UNKNOWN
UNKNOWN	TRUE	UNKNOWN	UNKNOWN

`NOT UNKNOWN` is UNKNOWN. Operator precedence is `NOT` > `AND` > `OR`, so add parentheses whenever you mix `AND` and `OR`.

Traps: `x <> 5` silently drops rows where `x` is NULL. `NOT IN (list containing NULL)` returns no rows (§2.5, §6.2). `BETWEEN '2024-01-01' AND '2024-01-31'` on a timestamp misses everything after midnight on the 31st. `OFFSET 100000` still reads and throws away 100,000 rows.

20.3 Functions and expressions

Covers Chapter 3.

Area	PostgreSQL	Notes
Cast	`CAST(x AS numeric(10,2))`, `x::int`	`::` is PostgreSQL-only. Bad input raises an error instead of guessing
Division	`7 / 2` → `3`, `7 / 2.0` → `3.5`, `7::numeric / 2` → `3.5`	integer ÷ integer truncates toward zero (`-7 / 2` → `-3`)
Remainder	`7 % 3`, `mod(7, 3)`	takes the sign of the dividend (`-7 % 2` → `-1`)
Rounding	`round(x, 2)`, `trunc(x)`, `ceil`, `floor`	`round(2.5)` → 3 on `numeric` (half away from zero), `round(2.5::float8)` → 2 (half to even)
Logs and powers	`ln(x)`, `log(x)` (base 10), `log(b, x)`, `power(a, b)`, `sqrt`	MySQL's `LOG(x)` is the natural log
Safe divide	`a / NULLIF(b, 0)`	`1 / 0` is an error (SQLSTATE 22012, a standard error code). The `NULLIF` version returns NULL. MySQL and SQLite return NULL for `x / 0` instead of failing (20.16)
Concatenate	`a || b`, `concat(a, b)`, `concat_ws(', ', a, b)`	`||` with a NULL gives NULL. `concat` skips NULLs
Length	`length(s)` / `char_length(s)` = characters, `octet_length(s)` = bytes	`'é'` is 1 character, 2 bytes in UTF-8 (the usual text encoding)
Pieces	`substring(s FROM 2 FOR 3)`, `left`, `right`, `split_part(s, ',', 2)`	positions are 1-based
Clean up	`trim`, `lower`, `upper`, `initcap`, `replace`, `translate`, `lpad`, `regexp_replace`	
Search	`position('x' IN s)`, `strpos(s, 'x')`	0 means "not found"
Now	`now()` = `current_timestamp` (fixed for the transaction), `clock_timestamp()` (real time)	`current_date` for today
Date arithmetic	`d + 7`, `ts + interval '2 hours'`, `date '2024-03-01' - date '2024-02-01'` → `29`	`date - date` is an integer number of days. Month addition clamps (Jan 31 + 1 month = Feb 29 in 2024)
Parts	`extract(year FROM ts)`, `date_part('dow', d)` (Sunday = 0), `date_trunc('month', ts)`, `age(a, b)`	
Format and parse	`to_char(ts, 'YYYY-MM')`, `to_date('03/05/2024', 'DD/MM/YYYY')`	
Time zones	`timestamptz` stores an instant. `ts AT TIME ZONE 'Europe/Paris'` converts	store `timestamptz`, not `timestamp`
Conditionals	`CASE WHEN … THEN … ELSE … END`, `COALESCE`, `NULLIF`, `GREATEST`, `LEAST`	no `ELSE` → NULL. PostgreSQL's `GREATEST`/`LEAST` ignore NULLs

Traps: integer averages and percentages (`100 * a / b` with integers truncates, so write `100.0 * a / b`), `float` for money (use `numeric`), `timestamp` without a time zone for events in several regions, and assuming `LOG` means the same thing in every database.

20.4 Aggregation

Covers Chapter 4.

```sql
SELECT   dept_id,
         count(*)                                  AS staff,        -- rows
         count(manager_id)                         AS with_manager, -- non-NULL values
         count(DISTINCT job)                       AS jobs,
         round(avg(salary), 2)                     AS avg_salary,   -- NULLs ignored
         count(*) FILTER (WHERE salary > 5000)     AS high_paid,    -- conditional aggregate
         sum(CASE WHEN remote THEN 1 ELSE 0 END)   AS remote_staff, -- portable version
         string_agg(name, ', ' ORDER BY name)      AS names
FROM     employees
WHERE    hired >= date '2020-01-01'                -- filters rows before grouping
GROUP BY dept_id
HAVING   count(*) >= 3                             -- filters groups after aggregation
ORDER BY staff DESC;
```

Function	Returns	NULL behaviour / empty input
`count(*)`	number of rows	0 for no rows
`count(x)`, `count(DISTINCT x)`	non-NULL values (distinct)	0 for no rows
`sum`, `avg`, `min`, `max`	as named	skip NULLs. **NULL** for no rows (use `COALESCE(sum(x), 0)`)
`avg(int_col)`	`numeric` in PostgreSQL	SQL Server returns an integer
`string_agg(x, sep ORDER BY …)`, `array_agg`, `json_agg`	concatenated value	`string_agg` skips NULLs
`bool_and`, `bool_or`	boolean	
`percentile_cont(0.5) WITHIN GROUP (ORDER BY x)`	median (interpolated)	`percentile_disc` returns an actual value; `mode()` the most common
`GROUPING SETS ((a), (b), ())`, `ROLLUP (a, b)`, `CUBE (a, b)`	subtotals in one query	`grouping(a)` = 1 marks a rolled-up row. `ROLLUP (a, b)` = (a,b), (a), ()

Rules: every selected column must be grouped or aggregated (or be functionally dependent on a grouped primary key, meaning its value is fixed once the key is known). NULLs form **one** group. `GROUP BY` does not sort. An aggregate query without `GROUP BY` always returns exactly one row, even over zero rows. That is why "return NULL if there is no answer" questions use an aggregate or scalar subquery (P39).

Traps: `WHERE count(*) > 1` (use `HAVING`). `count(*)` after a `LEFT JOIN` counts the padding row, so use `count(right_table.id)` to get zero (§5.9). Averages of averages. Joining two child tables before aggregating multiplies the rows (fan-out).

20.5 Joins

Covers Chapter 5.

```sql
SELECT c.name, o.id, o.total
FROM   customers c
LEFT   JOIN orders o
       ON  o.customer_id = c.id
       AND o.placed_at >= date '2024-01-01';   -- right-table filter belongs in ON
```

Join	Returns	Typical question
`CROSS JOIN`	every pair, m × n rows	"every student × every subject" grid
`[INNER] JOIN … ON`	only pairs that match	"orders with their customer"
`LEFT [OUTER] JOIN`	every left row, with matches or NULL padding	"all customers, with their orders if any"
`RIGHT [OUTER] JOIN`	mirror of `LEFT`	rewrite as `LEFT` for readability
`FULL [OUTER] JOIN`	all rows of both sides	reconciling two lists
Self join	a table joined to itself under two aliases	employee → manager, pairs, previous row
Semi-join: `EXISTS`, `IN`	left rows that have at least one match, each once	"customers who ordered"
Anti-join: `NOT EXISTS`, `LEFT JOIN … WHERE r.id IS NULL`, `EXCEPT`	left rows with no match	"customers who never ordered"
`JOIN … USING (col)`	equality on same-named columns, one output column	fine. `NATURAL JOIN` is not (it silently joins on every shared name)
`JOIN LATERAL (subquery) ON true`	per-row subquery that may reference the left row	"top 3 orders per customer" (§6.7)

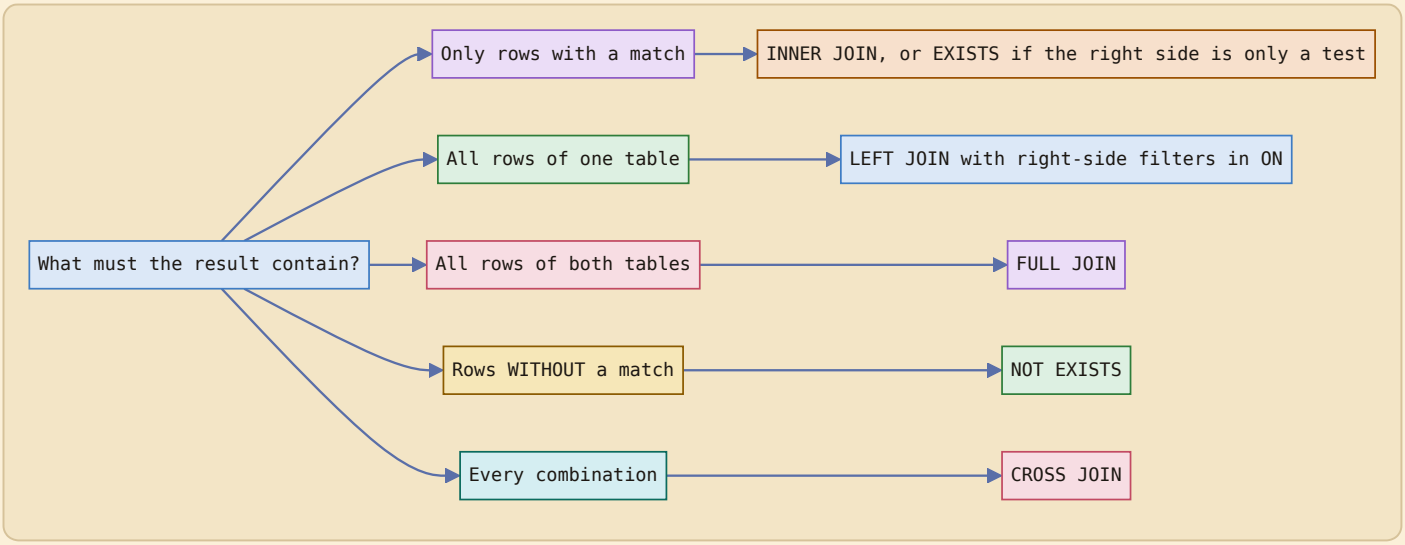


Figure 20.2 — Choosing a join.

Pitfalls (§5.9): fan-out when joining two one-to-many children, right-table conditions in `WHERE` after a `LEFT JOIN`, NULL join keys (never match), `count(*)` of padded rows, joining on names instead of keys, and `NOT IN` with NULLs. **How joins run:** nested loop (few outer rows, indexed inner key), hash join (large, equality only), merge join (both inputs sorted) (§12.15).

20.6 Subqueries and set operations

Covers Chapter 6.

Form	Example	Watch out for
Scalar subquery	`SELECT name, (SELECT max(total) FROM orders o WHERE o.cid = c.id) FROM customers c`	0 rows → NULL. 2+ rows → error
`IN` / `NOT IN`	`WHERE id IN (SELECT cid FROM orders)`	`NOT IN` returns nothing if the subquery yields a NULL
Correlated	inner query uses a column of the outer row	conceptually runs once per outer row. Qualify every column
`EXISTS` / `NOT EXISTS`	`WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.cid = c.id)`	the safe anti-join. The select list inside does not matter
`ANY` / `ALL`	`salary > ALL (SELECT …)`	`> ALL (empty)` is TRUE. `> ANY (empty)` is FALSE. `= ANY` = `IN`, `<> ALL` = `NOT IN`
Derived table	`FROM (SELECT …) AS t`	alias required before PostgreSQL 16
Row subquery	`WHERE (a, b) = (SELECT x, y …)`	also `(a, b) IN (…)`
`LATERAL`	`CROSS JOIN LATERAL (SELECT … WHERE o.cid = c.id LIMIT 3) t`	`LEFT JOIN LATERAL … ON true` keeps rows with no match

Set operator	Result	Notes
`UNION`	rows in either query, duplicates removed	costs a sort or hash
`UNION ALL`	everything, duplicates kept	the default choice when duplicates are impossible or wanted
`INTERSECT [ALL]`	rows in both	NULLs compare as equal here
`EXCEPT [ALL]`	rows in the first but not the second	not commutative. Oracle calls it `MINUS`

Rules: same number of columns, compatible types, column names come from the first query, `INTERSECT` binds tighter than `UNION`/`EXCEPT`, and `ORDER BY`/`LIMIT` apply to the whole combined result (§6.11).

20.7 CTEs, recursion and views

Covers Chapter 7.

```sql
WITH RECURSIVE chain AS (
    SELECT id, manager_id, name, 1 AS depth, ARRAY[id] AS path   -- anchor: the start rows
    FROM   employees WHERE manager_id IS NULL
  UNION ALL
    SELECT e.id, e.manager_id, e.name, c.depth + 1, c.path || e.id   -- recursive term
    FROM   employees e JOIN chain c ON e.manager_id = c.id
    WHERE  NOT e.id = ANY (c.path)                                    -- cycle guard
)
SELECT repeat('  ', depth - 1) || name AS org_chart FROM chain ORDER BY path;
```

Tool	What it is	Use it for	Remember
CTE `WITH x AS (…)`	a named subquery for one statement	readable step-by-step queries, reuse by name	PostgreSQL 12+ inlines (merges into the main query) single-use CTEs. `AS MATERIALIZED` / `NOT MATERIALIZED` override
Data-modifying CTE	`WITH moved AS (DELETE … RETURNING *) INSERT … SELECT * FROM moved`	several changes in one atomic statement	PostgreSQL only. Every part sees the same snapshot (frozen view of the data)
Recursive CTE	anchor `UNION [ALL]` recursive term	trees, graphs, number and date series	the recursive term sees only the previous round. Stops when a round adds no rows. Needs a stop condition
`CYCLE` / `SEARCH` clauses	built-in cycle detection and ordering	graphs	PostgreSQL 14+
`generate_series(a, b, step)`	a set-returning function (it returns rows)	calendars, filling gaps with zeros	inclusive of both ends. With `date` arguments and an `interval` step it returns `timestamptz`, so cast with `::date`
View	a stored query, no data	simplicity, security, stable interfaces	always current. Simple one-table views are updatable. `WITH CHECK OPTION`
Materialized view	a stored result	expensive reports	stale until `REFRESH MATERIALIZED VIEW [CONCURRENTLY]` (needs a unique index)

20.8 Window functions and frames

Covers Chapter 8.

```
function(args) OVER (
    PARTITION BY dept_id                  -- independent groups; rows are NOT collapsed
    ORDER BY     hired, id                -- order inside each partition
    ROWS BETWEEN 2 PRECEDING AND CURRENT ROW   -- frame: which rows the function sees
)
```

Function	Result on scores 30, 20, 20, 10 (descending)	Uses the frame?
`row_number()`	1, 2, 3, 4 (ties broken arbitrarily unless you add a tie-breaker)	no
`rank()`	1, 2, 2, 4 (gaps after ties)	no
`dense_rank()`	1, 2, 2, 3 (no gaps)	no
`ntile(n)`	bucket numbers 1..n. Larger buckets first (7 rows into 3 → 3, 2, 2)	no
`percent_rank()` / `cume_dist()`	(rank − 1)/(n − 1) / (rows sorted before the current row + its peers)/n	no
`lag(x, k, default)` / `lead(…)`	value k rows before / after (NULL or `default` past the edge)	no
`first_value`, `last_value`, `nth_value`	value at a position **in the frame**	**yes**
`sum`, `avg`, `count`, `min`, `max` … `OVER`	aggregate over the frame	**yes**

Frame	Meaning
no `ORDER BY` in `OVER`	frame = whole partition (`sum(x) OVER ()` = grand total on every row)
`ORDER BY` without a frame clause	**`RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`**, which includes all peers (ties) of the current row
`ROWS BETWEEN n PRECEDING AND CURRENT ROW`	physical rows. Use for moving averages and strict running totals
`RANGE BETWEEN interval '7 days' PRECEDING AND CURRENT ROW`	by value distance on the single `ORDER BY` key. Ties included
`GROUPS BETWEEN 1 PRECEDING AND CURRENT ROW`	by peer groups (PostgreSQL 11+)
`EXCLUDE CURRENT ROW / GROUP / TIES`	remove rows from the frame (PostgreSQL 11+)

Patterns: running total `sum(x) OVER (PARTITION BY g ORDER BY t)`, share of total `x / sum(x) OVER ()`, change vs previous `x - lag(x) OVER (…)`, 7-row moving average with `ROWS 6 PRECEDING`, top-N per group (rank in a CTE, filter outside), de-duplication (`row_number() = 1`), gaps and islands (`value − row_number()` is constant inside a run) (§8.7–§8.9).

Traps: `last_value` with the default frame returns the current row (give it `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`). The default `RANGE` frame gives tied rows the same running total. Windows cannot be used in `WHERE`, and PostgreSQL has no `QUALIFY`. `IGNORE NULLS` arrives only in PostgreSQL 19 (in beta as of September 2026), so on 14–18 emulate it (§8.6). Each distinct window specification may need its own sort (§8.10).

20.9 DDL, DML, constraints and upsert

Covers Chapter 9.

```sql
CREATE TABLE orders (
    id           bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    customer_id  bigint NOT NULL REFERENCES customers (id) ON DELETE RESTRICT,
    status       text   NOT NULL DEFAULT 'new'
                 CONSTRAINT orders_status_ck CHECK (status IN ('new', 'paid', 'shipped')),
    total        numeric(12, 2) NOT NULL CHECK (total >= 0),
    placed_at    timestamptz NOT NULL DEFAULT now(),
    UNIQUE (customer_id, placed_at)
);
CREATE INDEX ON orders (customer_id);                -- FKs are not indexed automatically

INSERT INTO orders (customer_id, total) VALUES (1, 20), (2, 35) RETURNING id;
UPDATE orders SET status = 'paid' WHERE id = 1 RETURNING *;
DELETE FROM orders o USING customers c WHERE o.customer_id = c.id AND c.banned;

INSERT INTO stock (sku, qty) VALUES ('A-1', 5)
ON CONFLICT (sku) DO UPDATE SET qty = stock.qty + EXCLUDED.qty;   -- upsert
```

Statement	Does	Remember
`CREATE TABLE [IF NOT EXISTS]`	defines a table	`CREATE TABLE t AS SELECT …` copies data but no constraints. `LIKE src INCLUDING ALL` copies structure
`ALTER TABLE … ADD / DROP / RENAME COLUMN`, `ALTER COLUMN … TYPE t USING expr`, `SET NOT NULL`, `ADD CONSTRAINT … NOT VALID` + `VALIDATE CONSTRAINT`	changes a table	many forms take an exclusive lock (blocking all other use of the table). Backfill (fill existing rows) before `SET NOT NULL`
`DROP TABLE [IF EXISTS] … [CASCADE]`	removes the table itself	`CASCADE` also drops dependent views and FKs
`TRUNCATE t [RESTART IDENTITY] [CASCADE]`	empties the table fast	no per-row triggers. Transactional in PostgreSQL
`DELETE FROM t WHERE …`	removes chosen rows	fires triggers, logs every row. Forget `WHERE` and every row goes
`INSERT … SELECT`	copies rows	list the target columns explicitly
`UPDATE … SET a = b, b = a`	swaps values	every `SET` expression sees the **old** row
`… RETURNING`	returns affected rows	PostgreSQL, SQLite 3.35+. SQL Server uses `OUTPUT`
`INSERT … ON CONFLICT (key) DO NOTHING / DO UPDATE SET c = EXCLUDED.c`	upsert	needs a unique index or constraint on the conflict target
`MERGE INTO t USING s ON … WHEN MATCHED … WHEN NOT MATCHED …`	standard upsert and sync	PostgreSQL 15+ (not on the book's server, 14)

Constraint	Rejects	Notes
`NOT NULL`	NULL	the most useful constraint
`PRIMARY KEY`	duplicates and NULLs	one per table. Creates a unique index
`UNIQUE`	duplicate non-NULL values	several NULLs allowed (SQL Server: one). `NULLS NOT DISTINCT` in PostgreSQL 15+
`CHECK (expr)`	rows where `expr` is FALSE	NULL passes. Cannot look at other rows
`FOREIGN KEY … REFERENCES`	child values with no parent	`ON DELETE CASCADE / SET NULL / SET DEFAULT / RESTRICT / NO ACTION` (default). NO ACTION can be deferred, RESTRICT cannot
`DEFAULT`	–	used when the column is omitted
`EXCLUDE USING gist (…)`	overlapping values	"no two bookings of one room overlap" (§17.2)

Key generation	Use
`GENERATED ALWAYS AS IDENTITY`	the standard choice. `serial` is the legacy form
Sequence (`nextval`)	shared or custom counters. Gaps are normal (rollbacks, caching)
`uuid` (`gen_random_uuid()`, PostgreSQL 13+)	ids made on many machines. Random UUIDs scatter B-tree inserts
`GENERATED ALWAYS AS (expr) STORED`	derived columns kept in sync automatically. PostgreSQL 18 adds `VIRTUAL` (computed on read), which becomes the default when neither keyword is given

Backups (§9.12): `pg_dump` makes a logical copy, the statements needed to rebuild the data (plain SQL, or `-Fc` custom / `-Fd` directory formats for `pg_restore`, which can restore selected objects and run in parallel). Physical base backups (file-level copies of the whole server) + archived WAL (the write-ahead log of every change) allow point-in-time recovery (PITR). Plan by RPO (how much data you may lose) and RTO (how long recovery may take), keep 3 copies on 2 media with 1 off-site, and test restores.

20.10 Data modeling and normalization quick rules

Covers Chapter 10, Chapter 11 and Chapter 17.

Why databases exist (§1.1). File-based systems suffered from redundancy and inconsistency (the same fact stored in several files, drifting apart), isolated data in different formats, no general way to ask a new question, integrity rules buried in program code, lost updates when two users wrote at once, no atomic recovery after a crash, and all-or-nothing security. A DBMS (database management system) answers each of these by mediating every access.

The three levels of a model (§10.2).

Level	Audience	Contains	Does not contain
Conceptual	business people	entities and relationships	data types, keys, DBMS details
Logical	designers	attributes, keys, cardinalities, normalized tables	a particular product's types or indexes
Physical	the DBMS	exact types, indexes, partitions, storage	–

ER notation (§10.5).

Concept	Chen notation	Crow's foot
Entity / weak entity	rectangle / double rectangle	box
Attribute	ellipse (underlined = key, double = multivalued, dashed = derived)	listed inside the box
Relationship / identifying relationship	diamond / double diamond	a line / a solid line to a weak entity
Cardinality	`1`, `N`, `M` on the lines	line ends: `||` exactly one, `o|` zero or one, `|{` one or many, `o{` zero or many
Participation	double line = total (mandatory)	inner mark: `|` mandatory, `o` optional

Mapping ER → tables (§10.15). Entity → table. Composite attribute → its parts. Multivalued attribute → its own table. Derived attribute → a view or generated column. 1:N → FK on the "many" side. M:N → junction table with a composite PK. 1:1 → FK + `UNIQUE`. Weak entity → owner's key + partial key as a composite PK, with `ON DELETE CASCADE`. Total participation → `NOT NULL` on the FK.

Keys (§10.13). Superkey (any column set that identifies a row) ⊇ candidate key (a minimal superkey) ⊇ the one chosen **primary key**. The other candidates are **alternate keys** (enforce with `UNIQUE`). A **natural key** comes from the data (email, ISBN), a **surrogate key** is made up (identity, UUID). A good default: surrogate PK + `UNIQUE` on the natural key. **Prime attribute** = belongs to some candidate key.

Functional dependencies (§11.1–§11.5). `X → Y` means rows that agree on X agree on Y. It is a business rule: data can disprove it but never prove it. **Closure X^+**: start with X, fire every FD whose left side is fully inside the set, repeat until nothing changes. X is a superkey when X^+ contains every attribute. **Candidate keys**: attributes that never appear on a right side must be in every key, so start from them. **Armstrong's axioms**: reflexivity, augmentation, transitivity (plus derived union, decomposition, pseudotransitivity). **Minimal cover**: single-attribute right sides, no extraneous left-side attributes, no redundant FDs.

Normal forms (§11.6–§11.13).

Form	Rule (informal)	Removes
1NF	every cell holds one atomic value, no repeating groups	lists in a column, `phone1, phone2, phone3`
2NF	1NF + no non-prime attribute depends on **part** of a composite key	partial dependencies
3NF	2NF + no non-prime attribute depends on another non-key attribute. Formally: for each FD X → A, X is a superkey or A is prime	transitive dependencies
BCNF	for every non-trivial FD X → Y, X is a superkey	the remaining FD redundancy
4NF	BCNF + no non-trivial multivalued dependency unless its left side is a superkey	independent lists stored together
5NF	every join dependency follows from the keys	facts that only make sense as three-way combinations

Two things people often get wrong about 1NF: atomicity depends on how the data is **used** (a full name is atomic if you never query first names separately), and a table is not in 1NF just because each cell holds one string. A comma-separated list in one string still breaks it.

Anomalies (why normalize): *insertion* (cannot record a fact without an unrelated one), *update* (one fact stored in many rows, so a change must touch all of them), *deletion* (removing one fact loses another). **Every decomposition must be lossless** (the shared columns are a key of at least one piece) and ideally **dependency-preserving**. 3NF can always have both. BCNF sometimes loses an FD, so most designs stop at 3NF (§11.10).

Design judgement calls (§17.14).

Decision	Rule of thumb
Type vs instance	separate the catalogue row from the physical item (title vs copy, room type vs room, seat vs seat-for-a-show)
Current state vs history	a status column for "now" plus an append-only events or ledger table for "what happened"
Snapshot vs reference	copy values that must not change later (price paid, shipping address) into the order row. That is a separate fact, not redundancy
Delete vs soft-delete	soft-delete (`deleted_at`) rows that history still references
Enforce where	rules in the schema (`CHECK` , `UNIQUE` , partial unique index, `EXCLUDE`) rather than only in application code
Denormalize when	only for a measured read problem, updated in the same transaction, with a reconciliation query (§11.16)
Scarce items	a guarded `UPDATE … WHERE status = 'free'` in a short transaction, or `FOR UPDATE SKIP LOCKED`

20.11 Indexes and reading EXPLAIN

Covers Chapter 12.

```
CREATE INDEX CONCURRENTLY orders_cust_date_ix ON orders (customer_id, placed_at DESC); -- composite
CREATE INDEX ON orders (customer_id) INCLUDE (total);              -- covering (index-only scans)
CREATE INDEX ON orders (placed_at) WHERE status = 'new';           -- partial
CREATE INDEX ON users (lower(email));                              -- expression
CREATE INDEX ON docs USING gin (body_tsv);                         -- full-text / jsonb / arrays
CREATE INDEX ON readings USING brin (taken_at);                    -- huge, append-only, time-ordered
EXPLAIN (ANALYZE, BUFFERS) SELECT … ;                              -- runs the query and shows the plan
```

Which index?

Query shape	Index	Why
`=`, `<`, `>`, `BETWEEN`, `ORDER BY`, `LIKE 'abc%'` (with `text_pattern_ops` or C collation, i.e. plain byte-order sorting)	**B-tree** (default)	sorted, supports ranges and order
`WHERE a = ? AND b > ?`	composite B-tree `(a, b)`	equality columns first, range column last
`WHERE a = ?` returning only a few columns	`(a) INCLUDE (x, y)`	index-only scan, no table visits
Only a small hot subset is queried (`status = 'new'`)	partial index `WHERE status = 'new'`	smaller, cheaper to maintain
`WHERE lower(email) = ?`	expression index on `lower(email)`	the index must match the expression
Equality only, very large keys	hash	cannot do ranges or ordering
JSONB `@>`, arrays `&&` / `@>`, full-text `@@`, trigram `LIKE '%x%'`	**GIN** (`pg_trgm` for trigrams)	inverted index: value → rows
Ranges overlapping, geometry, nearest neighbour, `EXCLUDE` constraints	**GiST**	tree of bounding boxes
Billions of rows physically ordered by time	**BRIN**	a few bytes per block range
Low-selectivity column (`is_active` true for 90 %)	usually **none**	a sequential scan is cheaper

Reading a plan (§12.10). Read from the innermost node outward. Each node shows `cost=startup..total` (planner units), estimated `rows`, and, with `ANALYZE`, `actual time`, actual `rows` and `loops` (multiply per-loop numbers by loops). `BUFFERS` shows pages hit in memory vs read.

Node	Means	Chosen when
Seq Scan	reads every page	a large fraction of the table is needed, or no usable index
Index Scan	walks the index, then fetches each row	few rows, or rows needed in index order
Index Only Scan	answers from the index alone	all needed columns are in the index and pages are all-visible (vacuumed)
Bitmap Index Scan + Bitmap Heap Scan	collects row locations, then reads pages in order	a medium number of scattered rows. Can combine indexes with AND/OR
Nested Loop	for each outer row, probe the inner	small outer side + index on the inner join key
Hash Join	builds a hash table on the smaller input	large inputs, equality joins
Merge Join	merges two sorted inputs	both sides already sorted (or cheap to sort)
Sort / Incremental Sort, HashAggregate / GroupAggregate	sorting and grouping	watch `Sort Method: external merge` (spilled to disk)

The big warning sign: estimated rows far from actual rows. Refresh statistics with `ANALYZE`, and consider extended statistics (statistics on combinations of columns) for correlated columns (§12.16).

Anti-patterns that disable an index (§12.13): a function or arithmetic on the indexed column (`WHERE date(ts) = …`, `WHERE price * 1.2 > …`), a leading wildcard (`LIKE '%x'`), implicit casts, `OR` across different columns (sometimes fixable with `UNION` or a bitmap), a composite index whose leading column is not filtered (PostgreSQL 18+, like Oracle and MySQL 8.0.13+, can sometimes skip-scan it when that column has few distinct values, but don't design around it), deep `OFFSET` paging (use keyset: `WHERE (ts, id) < (:last_ts, :last_id) ORDER BY ts DESC, id DESC LIMIT 20`), and N+1 query loops from application code (one extra query per item of a list). Every index slows writes and uses space, so index real query patterns, not columns.

20.12 Transactions, isolation and concurrency

Covers Chapter 13.

```
BEGIN ISOLATION LEVEL REPEATABLE READ;
  SELECT balance FROM accounts WHERE id = 1 FOR UPDATE;   -- lock the row you will change
  UPDATE accounts SET balance = balance - 100 WHERE id = 1;
  SAVEPOINT s1;
  UPDATE accounts SET balance = balance + 100 WHERE id = 2;
  -- ROLLBACK TO SAVEPOINT s1;   -- undo only the part after s1
COMMIT;                          -- or ROLLBACK
```

ACID. *Atomicity*: all or nothing (undo via log or, in PostgreSQL, by marking the transaction aborted). *Consistency*: constraints and application rules hold before and after. *Isolation*: concurrent transactions do not see each other's partial work (locks, MVCC = multiversion concurrency control, SSI = serializable snapshot isolation, both explained below). *Durability*: a committed change survives a crash (write-ahead log flushed at commit, checkpoints, replicas).

Anomalies (§13.4).

Anomaly	What happens	Example
Dirty write	overwrite another transaction's uncommitted change	two uncommitted updates on one row
Dirty read	read another transaction's uncommitted change	see a balance that is later rolled back
Non-repeatable read (read skew)	the same row read twice gives different values. Read skew: different rows seen at different moments	a report sees an account before a transfer and the other account after it
Phantom read	the same query returns a different **set** of rows	a count rises because someone inserted a matching row
Lost update	two read-modify-write cycles, the second overwrites the first	two sales both set stock 10 → 9
Write skew	two transactions read the same rows, then each updates a **different** row, breaking a rule both checked	two doctors both go off call, leaving nobody on call

Isolation levels — standard minimum vs PostgreSQL.

Level	Standard forbids	PostgreSQL actually prevents	Still possible in PostgreSQL
Read Uncommitted	—	dirty reads (it behaves as Read Committed)	as Read Committed
Read Committed (PostgreSQL default)	dirty read	dirty read and write	non-repeatable read, phantom, lost update, write skew
Repeatable Read (snapshot isolation)	+ non-repeatable read	+ phantoms. Lost updates abort with SQLSTATE 40001	write skew
Serializable (SSI)	+ phantom	everything. One transaction aborts with 40001	nothing, but you must retry

Which isolation level (and lock)?

Situation	Choose
Ordinary OLTP (many small transactions), single-statement changes (`SET qty = qty - 1`)	Read Committed
A consistent multi-query report or export	Repeatable Read, `READ ONLY`
Read a row, decide, write it back	`SELECT … FOR UPDATE` at Read Committed, or an atomic `UPDATE … WHERE`, or Repeatable Read + retry
Optimistic editing across user think-time	a `version` column: `UPDATE … SET version = version + 1 WHERE id = ? AND version = ?`
A rule spanning several rows (no double booking, one on call)	Serializable + retry, an `EXCLUDE`/unique constraint, or explicit locks
Many workers taking jobs from a queue	`SELECT … FOR UPDATE SKIP LOCKED LIMIT 1`
Must not wait at all	`FOR UPDATE NOWAIT` (errors immediately if locked)

Concurrency control families (§13.9). Pessimistic locking (shared/exclusive locks, two-phase locking: grow then shrink, strict 2PL holds write locks to commit). Optimistic (work freely, validate at commit, abort on conflict). Timestamp ordering (each transaction gets a timestamp, operations out of order are rejected). **MVCC** (writers create new row versions, readers see a snapshot, so readers never block writers. PostgreSQL marks versions with `xmin`/`xmax` and cleans dead ones with VACUUM).

Deadlocks (§13.11): two transactions each wait for a lock the other holds. PostgreSQL detects it after `deadlock_timeout` and aborts one (SQLSTATE 40P01). Prevent it by locking rows in a consistent order and keeping transactions short. Application code should **retry on 40001 and 40P01**.

20.13 Advanced PostgreSQL: PL/pgSQL, JSONB, full-text search, security

Covers Chapter 14.

```sql
CREATE FUNCTION order_total(p_id bigint) RETURNS numeric
LANGUAGE sql STABLE AS $$ SELECT sum(qty * price) FROM order_lines WHERE order_id = p_id $$;

CREATE PROCEDURE archive_old() LANGUAGE plpgsql AS $$
BEGIN
  LOOP
    DELETE FROM events WHERE id IN (SELECT id FROM events WHERE ts < now() - interval '1 year' LIMIT
10000);
    EXIT WHEN NOT FOUND;
    COMMIT;                                  -- procedures may commit (when CALLed outside a transaction)
  END LOOP;
END $$;
CALL archive_old();

CREATE FUNCTION touch() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN NEW.updated_at := now(); RETURN NEW; END $$;
CREATE TRIGGER t_touch BEFORE UPDATE ON products FOR EACH ROW EXECUTE FUNCTION touch();
```

Feature	Syntax	Remember
Function	`CREATE FUNCTION f(args) RETURNS t LANGUAGE sql / plpgsql`	declare volatility honestly: `IMMUTABLE`, `STABLE`, `VOLATILE` (default)
Procedure	`CREATE PROCEDURE`, `CALL p()`	can `COMMIT` / `ROLLBACK` inside. `INOUT` parameters
Trigger	`BEFORE` / `AFTER` / `INSTEAD OF`, `FOR EACH ROW` / `STATEMENT`	BEFORE can modify `NEW` or reject (return NULL). AFTER reacts. Keep them fast
JSONB	`->` (jsonb), `->>` (text), `#>> '{a,b}'`, `@>` (contains), `?` (key exists), `jsonb_set`, `jsonb_each`, `jsonb_array_elements`	binary, keys de-duplicated (last wins) and reordered. GIN index for `@>` and `?`. In a quick demo, `VACUUM` after bulk updates before building the index, or older snapshots may ignore it
Arrays	`ARRAY[1,2]`, `a[1]` (1-based), `= ANY(a)`, `@>`, `&&`, `unnest`, `array_agg`	out-of-range subscript returns NULL
Full-text search	`to_tsvector('english', body) @@ websearch_to_tsquery('english', 'fast db')`, `ts_rank`	stemming and stop words. Store a generated `tsvector` column with a GIN index
Pivot	`max(v) FILTER (WHERE k = 'a') AS a … GROUP BY id`, or `crosstab` (tablefunc)	unpivot with `CROSS JOIN LATERAL (VALUES ('a', a), ('b', b))`
Roles and privileges	`CREATE ROLE app LOGIN`, `GRANT SELECT, INSERT ON t TO app`, `REVOKE`	group roles, least privilege
Row-level security	`ALTER TABLE t ENABLE ROW LEVEL SECURITY`, `CREATE POLICY p ON t USING (tenant_id = current_setting('app.tenant')::int)`	`USING` filters reads, `WITH CHECK` filters writes. Superusers and `BYPASSRLS` roles always bypass it; the table owner does too unless `FORCE ROW LEVEL SECURITY`
SQL injection defence	bind parameters: psycopg `%s`, sqlite3 `?`, JDBC `PreparedStatement` `?`	never concatenate user input. Identifiers: allow-list or `format('%I', …)`
From code	explicit transactions, `Decimal`/`BigDecimal` for money, batching or `COPY` for bulk loads, stream large results	a connection pool (small, reused) or PgBouncer (a pooling proxy) in front of many clients
ORMs	fast for CRUD (create, read, update, delete)	watch for N+1 queries. Hand-write complex and hot queries (§14.13)

20.14 Scaling: partitioning, sharding, replication

Covers Chapter 15.

```
CREATE TABLE events (id bigint, ts timestamptz NOT NULL, payload jsonb)
PARTITION BY RANGE (ts);
CREATE TABLE events_2024_06 PARTITION OF events
  FOR VALUES FROM ('2024-06-01') TO ('2024-07-01');       -- upper bound is exclusive
-- also PARTITION BY LIST (region) and PARTITION BY HASH (id) … FOR VALUES WITH (MODULUS 4, REMAINDER 0)
```

Tool	What it does	Choose it when	Cost
Scale up	a bigger server	first, almost always	a ceiling, one failure domain
Read replicas	copies that serve reads	read-heavy workloads, reporting	replication lag, stale reads
Partitioning (one server)	split a table by range, list or hash	huge tables, time-based retention (drop a partition), pruning	unique keys must include the partition key
Sharding (many servers)	partitions on different machines	writes or data exceed one server	cross-shard joins and transactions, rebalancing, operations
Caching	keep hot results in memory	repeated reads of the same data	invalidation
Denormalized / summary tables	precomputed answers	expensive repeated aggregates	keeping copies correct

Concept	One-line rule
Shard key	high cardinality (many distinct values), spreads load evenly, matches the most common query filter
Consistent hashing	only about 1/n of keys move when a node joins. Virtual nodes smooth the load
Synchronous vs asynchronous replication	sync = no committed data lost but slower writes. Async = fast, with lag and possible loss on failover
Single-leader / multi-leader / leaderless	one writer (simple) / several writers (conflicts to resolve) / quorums with R + W > N
Consistency models	strong (linearizable) > sequential > causal > session (read-your-writes) > eventual
CAP	during a network partition choose consistency or availability
PACELC	if Partitioned choose A or C, Else choose Latency or Consistency
Consensus (Raft, Paxos)	a majority of 2f + 1 nodes tolerates f crashes. Byzantine faults (nodes that lie, handled by PBFT) need 3f + 1
Two-phase commit	prepare, then commit. Blocks if the coordinator dies after prepare
Recovery	WAL first, checkpoints bound replay, redo committed work, undo uncommitted work
High availability	heartbeat → quorum decides → promote a replica → fence (block writes from) the old leader → redirect clients
Backups vs replicas	a replica copies your mistakes too. Only backups (with PITR) undo a bad `DELETE`

20.15 Which database?

Covers Chapter 16. Default to PostgreSQL for transactional data and add a specialised store only when a requirement clearly exceeds it (§16.10).

Family	Data shape	Strength	Weakness	Examples
Relational (OLTP)	tables, keys, constraints	joins, ACID, ad-hoc SQL	scaling writes beyond one node takes work	PostgreSQL, MySQL, SQL Server, Oracle
Key-value	key → opaque value	fastest get/put, TTL (automatic expiry), simple scale-out	no queries by value	Redis, DynamoDB (also document-like), Memcached
Document	nested JSON per record	flexible schema, whole-object reads	cross-document joins and transactions are limited or costly	MongoDB, Couchbase, Firestore
Wide-column	partition key + sorted clustering columns	huge write rates, time-ordered data per key	must model around queries, no joins	Cassandra, ScyllaDB, HBase, Bigtable
Graph	nodes and edges	multi-hop traversals, paths	bulk analytics over all data, sharding	Neo4j, Amazon Neptune
Time-series	timestamped measurements	compression, retention, downsampling	general-purpose queries	TimescaleDB, InfluxDB, Prometheus
Search engine	inverted index of documents	relevance ranking, facets, fuzzy text	not a system of record	Elasticsearch, OpenSearch
Columnar warehouse	column-oriented tables	scans and aggregates over billions of rows	slow single-row writes	BigQuery, Snowflake, Redshift, ClickHouse
NewSQL / distributed SQL	SQL tables on consensus-replicated shards	SQL + ACID across nodes and regions	latency of consensus, operational complexity	Spanner, CockroachDB, YugabyteDB, TiDB
In-memory	data kept in RAM, persisted by snapshot + log	microsecond latency	cost per GB, durability settings	Redis, SAP HANA, VoltDB

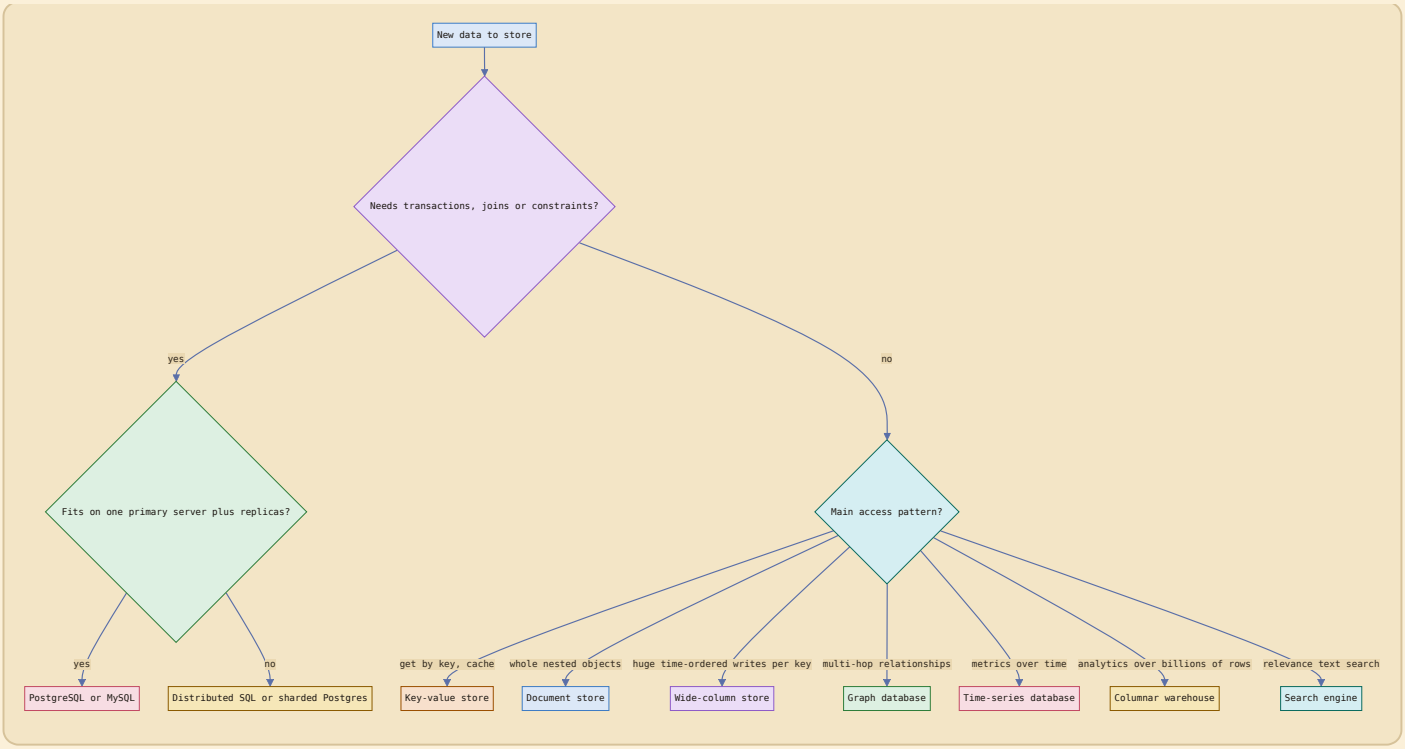


Figure 20.3 — A first-pass database choice. Real systems often combine several (polyglot persistence), fed from one source of truth by change data capture (streaming every committed change out of the main database).

ACID vs BASE. BASE = *Basically Available, Soft state, Eventually consistent*: stay up and accept writes, and let replicas converge later. It is a spectrum, not a switch: many NoSQL stores offer tunable or per-operation consistency.

20.16 Dialect differences

The book runs PostgreSQL 14. This table collects the differences you are most likely to hit when you move a query to another database. Versions in brackets are the first release with the feature. "—" means there is no built-in equivalent. Oracle means 19c, with features added in 21c or 23ai (renamed 26ai) marked. The SQLite column was checked on SQLite 3.43 by `code/02_sqlite_dialect.sqlite.sql` (20.17). The other columns come from each vendor's documentation and were not run here.

Syntax and basics

Feature	PostgreSQL	MySQL 8	SQL Server	SQLite	Oracle
First n rows	`LIMIT n` / `FETCH FIRST n ROWS ONLY`	`LIMIT n`	`TOP (n)` / `OFFSET 0 ROWS FETCH NEXT n ROWS ONLY` (2012, needs `ORDER BY`)	`LIMIT n`	`FETCH FIRST n ROWS ONLY` (12c). Older: `ROWNUM`
Quote an identifier	`"Name"`	backticks around the name (`"` only in ANSI_QUOTES mode)	`[Name]` or `"Name"`	`"Name"` (backticks and `[Name]` also accepted)	`"Name"`
Unquoted names are folded to	lower case	case of table names depends on the OS. Column names are case-insensitive	kept. Case-sensitivity follows the collation (insensitive by default)	kept, compared case-insensitively	UPPER case
String concatenation	`a || b` (NULL if either is NULL), `concat()`	`CONCAT(a, b)` (NULL if any is NULL). `||` means OR by default	`a + b`, `CONCAT()` (NULL treated as '')	`a || b`	`a || b` (NULL treated as '')
Empty string `''`	a value, not NULL	a value	a value	a value	**is NULL**
`7 / 2`	`3`	`3.5000` (`7 DIV 2` = 3)	`3`	`3`	`3.5`
`1 / 0` in a `SELECT`	error (22012)	NULL + warning (strict mode errors only in `INSERT`/`UPDATE`)	error (8134)	NULL	error (ORA-01476)
Cast	`CAST(x AS t)`, `x::t`	`CAST`, `CONVERT(x, t)`	`CAST`, `CONVERT(t, x)`, `TRY_CAST`	`CAST`	`CAST`, `TO_NUMBER`, `TO_DATE`
Boolean type	`boolean`	`BOOLEAN` = `TINYINT(1)`	`BIT`	integers 0/1 (keywords `TRUE`/`FALSE` accepted)	`BOOLEAN` (23ai). Before: none in SQL
`SELECT` without `FROM`	yes	yes (or `FROM DUAL`)	yes	yes	`FROM DUAL` required before 23ai
Type checking	strict	lenient unless strict SQL mode (the default in 8.0)	strict	dynamic typing (type affinity). `STRICT` tables (3.37)	strict
Auto-increment	`GENERATED … AS IDENTITY`, `serial`	`AUTO_INCREMENT`	`IDENTITY(1,1)`	`INTEGER PRIMARY KEY` (optionally `AUTOINCREMENT`)	`GENERATED … AS IDENTITY` (12c), sequences
Sequences	yes	–	yes (2012)	–	yes
`DROP TABLE IF EXISTS`	yes	yes	yes (2016)	yes	23ai

NULLs, comparison and sorting

Feature	PostgreSQL	MySQL 8	SQL Server	SQLite	Oracle
NULL-safe equality	`IS NOT DISTINCT FROM`	`<=>`	`IS NOT DISTINCT FROM` (2022)	`IS` (and `IS NOT DISTINCT FROM`, 3.39)	— (use `DECODE(a, b, 1, 0) = 1`)
NULLs in `ORDER BY … ASC`	last	first	first	first	last
`NULLS FIRST / LAST`	yes	— (sort by `x IS NULL` first)	— (sort by a `CASE` first)	yes (3.30)	yes
First non-NULL	`COALESCE`	`COALESCE`, `IFNULL`	`COALESCE`, `ISNULL`	`COALESCE`, `IFNULL`	`COALESCE`, `NVL`
`GREATEST(1, NULL)`	1 (NULLs ignored)	NULL	1 (2022)	use `max(a, b)` (NULL if any NULL)	NULL
`UNIQUE` column with several NULLs	allowed (`NULLS NOT DISTINCT` 15+)	allowed	**one** NULL only (use a filtered index)	allowed	allowed (all-NULL keys are not indexed)
`LIKE` case-sensitive?	yes (`ILIKE` for no)	no with the default collation	no with the default collation	no for ASCII	yes
Regular expressions	`~`, `~*`, `regexp_replace`	`REGEXP`, `REGEXP_LIKE` (8.0)	`REGEXP_LIKE`, `REGEXP_REPLACE` (2025). Before: `LIKE` patterns only	`REGEXP` only if a function is loaded (the shell has one)	`REGEXP_LIKE`, `REGEXP_REPLACE`

Joins, subqueries and set operations

Feature	PostgreSQL	MySQL 8	SQL Server	SQLite	Oracle
`FULL OUTER JOIN`	yes	— (emulate with `UNION`)	yes	yes (3.39)	yes
`RIGHT JOIN`	yes	yes	yes	yes (3.39)	yes
`USING`, `NATURAL JOIN`	yes	yes	—	yes	yes
`LATERAL`	yes	yes (8.0.14)	`CROSS APPLY` / `OUTER APPLY`	—	`LATERAL`, `CROSS/OUTER APPLY` (12c)
`INTERSECT`, `EXCEPT`	yes	yes (8.0.31)	yes	yes	`INTERSECT`, `MINUS` (`EXCEPT` 21c)
Row values `(a, b) IN (SELECT …)`	yes	yes	— (use `EXISTS`)	yes (3.15)	yes
`UPDATE` with a join	`UPDATE t SET … FROM s WHERE …`	`UPDATE t JOIN s ON … SET …`	`UPDATE t SET … FROM t JOIN s ON …`	`UPDATE … FROM` (3.33)	`MERGE`, or a correlated subquery (direct joins 23ai)

Aggregation, CTEs and windows

Feature	PostgreSQL	MySQL 8	SQL Server	SQLite	Oracle
String aggregation	`string_agg(x, ',' ORDER BY x)`	`GROUP_CONCAT(x ORDER BY x SEPARATOR ',')`	`STRING_AGG(x, ',') WITHIN GROUP (ORDER BY x)` (2017)	`group_concat(x, ',')` (`string_agg` 3.44)	`LISTAGG(x, ',') WITHIN GROUP (ORDER BY x)`
`FILTER (WHERE …)` on aggregates	yes	— (use `CASE`)	— (use `CASE`)	yes (3.30)	— (use `CASE`)
`avg()` of an integer column	exact `numeric`	decimal	**integer** (truncated)	floating point	exact `NUMBER`
Median	`percentile_cont(0.5) WITHIN GROUP (ORDER BY x)`	—	`PERCENTILE_CONT` as a window function only	—	`MEDIAN(x)`, `PERCENTILE_CONT`
`ROLLUP` / `CUBE` / `GROUPING SETS`	all three	`WITH ROLLUP` only	all three	—	all three
Recursive CTE keyword	`WITH RECURSIVE`	`WITH RECURSIVE`	plain `WITH`	`WITH RECURSIVE`	plain `WITH` (column list required)
Window functions	yes (8.4)	yes (8.0)	ranking 2005, frames 2012	yes (3.25)	yes (8i)
`GROUPS` frames, `EXCLUDE`	yes (11)	—	—	yes (3.28)	yes (21c)
`WINDOW w AS (…)` clause	yes	yes	yes (2022)	yes	yes (21c)
`IGNORE NULLS` in `lag` / `first_value`	— (19, in beta as of September 2026)	—	yes (2022)	—	yes
`QUALIFY`	—	—	—	—	26ai
`DISTINCT ON`	yes	—	—	—	—

DDL, DML and transactions

Feature	PostgreSQL	MySQL 8	SQL Server	SQLite	Oracle
Upsert	`INSERT … ON CONFLICT`, `MERGE` (15)	`INSERT … ON DUPLICATE KEY UPDATE`, `REPLACE`, `INSERT IGNORE`	`MERGE`	`INSERT … ON CONFLICT` (3.24), `INSERT OR REPLACE`	`MERGE`
Return changed rows	`RETURNING`	—	`OUTPUT inserted.*`	`RETURNING` (3.35)	`RETURNING … INTO` (PL/SQL)
DDL inside a transaction	yes, can be rolled back	no, causes an implicit commit	mostly yes	yes	no, DDL commits
`CHECK` enforced	yes	yes (8.0.16; parsed but ignored before)	yes	yes	yes
Default isolation level	Read Committed	Repeatable Read	Read Committed (locking)	Serializable	Read Committed
"Serializable" means	SSI (true serializability)	2PL: plain reads become locking reads	2PL (or `SNAPSHOT` level)	one writer at a time	snapshot isolation (write skew possible)
Stored-procedure language	PL/pgSQL (and others)	SQL/PSM-style	T-SQL	— (host language)	PL/SQL
Materialized views	yes, manual `REFRESH`	—	indexed views	—	yes, with incremental (fast) refresh

Indexes and plans

Feature	PostgreSQL	MySQL 8 (InnoDB)	SQL Server	SQLite	Oracle
Table storage	heap + separate indexes	clustered on the primary key	heap or clustered index	B-tree keyed by rowid	heap (index-organized tables optional)
Partial index	yes	—	filtered index	yes	— (function-based index returning NULL)
Covering `INCLUDE`	yes (11)	— (add to the key)	yes	—	—
Expression index	yes	functional key parts (8.0.13)	index a computed column	yes	function-based index
Show the plan	`EXPLAIN (ANALYZE, BUFFERS)`	`EXPLAIN ANALYZE` (8.0.18), `EXPLAIN FORMAT=TREE`	actual execution plan, `SET STATISTICS IO ON`	`EXPLAIN QUERY PLAN`	`EXPLAIN PLAN FOR` + `DBMS_XPLAN.DISPLAY`

Dates and functions

Task	PostgreSQL	MySQL 8	SQL Server	SQLite	Oracle
Now	`now()`	`NOW()`	`SYSDATETIME()`, `GETDATE()`	`datetime('now')`	`SYSTIMESTAMP`, `SYSDATE`
Add 3 days	`d + 3`, `ts + interval '3 days'`	`DATE_ADD(d, INTERVAL 3 DAY)`	`DATEADD(day, 3, d)`	`date(d, '+3 days')`	`d + 3`, `d + INTERVAL '3' DAY`
Days between	`d2 - d1`	`DATEDIFF(d2, d1)`	`DATEDIFF(day, d1, d2)`	`julianday(d2) - julianday(d1)`	`d2 - d1`
Truncate to month	`date_trunc('month', d)`	`DATE_FORMAT(d, '%Y-%m-01')`	`DATETRUNC(month, d)` (2022)	`date(d, 'start of month')`	`TRUNC(d, 'MM')`
Year of a date	`extract(year FROM d)`	`YEAR(d)`	`YEAR(d)`	`strftime('%Y', d)`	`EXTRACT(YEAR FROM d)`
Characters / bytes	`length` / `octet_length`	`CHAR_LENGTH` / `LENGTH`	`LEN` (ignores trailing spaces) / `DATALENGTH`	`length` / `length(CAST(s AS BLOB))`	`LENGTH` / `LENGTHB`
If-then-else shortcut	`CASE`	`IF(c, a, b)`	`IIF(c, a, b)`	`iif(c, a, b)` (3.32)	`DECODE`, `CASE`
Random number 0–1	`random()`	`RAND()`	`RAND()`	`random() / 18446744073709551616.0 + 0.5`	`DBMS_RANDOM.VALUE`
JSON	`json`, `jsonb` (+ GIN)	`JSON` (binary)	`nvarchar` + `JSON_VALUE` etc. (native `json` type 2025)	JSON functions on text (built in since 3.38)	`JSON` type (21c)
Arrays	yes	–	–	–	collection types (`VARRAY`) in PL/SQL

20.17 Verified behaviours

Every concrete behaviour quoted on these sheets, such as `7 / 2 = 3`, `NULL = NULL` being unknown, rank ties, the default window frame, `NOT IN` with a NULL, upsert, and `->>` returning text, is asserted by `code/01_cheatsheet_checks.sql`. The fixtures and a small checker live in `code/00_schema.sql`:

```
CREATE FUNCTION chk(p_area text, p_claim text, p_got text, p_expected text)
RETURNS void LANGUAGE plpgsql AS $$
BEGIN
  IF p_got IS DISTINCT FROM p_expected THEN
    RAISE EXCEPTION 'cheat-sheet claim FAILED [%] %: got %, expected %',
      p_area, p_claim, coalesce(p_got, 'NULL'), coalesce(p_expected, 'NULL');
  END IF;
  INSERT INTO check_log (area, claim, got, expected)
  VALUES (p_area, p_claim, coalesce(p_got, 'NULL'), coalesce(p_expected, 'NULL'));
END $$;
```

`chk` compares with `IS DISTINCT FROM`, so an expected NULL is checked properly too (a plain `<>` would let a NULL result slip through, which is exactly the trap of 20.2). The checks are grouped into `DO` blocks by area. A few examples:

```
DO $$ BEGIN
  PERFORM chk('NULL', '2 NOT IN (1, NULL) is NULL, not TRUE',     (2 NOT IN (1, NULL))::text, NULL);
  PERFORM chk('NULL', 'NOT IN over a list holding NULL keeps 0 rows',
              (SELECT count(*) FROM customers WHERE id NOT IN (SELECT customer_id FROM banned))::text,
'0');
  -- ...
END $$;

DO $$ BEGIN
  PERFORM chk('Windows', 'row_number / rank / dense_rank on 30,20,20,10',
              (SELECT string_agg(format('%s:%s/%s/%s', name, rn, rk, drk), ' ' ORDER BY rn)
                 FROM (SELECT name,
                              row_number() OVER (ORDER BY points DESC, name) AS rn,
                              rank()       OVER (ORDER BY points DESC)       AS rk,
                              dense_rank() OVER (ORDER BY points DESC)       AS drk
                         FROM scores WHERE points IS NOT NULL) t),
              'Al:1/1/1 Bo:2/2/2 Cy:3/2/2 Di:4/4/3');
  PERFORM chk('Windows', 'default frame with ORDER BY is RANGE: peers share the running total',
              (SELECT string_agg(s::text, ',' ORDER BY day, amount)
                 FROM (SELECT day, amount, sum(amount) OVER (ORDER BY day) AS s FROM sales) t),
              '10,40,40,45');
  -- ...
END $$;

-- transactional DDL: a TRUNCATE inside a failed sub-block is rolled back
BEGIN
  TRUNCATE t_users;
  RAISE EXCEPTION 'undo';
EXCEPTION WHEN raise_exception THEN NULL;
END;
PERFORM chk('DDL', 'TRUNCATE is transactional in PostgreSQL (rows came back)',
            (SELECT count(*) FROM t_users)::text, '3');
```

If a claim were wrong, the file would stop with, for example, `ERROR: cheat-sheet claim FAILED [Functions] …: got 2/2, expected 1/2`. (That exact message appeared once while this chapter was being written, when the scratch database was accidentally created with the `SQL_ASCII` encoding. In that encoding `length('é')` counts bytes. The runner always creates UTF-8 databases.) The file ends by listing what was verified.

Output (`bash tools_run_sql.sh sql '20-*'` → `OK`, and the final queries):

```
     area     | claims
--------------+--------
 NULL         |     18
 Functions    |     18
 Aggregation  |     12
 Joins        |      5
 Subqueries   |      3
 Set ops      |      5
 DISTINCT     |      1
 ORDER BY     |      4
 WHERE        |      4
 Windows      |     12
 CTEs         |      3
 Constraints  |      3
 Upsert       |      2
 DML          |      1
 DDL          |      1
 Transactions |      2
 JSONB        |      5
 Arrays       |      3
 FTS          |      2
(19 rows)

 total_claims_verified
-----------------------
                   104
(1 row)
```

The SQLite column of the dialect table is checked the same way by `code/02_sqlite_dialect.sqlite.sql`. SQLite has no `DO` blocks, so each claim is inserted into a table whose `CHECK (ok = 1)` fails the script when a claim is false:

```sql
CREATE TABLE claim (what TEXT, ok INTEGER CHECK (ok = 1));
INSERT INTO claim VALUES
 ('7/2 = 3 (integer division)',              7 / 2 = 3),
 ('a || NULL is NULL',                       ('a' || NULL) IS NULL),
 ('LIKE is case-insensitive for ASCII',      'Anna' LIKE 'a%'),
 ('NULLs sort first in ASC',                 (SELECT name FROM s ORDER BY pts LIMIT 1) = 'c'),
 ('IS DISTINCT FROM exists (3.39+)',         (NULL IS NOT DISTINCT FROM NULL) = 1),
 -- ...
 ('UNIQUE allows several NULLs',             (SELECT count(*) FROM u) = 2);
BEGIN; CREATE TABLE gone (x); ROLLBACK;       -- DDL is transactional in SQLite too
INSERT INTO claim SELECT 'DDL rolls back', (SELECT count(*) FROM sqlite_master WHERE name = 'gone') = 0;
```

Output:

```
28 SQLite claims verified
```

20.18 Interview pattern catalogue

Most SQL interview questions are one of about twenty patterns. Recognise the pattern from the wording, write the template, then adapt it. Method and edge-case checklist: §18.1 and §19.1. Full catalogues: §18.10 and §19.11.

Pattern	Wording that gives it away	Core technique	Solved examples
Plain filter	"find rows where …", "not equal to"	`WHERE` with NULL-aware logic (`IS DISTINCT FROM`)	P1, P2, P4, 19.2.1
String and regex filter	"valid e-mail", "longer than N characters"	`~` with anchors, `char_length`	P6, P7
Row-level classification	"label each row", "is it a triangle"	`CASE`	P8, 19.10.3
Group and count	"per", "for each", "how many"	`GROUP BY` + aggregate, `HAVING`	P10, P12, P14, 19.2.2
Zero counts included	"including those with none"	`LEFT JOIN` + `count(right.col)` or `CROSS JOIN` grid	P20, 19.2.3
Distinct lists per group	"list the products sold each day"	`count(DISTINCT)`, `string_agg(DISTINCT … ORDER BY …)`	P15, P16, 19.10.2
Semi-join	"who has at least one"	`EXISTS`	P22
Anti-join	"never", "without", "no sales"	`NOT EXISTS` (not `NOT IN`)	P23, P24, P25, P26
Conditional aggregation / static pivot	"one column per status"	`count(*) FILTER (WHERE …)`, `SUM(CASE …)`	P27, P28, 19.6.1
Dynamic pivot / unpivot	"one column per store", columns unknown	`format('%I')` + `EXECUTE`, `LATERAL (VALUES …)`, `jsonb_each_text`	19.6.2, 19.6.3
Date ranges	"in February 2020", "last 30 days"	half-open `>= start AND < next_start`	P23, P29, P30, P31
As-of value	"price on that date"	latest row `<=` the date (`DISTINCT ON` or `LIMIT 1`) + default	P33
Ratio / rate	"percentage", "rate", "share"	`100.0 * …`, `NULLIF(denominator, 0)`, round last	P34, P35, P36, P37, 19.9.1, 19.9.2, 19.9.3
Nth highest	"second highest", "NULL if none"	`DISTINCT … OFFSET n-1 LIMIT 1` in a scalar subquery, or `dense_rank`	P39, 19.3.1
Top-N per group	"top 3 in each department"	`dense_rank() OVER (PARTITION BY …)` in a CTE, filter outside	19.3.2
First / last row per group	"first login device", "latest status"	`DISTINCT ON`, `row_number() = 1`	P40, 19.3.3
Ties must be kept	"all winners", "include ties"	`rank() = 1`, `WITH TIES`, `= (SELECT max …)`	P21, 19.3.8, 19.3.9
Running total / cut-off	"so far", "cumulative", "who fits"	`sum() OVER (ORDER BY …)`	P41, 19.4.1, 19.4.2
Compare with previous row	"improved", "change since last"	`lag()`	19.4.3, 19.5.6
Compare with group	"above the department average"	`avg() OVER (PARTITION BY …)`	P42, 19.7.5
Gaps and islands / streaks	"N in a row", "consecutive", "longest streak"	`value − row_number()`, or difference of two row numbers	P32, 19.5.1, 19.5.2, 19.5.3, 19.5.5

Pattern	Wording that gives it away	Core technique	Solved examples
Pairs stored once	friends, calls, matches	`UNION ALL` both directions	19.7.1, 19.7.2, 19.7.4
Relational division	"took every course"	`HAVING count(DISTINCT x) = (SELECT count(*) …)`	19.7.6
Retention / cohorts	"came back the next day"	first date per user, then join on `first + 1`	P38, 19.8.1
Histogram with empty buckets	"include zero"	`generate_series` + `LEFT JOIN`	19.8.2
Largest unique value	"largest number that appears once"	`HAVING count(*) = 1`, then `max` (NULL when none)	P9

20.19 Interview questions and answers

Concept questions (the coding problems are in Chapters 18–19). Each answer is the short version. Follow the link for the long one.

Fundamentals

1. **What is the difference between a database, a DBMS and SQL?** A database is an organised, persistent collection of related data. A DBMS is the software that stores it and controls access (queries, concurrency, recovery, security). SQL is the declarative language used to talk to a relational DBMS. → §1.2, §1.3, §1.9
2. **What are DDL, DML, DQL, DCL and TCL?** Define structure (`CREATE`, `ALTER`, `DROP`), change data (`INSERT`, `UPDATE`, `DELETE`), query (`SELECT`), grant rights (`GRANT`, `REVOKE`) and control transactions (`BEGIN`, `COMMIT`, `ROLLBACK`). → §1.9
3. **Relational vs NoSQL — how do you choose?** By access pattern and consistency need. Relational for structured data with joins, constraints and transactions. NoSQL families when a specific shape (key lookups, nested documents, huge write rates, graphs) dominates and you accept weaker guarantees. → §1.8, §16.10
4. **In what order is a `SELECT` evaluated?** FROM/JOIN → WHERE → GROUP BY → aggregates → HAVING → SELECT (windows) → DISTINCT → set operations → ORDER BY → LIMIT. That is why `WHERE` cannot see aliases and `ORDER BY` can. → §4.5
5. **What does NULL mean, and why does `x = NULL` not work?** NULL marks an unknown or missing value. Any comparison with it is UNKNOWN, and `WHERE` keeps only TRUE, so use `IS NULL` or `IS DISTINCT FROM`. → §2.5
6. **Why can `NOT IN` return no rows?** `x NOT IN (a, NULL)` means `x <> a AND x <> NULL`. The second part is UNKNOWN, so the whole condition is never TRUE. Use `NOT EXISTS`. → §6.2
7. **`WHERE` vs `HAVING`?** `WHERE` filters rows before grouping and cannot use aggregates. `HAVING` filters groups after aggregation. → §4.3
8. **`count(*)` vs `count(col)` vs `count(DISTINCT col)`?** All rows / non-NULL values / distinct non-NULL values. → §4.1
9. **`UNION` vs `UNION ALL`?** `UNION` removes duplicates (extra sort or hash). `UNION ALL` keeps everything and is cheaper. → §6.8
10. **`DELETE` vs `TRUNCATE` vs `DROP`?** Chosen rows (triggers fire, each row logged) / all rows quickly / the table itself. In PostgreSQL all three can be rolled back. → §9.6

Joins, subqueries, CTEs, windows

11. **Explain the join types.** Inner = matching pairs. Left/right = all of one side plus matches. Full = all of both. Cross = every pair. Self = a table joined to itself. → §5.2

12. **Why did my `LEFT JOIN` behave like an inner join?** A condition on the right table in `WHERE` removes the NULL-padded rows. Move it into `ON`. → §5.9
13. **What is a semi-join and an anti-join?** Semi: left rows that have a match, each returned once (`EXISTS`). Anti: left rows with no match (`NOT EXISTS`). → §5.8
14. **What is a correlated subquery?** A subquery that refers to the outer row, so conceptually it runs once per outer row. The planner can often turn it into a join. → §6.3
15. **`EXISTS` vs `IN`?** Both express a semi-join and PostgreSQL usually plans them the same way. The difference is `NOT IN` vs `NOT EXISTS` when NULLs are possible. → §6.4
16. **CTE vs subquery vs view vs materialized view?** A CTE names a step inside one statement. A subquery is inline. A view is a stored query, always current. A materialized view stores the result and must be refreshed. → §7.9
17. **How does a recursive CTE work?** The anchor produces the first rows. The recursive term runs again and again on the rows of the previous round until a round returns nothing. → §7.3
18. **Window function vs `GROUP BY`?** `GROUP BY` collapses each group into one row. A window function computes over related rows but keeps every row. → §8.1
19. **`ROW_NUMBER` vs `RANK` vs `DENSE_RANK`?** 1,2,3,4 vs 1,2,2,4 vs 1,2,2,3 on a tie for second place. → §8.5
20. **Why does my running total repeat on tied dates?** With `ORDER BY` and no frame clause the default frame is `RANGE … CURRENT ROW`, which includes all peers. Add a unique tie-breaker or use `ROWS`. → §8.3
21. **Why is `LAST_VALUE` returning the current row?** The default frame ends at the current row. Extend it to `UNBOUNDED FOLLOWING`. → §8.6
22. **How do you find the Nth highest salary?** `SELECT DISTINCT salary … ORDER BY salary DESC OFFSET n-1 LIMIT 1`, wrapped in a scalar subquery to get NULL when there is none, or `dense_rank() = n`. → P39
23. **What is the gaps-and-islands trick?** In a consecutive run, `value − row_number()` stays constant, so grouping by it groups each run. → §8.9

Design

24. **Primary key vs unique key vs foreign key?** PK: the chosen identifier, unique and not null, one per table. Unique: another identifier (alternate key), NULLs usually allowed. FK: a column that must match a key in another table. → §9.3
25. **Superkey vs candidate key?** A superkey identifies rows. A candidate key is a superkey with nothing extra: drop any of its columns and it no longer identifies rows. → §10.13
26. **Natural vs surrogate key?** Natural keys come from the business data and can change. Surrogate keys are generated and meaningless. A common default is a surrogate PK plus `UNIQUE` on the natural key. → §10.13
27. **Explain 1NF, 2NF, 3NF and BCNF.** Atomic values / no partial dependency on a composite key / no transitive dependency / every determinant is a superkey. → §11.7–§11.11
28. **What anomalies does normalization prevent?** Insertion, update and deletion anomalies, all caused by storing one fact in more than one place. → §11.6
29. **When would you denormalize?** For a measured read bottleneck, with a way to keep the copies correct (same transaction, trigger, or materialized view) and a reconciliation check. → §11.16
30. **How do you model a many-to-many relationship?** A junction table with two FKs and a composite PK (or a surrogate PK + `UNIQUE` on the pair). Relationship attributes go there too. → §10.15
31. **What is a weak entity?** An entity that cannot be identified without its owner, such as an order line within an order. Its key is the owner's key + a partial key. → §10.9

Performance

32. **How does a B-tree index speed up a query?** It keeps keys sorted in a shallow balanced tree (3–4 levels for millions of rows), so finding a key or the start of a range takes a few page reads instead of a full scan. → §12.6
33. **Clustered vs non-clustered index?** A clustered index stores the table rows in index order (InnoDB's primary key, SQL Server's clustered index). A non-clustered index is separate and points to the rows. PostgreSQL tables are heaps. `CLUSTER` reorders once but does not maintain the order. → §12.4

34. **How do you choose column order in a composite index?** Leftmost prefix rule: equality columns first, then the range or sort column. → §12.11
35. **What is a covering index?** One that contains every column the query needs, allowing an index-only scan. → §12.12
36. **Why is my index not used?** A function or cast on the column, a leading wildcard, low selectivity, stale statistics, a non-leading composite column, or a table so small that a seq scan is cheaper. → §12.13
37. **How do you read `EXPLAIN ANALYZE`?** Innermost node first. Compare estimated with actual rows, look at loops, the biggest actual times, sorts spilling to disk, and seq scans on big tables with selective filters. → §12.10, §12.17
38. **Why is `OFFSET` paging slow, and what is the fix?** The database still reads and discards all the skipped rows. Keyset pagination (`WHERE (ts, id) < (last_ts, last_id)`) jumps straight there through an index. → §12.14
39. **Nested loop vs hash join vs merge join?** Small outer input with an indexed inner / large equality joins / inputs already sorted. → §12.15
40. **What is the N+1 problem?** One query for a list, then one more query per item. Fix it with a join, `= ANY($1)` with a batch of ids, or ORM eager loading. → §12.13, §14.13

Transactions and concurrency

41. **Explain ACID.** Atomicity (all or nothing), consistency (rules hold), isolation (concurrent transactions do not interfere), durability (committed data survives crashes). → §13.1
42. **Name the isolation levels and anomalies.** Read Uncommitted, Read Committed, Repeatable Read, Serializable. Anomalies: dirty read, non-repeatable read, phantom, lost update, write skew. → §13.4
43. **What does PostgreSQL's Repeatable Read prevent that the standard does not require?** Phantoms (it is snapshot isolation), and it aborts lost updates with a serialization error. It still allows write skew. → §13.4
44. **How do you prevent lost updates?** An atomic `UPDATE … SET x = x - 1`, `SELECT … FOR UPDATE`, Repeatable Read with retry, or optimistic locking with a version column. → §13.7
45. **What is write skew?** Two transactions read the same data, then update different rows, together breaking a rule both checked. Serializable (SSI) or explicit locks prevent it. → §13.8
46. **What is MVCC?** Writers create new row versions, and each transaction reads the versions visible in its snapshot, so readers do not block writers. Old versions are removed by VACUUM. → §13.14
47. **Optimistic vs pessimistic locking?** Pessimistic locks before touching data (waits, deadlocks possible). Optimistic works freely and checks for conflicts at commit (aborts and retries). Low contention favours optimistic. → §13.9
48. **How are deadlocks handled?** The database detects the wait cycle and aborts one transaction (40P01). Prevent them with a consistent lock order and short transactions, and retry in code. → §13.11
49. **How do you build a job queue in SQL?** `SELECT … FOR UPDATE SKIP LOCKED LIMIT 1` inside a transaction, so each worker takes a different unlocked row. → §13.12

Security, scaling, beyond

50. **How do you prevent SQL injection?** Parameterized queries (bind variables). Never concatenate input. Allow-list identifiers. Least-privilege roles. → §14.9
51. **Partitioning vs sharding vs replication?** Partitioning splits a table inside one server. Sharding puts partitions on different servers. Replication keeps copies of the same data for availability and read scaling. → §15.2, §15.3, §15.5
52. **Explain the CAP theorem.** During a network partition a distributed store must choose between consistency (refuse some requests) and availability (answer, possibly stale). PACELC adds that without a partition you trade latency against consistency. → §15.7
53. **Synchronous vs asynchronous replication?** Sync waits for the replica before commit (no loss, higher latency). Async commits first (fast, may lose recent writes on failover). → §15.5
54. **What is the difference between a backup and a replica?** A replica copies every change, including mistakes, within seconds. A backup is a point-in-time copy you can restore from. PITR combines a

base backup with archived WAL. → §15.11, §9.12

20.20 Glossary (A–Z)

Short definitions of the terms used across the book, in plain words. The last column links to the section that explains each term with examples. The terms from the course's key-term review cards (the problems that made databases necessary, ER notation, keys, functional dependencies, normal forms, modeling levels, and the case-study judgement calls) are included, with the judgement calls summarised as a table in 20.10.

595 terms. Jump to: 0–9 · A · B · C · D · E · F · G · H · I · J · K · L · M · N · O · P · Q · R · S · T · U · V · W · X · Y · Z

0–9

Term	Meaning	See
1NF (first normal form)	Every cell holds a single atomic value and there are no repeating groups of columns.	§11.7
2NF (second normal form)	1NF, and no non-prime attribute depends on only part of a composite candidate key.	§11.8
2PC (two-phase commit)	Distributed commit protocol: the coordinator asks every participant to prepare, then tells all to commit or all to abort. Blocks if the coordinator fails after prepare.	§15.9
2PL (two-phase locking)	A transaction acquires locks in a growing phase and releases them in a shrinking phase, never acquiring after releasing. Guarantees serializable schedules.	§13.10
3-2-1 backup rule	Keep three copies of the data on two different kinds of storage, with one copy off-site.	§9.12
3NF (third normal form)	2NF, and for every FD X → A either X is a superkey or A is a prime attribute. Removes transitive dependencies.	§11.9
4NF (fourth normal form)	BCNF, and every non-trivial multivalued dependency has a superkey on its left side.	§11.12
5NF (fifth normal form)	Every join dependency is implied by the candidate keys. Also called project-join normal form.	§11.13

A

Term	Meaning	See
ACID	Atomicity, consistency, isolation, durability: the four guarantees of a database transaction.	§13.1
Adjacency list	Storing a tree or graph as rows that each point to their parent (a `parent_id` column).	§7.4
Advisory lock	A PostgreSQL lock on a number the application chooses (`pg_advisory_lock`), not tied to any row or table.	§13.10
Aggregate function	A function that turns many input rows into one value, such as `count`, `sum`, `avg`, `min`, `max`.	§4.1
Aggregate push-down	Computing partial aggregates close to the data (on each shard, or before a join) so that fewer rows travel.	§15.4
Alias	A temporary name for a column (`AS total`) or a table (`FROM orders o`) within one query.	§2.1
Alternate key	Any candidate key other than the one picked as primary key. Enforced with `UNIQUE`.	§10.13
ALTER TABLE	DDL statement that changes an existing table: add, drop, rename or retype columns, add constraints.	§9.5
Analytic function	Another name for a window function (common in Oracle documentation).	§8.1
ANALYZE	Command that samples a table and refreshes the planner's statistics.	§12.16
Anchor (recursive CTE)	The non-recursive first part of a recursive CTE, which produces the starting rows.	§7.3
Anomaly (data)	An insertion, update or deletion problem caused by storing the same fact in more than one row.	§11.6
Anomaly (isolation)	A result that could not happen if transactions ran one at a time, such as a dirty read or write skew.	§13.4
ANSI / ISO SQL standard	The official SQL specification that every vendor implements, each with its own extensions and gaps.	§1.9
Anti-join	Returns rows of one table that have no match in another. Written `NOT EXISTS` or `LEFT JOIN … IS NULL`.	§5.8
ANY / SOME / ALL	Compare a value with a set: true if the comparison holds for at least one element (`ANY`, `SOME`) or for every element (`ALL`).	§6.5
AOF (append-only file)	Redis persistence mode that logs every write command so the data can be rebuilt after a restart.	§16.11
API	Application programming interface: the set of calls one program offers to others.	§1.8
Append-only table	A table that only ever gets new rows, never updates in place, such as an event log or a ledger.	§17.14
ARIES	A classic crash-recovery algorithm: analyse the log, redo all logged history, then undo transactions that never committed.	§15.9
Armstrong's axioms	Reflexivity, augmentation and transitivity: sound and complete rules for deriving functional dependencies.	§11.3
Array (PostgreSQL)	A column type holding an ordered list of values of one type, indexed from 1.	§14.5
Asynchronous replication	The primary commits without waiting for replicas. Fast, but a failover can lose the newest writes.	§15.5

Term	Meaning	See
Atomicity	A transaction's changes happen completely or not at all.	§13.2
Attribute	A named property of an entity or a column of a relation.	§10.6
Attribute closure (X^+)	Every attribute functionally determined by the set X under a given set of FDs.	§11.4
Audit columns	Columns such as `created_at`, `updated_at` and `created_by` that record when and by whom a row was changed.	§17.14
Augmentation (FD rule)	Armstrong's rule: if X → Y, then XZ → YZ for any attributes Z.	§11.3
Autocommit	Mode in which every statement is its own transaction and commits immediately.	§13.1
Autovacuum	PostgreSQL's background worker that runs VACUUM and ANALYZE automatically as tables change.	§12.16
Availability	The share of time a system answers requests. In CAP: every request to a working node gets an answer, possibly stale.	§15.1

B

Term	Meaning	See
Backfill	Filling in values for existing rows after adding a new column or rule.	§9.5
BASE	Basically available, soft state, eventually consistent: the looser guarantee model of many NoSQL systems.	§16.1
Base backup	A physical copy of a PostgreSQL server's whole data directory: the starting point for PITR or a new replica.	§9.12
BCNF (Boyce–Codd normal form)	For every non-trivial FD X → Y, X is a superkey.	§11.11
BEGIN / COMMIT / ROLLBACK	Start a transaction, make its changes permanent, or discard them.	§13.1
BETWEEN	Inclusive range test: `x BETWEEN a AND b` means `x >= a AND x <= b`.	§2.4
Big data	Data too large, too fast or too varied for one ordinary database server (volume, velocity, variety).	§16.9
BigQuery / Snowflake / Redshift	Managed cloud columnar data warehouses for analytical SQL over very large tables.	§16.8
Binary search	Finding a value in sorted data by repeatedly halving the search range: about $\log_2 n$ steps.	§12.5
Bind parameter (placeholder)	A marker such as `$1`, `%s` or `?` in the SQL text whose value the driver sends separately.	§14.9
BitmapAnd / BitmapOr	Plan nodes that combine the row bitmaps of several indexes with AND or OR before reading the table.	§12.10
Bitmap scan	Plan that collects matching row locations from one or more indexes into a bitmap, then reads the table pages in physical order.	§12.10
Bloat	Space wasted in a table or index by dead row versions and half-empty pages.	§13.14
Block	Another word for page: the fixed-size unit in which the disk and the DBMS read and write data.	§12.1
Blocking factor	How many records fit in one disk block (block size ÷ record size).	§12.5
Bloom filter	Compact probabilistic set that answers "definitely not present" or "probably present".	§12.8
BM25	The standard relevance score used by search engines such as Elasticsearch: rewards rare query words that appear often in a short document.	§16.11
BRIN index	Block range index: stores min/max per range of table pages. Tiny and effective when values follow physical order, such as time.	§12.9
BSON	MongoDB's binary form of JSON, with extra types such as dates and 64-bit integers.	§16.3
B-tree / B+ tree	Balanced, shallow, sorted tree index. In a B+ tree all keys live in linked leaf pages, which makes range scans cheap. PostgreSQL's default index.	§12.6
Bucket (hashing)	A slot, usually a page, that a hash function sends keys to.	§12.3
Buffer pool (shared buffers)	Memory area where the DBMS caches data pages read from disk.	§1.4
Bully algorithm	A leader-election method in which the live node with the highest id takes over.	§15.8
Byzantine fault	A failure in which a node keeps running but sends wrong or malicious messages, rather than simply stopping.	§15.8

C

Term	Meaning	See
Cache	A fast store, usually memory, that holds copies of often-read data so the slower source is asked less often.	§1.4
Cache invalidation	Removing or refreshing cached copies when the underlying data changes.	§7.8
Calendar table	A table (or `generate_series`) with one row per day, joined to data so that days with no activity still appear.	§7.6
Candidate key	A minimal set of attributes that uniquely identifies every row.	§10.13
Canonical pair	Storing an undirected pair (two friends) once in a fixed order, such as `user_a < user_b`.	§17.13
CAP theorem	If the network splits, a replicated store can stay fully consistent or stay fully available, but not both.	§15.7
Cardinality (ER)	The maximum number of instances on each side of a relationship: 1:1, 1:N or M:N.	§10.7
Cardinality estimate	The planner's guess of how many rows a plan node will produce.	§12.16
Cardinality (relation)	The number of rows in a relation.	§10.12
Cartesian product	Every row of one table paired with every row of another.	§5.1
CASCADE	Referential action: deleting or updating a parent row does the same to its child rows.	§9.3
CASE	Conditional expression returning the first `THEN` value whose `WHEN` is true, else the `ELSE` value (or NULL).	§3.5
Cassandra / ScyllaDB	Leaderless wide-column databases built for very high write rates spread over many nodes.	§16.4
Cast	Converting a value to another type: `CAST(x AS int)` or, in PostgreSQL, `x::int`.	§3.1
Catalog (data dictionary)	System tables describing all database objects: tables, columns, types, indexes, privileges.	§1.2
Causal consistency	Reads respect cause and effect: nobody sees a reply before the message it answers.	§15.6
CDC (change data capture)	Streaming committed row changes out of a database, usually by reading its log, into other systems.	§16.10
CHECK constraint	A row-level rule that rejects any row for which the expression is FALSE.	§9.3
CHECK OPTION (views)	`WITH CHECK OPTION` makes an updatable view reject inserts and updates whose rows the view would not show.	§7.7
Checkpoint	Point at which all dirty pages are flushed to disk, bounding how much log must be replayed after a crash.	§15.9
Chen notation	ER notation that draws entities as rectangles, attributes as ellipses and relationships as diamonds.	§10.5
Citus	A PostgreSQL extension that shards tables across several PostgreSQL servers and runs queries on them in parallel.	§15.3
ClickHouse	An open-source columnar database for fast analytical queries.	§12.1
Click-through rate (CTR)	Clicks divided by impressions (times the ad was shown), usually as a percentage.	§18.8

Term	Meaning	See
Closure table	A tree-storage pattern with one row for every ancestor–descendant pair, so subtree queries need no recursion.	§7.4
Clustered index	An index whose order is the physical order of the table rows (InnoDB primary key, SQL Server clustered index).	§12.4
Clustering column	In a wide-column store, the column that sorts the rows inside one partition.	§16.4
Clustering index	An index on a non-key field by which the data file is physically sorted.	§12.4
COALESCE	Returns the first non-NULL argument.	§3.5
CockroachDB	A distributed SQL database that speaks the PostgreSQL protocol and replicates data ranges with Raft.	§16.7
Cohort	A group of users who started in the same period, followed over time to measure retention.	§19.8
Collation	Rules that decide how text is compared and sorted, including case sensitivity.	§2.7
Co-location	Keeping related rows (one customer's orders and items) on the same shard so joins stay on one node.	§15.3
Column family	A group of columns stored together in wide-column stores such as HBase and Cassandra.	§16.4
Column store (columnar)	Storage that keeps each column's values together, making scans and compression of a few columns efficient.	§12.2
Compaction	Background merging of sorted files in an LSM-tree store, which throws away overwritten and deleted values.	§16.4
Compare-and-set (CAS)	An atomic "change it only if it still has the value I read" operation.	§13.1
Composite attribute	An attribute made of parts, such as an address made of street, city and postcode.	§10.6
Composite index	An index on several columns. Usable for filters on a leftmost prefix of its columns.	§12.11
Composite key	A key made of two or more columns.	§10.13
Compression	Storing data in fewer bytes. Column stores compress well because one column's values are alike.	§12.2
Conceptual model	Business-level model of entities and relationships, without types or DBMS details.	§10.2
Concurrency control	The mechanisms (locking, timestamps, MVCC, validation) that keep concurrent transactions correct.	§13.9
Conditional aggregation	Aggregating only the rows that meet a condition, with `FILTER (WHERE …)` or `sum(CASE …)`.	§4.4
Connection pool	A set of reusable open database connections shared by application threads.	§14.12
Consensus	Agreement of a group of nodes on one value or log order despite failures, as in Raft and Paxos.	§15.8
Conservative 2PL	A two-phase-locking variant that takes every lock a transaction needs before it starts, so it cannot deadlock.	§13.11
Consistency (ACID)	Every committed transaction leaves all constraints and business rules satisfied.	§13.3
Consistency (CAP)	In CAP: every read sees the most recent write, as if there were a single copy. Not the same as the C in ACID.	§15.7

Term	Meaning	See
Consistency model	The guarantee a distributed store gives about what reads can see: strong, causal, eventual and others.	§15.6
Consistent hashing	Placing nodes and keys on a hash ring so that adding a node moves only a small share of keys.	§15.3
Constraint	A rule enforced by the DBMS on the data: NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, CHECK, EXCLUDE.	§9.3
COPY	PostgreSQL's bulk command for loading or exporting many rows at once, far faster than row-by-row `INSERT`s.	§14.10
Correlated subquery	A subquery that refers to a column of the outer query and is logically evaluated per outer row.	§6.3
Cost (planner)	The planner's estimate of the work of a plan, in arbitrary units based on page reads and CPU.	§12.10
Covering index	An index containing every column a query needs, allowing an index-only scan.	§12.12
CPU	The processor that executes instructions. Sorting, hashing and expression evaluation cost CPU time.	§12.1
CRDT	Conflict-free replicated data type: a structure (counter, set) whose replicas can always be merged without conflicts.	§15.5
CROSS JOIN	Join that returns the Cartesian product.	§5.1
Crosstab	A function from PostgreSQL's `tablefunc` extension that pivots rows into columns.	§14.7
Crow's foot notation	ER notation that draws cardinality and optionality with symbols at the ends of relationship lines.	§10.5
CRUD	Create, read, update, delete: the four basic operations on stored data.	§14.13
CSV	Comma-separated values: a plain-text table format with one row per line.	§1.1
CTE (common table expression)	A named subquery defined with `WITH` and used by the statement that follows.	§7.1
CUBE	Grouping extension that produces subtotals for every combination of the listed columns.	§4.6
Cursor	A pointer for fetching a query result a batch of rows at a time.	§14.10
Cypher	Neo4j's graph query language, which writes patterns like `(a)-[:FOLLOWS]->(b)`.	§16.5

D

Term	Meaning	See
Database	An organised, persistent, self-describing collection of related data.	§1.2
Data independence	Changing how data is stored (physical) or parts of the schema (logical) without changing applications.	§1.3
Data integrity	The data is accurate and obeys its rules (types, keys, constraints).	§1.3
Data model	A description of the data's entities, attributes, relationships and rules.	§10.1
Data-modifying CTE	A `WITH` step containing `INSERT`, `UPDATE` or `DELETE … RETURNING` (PostgreSQL).	§7.2
Data type	The kind of value a column holds (integer, text, date …), which decides what operations are allowed.	§3.1
Data warehouse	A database built for analysis, loaded from operational systems and tuned for large scans and aggregates.	§11.16
DBMS	Software that defines, stores, queries, protects and shares a database.	§1.3
DCL	Data control language: `GRANT` and `REVOKE`.	§1.9
DDL	Data definition language: `CREATE`, `ALTER`, `DROP`, `TRUNCATE`.	§1.9
Deadlock	Two or more transactions each waiting for a lock held by another, so none can proceed.	§13.11
Dead tuple	An old row version that no transaction can see any more, waiting to be removed by VACUUM.	§13.14
Declarative	Saying what result you want and letting the system decide how to compute it. SQL is declarative.	§1.4
Decomposition	Splitting a relation into smaller ones during normalization.	§11.10
Deduplication	Keeping one row per group of duplicates, often with `row_number() = 1` or `DISTINCT ON`.	§8.8
Deferrable constraint	A constraint whose check can wait until `COMMIT` (`DEFERRABLE INITIALLY DEFERRED`).	§9.3
Degree (ER)	The number of entity types taking part in a relationship: unary, binary, ternary.	§10.7
Degree (relation)	The number of attributes (columns) of a relation.	§10.12
DELETE	DML statement that removes the rows matching its `WHERE` clause.	§9.10
Delta encoding	Storing each value as the difference from the previous one, giving small numbers for sorted ids or timestamps.	§12.2
Denormalization	Deliberately storing redundant data to speed up reads, with a mechanism to keep it correct.	§11.16
Dense index	An index with one entry for every record or key value.	§12.4
DENSE_RANK	Ranking function that gives ties the same rank and leaves no gaps (1, 2, 2, 3).	§8.5
Dependency preservation	Every functional dependency can be checked within a single table after decomposition.	§11.10
Derived attribute	An attribute that can be computed from others, such as age from birth date.	§10.6
Derived table	A subquery in the `FROM` clause.	§6.6

Term	Meaning	See
Determinant	The left side of a functional dependency.	§11.1
Dialect	One vendor's version of SQL, with its own syntax and functions.	§1.9
Dictionary encoding	Replacing repeated values with small integer codes plus one lookup table of the distinct values.	§12.2
Differential / incremental backup	A differential backup copies everything changed since the last full backup. An incremental copies changes since the last backup of any kind.	§9.12
Dirty page	A page changed in memory but not yet written to disk.	§15.9
Dirty read	Reading another transaction's uncommitted change.	§13.4
Dirty write	Overwriting another transaction's uncommitted change.	§13.4
Disaster recovery	The plans and off-site copies that let you restore service after losing a whole server, site or data set.	§15.11
DISTINCT	Removes duplicate rows from a result. `DISTINCT ON (cols)` keeps the first row per group (PostgreSQL).	§2.6
Distributed SQL	Another name for NewSQL: SQL and ACID transactions over data sharded and replicated across many nodes.	§16.7
Distributed transaction	A transaction that changes data on more than one node, so it needs a protocol such as two-phase commit.	§15.9
DML	Data manipulation language: `INSERT`, `UPDATE`, `DELETE`, `MERGE`.	§1.9
Document database	NoSQL store that keeps each record as a nested document, usually JSON.	§16.3
Dollar quoting	PostgreSQL's `$$ … $$` string syntax, used for function bodies so inner quotes need no escaping.	§14.1
Domain	The set of allowed values for an attribute.	§10.12
Double-entry bookkeeping	Recording each money movement as a debit and an equal credit, so all entries always sum to zero.	§17.3
Downsampling	Replacing old detailed measurements with coarser summaries, such as hourly averages.	§16.8
DQL	Data query language: `SELECT`.	§1.9
Driver	The library a program uses to talk to a particular database, such as psycopg or a JDBC driver.	§14.10
DROP	DDL statement that removes a database object entirely.	§9.6
Dual writes	An application writing the same change separately to two systems, which drift apart if one write fails.	§16.11
Durability	Once committed, a transaction's changes survive crashes.	§13.5
Dynamic SQL	SQL text built at run time and executed with `EXECUTE`. Identifiers must be quoted safely with `format('%I', …)`.	§19.6
DynamoDB	Amazon's managed key-value and document database.	§16.2

E

Term	Meaning	See
Elasticsearch / OpenSearch	Distributed search engines built on Lucene that index JSON documents for full-text search and log analytics.	§16.10
ELK stack	Elasticsearch, Logstash and Kibana: a common set of tools to collect, search and chart logs.	§16.11
Embedded database	A database library that runs inside the application process, with no separate server. SQLite is the usual example.	§1.7
Entity	A distinguishable thing the database stores facts about, such as a customer or an order.	§10.6
Entity integrity	No primary-key value may be NULL.	§10.14
Entity set / entity type	The entity type is the kind of thing (Employee). The entity set is all its instances stored right now.	§10.6
Enum type	A type whose values must come from a fixed list, such as `'new', 'paid', 'shipped'`.	§3.1
Equi-join	A join whose condition is equality between columns.	§5.3
Erasure coding	Storing data plus computed parity pieces so lost pieces can be rebuilt, using less space than full copies.	§15.11
ER diagram	A diagram of entities, attributes and relationships.	§10.5
ETL	Extract, transform, load: copying data from source systems, reshaping it, and loading it into a warehouse.	§11.16
Eventual consistency	If writes stop, all replicas eventually return the same value.	§15.6
Eviction policy	The rule a cache uses to decide what to drop when it is full, such as LRU or LFU.	§16.2
Exactly-once	A processing guarantee that each message affects the result exactly one time, even when it is retried.	§16.9
EXCEPT	Set operation returning rows of the first query that do not appear in the second.	§6.10
EXCLUDE constraint	PostgreSQL constraint that forbids two rows whose values conflict under given operators, such as overlapping ranges.	§17.2
EXCLUDED (ON CONFLICT)	In `INSERT … ON CONFLICT DO UPDATE`, the pseudo-row holding the values that were proposed for insertion.	§9.11
Exclusive lock (X)	A lock that only one transaction can hold. It blocks every other lock on the same item.	§13.10
Executor	The DBMS component that runs the chosen plan and produces the result rows.	§1.4
Existence dependency	An entity that cannot exist without another one, such as an order line without its order.	§10.9
EXISTS	True if the subquery returns at least one row.	§6.4
EXPLAIN / EXPLAIN ANALYZE	Show the chosen query plan, and with `ANALYZE` run it and report actual rows and times.	§12.10
Expression index	An index on the result of an expression, such as `lower(email)`.	§12.12
Extended statistics	PostgreSQL statistics on combinations of columns (`CREATE STATISTICS`), which help estimates for correlated columns.	§12.16

Term	Meaning	See
Extendible hashing	A dynamic hashing scheme that grows by doubling a directory of pointers and splitting only the bucket that overflowed.	§12.7
Extension (PostgreSQL)	An add-on package installed with `CREATE EXTENSION`, such as PostGIS or pg_trgm.	§15.2

F

Term	Meaning	See
Failover	Promoting a replica to primary when the primary fails.	§15.10
False positive	A "yes" that is actually wrong. A Bloom filter can give false positives but never false negatives.	§12.8
Fan-out	Row multiplication when a row joins to several matching rows, often inflating sums.	§5.9
Fan-out on write / on read	Two feed designs: push each new post into every follower's feed when it is written, or build the feed when it is read.	§17.4
Fault tolerance	Continuing to work correctly when some parts fail.	§15.1
Fencing	Preventing a deposed leader from continuing to write after failover.	§15.8
FETCH FIRST	Standard syntax for limiting rows: `FETCH FIRST n ROWS ONLY` or `WITH TIES`.	§2.8
File organization	How records are arranged in a file: heap, sorted, hashed or clustered.	§12.3
Fillfactor	How full PostgreSQL packs table or index pages, leaving the rest free for later updates.	§12.3
FILTER clause	Restricts the rows fed to one aggregate: `count(*) FILTER (WHERE …)`.	§4.4
Floating-point number	An approximate binary number (`real`, `double precision`) that cannot hold most decimal fractions exactly. Avoid it for money.	§3.1
Foreign data wrapper	A PostgreSQL extension that lets you query another database or a file as if it were a local table.	§15.4
Foreign key	Columns whose values must match a key in a referenced table (or be NULL).	§9.3
Frame (window frame)	The subset of the partition that an aggregate or value window function sees for the current row.	§8.4
fsync	The operating-system call that forces written data out of caches onto durable storage.	§15.9
FULL OUTER JOIN	Returns all rows from both sides, padding with NULLs where there is no match.	§5.6
Full-text search	Searching documents by words with stemming and ranking. `tsvector @@ tsquery` in PostgreSQL.	§14.6
Functional dependency	`X → Y`: any two rows with equal X values have equal Y values.	§11.1
Fuzzy checkpoint	A checkpoint that writes dirty pages out gradually while transactions keep running, instead of pausing all work.	§15.9

G

Term	Meaning	See
Gaps and islands	Problems about runs of consecutive values or dates, solved by grouping on `value − row_number()`.	§8.9
Generated column	A column computed from other columns of the same row (`GENERATED ALWAYS AS (expr) STORED`).	§9.4
generate_series	PostgreSQL function that returns a series of numbers or timestamps as rows.	§7.6
Geohash	A short string that encodes a map cell from latitude and longitude. Nearby places share a prefix.	§16.8
GIN index	Generalized inverted index: maps each element (word, key, array item) to the rows that contain it.	§12.9
GiST index	Generalized search tree for overlap and nearest-neighbour queries on ranges, geometry and more.	§12.9
Gossip protocol	Nodes spread cluster state by repeatedly swapping what they know with random peers.	§16.11
GQL	The ISO-standard graph query language (2024), close to Cypher.	§16.5
Graph database	NoSQL store of nodes and edges optimised for traversing relationships.	§16.5
Gremlin	A step-by-step graph traversal language used by Apache TinkerPop databases such as Amazon Neptune.	§16.5
GROUP BY	Groups rows that share values so aggregates are computed per group.	§4.2
GROUPING()	A function that returns 1 on a subtotal row where the column was rolled up, and 0 otherwise.	§4.6
GROUPING SETS	Computes several groupings in one query, such as per region, per product and a grand total.	§4.6
GROUPS frame	A window frame measured in peer groups rather than rows or values.	§8.4

H

Term	Meaning	See
Half-open interval	A range that includes its start but not its end (`>= start AND < end`), so neighbouring ranges never overlap.	§2.4
HashAggregate / GroupAggregate	Plan nodes that group rows, either with an in-memory hash table or by reading input already sorted on the group key.	§12.15
Hash function	A function that turns a key into a fixed-size number. The same key always gives the same number.	§12.3
Hash index	Index that hashes keys into buckets. Supports equality lookups only.	§12.7
Hash join	Join that builds a hash table on one input and probes it with the other. Equality only.	§12.15
Hash partitioning	Assigning rows to partitions by a hash of the key, spreading them evenly.	§15.2
Hash table	A structure that stores items in buckets chosen by a hash of the key, giving lookups in about constant time.	§12.7
HAVING	Filters groups after aggregation.	§4.3
HBase	A wide-column store on top of Hadoop's HDFS, modelled on Google's Bigtable.	§16.1
HDFS	Hadoop Distributed File System: stores huge files as large blocks copied to several machines.	§16.4
Heap	Unordered table storage: new rows go wherever there is space.	§12.1
Heartbeat	A periodic "I am alive" message. Missing heartbeats suggest that a node has failed.	§15.8
High availability	Keeping a service running through failures, usually with replicas and automatic failover.	§15.10
Hinted handoff	A node temporarily keeps writes meant for an unreachable replica and hands them over when it comes back.	§15.5
Histogram (statistics)	Buckets of a column's value distribution that the planner uses to estimate how many rows a condition keeps.	§12.16
Horizontal scaling	Adding more machines (scale out) rather than a bigger one.	§15.1
Hot partition (hotspot)	A partition or shard that receives far more traffic than the others and becomes the bottleneck.	§15.2
HTAP	Hybrid transactional/analytical processing: one system that serves OLTP and analytics together.	§16.6
Hybrid logical clock	A timestamp made of physical time plus a logical counter, used to order events across machines.	§16.7

I

Term	Meaning	See
IaaS / PaaS / SaaS	Cloud service models: rent machines (infrastructure), a managed platform, or finished software.	§1.5
Idempotent	Having the same effect whether it runs once or several times, which makes retries safe.	§17.3
Identifier	The name of a table, column or other object. Double quotes make it case-sensitive (`"Name"`).	§2.1
Identifying relationship	The relationship that connects a weak entity to its owner and supplies part of its key.	§10.9
Identity column	A column whose values are generated automatically: `GENERATED ALWAYS AS IDENTITY`.	§9.4
Impedance mismatch	The friction between objects in code (nested, linked by references) and flat relational tables.	§14.13
IN	True if a value equals any element of a list or subquery result.	§6.2
Index	An auxiliary structure that finds rows by key without scanning the whole table.	§12.4
Index-only scan	Answering a query from the index alone, without visiting table pages.	§12.10
Index scan	Walking an index to find matching entries, then fetching each row from the table.	§12.10
InfluxDB	A time-series database for metrics and sensor data.	§16.8
information_schema	The SQL-standard set of views that describe the database's tables, columns and constraints.	§1.2
Inlining (CTE)	The planner folding a CTE into the main query so it can be optimised together with it (PostgreSQL 12+).	§7.1
In-memory database	A database that keeps its working data in RAM and persists it with snapshots and logs.	§16.6
INNER JOIN	Returns only pairs of rows that satisfy the join condition.	§5.3
InnoDB	MySQL's default storage engine: transactional, with each table stored clustered on its primary key.	§1.4
INSERT	DML statement that adds rows.	§9.8
Integer division	Dividing two integers drops the fraction: in PostgreSQL `7 / 2` is `3`.	§2.3
Intention lock (IS, IX)	A table-level lock announcing that the transaction will lock rows inside the table, so table and row locks can be checked quickly.	§13.10
INTERSECT	Set operation returning rows present in both queries.	§6.9
Interval	A PostgreSQL type for a length of time, such as `interval '2 hours'`.	§3.4
Inverted index	An index that maps each word or element to the list of rows or documents that contain it.	§12.9
I/O	Input/output: reading from or writing to disk or the network, usually the slowest part of running a query.	§12.1
IoT	Internet of Things: networked sensors and devices that send streams of measurements.	§16.4
ISAM	Indexed sequential access method: an older file organization of sorted records with a static multilevel index.	§12.3

Term	Meaning	See
Isolation	Concurrent transactions do not observe each other's intermediate states.	§13.4
Isolation level	How strongly a transaction is isolated: Read Uncommitted, Read Committed, Repeatable Read or Serializable.	§13.4

J

Term	Meaning	See
JDBC	Java Database Connectivity: Java's standard API for running SQL through a database driver.	§14.11
Join	Combining rows of two tables according to a condition.	§5.2
Join dependency	A relation equals the join of some of its projections.	§11.13
JPA / Hibernate	Java's standard ORM interface and its most common implementation.	§12.13
JSON	JavaScript Object Notation: a text format of nested objects, arrays, strings, numbers, booleans and null.	§14.4
JSONB	PostgreSQL's binary JSON type, which supports indexing and containment queries.	§14.4
Junction table	A table that implements a many-to-many relationship with foreign keys to both sides.	§10.15

K

Term	Meaning	See
Kafka	Apache Kafka: a distributed, replicated log used to stream events between systems.	§16.9
Keyset pagination	Paging by remembering the last key seen (`WHERE key > :last`) instead of using `OFFSET`.	§12.14
Key-value store	NoSQL store that maps keys to values with get and put operations.	§16.2

L

Term	Meaning	See
LAG / LEAD	Window functions returning a value from a row before or after the current one.	§8.6
Last-write-wins (LWW)	A conflict rule that keeps the value with the latest timestamp and silently drops the others.	§15.5
Latency	The time a single operation takes, from request to response.	§1.5
LATERAL	A subquery in `FROM` that may refer to columns of tables listed before it.	§6.7
Lazy / eager loading	ORM strategies: fetch related objects only when they are first used (lazy, which risks N+1) or up front with a join (eager).	§12.13
Leader election	Choosing one node to coordinate writes, typically via a consensus system.	§15.8
Leader / follower	In replication, the leader (primary) accepts writes and the followers (replicas, standbys) copy its changes.	§15.5
Leaderless replication	Any replica accepts writes. Clients write to and read from several replicas and rely on quorums.	§15.5
Lease	A lock or leadership right that expires automatically after a set time unless renewed.	§15.8
Ledger	An append-only table of money or stock movements. Balances are sums over it.	§17.3
Leftmost prefix rule	A composite index on (a, b, c) helps filters on a, (a, b) or (a, b, c), not on b alone.	§12.11
LEFT OUTER JOIN	Returns every left row, with matching right rows or NULLs.	§5.4
Lexeme	A normalised word (after stemming) stored in a `tsvector`.	§14.6
LIKE / ILIKE	Pattern matching with `%` and `_`. `ILIKE` is case-insensitive (PostgreSQL).	§2.4
LIMIT / OFFSET	Return at most n rows, after skipping k rows.	§2.8
Linear hashing	A dynamic hashing scheme that splits buckets one at a time in a fixed order as the file grows.	§12.7
Linearizability	The strongest consistency model: each operation appears to take effect at one instant, in real-time order. Also called strong consistency.	§15.6
List partitioning	Assigning rows to partitions by a list of discrete values, such as region codes.	§15.2
Lock	A marker that restricts other transactions' access to a row or table until released.	§13.10
Lock manager	The DBMS component that grants, queues and releases locks.	§1.4
Logical model	Model with attributes, keys and normalized relations, independent of a particular DBMS.	§10.2
Logical query processing order	The conceptual order in which the clauses of a SELECT are evaluated, starting with FROM.	§4.5
Logical replication	Replicating row changes (insert, update, delete) for chosen tables, rather than copying raw WAL bytes.	§15.5
Lossless join	A decomposition whose pieces join back to exactly the original rows.	§11.10
Lost update	Two read-modify-write transactions where the second silently overwrites the first.	§13.7

Term	Meaning	See
LRU / LFU	Cache eviction policies: drop the least recently used, or the least frequently used, item first.	§16.2
LSM tree	Log-structured merge tree: writes go to memory and sorted files that are merged in the background.	§16.4
LSN (log sequence number)	A position in the write-ahead log, used to order changes and to track recovery and replication progress.	§13.5
Lucene	The Java search library inside Elasticsearch and OpenSearch.	§16.11

M

Term	Meaning	See
MapReduce	A batch-processing model: map each record to key-value pairs, group them by key, then reduce each group to a result.	§16.9
MariaDB	An open-source fork of MySQL.	§1.7
Materialized path	Storing each tree node's full ancestor path, such as `/1/4/9/`, in a column.	§7.4
Materialized view	A stored query result that must be refreshed to reflect new data.	§7.8
Memcached	A simple distributed in-memory key-value cache.	§16.6
MERGE	Standard statement that inserts, updates or deletes target rows based on a source (PostgreSQL 15+).	§9.11
Merge join	Join that walks two inputs sorted on the join key.	§12.15
Merkle tree	A tree of hashes in which each parent hashes its children, so two replicas can find differences by comparing a few hashes.	§15.5
Metadata	Data about data, such as table and column definitions.	§1.2
Minimal cover	An equivalent set of FDs with single-attribute right sides, no extraneous attributes and no redundant FDs.	§11.5
Mirroring	Keeping an exact, continuously updated copy of a disk or database on another device.	§15.11
MongoDB	A widely used document database that stores BSON documents in collections.	§16.3
Monotonic reads	Once you have seen a value, later reads never show you an older one.	§15.6
Moving average	The average of the last n values, recomputed for every row.	§8.7
Multi-leader replication	Several nodes accept writes and replicate to each other, so conflicting writes must be resolved.	§15.5
Multivalued attribute	An attribute that can hold several values for one entity, such as phone numbers.	§10.6
Multivalued dependency	`X ↠ Y`: for each X, the set of Y values is independent of the other attributes.	§11.12
MVCC	Multiversion concurrency control: keep several row versions so readers see a snapshot and do not block writers.	§13.14
MySQL	A widely used open-source relational database.	§1.7

N

Term	Meaning	See
N+1 query problem	Running one query for a list, then one more per item, instead of a single set-based query.	§12.13
Natural join	A join on all columns with the same name. Risky because it changes silently when columns are added.	§5.10
Natural key	A key taken from real-world data, such as an ISBN or email.	§10.13
Neo4j	A widely used graph database, queried with Cypher.	§16.10
Nested loop join	For each row of the outer input, look up matches in the inner input.	§12.15
Network partition	A network failure that splits the nodes into groups that cannot reach each other.	§15.1
NewSQL	Distributed databases that keep SQL and ACID while scaling out, such as Spanner and CockroachDB.	§16.7
Non-equi join	A join whose condition is not plain equality, such as `BETWEEN` or `<`.	§5.1
Non-repeatable read	Reading the same row twice in a transaction and getting different values.	§13.4
Normal form	A level of design quality (1NF, 2NF, 3NF, BCNF …) defined by which dependencies a table may contain.	§11.6
Normalization	Organising tables so each fact is stored once, guided by functional dependencies and normal forms.	§11.6
NoSQL	Non-relational databases: key-value, document, wide-column and graph stores.	§16.1
NOT NULL	Constraint forbidding NULL in a column.	§9.3
NOWAIT	A locking option that raises an error at once instead of waiting for a locked row.	§13.12
NTILE	Window function that splits ordered rows into n roughly equal buckets.	§8.5
NULL	A marker for a missing or unknown value. Comparisons with it yield UNKNOWN.	§2.5
NULLIF	Returns NULL if its two arguments are equal, otherwise the first.	§3.5
NULL padding	The NULLs an outer join fills into the columns of the side that had no match.	§5.4
numeric (decimal)	An exact decimal type with a chosen precision and scale, such as `numeric(12, 2)`. The right choice for money.	§3.1

O

Term	Meaning	See
OLAP	Online analytical processing: large scans and aggregates over historical data.	§11.16
OLTP	Online transaction processing: many small reads and writes of current data.	§11.16
ON CONFLICT	PostgreSQL clause that turns an `INSERT` into an upsert or skips conflicting rows.	§9.11
ON DELETE action	What happens to child rows when a parent is deleted: CASCADE, SET NULL, SET DEFAULT, RESTRICT, NO ACTION.	§9.3
Operator precedence	The order in which operators bind: `NOT` before `AND` before `OR`, so add parentheses when you mix them.	§2.2
Optimistic concurrency control	Proceed without locks, validate at commit, and abort or retry on conflict.	§13.9
Optimizer (planner)	The DBMS component that chooses an efficient execution plan for a query.	§1.4
Oracle Database	A commercial relational database. Its procedural language is PL/SQL.	§1.7
ORDER BY	Sorts the result. The only way to guarantee row order.	§2.7
Ordered-set aggregate	An aggregate that needs sorted input, given with `WITHIN GROUP (ORDER BY …)`, such as `percentile_cont`.	§4.1
ORM	Object-relational mapper: a library that maps classes to tables and generates SQL.	§14.13
Orphan row	A child row whose foreign key points to a parent that does not exist.	§10.14
Outbox pattern	Writing an event row to an outbox table in the same transaction as the data change, then publishing it from there. Avoids dual writes.	§14.3
OUTER JOIN	LEFT, RIGHT or FULL join, which keeps unmatched rows padded with NULLs.	§5.2
Overflow page	An extra page chained to a full bucket or page.	§12.7

P

Term	Meaning	See
PACELC	Extension of CAP: under Partition choose Availability or Consistency, Else choose Latency or Consistency.	§15.7
Page	Fixed-size unit of storage and I/O (8 KB in PostgreSQL).	§12.1
Pagination	Returning a result one page at a time, with `LIMIT` / `OFFSET` or keyset paging.	§12.14
Parameterized query	A query whose values are sent separately from the SQL text as bind parameters, preventing injection.	§14.9
Parser	The DBMS component that checks a statement's syntax and turns it into an internal tree for the planner.	§1.4
Partial dependency	A non-prime attribute determined by a proper subset of a composite key.	§11.8
Partial index	An index that covers only rows matching a `WHERE` condition.	§12.12
Partial key (discriminator)	The attribute that identifies a weak entity among the other entities of the same owner.	§10.9
Participation	Whether every entity instance must take part in a relationship (total) or may not (partial).	§10.7
PARTITION BY (window)	Splits rows into independent groups for a window function without collapsing them.	§8.2
Partitioning	Splitting one table into smaller physical pieces by range, list or hash.	§15.2
Partition key	The column whose value decides which partition, shard or node stores a row.	§15.2
Partition pruning	Skipping partitions that cannot contain matching rows.	§15.2
Partition tolerance	In CAP: the system keeps working even when messages between nodes are lost.	§15.7
Patroni	A tool that runs PostgreSQL high availability: it watches the primary and performs automatic failover using a consensus store.	§15.10
Paxos	A classic consensus algorithm in which proposers get a majority of acceptors to agree on a single value.	§15.8
PBFT	Practical Byzantine Fault Tolerance: consensus that survives f lying nodes using 3f + 1 nodes.	§15.8
Peer rows	Rows with equal values of the window `ORDER BY` key.	§8.3
Percentile / median	The value below which a given share of the data falls. The median is the 50th percentile.	§4.1
Pessimistic concurrency control	Lock data before using it so conflicting transactions wait.	§13.9
PgBouncer	A lightweight connection pooler placed between applications and PostgreSQL.	§14.12
pg_dump / pg_restore	PostgreSQL's tools to export a logical backup and to load it back.	§9.12
pg_trgm (trigrams)	A PostgreSQL extension that splits text into three-character pieces so indexes can speed up `LIKE '%x%'` and similarity search.	§12.13
Phantom read	Re-running a query in a transaction returns a different set of rows because of inserts or deletes.	§13.4
Physical model	Model with exact types, indexes, partitions and storage for a specific DBMS.	§10.2

Term	Meaning	See
PITR (point-in-time recovery)	Restoring a base backup and replaying archived WAL up to a chosen moment.	§9.12
Pivot	Turning row values into columns, usually with conditional aggregation.	§14.7
PL/pgSQL	PostgreSQL's procedural language for functions, procedures and triggers.	§14.1
Polyglot persistence	Using several kinds of database in one system, each for what it does best.	§16.10
Polymorphic association	One column that may point to rows of several different tables (a type + id pair). No foreign key can enforce it.	§17.10
PostGIS	A PostgreSQL extension that adds geometry and geography types, spatial functions and spatial indexes.	§16.8
PostgreSQL	The open-source object-relational database used throughout this book (version 14).	§1.7
Posting list	In an inverted index, the list of rows or documents that contain one term.	§16.11
Predicate lock	A lock on a condition rather than on existing rows, used to prevent phantoms.	§13.10
Predicate push-down	Applying filters as early as possible (inside a scan, a subquery or a remote shard) so fewer rows flow on.	§15.4
Prepared statement	A statement parsed once and executed many times with different parameters.	§14.9
Primary index	An index on the field by which a sorted file is ordered, usually with one entry per block (sparse).	§12.4
Primary key	The chosen candidate key: unique and not NULL, one per table.	§9.3
Prime attribute	An attribute appearing in some candidate key.	§10.13
Principle of least privilege	Give each user or program only the rights it needs, and nothing more.	§14.8
Privilege	A permission such as SELECT or INSERT on an object, granted to a role.	§14.8
Procedure (stored)	Named server-side routine called with `CALL`. Can control transactions.	§14.2
Prometheus	A monitoring system with a built-in time-series database for metrics.	§16.8
Pseudotransitivity	A derived FD rule: if X → Y and WY → Z, then WX → Z.	§11.3
psql	PostgreSQL's command-line client.	§1.10

Q

Term	Meaning	See
Quadtree	A spatial index that recursively divides an area into four squares.	§16.8
Query builder	A library that builds SQL from code calls, without the full object mapping of an ORM.	§14.13
Query plan	The tree of operations (scans, joins, sorts) the executor runs for a query.	§12.10
Quorum	The number of replicas that must acknowledge an operation. `R + W > N` makes reads see the latest write.	§15.5

R

Term	Meaning	See
Raft	A consensus algorithm with an elected leader that replicates a log to a majority.	§15.8
RAID	Combining several disks for safety or speed: RAID 1 mirrors, RAID 5 and 6 add parity, RAID 10 mirrors and stripes.	§15.11
RAM	The computer's main memory: much faster than disk, but its contents are lost when the power goes.	§12.1
RANGE frame	A window frame defined by value distance from the current row's `ORDER BY` value. Includes peers.	§8.4
Range partitioning	Assigning rows to partitions by ranges of a key, usually dates.	§15.2
Range type	A PostgreSQL type holding an interval of values, such as `daterange` or `tstzrange`, with operators like `&&` (overlaps).	§17.1
RANK	Ranking function that gives ties the same rank and skips the next ranks (1, 2, 2, 4).	§8.5
RDBMS	Relational database management system: a DBMS based on the relational model, such as PostgreSQL.	§1.7
RDB snapshot	Redis's point-in-time dump of its whole data set to a file.	§16.11
Read Committed	Isolation level where each statement sees only data committed before it began. PostgreSQL's default.	§13.4
Read replica	A copy of the database that serves read queries.	§15.5
Read skew	Seeing different parts of the data at different moments within one transaction.	§13.4
Read-your-writes	A guarantee that a user always sees their own earlier writes.	§15.6
Rebalancing	Moving data between nodes when nodes are added or removed, to keep the load even.	§15.3
Recovery	Restoring a consistent state after a crash, using the log to redo committed and undo uncommitted work.	§15.9
Recursive CTE	A CTE that refers to itself, used for hierarchies, graphs and series.	§7.3
Redis	An in-memory data-structure store (strings, hashes, lists, sets, sorted sets, streams) used as a cache, queue or database.	§16.2
Redo / undo	Redo re-applies committed changes from the log. Undo reverses changes made by transactions that did not commit.	§15.9
Redundancy	The same fact stored more than once.	§11.6
Referential integrity	Every foreign-key value matches an existing referenced key (or is NULL).	§10.14
Reflexivity (FD rule)	Armstrong's rule: if Y is a subset of X, then X → Y.	§11.3
Regular expression	A pattern language for text, such as `^[a-z]+@`. PostgreSQL matches with `~` and `~*`.	§2.4
Relation	The formal name for a table: a set of tuples over a set of attributes.	§10.12
Relational algebra	The formal operations on relations (selection, projection, join, union, difference) that SQL is built on.	§10.11
Relational division	Finding rows related to every member of another set, such as students who took all courses.	§19.7

Term	Meaning	See
Relational model	Data as relations of atomic values, linked by values and queried declaratively (Codd, 1970).	§1.6
Relation schema / relation instance	The schema is a relation's name and attributes. The instance is the set of tuples it holds at a moment.	§10.12
Repeatable Read	Isolation level where rows read once keep their values. In PostgreSQL, snapshot isolation.	§13.4
Replication	Keeping copies of the same data on several servers.	§15.5
Replication lag	The delay before a change made on the primary appears on a replica.	§15.5
Result set	The rows a query returns.	§2.1
Retention policy	A rule for how long data is kept before it is deleted or summarised.	§16.8
RETURNING	Clause that returns the rows affected by `INSERT`, `UPDATE` or `DELETE`.	§9.8
RIGHT OUTER JOIN	Returns every right row, with matching left rows or NULLs.	§5.5
Rigorous 2PL	Two-phase locking that holds every lock, read and write, until the transaction ends.	§13.10
Role	A PostgreSQL user or group that owns objects and holds privileges.	§14.8
ROLLUP	Grouping extension that adds hierarchical subtotals and a grand total.	§4.6
Round-robin	Handing items to nodes in turn: 1, 2, 3, 1, 2, 3 …	§15.2
Row-level security	Policies that filter which rows a role may read or write.	§14.8
ROW_NUMBER	Window function giving each row a unique sequential number in its partition.	§8.5
Row-oriented storage	Storage that keeps all the columns of a row together. Good for OLTP reads and writes of whole rows.	§12.1
ROWS frame	A window frame counted in physical rows.	§8.4
Row value (row constructor)	Several values written as one, `(a, b)`, compared left to right: `(ts, id) < (:ts, :id)`.	§6.6
RPO / RTO	Recovery point objective (data you may lose) and recovery time objective (downtime you may have).	§9.12
R-tree	A spatial index that groups nearby objects into nested bounding rectangles.	§16.8
Run-length encoding (RLE)	Compressing runs of a repeated value as (value, count) pairs.	§12.2
Running total	A cumulative sum from the first row up to the current row.	§8.3

S

Term	Meaning	See
Saga	A long business process split into local transactions, each with a compensating step that undoes it if a later step fails.	§15.9
Sargable	A predicate the database can answer with an index seek, such as `col >= value` with no function on `col`.	§12.13
Savepoint	A marker inside a transaction you can roll back to without abandoning the whole transaction.	§13.2
Scalar function	A function that returns one value per input row, such as `upper` or `round`.	§4.1
Scalar subquery	A subquery that returns one value. Zero rows give NULL, more than one row is an error.	§6.1
Scatter-gather	Sending a query to every shard and merging their partial results.	§15.4
Schema	The structure of a database (tables, columns, constraints), or a PostgreSQL namespace that groups objects.	§9.2
Schema-on-read / on-write	Structure checked when data is read (typical of NoSQL) or when it is written (relational).	§1.8
Secondary index	An index on columns other than the file's ordering key, with one entry per record (dense).	§12.4
Selectivity	The fraction of rows a predicate keeps. Low fractions favour index scans.	§12.16
SELF JOIN	Joining a table to itself under two aliases.	§5.7
Semi-join	Returns rows of one table that have at least one match in another, each once.	§5.8
Semi-structured data	Data that carries its own flexible structure, such as JSON or XML.	§14.4
Semi-synchronous replication	The primary waits until at least one replica has received the change before confirming the commit.	§15.5
Sequence	A database object that hands out increasing numbers.	§9.4
Sequential scan	Reading every page of a table.	§12.10
serial / bigserial	PostgreSQL's older auto-increment shorthand: an integer column with a sequence as its default.	§9.4
Serializable	The strongest isolation level: the outcome equals some serial order of the transactions.	§13.8
Serialization failure	Error 40001: the database aborted a transaction to preserve isolation. Retry it.	§13.7
Session consistency	Guarantees such as read-your-writes and monotonic reads that hold within one client session.	§15.6
Sessionization	Grouping a user's events into sessions separated by gaps of inactivity.	§8.9
Set operation	UNION, INTERSECT or EXCEPT, which combine the results of two queries.	§6.8
Set-returning function	A function that returns many rows, such as `generate_series` or `unnest`, used in `FROM`.	§7.6
Sharding	Placing partitions of the data on different servers.	§15.3
Shard key	The column that decides which shard stores a row.	§15.3
Shared lock (S)	A lock several transactions can hold at once for reading. It blocks exclusive locks.	§13.10

Term	Meaning	See
Single-leader replication	One node takes all the writes and streams them to its followers.	§15.5
SKIP LOCKED	A locking option that skips rows other transactions have locked, so many workers can take different jobs from one queue.	§13.12
Slotted page	A page layout with an array of line pointers in the header pointing to variable-length rows stored from the end of the page.	§12.1
Snapshot	The set of committed data a transaction or statement is allowed to see.	§13.4
Snapshot isolation	Each transaction reads from a consistent snapshot taken at its start. Allows write skew.	§13.4
Soft delete	Marking a row as deleted (`deleted_at`) instead of removing it.	§17.14
Spanner	Google's globally distributed SQL database, which uses TrueTime clocks to order transactions.	§16.7
Spark	Apache Spark: distributed in-memory processing of large data sets, with SQL and DataFrame APIs.	§16.9
Sparse index	An index with entries for only some records, typically one per block.	§12.4
Split brain	Two nodes both believe they are the leader and accept writes, so the data diverges.	§15.8
SQL	Structured Query Language: the declarative standard language of relational databases.	§1.9
SQLAlchemy	Python's widely used SQL toolkit and ORM.	§14.12
SQL injection	An attack that smuggles SQL into a query through unescaped input.	§14.9
SQLite	An embedded relational database stored in a single file and run inside the application.	§1.7
SQL Server	Microsoft's relational database. Its dialect and procedural language is T-SQL.	§1.7
SQLSTATE	The standard five-character error code, such as `23505` (unique violation) or `40001` (serialization failure).	§13.6
SSI	Serializable snapshot isolation: snapshot isolation plus tracking of read-write conflicts, aborting when a cycle is possible.	§13.8
SSTable	Sorted string table: an immutable file of sorted key-value pairs, written by LSM-tree stores.	§12.3
Star schema	A warehouse design: one central fact table of events joined to surrounding dimension tables (date, product, store).	§11.16
Starvation	A transaction waits forever because others keep getting the resource first.	§13.13
Statistics	Per-column summaries (distinct counts, common values, histograms) used by the planner.	§12.16
Stemming	Reducing words to a common root (running → run) so a search matches every form.	§14.6
Stop word	A very common word, such as "the" or "and", that full-text search ignores.	§14.6
Storage engine	The DBMS layer that stores pages and rows on disk.	§1.4
Streaming replication	PostgreSQL's physical replication: replicas receive WAL records continuously and replay them.	§15.5
Strict 2PL	Two-phase locking that holds write locks until commit or abort, so no one reads uncommitted data.	§13.10

Term	Meaning	See
Subquery	A query nested inside another statement.	§6.1
Superkey	Any set of attributes that uniquely identifies rows.	§10.13
Superuser	A role that bypasses all permission checks.	§14.8
Surrogate key	An artificial key with no business meaning, such as an identity number or UUID.	§10.13
Synchronous replication	The primary waits for a replica to confirm before reporting commit.	§15.5

T

Term	Meaning	See
Table	The SQL form of a relation: rows and named, typed columns.	§1.6
Tablespace	A directory on disk where PostgreSQL stores chosen tables and indexes.	§9.12
TCL	Transaction control language: `BEGIN`, `COMMIT`, `ROLLBACK`, `SAVEPOINT`.	§1.9
Temporary table	A table that exists only for the session (or transaction) that created it.	§7.1
Thomas write rule	In timestamp ordering, a write that is already outdated is skipped instead of aborting the transaction.	§13.13
Three-valued logic	SQL logic with TRUE, FALSE and UNKNOWN.	§2.5
Throughput	How much work a system completes per unit of time, such as transactions per second.	§16.2
TiDB	A MySQL-compatible distributed SQL database built on the TiKV key-value store.	§16.7
TimescaleDB	A PostgreSQL extension for time-series data that partitions tables by time automatically.	§16.8
Time-series database	A store optimised for timestamped measurements, retention and downsampling.	§16.8
Timestamp ordering	Concurrency control that orders conflicting operations by transaction timestamps.	§13.13
timestamptz vs timestamp	`timestamptz` stores an exact instant, shown in the session's time zone. `timestamp` stores a wall-clock reading with no zone.	§3.1
TOAST	PostgreSQL's mechanism for compressing or moving large values out of the main table page.	§12.1
Tombstone	A marker saying a key was deleted, so the deletion reaches every replica and survives compaction.	§16.4
Top-N per group	Returning the N best rows inside each group, usually by ranking in a CTE and filtering outside.	§8.8
Transaction	A unit of work of one or more statements that succeeds or fails as a whole.	§13.1
Transaction manager	The DBMS component that starts, commits and aborts transactions and coordinates locks and logging.	§1.4
Transitive dependency	A → B and B → C where B is not a key, so C depends on the key only through B.	§11.9
Transitivity (FD rule)	Armstrong's rule: if X → Y and Y → Z, then X → Z.	§11.3
Trigger	A function run automatically before or after changes to a table.	§14.3
Trivial FD	An FD whose right side is contained in its left side.	§11.2
TrueTime	Google's clock service that returns the current time as an interval with bounded uncertainty, used by Spanner.	§16.7
TRUNCATE	Quickly removes all rows from a table.	§9.6
T-SQL	Transact-SQL: SQL Server's dialect and procedural language.	§1.7
tsvector / tsquery	PostgreSQL full-text types: a document as a list of lexemes, and a search condition matched against it with `@@`.	§14.6
TTL (time to live)	An expiry time after which a key or row is deleted automatically.	§16.2
Tuple	The formal name for a row.	§10.12

Term	Meaning	See
Type vs instance	Keeping the catalogue description (room type, book title) separate from the physical item (room, copy).	§17.14

U

Term	Meaning	See
Undo log	A log of old values used to roll back the changes of transactions that do not commit.	§13.14
UNION / UNION ALL	Combine two results, removing duplicates (`UNION`) or keeping them (`UNION ALL`).	§6.8
UNIQUE constraint	Forbids duplicate non-NULL values in a column or column set.	§9.3
Unique index	An index that also rejects duplicate keys. It is how UNIQUE and PRIMARY KEY are enforced.	§9.3
unnest	A function that expands an array into a set of rows.	§14.5
Unpivot	Turning columns into rows.	§14.7
Updatable view	A simple view that accepts `INSERT` , `UPDATE` and `DELETE` , passing them to the underlying table.	§7.7
UPDATE	DML statement that changes column values in matching rows.	§9.9
Update anomaly	A change to a fact must be repeated in many rows, and missing one leaves the data inconsistent.	§11.6
Upsert	Insert a row, or update it if it already exists.	§9.11
UTC	Coordinated Universal Time: the zone-free reference clock that servers usually store times in.	§3.4
UTF-8	The common text encoding that stores each character in 1 to 4 bytes.	§1.10
UUID	128-bit identifier that can be generated anywhere without coordination.	§9.4
UUIDv7	A UUID version that starts with a timestamp, so new ids are roughly in order and kinder to B-tree indexes.	§9.4

V

Term	Meaning	See
VACUUM	PostgreSQL maintenance that reclaims dead row versions and updates the visibility map.	§13.14
Vector clock	A list of per-node counters attached to a value, used to tell whether two versions are ordered or concurrent.	§15.5
Version column	A counter increased on every update and checked in the `WHERE` clause, which implements optimistic locking.	§13.7
Vertical scaling	Moving to a bigger machine (scale up).	§15.1
View	A named, stored query that behaves like a table and holds no data of its own.	§7.7
Virtual node	One of many small positions a physical node owns on a hash ring, so load spreads evenly.	§15.3
Visibility map	A PostgreSQL bitmap marking pages whose rows are visible to every transaction, which makes index-only scans possible.	§12.10
Volatility (function)	How stable a function's result is: `IMMUTABLE` (never changes), `STABLE` (fixed within a statement) or `VOLATILE` (may change on every call).	§14.1

W

Term	Meaning	See
Wait-die / wound-wait	Deadlock-prevention schemes that use transaction age to decide whether a transaction waits for a lock or is aborted.	§13.11
WAL (write-ahead log)	A log of changes written and flushed before the data pages, used for durability, recovery and replication.	§13.5
Weak entity	An entity identified only together with its owner entity.	§10.9
Weighted average	An average in which each value counts in proportion to a weight, such as units sold.	§18.8
WHERE	Filters rows before grouping. Keeps only rows where the condition is TRUE.	§2.2
Wide-column store	NoSQL store with partition keys and sorted clustering columns, such as Cassandra.	§16.4
Wildcard	A pattern character: in `LIKE`, `%` matches any run of characters and `_` exactly one.	§2.4
Window function	A function computed over a set of rows related to the current row, without collapsing them.	§8.1
WITH TIES	Extends `FETCH FIRST n` to include rows tied with the last one.	§2.8
work_mem	PostgreSQL setting for the memory one sort or hash may use before it spills to disk.	§12.15
Write amplification	One logical write causing several physical writes (log, page, index, compaction).	§12.4
Write skew	Two transactions read overlapping data and write different rows, together breaking a rule each checked.	§13.8

X

Term	Meaning	See
xmin / xmax	Hidden PostgreSQL row columns that hold the ids of the transactions that created and deleted that row version.	§13.14

Y

Term	Meaning	See
YugabyteDB	A distributed SQL database with a PostgreSQL-compatible interface.	§16.7

Z

Term	Meaning	See
ZooKeeper	Apache ZooKeeper: a replicated coordination service (using the ZAB protocol) for configuration, locks and leader election.	§15.8

Chapter 20 TL;DR

- **Evaluation order** explains most errors: FROM → WHERE → GROUP BY → HAVING → SELECT (windows) → DISTINCT → set operations → ORDER BY → LIMIT (20.1).
- **NULL** is unknown: `WHERE` keeps only TRUE, `NOT IN` with a NULL returns nothing, `count(col)` and other aggregates skip NULLs, and `sum` over no rows is NULL (20.2, 20.4).
- **PostgreSQL arithmetic**: `7 / 2 = 3`, so write `100.0 *` before dividing and guard with `NULLIF(x, 0)` (20.3).
- **Joins**: right-table filters of a `LEFT JOIN` go in `ON`. `NOT EXISTS` is the safe anti-join. Watch fan-out (20.5, 20.6).
- **Windows**: `row_number` / `rank` / `dense_rank` differ only on ties. The default frame with `ORDER BY` is `RANGE … CURRENT ROW`, so give `last_value` a full frame (20.8).
- **Design**: model entities, keys and relationships first, normalize to 3NF/BCNF, denormalize only with a reason and a way to keep copies right (20.10).
- **Indexes**: B-tree by default, equality columns before range columns, no functions on indexed columns, keyset instead of deep `OFFSET`, and compare estimated with actual rows in `EXPLAIN ANALYZE` (20.11).
- **Transactions**: PostgreSQL defaults to Read Committed. Repeatable Read is snapshot isolation (write skew still possible). Serializable (SSI) prevents everything but needs retries on 40001 (20.12).
- **Dialects** differ most on paging, concatenation, NULL sorting, upsert, `RETURNING`, integer division and transactional DDL (20.16).
- About 100 PostgreSQL claims and 28 SQLite claims on these sheets are checked by runnable scripts (20.17). The pattern catalogue (20.18) and the fifty Q&A (20.19) link back to the full explanations.

This is the end of the SQL Handbook. For more on the design side of data systems, see the DDIA notes, and for how databases fit into whole systems, the case studies in Chapter 17.